

Active Directory Hardening Guidebook

Production-Grade Hardening Requirements & Guidelines for Air-Gapped Environments

Standards Alignment:

ANSSI (French National Agency for the Security of Information Systems)
CIS Benchmarks (Center for Internet Security)
Microsoft Security Baselines

Target Operating Systems:

Domain Controllers: Windows Server 2016 and above
Tier 2 Client Workstations: Windows 10 and above

Generated dynamically on: July 08, 2026

Active Directory Hardening Guidebook

Welcome to the **Active Directory Hardening Guidebook**. This repository houses a production-grade, offensive-aligned set of hardening requirements and guidelines specifically designed for securing **modern Active Directory (AD) environments** in **air-gapped (offline)** settings.

Core Philosophy & Design

In high-security, isolated environments, traditional cloud-based security feeds and agent-based telemetry may be unavailable or restricted. This guidebook addresses this challenge by providing a self-contained, deterministic hardening framework that:

- **Enforces Administrative Tiering:** Strictly isolates administrative identities, credentials, and systems to prevent privilege escalation.
- **Restricts Attack Surface:** Systematically disables legacy protocols, name resolution mechanisms, and vulnerable default options.
- **Ensures Deterministic Configuration:** Standardizes system baselines using Group Policy Objects (GPOs) and Desired State Configuration (DSC).

Scope & Target Systems

The guidebook covers the following scopes:

COMPONENT	TARGET OPERATING SYSTEM	PLACEMENT / TIER
Domain Controllers	Windows Server 2016 and above	Tier 0 (Core Identity)
Privileged Access Workstations (PAWs)	Windows 10 Enterprise and above	Tier 0/1 (Management Plane)
Tier 2 Client Workstations	Windows 10 Enterprise and above	Tier 2 (Standard Clients)

Target environment characteristics:

- **Air-Gapped / Isolated:** No active internet connectivity or external DNS dependencies.
- **On-Premises Focused:** No Azure AD / Entra ID hybrid integrations, relying purely on native AD DS.
- **Standard OS Baseline:** Built and tested against native Windows enterprise releases, with no dependence on third-party security agents for core functions.

Administrative Tiering Model

To prevent credential theft and lateral movement, the guidebook structures all policies around the 3-Tier administrative tiering model:

TIER 0

Identity & Control Plane
Direct or indirect administrative control over the Active Directory forest, identity systems, and key management infrastructure.

Domain ControllersPKI & Active Directory Certificate Services (ADCS)Privileged Access Workstations (PAWs)Domain/Forest Admins

TIER 1

Enterprise Servers & Services
Corporate application servers, database systems, and management infrastructure. High business value but no direct identity fabric control.

Enterprise Member ServersApplication Service AccountsServer AdministratorsWSUS & Configuration Managers

TIER 2

Endpoints & Standard Users
Standard corporate client workstations, laptops, mobile devices, and normal business users.

Client Workstations / LaptopsStandard Corporate UsersLocal Workstation Administrators

Guidebook Structure & Modules

The hardening guidelines are organized into eight functional modules:

MODULE	TARGET SCOPE	FOCUS AREAS
Module 1: Architecture & Administrative Tiering	Infrastructure Layout	Tier logons, administrative protocols, privileged group audits, forest functional levels, GPO management.
Module 2: Domain Controller Hardening	Domain Controllers (Tier 0)	SMBv1/NTLMv1 deprecation, LDAP signing/channel binding, Print Spooler disablement, LSA/Credential Guard.
Module 3: Identities & Services Hardening	Directory Identities	Password policies, LAPS, gMSAs, delegation restrictions, Protected Users, ADCS/PKI security.
Module 4: Network Configuration & Firewalling	Network Boundaries	Port matrices, RPC dynamic port restrictions, IPsec domain isolation, SMBv3 security, WinRM.
Module 5: Logging, Monitoring & SIEM	Directory Auditing	Security audit policies, PowerShell/CLI auditing, Sysmon deployment, secure log shipping.
Module 6: Secure Operations & Maintenance	Directory Operations	KRBGTGT password rotation, AD Recycle Bin, ADMX Central Store management, Tier 0 WSUS baseline.
Module 7: Privileged Access Workstations Hardening	Management Devices (Tier 0/1)	BitLocker with TPM/PIN, UEFI security, DMA protection, AppLocker, WDAC, kernel shadow stacks.
Module 8: Endpoint Hardening	Client Workstations (Tier 2)	UAC policies, LOLBins blocklists, Application Control (WDAC), system service disabling, Windows Defender.

Continuous Auditing & Compliance Framework

To ensure that the security controls documented in this guidebook remain enforced and do not experience configuration drift, this repository includes a two-pronged automated auditing framework:

1. PowerShell DSC Audit Baseline

A native [PowerShell DSC Audit Framework](#) that operates in [ApplyAndMonitor](#) mode, continuously checking target systems against the security baseline and logging details of any failing controls or drift. It is organized around targeting profiles for Common Controls, Domain Controllers, PAWs, and Endpoints.

For details on configuration and compilation, refer to the [DSC Audit Framework Documentation](#).

2. Automated SCAP Benchmarks (XCCDF & OVAL)

A standardized, declarative security checking mechanism using [SCAP XML Benchmarks](#):

- **XCCDF Benchmark** ([audit/scap/ad-hardening-xccdf.xml](#)): Defines the checklist groups, severities, and target profiles.
- **OVAL Definitions** ([audit/scap/ad-hardening-oval.xml](#)): Implements automated native checks (registry, services, privileges, account limits, and audit policies) to evaluate compliance without manual tasks.

For execution instructions using tools like [oscap](#) or enterprise compliance agents, refer to the [SCAP Compliance Documentation](#).

Standards & Compliance Mapping

All controls are mapped to international security standards to simplify audits and verify posture. For compliance mappings, see the dedicated matrices:

- [ANSSI Compliance Matrix](#): Aligned with ANSSI's *Hardening an Active Directory Directory Service* guidelines.
- [CIS Benchmarks Compliance Matrix](#): Aligned with the Center for Internet Security (CIS) Windows Server and Windows Client Benchmarks.
- [Microsoft Security Baselines Compliance Matrix](#): Aligned with the Microsoft Security Baselines specification.

Module 1: Architecture & Administrative Tiering

This directory contains the Active Directory Administrative Tiering Model definitions, theoretical designs, and technical enforcement controls for secure, air-gapped environments.

Architecture Treatise & Guidelines

- [Tiering and Architecture Overview](#) Detailed design treatise covering administrative boundaries (Tier 0, Tier 1, Tier 2), Organizational Unit layout structures, custom naming conventions, credentials hygiene, and management routing from PAWs to DCs via secure jump hosts.

Technical Hardening Controls

1. [REQ-ARCH-001 - Implement Active Directory Administrative Tiering Model](#) Enforces User Rights Assignment GPOs to implement the Active Directory administrative tiering model by blocking high-privilege administrators (Tier 0/1) from authenticating interactively or via network logon on lower-tier computers (Tier 1/2), preventing credential exposure in LSASS memory.
2. [REQ-ARCH-002 - Restrict Administrative Management Protocols](#) Restricts inbound Remote Desktop (RDP) and Windows Remote Management (WinRM) administrative protocols to dedicated, secure administrative subnets and jump hosts.
3. [REQ-ARCH-003 - Audit Privileged Groups](#) Implements automated auditing of Tier 0 administrative Active Directory groups to detect nested memberships and unauthorized additions.
4. [REQ-ARCH-004 - Keep Domain and Forest Functional Levels Up-To-Date](#) Recommends migrating Domain and Forest Functional Levels to Windows Server 2016 or higher to unlock critical security features like the Protected Users group, gMSAs, and Kerberos Armoring.
5. [REQ-ARCH-005 - Default Domain and Domain Controllers Policies Management](#) Provides structural guidelines to separate custom hardening policies into dedicated, modular GPOs rather than directly editing Default Domain/DC policies, protecting the forest baseline.
6. [REQ-ARCH-006 - Harden Active Directory Domain Trusts](#) Hardens trust relationships across forest and external boundaries by disabling SID History, enabling Quarantine (SID filtering), enforcing Selective Authentication, and blocking Kerberos TGT Delegation.
7. [REQ-ARCH-007 - Harden Microsoft Exchange Active Directory Permissions](#) Removes WriteDacl and WriteOwner permissions for Exchange groups on the domain root.

Module 1: Architecture and Administrative Tiering

This document details the design principles, theoretical concepts, threat vectors, and management rules governing the Active Directory Administrative Tiering Model in secure, air-gapped environments.

1. Rationale Behind the Tiering Model

Active Directory directory services run as a shared security database where trust is transitive across the entire domain. By default, any account with local administrator rights on a domain-joined machine can access memory components containing active authentication tokens.

If an enterprise administrator uses the same administrative credential (e.g., Domain Admin) to manage a Domain Controller (Tier 0) and log on to a standard workstation or application server (Tier 1/2), those high-privilege credentials remain stored in the Local Security Authority Subsystem Service (LSASS) process memory of the lower-tier system.

If an attacker compromises that lower-tier server or workstation, they can dump LSASS memory or the cache, steal the Domain Admin credentials, and achieve immediate, full control of the Active Directory database (Tier 0).

The administrative tiering model prevents this escalation pathway by establishing rigid identity boundaries. It restricts privileged accounts to logging on only to systems within their own security tier or higher, ensuring that Tier 0 credentials never reside on less-secure Tier 1 or Tier 2 machines.

2. Threat Model and Mitigated Attack Vectors

Implementing administrative tiering directly mitigates the most common and critical Active Directory exploit techniques:

Credential Harvesting from LSASS (Memory Dumping) * **Threat**: When a user logs on to a Windows machine, the LSASS process caches credentials (hashes, Kerberos tickets, or cleartext passwords depending on OS configuration) to support single sign-on. Tools like Mimikatz allow attackers with local administrator rights on that host to read LSASS memory and extract these secrets. * **Mitigation**: Under the tiering model, Tier 0 administrators are restricted from logging on to Tier 1 and Tier 2 machines. If an attacker dumps LSASS on a Tier 2 endpoint, they will only find Tier 2 user credentials, preventing them from escalating to domain administrator levels.

Pass-the-Hash (PtH) and Overpass-the-Hash * **Threat**: Windows uses NTLM hashes for local and network authentication. Attackers do not need to decrypt NTLM hashes to authenticate; they can present the captured hash directly to target systems to log on as that user. * **Mitigation**: Enforcing explicit boundaries prevents Tier 0 and Tier 1 hashes from being cached on Tier 2 endpoints, meaning there are no high-tier hashes on standard machines for attackers to pass or relay.

3. Theoretical Implementation

Enforcing administrative tiering requires structural separation in Active Directory at the Organizational Unit (OU) level, Group Policy level, and account naming level.

 Active Directory Administrative Tiering Model

Organizational Unit (OU) Structure Active Directory OUs must be organized to group assets logically by security level:

 Active Directory Organizational Unit Hierarchy

Group Policy Object (GPO) Separation Separate, distinct GPOs must be created and linked to each tier's OU. Cross-linking GPOs is prohibited: * **GPO_Hardening_Tier0**: Linked only to the Tier 0 OU. Defines strict Logon Rights, audits NTDS access, and enforces LSA/Credential Guard policies. * **GPO_Hardening_Tier1**: Linked only to the Tier 1 OU. Configures server security baselines, restricts incoming RDP, and disables legacy protocols. * **GPO_Hardening_Tier2**: Linked only to the Tier 2 OU. Enforces Workstation Isolation, blocks incoming RPC/SMB from peer workstations, and configures local UAC restrictions.

Administrative Account Design and Naming Prefixes Administrators must use distinct accounts depending on the tier they are managing. Standardizing account prefixes facilitates GPO targeting and automated auditing: * **Tier 0 Accounts** (Prefix: 'a0-'): Members of 'Domain Admins', 'Enterprise Admins', or custom groups delegated to modify Active Directory objects. E.g., 'a0-florian'. * **Tier 1 Accounts** (Prefix: 'a1-'): Used for managing member servers, IIS instances, SQL databases, and enterprise applications. E.g., 'a1-florian'. * **Tier 2 Accounts** (Prefix: 'a2-'): Used for desktop support, local administration on standard workstations, or active helpdesk work. E.g., 'a2-florian'.

4. Operational Management and Administrative Routing

Managing a tiered environment requires strict adherence to operational routing paths.

Credentials Lifecycle & Hygiene 1. **Interactive Logons**: Tier 0 administrator credentials ('a0-' prefix) must only be entered interactively on PAWs, Tier 0 Jump Hosts, and Domain Controllers. They must never be used in 'RunAs' contexts on Tier 1 or Tier 2 machines. 2. **Service Accounts**: Service accounts must be assigned to a specific tier. A service account running an application on a Tier 1 server must never have administrative rights in Active Directory (Tier 0). 3. **Emergency Access Accounts (Break-Glass)**: Enforce highly complex passwords for emergency accounts. Store these passwords in a physical vault (such as a safe) with dual-authorization access requirements, avoiding digital password managers running on standard network subnets.

5. Technical Hardening Controls

To enforce this theoretical architecture on Domain Controllers and client computers, you must configure the following technical controls:

1. [REQ-ARCH-001 - Implement Active Directory Administrative Tiering Model](#) Enforces User Rights Assignment GPOs to implement the Active Directory administrative tiering model by denying Tier 0 administrative accounts from logging on to Tier 1 and Tier 2 machines, and Tier 1 administrative accounts from logging on to Tier 2 machines.
2. [REQ-ARCH-002 - Restrict Administrative Management Protocols](#) Enforces network-level restriction of RDP (TCP 3389) and WinRM (TCP 5985/5986) management traffic to specific administrative IP ranges and Jump Host IP addresses.
3. [REQ-ARCH-003 - Audit Privileged Groups](#) Enforces regular automated checks on domain administrative groups to ensure no nested memberships or unapproved accounts are assigned Tier 0 rights.

[REQ-ARCH-001] Implement Active Directory Administrative Tiering Model

Target Scope * **Applicable Systems**: Tier 1 member servers and Tier 2 client workstations. * **Operating Systems**: Windows Server 2016 (and above), Windows 10 (and above) Enterprise/Professional.

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment

Rationale To successfully enforce the administrative tiering model, the boundaries must be programmatically restricted. Active Directory administrative groups (such as 'Domain Admins', 'Enterprise Admins', and 'Schema Admins') must be explicitly blocked from authenticating to lower-tier systems.

If these permissions are not restricted, a Tier 0 administrator might use their account to troubleshoot a Tier 1 server or Tier 2 workstation. This action caches their administrative password hash or Kerberos Ticket Granting Ticket (TGT) in the local memory of the target machine. If that machine has been compromised, an attacker can extract those credentials and compromise the entire Active Directory domain.

By configuring Group Policy objects to explicitly deny logon rights (interactive, network, and Remote Desktop) for high-tier administrative accounts on lower-tier systems, you prevent accidental or unauthorized exposure of high-privilege credentials.

Legacy Impact & Compatibility * **Administration Access**: Technicians cannot log on to workstations or member servers using Domain Admin accounts. They must use dedicated administrative credentials assigned to that specific tier (e.g., 'a2-florian' for workstation support) or use local administrator accounts managed by LAPS. * **Remote Support Tools**: Automated support tools or scripts that authenticate using Domain Admin accounts to query local client registry hives or modify system files will be blocked. These tools must be reconfigured to authenticate using dedicated service accounts or Tier 1/2 administrative accounts.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

To prevent Tier 0 credentials from being exposed on Tier 1 and Tier 2 systems, configure the logon restrictions using GPOs linked to the Tier 1 (Member Servers) and Tier 2 (Workstations) OUs.

1. Configure Tier 1 Logon Restrictions GPO 1. Open the **Group Policy Management Console** ('gpmc.msc'). 2. Create a GPO linked to the **Tier 1 Member Servers** OU (e.g., 'GPO_Restrictions_Tier1'). 3. Navigate to: 'Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment' 4. Define and configure the following policies: * **Deny access to this computer from the network**: Add 'Domain Admins', 'Enterprise Admins', 'Schema Admins'. * **Deny log on as a batch job**: Add 'Domain Admins', 'Enterprise Admins', 'Schema Admins'. * **Deny log on as a service**: Add 'Domain Admins', 'Enterprise Admins', 'Schema Admins'. * **Deny log on locally**: Add 'Domain Admins', 'Enterprise Admins', 'Schema Admins'. * **Deny log on through Remote Desktop Services**: Add 'Domain Admins', 'Enterprise Admins', 'Schema Admins'.

2. Configure Tier 2 Logon Restrictions GPO 1. Create a GPO linked to the **Tier 2 Workstations** OU (e.g., 'GPO_Restrictions_Tier2'). 2. Navigate to the same path: 'Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment' 3. Configure the policies to deny access for **both** Tier 0 and Tier 1 administrative groups: * **Deny access to this computer from the network**: Add Tier 0 groups ('Domain Admins', 'Enterprise Admins', 'Schema Admins') and Tier 1 administrative groups. * **Deny log on as a batch job**: Add Tier 0 and Tier 1 administrative groups. * **Deny log on as a service**: Add Tier 0 and Tier 1 administrative groups. * **Deny log on locally**: Add Tier 0 and Tier 1 administrative groups. * **Deny log on through Remote Desktop Services**: Add Tier 0 and Tier 1 administrative groups.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this script to configure local security database files via `secedit` to deny specified administrative groups from logging on locally, over the network, or via Remote Desktop.

[Download Script: Set-LocalLogonRestrictions.ps1](#)

```

# Set-LocalLogonRestrictions.ps1
# Configures secedit User Rights Assignment to block domain administrative groups.

# 1. Define groups to block (Tier 0 admin groups)
$DenyGroups = @("Domain Admins", "Enterprise Admins", "Schema Admins")

# 2. Translate group names to SID strings
$SIDs = foreach ($group in $DenyGroups) {
    try {
        $sid = (New-Object System.Security.Principal.NTAccount($group)).Translate([System.Security.Principal.SecurityIdentifier]).Value
        "$sid"
    } catch {
        Write-Warning "Could not resolve SID for group: $group. Skipping."
    }
}

if ($SIDs.Count -eq 0) {
    Write-Error "No valid group SIDs resolved. Exiting."
    exit 1
}

# 3. Define the temporary paths for secedit config
$TempDir = [System.IO.Path]::GetTempPath()
$SecConfigPath = Join-Path $TempDir "sec_config.inf"
$SecDbPath = Join-Path $TempDir "sec_db.sdb"

# 4. Export current local security policy
secedit /export /cfg $SecConfigPath /areas USER_RIGHTS /quiet

# 5. Modify exported config to inject our deny rules
$configContent = Get-Content -Path $SecConfigPath
$newContent = [System.Collections.Generic.List[string]]::new()
$inUserRights = $false

$PolicyEntries = @(
    "SeDenyInteractiveLogonRight",
    "SeDenyNetworkLogonRight",
    "SeDenyRemoteInteractiveLogonRight"
)

foreach ($line in $configContent) {
    if ($line -match '^\[Privilege Rights\]\s\{' {
        $inUserRights = $true
        $newContent.Add($line)
        continue
    }
    if ($inUserRights -and $line -match '^\s\{' {
        $inUserRights = $false
    }

    # Filter out existing lines for the policies we configure
    $matched = $false
    foreach ($entry in $PolicyEntries) {
        if ($line -match "^\s*$entry\s*=") {
            $matched = $true
            break
        }
    }

    if (-not $matched) {
        $newContent.Add($line)
    }
}

# Insert new rules into [Privilege Rights] section
$insertIndex = $newContent.FindIndex({ $args[0] -match '^\[Privilege Rights\]\s\{' })
if ($insertIndex -ge 0) {
    $offset = 1
    $formattedSIDs = $SIDs -join ","
    foreach ($policy in $PolicyEntries) {
        $newContent.Insert($insertIndex + $offset, "$policy = $formattedSIDs")
        $offset++
    }
}

```

```

}

# Save new configuration
$NewContent | Out-File -FilePath $SecConfigPath -Encoding utf16

# 6. Apply configuration using secedit
Write-Host "Applying User Rights Assignment restrictions via secedit..." -ForegroundColor Cyan
$process = Start-Process secedit -ArgumentList "/configure /db $SecDbPath /cfg $SecConfigPath /areas USER_RIGHTS /quiet" -Wait -NoNe
wWindow -PassThru

# Cleanup temporary database files
if (Test-Path $SecConfigPath) { Remove-Item $SecConfigPath -Force }
if (Test-Path $SecDbPath) { Remove-Item $SecDbPath -Force }

if ($process.ExitCode -eq 0) {
    Write-Host "Logon restrictions applied successfully." -ForegroundColor Green
} else {
    Write-Error "Failed to apply logon restrictions. Exit code: $($process.ExitCode)"
}

```

To audit active logon restriction settings locally: [Download Script: Test-LocalLogonRestrictions.ps1](#)

```

# Test-LocalLogonRestrictions.ps1
# Audits local security policies to check if SeDeny rights are populated.

Write-Host "--- Auditing Local User Rights Assignments ---" -ForegroundColor Cyan

$tempDir = [System.IO.Path]::GetTempPath()
$SecConfigPath = Join-Path $tempDir "sec_audit.inf"

# Export current configuration
secedit /export /cfg $SecConfigPath /areas USER_RIGHTS /quiet

$configContent = Get-Content -Path $SecConfigPath
$PoliciesToTest = @(
    "SeDenyInteractiveLogonRight",
    "SeDenyNetworkLogonRight",
    "SeDenyRemoteInteractiveLogonRight"
)

foreach ($policy in $PoliciesToTest) {
    $match = $configContent | Where-Object { $_ -match "^$policy\s*=" }
    if ($match) {
        # Check if it contains domain admin or other groups
        Write-Host "    - Policy '$policy' is configured: $match" -ForegroundColor Green
    } else {
        Write-Host "    - VULNERABLE: Policy '$policy' is not defined (No accounts denied)." -ForegroundColor Red
    }
}

if (Test-Path $SecConfigPath) { Remove-Item $SecConfigPath -Force }

```

📄 Sources & Compliance References * **ANSSI AD Hardening Guide***: Recommendations R1, R2, and R3 (Administrative isolation and account restriction rules) * **CIS Microsoft Windows Server 2016 Benchmark***: Section 18.2 (User Rights Assignment)

[REQ-ARCH-002] Restrict Administrative Management Protocols

Target Scope * **Applicable Systems**: Tier 0 assets (Domain Controllers, Jump Hosts) and Tier 1 member servers. * **Operating Systems**: Windows Server 2016 (and above), Windows 10 (and above) Enterprise/Professional.

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Windows Defender Firewall with Advanced Security

Rationale Remote management protocols like Remote Desktop Protocol (RDP) and Windows Remote Management (WinRM) are critical interfaces for directory administration. However, if these protocols are accessible from any host in the network, they become high-value targets for attackers.

If an attacker compromises a standard user workstation (Tier 2), they can run network scanners to identify all machines listening on port 3389 (RDP) or 5985/5986 (WinRM). They can then attempt password spraying or locate open administrative sessions.

Restricting administrative protocols at the network and local firewall levels to allow inbound connections *only* from designated administrative jump hosts or PAW IP ranges stops lateral movement and brute-force attempts from standard user networks.

Legacy Impact & Compatibility * **Administration Location**: Administrators cannot manage Domain Controllers or member servers from standard client subnets. They must be physically connected to the dedicated administrative subnet or log on from a designated PAW. * **Network Planning**: Any changes to administrative IP address ranges require updating GPO firewall rules. If IP assignments are incorrect, administrators can be locked out of remote management interfaces.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`).
2. Create or edit the GPO linked to the **Domain Controllers** OU (e.g., `GP0_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Windows Defender Firewall with Advanced Security`
4. Expand **Inbound Rules**.
5. Locate the rule **Remote Desktop - User Mode (TCP-In)** (or create a custom inbound port rule for TCP port 3389).
6. Double-click the rule, navigate to the **Scope** tab, and configure:
 - **Remote IP address**: Click **These IP addresses** and enter the specific IP range of the Tier 0 Jump Hosts and PAWs (e.g., `10.10.0.0/24`). Do not allow "Any IP address".
7. Locate the rule **Windows Remote Management (HTTP-In)** (TCP port 5985/5986).
8. Double-click the rule, navigate to the **Scope** tab, and enter the same administrative IP range under **Remote IP address**.
9. Enforce blocking of any other inbound connections by ensuring the default firewall action for the profile is set to block inbound connections that do not match an allow rule.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally on a Domain Controller or member server to create Windows Defender Firewall rules restricting WinRM and RDP to authorized subnets.

[Download Script: Set-AdminProtocolRestrictions.ps1](#)

```
# Set-AdminProtocolRestrictions.ps1
# Creates inbound firewall rules to restrict RDP and WinRM to designated management subnets.

Write-Host "--- Restricting Administrative Protocols Inbound ---" -ForegroundColor Cyan

# Define the authorized administrative network subnet
$AdminSubnet = "10.10.0.0/24" # Replace with your PAW / Jump Host subnet

# 1. Restrict RDP (TCP port 3389) Inbound
$RdpRule = Get-NetFirewallRule -DisplayName "Remote Desktop - User Mode (TCP-In)" -ErrorAction SilentlyContinue
if ($RdpRule) {
    Write-Host "[+] Restricting existing RDP firewall rule to admin subnet..." -ForegroundColor Gray
    Set-NetFirewallRule -DisplayName "Remote Desktop - User Mode (TCP-In)" -RemoteAddress $AdminSubnet
    Write-Host "    RDP rule restricted." -ForegroundColor Green
} else {
    Write-Host "[+] Creating new restricted RDP inbound rule..." -ForegroundColor Gray
    New-NetFirewallRule -DisplayName "Hardening: Restricted RDP Inbound" `
        -Direction Inbound `
        -Action Allow `
        -Protocol TCP `
        -LocalPort 3389 `
        -RemoteAddress $AdminSubnet `
        -Enabled True | Out-Null
    Write-Host "    Restricted RDP rule created." -ForegroundColor Green
}

# 2. Restrict WinRM HTTPS (TCP port 5986) Inbound
$WinRMRule = Get-NetFirewallRule -DisplayName "Windows Remote Management (HTTPS-In)" -ErrorAction SilentlyContinue
if ($WinRMRule) {
    Write-Host "[+] Restricting existing WinRM HTTPS rule to admin subnet..." -ForegroundColor Gray
    Set-NetFirewallRule -DisplayName "Windows Remote Management (HTTPS-In)" -RemoteAddress $AdminSubnet
    Write-Host "    WinRM rule restricted." -ForegroundColor Green
} else {
    Write-Host "[+] Creating new restricted WinRM HTTPS inbound rule..." -ForegroundColor Gray
    New-NetFirewallRule -DisplayName "Hardening: Restricted WinRM HTTPS Inbound" `
        -Direction Inbound `
        -Action Allow `
        -Protocol TCP `
        -LocalPort 5986 `
        -RemoteAddress $AdminSubnet `
        -Enabled True | Out-Null
    Write-Host "    Restricted WinRM rule created." -ForegroundColor Green
}
}
```

To audit the firewall restrictions: [Download Script: Test-AdminProtocolRestrictions.ps1](#)

```
# Test-AdminProtocolRestrictions.ps1
# Audits local firewall rules for RDP and WinRM to check remote address restrictions.

Write-Host "--- Auditing Administrative Port Firewall Rules ---" -ForegroundColor Cyan

$Rules = @(
    "Remote Desktop - User Mode (TCP-In)",
    "Windows Remote Management (HTTPS-In)",
    "Hardening: Restricted RDP Inbound",
    "Hardening: Restricted WinRM HTTPS Inbound"
)

foreach ($RuleName in $Rules) {
    $Rule = Get-NetFirewallRule -DisplayName $RuleName -ErrorAction SilentlyContinue
    if ($Rule) {
        $Address = Get-NetFirewallAddressFilter -AssociatedNetFirewallRule $Rule
        $color = if ($Address.RemoteAddress -ne "Any" -and $Address.RemoteAddress -ne "") { "Green" } else { "Red" }
        Write-Host "    - Firewall Rule: $($Rule.DisplayName) | Enabled: $($Rule.Enabled) | RemoteAddress Restriction: $($Address.RemoteAddress)" -ForegroundColor $color
    }
}
}
```

 Sources & Compliance References * **ANSI AD Hardening Guide**: Recommendation R8 (Dedicated management subnets and jump hosts) * **CIS Microsoft Windows Server 2016 Benchmark**: Section 19 (Windows Defender Firewall)

[REQ-ARCH-003] Audit Privileged Groups

Target Scope * **Applicable Systems**: Active Directory Domain. * **Operating Systems**: Windows Server 2016 (and above) Domain Controllers.

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * Active Directory Directory Service Changes Auditing (GPO: Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies\DS Access)

Rationale Attackers who gain initial access to an Active Directory domain attempt to elevate their privileges to Tier 0. A primary method for establishing domain persistence is adding compromised domain accounts to highly privileged administrative groups, such as 'Domain Admins', 'Enterprise Admins', 'Schema Admins', or 'Builtin\Administrators'.
If these groups are not audited continuously:

1. **Backdoor Persistence:** Attackers can add temporary users to administrative groups and remove them later, leaving backdoor accounts with administrative authority that go unnoticed.
2. **Privilege Creep:** Unmanaged administrative accounts accumulate over time, violating the principle of least privilege.
3. **Delegation Risks:** Administrative accounts can inherit unintended administrative rights if nested within other groups.

Enforcing advanced auditing on directory object changes, combined with a daily script to monitor privileged group members, ensures immediate visibility into unauthorized modifications.

Legacy Impact & Compatibility * **Auditing Load**: Directory Service changes logging (Event ID 5136) can generate a high volume of events on Domain Controllers in large environments. Ensure event logs have appropriate size allocations and logs are shipped to a local SIEM. *
Operational Workflows: Administrators must use standard change management procedures when adding or removing accounts from administrative groups to avoid generating false-positive security alerts.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

To generate events when administrative group membership is altered:

1. Open the **Group Policy Management Console** (`gpmc.msc`).
2. Create or edit the GPO linked to the **Domain Controllers** OU (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies\DS Access`
4. Configure the setting:
 - **Policy:** `Audit Directory Service Changes`
 - **Setting:** `Enabled` (Select `Success` and `Failure`)

This ensures that Event ID **5136** is logged in the Security log of the Domain Controller whenever an Active Directory object attribute (such as a group's `member` attribute) is changed.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run this script from a secure administrative workstation to audit nested and direct memberships in critical Tier 0 groups, highlighting any unexpected accounts.

[Download Script: Audit-ADAdminGroups.ps1](#)

```
# Audit-ADAdminGroups.ps1
# Queries memberships of privileged Tier 0 AD groups recursively.

Import-Module ActiveDirectory

$Tier0Groups = @(
    "Domain Admins",
    "Enterprise Admins",
    "Schema Admins",
    "Administrators",
    "Account Operators",
    "Server Operators",
    "Backup Operators"
)

Write-Host "--- Auditing Privileged AD Groups ---" -ForegroundColor Cyan

# Define the list of explicitly authorized accounts (e.g. emergency break-glass account)
$AuthorizedUsers = @("Administrator", "a0-breakglass")

foreach ($GroupName in $Tier0Groups) {
    try {
        $Group = Get-ADGroup -Identity $GroupName -ErrorAction Stop
        $Members = Get-ADGroupMember -Identity $GroupName -Recursive

        Write-Host "`n[+] Group: $($Group.Name)" -ForegroundColor Yellow
        if ($Members.Count -eq 0) {
            Write-Host "    No members found." -ForegroundColor Gray
        } else {
            foreach ($Member in $Members) {
                # Highlight unauthorized accounts in red
                if ($AuthorizedUsers -notcontains $Member.SamAccountName -and $Member.SamAccountName -notlike "a0-*") {
                    Write-Host "        - VULNERABLE: Unauthorized user '$($Member.SamAccountName)' in admin group!" -ForegroundColor Red
                } else {
                    Write-Host "        - Member: $($Member.SamAccountName) | Class: $($Member.objectClass)" -ForegroundColor Green
                }
            }
        }
    } catch {
        Write-Warning "Could not query group '$GroupName'. Ensure appropriate permissions."
    }
}
```

To verify that directory auditing is active on the local DC: [Download Script: Test-ADChangesAuditing.ps1](#)

```
# Test-ADChangesAuditing.ps1
# Audits local DC auditpol settings to verify Directory Service auditing is enabled.

Write-Host "--- Auditing Directory Service Audit Policy ---" -ForegroundColor Cyan

$rawOutput = auditpol.exe /get /subcategory:"Directory Service Changes" /r
# Parse CSV output: Machine,Subcategory,GUID,PolicyVal
if ($rawOutput -match "^.+,Directory Service Changes,.(+)\$" ) {
    $policyVal = $Matches[1]
    $color = if ($policyVal -match "Success") { "Green" } else { "Red" }
    Write-Host "    - Directory Service Changes Audit: $policyVal (Required = Success and Failure)" -ForegroundColor $color
} else {
    Write-Warning "    - Status: Could not parse DS auditpol status."
}
```

📄 Sources & Compliance References * **ANSSI AD Hardening Guide**²: Recommendation R57 (Vulnerability assessment of directory services), Section on administrative groups. * **CIS Microsoft Windows Server 2016 Benchmark**³: Section 9.2.1 (Audit Directory Service Access) * **Microsoft Security Baselines**⁴: Domain Controller baseline settings.

[REQ-ARCH-004] Keep Domain and Forest Functional Levels Up-To-Date

Target Scope * **Applicable Systems**: Domain Controllers (Forest-wide configuration) * **Operating Systems**: Windows Server 2016 and above

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: Active Directory Domains and Trusts (AD Schema modification)

Rationale The Domain Functional Level (DFL) and Forest Functional Level (FFL) dictate the capabilities of the Active Directory (AD) infrastructure. While upgrading the Operating System of Domain Controllers (DCs) improves the underlying OS security, the AD logic remains constrained by the functional level. Maintaining an obsolete DFL forces Domain Controllers to emulate legacy behaviors and protocols to ensure backward compatibility with non-existent older DCs. This emulation effectively creates a ceiling for your security posture, preventing the activation of modern identity protection mechanisms.

Raising the functional level is critical for hardening because it unlocks architectural security changes that mitigate credential theft and lateral movement. Specifically, higher functional levels are prerequisites for:

1. **Protected Users Group**: Mandates restrictions that prevent caching of NTLM credentials, disable Kerberos DES/RC4 keys, and prevent caching of plaintext passwords.
2. **Group Managed Service Accounts (gMSAs)**: Eliminates static service account passwords by automating 120-character key rotation.
3. **Kerberos Armoring (FAST)**: Protects Kerberos tickets and exchanges against offline dictionary attacks and ticket manipulation.
4. **Deprecating Legacy Replication**: Disables insecure File Replication Service (FRS) in favor of DFS Replication (DFSR).

Legacy Impact & Compatibility * **Irreversibility**: Raising the domain and forest functional levels is generally irreversible. Once raised to Windows Server 2016, you cannot demote the functional levels back to any previous Server version. * **Domain Controller Compatibility**: Any older Domain Controller running Windows Server 2012 R2 or earlier will fail to replicate and will be blocked from functioning. All Domain Controllers must run an OS version equal to or higher than the target functional level before it is raised. * **Testing Phase**: Prior to raising the functional level, administrators must inventory all Domain Controllers and decommission any legacy systems.

Implementation Steps

Option A: Active Directory Domains and Trusts (Preferred)

1. Log on to a Domain Controller or a management workstation with **Enterprise Admins** credentials.
2. Open the **Active Directory Domains and Trusts** console (`domain.msc`).
3. To raise the Domain Functional Level:
 - In the console tree, right-click the domain for which you want to raise the functional level, and then click **Raise Domain Functional Level**.
 - Select **Windows Server 2016** (or higher depending on your environment) from the list of available levels, and then click **Raise**.
 - Read the warning dialog and click **OK** to confirm.
4. To raise the Forest Functional Level:
 - In the console tree, right-click **Active Directory Domains and Trusts**, and then click **Raise Forest Functional Level**.
 - Select **Windows Server 2016** (or higher depending on your environment) from the list of available levels, and then click **Raise**.
 - Read the warning dialog and click **OK** to confirm.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts to audit and raise the functional levels.

1. Local Audit (Audit-ADFunctionalLevels.ps1)

[Download Script: Audit-ADFunctionalLevels.ps1](#)

```
# Audit-ADFunctionalLevels.ps1
# Description: Audits the current domain and forest functional levels.

Import-Module ActiveDirectory

Write-Host "--- Auditing Active Directory Functional Levels ---" -ForegroundColor Cyan

$Domain = Get-ADDomain -ErrorAction SilentlyContinue
$Forest = Get-ADForest -ErrorAction SilentlyContinue

if ($Domain -and $Forest) {
    $DomainMode = $Domain.DomainMode
    $ForestMode = $Forest.ForestMode

    $MinDomainMode = [Microsoft.ActiveDirectory.Management.ADDomainMode]::Windows2016Domain
    $MinForestMode = [Microsoft.ActiveDirectory.Management.ADForestMode]::Windows2016Forest

    $DomainCompliant = [int]$DomainMode -ge [int]$MinDomainMode
    $ForestCompliant = [int]$ForestMode -ge [int]$MinForestMode

    if ($DomainCompliant) {
        Write-Host "Status: Domain Functional Level is compliant ($DomainMode)." -ForegroundColor Green
    } else {
        Write-Host "VULNERABLE: Domain Functional Level ($DomainMode) is below Windows Server 2016." -ForegroundColor Red
    }

    if ($ForestCompliant) {
        Write-Host "Status: Forest Functional Level is compliant ($ForestMode)." -ForegroundColor Green
    } else {
        Write-Host "VULNERABLE: Forest Functional Level ($ForestMode) is below Windows Server 2016." -ForegroundColor Red
    }
} else {
    Write-Host "VULNERABLE: Could not retrieve Active Directory settings. Run on a domain-joined machine with AD module installed." -ForegroundColor Red
}
}
```

2. Local Remediation (Set-ADFunctionalLevels.ps1)

[Download Script: Set-ADFunctionalLevels.ps1](#)

```
# Set-ADFunctionalLevels.ps1
# Description: Raises Domain and Forest Functional Levels to Windows Server 2016.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Raise Functional Levels to Windows Server 2016..." -ForegroundColor Cyan

$Domain = Get-ADDomain -ErrorAction SilentlyContinue
$Forest = Get-ADForest -ErrorAction SilentlyContinue

if (-not $Domain -or -not $Forest) {
    Write-Error "Could not retrieve Active Directory settings. Run on a Domain Controller with administrative privileges."
    exit 1
}

# Raise Domain Functional Level
if ($Domain.DomainMode -lt [Microsoft.ActiveDirectory.Management.ADDomainMode]::Windows2016Domain) {
    try {
        Set-ADDomainMode -Identity $Domain.DNSRoot -DomainMode Windows2016Domain -Confirm:$false -ErrorAction Stop
        Write-Host "Domain Functional Level successfully raised to Windows Server 2016." -ForegroundColor Green
    } catch {
        Write-Error "Failed to raise Domain Functional Level. Error: $($_.Exception.Message)"
    }
} else {
    Write-Host "Domain Functional Level is already Windows Server 2016 or higher." -ForegroundColor Green
}

# Raise Forest Functional Level
if ($Forest.ForestMode -lt [Microsoft.ActiveDirectory.Management.ADForestMode]::Windows2016Forest) {
    try {
        Set-ADForestMode -Identity $Forest.Name -ForestMode Windows2016Forest -Confirm:$false -ErrorAction Stop
        Write-Host "Forest Functional Level successfully raised to Windows Server 2016." -ForegroundColor Green
    } catch {
        Write-Error "Failed to raise Forest Functional Level. Error: $($_.Exception.Message)"
    }
} else {
    Write-Host "Forest Functional Level is already Windows Server 2016 or higher." -ForegroundColor Green
}
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**: Section on raising Forest and Domain functional levels. * **Microsoft Security Guidance**: Active Directory Forest and Domain Functional Levels Overview.

[REQ-ARCH-005] Default Domain and Domain Controllers Policies Management

Target Scope * **Applicable Systems** : Domain Controllers and Domain Members (Forest-wide) * **Operating Systems** : Windows Server 2016 and above

Implementation Details * **Priority** : High * **GPO Path / Registry Location** : * **GPO Paths** : * `Default Domain Policy` (GUID: `{31B2F340-016D-11D2-945F-00C04FB984F9}`) * `Default Domain Controllers Policy` (GUID: `{6AC1786C-016F-11D2-945F-00C04FB984F9}`) * **EFS path** : `Computer Configuration\Policies\Windows Settings\Security Settings\Public Key Policies\Encrypting File System` * **Background Refresh path** : `Computer Configuration\Policies\Administrative Templates\System\Group Policy` * **Registry Locations** : * **EFS** : `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\CurrentVersion\EFS` * `EfsConfiguration` = `1` (REG_DWORD, 1 = Disabled) * **Background Refresh** : `HKLM\SOFTWARE\Policies\Microsoft\Windows\System` * `DisableBkGndGroupPolicy` = `0` (REG_DWORD, 0 = Active/Enabled)

Rationale Modifying the Default Domain Policy (DDP) and Default Domain Controllers Policy (DDCP) introduces significant operational risks. These default policies define the core directory baseline configurations required for Active Directory to initialize, replicate, and authenticate. However, a critical exception to modular Group Policy design applies to **Account Policies** (Password, Account Lockout, and Kerberos settings) and **Encrypting File System (EFS)** policies. Windows operating systems only process domain-wide Account Policies from the GPO linked directly to the domain root (by default, the Default Domain Policy). Custom GPOs linked at lower levels containing these settings will be ignored for domain accounts. Therefore, these specific baselines must be configured directly within the Default Domain Policy itself.

All other custom hardening settings (such as local User Rights Assignments, Audit Policies, and registry parameters) should be managed via dedicated modular GPOs (e.g., [SEC_DomainControllers_Hardening](#)) linked at higher precedence.

Integrating the following controls inside the DDP/DDCP and custom GPOs completes the Active Directory management baseline:

1. **Disabling Encrypting File System (EFS)**: EFS allows users to encrypt files on local drives. This makes files difficult to recover or back up securely. Enterprise environments should enforce full-disk encryption (BitLocker) rather than user-managed file-level encryption. Disabling EFS prevents unauthorized user-level encryption.
2. **Enforcing Group Policy Background Refresh**: Ensuring that Group Policy background refresh is active prevents unauthorized local overrides from persisting. Setting the policy [Turn off background refresh of Group Policy](#) to **Disabled** ensures that GPOs are reapplied every 90 minutes.
3. **Mandating GPO Comments**: Documenting the purpose, author, and revision history in the GPO comments field ensures accountability and prevents configuration drift.
4. **Restricting gpupdate /force Overuse**: Running `gpupdate /force` causes endpoints and servers to re-download all applied GPOs from Domain Controllers. In large environments, this can trigger severe network congestion and CPU spikes on DCs. Administrators should use standard `gpupdate` without the `/force` switch unless a full re-application of unchanged policies is required.

Legacy Impact & Compatibility * **EFS Disabling** : Prior to disabling EFS, administrators must scan domain-joined machines for active EFS-encrypted files and decrypt them. If EFS is disabled while encrypted files exist, users will lose access to those files. * **GPO Precedence** : The modular hardening GPOs must be linked at the root/OU with a lower link order number (higher precedence) than the default policies to ensure custom settings override defaults cleanly.

Implementation Steps

Option A: Group Policy Management Console (GPMC) (Preferred)

Step 1: Configure Default Domain Policy for Account and EFS Policies 1. Log on to a management workstation or Domain Controller with **Domain Admins** credentials. 2. Open the **Group Policy Management Console** (`gpmc.msc`). 3. Right-click the **Default Domain Policy** and select **Edit**. 4. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Public Key Policies`. 5. Right-click **Encrypting File System** and select **Properties**. 6. In the **General** tab, under **File encryption using Encrypting File System (EFS)**, select **Disabled** and click **OK**. 7. Navigate to the Account Policies node to configure domain-wide Password and Lockout parameters as detailed in [\[configure-account-policies.md\]\(#08-endpoints-configure-account-policies-md\)](#).

Step 2: Establish Modular GPOs for Non-Account Policies 1. In the `gpmc.msc` console tree, right-click **Group Policy Objects** and select **New**. 2. Name the GPO `SEC_DomainControllers_Hardening` (and `SEC_Domain_Hardening` for domain members) and click **OK**. 3. Right-click the **Domain Controllers** OU (or root domain) and select **Link an Existing GPO**. 4. Select `SEC_DomainControllers_Hardening` and click **OK**. 5. Select the **Domain Controllers** OU in the left pane, navigate to the **Linked Group Policy Objects** tab, select the custom hardening GPO, and use the green up arrow to set its **Link Order** to **1** (highest precedence).

Step 3: Configure Group Policy Background Refresh 1. Edit your modular hardening GPO (e.g., `SEC_DomainControllers_Hardening` or `SEC_Domain_Hardening`). 2. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Group Policy`. 3. Double-click the policy **Turn off background refresh of Group Policy** and set it to **Disabled**.

Step 4: Mandate GPO Comments for Accountability 1. In GPMC, right-click your GPO, select **Properties**, and navigate to the **Comment** tab. 2. Enter a structured comment containing: * **Purpose**: [Brief explanation of GPO controls] * **Author**: [Administrator Name or Security Team] * **Date**: [Creation/Modification Date] * **Reference Requirement**: [e.g., REQ-ARCH-005]

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts to audit and set up the GPO structure, EFS, and background refresh locally.

1. Local Audit (Audit-GPOPrecedence.ps1)

[Download Script: Audit-GPOPrecedence.ps1](#)

```
# Audit-GPOPrecedence.ps1
# Description: Verifies GPO precedence on DC OU, and checks local EFS and background refresh registry configuration.

Import-Module ActiveDirectory
Import-Module GroupPolicy

Write-Host "--- Auditing Default Policies and Precedence ---" -ForegroundColor Cyan

# 1. Audit GPO Precedence on Domain Controllers OU
$DomainInfo = Get-ADDomain
$DCOUDN = "OU=Domain Controllers,$($DomainInfo.DistinguishedName)"

try {
    $OUInfo = Get-GPInheritance -Target $DCOUDN -ErrorAction Stop

    Write-Host "`nLinked GPOs on Domain Controllers OU:" -ForegroundColor Yellow
    $HardeningGPOFound = $false
    $HardeningOrder = 999
    $DefaultDCOrder = 999

    foreach ($link in $OUInfo.GpoLinks) {
        $status = if ($link.Enabled) { "Enabled" } else { "Disabled" }
        Write-Host "    - Link Order: $($link.Order) | GPO Name: $($link.DisplayName) | Status: $status" -ForegroundColor White

        if ($link.DisplayName -like "*Hardening*" -and $link.Enabled) {
            $HardeningGPOFound = $true
            $HardeningOrder = $link.Order
        }
        if ($link.DisplayName -eq "Default Domain Controllers Policy") {
            $DefaultDCOrder = $link.Order
        }
    }

    if ($HardeningGPOFound -and $HardeningOrder -lt $DefaultDCOrder) {
        Write-Host "`n[+] GPO Precedence: Compliant. Custom hardening GPO has higher precedence (Order $HardeningOrder) than Default DC Policy (Order $DefaultDCOrder)." -ForegroundColor Green
    } else {
        Write-Host "`n[!] VULNERABLE: No active dedicated hardening GPO found with higher precedence than Default DC Policy." -ForegroundColor Red
    }
} catch {
    Write-Host "[!] Could not retrieve GPO information for DC OU. Error: $($_.Exception.Message)" -ForegroundColor Red
}

# 2. Audit EFS Registry status
$EfsPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\CurrentVersion\EFS"
$EfsVal = Get-ItemProperty -Path $EfsPath -Name "EfsConfiguration" -ErrorAction SilentlyContinue
if ($EfsVal -and $EfsVal.EfsConfiguration -eq 1) {
    Write-Host "[+] EFS Configuration: Secure (Disabled)." -ForegroundColor Green
} else {
    Write-Host "[!] VULNERABLE: EFS is not disabled in registry policies." -ForegroundColor Red
}

# 3. Audit Background Refresh status
$SysPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System"
$BkgVal = Get-ItemProperty -Path $SysPath -Name "DisableBkgndGroupPolicy" -ErrorAction SilentlyContinue
if ($BkgVal -and $BkgVal.DisableBkgndGroupPolicy -eq 0) {
    Write-Host "[+] Group Policy Background Refresh: Secure (Active)." -ForegroundColor Green
} else {
    Write-Host "[!] VULNERABLE: Group Policy background refresh is turned off in registry." -ForegroundColor Red
}
```

2. Local Remediation (Set-ADModularGPO.ps1)

[Download Script: Set-ADModularGPO.ps1](#)

```
# Set-ADModularGPO.ps1
# Description: Creates DC hardening GPO with top precedence, disables EFS, and enables background refresh locally.

Import-Module ActiveDirectory
Import-Module GroupPolicy

Write-Host "Applying Default Policies hardening baseline..." -ForegroundColor Cyan

# 1. Create and Link DC Hardening GPO
$DomainInfo = Get-ADDomain
$DCOUDN = "OU=Domain Controllers,$($DomainInfo.DistinguishedName)"
$GPOName = "SEC_DomainControllers_Hardening"

try {
    $GPO = Get-GPO -Name $GPOName -ErrorAction SilentlyContinue
    if (-not $GPO) {
        $GPO = New-GPO -Name $GPOName -Comment "Dedicated GPO for Domain Controllers hardening. Requirements: REQ-ARCH-005." -ErrorAction Stop
        Write-Host "[+] GPO '$GPOName' created successfully." -ForegroundColor Green
    } else {
        Write-Host "[+] GPO '$GPOName' already exists." -ForegroundColor Yellow
    }

    $Links = (Get-GPInheritance -Target $DCOUDN).GpoLinks
    $IsLinked = $false
    foreach ($link in $Links) {
        if ($link.DisplayName -eq $GPOName) {
            $IsLinked = $true
            break
        }
    }

    if (-not $IsLinked) {
        New-GPLink -Name $GPOName -Target $DCOUDN -LinkEnabled Yes -ErrorAction Stop | Out-Null
        Write-Host "[+] GPO '$GPOName' linked to Domain Controllers OU." -ForegroundColor Green
    } else {
        Write-Host "[+] GPO '$GPOName' is already linked to Domain Controllers OU." -ForegroundColor Yellow
    }

    Set-GPLink -Name $GPOName -Target $DCOUDN -Order 1 -ErrorAction Stop | Out-Null
    Write-Host "[+] GPO '$GPOName' set to link order 1 (highest precedence)." -ForegroundColor Green
} catch {
    Write-Host "[!] Failed to configure GPO structure. Error: $($_.Exception.Message)" -ForegroundColor Red
}

# 2. Configure Local Registry for EFS (Disable)
$EfsPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\CurrentVersion\EFS"
if (-not (Test-Path $EfsPath)) {
    New-Item -Path $EfsPath -Force | Out-Null
}
Set-ItemProperty -Path $EfsPath -Name "EfsConfiguration" -Value 1 -Type DWord -Force
Write-Host "[+] EFS registry policy configured to Disabled." -ForegroundColor Green

# 3. Configure Local Registry for GP Background Refresh (Active)
$SysPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System"
if (-not (Test-Path $SysPath)) {
    New-Item -Path $SysPath -Force | Out-Null
}
Set-ItemProperty -Path $SysPath -Name "DisableBkGndGroupPolicy" -Value 0 -Type DWord -Force
Write-Host "[+] Group Policy background refresh registry policy enabled." -ForegroundColor Green
```

Sources & Compliance References * **CIS Active Directory and Group Policy Management Best Practices**:

Section "Default Domain Policy and Default Domain Controller Policy" (Pages 18, 22-24, 37-38) * **ANSSI AD Hardening Guide**:

Recommendations on directory service configuration management.

[REQ-ARCH-006] Harden Active Directory Domain Trusts

Target Scope * **Applicable Systems**: Domain Controllers * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: Active Directory trust object attributes (`msDS-TrustSettings` / `trustAttributes` on trusted domain objects)

Rationale Active Directory trust relationships permit authentication and resource access across domain or forest boundaries. However, weak trust configurations can serve as transit routes for attackers to compromise trusting domains.

Specifically:

1. **SID History**: Enabling SID History allows users from a trusted forest to present security identifiers (SIDs) from other domains, bypass SID filtering, and potentially impersonate high-privilege administrators in the trusting forest. Disabling SID History on external/forest trusts prevents this path of escalation.
2. **SID Filtering / Quarantine**: Enabling Quarantine (SID Filtering) on external trusts ensures that the trusting domain filters out unauthorized SIDs presented in authorization packets, restricting access only to SIDs originating from the trusted domain itself.
3. **Kerberos TGT Delegation**: If TGT delegation is allowed on inbound trusts, a user authenticating from the trusted forest to a service in the trusting domain can have their Kerberos Ticket Granting Ticket (TGT) delegated to that service. If that service or host is compromised, the attacker can harvest the user's TGT and impersonate them. Blocking TGT delegation is critical to prevent credential exposure.
4. **Selective Authentication**: Enforcing selective authentication restricts cross-forest access, allowing administrators to explicitly define which users/groups from the trusted forest can authenticate to specific resources in the trusting forest.

Legacy Impact & Compatibility * **Operational Disruption**: Disabling SID History may disrupt access for users who migrated from old domains but still rely on legacy SIDs for resource permissions. Ensure that all migrated resources have updated Access Control Lists (ACLs) before disabling SID History. * **Authentication Failure**: Enforcing selective authentication will block all cross-forest access by default until explicit "Allowed to Authenticate" permissions are configured on computer objects for target external security groups.

Implementation Steps

Option A: Active Directory Domains and Trusts GUI Configuration

1. Open **Active Directory Domains and Trusts** (`domain.msc`) on a Domain Controller.
2. Right-click the trusting domain and select **Properties**.
3. Select the **Trusts** tab.
4. Under either **Domains that trust this domain (Inbound)** or **Domains trusted by this domain (Outbound)**, select the target trust and click **Properties**.
5. Configure selective authentication:
 - Select the **Authentication** tab.
 - Change the option from **Forest-wide authentication** to **Selective authentication**.
6. Enforce SID filtering and quarantine rules (usually performed automatically when establishing external trusts).

Option B: PowerShell & netdom Configuration (Remediation / Non-GPO)

Run the following script block to audit and harden trust relationships. The `netdom` utility is used to query and apply the trust settings.

[Download Script: Set-ADTrustHardening.ps1](#)

```
# Set-ADTrustHardening.ps1
# Description: Hardens trust relationships by disabling SID History and TGT Delegation, and enabling Quarantine.

Write-Host "Applying hardening requirement: Harden Active Directory Domain Trusts..." -ForegroundColor Cyan

# Set target trust variables (replace with your domain names)
$TrustingDomain = "corp.local"
$TrustedDomain = "partner.local"

# 1. Disable SID History on Forest/External Trust
Write-Host "Disabling SID History on trust from $($TrustingDomain) to $($TrustedDomain)..." -ForegroundColor White
netdom trust $TrustingDomain /domain:$TrustedDomain /EnableSIDHistory:no

# 2. Enable Quarantine (SID Filtering) on External Domain Trust
Write-Host "Enabling Quarantine on trust from $($TrustingDomain) to $($TrustedDomain)..." -ForegroundColor White
netdom trust $TrustingDomain /domain:$TrustedDomain /Quarantine:yes

# 3. Disable TGT Delegation over Inbound Trust
Write-Host "Disabling Kerberos TGT Delegation on trust from $($TrustingDomain) to $($TrustedDomain)..." -ForegroundColor White
netdom trust $TrustingDomain /domain:$TrustedDomain /EnableTGTDelagation:no

Write-Host "Trust hardening commands executed." -ForegroundColor Green
```

To verify the trust configuration state: [Download Script: Get-ADTrustStatus.ps1](#)

```
# Get-ADTrustStatus.ps1
# Description: Audits trust attributes and configuration settings.

Import-Module ActiveDirectory

Write-Host "--- Auditing Trust Relationships ---" -ForegroundColor Cyan

$Trusts = Get-ADTrust -Filter * -Properties *

if ($Trusts) {
    foreach ($Trust in $Trusts) {
        Write-Host "[+] Trust Name: $($Trust.Name)" -ForegroundColor Green
        Write-Host "    - Trust Type: $($Trust.TrustType)" -ForegroundColor White
        Write-Host "    - Direction: $($Trust.TrustDirection)" -ForegroundColor White
        Write-Host "    - Selective Authentication: $($Trust.SelectiveAuthentication)" -ForegroundColor White
        Write-Host "    - Disallow Transitivity: $($Trust.DisallowTransitivity)" -ForegroundColor White

        # Verify specific settings using netdom query
        Write-Host "    - netdom Configuration Details:" -ForegroundColor White
        netdom trust $Trust.Source /domain:$Trust.Target /Query
    }
} else {
    Write-Host "[-] No trust relationships found in the domain." -ForegroundColor Yellow
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**²⁴: Recommendations R24, R25+, R26 (Section 3.2.3) * **ANSSI Remediation of Active Directory Tier 0 Guide**²⁵: Section 8 (Page 34), Section 4.e (Page 27) * **Microsoft Security Guidance**²⁶: Security Considerations for Trusts

[REQ-ARCH-007] Harden Microsoft Exchange Active Directory Permissions

Target Scope * **Applicable Systems**: Active Directory Domain Controllers and Domain Root Object (Tier 0). * **Operating Systems**: Active Directory Domain Services (All supported functional levels).

Implementation Details * **Priority**: High (Prevents domain-wide privilege escalation from compromised Exchange servers). * **GPO Path / Registry Location**: N/A (Direct Active Directory ACL modification on the domain root object).

Rationale By default, installing Microsoft Exchange Server in an Active Directory forest modifies the permissions of the domain root object. It grants the **Exchange Windows Permissions** group (and sometimes **Exchange Servers**) write permissions ('WriteDACL' and 'WriteOwner') over the domain root container. This configuration presents a critical security risk:

1. **Privilege Escalation:** Any user or service account with administrative control over Exchange, or any compromised Exchange server itself, can write new permissions to the domain root.
2. **DCSync Exploitation:** The attacker can grant their own account the `ds-Replication-Get-Changes` and `ds-Replication-Get-Changes-All` (DCSync) extended rights. This allows the attacker to dump password hashes directly from the domain controller database (NTDS.dit), leading to a complete forest compromise.

Restricting these write permissions ensures that Exchange servers cannot modify domain-level security descriptors.

Legacy Impact & Compatibility * **Exchange Operations**: Restricting 'WriteDACL' on the domain root may prevent Exchange from performing certain automatic administrative operations, such as modifying schema attributes, auto-provisioning objects in default containers, or updating domain-wide permissions during subsequent Exchange cumulative updates (CUs). * **Workaround**: If Exchange database or organization adjustments are required, a member of the Domain Admins or Enterprise Admins group must run Exchange setup with the '/PrepareAD' flag to temporarily apply permissions, or perform the modifications manually.

Implementation Steps

Option A: Active Directory Users & Computers (GUI Configuration)

1. Open **Active Directory Users and Computers** (`dsa.msc`) on a management console with Domain Admin privileges.
2. Enable **Advanced Features** under the **View** menu.
3. Right-click the root domain object (e.g., `domain.local`) and select **Properties**.
4. Select the **Security** tab, then click **Advanced**.
5. Locate the permission entries for **Exchange Windows Permissions** and **Exchange Servers**.
6. Review the permissions:
 - Remove any entries granting **Modify permissions** (`WriteDACL`) or **Modify owner** (`WriteOwner`) on the domain root.
 - Ensure standard read and object creation permissions (such as writing specific user properties for mailbox management) remain unchanged.
7. Click **Apply**, then **OK**.

Option B: PowerShell & Active Directory Configuration (Remediation / Non-GPO)

To automate verification and remediation of the domain root ACL:

[Download Script: Harden-ExchangePermissions.ps1](#)

```
# Harden-ExchangePermissions.ps1
# Description: Removes WriteDacl and WriteOwner permissions for Exchange groups on the domain root.

Import-Module ActiveDirectory

$DomainDN = (Get-ADDomain).DistinguishedName
$DomainPath = "AD:\$DomainDN"

# Retrieve existing ACL
$Acl = Get-Acl -Path $DomainPath
$DangerousAcesToRemove = @()

# Define Exchange groups to target
$TargetGroups = @("Exchange Windows Permissions", "Exchange Servers")
$TargetSids = @()

foreach ($groupName in $TargetGroups) {
    try {
        $sid = (Get-ADGroup -Identity $groupName).SID.Value
        $TargetSids += $sid
    }
    catch {
        Write-Host "Group '$groupName' not found in this domain. Skipping." -ForegroundColor Yellow
    }
}

if ($TargetSids.Count -eq 0) {
    Write-Host "No Exchange groups detected. No permissions to harden." -ForegroundColor Green
    exit 0
}

# Scan ACL for dangerous ACEs
foreach ($ace in $Acl.Access) {
    $identity = $ace.IdentityReference
    try {
        $sid = $identity.Translate([System.Security.Principal.SecurityIdentifier]).Value
    }
    catch {
        continue
    }

    if ($TargetSids -contains $sid) {
        # Check for WriteDacl or WriteOwner rights
        $hasWriteDacl = ($ace.ActiveDirectoryRights -band [System.DirectoryServices.ActiveDirectoryRights]::WriteDacl) -eq [System.DirectoryServices.ActiveDirectoryRights]::WriteDacl
        $hasWriteOwner = ($ace.ActiveDirectoryRights -band [System.DirectoryServices.ActiveDirectoryRights]::WriteOwner) -eq [System.DirectoryServices.ActiveDirectoryRights]::WriteOwner

        if ($hasWriteDacl -or $hasWriteOwner) {
            $DangerousAcesToRemove += $ace
        }
    }
}

if ($DangerousAcesToRemove.Count -eq 0) {
    Write-Host "[+] Domain root ACL is compliant. No dangerous Exchange write permissions found." -ForegroundColor Green
} else {
    Write-Host "[-] Found $($DangerousAcesToRemove.Count) dangerous Exchange permissions. Removing..." -ForegroundColor Yellow
    foreach ($ace in $DangerousAcesToRemove) {
        $Acl.RemoveAccessRule($ace) | Out-Null
    }
    Set-Acl -Path $DomainPath -AclObject $Acl
    Write-Host "[+] Successfully removed dangerous Exchange WriteDacl/WriteOwner permissions from domain root." -ForegroundColor Green
}
}
```

To verify the domain root permissions:

[Download Script: Get-ExchangePermissionsStatus.ps1](#)

```
# Get-ExchangePermissionsStatus.ps1
# Check if Exchange groups hold WriteDacl/WriteOwner permissions on the domain root.

Import-Module ActiveDirectory

$DomainDN = (Get-ADDomain).DistinguishedName
$DomainPath = "AD:\$DomainDN"

$Acl = Get-Acl -Path $DomainPath
$TargetGroups = @("Exchange Windows Permissions", "Exchange Servers")
$TargetSids = @()

foreach ($groupName in $TargetGroups) {
    try {
        $sid = (Get-ADGroup -Identity $groupName).SID.Value
        $TargetSids += $sid
    }
    catch {
        Write-Verbose "Group '$groupName' not found in Active Directory."
    }
}

if ($TargetSids.Count -eq 0) {
    Write-Host "[+] COMPLIANT: Exchange groups are not present in this domain."
    exit 0
}

$nonCompliantAces = 0

foreach ($ace in $Acl.Access) {
    $identity = $ace.IdentityReference
    try {
        $sid = $identity.Translate([System.Security.Principal.SecurityIdentifier]).Value
    }
    catch {
        continue
    }

    if ($TargetSids -contains $sid) {
        $hasWriteDacl = ($ace.ActiveDirectoryRights -band [System.DirectoryServices.ActiveDirectoryRights]::WriteDacl) -eq [System.DirectoryServices.ActiveDirectoryRights]::WriteDacl
        $hasWriteOwner = ($ace.ActiveDirectoryRights -band [System.DirectoryServices.ActiveDirectoryRights]::WriteOwner) -eq [System.DirectoryServices.ActiveDirectoryRights]::WriteOwner

        if ($hasWriteDacl -or $hasWriteOwner) {
            Write-Host "[!] NON-COMPLIANT: Group '$($identity.Value)' has permissions: $($ace.ActiveDirectoryRights)" -ForegroundColor Red
            $nonCompliantAces++
        }
    }
}

if ($nonCompliantAces -eq 0) {
    Write-Host "[+] COMPLIANT: No Exchange groups have WriteDacl or WriteOwner permissions on the domain root." -ForegroundColor Green
    exit 0
} else {
    Write-Host "[!] NON-COMPLIANT: Dangerous Exchange write permissions detected on the domain root." -ForegroundColor Red
    exit 1
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**: Recommendation R13 (Restricting permissions on domain objects) * **CIS Benchmark**: Section 1.2 (Active Directory Permissions Audit) * **PingCastle Rule**: 'P-ExchangePrivEsc' (Ensure that Exchange did not introduce security vulnerabilities)

Module 2: Domain Controller Hardening

This directory contains security baselines for Domain Controllers running Windows Server 2016 and above in high-security, air-gapped Active Directory environments.

Technical Hardening Controls

- [REQ-DC-001 - Disable SMBv1](#) Requirement to disable the legacy SMBv1 protocol and its associated client-side driver to prevent remote code execution and spoofing vulnerabilities.
- [REQ-DC-002 - Disable Multicast Name Resolution](#) Requirement to disable LLMNR, NetBIOS (NBT-NS), and mDNS to prevent local name resolution spoofing and credential harvesting.
- [REQ-DC-003 - Disable NTLMv1](#) Requirement to restrict NTLM authentication to NTLMv2 or Kerberos to protect credentials from offline brute-force cracking.
- [REQ-DC-004 - Enforce LDAP Server Signing](#) Requirement to enforce packet signing on LDAP cleartext traffic to protect directory transactions from man-in-the-middle attacks.
- [REQ-DC-005 - Enforce LDAP Channel Binding](#) Requirement to enforce LDAP Channel Binding Tokens (CBT) over secure LDAPS connections to prevent authentication relay attacks.
- [REQ-DC-006 - Enable LSA Protection](#) Requirement to configure the Local Security Authority (LSA) process to run as a Protected Process Light (PPL) to protect credential secrets from LSASS memory dumps.
- [REQ-DC-007 - Disable Credential Guard](#) Requirement to disable Windows Defender Credential Guard on Domain Controllers in accordance with Microsoft recommendations while keeping Virtualization-Based Security (VBS) enabled.
- [REQ-DC-008 - Disable Print Spooler Service](#) Requirement to stop and disable the Print Spooler service on Domain Controllers to prevent remote execution and coercive authentication attacks.
- [REQ-DC-009 - Enforce SMB Message Signing](#) Requirement to enforce SMB client and server signing to protect file transfer data and block SMB relay attacks.
- [REQ-DC-010 - Restrict Kerberos Encryption Types](#) Requirement to configure allowed Kerberos encryption types, restricting to AES128/AES256 and disabling legacy DES and RC4 to prevent Kerberoasting.
- [REQ-DC-011 - Restrict Remote SAM API Access](#) Requirement to restrict remote RPC access to the SAM database to local Administrators, preventing remote recon and user enumeration.
- [REQ-DC-012 - Disable Unnecessary Services on Domain Controllers](#) Requirement to disable unnecessary system services (such as Xbox services and other non-essential services) on Domain Controllers to minimize the attack surface.
 - [REQ-DC-035 - Disable Xbox Live Auth Manager \(XblAuthManager\)](#)
 - [REQ-DC-036 - Disable Xbox Live Game Save \(XblGameSave\)](#)
 - [REQ-DC-037 - Disable ActiveX Installer \(AxInstSV\)](#)
 - [REQ-DC-038 - Disable Bluetooth Support Service \(bthserv\)](#)
 - [REQ-DC-039 - Disable Connected Devices Platform User Service \(CDPUserSvc\)](#)
 - [REQ-DC-040 - Disable Contact Data \(PimIndexMaintenanceSvc\)](#)
 - [REQ-DC-041 - Disable WAP Push Message Routing Service \(dmwappushservice\)](#)
 - [REQ-DC-042 - Disable Downloaded Maps Manager \(MapsBroker\)](#)
 - [REQ-DC-043 - Disable Geolocation Service \(Ifsvc\)](#)
 - [REQ-DC-044 - Disable Internet Connection Sharing \(ICS\) \(SharedAccess\)](#)
 - [REQ-DC-045 - Disable Link-Layer Topology Discovery Mapper \(lltdsvc\)](#)
 - [REQ-DC-046 - Disable Microsoft Account Sign-in Assistant \(wlidsvc\)](#)
 - [REQ-DC-047 - Disable Microsoft Passport \(NgcSvc\)](#)
 - [REQ-DC-048 - Disable Microsoft Passport Container \(NgcCtnrSvc\)](#)
 - [REQ-DC-049 - Disable Network Connection Broker \(NcbService\)](#)
 - [REQ-DC-050 - Disable Phone Service \(PhoneSvc\)](#)
 - [REQ-DC-051 - Disable Printer Extensions and Notifications \(PrintNotify\)](#)
 - [REQ-DC-052 - Disable Program Compatibility Assistant Service \(PcaSvc\)](#)
 - [REQ-DC-053 - Disable Quality Windows Audio Video Experience \(QWAVE\)](#)
 - [REQ-DC-054 - Disable Radio Management Service \(RmSvc\)](#)
 - [REQ-DC-055 - Disable Sensor Data Service \(SensorDataService\)](#)
 - [REQ-DC-056 - Disable Sensor Monitoring Service \(SensrSvc\)](#)
 - [REQ-DC-057 - Disable Sensor Service \(SensorService\)](#)
 - [REQ-DC-058 - Disable Shell Hardware Detection \(ShellHWDetection\)](#)
 - [REQ-DC-059 - Disable Smart Card Device Enumeration Service \(ScDeviceEnum\)](#)

- [REQ-DC-060 - Disable SSDP Discovery \(SSDPSRV\)](#)
- [REQ-DC-061 - Disable Still Image Acquisition Events \(WiaRpc\)](#)
- [REQ-DC-062 - Disable Sync Host \(OneSyncSvc\)](#)
- [REQ-DC-063 - Disable UPnP Device Host \(upnphost\)](#)
- [REQ-DC-064 - Disable User Data Access \(UserDataSvc\)](#)
- [REQ-DC-065 - Disable User Data Storage \(UnistoreSvc\)](#)
- [REQ-DC-066 - Disable WalletService \(WalletService\)](#)
- [REQ-DC-067 - Disable Windows Audio \(Audiosrv\)](#)
- [REQ-DC-068 - Disable Windows Audio Endpoint Builder \(AudioEndpointBuilder\)](#)
- [REQ-DC-069 - Disable Windows Camera Frame Server \(FrameServer\)](#)
- [REQ-DC-070 - Disable Windows Image Acquisition \(WIA\) \(stisvc\)](#)
- [REQ-DC-071 - Disable Windows Insider Service \(wisvc\)](#)
- [REQ-DC-072 - Disable Windows Mobile Hotspot Service \(icssvc\)](#)
- [REQ-DC-073 - Disable Windows Push Notifications System Service \(WpnService\)](#)
- [REQ-DC-074 - Disable Windows Push Notifications User Service \(WpnUserService\)](#)
- [REQ-DC-013 - Enable Kerberos Armoring](#) Requirement to enable Kerberos Armoring (FAST) on Domain Controllers and client endpoints to encrypt pre-authentication exchanges and protect credentials from offline brute-force attacks.
- [REQ-DC-014 - Restrict NTLM](#) Requirement to audit and restrict NTLMv2 and domain-wide NTLM authentication to prevent credential relaying and force the transition to Kerberos.
- [REQ-DC-015 - Migrate SYSVOL Replication to DFSR](#) Requirement to migrate SYSVOL folder replication from legacy FRS to secure DFSR to ensure replication integrity and disable deprecated services.
- [REQ-DC-016 - Harden adminSDHolder Permissions](#) Requirement to secure the adminSDHolder object's Access Control List to prevent privilege escalation backdoors on protected accounts.
- [REQ-DC-017 - Harden Microsoft DNS AD Container Permissions](#) Requirement to secure CN=MicrosoftDNS,CN=System container permissions and block DNS service DLL hijacking (ServerLevelPluginDll).
- [REQ-DC-018 - Harden Virtualization Hosts for Domain Controllers](#) Requirement to treat virtualization hypervisors hosting Domain Controllers as Tier 0 systems, separating host hardware and enforcing VM encryption.
- [REQ-DC-019 - Enforce RDP Restricted Admin Mode](#) Requirement to configure and require RDP Restricted Admin Mode on administrative clients and servers to protect credentials in host memory.
- [REQ-DC-020 - Windows Defender Antivirus Domain Controller Baseline and Exploit Guard](#) Requirement to configure and harden Windows Defender Antivirus on Domain Controllers, enabling real-time scanning, preventing local exclusion modifications, enforcing server-compatible ASR rules (including LSASS protection), activating Tamper Protection, and sandboxing execution.
 - [REQ-DC-075 - Disable Real-Time Monitoring and Behavior Monitoring Override on Domain Controllers](#)
 - [REQ-DC-076 - Configure Potentially Unwanted Applications \(PUA\) Protection on Domain Controllers](#)
 - [REQ-DC-077 - Prevent Local List Merging and Exclusions Configuration on Domain Controllers](#)
 - [REQ-DC-078 - Configure Auto Exclusions Configuration on Domain Controllers](#)
 - [REQ-DC-079 - Prevent MAPS Local Setting Override on Domain Controllers](#)
 - [REQ-DC-080 - Enable EDR in Block Mode on Domain Controllers](#)
 - [REQ-DC-081 - Allow Network Protection on Windows Server on Domain Controllers](#)
 - [REQ-DC-082 - Enable File Hash Computation on Domain Controllers](#)
 - [REQ-DC-083 - Configure Network Inspection System \(NIS\) settings on Domain Controllers](#)
 - [REQ-DC-084 - Configure OOB Real-Time Protection and Security Intelligence on Domain Controllers](#)
 - [REQ-DC-085 - Enable Dynamic Signature Dropped Event Reporting on Domain Controllers](#)
 - [REQ-DC-086 - Disable Generic Reports on Domain Controllers](#)
 - [REQ-DC-087 - Configure Behavioral Network Brute Force Protection Aggressiveness on Domain Controllers](#)
 - [REQ-DC-088 - Configure Behavioral Network Remote Encryption Protection Aggressiveness on Domain Controllers](#)
 - [REQ-DC-089 - Configure Quick Scan and Scanning Exclusions on Domain Controllers](#)
 - [REQ-DC-090 - Configure Scheduled Scan Parameters on Domain Controllers](#)
 - [REQ-DC-091 - Configure Security Intelligence Update Schedule on Domain Controllers](#)
 - [REQ-DC-092 - Configure Attack Surface Reduction Rules on Domain Controllers](#)
 - [REQ-DC-098 - ASR: Block abuse of exploited vulnerable signed drivers on Domain Controllers](#)
 - [REQ-DC-099 - ASR: Block credential stealing from the Windows local security authority subsystem on Domain Controllers](#)

- [REQ-DC-100 - ASR: Block execution of potentially obfuscated scripts on Domain Controllers](#)
- [REQ-DC-101 - ASR: Block persistence through WMI event subscription on Domain Controllers](#)
- [REQ-DC-102 - ASR: Block process creations originating from PSEXEC and WMI commands on Domain Controllers](#)
- [REQ-DC-103 - ASR: Use advanced protection against ransomware on Domain Controllers](#)
- [REQ-DC-093 - Configure Threat Severity Default Quarantine Actions on Domain Controllers](#)
- [REQ-DC-094 - Configure Family Options UI Lockdown on Domain Controllers](#)
- [REQ-DC-095 - Configure Tamper Protection on Domain Controllers](#)
- [REQ-DC-096 - Configure Sandbox Execution Environment on Domain Controllers](#)
- [REQ-DC-097 - Configure AMSI Authenticode Signature Verification on Domain Controllers](#)
- [REQ-DC-021 - Configure AppLocker Policies on Domain Controllers](#) Requirement to configure strict AppLocker rules on Domain Controllers to prevent administrative users from executing unapproved binaries, scripts, installers, or web browsers on Tier 0 systems.
- [REQ-DC-022 - Enable WDAC Driver Blocklist](#) Requirement to configure the Windows Defender Application Control (WDAC) driver blocklist to protect kernel memory from Bring Your Own Vulnerable Driver (BYOVD) attacks.
- [REQ-DC-023 - Configure User Rights Assignments for Domain Controllers](#) Requirement to restrict local user rights assignments on Domain Controllers to prevent default operator groups (Print Operators, Server Operators, Backup Operators) from logging on locally, backing up/restoring files, or shutting down Domain Controllers.
 - [REQ-DC-104 - Configure User Rights: Access this computer from the network on Domain Controllers](#)
 - [REQ-DC-105 - Configure User Rights: Act as part of the operating system on Domain Controllers](#)
 - [REQ-DC-106 - Configure User Rights: Add workstations to domain on Domain Controllers](#)
 - [REQ-DC-107 - Configure User Rights: Adjust memory quotas for a process on Domain Controllers](#)
 - [REQ-DC-108 - Configure User Rights: Allow log on locally on Domain Controllers](#)
 - [REQ-DC-109 - Configure User Rights: Allow log on through Remote Desktop Services on Domain Controllers](#)
 - [REQ-DC-110 - Configure User Rights: Back up files and directories on Domain Controllers](#)
 - [REQ-DC-111 - Configure User Rights: Bypass traverse checking on Domain Controllers](#)
 - [REQ-DC-112 - Configure User Rights: Change the system time on Domain Controllers](#)
 - [REQ-DC-113 - Configure User Rights: Create a pagefile on Domain Controllers](#)
 - [REQ-DC-114 - Configure User Rights: Create a token object on Domain Controllers](#)
 - [REQ-DC-115 - Configure User Rights: Create permanent shared objects on Domain Controllers](#)
 - [REQ-DC-116 - Configure User Rights: Debug programs on Domain Controllers](#)
 - [REQ-DC-117 - Configure User Rights: Deny access to this computer from the network on Domain Controllers](#)
 - [REQ-DC-118 - Configure User Rights: Deny log on as a batch job on Domain Controllers](#)
 - [REQ-DC-119 - Configure User Rights: Deny log on as a service on Domain Controllers](#)
 - [REQ-DC-120 - Configure User Rights: Deny log on locally on Domain Controllers](#)
 - [REQ-DC-121 - Configure User Rights: Deny log on through Remote Desktop Services on Domain Controllers](#)
 - [REQ-DC-122 - Configure User Rights: Enable computer and user accounts to be trusted for delegation on Domain Controllers](#)
 - [REQ-DC-123 - Configure User Rights: Force shutdown from a remote system on Domain Controllers](#)
 - [REQ-DC-124 - Configure User Rights: Generate security audits on Domain Controllers](#)
 - [REQ-DC-125 - Configure User Rights: Load and unload device drivers on Domain Controllers](#)
 - [REQ-DC-126 - Configure User Rights: Lock pages in memory on Domain Controllers](#)
 - [REQ-DC-127 - Configure User Rights: Log on as a batch job on Domain Controllers](#)
 - [REQ-DC-128 - Configure User Rights: Log on as a service on Domain Controllers](#)
 - [REQ-DC-129 - Configure User Rights: Manage auditing and security log on Domain Controllers](#)
 - [REQ-DC-130 - Configure User Rights: Modify firmware environment values on Domain Controllers](#)
 - [REQ-DC-131 - Configure User Rights: Profile single process on Domain Controllers](#)
 - [REQ-DC-132 - Configure User Rights: Restore files and directories on Domain Controllers](#)
 - [REQ-DC-133 - Configure User Rights: Shut down the system on Domain Controllers](#)
 - [REQ-DC-134 - Configure User Rights: Synchronize directory service data on Domain Controllers](#)
 - [REQ-DC-135 - Configure User Rights: Take ownership of files or other objects on Domain Controllers](#)
- [REQ-DC-024 - Configure dSHeuristics](#) Requirement to audit and configure the dSHeuristics forest-wide attribute to reach maximum Level 5 security, blocking anonymous LDAP and NSPI operations, securing adminSDHolder, and enforcing KB5008383 owner implicit rights

protections.

- [REQ-DC-025 - Configure Security Options for Domain Controllers](#) Requirement to configure baseline administrative template Security Options, disabling anonymous access to SAM/shares and enforcing credential policies.
- [REQ-DC-026 - Configure TCP/IP and Network Parameter Hardening for Domain Controllers](#) Requirement to configure hardened network configurations, TCP/IP MSS parameters, disabling LLTDIO/RSPNDR drivers, Peer-to-Peer, and Windows Connect Now.
- [REQ-DC-027 - Configure Telemetry, Diagnostics and Privacy Options for Domain Controllers](#) Requirement to restrict telemetry collection, online diagnostics, advertising IDs, diagnostic tools, and cloud content integration.
- [REQ-DC-028 - Configure Untrusted Font Blocking for Domain Controllers](#) Requirement to configure the Untrusted Font Blocking mitigation on Domain Controllers to prevent kernel font parser exploits.
- [REQ-DC-029 - Configure svchost.exe Mitigation Options](#) Requirement to configure svchost.exe mitigation options on Domain Controllers and Member Servers to restrict binary loading to Microsoft-signed code and block dynamic code execution.
- [REQ-DC-030 - Secure Directory Services Restore Mode \(DSRM\) and Recovery Parameters](#) Requirement to secure DSRM restore mode logon behavior and recovery credentials parameters.
- [REQ-DC-031 - Configure NTP Time Synchronization on the PDC Emulator](#) Requirement to configure NTP time synchronization on the PDC Emulator to serve as a reliable time source and secure Kerberos exchanges.
- [REQ-DC-032 - Enable UEFI Secure Boot](#) Requirement to enforce hardware-rooted platform integrity checks, verifying that UEFI Secure Boot is active on Domain Controllers.
- [REQ-DC-033 - Configure Secure Boot Revocations and Bootloader Updates](#) Requirement to configure and enforce BlackLotus revocation updates and bootloader integrity verification policy variables in system firmware.
- [REQ-DC-034 - Configure Windows Defender Application Control](#) Requirement to deploy Windows Defender Application Control (WDAC) on Domain Controllers in Audit Mode to block unauthorized system-level binaries and scripts.
- [REQ-DC-135 - Configure Advanced Security Audit Policies for Domain Controllers](#) Enforces granular Windows security audit policies (including logons, group memberships, directory service modifications, and object access) to log critical threat telemetry on Domain Controllers.
 - [REQ-DC-136 - Audit Policy: Advanced Audit Policy Overrides](#)
 - [REQ-DC-137 - Audit Policy: Account Logon Auditing](#)
 - [REQ-DC-138 - Audit Policy: Account Management Auditing](#)
 - [REQ-DC-139 - Audit Policy: Detailed Tracking Auditing](#)
 - [REQ-DC-140 - Audit Policy: Directory Service Access Auditing](#)
 - [REQ-DC-141 - Audit Policy: Logon and Logoff Auditing](#)
 - [REQ-DC-142 - Audit Policy: Object Access Auditing](#)
 - [REQ-DC-143 - Audit Policy: Policy Change Auditing](#)
 - [REQ-DC-144 - Audit Policy: Privilege Use Auditing](#)
 - [REQ-DC-145 - Audit Policy: System Events Auditing](#)

[REQ-DC-001] Disable SMBv1

Target Scope * **Applicable Systems**: Domain Controllers, Member Servers, Tier 2 Clients * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10, Windows 11

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **Disable SMBv1 Server**: * Path: `Computer Configuration\Preferences\Windows Settings\Registry` * Key Path: `SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters` * Value Name: `SMB1` * Value Type: `REG\_DWORD` * Value Data: `0` * **Disable SMBv1 Client**: * Path: `Computer Configuration\Preferences\Windows Settings\Registry` * Key Path: `SYSTEM\CurrentControlSet\Services\mrxsmb10` * Value Name: `Start` * Value Type: `REG\_DWORD` * Value Data: `4`

Rationale SMBv1 (Server Message Block version 1) is a legacy networking protocol designed over 30 years ago. It lacks modern security features such as packet encryption, cryptographic signing enforcement, and robust integrity verification. SMBv1 contains severe remote code execution (RCE) vulnerabilities (e.g., the EternalBlue vulnerability mitigated in MS17-010). Attackers can exploit SMBv1 to execute commands remotely, capture credentials, or perform lateral movement across the domain network. Domain Controllers (Tier 0) are highly sensitive targets, and keeping SMBv1 enabled presents an unacceptable risk. Disabling both the server protocol and the client driver eliminates this entire attack surface.

Legacy Impact & Compatibility * **Authentication Failures**: Disabling SMBv1 breaks network sharing and file transfer with outdated systems, including Windows XP, Windows Server 2003, early Linux distributions using old Samba versions, older network-attached storage (NAS) appliances, and legacy printers or scanners. * **Pre-requisite Audit**: In legacy environments, network administrators should audit SMBv1 usage using Windows Event Logs (Microsoft-Windows-SMBServer/Analytic log) before disabling the protocol. If legacy clients exist, they must be upgraded or isolated.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Configure Group Policy Preferences to enforce the registry settings:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host.
2. Edit the appropriate hardening GPO (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference to disable the SMBv1 Server (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters`
 - **Value name**: `SMB1`
 - **Value type**: `REG_DWORD`
 - **Value data**: `0`
5. Create a second Registry Preference to disable the SMBv1 Client Driver (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\mrxsmb10`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`
6. Link the GPO to the appropriate Organizational Unit (OU) containing the target assets.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally or if the control is not manageable via standard GPO GUI interfaces.

[Download Script: Configure-DisableSMBv1.ps1](#)

```
# Configure-DisableSMBv1.ps1
# Description: Disables SMBv1 server protocol and mrxsmbl0 client driver.

Write-Host "Applying hardening requirement: Disable SMBv1..." -ForegroundColor Cyan

# 1. Disable SMBv1 Server
$srvRegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters"
if (-not (Test-Path $srvRegPath)) {
    New-Item -Path $srvRegPath -Force | Out-Null
}
Set-ItemProperty -Path $srvRegPath -Name "SMB1" -Value 0 -Type DWord
Write-Host "SMBv1 Server registry configuration applied." -ForegroundColor Green

# Use standard cmdlet if available
if (Get-Command -Name Set-SmbServerConfiguration -ErrorAction SilentlyContinue) {
    Set-SmbServerConfiguration -EnableSMB1Protocol $false -Force
    Write-Host "SMBv1 Server protocol disabled via cmdlet." -ForegroundColor Green
}

# 2. Disable SMBv1 Client Driver
$clientRegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\mrxsmbl0"
if (Test-Path $clientRegPath) {
    Set-ItemProperty -Path $clientRegPath -Name "Start" -Value 4 -Type DWord
    Write-Host "SMBv1 Client mrxsmbl0 driver disabled." -ForegroundColor Green
} else {
    Write-Host "mrxsmbl0 driver registry key not found (may already be removed)." -ForegroundColor Yellow
}

Write-Host "Hardening applied successfully. A system reboot is required." -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-SMBv1Status.ps1](#)

```
# Get-SMBv1Status.ps1
# Description: Audits the registry configuration of SMBv1 server and client components.

Write-Host "--- Auditing SMBv1 Configuration ---" -ForegroundColor Cyan
$Vulnerable = $false

# Check Server configuration
if (Get-Command -Name Get-SmbServerConfiguration -ErrorAction SilentlyContinue) {
    $SmbConfig = Get-SmbServerConfiguration
    if ($SmbConfig.EnableSMB1Protocol -eq $true) {
        Write-Host "[!] VULNERABLE: SMBv1 Server protocol is enabled via configuration." -ForegroundColor Red
        $Vulnerable = $true
    } else {
        Write-Host "[+] SMBv1 Server protocol is disabled." -ForegroundColor Green
    }
} else {
    $SrvReg = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters" -Name "SMB1" -ErrorAction SilentlyContinue
    if ($SrvReg -and $SrvReg.SMB1 -eq 1) {
        Write-Host "[!] VULNERABLE: SMB1 registry parameter is set to 1 (Enabled)." -ForegroundColor Red
        $Vulnerable = $true
    } else {
        Write-Host "[+] SMB1 registry parameter is disabled or not present." -ForegroundColor Green
    }
}

# Check Client configuration
$DriverReg = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\mrxsmb10" -Name "Start" -ErrorAction SilentlyContinue
if ($DriverReg -and $DriverReg.Start -ne 4) {
    Write-Host "[!] VULNERABLE: mrxsmb10 client driver is not disabled (Start value: $($DriverReg.Start))." -ForegroundColor Red
    $Vulnerable = $true
} else {
    Write-Host "[+] mrxsmb10 client driver is disabled." -ForegroundColor Green
}

if ($Vulnerable) {
    Write-Host "Audit result: VULNERABLE" -ForegroundColor Red
} else {
    Write-Host "Audit result: SECURE" -ForegroundColor Green
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**: Recommendation R13 (Disabling obsolete and insecure protocols) *
 CIS Benchmark: CIS Microsoft Windows Server Benchmark - Section 18.9.72 (Ensure 'Configure SMBv1 client driver' is set to 'Enabled: Disable driver') * **Microsoft Security Baseline**: Domain Controller / Member Server Baseline configurations

[REQ-DC-002] Disable Multicast Name Resolution

Target Scope * **Applicable Systems**: Domain Controllers, Member Servers, Tier 2 Clients * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10, Windows 11

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **Disable LLMNR**: * Path: `Computer Configuration\Policies\Administrative Templates\Network\DNS Client` * Policy: `Turn off multicast name resolution` * Setting: `Enabled` * Registry: `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\DNSClient` → `EnableMulticast` = `0` (REG_DWORD) * **Disable mDNS**: * Path: `Computer Configuration\Preferences\Windows Settings\Registry` * Key Path: `SYSTEM\CurrentControlSet\Services\Dnscache\Parameters` * Value Name: `EnableMDNS` * Value Type: `REG\_DWORD` * Value Data: `0` * **Disable NetBIOS (NBT-NS)**: * GPO Path: `Computer Configuration\Policies\Administrative Templates\Network\DNS Client` * Policy: `Configure NetBIOS settings` * Setting: `Enabled: Disable NetBIOS name resolution on public networks` (Backed by Registry: `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\DNSClient` → `EnableNetbios` = `2` [REG_DWORD]) * GPO Preference Path: `Computer Configuration\Preferences\Windows Settings\Registry` * Key Path: `SYSTEM\CurrentControlSet\Services\NetBT\Parameters\Interfaces` * Value Name: `NetbiosOptions` * Value Type: `REG\_DWORD` * Value Data: `2` (Disables NetBIOS over TCP/IP on adapters)

Rationale Link-Local Multicast Name Resolution (LLMNR), NetBIOS Name Service (NBT-NS), and multicast DNS (mDNS) are fallback name resolution protocols. When a Windows host is unable to resolve a name via standard DNS, it broadcasts the query to the local subnet using these protocols.

Adversaries on the same subnet can easily sniff these broadcast/multicast queries and respond with their own IP address (using tools such as Responder). When the requesting client attempts to authenticate to the fake host, its NTLM credentials (specifically NTLMv2 hashes) are captured by the attacker. These hashes can then be cracked offline or relayed to other hosts on the network (e.g., Active Directory Certificate Services or SMB servers) to achieve unauthorized administrative access. Disabling these legacy fallback protocols eliminates these name resolution spoofing attack vectors.

Legacy Impact & Compatibility * **DNS Dependency**: Once these protocols are disabled, hosts rely entirely on DNS. If DNS registration, DNS suffixes, or DNS servers are misconfigured, systems may fail to resolve names of adjacent local network resources. * **Legacy System Impact**: Legacy applications or operating systems that do not use DNS for local hostname resolution will lose the ability to locate resources. A robust DNS infrastructure with dynamic registration enabled is a pre-requisite.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Configure Group Policy to disable LLMNR, and Group Policy Preferences to disable NetBIOS and mDNS:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host.
2. Edit the appropriate hardening GPO (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\Network\DNS Client`
4. Configure the following policies:
 - **Policy:** `Turn off multicast name resolution`
 - **Setting:** `Enabled`
 - **Policy:** `Configure NetBIOS settings`
 - **Setting:** `Enabled: Disable NetBIOS name resolution on public networks` (Note: Requires newer Windows 11 / Server 2022 ADMX templates)
5. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
6. Create a Registry Preference to disable mDNS (Right-click **Registry** → **New** → **Registry Item**):
 - **Action:** `Update`
 - **Hive:** `HKEY_LOCAL_MACHINE`
 - **Key Path:** `SYSTEM\CurrentControlSet\Services\Dnscache\Parameters`
 - **Value name:** `EnableMDNS`
 - **Value type:** `REG_DWORD`
 - **Value data:** `0`
7. For NetBIOS, because adapter GUIDs vary, Group Policy Preferences can be configured with target registry keys, but NetBIOS disabling is often handled dynamically via DHCP scope options (Option 046 or by setting NetBIOS options via DHCP) or locally via scripting on server endpoints. Alternatively, create Registry Preferences for common interfaces or apply Option B locally.
8. Link the GPO to the appropriate Organizational Unit (OU) containing the target assets.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the settings locally.

[Download Script: Configure-DisableMulticastNameResolution.ps1](#)

```
# Configure-DisableMulticastNameResolution.ps1
# Description: Disables LLmnr, NetBIOS over TCP/IP, and mDNS on all interfaces.

Write-Host "Applying hardening requirement: Disable Multicast Name Resolution..." -ForegroundColor Cyan

# 1. Disable LLmnr
$LLmnrPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\DnsClient"
if (-not (Test-Path $LLmnrPath)) {
    New-Item -Path $LLmnrPath -Force | Out-Null
}
Set-ItemProperty -Path $LLmnrPath -Name "EnableMulticast" -Value 0 -Type DWord
Write-Host "LLMNR disabled via registry policy." -ForegroundColor Green

# 2. Disable mDNS
$mdnsPath = "HKLM:\SYSTEM\CurrentControlSet\Services\Dnscache\Parameters"
if (-not (Test-Path $mdnsPath)) {
    New-Item -Path $mdnsPath -Force | Out-Null
}
Set-ItemProperty -Path $mdnsPath -Name "EnableMDNS" -Value 0 -Type DWord
Write-Host "mDNS disabled via registry." -ForegroundColor Green

# 3. Disable NetBIOS over TCP/IP on all Network Adapters
$interfacesPath = "HKLM:\SYSTEM\CurrentControlSet\Services\NetBT\Parameters\Interfaces"
if (Test-Path $interfacesPath) {
    $interfaces = Get-ChildItem -Path $interfacesPath
    foreach ($interface in $interfaces) {
        $intName = $interface.PSChildName
        $intPath = "$($interfacesPath)\($intName)"
        Set-ItemProperty -Path $intPath -Name "NetbiosOptions" -Value 2 -Type DWord
        Write-Host " NetBIOS disabled on interface: $($intName)" -ForegroundColor Gray
    }
    Write-Host "NetBIOS over TCP/IP disabled on all active interfaces." -ForegroundColor Green
} else {
    Write-Host "NetBIOS interfaces registry path not found." -ForegroundColor Yellow
}

Write-Host "Hardening applied successfully." -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-MulticastNameResolutionStatus.ps1](#)

```
# Get-MulticastNameResolutionStatus.ps1
# Description: Audits LLMNR, NetBIOS, and mDNS registry settings.

Write-Host "--- Auditing Multicast Name Resolution ---" -ForegroundColor Cyan
$Vulnerable = $false

# 1. Audit LLMNR
$LLMnrReg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\DNSClient" -Name "EnableMulticast" -ErrorAction SilentlyContinue
if ($LLMnrReg) {
    if ($LLMnrReg.EnableMulticast -eq 1) {
        Write-Host "[!] VULNERABLE: LLMNR is explicitly enabled." -ForegroundColor Red
        $Vulnerable = $true
    } else {
        Write-Host "[+] LLMNR is disabled." -ForegroundColor Green
    }
} else {
    Write-Host "[!] VULNERABLE: LLMNR policy key 'EnableMulticast' does not exist (default is enabled)." -ForegroundColor Red
    $Vulnerable = $true
}

# 2. Audit mDNS
$mdnsReg = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Dnscache\Parameters" -Name "EnableMDNS" -ErrorAction SilentlyContinue
if ($mdnsReg) {
    if ($mdnsReg.EnableMDNS -ne 0) {
        Write-Host "[!] VULNERABLE: mDNS is enabled." -ForegroundColor Red
        $Vulnerable = $true
    } else {
        Write-Host "[+] mDNS is disabled." -ForegroundColor Green
    }
} else {
    # Default is enabled on Windows Server 2022 / Windows 11
    Write-Host "[!] VULNERABLE: mDNS key 'EnableMDNS' is missing (default is enabled)." -ForegroundColor Red
    $Vulnerable = $true
}

# 3. Audit NetBIOS over TCP/IP
$interfacesPath = "HKLM:\SYSTEM\CurrentControlSet\Services\NetBT\Parameters\Interfaces"
if (Test-Path $interfacesPath) {
    $interfaces = Get-ChildItem -Path $interfacesPath
    $netbiosEnabledCount = 0
    foreach ($interface in $interfaces) {
        $intName = $interface.PSChildName
        $intPath = "$($interfacesPath)\$($intName)"
        $optVal = Get-ItemProperty -Path $intPath -Name "NetbiosOptions" -ErrorAction SilentlyContinue
        if ($optVal) {
            if ($optVal.NetbiosOptions -ne 2) {
                Write-Host "[!] VULNERABLE: NetBIOS is enabled/default on interface: $($intName) (NetbiosOptions = $($optVal.NetbiosOptions))" -ForegroundColor Red
                $netbiosEnabledCount = $netbiosEnabledCount + 1
            }
        } else {
            Write-Host "[!] VULNERABLE: NetbiosOptions value missing (default enabled) on interface: $($intName)" -ForegroundColor Red
            $netbiosEnabledCount = $netbiosEnabledCount + 1
        }
    }

    if ($netbiosEnabledCount -gt 0) {
        Write-Host "[!] VULNERABLE: NetBIOS over TCP/IP is active on $($netbiosEnabledCount) interface(s)." -ForegroundColor Red
        $Vulnerable = $true
    } else {
        Write-Host "[+] NetBIOS over TCP/IP is disabled on all interfaces." -ForegroundColor Green
    }
}

if ($Vulnerable) {
    Write-Host "Audit result: VULNERABLE" -ForegroundColor Red
} else {
    Write-Host "Audit result: SECURE" -ForegroundColor Green
}
}
```

Sources & Compliance References * **ANSI AD Hardening Guide**: Recommendation R13 (Disabling obsolete and insecure protocols) *
CIS Benchmark: CIS Microsoft Windows Server Benchmark - Section 18.8.19.1.1 (Ensure 'Turn off multicast name resolution' is set to
'Enabled') * **Microsoft Security Guidance**: Recommendations for disabling NetBIOS on network adapters

[REQ-DC-003] Disable NTLMv1

Target Scope * **Applicable Systems**: Domain Controllers, Member Servers, Tier 2 Clients * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10, Windows 11

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` * **Policy**: `Network security: LAN Manager authentication level` * **Setting**: `Send NTLMv2 response only. Refuse LM & NTLM` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Control\Lsa` → `LmCompatibilityLevel` = `5` (REG_DWORD)

Rationale NTLMv1 (NT LAN Manager version 1) is a legacy authentication protocol that relies on weak cryptographic primitives (specifically MD4 and DES). Because of these mathematical weaknesses, an attacker who intercepts NTLMv1 network authentication traffic can decrypt the responses offline in a matter of minutes, recovering the user's plaintext password or NT hash.

By configuring the system to send only NTLMv2 responses and refuse both LM and NTLMv1 negotiations (corresponding to LAN Manager Compatibility Level 5), the system enforces the use of NTLMv2, which implements HMAC-MD5 and provides significantly stronger protection against offline cryptographic analysis. It also ensures that the directory environment moves closer to Kerberos-exclusive authentication.

Legacy Impact & Compatibility * **Legacy Clients**: Operating systems older than Windows 7 / Windows Server 2008 R2, along with outdated non-Windows systems (such as older Samba integrations, legacy network scanners, or ancient printers), might not support NTLMv2. Disabling NTLMv1 will block these systems from authenticating. * **Pre-remediation Audit**: Before applying this control, administrators should monitor NTLMv1 authentications by checking the Windows Security Log (Event ID 4624, checking the authentication package details) or enabling NTLM auditing via GPO (`Network security: Restrict NTLM: Audit NTLM authentication in this domain`).

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host.
2. Edit the appropriate hardening GPO (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options`
4. Configure the following setting:
 - **Policy**: `Network security: LAN Manager authentication level`
 - **Setting**: `Send NTLMv2 response only. Refuse LM & NTLM`
5. Link the GPO to the appropriate Organizational Unit (OU) containing the target assets.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally.

[Download Script: Configure-DisableNTLMv1.ps1](#)

```
# Configure-DisableNTLMv1.ps1
# Description: Restricts NTLM authentication to NTLMv2 and refuses NTLMv1 / LM.

Write-Host "Applying hardening requirement: Disable NTLMv1..." -ForegroundColor Cyan

$regPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"
if (-not (Test-Path $regPath)) {
    New-Item -Path $regPath -Force | Out-Null
}

Set-ItemProperty -Path $regPath -Name "LmCompatibilityLevel" -Value 5 -Type DWord
Write-Host "LM Compatibility Level set to 5 (Send NTLMv2 response only. Refuse LM & NTLM)." -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-NTLMv1Status.ps1](#)

```
# Get-NTLMV1Status.ps1
# Description: Audits the LM Compatibility Level setting in the registry.

Write-Host "--- Auditing NTLMv1 Restriction ---" -ForegroundColor Cyan

$regPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"
$LsaReg = Get-ItemProperty -Path $regPath -Name "LmCompatibilityLevel" -ErrorAction SilentlyContinue

if ($LsaReg) {
    $lmVal = $LsaReg.LmCompatibilityLevel
    if ($lmVal -eq 5) {
        Write-Host "[+] NTLMv1 is disabled. LM Compatibility Level is set to $($lmVal) (Secure)." -ForegroundColor Green
    } else {
        Write-Host "[!] VULNERABLE: LM Compatibility Level is set to $($lmVal) (Required: 5)." -ForegroundColor Red
    }
} else {
    Write-Host "[!] VULNERABLE: LmCompatibilityLevel key is missing. System is using the default value (allows NTLMv1)." -ForegroundColor Red
}
```

Sources & Compliance References * **ANSI AD Hardening Guide**: Recommendation R13 (Disabling obsolete and insecure protocols) *
 CIS Benchmark: CIS Microsoft Windows Server Benchmark - Section 2.3.7.4 (Ensure 'Network security: LAN Manager authentication level' is set to 'Send NTLMv2 response only. Refuse LM & NTLM') * **Microsoft Security Guidance**: LAN Manager Authentication Level configurations

[REQ-DC-004] Enforce LDAP Server Signing

Target Scope * **Applicable Systems**: Domain Controllers * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` * **Policy**: `Domain controller: LDAP server signing requirements` * **Setting**: `Require signing` * **Registry Location**: `HKLM\System\CurrentControlSet\Services\NTDS\Parameters` → `LDAPServerIntegrity` = `2` (REG_DWORD)

Rationale Lightweight Directory Access Protocol (LDAP) traffic transmitted over cleartext (TCP port 389) without signing is vulnerable to eavesdropping and man-in-the-middle (MitM) attacks. An adversary in a position to intercept network traffic can inject malicious payload packets, modify directory responses, or perform session hijacking.

Enforcing LDAP signing ensures that the LDAP server (Domain Controller) rejects simple binds that are not encrypted or signed. It mandates data integrity verification via cryptographically secure signatures on the network packets. This directly mitigates the threat of LDAP relay and injection attacks, securing the communication path between directory clients and Domain Controllers.

Legacy Impact & Compatibility * **Client Compatibility**: Enforcing LDAP signing on Domain Controllers requires that all clients connecting via cleartext LDAP support and negotiate signing (e.g., SASL GSS-API). Simple LDAP binds that pass passwords in cleartext without SSL/TLS or signing will fail. * **Non-Windows Systems**: Many third-party integrations, older Linux/Unix servers (running older SSSD or PAM LDAP modules), network appliances (e.g., printers, scanners, firewalls), and custom legacy applications do not support LDAP signing. These devices must be updated to support signing or reconfigured to use secure LDAP (LDAPS) over port 636. * **Audit Phase**: Prior to enforcement, check directory service event logs (Event ID 2887 is logged every 24 hours indicating how many unsigned and cleartext LDAP binds occurred; Event ID 2889 can be enabled to identify the specific IP addresses and account names of clients performing unsigned binds).

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host.
2. Edit the GPO linked to the Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options`
4. Configure the following setting:
 - **Policy**: `Domain controller: LDAP server signing requirements`
 - **Setting**: `Require signing`
5. Link the GPO to the Domain Controllers OU.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally.

[Download Script: Configure-LDAPSigning.ps1](#)

```
# Configure-LDAPSigning.ps1
# Description: Configures the LDAP server signing requirement to Require Signing.

Write-Host "Applying hardening requirement: Enforce LDAP Server Signing..." -ForegroundColor Cyan

$regPath = "HKLM:\System\CurrentControlSet\Services\NTDS\Parameters"
if (-not (Test-Path $regPath)) {
    New-Item -Path $regPath -Force | Out-Null
}

Set-ItemProperty -Path $regPath -Name "LDAPServerIntegrity" -Value 2 -Type DWord
Write-Host "LDAP Server Integrity set to 2 (Require Signing)." -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-LDAPSigningStatus.ps1](#)

```
# Get-LDAPSigningStatus.ps1
# Description: Audits the LDAP server signing configuration in the registry.

Write-Host "--- Auditing LDAP Server Signing ---" -ForegroundColor Cyan

$regPath = "HKLM:\System\CurrentControlSet\Services\NTDS\Parameters"
$ntdsReg = Get-ItemProperty -Path $regPath -Name "LDAPServerIntegrity" -ErrorAction SilentlyContinue

if ($ntdsReg) {
    $integrityVal = $ntdsReg.LDAPServerIntegrity
    if ($integrityVal -eq 2) {
        Write-Host "[+] LDAP Server Signing is secure. LDAPServerIntegrity is set to $($integrityVal) (Require Signing)." -Foregroun
dColor Green
    } else {
        Write-Host "[!] VULNERABLE: LDAPServerIntegrity is set to $($integrityVal) (Required: 2)." -ForegroundColor Red
    }
} else {
    Write-Host "[!] VULNERABLE: LDAPServerIntegrity registry value is missing. The system uses default negotiation (allows unsigned
connections)." -ForegroundColor Red
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**: Recommendation R19 (LDAP Signing) * **CIS Benchmark**: CIS Microsoft Windows Server Benchmark - Section 2.3.3.1 (Ensure 'Domain controller: LDAP server signing requirements' is set to 'Require signing') * **Microsoft Security Guidance**: Active Directory LDAP signing requirements

[REQ-DC-005] Enforce LDAP Channel Binding

Target Scope * **Applicable Systems**: Domain Controllers * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` * **Policy**: `Domain controller: LDAP server channel binding token requirements` * **Setting**: `Always` * **Registry Location**:
`HKLM\System\CurrentControlSet\Services\NTDS\Parameters` → `LdapEnforceChannelBinding` = `2` (REG_DWORD)

Rationale Adversaries use credential relay attacks (such as NTLM relaying) to intercept authentication challenges and replay them to other network services. In a coercion attack (e.g., the PetitPotam technique), an attacker forces a Domain Controller to authenticate to a malicious listener using NTLM. The attacker then relays these credentials to Active Directory Certificate Services (ADCS) or an LDAPS server to issue administrative certificates or modify directory databases.

LDAP Channel Binding Tokens (CBT) mitigate these relay attacks. CBT establishes a cryptographic link between the transport-level security channel (TLS/SSL) and the application-level authentication protocol (SASL/NTLM/Kerberos). By requiring CBT verification on the LDAP server, the Domain Controller verifies that the authentication request originated from within the specific TLS channel used to send it. If an attacker attempts to relay credentials from a different session, the channel parameters will not match, and the Domain Controller will reject the authentication request.

Legacy Impact & Compatibility * **Client and Software Compatibility**: Enforcing Channel Binding requirements to `Always` (level 2) means that any LDAP client connecting over SSL/TLS (LDAPS on port 636 or LDAP startTLS on port 389) must support and submit CBT. Older Windows clients without security patches, third-party LDAP integration clients, Java-based applications, or Linux/Unix systems using outdated LDAP packages may fail to bind if they do not support channel binding. * **Pre-requisite Patching**: Ensure that all clients and Domain Controllers have the latest security updates installed. * **Audit and Phased Deployment**: Administrators can configure `LdapEnforceChannelBinding` to `1` (When Supported) during a testing phase. Event logs should be reviewed (specifically Directory Service Event ID 3039, which indicates client compatibility failures) before enforcing the setting to `2` (Always).

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host.
2. Edit the GPO linked to the Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options`
4. Configure the following setting:
 - **Policy**: `Domain controller: LDAP server channel binding token requirements`
 - **Setting**: `Always`
5. Link the GPO to the Domain Controllers OU.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally.

[Download Script: Configure-LDAPChannelBinding.ps1](#)

```
# Configure-LDAPChannelBinding.ps1
# Description: Enforces LDAP Channel Binding Token requirements to Always.

Write-Host "Applying hardening requirement: Enforce LDAP Channel Binding..." -ForegroundColor Cyan

$regPath = "HKLM:\System\CurrentControlSet\Services\NTDS\Parameters"
if (-not (Test-Path $regPath)) {
    New-Item -Path $regPath -Force | Out-Null
}

Set-ItemProperty -Path $regPath -Name "LdapEnforceChannelBinding" -Value 2 -Type DWord
Write-Host "LDAP Channel Binding requirements set to 2 (Always)." -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-LDAPChannelBindingStatus.ps1](#)

```
# Get-LDAPChannelBindingStatus.ps1
# Description: Audits the LDAP Channel Binding Token configuration in the registry.

Write-Host "--- Auditing LDAP Channel Binding ---" -ForegroundColor Cyan

$regPath = "HKLM:\System\CurrentControlSet\Services\NTDS\Parameters"
$ntdsReg = Get-ItemProperty -Path $regPath -Name "LdapEnforceChannelBinding" -ErrorAction SilentlyContinue

if ($ntdsReg) {
    $cbtVal = $ntdsReg.LdapEnforceChannelBinding
    if ($cbtVal -eq 2) {
        Write-Host "[+] LDAP Channel Binding is secure. LdapEnforceChannelBinding is set to $($cbtVal) (Always)." -ForegroundColor Green
    } else {
        Write-Host "[!] VULNERABLE: LdapEnforceChannelBinding is set to $($cbtVal) (Required: 2)." -ForegroundColor Red
    }
} else {
    Write-Host "[!] VULNERABLE: LdapEnforceChannelBinding registry value is missing. The system uses default settings (does not enforce CBT)." -ForegroundColor Red
}
```

Sources & Compliance References * **ANSI AD Hardening Guide**: Recommendation R20 (LDAP Channel Binding) * **CIS Benchmark**: CIS Microsoft Windows Server Benchmark - Section 2.3.3.2 (Ensure 'Domain controller: LDAP server channel binding token requirements' is set to 'Always') * **Microsoft Security Advisory**: CVE-2017-8563 (LDAP Channel Binding security update details)

[REQ-DC-006] Enable LSA Protection

Target Scope * **Applicable Systems**: Domain Controllers, Member Servers, Tier 2 Clients * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10, Windows 11

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Administrative Templates\System\Local Security Authority` * **Policy**: `Configure LSA to run as a protected process` * **Setting**: `Enabled` (Value: `Enabled with LSA Protection` or `1`) * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Control\Lsa` → `RunAsPPL` = `1` (REG_DWORD)

Rationale The Local Security Authority Subsystem Service (LSASS) process (`lsass.exe`) is responsible for enforcing security policies, handling user authentication, and storing sensitive credential secrets (such as Kerberos tickets, NT hashes, and cached credentials) in memory. Adversaries who gain local administrative rights frequently target LSASS using memory-dumping tools (e.g., Mimikatz, Procdump) to extract these credentials, leading to domain-wide compromise and lateral movement. Enabling LSA Protection configures LSASS to run as a Protected Process Light (PPL). When running as a PPL, the operating system uses security boundaries to prevent non-protected processes (even those running with local administrator or SYSTEM privileges) from accessing LSASS memory space via debugging APIs ([OpenProcess](#) with read/write permissions) or injecting DLLs. This significantly increases the difficulty of offline credential harvesting.

Legacy Impact & Compatibility * **Third-Party Plug-ins**: Any third-party authentication plug-in, custom credential provider, smart card reader driver, or security agent (such as older antivirus or host-intrusion prevention systems) that interacts directly with LSASS must be digitally signed with a Microsoft signature. Unsigned binaries will fail to load into LSASS, potentially breaking multi-factor authentication or smart card logon. * **Audit Mode**: Administrators should audit the system for unsigned LSA plug-ins before enforcing LSA Protection. Event ID 3065 and 3066 in the `Microsoft-Windows-CodeIntegrity/Operational` log will list any unsigned drivers or DLLs that would have been blocked from loading into LSASS. * **Reboot Requirement**: Applying this setting requires a system reboot to take effect.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ([gpmc.msc](#)) on a management host.
2. Edit the appropriate hardening GPO (e.g., [GPO_Hardening_DomainControllers](#)).
3. Navigate to: [Computer Configuration\Policies\Administrative Templates\System\Local Security Authority](#)
4. Set the following policy:
 - o **Policy**: [Configure LSA to run as a protected process](#)
 - o **Setting**: [Enabled](#)
 - o **Choose one of the following options**: [Enabled with LSA Protection](#)
5. Link the GPO to the appropriate Organizational Unit (OU) containing the target assets.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally.

[Download Script: Configure-LSAProtection.ps1](#)

```
# Configure-LSAProtection.ps1
# Description: Enables LSA Protection (RunAsPPL) in the registry.

Write-Host "Applying hardening requirement: Enable LSA Protection..." -ForegroundColor Cyan

$regPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"
if (-not (Test-Path $regPath)) {
    New-Item -Path $regPath -Force | Out-Null
}

Set-ItemProperty -Path $regPath -Name "RunAsPPL" -Value 1 -Type DWord
Write-Host "LSA Protection registry configuration applied. A reboot is required to activate." -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-LSAProtectionStatus.ps1](#)

```
# Get-LSAProtectionStatus.ps1
# Description: Audits LSA Protection (RunAsPPL) in the registry.

Write-Host "--- Auditing LSA Protection ---" -ForegroundColor Cyan

$regPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"
$LsaReg = Get-ItemProperty -Path $regPath -Name "RunAsPPL" -ErrorAction SilentlyContinue

if ($LsaReg) {
    $pplVal = $LsaReg.RunAsPPL
    if ($pplVal -eq 1) {
        Write-Host "[+] LSA Protection is enabled. RunAsPPL is set to $($pplVal) (Secure)." -ForegroundColor Green
    } else {
        Write-Host "[!] VULNERABLE: RunAsPPL is set to $($pplVal) (Required: 1)." -ForegroundColor Red
    }
} else {
    Write-Host "[!] VULNERABLE: RunAsPPL registry value is missing. LSA is not running as a protected process." -ForegroundColor Red
}
```

Sources & Compliance References * **ANSI AD Hardening Guide**: Recommendation R14 (LSA Protection) * **CIS Benchmark**: CIS Microsoft Windows Server Benchmark - Section 18.9.50.1 (Ensure 'Configure LSA to run as a protected process' is set to 'Enabled: Enabled with LSA Protection') * **Microsoft Security Guidance**: Configuring Additional LSA Protection

[REQ-DC-007] Disable Credential Guard

Target Scope * **Applicable Systems**: Domain Controllers * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows Server 2025

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Administrative Templates\System\Device Guard` * **Policy**: `Turn On Virtualization-Based Security` * **Setting**: `Enabled` * **Virtualization Based Protection of Code Integrity**: `Enabled with UEFI lock` * **Credential Guard Configuration**: `Disabled` * **Require UEFI Memory Attributes Table**: `Enabled` * **Secure Launch Configuration**: `Enabled` * **Select Platform Security Level**: `Secure Boot` * **Registry Location (VBS)**: `HKLM\SYSTEM\CurrentControlSet\Control\DeviceGuard` for: * `EnableVirtualizationBasedSecurity` = `1` (REG_DWORD) * `HVCIMATRequired` = `1` (REG_DWORD) * `ConfigureSystemGuardLaunch` = `1` (REG_DWORD) * `RequirePlatformSecurityFeatures` = `1` (REG_DWORD) * `HypervisorEnforcedCodeIntegrity` = `1` (REG_DWORD) * **Registry Location (Credential Guard)**: `HKLM\SYSTEM\CurrentControlSet\Control\Lsa` → `LsaCfgFlags` = `0` (REG_DWORD)

Rationale Virtualization-Based Security (VBS) and Hypervisor-Protected Code Integrity (HVCI) should be enabled on Domain Controllers to protect the integrity of the operating system kernel and enforce driver blocklists.

However, Windows Defender Credential Guard must **not** be deployed on Active Directory domain controllers. Credential Guard is designed to isolate LSA secrets to prevent credential-dumping tools from harvesting password hashes from local memory. Domain controllers do not store user credentials in LSASS memory in the same way member servers do; instead, credentials are stored securely in the Active Directory database (ntds.dit). Furthermore, domain controllers run LSASS in a manner that requires active cryptographic operations and delegation capabilities that are incompatible with the restrictions imposed by Credential Guard.

Enabling Credential Guard on a domain controller can lead to authentication failures, block Kerberos delegation features, and cause operational instability without providing any security benefits.

For official warnings and product specifications, refer to the Microsoft documentation: [Microsoft Security Guidance: Windows Defender Credential Guard Warnings](#)

Legacy Impact & Compatibility * **Virtualization-Based Security**: Enabling VBS and Memory Integrity (HVCI) requires compliant virtualization hardware (Secure Boot, SLAT, IOMMU). Ensure hypervisors and host environments are fully compatible prior to deployment. * **Domain Controller Functionality**: Disabling Credential Guard on domain controllers is the standard, officially supported Microsoft configuration. It ensures that critical Active Directory services, authentication protocols, and delegation features operate without restriction or compatibility failures.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host.
2. Edit the appropriate hardening GPO (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Device Guard`
4. Set the following policy:
 - **Policy**: `Turn On Virtualization-Based Security`
 - **Setting**: `Enabled`
 - **Virtualization Based Protection of Code Integrity**: `Enabled with UEFI lock`
 - **Credential Guard Configuration**: `Disabled`
 - **Require UEFI Memory Attributes Table**: `Enabled`
 - **Secure Launch Configuration**: `Enabled`
 - **Select Platform Security Level**: `Secure Boot`

5. Link the GPO to the appropriate Organizational Unit (OU) containing the target assets.
-

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the settings locally.

[Download Script: Configure-DisableCredentialGuard.ps1](#)

```
# Configure-DisableCredentialGuard.ps1
# Description: Enables Virtualization-Based Security (VBS) and disables Credential Guard in the registry.

Write-Host "Applying hardening requirement: Enable VBS Baseline and Disable Credential Guard..." -ForegroundColor Cyan

# 1. Enable Virtualization-Based Security and related hypervisor options
$vbsPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard"
if (-not (Test-Path $vbsPath)) {
    New-Item -Path $vbsPath -Force | Out-Null
}

$vbsSettings = @{
    "EnableVirtualizationBasedSecurity" = 1
    "HVCIMATRequired"                  = 1
    "ConfigureSystemGuardLaunch"       = 1
    "RequirePlatformSecurityFeatures"  = 1
    "HypervisorEnforcedCodeIntegrity"  = 1
}

foreach ($Setting in $vbsSettings.Keys) {
    Set-ItemProperty -Path $vbsPath -Name $Setting -Value $vbsSettings[$Setting] -Type DWord -ErrorAction Stop
}
Write-Host "Virtualization-Based Security parameters enabled in registry." -ForegroundColor Green

# 2. Disable Credential Guard (LsaCfgFlags: 0 = Disabled)
$lsaPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"
if (-not (Test-Path $lsaPath)) {
    New-Item -Path $lsaPath -Force | Out-Null
}
Set-ItemProperty -Path $lsaPath -Name "LsaCfgFlags" -Value 0 -Type DWord
Write-Host "Credential Guard configured to Disabled in registry." -ForegroundColor Green

Write-Host "Hardening applied successfully. A system reboot is required." -ForegroundColor Green
```

To audit VBS and Credential Guard status using Registry and WMI:

[Download Script: Get-CredentialGuardStatus.ps1](#)

```
# Get-CredentialGuardStatus.ps1
# Description: Audits the configuration and operational status of VBS and ensures Credential Guard is disabled.

Write-Host "--- Auditing VBS and Credential Guard Status ---" -ForegroundColor Cyan
$Vulnerable = $false

# 1. Audit Registry Settings
$VbsRegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard"
$LsaReg = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa" -Name "LsaCfgFlags" -ErrorAction SilentlyContinue

$ExpectedVbsSettings = @{
    "EnableVirtualizationBasedSecurity" = 1
    "HVCIAMATRequired"                 = 1
    "ConfigureSystemGuardLaunch"        = 1
    "RequirePlatformSecurityFeatures"   = 1
    "HypervisorEnforcedCodeIntegrity"   = 1
}

if (Test-Path $VbsRegPath) {
    $VbsValues = Get-ItemProperty -Path $VbsRegPath -ErrorAction SilentlyContinue
    foreach ($Setting in $ExpectedVbsSettings.Keys) {
        $Val = $VbsValues.$Setting
        $Expected = $ExpectedVbsSettings[$Setting]
        if ($Val -eq $Expected) {
            Write-Host "[+] VBS setting '$Setting' is correctly configured ($Val)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: VBS setting '$Setting' is missing or incorrect ($Val)." -ForegroundColor Red
            $Vulnerable = $true
        }
    }
} else {
    Write-Host "[!] VULNERABLE: Virtualization-Based Security registry path does not exist." -ForegroundColor Red
    $Vulnerable = $true
}

# For DCs, Credential Guard (LsaCfgFlags) must be set to 0 (Disabled)
if ($null -ne $LsaReg -and $LsaReg.LsaCfgFlags -ne 0) {
    Write-Host "[!] VULNERABLE: Credential Guard is enabled in registry (LsaCfgFlags = $($LsaReg.LsaCfgFlags))." -ForegroundColor Red
    $Vulnerable = $true
} else {
    $Val = if ($null -eq $LsaReg) { "Not configured (Disabled)" } else { $LsaReg.LsaCfgFlags }
    Write-Host "[+] Credential Guard registry key 'LsaCfgFlags' is correctly set to disabled ($Val)." -ForegroundColor Green
}

# 2. Audit WMI Operational State (if running)
$DeviceGuard = Get-CimInstance -Namespace "root\cimv2" -ClassName "Win32_DeviceGuard" -ErrorAction SilentlyContinue
if ($DeviceGuard) {
    # SecurityServicesRunning: 1 = Credential Guard
    $ServicesRunning = $DeviceGuard.SecurityServicesRunning
    $CgRunning = $false
    foreach ($Service in $ServicesRunning) {
        if ($Service -eq 1) { $CgRunning = $true }
    }

    if ($CgRunning) {
        Write-Host "[!] VULNERABLE: Credential Guard is running operationally on this Domain Controller." -ForegroundColor Red
        $Vulnerable = $true
    } else {
        Write-Host "[+] Credential Guard is not running on this Domain Controller." -ForegroundColor Green
    }
} else {
    Write-Host "[-] WMI class Win32_DeviceGuard is not available." -ForegroundColor Yellow
}

if ($Vulnerable) {
    Write-Host "Audit result: VULNERABLE (Credential Guard enabled or VBS misconfigured)" -ForegroundColor Red
} else {
    Write-Host "Audit result: SECURE (Credential Guard disabled and VBS configured)" -ForegroundColor Green
}
}
```

Sources & Compliance References * **Microsoft Security Guidance**: [Windows Defender Credential Guard Warnings and Exclusions]

(<https://learn.microsoft.com/en-us/windows/security/identity-protection/credential-guard/>) * **ANSSI AD Hardening Guide**:

Recommendation R14 (acknowledging that Credential Guard is excluded from Domain Controllers due to compatibility constraints, while LSA protection remains active) * **CIS Benchmark**:

CIS Microsoft Windows Server Benchmark - Section 18.9.31.2 (noting that Credential Guard is restricted to compatible systems and excludes Active Directory Domain Controllers)

[REQ-DC-008] Disable Print Spooler Service

Target Scope * **Applicable Systems**: Domain Controllers * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` * **Service**: `Print Spooler` * **Setting**: `Define this policy setting: Disabled` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\Spooler` → `Start` = `4` (REG_DWORD)

Rationale The Windows Print Spooler service (`Spooler`) is enabled and running by default on Windows Server installations, including Domain Controllers. However, Domain Controllers do not print and should never act as print servers. The Print Spooler service has a history of high-severity vulnerabilities, including remote code execution exploits (e.g., the PrintNightmare vulnerability family - CVE-2021-1675 / CVE-2021-34527). Additionally, the service is exploited in coercion attacks such as the PetitPotam technique or printer-based authentication coercion. An attacker with low-privilege network access can send an RPC request to the DC's Print Spooler service (specifically utilizing APIs such as [RpcRemoteFindFirstPrinterChangeNotificationEx](#)), forcing the Domain Controller to authenticate to a malicious listener over NTLM. The attacker can then relay this authentication to take over the Active Directory domain. Disabling the service completely closes these high-risk vectors.

Legacy Impact & Compatibility * **No Functional Impact**: Disabling the Print Spooler service on a Domain Controller has no negative impact on Active Directory replication, client logins, DNS, or directory services. * **Pruning Shared Printers**: If the Domain Controller was historically used to host shared printers (which violates security best practices), those print queues will become unavailable and must be migrated to dedicated Member Servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ([gpmc.msc](#)) on a management host.
2. Edit the GPO linked to the Domain Controllers Organizational Unit (e.g., [GPO_Hardening_DomainControllers](#)).
3. Navigate to: [Computer Configuration\Policies\Windows Settings\Security Settings\System Services](#)
4. Find the **Print Spooler** service in the list and double-click it.
5. Check **Define this policy setting** and select **Disabled**.
6. Link the GPO to the Domain Controllers OU.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally.

[Download Script: Configure-DisablePrintSpooler.ps1](#)

```
# Configure-DisablePrintSpooler.ps1
# Description: Stops and disables the Print Spooler service.

Write-Host "Applying hardening requirement: Disable Print Spooler..." -ForegroundColor Cyan

$serviceName = "Spooler"
$service = Get-Service -Name $serviceName -ErrorAction SilentlyContinue

if ($service) {
    if ($service.Status -eq "Running") {
        Write-Host "Stopping service $($serviceName)..." -ForegroundColor Gray
        Stop-Service -Name $serviceName -Force -ErrorAction SilentlyContinue
    }

    # Configure startup type to disabled
    Set-Service -Name $serviceName -StartupType Disabled
    Write-Host "Service $($serviceName) has been stopped and disabled." -ForegroundColor Green
} else {
    Write-Host "Service $($serviceName) not found." -ForegroundColor Yellow
}
```

To verify the setting has been applied: [Download Script: Get-PrintSpoolerStatus.ps1](#)

```
# Get-PrintSpoolerStatus.ps1
# Description: Audits the operational status and startup type of the Print Spooler service.

Write-Host "--- Auditing Print Spooler Service ---" -ForegroundColor Cyan

$serviceName = "Spooler"
$service = Get-Service -Name $serviceName -ErrorAction SilentlyContinue

if ($service) {
    $status = $service.Status
    $startType = $service.StartType

    if ($status -eq "Stopped" -and $startType -eq "Disabled") {
        Write-Host "[+] Print Spooler is secure (Stopped and Disabled)." -ForegroundColor Green
    } else {
        Write-Host "[!] VULNERABLE: Print Spooler service status is $($status) and StartType is $($startType) (Required: Stopped & Disabled)." -ForegroundColor Red
    }
} else {
    Write-Host "[+] Print Spooler service is not installed on this system (Secure)." -ForegroundColor Green
}
```

Sources & Compliance References * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution and software installation) * **CIS Benchmark**: CIS Microsoft Windows Server Benchmark - Section 2.2.33 (Ensure 'Print Spooler' is set to 'Disabled') * **Microsoft Security Guidance**: Recommendations for disabling Print Spooler on Domain Controllers * **Other Reference**: CVE-2021-1675 / CVE-2021-34527 (PrintNightmare mitigation guidance)

[REQ-DC-009] Enforce SMB Message Signing

Target Scope * **Applicable Systems**: Domain Controllers, Member Servers, Tier 2 Clients * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10, Windows 11

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **Microsoft network server: Digitally sign communications (always)**: * Path: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` * Policy: `Microsoft network server: Digitally sign communications (always)` * Setting: `Enabled` * Registry: `HKLM\System\CurrentControlSet\Services\LanmanServer\Parameters` → `RequireSecuritySignature` = `1` (REG_DWORD) * **Microsoft network client: Digitally sign communications (always)**: * Path: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` * Policy: `Microsoft network client: Digitally sign communications (always)` * Setting: `Enabled` * Registry: `HKLM\System\CurrentControlSet\Services\LanmanWorkstation\Parameters` → `RequireSecuritySignature` = `1` (REG_DWORD)

Rationale Server Message Block (SMB) authentication is vulnerable to man-in-the-middle (MitM) and relay attacks. If SMB signing is not enforced, an attacker positioned on the local network can intercept SMB authentication sessions from client systems and relay them to another host (e.g., a Domain Controller or high-value member server). If the relayed user credential possesses administrative rights on the target host, the attacker can execute commands remotely (e.g., via PsExec/WMI) and compromise the system without knowing the password. Enforcing SMB signing ensures that all SMB packets are digitally signed using session keys. This guarantees the authenticity of both the sender and the receiver and ensures packet integrity. If an attacker attempts to relay or modify the packets, the cryptographic signature check fails, and the session is terminated. Enforcing this on both server and client roles is a fundamental defense against lateral movement and domain takeover.

Legacy Impact & Compatibility * **Performance Overhead**: Enforcing SMB signing introduces a slight CPU processing overhead for signing and verifying packets. On modern hardware supporting instruction sets such as AES-NI, this performance impact is negligible. * **Legacy Compatibility**: Legacy operating systems (e.g., Windows 98/NT4) or outdated third-party SMB client implementations that do not support SMB signing will be blocked from accessing SYSVOL, NETLOGON, or other file shares on Domain Controllers. Ensure all network systems support SMBv2 or higher with signing capabilities.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host.
2. Edit the appropriate hardening GPO (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options`
4. Configure the following policies:
 - **Policy**: `Microsoft network server: Digitally sign communications (always)`
 - **Setting**: `Enabled`
 - **Policy**: `Microsoft network client: Digitally sign communications (always)`
 - **Setting**: `Enabled`
5. Link the GPO to the appropriate Organizational Unit (OU) containing the target assets.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the settings locally.

[Download Script: Configure-SMBSigning.ps1](#)

```
# Configure-SMBSigning.ps1
# Description: Enforces SMB signing for both SMB server and client.

Write-Host "Applying hardening requirement: Enforce SMB Message Signing..." -ForegroundColor Cyan

# 1. Enforce Server SMB Signing
$srvRegPath = "HKLM:\System\CurrentControlSet\Services\LanmanServer\Parameters"
if (-not (Test-Path $srvRegPath)) {
    New-Item -Path $srvRegPath -Force | Out-Null
}
Set-ItemProperty -Path $srvRegPath -Name "RequireSecuritySignature" -Value 1 -Type DWord
Write-Host "SMB Server signing (always) enabled." -ForegroundColor Green

# 2. Enforce Client SMB Signing
$cliRegPath = "HKLM:\System\CurrentControlSet\Services\LanmanWorkstation\Parameters"
if (-not (Test-Path $cliRegPath)) {
    New-Item -Path $cliRegPath -Force | Out-Null
}
Set-ItemProperty -Path $cliRegPath -Name "RequireSecuritySignature" -Value 1 -Type DWord
Write-Host "SMB Client signing (always) enabled." -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-SMBSigningStatus.ps1](#)

```
# Get-SMBSigningStatus.ps1
# Description: Audits the registry settings for SMB server and client signing.

Write-Host "--- Auditing SMB Message Signing ---" -ForegroundColor Cyan
$Vulnerable = $false

# 1. Audit Server-side SMB Signing
$SrvReg = Get-ItemProperty -Path "HKLM:\System\CurrentControlSet\Services\LanmanServer\Parameters" -Name "RequireSecuritySignature" -ErrorAction SilentlyContinue
if ($SrvReg -and $SrvReg.RequireSecuritySignature -eq 1) {
    Write-Host "[+] SMB Server signing is enforced (RequireSecuritySignature = 1)." -ForegroundColor Green
} else {
    Write-Host "[!] VULNERABLE: SMB Server signing is NOT enforced." -ForegroundColor Red
    $Vulnerable = $true
}

# 2. Audit Client-side SMB Signing
$CliReg = Get-ItemProperty -Path "HKLM:\System\CurrentControlSet\Services\LanmanWorkstation\Parameters" -Name "RequireSecuritySignature" -ErrorAction SilentlyContinue
if ($CliReg -and $CliReg.RequireSecuritySignature -eq 1) {
    Write-Host "[+] SMB Client signing is enforced (RequireSecuritySignature = 1)." -ForegroundColor Green
} else {
    Write-Host "[!] VULNERABLE: SMB Client signing is NOT enforced." -ForegroundColor Red
    $Vulnerable = $true
}

if ($Vulnerable) {
    Write-Host "Audit result: VULNERABLE" -ForegroundColor Red
} else {
    Write-Host "Audit result: SECURE" -ForegroundColor Green
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**: Recommendation R13 (Disabling obsolete and insecure protocols and configuring transport security) * **CIS Benchmark**: CIS Microsoft Windows Server Benchmark - Sections 2.3.9.2 and 2.3.9.5 (Ensure 'Microsoft network server/client: Digitally sign communications (always)' is set to 'Enabled') * **Microsoft Security Guidance**: SMB Signing configurations and security implications

[REQ-DC-010] Restrict Kerberos Encryption Types

Target Scope * **Applicable Systems**: Domain Controllers, Member Servers, Tier 2 Clients * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10, Windows 11

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` * **Policy**: `Network security: Configure encryption types allowed for Kerberos` * **Setting**: `AES128\_HMAC\_SHA1, AES256\_HMAC\_SHA1, Future encryption types` (Make sure DES and RC4 are unchecked) * **Registry Location**: `HKLM\Software\Microsoft\Windows\CurrentVersion\Policies\System\Kerberos\Parameters` -
> `SupportedEncryptionTypes` = `2147483640` (REG_DWORD, decimal representation of `0x7FFFFFF8` or `0x7FFFFFFC` which restricts to AES and future types)

Rationale Active Directory uses Kerberos as its primary authentication protocol. By default, older cryptographic algorithms such as Data Encryption Standard (DES) and Rivest Cipher 4 (RC4) are supported for compatibility reasons. RC4-HMAC uses MD4 and MD5 hashing, which are cryptographically broken. When RC4 is enabled, adversaries can perform Kerberoasting attacks (requesting Service Principal Name - SPN - tickets from the domain) and easily crack the resulting tickets offline using GPU arrays. Similarly, DES is vulnerable to brute-force cracking.

Restricting allowed Kerberos encryption types to Advanced Encryption Standard (AES-128 and AES-256) forces the Kerberos Key Distribution Center (KDC) on Domain Controllers and the local systems to negotiate only AES encryption. This significantly increases the cryptographic strength of Kerberos tickets, making offline password extraction (Kerberoasting) or golden ticket forgery highly difficult.

Legacy Impact & Compatibility * **Account Compatibility**: User and computer accounts in Active Directory must support AES encryption. If a legacy service account does not have "This account supports Kerberos AES 128 bit/256 bit encryption" checked in its account options in Active Directory, authentication will fail once RC4 is disabled. * **Domain Trust Impact**: If the domain is joined in a trust relationship with an external domain (e.g., legacy MIT Kerberos realms or older Windows forests) that does not support AES, trust authentication will fail. * **Pre-remediation Audit**: Administrators should query the active directory environment for accounts that do not support AES using PowerShell: `Get-ADUser -Filter * -Properties msDS-SupportedEncryptionTypes | Where-Object { -not $_.msDS-SupportedEncryptionTypes }` Ensure these accounts are updated to support AES before enforcing this control.

Implementation Steps**### Option A: Group Policy Object (GPO) Configuration (Preferred)**

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host.
2. Edit the appropriate hardening GPO (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options`
4. Configure the following setting:
 - **Policy**: `Network security: Configure encryption types allowed for Kerberos`
 - **Setting**: Check only the following boxes:
 - `AES128_HMAC_SHA1`
 - `AES256_HMAC_SHA1`
 - `Future encryption types`
5. Link the GPO to the appropriate Organizational Unit (OU) containing the target assets.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally.

[Download Script: Configure-KerberosEncryptionTypes.ps1](#)

```
# Configure-KerberosEncryptionTypes.ps1
# Description: Restricts Kerberos encryption types to AES128, AES256, and Future types.

Write-Host "Applying hardening requirement: Restrict Kerberos Encryption Types..." -ForegroundColor Cyan

$regPath = "HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies\System\Kerberos\Parameters"
if (-not (Test-Path $regPath)) {
    New-Item -Path $regPath -Force | Out-Null
}

# 2147483640 (0x7FFFFFF8) enables AES128, AES256, and Future encryption types
Set-ItemProperty -Path $regPath -Name "SupportedEncryptionTypes" -Value 2147483640 -Type DWord
Write-Host "Kerberos encryption types restricted to AES and future types." -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-KerberosEncryptionStatus.ps1](#)

```
# Get-KerberosEncryptionStatus.ps1
# Description: Audits the allowed Kerberos encryption types in the registry.

Write-Host "--- Auditing Kerberos Encryption Types ---" -ForegroundColor Cyan

$regPath = "HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies\System\Kerberos\Parameters"
$regVal = Get-ItemProperty -Path $regPath -Name "SupportedEncryptionTypes" -ErrorAction SilentlyContinue

if ($regVal) {
    $encTypes = $regVal.SupportedEncryptionTypes

    # Check if weak algorithms are enabled (DES = 0x1, RC4 = 0x2)
    $hasDES = ($encTypes -band 0x1) -or ($encTypes -band 0x2)
    $hasRC4 = ($encTypes -band 0x4)
    $hasAES128 = ($encTypes -band 0x8)
    $hasAES256 = ($encTypes -band 0x10)

    if ($hasDES -or $hasRC4) {
        Write-Host "[!] VULNERABLE: Weak Kerberos encryption algorithms are allowed (DES: $($hasDES), RC4: $($hasRC4)). SupportedEncryptionTypes raw value: $($encTypes)." -ForegroundColor Red
    } else {
        if ($hasAES128 -and $hasAES256) {
            Write-Host "[+] Kerberos encryption is secure. Restricting to AES128/AES256 (SupportedEncryptionTypes: $($encTypes))." -ForegroundColor Green
        } else {
            Write-Host "[-] Kerberos encryption configuration is custom. AES128: $($hasAES128), AES256: $($hasAES256) (SupportedEncryptionTypes: $($encTypes))." -ForegroundColor Yellow
        }
    }
} else {
    Write-Host "[!] VULNERABLE: SupportedEncryptionTypes registry value is missing. Default behavior allows insecure RC4." -ForegroundColor Red
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**: Recommendation R13 (Disabling obsolete and insecure protocols) *
 CIS Benchmark: CIS Microsoft Windows Server Benchmark - Section 2.3.7.5 (Ensure 'Network security: Configure encryption types allowed for Kerberos' is configured) * **Microsoft Security Guidance**: Decrypting the Selection of Supported Kerberos Encryption Types

[REQ-DC-011] Restrict Remote SAM API Access

Target Scope * **Applicable Systems**: Domain Controllers, Member Servers * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10 (1607+), Windows 11

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` * **Policy**: `Network access: Restrict clients allowed to make remote calls to SAM` * **Setting**: `O:BAG:BAD:(A;;RC;;;BA)` (Security Descriptor definition in SDDL format, which grants Remote Access/Read Control only to the built-in Administrators group - BA) * **Registry Location**: `HKLM\System\CurrentControlSet\Control\Lsa` → `RestrictRemoteSAM` = `O:BAG:BAD:(A;;RC;;;BA)` (REG_SZ)

Rationale By default, the Security Account Manager (SAM) and NT Directory Services (NTDS) allow remote RPC connections from non-privileged accounts. Using these connections, an attacker who has established a foothold in the network (even with a basic, non-administrative domain user account) can query the Domain Controller or member servers to enumerate local users, group memberships, and security policies.

Tools like BloodHound/SharpHound, or simple commands like `net user /domain`, rely on these remote SAM RPC interfaces to extract information for network profiling and lateral movement mapping. Restricting remote client access to the SAM API (utilizing `RestrictRemoteSAM`) ensures that only members of the built-in Administrators group can make remote RPC queries. This significantly reduces the recon capabilities of an internal attacker.

Legacy Impact & Compatibility * **Monitoring Tool Impact**: Monitoring, configuration management, vulnerability scanning, or security audit systems that run under non-administrative accounts and remotely query local user or group lists on servers will fail. These tools must be updated to run under administrative credentials, or their service accounts must be explicitly granted access in the custom SDDL security descriptor. * **Compatibility Testing**: Prior to wide-scale enforcement, check the `System` event log on Domain Controllers for event sources relating to SAM RPC blocks (Event ID 16962 and 16969, which log blocked remote calls to SAM with details on the calling user account and IP).

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host.
2. Edit the appropriate hardening GPO (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options`
4. Configure the following setting:
 - **Policy**: `Network access: Restrict clients allowed to make remote calls to SAM`
 - **Setting**: Click **Edit Security** and configure the permissions. By default, only local Administrators are granted access. Ensure the SDDL translates to `O:BAG:BAD:(A;;RC;;;BA)` .
5. Link the GPO to the appropriate Organizational Unit (OU) containing the target assets.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally.

[Download Script: Configure-RestrictRemoteSAM.ps1](#)

```
# Configure-RestrictRemoteSAM.ps1
# Description: Restricts remote RPC access to the SAM database to local Administrators.

Write-Host "Applying hardening requirement: Restrict Remote SAM API Access..." -ForegroundColor Cyan

$regPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
if (-not (Test-Path $regPath)) {
    New-Item -Path $regPath -Force | Out-Null
}

$sddl = "O:BAG:BAD:(A;;RC;;;BA)"
Set-ItemProperty -Path $regPath -Name "RestrictRemoteSAM" -Value $sddl -Type String
Write-Host "SAM remote API access restricted to Administrators (SDDL applied)." -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-RestrictRemoteSAMStatus.ps1](#)

```
# Get-RestrictRemoteSAMStatus.ps1
# Description: Audits the RestrictRemoteSAM registry value.

Write-Host "--- Auditing RestrictRemoteSAM ---" -ForegroundColor Cyan

$regPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
$LsaReg = Get-ItemProperty -Path $regPath -Name "RestrictRemoteSAM" -ErrorAction SilentlyContinue

if ($LsaReg) {
    $sddlVal = $LsaReg.RestrictRemoteSAM
    if ($sddlVal -eq "0:BAG:BAD:(A;;RC;;;BA)") {
        Write-Host "[+] Remote SAM access is secure. RestrictRemoteSAM matches expected SDDL: $($sddlVal)." -ForegroundColor Green
    } else {
        Write-Host "[!] VULNERABLE: RestrictRemoteSAM is configured but has a different SDDL: $($sddlVal) (Expected: 0:BAG:BAD:(A;;RC;;;BA))." -ForegroundColor Red
    }
} else {
    Write-Host "[!] VULNERABLE: RestrictRemoteSAM registry key is missing. System allows remote SAM enumeration by standard users." -ForegroundColor Red
}
```

Sources & Compliance References * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution and interface exposures) * **CIS Benchmark**: CIS Microsoft Windows Server Benchmark - Section 2.3.10.10 (Ensure 'Network access: Restrict clients allowed to make remote calls to SAM' is set to 'Administrators: Remote Access: Allowed') * **Microsoft Security Guidance**: Network access: Restrict clients allowed to make remote calls to SAM configuration

Disable Unnecessary Services on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * Computer Configuration\Preferences\Windows Settings\Registry * HKLM\SYSTEM\CurrentControlSet\Services\Start

Rationale Domain Controllers are the core authority within an Active Directory forest (Tier 0). Minimizing the running services on these critical systems reduces the overall attack surface and limits potential targets for local privilege escalation, remote exploit execution, or credential extraction.

Disabling non-essential services aligns with the principle of least functionality. This submodule contains individual requirement controls for each non-essential server service.

Legacy Impact & Compatibility * **No Functional Impact**: Disabling the specified services has no operational impact on Active Directory Domain Services, replication, group policy processing, client authentication, or administrative management tools. * **Smart Cards**: If your domain relies on Smart Card authentication, the Smart Card Device Enumeration Service ('ScDeviceEnum') and Smart Card ('SCardSvr') services must remain enabled. * **Desktop Experience Feature Scope**: The specified services are primarily present on Windows Server installations with Desktop Experience. On Windows Server Core installations, most of these services are not installed by default, and attempts to stop or disable them will simply be skipped.

Service Hardening Requirements

The following services must be stopped and disabled:

1. [REQ-DC-035 - Disable Xbox Live Auth Manager \(XblAuthManager\)](#)
2. [REQ-DC-036 - Disable Xbox Live Game Save \(XblGameSave\)](#)
3. [REQ-DC-037 - Disable ActiveX Installer \(AxInstSV\)](#)
4. [REQ-DC-038 - Disable Bluetooth Support Service \(bthserv\)](#)
5. [REQ-DC-039 - Disable Connected Devices Platform User Service \(CDPUserSvc\)](#)
6. [REQ-DC-040 - Disable Contact Data \(PimIndexMaintenanceSvc\)](#)
7. [REQ-DC-041 - Disable WAP Push Message Routing Service \(dmwappushservice\)](#)
8. [REQ-DC-042 - Disable Downloaded Maps Manager \(MapsBroker\)](#)
9. [REQ-DC-043 - Disable Geolocation Service \(lfsvc\)](#)
10. [REQ-DC-044 - Disable Internet Connection Sharing \(ICS\) \(SharedAccess\)](#)
11. [REQ-DC-045 - Disable Link-Layer Topology Discovery Mapper \(lltdsvc\)](#)
12. [REQ-DC-046 - Disable Microsoft Account Sign-in Assistant \(wlidsvc\)](#)
13. [REQ-DC-047 - Disable Microsoft Passport \(NgcSvc\)](#)
14. [REQ-DC-048 - Disable Microsoft Passport Container \(NgcCntrSvc\)](#)
15. [REQ-DC-049 - Disable Network Connection Broker \(NcbService\)](#)
16. [REQ-DC-050 - Disable Phone Service \(PhoneSvc\)](#)
17. [REQ-DC-051 - Disable Printer Extensions and Notifications \(PrintNotify\)](#)
18. [REQ-DC-052 - Disable Program Compatibility Assistant Service \(PcaSvc\)](#)
19. [REQ-DC-053 - Disable Quality Windows Audio Video Experience \(QWAVE\)](#)
20. [REQ-DC-054 - Disable Radio Management Service \(RmSvc\)](#)
21. [REQ-DC-055 - Disable Sensor Data Service \(SensorDataService\)](#)
22. [REQ-DC-056 - Disable Sensor Monitoring Service \(SensrSvc\)](#)
23. [REQ-DC-057 - Disable Sensor Service \(SensorService\)](#)
24. [REQ-DC-058 - Disable Shell Hardware Detection \(ShellHWDetection\)](#)
25. [REQ-DC-059 - Disable Smart Card Device Enumeration Service \(ScDeviceEnum\)](#)
26. [REQ-DC-060 - Disable SSDP Discovery \(SSDPService\)](#)
27. [REQ-DC-061 - Disable Still Image Acquisition Events \(WiaRpc\)](#)
28. [REQ-DC-062 - Disable Sync Host \(OneSyncSvc\)](#)
29. [REQ-DC-063 - Disable UPnP Device Host \(upnphost\)](#)
30. [REQ-DC-064 - Disable User Data Access \(UserDataSvc\)](#)
31. [REQ-DC-065 - Disable User Data Storage \(UnistoreSvc\)](#)
32. [REQ-DC-066 - Disable WalletService \(WalletService\)](#)

- 33. [REQ-DC-067 - Disable Windows Audio \(Audiosrv\)](#)
- 34. [REQ-DC-068 - Disable Windows Audio Endpoint Builder \(AudioEndpointBuilder\)](#)
- 35. [REQ-DC-069 - Disable Windows Camera Frame Server \(FrameServer\)](#)
- 36. [REQ-DC-070 - Disable Windows Image Acquisition \(WIA\) \(stisvc\)](#)
- 37. [REQ-DC-071 - Disable Windows Insider Service \(wisvc\)](#)
- 38. [REQ-DC-072 - Disable Windows Mobile Hotspot Service \(icssvc\)](#)
- 39. [REQ-DC-073 - Disable Windows Push Notifications System Service \(WpnService\)](#)
- 40. [REQ-DC-074 - Disable Windows Push Notifications User Service \(WpnUserService\)](#)

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: [Security guidelines for disabling system services in Windows Server](https://learn.microsoft.com/en-us/windows-server/security/windows-services/security-guidelines-for-disabling-system-services-in-windows-server) * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution and software installation) * **CIS Microsoft Windows Server Benchmark**: Section 2.2 and general service minimization baselines

[REQ-DC-035] Disable Xbox Live Auth Manager on Domain Controllers (XblAuthManager)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\XblAuthManager\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Xbox Live Auth Manager (XblAuthManager) service directly supports this:

1. Provides gaming authentication functions; irrelevant to server infrastructure.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\XblAuthManager`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableXblAuthManager.ps1](#)

```
# Configure-DisableXblAuthManager.ps1
# Description: Disables the unnecessary Xbox Live Auth Manager (XblAuthManager) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Xbox Live Auth Manager service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "XblAuthManager"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-XblAuthManagerStatus.ps1](#)

```
# Get-XblAuthManagerStatus.ps1
# Description: Audits the registry startup state of unnecessary Xbox Live Auth Manager (XblAuthManager) service.

Write-Host "--- Auditing Xbox Live Auth Manager (XblAuthManager) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "XblAuthManager"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```


Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-036] Disable Xbox Live Game Save on Domain Controllers (XblGameSave)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\XblGameSave\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Xbox Live Game Save (XblGameSave) service directly supports this:

1. Provides gaming save functions; irrelevant to server infrastructure.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\XblGameSave`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableXblGameSave.ps1](#)

```
# Configure-DisableXblGameSave.ps1
# Description: Disables the unnecessary Xbox Live Game Save (XblGameSave) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Xbox Live Game Save service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "XblGameSave"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-XblGameSaveStatus.ps1](#)

```
# Get-XblGameSaveStatus.ps1
# Description: Audits the registry startup state of unnecessary Xbox Live Game Save (XblGameSave) service.

Write-Host "--- Auditing Xbox Live Game Save (XblGameSave) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "XblGameSave"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-037] Disable ActiveX Installer (AxInstSV) on Domain Controllers (AxInstSV)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\AxInstSV\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the ActiveX Installer (AxInstSV) (AxInstSV) service directly supports this:

1. Validates ActiveX controls. Domain Controllers should never run ActiveX controls.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\AxInstSV`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableAxInstSV.ps1](#)

```
# Configure-DisableAxInstSV.ps1
# Description: Disables the unnecessary ActiveX Installer (AxInstSV) (AxInstSV) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable ActiveX Installer (AxInstSV) service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "AxInstSV"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-AxInstSVStatus.ps1](#)

```
# Get-AxInstSVStatus.ps1
# Description: Audits the registry startup state of unnecessary ActiveX Installer (AxInstSV) (AxInstSV) service.

Write-Host "--- Auditing ActiveX Installer (AxInstSV) (AxInstSV) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "AxInstSV"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-038] Disable Bluetooth Support Service on Domain Controllers (bthserv)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\bthserv\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Bluetooth Support Service (bthserv) service directly supports this:

1. Supports Bluetooth devices; unnecessary on server systems.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\bthserv`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disablebthserv.ps1](#)


```
# Configure-Disablebthserv.ps1
# Description: Disables the unnecessary Bluetooth Support Service (bthserv) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Bluetooth Support Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "bthserv"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-bthservStatus.ps1](#)

```
# Get-bthservStatus.ps1
# Description: Audits the registry startup state of unnecessary Bluetooth Support Service (bthserv) service.

Write-Host "--- Auditing Bluetooth Support Service (bthserv) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "bthserv"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-039] Disable Connected Devices Platform User Service on Domain Controllers (CDPUserSvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\CDPUserSvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Connected Devices Platform User Service (CDPUserSvc) service directly supports this:

1. Connected Devices Platform User Service; syncs user activity data across devices.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\CDPUserSvc`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableCDPUserSvc.ps1](#)

```
# Configure-DisableCDPUserSvc.ps1
# Description: Disables the unnecessary Connected Devices Platform User Service (CDPUserSvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Connected Devices Platform User Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "CDPUserSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-CDPUserSvcStatus.ps1](#)

```
# Get-CDPUserSvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Connected Devices Platform User Service (CDPUserSvc) service.

Write-Host "--- Auditing Connected Devices Platform User Service (CDPUserSvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "CDPUserSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-040] Disable Contact Data on Domain Controllers (PimIndexMaintenanceSvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\PimIndexMaintenanceSvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Contact Data (PimIndexMaintenanceSvc) service directly supports this:

1. Manages contact data indexing; unnecessary on directory servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\PimIndexMaintenanceSvc`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePimIndexMaintenanceSvc.ps1](#)

```
# Configure-DisablePimIndexMaintenanceSvc.ps1
# Description: Disables the unnecessary Contact Data (PimIndexMaintenanceSvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Contact Data service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "PimIndexMaintenanceSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-PimIndexMaintenanceSvcStatus.ps1](#)

```
# Get-PimIndexMaintenanceSvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Contact Data (PimIndexMaintenanceSvc) service.

Write-Host "--- Auditing Contact Data (PimIndexMaintenanceSvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "PimIndexMaintenanceSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-041] Disable WAP Push Message Routing Service on Domain Controllers (dmwappushservice)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\dmwappushservice\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the WAP Push Message Routing Service (dmwappushservice) service directly supports this:

1. WAP Push Message Routing Service; used for diagnostics and telemetry.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
 2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
 3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
 4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\dmwappushservice`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`
-

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disabledmwappushservice.ps1](#)

```
# Configure-Disabledmwappushservice.ps1
# Description: Disables the unnecessary WAP Push Message Routing Service (dmwappushservice) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable WAP Push Message Routing Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "dmwappushservice"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-dmwappushserviceStatus.ps1](#)

```
# Get-dmwappushserviceStatus.ps1
# Description: Audits the registry startup state of unnecessary WAP Push Message Routing Service (dmwappushservice) service.

Write-Host "--- Auditing WAP Push Message Routing Service (dmwappushservice) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "dmwappushservice"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-042] Disable Downloaded Maps Manager on Domain Controllers (MapsBroker)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\MapsBroker\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Downloaded Maps Manager (MapsBroker) service directly supports this:

1. Enables access to downloaded maps; irrelevant to server roles.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\MapsBroker`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableMapsBroker.ps1](#)

```
# Configure-DisableMapsBroker.ps1
# Description: Disables the unnecessary Downloaded Maps Manager (MapsBroker) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Downloaded Maps Manager service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "MapsBroker"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\[${ServiceName}]"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '${ServiceName}' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '${ServiceName}' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '${ServiceName}' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '${ServiceName}'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-MapsBrokerStatus.ps1](#)

```
# Get-MapsBrokerStatus.ps1
# Description: Audits the registry startup state of unnecessary Downloaded Maps Manager (MapsBroker) service.

Write-Host "--- Auditing Downloaded Maps Manager (MapsBroker) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "MapsBroker"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\[${ServiceName}]"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '${ServiceName}' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '${ServiceName}' startup type is not Disabled (Start value is ${Start})." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '${ServiceName}' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '${ServiceName}' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-043] Disable Geolocation Service on Domain Controllers (lfsvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\lfsvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Geolocation Service (lfsvc) service directly supports this:

1. Monitors system location; represents a privacy and security risk.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\lfsvc`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disablelfsvc.ps1](#)

```
# Configure-Disablelfsvc.ps1
# Description: Disables the unnecessary Geolocation Service (lfsvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Geolocation Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "lfsvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-lfsvcStatus.ps1](#)

```
# Get-lfsvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Geolocation Service (lfsvc) service.

Write-Host "--- Auditing Geolocation Service (lfsvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "lfsvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance***: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide***: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark***: Service minimization baselines

[REQ-DC-044] Disable Internet Connection Sharing (ICS) on Domain Controllers (SharedAccess)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\SharedAccess\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Internet Connection Sharing (ICS) (SharedAccess) service directly supports this:

1. Provides network address translation; represents a networking security risk.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\SharedAccess`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableSharedAccess.ps1](#)

```
# Configure-DisableSharedAccess.ps1
# Description: Disables the unnecessary Internet Connection Sharing (ICS) (SharedAccess) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Internet Connection Sharing (ICS) service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "SharedAccess"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-SharedAccessStatus.ps1](#)

```
# Get-SharedAccessStatus.ps1
# Description: Audits the registry startup state of unnecessary Internet Connection Sharing (ICS) (SharedAccess) service.

Write-Host "--- Auditing Internet Connection Sharing (ICS) (SharedAccess) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "SharedAccess"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-045] Disable Link-Layer Topology Discovery Mapper on Domain Controllers (lltdsvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\lltdsvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Link-Layer Topology Discovery Mapper (lltdsvc) service directly supports this:

1. Discovers network topology; unnecessary exposure of server network location.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\lltdsvc`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disablelltdsvc.ps1](#)

```
# Configure-Disablelltdsvc.ps1
# Description: Disables the unnecessary Link-Layer Topology Discovery Mapper (lltdsvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Link-Layer Topology Discovery Mapper service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "lltdsvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-lltdsvcStatus.ps1](#)

```
# Get-lltdsvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Link-Layer Topology Discovery Mapper (lltdsvc) service.

Write-Host "--- Auditing Link-Layer Topology Discovery Mapper (lltdsvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "lltdsvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-046] Disable Microsoft Account Sign-in Assistant on Domain Controllers (wlidsvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\wlidsvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Microsoft Account Sign-in Assistant (wlidsvc) service directly supports this:

1. Enables user signing with Microsoft Accounts; unnecessary on directory servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\wlidsvc`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disablewlidsvc.ps1](#)

```
# Configure-DisableWlidsvc.ps1
# Description: Disables the unnecessary Microsoft Account Sign-in Assistant (wlidsvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Microsoft Account Sign-in Assistant service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "wlidsvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-wlidsvcStatus.ps1](#)

```
# Get-wlidsvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Microsoft Account Sign-in Assistant (wlidsvc) service.

Write-Host "--- Auditing Microsoft Account Sign-in Assistant (wlidsvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "wlidsvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-047] Disable Microsoft Passport on Domain Controllers (NgcSvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\NgcSvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Microsoft Passport (NgcSvc) service directly supports this:

1. Part of Windows Hello for Business; not needed if Windows Hello is not deployed.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\NgcSvc`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableNgcSvc.ps1](#)

```
# Configure-DisableNgcSvc.ps1
# Description: Disables the unnecessary Microsoft Passport (NgcSvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Microsoft Passport service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "NgcSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-NgcSvcStatus.ps1](#)

```
# Get-NgcSvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Microsoft Passport (NgcSvc) service.

Write-Host "--- Auditing Microsoft Passport (NgcSvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "NgcSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-048] Disable Microsoft Passport Container on Domain Controllers (NgcCtrSvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\NgcCtrSvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Microsoft Passport Container (NgcCtrSvc) service directly supports this:

1. Part of Windows Hello for Business container management.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
 2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
 3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
 4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\NgcCtrSvc`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`
-

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableNgcCtrSvc.ps1](#)

```
# Configure-DisableNgcCtrSvc.ps1
# Description: Disables the unnecessary Microsoft Passport Container (NgcCtrSvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Microsoft Passport Container service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "NgcCtrSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-NgcCtrSvcStatus.ps1](#)

```
# Get-NgcCtrSvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Microsoft Passport Container (NgcCtrSvc) service.

Write-Host "--- Auditing Microsoft Passport Container (NgcCtrSvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "NgcCtrSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-049] Disable Network Connection Broker on Domain Controllers (NcbService)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\NcbService\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Network Connection Broker (NcbService) service directly supports this:

1. Broker for background network connections for modern apps.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\NcbService`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableNcbService.ps1](#)

```
# Configure-DisableNcbService.ps1
# Description: Disables the unnecessary Network Connection Broker (NcbService) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Network Connection Broker service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "NcbService"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-NcbServiceStatus.ps1](#)

```
# Get-NcbServiceStatus.ps1
# Description: Audits the registry startup state of unnecessary Network Connection Broker (NcbService) service.

Write-Host "--- Auditing Network Connection Broker (NcbService) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "NcbService"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-050] Disable Phone Service on Domain Controllers (PhoneSvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\PhoneSvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Phone Service (PhoneSvc) service directly supports this:

1. Manages telephony state; irrelevant to server infrastructure.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\PhoneSvc`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePhoneSvc.ps1](#)

```
# Configure-DisablePhoneSvc.ps1
# Description: Disables the unnecessary Phone Service (PhoneSvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Phone Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "PhoneSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-PhoneSvcStatus.ps1](#)

```
# Get-PhoneSvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Phone Service (PhoneSvc) service.

Write-Host "--- Auditing Phone Service (PhoneSvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "PhoneSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-051] Disable Printer Extensions and Notifications on Domain Controllers (PrintNotify)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\PrintNotify\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Printer Extensions and Notifications (PrintNotify) service directly supports this:

1. Handles printer notification dialogs; unnecessary on servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\PrintNotify`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePrintNotify.ps1](#)

```
# Configure-DisablePrintNotify.ps1
# Description: Disables the unnecessary Printer Extensions and Notifications (PrintNotify) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Printer Extensions and Notifications service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "PrintNotify"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-PrintNotifyStatus.ps1](#)

```
# Get-PrintNotifyStatus.ps1
# Description: Audits the registry startup state of unnecessary Printer Extensions and Notifications (PrintNotify) service.

Write-Host "--- Auditing Printer Extensions and Notifications (PrintNotify) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "PrintNotify"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-052] Disable Program Compatibility Assistant Service on Domain Controllers (PcaSvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\PcaSvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Program Compatibility Assistant Service (PcaSvc) service directly supports this:

1. Detects application compatibility issues; unnecessary on highly controlled DCs.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\PcaSvc`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePcaSvc.ps1](#)

```
# Configure-DisablePcaSvc.ps1
# Description: Disables the unnecessary Program Compatibility Assistant Service (PcaSvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Program Compatibility Assistant Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "PcaSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-PcaSvcStatus.ps1](#)

```
# Get-PcaSvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Program Compatibility Assistant Service (PcaSvc) service.

Write-Host "--- Auditing Program Compatibility Assistant Service (PcaSvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "PcaSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-053] Disable Quality Windows Audio Video Experience on Domain Controllers (QWAVE)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\QWAVE\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Quality Windows Audio Video Experience (QWAVE) service directly supports this:

1. Multimedia streaming quality service; irrelevant to server systems.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\QWAVE`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableQWAVE.ps1](#)

```
# Configure-DisableQWAVE.ps1
# Description: Disables the unnecessary Quality Windows Audio Video Experience (QWAVE) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Quality Windows Audio Video Experience service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "QWAVE"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-QWAVEStatus.ps1](#)

```
# Get-QWAVEStatus.ps1
# Description: Audits the registry startup state of unnecessary Quality Windows Audio Video Experience (QWAVE) service.

Write-Host "--- Auditing Quality Windows Audio Video Experience (QWAVE) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "QWAVE"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-054] Disable Radio Management Service on Domain Controllers (RmSvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\RmSvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Radio Management Service (RmSvc) service directly supports this:

1. Controls radio transmitters (cellular, Wi-Fi); irrelevant to servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\RmSvc`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableRmSvc.ps1](#)

```
# Configure-DisableRmSvc.ps1
# Description: Disables the unnecessary Radio Management Service (RmSvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Radio Management Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "RmSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-RmSvcStatus.ps1](#)

```
# Get-RmSvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Radio Management Service (RmSvc) service.

Write-Host "--- Auditing Radio Management Service (RmSvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "RmSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-055] Disable Sensor Data Service on Domain Controllers (SensorDataService)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\SensorDataService\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Sensor Data Service (SensorDataService) service directly supports this:

1. Handles data from system sensors; irrelevant to servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\SensorDataService`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableSensorDataService.ps1](#)

```
# Configure-DisableSensorDataService.ps1
# Description: Disables the unnecessary Sensor Data Service (SensorDataService) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Sensor Data Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "SensorDataService"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-SensorDataServiceStatus.ps1](#)

```
# Get-SensorDataServiceStatus.ps1
# Description: Audits the registry startup state of unnecessary Sensor Data Service (SensorDataService) service.

Write-Host "--- Auditing Sensor Data Service (SensorDataService) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "SensorDataService"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```


Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-056] Disable Sensor Monitoring Service on Domain Controllers (SensrSvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\SensrSvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Sensor Monitoring Service (SensrSvc) service directly supports this:

1. Monitors system sensors; irrelevant to servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\SensrSvc`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableSensrSvc.ps1](#)

```
# Configure-DisableSensrSvc.ps1
# Description: Disables the unnecessary Sensor Monitoring Service (SensrSvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Sensor Monitoring Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "SensrSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-SensrSvcStatus.ps1](#)

```
# Get-SensrSvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Sensor Monitoring Service (SensrSvc) service.

Write-Host "--- Auditing Sensor Monitoring Service (SensrSvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "SensrSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-057] Disable Sensor Service on Domain Controllers (SensorService)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\SensorService\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Sensor Service (SensorService) service directly supports this:

1. Core sensor service; irrelevant to servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\SensorService`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableSensorService.ps1](#)

```
# Configure-DisableSensorService.ps1
# Description: Disables the unnecessary Sensor Service (SensorService) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Sensor Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "SensorService"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-SensorServiceStatus.ps1](#)

```
# Get-SensorServiceStatus.ps1
# Description: Audits the registry startup state of unnecessary Sensor Service (SensorService) service.

Write-Host "--- Auditing Sensor Service (SensorService) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "SensorService"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-058] Disable Shell Hardware Detection on Domain Controllers (ShellHWDetection)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\ShellHWDetection\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Shell Hardware Detection (ShellHWDetection) service directly supports this:

1. Provides notifications for AutoPlay hardware events; can be disabled.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\ShellHWDetection`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableShellHWDetection.ps1](#)


```
# Configure-DisableShellHWDetection.ps1
# Description: Disables the unnecessary Shell Hardware Detection (ShellHWDetection) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Shell Hardware Detection service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "ShellHWDetection"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-ShellHWDetectionStatus.ps1](#)

```
# Get-ShellHWDetectionStatus.ps1
# Description: Audits the registry startup state of unnecessary Shell Hardware Detection (ShellHWDetection) service.

Write-Host "--- Auditing Shell Hardware Detection (ShellHWDetection) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "ShellHWDetection"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance***: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide***: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark***: Service minimization baselines

[REQ-DC-059] Disable Smart Card Device Enumeration Service on Domain Controllers (ScDeviceEnum)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\ScDeviceEnum\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Smart Card Device Enumeration Service (ScDeviceEnum) service directly supports this:

1. Detects smart cards; can be disabled if smart cards are not used for authentication.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
 2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
 3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
 4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\ScDeviceEnum`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`
-

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableScDeviceEnum.ps1](#)

```
# Configure-DisableScDeviceEnum.ps1
# Description: Disables the unnecessary Smart Card Device Enumeration Service (ScDeviceEnum) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Smart Card Device Enumeration Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "ScDeviceEnum"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-ScDeviceEnumStatus.ps1](#)

```
# Get-ScDeviceEnumStatus.ps1
# Description: Audits the registry startup state of unnecessary Smart Card Device Enumeration Service (ScDeviceEnum) service.

Write-Host "--- Auditing Smart Card Device Enumeration Service (ScDeviceEnum) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "ScDeviceEnum"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-060] Disable SSDP Discovery on Domain Controllers (SSDPSRV)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\SSDPSRV\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the SSDP Discovery (SSDPSRV) service directly supports this:

1. Discovers UPnP devices; introduces broadcast name discovery vulnerabilities.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\SSDPSRV`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableSSDPSRV.ps1](#)

```
# Configure-DisableSSDPSRV.ps1
# Description: Disables the unnecessary SSDP Discovery (SSDPSRV) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable SSDP Discovery service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "SSDPSRV"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-SSDPSRVStatus.ps1](#)

```
# Get-SSDPSRVStatus.ps1
# Description: Audits the registry startup state of unnecessary SSDP Discovery (SSDPSRV) service.

Write-Host "--- Auditing SSDP Discovery (SSDPSRV) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "SSDPSRV"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\[($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '[($ServiceName)]' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '[($ServiceName)]' startup type is not Disabled (Start value is [($Start)])." -Foregro
undColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '[($ServiceName)]' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '[($ServiceName)]' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-061] Disable Still Image Acquisition Events on Domain Controllers (WiaRpc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\WiaRpc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Still Image Acquisition Events (WiaRpc) service directly supports this:

1. Still image capturing events; irrelevant to servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\WiaRpc`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableWiaRpc.ps1](#)


```
# Configure-DisableWiaRpc.ps1
# Description: Disables the unnecessary Still Image Acquisition Events (WiaRpc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Still Image Acquisition Events service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "WiaRpc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-WiaRpcStatus.ps1](#)

```
# Get-WiaRpcStatus.ps1
# Description: Audits the registry startup state of unnecessary Still Image Acquisition Events (WiaRpc) service.

Write-Host "--- Auditing Still Image Acquisition Events (WiaRpc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "WiaRpc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-062] Disable Sync Host on Domain Controllers (OneSyncSvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\OneSyncSvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Sync Host (OneSyncSvc) service directly supports this:

1. Synchronizes mail, contacts, calendar; irrelevant to servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\OneSyncSvc`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableOneSyncSvc.ps1](#)

```
# Configure-DisableOneSyncSvc.ps1
# Description: Disables the unnecessary Sync Host (OneSyncSvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Sync Host service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "OneSyncSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-OneSyncSvcStatus.ps1](#)

```
# Get-OneSyncSvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Sync Host (OneSyncSvc) service.

Write-Host "--- Auditing Sync Host (OneSyncSvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "OneSyncSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance***: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide***: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark***: Service minimization baselines

[REQ-DC-063] Disable UPnP Device Host on Domain Controllers (upnphost)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\upnphost\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the UPnP Device Host (upnphost) service directly supports this:

1. Allows hosting of UPnP devices; represents unnecessary network exposure.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\upnphost`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disableupnphost.ps1](#)

```
# Configure-Disableupnphost.ps1
# Description: Disables the unnecessary UPnP Device Host (upnphost) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable UPnP Device Host service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "upnphost"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-upnphostStatus.ps1](#)

```
# Get-upnphostStatus.ps1
# Description: Audits the registry startup state of unnecessary UPnP Device Host (upnphost) service.

Write-Host "--- Auditing UPnP Device Host (upnphost) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "upnphost"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-064] Disable User Data Access on Domain Controllers (UserDataSvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\UserDataSvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the User Data Access (UserDataSvc) service directly supports this:

1. Manages user structured data; irrelevant to servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\UserDataSvc`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableUserDataSvc.ps1](#)


```
# Configure-DisableUserDataSvc.ps1
# Description: Disables the unnecessary User Data Access (UserDataSvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable User Data Access service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "UserDataSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-UserDataSvcStatus.ps1](#)

```
# Get-UserDataSvcStatus.ps1
# Description: Audits the registry startup state of unnecessary User Data Access (UserDataSvc) service.

Write-Host "--- Auditing User Data Access (UserDataSvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "UserDataSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-065] Disable User Data Storage on Domain Controllers (UnistoreSvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\UnistoreSvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the User Data Storage (UnistoreSvc) service directly supports this:

1. Stores user data; irrelevant to servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\UnistoreSvc`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableUnistoreSvc.ps1](#)

```
# Configure-DisableUnistoreSvc.ps1
# Description: Disables the unnecessary User Data Storage (UnistoreSvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable User Data Storage service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "UnistoreSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-UnistoreSvcStatus.ps1](#)

```
# Get-UnistoreSvcStatus.ps1
# Description: Audits the registry startup state of unnecessary User Data Storage (UnistoreSvc) service.

Write-Host "--- Auditing User Data Storage (UnistoreSvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "UnistoreSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance***: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide***: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark***: Service minimization baselines

[REQ-DC-066] Disable WalletService on Domain Controllers (WalletService)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\WalletService\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the WalletService (WalletService) service directly supports this:

1. Used by Wallet application; irrelevant to servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\WalletService`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableWalletService.ps1](#)

```
# Configure-DisableWalletService.ps1
# Description: Disables the unnecessary WalletService (WalletService) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable WalletService service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "WalletService"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-WalletServiceStatus.ps1](#)

```
# Get-WalletServiceStatus.ps1
# Description: Audits the registry startup state of unnecessary WalletService (WalletService) service.

Write-Host "--- Auditing WalletService (WalletService) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "WalletService"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-067] Disable Windows Audio on Domain Controllers (Audiosrv)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\Audiosrv\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Windows Audio (Audiosrv) service directly supports this:

1. Manages system audio; servers do not require audio capabilities.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\Audiosrv`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableAudiosrv.ps1](#)

```
# Configure-DisableAudiosrv.ps1
# Description: Disables the unnecessary Windows Audio (Audiosrv) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Windows Audio service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "Audiosrv"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-AudiosrvStatus.ps1](#)

```
# Get-AudiosrvStatus.ps1
# Description: Audits the registry startup state of unnecessary Windows Audio (Audiosrv) service.

Write-Host "--- Auditing Windows Audio (Audiosrv) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "Audiosrv"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-068] Disable Windows Audio Endpoint Builder on Domain Controllers (AudioEndpointBuilder)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\AudioEndpointBuilder\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Windows Audio Endpoint Builder (AudioEndpointBuilder) service directly supports this:

1. Manages audio endpoints; servers do not require audio capabilities.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\AudioEndpointBuilder`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableAudioEndpointBuilder.ps1](#)

```
# Configure-DisableAudioEndpointBuilder.ps1
# Description: Disables the unnecessary Windows Audio Endpoint Builder (AudioEndpointBuilder) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Windows Audio Endpoint Builder service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "AudioEndpointBuilder"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-AudioEndpointBuilderStatus.ps1](#)

```
# Get-AudioEndpointBuilderStatus.ps1
# Description: Audits the registry startup state of unnecessary Windows Audio Endpoint Builder (AudioEndpointBuilder) service.

Write-Host "--- Auditing Windows Audio Endpoint Builder (AudioEndpointBuilder) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "AudioEndpointBuilder"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-069] Disable Windows Camera Frame Server on Domain Controllers (FrameServer)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\FrameServer\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Windows Camera Frame Server (FrameServer) service directly supports this:

1. Enables access to system camera feeds; irrelevant to servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\FrameServer`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableFrameServer.ps1](#)

```
# Configure-DisableFrameServer.ps1
# Description: Disables the unnecessary Windows Camera Frame Server (FrameServer) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Windows Camera Frame Server service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "FrameServer"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-FrameServerStatus.ps1](#)

```
# Get-FrameServerStatus.ps1
# Description: Audits the registry startup state of unnecessary Windows Camera Frame Server (FrameServer) service.

Write-Host "--- Auditing Windows Camera Frame Server (FrameServer) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "FrameServer"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-070] Disable Windows Image Acquisition (WIA) on Domain Controllers (stisvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\stisvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Windows Image Acquisition (WIA) (stisvc) service directly supports this:

1. Image acquisition from scanners/cameras; irrelevant to servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\stisvc`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disablestisvc.ps1](#)

```
# Configure-Disablestisvc.ps1
# Description: Disables the unnecessary Windows Image Acquisition (WIA) (stisvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Windows Image Acquisition (WIA) service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "stisvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-stisvcStatus.ps1](#)

```
# Get-stisvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Windows Image Acquisition (WIA) (stisvc) service.

Write-Host "--- Auditing Windows Image Acquisition (WIA) (stisvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "stisvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-071] Disable Windows Insider Service on Domain Controllers (wisvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\wisvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Windows Insider Service (wisvc) service directly supports this:

1. Handles Windows Insider settings; unnecessary on production servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\wisvc`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disablewisvc.ps1](#)

```
# Configure-Disablewisvc.ps1
# Description: Disables the unnecessary Windows Insider Service (wisvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Windows Insider Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "wisvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-wisvcStatus.ps1](#)

```
# Get-wisvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Windows Insider Service (wisvc) service.

Write-Host "--- Auditing Windows Insider Service (wisvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "wisvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-072] Disable Windows Mobile Hotspot Service on Domain Controllers (icssvc)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\icssvc\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Windows Mobile Hotspot Service (icssvc) service directly supports this:

1. Shares internet connection as hotspot; introduces wireless routing security risk.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\icssvc`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disableicssvc.ps1](#)

```
# Configure-Disableicssvc.ps1
# Description: Disables the unnecessary Windows Mobile Hotspot Service (icssvc) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Windows Mobile Hotspot Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "icssvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-icssvcStatus.ps1](#)

```
# Get-icssvcStatus.ps1
# Description: Audits the registry startup state of unnecessary Windows Mobile Hotspot Service (icssvc) service.

Write-Host "--- Auditing Windows Mobile Hotspot Service (icssvc) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "icssvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-073] Disable Windows Push Notifications System Service on Domain Controllers (WpnService)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\WpnService\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Windows Push Notifications System Service (WpnService) service directly supports this:

1. System service for push notifications; irrelevant to servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - o **Action**: `Update`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Services\WpnService`
 - o **Value name**: `Start`
 - o **Value type**: `REG_DWORD`
 - o **Value data**: `4`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableWpnService.ps1](#)

```
# Configure-DisableWpnService.ps1
# Description: Disables the unnecessary Windows Push Notifications System Service (WpnService) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Windows Push Notifications System Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "WpnService"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-WpnServiceStatus.ps1](#)

```
# Get-WpnServiceStatus.ps1
# Description: Audits the registry startup state of unnecessary Windows Push Notifications System Service (WpnService) service.

Write-Host "--- Auditing Windows Push Notifications System Service (WpnService) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "WpnService"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-074] Disable Windows Push Notifications User Service on Domain Controllers (WpnUserService)

Target Scope * **Applicable Systems**: Domain Controllers (DCs) running Windows Server. * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022.

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\WpnUserService\Start`

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in the Active Directory forest. Minimizing the execution footprint on these critical servers is an essential hardening guideline. Disabling the Windows Push Notifications User Service (WpnUserService) service directly supports this:

1. User service for push notifications; irrelevant to servers.
2. Reducing running background services closes potential vectors for local privilege escalation and memory space exploits on the directory controllers.

Legacy Impact & Compatibility * **Normal Operations**: Disabling this service has no impact on core Active Directory Domain Services (AD DS), Kerberos, DNS, or SYSVOL replication functionality. * **Special Hardware/Configurations**: Smart card services or time sync should remain enabled if strictly required, but the services listed in the baseline can be safely disabled on standard directory servers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Because these services are not managed through standard GPO Administrative Templates, configure Group Policy Preferences (GPP) for the registry:

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
 2. Edit the GPO linked to your Domain Controllers Organizational Unit (e.g., `GPO_Hardening_DomainControllers`).
 3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
 4. Create a new Registry Preference (Right-click **Registry** → **New** → **Registry Item**):
 - **Action**: `Update`
 - **Hive**: `HKEY_LOCAL_MACHINE`
 - **Key Path**: `SYSTEM\CurrentControlSet\Services\WpnUserService`
 - **Value name**: `Start`
 - **Value type**: `REG_DWORD`
 - **Value data**: `4`
-

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableWpnUserService.ps1](#)

```
# Configure-DisableWpnUserService.ps1
# Description: Disables the unnecessary Windows Push Notifications User Service (WpnUserService) service on Domain Controllers.

Write-Host "Applying hardening requirement: Disable Windows Push Notifications User Service service on Domain Controllers..." -ForegroundColor Cyan

$ServiceName = "WpnUserService"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
    if ($Service.StartType -ne "Disabled") {
        if ($Service.Status -eq "Running") {
            Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
        }
        Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
        Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
    } else {
        Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
    }
} else {
    Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
    Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
    Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the Domain Controller:

[Download Script: Get-WpnUserServiceStatus.ps1](#)

```
# Get-WpnUserServiceStatus.ps1
# Description: Audits the registry startup state of unnecessary Windows Push Notifications User Service (WpnUserService) service.

Write-Host "--- Auditing Windows Push Notifications User Service (WpnUserService) Service on Domain Controller ---" -ForegroundColor Cyan

$ServiceName = "WpnUserService"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
    $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
    if ($null -ne $StartVal) {
        $Start = $StartVal.Start
        if ($Start -eq 4) {
            Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
        } else {
            Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
            $IsVulnerable = $true
        }
    } else {
        Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
        $IsVulnerable = $true
    }
} else {
    Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **Microsoft Windows Server Security Guidance**: Guidelines for disabling system services in Windows Server * **ANSSI AD Hardening Guide**: Recommendation R4 (Minimization of service execution) * **CIS Microsoft Windows Server Benchmark**: Service minimization baselines

[REQ-DC-013] Enable Kerberos Armoring

Target Scope * **Applicable Systems**: Domain Controllers, Member Servers, Tier 2 Clients * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10, Windows 11

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **KDC Settings (Domain Controllers)**: * **GPO Path**: `Computer Configuration\Policies\Administrative Templates\System\KDC` * **Policy**: `KDC support for claims, compound authentication and Kerberos armoring` * **Setting**: `Supported` (or `Fail unarmored authentication requests` for strict enforcement) * **Registry Location**: `HKLM\Software\Microsoft\Windows\CurrentVersion\Policies\System\KDC\Parameters` → `EnableCbacAndArmor` = `1` (Supported) or `3` (Fail unarmored) (REG_DWORD) * **Client Settings (Workstations & Member Servers)**: * **GPO Path**: `Computer Configuration\Policies\Administrative Templates\System\Kerberos` * **Policies**: * `Kerberos client support for claims, compound authentication and Kerberos armoring` → Enabled * `Support device authentication using certificate` → Enabled: Automatic * **Registry Location**: `HKLM\Software\Microsoft\Windows\CurrentVersion\Policies\System\Kerberos\Parameters` * `EnableCbacAndArmor` = `1` (REG_DWORD) * `DevicePKInitEnabled` = `1` (REG_DWORD) * `DevicePKInitBehavior` = `0` (REG_DWORD)

Rationale Active Directory environments relying on standard Kerberos authentication are susceptible to offline brute-force, dictionary attacks, and credential harvesting. During the initial Kerberos pre-authentication phase, the client requests a Ticket Granting Ticket (TGT) in clear text by sending an AS-REQ containing encrypted timestamps. Attackers monitoring network traffic can intercept these exchanges, or perform AS-REP roasting against accounts that do not require pre-authentication, conducting offline password cracking to compromise credentials.

Kerberos Armoring, also known as Flexible Authentication Secure Tunneling (FAST), mitigates this vulnerability by establishing an encrypted channel (a secure tunnel) between the Kerberos client and the Key Distribution Center (KDC) on the Domain Controller. This tunnel is encrypted using the computer account's credential (or the local system's credential), protecting the pre-authentication messages (AS-REQ and AS-REP) from eavesdropping and tampering.

Additionally, Kerberos Armoring is a strict prerequisite for Dynamic Access Control (DAC) and Compound Authentication (which validates both the user's and the device's identities before granting access). Implementing Kerberos Armoring significantly enhances the directory service's resistance to credential relaying, user enumeration, and offline brute-force cracking.

Legacy Impact & Compatibility * **Operating System Requirements**: Kerberos Armoring requires at least a Windows Server 2012 domain functional level. Target clients and servers must run Windows 8 / Windows Server 2012 or newer. * **Enforcement Risk**: Setting KDC support to "Fail unarmored authentication requests" (Value 3) too early will prevent systems that are not configured for FAST, legacy operating systems (e.g., Windows 7, Windows Server 2008 R2), and non-domain-joined devices from authenticating, resulting in complete denial of service. * **Staged Deployment**: A phased rollout is highly recommended. Administrators must first configure all client systems to support claims and armoring. Once client compliance is verified, KDC support should be set to "Supported" (Value 1) to allow armored connections without failing unarmored ones. After validating the environment and verifying that no authentication errors occur, KDC support can be transitioned to "Fail unarmored authentication requests" (Value 3) for maximum security.

Implementation Steps**### Option A: Group Policy Object (GPO) Configuration (Preferred)**

Configure Domain Controller KDC Policy 1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host. 2. Edit the appropriate Domain Controllers hardening GPO (e.g., `GPO\_Hardening\_DomainControllers`). 3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\KDC` 4. Configure the following setting: * **Policy**: `KDC support for claims, compound authentication and Kerberos armoring` * **Setting**: `Enabled` * **Options**: Select `Supported` from the dropdown list (upgrade to `Fail unarmored authentication requests` only after full client rollout and validation). 5. Link the GPO to the Domain Controllers Organizational Unit (OU).

Configure Client & Member Server Policy 1. In the **Group Policy Management Console**, edit the GPO applied to clients and member servers (e.g., `GPO\_Hardening\_Clients`). 2. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Kerberos` 3. Configure the following settings: * **Policy**: `Kerberos client support for claims, compound authentication and Kerberos armoring` → **Enabled** * **Policy**: `Support device authentication using certificate` → **Enabled** (Select `Automatic` in options) 4. Link the GPO to the appropriate OUs containing workstations and member servers.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally (for testing or standalone systems) or if the control is not manageable via standard GPO GUI interfaces.

[Download Script: Configure-KerberosArmoring.ps1](#)

```
# Configure-KerberosArmoring.ps1
# Description: Configures Kerberos Armoring (FAST) registry settings on Domain Controllers and clients.

Write-Host "Applying hardening requirement: Enable Kerberos Armoring (FAST)..." -ForegroundColor Cyan

$ClientRegPath = "HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies\System\Kerberos\Parameters"
$KdcRegPath = "HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies\System\KDC\Parameters"

# Configure client-side setting (applicable to all systems, including DCs)
if (-not (Test-Path $ClientRegPath)) {
    New-Item -Path $ClientRegPath -Force | Out-Null
}
Set-ItemProperty -Path $ClientRegPath -Name "EnableCbacAndArmor" -Value 1 -Type DWord
Set-ItemProperty -Path $ClientRegPath -Name "DevicePKInitEnabled" -Value 1 -Type DWord
Set-ItemProperty -Path $ClientRegPath -Name "DevicePKInitBehavior" -Value 0 -Type DWord
Write-Host "Client-side Kerberos Armoring and Device Certificate Authentication enabled successfully." -ForegroundColor Green

# Determine if the host is a Domain Controller
$DomainRole = (Get-CimInstance -ClassName Win32_ComputerSystem).DomainRole
$IsDC = ($DomainRole -eq 4) -or ($DomainRole -eq 5)

if ($IsDC) {
    Write-Host "Domain Controller detected. Enabling KDC support for Kerberos Armoring..." -ForegroundColor Cyan
    if (-not (Test-Path $KdcRegPath)) {
        New-Item -Path $KdcRegPath -Force | Out-Null
    }

    # Value 1 = Supported (Safe deployment baseline)
    # Value 3 = Fail unarmored authentication requests (Strict/Enforced state)
    Set-ItemProperty -Path $KdcRegPath -Name "EnableCbacAndArmor" -Value 1 -Type DWord
    Write-Host "KDC support for claims and armoring set to Supported." -ForegroundColor Green
}
}
```

To verify the setting has been applied: [Download Script: Get-KerberosArmoringStatus.ps1](#)


```
# Get-KerberosArmoringStatus.ps1
# Description: Audits the Kerberos Armoring (FAST) configuration on DCs and clients.

Write-Host "--- Auditing Kerberos Armoring (FAST) Configuration ---" -ForegroundColor Cyan

$DomainRole = (Get-CimInstance -ClassName Win32_ComputerSystem).DomainRole
$IsDC = ($DomainRole -eq 4) -or ($DomainRole -eq 5)
$ClientRegPath = "HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies\System\Kerberos\Parameters"
$KdcRegPath = "HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies\System\KDC\Parameters"

# 1. Audit Client-side support
$ClientValue = Get-ItemProperty -Path $ClientRegPath -Name "EnableCbacAndArmor" -ErrorAction SilentlyContinue
$DevicePKInit = Get-ItemProperty -Path $ClientRegPath -Name "DevicePKInitEnabled" -ErrorAction SilentlyContinue
$DeviceBehavior = Get-ItemProperty -Path $ClientRegPath -Name "DevicePKInitBehavior" -ErrorAction SilentlyContinue

if ($null -ne $ClientValue -and $ClientValue.EnableCbacAndArmor -eq 1) {
    Write-Host "[+] Client-side Kerberos Armoring is ENABLED (EnableCbacAndArmor = 1)." -ForegroundColor Green
} else {
    Write-Host "[!] Client-side Kerberos Armoring is DISABLED/MISSING." -ForegroundColor Red
}

if ($null -ne $DevicePKInit -and $DevicePKInit.DevicePKInitEnabled -eq 1 -and $null -ne $DeviceBehavior -and $DeviceBehavior.DevicePKInitBehavior -eq 0) {
    Write-Host "[+] Certificate device authentication is ENABLED: Automatic." -ForegroundColor Green
} else {
    Write-Host "[!] Certificate device authentication is NOT compliant or not configured." -ForegroundColor Red
}

# 2. Audit KDC support if Domain Controller
if ($IsDC) {
    Write-Host "Domain Controller detected. Auditing KDC support..." -ForegroundColor Cyan
    $KdcValue = Get-ItemProperty -Path $KdcRegPath -Name "EnableCbacAndArmor" -ErrorAction SilentlyContinue
    if ($null -ne $KdcValue) {
        $KdcState = $KdcValue.EnableCbacAndArmor
        if ($KdcState -eq 1) {
            Write-Host "[+] KDC support for claims and armoring is ENABLED (Supported: 1)." -ForegroundColor Green
        } elseif ($KdcState -eq 2) {
            Write-Host "[+] KDC support for claims and armoring is ENABLED (Always provide claims: 2)." -ForegroundColor Green
        } elseif ($KdcState -eq 3) {
            Write-Host "[+] KDC support for claims and armoring is ENABLED and ENFORCED (Fail unarmored: 3)." -ForegroundColor Green
        } else {
            Write-Host "[!] KDC support for claims and armoring is configured with unrecognized value: $($KdcState)." -ForegroundColor Red
        }
    } else {
        Write-Host "[!] KDC support for claims and armoring configuration is MISSING (Disabled by default)." -ForegroundColor Red
    }
}
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**: Recommendation R20 (Claims, compound authentication, and Kerberos armoring) * **CIS Benchmark**: CIS Microsoft Windows Server 2016 Benchmark - Section 18.9.4.1 (Ensure 'KDC support for claims, compound authentication and Kerberos armoring' is configured) & Section 18.9.11.1 (Ensure 'Kerberos client support for claims, compound authentication and Kerberos armoring' is configured) & Section 18.9.11.2 (Ensure 'Support device authentication using certificate' is configured) * **Microsoft Security Baseline Focus**: KDC and Kerberos Administrative Templates

[REQ-DC-014] Restrict NTLM

Target Scope * **Applicable Systems**: Domain Controllers, Member Servers, Tier 2 Clients * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10, Windows 11

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` * **Policies**: * `Network security: Restrict NTLM: Audit NTLM authentication in this domain` * `Network security: Restrict NTLM: NTLM authentication in this domain` * `Network security: Restrict NTLM: Audit Incoming NTLM Traffic` * `Network security: Restrict NTLM: Incoming NTLM Traffic` * `Network security: Restrict NTLM: Outgoing NTLM traffic to remote servers` * `Network security: Restrict NTLM: Add remote server exceptions for NTLM authentication` * **Registry Location**: * `HKLM\SYSTEM\CurrentControlSet\Services\Netlogon\Parameters` → `AuditNTLMInDomain` = `7` (REG_DWORD) * `HKLM\SYSTEM\CurrentControlSet\Services\Netlogon\Parameters` → `RestrictNTLMInDomain` = `3` (REG_DWORD) * `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\MSV1\_0` → `AuditReceivingNTLMTraffic` = `2` (REG_DWORD) * `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\MSV1\_0` → `RestrictReceivingNTLMTraffic` = `2` (REG_DWORD) * `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\MSV1\_0` → `RestrictSendingNTLMTraffic` = `2` (REG_DWORD)

Rationale The NT LAN Manager (NTLM) authentication protocol is legacy, cryptographically weak, and lacks support for modern security primitives such as mutual authentication. Adversaries exploit NTLM through credential relaying attacks (e.g., replaying captured authentication responses to other network services) and coercion techniques (e.g., PetitPotam or printer spooler RPC abuse).

By auditing and subsequently restricting NTLM authentication incoming to, outgoing from, and within the Active Directory domain, organizations significantly mitigate the risks of credential relaying, offline password cracking, and unauthorized lateral movement. Restricting NTLM pushes client machines and application servers to utilize Kerberos, which provides robust mutual authentication and support for advanced cryptographic algorithms. Microsoft has also announced the eventual deprecation of NTLM, making active restriction an essential step in future-proofing active directory services.

Legacy Impact & Compatibility * **Auditing Requirement**: Administrators must enable auditing policies prior to enforcing restriction policies. Review the operational logs at `Applications and Services Logs > Microsoft > Windows > NTLM > Operational` for Event ID `8004` (Domain Controller NTLM validation), `8001` (incoming NTLM block auditing), and `8003` (outgoing NTLM block auditing) to identify legitimate business applications requiring NTLM. * **Authentication Failures**: Hard enforcement of NTLM restrictions will block applications that use hardcoded IP addresses (preventing Kerberos SPN mapping), non-domain joined assets, and legacy appliances. * **Exceptions**: Remote servers that cannot be transitioned to Kerberos must be explicitly added to the GPO exception list (`Network security: Restrict NTLM: Add remote server exceptions for NTLM authentication`) to prevent application outages.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Phase 1: Enable Auditing (Apply to Domain Controllers, Servers, and Clients) 1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host. 2. Edit the appropriate hardening GPO (e.g., `GPO\_Hardening\_DomainControllers`). 3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` 4. Configure the following audit policies: * **Policy**: `Network security: Restrict NTLM: Audit NTLM authentication in this domain` * **Setting**: `Enable all` (Apply to Domain Controllers) * **Policy**: `Network security: Restrict NTLM: Audit Incoming NTLM Traffic` * **Setting**: `Enable auditing for all accounts` (Apply to DCs, member servers, and clients) * **Policy**: `Network security: Restrict NTLM: Outgoing NTLM traffic to remote servers` * **Setting**: `Audit all` (Apply to DCs, member servers, and clients)

Phase 2: Enforce Restrictions (Apply after logs have been verified and exception lists populated) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` 2. Configure the following restriction policies: * **Policy**: `Network security: Restrict NTLM: NTLM authentication in this domain` * **Setting**: `Deny for domain accounts` (Apply to Domain Controllers) * **Policy**: `Network security: Restrict NTLM: Incoming NTLM Traffic` * **Setting**: `Deny all accounts` (Apply to member servers and clients) * **Policy**: `Network security: Restrict NTLM: Outgoing NTLM traffic to remote servers` * **Setting**: `Deny all` (Apply to member servers and clients) * **Policy**: `Network security: Restrict NTLM: Add remote server exceptions for NTLM authentication` * **Setting**: Configure the server hostnames or FQDNs that require exemption.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally (for testing or standalone systems) or if the control is not manageable via standard GPO GUI interfaces.

[Download Script: Configure-RestrictNTLM.ps1](#)

```
# Configure-RestrictNTLM.ps1
# Description: Configures local registry values to enforce NTLM restrictions and auditing.

Write-Host "Applying hardening requirement: Restrict NTLM..." -ForegroundColor Cyan

$LsaPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa\MSV1_0"
$NetLogonPath = "HKLM:\SYSTEM\CurrentControlSet\Services\Netlogon\Parameters"

# Ensure LSA MSV1_0 path exists and apply settings
if (-not (Test-Path $LsaPath)) {
    New-Item -Path $LsaPath -Force | Out-Null
}

Set-ItemProperty -Path $LsaPath -Name "AuditReceivingNTLMTraffic" -Value 2 -Type DWord
Set-ItemProperty -Path $LsaPath -Name "RestrictReceivingNTLMTraffic" -Value 2 -Type DWord
Set-ItemProperty -Path $LsaPath -Name "RestrictSendingNTLMTraffic" -Value 2 -Type DWord

# Ensure Netlogon Parameters path exists and apply settings
if (-not (Test-Path $NetLogonPath)) {
    New-Item -Path $NetLogonPath -Force | Out-Null
}

Set-ItemProperty -Path $NetLogonPath -Name "AuditNTLMInDomain" -Value 7 -Type DWord
Set-ItemProperty -Path $NetLogonPath -Name "RestrictNTLMInDomain" -Value 3 -Type DWord

Write-Host "NTLM restriction registry configurations applied successfully." -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-RestrictNTLMStatus.ps1](#)

```
# Get-RestrictNTLMStatus.ps1
# Description: Audits the registry configuration for NTLM auditing and restrictions.

Write-Host "--- Auditing NTLM Restrictions ---" -ForegroundColor Cyan

$LsaPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa\MSV1_0"
$NetlogonPath = "HKLM:\SYSTEM\CurrentControlSet\Services\Netlogon\Parameters"

if (Test-Path $LsaPath) {
    $AuditRecv = Get-ItemProperty -Path $LsaPath -Name "AuditReceivingNTLMTraffic" -ErrorAction SilentlyContinue
    if ($AuditRecv) {
        Write-Host "[+] AuditReceivingNTLMTraffic is set to $($AuditRecv.AuditReceivingNTLMTraffic) (Secure)." -ForegroundColor Green
    } else {
        Write-Host "[!] VULNERABLE: AuditReceivingNTLMTraffic is not configured." -ForegroundColor Red
    }

    $RestrictRecv = Get-ItemProperty -Path $LsaPath -Name "RestrictReceivingNTLMTraffic" -ErrorAction SilentlyContinue
    if ($RestrictRecv) {
        Write-Host "[+] RestrictReceivingNTLMTraffic is set to $($RestrictRecv.RestrictReceivingNTLMTraffic) (Secure)." -ForegroundColor Green
    } else {
        Write-Host "[!] VULNERABLE: RestrictReceivingNTLMTraffic is not configured." -ForegroundColor Red
    }

    $RestrictSend = Get-ItemProperty -Path $LsaPath -Name "RestrictSendingNTLMTraffic" -ErrorAction SilentlyContinue
    if ($RestrictSend) {
        Write-Host "[+] RestrictSendingNTLMTraffic is set to $($RestrictSend.RestrictSendingNTLMTraffic) (Secure)." -ForegroundColor Green
    } else {
        Write-Host "[!] VULNERABLE: RestrictSendingNTLMTraffic is not configured." -ForegroundColor Red
    }
} else {
    Write-Host "[!] LSA MSV1_0 path does not exist." -ForegroundColor Red
}

if (Test-Path $NetlogonPath) {
    $AuditDomain = Get-ItemProperty -Path $NetlogonPath -Name "AuditNTLMInDomain" -ErrorAction SilentlyContinue
    if ($AuditDomain) {
        Write-Host "[+] AuditNTLMInDomain is set to $($AuditDomain.AuditNTLMInDomain) (Secure)." -ForegroundColor Green
    } else {
        Write-Host "[!] VULNERABLE: AuditNTLMInDomain is not configured." -ForegroundColor Red
    }

    $RestrictDomain = Get-ItemProperty -Path $NetlogonPath -Name "RestrictNTLMInDomain" -ErrorAction SilentlyContinue
    if ($RestrictDomain) {
        Write-Host "[+] RestrictNTLMInDomain is set to $($RestrictDomain.RestrictNTLMInDomain) (Secure)." -ForegroundColor Green
    } else {
        Write-Host "[!] VULNERABLE: RestrictNTLMInDomain is not configured." -ForegroundColor Red
    }
} else {
    Write-Host "[!] Netlogon Parameters path does not exist." -ForegroundColor Red
}
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**: Recommendation R13 (Disabling obsolete and insecure protocols) *
 CIS Benchmark: CIS Microsoft Windows Server Benchmark - Section 2.3.7.5 to 2.3.7.9 (Restrict NTLM policies) * **Microsoft Security
 Guidance**: Restrict NTLM Audit and Restrict Policies

[REQ-DC-015] Migrate SYSVOL Replication to DFSR

Target Scope * **Applicable Systems**: Domain Controllers * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: Domain Controller replication system attributes (managed via `dfsrmig.exe` tool and the DFSR service)

Rationale Active Directory domain environments rely on replication to synchronize GPO templates and scripts stored in the `SYSVOL` share across all Domain Controllers. Historically, this replication was managed by the File Replication Service (FRS).

However, FRS is obsolete, does not support modern transport-layer security features, and is prone to replication database corruption.

Migrating to Distributed File System Replication (DFSR):

1. **Ensures Replication Integrity:** DFSR uses remote differential compression algorithms and hash validation to verify that files are replicated securely and without corruption.
2. **Minimizes Attack Surface:** Transitioning to DFSR allows security administrators to completely disable and deprecate the legacy, insecure FRS service and its associated RPC endpoints.
3. **Improves Diagnostics:** DFSR provides comprehensive logging, system health auditing, and diagnostic reports, allowing quick identification of synchronization failures.

Legacy Impact & Compatibility * **Replication Health Requirement**: Prior to migration, Active Directory and file replication must be fully synchronized and healthy. Any existing replication errors will prevent successful migration and can lead to domain-wide split-brain scenarios. * **One-Way Committal**: The final stage of the migration (State 3: Eliminated) is permanent. FRS databases are deleted, and it is impossible to roll back to FRS without restoring Domain Controllers from backups. * **System Requirements**: All Domain Controllers in the domain must run Windows Server 2008 or later, and the Domain Functional Level (DFL) must be raised to at least Windows Server 2008.

Implementation Steps

Option A: Manual Step-by-Step Migration Command Line

The migration process consists of transitioning the domain through four replication states (from State 0 to State 3) using the `dfsrmig` command line tool on a writable Domain Controller (typically the PDC Emulator).

1. **Verify Current State:** Open an elevated command prompt on the DC and run: `dfsrmig /getglobalstate` *Expected starting output:* `Current DFSR global state: 'Start'` (State 0).
2. **Transition to Prepared State (State 1):** This state creates a clone of the `SYSVOL` folder named `SYSVOL_DFSR` and begins DFSR replication in the background while keeping the original `SYSVOL` folder active via FRS. `dfsrmig /setglobalstate 1`
3. **Verify State 1 Reachability:** Query all Domain Controllers to ensure they have successfully transitioned to the Prepared state: `dfsrmig /getmigrationstate` *Do not proceed to the next step until the output confirms:* `All Domain Controllers have migrated successfully to the Global state ('Prepared')`.
4. **Transition to Redirected State (State 2):** This state redirects the active `SYSVOL` network share mapping to point to the new `SYSVOL_DFSR` folder replicated by DFSR. FRS continues replication in the background for backward compatibility/rollback capability. `dfsrmig /setglobalstate 2`
5. **Verify State 2 Reachability:** Query the status to confirm all DCs have successfully redirected: `dfsrmig /getmigrationstate` *Do not proceed until the output confirms:* `All Domain Controllers have migrated successfully to the Global state ('Redirected')`.
6. **Transition to Eliminated State (State 3):** This final stage commits the migration. It deletes the original `SYSVOL` directory, stops and disables the FRS service, and deletes the FRS replication configuration from Active Directory. `dfsrmig /setglobalstate 3`
7. **Verify State 3 Committal:** Confirm the migration is complete: `dfsrmig /getmigrationstate` *Expected Output:* `All Domain Controllers have migrated successfully to the Global state ('Eliminated')`.

Option B: PowerShell Migration Status Auditing

Use this PowerShell script to monitor the progress of the DFSR migration across all Domain Controllers in the forest.

[Download Script: Get-SYSVOLDfsrMigrationStatus.ps1](#)

```
# Get-SYSVOLDfsrMigrationStatus.ps1
# Description: Checks the current SYSVOL replication migration state.

Import-Module ActiveDirectory

Write-Host "--- Auditing SYSVOL DFSR Migration Status ---" -ForegroundColor Cyan

# Check if the FRS service is still running on this DC
$FrsService = Get-Service -Name "NtFrs" -ErrorAction SilentlyContinue

if ($null -ne $FrsService) {
    Write-Host "[*] FRS Service State: $($FrsService.Status)" -ForegroundColor White
} else {
    Write-Host "[+] FRS Service is not installed (expected in modern server configurations)." -ForegroundColor Green
}

# Run dfsrmig validation checks
$DfsMigOutput = & dfsrmig.exe /getglobalstate 2>&1

if ($DfsMigOutput -like "*Eliminated*") {
    Write-Host "[+] SYSVOL migration to DFSR is complete and finalized (State 3: Eliminated)." -ForegroundColor Green
} else {
    Write-Host "[-] WARNING: SYSVOL replication is not fully migrated to DFSR." -ForegroundColor Red
    Write-Host "    Current Status: $DfsMigOutput" -ForegroundColor Yellow
}
}
```

To verify active DFSR health on the server: [Download Script: Get-DfsrHealthStatus.ps1](#)

```
# Get-DfsrHealthStatus.ps1
# Description: Checks the event logs for DFSR replication errors.

Write-Host "Checking DFSR replication event logs..." -ForegroundColor Cyan

$DfsrEvents = Get-WinEvent -LogName "DFS Replication" -MaxEvents 10 -ErrorAction SilentlyContinue

if ($DfsrEvents) {
    foreach ($DfsrEvent in $DfsrEvents) {
        $EventColor = if ($DfsrEvent.LevelDisplayName -eq "Error") { "Red" } else { "White" }
        Write-Host "[$($DfsrEvent.TimeCreated)] [$($DfsrEvent.LevelDisplayName)] ID: $($DfsrEvent.Id) - $($DfsrEvent.Message)" -ForegroundColor $EventColor
    }
} else {
    Write-Host "[+] No recent DFSR events or errors detected." -ForegroundColor Green
}
}
```

Sources & Compliance References * **ANSSI Remediation of Active Directory Tier 0 Guide***: Section 5.e (Page 31) * **ANSSI AD Hardening Guide***: Section 3.1.1 (Niveaux fonctionnels) * **Microsoft Security Guidance***: Migrate SYSVOL Replication to DFS Replication (DFSR) Guide

[REQ-DC-016] Harden adminSDHolder Permissions

Target Scope * **Applicable Systems**: Domain Controllers * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: Active Directory Object Path: 'CN=adminSDHolder,CN=System,DC=[Domain]'

Rationale In Active Directory, the 'adminSDHolder' object acts as a security template for administrative accounts and groups (known as protected objects). Every hour, a background system thread (the 'AdminSDHolder' task, also referred to as the 'ProtectAdminGroups' task) running on the Domain Controller holding the PDC Emulator role compares the Access Control Lists (ACLs) of all protected objects against the ACL of the 'adminSDHolder' object. If they differ, the ACL on the protected object is overwritten by the ACL on 'adminSDHolder', and security inheritance is disabled.

A common misconception is that the Security Descriptor Propagator (SDProp) thread enforces this protection. In reality, SDProp is a separate background thread on all Domain Controllers whose sole job is to propagate inheritable Access Control Entries (ACEs) when an object is moved or a parent ACL is modified. The actual enforcement of the adminSDHolder template is performed exclusively by the AdminSDHolder background task.

If an attacker gains temporary write permissions on the adminSDHolder object, they can inject a backdoor Access Control Entry (ACE) granting their account write permissions. Within an hour, the AdminSDHolder background task will apply this backdoor ACE to all protected groups (e.g., Domain Admins, Schema Admins, Enterprise Admins). Even if the administrator cleans up permissions on a Domain Admin account directly, the AdminSDHolder task will restore the backdoor ACE on its next run.

The Importance of Container Inheritance Protection (DACL_Protected) By default, the 'adminSDHolder' container itself ('CN=adminSDHolder,CN=System,DC=[Domain]') has its DACL protected (inheritance blocked). This prevents permissions delegated on the parent 'CN=System' container or the Domain Root from inheriting down to the 'adminSDHolder' object. If inheritance is accidentally enabled on 'CN=adminSDHolder', any account with write permissions delegated on parent containers would implicitly gain control over 'adminSDHolder', resulting in full domain compromise when those permissions are propagated to all administrative accounts.

Protection of Non-User Objects The 'AdminSDHolder' task is not restricted to user objects and groups. It will also protect computer objects, Managed Service Accounts (MSAs), and Group Managed Service Accounts (gMSAs) if they are added to a protected group (such as 'Administrators' or 'Domain Admins'), provided they are not members of an excluded group like 'Domain Controllers' or 'Read-Only Domain Controllers' (which are excluded to prevent replication and authentication breakages).

Therefore:

- 1. Prevents Privilege Escalation Backdoors:** Auditing and hardening the adminSDHolder ACL ensures that unauthorized accounts cannot establish persistent, self-healing backdoors.
- 2. Maintains Tier 0 Isolation:** Restricting write access on adminSDHolder strictly to Tier 0 accounts keeps the tiered boundary intact.
- 3. Protects Against Inherited Backdoors:** Enforcing inheritance blocking on the adminSDHolder container prevents lateral permission leakage from parent containers.

Legacy Impact & Compatibility * **Administrative Operations**: If custom scripts or third-party provisioning systems rely on writing directly to protected accounts without belonging to the built-in protected groups, modifying the 'adminSDHolder' ACL may disrupt their operations. Ensure all automated directory tools are audited before removing custom ACL entries. * **Pre-Remediation Check**: Ensure that only trusted, built-in system groups (such as Domain Admins, Enterprise Admins, and SYSTEM) have Write and Modify permissions on the 'adminSDHolder' object. * **Service Accounts and gMSAs**: If service accounts or gMSAs are placed in administrative groups, their object DACLs will also be protected and set to block inheritance. Ensure this does not break service delegation or management permissions.

Implementation Steps

Option A: Active Directory Users and Computers (ADUC) Console Configuration

1. Open **Active Directory Users and Computers** (`dsa.msc`) on a Domain Controller.
2. Select **View** and click **Advanced Features** (if not already enabled).
3. Navigate to: `System\adminSDHolder`
4. Right-click **adminSDHolder** and select **Properties**.
5. Select the **Security** tab.
6. Click **Advanced**.
7. Enforce inheritance blocking:
 - Verify that the button at the bottom reads **Enable Inheritance**. If it reads **Disable Inheritance**, click it to block inheritance.
 - When prompted, choose to convert inherited permissions into explicit permissions to avoid losing default settings, then prune any non-compliant explicit permissions.
8. Review the permission entries:
 - Ensure only highly privileged built-in groups (e.g., `Domain Admins` , `Enterprise Admins` , `SYSTEM`) have Write, Modify, or Full Control permissions.

- Remove any entries granting permissions to non-Tier 0 accounts, such as delegated helpdesk groups, `Account Operators`, `Print Operators`, or custom service accounts.

9. Click **OK** to save changes.

Option B: PowerShell Configuration (Remediation / Non-GPO)

Run the following script to audit and remediate unauthorized permissions on the `adminSDHolder` object.

[Download Script: Harden-AdminSDHolder.ps1](#)


```
# Harden-AdminSDHolder.ps1
# Description: Hardens the adminSDHolder ACL by auditing permissions, blocking inheritance, and removing delegated helpdesk groups.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Harden adminSDHolder Permissions..." -ForegroundColor Cyan

# Set target DN
$DomainDN = (Get-ADRootDSE).defaultNamingContext
$AdminSDPath = "AD:\CN=adminSDHolder,CN=System,$($DomainDN)"

# Define allowed high-privilege built-in identities (SIDs or names)
$AllowedTrustees = @(
    "SYSTEM",
    "Domain Admins",
    "Enterprise Admins",
    "Administrators"
)

# Fetch ACL
$Acl = Get-Acl -Path $AdminSDPath
$AclModified = $false

# 1. Enforce inheritance blocking (DACL_Protected flag)
if ($Acl.AreAccessRulesProtected) {
    Write-Host "[+] Inheritance is already blocked (protected) on the adminSDHolder container." -ForegroundColor Green
} else {
    Write-Host "[-] adminSDHolder container has inheritance enabled. Blocking inheritance..." -ForegroundColor Yellow
    # Block inheritance ($true) and copy existing rules as explicit ($true)
    $Acl.SetAccessRuleProtection($true, $true)
    $AclModified = $true
}

# 2. Prune non-compliant permissions
foreach ($Rule in $Acl.Access) {
    $Identity = $Rule.IdentityReference.Value

    # Check if trustee is allowed to write
    if ($Rule.ActiveDirectoryRights -match "WriteProperty|WriteDacl|WriteOwner|GenericAll|GenericWrite") {
        $IsAllowed = $false
        foreach ($Allowed in $AllowedTrustees) {
            if ($Identity -match $Allowed) {
                $IsAllowed = $true
                break
            }
        }

        if (-not $IsAllowed) {
            Write-Host "[-] Unauthorized write permission found: Account '$($Identity)' has rights: $($Rule.ActiveDirectoryRights)" -ForegroundColor Yellow

            # Remove rule
            $Acl.RemoveAccessRule($Rule) | Out-Null
            $AclModified = $true
        }
    }
}

if ($AclModified) {
    Set-Acl -Path $AdminSDPath -AclObject $Acl -ErrorAction Stop
    Write-Host "[+] adminSDHolder ACL hardened successfully." -ForegroundColor Green
} else {
    Write-Host "[+] adminSDHolder ACL is already clean." -ForegroundColor Green
}
```

To verify current adminSDHolder permissions: [Download Script: Get-AdminSDHolderAudit.ps1](#)

```
# Get-AdminSDHolderAudit.ps1
# Description: Audits and prints all active permission entries and the inheritance block status on adminSDHolder.

Import-Module ActiveDirectory

Write-Host "--- Auditing adminSDHolder Permissions ---" -ForegroundColor Cyan

$DomainDN = (Get-ADRootDSE).defaultNamingContext
$AdminSDPath = "AD:\CN=adminSDHolder,CN=System,$($DomainDN)"
$Acl = Get-Acl -Path $AdminSDPath

# Check inheritance block status
if ($Acl.AreAccessRulesProtected) {
    Write-Host "[+] Inheritance is BLOCKED (Protected) on the adminSDHolder container (Secure)." -ForegroundColor Green
} else {
    Write-Host "[!] VULNERABLE: Inheritance is ENABLED (Unprotected) on the adminSDHolder container. Permissions may inherit from parent objects." -ForegroundColor Red
}

foreach ($Rule in $Acl.Access) {
    $Identity = $Rule.IdentityReference.Value
    $Rights = $Rule.ActiveDirectoryRights
    $Inheritance = $Rule.InheritanceType

    $Color = if ($Rights -match "WriteProperty|WriteDacl|WriteOwner|GenericAll") { "Yellow" } else { "Gray" }

    Write-Host "[*] Trustee: $($Identity)" -ForegroundColor White
    Write-Host "    - Rights: $($Rights)" -ForegroundColor $Color
    Write-Host "    - Inheritance: $($Inheritance)" -ForegroundColor Gray
}
```

Sources & Compliance References * **ANSSI Remediation of Active Directory Tier 0 Guide**¹: Section 5.d (Page 31) * **ANSSI AD Hardening Guide**²: Recommendation R23 * **Microsoft Security Guidance**³: Protected Accounts and Groups in Active Directory * **CIS Benchmark**⁴: Section 2.2 (Active Directory Object Access Permissions) * **SpecterOps Research**⁵: AdminSDHolder: Misconceptions, Misconfigurations, and Myths (October 2025)

[REQ-DC-017] Harden Microsoft DNS AD Container Permissions

Target Scope * **Applicable Systems**: Domain Controllers * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **Active Directory Path**:

`CN=MicrosoftDNS,CN=System,DC=[Domain]` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Services\DNS\Parameters` * `ServerLevelPluginDll` = `""` (REG_SZ)

Rationale In Active Directory-integrated DNS zones, DNS server configurations and zone data are stored inside the directory. By default, members of the built-in `DnsAdmins` group have write permissions over the properties of the `CN=MicrosoftDNS,CN=System` container. This default configuration presents a critical privilege escalation vector:

1. **DLL Hijacking via DNS Service:** An account with write permissions on the DNS container can specify a path to a malicious DLL in the `ServerLevelPluginDll` parameter. The DNS server service (which runs as `SYSTEM` on Domain Controllers) will load this DLL upon restart, executing arbitrary code with system privileges and leading to full Domain Controller takeover.
2. **Restricts DNS Management Boundary:** Restricting write access on the container properties and auditing the membership of the `DnsAdmins` group prevents lower-tier administrators (such as Tier 1 network administrators) from escalating their privileges to Tier 0.

Legacy Impact & Compatibility * **DNS Administrative Workloads**: Delegated network administrators who are not members of Tier 0 but manage DNS records may fail to perform some zone maintenance if they rely on membership in the default `DnsAdmins` group. They should instead be granted delegated rights over specific DNS zones rather than write access on the main DNS system container. * **Vulnerability Mitigation**: Ensure that Microsoft security update CVE-2021-40469 is installed on all Domain Controllers to harden the DNS plugin loading behavior.

Implementation Steps

Option A: Active Directory Users and Computers (ADUC) Console Configuration

1. Open **Active Directory Users and Computers** (`dsa.msc`) on a Domain Controller.
2. Ensure **View** → **Advanced Features** is checked.
3. Navigate to: `System\MicrosoftDNS`
4. Right-click **MicrosoftDNS** and select **Properties**.
5. Select the **Security** tab.
6. Select the **DnsAdmins** group or any non-Tier 0 administrative user/group.
7. Click **Advanced**.
8. Ensure they do not possess **Write all properties** or **Full Control** over the container.
9. Click **OK** to apply.
10. Open **Active Directory Users and Computers** and navigate to the `Builtin` or `Users` container.
11. Double-click the **DnsAdmins** group and ensure only Tier 0 accounts are members. Remove any non-Tier 0 identities.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script block to audit and remove the `ServerLevelPluginDll` registry backdoor on Domain Controllers, and verify DnsAdmins membership.

Download Script: [Harden-DnsServerConfiguration.ps1](#)

```
# Harden-DnsServerConfiguration.ps1
# Description: Deletes any ServerLevelPluginDll entry to block DNS DLL hijacking, and checks DnsAdmins group.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Harden Microsoft DNS AD Container..." -ForegroundColor Cyan

# 1. Clean up ServerLevelPluginDll Registry Key
$RegPath = "HKLM:\System\CurrentControlSet\Services\DNS\Parameters"
$ValueName = "ServerLevelPluginDll"

if (Test-Path $RegPath) {
    $PluginDll = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
    if ($null -ne $PluginDll) {
        Write-Host "[-] WARNING: Potentially unauthorized DNS plugin detected: $($PluginDll.ServerLevelPluginDll)" -ForegroundColor
        Yellow
        Remove-ItemProperty -Path $RegPath -Name $ValueName -Force -ErrorAction Stop
        Write-Host "[+] ServerLevelPluginDll registry parameter removed successfully." -ForegroundColor Green
    } else {
        Write-Host "[+] No ServerLevelPluginDll registry parameter found (clean configuration)." -ForegroundColor Green
    }
}

# 2. Audit DnsAdmins Membership
$DnsAdminsGroup = Get-ADGroup -Filter "Name -eq 'DnsAdmins'" -ErrorAction SilentlyContinue

if ($null -ne $DnsAdminsGroup) {
    $Members = Get-ADGroupMember -Identity $DnsAdminsGroup
    if ($Members.Count -gt 0) {
        Write-Host "[-] WARNING: The DnsAdmins group contains active members. Please verify that all members are Tier 0 identities."
        -ForegroundColor Yellow
        foreach ($Member in $Members) {
            Write-Host "    - Member: $($Member.SamAccountName) ($($Member.objectClass))" -ForegroundColor White
        }
    } else {
        Write-Host "[+] The DnsAdmins group is empty (recommended)." -ForegroundColor Green
    }
}
}
```

To verify active DNS parameters and container permissions: [Download Script: Get-DnsAuditStatus.ps1](#)

```
# Get-DnsAuditStatus.ps1
# Description: Queries the DNS registry parameter settings and AD container ACLs.

Import-Module ActiveDirectory

Write-Host "--- Auditing DNS Security Parameters ---" -ForegroundColor Cyan

# Check Registry
$RegPath = "HKLM:\System\CurrentControlSet\Services\DNS\Parameters"
$ValueName = "ServerLevelPluginDll"

if (Test-Path $RegPath) {
    $Val = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
    if ($null -ne $Val) {
        Write-Host "[!] Danger: ServerLevelPluginDll is configured: $($Val.ServerLevelPluginDll)" -ForegroundColor Red
    } else {
        Write-Host "[+] ServerLevelPluginDll: Not configured (Secure)." -ForegroundColor Green
    }
}

# Check AD Container Write ACLs
$DomainDN = (Get-ADRootDSE).defaultNamingContext
$DnsPath = "AD:\CN=MicrosoftDNS,CN=System,$($DomainDN)"
$Acl = Get-Acl -Path $DnsPath

Write-Host "Reviewing MicrosoftDNS AD container access permissions..." -ForegroundColor White
foreach ($Rule in $Acl.Access) {
    if ($Rule.ActiveDirectoryRights -match "WriteProperty|GenericAll|GenericWrite") {
        Write-Host "    - Trustee: $($Rule.IdentityReference.Value) | Rights: $($Rule.ActiveDirectoryRights)" -ForegroundColor Yellow
    }
}
}
```

Sources & Compliance References * **ANSSI Remediation of Active Directory Tier 0 Guide***: Section 3.e (Page 23) * **ANSSI AD Hardening Guide***: Section 3.2.1, Section 3.6, Section 9 * **Microsoft Security Response Center***: CVE-2021-40469 Mitigation * **Other Reference***: CVE-2021-40469 (Windows DNS Server Remote Code Execution Vulnerability)

[REQ-DC-018] Harden Virtualization Hosts for Domain Controllers

Target Scope * **Applicable Systems**: Virtualization Hosts (Hyper-V / VMware ESXi hosting Domain Controllers) * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, VMware vSphere ESXi 6.7+

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: Hypervisor Host Group Policies, Local Host Firewall Configuration, and Virtualization Management boundaries

Rationale In modern IT environments, Domain Controllers (DCs) are frequently virtualized. However, a virtualized DC is only as secure as the physical host and hypervisor running it.

If a hypervisor hosting a DC is compromised, an attacker can:

1. **Harvest NTDS.dit Offline:** Copy the virtual hard disk file (VHDX/VMDK) of the running DC and extract all domain password hashes offline.
2. **Manipulate Memory:** Dump the guest DC's LSASS memory directly from the hypervisor console, exposing active Tier 0 administrator credentials.
3. **Inject Arbitrary Commands:** Use integration tools (like Hyper-V Integration Services or VMware Tools) to run code within the guest OS without authenticating.

To mitigate these risks:

- **Host Segregation:** Hypervisors hosting Tier 0 Domain Controllers must be dedicated exclusively to Tier 0 workloads. Lower-tier virtual machines (Tier 1/2) must never run on the same physical host clusters.
- **Administrative Isolation:** The virtualization hosts and management consoles (e.g., vCenter, Hyper-V Manager) must be managed exclusively by Tier 0 administrative accounts.
- **Virtual Machine Security:** Enable shielded VMs or VM encryption options to cryptographically lock the guest OS resources and protect virtual disks from unauthorized access.

Legacy Impact & Compatibility * **Infrastructure Cost**: Dedicating specific physical hardware hosts solely for Domain Controllers and other Tier 0 VMs reduces server hardware resource utilization, potentially increasing cost. * **Management Disruption**: Standard virtualization administrators will lose the ability to manage the hosts running Domain Controllers, requiring a separate, dedicated administrative pool.

Implementation Steps

Option A: Manual Virtualization Host Isolation (Preferred)

1. Identify all physical hosts running virtualized Domain Controllers.
2. Move all non-Tier 0 virtual machines off these hosts using live migration (vMotion/Live Migration).
3. Place these hosts into a dedicated, isolated hypervisor cluster (e.g., `Cluster-Tier0`).
4. Reconfigure management permissions on the virtualization console:
 - Remove standard admin permissions from the cluster.
 - Restrict access rights exclusively to a dedicated Tier 0 virtualization administrator group.
5. Disable unnecessary virtual integration services in the VM settings:
 - In Hyper-V Manager, open DC VM Properties → **Integration Services** → Uncheck **Guest services** and **Time synchronization** (if relying on NTP domain hierarchy).

Option B: PowerShell Host Configuration Auditing

Run the following script block on a Domain Controller to determine if it is virtualized, identify the hypervisor type, and audit key integration parameters.

[Download Script: Get-DcVirtualizationStatus.ps1](#)

```
# Get-DcVirtualizationStatus.ps1
# Description: Audits the virtualization environment of the Domain Controller.

Write-Host "--- Auditing DC Virtualization Status ---" -ForegroundColor Cyan

# 1. Determine if running on physical or virtual hardware
$ComputerSystem = Get-CimInstance -ClassName Win32_ComputerSystem -ErrorAction Stop
$Model = $ComputerSystem.Model
$Manufacturer = $ComputerSystem.Manufacturer

Write-Host "[*] Host Manufacturer: $($Manufacturer)" -ForegroundColor White
Write-Host "[*] System Model:      $($Model)" -ForegroundColor White

$IsVirtual = $false
$HypervisorType = "Unknown"

if ($Model -match "Virtual Machine|VMware|VirtualBox|Xen") {
    $IsVirtual = $true
    if ($Manufacturer -match "Microsoft") { $HypervisorType = "Hyper-V" }
    elseif ($Manufacturer -match "VMware") { $HypervisorType = "VMware ESXi" }
}

if ($IsVirtual) {
    Write-Host "[-] WARNING: Domain Controller is virtualized on $($HypervisorType)." -ForegroundColor Yellow
    Write-Host "    Ensure the underlying host is secured as a Tier 0 asset." -ForegroundColor Yellow

    # 2. Check VM integration service settings if Hyper-V guest
    if ($HypervisorType -eq "Hyper-V") {
        $IntegrationServices = Get-Service -Name "vm*" -ErrorAction SilentlyContinue
        if ($IntegrationServices) {
            Write-Host "    Integration Services detected:" -ForegroundColor White
            foreach ($Svc in $IntegrationServices) {
                Write-Host "        - $($Svc.Name): $($Svc.Status)" -ForegroundColor White
            }
        }
    }
} else {
    Write-Host "[+] Domain Controller is running on physical hardware (secure boundary)." -ForegroundColor Green
}
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**:

- Recommendation R47 (Risques relatifs aux infrastructures de virtualisation)
- * **ANSSI Remediation of Active Directory Tier 0 Guide**:
- Section 3.d (Page 23), Section 4.d (Page 27)
- * **Microsoft Security Guidance**:
- Securing Domain Controllers Against Virtualization Attacks
- * **CIS Benchmark**:
- Section 2.3.1.2 (Harden Hypervisor Access Control)

[REQ-DC-019] Enforce RDP Restricted Admin Mode

Target Scope * **Applicable Systems**: Domain Controllers, Member Servers, Tier 2 Clients, PAWs * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10/11 Enterprise

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **Client Configuration (GPO)**: `Computer Configuration\Policies\Administrative Templates\System\Credentials Delegation\Restrict delegation of credentials to remote servers` → Enabled (Require Restricted Admin) * **RDP Session Security GPO**: * `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Connection Client\Do not allow passwords to be saved` → Enabled * `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Connections\Restrict Remote Desktop Services users to a single Remote Desktop Services session` → Enabled * `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Device and Resource Redirection\Do not allow drive redirection` → Enabled * `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Device and Resource Redirection\Do not allow COM port redirection` → Enabled * `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Device and Resource Redirection\Do not allow LPT port redirection` → Enabled * `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Device and Resource Redirection\Do not allow supported Plug and Play device redirection` → Enabled * `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Security\Always prompt for password upon connection` → Enabled * `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Security\Require secure RPC communication` → Enabled * `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Security\Set client connection encryption level` → Enabled (Encryption Level: High Level) * `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Session Time Limits\Set time limit for active but idle Remote Desktop Services sessions` → Enabled: 15 minutes or less, but not Never (0) * `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Session Time Limits\Set time limit for disconnected sessions` → Enabled: 1 minute * **Server Configuration (Registry)**: * `HKLM\System\CurrentControlSet\Control\Lsa\DisableRestrictedAdmin` → `0` (DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services` → `DisablePasswordSaving` = `1` (DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services` → `fSingleSessionPerUser` = `1` (DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services` → `fDisableCdm` = `1` (DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services` → `fDisableCcm` = `1` (DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services` → `fDisableLpt` = `1` (DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services` → `fDisablePNPRedir` = `1` (DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services` → `fPromptForPassword` = `1` (DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services` → `fEncryptRPCTraffic` = `1` (DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services` → `MinEncryptionLevel` = `3` (DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services` → `MaxIdleTime` = `900000` (DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services` → `MaxDisconnectionTime` = `60000` (DWORD)

Rationale Remote Desktop Protocol (RDP) is a standard tool for administrative sessions. However, by default, RDP authenticates users by placing their credentials (NTLM hashes or Kerberos tickets) directly in the Local Security Authority Subsystem Service (LSASS) memory of the destination host.

If an administrator connects to a compromised host (such as a Tier 1 server or Tier 2 workstation) from their workstation using standard RDP, an attacker with local administrator privileges on that target system can dump LSASS and harvest the administrator's credentials. This allows the attacker to compromise the administrator's account and move laterally or escalate privileges.

Enforcing RDP Restricted Admin Mode (RDP RA) prevents credential harvesting:

1. **Blocks Credential Transmission**: Under Restricted Admin mode, the client does not send the user's plaintext password, NTLM hash, or Kerberos TGT to the remote host. The host validates the connection without caching reusable credentials in its LSASS database.
 2. **Limits Remote Session Privilege**: In network access attempts initiated from within the RDP session, the remote session runs in the security context of the destination machine's computer account (`$MachineName$`) rather than the administrator's user account.
-

Legacy Impact & Compatibility * **No Network SSO**: Because the remote session lacks the user's credentials, the administrator cannot access other network shares or Active Directory resources (such as mapped drives or remote management tools) from *within* the RDP session. Any cross-machine administrative tasks must be executed from the administrator's local management host using tools like RSAT or PowerShell Remoting rather than nested RDP. * **Server Compatibility**: Target servers must support Restricted Admin Mode (supported natively on Windows Server 2012 R2 and later).

Implementation Steps**### Option A: Group Policy Object (GPO) Configuration**

1. Enforce Client-Side Connection Restriction (PAWs & Workstations) To force administrative workstations to use Restricted Admin mode for all remote RDP sessions:

1. Open the **Group Policy Management Console** (`gpmc.msc`).
2. Create or edit a GPO targeting administrative hosts (e.g., `GPO_Workstation_Hardening`).

3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Credentials Delegation`
4. Double-click the **Restrict delegation of credentials to remote servers** policy.
5. Set it to **Enabled**.
6. In the options dropdown, select **Require Restricted Admin**.
7. Click **OK** and link the GPO to your PAW / Workstation OUs.

2. Configure Server-Side Support (Domain Controllers & Member Servers) Ensure that all target hosts are configured to permit Restricted Admin connections:

1. Open the **Group Policy Management Console** (`gpmc.msc`).
2. Create or edit a GPO targeting servers (e.g., `GPO_Server_Hardening`).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Credentials Delegation`
4. Double-click **Restrict delegation of credentials to remote servers**.
5. Set it to **Enabled**.
6. Select **Require Remote Credential Guard or Restricted Admin** (or **Require Restricted Admin**).
7. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Connection Client`
8. Configure the setting:
 - **Policy:** `Do not allow passwords to be saved`
 - **Setting:** `Enabled` 8b. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Connections` 8c. Configure the setting:
 - **Policy:** `Restrict Remote Desktop Services users to a single Remote Desktop Services session`
 - **Setting:** `Enabled`
9. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Device and Resource Redirection`
10. Configure the settings:
 - **Policy:** `Do not allow drive redirection` → **Enabled**
 - **Policy:** `Do not allow COM port redirection` → **Enabled**
 - **Policy:** `Do not allow LPT port redirection` → **Enabled**
 - **Policy:** `Do not allow supported Plug and Play device redirection` → **Enabled**
11. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Security`
12. Configure the following settings:
 - **Policy:** `Always prompt for password upon connection` → **Enabled**
 - **Policy:** `Require secure RPC communication` → **Enabled**
 - **Policy:** `Set client connection encryption level` → **Enabled** (Select `High Level` in the options dropdown) 12b. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Session Time Limits` 12c. Configure the following settings:
 - **Policy:** `Set time limit for active but idle Remote Desktop Services sessions` → **Enabled** (Select `15 minutes` in the options dropdown)
 - **Policy:** `Set time limit for disconnected sessions` → **Enabled** (Select `1 minute` in the options dropdown)
13. Click **OK** and link the GPO to your Domain Controllers and Member Servers OUs.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script to configure the local host registry to allow and enforce Restricted Admin mode.

[Download Script: Set-RdpRestrictedAdmin.ps1](#)

```
# Set-RdpRestrictedAdmin.ps1
# Description: Enables RDP Restricted Admin mode support and hardens RDP session options.

Write-Host "Applying hardening requirement: Enforce RDP Restricted Admin Mode and Session Controls..." -ForegroundColor Cyan

# 1. Enable Restricted Admin support
$LsaPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
$ValueName = "DisableRestrictedAdmin"
$ValueData = 0

if (Test-Path $LsaPath) {
    Set-ItemProperty -Path $LsaPath -Name $ValueName -Value $ValueData -Type DWord -ErrorAction Stop
    Write-Host "[+] Local system configured to accept RDP Restricted Admin connections." -ForegroundColor Green
} else {
    Write-Warning "LSA Registry path not found."
}

# 2. Harden RDP Session options in registry
$RdpPolicyPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services"
if (-not (Test-Path $RdpPolicyPath)) {
    New-Item -Path $RdpPolicyPath -Force | Out-Null
}

$RdpSettings = @{
    "DisablePasswordSaving" = 1
    "fSingleSessionPerUser" = 1
    "fDisableCdm"           = 1
    "fDisableCcm"           = 1
    "fDisableLpt"           = 1
    "fDisablePNPRedir"      = 1
    "fPromptForPassword"    = 1
    "fEncryptRPCTraffic"    = 1
    "MinEncryptionLevel"    = 3
    "MaxIdleTime"           = 900000
    "MaxDisconnectionTime" = 60000
}

foreach ($Setting in $RdpSettings.Keys) {
    Set-ItemProperty -Path $RdpPolicyPath -Name $Setting -Value $RdpSettings[$Setting] -Type DWord -ErrorAction Stop
}
Write-Host "[+] RDP session security controls applied to registry." -ForegroundColor Green
```

To verify the local RDP configuration status: [Download Script: Get-RdpRestrictedAdminStatus.ps1](#)

```
# Get-RdpRestrictedAdminStatus.ps1
# Description: Checks the configuration state of RDP Restricted Admin and session security settings.

$LsaPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
$ValueName = "DisableRestrictedAdmin"

Write-Host "Checking local LSA registry settings..." -ForegroundColor Cyan

if (Test-Path $LsaPath) {
    $Value = Get-ItemProperty -Path $LsaPath -Name $ValueName -ErrorAction SilentlyContinue
    if ($null -ne $Value) {
        if ($Value.DisableRestrictedAdmin -eq 0) {
            Write-Host "[+] RDP Restricted Admin Mode: Enabled (Value = 0)." -ForegroundColor Green
        } else {
            Write-Host "[-] RDP Restricted Admin Mode: Disabled (Value = $($Value.DisableRestrictedAdmin))." -ForegroundColor Red
        }
    } else {
        Write-Host "[+] RDP Restricted Admin Mode: Enabled (Default state: No registry restriction)." -ForegroundColor Green
    }
}

Write-Host "Checking RDP Session Security registry settings..." -ForegroundColor Cyan
$RdpPolicyPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services"

$ExpectedRdpSettings = @{
    "DisablePasswordSaving" = 1
    "fSingleSessionPerUser" = 1
    "fDisableCdm"           = 1
    "fDisableCcm"           = 1
    "fDisableLpt"           = 1
    "fDisablePNPRedir"      = 1
    "fPromptForPassword"    = 1
    "fEncryptRPCTraffic"    = 1
    "MinEncryptionLevel"    = 3
    "MaxIdleTime"           = 900000
    "MaxDisconnectionTime"  = 60000
}

if (Test-Path $RdpPolicyPath) {
    $PolicyValues = Get-ItemProperty -Path $RdpPolicyPath -ErrorAction SilentlyContinue
    foreach ($Setting in $ExpectedRdpSettings.Keys) {
        $Val = $PolicyValues.$Setting
        $Expected = $ExpectedRdpSettings[$Setting]
        $Color = if ($Val -eq $Expected) { "Green" } else { "Red" }
        Write-Host "    - $($Setting): $Val (Expected: $Expected)" -ForegroundColor $Color
    }
} else {
    Write-Host "[-] RDP Session Policies path not found." -ForegroundColor Red
}
}
```

To establish a remote connection with Restricted Admin manually from the command line:

```
mstsc.exe /RestrictedAdmin
```

Sources & Compliance References * ****ANSSI AD Hardening Guide****: Section 4.7, Section 4.9 (Connexions distantes), Annexe E * ****ANSSI Remediation of Active Directory Tier 0 Guide****: Section 10.g (Page 40), Section 5.1 (Page 104) * ****Microsoft Security Guidance****: Mitigating Pass-the-Hash Attacks - Section 4.3 (Restricted Admin RDP) * ****CIS Benchmark****: Section 18.4 (Credentials Delegation Configuration), Section 18.10.57.3.2.1, Section 18.10.57.3.3.1, Section 18.10.57.3.3.3, Section 18.10.57.3.3.4, Section 18.10.57.3.10.1, Section 18.10.57.3.10.2

Windows Defender Antivirus Domain Controller Baseline and Exploit Guard

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus

Rationale Domain Controllers are the most critical assets in an Active Directory environment, containing the Active Directory database (NTDS.dit) and credential material for the entire enterprise. Because Domain Controllers are Tier 0 assets, protective security agents running on them must be hardened to prevent credential access, lateral movement, and tampering.

This submodule contains individual requirement controls for each advanced protective mechanism and configuration parameter within Windows Defender Antivirus for Domain Controllers.

Legacy Impact & Compatibility * **ASR PSEXec and WMI Rule**: Enforcing "Block process creations originating from PSEXec and WMI commands" ('d1e49aac-8f56-4280-b9ba-993a6d77406c') can disrupt enterprise remote administration, monitoring agents, and backup orchestrators. In environments utilizing such orchestrators, this rule should be set to **Audit** mode or configured with explicit process exclusions rather than hard Block mode. * **Office Application Rules**: Rules targeting Microsoft Office or Adobe applications are documented but will not affect Domain Controllers, as these applications must never be installed on Tier 0 systems. * **Reboot Requirement**: Activating Sandbox Execution via 'MP_FORCE_USE_SANDBOX' requires a reboot of the Domain Controllers to take effect. This should be scheduled during standard maintenance windows. * **AMSI Provider Signatures**: Any third-party security agents registering as AMSI providers must have a valid, trusted Authenticode signature. Unsigned or self-signed providers will be prevented from loading.

Defender Hardening Requirements for Domain Controllers

The following Defender Antivirus configurations must be enforced on Domain Controllers:

1. [REQ-DC-075 - Disable Real-Time Monitoring and Behavior Monitoring Override on Domain Controllers](#)
2. [REQ-DC-076 - Configure Potentially Unwanted Applications \(PUA\) Protection on Domain Controllers](#)
3. [REQ-DC-077 - Prevent Local List Merging and Exclusions Configuration on Domain Controllers](#)
4. [REQ-DC-078 - Configure Auto Exclusions Configuration on Domain Controllers](#)
5. [REQ-DC-079 - Prevent MAPS Local Setting Override on Domain Controllers](#)
6. [REQ-DC-080 - Enable EDR in Block Mode on Domain Controllers](#)
7. [REQ-DC-081 - Allow Network Protection on Windows Server on Domain Controllers](#)
8. [REQ-DC-082 - Enable File Hash Computation on Domain Controllers](#)
9. [REQ-DC-083 - Configure Network Inspection System \(NIS\) settings on Domain Controllers](#)
10. [REQ-DC-084 - Configure OOBE Real-Time Protection and Security Intelligence on Domain Controllers](#)
11. [REQ-DC-085 - Enable Dynamic Signature Dropped Event Reporting on Domain Controllers](#)
12. [REQ-DC-086 - Disable Generic Reports on Domain Controllers](#)
13. [REQ-DC-087 - Configure Behavioral Network Brute Force Protection Aggressiveness on Domain Controllers](#)
14. [REQ-DC-088 - Configure Behavioral Network Remote Encryption Protection Aggressiveness on Domain Controllers](#)
15. [REQ-DC-089 - Configure Quick Scan and Scanning Exclusions on Domain Controllers](#)
16. [REQ-DC-090 - Configure Scheduled Scan Parameters on Domain Controllers](#)
17. [REQ-DC-091 - Configure Security Intelligence Update Schedule on Domain Controllers](#)
18. [REQ-DC-092 - Configure Attack Surface Reduction Rules on Domain Controllers](#)
19. [REQ-DC-093 - Configure Threat Severity Default Quarantine Actions on Domain Controllers](#)
20. [REQ-DC-094 - Configure Family Options UI Lockdown on Domain Controllers](#)
21. [REQ-DC-095 - Configure Tamper Protection on Domain Controllers](#)
22. [REQ-DC-096 - Configure Sandbox Execution Environment on Domain Controllers](#)
23. [REQ-DC-097 - Configure AMSI Authenticode Signature Verification on Domain Controllers](#)

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-075] Disable Real-Time Monitoring and Behavior Monitoring Override on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Real-time Protection` * **Registry Location**: *
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender` * `DisableAntiSpyware` = `0` (REG_DWORD) *
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` * `DisableRealtimeMonitoring` = `0` (REG_DWORD) *
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` * `DisableBehaviorMonitoring` = `0` (REG_DWORD) *
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` * `DisableIOAVProtection` = `0` (REG_DWORD) *
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` * `DisableScriptScanning` = `0` (REG_DWORD)

Rationale Real-time scanning, behavior monitoring, and script checking are the core dynamic defense mechanisms of Windows Defender. Disabling or bypassing these controls allows malicious scripts, file-based attacks, and unauthorized in-memory activities to execute undetected.

Legacy Impact & Compatibility Negligible operational impact. Real-time scanning introduces standard CPU overhead during file read/write operations.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Real-time Protection 2. Set 'Turn off real-time protection' to 'Disabled' 3. Set 'Turn on behavior monitoring' to 'Enabled' 4. Set 'Scan all downloaded files and attachments' to 'Enabled' 5. Set 'Turn on script scanning' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderRtpDC.ps1](#)] ([../implementation_scripts/Configure-DefenderRtpDC.ps1](#))

```
# Configure-DefenderRtpDC.ps1
Set-MpPreference -DisableRealtimeMonitoring $false
Set-MpPreference -DisableBehaviorMonitoring $false
Set-MpPreference -DisableIOAVProtection $false
Set-MpPreference -DisableScriptScanning $false
$DefenderPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender"
if (-not (Test-Path $DefenderPath)) { New-Item -Path $DefenderPath -Force | Out-Null }
Set-ItemProperty -Path $DefenderPath -Name "DisableAntiSpyware" -Value 0 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderRtpDCStatus.ps1](#)

```
# Get-DefenderRtpDCStatus.ps1
$Pref = Get-MpPreference
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender" -Name "DisableAntiSpyware" -ErrorAction SilentlyContinue
if ($Pref.DisableRealtimeMonitoring -eq $false -and $Pref.DisableBehaviorMonitoring -eq $false -and $Pref.DisableIOAVProtection -eq $false -and $Pref.DisableScriptScanning -eq $false -and ($Reg -and $Reg.DisableAntiSpyware -eq 0)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-076] Configure Potentially Unwanted Applications (PUA) Protection on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender` * `PUAProtection` = `1` (REG_DWORD)

Rationale Potentially Unwanted Applications (PUA) include adware, torrent clients, cryptominers, and system optimizers that increase risk and resource consumption. Forcing PUA blocking stops standard vectors of shadow IT and unauthorized utility tool execution.

Legacy Impact & Compatibility Legitimate but unrecognized administrative utilities may be blocked. Standard exclusions must be configured for approved utilities.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus 2. Set 'Configure detection for potentially unwanted applications' to 'Enabled' 3. Select 'Block' in the options dropdown list

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderPUA.ps1](#)]

(../implementation_scripts/Configure-DefenderPUA.ps1)

```
# Configure-DefenderPUA.ps1
Set-MpPreference -PUAProtection 1
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender" -Name "PUAProtection" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderPUAStatus.ps1](#)

```
# Get-DefenderPUAStatus.ps1
$Pref = Get-MpPreference
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender" -Name "PUAProtection" -ErrorAction SilentlyContinue
if ($Pref.PUAProtection -eq 1 -or ($Reg -and $Reg.PUAProtection -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-077] Prevent Local List Merging and Exclusions Configuration on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender` * `DisableLocalAdminMerge` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender` * `HideExclusionsFromLocalAdmins` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions` * `DisableLocalAdminConfiguration` = `1` (REG_DWORD)

Rationale If local administrators or compromised administrative accounts can modify Defender exclusions or merge local lists, they can authorize malicious folders or tools. Restricting list configuration to central GPOs ensures consistent security enforcement.

Legacy Impact & Compatibility Local administrators will be unable to add folders or scripts to Defender exclusions. All exclusions must be managed centrally via GPO.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus 2. Set 'Configure local administrator merge behavior for lists' to 'Disabled' 3. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Exclusions 4. Set 'Prevent users from configuring exclusions' to 'Enabled' 5. Set 'Control whether or not exclusions are visible to Local Admins' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderLocalExclusions.ps1](#)] ([../implementation_scripts/Configure-DefenderLocalExclusions.ps1](#))

```
# Configure-DefenderLocalExclusions.ps1
Set-MpPreference -DisableLocalAdminMerge $true
Set-MpPreference -DisableExclusionRestriction $false
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender"
if (-not (Test-Path $Path)) { New-Item -Path $Path -Force | Out-Null }
Set-ItemProperty -Path $Path -Name "DisableLocalAdminMerge" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $Path -Name "HideExclusionsFromLocalAdmins" -Value 1 -Type DWord -Force
$ExclPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions"
if (-not (Test-Path $ExclPath)) { New-Item -Path $ExclPath -Force | Out-Null }
Set-ItemProperty -Path $ExclPath -Name "DisableLocalAdminConfiguration" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderLocalExclusionsStatus.ps1](#)

```
# Get-DefenderLocalExclusionsStatus.ps1
$Pref = Get-MpPreference
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender" -Name "DisableLocalAdminMerge" -ErrorAction SilentlyContinue
$RegHide = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender" -Name "HideExclusionsFromLocalAdmins" -ErrorAction SilentlyContinue
$RegConfig = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions" -Name "DisableLocalAdminConfiguration" -ErrorAction SilentlyContinue
if (($Pref.DisableLocalAdminMerge -eq $true -or ($Reg -and $Reg.DisableLocalAdminMerge -eq 1)) -and
    ($RegHide -and $RegHide.HideExclusionsFromLocalAdmins -eq 1) -and
    ($RegConfig -and $RegConfig.DisableLocalAdminConfiguration -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-078] Configure Auto Exclusions Configuration on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Exclusions` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions` * `DisableAutoExclusions` = `0` (REG_DWORD)

Rationale Auto Exclusions automatically configure exclusions for known safe system folders or server roles to reduce performance overhead. Enforcing that auto exclusions are not disabled ensures server performance stability and proper system scanning.

Legacy Impact & Compatibility None.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Exclusions 2. Set 'Turn off Auto Exclusions' to 'Disabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderAutoExclusions.ps1](#)] ([../implementation_scripts/Configure-DefenderAutoExclusions.ps1](#))

```
# Configure-DefenderAutoExclusions.ps1
$ExclPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions"
if (-not (Test-Path $ExclPath)) { New-Item -Path $ExclPath -Force | Out-Null }
Set-ItemProperty -Path $ExclPath -Name "DisableAutoExclusions" -Value 0 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderAutoExclusionsStatus.ps1](#)

```
# Get-DefenderAutoExclusionsStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions" -Name "DisableAutoExclusions" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.DisableAutoExclusions -eq 0) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-079] Prevent MAPS Local Setting Override on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\MAPS` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Spynet` * `SpynetReporting` = `0` (REG_DWORD)

Rationale Preventing local overrides of Microsoft Active Protection Service (MAPS) reporting ensures that Domain Controllers consistently report telemetry and signature feedback to cloud resources, preserving centralized protective visibility.

Legacy Impact & Compatibility Local system administrators cannot opt-out of telemetry reporting.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\MAPS 2. Set 'Join Microsoft MAPS' to 'Disabled' (or prevent override to SpynetReporting = 0)

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderSpynetDC.ps1](#)] ([../implementation_scripts/Configure-DefenderSpynetDC.ps1](#))

```
# Configure-DefenderSpynetDC.ps1
$SpynetPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Spynet"
if (-not (Test-Path $SpynetPath)) { New-Item -Path $SpynetPath -Force | Out-Null }
Set-ItemProperty -Path $SpynetPath -Name "SpynetReporting" -Value 0 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderSpynetDCStatus.ps1](#)

```
# Get-DefenderSpynetDCStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Spynet" -Name "SpynetReporting" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.SpynetReporting -eq 0) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-080] Enable EDR in Block Mode on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Features` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Features` * `PassiveRemediation` = `1` (REG_DWORD)

Rationale Endpoint Detection and Response (EDR) in Block Mode allows Defender to take remediation actions on malicious artifacts detected by Microsoft Defender for Endpoint even if another non-Microsoft antivirus is primary. This establishes secondary defensive block capabilities.

Legacy Impact & Compatibility None. Extends passive monitoring to execute block operations when necessary.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Features 2. Set 'Enable EDR in block mode' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderEdrBlockMode.ps1](#)] ([../implementation_scripts/Configure-DefenderEdrBlockMode.ps1](#))

```
# Configure-DefenderEdrBlockMode.ps1
$FeaturesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Features"
if (-not (Test-Path $FeaturesPath)) { New-Item -Path $FeaturesPath -Force | Out-Null }
Set-ItemProperty -Path $FeaturesPath -Name "PassiveRemediation" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderEdrBlockModeStatus.ps1](#)

```
# Get-DefenderEdrBlockModeStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Features" -Name "PassiveRemediation" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.PassiveRemediation -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-081] Allow Network Protection on Windows Server on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Network Protection` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\Network Protection` * `AllowNetworkProtectionOnWinServer` = `1` (REG_DWORD)

Rationale Network Protection blocks processes from accessing malicious domains, phishing sites, and host IP ranges. Allowing Network Protection on Windows Server ensures that member servers running server workloads possess the same IP filter protections as client platforms.

Legacy Impact & Compatibility Unrecognized or legacy client-server connections to custom web resources may be filtered. Exclusions must be mapped under DFE.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Network Protection 2. Set 'This setting controls whether Network Protection is allowed to be configured into block or audit mode on Windows Server' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-DefenderNetworkProtectionServer.ps1](../implementation_scripts/Configure-DefenderNetworkProtectionServer.ps1)

```
# Configure-DefenderNetworkProtectionServer.ps1
$NetProtPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\Network Protection"
if (-not (Test-Path $NetProtPath)) { New-Item -Path $NetProtPath -Force | Out-Null }
Set-ItemProperty -Path $NetProtPath -Name "AllowNetworkProtectionOnWinServer" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderNetworkProtectionServerStatus.ps1](#)

```
# Get-DefenderNetworkProtectionServerStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\Network Protection" -Name "AllowNetworkProtectionOnWinServer" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.AllowNetworkProtectionOnWinServer -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-082] Enable File Hash Computation on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\MpEngine` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\MpEngine` * `EnableFileHashComputation` = `1` (REG_DWORD)

Rationale Computing cryptographic file hashes allows Defender to pass hashes of scanned files to cloud and SIEM Domain Controllers. This enables precise IOC matches, file tracking, and correlation with threat intelligence repositories.

Legacy Impact & Compatibility Minor CPU/Disk I/O impact when scanning large directories.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\MpEngine 2. Set 'Enable file hash computation feature' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderFileHash.ps1](#)] ([../implementation_scripts/Configure-DefenderFileHash.ps1](#))

```
# Configure-DefenderFileHash.ps1
Set-MpPreference -EnableFileHashComputation $true
$MpEnginePath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\MpEngine"
if (-not (Test-Path $MpEnginePath)) { New-Item -Path $MpEnginePath -Force | Out-Null }
Set-ItemProperty -Path $MpEnginePath -Name "EnableFileHashComputation" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderFileHashStatus.ps1](#)

```
# Get-DefenderFileHashStatus.ps1
$Pref = Get-MpPreference
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\MpEngine" -Name "EnableFileHashComputation" -Error
Action SilentlyContinue
if ($Pref.EnableFileHashComputation -eq $true -or ($Reg -and $Reg.EnableFileHashComputation -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-083] Configure Network Inspection System (NIS) settings on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Network Inspection System` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\NIS` * `EnableConvertWarnToBlock` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\NIS` * `AllowSwitchToAsyncInspection` = `1` (REG_DWORD)

Rationale The Network Inspection System (NIS) inspects network traffic patterns for known exploits. Converting warning verdicts to block enforces inline blocking of zero-day exploits, while allowing async inspection prevents performance overhead from slowing local network interfaces.

Legacy Impact & Compatibility None.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Network Inspection System 2. Set 'Convert warn verdict to block' to 'Enabled' 3. Set 'Turn on asynchronous inspection' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderNis.ps1](#)] ([./implementation_scripts/Configure-DefenderNis.ps1](#))

```
# Configure-DefenderNis.ps1
$NisPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\NIS"
if (-not (Test-Path $NisPath)) { New-Item -Path $NisPath -Force | Out-Null }
Set-ItemProperty -Path $NisPath -Name "EnableConvertWarnToBlock" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $NisPath -Name "AllowSwitchToAsyncInspection" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderNisStatus.ps1](#)

```
# Get-DefenderNisStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\NIS" -Name "EnableConvertWarnToBlock" -ErrorAction SilentlyContinue
$RegAsync = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\NIS" -Name "AllowSwitchToAsyncInspection" -ErrorAction SilentlyContinue
if (($Reg -and $Reg.EnableConvertWarnToBlock -eq 1) -and ($RegAsync -and $RegAsync.AllowSwitchToAsyncInspection -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-084] Configure OOBE Real-Time Protection and Security Intelligence on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Real-time Protection` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` * `OobeEnableRtpAndSigUpdate` = `1` (REG_DWORD)

Rationale Enabling real-time protection and intelligence updates during the Out-of-Box Experience (OOBE) ensures that the system is fully updated and protected before the initial administrative user signs in or connects to enterprise network nodes.

Legacy Impact & Compatibility Negligible. Minor network traffic during initial provisioning phases.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Real-time Protection 2. Set 'Configure real-time protection and Security Intelligence Updates during OOBE' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderOobeRtp.ps1](#)] ([../implementation_scripts/Configure-DefenderOobeRtp.ps1](#))

```
# Configure-DefenderOobeRtp.ps1
$RtpPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection"
if (-not (Test-Path $RtpPath)) { New-Item -Path $RtpPath -Force | Out-Null }
Set-ItemProperty -Path $RtpPath -Name "OobeEnableRtpAndSigUpdate" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderOobeRtpStatus.ps1](#)

```
# Get-DefenderOobeRtpStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection" -Name "OobeEnableRtpAndSigUpdate" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.OobeEnableRtpAndSigUpdate -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-085] Enable Dynamic Signature Dropped Event Reporting on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Low * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Reporting` * **Registry Location**: *
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Reporting` * `EnableDynamicSignatureDroppedEventReporting` = `1` (REG_DWORD)

Rationale Enabling this log report generation triggers explicit events when a dynamic scan ruleset signature is dropped. This ensures SIEM integrations can immediately log changes in the local threat signatures dataset.

Legacy Impact & Compatibility None.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Reporting 2. Set 'Configure whether to report Dynamic Signature dropped events' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderDynamicReporting.ps1](#)] ([../implementation_scripts/Configure-DefenderDynamicReporting.ps1](#))

```
# Configure-DefenderDynamicReporting.ps1
$RepPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Reporting"
if (-not (Test-Path $RepPath)) { New-Item -Path $RepPath -Force | Out-Null }
Set-ItemProperty -Path $RepPath -Name "EnableDynamicSignatureDroppedEventReporting" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderDynamicReportingStatus.ps1](#)

```
# Get-DefenderDynamicReportingStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Reporting" -Name "EnableDynamicSignatureDroppedEventReporting" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.EnableDynamicSignatureDroppedEventReporting -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-095] Disable Generic Reports on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Reporting` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender` * `DisableGenericRePorts` = `1` (REG_DWORD)

Rationale Disabling generic telemetry and reports on Domain Controllers restricts the transmission of potentially sensitive metadata or environment data regarding Tier 0 directory services to external cloud resources.

Legacy Impact & Compatibility Limits some automated telemetry diagnostics sent to Microsoft support logs.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Reporting 2. Set 'Configure generic reports' to 'Disabled' (or set DisableGenericRePorts = 1)

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderDisableGenericReports.ps1](#)](../implementation_scripts/Configure-DefenderDisableGenericReports.ps1)

```
# Configure-DefenderDisableGenericReports.ps1
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender"
if (-not (Test-Path $Path)) { New-Item -Path $Path -Force | Out-Null }
Set-ItemProperty -Path $Path -Name "DisableGenericRePorts" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderDisableGenericReportsStatus.ps1](#)

```
# Get-DefenderDisableGenericReportsStatus.ps1
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender"
$Reg = Get-ItemProperty -Path $Path -Name "DisableGenericRePorts" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.DisableGenericRePorts -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-096] Configure Behavioral Network Brute Force Protection Aggressiveness on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Remediation\Behavioral Network Blocks` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Brute Force Protection` * `BruteForceProtectionAggressiveness` = `1` (REG_DWORD)

Rationale Active Directory Domain Controllers are prime targets for automated password brute-forcing and Kerberos pre-authentication spraying attacks. Setting brute force protection aggressiveness to 1 enables immediate behavioral blocks against network authentication flood sources.

Legacy Impact & Compatibility May occasionally trigger blocks on legitimate tools performing aggressive directory scans or rapid administrative sync scripts if not scoped correctly.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Remediation\Behavioral Network Blocks 2. Set 'Configure behavioral network brute force protection aggressiveness' to 'Enabled' (Select '1')

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderBruteForceProtection.ps1](#)] ([../implementation_scripts/Configure-DefenderBruteForceProtection.ps1](#))

```
# Configure-DefenderBruteForceProtection.ps1
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Brute Force Protection"
if (-not (Test-Path $Path)) { New-Item -Path $Path -Force | Out-Null }
Set-ItemProperty -Path $Path -Name "BruteForceProtectionAggressiveness" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderBruteForceProtectionStatus.ps1](#)

```
# Get-DefenderBruteForceProtectionStatus.ps1
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Brute Force Protection"
$Reg = Get-ItemProperty -Path $Path -Name "BruteForceProtectionAggressiveness" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.BruteForceProtectionAggressiveness -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-097] Configure Behavioral Network Remote Encryption Protection Aggressiveness on Domain Controllers
 ## Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Remediation\Behavioral Network Blocks` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Remote Encryption Protection` * `RemoteEncryptionProtectionAggressiveness` = `1` (REG_DWORD)

Rationale Ransomware groups target SYSVOL and SYSVOL shares on Domain Controllers to deploy encrypted templates or payloads. Enabling remote encryption protection aggressiveness blocks remote network-driven encryption attempts immediately.

Legacy Impact & Compatibility None. Blocks behavioral patterns matching ransomware network encryption engines.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Remediation\Behavioral Network Blocks 2. Set 'Configure behavioral network remote encryption protection aggressiveness' to 'Enabled' (Select '1')

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-DefenderRemoteEncryptionProtection.ps1](../implementation_scripts/Configure-DefenderRemoteEncryptionProtection.ps1)

```
# Configure-DefenderRemoteEncryptionProtection.ps1
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Remote Encryption Protection"
if (-not (Test-Path $Path)) { New-Item -Path $Path -Force | Out-Null }
Set-ItemProperty -Path $Path -Name "RemoteEncryptionProtectionAggressiveness" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderRemoteEncryptionProtectionStatus.ps1](#)

```
# Get-DefenderRemoteEncryptionProtectionStatus.ps1
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Remote Encryption Protection"
$Reg = Get-ItemProperty -Path $Path -Name "RemoteEncryptionProtectionAggressiveness" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.RemoteEncryptionProtectionAggressiveness -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-086] Configure Quick Scan and Scanning Exclusions on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` * `QuickScanIncludeExclusions` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` * `DisablePackedExeScanning` = `0` (REG_DWORD)

Rationale Malware frequently tries to establish persistence in excluded directories or inside packed/compressed executables. Forcing quick scans to include excluded files and ensuring packed file structures are recursively scanned prevents malware evasion.

Legacy Impact & Compatibility Increased quick scan times depending on folder sizes and compression ratios.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan 2. Set 'Scan excluded files and directories during quick scans' to 'Enabled' 3. Set 'Turn off scanning of packed executables' to 'Disabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderQuickScan.ps1](#)] (./implementation_scripts/Configure-DefenderQuickScan.ps1)

```
# Configure-DefenderQuickScan.ps1
Set-MpPreference -DisablePackedExeScanning $false
$ScanPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan"
if (-not (Test-Path $ScanPath)) { New-Item -Path $ScanPath -Force | Out-Null }
Set-ItemProperty -Path $ScanPath -Name "QuickScanIncludeExclusions" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $ScanPath -Name "DisablePackedExeScanning" -Value 0 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderQuickScanStatus.ps1](#)

```
# Get-DefenderQuickScanStatus.ps1
$Pref = Get-MpPreference
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -Name "QuickScanIncludeExclusions" -ErrorAction SilentlyContinue
$RegPack = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -Name "DisablePackedExeScanning" -ErrorAction SilentlyContinue
if (($Pref.DisablePackedExeScanning -eq $false -or ($RegPack -and $RegPack.DisablePackedExeScanning -eq 0)) -and ($Reg -and $Reg.QuickScanIncludeExclusions -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-087] Configure Scheduled Scan Parameters on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan` * **Registry Location**: *
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` * `ScheduleDay` = `0` (REG_DWORD) *
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` * `DisableEmailScanning` = `0` (REG_DWORD) *
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` * `DisableHeuristics` = `0` (REG_DWORD) *
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` * `DaysWithoutCatchupQuickScan` = `7` (REG_DWORD)

Rationale Ensuring daily scheduled scans, enabling heuristics for behavioral anomaly detection, scan mail attachments, and forcing a catchup scan after at most 7 days ensures system integrity is continually validated.

Legacy Impact & Compatibility Slight scanning overhead.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan 2. Set 'Specify the day of the week to run a scheduled scan' to 'Enabled' (Select 'Every day' or '0') 3. Set 'Turn on e-mail scanning' to 'Enabled' 4. Set 'Turn on heuristics' to 'Enabled' 5. Set 'Trigger a quick scan after X days without any scans' to 'Enabled' (7 days)

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderScheduledScan.ps1](#)] ([../implementation_scripts/Configure-DefenderScheduledScan.ps1](#))

```
# Configure-DefenderScheduledScan.ps1
Set-MpPreference -DisableEmailScanning $false
Set-MpPreference -DisableHeuristics $false
$ScanPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan"
if (-not (Test-Path $ScanPath)) { New-Item -Path $ScanPath -Force | Out-Null }
Set-ItemProperty -Path $ScanPath -Name "ScheduleDay" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $ScanPath -Name "DisableEmailScanning" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $ScanPath -Name "DisableHeuristics" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $ScanPath -Name "DaysWithoutCatchupQuickScan" -Value 7 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderScheduledScanStatus.ps1](#)

```
# Get-DefenderScheduledScanStatus.ps1
$Pref = Get-MpPreference
$RegDays = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -Name "DaysWithoutCatchupQuickScan" -ErrorAction SilentlyContinue
if ($Pref.DisableEmailScanning -eq $false -and $Pref.DisableHeuristics -eq $false -and ($RegDays -and $RegDays.DaysWithoutCatchupQuickScan -eq 7)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-088] Configure Security Intelligence Update Schedule on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Security Intelligence Updates` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates` * `ASSignatureDue` = `7` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates` * `AVSignatureDue` = `7` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates` * `ScheduleDay` = `0` (REG_DWORD)

Rationale Antivirus signatures must remain fresh to block the latest published threats. Mandating daily checks for updates and marking signatures older than 7 days as out-of-date ensures continuous defense parity.

Legacy Impact & Compatibility Requires continuous outbound connectivity to WSUS or Microsoft Update endpoints.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Security Intelligence Updates 2. Set 'Define the number of days before spyware security intelligence is considered out of date' to 'Enabled' (7 days) 3. Set 'Define the number of days before virus security intelligence is considered out of date' to 'Enabled' (7 days) 4. Set 'Specify the day of the week to check for security intelligence updates' to 'Enabled' (Every day or 0)

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderUpdateSchedule.ps1](#)] ([../implementation_scripts/Configure-DefenderUpdateSchedule.ps1](#))

```
# Configure-DefenderUpdateSchedule.ps1
$SigPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates"
if (-not (Test-Path $SigPath)) { New-Item -Path $SigPath -Force | Out-Null }
Set-ItemProperty -Path $SigPath -Name "ASSignatureDue" -Value 7 -Type DWord -Force
Set-ItemProperty -Path $SigPath -Name "AVSignatureDue" -Value 7 -Type DWord -Force
Set-ItemProperty -Path $SigPath -Name "ScheduleDay" -Value 0 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderUpdateScheduleStatus.ps1](#)

```
# Get-DefenderUpdateScheduleStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates" -Name "ASSignatureDue" -ErrorAction SilentlyContinue
$RegAV = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates" -Name "AVSignatureDue" -ErrorAction SilentlyContinue
$RegDay = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates" -Name "ScheduleDay" -ErrorAction SilentlyContinue
if (($Reg -and $Reg.ASSignatureDue -eq 7) -and ($RegAV -and $RegAV.AVSignatureDue -eq 7) -and ($RegDay -and $RegDay.ScheduleDay -eq 0)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

Configure Attack Surface Reduction Rules on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction

Rationale Attack Surface Reduction (ASR) rules block threat behaviors on Domain Controllers, focusing on preventing local credential hijacking from LSASS memory, vulnerable driver execution, and malicious persistence actions (WMI).

This submodule contains individual requirement rules for each ASR control configured on Active Directory Domain Controllers.

Legacy Impact & Compatibility * **PSEXEC / WMI Command Block**: Creating processes via remote WMI or PSEXEC commands is set to **Audit** mode (`2`) rather than Block to prevent breaking automated server monitoring, backup scripts, and administrative replication checks.

Enforced Attack Surface Reduction Rules on Domain Controllers

The following individual ASR rules must be configured:

1. [REQ-DC-098 - ASR: Block abuse of exploited vulnerable signed drivers on Domain Controllers](#) (Block)
 2. [REQ-DC-099 - ASR: Block credential stealing from the Windows local security authority subsystem on Domain Controllers](#) (Block)
 3. [REQ-DC-100 - ASR: Block execution of potentially obfuscated scripts on Domain Controllers](#) (Block)
 4. [REQ-DC-101 - ASR: Block persistence through WMI event subscription on Domain Controllers](#) (Block)
 5. [REQ-DC-102 - ASR: Block process creations originating from PSEXEC and WMI commands on Domain Controllers](#) (Audit)
 6. [REQ-DC-103 - ASR: Use advanced protection against ransomware on Domain Controllers](#) (Block)
-

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (ASR Rules) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-098] ASR: Block abuse of exploited vulnerable signed drivers on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `56a863a9-875e-4185-98a7-b882c64b5ce5` = `1` (REG_SZ)

Rationale Prevents an application from writing a vulnerable signed driver to disk. Attackers use Bring Your Own Vulnerable Driver (BYOVD) techniques to bypass Windows kernel protections by loading legitimate, signed third-party drivers that contain known vulnerabilities, allowing them to disable security agents and gain kernel-level privileges.

Legacy Impact & Compatibility Administrators or developers who deliberately load older or specific signed debugging/hardware diagnostics drivers containing known security bugs will be blocked. Approvals or exclusions must be configured for these specific drivers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `56a863a9-875e-4185-98a7-b882c64b5ce5` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAsrVulnerableDrivers.ps1](#)] ([./../implementation_scripts/Configure-DcAsrVulnerableDrivers.ps1](#))

```
# Configure-DcAsrVulnerableDrivers.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "56a863a9-875e-4185-98a7-b882c64b5ce5" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-DcAsrVulnerableDriversStatus.ps1](#)

```
# Get-DcAsrVulnerableDriversStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "56a863a9-875e-4185-98a7-b882c64b5ce5" -ErrorAction SilentlyContinue
if ($Value -and ($Value."56a863a9-875e-4185-98a7-b882c64b5ce5" -eq "1" -or $Value."56a863a9-875e-4185-98a7-b882c64b5ce5" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-099] ASR: Block credential stealing from the Windows local security authority subsystem on Domain Controllers
 ## Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2` = `1` (REG_SZ)

Rationale Blocks attempts to open or dump the memory of the Local Security Authority Subsystem Service (lsass.exe). Attackers dump LSASS memory using tools like Mimikatz or Task Manager to extract plaintext credentials, Kerberos tickets, or NTLM password hashes from system memory for lateral movement.

Legacy Impact & Compatibility Administrative diagnostic tools, customized authentication packages, or security scanners that attempt to open the LSASS process handle to read user tokens or perform access auditing will be blocked.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAsrLsassDump.ps1](#)] ([../implementation_scripts/Configure-DcAsrLsassDump.ps1](#))

```
# Configure-DcAsrLsassDump.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-DcAsrLsassDumpStatus.ps1](#)

```
# Get-DcAsrLsassDumpStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2" -ErrorAction SilentlyContinue
if ($Value -and ($Value."9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2" -eq "1" -or $Value."9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-100] ASR: Block execution of potentially obfuscated scripts on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `5beb7efe-fd9a-4556-801d-275e5ffc04cc` = `1` (REG_SZ)

Rationale Blocks execution of obfuscated or encrypted scripts (such as PowerShell, VBScript, or JavaScript). Threat actors obfuscate their scripts using base64 encoding, custom string manipulation, or encryption to hide the intent of their code and bypass static file scanning and network detection engines.

Legacy Impact & Compatibility Legitimate administrative tools or third-party provisioning scripts that use heavy minification, obfuscation, or custom encoding wrappers will be blocked. Scripts must be refactored to use readable, signed code.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `5beb7efe-fd9a-4556-801d-275e5ffc04cc` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAsrObfuscatedScripts.ps1](#)] ([../implementation_scripts/Configure-DcAsrObfuscatedScripts.ps1](#))

```
# Configure-DcAsrObfuscatedScripts.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "5beb7efe-fd9a-4556-801d-275e5ffc04cc" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-DcAsrObfuscatedScriptsStatus.ps1](#)

```
# Get-DcAsrObfuscatedScriptsStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "5beb7efe-fd9a-4556-801d-275e5ffc04cc" -ErrorAction SilentlyContinue
if ($Value -and ($Value."5beb7efe-fd9a-4556-801d-275e5ffc04cc" -eq "1" -or $Value."5beb7efe-fd9a-4556-801d-275e5ffc04cc" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-101] ASR: Block persistence through WMI event subscription on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `e6db77e5-3df2-4cf1-b95a-636979351e5b` = `1` (REG_SZ)

Rationale Blocks threat actors from achieving system persistence by registering permanent Windows Management Instrumentation (WMI) event subscriptions. WMI event subscriptions allow attackers to automatically launch malicious payloads when system triggers occur (like system boot or user logon) without using traditional startup registry keys.

Legacy Impact & Compatibility Management software or diagnostic monitoring tools that legitimately register permanent WMI event filters to track system telemetry will be blocked. Alternatives like Windows services or scheduled tasks must be used.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `e6db77e5-3df2-4cf1-b95a-636979351e5b` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAsrWmiPersistence.ps1](#)] ([./../implementation_scripts/Configure-DcAsrWmiPersistence.ps1](#))

```
# Configure-DcAsrWmiPersistence.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "e6db77e5-3df2-4cf1-b95a-636979351e5b" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-DcAsrWmiPersistenceStatus.ps1](#)

```
# Get-DcAsrWmiPersistenceStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "e6db77e5-3df2-4cf1-b95a-636979351e5b" -ErrorAction SilentlyContinue
if ($Value -and ($Value."e6db77e5-3df2-4cf1-b95a-636979351e5b" -eq "1" -or $Value."e6db77e5-3df2-4cf1-b95a-636979351e5b" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-102] ASR: Block process creations originating from PSEXec and WMI commands on Domain Controllers
 ## Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `d1e49aac-8f56-4280-b9ba-993a6d77406c` = `2` (REG_SZ)

Rationale Blocks processes created via WMI commands or PSEXec remote execution utilities. This directly stops lateral movement attacks where compromised accounts or threat actors attempt to start commands, backdoors, or credential dumpers remotely across domain-joined servers and Domain Controllers.

Legacy Impact & Compatibility Configured in Audit mode (2) to monitor remote execution commands, ensuring administrative agents and orchestration tools are logged without operational block impact.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `d1e49aac-8f56-4280-b9ba-993a6d77406c` as Value Name, with Value set to `2` (Audit).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAsrPsexecWmi.ps1](#)] ([./../implementation_scripts/Configure-DcAsrPsexecWmi.ps1](#))

```
# Configure-DcAsrPsexecWmi.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "d1e49aac-8f56-4280-b9ba-993a6d77406c" -Value "2" -Type String -Force
```

To audit the hardening status: [Download Script: Get-DcAsrPsexecWmiStatus.ps1](#)

```
# Get-DcAsrPsexecWmiStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "d1e49aac-8f56-4280-b9ba-993a6d77406c" -ErrorAction SilentlyContinue
if ($Value -and ($Value."d1e49aac-8f56-4280-b9ba-993a6d77406c" -eq "2" -or $Value."d1e49aac-8f56-4280-b9ba-993a6d77406c" -eq 2)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-103] ASR: Use advanced protection against ransomware on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `c1db55ab-c21a-4637-bb3f-a12568109d35` = `1` (REG_SZ)

Rationale Enables advanced behavioral heuristics and cloud analytics checks on files that attempt to modify multiple user files, detect signature-less encryption behavior, and block rapid write activity to prevent ransomware from encrypting system and user documents.

Legacy Impact & Compatibility Minor performance overhead when scanning active files during large batch edits or heavy database updates.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `c1db55ab-c21a-4637-bb3f-a12568109d35` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAsrRansomware.ps1](#)] ([./../implementation_scripts/Configure-DcAsrRansomware.ps1](#))

```
# Configure-DcAsrRansomware.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "c1db55ab-c21a-4637-bb3f-a12568109d35" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-DcAsrRansomwareStatus.ps1](#)

```
# Get-DcAsrRansomwareStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "c1db55ab-c21a-4637-bb3f-a12568109d35" -ErrorAction SilentlyContinue
if ($Value -and ($Value."c1db55ab-c21a-4637-bb3f-a12568109d35" -eq "1" -or $Value."c1db55ab-c21a-4637-bb3f-a12568109d35" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-090] Configure Threat Severity Default Quarantine Actions on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Threats` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Threats` * `Threats\_ThreatSeverityDefaultAction` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Threats\ThreatSeverityDefaultAction` * `1` = `2` (REG_DWORD)

Rationale By default, Defender may prompt users or take actions (like clean/ignore) that leave malware remnants on the filesystem. Configuring default quarantine actions for all severities (low, medium, high, severe) ensures automated containment.

Legacy Impact & Compatibility None.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Threats 2. Set 'Specify threat alert levels at which default action should not be taken when detected' to 'Enabled' 3. Click 'Show...' and enter threat levels (1 → 2, 2 → 2, 4 → 2, 5 → 2)

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderThreatActions.ps1](#)] (./implementation_scripts/Configure-DefenderThreatActions.ps1)

```
# Configure-DefenderThreatActions.ps1
$ThreatsPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Threats"
if (-not (Test-Path $ThreatsPath)) { New-Item -Path $ThreatsPath -Force | Out-Null }
Set-ItemProperty -Path $ThreatsPath -Name "Threats_ThreatSeverityDefaultAction" -Value 1 -Type DWord -Force
$ThreatsSevPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Threats\ThreatSeverityDefaultAction"
if (-not (Test-Path $ThreatsSevPath)) { New-Item -Path $ThreatsSevPath -Force | Out-Null }
Set-ItemProperty -Path $ThreatsSevPath -Name "1" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $ThreatsSevPath -Name "2" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $ThreatsSevPath -Name "4" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $ThreatsSevPath -Name "5" -Value 2 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderThreatActionsStatus.ps1](#)

```
# Get-DefenderThreatActionsStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Threats" -Name "Threats_ThreatSeverityDefaultAction" -ErrorAction SilentlyContinue
$RegSev = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Threats\ThreatSeverityDefaultAction" -ErrorAction SilentlyContinue
if (($Reg -and $Reg.Threats_ThreatSeverityDefaultAction -eq 1) -and
    ($RegSev -and $RegSev.1 -eq 2 -and $RegSev.2 -eq 2 -and $RegSev.4 -eq 2 -and $RegSev.5 -eq 2)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-091] Configure Family Options UI Lockdown on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Low * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Security\Family options` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\Family options` * `UILockdown` = `1` (REG_DWORD)

Rationale Locking down non-essential components of the Windows Security Center interface prevents users from tampering with parental or diagnostic UI controls on enterprise assets.

Legacy Impact & Compatibility None.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Security\Family options 2. Set 'Hide the Family options area' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderFamilyLockdown.ps1](#)] ([../implementation_scripts/Configure-DefenderFamilyLockdown.ps1](#))

```
# Configure-DefenderFamilyLockdown.ps1
$FamilyPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\Family options"
if (-not (Test-Path $FamilyPath)) { New-Item -Path $FamilyPath -Force | Out-Null }
Set-ItemProperty -Path $FamilyPath -Name "UILockdown" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderFamilyLockdownStatus.ps1](#)

```
# Get-DefenderFamilyLockdownStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\Family options" -Name "UILockdown"
-ErrorAction SilentlyContinue
if ($Reg -and $Reg.UILockdown -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-092] Configure Tamper Protection on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Security\Tamper Protection` * **Registry Location**: * `HKLM\SOFTWARE\Microsoft\Windows Defender\Features` * `TamperProtection` = `5` (REG_DWORD)

Rationale Tamper Protection prevents local administrators or compromised system accounts from disabling Windows Defender services, real-time scanning, or modifying active exclusions locally. This blocks a primary malware persistence vector.

Legacy Impact & Compatibility Local registry modifications to Defender configurations will be ignored. All changes must originate from central templates or cloud console panels.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Security\Tamper Protection 2. Set 'Protect Windows Security settings from tampering' to 'Enabled' (Block or On depending on ADMX version)

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderTamperProtection.ps1](#)] ([./implementation_scripts/Configure-DefenderTamperProtection.ps1](#))

```
# Configure-DefenderTamperProtection.ps1
$FeaturesPath = "HKLM:\SOFTWARE\Microsoft\Windows Defender\Features"
if (-not (Test-Path $FeaturesPath)) { New-Item -Path $FeaturesPath -Force | Out-Null }
try {
    Set-ItemProperty -Path $FeaturesPath -Name "TamperProtection" -Value 5 -Type DWord -ErrorAction Stop -Force
} catch {
    Write-Warning "Registry blocked. Tamper Protection registry key is normally protected by TrustedInstaller. Ensure GPO setting is applied."
}
```

To audit the hardening status: [Download Script: Get-DefenderTamperProtectionStatus.ps1](#)

```
# Get-DefenderTamperProtectionStatus.ps1
$FeaturesPath = "HKLM:\SOFTWARE\Microsoft\Windows Defender\Features"
$TamperVal = Get-ItemProperty -Path $FeaturesPath -Name "TamperProtection" -ErrorAction SilentlyContinue
if ($TamperVal -and $TamperVal.TamperProtection -eq 5) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-093] Configure Sandbox Execution Environment on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Environment` * **Registry Location**: * `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Environment` * `MP\_FORCE\_USE\_SANDBOX` = `1` (REG_SZ)

Rationale Forcing the Windows Defender scanning service (MsMpEng.exe) to run in a restricted AppContainer sandbox prevents privilege escalation. If an attacker exploits a parsing vulnerability in the engine, the compromise is contained inside the sandbox.

Legacy Impact & Compatibility A system reboot is required to initialize the scanning process within the AppContainer sandbox.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Preferences\Windows Settings\Environment 2. Right-click and select New → Environment Variable 3. Configure Action: Update, Type: System, Name: MP_FORCE_USE_SANDBOX, Value: 1

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderSandbox.ps1](#)] ([../implementation_scripts/Configure-DefenderSandbox.ps1](#))

```
# Configure-DefenderSandbox.ps1
$EnvPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Environment"
if (-not (Test-Path $EnvPath)) { New-Item -Path $EnvPath -Force | Out-Null }
Set-ItemProperty -Path $EnvPath -Name "MP_FORCE_USE_SANDBOX" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-DefenderSandboxStatus.ps1](#)

```
# Get-DefenderSandboxStatus.ps1
$EnvPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Environment"
$SandboxVar = Get-ItemProperty -Path $EnvPath -Name "MP_FORCE_USE_SANDBOX" -ErrorAction SilentlyContinue
if ($SandboxVar -and $SandboxVar.MP_FORCE_USE_SANDBOX -eq "1") {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-094] Configure AMSI Authenticode Signature Verification on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: * `HKLM\SOFTWARE\Microsoft\AMSI` * `FeatureBits` = `2` (REG_DWORD)

Rationale Enforcing signature checks on registered Antimalware Scan Interface (AMSI) providers blocks attackers from registering unsigned rogue AMSI provider DLLs to bypass script analysis.

Legacy Impact & Compatibility Third-party antivirus/security providers must register using signed Authenticode binaries to prevent being blocked.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Preferences\Windows Settings\Registry 2. Right-click and select New → Registry Item 3. Configure Action: Update, Hive: HKEY_LOCAL_MACHINE, Key Path: SOFTWARE\Microsoft\AMSI, Value Name: FeatureBits, Value Type: REG_DWORD, Value Data: 2

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderAmsiSignature.ps1](#)] (./implementation_scripts/Configure-DefenderAmsiSignature.ps1)

```
# Configure-DefenderAmsiSignature.ps1
$AmsiPath = "HKLM:\SOFTWARE\Microsoft\AMSI"
if (-not (Test-Path $AmsiPath)) { New-Item -Path $AmsiPath -Force | Out-Null }
Set-ItemProperty -Path $AmsiPath -Name "FeatureBits" -Value 2 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderAmsiSignatureStatus.ps1](#)

```
# Get-DefenderAmsiSignatureStatus.ps1
$AmsiPath = "HKLM:\SOFTWARE\Microsoft\AMSI"
if (Test-Path $AmsiPath) {
    $AmsiBits = Get-ItemProperty -Path $AmsiPath -Name "FeatureBits" -ErrorAction SilentlyContinue
    if ($AmsiBits -and $AmsiBits.FeatureBits -eq 2) {
        Write-Output "Compliant"
        exit 0
    }
}
Write-Output "Non-Compliant"
exit 1
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers

[REQ-DC-021] Configure AppLocker Policies on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

```
## Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **Service Configuration (GPO)**: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services\Application Identity` → Automatic * **AppLocker Path (GPO)**: `Computer Configuration\Policies\Windows Settings\Security Settings\Application Control Policies\AppLocker` * **Registry Locations**: *
`HKLM\SYSTEM\CurrentControlSet\Services\AppIDSvc` * `Start` = `2` (REG_DWORD) *
`HKLM\Software\Policies\Microsoft\Windows\SrpV2\Exe` * `EnforcementMode` = `1` (REG_DWORD) *
`HKLM\Software\Policies\Microsoft\Windows\SrpV2\Msi` * `EnforcementMode` = `1` (REG_DWORD) *
`HKLM\Software\Policies\Microsoft\Windows\SrpV2\Script` * `EnforcementMode` = `1` (REG_DWORD) *
`HKLM\Software\Policies\Microsoft\Windows\SrpV2\Appx` * `EnforcementMode` = `1` (REG_DWORD) *
`HKLM\SOFTWARE\Policies\Microsoft\Windows\AppCompat` * `Prevent16BitApp` = `1` (REG_DWORD)
```

Rationale Domain Controllers are Tier 0 administrative assets and must never be used for general-purpose tasks like web browsing, document viewing, or running unapproved utilities. Attackers who compromise a Domain Controller or obtain administrative access often attempt to execute custom binaries, remote access tools (RATs), or script-based tools to pivot, establish persistence, or extract the Active Directory database (NTDS.dit).

Enforcing AppLocker policies on Domain Controllers provides the following defense-in-depth security benefits:

- 1. Restricts Execution to Authorized Software:** Prevents execution of unapproved software, preventing standard user directories (such as `C:\Users\` or `C:\Windows\Temp\`) from being used to launch malicious binaries or scripts.
 - 2. Blocks Browser Execution:** Prevents administrative users from launching web browsers (Chrome, Edge, Firefox, Internet Explorer) directly on Domain Controllers, shutting down web-based drive-by downloads and browser-based credential leakage.
 - 3. Restricts Windows Installer and Script Execution:** Prevents unauthorized `.msi` installations and unauthorized PowerShell or VBScript scripts from running, reducing the likelihood of successful exploitation via living-off-the-land techniques.
 - 4. Defends Against AppLocker Bypasses:** Abusing trusted, signed Microsoft binaries (such as `msbuild.exe`, `installutil.exe`, `regasm.exe`, `regsvcs.exe`, `mshta.exe`, `regsvr32.exe`, `rundll32.exe`) allows attackers to execute arbitrary code bypassing default AppLocker rules. This control blocks these "Living off the Land" binaries (LOLBins) and prevents execution from user-writable paths under `%WINDIR%` (such as `Tasks`, `Temp`, `tracing`, `spool\drivers\color`, etc.).
-

Legacy Impact & Compatibility * **Third-Party Administrative Tools**: Monitoring agents, backup orchestrators, and system management tools that run from custom directories (outside `%ProgramFiles%` or `%WinDir%`) will be blocked unless explicit path, publisher, or hash rules are created to whitelist them. * **Audit Mode Verification**: It is highly recommended to deploy AppLocker in **Audit Only** mode for a baseline period (e.g., 30 days) to identify all legitimate software and administrative scripts. Analyze the event log (`Applications and Services Logs\Microsoft\Windows\AppLocker`) to create the necessary whitelist rules before switching to **Enforce rules** mode. * **AppIDSvc Service**: AppLocker relies on the **Application Identity** service (`AppIDSvc`) to evaluate rule enforcement. If the service is not running, rules will not be enforced.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Enable Application Identity Service 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Create or edit a GPO targeting Domain Controllers (e.g., `GPO_Hardening_DomainControllers`). 3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` 4. Double-click **Application Identity**. 5. Select **Define this policy setting** and configure the startup mode to **Automatic**.

2. Configure AppLocker Enforcement 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Application Control Policies\AppLocker` 2. Right-click **AppLocker** and select **Properties**. 3. Under the **Enforcement** tab, check **Configured** for the following rule collections: **Executable rules** → Select **Enforce rules** (or **Audit only** for baseline testing) * **Windows Installer rules** → Select **Enforce rules** (or **Audit only**) * **Script rules** → Select **Enforce rules** (or **Audit only**) * **Packaged app rules** → Select **Enforce rules** (or **Audit only**) 4. Click **OK**.

3. Create Default and Exception Rules 1. Expand **AppLocker** and select **Executable Rules**. 2. Right-click **Executable Rules** and select **Create Default Rules** (allows all files in Windows and Program Files directories, and allows local Administrators to run all files). 3. Per **ANSSI R2** recommendation, do not create standalone Deny rules. Instead, configure the following path **Exceptions** on the default Allow rule for the Windows folder: * Right-click the rule `(Default Rule) All files located in the Windows folder` and select **Properties**. * On the **Exceptions** tab, add path exceptions for writable directories under `%WINDIR%: %WINDIR%\Tasks %WINDIR%\Temp %WINDIR%\tracing %WINDIR%\System32\spool\drivers\color %WINDIR%\System32\Tasks\Microsoft\Windows\SyncCenter %WINDIR%\SysWOW64\Tasks\Microsoft\Windows\SyncCenter` * On the same **Exceptions** tab, add path exceptions for the following bypass binaries (LOLBins): `%*\msbuild.exe %*\installutil.exe %*\mshta.exe %*\regasm.exe %*\regsvcs.exe %*\regsvr32.exe %*\rundll32.exe %*\bginfo.exe %*\cdb.exe %*\cmstp.exe %*\control.exe %*\csi.exe %*\dfsvc.exe %*\dnx.exe %*\fsi.exe %*\ie4unit.exe %*\ieexec.exe %*\infdefaultinstall.exe %*\mavinject.exe %*\msdeploy.exe %*\msdt.exe %*\msxml.exe %*\odbcconf.exe %*\presentationhost.exe %*\rcsi.exe %*\rsi.exe %*\runscripthelper.exe %*\te.exe %*\tracker.exe %*\xwizard.exe` 4. Repeat the process for **Script Rules** by creating default rules and adding exceptions to the `%WINDIR%` Allow rule for script execution from user-writable paths (such as `%WINDIR%\Temp` and `%WINDIR%\Tasks`). 5. Disable NTVDm (16-bit application support) to prevent AppLocker

bypasses via 16-bit binaries: * Navigate to: `Computer Configuration\Administrative Templates\System\16-bit Application Compatibility` * Configure **Prevent access to 16-bit applications** to **Enabled**.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to configure the Application Identity service and import a robust local AppLocker policy.

[Download Script: Set-AppLockerDCPolicy.ps1](#)

```
# Set-AppLockerDCPolicy.ps1
# Description: Configures the Application Identity service and imports a robust local AppLocker XML policy.

Write-Host "Applying hardening requirement: Configure AppLocker on Domain Controllers..." -ForegroundColor Cyan

# 1. Enable Application Identity service (AppIDSvc)
$Service = Get-Service -Name AppIDSvc -ErrorAction SilentlyContinue
if ($Service) {
    Set-Service -Name AppIDSvc -StartupType Automatic
    if ($Service.Status -ne "Running") {
        Start-Service -Name AppIDSvc
    }
    Write-Host "[+] Application Identity service configured to start automatically and is running." -ForegroundColor Green
} else {
    Write-Error "Application Identity service (AppIDSvc) is not present on this system."
}

# 2. Configure local AppLocker policy XML content
$AppLockerXml = @"
<AppLockerPolicy Version="1">
  <RuleCollection Type="Exe" EnforcementMode="Enabled">
    <FilePathRule Id="921cc481-6e1e-453f-b3a5-bc4f4a38674d" Name="(Default Rule) All files located in the Program Files folder" Description="Allows members of the Everyone group to run applications that are located in the Program Files folder." UserOrGroupSid="S-1-1-0" Action="Allow">
      <Conditions>
        <FilePathCondition Path="%PROGRAMFILES%\*" />
      </Conditions>
    </FilePathRule>
    <FilePathRule Id="a61c8b2c-6d8f-4ad9-acbc-467b78a7f7b4" Name="(Default Rule) All files located in the Windows folder" Description="Allows members of the Everyone group to run applications that are located in the Windows folder." UserOrGroupSid="S-1-1-0" Action="Allow">
      <Conditions>
        <FilePathCondition Path="%WINDIR%\*" />
      </Conditions>
      <Exceptions>
        <FilePathCondition Path="%WINDIR%\Temp\*" />
        <FilePathCondition Path="%WINDIR%\Tasks\*" />
        <FilePathCondition Path="%WINDIR%\tracing\*" />
        <FilePathCondition Path="%WINDIR%\System32\spool\drivers\color\*" />
        <FilePathCondition Path="%WINDIR%\System32\Tasks\Microsoft\Windows\SyncCenter\*" />
        <FilePathCondition Path="%WINDIR%\SysWOW64\Tasks\Microsoft\Windows\SyncCenter\*" />
        <FilePathCondition Path="*\msbuild.exe" />
        <FilePathCondition Path="*\installutil.exe" />
        <FilePathCondition Path="*\mshta.exe" />
        <FilePathCondition Path="*\regasm.exe" />
        <FilePathCondition Path="*\regsvcs.exe" />
        <FilePathCondition Path="*\regsvr32.exe" />
        <FilePathCondition Path="*\rundll32.exe" />
        <FilePathCondition Path="*\bginfo.exe" />
        <FilePathCondition Path="*\cdb.exe" />
        <FilePathCondition Path="*\cmstp.exe" />
        <FilePathCondition Path="*\control.exe" />
        <FilePathCondition Path="*\csi.exe" />
        <FilePathCondition Path="*\dfsvc.exe" />
        <FilePathCondition Path="*\dnx.exe" />
        <FilePathCondition Path="*\fsi.exe" />
        <FilePathCondition Path="*\ie4unit.exe" />
        <FilePathCondition Path="*\ieexec.exe" />
        <FilePathCondition Path="*\infdefaultinstall.exe" />
        <FilePathCondition Path="*\mavinject.exe" />
        <FilePathCondition Path="*\msdeploy.exe" />
        <FilePathCondition Path="*\msdt.exe" />
        <FilePathCondition Path="*\msxsl.exe" />
        <FilePathCondition Path="*\odbcconf.exe" />
        <FilePathCondition Path="*\presentationhost.exe" />
        <FilePathCondition Path="*\rcsi.exe" />
        <FilePathCondition Path="*\rsi.exe" />
        <FilePathCondition Path="*\runscripthelper.exe" />
        <FilePathCondition Path="*\te.exe" />
        <FilePathCondition Path="*\tracker.exe" />
        <FilePathCondition Path="*\xwizard.exe" />
      </Exceptions>
    </FilePathRule>
    <FilePathRule Id="fd686d83-a829-4351-8ff4-27c1de5732e9" Name="(Default Rule) All files" Description="Allows members of the local
```

```

Administrators group to run all applications." UserOrGroupSid="S-1-5-32-544" Action="Allow">
  <Conditions>
    <FilePathCondition Path="*" />
  </Conditions>
</FilePathRule>
</RuleCollection>
<RuleCollection Type="Msi" EnforcementMode="Enabled">
  <FilePathRule Id="5b8fa8b3-3a5e-4c7a-9cb8-b223ff9db271" Name="(Default Rule) All Windows Installer files in Program Files" Description="Allows everyone to run Windows Installer files in Program Files." UserOrGroupSid="S-1-1-0" Action="Allow">
    <Conditions>
      <FilePathCondition Path="%PROGRAMFILES%*" />
    </Conditions>
  </FilePathRule>
  <FilePathRule Id="6b8fa8b3-3a5e-4c7a-9cb8-b223ff9db272" Name="(Default Rule) All Windows Installer files in Windows" Description="Allows everyone to run Windows Installer files in Windows." UserOrGroupSid="S-1-1-0" Action="Allow">
    <Conditions>
      <FilePathCondition Path="%WINDIR%*" />
    </Conditions>
  </FilePathRule>
  <FilePathRule Id="7b8fa8b3-3a5e-4c7a-9cb8-b223ff9db273" Name="(Default Rule) All Windows Installer files" Description="Allows administrators to run all Windows Installer files." UserOrGroupSid="S-1-5-32-544" Action="Allow">
    <Conditions>
      <FilePathCondition Path="*" />
    </Conditions>
  </FilePathRule>
</RuleCollection>
<RuleCollection Type="Script" EnforcementMode="Enabled">
  <FilePathRule Id="1c8fa8b3-3a5e-4c7a-9cb8-b223ff9db274" Name="(Default Rule) All scripts located in the Program Files folder" Description="Allows everyone to run scripts in Program Files." UserOrGroupSid="S-1-1-0" Action="Allow">
    <Conditions>
      <FilePathCondition Path="%PROGRAMFILES%*" />
    </Conditions>
  </FilePathRule>
  <FilePathRule Id="2c8fa8b3-3a5e-4c7a-9cb8-b223ff9db275" Name="(Default Rule) All scripts located in the Windows folder" Description="Allows everyone to run scripts in Windows." UserOrGroupSid="S-1-1-0" Action="Allow">
    <Conditions>
      <FilePathCondition Path="%WINDIR%*" />
    </Conditions>
    <Exceptions>
      <FilePathCondition Path="%WINDIR%\Temp%*" />
      <FilePathCondition Path="%WINDIR%\Tasks%*" />
    </Exceptions>
  </FilePathRule>
  <FilePathRule Id="3c8fa8b3-3a5e-4c7a-9cb8-b223ff9db276" Name="(Default Rule) All scripts" Description="Allows administrators to run all scripts." UserOrGroupSid="S-1-5-32-544" Action="Allow">
    <Conditions>
      <FilePathCondition Path="*" />
    </Conditions>
  </FilePathRule>
</RuleCollection>
<RuleCollection Type="Appx" EnforcementMode="Enabled">
  <FilePublisherRule Id="1d8fa8b3-3a5e-4c7a-9cb8-b223ff9db279" Name="(Default Rule) All signed packaged apps" Description="Allows everyone to run signed packaged apps." UserOrGroupSid="S-1-1-0" Action="Allow">
    <Conditions>
      <FilePublisherCondition PublisherName="*" ProductName="*" BinaryName="*">
        <BinaryVersionRange LowSection="0.0.0.0" HighSection="*" />
      </FilePublisherCondition>
    </Conditions>
  </FilePublisherRule>
</RuleCollection>
</AppLockerPolicy>
"@

# Write the temporary XML and import it
$TempPath = Join-Path -Path $env:TEMP -ChildPath "AppLockerDCPolicy.xml"
$AppLockerXml | Out-File -FilePath $TempPath -Encoding UTF8 -Force

# 3. Validate policy using Test-AppLockerPolicy before importing
try {
  Import-Module AppLocker -ErrorAction Stop
} catch {
  Write-Error "AppLocker module is not available on this system. Cannot configure or validate policy."
  if (Test-Path $TempPath) {
    Remove-Item -Path $TempPath -Force
  }
  return
}

```

```

$TestPaths = @(
    # Expected: Allowed
    "$env:windir\System32\cmd.exe",
    "$env:windir\System32\WindowsPowerShell\v1.0\powershell.exe",
    # Expected: DeniedByDefault or ExplicitlyDenied (since it is an exception to an Allow rule)
    "$env:USERPROFILE\Downloads\tool.exe",
    "$env:windir\Temp\malware.exe",
    "$env:windir\Tasks\evil.exe",
    "$env:windir\System32\msbuild.exe"
)

$ValidationFailed = $false
try {
    $TestResults = Test-AppLockerPolicy -XmlPolicy $TempPath -Path $TestPaths -User Everyone -ErrorAction Stop
    $ExpectedAllow = @(
        "$env:windir\System32\cmd.exe",
        "$env:windir\System32\WindowsPowerShell\v1.0\powershell.exe"
    )
    $ExpectedDeny = @(
        "$env:USERPROFILE\Downloads\tool.exe",
        "$env:windir\Temp\malware.exe",
        "$env:windir\Tasks\evil.exe",
        "$env:windir\System32\msbuild.exe"
    )

    foreach ($Result in $TestResults) {
        $Path = $Result.FilePath
        $Decision = $Result.PolicyDecision
        if ($ExpectedAllow -contains $Path) {
            if ($Decision -ne "Allowed") {
                Write-Warning "[VALIDATION FAIL] Expected Allow for: $Path (got: $Decision)"
                $ValidationFailed = $true
            }
        }
        if ($ExpectedDeny -contains $Path) {
            if ($Decision -eq "Allowed") {
                Write-Warning "[VALIDATION FAIL] Expected Deny/Not Allowed for: $Path (got: $Decision)"
                $ValidationFailed = $true
            }
        }
    }
} catch {
    Write-Warning "Could not perform policy validation tests: $($_.Exception.Message)"
    $ValidationFailed = $true
}

if ($ValidationFailed) {
    Write-Error "AppLocker policy validation failed. Policy was NOT imported."
    if (Test-Path $TempPath) {
        Remove-Item -Path $TempPath -Force
    }
    return
}

Write-Host "[+] AppLocker policy validation passed. Proceeding with import." -ForegroundColor Green

# 4. Import the validated AppLocker policy
try {
    Set-AppLockerPolicy -XmlPolicy $TempPath -ErrorAction Stop
    Write-Host "[+] Local AppLocker policy imported and enforced successfully." -ForegroundColor Green
} catch {
    Write-Error "Failed to import AppLocker policy: $($_.Exception.Message)"
} finally {
    if (Test-Path $TempPath) {
        Remove-Item -Path $TempPath -Force
    }
}

# 5. Disable NTVDM (16-bit compatibility) via Registry
$Ntvdmpath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppCompat"
if (-not (Test-Path $Ntvdmpath)) {
    New-Item -Path $Ntvdmpath -Force | Out-Null
}
Set-ItemProperty -Path $Ntvdmpath -Name "Prevent16BitApp" -Value 1 -Type DWord
Write-Host "[+] 16-bit NTVDM compatibility disabled in registry." -ForegroundColor Green

```

To audit the Application Identity service and AppLocker registry configuration: [Download Script: Get-AppLockerDCStatus.ps1](#)

```
# Get-AppLockerDCStatus.ps1
# Description: Checks the configuration state of the AppIDSvc service and AppLocker registry paths.

Write-Host "--- Auditing AppLocker Configuration ---" -ForegroundColor Cyan

# 1. Audit service state
$AppIDSvc = Get-Service -Name AppIDSvc -ErrorAction SilentlyContinue
if ($AppIDSvc) {
    $SvcColor = if ($AppIDSvc.Status -eq "Running" -and $AppIDSvc.StartType -eq "Automatic") { "Green" } else { "Yellow" }
    Write-Host "    - Application Identity Service: $($AppIDSvc.Status) | Startup: $($AppIDSvc.StartType) (Expected: Running | Autom
atic)" -ForegroundColor $SvcColor
} else {
    Write-Host "    - Application Identity Service: NOT INSTALLED" -ForegroundColor Red
}

# 2. Audit enforcement registry settings
$SrpPath = "HKLM:\Software\Policies\Microsoft\SrpV2"
$Collections = @("Exe", "Msi", "Script", "Appx")

if (Test-Path $SrpPath) {
    foreach ($Col in $Collections) {
        $ColPath = "$SrpPath\$Col"
        if (Test-Path $ColPath) {
            $Val = Get-ItemProperty -Path $ColPath -Name "EnforcementMode" -ErrorAction SilentlyContinue
            if ($null -ne $Val) {
                $Mode = if ($Val.EnforcementMode -eq 1) { "Enforced" } else { "Audit Only" }
                $Color = if ($Val.EnforcementMode -eq 1) { "Green" } else { "Yellow" }
                Write-Host "    - Collection $Col Enforcement: $Mode (Value: $($Val.EnforcementMode))" -ForegroundColor $Color
            } else {
                Write-Host "    - Collection $Col Enforcement: NOT CONFIGURED" -ForegroundColor Red
            }
        } else {
            Write-Host "    - Collection $Col Path: NOT FOUND" -ForegroundColor Red
        }
    }
} else {
    Write-Host "[-] AppLocker registry base path (SrpV2) not found. Policy is not deployed." -ForegroundColor Red
}

# 3. Audit NTVDm Disable Status
$NtvdmPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppCompat"
if (Test-Path $NtvdmPath) {
    $AppCompatVal = Get-ItemProperty -Path $NtvdmPath -Name "Prevent16BitApp" -ErrorAction SilentlyContinue
    if ($null -ne $AppCompatVal -and $AppCompatVal.Prevent16BitApp -eq 1) {
        Write-Host "    - NTVDm (16-bit AppCompat): Disabled (Secure)" -ForegroundColor Green
    } else {
        Write-Host "    - NTVDm (16-bit AppCompat): Enabled or Not Configured (Expected: Disabled)" -ForegroundColor Yellow
    }
} else {
    Write-Host "    - NTVDm (16-bit AppCompat): Not Configured (Expected: Disabled)" -ForegroundColor Yellow
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**: Recommendations Section 3.1.2 (System hardening and configuration baseline controls), DAT-NT-13 Note Technique (R2, R8, R10, R15, R16, R20) * **CIS Benchmark**: CIS Microsoft Windows Server Benchmark - Section 18.9 (Application Control Policies / AppLocker) * **Microsoft Security Baseline Focus**: Domain Controller Security baseline - AppLocker configurations * **Ultimate AppLocker Bypass List**: Generic & Verified AppLocker Bypasses

[REQ-DC-022] Enable WDAC Driver Blocklist

Target Scope * **Applicable Systems** : Domain Controllers * **Operating Systems** : Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows Server 2025

Implementation Details * **Priority** : High * **GPO Path / Registry Location** : * **GPO Path** : Computer Configuration\Administrative Templates\System\Device Guard\Deploy Windows Defender Application Control * **Registry Location** : 'HKLM\SYSTEM\CurrentControlSet\Control\CI\Config' → 'VulnerableDriverBlocklistEnable' = '1' (REG_DWORD)

Rationale Attackers frequently employ "Bring Your Own Vulnerable Driver" (BYOVD) attacks to bypass Windows kernel protections. In a BYOVD attack, an adversary with administrative privileges installs a legitimate, cryptographically signed third-party driver that contains a known, exploitable vulnerability. The attacker then exploits this vulnerability to execute arbitrary code with kernel privileges, allowing them to disable security agents, dump LSASS memory, or tamper with system integrity.

Enforcing the **Microsoft Vulnerable Driver Blocklist** via Windows Defender Application Control (WDAC) prevents known vulnerable or malicious drivers from loading in kernel space. By restricting the WDAC policy to **Kernel Mode Code Integrity (KMCI) only** (omitting user-mode enforcement), the control shields the system kernel from driver-based exploits without introducing administrative overhead or blocking standard user-mode server applications and utilities.

Legacy Impact & Compatibility * **Pre-requisite (Memory Integrity/HVCI)** : The vulnerable driver blocklist requires Hypervisor-Protected Code Integrity (HVCI) for secure, hypervisor-enforced validation. Refer to [REQ-DC-007 - Disable Credential Guard](#02-domain-controllers-disable-credential-guard-md) to ensure Virtualization-Based Security (VBS) and Memory Integrity (HVCI) are fully enabled (while ensuring Credential Guard is kept disabled). Enabling Secure Boot and CPU virtualization features is a strict pre-requisite; refer to [REQ-PAW-005 - UEFI Firmware Security Hardening](#07-paws-configure-uefi-security-md) and [REQ-PAW-006 - Enable Hardware Virtualization and DMA Protection](#07-paws-enable-hardware-virtualization-and-dma-protection-md) for firmware settings. * **Compatibility with Legacy Drivers** : Third-party backup, monitoring, or hardware administration software running deprecated, vulnerable drivers may fail to load. All such software must be updated to use secure, modern drivers. * **Deployment Testing** : To prevent system instability, the WDAC blocklist policy should be deployed in **Audit Mode** initially to verify that no critical operational drivers are blocked in production before shifting to enforcement mode.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

To enforce the driver blocklist across all Domain Controllers, you can deploy the Microsoft recommended block rules as a custom WDAC policy.

1. Download the Microsoft recommended driver block rules XML from the official Microsoft documentation.
2. Edit the XML to ensure it operates in **Audit Mode** first, then convert the XML configuration into a binary format:

```
ConvertFrom-CIPolicy -XmlFilePath "C:\WDAC\DriverBlocklist.xml" -BinaryFilePath "C:\WDAC\SIPolicy.p7b"
```

3. Copy the compiled `SIPolicy.p7b` file to a secure local path on all target Domain Controllers (e.g., `C:\Windows\System32\CodeIntegrity\SIPolicy.p7b`) or a share.
4. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain management host.
5. Create or edit a GPO linked to the Domain Controllers OU (e.g., `GPO_Hardening_DomainControllers`).
6. Navigate to: `Computer Configuration\Administrative Templates\System\Device Guard`
7. Configure the following setting:
 - o **Policy**: `Deploy Windows Defender Application Control`
 - o **Setting**: `Enabled`
 - o **Code Integrity Policy File Path**: Enter the local or network path to the policy file (e.g., `C:\Windows\System32\CodeIntegrity\SIPolicy.p7b`).
8. To ensure the built-in system driver blocklist is active on modern builds, configure the following registry setting via Group Policy Preferences:
 - o **Path**: `Computer Configuration\Preferences\Windows Settings\Registry`
 - o **Hive**: `HKEY_LOCAL_MACHINE`
 - o **Key Path**: `SYSTEM\CurrentControlSet\Control\CI\Config`
 - o **Value Name**: `VulnerableDriverBlocklistEnable`
 - o **Value Type**: `REG_DWORD`
 - o **Value Data**: `1`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to enable the Vulnerable Driver Blocklist registry key and ensure proper configuration.

[Download Script: Configure-DriverBlocklist.ps1](#)

```
# Configure-DriverBlocklist.ps1
# Description: Enables the Microsoft Vulnerable Driver Blocklist in the registry and validates VBS/HVCI settings.

Write-Host "Applying hardening requirement: Enable WDAC Driver Blocklist..." -ForegroundColor Cyan

# 1. Configure the registry settings to enable the blocklist
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\CI\Config"
$ValueName = "VulnerableDriverBlocklistEnable"

if (-not (Test-Path $RegPath)) {
    Write-Host "[+] Creating registry path: $RegPath" -ForegroundColor Gray
    New-Item -Path $RegPath -Force | Out-Null
}

Write-Host "[+] Setting registry value: $ValueName = 1" -ForegroundColor Gray
Set-ItemProperty -Path $RegPath -Name $ValueName -Value 1 -Type DWord -ErrorAction Stop

# 2. Validate VBS / HVCI Configuration
$ScenariosPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\HypervisorEnforcedCodeIntegrity"
if (Test-Path $ScenariosPath) {
    $HvciStatus = Get-ItemProperty -Path $ScenariosPath -Name "Enabled" -ErrorAction SilentlyContinue
    if ($null -ne $HvciStatus -and $HvciStatus.Enabled -eq 1) {
        Write-Host "[+] Pre-requisite Check: Memory Integrity (HVCI) is enabled." -ForegroundColor Green
    } else {
        Write-Host "[!] Warning: Memory Integrity (HVCI) is disabled. The blocklist requires HVCI for hypervisor enforcement." -ForegroundColor Yellow
    }
} else {
    Write-Host "[!] Warning: Memory Integrity scenario configuration not found. Check VBS settings." -ForegroundColor Yellow
}

Write-Host "[+] Configuration applied successfully. A reboot is required to activate the blocklist." -ForegroundColor Green
```

To verify the setting has been applied:

[Download Script: Get-DriverBlocklistStatus.ps1](#)

```

# Get-DriverBlocklistStatus.ps1
# Description: Audits the configuration of the Microsoft Vulnerable Driver Blocklist and HVCI state.

Write-Host "--- Auditing Vulnerable Driver Blocklist ---" -ForegroundColor Cyan
$Vulnerable = $false

# 1. Check registry value
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\CI\Config"
$ValueName = "VulnerableDriverBlocklistEnable"

if (Test-Path $RegPath) {
    $RegValue = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
    if ($null -ne $RegValue -and $RegValue.$ValueName -eq 1) {
        Write-Host "[+] Vulnerable Driver Blocklist is enabled in the registry." -ForegroundColor Green
    } else {
        Write-Host "[!] VULNERABLE: Vulnerable Driver Blocklist is disabled or not set in the registry." -ForegroundColor Red
        $Vulnerable = $true
    }
} else {
    Write-Host "[!] VULNERABLE: Code Integrity Config registry key does not exist." -ForegroundColor Red
    $Vulnerable = $true
}

# 2. Check HVCI Status
$ScenariosPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\HypervisorEnforcedCodeIntegrity"
if (Test-Path $ScenariosPath) {
    $HvciStatus = Get-ItemProperty -Path $ScenariosPath -Name "Enabled" -ErrorAction SilentlyContinue
    if ($null -ne $HvciStatus -and $HvciStatus.Enabled -eq 1) {
        Write-Host "[+] Memory Integrity (HVCI) is enabled." -ForegroundColor Green
    } else {
        Write-Host "[!] VULNERABLE: Memory Integrity (HVCI) is disabled in the registry." -ForegroundColor Red
        $Vulnerable = $true
    }
} else {
    Write-Host "[!] VULNERABLE: Memory Integrity scenario registry path does not exist." -ForegroundColor Red
    $Vulnerable = $true
}

# 3. Final Verdict
if ($Vulnerable) {
    Write-Host "`n[!] Verification FAILED: The Vulnerable Driver Blocklist is not fully secured." -ForegroundColor Red
} else {
    Write-Host "`n[+] Verification PASSED: The Vulnerable Driver Blocklist and HVCI are correctly configured." -ForegroundColor Green
}
}

```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Recommendations on system component code integrity and driver signature enforcement. * **CIS Microsoft Windows Server Benchmark***: Section 18.8.14.3 / 18.9.31.2 (Deploy Windows Defender Application Control / Memory Integrity). * **Microsoft Security Guidance***: Microsoft recommended driver block rules documentation.

Configure User Rights Assignments for Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers (DCs) * **Operating Systems**: Windows Server 2016 and above

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` * **Registry Location**: Stored inside local security database under privilege definitions.

Rationale User Rights Assignments on Domain Controllers represent a primary defense boundary to secure Active Directory directory services (Tier 0). Restricting who can perform raw volume access, time adjustments, or credential delegations prevents local privilege hijacking.

This submodule contains individual requirement rules for each User Rights Assignment control enforced on Domain Controllers.

Legacy Impact & Compatibility * **Operational Impact**: Restricting privileges on Domain Controllers impacts replication, system tasks, or third-party AD monitoring agents. Thoroughly validate service accounts before deploying constraints.

Enforced User Rights Assignments on Domain Controllers

The following individual URA rules must be configured:

1. [REQ-DC-104 - Configure User Rights: Access this computer from the network on Domain Controllers](#)
 2. [REQ-DC-105 - Configure User Rights: Act as part of the operating system on Domain Controllers](#)
 3. [REQ-DC-106 - Configure User Rights: Add workstations to domain on Domain Controllers](#)
 4. [REQ-DC-107 - Configure User Rights: Adjust memory quotas for a process on Domain Controllers](#)
 5. [REQ-DC-108 - Configure User Rights: Allow log on locally on Domain Controllers](#)
 6. [REQ-DC-109 - Configure User Rights: Allow log on through Remote Desktop Services on Domain Controllers](#)
 7. [REQ-DC-110 - Configure User Rights: Back up files and directories on Domain Controllers](#)
 8. [REQ-DC-111 - Configure User Rights: Bypass traverse checking on Domain Controllers](#)
 9. [REQ-DC-112 - Configure User Rights: Change the system time on Domain Controllers](#)
 10. [REQ-DC-113 - Configure User Rights: Create a pagefile on Domain Controllers](#)
 11. [REQ-DC-114 - Configure User Rights: Create a token object on Domain Controllers](#)
 12. [REQ-DC-115 - Configure User Rights: Create permanent shared objects on Domain Controllers](#)
 13. [REQ-DC-116 - Configure User Rights: Debug programs on Domain Controllers](#)
 14. [REQ-DC-117 - Configure User Rights: Deny access to this computer from the network on Domain Controllers](#)
 15. [REQ-DC-118 - Configure User Rights: Deny log on as a batch job on Domain Controllers](#)
 16. [REQ-DC-119 - Configure User Rights: Deny log on as a service on Domain Controllers](#)
 17. [REQ-DC-120 - Configure User Rights: Deny log on locally on Domain Controllers](#)
 18. [REQ-DC-121 - Configure User Rights: Deny log on through Remote Desktop Services on Domain Controllers](#)
 19. [REQ-DC-122 - Configure User Rights: Enable computer and user accounts to be trusted for delegation on Domain Controllers](#)
 20. [REQ-DC-123 - Configure User Rights: Force shutdown from a remote system on Domain Controllers](#)
 21. [REQ-DC-124 - Configure User Rights: Generate security audits on Domain Controllers](#)
 22. [REQ-DC-125 - Configure User Rights: Load and unload device drivers on Domain Controllers](#)
 23. [REQ-DC-126 - Configure User Rights: Lock pages in memory on Domain Controllers](#)
 24. [REQ-DC-127 - Configure User Rights: Log on as a batch job on Domain Controllers](#)
 25. [REQ-DC-128 - Configure User Rights: Log on as a service on Domain Controllers](#)
 26. [REQ-DC-129 - Configure User Rights: Manage auditing and security log on Domain Controllers](#)
 27. [REQ-DC-130 - Configure User Rights: Modify firmware environment values on Domain Controllers](#)
 28. [REQ-DC-131 - Configure User Rights: Profile single process on Domain Controllers](#)
 29. [REQ-DC-132 - Configure User Rights: Restore files and directories on Domain Controllers](#)
 30. [REQ-DC-133 - Configure User Rights: Shut down the system on Domain Controllers](#)
 31. [REQ-DC-134 - Configure User Rights: Synchronize directory service data on Domain Controllers](#)
 32. [REQ-DC-135 - Configure User Rights: Take ownership of files or other objects on Domain Controllers](#)
-

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-104] Configure User Rights: Access this computer from the network on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Access this computer from the network` * **Registry Location**: Stored inside local security database under privilege `SeNetworkLogonRight` set to `\*S-1-5-9 (Enterprise Domain Controllers), \*S-1-5-11 (Authenticated Users), \*S-1-5-32-544 (Administrators)`.

Rationale Allows users to connect to the Domain Controller over the network. Restricting this to Enterprise Domain Controllers, authenticated users, and Administrators protects Tier 0 interfaces.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeNetworkLogonRight` to `\*S-1-5-9 (Enterprise Domain Controllers), \*S-1-5-11 (Authenticated Users), \*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Access this computer from the network`. 3. Configure the security principal allocation to: `\*S-1-5-9 (Enterprise Domain Controllers), \*S-1-5-11 (Authenticated Users), \*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeNetworkLogonRight.ps1](#)]

(../implementation_scripts/Configure-DcUraSeNetworkLogonRight.ps1)

```
# Configure-DcUraSeNetworkLogonRight.ps1
# Configure-DcUraSeNetworkLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_senetworklogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_senetworklogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.]*)\$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeNetworkLogonRight\s*=") {
        $NewLines += "SeNetworkLogonRight = *S-1-5-9,*S-1-5-11,*S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeNetworkLogonRight = *S-1-5-9,*S-1-5-11,*S-1-5-32-544")
    } else {
        $NewLines += "SeNetworkLogonRight = *S-1-5-9,*S-1-5-11,*S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeNetworkLogonRightStatus.ps1](#)

```
# Get-DcUraSeNetworkLogonRightStatus.ps1
# Get-DcUraSeNetworkLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_senetworklogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeNetworkLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-9,*S-1-5-11,*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline*: User Rights Configuration specifications

[REQ-DC-105] Configure User Rights: Act as part of the operating system on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Act as part of the operating system` * **Registry Location**: Stored inside local security database under privilege `SeTcbPrivilege` set to `No one (Empty)`.

Rationale Allows a process to impersonate any user without credentials. This privilege must be empty on Domain Controllers to prevent Tier 0 privilege escalation.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeTcbPrivilege` to `No one (Empty)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Act as part of the operating system`. 3. Configure the security principal allocation to: `No one (Empty)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeTcbPrivilege.ps1](#)]

(../implementation_scripts/Configure-DcUraSeTcbPrivilege.ps1)

```
# Configure-DcUraSeTcbPrivilege.ps1
# Configure-DcUraSeTcbPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_setcbprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_setcbprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[.*\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeTcbPrivilege\s*=") {
        $NewLines += "SeTcbPrivilege = "
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeTcbPrivilege = ")
    } else {
        $NewLines += "SeTcbPrivilege = "
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeTcbPrivilegeStatus.ps1](#)


```
# Get-DcUraSeTcbPrivilegeStatus.ps1
# Get-DcUraSeTcbPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_setcbprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeTcbPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-106] Configure User Rights: Add workstations to domain on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Add workstations to domain` * **Registry Location**: Stored inside local security database under privilege `SeMachineAccountPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Allows users to add computer accounts to the domain. Restricting this to Administrators prevents standard users from creating unlimited computer accounts, mitigating AD object exhaustion and domain spoofing.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeMachineAccountPrivilege` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Add workstations to domain`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeMachineAccountPrivilege.ps1](../implementation_scripts/Configure-DcUraSeMachineAccountPrivilege.ps1)

```
# Configure-DcUraSeMachineAccountPrivilege.ps1
# Configure-DcUraSeMachineAccountPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_semachineaccountprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_semachineaccountprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.*])$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeMachineAccountPrivilege\s*=") {
        $NewLines += "SeMachineAccountPrivilege = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeMachineAccountPrivilege = *S-1-5-32-544")
    } else {
        $NewLines += "SeMachineAccountPrivilege = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeMachineAccountPrivilegeStatus.ps1](#)

```
# Get-DcUraSeMachineAccountPrivilegeStatus.ps1
# Get-DcUraSeMachineAccountPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_semachineaccountprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeMachineAccountPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-107] Configure User Rights: Adjust memory quotas for a process on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer

Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Adjust memory quotas for a process` *

Registry Location: Stored inside local security database under privilege `SeIncreaseQuotaPrivilege` set to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators)`.

Rationale Allows a process to adjust memory quotas for other processes. Restricting this to Administrators, Network Service, and Local Service prevents denial-of-service memory manipulation on Domain Controllers.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeIncreaseQuotaPrivilege` to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Adjust memory quotas for a process`. 3. Configure the security principal allocation to: `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeIncreaseQuotaPrivilege.ps1[../implementation_scripts/Configure-DcUraSeIncreaseQuotaPrivilege.ps1]

```
# Configure-DcUraSeIncreaseQuotaPrivilege.ps1
# Configure-DcUraSeIncreaseQuotaPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seincreasequotaprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seincreasequotaprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.]*\)$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeIncreaseQuotaPrivilege\s*=") {
        $NewLines += "SeIncreaseQuotaPrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeIncreaseQuotaPrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544")
    } else {
        $NewLines += "SeIncreaseQuotaPrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeIncreaseQuotaPrivilegeStatus.ps1](#)

```
# Get-DcUraSeIncreaseQuotaPrivilegeStatus.ps1
# Get-DcUraSeIncreaseQuotaPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seincreasequotaprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeIncreaseQuotaPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-19,*S-1-5-20,*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline*: User Rights Configuration specifications

[REQ-DC-108] Configure User Rights: Allow log on locally on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Allow log on locally` * **Registry Location**: Stored inside local security database under privilege `SeInteractiveLogonRight` set to `\*S-1-5-9 (Enterprise Domain Controllers), \*S-1-5-32-544 (Administrators)`.

Rationale Allows users to sign in locally at the Domain Controller console. Restricting this to Administrators and Enterprise Domain Controllers protects directory database physical access.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeInteractiveLogonRight` to `\*S-1-5-9 (Enterprise Domain Controllers), \*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Allow log on locally`. 3. Configure the security principal allocation to: `\*S-1-5-9 (Enterprise Domain Controllers), \*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeInteractiveLogonRight.ps1](#)]

(../implementation_scripts/Configure-DcUraSeInteractiveLogonRight.ps1)

```
# Configure-DcUraSeInteractiveLogonRight.ps1
# Configure-DcUraSeInteractiveLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seinteractivelogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seinteractivelogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[.*\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeInteractiveLogonRight\s*=") {
        $NewLines += "SeInteractiveLogonRight = *S-1-5-9,*S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeInteractiveLogonRight = *S-1-5-9,*S-1-5-32-544")
    } else {
        $NewLines += "SeInteractiveLogonRight = *S-1-5-9,*S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeInteractiveLogonRightStatus.ps1](#)

```
# Get-DcUraSeInteractiveLogonRightStatus.ps1
# Get-DcUraSeInteractiveLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seinteractiveLogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeInteractiveLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-9,*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-109] Configure User Rights: Allow log on through Remote Desktop Services on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Allow log on through Remote Desktop Services` * **Registry Location**: Stored inside local security database under privilege `SeRemoteInteractiveLogonRight` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Allows RDP access to Domain Controllers. Enforcing restriction to Administrators prevents lateral RDP access by non-tier-0 accounts.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeRemoteInteractiveLogonRight` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Allow log on through Remote Desktop Services`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeRemoteInteractiveLogonRight.ps1](../implementation_scripts/Configure-DcUraSeRemoteInteractiveLogonRight.ps1)

```
# Configure-DcUraSeRemoteInteractiveLogonRight.ps1
# Configure-DcUraSeRemoteInteractiveLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seremoteinteractiveLogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seremoteinteractiveLogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.]*)\$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeRemoteInteractiveLogonRight\s*=") {
        $NewLines += "SeRemoteInteractiveLogonRight = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeRemoteInteractiveLogonRight = *S-1-5-32-544")
    } else {
        $NewLines += "SeRemoteInteractiveLogonRight = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeRemoteInteractiveLogonRightStatus.ps1](#)

```
# Get-DcUraSeRemoteInteractiveLogonRightStatus.ps1
# Get-DcUraSeRemoteInteractiveLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seremoteinteractivelogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeRemoteInteractiveLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-110] Configure User Rights: Back up files and directories on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Back up files and directories` * **Registry Location**: Stored inside local security database under privilege `SeBackupPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Allows users to read all files on Domain Controllers. Restricting this prevents backup operators from dumping NTDS.dit or registry hives without domain admin authorization.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeBackupPrivilege` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Back up files and directories`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeBackupPrivilege.ps1](#)]

(../implementation_scripts/Configure-DcUraSeBackupPrivilege.ps1)

```
# Configure-DcUraSeBackupPrivilege.ps1
# Configure-DcUraSeBackupPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sebackupprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sebackupprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.]*)\$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "\s*SeBackupPrivilege\s*=") {
        $NewLines += "SeBackupPrivilege = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeBackupPrivilege = *S-1-5-32-544")
    } else {
        $NewLines += "SeBackupPrivilege = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeBackupPrivilegeStatus.ps1](#)

```
# Get-DcUraSeBackupPrivilegeStatus.ps1
# Get-DcUraSeBackupPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sebackupprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeBackupPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-111] Configure User Rights: Bypass traverse checking on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Bypass traverse checking` * **Registry Location**: Stored inside local security database under privilege `SeChangeNotifyPrivilege` set to `\*S-1-5-32-554 (Pre-Windows 2000 Compatible Access), \*S-1-5-11 (Authenticated Users), \*S-1-5-32-544 (Administrators), \*S-1-5-20 (NetworkService), \*S-1-5-19 (LocalService), \*S-1-1-0 (Everyone)`.

Rationale Allows users to pass through directories without checking permissions. Restricting it to system services, Administrators, and authenticated users ensures SYSVOL accessibility while preventing unauthorized path traversal.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeChangeNotifyPrivilege` to `\*S-1-5-32-554 (Pre-Windows 2000 Compatible Access), \*S-1-5-11 (Authenticated Users), \*S-1-5-32-544 (Administrators), \*S-1-5-20 (NetworkService), \*S-1-5-19 (LocalService), \*S-1-1-0 (Everyone)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Bypass traverse checking`. 3. Configure the security principal allocation to: `\*S-1-5-32-554 (Pre-Windows 2000 Compatible Access), \*S-1-5-11 (Authenticated Users), \*S-1-5-32-544 (Administrators), \*S-1-5-20 (NetworkService), \*S-1-5-19 (LocalService), \*S-1-1-0 (Everyone)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeChangeNotifyPrivilege.ps1](#)]

(../implementation_scripts/Configure-DcUraSeChangeNotifyPrivilege.ps1)

```
# Configure-DcUraSeChangeNotifyPrivilege.ps1
# Configure-DcUraSeChangeNotifyPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sechangenotifyprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sechangenotifyprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[.*\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "\s*SeChangeNotifyPrivilege\s*=") {
        $NewLines += "SeChangeNotifyPrivilege = *S-1-5-32-554,*S-1-5-11,*S-1-5-32-544,*S-1-5-20,*S-1-5-19,*S-1-1-0"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeChangeNotifyPrivilege = *S-1-5-32-554,*S-1-5-11,*S-1-5-32-544,*S-1-5-20,*S-1-5-19,*S-1-1-0")
    } else {
        $NewLines += "SeChangeNotifyPrivilege = *S-1-5-32-554,*S-1-5-11,*S-1-5-32-544,*S-1-5-20,*S-1-5-19,*S-1-1-0"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeChangeNotifyPrivilegeStatus.ps1](#)

```
# Get-DcUraSeChangeNotifyPrivilegeStatus.ps1
# Get-DcUraSeChangeNotifyPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sechangenotifyprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeChangeNotifyPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-554,*S-1-5-11,*S-1-5-32-544,*S-1-5-20,*S-1-5-19,*S-1-1-0"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline*: User Rights Configuration specifications

[REQ-DC-112] Configure User Rights: Change the system time on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Change the system time` * **Registry Location**: Stored inside local security database under privilege `SeSystemtimePrivilege` set to `\*S-1-5-32-544 (Administrators), \*S-1-5-19 (LocalService)`.

Rationale Enforces that only Administrators and Local Service can change the clock on Domain Controllers, preserving Kerberos ticket lifetimes and authentications.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeSystemtimePrivilege` to `\*S-1-5-32-544 (Administrators), \*S-1-5-19 (LocalService)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Change the system time`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators), \*S-1-5-19 (LocalService)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeSystemtimePrivilege.ps1](#)]

(../implementation_scripts/Configure-DcUraSeSystemtimePrivilege.ps1)

```
# Configure-DcUraSeSystemtimePrivilege.ps1
# Configure-DcUraSeSystemtimePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sesystemtimeprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sesystemtimeprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\\[Privilege Rights\\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\\[\\.\\*\\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "\\s*SeSystemtimePrivilege\\s*=") {
        $NewLines += "SeSystemtimePrivilege = *S-1-5-32-544,*S-1-5-19"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeSystemtimePrivilege = *S-1-5-32-544,*S-1-5-19")
    } else {
        $NewLines += "SeSystemtimePrivilege = *S-1-5-32-544,*S-1-5-19"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeSystemtimePrivilegeStatus.ps1](#)

```
# Get-DcUraSeSystemtimePrivilegeStatus.ps1
# Get-DcUraSeSystemtimePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sesystemtimeprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeSystemtimePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544,*S-1-5-19"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-113] Configure User Rights: Create a pagefile on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Low * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Create a pagefile` * **Registry Location**: Stored inside local security database under privilege `SeCreatePagefilePrivilege` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Allows creation of system paging files. Restricting to Administrators prevents storage exhaustion on DCs.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeCreatePagefilePrivilege` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Create a pagefile`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeCreatePagefilePrivilege.ps1][../implementation_scripts/Configure-DcUraSeCreatePagefilePrivilege.ps1)

```
# Configure-DcUraSeCreatePagefilePrivilege.ps1
# Configure-DcUraSeCreatePagefilePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_secreatepagefileprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_secreatepagefileprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\\[Privilege Rights\\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.\\*])$" ) {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^\\s*SeCreatePagefilePrivilege\\s*=") {
        $NewLines += "SeCreatePagefilePrivilege = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeCreatePagefilePrivilege = *S-1-5-32-544")
    } else {
        $NewLines += "SeCreatePagefilePrivilege = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeCreatePagefilePrivilegeStatus.ps1](#)


```
# Get-DcUraSeCreatePagefilePrivilegeStatus.ps1
# Get-DcUraSeCreatePagefilePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_secreatepagefileprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeCreatePagefilePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-114] Configure User Rights: Create a token object on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Create a token object` * **Registry Location**: Stored inside local security database under privilege `SeCreateTokenPrivilege` set to `No one (Empty)`.

Rationale Allows creation of arbitrary access tokens. Must be completely empty on Domain Controllers to prevent credential forge attacks.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeCreateTokenPrivilege` to `No one (Empty)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Create a token object`. 3. Configure the security principal allocation to: `No one (Empty)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeCreateTokenPrivilege.ps1](#)]

(../implementation_scripts/Configure-DcUraSeCreateTokenPrivilege.ps1)

```
# Configure-DcUraSeCreateTokenPrivilege.ps1
# Configure-DcUraSeCreateTokenPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_secreatetokenprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_secreatetokenprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[.*\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeCreateTokenPrivilege\s*=") {
        $NewLines += "SeCreateTokenPrivilege = "
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeCreateTokenPrivilege = ")
    } else {
        $NewLines += "SeCreateTokenPrivilege = "
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeCreateTokenPrivilegeStatus.ps1](#)

```
# Get-DcUraSeCreateTokenPrivilegeStatus.ps1
# Get-DcUraSeCreateTokenPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_secreatetokenprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeCreateTokenPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-115] Configure User Rights: Create permanent shared objects on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Create permanent shared objects` * **Registry Location**: Stored inside local security database under privilege `SeCreatePermanentPrivilege` set to `No one (Empty)`.

Rationale Must be empty on DCs to prevent kernel object mapping modifications.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeCreatePermanentPrivilege` to `No one (Empty)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Create permanent shared objects`. 3. Configure the security principal allocation to: `No one (Empty)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeCreatePermanentPrivilege.ps1[../implementation_scripts/Configure-DcUraSeCreatePermanentPrivilege.ps1]

```
# Configure-DcUraSeCreatePermanentPrivilege.ps1
# Configure-DcUraSeCreatePermanentPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_secreatepermanentprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_secreatepermanentprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.]*)\$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeCreatePermanentPrivilege\s*=") {
        $NewLines += "SeCreatePermanentPrivilege = "
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeCreatePermanentPrivilege = ")
    } else {
        $NewLines += "SeCreatePermanentPrivilege = "
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeCreatePermanentPrivilegeStatus.ps1](#)

```
# Get-DcUraSeCreatePermanentPrivilegeStatus.ps1
# Get-DcUraSeCreatePermanentPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_secreatepermanentprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeCreatePermanentPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-116] Configure User Rights: Debug programs on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Debug programs` * **Registry Location**: Stored inside local security database under privilege `SeDebugPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Prevents tools from attaching to lsass.exe to dump AD domain hash tables (ntds.dit hashes). Must be strictly restricted to Administrators.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeDebugPrivilege` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Debug programs`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeDebugPrivilege.ps1](#)]

(../implementation_scripts/Configure-DcUraSeDebugPrivilege.ps1)

```
# Configure-DcUraSeDebugPrivilege.ps1
# Configure-DcUraSeDebugPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sedebugprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sedebugprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.]*)\$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeDebugPrivilege\s*=") {
        $NewLines += "SeDebugPrivilege = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeDebugPrivilege = *S-1-5-32-544")
    } else {
        $NewLines += "SeDebugPrivilege = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeDebugPrivilegeStatus.ps1](#)

```
# Get-DcUraSeDebugPrivilegeStatus.ps1
# Get-DcUraSeDebugPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sedebugprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeDebugPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline*: User Rights Configuration specifications

[REQ-DC-117] Configure User Rights: Deny access to this computer from the network on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Deny access to this computer from the network` * **Registry Location**: Stored inside local security database under privilege `SeDenyNetworkLogonRight` set to `\*S-1-5-32-546 (Guests)`.

Rationale Explicitly denies network logon access to Guests group members to block anonymous network mapping.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeDenyNetworkLogonRight` to `\*S-1-5-32-546 (Guests)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Deny access to this computer from the network`. 3. Configure the security principal allocation to: `\*S-1-5-32-546 (Guests)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeDenyNetworkLogonRight.ps1[../implementation_scripts/Configure-DcUraSeDenyNetworkLogonRight.ps1]

```
# Configure-DcUraSeDenyNetworkLogonRight.ps1
# Configure-DcUraSeDenyNetworkLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sedenynetworklogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sedenynetworklogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\\[Privilege Rights\\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\\([.\\*]\\)$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "\\s*SeDenyNetworkLogonRight\\s*=") {
        $NewLines += "SeDenyNetworkLogonRight = *S-1-5-32-546"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeDenyNetworkLogonRight = *S-1-5-32-546")
    } else {
        $NewLines += "SeDenyNetworkLogonRight = *S-1-5-32-546"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeDenyNetworkLogonRightStatus.ps1](#)

```
# Get-DcUraSeDenyNetworkLogonRightStatus.ps1
# Get-DcUraSeDenyNetworkLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sedenynetworklogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeDenyNetworkLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-546"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-118] Configure User Rights: Deny log on as a batch job on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Deny log on as a batch job` * **Registry Location**: Stored inside local security database under privilege `SeDenyBatchLogonRight` set to `\*S-1-5-32-546 (Guests)`.

Rationale Denies Guest account batch execution to block scheduled tasks persistence.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeDenyBatchLogonRight` to `\*S-1-5-32-546 (Guests)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Deny log on as a batch job`. 3. Configure the security principal allocation to: `\*S-1-5-32-546 (Guests)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeDenyBatchLogonRight.ps1](#)]

(../implementation_scripts/Configure-DcUraSeDenyBatchLogonRight.ps1)

```
# Configure-DcUraSeDenyBatchLogonRight.ps1
# Configure-DcUraSeDenyBatchLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sedenybatchlogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sedenybatchlogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[.*\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeDenyBatchLogonRight\s*=") {
        $NewLines += "SeDenyBatchLogonRight = *S-1-5-32-546"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeDenyBatchLogonRight = *S-1-5-32-546")
    } else {
        $NewLines += "SeDenyBatchLogonRight = *S-1-5-32-546"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeDenyBatchLogonRightStatus.ps1](#)

```
# Get-DcUraSeDenyBatchLogonRightStatus.ps1
# Get-DcUraSeDenyBatchLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sedenybatchlogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeDenyBatchLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-546"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline*: User Rights Configuration specifications

[REQ-DC-119] Configure User Rights: Deny log on as a service on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Deny log on as a service` * **Registry Location**: Stored inside local security database under privilege `SeDenyServiceLogonRight` set to `\*S-1-5-32-546 (Guests)`.

Rationale Prevents Guests from running system services on Domain Controllers.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeDenyServiceLogonRight` to `\*S-1-5-32-546 (Guests)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Deny log on as a service`. 3. Configure the security principal allocation to: `\*S-1-5-32-546 (Guests)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeDenyServiceLogonRight.ps1](../implementation_scripts/Configure-DcUraSeDenyServiceLogonRight.ps1)

```
# Configure-DcUraSeDenyServiceLogonRight.ps1
# Configure-DcUraSeDenyServiceLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sedenyserviceologonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sedenyserviceologonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[.*\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeDenyServiceLogonRight\s*=") {
        $NewLines += "SeDenyServiceLogonRight = *S-1-5-32-546"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeDenyServiceLogonRight = *S-1-5-32-546")
    } else {
        $NewLines += "SeDenyServiceLogonRight = *S-1-5-32-546"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeDenyServiceLogonRightStatus.ps1](#)

```
# Get-DcUraSeDenyServiceLogonRightStatus.ps1
# Get-DcUraSeDenyServiceLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sedenyserviceLogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeDenyServiceLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-546"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline*: User Rights Configuration specifications

[REQ-DC-120] Configure User Rights: Deny log on locally on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Deny log on locally` * **Registry Location**: Stored inside local security database under privilege `SeDenyInteractiveLogonRight` set to `\*S-1-5-32-546 (Guests)`.

Rationale Denies Guests local interactive logons to Domain Controllers.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeDenyInteractiveLogonRight` to `\*S-1-5-32-546 (Guests)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Deny log on locally`. 3. Configure the security principal allocation to: `\*S-1-5-32-546 (Guests)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeDenyInteractiveLogonRight.ps1[../implementation_scripts/Configure-DcUraSeDenyInteractiveLogonRight.ps1]

```
# Configure-DcUraSeDenyInteractiveLogonRight.ps1
# Configure-DcUraSeDenyInteractiveLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sedenyinteractivelogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sedenyinteractivelogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.]*)\$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeDenyInteractiveLogonRight\s*=") {
        $NewLines += "SeDenyInteractiveLogonRight = *S-1-5-32-546"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeDenyInteractiveLogonRight = *S-1-5-32-546")
    } else {
        $NewLines += "SeDenyInteractiveLogonRight = *S-1-5-32-546"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeDenyInteractiveLogonRightStatus.ps1](#)

```
# Get-DcUraSeDenyInteractiveLogonRightStatus.ps1
# Get-DcUraSeDenyInteractiveLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sedenyinteractivelogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeDenyInteractiveLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-546"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-121] Configure User Rights: Deny log on through Remote Desktop Services on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Deny log on through Remote Desktop Services` * **Registry Location**: Stored inside local security database under privilege `SeDenyRemoteInteractiveLogonRight` set to `\*S-1-5-32-546 (Guests)`.

Rationale Explicitly blocks Remote Desktop logons for Guests on Domain Controllers.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeDenyRemoteInteractiveLogonRight` to `\*S-1-5-32-546 (Guests)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Deny log on through Remote Desktop Services`. 3. Configure the security principal allocation to: `\*S-1-5-32-546 (Guests)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeDenyRemoteInteractiveLogonRight.ps1](../implementation_scripts/Configure-DcUraSeDenyRemoteInteractiveLogonRight.ps1)

```
# Configure-DcUraSeDenyRemoteInteractiveLogonRight.ps1
# Configure-DcUraSeDenyRemoteInteractiveLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sedenyremoteinteractivelogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sedenyremoteinteractivelogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[.*\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeDenyRemoteInteractiveLogonRight\s*=") {
        $NewLines += "SeDenyRemoteInteractiveLogonRight = *S-1-5-32-546"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeDenyRemoteInteractiveLogonRight = *S-1-5-32-546")
    } else {
        $NewLines += "SeDenyRemoteInteractiveLogonRight = *S-1-5-32-546"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeDenyRemoteInteractiveLogonRightStatus.ps1](#)


```
# Get-DcUraSeDenyRemoteInteractiveLogonRightStatus.ps1
# Get-DcUraSeDenyRemoteInteractiveLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sedenyremoteinteractivelogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeDenyRemoteInteractiveLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-546"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-122] Configure User Rights: Enable computer and user accounts to be trusted for delegation on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Enable computer and user accounts to be trusted for delegation` * **Registry Location**: Stored inside local security database under privilege `SeEnableDelegationPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Restricting to Administrators prevents unauthorized Kerberos delegation setups on domain computer objects.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeEnableDelegationPrivilege` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Enable computer and user accounts to be trusted for delegation`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeEnableDelegationPrivilege.ps1[../implementation_scripts/Configure-DcUraSeEnableDelegationPrivilege.ps1]

```
# Configure-DcUraSeEnableDelegationPrivilege.ps1
# Configure-DcUraSeEnableDelegationPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seenabledelegationprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seenabledelegationprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.]*)\$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeEnableDelegationPrivilege\s*=") {
        $NewLines += "SeEnableDelegationPrivilege = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeEnableDelegationPrivilege = *S-1-5-32-544")
    } else {
        $NewLines += "SeEnableDelegationPrivilege = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeEnableDelegationPrivilegeStatus.ps1](#)

```
# Get-DcUraSeEnableDelegationPrivilegeStatus.ps1
# Get-DcUraSeEnableDelegationPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seenabledelegationprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeEnableDelegationPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-123] Configure User Rights: Force shutdown from a remote system on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Force shutdown from a remote system` * **Registry Location**: Stored inside local security database under privilege `SeRemoteShutdownPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Enforces that remote shutdowns can only be triggered by Administrators on Domain Controllers.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeRemoteShutdownPrivilege` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Force shutdown from a remote system`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeRemoteShutdownPrivilege.ps1](../implementation_scripts/Configure-DcUraSeRemoteShutdownPrivilege.ps1)

```
# Configure-DcUraSeRemoteShutdownPrivilege.ps1
# Configure-DcUraSeRemoteShutdownPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seremotesshutdownprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seremotesshutdownprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.*])$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeRemoteShutdownPrivilege\s*=") {
        $NewLines += "SeRemoteShutdownPrivilege = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeRemoteShutdownPrivilege = *S-1-5-32-544")
    } else {
        $NewLines += "SeRemoteShutdownPrivilege = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeRemoteShutdownPrivilegeStatus.ps1](#)

```
# Get-DcUraSeRemoteShutdownPrivilegeStatus.ps1
# Get-DcUraSeRemoteShutdownPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seremoteshutdownprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeRemoteShutdownPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-124] Configure User Rights: Generate security audits on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Generate security audits` * **Registry Location**: Stored inside local security database under privilege `SeAuditPrivilege` set to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService)`.

Rationale Restricting to Local Service and Network Service prevents rogue processes from writing fake events to security logs on DCs.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeAuditPrivilege` to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Generate security audits`. 3. Configure the security principal allocation to: `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeAuditPrivilege.ps1](#)]

(../implementation_scripts/Configure-DcUraSeAuditPrivilege.ps1)

```
# Configure-DcUraSeAuditPrivilege.ps1
# Configure-DcUraSeAuditPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seauditprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seauditprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[(.*)\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^\s*SeAuditPrivilege\s*=") {
        $NewLines += "SeAuditPrivilege = *S-1-5-19,*S-1-5-20"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeAuditPrivilege = *S-1-5-19,*S-1-5-20")
    } else {
        $NewLines += "SeAuditPrivilege = *S-1-5-19,*S-1-5-20"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeAuditPrivilegeStatus.ps1](#)

```
# Get-DcUraSeAuditPrivilegeStatus.ps1
# Get-DcUraSeAuditPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seauditprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeAuditPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-19,*S-1-5-20"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-125] Configure User Rights: Load and unload device drivers on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Load and unload device drivers` * **Registry Location**: Stored inside local security database under privilege `SeLoadDriverPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Restricting to Administrators blocks unauthorized kernel-mode driver loads (BYOVD) on DCs.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeLoadDriverPrivilege` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Load and unload device drivers`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeLoadDriverPrivilege.ps1](#)]

(../implementation_scripts/Configure-DcUraSeLoadDriverPrivilege.ps1)

```
# Configure-DcUraSeLoadDriverPrivilege.ps1
# Configure-DcUraSeLoadDriverPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seloaddriverprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seloaddriverprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[(.*)\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "\s*SeLoadDriverPrivilege\s*=") {
        $NewLines += "SeLoadDriverPrivilege = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeLoadDriverPrivilege = *S-1-5-32-544")
    } else {
        $NewLines += "SeLoadDriverPrivilege = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeLoadDriverPrivilegeStatus.ps1](#)

```
# Get-DcUraSeLoadDriverPrivilegeStatus.ps1
# Get-DcUraSeLoadDriverPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seloaddriverprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeLoadDriverPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline*: User Rights Configuration specifications

[REQ-DC-126] Configure User Rights: Lock pages in memory on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Lock pages in memory` * **Registry Location**: Stored inside local security database under privilege `SeLockMemoryPrivilege` set to `No one (Empty)`.

Rationale Must be empty on DCs to prevent locking memory segments and resource exhaustion.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeLockMemoryPrivilege` to `No one (Empty)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Lock pages in memory`. 3. Configure the security principal allocation to: `No one (Empty)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeLockMemoryPrivilege.ps1](#)]

(../implementation_scripts/Configure-DcUraSeLockMemoryPrivilege.ps1)

```
# Configure-DcUraSeLockMemoryPrivilege.ps1
# Configure-DcUraSeLockMemoryPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_selockmemoryprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_selockmemoryprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[.*\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "\s*SeLockMemoryPrivilege\s*=") {
        $NewLines += "SeLockMemoryPrivilege = "
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeLockMemoryPrivilege = ")
    } else {
        $NewLines += "SeLockMemoryPrivilege = "
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeLockMemoryPrivilegeStatus.ps1](#)

```
# Get-DcUraSeLockMemoryPrivilegeStatus.ps1
# Get-DcUraSeLockMemoryPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_selockmemoryprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeLockMemoryPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline*: User Rights Configuration specifications

[REQ-DC-127] Configure User Rights: Log on as a batch job on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Log on as a batch job` * **Registry Location**: Stored inside local security database under privilege `SeBatchLogonRight` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Restricting batch logons to Administrators prevents unauthorized automated task runs on Domain Controllers.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeBatchLogonRight` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Log on as a batch job`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeBatchLogonRight.ps1](#)]

(../implementation_scripts/Configure-DcUraSeBatchLogonRight.ps1)

```
# Configure-DcUraSeBatchLogonRight.ps1
# Configure-DcUraSeBatchLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sebatchlogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sebatchlogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.]*\)$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeBatchLogonRight\s*=") {
        $NewLines += "SeBatchLogonRight = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeBatchLogonRight = *S-1-5-32-544")
    } else {
        $NewLines += "SeBatchLogonRight = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeBatchLogonRightStatus.ps1](#)

```
# Get-DcUraSeBatchLogonRightStatus.ps1
# Get-DcUraSeBatchLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sebatchlogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeBatchLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-128] Configure User Rights: Log on as a service on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Log on as a service` * **Registry Location**: Stored inside local security database under privilege `SeServiceLogonRight` set to `No one (Empty)`.

Rationale Enforcing that no accounts can log on as a service on DCs by default (except dedicated managed service accounts configured explicitly).

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeServiceLogonRight` to `No one (Empty)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Log on as a service`. 3. Configure the security principal allocation to: `No one (Empty)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeServiceLogonRight.ps1](#)]

(../implementation_scripts/Configure-DcUraSeServiceLogonRight.ps1)

```
# Configure-DcUraSeServiceLogonRight.ps1
# Configure-DcUraSeServiceLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seserviceLogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seserviceLogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\\[Privilege Rights\\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.*]\\$)") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^\\s*SeServiceLogonRight\\s*=") {
        $NewLines += "SeServiceLogonRight = "
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeServiceLogonRight = ")
    } else {
        $NewLines += "SeServiceLogonRight = "
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeServiceLogonRightStatus.ps1](#)

```
# Get-DcUraSeServiceLogonRightStatus.ps1
# Get-DcUraSeServiceLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seservicelogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeServiceLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline*: User Rights Configuration specifications

[REQ-DC-129] Configure User Rights: Manage auditing and security log on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Manage auditing and security log` * **Registry Location**: Stored inside local security database under privilege `SeSecurityPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Restricting security log management to Administrators ensures audit trails cannot be altered on DCs.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeSecurityPrivilege` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Manage auditing and security log`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeSecurityPrivilege.ps1](#)]

(../implementation_scripts/Configure-DcUraSeSecurityPrivilege.ps1)

```
# Configure-DcUraSeSecurityPrivilege.ps1
# Configure-DcUraSeSecurityPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sesecurityprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sesecurityprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[(.*)\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^\s*SeSecurityPrivilege\s*=") {
        $NewLines += "SeSecurityPrivilege = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeSecurityPrivilege = *S-1-5-32-544")
    } else {
        $NewLines += "SeSecurityPrivilege = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeSecurityPrivilegeStatus.ps1](#)


```
# Get-DcUraSeSecurityPrivilegeStatus.ps1
# Get-DcUraSeSecurityPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sesecurityprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeSecurityPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-130] Configure User Rights: Modify firmware environment values on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Modify firmware environment values` * **Registry Location**: Stored inside local security database under privilege `SeSystemEnvironmentPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Restricting firmware variable modification to Administrators protects boot integrity on DCs.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeSystemEnvironmentPrivilege` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Modify firmware environment values`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeSystemEnvironmentPrivilege.ps1](../implementation_scripts/Configure-DcUraSeSystemEnvironmentPrivilege.ps1)

```
# Configure-DcUraSeSystemEnvironmentPrivilege.ps1
# Configure-DcUraSeSystemEnvironmentPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sesystemenvironmentprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sesystemenvironmentprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.]*)\$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeSystemEnvironmentPrivilege\s*=") {
        $NewLines += "SeSystemEnvironmentPrivilege = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeSystemEnvironmentPrivilege = *S-1-5-32-544")
    } else {
        $NewLines += "SeSystemEnvironmentPrivilege = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeSystemEnvironmentPrivilegeStatus.ps1](#)

```
# Get-DcUraSeSystemEnvironmentPrivilegeStatus.ps1
# Get-DcUraSeSystemEnvironmentPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sesystemenvironmentprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeSystemEnvironmentPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-131] Configure User Rights: Profile single process on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Low * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Profile single process` * **Registry Location**: Stored inside local security database under privilege `SeProfileSingleProcessPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Restricting profiling to Administrators prevents profiling sensitive processes on DCs.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeProfileSingleProcessPrivilege` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Profile single process`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeProfileSingleProcessPrivilege.ps1](../implementation_scripts/Configure-DcUraSeProfileSingleProcessPrivilege.ps1)

```
# Configure-DcUraSeProfileSingleProcessPrivilege.ps1
# Configure-DcUraSeProfileSingleProcessPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seprofilesinglprocessprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seprofilesinglprocessprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.*])$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "^s*SeProfileSingleProcessPrivilege\s*=") {
        $NewLines += "SeProfileSingleProcessPrivilege = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeProfileSingleProcessPrivilege = *S-1-5-32-544")
    } else {
        $NewLines += "SeProfileSingleProcessPrivilege = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeProfileSingleProcessPrivilegeStatus.ps1](#)

```
# Get-DcUraSeProfileSingleProcessPrivilegeStatus.ps1
# Get-DcUraSeProfileSingleProcessPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seprofilesinglprocessprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeProfileSingleProcessPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-132] Configure User Rights: Restore files and directories on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Restore files and directories` * **Registry Location**: Stored inside local security database under privilege `SeRestorePrivilege` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Restricting restore bypass privileges to Administrators prevents overwriting directory binaries or restoring unauthorized AD backups.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeRestorePrivilege` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Restore files and directories`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeRestorePrivilege.ps1](#)]

(../implementation_scripts/Configure-DcUraSeRestorePrivilege.ps1)

```
# Configure-DcUraSeRestorePrivilege.ps1
# Configure-DcUraSeRestorePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_serestoreprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_serestoreprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^([.]*)\$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "\s*SeRestorePrivilege\s*=") {
        $NewLines += "SeRestorePrivilege = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeRestorePrivilege = *S-1-5-32-544")
    } else {
        $NewLines += "SeRestorePrivilege = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeRestorePrivilegeStatus.ps1](#)

```
# Get-DcUraSeRestorePrivilegeStatus.ps1
# Get-DcUraSeRestorePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_serestoreprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeRestorePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-133] Configure User Rights: Shut down the system on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Shut down the system` * **Registry Location**: Stored inside local security database under privilege `SeShutdownPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Restricting local shutdown privilege to Administrators prevents denial-of-service shutdowns on DCs.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeShutdownPrivilege` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Shut down the system`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeShutdownPrivilege.ps1](#)]

(../implementation_scripts/Configure-DcUraSeShutdownPrivilege.ps1)

```
# Configure-DcUraSeShutdownPrivilege.ps1
# Configure-DcUraSeShutdownPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seshutdownprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seshutdownprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[([.*])\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "\s*SeShutdownPrivilege\s*=") {
        $NewLines += "SeShutdownPrivilege = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeShutdownPrivilege = *S-1-5-32-544")
    } else {
        $NewLines += "SeShutdownPrivilege = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeShutdownPrivilegeStatus.ps1](#)

```
# Get-DcUraSeShutdownPrivilegeStatus.ps1
# Get-DcUraSeShutdownPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seshutdownprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeShutdownPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline*: User Rights Configuration specifications

[REQ-DC-134] Configure User Rights: Synchronize directory service data on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Synchronize directory service data` * **Registry Location**: Stored inside local security database under privilege `SeSyncAgentPrivilege` set to `No one (Empty)`.

Rationale Allows replication of directory changes. This should be empty on Domain Controllers (only AD replication engine uses it natively, no separate account should hold it).

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeSyncAgentPrivilege` to `No one (Empty)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Synchronize directory service data`. 3. Configure the security principal allocation to: `No one (Empty)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcUraSeSyncAgentPrivilege.ps1](#)]

(../implementation_scripts/Configure-DcUraSeSyncAgentPrivilege.ps1)

```
# Configure-DcUraSeSyncAgentPrivilege.ps1
# Configure-DcUraSeSyncAgentPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sesyncagentprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sesyncagentprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[.*\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "\s*SeSyncAgentPrivilege\s*=") {
        $NewLines += "SeSyncAgentPrivilege = "
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeSyncAgentPrivilege = ")
    } else {
        $NewLines += "SeSyncAgentPrivilege = "
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeSyncAgentPrivilegeStatus.ps1](#)

```
# Get-DcUraSeSyncAgentPrivilegeStatus.ps1
# Get-DcUraSeSyncAgentPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sesyncagentprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeSyncAgentPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline: User Rights Configuration specifications

[REQ-DC-135] Configure User Rights: Take ownership of files or other objects on Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Take ownership of files or other objects` * **Registry Location**: Stored inside local security database under privilege `SeTakeOwnershipPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

Rationale Restricting ownership takeover to Administrators protects Domain Controller filesystem hierarchy.

Legacy Impact & Compatibility * **Operational Impact**: Restricting `SeTakeOwnershipPrivilege` to `\*S-1-5-32-544 (Administrators)` protects Domain Controllers filesystem and service execution interfaces. Ensure core directory sync or backup agents do not lose validation access.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Take ownership of files or other objects`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

DcUraSeTakeOwnershipPrivilege.ps1](../implementation_scripts/Configure-DcUraSeTakeOwnershipPrivilege.ps1)

```
# Configure-DcUraSeTakeOwnershipPrivilege.ps1
# Configure-DcUraSeTakeOwnershipPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_setakeownershipprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_setakeownershipprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
    $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
    if ($Line -match "^\[.*\]$") {
        if ($Matches[1] -eq "Privilege Rights") {
            $InPriv = $true
        } else {
            $InPriv = $false
        }
    }
    if ($InPriv -and $Line -match "\s*SeTakeOwnershipPrivilege\s*=") {
        $NewLines += "SeTakeOwnershipPrivilege = *S-1-5-32-544"
        $KeyAdded = $true
    } else {
        $NewLines += $Line
    }
}

if (-not $KeyAdded) {
    # Find Privilege Rights section index and insert it right after
    $Idx = $NewLines.IndexOf("[Privilege Rights]")
    if ($Idx -ge 0) {
        $NewLines.Insert($Idx + 1, "SeTakeOwnershipPrivilege = *S-1-5-32-544")
    } else {
        $NewLines += "SeTakeOwnershipPrivilege = *S-1-5-32-544"
    }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-DcUraSeTakeOwnershipPrivilegeStatus.ps1](#)

```
# Get-DcUraSeTakeOwnershipPrivilegeStatus.ps1
# Get-DcUraSeTakeOwnershipPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_setakeownershipprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
    Write-Output "Non-Compliant"
    exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeTakeOwnershipPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
    $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Protective controls baselines on Domain Controllers *
 Microsoft Security Baseline*: User Rights Configuration specifications

[REQ-DC-024] Configure dSHeuristics Attribute

Target Scope * **Applicable Systems**: Domain Controllers * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **Active Directory Object**: `CN=Directory Service,CN=Windows NT,CN=Services,CN=Configuration,DC=` * **Attribute**: `dSHeuristics` * **Recommended Value for Level 5 Security**: `0000000001000000000200000001130` (or a customized string with indices 7, 8, 16, 21, 28, 29, 31 set to secure values)

Rationale The `dSHeuristics` attribute is a Unicode string that defines forest-wide heuristic configuration settings for Active Directory. Individual characters at specific indices (1-based) modify security and protocol behaviors on Domain Controllers:

1. **fLDAPBlockAnonOps** (7th character): Controls anonymous LDAP operations. If set to **2**, anonymous binds and searches are permitted, allowing unauthorized users to map out directory structures. Setting it to **0** blocks anonymous operations.
2. **fAllowAnonNSPI** (8th character): Controls anonymous access to the Name Service Provider Interface (NSPI). If set to **1** or any value other than **0**, anonymous clients can query address books, which allows user enumeration. Setting it to **0** restricts NSPI queries to authenticated users.
3. **dwAdminSDExMask** (16th character): Excludes administrative groups from the automatic security descriptor protection mechanism (the **AdminSDHolder** background task). By default (**0**), groups like Account Operators, Server Operators, Print Operators, and Backup Operators are protected. If set to non-zero, this protection is bypassed, risking privilege escalation.
4. **DoNotVerifyUPNAndOrSPNUniqueness** (21st character): Controls uniqueness enforcement for User Principal Names (UPN) and Service Principal Names (SPN). Disabling this check (**1** or non-zero) can lead to identity spoofing or credential hijacking by registering duplicate names (KB5008382).
5. **AttributeAuthorizationOnLDAPAdd** (28th character) and **BlockOwnerImplicitRights** (29th character): Introduced in KB5008383. Setting these to **1** enforces strict authorization validations and auditing during LDAP Add operations, preventing malicious creators from abusing implicit owner privileges.
6. **DisableConfidentialAttributeEncryptionRequirements** (31st character): Controls connection security requirements for retrieving or writing confidential attributes. Allowing unencrypted connections (non-zero) risks exposing sensitive information such as password hashes or BitLocker recovery keys on the network.

To reach the maximum Level 5 security state, all dangerous features must be disabled, and KB5008383 protections must be explicitly set to **1**.

Legacy Impact & Compatibility * **Anonymous Access**: Restricting `fLDAPBlockAnonOps` and `fAllowAnonNSPI` will prevent legacy mail clients (such as Outlook 2003) or older third-party applications from querying the Global Address List (GAL) anonymously. Ensure all applications querying the directory authenticate securely. * **KB5008383 Controls**: Explicitly setting `AttributeAuthorizationOnLDAPAdd` and `BlockOwnerImplicitRights` to `1` changes the security validation during LDAP Add operations. Provisions and automation workflows that rely on implicit owner permissions when creating objects must be verified in a staging environment.

Implementation Steps

Option A: Graphical Tools (ADSI Edit / Ldp.exe)

1. Open **ADSI Edit** (`adsiedit.msc`) with Enterprise Administrator or Domain Administrator (of the forest root) privileges.
2. Right-click **ADSI Edit** in the left pane and select **Connect to...**
3. Under **Connection Point**, choose **Select a well-known Naming Context** and select **Configuration**. Click **OK**.
4. In the left pane, expand the tree: **Configuration** → **CN=Configuration,DC=...** → **CN=Services** → **CN=Windows NT** → **CN=Directory Service**
5. Right-click **CN=Directory Service** and select **Properties**.
6. Locate the **dSHeuristics** attribute in the Attribute Editor list and click **Edit**.
7. If the current value is **<not set>**, set it to the standard Level 5 string: `0000000001000000000200000001130`
8. If a value is already present, modify it carefully by preserving existing non-security related positions, updating only the specific security positions:
 - **7th character** (fLDAPBlockAnonOps): Must not be **2** (change to **0**)
 - **8th character** (fAllowAnonNSPI): Must be **0**
 - **16th character** (dwAdminSDExMask): Must be **0**
 - **21st character** (DoNotVerifyUPNAndOrSPNUniqueness): Must be **0**
 - **28th character** (AttributeAuthorizationOnLDAPAdd): Must be **1**
 - **29th character** (BlockOwnerImplicitRights): Must be **1**
 - **31st character** (DisableConfidentialAttributeEncryptionRequirements): Must be **0**
 - Ensure control characters **10th** is **1**, **20th** is **2**, and **30th** is **3** if the string length reaches those values.
9. Click **OK** and apply the changes.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)
Use this method to apply the setting programmatically.

[Download Script: Configure-dSHeuristics.ps1](#)

```
# Configure-dSHeuristics.ps1
# Description: Configures the dSHeuristics attribute to reach Level 5 security.

Write-Host "Applying hardening requirement: Configure dSHeuristics..." -ForegroundColor Cyan

# Ensure we can read the Configuration naming context
$rootDSE = [ADSI]"LDAP://RootDSE"
$configNamingContext = $rootDSE.configurationNamingContext[0]
$dsPath = "LDAP://CN=Directory Service,CN=Windows NT,CN=Services,CN=Configuration,$configNamingContext"

$dsObject = [ADSI]$dsPath
$currentHeuristics = $dsObject.Properties["dSHeuristics"].Value

$newHeuristics = ""

if ($null -eq $currentHeuristics -or $currentHeuristics -eq "") {
    # Initialize with default 31-character Level 5 string if not already configured
    $newHeuristics = "0000000001000000000200000001130"
} else {
    # If already set, we need to modify specific indices while preserving other values
    $charArray = $currentHeuristics.ToCharArray()

    # Pad array to at least 31 characters to support all configurations
    if ($charArray.Length -lt 31) {
        $tempArray = [char[]](New-Object char[] 31)
        for ($i = 0; $i -lt 31; $i++) {
            if ($i -lt $charArray.Length) {
                $tempArray[$i] = $charArray[$i]
            } else {
                $tempArray[$i] = [char]"0"
            }
        }
        $charArray = $tempArray
    }

    # 7: fLDAPBlockAnonOps (Index 6)
    if ($charArray[6] -eq [char]"2") {
        $charArray[6] = [char]"0"
    }

    # 8: fAllowAnonNSPI (Index 7) → must be "0"
    $charArray[7] = [char]"0"

    # 10: tenthChar (Index 9) → control character, must be "1"
    $charArray[9] = [char]"1"

    # 16: dwAdminSDExMask (Index 15) → must be "0"
    $charArray[15] = [char]"0"

    # 20: twentiethChar (Index 19) → control character, must be "2"
    $charArray[19] = [char]"2"

    # 21: DoNotVerifyUPNAndOrSPNUniqueness (Index 20) → must be "0"
    $charArray[20] = [char]"0"

    # 28: AttributeAuthorizationOnLDAPAdd (Index 27) → must be "1" for Level 5
    $charArray[27] = [char]"1"

    # 29: BlockOwnerImplicitRights (Index 28) → must be "1" for Level 5
    $charArray[28] = [char]"1"

    # 30: thirtiethChar (Index 29) → control character, must be "3"
    $charArray[29] = [char]"3"

    # 31: DisableConfidentialAttributeEncryptionRequirements (Index 30) → must be "0"
    $charArray[30] = [char]"0"

    $newHeuristics = [string]::new($charArray)
}

if ($currentHeuristics -ne $newHeuristics) {
    Write-Host "Updating dSHeuristics from '$currentHeuristics' to '$newHeuristics'..." -ForegroundColor Yellow
    $dsObject.Properties["dSHeuristics"].Value = $newHeuristics
    $dsObject.CommitChanges()
}
```

```
        Write-Host "dSHeuristics updated successfully." -ForegroundColor Green
    } else {
        Write-Host "dSHeuristics is already configured securely ('$currentHeuristics'). No action required." -ForegroundColor Green
    }
```

To verify the setting has been applied:

[Download Script: Get-dSHeuristicsStatus.ps1](#)

```
# Get-dSHeuristicsStatus.ps1
# Description: Audits the dSHeuristics attribute settings for security compliance.

Write-Host "--- Auditing dSHeuristics Configuration ---" -ForegroundColor Cyan

$rootDSE = [ADSI]"LDAP://RootDSE"
$configNamingContext = $rootDSE.configurationNamingContext[0]
$dsPath = "LDAP://CN=Directory Service,CN=Windows NT,CN=Services,CN=Configuration,$configNamingContext"

$dsObject = [ADSI]$dsPath
$dsHeuristics = $dsObject.Properties["dSHeuristics"].Value

function Get-HeuristicChar {
    param(
        [string]$String,
        [int]$Index, # 0-based index
        [char]$Default = [char]"0"
    )
    if ($null -ne $String -and $String.Length -gt $Index) {
        return $String.Substring($Index, 1)
    }
    return $Default
}

$vulnerable = $false
$nonLevel5 = $false

if ($null -eq $dsHeuristics -or $dsHeuristics -eq "") {
    Write-Host "[!] dSHeuristics is not set. Default settings apply." -ForegroundColor Yellow
    Write-Host "    - AttributeAuthorizationOnLDAPAdd: Not Set (defaults to 0, which is Level 3/4 but NOT Level 5)" -ForegroundColor Yellow
    Write-Host "    - BlockOwnerImplicitRights: Not Set (defaults to 0, which is Level 3/4 but NOT Level 5)" -ForegroundColor Yellow
    $nonLevel5 = $true
} else {
    Write-Host "[+] Current dSHeuristics string: $dsHeuristics" -ForegroundColor Green

    # 7. fLDAPBlockAnonOps (Index 6)
    $val = Get-HeuristicChar -String $dsHeuristics -Index 6
    if ($val -eq "2") {
        Write-Host "[!] VULNERABLE: fLDAPBlockAnonOps is set to '2' (Allows anonymous LDAP operations)." -ForegroundColor Red
        $vulnerable = $true
    } else {
        Write-Host "[+] fLDAPBlockAnonOps (Index 6): Set to '$val' (Anonymous LDAP operations blocked)." -ForegroundColor Green
    }

    # 8. fAllowAnonNSPI (Index 7)
    $val = Get-HeuristicChar -String $dsHeuristics -Index 7
    if ($val -ne "0") {
        Write-Host "[!] VULNERABLE: fAllowAnonNSPI is set to '$val' (Allows anonymous NSPI access; must be 0)." -ForegroundColor Red
        $vulnerable = $true
    } else {
        Write-Host "[+] fAllowAnonNSPI (Index 7): Set to '0' (Anonymous NSPI blocked)." -ForegroundColor Green
    }

    # 16. dwAdminSDExMask (Index 15)
    $val = Get-HeuristicChar -String $dsHeuristics -Index 15
    if ($val -ne "0") {
        Write-Host "[!] VULNERABLE: dwAdminSDExMask is set to '$val' (Disables protection for administrative groups; must be 0)." -ForegroundColor Red
        $vulnerable = $true
    } else {
        Write-Host "[+] dwAdminSDExMask (Index 15): Set to '0' (All default admin groups protected)." -ForegroundColor Green
    }

    # 21. DoNotVerifyUPNAndOrSPNUniqueness (Index 20)
    $val = Get-HeuristicChar -String $dsHeuristics -Index 20
    if ($val -ne "0") {
        Write-Host "[!] VULNERABLE: DoNotVerifyUPNAndOrSPNUniqueness is set to '$val' (Bypasses UPN/SPN uniqueness checks; must be 0)." -ForegroundColor Red
        $vulnerable = $true
    } else {
        Write-Host "[+] DoNotVerifyUPNAndOrSPNUniqueness (Index 20): Set to '0' (Uniqueness checks active)." -ForegroundColor Green
    }
}
```



```

# 28. AttributeAuthorizationOnLDAPAdd (Index 27)
$val = Get-HeuristicChar -String $dsHeuristics -Index 27
if ($val -eq "2") {
    Write-Host "[!] VULNERABLE: AttributeAuthorizationOnLDAPAdd is set to '2' (Bypasses LDAP Add authorization checks)." -ForegroundColor Red
    $vulnerable = $true
} elseif ($val -ne "1") {
    Write-Host "[!] WARNING: AttributeAuthorizationOnLDAPAdd is set to '$val' (Must be set to '1' for Level 5 security)." -ForegroundColor Yellow
    $nonLevel5 = $true
} else {
    Write-Host "[+] AttributeAuthorizationOnLDAPAdd (Index 27): Set to '1' (Level 5 secure)." -ForegroundColor Green
}

# 29. BlockOwnerImplicitRights (Index 28)
$val = Get-HeuristicChar -String $dsHeuristics -Index 28
if ($val -eq "2") {
    Write-Host "[!] VULNERABLE: BlockOwnerImplicitRights is set to '2' (Bypasses owner implicit rights protection)." -ForegroundColor Red
    $vulnerable = $true
} elseif ($val -ne "1") {
    Write-Host "[!] WARNING: BlockOwnerImplicitRights is set to '$val' (Must be set to '1' for Level 5 security)." -ForegroundColor Yellow
    $nonLevel5 = $true
} else {
    Write-Host "[+] BlockOwnerImplicitRights (Index 28): Set to '1' (Level 5 secure)." -ForegroundColor Green
}

# 31. DisableConfidentialAttributeEncryptionRequirements (Index 30)
$val = Get-HeuristicChar -String $dsHeuristics -Index 30
if ($val -ne "0") {
    Write-Host "[!] VULNERABLE: DisableConfidentialAttributeEncryptionRequirements is set to '$val' (Allows unencrypted transmission of confidential attributes; must be 0)." -ForegroundColor Red
    $vulnerable = $true
} else {
    Write-Host "[+] DisableConfidentialAttributeEncryptionRequirements (Index 30): Set to '0' (Requires encrypted connection)." -ForegroundColor Green
}

if ($vulnerable) {
    Write-Host "[!] Result: VULNERABLE (Dangerous settings detected in dSHeuristics)." -ForegroundColor Red
} elseif ($nonLevel5) {
    Write-Host "[!] Result: Partially Secure (No highly dangerous settings, but Level 5 maximum security is not reached)." -ForegroundColor Yellow
} else {
    Write-Host "[+] Result: SECURE (Level 5 security reached)." -ForegroundColor Green
}

```

Sources & Compliance References * **ANSSI AD Hardening Guide***: Section on dSHeuristics Settings & Tier 0 Protection * **CIS Benchmark***: CIS Microsoft Windows Server Benchmark - Section 2.3 (Directory Services / Security Options) * **Microsoft Technical Specification***: [MS-ADTS] Section 6.1.1.2.4.1.2 (dSHeuristics) * **Microsoft Security Guidance***: KB5008382 (UPN/SPN Uniqueness), KB5008383 (LDAP Add ownership protections)

[REQ-DC-025] Configure Security Options for Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers, Member Servers. * **Operating Systems**: Windows Server 2016 and above.

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` * **Registry Locations**: *

```
'HKLM\System\CurrentControlSet\Control\Lsa\SubmitQueue' * 'HKLM\System\CurrentControlSet\Control\Lsa\DisableDomainCreds' *
'HKLM\System\CurrentControlSet\Services\Netlogon\Parameters\AllowVulnerableChannel' *
'HKLM\System\CurrentControlSet\Services\Netlogon\Parameters\RefusePasswordChange' *
'HKLM\System\CurrentControlSet\Services\Netlogon\Parameters\DisablePasswordChange' *
'HKLM\System\CurrentControlSet\Services\Netlogon\Parameters\MaximumPasswordAge' *
'HKLM\System\CurrentControlSet\Services\Netlogon\Parameters\RequireStrongKey' *
'HKLM\System\CurrentControlSet\Services\LanmanServer\Parameters\NullSessionPipes' *
'HKLM\System\CurrentControlSet\Control\SecurePipeServers\winreg\AllowedExactPaths\Machine' *
'HKLM\System\CurrentControlSet\Control\SecurePipeServers\winreg\AllowedPaths\Machine'
```

Rationale Windows Security Options control critical security settings such as anonymous access to Named Pipes, remote access to the registry (winreg), and domain member secure channel parameters. Restricting these options prevents credential sniffing, service enumeration, and remote unauthorized inspection of local configurations.

Specifically, the following settings are configured to protect the Tier 0 administrative boundary:

1. **Server Operator Task Scheduling (`SubmitQueue`)**: Restricting Server Operators from scheduling tasks on Domain Controllers prevents privilege escalation and command execution pathways.
2. **Secure Channel Protection (`RequireStrongKey` / `AllowVulnerableChannel`)**: Restricting secure channels to strong session keys and blocking vulnerable connections mitigates coercion and impersonation attacks.
3. **Machine Password Change (`RefusePasswordChange` / `DisablePasswordChange` / `MaximumPasswordAge`)**: Ensuring domain members rotate machine account passwords at regular intervals prevents offline account hijacking while forcing the DC to process password changes correctly.
4. **Anonymous Named Pipe Restricting (`NullSessionPipes`)**: Setting `NullSessionPipes` to a minimum required list prevents anonymous callers from enumerating user SIDs or directories on Domain Controllers.
5. **Remote Registry Restrictions (`winreg` Exact Paths and Paths)**: Restricting remote WinReg operations prevents information disclosure and configuration scanning.
6. **Network Credentials Storage Restrictions (`DisableDomainCreds`)**: Blocking the local storage of credentials or passwords for network authentication prevents local security databases from caching reusable network hashes, hindering lateral movement.

Legacy Impact & Compatibility * **Remote Management Tools**: Third-party remote monitoring or inventory systems that rely on remote registry scanning (WinReg) may fail. Configure these systems to authenticate using dedicated service accounts or verify they use modern APIs. * **Anonymous Access**: Legacy applications attempting to query Named Pipes anonymously will be blocked. Ensure applications authenticate using standard domain credentials.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management workstation.
2. Edit the modular Domain Controllers GPO (e.g., `SEC_DomainControllers_Hardening`).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options`
4. Configure the following policies as specified:

POLICY SETTING	SETTING VALUE
Domain controller: Allow server operators to schedule tasks	Disabled
Domain controller: Allow vulnerable Netlogon secure channel connections	Not Configured
Domain controller: Refuse machine account password changes	Disabled
Domain member: Disable machine account password changes	Disabled
Domain member: Maximum machine account password age	30
Domain member: Require strong (Windows 2000 or later) session key	Enabled
Network access: Do not allow storage of passwords and credentials for network authentication	Enabled
Network access: Named Pipes that can be accessed anonymously	netlogon , samr , lsarpc
Network access: Remotely accessible registry paths	System\CurrentControlSet\Control\ProductOptions , System\CurrentControlSet\Control\Server Applications , Software\Microsoft\Windows NT\CurrentVersion
Network access: Remotely accessible registry paths and sub-paths	System\CurrentControlSet\Control\Print\Printers , System\CurrentControlSet\Services\Eventlog , Software\Microsoft\OLAP Server , Software\Microsoft\Windows NT\CurrentVersion\Print , Software\Microsoft\Windows NT\CurrentVersion\Windows , System\CurrentControlSet\Control\ContentIndex , System\CurrentControlSet\Control\Terminal Server , System\CurrentControlSet\Control\Terminal Server\UserConfig , System\CurrentControlSet\Control\Terminal Server\DefaultUserConfiguration , Software\Microsoft\Windows NT\CurrentVersion\PerfLib , System\CurrentControlSet\Services\SysmonLog

5. Link the GPO to the appropriate Organizational Unit.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the registry configuration locally.

[Download Script: Configure-DCSecurityOptions.ps1](#)

```
# Configure-DCSecurityOptions.ps1
# Description: Configures GPO Security Options registry keys for Domain Controllers.

Write-Host "Applying hardening requirement: Configure Security Options for Domain Controllers..." -ForegroundColor Cyan

# 1. Domain controller: Allow server operators to schedule tasks = Disabled (SubmitQueue = 0)
$LsaPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
if (-not (Test-Path $LsaPath)) {
    New-Item -Path $LsaPath -Force | Out-Null
}
Set-ItemProperty -Path $LsaPath -Name "SubmitQueue" -Value 0 -Type DWord -Force
Write-Host "    Domain controller: Allow server operators to schedule tasks set to Disabled." -ForegroundColor Green

# 1b. Network access: Do not allow storage of passwords and credentials for network authentication = Enabled (DisableDomainCreds = 1)
Set-ItemProperty -Path $LsaPath -Name "DisableDomainCreds" -Value 1 -Type DWord -Force
Write-Host "    Network access: Do not allow storage of credentials set to Enabled." -ForegroundColor Green

# 2. Domain controller: Allow vulnerable Netlogon secure channel connections = Not Configured / Explicitly Blocked
$NetlogonParamsPath = "HKLM:\System\CurrentControlSet\Services\Netlogon\Parameters"
if (-not (Test-Path $NetlogonParamsPath)) {
    New-Item -Path $NetlogonParamsPath -Force | Out-Null
}
Set-ItemProperty -Path $NetlogonParamsPath -Name "AllowVulnerableChannel" -Value 0 -Type DWord -Force
Write-Host "    Domain controller: Allow vulnerable Netlogon connections set to Disabled." -ForegroundColor Green

# 3. Domain controller: Refuse machine account password changes = Disabled
Set-ItemProperty -Path $NetlogonParamsPath -Name "RefusePasswordChange" -Value 0 -Type DWord -Force
Write-Host "    Domain controller: Refuse machine account password changes set to Disabled." -ForegroundColor Green

# 4. Domain member: Disable machine account password changes = Disabled
Set-ItemProperty -Path $NetlogonParamsPath -Name "DisablePasswordChange" -Value 0 -Type DWord -Force
Write-Host "    Domain member: Disable machine account password changes set to Disabled." -ForegroundColor Green

# 5. Domain member: Maximum machine account password age = 30
Set-ItemProperty -Path $NetlogonParamsPath -Name "MaximumPasswordAge" -Value 30 -Type DWord -Force
Write-Host "    Domain member: Maximum machine account password age set to 30." -ForegroundColor Green

# 6. Domain member: Require strong (Windows 2000 or later) session key = Enabled
Set-ItemProperty -Path $NetlogonParamsPath -Name "RequireStrongKey" -Value 1 -Type DWord -Force
Write-Host "    Domain member: Require strong session key set to Enabled." -ForegroundColor Green

# 7. Network access: Named Pipes that can be accessed anonymously (netlogon, samr, lsarpc)
$LanmanServerParamsPath = "HKLM:\System\CurrentControlSet\Services\LanmanServer\Parameters"
if (-not (Test-Path $LanmanServerParamsPath)) {
    New-Item -Path $LanmanServerParamsPath -Force | Out-Null
}
$NullSessionPipes = @("netlogon", "samr", "lsarpc")
Set-ItemProperty -Path $LanmanServerParamsPath -Name "NullSessionPipes" -Value $NullSessionPipes -Type MultiString -Force
Write-Host "    Network access: Named Pipes that can be accessed anonymously configured." -ForegroundColor Green

# 8. Network access: Remotely accessible registry paths
$WinregExactPath = "HKLM:\System\CurrentControlSet\Control\SecurePipeServers\winreg\AllowedExactPaths"
if (-not (Test-Path $WinregExactPath)) {
    New-Item -Path $WinregExactPath -Force | Out-Null
}
$AllowedExactPaths = @(
    "System\CurrentControlSet\Control\ProductOptions",
    "System\CurrentControlSet\Control\Server Applications",
    "Software\Microsoft\Windows NT\CurrentVersion"
)
Set-ItemProperty -Path $WinregExactPath -Name "Machine" -Value $AllowedExactPaths -Type MultiString -Force
Write-Host "    Network access: Remotely accessible registry paths configured." -ForegroundColor Green

# 9. Network access: Remotely accessible registry paths and sub-paths
$WinregPath = "HKLM:\System\CurrentControlSet\Control\SecurePipeServers\winreg\AllowedPaths"
if (-not (Test-Path $WinregPath)) {
    New-Item -Path $WinregPath -Force | Out-Null
}
$AllowedPaths = @(
    "System\CurrentControlSet\Control\Print\Printers",
    "System\CurrentControlSet\Services\Eventlog",
    "Software\Microsoft\OLAP Server",
    "Software\Microsoft\Windows NT\CurrentVersion\Print",

```

```

    "Software\Microsoft\Windows NT\CurrentVersion\Windows",
    "System\CurrentControlSet\Control\ContentIndex",
    "System\CurrentControlSet\Control\Terminal Server",
    "System\CurrentControlSet\Control\Terminal Server\UserConfig",
    "System\CurrentControlSet\Control\Terminal Server\DefaultUserConfiguration",
    "Software\Microsoft\Windows NT\CurrentVersion\Perflib",
    "System\CurrentControlSet\Services\SysmonLog"
)

# Optional AD CS or WINS sub-paths
$CertSvc = Get-Service -Name "CertSvc" -ErrorAction SilentlyContinue
if ($null -ne $CertSvc) {
    $AllowedPaths += "System\CurrentControlSet\Services\CertSvc"
    Write-Host "    AD CS detected: adding CertSvc registry path." -ForegroundColor Gray
}
$WinsSvc = Get-Service -Name "WINS" -ErrorAction SilentlyContinue
if ($null -ne $WinsSvc) {
    $AllowedPaths += "System\CurrentControlSet\Services\WINS"
    Write-Host "    WINS detected: adding WINS registry path." -ForegroundColor Gray
}

Set-ItemProperty -Path $WinregPath -Name "Machine" -Value $AllowedPaths -Type MultiString -Force
Write-Host "    Network access: Remotely accessible registry paths and sub-paths configured." -ForegroundColor Green

Write-Host "Domain Controller Security Options configuration completed." -ForegroundColor Green

```

To audit local configurations, execute the following script:

[Download Script: Get-DCSecurityOptionsStatus.ps1](#)

```

# Get-DCSecurityOptionsStatus.ps1
# Description: Audits GPO Security Options registry keys for Domain Controllers.

Write-Host "--- Auditing Domain Controller Security Options ---" -ForegroundColor Cyan
$Vulnerable = $false

# Helper function to check DWORD value
function Test-DwordValue {
    param(
        [string]$Path,
        [string]$ValueName,
        [int]$ExpectedValue
    )
    $Val = Get-ItemProperty -Path $Path -Name $ValueName -ErrorAction SilentlyContinue
    if ($null -eq $Val) {
        Write-Host "    [!] VULNERABLE: $($ValueName) under $($Path) is not configured." -ForegroundColor Red
        return $true
    }
    $ActualVal = $Val.$ValueName
    if ($ActualVal -ne $ExpectedValue) {
        Write-Host "    [!] VULNERABLE: $($ValueName) is set to $($ActualVal) (Expected: $($ExpectedValue))" -ForegroundColor Red
        return $true
    }
    Write-Host "    [+] $($ValueName) is set to $($ActualVal) (Compliant)." -ForegroundColor Green
    return $false
}

# 1. Domain controller: Allow server operators to schedule tasks (SubmitQueue = 0)
if (Test-DwordValue -Path "HKLM:\System\CurrentControlSet\Control\Lsa" -ValueName "SubmitQueue" -ExpectedValue 0) {
    $Vulnerable = $true
}

# 1b. Network access: Do not allow storage of passwords and credentials for network authentication (DisableDomainCreds = 1)
if (Test-DwordValue -Path "HKLM:\System\CurrentControlSet\Control\Lsa" -ValueName "DisableDomainCreds" -ExpectedValue 1) {
    $Vulnerable = $true
}

# 2. Domain controller: Allow vulnerable Netlogon connections (AllowVulnerableChannel = 0)
if (Test-DwordValue -Path "HKLM:\System\CurrentControlSet\Services\Netlogon\Parameters" -ValueName "AllowVulnerableChannel" -ExpectedValue 0) {
    $Vulnerable = $true
}

# 3. Domain controller: Refuse machine account password changes (RefusePasswordChange = 0)
if (Test-DwordValue -Path "HKLM:\System\CurrentControlSet\Services\Netlogon\Parameters" -ValueName "RefusePasswordChange" -ExpectedValue 0) {
    $Vulnerable = $true
}

# 4. Domain member: Disable machine account password changes (DisablePasswordChange = 0)
if (Test-DwordValue -Path "HKLM:\System\CurrentControlSet\Services\Netlogon\Parameters" -ValueName "DisablePasswordChange" -ExpectedValue 0) {
    $Vulnerable = $true
}

# 5. Domain member: Maximum machine account password age (MaximumPasswordAge = 30)
$MaxAgeVal = Get-ItemProperty -Path "HKLM:\System\CurrentControlSet\Services\Netlogon\Parameters" -Name "MaximumPasswordAge" -ErrorAction SilentlyContinue
if ($null -eq $MaxAgeVal) {
    Write-Host "    [!] VULNERABLE: MaximumPasswordAge is not configured." -ForegroundColor Red
    $Vulnerable = $true
} else {
    $ActualAge = $MaxAgeVal.MaximumPasswordAge
    if ($ActualAge -gt 30 -or $ActualAge -eq 0) {
        Write-Host "    [!] VULNERABLE: MaximumPasswordAge is $($ActualAge) (Expected: 30 or fewer, but not 0)" -ForegroundColor Red
        $Vulnerable = $true
    } else {
        Write-Host "    [+] MaximumPasswordAge is $($ActualAge) (Compliant)." -ForegroundColor Green
    }
}

# 6. Domain member: Require strong session key (RequireStrongKey = 1)
if (Test-DwordValue -Path "HKLM:\System\CurrentControlSet\Services\Netlogon\Parameters" -ValueName "RequireStrongKey" -ExpectedValue 1) {

```

```

    $vulnerable = $true
}

# Helper function to check MultiString value
function Test-MultiStringValue {
    param(
        [string]$Path,
        [string]$ValueName,
        [string[]]$ExpectedElements
    )
    $Val = Get-ItemProperty -Path $Path -Name $ValueName -ErrorAction SilentlyContinue
    if ($null -eq $Val) {
        Write-Host "    [!] VULNERABLE: $($ValueName) under $($Path) is not configured." -ForegroundColor Red
        return $true
    }
    $ActualList = $Val.$ValueName
    $Missing = @()
    foreach ($E in $ExpectedElements) {
        $Found = $false
        foreach ($A in $ActualList) {
            if ($A.Trim().ToLower() -eq $E.ToLower()) {
                $Found = $true
                break
            }
        }
        if (-not $Found) {
            $Missing += $E
        }
    }
    if ($Missing.Count -gt 0) {
        Write-Host "    [!] VULNERABLE: $($ValueName) is missing elements: $($Missing -join ', ')" -ForegroundColor Red
        return $true
    }
    Write-Host "    [+] $($ValueName) contains all required elements (Compliant)." -ForegroundColor Green
    return $false
}

# 7. Network access: Named Pipes that can be accessed anonymously
$RequiredPipes = @("netlogon", "samr", "lsarpc")
if (Test-MultiStringValue -Path "HKLM:\System\CurrentControlSet\Services\LanmanServer\Parameters" -ValueName "NullSessionPipes" -ExpectedElements $RequiredPipes) {
    $vulnerable = $true
}

# 8. Network access: Remotely accessible registry paths
$RequiredExactPaths = @(
    "System\CurrentControlSet\Control\ProductOptions",
    "System\CurrentControlSet\Control\Server Applications",
    "Software\Microsoft\Windows NT\CurrentVersion"
)
if (Test-MultiStringValue -Path "HKLM:\System\CurrentControlSet\Control\SecurePipeServers\winreg\AllowedExactPaths" -ValueName "Machine" -ExpectedElements $RequiredExactPaths) {
    $vulnerable = $true
}

# 9. Network access: Remotely accessible registry paths and sub-paths
$RequiredPaths = @(
    "System\CurrentControlSet\Control\Print\Printers",
    "System\CurrentControlSet\Services\Eventlog",
    "Software\Microsoft\OLAP Server",
    "Software\Microsoft\Windows NT\CurrentVersion\Print",
    "Software\Microsoft\Windows NT\CurrentVersion\Windows",
    "System\CurrentControlSet\Control\ContentIndex",
    "System\CurrentControlSet\Control\Terminal Server",
    "System\CurrentControlSet\Control\Terminal Server\UserConfig",
    "System\CurrentControlSet\Control\Terminal Server\DefaultUserConfiguration",
    "Software\Microsoft\Windows NT\CurrentVersion\Perflib",
    "System\CurrentControlSet\Services\SysmonLog"
)

# Optional AD CS or WINS sub-paths
$CertSvc = Get-Service -Name "CertSvc" -ErrorAction SilentlyContinue
if ($null -ne $CertSvc) {
    $RequiredPaths += "System\CurrentControlSet\Services\CertSvc"
}
$WinsSvc = Get-Service -Name "WINS" -ErrorAction SilentlyContinue
if ($null -ne $WinsSvc) {

```

```
$RequiredPaths += "System\CurrentControlSet\Services\WINS"
}

if (Test-MultiStringValue -Path "HKLM:\System\CurrentControlSet\Control\SecurePipeServers\winreg\AllowedPaths" -ValueName "Machine"
-ExpectedElements $RequiredPaths) {
    $vulnerable = $true
}

if ($vulnerable) {
    Write-Host "Audit result: VULNERABLE" -ForegroundColor Red
} else {
    Write-Host "Audit result: SECURE" -ForegroundColor Green
}
```

Sources & Compliance References * **CIS Benchmark**: CIS Microsoft Windows Server 2016 Benchmark v2.0.0 - Section 2.3.5.1, Section 2.3.5.2, Section 2.3.5.5, Section 2.3.6.4, Section 2.3.6.5, Section 2.3.6.6, Section 2.3.10.4, Section 2.3.10.6, Section 2.3.10.8, Section 2.3.10.9 *
ANSSI AD Hardening Guide: Section on Active Directory configuration and secure channels

[REQ-DC-026] Configure TCP/IP and Network Parameter Hardening for Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

```
## Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **TCP/IP Settings (MSS)**: * Path: 'Computer Configuration\Preferences\Windows Settings\Registry' * Registry Key: 'HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters' * 'KeepAliveTime' = '300000' (REG_DWORD) * 'PerformRouterDiscovery' = '0' (REG_DWORD) * 'TcpMaxDataRetransmissions' = '3' (REG_DWORD) * Registry Key: 'HKLM\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters' * 'TcpMaxDataRetransmissions' = '3' (REG_DWORD) * **IPv6 DNS Settings**: * GPO Path: 'Computer Configuration\Policies\Administrative Templates\Network\DNS Client' * 'Turn off default IPv6 DNS Servers' → Enabled * Registry Key: 'HKLM\SOFTWARE\Policies\Microsoft\Windows NT\DNSClient' * 'DisablePv6DefaultDnsServers' = '1' (REG_DWORD) * **Link-Layer Topology Discovery (LLTD)**: * GPO Path: 'Computer Configuration\Policies\Administrative Templates\Network\Link-Layer Topology Discovery' * 'Turn on Mapper I/O (LLTDIO) driver' → Disabled * 'Turn on Responder (RSPNDR) driver' → Disabled * Registry Key: 'HKLM\SOFTWARE\Policies\Microsoft\Windows\LLTD' * 'AllowLLTDIOOnDomain' = '0' (REG_DWORD) * 'AllowLLTDIOOnPublicNet' = '0' (REG_DWORD) * 'EnableLLTDIO' = '0' (REG_DWORD) * 'ProhibitLLTDIOOnPrivateNet' = '0' (REG_DWORD) * 'AllowRspndrOnDomain' = '0' (REG_DWORD) * 'AllowRspndrOnPublicNet' = '0' (REG_DWORD) * 'EnableRspndr' = '0' (REG_DWORD) * 'ProhibitRspndrOnPrivateNet' = '0' (REG_DWORD) * **Peer-to-Peer Networking**: * GPO Path: 'Computer Configuration\Policies\Administrative Templates\Network\Microsoft Peer-to-Peer Networking Services' * 'Turn off Microsoft Peer-to-Peer Networking Services' → Enabled * Registry Key: 'HKLM\SOFTWARE\Policies\Microsoft\Peernet' * 'Disabled' = '1' (REG_DWORD) * **Windows Connect Now (WCN)**: * GPO Path: 'Computer Configuration\Policies\Administrative Templates\Network\Windows Connect Now' * 'Configuration of wireless settings using Windows Connect Now' → Disabled * 'Prohibit access of the Windows Connect Now wizards' → Enabled * Registry Keys: * Path: 'HKLM\SOFTWARE\Policies\Microsoft\Windows\WCN\Registrars' * 'EnableRegistrars' = '0' (REG_DWORD) * 'DisableUPnPRegistrar' = '1' (REG_DWORD) * 'DisableInBand802DOT11Registrar' = '1' (REG_DWORD) * 'DisableFlashConfigRegistrar' = '1' (REG_DWORD) * 'DisableWPDRegistrar' = '1' (REG_DWORD) * Path: 'HKLM\SOFTWARE\Policies\Microsoft\Windows\WCN\UI' * 'DisableWcnUI' = '1' (REG_DWORD)
```

Rationale Securing low-level TCP/IP parameters and network-layer advertisement protocols on Domain Controllers is essential to block local discovery, routing redirection, and denial of service: 1. **MSS TCP/IP Tuning**: Restricting TCP data retransmissions ('TcpMaxDataRetransmissions') limits resources allocated to unacknowledged TCP segments during connection drops, mitigating denial-of-service attempts. Disabling Internet Router Discovery Protocol (IRDP) ('PerformRouterDiscovery') blocks gateway advertisement redirection or man-in-the-middle attacks where hosts are coerced into routing traffic through a rogue gateway. Adjusting 'KeepAliveTime' optimizes connection checks. 2. **LLTD Mapper and Responder**: Disabling Link-Layer Topology Discovery (LLTD) components ('LLTDIO' and 'RSPNDR') prevents local attackers from mapping Domain Controllers on physical subnetworks or running topological discovery queries. 3. **Microsoft Peer-to-Peer and Windows Connect Now**: Peer-to-Peer protocols (PNRP) and Windows Connect Now (WCN) wizards introduce unnecessary background communication channels, local discovery broadcasts, and wireless profile exposure vectors. These services must be completely disabled on Tier 0 assets to minimize attack surface. 4. **Disable Default IPv6 DNS Servers**: Prevents automated fallback to unauthenticated, dynamic local IPv6 DNS servers advertised by third-party routers, mitigating DNS redirection or spoofing vectors.

Legacy Impact & Compatibility * **Network Discovery**: Disabling LLTD Mapper and Responder will prevent the Domain Controller from appearing in the visual "Network Map" GUI of neighboring systems. This does not affect active directory replication, client authentication, DNS resolution, or administrative connections. * **Peer-to-Peer Applications**: Disabling Peernet blocks local administrative tools or collaboration applications that rely on Microsoft Peer-to-Peer services. These are not supported on Tier 0 servers. * **WCN Configuration**: Windows Connect Now is designed for wireless device configuration. Since Domain Controllers must run on dedicated, wired server backbones, disabling WCN has zero operational impact.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`).
2. Edit the GPO linked to the Domain Controllers OU (e.g., `GPO_Hardening_DomainControllers`).
3. Configure GPO templates:
 - Navigate to: `Computer Configuration\Policies\Administrative Templates\Network\DNS Client`
 - **Policy:** `Turn off default IPv6 DNS Servers` → **Enabled**
 - Navigate to: `Computer Configuration\Policies\Administrative Templates\Network\Link-Layer Topology Discovery`
 - **Policy:** `Turn on Mapper I/O (LLTDIO) driver` → **Disabled**
 - **Policy:** `Turn on Responder (RSPNDR) driver` → **Disabled**
 - Navigate to: `Computer Configuration\Policies\Administrative Templates\Network\Microsoft Peer-to-Peer Networking Services`
 - **Policy:** `Turn off Microsoft Peer-to-Peer Networking Services` → **Enabled**
 - Navigate to: `Computer Configuration\Policies\Administrative Templates\Network\Windows Connect Now`
 - **Policy:** `Configuration of wireless settings using Windows Connect Now` → **Disabled**
 - **Policy:** `Prohibit access of the Windows Connect Now wizards` → **Enabled**
4. Configure TCP/IP Registry Preferences:
 - Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`

- Create or update the following Registry Items (Right-click **Registry** → **New** → **Registry Item**):

- Hive: `HKEY_LOCAL_MACHINE` | Key Path: `SYSTEM\CurrentControlSet\Services\Tcpip\Parameters` | Value name: `KeepAliveTime` | Value type: `REG_DWORD` | Value data: `300000`
- Hive: `HKEY_LOCAL_MACHINE` | Key Path: `SYSTEM\CurrentControlSet\Services\Tcpip\Parameters` | Value name: `PerformRouterDiscovery` | Value type: `REG_DWORD` | Value data: `0`
- Hive: `HKEY_LOCAL_MACHINE` | Key Path: `SYSTEM\CurrentControlSet\Services\Tcpip\Parameters` | Value name: `TcpMaxDataRetransmissions` | Value type: `REG_DWORD` | Value data: `3`
- Hive: `HKEY_LOCAL_MACHINE` | Key Path: `SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters` | Value name: `TcpMaxDataRetransmissions` | Value type: `REG_DWORD` | Value data: `3`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to enforce network parameter hardening.

[Download Script: Configure-DCNetworkHardening.ps1](#)

```
# Configure-DCNetworkHardening.ps1
# Description: Configures TCP/IP parameters, LLTD, Peer-to-Peer, and WCN hardening on Domain Controllers.

Write-Host "Applying Network Parameter Hardening..." -ForegroundColor Cyan

# 1. TCP/IP Parameters (MSS)
$TcpipParamsPath = "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters"
if (-not (Test-Path $TcpipParamsPath)) {
    New-Item -Path $TcpipParamsPath -Force | Out-Null
}
Set-ItemProperty -Path $TcpipParamsPath -Name "KeepAliveTime" -Value 300000 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $TcpipParamsPath -Name "PerformRouterDiscovery" -Value 0 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $TcpipParamsPath -Name "TcpMaxDataRetransmissions" -Value 3 -Type DWord -ErrorAction Stop

$Tcpip6ParamsPath = "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters"
if (-not (Test-Path $Tcpip6ParamsPath)) {
    New-Item -Path $Tcpip6ParamsPath -Force | Out-Null
}
Set-ItemProperty -Path $Tcpip6ParamsPath -Name "TcpMaxDataRetransmissions" -Value 3 -Type DWord -ErrorAction Stop

# 2. IPv6 Default DNS Servers
$DnsClientPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\DNSClient"
if (-not (Test-Path $DnsClientPath)) {
    New-Item -Path $DnsClientPath -Force | Out-Null
}
Set-ItemProperty -Path $DnsClientPath -Name "DisableIPv6DefaultDnsServers" -Value 1 -Type DWord -ErrorAction Stop

# 3. LLTD Mapper and Responder
$LLtdPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\LLTD"
if (-not (Test-Path $LLtdPath)) {
    New-Item -Path $LLtdPath -Force | Out-Null
}
Set-ItemProperty -Path $LLtdPath -Name "AllowLLTDIOOnDomain" -Value 0 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $LLtdPath -Name "AllowLLTDIOOnPublicNet" -Value 0 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $LLtdPath -Name "EnableLLTDIO" -Value 0 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $LLtdPath -Name "ProhibitLLTDIOOnPrivateNet" -Value 0 -Type DWord -ErrorAction Stop

Set-ItemProperty -Path $LLtdPath -Name "AllowRspndrOnDomain" -Value 0 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $LLtdPath -Name "AllowRspndrOnPublicNet" -Value 0 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $LLtdPath -Name "EnableRspndr" -Value 0 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $LLtdPath -Name "ProhibitRspndrOnPrivateNet" -Value 0 -Type DWord -ErrorAction Stop

# 4. Peer-to-Peer Networking
$PeernetPath = "HKLM:\SOFTWARE\Policies\Microsoft\Peernet"
if (-not (Test-Path $PeernetPath)) {
    New-Item -Path $PeernetPath -Force | Out-Null
}
Set-ItemProperty -Path $PeernetPath -Name "Disabled" -Value 1 -Type DWord -ErrorAction Stop

# 5. Windows Connect Now (WCN)
$WcnRegsPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WCN\Registrars"
if (-not (Test-Path $WcnRegsPath)) {
    New-Item -Path $WcnRegsPath -Force | Out-Null
}
Set-ItemProperty -Path $WcnRegsPath -Name "EnableRegistrars" -Value 0 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $WcnRegsPath -Name "DisableUPnPRegistrar" -Value 1 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $WcnRegsPath -Name "DisableInBand802DOT11Registrar" -Value 1 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $WcnRegsPath -Name "DisableFlashConfigRegistrar" -Value 1 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $WcnRegsPath -Name "DisableWPDRegistrar" -Value 1 -Type DWord -ErrorAction Stop

$WcnUiPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WCN\UI"
if (-not (Test-Path $WcnUiPath)) {
    New-Item -Path $WcnUiPath -Force | Out-Null
}
Set-ItemProperty -Path $WcnUiPath -Name "DisableWcnUi" -Value 1 -Type DWord -ErrorAction Stop

Write-Host "Network parameters hardening completed successfully. A system restart is required for changes to take effect." -Foregrou
ndColor Green
```

To verify the setting has been applied:

[Download Script: Get-DCNetworkHardeningStatus.ps1](#)

```
# Get-DCNetworkHardeningStatus.ps1
# Description: Audits registry configuration of TCP/IP parameters, LLTD, Peer-to-Peer, and WCN on Domain Controllers.

Write-Host "--- Auditing Domain Controller Network Parameter Hardening ---" -ForegroundColor Cyan

# 1. Audit TCP/IP Parameters (MSS)
$TcpipParamsPath = "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters"
$TcpipExpected = @{
    "KeepAliveTime"           = 300000
    "PerformRouterDiscovery"   = 0
    "TcpMaxDataRetransmissions" = 3
}

if (Test-Path $TcpipParamsPath) {
    $TcpipReg = Get-ItemProperty -Path $TcpipParamsPath -ErrorAction SilentlyContinue
    foreach ($K in $TcpipExpected.Keys) {
        $Val = $TcpipReg.$K
        $Exp = $TcpipExpected[$K]
        $Color = if ($Val -eq $Exp) { "Green" } else { "Red" }
        Write-Host "    - TCP/IP $($K): $($Val) (Expected: $($Exp))" -ForegroundColor $Color
    }
} else {
    Write-Host "    - TCP/IP Parameters Registry Path: NOT FOUND" -ForegroundColor Red
}

$Tcpip6ParamsPath = "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters"
if (Test-Path $Tcpip6ParamsPath) {
    $Tcpip6Reg = Get-ItemProperty -Path $Tcpip6ParamsPath -ErrorAction SilentlyContinue
    $Val = $Tcpip6Reg.TcpMaxDataRetransmissions
    $Color = if ($Val -eq 3) { "Green" } else { "Red" }
    Write-Host "    - TCP/IP6 TcpMaxDataRetransmissions: $($Val) (Expected: 3)" -ForegroundColor $Color
} else {
    Write-Host "    - TCP/IP6 Parameters Registry Path: NOT FOUND" -ForegroundColor Red
}

# 2. Audit IPv6 Default DNS Servers
$DnsClientPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\DNSClient"
if (Test-Path $DnsClientPath) {
    $DnsClientReg = Get-ItemProperty -Path $DnsClientPath -ErrorAction SilentlyContinue
    $Val = $DnsClientReg.DisableIPv6DefaultDnsServers
    $Color = if ($Val -eq 1) { "Green" } else { "Red" }
    Write-Host "    - DNS Client DisableIPv6DefaultDnsServers: $($Val) (Expected: 1)" -ForegroundColor $Color
} else {
    Write-Host "    - DNS Client Registry Path: NOT FOUND" -ForegroundColor Red
}

# 3. Audit LLTD
$LLtdPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\LLTD"
$LLtdExpected = @{
    "AllowLLTDIOOnDomain"       = 0
    "AllowLLTDIOOnPublicNet"    = 0
    "EnableLLTDIO"              = 0
    "ProhibitLLTDIOOnPrivateNet" = 0
    "AllowRspndrOnDomain"       = 0
    "AllowRspndrOnPublicNet"    = 0
    "EnableRspndr"              = 0
    "ProhibitRspndrOnPrivateNet" = 0
}

if (Test-Path $LLtdPath) {
    $LLtdReg = Get-ItemProperty -Path $LLtdPath -ErrorAction SilentlyContinue
    foreach ($K in $LLtdExpected.Keys) {
        $Val = $LLtdReg.$K
        $Exp = $LLtdExpected[$K]
        $Color = if ($Val -eq $Exp) { "Green" } else { "Red" }
        Write-Host "    - LLTD $($K): $($Val) (Expected: $($Exp))" -ForegroundColor $Color
    }
} else {
    Write-Host "    - LLTD Registry Path: NOT FOUND" -ForegroundColor Red
}

# 4. Audit Peer-to-Peer
$PeernetPath = "HKLM:\SOFTWARE\Policies\Microsoft\Peernet"
if (Test-Path $PeernetPath) {
    $PeernetReg = Get-ItemProperty -Path $PeernetPath -ErrorAction SilentlyContinue

```

```

    $Val = $PeernetReg.Disabled
    $Color = if ($Val -eq 1) { "Green" } else { "Red" }
    Write-Host "        - Peernet Disabled: $($Val) (Expected: 1)" -ForegroundColor $Color
} else {
    Write-Host "        - Peernet Registry Path: NOT FOUND" -ForegroundColor Red
}

# 5. Audit Windows Connect Now (WCN)
$WcnRegsPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WCN\Registrars"
$WcnRegsExpected = @{
    "EnableRegistrars"           = 0
    "DisableUPnPRegistrar"       = 1
    "DisableInBand802DOT11Registrar" = 1
    "DisableFlashConfigRegistrar" = 1
    "DisableWPDRegistrar"        = 1
}
if (Test-Path $WcnRegsPath) {
    $WcnRegsReg = Get-ItemProperty -Path $WcnRegsPath -ErrorAction SilentlyContinue
    foreach ($K in $WcnRegsExpected.Keys) {
        $Val = $WcnRegsReg.$K
        $Exp = $WcnRegsExpected[$K]
        $Color = if ($Val -eq $Exp) { "Green" } else { "Red" }
        Write-Host "        - WCN Registrars $($K): $($Val) (Expected: $($Exp))" -ForegroundColor $Color
    }
} else {
    Write-Host "        - WCN Registrars Registry Path: NOT FOUND" -ForegroundColor Red
}

$WcnUiPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WCN\UI"
if (Test-Path $WcnUiPath) {
    $WcnUiReg = Get-ItemProperty -Path $WcnUiPath -ErrorAction SilentlyContinue
    $Val = $WcnUiReg.DisableWcnUi
    $Color = if ($Val -eq 1) { "Green" } else { "Red" }
    Write-Host "        - WCN UI DisableWcnUi: $($Val) (Expected: 1)" -ForegroundColor $Color
} else {
    Write-Host "        - WCN UI Registry Path: NOT FOUND" -ForegroundColor Red
}
}

```

Sources & Compliance References * **CIS Benchmark**:

CIS Microsoft Windows Server Benchmark - Section 18.5 (MSS Parameters), Section 18.6.4 (DNS Client), Section 18.6.9 (LLTD), Section 18.6.10 (Peer-to-Peer), Section 18.6.20 (WCN) * **ANSSI AD Hardening Guide**:

Security guidelines to disable unnecessary interfaces and protocols on Domain Controllers.

[REQ-DC-027] Configure Telemetry, Diagnostics and Privacy Options for Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Low * **GPO Path / Registry Location**: * **Allow Online Tips**: * GPO: 'Computer Configuration\Policies\Administrative Templates\Control Panel\Allow Online Tips' → Disabled * Registry: 'HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer' → 'AllowOnlineTips' = '0' (REG_DWORD) * **Enable Font Providers**: * GPO: 'Computer Configuration\Policies\Administrative Templates\Network\Fonts\Enable Font Providers' → Disabled * Registry: 'HKLM\SOFTWARE\Policies\Microsoft\Windows\System' → 'EnableFontProviders' = '0' (REG_DWORD) * **Turn off notifications network usage**: * GPO: 'Computer Configuration\Policies\Administrative Templates\Start Menu and Taskbar\Turn off notifications network usage' → Enabled * Registry: 'HKLM\SOFTWARE\Policies\Microsoft\Windows\CurrentVersion\PushNotifications' → 'NoCloudApplicationNotification' = '1' (REG_DWORD) * **Handwriting Personalization & Error Reporting**: * GPO: 'Computer Configuration\Policies\Administrative Templates\System\Internet Communication Management\Internet Communication settings' * 'Turn off handwriting personalization data sharing' → Enabled * 'Turn off handwriting recognition error reporting' → Enabled * Registry: * 'HKLM\SOFTWARE\Policies\Microsoft\Windows\TabletPC' → 'PreventHandwritingDataSharing' = '1' (REG_DWORD) * 'HKLM\SOFTWARE\Policies\Microsoft\Windows\HandwritingErrorReports' → 'PreventHandwritingErrorReports' = '1' (REG_DWORD) * **Internet Communication Settings**: * GPO: 'Computer Configuration\Policies\Administrative Templates\System\Internet Communication Management\Internet Communication settings' * 'Turn off printing over HTTP' → Enabled * 'Turn off Search Companion content file updates' -> Enabled * 'Turn off the "Order Prints" picture task' → Enabled * 'Turn off the "Publish to Web" task for files and folders' → Enabled * 'Turn off the Windows Messenger Customer Experience Improvement Program' → Enabled * 'Turn off Windows Error Reporting' → Enabled * Registry: * 'HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Printers' → 'DisableHTTPPrinting' = '1' (REG_DWORD) * 'HKLM\SOFTWARE\Policies\Microsoft\SearchCompanion' → 'DisableContentFileUpdates' = '1' (REG_DWORD) * 'HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer' → 'NoOnlinePrintsWizard' = '1' (REG_DWORD) * 'HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer' → 'NoPublishingWizard' = '1' (REG_DWORD) * 'HKLM\SOFTWARE\Policies\Microsoft\Messenger\Client' → 'CEIP' = '2' (REG_DWORD) * 'HKLM\SOFTWARE\Policies\Microsoft\SQMClient\Windows' → 'CEIPEnable' = '0' (REG_DWORD) * 'HKLM\SOFTWARE\Policies\Microsoft\Windows\Windows Error Reporting' → 'Disabled' = '1' (REG_DWORD) * 'HKLM\SOFTWARE\Policies\Microsoft\PCHealth\ErrorReporting' → 'DoReport' = '0' (REG_DWORD) * **Microsoft Support Diagnostic Tool (MSDT)**: * GPO: 'Computer Configuration\Policies\Administrative Templates\System\Troubleshooting and Diagnostics\Microsoft Support Diagnostic Tool\Microsoft Support Diagnostic Tool: Turn on MSDT interactive communication with support provider' → Disabled * Registry: 'HKLM\SOFTWARE\Policies\Microsoft\Windows\ScriptedDiagnosticsProvider\Policy' → 'DisableQueryRemoteServer' = '0' (REG_DWORD) * **Turn off the advertising ID**: * GPO: 'Computer Configuration\Policies\Administrative Templates\System\User Profiles\Turn off the advertising ID' → Enabled * Registry: 'HKLM\SOFTWARE\Policies\Microsoft\Windows\AdvertisingInfo' → 'DisabledByGroupPolicy' = '1' (REG_DWORD) * **Allow app data sharing / Camera**: * GPO: * 'Computer Configuration\Policies\Administrative Templates\Windows Components\App Package Deployment\Allow a Windows app to share application data between users' → Disabled * 'Computer Configuration\Policies\Administrative Templates\Windows Components\Camera\Allow Use of Camera' → Disabled * Registry: * 'HKLM\SOFTWARE\Policies\Microsoft\Windows\CurrentVersion\AppModel\StateManager' → 'AllowSharedLocalAppData' = '0' (REG_DWORD) * 'HKLM\SOFTWARE\Policies\Microsoft\Camera' → 'AllowCamera' = '0' (REG_DWORD) * **Connected User Experience and Telemetry / Location / Messaging**: * GPO: * 'Computer Configuration\Policies\Administrative Templates\Windows Components\Data Collection and Preview Builds\Configure Authenticated Proxy usage for the Connected User Experience and Telemetry service' → Enabled: Disable Authenticated Proxy usage * 'Computer Configuration\Policies\Administrative Templates\Windows Components\Location and Sensors\Turn off location' → Enabled * 'Computer Configuration\Policies\Administrative Templates\Windows Components\Messaging\Allow Message Service Cloud Sync' → Disabled * Registry: * 'HKLM\SOFTWARE\Policies\Microsoft\Windows\DataCollection' → 'DisableEnterpriseAuthProxy' = '1' (REG_DWORD) * 'HKLM\SOFTWARE\Policies\Microsoft\Windows\LocationAndSensors' → 'DisableLocation' = '1' (REG_DWORD) * 'HKLM\SOFTWARE\Policies\Microsoft\Windows\Messaging' → 'AllowMessageSync' = '0' (REG_DWORD) * **Push to Install / Search Cloud / KMS Online / Ink Workspace**: * GPO: * 'Computer Configuration\Policies\Administrative Templates\Windows Components\Push to Install\Turn off Push To Install service' → Enabled * 'Computer Configuration\Policies\Administrative Templates\Windows Components\Search\Allow Cloud Search' → Enabled: Disable Cloud Search * 'Computer Configuration\Policies\Administrative Templates\Windows Components\Search\Allow search highlights' → Disabled * 'Computer Configuration\Policies\Administrative Templates\Windows Components\Software Protection Platform\Turn off KMS Client Online AVS Validation' → Enabled * 'Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Ink Workspace\Allow suggested apps in Windows Ink Workspace' → Disabled * Registry: * 'HKLM\SOFTWARE\Policies\Microsoft\PushToInstall' → 'DisablePushToInstall' = '1' (REG_DWORD) * 'HKLM\SOFTWARE\Policies\Microsoft\Windows\Windows Search' → 'AllowCloudSearch' = '0' (REG_DWORD) * 'HKLM\SOFTWARE\Policies\Microsoft\Windows\Windows Search' → 'EnableDynamicContentInWSB' = '0' (REG_DWORD) * 'HKLM\SOFTWARE\Policies\Microsoft\Windows NT\CurrentVersion\Software Protection Platform' → 'NoGenTicket' = '1' (REG_DWORD) * 'HKLM\SOFTWARE\Policies\Microsoft\WindowsInkWorkspace' → 'AllowSuggestedAppsInWindowsInkWorkspace' = '0' (REG_DWORD) * **User Configuration (Privacy & Feedback)**: * GPO (User Configuration): * 'User Configuration\Policies\Administrative Templates\System\Internet Communication Management\Internet Communication Settings\Turn off Help Experience Improvement Program' → Enabled * 'User Configuration\Policies\Administrative Templates\Windows Components\Cloud Content\Do not use diagnostic data for tailored experiences' → Enabled * 'User Configuration\Policies\Administrative Templates\Windows Components\Cloud Content\Turn off all Windows spotlight features' → Enabled * 'User Configuration\Policies\Administrative Templates\Windows Components\Windows Media Player\Playback\Prevent Codec Download' → Enabled * Registry: * 'HKCU\Software\Policies\Microsoft\Assistance\Client\1.0' → 'NoImplicitFeedback' = '1' (REG_DWORD) * 'HKCU\Software\Policies\Microsoft\Windows\CloudContent' →


```
'DisableTailoredExperiencesWithDiagnosticData' = '1' (REG_DWORD) * 'HKCU\Software\Policies\Microsoft\Windows\CloudContent' →
'DisableWindowsSpotlightFeatures' = '1' (REG_DWORD) * 'HKCU\Software\Policies\Microsoft\WindowsMediaPlayer' →
'PreventCodecDownload' = '1' (REG_DWORD)
```

Rationale Restricting telemetry, remote help channels, data sharing features, and diagnostic logs on Domain Controllers limits target exposure and data leakage:

- 1. **Minimize Telemetry and Personalization Leakage**:** Domain Controllers process highly sensitive security directory events, administrative tasks, and structural secrets. Personalization telemetry (such as handwriting sharing, speech data, and Customer Experience Improvement Programs) sends host telemetry to external cloud endpoints.
- 2. **Disable Non-Essential Network Services**:** Features like Online Help, printing over HTTP, and Windows Spotlight create unauthenticated outbound connections to cloud platforms.
- 3. **Restrict Diagnostic Tools**:** Interactive diagnostic channels like the Microsoft Support Diagnostic Tool (MSDT) have been targeted in remote execution exploits (e.g., Follina). Restricting interactive troubleshooting tools prevents adversaries from utilizing diagnostic capabilities for code execution.
- 4. **Prevent Cloud Synchronization**:** Message cloud synchronization, push-to-install services, and cloud-based search highlights run background processes that expose local queries and operations to external networks.

Legacy Impact & Compatibility

- **Interactive Troubleshooting**:** Automated help diagnostics or Microsoft Support online troubleshooting utilities will be blocked. Administrators must identify issues manually using local event viewer logs and offline diagnostics.
- **Search Highlights and Cloud Search**:** Local Start menu searches will only index local server content and folders, ignoring web or cloud highlights, which is the preferred behavior for hardened server nodes.
- **KMS Online Validation**:** The server will not attempt online KMS validation checks via Active Validation Service (AVS) client prompts. Activation should be managed using local Active Directory-Based Activation (ADBA) or KMS host infrastructure.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Configure Group Policy settings to disable telemetry and cloud services:

1. Open the **Group Policy Management Console** (`gpmc.msc`).
2. Edit the appropriate GPO targeting the Domain Controllers (e.g., `GPO_Hardening_DomainControllers`).
3. Configure the following settings under **Computer Configuration**:
 - `Control Panel\Allow Online Tips` → **Disabled**
 - `Network\Fonts\Enable Font Providers` → **Disabled**
 - `Start Menu and Taskbar\Turn off notifications network usage` → **Enabled**
 - `System\Internet Communication Management\Internet Communication settings`:
 - `Turn off handwriting personalization data sharing` → **Enabled**
 - `Turn off handwriting recognition error reporting` → **Enabled**
 - `Turn off printing over HTTP` → **Enabled**
 - `Turn off Search Companion content file updates` → **Enabled**
 - `Turn off the "Order Prints" picture task` → **Enabled**
 - `Turn off the "Publish to Web" task for files and folders` → **Enabled**
 - `Turn off the Windows Messenger Customer Experience Improvement Program` → **Enabled**
 - `Turn off Windows Customer Experience Improvement Program` → **Enabled**
 - `Turn off Windows Error Reporting` → **Enabled**
 - `System\Troubleshooting and Diagnostics\Microsoft Support Diagnostic Tool\Microsoft Support Diagnostic Tool: Turn on MSDT interactive communication with support provider` → **Disabled**
 - `System\User Profiles\Turn off the advertising ID` → **Enabled**
 - `Windows Components\App Package Deployment\Allow a Windows app to share application data between users` → **Disabled**
 - `Windows Components\Camera\Allow Use of Camera` → **Disabled**
 - `Windows Components\Data Collection and Preview Builds\Configure Authenticated Proxy usage for the Connected User Experience and Telemetry service` → **Enabled: Disable Authenticated Proxy usage**
 - `Windows Components\Location and Sensors\Turn off location` → **Enabled**
 - `Windows Components\Messaging\Allow Message Service Cloud Sync` → **Disabled**
 - `Windows Components\Push to Install\Turn off Push To Install service` → **Enabled**
 - `Windows Components\Search\Allow Cloud Search` → **Enabled: Disable Cloud Search**
 - `Windows Components\Search\Allow search highlights` → **Disabled**
 - `Windows Components\Software Protection Platform\Turn off KMS Client Online AVS Validation` → **Enabled**
 - `Windows Components\Windows Ink Workspace\Allow suggested apps in Windows Ink Workspace` → **Disabled**

4. Configure the following settings under **User Configuration** (or configure via GPO Preferences Registry if Loopback Processing is not active):

- System\Internet Communication Management\Internet Communication Settings\Turn off Help Experience Improvement Program → **Enabled**
- Windows Components\Cloud Content\Do not use diagnostic data for tailored experiences → **Enabled**
- Windows Components\Cloud Content\Turn off all Windows spotlight features → **Enabled**
- Windows Components\Windows Media Player\Playback\Prevent Codec Download → **Enabled**

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to enforce telemetry and privacy registry hardening.

[Download Script: Configure-DCTelemetryPrivacy.ps1](#)


```
# Configure-DCTelemetryPrivacy.ps1
# Description: Configures telemetry, diagnostic, and privacy options for Domain Controllers.

Write-Host "Applying Telemetry and Privacy baseline settings..." -ForegroundColor Cyan

# Helper function to create keys and set values safely
function Set-RegDWord {
    [CmdletBinding(SupportsShouldProcess)]
    param (
        [string]$path,
        [string]$name,
        [int]$value
    )
    if ($PSCmdlet.ShouldProcess($path, "Set registry DWORD value $name to $value")) {
        $parent = Split-Path -Path $path
        if (-not (Test-Path $parent)) {
            New-Item -Path $parent -Force | Out-Null
        }
        if (-not (Test-Path $path)) {
            New-Item -Path $path -Force | Out-Null
        }
        Set-ItemProperty -Path $path -Name $name -Value $value -Type DWord -Force
    }
}

# 1. HKLM Policy Configurations
Set-RegDWord "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" "AllowOnlineTips" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" "EnableFontProviders" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CurrentVersion\PushNotifications" "NoCloudApplicationNotification" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\TabletPC" "PreventHandwritingDataSharing" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\HandwritingErrorReports" "PreventHandwritingErrorReports" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers" "DisableHTTPPrinting" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\SearchCompanion" "DisableContentFileUpdates" 1
Set-RegDWord "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" "NoOnlinePrintsWizard" 1
Set-RegDWord "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" "NoPublishingWizard" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Messenger\Client" "CEIP" 2
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\SQMClient\Windows" "CEIPEnable" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Error Reporting" "Disabled" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\PCHealth\ErrorReporting" "DoReport" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\ScriptedDiagnosticsProvider\Policy" "DisableQueryRemoteServer" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AdvertisingInfo" "DisabledByGroupPolicy" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CurrentVersion\AppModel\StateManager" "AllowSharedLocalAppData" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Camera" "AllowCamera" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" "DisableEnterpriseAuthProxy" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\LocationAndSensors" "DisableLocation" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Messaging" "AllowMessageSync" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\PushToInstall" "DisablePushToInstall" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" "AllowCloudSearch" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" "EnableDynamicContentInWSB" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\CurrentVersion\Software Protection Platform" "NoGenTicket" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\WindowsInkWorkspace" "AllowSuggestedAppsInWindowsInkWorkspace" 0

Write-Host "HKLM telemetry parameters configured." -ForegroundColor Green

# 2. Configure Current User Settings (HKCU)
$AssistancePath = "HKCU:\Software\Policies\Microsoft\Assistance\Client\1.0"
Set-RegDWord $AssistancePath "NoImplicitFeedback" 1

$CloudContentPath = "HKCU:\Software\Policies\Microsoft\Windows\CloudContent"
Set-RegDWord $CloudContentPath "DisableTailoredExperiencesWithDiagnosticData" 1
Set-RegDWord $CloudContentPath "DisableWindowsSpotlightFeatures" 1

$MediaPlayerPath = "HKCU:\Software\Policies\Microsoft\WindowsMediaPlayer"
Set-RegDWord $MediaPlayerPath "PreventCodecDownload" 1

Write-Host "HKCU telemetry parameters configured." -ForegroundColor Green

# 3. Configure Default User Hive Settings (HKU\DefaultUser)
$DefaultUserHive = "C:\Users\Default\NTUSER.DAT"
if (Test-Path $DefaultUserHive) {
    Write-Host "Loading Default User hive..." -ForegroundColor Gray
    reg load HKU\DefaultUser $DefaultUserHive | Out-Null

    Set-RegDWord "Registry::HKU\DefaultUser\Software\Policies\Microsoft\Assistance\Client\1.0" "NoImplicitFeedback" 1
}
```

```

Set-RegDWord "Registry::HKU\DefaultUser\Software\Policies\Microsoft\Windows\CloudContent" "DisableTailoredExperiencesWithDiagnosticData" 1
Set-RegDWord "Registry::HKU\DefaultUser\Software\Policies\Microsoft\Windows\CloudContent" "DisableWindowsSpotLightFeatures" 1
Set-RegDWord "Registry::HKU\DefaultUser\Software\Policies\Microsoft\WindowsMediaPlayer" "PreventCodecDownload" 1

[GC]::Collect()
[GC]::WaitForPendingFinalizers()
reg unload HKU\DefaultUser | Out-Null
Write-Host "Default User hive configurations applied." -ForegroundColor Green
} else {
    Write-Warning "Default User hive not found."
}

Write-Host "Telemetry and privacy settings applied successfully." -ForegroundColor Green

```

To verify the settings have been applied:

[Download Script: Get-DCTelemetryPrivacyStatus.ps1](#)

```
# Get-DCTelemetryPrivacyStatus.ps1
# Description: Audits registry configuration of telemetry, diagnostic, and privacy settings on Domain Controllers.

Write-Host "--- Auditing Domain Controller Telemetry and Privacy Settings ---" -ForegroundColor Cyan

$script:Vulnerable = $false

# Helper function to audit registry properties
function Test-RegistryValue ($path, $name, $expectedValue) {
    $val = Get-ItemProperty -Path $path -Name $name -ErrorAction SilentlyContinue
    $actual = if ($val) { $val.$name } else { "" }
    $color = "Red"
    if ($actual -eq $expectedValue) {
        $color = "Green"
    } else {
        $script:Vulnerable = $true
    }
    Write-Host "    - Registry Setting: $($name) | Actual: '$($actual)' (Expected: '$($expectedValue)')" -ForegroundColor $color
}

# 1. HKLM Audits
Write-Host "Auditing HKLM Settings..." -ForegroundColor Gray
Test-RegistryValue "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" "AllowOnlineTips" 0
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" "EnableFontProviders" 0
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CurrentVersion\PushNotifications" "NoCloudApplicationNotification" 1
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\TabletPC" "PreventHandwritingDataSharing" 1
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\HandwritingErrorReports" "PreventHandwritingErrorReports" 1
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers" "DisableHTTPPrinting" 1
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\SearchCompanion" "DisableContentFileUpdates" 1
Test-RegistryValue "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" "NoOnlinePrintsWizard" 1
Test-RegistryValue "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" "NoPublishingWizard" 1
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Messenger\Client" "CEIP" 2
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\SQMClient\Windows" "CEIPEnable" 0
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Error Reporting" "Disabled" 1
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\PCHealth\ErrorReporting" "DoReport" 0
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\ScriptedDiagnosticsProvider\Policy" "DisableQueryRemoteServer" 0
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AdvertisingInfo" "DisabledByGroupPolicy" 1
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CurrentVersion\AppModel\StateManager" "AllowSharedLocalAppData" 0
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Camera" "AllowCamera" 0
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" "DisableEnterpriseAuthProxy" 1
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\LocationAndSensors" "DisableLocation" 1
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Messaging" "AllowMessageSync" 0
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\PushToInstall" "DisablePushToInstall" 1
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" "AllowCloudSearch" 0
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" "EnableDynamicContentInWSB" 0
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\CurrentVersion\Software Protection Platform" "NoGenTicket" 1
Test-RegistryValue "HKLM:\SOFTWARE\Policies\Microsoft\WindowsInkWorkspace" "AllowSuggestedAppsInWindowsInkWorkspace" 0

# 2. HKCU Audits
Write-Host "Auditing HKCU Settings..." -ForegroundColor Gray
Test-RegistryValue "HKCU:\Software\Policies\Microsoft\Assistance\Client\1.0" "NoImplicitFeedback" 1
Test-RegistryValue "HKCU:\Software\Policies\Microsoft\Windows\CloudContent" "DisableTailoredExperiencesWithDiagnosticData" 1
Test-RegistryValue "HKCU:\Software\Policies\Microsoft\Windows\CloudContent" "DisableWindowsSpotlightFeatures" 1
Test-RegistryValue "HKCU:\Software\Policies\Microsoft\WindowsMediaPlayer" "PreventCodecDownload" 1

if ($script:Vulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
}
```

Sources & Compliance References * **CIS Benchmark**: CIS Microsoft Windows Server Benchmark - Section 18.1.3 (Online Tips), Section 18.6.5 (Fonts), Section 18.8 (Push Notifications), Section 18.9.20 (Internet Communication Management), Section 18.9.47 (MSDT), Section 18.9.49 (Advertising ID), Section 18.10.4 (App Data Sharing), Section 18.10.11 (Camera), Section 18.10.16 (Telemetry Proxy), Section 18.10.37 (Location), Section 18.10.41 (Messaging Sync), Section 18.10.56 (Push to Install), Section 18.10.59 (Cloud Search), Section 18.10.63 (KMS Online AVS), Section 18.10.80 (Ink Workspace), Section 19.6.6 (Help Experience), Section 19.7.8 (Spotlight & Tailored Experiences), Section 19.7.46 (Codec Downloads) * **ANSSI AD Hardening Guide**: Technical security baseline recommendations to restrict diagnostic tools, telemetry collection, and background service exposure.

[REQ-DC-028] Configure Untrusted Font Blocking for Domain Controllers

Target Scope * **Applicable Systems**: Domain Controllers and Member Servers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Administrative Templates\System\Mitigation Options\Untrusted Font Blocking` * **Registry Location**: `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\MitigationOptions` * **Value Name**: `MitigationOptions\_FontBlocking` * **Value Type**: `REG\_SZ` * **Value Data**: `1000000000000`

Rationale Font files (TrueType, OpenType, and others) are highly complex formats that require advanced parsing logic. Historically, font parsing in Windows was performed by the Graphics Device Interface (GDI) within the operating system kernel. Vulnerabilities in the kernel-mode font parser (such as buffer overflows or remote code execution) have been frequently exploited by threat actors to execute arbitrary code with kernel-level privileges.

Enabling Untrusted Font Blocking limits the attack surface of the graphics subsystem on Domain Controllers:

1. **Kernel Attack Surface Reduction:** Restricting the system to only load trusted fonts installed in the `%windir%\Fonts` system directory prevents the processing of malicious, web-delivered, or embedded font files.
2. **Mitigation of Document-Based Exploits:** Prevents malicious font files embedded in Microsoft Office documents, PDFs, or web pages from triggering parsing vulnerabilities in the context of administrative sessions on the Domain Controller.

Legacy Impact & Compatibility * **Web Browsers & Web Fonts**: Web browsers and applications running on Domain Controllers/Member Servers that dynamically download custom fonts may fail to render those fonts, reverting to system default fallback fonts. However, web browsing should not be conducted on Domain Controllers as a baseline practice. * **Embedded Fonts in Documents**: Document viewers utilizing custom embedded fonts that are not locally installed in the system directory will render using default system fonts, potentially affecting layout alignment. * **Deployment Validation**: It is recommended to deploy this setting in Audit Mode (`3000000000000`) initially to monitor event log entries (Event ID 305 inside the `Microsoft-Windows-Security-Mitigations/KernelMode` log) before fully enforcing the block policy.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain controller or management host.
2. Create a new GPO or edit an existing one (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Mitigation Options`
4. Configure the following setting:
 - **Policy:** `Untrusted Font Blocking`
 - **Setting:** `Enabled`
 - **Mitigation Options:** `Block untrusted fonts and log events`
5. Link the GPO to the Domain Controllers Organizational Unit (OU) containing the target servers.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally on standalone systems or during reference image build phases.

[Download Script: Configure-UntrustedFontBlocking.ps1](#)

```
# Configure-UntrustedFontBlocking.ps1
# Description: Configures Untrusted Font Blocking mitigation to block untrusted fonts and log events on Domain Controllers.

$RegPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\MitigationOptions"
$ValueName = "MitigationOptions_FontBlocking"
$ValueData = "1000000000000"

Write-Host "Applying hardening requirement: Configure Untrusted Font Blocking..." -ForegroundColor Cyan

if (-not (Test-Path $RegPath)) {
    New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value $ValueData -Type String -Force | Out-Null
Write-Host "Hardening applied successfully." -ForegroundColor Green
```

To verify the setting has been applied:

Download Script: [Get-UntrustedFontBlockingStatus.ps1](#)

```
# Get-UntrustedFontBlockingStatus.ps1
# Description: Checks the current configuration state of Untrusted Font Blocking registry setting on Domain Controllers.

$RegPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\MitigationOptions"
$ValueName = "MitigationOptions_FontBlocking"
$ExpectedValue = "10000000000000"

Write-Host "Auditing hardening requirement: Configure Untrusted Font Blocking..." -ForegroundColor Cyan

if (Test-Path $RegPath) {
    $value = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
    if ($null -ne $value -and $value.$ValueName -eq $ExpectedValue) {
        Write-Host "Audit Result: Compliant. Untrusted fonts are blocked and logged ($($ValueName) = $($ExpectedValue))." -Foregroun
dColor Green
        exit 0
    }
}

Write-Host "Audit Result: Non-Compliant. Untrusted fonts are not configured to block and log." -ForegroundColor Red
exit 1
```

Sources & Compliance References * **Microsoft Learn**: Block untrusted fonts in an enterprise (MitigationOptions registry configurations)
 * **CIS Benchmark**: CIS Windows Server Benchmark (Section 18.9.30.1 - Mitigation Options)

[REQ-DC-029] Configure svchost.exe Mitigation Options

Target Scope * **Applicable Systems**: Domain Controllers and Member Servers. * **Operating Systems**: Windows Server 2022 (and above), Windows Server Semi-Annual Channel (1903 and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Administrative Templates\System\Service Control Manager Settings\Security Settings\Enable svchost.exe mitigation options` * **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Control\SCMConfig` * **Value Name**: `EnableSvchostMitigationPolicy` * **Value Type**: `REG\_DWORD` * **Value Data**: `1`

Rationale The Service Host (`svchost.exe`) process is a critical system component responsible for hosting multiple Windows services. Because `svchost.exe` runs with elevated privileges (such as `SYSTEM`, `Network Service`, or `Local Service`) and handles sensitive system tasks, it is a prime target for security evasion techniques, process hollowing, and DLL injection.

Enabling `svchost.exe` mitigation options restricts the behavior of the `svchost.exe` process to enhance security:

1. **Microsoft-Only Binary Enforcement:** Requires all binaries and dynamic-link libraries (DLLs) loaded into `svchost.exe` to be digitally signed by Microsoft. This prevents attackers from injecting custom, unsigned malicious DLLs into `svchost.exe` instances.
2. **Dynamic Code Blocking:** Prevents the execution of dynamically generated code (such as Just-In-Time compiled code) within `svchost.exe` processes, neutralizing typical in-memory exploitation vectors.

Legacy Impact & Compatibility * **Third-Party Compatibility**: This policy requires all binaries loaded by `svchost.exe` to be Microsoft-signed. Any third-party software, security agents, or system drivers that attempt to run services inside the `svchost.exe` process space using non-Microsoft DLLs will fail to load. This has historically caused issues with legacy antivirus, third-party authentication plugins, or specialized management utilities. * **Operating System Support**: This policy has no effect on Windows Server versions prior to 1903 (such as Windows Server 2016 and Windows Server 2019). * **Deployment Validation**: It is recommended to perform extensive baseline testing on a representative subset of member servers running third-party software before deploying this configuration across the entire production domain.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain controller or management host.
2. Create a new GPO or edit an existing one (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Service Control Manager Settings\Security Settings`
4. Configure the following setting:
 - **Policy:** `Enable svchost.exe mitigation options`
 - **Setting:** `Enabled`
5. Link the GPO to the Domain Controllers and Member Servers Organizational Units (OUs) containing the target systems.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally on standalone systems or during reference image build phases.

Download Script: [Configure-SvchostMitigation.ps1](#)

```
# Configure-SvchostMitigation.ps1
# Description: Configures svchost.exe mitigation options to enforce Microsoft-signed binaries and block dynamic code.

$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\SCMConfig"
$ValueName = "EnableSvchostMitigationPolicy"
$ValueData = 1

Write-Host "Applying hardening requirement: Configure svchost.exe mitigation options..." -ForegroundColor Cyan

if (-not (Test-Path $RegPath)) {
    New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value $ValueData -Type DWord -Force | Out-Null
Write-Host "Hardening applied successfully." -ForegroundColor Green
```

To verify the setting has been applied:

Download Script: [Get-SvchostMitigationStatus.ps1](#)

```
# Get-SvchostMitigationStatus.ps1
# Description: Audits the configuration state of svchost.exe mitigation options.

$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\SCM\Config"
$ValueName = "EnableSvchostMitigationPolicy"
$ExpectedValue = 1

Write-Host "Auditing hardening requirement: Configure svchost.exe mitigation options..." -ForegroundColor Cyan

if (Test-Path $RegPath) {
    $value = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
    if ($null -ne $value -and $value.$ValueName -eq $ExpectedValue) {
        Write-Host "Audit Result: Compliant. svchost.exe mitigation options are enabled." -ForegroundColor Green
        exit 0
    }
}

Write-Host "Audit Result: Non-Compliant. svchost.exe mitigation options are disabled or not configured." -ForegroundColor Red
exit 1
```

Sources & Compliance References * **Microsoft Learn**: Group Policy settings reference - Service Control Manager Settings *
Microsoft Security Guidance: Removing "Enable svchost.exe mitigation options" from baseline recommendations (for compatibility awareness)

[REQ-DC-030] Secure Directory Services Restore Mode (DSRM) and Recovery Parameters

Target Scope * **Applicable Systems**: Domain Controllers (Tier 0). * **Operating Systems**: Windows Server 2016, 2019, 2022, 2025.

Implementation Details * **Priority**: High (Reduces offline DC attack surface). * **GPO Path / Registry Location**: * Registry Path: `HKLM\System\CurrentControlSet\Control\Lsa` * Value Name: `DsrAdminLogonBehavior` * Value Type: `REG\_DWORD` * Value Data: `1` (Allows DSRM Admin logon only when booted in DSRM).

Rationale Directory Services Restore Mode (DSRM) is a special boot mode for Domain Controllers that allows administrators to repair or restore the Active Directory database (NTDS.dit). DSRM uses a local Administrator account separate from the AD directory.

If DSRM is not secured:

1. **Network Authentication Abuse:** By default, or if misconfigured (e.g. `DsrAdminLogonBehavior` set to `0` or `2`), the local DSRM administrator account can authenticate over the network to the Domain Controller. Since this account has a static password (often never changed since DC promotion), it can be targeted for brute-forcing, pass-the-hash, or DCSync credential retrieval.
2. **Offline Recovery Attacks:** If the DC is booted in Safe Mode/DSRM, local controls are reduced.

Setting `DsrAdminLogonBehavior` to `1` ensures the DSRM Administrator account can only log on locally, and only when the DC is booted into DSRM mode. Setting it to `2` allows logon when the AD service is stopped, which is also a risk.

Legacy Impact & Compatibility * **Offline Recovery Access**: To use DSRM for database maintenance, administrators must have physical console access (or out-of-band console access like iDRAC/iLO/Hyper-V console) since remote RDP or network logons will be rejected. * **Credential Synchronization**: The DSRM password must be rotated periodically and synchronized with a highly secure Domain Admin credential.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host.
2. Create or edit a GPO linked to the **Domain Controllers** OU (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a new **Registry Item** with the following properties:
 - **Action:** `Update`
 - **Hive:** `HKEY_LOCAL_MACHINE`
 - **Key Path:** `System\CurrentControlSet\Control\Lsa`
 - **Value Name:** `DsrAdminLogonBehavior`
 - **Value Type:** `REG_DWORD`
 - **Value Data:** `00000001` (Hexadecimal)
5. Deploy and link the GPO to enforce the registry setting domain-wide.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

To automate verification and local remediation of the DSRM configuration:

[Download Script: Set-DsrHardening.ps1](#)


```
# Set-DsrmHardening.ps1
# Description: Configures DsrmAdminLogonBehavior to restrict network logons.

$RegPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
$ValueName = "DsrmAdminLogonBehavior"
$ValueData = 1 # Restrict network logons

Write-Host "Applying hardening: Restricting DSRM Admin Logon Behavior..." -ForegroundColor Cyan

if (-not (Test-Path $RegPath)) {
    New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value $ValueData -Type DWord
Write-Host "[+] Registry parameter set successfully: $ValueName = $ValueData" -ForegroundColor Green

# Instructions for DSRM password sync
Write-Host "`n[NOTE] Ensure that you synchronize the DSRM Administrator password with the Domain Administrator account." -ForegroundColor Yellow
Write-Host "Run the following command to sync passwords:" -ForegroundColor Yellow
Write-Host " ntdsutil `set dsrm password` `sync from domain account administrator` q q" -ForegroundColor Yellow
```

To verify the DSRM configuration status:

[Download Script: Get-DsrmHardeningStatus.ps1](#)

```
# Get-DsrmHardeningStatus.ps1
# Check if DsrmAdminLogonBehavior registry parameter is set to 1.

$RegPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
$ValueName = "DsrmAdminLogonBehavior"

if (-not (Test-Path $RegPath)) {
    Write-Host "[!] NON-COMPLIANT: Registry key '$RegPath' does not exist." -ForegroundColor Red
    exit 1
}

$Value = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue

if ($null -eq $Value -or $Value.$ValueName -ne 1) {
    Write-Host "[!] NON-COMPLIANT: DSRM network logon is not restricted (DsrmAdminLogonBehavior is not 1)." -ForegroundColor Red
    exit 1
} else {
    Write-Host "[+] COMPLIANT: DSRM network logon is restricted (DsrmAdminLogonBehavior = 1)." -ForegroundColor Green
    exit 0
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**: Recommendation R14 (Harden directory services restore mode (DSRM)) * **CIS Benchmark**: Section 2.3.1.2 (LSA Security Settings) * **PingCastle Rule**: 'P-RecoveryModeUnprotected' (Ensure the \"automatic administrative logon\" feature of the recovery mode is not enabled)

[REQ-DC-031] Configure NTP Time Synchronization on the PDC Emulator

Target Scope * **Applicable Systems**: Domain Controllers (specifically the PDC Emulator FSMO role owner) * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows Server 2025

Implementation Details * **Priority**: High (Essential for secure Kerberos authentication and time synchronization) * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Administrative Templates\System\Windows Time Service` * **Registry Locations**: * `HKLM\System\CurrentControlSet\Services\W32Time\Parameters` → `Type` = `"NTP"` * `HKLM\System\CurrentControlSet\Services\W32Time\Config` → `AnnounceFlags` = `5` (REG_DWORD) * `HKLM\System\CurrentControlSet\Services\W32Time\Parameters` → `NtpServer` = `"[NtpServerAddress],0x8"` (REG_SZ)

Rationale Active Directory relies heavily on Kerberos authentication, which has a default maximum clock skew limit of 5 minutes to prevent replay attacks. If the system clocks of Domain Controllers and member endpoints drift, authentication will fail, causing directory service outages.

The Domain Controller holding the Primary Domain Controller (PDC) Emulator FSMO role acts as the root time source for the entire Active Directory forest. All other Domain Controllers synchronize their time from the PDC Emulator, and member servers and workstations synchronize their time from their local authenticating Domain Controllers (using the NT5DS domain hierarchy).

If the PDC Emulator is not configured to synchronize with a reliable external time source or hardware clock:

1. **Clock Drift Outages:** The entire forest clock can drift over time, eventually exceeding the 5-minute skew limit for external integrations or causing authentication failures.
2. **Replay Attacks:** A lack of synchronized time compromises the security of Kerberos tokens, leaving the environment vulnerable to replay attacks.

Configuring the PDC Emulator as a reliable NTP server synchronizing with a secure, trusted reference clock prevents time drift and secures Kerberos transactions.

Legacy Impact & Compatibility * **Integration Services Conflict**: When Domain Controllers run as virtual machines (e.g., on Hyper-V or VMware ESXi), the hypervisor's integration services may try to synchronize the guest's clock with the physical host. This conflict can lead to time synchronization issues. Hyper-V/VMware time synchronization must be disabled on virtual Domain Controllers if they are configured to synchronize via NTP. * **Network Connectivity**: In air-gapped environments, the PDC Emulator must be configured to synchronize from a local hardware clock (such as a GPS-based NTP appliance) rather than an internet-based server.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

To configure the PDC Emulator time synchronization via a dedicated GPO linked to the Domain Controllers OU (utilizing WMI filtering to target only the PDC Emulator):

1. Create a WMI Filter for the PDC Emulator Role 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Right-click **WMI Filters** in the console tree and select **New**. 3. Name the filter **Target PDC Emulator**. 4. Add the following WQL query: ``sql Select \* from Win32\_DirectoryServerInfo where DomainRole = 5 `` 5. Click **Save**.

2. Configure Windows Time Service Policy 1. Create a new GPO named **SEC_DomainControllers_TimeSync** and link it to the **Domain Controllers** OU. 2. Under the WMI Filtering section of the GPO, select the **Target PDC Emulator** filter. 3. Edit the GPO and navigate to: **Computer Configuration\Policies\Administrative Templates\System\Windows Time Service\Time Providers** 4. Configure the following policies: * **Configure Windows NTP Client**: * Set to **Enabled**. * **NtpServer**: `time.windows.com,0x8` (or the IP/FQDN of your local hardware NTP server in air-gapped systems). * **Type**: **NTP**. * **CrossSiteSyncFlags**: **2** (All). * **ResolvePeerBackoffMinutes**: **15**. * **ResolvePeerBackoffMaxTimes**: **7**. * **SpecialPollInterval**: **3600**. * **EventLogFlags**: **3**. * **Enable Windows NTP Client**: * Set to **Enabled**. * **Enable Windows NTP Server**: * Set to **Enabled**. 5. Navigate to: **Computer Configuration\Policies\Administrative Templates\System\Windows Time Service** 6. Configure the settings: * **Global Configuration Settings**: * Set to **Enabled**. * **AnnounceFlags**: **5** (Reliable Time Server). 7. Force update policy on the PDC Emulator.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally on the active PDC Emulator.

[Download Script: Configure-PdcTimeSync.ps1](#)

```
# Configure-PdcTimeSync.ps1
# Description: Configures Windows Time Service on the PDC Emulator or default DC time settings.

Write-Host "Applying hardening: Configure NTP Time Synchronization on PDC Emulator..." -ForegroundColor Cyan

# 1. Determine if local computer is the PDC Emulator
try {
    $Domain = [System.DirectoryServices.ActiveDirectory.Domain]::GetCurrentDomain()
    $PdcName = $Domain.PdcRoleOwner.Name
    $ComputerFQDN = "$env:COMPUTERNAME.$env:USERDNSDOMAIN"
    $IsPdc = ($PdcName -eq $ComputerFQDN) -or ($PdcName.Split(".")[0] -eq $env:COMPUTERNAME)
} catch {
    Write-Warning "Could not dynamically determine PDC Emulator FSMO role owner. Defaulting to NT5DS client mode."
    $IsPdc = $false
}

$W32TimeParams = "HKLM:\System\CurrentControlSet\Services\W32Time\Parameters"
$W32TimeConfig = "HKLM:\System\CurrentControlSet\Services\W32Time\Config"

if ($IsPdc) {
    Write-Host "[+] This system is the active PDC Emulator. Configuring as reliable NTP source..." -ForegroundColor Green

    # Configure parameters
    Set-ItemProperty -Path $W32TimeParams -Name "Type" -Value "NTP" -Type String -Force
    # Set default external NTP source (e.g. time.windows.com or local air-gapped clock)
    Set-ItemProperty -Path $W32TimeParams -Name "NtpServer" -Value "time.windows.com,0x8" -Type String -Force
    # Configure AnnounceFlags to 5 (Reliable Time Server)
    Set-ItemProperty -Path $W32TimeConfig -Name "AnnounceFlags" -Value 5 -Type DWord -Force

    # Apply changes to w32time service
    $Null = Start-Process w32tm -ArgumentList "/config /manualpeerlist:`"time.windows.com,0x8`" /syncfromflags:manual /reliable:yes /update" -Wait -NoNewWindow
    Write-Host "[+] System w32tm manual peer list updated to time.windows.com." -ForegroundColor Green
} else {
    Write-Host "[-] This system is NOT the PDC Emulator. Enforcing NT5DS domain hierarchy sync..." -ForegroundColor Yellow

    # Configure parameters
    Set-ItemProperty -Path $W32TimeParams -Name "Type" -Value "NT5DS" -Type String -Force
    Set-ItemProperty -Path $W32TimeConfig -Name "AnnounceFlags" -Value 10 -Type DWord -Force

    # Apply changes to w32time service
    $Null = Start-Process w32tm -ArgumentList "/config /syncfromflags:domhier /reliable:no /update" -Wait -NoNewWindow
    Write-Host "[+] System w32tm configured to synchronize from domain hierarchy." -ForegroundColor Green
}

# Restart Windows Time Service to apply settings
Restart-Service w32time -Force
Write-Host "[+] Windows Time Service (w32time) restarted." -ForegroundColor Green
```

To verify time synchronization status:

[Download Script: Get-PdcTimeSyncStatus.ps1](#)

```
# Get-PdcTimeSyncStatus.ps1
# Description: Audits Windows Time Service configuration on the local Domain Controller.

Write-Host "--- Auditing PDC Time Synchronization Status ---" -ForegroundColor Cyan

# 1. Determine if local computer is the PDC Emulator
try {
    $Domain = [System.DirectoryServices.ActiveDirectory.Domain]::GetCurrentDomain()
    $PdcName = $Domain.PdcRoleOwner.Name
    $ComputerFQDN = "$env:COMPUTERNAME.$env:USERDNSDOMAIN"
    $IsPdc = ($PdcName -eq $ComputerFQDN) -or ($PdcName.Split(".")[0] -eq $env:COMPUTERNAME)
} catch {
    Write-Warning "Could not dynamically determine PDC Emulator FSMO role owner."
    $IsPdc = $false
}

$W32TimeParams = "HKLM:\System\CurrentControlSet\Services\W32Time\Parameters"
$W32TimeConfig = "HKLM:\System\CurrentControlSet\Services\W32Time\Config"

$TypeVal = Get-ItemProperty -Path $W32TimeParams -Name "Type" -ErrorAction SilentlyContinue
$AnnounceVal = Get-ItemProperty -Path $W32TimeConfig -Name "AnnounceFlags" -ErrorAction SilentlyContinue

$Service = Get-Service -Name w32time -ErrorAction SilentlyContinue

if ($null -eq $Service -or $Service.Status -ne "Running") {
    Write-Host "[!] VULNERABLE: Windows Time Service (w32time) is not running." -ForegroundColor Red
    exit 1
}

if ($IsPdc) {
    Write-Host "[*] Active role: PDC Emulator FSMO owner." -ForegroundColor White
    if ($TypeVal.Type -eq "NTP" -and $AnnounceVal.AnnounceFlags -eq 5) {
        Write-Host "[+] Compliant: PDC Emulator configured as reliable NTP source (Type: NTP, Announce: 5)." -ForegroundColor Green
        exit 0
    } else {
        Write-Host "[!] NON-COMPLIANT: PDC Emulator has incorrect settings (Type: $($TypeVal.Type), Announce: $($AnnounceVal.AnnounceFlags))." -ForegroundColor Red
        exit 1
    }
} else {
    Write-Host "[*] Active role: Standard Domain Controller." -ForegroundColor White
    if ($TypeVal.Type -eq "NT5DS" -and $AnnounceVal.AnnounceFlags -eq 10) {
        Write-Host "[+] Compliant: Standard Domain Controller using NT5DS domain hierarchy." -ForegroundColor Green
        exit 0
    } else {
        Write-Host "[!] NON-COMPLIANT: DC is not using standard NT5DS settings (Type: $($TypeVal.Type), Announce: $($AnnounceVal.AnnounceFlags))." -ForegroundColor Red
        exit 1
    }
}
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**: Section 2.1 (Domain Controller requirements) * **CIS Benchmark**: CIS Microsoft Windows Server Benchmark - Section 18.9.51.1 (Ensure 'Configure Windows NTP Client' is set to 'Enabled') * **Microsoft Security Guidance**: Windows Time Service Technical Reference

[REQ-DC-032] Enable UEFI Secure Boot

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * UEFI Firmware configuration (Hardware/BIOS level or Hypervisor/VM level) * HKLM\SYSTEM\CurrentControlSet\Control\SecureBoot\State * `UEFISecureBootEnabled` = `1` (REG_DWORD)

Rationale Secure Boot is a security standard developed by members of the PC industry to help ensure that a device boots using only software that is trusted by the Original Equipment Manufacturer (OEM).

When the PC starts, the firmware checks the signature of each piece of boot software, including UEFI firmware drivers (also known as Option ROMs), EFI applications, and the operating system. If the signatures are valid, the PC boots, and the firmware gives control to the operating system.

For Tier 0 assets like Domain Controllers, firmware integrity is critical. If Secure Boot is disabled:

1. **Bootkits & Rootkits:** Attackers with physical access or hosting infrastructure control (in virtual environments) can replace the boot manager with a malicious bootloader (bootkit) to bypass LSASS protections and all OS security controls before the Windows kernel loads.
2. **Virtualization-Based Security:** Advanced security boundaries (like LSA Protection and Device Guard) depend on hardware-rooted trust. Without UEFI Secure Boot active, virtualization-based security capabilities cannot execute with proper system measurements.

Legacy Impact & Compatibility * **Virtual Domain Controllers**: When running Domain Controllers as virtual machines, UEFI and Secure Boot must be enabled in the hypervisor settings (e.g., Gen 2 VMs in Hyper-V with Secure Boot enabled, or EFI boot with Secure Boot enabled in VMware vSphere). * **BIOS Mode Conversion**: Physical Domain Controllers running in legacy BIOS mode (CSM) instead of Native UEFI cannot use Secure Boot. Converting these systems requires partition style migration (MBR to GPT) and BIOS changes; refer to [REQ-DC-018 - Harden Virtualization Hosts for Domain Controllers](#02-domain-controllers-harden-dc-virtualization-hosts-md) for virtualization host hardening requirements.

Implementation Steps

Option A: Firmware / Hypervisor Configuration (Preferred)

UEFI Secure Boot must be configured at the hardware or hypervisor layer:

- **Physical Servers:** Access the UEFI firmware configuration utility during POST (Delete, F2, F10, or F12) and enable **Secure Boot** in the Security settings.
- **Hyper-V Gen 2 Virtual Machines:**
 1. Open **Hyper-V Manager**.
 2. Select the Domain Controller VM and open its **Settings**.
 3. Navigate to **Security** → check **Enable Secure Boot** → select **Microsoft Windows** template.
- **VMware vSphere Virtual Machines:**
 1. Open the **vSphere Client**.
 2. Edit settings of the Domain Controller VM → go to **VM Options** → **Boot Options**.
 3. Set Firmware to **EFI** and check **Enable UEFI Secure Boot**.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Secure Boot cannot be configured from within the OS using registry settings. However, you must programmatically audit the state of Secure Boot to flag non-compliant hardware.

Run the following script to check the status of Secure Boot on the Domain Controller:

[Download Script: Audit-DcSecureBoot.ps1](#)

```
# Audit-DcSecureBoot.ps1
# Description: Queries UEFI Secure Boot parameters and audits UEFI Secure Boot status.

Write-Host "--- Auditing UEFI Secure Boot ---" -ForegroundColor Cyan

$script:NonCompliant = $false

# 1. Verify boot environment type
if ($env:firmware_type -eq "UEFI") {
    Write-Host "    - Boot Environment Type: UEFI" -ForegroundColor Green
} else {
    Write-Host "    - VULNERABLE: System booted in Legacy BIOS mode (CSM enabled) or firmware type is unrecognized." -ForegroundColor Red
    $script:NonCompliant = $true
}

# 2. Verify Secure Boot status
try {
    # Confirm-SecureBootUEFI returns $true if Secure Boot is active, $false if disabled,
    # and throws an exception if the platform does not support UEFI or Secure Boot.
    $SecureBootState = Confirm-SecureBootUEFI -ErrorAction Stop

    $Color = if ($SecureBootState -eq $true) { "Green" } else { "Red" }
    Write-Host "    - Secure Boot Active: $SecureBootState" -ForegroundColor $Color
    if ($SecureBootState -eq $false) { $script:NonCompliant = $true }
} catch [System.PlatformNotSupportedException] {
    Write-Host "    - VULNERABLE: UEFI Secure Boot is not supported on this platform (Legacy BIOS mode)." -ForegroundColor Red
    $script:NonCompliant = $true
} catch {
    # If cmdlet throws unauthorized access or not enabled error
    Write-Host "    - VULNERABLE: Secure Boot is disabled in firmware or cannot be verified. Error: $($_.Exception.Message)" -ForegroundColor Red
    $script:NonCompliant = $true
}

if ($script:NonCompliant) {
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows Server Benchmark***: Section 18.8 (Virtualization Based Security Prerequisites) * **ANSSI AD Hardening Guide***: Recommendations regarding hardware platform integrity.

[REQ-DC-033] Configure Secure Boot Revocations and Bootloader Updates

Target Scope * **Applicable Systems**: Domain Controllers. * **Operating Systems**: Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **BlackLotus Revocation Updates (CVE-2023-24932)**: 'HKLM\SYSTEM\CurrentControlSet\Control\Secureboot' * 'AvailableUpdates' = '>= 0x4000' (REG_DWORD)

Rationale A vulnerability in the Windows Boot Manager allows an attacker with physical access or local administrative rights to bypass UEFI Secure Boot and execute unsigned code during the boot process (BlackLotus bootkit).

To fully mitigate this threat (CVE-2023-24932), Windows update revocations must be applied to the UEFI variables (DBX list) and code integrity SVN policies must be updated. This is managed via the `AvailableUpdates` registry key, which instructs the OS boot manager to write the revocation variables to firmware.

According to the latest Microsoft guidelines, the recommended trigger value for enterprise deployments to apply all security updates (including the new Windows UEFI CA 2023 certificates and boot manager updates) is `0x5944` (hex) / `22852` (decimal). As the OS processes this bitmask, the value is cleared incrementally, ending up at `0x4000` (hex) / `16384` (decimal) upon successful completion.

Legacy Impact & Compatibility * **BlackLotus Mitigation Risks**: Enforcing the BlackLotus DBX and SVN updates is a permanent, non-reversible action once written to the device firmware. If an administrator attempts to boot the machine using older, unpatched Windows installation media or recovery disks, the system will reject the boot manager and fail to boot. All recovery and deployment media must be updated with current security patches before applying these mitigations.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

To configure the update triggers for the DBX and Code Integrity boot manager revocations, define Registry GPO Preferences inside the Domain Controllers GPO:

1. Open the **Group Policy Management Console** (`gpmc.msc`).
2. Create or edit a GPO linked to the Domain Controllers OU (e.g., `GPO_Hardening_DomainControllers`).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a registry item to deploy the `AvailableUpdates` DWORD under `HKLM\SYSTEM\CurrentControlSet\Control\Secureboot` .
 - o **Action:** `Update`
 - o **Hive:** `HKEY_LOCAL_MACHINE`
 - o **Key Path:** `SYSTEM\CurrentControlSet\Control\Secureboot`
 - o **Value Name:** `AvailableUpdates`
 - o **Value Type:** `REG_DWORD`
 - o **Value Data:** `22852` (Decimal) or `5944` (Hex)

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script locally to configure the BlackLotus mitigation update trigger:

[Download Script: Set-DcSecureBootRevocations.ps1](#)

```
# Set-DcSecureBootRevocations.ps1
# Description: Triggers Secure Boot DBX and Code Integrity revocation updates for BlackLotus mitigation.

Write-Host "--- Configuring BlackLotus Secure Boot Mitigations ---" -ForegroundColor Cyan

$Path = "HKLM:\SYSTEM\CurrentControlSet\Control\Secureboot"
if (-not (Test-Path $Path)) {
    New-Item -Path $Path -Force | Out-Null
}

# Trigger updates (0x5944 = 22852)
Set-ItemProperty -Path $Path -Name "AvailableUpdates" -Value 22852 -Type DWord -Force | Out-Null
Write-Host "[+] BlackLotus DBX and 2023 CA revocation updates configured in registry. A system reboot is required." -ForegroundColor Green
```

Audit Script

Run the following script to check the status of Secure Boot revocations on the local machine:

[Download Script: Audit-DcSecureBootRevocations.ps1](#)

```
# Audit-DcSecureBootRevocations.ps1
# Description: Queries UEFI Secure Boot parameters and audits BlackLotus mitigation registry settings.

Write-Host "--- Auditing BlackLotus Mitigations ---" -ForegroundColor Cyan

$script:NonCompliant = $false

# 1. Audit AvailableUpdates registry key
$Path = "HKLM:\SYSTEM\CurrentControlSet\Control\Secureboot"
if (Test-Path $Path) {
    $Val = Get-ItemProperty -Path $Path -Name "AvailableUpdates" -ErrorAction SilentlyContinue
    $UpdateVal = if ($Val) { $Val.AvailableUpdates } else { 0 }

    # Check if configured (≥ 0x4000 / 16384)
    if ($UpdateVal -ge 16384) {
        Write-Host "    - BlackLotus Revocation Updates (AvailableUpdates): $UpdateVal (Compliant)" -ForegroundColor Green
    } else {
        Write-Host "    - BlackLotus Revocation Updates (AvailableUpdates): $UpdateVal (Non-Compliant - DBX/SVN revocations not triggered)" -ForegroundColor Red
        $script:NonCompliant = $true
    }
} else {
    Write-Host "    - BlackLotus Revocation Updates: Registry path not found (Non-Compliant)" -ForegroundColor Red
    $script:NonCompliant = $true
}

# 2. Audit UEFICA2023Status (if present)
$ServicingPath = "HKLM:\SYSTEM\CurrentControlSet\Control\SecureBoot\Servicing"
if (Test-Path $ServicingPath) {
    $ServVal = Get-ItemProperty -Path $ServicingPath -Name "UEFICA2023Status" -ErrorAction SilentlyContinue
    if ($ServVal) {
        $Status = $ServVal.UEFICA2023Status
        $Color = if ($Status -eq "Updated") { "Green" } else { "Yellow" }
        Write-Host "    - UEFI CA 2023 Update Status: $Status" -ForegroundColor $Color
    }
}

if ($script:NonCompliant) {
    exit 1
}
```

Sources & Compliance References * **Microsoft KB5068202***: Registry key updates for Secure Boot: Windows devices with IT-managed updates * **ANSI AD Hardening Guide***: Recommendations regarding hardware platform integrity.

[REQ-DC-034] Configure Windows Defender Application Control

Target Scope * **Applicable Systems**: Domain Controllers * **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows Server 2025

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: Computer Configuration\Administrative Templates\System\Device Guard\Deploy Windows Defender Application Control * **Registry Location**:
'HKLM\SOFTWARE\Policies\Microsoft\Windows\DeviceGuard\CodeIntegrityPolicyPaths'

Rationale Domain Controllers represent the highest privilege tier (Tier 0) in an Active Directory forest. Traditional signature-based antivirus solutions are easily bypassed by custom, compiled executables, memory injection scripts, or zero-day payloads.

Windows Defender Application Control (WDAC) enforces a strict trust-based model for binary and script execution. By restricting the operating system to only run signed, trusted system files and administrative utilities, WDAC blocks unauthorized software, remote access tools, and custom malware. Deploying WDAC on Domain Controllers mitigates administrative credential dumping, domain compromises, and malware execution in kernel or user space.

Legacy Impact & Compatibility * **Pre-requisite (Memory Integrity/HVCI)**: Hypervisor-Protected Code Integrity (HVCI) must be active to enforce code integrity policies at the hypervisor layer. Refer to [REQ-DC-007 - Disable Credential Guard](#02-domain-controllers-disable-credential-guard-md) to ensure Memory Integrity (HVCI) and Virtualization-Based Security (VBS) are fully active. * **Administrative Overhead**: Any new software, agents, or drivers deployed to Domain Controllers must be digitally signed by a trusted publisher or explicitly allowed by the WDAC code integrity policy. Unsigned administrative scripts will be blocked. * **Audit Mode Deployment**: To prevent service disruption, the WDAC baseline policy must be deployed in **Audit Mode** initially. This allows administrators to verify that no critical server roles (DNS, AD DS, DHCP, backup agents) or monitoring services are blocked in production before shifting to enforcement mode.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

To deploy WDAC via Group Policy, the policy XML must first be generated, compiled, and placed in a secure shared intranet network path or local path on Domain Controllers.

1. Generate and Compile the Policy (on a Reference Domain Controller) Run the following PowerShell commands to generate the Microsoft Default Windows baseline policy: ``powershell # Generate the baseline policy XML New-CIPolicy -MultiplePolicyFormat -Level FilePublisher -FilePath "C:\WDAC\DCBaselinePolicy.xml" -UserPEs

Compile the XML policy into a binary CIP file

ConvertFrom-CIPolicy -XmlFilePath "C:\WDAC\DCBaselinePolicy.xml" -BinaryFilePath "C:\WDAC\DCBaselinePolicy.cip"

```
<div id="02-domain-controllers-configure-wdac-md-2-deploy-the-policy-via-gpo"></div>
#### 2. Deploy the Policy via GPO
1. Copy the compiled `DCBaselinePolicy.cip` file to a local secure directory on all target Domain Controllers (e.g., `C:\Windows\System32\CodeIntegrity\SIPolicy.p7b`) or host it on a network share.
2. Open the **Group Policy Management Console** (`gpmc.msc`).
3. Create or edit a GPO linked to the Domain Controllers OU (e.g., `GPO_Hardening_DomainControllers`).
4. Navigate to:
    `Computer Configuration\Administrative Templates\System\Device Guard`
5. Configure the following setting:
    * **Policy**: `Deploy Windows Defender Application Control`
    * **Setting**: `Enabled`
    * **Code Integrity Policy File Path**: Enter the local path (e.g., `C:\Windows\System32\CodeIntegrity\SIPolicy.p7b`) or network share path.

---

<div id="02-domain-controllers-configure-wdac-md-option-b-powershell-registry-configuration-remediation-non-gpo"></div>
### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to generate a baseline WDAC policy, enable Audit Mode, and configure local parameters.

<div id="02-domain-controllers-configure-wdac-md-configure-dcwdaclocalpolicy.ps1"></div>
# Configure-DCWDACLocalPolicy.ps1

[Download Script: Configure-DCWDACLocalPolicy.ps1](implementation_scripts/Configure-DCWDACLocalPolicy.ps1)

```powershell
Configure-DCWDACLocalPolicy.ps1
Description: Generates a baseline local Code Integrity policy for Domain Controllers, sets it to Audit Mode, and compiles it.

Write-Host "--- Configuring Domain Controller WDAC Local Policy Baseline ---" -ForegroundColor Cyan

Create working directories
$WdacDir = "C:\Windows\System32\CodeIntegrity"
if (-not (Test-Path $WdacDir)) {
 New-Item -Path $WdacDir -ItemType Directory -Force | Out-Null
}

1. Generate the Default Windows Policy
Write-Host "[+] Generating Default Windows code integrity rules..." -ForegroundColor Gray
$PolicyXml = "C:\Windows\Temp\DCDefaultWindows.xml"
$PolicyBin = "$WdacDir\SIPolicy.p7b"

Create a policy based on Microsoft's default rules (trusts Windows, Store, and Driver files)
New-CIPolicy -FilePath $PolicyXml -Level Windows -UserPEs -ErrorAction Stop

2. Set Policy to Audit Mode (Rule Option 3 represents Audit Mode)
Write-Host "[+] Setting WDAC policy to Audit Mode for baseline logging..." -ForegroundColor Gray
Set-RuleOption -FilePath $PolicyXml -Option 3 -ErrorAction SilentlyContinue

3. Compile the XML into the binary policy expected by the bootloader
Write-Host "[+] Compiling Code Integrity XML into SIPolicy.p7b..." -ForegroundColor Gray
ConvertFrom-CIPolicy -XmlFilePath $PolicyXml -BinaryFilePath $PolicyBin -ErrorAction Stop

Cleanup temp files
if (Test-Path $PolicyXml) { Remove-Item $PolicyXml -Force }

Write-Host "[+] Local WDAC baseline policy configured. Reboot required." -ForegroundColor Green
```

To verify the setting has been applied:

```
Test-DCWDACStatus.ps1
```

[Download Script: Test-DCWDACStatus.ps1](#)

```
Test-DCWDACStatus.ps1
Description: Audits the local Domain Controller to check if Code Integrity policies and HVCI are active.

Write-Host "--- Auditing Domain Controller WDAC State ---" -ForegroundColor Cyan
$Vulnerable = $false

1. Query WMI class for Code Integrity status
try {
 $CI = Get-CimInstance -Namespace "Root\Microsoft\Windows\CI" -ClassName "MSFT_Sipolicy" -ErrorAction Stop
 if ($null -ne $CI -and $CI.Count -gt 0) {
 Write-Host "`n[+] Found $($CI.Count) active Code Integrity policies." -ForegroundColor Green
 foreach ($Policy in $CI) {
 Write-Host " - Policy: $($Policy.FriendlyName) | ID: $($Policy.PolicyID) | Enforced: $($Policy.EnforcementMode)" -ForegroundColor Green
 }
 } else {
 Write-Host "`n[-] No active Code Integrity / WDAC policies detected via WMI." -ForegroundColor Yellow
 }
} catch {
 Write-Host "`n[-] Could not query WMI MSFT_Sipolicy. This is expected if no WDAC policies are currently deployed." -ForegroundColor Gray
}

2. Check Memory Integrity (HVCI) configuration
$ScenariosPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\HypervisorEnforcedCodeIntegrity"
if (Test-Path $ScenariosPath) {
 $HvciStatus = Get-ItemProperty -Path $ScenariosPath -Name "Enabled" -ErrorAction SilentlyContinue
 if ($null -ne $HvciStatus -and $HvciStatus.Enabled -eq 1) {
 Write-Host "[+] Memory Integrity (HVCI) is enabled." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Memory Integrity (HVCI) is disabled in the registry." -ForegroundColor Red
 $Vulnerable = $true
 }
} else {
 Write-Host "[!] VULNERABLE: Memory Integrity scenario registry path does not exist." -ForegroundColor Red
 $Vulnerable = $true
}

3. Final Verdict
if ($Vulnerable) {
 Write-Host "`n[!] Verification FAILED: One or more driver security controls are not configured." -ForegroundColor Red
} else {
 Write-Host "`n[+] Verification PASSED: WDAC driver settings and HVCI are correctly configured." -ForegroundColor Green
}
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendations on system component code integrity and application control restrictions. \* \*\*CIS Microsoft Windows Server Benchmark\*\*\*: Section 18.8.14.3 (Deploy Windows Defender Application Control / Memory Integrity). \* \*\*Microsoft Security Guidance\*\*\*: Windows Defender Application Control Deployment Guide.

# [REQ-DC-135] Configure Advanced Security Audit Policies for Domain Controllers

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 (and above)

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` \* \*\*Policy\*\*: `Audit: Force audit policy subcategory settings (Windows Vista or later) to override audit policy category settings` \* \*\*Setting\*\*: `Enabled` \* \*\*Registry Location\*\*: `HKLM\System\CurrentControlSet\Control\Lsa\SCENoApplyLegacyAuditPolicy` = `1` (DWord)

## Rationale Enforcing category overrides on Domain Controllers is a prerequisite to ensure that the refined audit subcategories (such as Directory Service Access, Kerberos operations, etc.) are correctly logged and not masked by basic legacy policies.

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Ensure the Security log size is at least 1GB to prevent rollover.

## Submodule Requirements The child policies under this parent requirement: \* \*\*[REQ-DC-136 - Audit Policy: Advanced Audit Policy Overrides](#02-domain-controllers-audit-policy-configure-dc-audit-audit-override-md)\*\* \* \*\*[REQ-DC-137 - Audit Policy: Account Logon Auditing](#02-domain-controllers-audit-policy-configure-dc-audit-account-logon-md)\*\* \* \*\*[REQ-DC-138 - Audit Policy: Account Management Auditing](#02-domain-controllers-audit-policy-configure-dc-audit-account-management-md)\*\* \* \*\*[REQ-DC-139 - Audit Policy: Detailed Tracking Auditing](#02-domain-controllers-audit-policy-configure-dc-audit-detailed-tracking-md)\*\* \* \*\*[REQ-DC-140 - Audit Policy: Directory Service Access Auditing](#02-domain-controllers-audit-policy-configure-dc-audit-ds-access-md)\*\* \* \*\*[REQ-DC-141 - Audit Policy: Logon and Logoff Auditing](#02-domain-controllers-audit-policy-configure-dc-audit-logon-logoff-md)\*\* \* \*\*[REQ-DC-142 - Audit Policy: Object Access Auditing](#02-domain-controllers-audit-policy-configure-dc-audit-object-access-md)\*\* \* \*\*[REQ-DC-143 - Audit Policy: Policy Change Auditing](#02-domain-controllers-audit-policy-configure-dc-audit-policy-change-md)\*\* \* \*\*[REQ-DC-144 - Audit Policy: Privilege Use Auditing](#02-domain-controllers-audit-policy-configure-dc-audit-privilege-use-md)\*\* \* \*\*[REQ-DC-145 - Audit Policy: System Events Auditing](#02-domain-controllers-audit-policy-configure-dc-audit-system-events-md)\*\*

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` 2. Configure the following setting: \* \*\*Policy\*\*: `Audit: Force audit policy subcategory settings (Windows Vista or later) to override audit policy category settings` \* \*\*Setting\*\*: `Enabled`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAuditPoliciesParent.ps1](#)] ([./implementation\\_scripts/Configure-DcAuditPoliciesParent.ps1](#))

```
Configure-DcAuditPoliciesParent.ps1
Description: Enforces Advanced Audit Policy Overrides registry value.

$RegPath = "reg:\HKLM\System\CurrentControlSet\Control\Lsa"
$ValueName = "SCENoApplyLegacyAuditPolicy"

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value 1 -Type DWord -Force
Write-Host "Enforced SCENoApplyLegacyAuditPolicy = 1 on Domain Controller" -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-DcAuditPoliciesParentStatus.ps1](#)

```
Get-DcAuditPoliciesParentStatus.ps1
Description: Audits the Advanced Audit Policy Overrides registry value.

$RegPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
$ValueName = "SCENoApplyLegacyAuditPolicy"

$val = Get-ItemPropertyValue -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
if ($val -eq 1) {
 Write-Host "Advanced Security Audit Policy Overrides are correctly enabled." -ForegroundColor Green
} else {
 Write-Host "Advanced Security Audit Policy Overrides are disabled!" -ForegroundColor Red
}
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R48 \* \*\*CIS Benchmark\*\*: Section 9 and 17

# [REQ-DC-136] Audit Policy: Advanced Audit Policy Overrides on Domain Controllers

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 (and above)

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Registry: `HKLM\System\CurrentControlSet\Control\Lsa\SCENoApplyLegacyAuditPolicy` = `1` (DWord) \* Registry: `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters\LogLevel` = `0` (DWord)

## Rationale Enforcing advanced audit policy overrides prevents legacy category settings from overriding refined subcategory policies, and disabling verbose Kerberos logging ensures that event logs are not flooded with diagnostic events.

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Verify that the local Security Event Log size is set to a minimum of 1GB on Domain Controllers to prevent rapid log rollover.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Registry: `HKLM\System\CurrentControlSet\Control\Lsa\SCENoApplyLegacyAuditPolicy` = `1` (DWord) \* Registry: `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters\LogLevel` = `0` (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAuditAuditoverride.ps1](#)] ([./implementation\\_scripts/Configure-DcAuditAuditoverride.ps1](#))

```
Configure-DcAuditAuditoverride.ps1
Write-Host "Applying Audit Policy category: audit-override..." -ForegroundColor Cyan

Set Registry Override: SCENoApplyLegacyAuditPolicy
if (-not (Test-Path "reg:\HKLM\System\CurrentControlSet\Control\Lsa")) { New-Item -Path "reg:\HKLM\System\CurrentControlSet\Control\Lsa" -Force | Out-Null }
Set-ItemProperty -Path "reg:\HKLM\System\CurrentControlSet\Control\Lsa" -Name "SCENoApplyLegacyAuditPolicy" -Value 1 -Type DWord -Force
Write-Host " Enforced SCENoApplyLegacyAuditPolicy = 1" -ForegroundColor Green

Set Registry Override: LogLevel
if (-not (Test-Path "reg:\HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters")) { New-Item -Path "reg:\HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters" -Force | Out-Null }
Set-ItemProperty -Path "reg:\HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters" -Name "LogLevel" -Value 0 -Type DWord -Force
Write-Host " Enforced LogLevel = 0" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-DcAuditAuditoverrideStatus.ps1](#)

```
Get-DcAuditAuditoverrideStatus.ps1
$script:Vulnerable = $false

Audit Registry: SCENoApplyLegacyAuditPolicy
$RegVal = Get-ItemProperty -Path "reg:\HKLM\System\CurrentControlSet\Control\Lsa" -Name "SCENoApplyLegacyAuditPolicy" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.SCENoApplyLegacyAuditPolicy -ne 1) {
 $script:Vulnerable = $true
}

Audit Registry: LogLevel
$RegVal = Get-ItemProperty -Path "reg:\HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters" -Name "LogLevel" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.LogLevel -ne 0) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendation R48 \* \*\*CIS Windows Server Benchmark\*\*: Section 9 (Audit Policy)

# [REQ-DC-137] Audit Policy: Account Logon Auditing on Domain Controllers

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 (and above)

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Kerberos Authentication Service` → `Success and Failure` \* Subcategory: `Kerberos Service Ticket Operations` → `Success and Failure` \* Subcategory: `Credential Validation` → `Success and Failure`

## Rationale Auditing account logon events captures authentication requests processed by the local system or the domain controller, which is critical for identifying Kerberoasting, NTLM relaying, and brute-force attempts.

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Verify that the local Security Event Log size is set to a minimum of 1GB on Domain Controllers to prevent rapid log rollover.

### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Kerberos Authentication Service` → `Success and Failure` \* Subcategory: `Kerberos Service Ticket Operations` → `Success and Failure` \* Subcategory: `Credential Validation` → `Success and Failure`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAuditAccountlogon.ps1](#)] ([./implementation\\_scripts/Configure-DcAuditAccountlogon.ps1](#))

```
Configure-DcAuditAccountlogon.ps1
Write-Host "Applying Audit Policy category: account-logon..." -ForegroundColor Cyan

Set Audit Subcategory: Kerberos Authentication Service
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Kerberos Authentication Service`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Kerberos Authentication Service to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Kerberos Authentication Service. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Kerberos Service Ticket Operations
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Kerberos Service Ticket Operations`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Kerberos Service Ticket Operations to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Kerberos Service Ticket Operations. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Credential Validation
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Credential Validation`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Credential Validation to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Credential Validation. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-DcAuditAccountlogonStatus.ps1](#)

```
Get-DcAuditAccountLogonStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Kerberos Authentication Service
$RawOutput = auditpol.exe /get /subcategory:"Kerberos Authentication Service" /r
if ($RawOutput -notmatch ",Kerberos Authentication Service,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Kerberos Service Ticket Operations
$RawOutput = auditpol.exe /get /subcategory:"Kerberos Service Ticket Operations" /r
if ($RawOutput -notmatch ",Kerberos Service Ticket Operations,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Credential Validation
$RawOutput = auditpol.exe /get /subcategory:"Credential Validation" /r
if ($RawOutput -notmatch ",Credential Validation,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendation R48 \* \*\*CIS Windows Server Benchmark\*\*\*: Section 9 (Audit Policy)



# [REQ-DC-138] Audit Policy: Account Management Auditing on Domain Controllers

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 (and above)

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `User Account Management` → `Success and Failure` \* Subcategory: `Security Group Management` → `Success and Failure` \* Subcategory: `Application Group Management` → `Success and Failure` \* Subcategory: `Computer Account Management` → `Success and Failure` \* Subcategory: `Distribution Group Management` → `Success` \* Subcategory: `Other Account Management Events` → `Success and Failure`

---

## Rationale Auditing account management logs security principal modifications (creations, deletions, password resets, group modifications) to detect privilege escalation attempts on domain or local administrative groups.

---

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Verify that the local Security Event Log size is set to a minimum of 1GB on Domain Controllers to prevent rapid log rollover.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `User Account Management` → `Success and Failure` \* Subcategory: `Security Group Management` → `Success and Failure` \* Subcategory: `Application Group Management` → `Success and Failure` \* Subcategory: `Computer Account Management` → `Success and Failure` \* Subcategory: `Distribution Group Management` → `Success` \* Subcategory: `Other Account Management Events` → `Success and Failure`

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAuditAccountmanagement.ps1](#)]

(../implementation\_scripts/Configure-DcAuditAccountmanagement.ps1)

```
Configure-DcAuditAccountmanagement.ps1
Write-Host "Applying Audit Policy category: account-management..." -ForegroundColor Cyan

Set Audit Subcategory: User Account Management
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"User Account Management`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory User Account Management to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory User Account Management. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Security Group Management
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Security Group Management`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Security Group Management to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Security Group Management. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Application Group Management
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Application Group Management`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Application Group Management to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Application Group Management. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Computer Account Management
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Computer Account Management`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Computer Account Management to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Computer Account Management. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Distribution Group Management
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Distribution Group Management`" /success:enable /failure:disable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Distribution Group Management to Success" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Distribution Group Management. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Other Account Management Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Other Account Management Events`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Other Account Management Events to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Other Account Management Events. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-DcAuditAccountmanagementStatus.ps1](#)

```

Get-DcAuditAccountmanagementStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: User Account Management
$RawOutput = auditpol.exe /get /subcategory:"User Account Management" /r
if ($RawOutput -notmatch ",User Account Management,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Security Group Management
$RawOutput = auditpol.exe /get /subcategory:"Security Group Management" /r
if ($RawOutput -notmatch ",Security Group Management,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Application Group Management
$RawOutput = auditpol.exe /get /subcategory:"Application Group Management" /r
if ($RawOutput -notmatch ",Application Group Management,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Computer Account Management
$RawOutput = auditpol.exe /get /subcategory:"Computer Account Management" /r
if ($RawOutput -notmatch ",Computer Account Management,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Distribution Group Management
$RawOutput = auditpol.exe /get /subcategory:"Distribution Group Management" /r
if ($RawOutput -notmatch ",Distribution Group Management,.*,Success") {
 $script:Vulnerable = $true
}

Audit Subcategory: Other Account Management Events
$RawOutput = auditpol.exe /get /subcategory:"Other Account Management Events" /r
if ($RawOutput -notmatch ",Other Account Management Events,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}

```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendation R48 \* \*\*CIS Windows Server Benchmark\*\*\*: Section 9 (Audit Policy)

# [REQ-DC-139] Audit Policy: Detailed Tracking Auditing on Domain Controllers

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 (and above)

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Process Creation` → `Success and Failure` \* Subcategory: `DPAPI Activity` → `Success and Failure` \* Subcategory: `PNP Activity` → `Success`

## Rationale Detailed tracking records process creations and device arrivals to ensure EDR/SIEM visibility into executable command lines and hardware plug events.

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Verify that the local Security Event Log size is set to a minimum of 1GB on Domain Controllers to prevent rapid log rollover.

### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Process Creation` → `Success and Failure` \* Subcategory: `DPAPI Activity` → `Success and Failure` \* Subcategory: `PNP Activity` → `Success`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAuditDetailedtracking.ps1](#)] (./implementation\_scripts/Configure-DcAuditDetailedtracking.ps1)

```
Configure-DcAuditDetailedtracking.ps1
Write-Host "Applying Audit Policy category: detailed-tracking..." -ForegroundColor Cyan

Set Audit Subcategory: Process Creation
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Process Creation`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Process Creation to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Process Creation. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: DPAPI Activity
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`DPAPI Activity`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory DPAPI Activity to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory DPAPI Activity. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: PNP Activity
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`PNP Activity`" /success:enable /failure:disable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory PNP Activity to Success" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory PNP Activity. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-DcAuditDetailedtrackingStatus.ps1](#)

```
Get-DcAuditDetailedtrackingStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Process Creation
$RawOutput = auditpol.exe /get /subcategory:"Process Creation" /r
if ($RawOutput -notmatch ",Process Creation,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: DPAPI Activity
$RawOutput = auditpol.exe /get /subcategory:"DPAPI Activity" /r
if ($RawOutput -notmatch ",DPAPI Activity,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: PNP Activity
$RawOutput = auditpol.exe /get /subcategory:"PNP Activity" /r
if ($RawOutput -notmatch ",PNP Activity,.*,Success") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*ANSI Active Directory Hardening Guide\*\*: Recommendation R48 \* \*\*CIS Windows Server Benchmark\*\*: Section 9 (Audit Policy)

# [REQ-DC-140] Audit Policy: Directory Service Access Auditing on Domain Controllers

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 (and above)

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Directory Service Changes` → `Success and Failure` \* Subcategory: `Directory Service Access` → `Success and Failure`

## Rationale Auditing Directory Service changes captures creations, modifications, and deletions of Active Directory objects, providing an essential trail for monitoring structural domain modifications.

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Verify that the local Security Event Log size is set to a minimum of 1GB on Domain Controllers to prevent rapid log rollover.

### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Directory Service Changes` → `Success and Failure` \* Subcategory: `Directory Service Access` → `Success and Failure`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAuditDsaccess.ps1](#)] ([./implementation\\_scripts/Configure-DcAuditDsaccess.ps1](#))

```
Configure-DcAuditDsaccess.ps1
Write-Host "Applying Audit Policy category: ds-access..." -ForegroundColor Cyan

Set Audit Subcategory: Directory Service Changes
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Directory Service Changes`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Directory Service Changes to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Directory Service Changes. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Directory Service Access
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Directory Service Access`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Directory Service Access to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Directory Service Access. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-DcAuditDsaccessStatus.ps1](#)

```
Get-DcAuditDsaccessStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Directory Service Changes
$RawOutput = auditpol.exe /get /subcategory:"Directory Service Changes" /r
if ($RawOutput -notmatch ",Directory Service Changes,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Directory Service Access
$RawOutput = auditpol.exe /get /subcategory:"Directory Service Access" /r
if ($RawOutput -notmatch ",Directory Service Access,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*  
Recommendation R48 \* \*\*CIS Windows Server Benchmark\*\*  
Section 9 (Audit Policy)

# [REQ-DC-141] Audit Policy: Logon and Logoff Auditing on Domain Controllers

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 (and above)

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Logon` → `Success and Failure` \* Subcategory: `Logoff` → `Success` \* Subcategory: `Special Logon` → `Success and Failure` \* Subcategory: `Account Lockout` → `Success and Failure` \* Subcategory: `Other Logon/Logoff Events` → `Success and Failure`

---

## Rationale Auditing logon/logoff events monitors administrative session states, special elevations, and failed logon attempts, which is critical for finding unauthorized remote access or lateral movement.

---

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Verify that the local Security Event Log size is set to a minimum of 1GB on Domain Controllers to prevent rapid log rollover.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Logon` → `Success and Failure` \* Subcategory: `Logoff` → `Success` \* Subcategory: `Special Logon` → `Success and Failure` \* Subcategory: `Account Lockout` → `Success and Failure` \* Subcategory: `Other Logon/Logoff Events` → `Success and Failure`

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAuditLogonlogoff.ps1](#)]



(../implementation\_scripts/Configure-DcAuditLogonlogoff.ps1)

```
Configure-DcAuditLogonlogoff.ps1
Write-Host "Applying Audit Policy category: logon-logoff..." -ForegroundColor Cyan

Set Audit Subcategory: Logon
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Logon`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Logon to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Logon. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Logoff
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Logoff`" /success:enable /failure:disable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Logoff to Success" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Logoff. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Special Logon
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Special Logon`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Special Logon to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Special Logon. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Account Lockout
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Account Lockout`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Account Lockout to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Account Lockout. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Other Logon/Logoff Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Other Logon/Logoff Events`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Other Logon/Logoff Events to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Other Logon/Logoff Events. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-DcAuditLogonlogoffStatus.ps1](#)

```

Get-DcAuditLogonLogoffStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Logon
$RawOutput = auditpol.exe /get /subcategory:"Logon" /r
if ($RawOutput -notmatch ",Logon,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Logoff
$RawOutput = auditpol.exe /get /subcategory:"Logoff" /r
if ($RawOutput -notmatch ",Logoff,.*, Success") {
 $script:Vulnerable = $true
}

Audit Subcategory: Special Logon
$RawOutput = auditpol.exe /get /subcategory:"Special Logon" /r
if ($RawOutput -notmatch ",Special Logon,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Account Lockout
$RawOutput = auditpol.exe /get /subcategory:"Account Lockout" /r
if ($RawOutput -notmatch ",Account Lockout,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Other Logon/Logoff Events
$RawOutput = auditpol.exe /get /subcategory:"Other Logon/Logoff Events" /r
if ($RawOutput -notmatch ",Other Logon/Logoff Events,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}

```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendation R48 \* \*\*CIS Windows Server Benchmark\*\*: Section 9 (Audit Policy)

# [REQ-DC-142] Audit Policy: Object Access Auditing on Domain Controllers

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 (and above)

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Handle Manipulation` → `Success and Failure` \* Subcategory: `Registry` → `Success and Failure` \* Subcategory: `File Share` → `Success and Failure` \* Subcategory: `Detailed File Share` → `Failure` \* Subcategory: `Other Object Access Events` → `Success and Failure`

---

## Rationale Auditing object access (files, registry keys, and shares) helps monitor unauthorized modifications to system configuration files and access to restricted shares.

---

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Verify that the local Security Event Log size is set to a minimum of 1GB on Domain Controllers to prevent rapid log rollover.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Handle Manipulation` → `Success and Failure` \* Subcategory: `Registry` → `Success and Failure` \* Subcategory: `File Share` → `Success and Failure` \* Subcategory: `Detailed File Share` → `Failure` \* Subcategory: `Other Object Access Events` → `Success and Failure`

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAuditObjectaccess.ps1](#)]

(../implementation\_scripts/Configure-DcAuditObjectaccess.ps1)

```
Configure-DcAuditObjectaccess.ps1
Write-Host "Applying Audit Policy category: object-access..." -ForegroundColor Cyan

Set Audit Subcategory: Handle Manipulation
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Handle Manipulation`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Handle Manipulation to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Handle Manipulation. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Registry
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Registry`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Registry to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Registry. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: File Share
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"File Share`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory File Share to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory File Share. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Detailed File Share
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Detailed File Share`" /success:disable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Detailed File Share to Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Detailed File Share. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Other Object Access Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Other Object Access Events`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Other Object Access Events to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Other Object Access Events. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-DcAuditObjectaccessStatus.ps1](#)

```
Get-DcAuditObjectaccessStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Handle Manipulation
$RawOutput = auditpol.exe /get /subcategory:"Handle Manipulation" /r
if ($RawOutput -notmatch ",Handle Manipulation,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Registry
$RawOutput = auditpol.exe /get /subcategory:"Registry" /r
if ($RawOutput -notmatch ",Registry,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: File Share
$RawOutput = auditpol.exe /get /subcategory:"File Share" /r
if ($RawOutput -notmatch ",File Share,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Detailed File Share
$RawOutput = auditpol.exe /get /subcategory:"Detailed File Share" /r
if ($RawOutput -notmatch ",Detailed File Share,.*,Failure") {
 $script:Vulnerable = $true
}

Audit Subcategory: Other Object Access Events
$RawOutput = auditpol.exe /get /subcategory:"Other Object Access Events" /r
if ($RawOutput -notmatch ",Other Object Access Events,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendation R48 \* \*\*CIS Windows Server Benchmark\*\*: Section 9 (Audit Policy)

# [REQ-DC-143] Audit Policy: Policy Change Auditing on Domain Controllers

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 (and above)

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Policy Change` → `Success and Failure` \* Subcategory: `Authentication Policy Change` → `Success` \* Subcategory: `Authorization Policy Change` → `Success` \* Subcategory: `MPSSVC Rule-Level Policy Change` → `Success and Failure` \* Subcategory: `Other Policy Change Events` → `Failure`

---

## Rationale Auditing policy changes tracks attempts to modify authorization policies, auditing configuration changes, or firewall rule alterations to hide adversarial tracks.

---

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Verify that the local Security Event Log size is set to a minimum of 1GB on Domain Controllers to prevent rapid log rollover.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Policy Change` → `Success and Failure` \* Subcategory: `Authentication Policy Change` → `Success` \* Subcategory: `Authorization Policy Change` → `Success` \* Subcategory: `MPSSVC Rule-Level Policy Change` → `Success and Failure` \* Subcategory: `Other Policy Change Events` → `Failure`

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAuditPolicychange.ps1](#)]

(../implementation\_scripts/Configure-DcAuditPolicychange.ps1)

```
Configure-DcAuditPolicychange.ps1
Write-Host "Applying Audit Policy category: policy-change..." -ForegroundColor Cyan

Set Audit Subcategory: Policy Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Policy Change`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Policy Change to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Policy Change. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Authentication Policy Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Authentication Policy Change`" /success:enable /failure:disable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Authentication Policy Change to Success" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Authentication Policy Change. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Authorization Policy Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Authorization Policy Change`" /success:enable /failure:disable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Authorization Policy Change to Success" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Authorization Policy Change. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: MPSSVC Rule-Level Policy Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`MPSSVC Rule-Level Policy Change`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory MPSSVC Rule-Level Policy Change to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory MPSSVC Rule-Level Policy Change. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Other Policy Change Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Other Policy Change Events`" /success:disable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Other Policy Change Events to Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Other Policy Change Events. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-DcAuditPolicychangeStatus.ps1](#)

```

Get-DcAuditPolicychangeStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Policy Change
$RawOutput = auditpol.exe /get /subcategory:"Policy Change" /r
if ($RawOutput -notmatch ",Policy Change,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Authentication Policy Change
$RawOutput = auditpol.exe /get /subcategory:"Authentication Policy Change" /r
if ($RawOutput -notmatch ",Authentication Policy Change,.*,Success") {
 $script:Vulnerable = $true
}

Audit Subcategory: Authorization Policy Change
$RawOutput = auditpol.exe /get /subcategory:"Authorization Policy Change" /r
if ($RawOutput -notmatch ",Authorization Policy Change,.*,Success") {
 $script:Vulnerable = $true
}

Audit Subcategory: MPSSVC Rule-Level Policy Change
$RawOutput = auditpol.exe /get /subcategory:"MPSSVC Rule-Level Policy Change" /r
if ($RawOutput -notmatch ",MPSSVC Rule-Level Policy Change,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Other Policy Change Events
$RawOutput = auditpol.exe /get /subcategory:"Other Policy Change Events" /r
if ($RawOutput -notmatch ",Other Policy Change Events,.*,Failure") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}

```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendation R48 \* \*\*CIS Windows Server Benchmark\*\*\*: Section 9 (Audit Policy)



# [REQ-DC-144] Audit Policy: Privilege Use Auditing on Domain Controllers

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 (and above)

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Sensitive Privilege Use` → `Success and Failure`

## Rationale Auditing sensitive privilege use logs attempts by processes or users to exercise rights like ActAsPartOfTypeOperatingSystem or LoadDrivers, identifying potential privilege escalations.

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Verify that the local Security Event Log size is set to a minimum of 1GB on Domain Controllers to prevent rapid log rollover.

### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Sensitive Privilege Use` → `Success and Failure`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAuditPrivilegeuse.ps1](#)] ([../implementation\\_scripts/Configure-DcAuditPrivilegeuse.ps1](#))

```
Configure-DcAuditPrivilegeuse.ps1
Write-Host "Applying Audit Policy category: privilege-use..." -ForegroundColor Cyan

Set Audit Subcategory: Sensitive Privilege Use
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Sensitive Privilege Use`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Sensitive Privilege Use to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Sensitive Privilege Use. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-DcAuditPrivilegeuseStatus.ps1](#)

```
Get-DcAuditPrivilegeuseStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Sensitive Privilege Use
$RawOutput = auditpol.exe /get /subcategory:"Sensitive Privilege Use" /r
if ($RawOutput -notmatch ",Sensitive Privilege Use,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendation R48 \* \*\*CIS Windows Server Benchmark\*\*: Section 9 (Audit Policy)

# [REQ-DC-145] Audit Policy: System Events Auditing on Domain Controllers

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 (and above)

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `IPsec Driver` → `Success and Failure` \* Subcategory: `Other System Events` → `Success and Failure` \* Subcategory: `Security State Change` → `Success and Failure` \* Subcategory: `Security System Extension` → `Success and Failure` \* Subcategory: `System Integrity` → `Success and Failure`

---

## Rationale Auditing system security extensions, integrity violations, and driver arrivals monitors boot health and tampering of host security services.

---

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Verify that the local Security Event Log size is set to a minimum of 1GB on Domain Controllers to prevent rapid log rollover.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `IPsec Driver` → `Success and Failure` \* Subcategory: `Other System Events` → `Success and Failure` \* Subcategory: `Security State Change` → `Success and Failure` \* Subcategory: `Security System Extension` → `Success and Failure` \* Subcategory: `System Integrity` → `Success and Failure`

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DcAuditSystemevents.ps1](#)]

(../implementation\_scripts/Configure-DcAuditSystemevents.ps1)

```
Configure-DcAuditSystemevents.ps1
Write-Host "Applying Audit Policy category: system-events..." -ForegroundColor Cyan

Set Audit Subcategory: IPsec Driver
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`IPsec Driver`" /success:enable /failure:enable" -Wait -NoNewWin
dow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory IPsec Driver to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory IPsec Driver. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Other System Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Other System Events`" /success:enable /failure:enable" -Wait -N
oNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Other System Events to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Other System Events. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Security State Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Security State Change`" /success:enable /failure:enable" -Wait
-NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Security State Change to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Security State Change. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Security System Extension
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Security System Extension`" /success:enable /failure:enable" -W
ait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Security System Extension to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Security System Extension. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: System Integrity
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`System Integrity`" /success:enable /failure:enable" -Wait -NoNe
wWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory System Integrity to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory System Integrity. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-DcAuditSystemeventsStatus.ps1](#)

```

Get-DcAuditSystemeventsStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: IPsec Driver
$RawOutput = auditpol.exe /get /subcategory:"IPsec Driver" /r
if ($RawOutput -notmatch ",IPsec Driver,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Other System Events
$RawOutput = auditpol.exe /get /subcategory:"Other System Events" /r
if ($RawOutput -notmatch ",Other System Events,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Security State Change
$RawOutput = auditpol.exe /get /subcategory:"Security State Change" /r
if ($RawOutput -notmatch ",Security State Change,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Security System Extension
$RawOutput = auditpol.exe /get /subcategory:"Security System Extension" /r
if ($RawOutput -notmatch ",Security System Extension,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: System Integrity
$RawOutput = auditpol.exe /get /subcategory:"System Integrity" /r
if ($RawOutput -notmatch ",System Integrity,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}

```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendation R48 \* \*\*CIS Windows Server Benchmark\*\*\*: Section 9 (Audit Policy)

## # Module 3: Identities & Services Hardening

This directory contains security requirements and policies designed to protect administrative identities, user credentials, and critical network service accounts in the Active Directory domain.

### ## Technical Hardening Controls

1. [REQ-ID-001 - Enforce Fine-Grained Password Policies](#) Enforces Password Settings Objects (PSOs) with strong password length and lockout settings for administrative groups.
2. [REQ-ID-002 - Enable Local Administrator Password Solution \(LAPS\)](#) Implements Windows LAPS or Classic LAPS to rotate local administrator passwords periodically.
3. [REQ-ID-003 - Implement Group Managed Service Accounts \(gMSA\)](#) Replaces static passwords with auto-managed complex service account credentials.
4. [REQ-ID-004 - Restrict Kerberos Delegation](#) Bans unconstrained delegation and mandates constrained/resource-based constrained delegation.
5. [REQ-ID-005 - Configure and Populate Protected Users Group](#) Enforces strict caching and authentication restrictions on high-privilege identities to prevent credential theft.
6. [REQ-ID-006 - Rename and Disable Default Administrator and Guest Accounts](#) Mitigates automated scanning and brute-force attempts on built-in OS accounts.
7. [REQ-ID-007 - Restrict Interactive Logons for Service Accounts](#) Blocks interactive local and remote desktop logons for service accounts via User Rights Assignment GPOs.
8. [REQ-ID-008 - Enforce User and Service Account Kerberos Encryption \(AES-Only\)](#) Sets the msDS-SupportedEncryptionTypes attribute to AES-only to mitigate Kerberoasting and session hijacking.
9. [REQ-ID-009 - Enforce Kerberos Pre-Authentication](#) Mandates Kerberos pre-authentication on all active user accounts to mitigate AS-REP Roasting attacks.
10. [REQ-ID-010 - Restrict Schema Administrators Group Membership](#) Automates Schema Admins membership audit and locking using Restricted Groups GPO to minimize the attack surface.
11. [REQ-ID-011 - Enforce Accidental Deletion Protection on Organizational Units](#) Safeguards OUs from deletion errors or malicious administrative actions via the `ProtectedFromAccidentalDeletion` attribute.
12. [REQ-ID-012 - Configure Active Directory Authentication Silos and Policies](#) Enforces logical boundaries restricting where Tier 0 administrator and host accounts can authenticate, preventing credential theft.
13. [REQ-ID-013 - Clean Up adminCount Attribute Orphans](#) Identifies and remediates orphan accounts with disabled security descriptor inheritance, resetting adminCount to 0 and re-enabling inheritance.
14. [REQ-ID-014 - Renew KDS Root Keys and gMSA Secrets](#) Enforces KDS root key rotation and triggers password regeneration for Group Managed Service Accounts to mitigate exfiltration backdoors.
15. [REQ-ID-015 - Harden Active Directory Certificate Services \(ADCS\) and PKI](#) Hardens ADCS templates to block ESC1 SAN enrollment bypasses, mandates manager approval, and secures CA Web Enrollment endpoints.
16. [REQ-ID-016 - Configure Logon Screen and Credentials Delegation](#) Restricts logon screen user enumeration, and hardens CredSSP/credentials delegation.
17. [REQ-ID-017 - Disable Machine Account Quota](#) Restricts the ms-DS-MachineAccountQuota attribute to 0 and limits the SeMachineAccountPrivilege user right to prevent unauthorized computer object creation by standard domain users.
18. [REQ-ID-018 - Restrict Pre-Windows 2000 Compatible Access Group](#) Limits the memberships of the legacy "Pre-Windows 2000 Compatible Access" group and restricts anonymous query options to prevent directory enumeration.
19. [REQ-ID-019 - Enforce Smart Card Authentication for Privileged Users](#) Enforces the 'Smart card is required for interactive logon' setting on administrative accounts to invalidate password hashes and force Kerberos PKINIT.
20. [REQ-ID-020 - Clean Up Legacy Group Policy Preferences and SYSVOL Passwords](#) Identifies and remediates Group Policy Preferences (GPP) XML files with `cpassword` properties and legacy scripts containing cleartext credentials inside SYSVOL.

# [REQ-ID-001] Enforce Fine-Grained Password Policies

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: Active Directory System Container: `CN=Password Settings Container,CN=System,DC=[Domain]`

## Rationale Active Directory default domain password policies apply globally to all user accounts. These global policies are often configured with lower complexity and length requirements to avoid overwhelming standard users. However, such settings are inadequate for highly privileged accounts (Tier 0 and Tier 1 administrators), which are primary targets for credential stuffing, brute-force, and offline cracking attacks.

Enforcing Fine-Grained Password Policies (FGPP) via Password Settings Objects (PSOs) allows administrators to apply distinct, highly restrictive password and account lockout policies to specific users or groups. By mandating longer password lengths and stricter lockout thresholds for privileged identities, the domain's defense-in-depth posture is significantly bolstered without affecting standard users.

## Legacy Impact & Compatibility \* \*\*Application Compatibility\*\*: Legacy applications, automated scripts, or administrative tools that authenticate using service accounts or administrative credentials might fail if they do not support passwords longer than 14 characters or are hardcoded to fail on password complexity. \* \*\*Pre-remediation Audit\*\*: Before applying the policy to administrative groups, audit administrative accounts to ensure their current passwords comply with the new length requirements, or force a password reset upon applying the PSO.

#### ## Implementation Steps

##### #### Option A: Active Directory Administrative Center (ADAC) Configuration (Preferred)

1. Open the **Active Directory Administrative Center** ( `dsac.exe` ) on a Domain Controller or management server.
2. Switch to the Tree View, select your domain, and navigate to: `System\Password Settings Container`
3. Right-click the **Password Settings Container**, select **New**, and then click **Password Settings**.
4. Configure the Password Settings Object (PSO):
  - **Name**: `Tier0-Admin-PSO`
  - **Precedence**: `10`
  - **Enforce minimum password length**: `20`
  - **Password must meet complexity requirements**: Checked
  - **Enforce password history**: `24`
  - **Store password using reversible encryption**: Unchecked
  - **Number of failed logon attempts allowed (lockout threshold)**: `5`
  - **Reset failed logon attempts count after**: `30 minutes`
  - **Account lockout duration**: `30 minutes`
5. Under **Directly Applies To**, click **Add** and select the security group containing your Tier 0 and Tier 1 administrators (e.g., `Grp_Tier0_Admins` ).
6. Click **OK** to save and apply the PSO.

##### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script to create and apply the Fine-Grained Password Policy using PowerShell.

[Download Script: Set-AdminPasswordPolicy.ps1](#)

```
Set-AdminPasswordPolicy.ps1
Description: Creates a secure Fine-Grained Password Policy for administrative accounts.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Enforce Fine-Grained Password Policies..." -ForegroundColor Cyan

$AdminPSOName = "Tier0-Admin-PSO"
$ExistingPSO = Get-ADFineGrainedPasswordPolicy -Filter "Name -eq '$AdminPSOName'"

if (-not $ExistingPSO) {
 # Create the Fine-Grained Password Policy (PSO)
 New-ADFineGrainedPasswordPolicy -Name $AdminPSOName `
 -Precedence 10 `
 -ComplexityEnabled $true `
 -MinPasswordLength 20 `
 -PasswordHistoryCount 24 `
 -ReversibleEncryptionEnabled $false `
 -LockoutDuration "00:30:00" `
 -LockoutObservationWindow "00:30:00" `
 -LockoutThreshold 5 `
 -MinPasswordAge "1.00:00:00" `
 -MaxPasswordAge "60.00:00:00"

 Write-Host "PSO '$AdminPSOName' created successfully." -ForegroundColor Green
} else {
 Write-Host "PSO '$AdminPSOName' already exists." -ForegroundColor Yellow
}
```

To verify the applied Fine-Grained Password Policies: [Download Script: Get-AdminPasswordPolicyStatus.ps1](#)

```
Get-AdminPasswordPolicyStatus.ps1
Description: Audits Fine-Grained Password Policies in the Active Directory domain.

Import-Module ActiveDirectory

Write-Host "--- Auditing Fine-Grained Password Policies ---" -ForegroundColor Cyan

$psolist = Get-ADFineGrainedPasswordPolicy -Filter *

if ($psolist) {
 foreach ($pso in $psolist) {
 Write-Host "[+] PSO Name: $($pso.Name)" -ForegroundColor Green
 Write-Host " - Precedence: $($pso.Precedence)" -ForegroundColor White
 Write-Host " - MinPasswordLength: $($pso.MinPasswordLength)" -ForegroundColor White
 Write-Host " - LockoutThreshold: $($pso.LockoutThreshold)" -ForegroundColor White
 Write-Host " - LockoutDuration: $($pso.LockoutDuration)" -ForegroundColor White
 }
} else {
 Write-Host "[-] No Fine-Grained Password Policies found in the domain." -ForegroundColor Yellow
}
```

---

**## Sources & Compliance References** \* **\*\*ANSI AD Hardening Guide\*\***: Section 3.1 (User Password policies) \* **\*\*CIS Benchmark\*\***: CIS Microsoft Windows Server Benchmark - Section 1.1 (Password Policy) \* **\*\*Microsoft Security Guidance\*\***: AD DS Fine-Grained Password Policies Step-by-Step Guide

## # [REQ-ID-002] Enable Local Administrator Password Solution (LAPS)

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients, Domain Controllers \* \*\*Operating Systems\*\*: \* \*\*Modern Windows LAPS\*\*: Windows Server 2019/2022/2025 (with April 11, 2023 cumulative update or later), Windows 10/11 (with April 11, 2023 cumulative update or later) \* \*\*Legacy Microsoft LAPS\*\*: Windows Server 2016 (and older), Windows 7/8/8.1, Windows Server 2008 R2/2012/2012 R2

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\System\LAPS` \* \*\*Registry Location (GPO Policies)\*\*: `HKLM\Software\Policies\Microsoft\Windows\LAPS` \* \*\*Registry Location (Local / CSP Settings)\*\*: `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\LAPS`

---

## Rationale In standard Active Directory setups, local administrator accounts on member servers and client workstations often share the same password. If a single machine is compromised and the local administrator password hash is extracted (e.g., from LSASS memory or SAM database), attackers can leverage Pass-the-Hash (PtH) techniques to log on to other domain machines laterally.

Implementing the Local Administrator Password Solution (LAPS) completely mitigates this lateral movement vector by automatically generating a unique, complex password for the specified local administrator account on each machine. These passwords are changed periodically and stored securely in a confidential attribute ( `msLAPS-Password` or `ms-Mcs-AdmPwd` ) on the computer's Active Directory object. Read access is restricted to authorized administrative groups.

#### Modern Windows LAPS vs. Legacy Microsoft LAPS Microsoft introduced modern **Windows LAPS** as a native OS feature in April 2023, deprecating the older, installer-based **Legacy Microsoft LAPS** (MSI package). Key benefits of modern Windows LAPS include: 1.

**Password Encryption**: Cryptographically protects passwords stored in Active Directory, preventing exposure to unauthorized users who might inspect directory attributes. 2. **Directory Services Restore Mode (DSRM) Backup**: Supports backing up DSRM account passwords on Domain Controllers (running Windows Server 2019 and newer). 3. **Password History**: Allows retrieval of previous passwords to support system restores. 4. **Post-Authentication Actions**: Forces an automated password reset and logs off the managed administrator account after interactive use, restricting the lifetime of exposed credentials on a device. 5. **Azure AD / Microsoft Entra ID Support**: Natively supports backing up passwords directly to the cloud for cloud-joined or hybrid-joined devices.

---

## Legacy Impact & Compatibility \* \*\*LAPS Client Dependency\*\*: Devices must have the Windows LAPS extension installed (native to modern Windows updates) or have the Classic LAPS client installed. \* \*\*Schema Extension\*\*: The Active Directory schema must be extended to include LAPS attributes before clients can write passwords. \* \*\*AD Permissions\*\*: Ensure permissions on computer objects are configured to block non-administrator users from reading the confidential password attributes. \* \*\*Encryption Prerequisite\*\*: Encryption of LAPS passwords requires a Domain Functional Level of at least Windows Server 2016. If the functional level is lower, passwords must be stored in clear text. \* \*\*DSRM Prerequisite\*\*: DSRM password backup is only supported on Domain Controllers running Windows Server 2019 or later. \* \*\*Windows Server 2016 Warning\*\*: Windows Server 2016 domain controllers and member servers do not support modern Windows LAPS natively. For Windows Server 2016 systems, the **Legacy Microsoft LAPS** client (MSI-based) must be deployed and maintained.

---

## ## Prerequisites & Active Directory Preparation

Before configuring Windows LAPS client policies, the Active Directory forest must be prepared to support modern LAPS attributes and access controls.

#### Step 1: Update Active Directory Schema Extend the Active Directory schema to add the modern Windows LAPS attributes. This must be run by an administrator with Schema Admins privileges in the forest root domain. Run this on a Domain Controller running Windows Server 2019 or later, or any system with the LAPS PowerShell module installed: ``powershell Import-Module Laps Update-LapsADSchema -Verbose ``

#### Step 2: Configure Organizational Unit Permissions Computers must be authorized to write their own local administrator passwords to their corresponding computer object in Active Directory. Grant this permission by running the following command against the Organizational Units (OUs) that contain the target computers: ``powershell Set-LapsADComputerSelfPermission -Identity "OU=Workstations,DC=domain,DC=local" -Verbose ``

#### Step 3: Grant Password Query Permissions By default, only Domain Admins have permissions to read LAPS passwords. Grant password query permissions to dedicated administrative groups (e.g., tier-2 support technicians): ``powershell Set-LapsADReadPasswordPermission -Identity "OU=Workstations,DC=domain,DC=local" -AllowedPrincipals @("DOMAIN\Tier2-Support") -Verbose ``

#### Step 4: Grant Password Reset (Expiration) Permissions Grant permissions to authorize administrative groups to force an early password rotation (by marking the password as expired in AD): ``powershell Set-LapsADResetPasswordPermission -Identity "OU=Workstations,DC=domain,DC=local" -AllowedPrincipals @("DOMAIN\Tier2-Support") -Verbose ``

#### Step 5: Audit Extended Rights Audit and verify that no unauthorized groups or users possess extended rights on the OU which would allow them to bypass confidentiality settings and read password attributes: ``powershell Find-LapsADExtendedRights -Identity "OU=Workstations,DC=domain,DC=local" ``

---

## ## Implementation Steps

### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a management host.
2. Edit the appropriate hardening GPO (e.g., `GPO_Hardening_MemberServers` ).
3. Navigate to: `Computer Configuration\Administrative Templates\System\LAPS`
4. Configure the following settings:



- **Policy:** `Configure password backup directory`
  - **Setting:** `Enabled`
  - **Options:** Set backup directory to `Active Directory`.
- **Policy:** `Do not allow password expiration time longer than required by policy`
  - **Setting:** `Enabled`
- **Policy:** `Enable password encryption`
  - **Setting:** `Enabled`
- **Policy:** `Password Settings`
  - **Setting:** `Enabled`
  - **Options:** Set complexity to `Large letters + small letters + numbers + special characters`, length to `20`, and age to `30` days.
- **Policy:** `Post-authentication actions`
  - **Setting:** `Enabled`
  - **Options:** Set Grace period (hours) to `8`, Actions to `Reset the password and logoff the managed account`.
- **Policy:** `Enable local admin password management`
  - **Setting:** `Enabled`

5. Link the GPO to the Organizational Units (OUs) containing member servers and client workstations.

---

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally (for testing or standalone systems) or if the control is not manageable via standard GPO GUI interfaces.

#### Download Script: [Configure-LAPS.ps1](#)

```
Configure-LAPS.ps1
Description: Configures Windows LAPS parameters in the registry.

Write-Host "Applying hardening requirement: Enable Local Administrator Password Solution..." -ForegroundColor Cyan

$RegPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\LAPS"

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Enable LAPS management
Set-ItemProperty -Path $RegPath -Name "EnableLAPS" -Value 1 -Type DWord
2 = Backup to Active Directory
Set-ItemProperty -Path $RegPath -Name "BackupDirectory" -Value 2 -Type DWord
1 = Do not allow password expiration time longer than required by policy
Set-ItemProperty -Path $RegPath -Name "PasswordExpirationProtectionEnabled" -Value 1 -Type DWord
1 = Enable password encryption
Set-ItemProperty -Path $RegPath -Name "ADPasswordEncryptionEnabled" -Value 1 -Type DWord
4 = Letters + numbers + special characters
Set-ItemProperty -Path $RegPath -Name "PasswordComplexity" -Value 4 -Type DWord
Set-ItemProperty -Path $RegPath -Name "PasswordLength" -Value 20 -Type DWord
Set-ItemProperty -Path $RegPath -Name "PasswordAgeDays" -Value 30 -Type DWord
8 = Grace period of 8 hours
Set-ItemProperty -Path $RegPath -Name "PostAuthenticationResetDelay" -Value 8 -Type DWord
3 = Reset the password and logoff the managed account
Set-ItemProperty -Path $RegPath -Name "PostAuthenticationActions" -Value 3 -Type DWord

Write-Host "Windows LAPS configuration registry settings applied successfully." -ForegroundColor Green
```

---

#### ## Operation & Monitoring

### Force Policy Processing Trigger Windows LAPS to immediately query GPO and process the current policy (e.g., generate a new password if expired): ``powershell Invoke-LapsPolicyProcessing ``

### Retrieve Password from Active Directory Query and retrieve the managed administrator password in plain text: ``powershell Get-LapsADPassword -Identity "COMPUTER-NAME" -AsPlainText ``

### Force Early Password Rotation Set the scheduled password expiration to the current time, forcing the computer to rotate the password

during its next processing cycle: ``powershell Set-LapsADPasswordExpirationTime -Identity "COMPUTER-NAME" `` Or force immediate local rotation directly on the managed client system: ``powershell Reset-LapsPassword ``

### Diagnostics & Event Logs Audit Windows LAPS actions in the Windows Event Viewer: \* \*\*Log Channel Path\*\*: `Applications and Services Logs \ Microsoft \ Windows \ LAPS \ Operational` \* \*\*Key Event IDs\*\*: \* `10018`: LAPS password successfully backed up to Active Directory. \* `10019`: LAPS password backup to Active Directory failed. \* `10020`: LAPS password encryption failed.

---

#### ## Audit verification

Use the following script to verify the setting has been applied:

[Download Script: Get-LAPSStatus.ps1](#)

```
Get-LAPSStatus.ps1
Description: Checks the Windows LAPS registry parameters.

Write-Host "--- Auditing LAPS Registry Configuration ---" -ForegroundColor Cyan
$script:Vulnerable = $false

$RegPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\LAPS"

if (Test-Path $RegPath) {
 $enableLAPS = Get-ItemProperty -Path $RegPath -Name "EnableLAPS" -ErrorAction SilentlyContinue
 $backupDir = Get-ItemProperty -Path $RegPath -Name "BackupDirectory" -ErrorAction SilentlyContinue
 $expirationProtection = Get-ItemProperty -Path $RegPath -Name "PasswordExpirationProtectionEnabled" -ErrorAction SilentlyContinue
 $encryption = Get-ItemProperty -Path $RegPath -Name "ADPasswordEncryptionEnabled" -ErrorAction SilentlyContinue
 $complexity = Get-ItemProperty -Path $RegPath -Name "PasswordComplexity" -ErrorAction SilentlyContinue
 $length = Get-ItemProperty -Path $RegPath -Name "PasswordLength" -ErrorAction SilentlyContinue
 $age = Get-ItemProperty -Path $RegPath -Name "PasswordAgeDays" -ErrorAction SilentlyContinue
 $resetDelay = Get-ItemProperty -Path $RegPath -Name "PostAuthenticationResetDelay" -ErrorAction SilentlyContinue
 $actions = Get-ItemProperty -Path $RegPath -Name "PostAuthenticationActions" -ErrorAction SilentlyContinue

 Write-Host "[+] LAPS Configuration Found under HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\LAPS" -ForegroundColor Green

 # Audit EnableLAPS
 if ($null -ne $enableLAPS -and $enableLAPS.EnableLAPS -eq 1) {
 Write-Host " - LAPS Management: Enabled" -ForegroundColor White
 } else {
 Write-Host " - LAPS Management: NOT ENABLED (EnableLAPS = 0 or missing)" -ForegroundColor Red
 $script:Vulnerable = $true
 }

 # Audit BackupDirectory
 if ($null -ne $backupDir -and $backupDir.BackupDirectory -eq 2) {
 Write-Host " - Backup Directory: $($backupDir.BackupDirectory) (2 = Active Directory)" -ForegroundColor White
 } else {
 $val = "Missing"
 if ($null -ne $backupDir) { $val = $backupDir.BackupDirectory }
 Write-Host " - Backup Directory: $($val) (Expected: 2 = Active Directory)" -ForegroundColor Red
 $script:Vulnerable = $true
 }

 # Audit Expiration Protection
 if ($null -ne $expirationProtection -and $expirationProtection.PasswordExpirationProtectionEnabled -eq 1) {
 Write-Host " - Expiration Protection: Enabled" -ForegroundColor White
 } else {
 $val = "Missing"
 if ($null -ne $expirationProtection) { $val = $expirationProtection.PasswordExpirationProtectionEnabled }
 Write-Host " - Expiration Protection: $($val) (Expected: 1)" -ForegroundColor Red
 $script:Vulnerable = $true
 }

 # Audit Encryption
 if ($null -ne $encryption -and $encryption.ADPasswordEncryptionEnabled -eq 1) {
 Write-Host " - Password Encryption: Enabled" -ForegroundColor White
 } else {
 $val = "Missing"
 if ($null -ne $encryption) { $val = $encryption.ADPasswordEncryptionEnabled }
 Write-Host " - Password Encryption: $($val) (Expected: 1)" -ForegroundColor Red
 $script:Vulnerable = $true
 }

 # Audit Complexity
 if ($null -ne $complexity -and $complexity.PasswordComplexity -eq 4) {
 Write-Host " - Password Complexity: $($complexity.PasswordComplexity) (4 = Large + small + numbers + special characters)" -ForegroundColor White
 } else {
 $val = "Missing"
 if ($null -ne $complexity) { $val = $complexity.PasswordComplexity }
 Write-Host " - Password Complexity: $($val) (Expected: 4)" -ForegroundColor Red
 $script:Vulnerable = $true
 }

 # Audit Length
 if ($null -ne $length -and $length.PasswordLength -ge 15) {
```

```

 Write-Host " - Password Length: $($length.PasswordLength) characters (Secure, ≥ 15)" -ForegroundColor White
 } else {
 $val = "Missing"
 if ($null -ne $length) { $val = $length.PasswordLength }
 Write-Host " - Password Length: $($val) (Expected: ≥ 15)" -ForegroundColor Red
 $script:Vulnerable = $true
 }

 # Audit Age
 if ($null -ne $age -and $age.PasswordAgeDays -le 30) {
 Write-Host " - Password Rotation Interval: $($age.PasswordAgeDays) days (Secure, ≤ 30)" -ForegroundColor White
 } else {
 $val = "Missing"
 if ($null -ne $age) { $val = $age.PasswordAgeDays }
 Write-Host " - Password Rotation Interval: $($val) (Expected: ≤ 30)" -ForegroundColor Red
 $script:Vulnerable = $true
 }

 # Audit Post-Authentication Reset Delay
 if ($null -ne $resetDelay -and $resetDelay.PostAuthenticationResetDelay -le 8 -and $resetDelay.PostAuthenticationResetDelay -gt 0) {
 Write-Host " - Post-Auth Reset Delay: $($resetDelay.PostAuthenticationResetDelay) hours (Secure, ≤ 8)" -ForegroundColor White
 } else {
 $val = "Missing"
 if ($null -ne $resetDelay) { $val = $resetDelay.PostAuthenticationResetDelay }
 Write-Host " - Post-Auth Reset Delay: $($val) (Expected: ≤ 8 and > 0)" -ForegroundColor Red
 $script:Vulnerable = $true
 }

 # Audit Post-Authentication Actions
 if ($null -ne $actions -and $actions.PostAuthenticationActions -eq 3) {
 Write-Host " - Post-Auth Actions: $($actions.PostAuthenticationActions) (3 = Reset and logoff)" -ForegroundColor White
 } else {
 $val = "Missing"
 if ($null -ne $actions) { $val = $actions.PostAuthenticationActions }
 Write-Host " - Post-Auth Actions: $($val) (Expected: 3 = Reset and logoff)" -ForegroundColor Red
 $script:Vulnerable = $true
 }

} else {
 Write-Host "[!] VULNERABLE: Windows LAPS registry path does not exist. LAPS is not configured." -ForegroundColor Red
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}

```

## Sources & Compliance References \* \*\*Microsoft Documentation\*\*: [Windows LAPS Overview](https://learn.microsoft.com/en-us/windows-server/identity/laps/laps-overview) \* \*\*Microsoft Documentation\*\*: [Get started with Windows LAPS and Windows Server Active Directory](https://learn.microsoft.com/en-us/windows-server/identity/laps/laps-scenarios-windows-server-active-directory) \* \*\*ANSSI AD Hardening Guide\*\*: Section on Local account management and password randomization \* \*\*CIS Benchmark\*\*: CIS Microsoft Windows Server Benchmark - Section 18.9.11 (LAPS Configuration) \* \*\*CIS Benchmark\*\*: CIS Microsoft Windows Client Benchmark - Section 18.9.25 (LAPS Configuration)

# [REQ-ID-003] Implement Group Managed Service Accounts (gMSA)

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: Active Directory Object Management (Managed Service Accounts container: 'CN=Managed Service Accounts,DC=[Domain]')

## Rationale Traditional service accounts in Active Directory are standard user accounts with static, often long-lived passwords. Because service passwords are rarely rotated, they are prime targets for offline brute-force attacks known as **Kerberoasting**. An attacker with domain access can request a Kerberos service ticket (TGS) for any account with a Service Principal Name (SPN) and attempt to crack the password hash offline.

Group Managed Service Accounts (gMSAs) address this risk by delegating password management to the operating system and Domain Controllers. Windows automatically generates a complex 120-character password for each gMSA and rotates it every 30 days. Additionally, gMSAs cannot be used for interactive logons, preventing administrative session hijacking or remote administrative access via service accounts.

However, gMSAs introduce specific security boundaries that must be strictly enforced:

1. **Password Retrieval Delegation (GMSA Password Access)**: The attribute `msDS-GroupMSAMembership` ( `PrincipalsAllowedToRetrieveManagedPassword` ) defines which security principals can query Active Directory to retrieve the clear-text gMSA password. If human user accounts or groups containing human users are added to this attribute, any compromise of those user credentials allows an attacker to fetch the clear-text password blob and convert it to an NT hash.
2. **Credential Dumping from memory (LSASS ekeys)**: While LSASS does not cache the clear-text password of a gMSA under standard `sekurlsa::logonpasswords` dumps, the active Kerberos keys (NT hash, AES-128/256 keys) are stored in memory on the host computer running the service. An attacker with administrative/SYSTEM access to the host server can extract these keys using Mimikatz `sekurlsa::ekeys` and use them for pass-the-hash (PTH) or pass-the-ticket (PTT) attacks.
3. **Tier Alignment (Tier-Matching)**: Because compromising the host server hosting a gMSA compromises the gMSA itself, and retrieving the gMSA password grants full control over its permissions, hosts running gMSAs must be secured to the same level (Tier) as the privileges granted to the gMSA. A Tier 0 gMSA must only run on Tier 0 systems (Domain Controllers or Tier 0 Admin Hosts), and only Tier 0 computer accounts/groups must be allowed to retrieve its password.

## Legacy Impact & Compatibility \* \*\*OS Compatibility\*\*: gMSAs require a domain functional level of Windows Server 2012 or higher. Client hosts running the service must run Windows Server 2012/Windows 8 or higher. \* \*\*Application Support\*\*: While major enterprise services such as Microsoft SQL Server, IIS, and Windows Services support gMSAs native, some legacy third-party applications do not support managing authentication without standard credentials. \* \*\*Active Directory KDS Root Key\*\*: A Key Distribution Service (KDS) Root Key must be generated once in the forest before any gMSA can be created.

#### ## Implementation Steps

### Option A: Active Directory Management Console Configuration (Preferred)

gMSAs are primarily created and managed using administrative consoles or PowerShell.

1. Open **Active Directory Users and Computers** ( `dsa.msc` ).
2. Verify the presence of the default **Managed Service Accounts** container.
3. Because gMSA creation requires AD schema and principal mapping, PowerShell is the primary method used to initialize and link the account to the host server. Follow the steps in Option B.
4. Once created, configure the target service (e.g., in Services Console `services.msc`):
  - Set **Log On As to This account**.
  - Enter the name of the gMSA with a trailing dollar sign (e.g., `domain\gmsa-sqlservice$` ).
  - Clear the Password fields and click **OK**.
  - Restart the service.

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use the following PowerShell script to initialize the KDS root key (if not already done) and create a gMSA.

[Download Script: Set-gMSAServiceAccount.ps1](#)

```
Set-gMSAServiceAccount.ps1
Description: Generates the KDS root key and registers a new gMSA.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Implement Group Managed Service Accounts..." -ForegroundColor Cyan

1. Initialize KDS Root Key (Required once in the forest)
In standard setups, there is a 10-hour delay for propagation.
-EffectiveImmediately is used for lab configurations.
try {
 Add-KdsRootKey -EffectiveImmediately -ErrorAction SilentlyContinue
 Write-Host "[+] KDS Root Key creation initiated/verified." -ForegroundColor Green
} catch {
 Write-Warning "Could not configure KDS Root Key. It may already exist."
}

2. Create the gMSA
$gMSAName = "gmsa-sqlservice"
$existingMSA = Get-ADServiceAccount -Filter "Name -eq '$gMSAName'"

if (-not $existingMSA) {
 # Specify the name, DNS, and which principals (member servers running the service) can retrieve the password.
 # CRITICAL: Do NOT allow user accounts or groups containing users (Like Domain Admins or Schema Admins)
 # to retrieve the password. Only allow the specific computer account(s) hosting the service.
 $targetHostComputer = "SQLServerHost$"

 New-ADServiceAccount -Name $gMSAName `
 -DNSTargetName "$gMSAName.domain.local" `
 -ManagedPasswordIntervalInDays 30 `
 -PrincipalsAllowedToRetrieveManagedPassword $targetHostComputer

 Write-Host "[+] gMSA '$gMSAName' created successfully." -ForegroundColor Green
} else {
 Write-Host "[-] gMSA '$gMSAName' already exists." -ForegroundColor Yellow
}
}
```

To audit registered Managed Service Accounts: [Download Script: Get-gMSAStatus.ps1](#)

```

Get-gMSAStatus.ps1
Description: Lists all registered gMSAs and audits password retrieval delegation permissions.

Import-Module ActiveDirectory

Write-Host "--- Auditing Group Managed Service Accounts ---" -ForegroundColor Cyan

$gMSAs = Get-ADServiceAccount -Filter * -Properties Name, DNSHostName, Enabled, PrincipalsAllowedToRetrieveManagedPassword

if ($gMSAs) {
 foreach ($sa in $gMSAs) {
 $nonCompliant = $false
 Write-Host "[*] gMSA Account: $($sa.Name)" -ForegroundColor White
 Write-Host " - DNS Name: $($sa.DNSHostName)" -ForegroundColor White
 Write-Host " - Enabled: $($sa.Enabled)" -ForegroundColor White

 $principals = $sa.PrincipalsAllowedToRetrieveManagedPassword
 if ($null -ne $principals) {
 Write-Host " - Principals Allowed to Retrieve Password:" -ForegroundColor White
 foreach ($p in $principals) {
 # Get the AD Object to verify class and name
 $adObj = Get-ADObject -Identity $p.DistinguishedName -Properties ObjectClass, Name
 if ($null -ne $adObj) {
 $objType = $adObj.ObjectClass
 $name = $adObj.Name

 if ($objType -eq "user") {
 Write-Host " [-] WARNING: User account '$($name)' is explicitly allowed to retrieve password (HIGH RIS
K)" -ForegroundColor Red
 $nonCompliant = $true
 } elseif ($objType -eq "group") {
 Write-Host " [-] Group: '$($name)'" -ForegroundColor Yellow

 # Recursively resolve members to check for users
 $members = Get-ADGroupMember -Identity $p.DistinguishedName -Recursive
 $userMembers = $members | Where-Object { $_.objectClass -eq "user" }

 if ($userMembers) {
 $userNames = @()
 foreach ($user in $userMembers) {
 $userNames += $user.Name
 }
 $usersList = $userNames -join ", "
 Write-Host " [-] WARNING: Group '$($name)' contains human user accounts: $($usersList) (HIGH RIS
K)" -ForegroundColor Red
 $nonCompliant = $true
 } else {
 Write-Host " [+] Group contains only computer/service accounts." -ForegroundColor Green
 }
 } else {
 Write-Host " [+] Computer/Host: '$($name)'" -ForegroundColor Green
 }
 }
 }
 }
 }
} else {
 Write-Host " [-] WARNING: No principals allowed to retrieve password (gMSA will not function)." -ForegroundColor Yell
ow
 $nonCompliant = $true
}

if ($nonCompliant) {
 Write-Host " [-] STATUS: NON-COMPLIANT (Insecure password retrieval delegation)" -ForegroundColor Red
} else {
 Write-Host " [+] STATUS: COMPLIANT" -ForegroundColor Green
}
Write-Host ""
}
} else {
 Write-Host "[-] No Group Managed Service Accounts found in the Active Directory domain." -ForegroundColor Yellow
}
}

```

Benchmark\*\*: CIS Microsoft Windows Server Benchmark - Section on Managed Service Accounts \* \*\*Microsoft Security Guidance\*\*: Group Managed Service Accounts Overview \* \*\*Trimarc ADSecurity\*\*: [Attacking Active Directory Group Managed Service Accounts (GMSAs)] (<https://adsecurity.org/?p=4367>)



**# [REQ-ID-004] Restrict Kerberos Delegation**

**## Target Scope** \* **Applicable Systems**: Domain Controllers, Member Servers \* **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022

**## Implementation Details** \* **Priority**: High \* **GPO Path / Registry Location**: Active Directory Object Attributes ('userAccountControl' flags: 'ADS\_UF\_TRUSTED\_FOR\_DELEGATION' = '0x80000')

**## Rationale** Kerberos delegation allows a service to impersonate a user to access downstream resources on behalf of that user. In **Unconstrained Delegation**, when a user authenticates to a service, the user's Ticket Granting Ticket (TGT) is sent to the service server and stored in LSASS memory. If an attacker compromises that service server, they can extract the cached TGTs of all users who have authenticated to it (including Domain Admins) and impersonate them across the entire domain.

To prevent this critical privilege escalation path, **Unconstrained Delegation must be banned entirely**. Any required delegation should be restricted to **Constrained Delegation** or **Resource-Based Constrained Delegation (RBCD)**, which specify exactly which target services can receive delegated credentials.

**## Legacy Impact & Compatibility** \* **Application Functionality**: Disabling unconstrained delegation on legacy servers might break multi-tier applications (e.g., Web Frontend → SQL Backend) if they have not been configured for constrained delegation or RBCD. \* **Transition Plan**: Before disabling unconstrained delegation, identify the specific Service Principal Names (SPNs) and destination services required, and configure Constrained Delegation (S4U2Proxy) to allow only those pathways.

**## Implementation Steps****### Option A: Active Directory Users and Computers Console Configuration (Preferred)**

1. Open **Active Directory Users and Computers** ( `dsa.msc` ).
2. Locate the computer or user account that has unconstrained delegation enabled.
3. Right-click the object and select **Properties**.
4. Go to the **Delegation** tab.
5. Select **Do not trust this computer for delegation** to disable unconstrained delegation.
6. Alternatively, to configure Constrained Delegation:
  - Select **Trust this computer for delegation to specified services only**.
  - Select **Use Kerberos only** or **Use any authentication protocol** (S4U2Self/Protocol Transition).
  - Click **Add** to specify the target services.
7. Click **Apply** and then **OK**.

**### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)**

Use the following PowerShell script to audit and disable unconstrained delegation on all computers and users.

[Download Script: Set-RestrictDelegation.ps1](#)

```
Set-RestrictDelegation.ps1
Description: Disables unconstrained delegation on computer and user accounts.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Restrict Kerberos Delegation..." -ForegroundColor Cyan

Find all computer accounts with Unconstrained Delegation
$unconstrainedComputers = Get-ADComputer -Filter {TrustedForDelegation -eq $true}
foreach ($comp in $unconstrainedComputers) {
 Write-Host "[*] Disabling Unconstrained Delegation on Computer: $($comp.SamAccountName)" -ForegroundColor Gray
 Set-ADComputer -Identity $comp -TrustedForDelegation $false
}

Find all user accounts with Unconstrained Delegation
$unconstrainedUsers = Get-ADUser -Filter {TrustedForDelegation -eq $true}
foreach ($user in $unconstrainedUsers) {
 Write-Host "[*] Disabling Unconstrained Delegation on User: $($user.SamAccountName)" -ForegroundColor Gray
 Set-ADUser -Identity $user -TrustedForDelegation $false
}

Write-Host "Unconstrained delegation has been disabled on all identified accounts." -ForegroundColor Green
```

To audit delegation settings in the domain: [Download Script: Get-KerberosDelegationStatus.ps1](#)

```
Get-KerberosDelegationStatus.ps1
Description: Audits accounts with unconstrained delegation in the Active Directory domain.

Import-Module ActiveDirectory

Write-Host "--- Auditing Kerberos Delegation Settings ---" -ForegroundColor Cyan

$unconstrainedComputers = Get-ADComputer -Filter {TrustedForDelegation -eq $true}
$unconstrainedUsers = Get-ADUser -Filter {TrustedForDelegation -eq $true}

$totalUnconstrained = $unconstrainedComputers.Count + $unconstrainedUsers.Count

if ($totalUnconstrained -eq 0) {
 Write-Host "[+] Secure: No accounts found with Unconstrained Delegation." -ForegroundColor Green
} else {
 foreach ($comp in $unconstrainedComputers) {
 Write-Host "[!] VULNERABLE: Computer with Unconstrained Delegation: $($comp.SamAccountName)" -ForegroundColor Red
 }
 foreach ($user in $unconstrainedUsers) {
 Write-Host "[!] VULNERABLE: User with Unconstrained Delegation: $($user.SamAccountName)" -ForegroundColor Red
 }
}
```

---

**## Sources & Compliance References** \* **\*\*ANSI AD Hardening Guide\*\***: Recommendation R15 (Unconstrained Delegation ban) and R16 (Restricting Constrained Delegation) \* **\*\*CIS Benchmark\*\***: CIS Microsoft Windows Server Benchmark - Section on Kerberos Delegation \* **\*\*Microsoft Security Guidance\*\***: Kerberos Delegation Overview and Security Risks

# [REQ-ID-005] Configure and Populate Protected Users Group

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers, Tier 2 Clients \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10, Windows 11

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: Active Directory Built-in Group: `CN=Protected Users,CN=Users,DC=[Domain]`

## Rationale Standard administrative accounts are highly vulnerable to credential harvesting attacks. If a Domain Admin or other high-privilege account authenticates to a compromised workstation or member server, their credentials (passwords, Kerberos TGTs, NTLM hashes) remain cached in the Local Security Authority Subsystem Service (LSASS) memory. Attackers can extract these credentials using tools like Mimikatz to escalate privileges or move laterally.

The **Protected Users** security group (introduced in Windows Server 2012 R2) enforces non-configurable, highly secure authentication restrictions on its members. These protections include:

1. **No NTLM caching:** NTLM password hashes are not cached locally, and members cannot authenticate via NTLM.
2. **Short Kerberos TGT lifetimes:** Ticket Granting Tickets (TGTs) are limited to 4 hours and cannot be renewed beyond that.
3. **No weak encryption:** Members cannot use DES or RC4 encryption for Kerberos pre-authentication.
4. **No CredSSP or WDigest caching:** Cleartext credentials are never cached by the local system.
5. **No delegation:** Kerberos delegation (constrained or unconstrained) is blocked for accounts in this group.

Placing Tier 0 and Tier 1 administrative accounts into the Protected Users group significantly reduces the threat of credential harvesting.

## Legacy Impact & Compatibility \* \*\*NTLM Authentication Blocked\*\*: Any application or administrative process that authenticates using a Protected User account via NTLM (instead of Kerberos) will fail. Ensure DNS resolution and Kerberos pathways are fully functional. \* \*\*Short Session Lifetimes\*\*: Administrative sessions will require re-authentication after 4 hours due to the non-configurable TGT limit. \* \*\*No Delegation\*\*: If an administrator needs to manage services that rely on delegation, secondary non-delegated accounts must be used, or delegation must be re-architected.

#### ## Implementation Steps

### Option A: Active Directory Users and Computers Console Configuration (Preferred)

1. Open **Active Directory Users and Computers** ( `dsa.msc` ).
2. Navigate to the **Users** container (or where the built-in groups are located).
3. Double-click the **Protected Users** security group.
4. Click the **Members** tab.
5. Click **Add**.
6. Type the names of the Tier 0 and Tier 1 administrative accounts (e.g., `admin-t0-user` ) and click **OK**.
7. Click **Apply** and then **OK**.
8. Ask the administrators to sign out and sign back in for the security group memberships to take effect.

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script to add administrative accounts to the Protected Users security group using PowerShell.

[Download Script: Set-ProtectedUsers.ps1](#)

```
Set-ProtectedUsers.ps1
Description: Adds privileged accounts to the Protected Users group.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Populate Protected Users Group..." -ForegroundColor Cyan

$GroupName = "Protected Users"
$TargetAdmins = @("admin-t0-user", "admin-t1-user")

foreach ($Admin in $TargetAdmins) {
 $User = Get-ADUser -Filter "SamAccountName -eq '$Admin'"

 if ($User) {
 # Check if already a member
 $isMember = Get-ADGroupMember -Identity $GroupName | Where-Object { $_.SamAccountName -eq $Admin }

 if (-not $isMember) {
 Add-ADGroupMember -Identity $GroupName -Members $User
 Write-Host "[+] Added $Admin to Protected Users group." -ForegroundColor Green
 } else {
 Write-Host "[-] User $Admin is already a member of Protected Users group." -ForegroundColor Yellow
 }
 } else {
 Write-Warning "User '$Admin' not found in Active Directory."
 }
}
}
```

To audit the members of the Protected Users group: [Download Script: Get-ProtectedUsersStatus.ps1](#)

```
Get-ProtectedUsersStatus.ps1
Description: Lists all members of the Protected Users security group.

Import-Module ActiveDirectory

Write-Host "--- Auditing Protected Users Group Members ---" -ForegroundColor Cyan

$GroupName = "Protected Users"
$Members = Get-ADGroupMember -Identity $GroupName -ErrorAction SilentlyContinue

if ($Members) {
 Write-Host "[+] Members of the Protected Users group:" -ForegroundColor Green
 foreach ($Member in $Members) {
 Write-Host " - $($Member.SamAccountName) (Type: $($Member.objectClass))" -ForegroundColor White
 }
} else {
 Write-Host "[!] VULNERABLE: No members found in the Protected Users group. Administrative accounts may be unprotected." -ForegroundColor Red
}
}
```

---

## Sources & Compliance References \* **ANSI AD Hardening Guide**: Recommendation R14 (Use of Protected Users group) \* **CIS Benchmark**: CIS Microsoft Windows Server Benchmark - Section on Account Lockout and Protected Groups \* **Microsoft Security Guidance**: Protected Users Security Group Technical Reference

# [REQ-ID-006] Rename and Disable Default Administrator and Guest Accounts

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers, Tier 2 Clients \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10, Windows 11

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` \* \*\*Policies\*\*: \* `Accounts: Administrator account status`: `Disabled` (Note: Ensure LAPS or an alternative local admin exists first) \* `Accounts: Guest account status`: `Disabled` \* `Accounts: Rename administrator account`: `[CustomName]` (e.g., `SvcLocalAdmin`) \* `Accounts: Rename guest account`: `[CustomName]` (e.g., `SvcLocalGuest`) \* \*\*Registry Location\*\*: \* Disable Admin: `HKLM\SAM\SAM\Domains\Account\Users\000001F4` (Managed via Local Security Policy / GPO) \* Disable Guest: `HKLM\SAM\SAM\Domains\Account\Users\000001F5` (Managed via Local Security Policy / GPO)

---

## Rationale Active Directory and local Windows environments initialize built-in accounts with fixed Relative Identifiers (RIDs). The default Administrator account always has RID 500, and the Guest account always has RID 501.

Because these accounts are well-known, they are frequent targets for automated brute-force, password guessing, and identity enumeration attacks. In many environments, the built-in local administrator account has the same password across multiple systems, allowing attackers to move laterally if they crack one machine. Renaming these accounts increases the complexity of target identification, while disabling them prevents unauthorized logons entirely. If LAPS is active, it can rotate the password of the local administrator account even when the account is disabled, preserving safe recovery options.

---

## Legacy Impact & Compatibility \* \*\*Script Dependencies\*\*: Older scripts, setup packages, or orchestration tools that rely on the hardcoded username `Administrator` or `Guest` for authentication will fail. Update scripts to query by SID (e.g., `S-1-5-21-...-500`) or target the updated username. \* \*\*Safe Mode Access\*\*: If the local Administrator account is disabled and no other administrative account is functional, Safe Mode might limit recovery options. However, Windows automatically enables the built-in Administrator account in Safe Mode if no other local administrator accounts exist.

---

#### ## Implementation Steps

##### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a management host.
2. Edit the appropriate hardening GPO (e.g., `GPO_Hardening_DomainControllers` or `GPO_Hardening_MemberServers` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options`
4. Configure the following settings:
  - **Policy:** `Accounts: Administrator account status`
    - **Setting:** `Disabled`
  - **Policy:** `Accounts: Guest account status`
    - **Setting:** `Disabled`
  - **Policy:** `Accounts: Rename administrator account`
    - **Setting:** Enter a non-obvious custom name (e.g., `LocalMgmtAdmin` ).
  - **Policy:** `Accounts: Rename guest account`
    - **Setting:** Enter a non-obvious custom name (e.g., `LocalMgmtGuest` ).
5. Link the GPO to the appropriate Organizational Units.

---

##### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script to disable the local Administrator and Guest accounts locally using PowerShell.

[Download Script: Set-HardenDefaultAccounts.ps1](#)

```
Set-HardenDefaultAccounts.ps1
Description: Disables and renames the built-in local Administrator and Guest accounts locally.

Write-Host "Applying hardening requirement: Rename and Disable Default Accounts..." -ForegroundColor Cyan

1. Disable and rename built-in local Administrator account
$adminAccount = Get-LocalUser | Where-Object { $_.SID -like "*-500" }
if ($adminAccount) {
 if ($adminAccount.Enabled) {
 Disable-LocalUser -Name $adminAccount.Name
 Write-Host "[+] Local Administrator account ($($adminAccount.Name)) disabled." -ForegroundColor Green
 } else {
 Write-Host "[-] Local Administrator account ($($adminAccount.Name)) is already disabled." -ForegroundColor Yellow
 }

 if ($adminAccount.Name -eq "Administrator") {
 Rename-LocalUser -Name "Administrator" -NewName "LocalMgmtAdmin"
 Write-Host "[+] Local Administrator account renamed to LocalMgmtAdmin." -ForegroundColor Green
 } else {
 Write-Host "[-] Local Administrator account is already renamed ($($adminAccount.Name))." -ForegroundColor Yellow
 }
} else {
 Write-Warning "Built-in local Administrator account not found."
}

2. Disable and rename built-in local Guest account
$guestAccount = Get-LocalUser | Where-Object { $_.SID -like "*-501" }
if ($guestAccount) {
 if ($guestAccount.Enabled) {
 Disable-LocalUser -Name $guestAccount.Name
 Write-Host "[+] Local Guest account ($($guestAccount.Name)) disabled." -ForegroundColor Green
 } else {
 Write-Host "[-] Local Guest account ($($guestAccount.Name)) is already disabled." -ForegroundColor Yellow
 }

 if ($guestAccount.Name -eq "Guest") {
 Rename-LocalUser -Name "Guest" -NewName "LocalMgmtGuest"
 Write-Host "[+] Local Guest account renamed to LocalMgmtGuest." -ForegroundColor Green
 } else {
 Write-Host "[-] Local Guest account is already renamed ($($guestAccount.Name))." -ForegroundColor Yellow
 }
} else {
 Write-Warning "Built-in local Guest account not found."
}
}
```

To audit default accounts status locally: [Download Script: Get-DefaultAccountsStatus.ps1](#)

```
Get-DefaultAccountsStatus.ps1
Description: Audits the enabled status of the built-in local Administrator and Guest accounts.

Write-Host "--- Auditing Default Accounts Status ---" -ForegroundColor Cyan

$adminAccount = Get-LocalUser | Where-Object { $_.SID -like "*-500" }
$guestAccount = Get-LocalUser | Where-Object { $_.SID -like "*-501" }

if ($adminAccount) {
 $adminColor = if ($adminAccount.Enabled) { "Red" } else { "Green" }
 Write-Host " - Local Administrator ($($adminAccount.Name)): Enabled = $($adminAccount.Enabled)" -ForegroundColor $adminColor
}

if ($guestAccount) {
 $guestColor = if ($guestAccount.Enabled) { "Red" } else { "Green" }
 Write-Host " - Local Guest ($($guestAccount.Name)): Enabled = $($guestAccount.Enabled)" -ForegroundColor $guestColor
}
}
```

---

## Sources & Compliance References \* \*\*ANSI AD Hardening Guide\*\*: Section on Default accounts and local user restriction \* \*\*CIS

Benchmark\*\*: CIS Microsoft Windows Server Benchmark - Section 2.3.1.1 (Accounts: Administrator account status) and Section 2.3.1.2 (Accounts: Guest account status) \* \*\*Microsoft Security Guidance\*\*: Securing Built-in Administrator Accounts in Windows

# [REQ-ID-007] Restrict Interactive Logons for Service Accounts

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers, Tier 2 Clients \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10, Windows 11

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` \* \*\*Policies\*\*: \* `Deny log on locally`: Add Service Accounts security group (e.g., `Grp\_ServiceAccounts`) \* `Deny log on through Remote Desktop Services`: Add Service Accounts security group (e.g., `Grp\_ServiceAccounts`)

---

## Rationale Service accounts are frequent targets of brute-force and Kerberoasting attacks. If an adversary successfully cracks a service account's password offline, their immediate next step is to use those credentials to log on to domain systems to establish a footprint, dump credentials from memory, or perform administrative tasks.

By enforcing User Rights Assignment policies that explicitly deny service accounts the ability to log on locally (at the physical console) or through Remote Desktop Services (RDP), the threat of an interactive domain compromise via service credentials is neutralized. Even if a service account password is compromised, the attacker cannot utilize it to gain an interactive command shell or graphical desktop session on network endpoints or servers.

---

## Legacy Impact & Compatibility \* \*\*Scheduled Tasks\*\*: If service accounts are configured to run local scheduled tasks, they might require "Log on as a batch job" permissions. Make sure not to block "Log on as a batch job" if the service account has legitimate batch requirements. \* \*\*Testing Phase\*\*: Prior to implementing these deny policies in production GPOs, identify all service accounts, group them into a central security group, and verify that none of these accounts are used by administrators to log on to servers manually for management purposes.

---

#### ## Implementation Steps

##### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a management host.
2. Edit the GPO linked to all domain systems (e.g., `GPO_Hardening_DomainMembers` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment`
4. Configure the following settings:
  - **Policy:** `Deny log on locally`
    - **Setting:** Define this policy and click **Add User or Group**. Add the dedicated security group containing all service accounts (e.g., `domain\Grp_ServiceAccounts` or `domain\Domain Users` if restricting specific sub-groups).
  - **Policy:** `Deny log on through Remote Desktop Services`
    - **Setting:** Define this policy and click **Add User or Group**. Add the dedicated security group containing all service accounts.
5. Click **Apply** and then **OK**.
6. Force a policy update on target servers ( `gpupdate /force` ).

---

##### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Since User Rights Assignment is typically controlled by GPO or local security policy database ( `secedit.sdb` ), local changes can be made using the `secedit` command-line utility.

[Download Script: Set-DenyServiceLogons.ps1](#)



```

Set-DenyServiceLogons.ps1
Description: Configures local security database to deny interactive logons for a service account group.

Write-Host "Applying hardening requirement: Restrict Interactive Logons for Service Accounts..." -ForegroundColor Cyan

$SecDb = "$($env:temp)\localpolicy.sdb"
$SecCfg = "$($env:temp)\localpolicy.inf"
$GroupName = "Grp_ServiceAccounts" # Replace with the target security group name

Export current security policy
secedit /export /cfg $SecCfg /quiet

Read configuration file
$cfgContent = Get-Content -Path $SecCfg

Define logon rights lines
$DenyLocalLine = "SeDenyInteractiveLogonRight = $GroupName"
$DenyRdpLine = "SeDenyRemoteInteractiveLogonRight = $GroupName"

Check and update policy lines
$hasDenyLocal = $false
$hasDenyRdp = $false

$newCfg = New-Object System.Collections.Generic.List[string]

foreach ($line in $cfgContent) {
 if ($line -like "SeDenyInteractiveLogonRight*") {
 if ($line -notlike "*$GroupName*") {
 $line = "$($line), $GroupName"
 }
 $hasDenyLocal = $true
 }
 if ($line -like "SeDenyRemoteInteractiveLogonRight*") {
 if ($line -notlike "*$GroupName*") {
 $line = "$($line), $GroupName"
 }
 $hasDenyRdp = $true
 }
 $newCfg.Add($line) | Out-Null
}

if (-not $hasDenyLocal) {
 # Add to [Privilege Rights] section
 $privIndex = $newCfg.IndexOf("[Privilege Rights]")
 if ($privIndex -ge 0) {
 $newCfg.Insert($privIndex + 1, $DenyLocalLine)
 }
}

if (-not $hasDenyRdp) {
 $privIndex = $newCfg.IndexOf("[Privilege Rights]")
 if ($privIndex -ge 0) {
 $newCfg.Insert($privIndex + 1, $DenyRdpLine)
 }
}

Save updated configuration
$newCfg | Set-Content -Path $SecCfg

Configure local security policy
secedit /configure /db $SecDb /cfg $SecCfg /areas USER_RIGHTS /quiet

Cleanup temporary files
Remove-Item -Path $SecCfg -Force
Remove-Item -Path $SecDb -Force

Write-Host "Local security policy updated: Deny log on locally/RDP applied to group $GroupName." -ForegroundColor Green

```

To audit local User Rights Assignment settings: [Download Script: Get-DenyServiceLogonsStatus.ps1](#)

```
Get-DenyServiceLogonsStatus.ps1
Description: Audits the Deny logon rights configurations locally.

Write-Host "--- Auditing Deny Logon Rights ---" -ForegroundColor Cyan

$SecCfg = "$(env:temp)\auditpolicy.inf"
secedit /export /cfg $SecCfg /quiet

$cfgContent = Get-Content -Path $SecCfg
$denyLocal = $cfgContent | Where-Object { $_ -like "SeDenyInteractiveLogonRight*" }
$denyRdp = $cfgContent | Where-Object { $_ -like "SeDenyRemoteInteractiveLogonRight*" }

Write-Host "[+] Local Deny Logon Rights settings:" -ForegroundColor Green
if ($denyLocal) {
 Write-Host " - $denyLocal" -ForegroundColor White
} else {
 Write-Host " - SeDenyInteractiveLogonRight is not configured." -ForegroundColor Yellow
}

if ($denyRdp) {
 Write-Host " - $denyRdp" -ForegroundColor White
} else {
 Write-Host " - SeDenyRemoteInteractiveLogonRight is not configured." -ForegroundColor Yellow
}

Remove-Item -Path $SecCfg -Force
```

---

**## Sources & Compliance References** \* **ANSI AD Hardening Guide**: Recommendation on limiting administrative and service account interactive logons \* **CIS Benchmark**: CIS Microsoft Windows Server Benchmark - Section 2.2.14 (Deny log on locally) and Section 2.2.16 (Deny log on through Remote Desktop Services) \* **Microsoft Security Guidance**: User Rights Assignment Policies and Service Account Security

# [REQ-ID-008] Enforce User and Service Account Kerberos Encryption (AES-Only)

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: Active Directory Account Attribute: `msDS-SupportedEncryptionTypes` (Set to `24`)

## Rationale In Active Directory, even when Group Policies restrict Kerberos encryption algorithms on domain members, individual user and service accounts can override these restrictions during authentication negotiation. If an account has obsolete encryption types enabled (e.g., RC4 or DES) or has the `msDS-SupportedEncryptionTypes` attribute set to `0` (default, which defaults to domain controllers' allowed options), the Key Distribution Center (KDC) may issue Service tickets (TGS) using the RC4 algorithm.

Because RC4 utilizes weaker, legacy cryptography, tickets encrypted using RC4 can be easily extracted and cracked offline (Kerberoasting) by adversaries. Explicitly configuring the `msDS-SupportedEncryptionTypes` attribute to `24` (AES128 = 8 + AES256 = 16) on Active Directory accounts ensures that the KDC only negotiates AES encryption types, securing the authentication credentials against offline brute-forcing and ticket forgery.

## Legacy Impact & Compatibility \* \*\*Client and Service Compatibility\*\*: Host systems running services under these accounts, as well as the clients connecting to them, must support AES Kerberos encryption. Windows 7/Server 2008 and newer support AES natively. \* \*\*Pre-remediation Audit\*\*: Ensure that any legacy non-Windows servers (e.g., older UNIX/Linux systems running Samba or Java applications) are verified to support AES Kerberos encryption before enforcing this setting.

#### ## Implementation Steps

### Option A: Active Directory Users and Computers Attribute Editor (Preferred)

1. Open **Active Directory Users and Computers** ( `dsa.msc` ).
2. Select **View** from the menu bar and check **Advanced Features**.
3. Locate the target user or service account.
4. Right-click the object and select **Properties**.
5. Navigate to the **Attribute Editor** tab.
6. Scroll down and double-click the `msDS-SupportedEncryptionTypes` attribute.
7. Set the value to `24` (Decimal) and click **OK**.
8. Go to the **Account** tab and verify that under **Account options**, `This account supports Kerberos AES 128 bit encryption` and `This account supports Kerberos AES 256 bit encryption` are checked, while RC4/DES are not forced.
9. Click **Apply** and then **OK**.

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script to enforce AES-only encryption on active user accounts in the domain.

[Download Script: Set-AccountAESEncryption.ps1](#)

```
Set-AccountAESEncryption.ps1
Description: Configures the msDS-SupportedEncryptionTypes attribute to AES-only (24) on active user accounts.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Enforce AES-Only Kerberos Encryption on Accounts..." -ForegroundColor Cyan

24 represents AES128 (8) + AES256 (16)
$AESValue = 24
$TargetUsers = Get-ADUser -Filter {Enabled -eq $true}

foreach ($User in $TargetUsers) {
 # Retrieve current attribute value
 $currUser = Get-ADUser -Identity $User -Properties msDS-SupportedEncryptionTypes
 $currVal = $currUser."msDS-SupportedEncryptionTypes"

 if ($currVal -ne $AESValue) {
 Write-Host "[*] Enforcing AES encryption on account: $($User.SamAccountName)" -ForegroundColor Gray
 Set-ADUser -Identity $User -Replace @{msDS-SupportedEncryptionTypes = $AESValue}
 }
}

Write-Host "AES encryption has been successfully enforced on active accounts." -ForegroundColor Green
```

To audit account Kerberos encryption configuration: [Download Script: Get-AccountEncryptionStatus.ps1](#)

```
Get-AccountEncryptionStatus.ps1
Description: Identifies accounts that do not have msDS-SupportedEncryptionTypes set to 24 (AES-only).

Import-Module ActiveDirectory

Write-Host "--- Auditing Account Kerberos Encryption Configuration ---" -ForegroundColor Cyan

$VulnerableAccounts = Get-ADUser -Filter {Enabled -eq $true} -Properties msDS-SupportedEncryptionTypes | Where-Object { $_."msDS-Sup
portedEncryptionTypes" -ne 24 }

if ($VulnerableAccounts) {
 Write-Host "[!] Accounts not configured for AES-only (msDS-SupportedEncryptionTypes ≠ 24):" -ForegroundColor Red
 foreach ($Acct in $VulnerableAccounts) {
 $Val = $Acct."msDS-SupportedEncryptionTypes"
 if ($null -eq $Val) { $Val = "Not Set (0)" }
 Write-Host " - $($Acct.SamAccountName) | Value: $Val" -ForegroundColor White
 }
} else {
 Write-Host "[+] Secure: All active accounts are configured for AES-only Kerberos encryption." -ForegroundColor Green
}
```

---

## Sources & Compliance References \* **\*\*ANSSI AD Hardening Guide\*\***: Recommendation R13 (Disabling obsolete encryption algorithms) \*  
**\*\*CIS Benchmark\*\***: CIS Microsoft Windows Server Benchmark - Section on Kerberos Encryption Strength \* **\*\*Microsoft Security Guidance\*\***:  
 Decrypting the Selection of Supported Kerberos Encryption Types

# [REQ-ID-009] Enforce Kerberos Pre-Authentication

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers (Active Directory User Accounts) \* \*\*Operating Systems\*\*: Windows Server 2016 and above

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: Active Directory User Account Control Attribute (flag 'DONOTREQ\_PREAUTH' / 0x400000)

---

## Rationale Kerberos Pre-Authentication serves as the primary line of defense against AS-REP Roasting. AS-REP Roasting is a credential theft technique where attackers target accounts that do not require Kerberos pre-authentication.

Without pre-authentication:

1. **Unauthenticated Requesting:** The Key Distribution Center (KDC) will issue a Ticket Granting Ticket (TGT) encrypted with the user's secret key (derived from their password) to any client that requests it, without requiring the client to authenticate or prove identity first.
2. **Offline Password Cracking:** An attacker can request a TGT for a target user, intercept the KDC's response (AS-REP payload), and take the encrypted data offline. They can then perform brute-force or dictionary attacks to crack the password hash without triggering account lockout policies or generating logon failure logs.

By enforcing pre-authentication, the KDC requires the client to encrypt a timestamp using their password hash before issuing the TGT. This proves the client possesses the password, preventing attackers from retrieving the encrypted AS-REP token for offline cracking.

---

## Legacy Impact & Compatibility \* \*\*Authentication Failure\*\*: Accounts linked to very old or poorly designed application integrations, legacy mainframes, or third-party appliances that do not support Kerberos pre-authentication will fail to authenticate. These service integrations must be upgraded to support standard Kerberos pre-authentication or transitioned to secure modern service account models (like gMSAs). \* \*\*Initial Audit\*\*: Administrators must audit the environment to locate and analyze why any accounts have pre-authentication disabled prior to enforcing the setting.

---

## Implementation Steps

### Option A: Active Directory Users and Computers (Preferred)

1. Log on to a Domain Controller or administrative workstation with **Account Operators** or **Domain Admins** credentials.
2. Open **Active Directory Users and Computers** ( [dsa.msc](#) ).
3. Locate the target user account, right-click it, and select **Properties**.
4. Navigate to the **Account** tab.
5. In the **Account options** list, scroll down and ensure that the checkbox for **Do not require Kerberos preauthentication** is **unchecked** (disabled).
6. Click **Apply** and **OK**.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts to audit and remediate accounts in the forest.

#### 1. Local AD Audit (Audit-KerberosPreAuth.ps1)

[Download Script: Audit-KerberosPreAuth.ps1](#)

```
Audit-KerberosPreAuth.ps1
Description: Audits active user accounts to find any with pre-authentication disabled.

Import-Module ActiveDirectory

Write-Host "--- Auditing Kerberos Pre-Authentication Status ---" -ForegroundColor Cyan

try {
 # Search for enabled accounts with DONOTREQ_PREAUTH (0x400000) active
 $VulnerableAccounts = Get-ADUser -Filter "DoesNotRequirePreAuth -eq '$true'" -Properties DoesNotRequirePreAuth, Enabled | Where-Object { $_.Enabled -eq $true }

 if ($VulnerableAccounts) {
 Write-Host "`nVULNERABLE: Found $($VulnerableAccounts.Count) enabled user account(s) with Kerberos Pre-Authentication disabled:" -ForegroundColor Red
 foreach ($acc in $VulnerableAccounts) {
 Write-Host " - User: $($acc.SamAccountName) | DN: $($acc.DistinguishedName)" -ForegroundColor White
 }
 } else {
 Write-Host "`nStatus: Compliant. All enabled user accounts require Kerberos Pre-Authentication." -ForegroundColor Green
 }
} catch {
 Write-Host "VULNERABLE: Could not audit accounts. Error: $($_.Exception.Message)" -ForegroundColor Red
}
```

#### #### 2. Local AD Remediation (Set-KerberosPreAuth.ps1)

[Download Script: Set-KerberosPreAuth.ps1](#)

```
Set-KerberosPreAuth.ps1
Description: Enforces Kerberos Pre-Authentication on all active user accounts.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Enforce Kerberos Pre-Authentication..." -ForegroundColor Cyan

try {
 $VulnerableAccounts = Get-ADUser -Filter "DoesNotRequirePreAuth -eq '$true'" -Properties DoesNotRequirePreAuth, Enabled | Where-Object { $_.Enabled -eq $true }

 if ($VulnerableAccounts) {
 Write-Host "[+] Found $($VulnerableAccounts.Count) accounts requiring remediation." -ForegroundColor Yellow
 foreach ($acc in $VulnerableAccounts) {
 Set-ADAccountControl -Identity $acc.SamAccountName -DoesNotRequirePreAuth $false -ErrorAction Stop
 Write-Host " Remediated: $($acc.SamAccountName)" -ForegroundColor Green
 }
 Write-Host "[+] All target accounts successfully remediated." -ForegroundColor Green
 } else {
 Write-Host "[+] No vulnerable accounts found. Pre-Authentication is already enforced." -ForegroundColor Green
 }
} catch {
 Write-Error "Failed to enforce Kerberos Pre-Authentication. Error: $($_.Exception.Message)"
}
```

---

## Sources & Compliance References \* **ANSI AD Hardening Guide**: Technical recommendations regarding Kerberos protocol security. \* **CIS Microsoft Windows Server Benchmark**: User Account Control security baseline configuration. \* **MITRE ATT&CK**: Technique T1558.004 (AS-REP Roasting).

# [REQ-ID-010] Restrict Schema Administrators Group Membership

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 and above

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Restricted Groups`

---

## Rationale The Schema Admins group is one of the most critical security groups within an Active Directory forest. This group controls the underlying structure of the directory database, defining every class of object and every attribute that those objects can possess.

A compromised Schema Admin account poses a massive risk to the forest:

1. **Schema Modifications:** Attackers can modify class definitions, introduce rogue attributes, or insert persistent directory-level backdoors that survive standard OS-level remediation.
2. **Low Operational Frequency:** Schema modifications are extremely rare, typically occurring only during major enterprise software installations (such as Exchange, SCCM) or AD functional level upgrades.

To minimize the attack surface, standard administrative accounts must not have permanent membership in the Schema Admins group. Instead, membership must be granted strictly on a Just-In-Time (JIT) basis and revoked immediately after schema changes are completed. Locking the group membership to empty using a Restricted Groups GPO ensures that any unauthorized or accidental additions are automatically cleared.

---

## Legacy Impact & Compatibility \* \*\*Schema Updates Blocked\*\*: When executing legitimate schema updates, the Restricted Groups GPO will automatically remove installers from the group, causing setup failures. During scheduled upgrades, administrators must temporarily disable the link for this GPO (or modify the policy) and enable it immediately once the upgrade completes. \* \*\*Administrative Training\*\*: Administrators must be trained on JIT procedures for schema changes.

---

#### ## Implementation Steps

##### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Log on to a management workstation or Domain Controller with **Domain Admins** credentials.
  2. Open the **Group Policy Management Console** ( `gpmc.msc` ).
  3. Create a new GPO named `SEC_Forest_RestrictedGroups` and link it to the **Domain Controllers** OU in the forest root domain.
  4. Right-click the GPO and select **Edit** to open the Group Policy Management Editor.
  5. Navigate to: `Computer Configuration > Policies > Windows Settings > Security Settings > Restricted Groups`
  6. Right-click **Restricted Groups** and select **Add Group**.
  7. In the Group box, type or browse for **Schema Admins** and click **OK**.
  8. In the properties dialog for Schema Admins:
    - Leave the list under **Members of this group** completely blank.
    - Leave the list under **This group is a member of** completely blank.
  9. Click **Apply** and **OK**.
  10. The group membership will be automatically checked and cleared on Domain Controllers during Group Policy refresh cycles.
- 

##### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts to audit and clear the group membership.

##### #### 1. Local Audit (Audit-SchemaAdminsGroup.ps1)

[Download Script: Audit-SchemaAdminsGroup.ps1](#)

```
Audit-SchemaAdminsGroup.ps1
Description: Audits the Schema Admins group membership.

Import-Module ActiveDirectory

Write-Host "--- Auditing Schema Admins Group Membership ---" -ForegroundColor Cyan

try {
 $Group = Get-ADGroup -Identity "Schema Admins" -Properties Members -ErrorAction Stop
 $MembersCount = $Group.Members.Count

 if ($MembersCount -gt 0) {
 Write-Host "`nVULNERABLE: Schema Admins group is NOT empty. Found $MembersCount member(s):" -ForegroundColor Red
 foreach ($memberDN in $Group.Members) {
 $memberObj = Get-ADObject -Identity $memberDN -ErrorAction SilentlyContinue
 Write-Host " - Member: $($memberObj.Name) | DN: $memberDN" -ForegroundColor White
 }
 } else {
 Write-Host "`nStatus: Compliant. Schema Admins group is empty." -ForegroundColor Green
 }
} catch {
 Write-Host "VULNERABLE: Could not query Schema Admins group. Error: $($_.Exception.Message)" -ForegroundColor Red
}
```

#### #### 2. Local Remediation (Clear-SchemaAdminsGroup.ps1)

[Download Script: Clear-SchemaAdminsGroup.ps1](#)

```
Clear-SchemaAdminsGroup.ps1
Description: Removes all members from the Schema Admins group.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Clear Schema Admins group membership..." -ForegroundColor Cyan

try {
 $Group = Get-ADGroup -Identity "Schema Admins" -Properties Members -ErrorAction Stop

 if ($Group.Members.Count -gt 0) {
 Write-Host "[+] Found $($Group.Members.Count) members in Schema Admins group." -ForegroundColor Yellow
 foreach ($memberDN in $Group.Members) {
 $memberObj = Get-ADObject -Identity $memberDN
 Remove-ADGroupMember -Identity "Schema Admins" -Members $memberDN -Confirm:$false -ErrorAction Stop
 Write-Host " Removed member: $($memberObj.Name)" -ForegroundColor Green
 }
 Write-Host "[+] Schema Admins group cleared successfully." -ForegroundColor Green
 } else {
 Write-Host "[+] Schema Admins group is already empty." -ForegroundColor Green
 }
} catch {
 Write-Error "Failed to clear Schema Admins group. Error: $($_.Exception.Message)"
}
```

---

**## Sources & Compliance References** \* **\*\*ANSI AD Hardening Guide\*\***: Technical recommendations regarding privileged group membership management. \* **\*\*Microsoft Architecture Guide\*\***: Design recommendations for administrative group hygiene.



# [REQ-ID-011] Enforce Accidental Deletion Protection on Organizational Units

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 and above

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: Active Directory Object Access Control Lists  
(`ProtectedFromAccidentalDeletion` attribute)

## Rationale Organizational Units (OUs) act as the logical containers for structuring users, groups, and computers in Active Directory, and are the targets for linking Group Policy Objects (GPOs).

Enforcing the accidental deletion protection property provides the following security and availability benefits:

1. **Administrative Safeguard:** Drag-and-drop mistakes or batch scripting errors can lead to the deletion of an entire OU hierarchy, causing severe outages and loss of access controls. This feature places a "Deny" Access Control Entry (ACE) for the "Everyone" group on the "Delete" and "Delete Subtree" permissions of the object.
2. **Operational Continuity:** While it does not prevent a malicious administrator from intentionally disabling the setting and deleting the OU, it forces a deliberate, two-step verification process before any destructive actions can be performed.

## Legacy Impact & Compatibility \* \*\*Administrative Operations\*\*: When moving an OU, or renaming/deleting it during structural reorganizations, administrators must manually uncheck the protection box (or disable the property via PowerShell) before performing the action. \* \*\*Scripted Creation\*\*: Any custom PowerShell scripts used to provision new OUs should explicitly set the `ProtectedFromAccidentalDeletion \$true` parameter during creation to ensure consistent compliance.

#### ## Implementation Steps

##### ### Option A: Active Directory Users and Computers (Preferred)

1. Log on to a management workstation or Domain Controller with **Domain Admins** or **Account Operators** credentials.
2. Open **Active Directory Users and Computers** ( [dsa.msc](#) ).
3. In the top menu, click **View** and ensure that **Advanced Features** is checked. This is required to expose the Object tab in properties.
4. Locate the target Organizational Unit, right-click it, and select **Properties**.
5. Navigate to the **Object** tab.
6. Check the box for **Protect object from accidental deletion**.
7. Click **Apply** and **OK**.

##### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts to audit and configure the setting domain-wide.

##### #### 1. Local Audit (Audit-OUAccidentalDeletion.ps1)

[Download Script: Audit-OUAccidentalDeletion.ps1](#)

```
Audit-OUAccidentalDeletion.ps1
Description: Audits all OUs to find any without accidental deletion protection.

Import-Module ActiveDirectory

Write-Host "--- Auditing OU Accidental Deletion Protection ---" -ForegroundColor Cyan

try {
 $UnprotectedOUs = Get-ADOrganizationalUnit -Filter "ProtectedFromAccidentalDeletion -eq '$false'" -ErrorAction Stop

 if ($UnprotectedOUs) {
 Write-Host "`nVULNERABLE: Found $($UnprotectedOUs.Count) Organizational Unit(s) without accidental deletion protection:" -ForegroundColor Red
 foreach ($ou in $UnprotectedOUs) {
 Write-Host " - OU: $($ou.Name) | DN: $($ou.DistinguishedName)" -ForegroundColor White
 }
 } else {
 Write-Host "`nStatus: Compliant. All Organizational Units are protected from accidental deletion." -ForegroundColor Green
 }
} catch {
 Write-Host "VULNERABLE: Could not audit OUs. Error: $($_.Exception.Message)" -ForegroundColor Red
}
```

##### #### 2. Local Remediation (Enforce-OUAccidentalDeletion.ps1)

[Download Script: Enforce-OUAccidentalDeletion.ps1](#)

```
Enforce-OUAccidentalDeletion.ps1
Description: Enables accidental deletion protection on all OUs in the domain.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Enforce OU Accidental Deletion Protection..." -ForegroundColor Cyan

try {
 $UnprotectedOUs = Get-ADOrganizationalUnit -Filter "ProtectedFromAccidentalDeletion -eq '$false'" -ErrorAction Stop

 if ($UnprotectedOUs) {
 Write-Host "[+] Found $($UnprotectedOUs.Count) OUs requiring protection." -ForegroundColor Yellow
 foreach ($ou in $UnprotectedOUs) {
 Set-ADOrganizationalUnit -Identity $ou.DistinguishedName -ProtectedFromAccidentalDeletion $true -ErrorAction Stop
 Write-Host " Protected OU: $($ou.Name)" -ForegroundColor Green
 }
 Write-Host "[+] All Organizational Units are now protected." -ForegroundColor Green
 } else {
 Write-Host "[+] No unprotected OUs found." -ForegroundColor Green
 }
} catch {
 Write-Error "Failed to enable protection on OUs. Error: $($_.Exception.Message)"
}
```

---

## Sources & Compliance References \* \*\*Microsoft Best Practices\*\*: Protecting Organizational Units from Accidental Deletion. \* \*\*ANSI AD Hardening Guide\*\*: Operational integrity and directory maintenance recommendations.

# [REQ-ID-012] Configure Active Directory Authentication Silos and Policies

## Target Scope \* **Applicable Systems**: Domain Controllers, Tier 0 Administration Workstations (PAWs), Tier 0 Administrator Accounts \* **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022

## Implementation Details \* **Priority**: High \* **GPO Path / Registry Location**: \* **Active Directory Path**: `CN=AuthN Silos,CN=Directory Service,CN=Windows NT,CN=Services,CN=Configuration,DC=[Domain]` \* **KDC GPO Policy**: `Computer Configuration\Policies\Administrative Templates\System\KDC\Support for claims, compound authentication and Kerberos armoring` → Enabled (Supported)

## Rationale Standard Active Directory group memberships define access permissions but do not prevent highly privileged accounts from authenticating to lower-trust systems. If an administrator accidentally logs on to a compromised Tier 1/2 machine, their credentials can be stolen.

Authentication Silos and Policies provide cryptographic containment:

1. **Enforces Logon Boundaries**: An Authentication Silo bounds a set of user accounts and computer accounts. Silo members are technically blocked from obtaining Kerberos tickets (TGT/TGS) to authenticate to hosts outside the silo.
2. **Shortens Session Exposure**: Authentication policies can restrict Kerberos Ticket Granting Ticket (TGT) lifetimes for silo members (e.g., down to 120 minutes), minimizing the window of exposure for hijacked sessions.
3. **Cryptographic Validation**: Uses Kerberos claims and Dynamic Access Control (DAC) to dynamically evaluate and enforce authentication paths at the domain controller level.

## Legacy Impact & Compatibility \* **Protected Users Prerequisite**: Users placed in an Authentication Silo must also be members of the **Protected Users** security group. Silos apply only to the Kerberos protocol; if NTLM is not blocked for these users, the silo restrictions can be bypassed using NTLM fallbacks. \* **Domain Functional Level (DFL)**: The AD environment must be at a minimum functional level of Windows Server 2012 R2. \* **Strict Lockout Risk**: If a Tier 0 administrator tries to log on to a PAW that has not been explicitly added to the Tier 0 Silo, their logon attempt will be rejected by the Domain Controller. Administrators must ensure all management hosts are enrolled in the silo before enforcing the policy.

## ## Implementation Steps

#### Option A: Active Directory Administrative Center (ADAC) Configuration

#### 1. Enable KDC Claims and Armoring via Group Policy Before configuring silos, Domain Controllers must be configured to support claims:

1. Open **Group Policy Management** ( `gpmc.msc` ).
2. Edit the GPO linked to the **Domain Controllers** OU (e.g., `GPO_Hardening_DomainControllers` ).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\KDC`
4. Double-click the **Support for claims, compound authentication and Kerberos armoring** policy.
5. Set it to **Enabled**.
6. Under options, select **Supported** (or **Always support**).
7. Save the GPO and run `gpupdate /force` on all Domain Controllers.

#### 2. Create the Authentication Policy and Silo in ADAC 1. Open **Active Directory Administrative Center** ( `dsac.exe` ). 2. In the left pane, click the tree view, expand your domain, and select the **Authentication** container. 3. Right-click **Authentication Policies**, click **New**, and then click **Authentication Policy**: \* **Name**: `TO\_AuthPol` \* Check **Enforce user ticket lifetime restrictions** and set the TGT lifetime value to `120` minutes. 4. Right-click **Authentication Policy Silos**, click **New**, and then click **Authentication Policy Silo**: \* **Name**: `TO\_Silo` \* Select **Enforce the silo policies**: \* Under **Permitted Accounts**, click **Add** to specify the user accounts and computer objects (PAWs and DCs) that belong to Tier 0. \* Under the **User** policy, select `TO\_AuthPol`. Under the **Computer** policy, select `TO\_AuthPol`. 5. Click **OK** to save and apply the Silo.

#### Option B: PowerShell Configuration (Remediation / Non-GPO)

Run the following script block to programmatically define the Authentication Policy, create the Silo, and enroll Tier 0 members.

[Download Script: Set-ADAuthenticationSilo.ps1](#)

```
Set-ADAuthenticationSilo.ps1
Description: Creates a Tier 0 Authentication Policy Silo and assigns accounts.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Configure Active Directory Authentication Silos..." -ForegroundColor Cyan

$PolicyName = "T0_AuthPol"
$SiloName = "T0_Silo"
$UserGroupName = "Grp_Tier0_Admins" # AD group containing Tier 0 admin users
$ComputerGroupName = "Grp_Tier0_PAWs" # AD group containing Tier 0 PAW computers

1. Create the Authentication Policy if it does not exist
$ExistPolicy = Get-ADAuthenticationPolicy -Filter "Name -eq '$PolicyName'" -ErrorAction SilentlyContinue

if (-not $ExistPolicy) {
 # Create policy with 120-minute (2 hour) TGT lifetime
 New-ADAuthenticationPolicy -Name $PolicyName `
 -Description "Authentication Policy for Tier 0 Administrators" `
 -UserTGTLifetimeMins 120 `
 -Enforce $true `
 -ProtectedFromAccidentalDeletion $true `
 -ErrorAction Stop
 Write-Host "[+] Authentication Policy '$PolicyName' created." -ForegroundColor Green
} else {
 Write-Host "[*] Authentication Policy '$PolicyName' already exists." -ForegroundColor Yellow
}

2. Create the Authentication Policy Silo
$ExistSilo = Get-ADAuthenticationPolicySilo -Filter "Name -eq '$SiloName'" -ErrorAction SilentlyContinue

if (-not $ExistSilo) {
 New-ADAuthenticationPolicySilo -Name $SiloName `
 -Description "Authentication Policy Silo for Tier 0 Containment" `
 -UserAuthenticationPolicy $PolicyName `
 -ComputerAuthenticationPolicy $PolicyName `
 -ServiceAuthenticationPolicy $PolicyName `
 -Enforce $true `
 -ProtectedFromAccidentalDeletion $true `
 -ErrorAction Stop
 Write-Host "[+] Authentication Policy Silo '$SiloName' created." -ForegroundColor Green
} else {
 Write-Host "[*] Authentication Policy Silo '$SiloName' already exists." -ForegroundColor Yellow
}

3. Grant Silo Access to Users and Computers
Write-Host "Granting silo access to members of group '$UserGroupName'..." -ForegroundColor White
$AdminUsers = Get-ADGroupMember -Identity $UserGroupName -Recursive | Where-Object { $_.objectClass -eq "user" }
foreach ($User in $AdminUsers) {
 Grant-ADAuthenticationPolicySiloAccess -Identity $SiloName -Account $User.DistinguishedName -ErrorAction SilentlyContinue
 Set-ADAccountAuthenticationPolicySilo -Identity $User.DistinguishedName -AuthenticationPolicySilo $SiloName -ErrorAction SilentlyContinue
}

Write-Host "Granting silo access to members of group '$ComputerGroupName'..." -ForegroundColor White
$PawComputers = Get-ADGroupMember -Identity $ComputerGroupName -Recursive | Where-Object { $_.objectClass -eq "computer" }
foreach ($Comp in $PawComputers) {
 Grant-ADAuthenticationPolicySiloAccess -Identity $SiloName -Account $Comp.DistinguishedName -ErrorAction SilentlyContinue
 Set-ADAccountAuthenticationPolicySilo -Identity $Comp.DistinguishedName -AuthenticationPolicySilo $SiloName -ErrorAction SilentlyContinue
}

4. Grant access to Domain Controllers (writable DCs must be part of the silo)
$DCs = Get-ADDomainController -Filter "IsReadOnly -eq '$false'"
foreach ($DC in $DCs) {
 $DcDN = $DC.ComputerObjectDN
 Grant-ADAuthenticationPolicySiloAccess -Identity $SiloName -Account $DcDN -ErrorAction SilentlyContinue
 Set-ADAccountAuthenticationPolicySilo -Identity $DcDN -AuthenticationPolicySilo $SiloName -ErrorAction SilentlyContinue
}

Write-Host "[+] Authentication Silo membership initialized." -ForegroundColor Green
```

To verify active Authentication Silo status: [Download Script: Get-AuthSiloAuditStatus.ps1](#)

```
Get-AuthSiloAuditStatus.ps1
Description: Queries the active Authentication Silos and lists their configuration settings.

Import-Module ActiveDirectory

Write-Host "--- Auditing Authentication Silos ---" -ForegroundColor Cyan

$Silos = Get-ADAuthenticationPolicySilo -Filter * -Properties *

if ($Silos) {
 foreach ($Silo in $Silos) {
 Write-Host "[+] Silo Name: $($Silo.Name)" -ForegroundColor Green
 Write-Host " - Enforced: $($Silo.Enforce)" -ForegroundColor White
 Write-Host " - User Policy: $($Silo.UserAuthenticationPolicy)" -ForegroundColor White
 Write-Host " - Computer Policy: $($Silo.ComputerAuthenticationPolicy)" -ForegroundColor White
 }
} else {
 Write-Host "[-] No Authentication Policy Silos configured in this domain." -ForegroundColor Yellow
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendations R8, R64, R80, Annexe C (Mise en œuvre d'un silo)  
 \* \*\*ANSSI Remediation of Active Directory Tier 0 Guide\*\*: Section 10.g (Page 40), Section 11 (Page 49) \* \*\*Microsoft Security Guidance\*\*:  
 Authentication Policies and Authentication Policy Silos Overview

# [REQ-ID-013] Clean Up adminCount Attribute Orphans

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Identity Management \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: Active Directory User Object attributes ('adminCount' and Security Descriptor inheritance flag)

## Rationale In Active Directory, when an account is added to a protected group (such as 'Domain Admins', 'Schema Admins', or 'Account Operators'), a forest-wide background thread (the 'AdminSDHolder' task, running on the Domain Controller holding the PDC Emulator role) automatically sets the account's 'adminCount' attribute to '1' and disables security descriptor inheritance. This is done to ensure the account only inherits permissions defined by the secure 'adminSDHolder' template rather than any insecure permissions on the parent Organizational Unit (OU).

A common myth is that Active Directory uses the `adminCount` attribute to determine which accounts are protected. In reality, the `AdminSDHolder` background task evaluates group membership (direct or nested association with a protected group) to trigger protection, not `adminCount`. The `adminCount=1` attribute is merely a metadata flag stamped by the task.

However, if that user is later removed from the protected group:

1. **adminCount Remains Active:** Active Directory does not automatically reset the `adminCount` attribute to `0` or empty, nor does it re-enable security descriptor inheritance on the user object. Although the `AdminSDHolder` task stops protecting the object (it no longer overwrites its DACL), inheritance remains permanently disabled.
2. **Leaves Account Insecurely Unmanaged:** The user object remains orphaned, with inheritance permanently disabled. This blocks future legitimate GPO-based permission updates and can allow persistent, hidden permissions (backdoors) on the object to remain unmitigated.
3. **Breaks Management Consistency:** Security administrators auditing protected accounts will see false positives, as accounts appear to have administrative attributes when they do not have administrative group memberships.

Auditing and resetting these orphan accounts restores proper security inheritance and cleans up directory metadata.

## Legacy Impact & Compatibility \* \*\*Parent OU Access Control\*\*: Re-enabling security inheritance on a user object will immediately apply the Access Control Lists (RLs) of its parent Organizational Unit. Before re-enabling inheritance, ensure that the target user's parent OU is properly secured and does not grant excessive delegation permissions to non-Tier 0 accounts. \* \*\*Service Interruption\*\*: No service interruption is expected, as this is an attribute cleanup task.

## Implementation Steps

#### Option A: Active Directory Administrative Center (ADAC) Manual Modification

1. Open **Active Directory Administrative Center** ( `dsac.exe` ) on a Domain Controller.
2. Navigate to your domain and locate the target orphan user account.
3. Right-click the user account and select **Properties**.
4. Click **Attribute Editor** (ensure Advanced features are active).
5. Locate the `adminCount` attribute and click **Clear** or set the value to `<not set>`.
6. Select the **Security** tab, click **Advanced**, and click **Enable Inheritance**.
7. Click **OK** to save and commit changes.

#### Option B: PowerShell Configuration (Remediation / Non-GPO)

Run the following script to automatically identify all user accounts with `adminCount=1` that are no longer members of any protected AD group, reset the attribute, and re-enable security inheritance.

[Download Script: Cleanup-AdminCountOrphans.ps1](#)

```
Cleanup-AdminCountOrphans.ps1
Description: Resets adminCount and re-enables inheritance on user accounts that are no longer in protected groups.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Clean Up adminCount Attribute Orphans..." -ForegroundColor Cyan

Define the list of built-in protected AD groups (sAMAccountNames)
$ProtectedGroups = @(
 "Administrators",
 "Domain Admins",
 "Enterprise Admins",
 "Schema Admins",
 "Account Operators",
 "Backup Operators",
 "Print Operators",
 "Server Operators",
 "Cert Publishers",
 "Group Policy Creator Owners"
)

Fetch all user objects with adminCount set to 1
Write-Host "Scanning domain for accounts with adminCount = 1..." -ForegroundColor White
$Orphans = Get-ADUser -Filter "adminCount -eq 1" -Properties MemberOf, adminCount

$CleanedCount = 0

foreach ($User in $Orphans) {
 $IsStillProtected = $false

 # Check if the user is currently in any of the protected groups
 foreach ($Group in $ProtectedGroups) {
 $GroupObj = Get-ADGroup -Filter "Name -eq '$Group'" -ErrorAction SilentlyContinue
 if ($null -ne $GroupObj) {
 # Check membership
 $IsMember = Get-ADGroupMember -Identity $GroupObj -Recursive | Where-Object { $_.distinguishedName -eq $User.distinguishedName }
 if ($null -ne $IsMember) {
 $IsStillProtected = $true
 break
 }
 }
 }

 # If the user is no longer in a protected group, perform cleanup
 if (-not $IsStillProtected) {
 Write-Host "[] Found orphan account: $($User.SamAccountName)" -ForegroundColor Yellow

 # 1. Clear the adminCount attribute
 Set-ADUser -Identity $User.distinguishedName -Clear "adminCount" -ErrorAction Stop

 # 2. Re-enable ACL inheritance on the object
 $UserDN = $User.distinguishedName
 $AclPath = "AD:\($UserDN)"
 $Acl = Get-Acl -Path $AclPath

 if ($Acl.AreAccessRulesProtected) {
 # Disable protection, copying existing rules as inherited
 $Acl.SetAccessRuleProtection($false, $true)
 Set-Acl -Path $AclPath -AclObject $Acl -ErrorAction Stop
 Write-Host " - Reset adminCount and enabled security inheritance." -ForegroundColor Green
 } else {
 Write-Host " - Reset adminCount (security inheritance was already enabled)." -ForegroundColor Green
 }

 $CleanedCount++
 }
}

Write-Host "[+] Cleanup complete. Total orphan accounts remediated: $($CleanedCount)." -ForegroundColor Green
```

To audit and list orphan accounts without making changes: [Download Script: Get-AdminCountOrphansAudit.ps1](#)

```
Get-AdminCountOrphansAudit.ps1
Description: Scans the domain and prints all orphan adminCount accounts.

Import-Module ActiveDirectory

Write-Host "--- Auditing adminCount Orphans ---" -ForegroundColor Cyan

$ProtectedGroups = @("Administrators","Domain Admins","Enterprise Admins","Schema Admins","Account Operators","Backup Operators","Print Operators","Server Operators")
$Users = Get-ADUser -Filter "adminCount -eq 1"

$OrphanCount = 0

foreach ($U in $Users) {
 $Member = $false
 foreach ($Grp in $ProtectedGroups) {
 $GrpObj = Get-ADGroup -Filter "Name -eq '$Grp'"
 if ($GrpObj) {
 $Check = Get-ADGroupMember -Identity $GrpObj -Recursive | Where-Object { $_.distinguishedName -eq $U.distinguishedName }
 if ($Check) {
 $Member = $true
 break
 }
 }
 }

 if (-not $Member) {
 Write-Host "[!] Orphan: $($U.SamAccountName) has adminCount=1 but is not in protected groups." -ForegroundColor Yellow
 $OrphanCount++
 }
}

Write-Host "[*] Total adminCount orphans detected: $($OrphanCount)." -ForegroundColor White
```

---

## Sources & Compliance References \* \*\*ANSSI Remediation of Active Directory Tier 0 Guide\*\*\*: Section 6.b (Page 32) \* \*\*ANSSI AD Hardening Guide\*\*\*: Section 3.2.2 (adminSDHolder Context) \* \*\*Microsoft Security Guidance\*\*\*: Active Directory Orphaned adminCount Cleanup Procedures \* \*\*SpecterOps Research\*\*\*: AdminSDHolder: Misconceptions, Misconfigurations, and Myths (October 2025)



# [REQ-ID-014] Renew KDS Root Keys and gMSA Secrets

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Group Managed Service Accounts (gMSAs) \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: Active Directory KDS Configuration container: `CN=Microsoft Key Distribution Service,CN=Services,CN=Configuration,DC=[Domain]`

## Rationale Group Managed Service Accounts (gMSAs) offer secure, automated password management (complex 120-character passwords rotated every 30 days) for services running on domain member systems. The passwords for these accounts are generated by the Key Distribution Service (KDS) running on Domain Controllers. However, the password generation algorithm relies on KDS root keys stored in the AD Configuration partition.

If an attacker compromises or steals the Active Directory database (e.g., via NTDS.dit exfiltration), they obtain the active KDS root keys. Using these keys, the attacker can recalculate the passwords for any gMSA at any time, establishing an invisible, persistent backdoor to any service running under a gMSA.

Therefore:

1. **Interrupts Attacker Persistence:** Generating a new KDS root key and forcing all gMSA accounts to rotate their passwords invalidates any previously compromised password generation seeds.
2. **Harden Service Isolation:** Ensures that service account security boundaries remain intact post-remediation.

## Legacy Impact & Compatibility \* \*\*Replication Sync Delay\*\*: By default, Active Directory enforces a 10-hour delay when adding a new KDS root key. This is done to ensure that all Domain Controllers have successfully replicated the new root key before it is used to generate gMSA passwords. Bypassing this delay during remediation using `-EffectiveImmediately` is possible, but requires verifying that AD replication is functioning properly. \* \*\*Service Downtime\*\*: gMSA password rotation is handled automatically by the host systems. However, triggering a force-reset of the password requires restarting the associated service on the host system to ensure it fetches the new password.

## ## Implementation Steps

### ### Option A: Active Directory PowerShell Configuration (Remediation / Non-GPO)

Because KDS keys and gMSAs do not have standard GPO management interfaces, the creation of root keys and service account password rotation must be executed via PowerShell.

##### 1. Generate a New KDS Root Key Open an elevated PowerShell console on the PDC Emulator Domain Controller and run:

[Download Script: New-KdsKey.ps1](#)

```
New-KdsKey.ps1
Description: Generates a new KDS Root Key.

Import-Module Kds

Write-Host "Creating new KDS Root Key..." -ForegroundColor Cyan

Create new KDS key effective immediately (backdated by 10 hours to bypass replication delay)
$NewKey = Add-KdsRootKey -EffectiveTime ((Get-Date).AddHours(-10)) -ErrorAction Stop

if ($null -ne $NewKey) {
 Write-Host "[+] New KDS Root Key created successfully. Key ID: $NewKey" -ForegroundColor Green
}
```

##### 2. Force gMSA Password Rotation Once the new root key is active, force password rotation for all gMSAs:

[Download Script: Rotate-gMSAPasswords.ps1](#)

```
Rotate-gMSAPasswords.ps1
Description: Forces password rotation for all gMSAs.

Import-Module ActiveDirectory

Write-Host "Locating all gMSAs in the domain..." -ForegroundColor Cyan

$gMSAs = Get-ADServiceAccount -Filter "ObjectClass -eq 'msDS-GroupManagedServiceAccount'"

if ($gMSAs) {
 foreach ($Acct in $gMSAs) {
 Write-Host "[*] Rotating password for gMSA: $($Acct.Name)..." -ForegroundColor White

 # Reset password (forces rotation on the next request by the host computer)
 Reset-ADServiceAccountPassword -Identity $Acct.DistinguishedName -ErrorAction Stop

 Write-Host "[+] Password rotated successfully for $($Acct.Name)." -ForegroundColor Green
 }
} else {
 Write-Host "[-] No Group Managed Service Accounts found." -ForegroundColor Yellow
}
```

### ### Option B: PowerShell Auditing Status

Use this PowerShell script to audit current KDS root keys and gMSA replication status.

[Download Script: Get-KdsAndGmsaAudit.ps1](#)

```
Get-KdsAndGmsaAudit.ps1
Description: Audits active KDS root keys and checks gMSA accounts.

Import-Module ActiveDirectory
Import-Module Kds

Write-Host "--- Auditing KDS Root Keys ---" -ForegroundColor Cyan

$KdsKeys = Get-KdsRootKey -ErrorAction SilentlyContinue

if ($KdsKeys) {
 foreach ($Key in $KdsKeys) {
 Write-Host "[+] KDS Key ID: $($Key.KeyId)" -ForegroundColor Green
 Write-Host " - Created: $($Key.CreateTime)" -ForegroundColor White
 Write-Host " - Effective: $($Key.EffectiveTime)" -ForegroundColor White
 }
} else {
 Write-Host "[-] No KDS Root Keys found in the forest." -ForegroundColor Red
}

Write-Host "--- Auditing gMSA Accounts ---" -ForegroundColor Cyan

$Accounts = Get-ADServiceAccount -Filter * -Properties msDS-ManagedPasswordInterval, msDS-HostSecurityGroupsScope

if ($Accounts) {
 foreach ($Acct in $Accounts) {
 Write-Host "[*] gMSA: $($Acct.Name)" -ForegroundColor White
 Write-Host " - Rotation Interval: $($Acct.'msDS-ManagedPasswordInterval') days" -ForegroundColor Gray
 }
} else {
 Write-Host "[-] No service accounts found." -ForegroundColor Yellow
}
```

## Sources & Compliance References \* \*\*ANSSI Remediation of Active Directory Tier 0 Guide\*\*<sup>2</sup>: Section 4.d (Page 27) \* \*\*ANSSI AD Hardening Guide\*\*<sup>3</sup>: Section 4.14 (Managed Service Accounts) \* \*\*Microsoft Security Guidance\*\*<sup>4</sup>: Group Managed Service Accounts (gMSAs) and KDS Root Key Management

# [REQ-ID-015] Harden Active Directory Certificate Services (ADCS) and PKI

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers (Certification Authorities), Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*AD CS Template Container\*\*: `CN=Certificate Templates,CN=Public Key Services,CN=Services,CN=Configuration,DC=[Domain]` \* \*\*IIS Web Enrollment Server\*\*: CA Web Enrollment IIS virtual directory configuration (`CertSrv`)

## Rationale Active Directory Certificate Services (ADCS) is a built-in Public Key Infrastructure (PKI) solution widely used for issuing certificates for computer and user authentication. However, misconfigured certificate templates and CA web endpoints present severe privilege escalation vectors (collectively referred to as ESC1 through ESC8).

Key vulnerabilities include:

1. **ESC1 (Enrollee Supplies Subject / SAN Exploitation)**: If a certificate template allows the client requesting the certificate to supply the subject name (Subject Alternative Name - SAN) in the enrollment request, and that template allows client authentication, any unprivileged domain user can request a certificate in the name of a Domain Administrator or Domain Controller. Upon receiving the certificate, the attacker can authenticate as that administrator, resulting in instant forest compromise.
2. **ESC8 (IIS Web Enrollment NTLM Relay)**: The default ADCS HTTP Web Enrollment pages ( `/certsrv` ) do not enforce HTTPS and support NTLM authentication without protection. Attackers can coerce NTLM authentication from a Domain Controller (e.g., using printer spooler coercion) and relay that authentication to the CA web enrollment endpoint to request a DC certificate, taking over the domain.

Hardening ADCS templates and endpoints is critical to secure the Tier 0 boundary.

## Legacy Impact & Compatibility \* \*\*Enrollment Disruption\*\*: Disabling template configurations that allow client-specified SANs will block applications (such as third-party firewalls, load balancers, or web servers) that legitimately use this configuration to automatically request certificates with custom SANs. These systems should be migrated to secure enrollment agents or manual enrollment templates with manager approval. \* \*\*Authentication Failures\*\*: Disabling HTTP Web Enrollment ( `/certsrv` ) entirely is recommended. If it must remain active, IIS must be configured to enforce HTTPS and Extended Protection for Authentication (EPA), which will block legacy non-channel-bound clients.

## Implementation Steps

### Option A: Certificate Templates Console Configuration

##### 1. Mitigate ESC1 (Disable Enrollee Supplies Subject) 1. Open the **Certificate Templates Console** ( `certtmpl.msc` ) on the CA server or a management host. 2. Locate the active templates used for authentication (e.g., `User`, `Computer`, or custom templates). 3. Right-click the template and select **Properties**. 4. Select the **Subject Name** tab. 5. Ensure the option **Build from this Active Directory information** is selected. 6. **Do NOT select** the option **Supply in the request**. If a template **must** allow user-supplied subjects (e.g., web server SSL templates), ensure that **Client Authentication** is **not** present in the Extended Key Usage (EKU) list, and enforce manager approval (see below).

##### 2. Enforce Manager Approval on Sensitive Templates 1. In the template properties, select the **Issuance Requirements** tab. 2. Check the box for **CA administrator approval**. 3. Under **Require the following for enrollment**, set the authorized signatures to `1` if an enrollment agent is required. 4. Save the template.

##### 3. Disable or Secure HTTP Web Enrollment (Mitigate ESC8) 1. Log on to the CA server hosting the Web Enrollment role. 2. Open **Internet Information Services (IIS) Manager** ( `inetmgr.exe` ). 3. In the left tree view, navigate to: `Sites\Default Web Site\CertSrv` 4. In the middle pane, double-click **Authentication**. 5. Select **Windows Authentication** and click **Advanced Settings** in the right pane. 6. Set **Extended Protection** to **Required** (or **Accept**). 7. Ensure that the **Default Web Site** binds exclusively to HTTPS (port 443) and redirect all HTTP traffic to HTTPS. \* Ideally, if Web Enrollment is not required, uninstall the "Active Directory Certificate Services Web Enrollment" role entirely via Server Manager.\*

### Option B: PowerShell Configuration (Remediation / Non-GPO)

Run the following script block to audit active certificate templates for vulnerable configurations (ESC1).

[Download Script: Get-ADCSTemplateAudit.ps1](#)

```
Get-ADCS_TemplateAudit.ps1
Description: Audits Active Directory certificate templates for SAN and authentication misconfigurations.

Import-Module ActiveDirectory

Write-Host "--- Auditing AD CS Certificate Templates ---" -ForegroundColor Cyan

$ConfigDN = (Get-ADRootDSE).configurationNamingContext
$TemplatesPath = "LDAP://CN=Certificate Templates,CN=Public Key Services,CN=Services,$($ConfigDN)"
$Searcher = New-Object System.DirectoryServices.DirectorySearcher([ADSI]$TemplatesPath)
$Searcher.Filter = "(objectClass=pkipastructure)"
$Templates = $Searcher.FindAll()

$VulnerableCount = 0

foreach ($Result in $Templates) {
 $Template = $Result.GetDirectoryEntry()
 $TemplateName = $Template.cn.Value

 # Check if enrollee supplies subject name (SAN flag: 0x00010000)
 $NameFlags = $Template.'msPKI-Certificate-Name-Flag'.Value
 $SuppliesSubject = ($NameFlags -band 0x00010000) -eq 0x00010000

 # Check if template is used for Client Authentication (EKU OID: 1.3.6.1.5.5.7.3.2)
 $EKUs = $Template.'pKIExtendedKeyUsage'.Value
 $AllowsClientAuth = $false
 foreach ($Eku in $EKUs) {
 if ($Eku -eq "1.3.6.1.5.5.7.3.2" -or $Eku -eq "1.3.6.1.4.1.311.20.2.2") { # Client Auth or Smartcard Logon
 $AllowsClientAuth = $true
 }
 }

 # Check if manager approval is required (Enrollment flag: 0x00000002)
 $EnrollFlags = $Template.'msPKI-Enrollment-Flag'.Value
 $RequiresApproval = ($EnrollFlags -band 0x00000002) -eq 0x00000002

 if ($SuppliesSubject -and $AllowsClientAuth -and -not $RequiresApproval) {
 Write-Host "[!] VULNERABLE TEMPLATE DETECTED (ESC1): $TemplateName" -ForegroundColor Red
 Write-Host " - Allows Client Authentication" -ForegroundColor White
 Write-Host " - Enrollee supplies Subject/SAN" -ForegroundColor White
 Write-Host " - Requires NO Manager Approval" -ForegroundColor White
 $VulnerableCount++
 }
}

if ($VulnerableCount -eq 0) {
 Write-Host "[+] No vulnerable ESC1 certificate templates found." -ForegroundColor Green
} else {
 Write-Host "[-] Action Required: Resolve the above vulnerable templates immediately." -ForegroundColor Red
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendations R36, R37 (Section 3.3.4) \* \*\*ANSSI Remediation of Active Directory Tier 0 Guide\*\*: Section 9 (Page 35) \* \*\*Microsoft Security Advisory\*\*: KB5014754 (Certificate-based authentication changes) \* \*\*Other Reference\*\*: CVE-2022-26923 (Active Directory Domain Services Privilege Escalation)

# [REQ-ID-016] Configure Logon Screen and Credentials Delegation

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers, Tier 2 Clients. \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Logon screen local users enumeration GPO\*\*: `Computer Configuration\Policies\Administrative Templates\System\Logon\Enumerate local users on domain-joined computers` → Disabled \* \*\*CredSSP Encryption GPO\*\*: `Computer Configuration\Policies\Administrative Templates\System\Credentials Delegation\Encryption Oracle Remediation` → Enabled (Select \*\*Force Updated Clients\*\* in options) \* \*\*Protected Credentials Delegation GPO\*\*: `Computer Configuration\Policies\Administrative Templates\System\Credentials Delegation\Remote host allows delegation of non-exportable credentials` -

> Enabled \* \*\*Registry Location (Logon)\*\*: `HKLM\SOFTWARE\Policies\Microsoft\Windows\System` → `EnumerateLocalUsers` = `0` (REG\_DWORD) \* \*\*Registry Location (CredSSP)\*\*: `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\CredSSP\Parameters` → `AllowEncryptionOracle` = `0` (REG\_DWORD) \* \*\*Registry Location (Credentials Delegation)\*\*: `HKLM\SOFTWARE\Policies\Microsoft\Windows\CredentialsDelegation` → `AllowProtectedCreds` = `1` (REG\_DWORD)

---

## Rationale Securing interactive logons and connection pathways is critical to preventing identity leaks, unauthorized physical user identification, and credential theft during remote administration:

1. **Logon Screen Reconnaissance:** Allowing the local login screen to enumerate local and domain users exposes valid usernames to physical shoulder-surfers or unauthorized operators. Disabling local user enumeration hides username lists at logon.
2. **CredSSP Vulnerabilities (CVE-2018-0886):** The Credential Security Support Provider protocol (CredSSP) had a logical remote code execution flaw. Enforcing Encryption Oracle Remediation in updated mode blocks connections from unpatched clients and servers.
3. **Delegated Credential Extraction:** When users connect to remote hosts, delegating exportable credentials exposes their authentication materials in remote LSASS memory. Forcing the delegation of non-exportable credentials ensures authentication materials cannot be exported by administrative attackers on the remote system.

---

## Legacy Impact & Compatibility \* \*\*CredSSP Connection Failures\*\*: Administrative RDP sessions to legacy, unpatched servers (e.g., Windows Server 2008 / Windows Server 2003) will be blocked if those targets do not support patched CredSSP. These targets must be decommissioned or patched.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

#### 1. Disable Logon Screen Username Enumeration 1. Open the \*\*Group Policy Management Console\*\* (`gpmc.msc`). 2. Create or edit a GPO linked to your computer OUs (e.g., `GPO\_Computer\_Hardening\_Baseline`). 3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Logon` 4. Double-click \*\*Enumerate local users on domain-joined computers\*\*. 5. Set it to \*\*Disabled\*\* and click \*\*OK\*\*.

#### 2. Configure CredSSP and Credentials Delegation 1. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Credentials Delegation` 2. Double-click \*\*Encryption Oracle Remediation\*\*. 3. Set it to \*\*Enabled\*\*, and select \*\*Force Updated Clients\*\* in the options dropdown. Click \*\*OK\*\*. 4. Double-click \*\*Remote host allows delegation of non-exportable credentials\*\*. 5. Set it to \*\*Enabled\*\* and click \*\*OK\*\*.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to apply logon screen and delegation settings to the registry.

[Download Script: Configure-CredentialDelegationAndLogon.ps1](#)

```
Configure-CredentialDelegationAndLogon.ps1
Description: Hardens logon screen user enumeration, CredSSP encryption oracle remediation, and remote host non-exportable credentials delegation.

Write-Host "Applying logon screen and credentials delegation registry controls..." -ForegroundColor Cyan

1. Disable Logon Screen User Enumeration
$SystemPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System"
if (-not (Test-Path $SystemPath)) {
 New-Item -Path $SystemPath -Force | Out-Null
}
Set-ItemProperty -Path $SystemPath -Name "EnumerateLocalUsers" -Value 0 -Type DWord -ErrorAction Stop
Write-Host "[+] Logon screen local user enumeration disabled." -ForegroundColor Green

2. Enforce CredSSP Encryption Oracle Remediation
$CredSspPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\CredSSP\Parameters"
if (-not (Test-Path $CredSspPath)) {
 New-Item -Path $CredSspPath -Force | Out-Null
}
Set-ItemProperty -Path $CredSspPath -Name "AllowEncryptionOracle" -Value 0 -Type DWord -ErrorAction Stop
Write-Host "[+] CredSSP Encryption Oracle Remediation configured to Force Updated Clients." -ForegroundColor Green

3. Remote Host Allows Delegation of Non-Exportable Credentials
$DelegPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CredentialsDelegation"
if (-not (Test-Path $DelegPath)) {
 New-Item -Path $DelegPath -Force | Out-Null
}
Set-ItemProperty -Path $DelegPath -Name "AllowProtectedCreds" -Value 1 -Type DWord -ErrorAction Stop
Write-Host "[+] Delegation of non-exportable credentials enabled." -ForegroundColor Green
```

*To audit these logon screen and delegation settings:*

[Download Script: Get-CredentialDelegationAndLogonStatus.ps1](#)

```

Get-CredentialDelegationAndLogonStatus.ps1
Description: Audits registry configuration of user enumeration, CredSSP, and delegation settings.

Write-Host "--- Auditing Credentials Delegation and Logon Settings ---" -ForegroundColor Cyan

$script:Vulnerable = $false

Helper function to check registry settings
function Confirm-RegValue ($Path, $Name, $Expected) {
 if (Test-Path $Path) {
 $Reg = Get-ItemProperty -Path $Path -ErrorAction SilentlyContinue
 $Val = $Reg.$Name
 if ($Val -eq $Expected) {
 Write-Host " [+] Path $($Path) | $($Name): $Val (Expected: $Expected)" -ForegroundColor Green
 } else {
 Write-Host " [!] MISMATCH: Path $($Path) | $($Name): $Val (Expected: $Expected)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
 } else {
 Write-Host " [!] NOT FOUND: Path $($Path) (Expected: $Name = $Expected)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
}

1. User Enumeration
Confirm-RegValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" "EnumerateLocalUsers" 0

2. CredSSP AllowEncryptionOracle
Confirm-RegValue "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\CredSSP\Parameters" "AllowEncryptionOracle" 0

3. Protected Credentials Delegation
Confirm-RegValue "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CredentialsDelegation" "AllowProtectedCreds" 1

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}

```

## Sources & Compliance References \* \*\*CIS Benchmark\*\*\*: CIS Microsoft Windows Server Benchmark - Section 18.8 (Credentials Delegation)  
 \* \*\*ANSSI AD Hardening Guide\*\*\*: Security guidelines regarding credential delegation and local machine access configuration. \* \*\*Microsoft Security Guidance\*\*\*: Mitigating CredSSP vulnerabilities (CVE-2018-0886)



# [REQ-ID-017] Disable Machine Account Quota

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Domain Environment \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*AD Attribute\*\*: `ms-DS-MachineAccountQuota` (Domain Root object) \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` \* \*\*Policy\*\*: `Add workstations to domain` \* \*\*Registry Location (SecEdit / User Rights Assignment)\*\*: `SeMachineAccountPrivilege`

## Rationale By default, Active Directory grants all authenticated users the ability to add up to 10 computer objects to the domain. This default configuration (defined by the `ms-DS-MachineAccountQuota` attribute on the domain head) poses a severe security risk:

1. **Privilege Escalation & RBCD**: Non-privileged domain users can create rogue computer accounts. An attacker with a compromised domain account can register a computer and use it to exploit Resource-Based Constrained Delegation (RBCD) or Kerberos relay attacks to escalate privileges to Domain Admin.
2. **Credential Harvesting**: Attackers can use rogue computer accounts to request Kerberos tickets, perform Kerberoasting, or execute AS-REP Roasting attacks on accounts mapped to those computer objects.
3. **Bypass of Network Controls**: Allowing unmanaged, non-compliant, or rogue machines to join the domain increases the overall attack surface and permits unauthorized systems to access internal resources.
4. **Name Squatting**: Unauthorized users can register computer accounts with names matching sensitive or prospective servers, hijacking traffic or interfering with legitimate resource provisioning.

Restricting this behavior by setting `ms-DS-MachineAccountQuota` to 0 and removing `Authenticated Users` from the "Add workstations to domain" user right ensures that only authorized administrators can introduce computer accounts to the directory.

## Legacy Impact & Compatibility \* \*\*Workstation Joins\*\*: Standard users will no longer be able to join their own computers to the domain using their standard user accounts. \* \*\*Automated Provisioning\*\*: Automated deployment tools, OS imaging scripts, or deployment servers (such as MDT or SCCM) that rely on generic domain user accounts to join target systems to the domain will fail. \* \*\*Mitigations\*\*: \* \*\*Pre-Staging\*\*: Administrators should pre-create the computer account object in the target Organizational Unit (OU) and grant specific domain users or groups (e.g., workstation technicians) permissions to join the computer to the domain (specifically, write validation to change host name, and reset password rights on the pre-created object). \* \*\*OU Delegation\*\*: Explicitly delegate permissions to create and delete computer objects on specific OUs (e.g., workstations OU) to dedicated IT operations or technician security groups.

#### ## Implementation Steps

##### #### Option A: Active Directory Administrative Center & GPMC (Preferred)

##### Step 1: Set ms-DS-MachineAccountQuota to 0 1. Log on to a Domain Controller or administrative host with **Domain Admins** credentials. 2. Open **Active Directory Users and Computers** (`dsa.msc`). 3. Click **View** in the top menu and ensure **Advanced Features** is checked. 4. Right-click the root domain object (e.g., `domain.local`) and select **Properties**. 5. Navigate to the **Attribute Editor** tab. 6. Scroll down to select the **ms-DS-MachineAccountQuota** attribute and click **Edit**. 7. Change the value to **0** and click **OK**. 8. Click **Apply** and then **OK**.

##### Step 2: Restrict 'Add workstations to domain' GPO 1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host. 2. Edit the **Default Domain Controllers Policy** or another GPO applying to all Domain Controllers. 3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 4. Locate the policy **Add workstations to domain**. 5. Double-click the policy, check **Define these policy settings**, and ensure that only authorized administrative groups (e.g., `Administrators`) are added. Remove **Authenticated Users**. 6. Click **Apply** and then **OK**.

##### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts to audit and remediate these settings.

#### 1. Local Audit (Audit-MachineAccountQuota.ps1)

[Download Script: Audit-MachineAccountQuota.ps1](#)



```
Audit-MachineAccountQuota.ps1
Description: Audits the domain-wide machine account quota attribute and local Add workstations to domain user right assignment.

Import-Module ActiveDirectory

Write-Host "--- Auditing Machine Account Quota Settings ---" -ForegroundColor Cyan

1. Audit domain-wide ms-DS-MachineAccountQuota
try {
 $Domain = Get-ADDomain -ErrorAction Stop
 $Quota = $Domain.MachineAccountQuota

 if ($Quota -ne 0) {
 Write-Host "VULNERABLE: Domain-wide ms-DS-MachineAccountQuota is set to $($Quota) (should be 0)." -ForegroundColor Red
 } else {
 Write-Host "Status: Compliant. Domain-wide ms-DS-MachineAccountQuota is set to 0." -ForegroundColor Green
 }
} catch {
 Write-Host "VULNERABLE: Could not audit ms-DS-MachineAccountQuota. Error: $($_.Exception.Message)" -ForegroundColor Red
}

2. Audit local User Rights Assignment for SeMachineAccountPrivilege
try {
 $SecCfg = "$($env:temp)\auditpolicy.inf"
 secedit /export /cfg $SecCfg /quiet

 if (Test-Path $SecCfg) {
 $cfgContent = Get-Content -Path $SecCfg
 $privilege = $cfgContent | Where-Object { $_ -like "SeMachineAccountPrivilege*" }

 if ($privilege) {
 $parts = $privilege -split "="
 if ($parts.Count -eq 2) {
 $value = $parts[1].Trim()
 if ($value -eq "*S-1-5-32-544") {
 Write-Host "Status: Compliant. SeMachineAccountPrivilege is restricted to Administrators." -ForegroundColor Green
 } elseif ($value -eq "") {
 Write-Host "Status: Compliant. SeMachineAccountPrivilege is empty (no one has the privilege)." -ForegroundColor Green
 } else {
 Write-Host "VULNERABLE: SeMachineAccountPrivilege is assigned to: $($value) (should be restricted to Administrators or empty)." -ForegroundColor Red
 }
 } else {
 Write-Host "VULNERABLE: Could not parse SeMachineAccountPrivilege line: $($privilege)" -ForegroundColor Red
 }
 } else {
 Write-Host "VULNERABLE: SeMachineAccountPrivilege line not defined in exported local policy (defaults to Authenticated Users)." -ForegroundColor Red
 }
 Remove-Item -Path $SecCfg -Force
 } else {
 Write-Host "VULNERABLE: Could not export local security policy database using secedit." -ForegroundColor Red
 }
} catch {
 Write-Host "VULNERABLE: Could not audit SeMachineAccountPrivilege. Error: $($_.Exception.Message)" -ForegroundColor Red
}
```

#### 2. Local Remediation (Set-MachineAccountQuota.ps1)

[Download Script: Set-MachineAccountQuota.ps1](#)

```
Set-MachineAccountQuota.ps1
Description: Sets the domain-wide machine account quota to 0 and restricts the local Add workstations to domain user right to Administrators.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Disable Machine Account Quota..." -ForegroundColor Cyan

1. Remediate domain-wide ms-DS-MachineAccountQuota
try {
 $Domain = Get-ADDomain -ErrorAction Stop
 if ($Domain.MachineAccountQuota -ne 0) {
 Set-ADDomain -Identity $Domain.DistinguishedName -Replace @{"ms-DS-MachineAccountQuota" = 0} -ErrorAction Stop
 Write-Host "[+] Domain-wide ms-DS-MachineAccountQuota successfully set to 0." -ForegroundColor Green
 } else {
 Write-Host "[-] Domain-wide ms-DS-MachineAccountQuota is already set to 0." -ForegroundColor Yellow
 }
} catch {
 Write-Error "Failed to set ms-DS-MachineAccountQuota. Error: ${_.Exception.Message}"
}

2. Remediate local User Rights Assignment (SeMachineAccountPrivilege)
try {
 $SecDb = "$(env:temp)\localpolicy.sdb"
 $SecCfg = "$(env:temp)\localpolicy.inf"

 # Export current security policy
 secedit /export /cfg $SecCfg /quiet

 if (Test-Path $SecCfg) {
 $CfgContent = Get-Content -Path $SecCfg
 $NewCfg = New-Object System.Collections.Generic.List[string]
 $HasPrivilege = $false

 foreach ($Line in $CfgContent) {
 if ($Line -like "SeMachineAccountPrivilege*") {
 $Line = "SeMachineAccountPrivilege = *S-1-5-32-544"
 $HasPrivilege = $true
 }
 $NewCfg.Add($Line) | Out-Null
 }

 if (-not $HasPrivilege) {
 # Add to [Privilege Rights] section
 $PrivIndex = $NewCfg.IndexOf("[Privilege Rights]")
 if ($PrivIndex -ge 0) {
 $NewCfg.Insert($PrivIndex + 1, "SeMachineAccountPrivilege = *S-1-5-32-544")
 } else {
 # Fallback: append section and value
 $NewCfg.Add("[Privilege Rights]") | Out-Null
 $NewCfg.Add("SeMachineAccountPrivilege = *S-1-5-32-544") | Out-Null
 }
 }
 }

 # Save updated configuration
 $NewCfg | Set-Content -Path $SecCfg

 # Configure local security policy
 secedit /configure /db $SecDb /cfg $SecCfg /areas USER_RIGHTS /quiet

 # Cleanup temporary files
 Remove-Item -Path $SecCfg -Force
 Remove-Item -Path $SecDb -Force

 Write-Host "[+] Local User Rights Assignment SeMachineAccountPrivilege successfully restricted to Administrators (*S-1-5-32-544)." -ForegroundColor Green
} else {
 Write-Error "Failed to export local security policy for remediation."
}
} catch {
 Write-Error "Failed to configure SeMachineAccountPrivilege. Error: ${_.Exception.Message}"
}
```

## Sources & Compliance References \* \*\*ANSI Active Directory Hardening Guide\*\*: Section 3.1.2 / CERT-FR AD Checklist  
(vuln\_user\_accounts\_machineaccountquota) \* \*\*CIS Benchmark\*\*: CIS Microsoft Windows Server 2016 Benchmark v2.0.0 - Section 2.2.4  
(Add workstations to domain) \* \*\*Microsoft Security Guidance\*\*: Active Directory ms-DS-MachineAccountQuota mitigation guidance

**# [REQ-ID-018] Restrict Pre-Windows 2000 Compatible Access Group**

**## Target Scope** \* **Applicable Systems**: Domain Controllers, Domain Environment \* **Operating Systems**: Windows Server 2016, Windows Server 2019, Windows Server 2022

---

**## Implementation Details** \* **Priority**: High \* **GPO Path / Registry Location**: \* **Restricted Groups GPO**: 'Computer Configuration\Policies\Windows Settings\Security Settings\Restricted Groups' → Group: 'Pre-Windows 2000 Compatible Access' (SID: 'S-1-5-32-554') \* **GPO Security Options**: \* 'Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options' → 'Network access: Let Everyone permissions apply to anonymous users' (Registry: 'HKLM:\SYSTEM\CurrentControlSet\Control\Lsa\EveryoneIncludesAnonymous' = '0') \* 'Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options' → 'Network access: Do not allow anonymous enumeration of SAM accounts and shares' (Registry: 'HKLM:\SYSTEM\CurrentControlSet\Control\Lsa\RestrictAnonymous' = '1') \* 'Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options' → 'Network access: Do not allow anonymous enumeration of SAM accounts' (Registry: 'HKLM:\SYSTEM\CurrentControlSet\Control\Lsa\RestrictAnonymousSAM' = '1')

---

**## Rationale** The "Pre-Windows 2000 Compatible Access" group (SID: 'S-1-5-32-554') is a legacy Active Directory security group designed to provide backward compatibility for NT4-era operating systems. By default, this group has broad read permissions to all user and group object attributes within the domain.

Historically, groups like "Everyone" ( S-1-1-0 ), "Anonymous Logon" ( S-1-5-7 ), or "Authenticated Users" ( S-1-5-11 ) were added to this group to maintain compatibility. The security risks include:

1. **Information Enumeration**: Any authenticated user (or an unauthenticated attacker via anonymous/everyone permissions) can query the Active Directory database to enumerate user lists, group memberships, trust details, and account metadata.
2. **Reconnaissance Surface**: Attackers use null-sessions or low-privileged domain accounts to profile the entire AD infrastructure, mapping target groups (like Domain Admins) and identifying service accounts for targeted attacks like Kerberoasting.
3. **Implicit Trust Abuse**: Relying on legacy broad-read access bypasses modern Active Directory object-level Access Control List (ACL) restrictions.

Removing insecure principals from this group and enforcing anonymous access restrictions restricts AD object access to authenticated, authorized accounts only.

---

**## Legacy Impact & Compatibility** \* **Legacy Integrations**: Non-Windows devices or legacy operating systems (such as older Linux integrations utilizing SSSD or Samba, Cisco ISE, or third-party reporting tools) may rely on the broad permissions of this group to query domain objects. Removing "Authenticated Users" can cause these services to fail to authenticate users or map group memberships. \* **Active Directory Certificate Services (AD CS)**: Integrated Enterprise CAs are added to this group by default to enable certificate management policies. If the restricted certificate manager feature is not in use, these CA servers can be safely removed; otherwise, removing them can impact enrollment processes. \* **Mitigations**: \* Prior to removing "Authenticated Users", verify whether any applications fail to query AD attributes. \* If a specific service or application fails, do not re-add broad groups to the Pre-Windows 2000 group. Instead, grant the specific application service account or computer account the necessary explicit "Read" permissions on target Organizational Units (OUs).

---

**## Implementation Steps****### Option A: Group Policy Object (GPO) Configuration (Preferred)**

**#### Step 1: Restrict Group Membership** 1. Open the **Group Policy Management Console** ('gpmc.msc') on a management host. 2. Create or edit a GPO linked to the **Domain Controllers** OU (e.g., 'GPO\_Hardening\_DomainControllers'). 3. Navigate to: 'Computer Configuration\Policies\Windows Settings\Security Settings\Restricted Groups'. 4. Right-click **Restricted Groups** and select **Add Group...** 5. Type **Pre-Windows 2000 Compatible Access** and click **OK**. 6. In the group properties dialog, under **Members of this group**, ensure the list is empty (or contains only authorized service accounts/CA computer accounts). Ensure **Everyone**, **Anonymous Logon**, and **Authenticated Users** are not listed. 7. Click **Apply** and then **OK**.

**#### Step 2: Configure Supporting Anonymous Access Policies** 1. In the same GPO, navigate to: 'Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options'. 2. Configure the following policies: \* **Policy**: 'Network access: Let Everyone permissions apply to anonymous users' \* **Setting**: 'Disabled' \* **Policy**: 'Network access: Do not allow anonymous enumeration of SAM accounts and shares' \* **Setting**: 'Enabled' \* **Policy**: 'Network access: Do not allow anonymous enumeration of SAM accounts' \* **Setting**: 'Enabled' 3. Link the GPO to the appropriate Organizational Unit (OU) containing the target assets.

---

**### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)**

Run the following scripts to audit and remediate these configurations.

**#### 1. Local Audit (Audit-PreWin2000Group.ps1)**

[Download Script: Audit-PreWin2000Group.ps1](#)

```

Audit-PreWin2000Group.ps1
Description: Audits the Pre-Windows 2000 Compatible Access group membership and local LSA registry configurations.

Import-Module ActiveDirectory

Write-Host "--- Auditing Pre-Windows 2000 Compatible Access Settings ---" -ForegroundColor Cyan

$GroupSid = "S-1-5-32-554"
$CriticalNonCompliantSids = @("S-1-1-0", "S-1-5-7") # Everyone, Anonymous Logon
$RestrictedNonCompliantSids = @("S-1-5-11") # Authenticated Users (default in modern systems, high legacy impact if removed)
$Vulnerable = $false
$HasWarning = $false

1. Audit Group Membership
try {
 $Group = Get-ADGroup -Identity $GroupSid -Properties Members -ErrorAction Stop
 $MembersSids = New-Object System.Collections.Generic.List[string]

 foreach ($MemberDN in $Group.Members) {
 $MemberObj = Get-ADObject -Identity $MemberDN -ErrorAction SilentlyContinue
 if ($null -ne $MemberObj) {
 $MembersSids.Add($MemberObj.SID.Value) | Out-Null
 }
 }

 # Check for Critical SIDs (Everyone, Anonymous Logon)
 foreach ($Sid in $CriticalNonCompliantSids) {
 if ($MembersSids.Contains($Sid)) {
 $Vulnerable = $true
 $Name = ""
 if ($Sid -eq "S-1-1-0") { $Name = "Everyone" }
 elseif ($Sid -eq "S-1-5-7") { $Name = "Anonymous Logon" }

 Write-Host "VULNERABLE: '$($Name)' ($($Sid)) is a member of the Pre-Windows 2000 Compatible Access group." -ForegroundColor Red
 }
 }

 # Check for Authenticated Users (High legacy impact warning)
 foreach ($Sid in $RestrictedNonCompliantSids) {
 if ($MembersSids.Contains($Sid)) {
 $HasWarning = $true
 Write-Host "WARNING: 'Authenticated Users' ($($Sid)) is a member of the Pre-Windows 2000 Compatible Access group. (This is default in modern systems; remove with caution)." -ForegroundColor Yellow
 }
 }

 if (-not $Vulnerable -and -not $HasWarning) {
 Write-Host "Status: Compliant. Pre-Windows 2000 Compatible Access group membership is restricted." -ForegroundColor Green
 } elseif (-not $Vulnerable) {
 Write-Host "Status: Compliant (with warnings). Critical groups are removed, but legacy/default Authenticated Users remains." -ForegroundColor Green
 }
} catch {
 Write-Host "VULNERABLE: Could not query Pre-Windows 2000 Compatible Access group membership. Error: $($_.Exception.Message)" -ForegroundColor Red
}

2. Audit LSA Registry Security Settings
$LsaPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"

$Settings = @{
 "EveryoneIncludesAnonymous" = 0
 "RestrictAnonymous" = 1
 "RestrictAnonymousSAM" = 1
}

foreach ($Key in $Settings.Keys) {
 try {
 if (Test-Path $LsaPath) {
 $Value = Get-ItemPropertyValue -Path $LsaPath -Name $Key -ErrorAction Stop
 $TargetValue = $Settings[$Key]

```

```

 if ($Value -ne $TargetValue) {
 Write-Host "VULNERABLE: LSA Registry Key '$($Key)' is set to $($Value) (should be $($TargetValue))." -ForegroundColor Red
 } else {
 Write-Host "Status: Compliant. LSA Registry Key '$($Key)' is set to $($TargetValue)." -ForegroundColor Green
 }
 } else {
 Write-Host "VULNERABLE: LSA registry path does not exist." -ForegroundColor Red
 }
} catch {
 Write-Host "VULNERABLE: Could not audit LSA key '$($Key)'. Error: $($_.Exception.Message)" -ForegroundColor Red
}
}

```

#### #### 2. Local Remediation (Set-PreWin2000Group.ps1)

[Download Script: Set-PreWin2000Group.ps1](#)

```
Set-PreWin2000Group.ps1
Description: Restricts Pre-Windows 2000 Compatible Access group membership and configures LSA registry security keys.

Import-Module ActiveDirectory

Configuration Switch: Set to $true if you want to remove 'Authenticated Users' (S-1-5-11).
WARNING: Removing Authenticated Users can break Legacy Samba/SSSD clients, Cisco ISE, and other querying integrations.
$RemoveAuthenticatedUsers = $false

Write-Host "Applying hardening requirement: Restrict Pre-Windows 2000 Compatible Access..." -ForegroundColor Cyan

$GroupSid = "S-1-5-32-554"
$NonCompliantSids = @("S-1-1-0", "S-1-5-7") # Critical SIDs to remove (Everyone, Anonymous Logon)

if ($RemoveAuthenticatedUsers) {
 $NonCompliantSids += "S-1-5-11" # Add Authenticated Users to the removal list
}

1. Remediate Group Membership
try {
 $Group = Get-ADGroup -Identity $GroupSid -Properties Members -ErrorAction Stop
 $MembersToRemove = New-Object System.Collections.Generic.List[string]

 foreach ($MemberDN in $Group.Members) {
 $MemberObj = Get-ADObject -Identity $MemberDN -ErrorAction SilentlyContinue
 if ($null -ne $MemberObj) {
 $Sid = $MemberObj.SID.Value
 if ($NonCompliantSids -contains $Sid) {
 $MembersToRemove.Add($MemberDN) | Out-Null
 }
 }
 }

 if ($MembersToRemove.Count -gt 0) {
 foreach ($MemberDN in $MembersToRemove) {
 try {
 Remove-ADGroupMember -Identity $GroupSid -Members $MemberDN -Confirm:$false -ErrorAction Stop
 Write-Host "[+] Successfully removed '$($MemberDN)' from the group." -ForegroundColor Green
 } catch {
 Write-Host "[-] Failed to remove '$($MemberDN)'. Error: $($_.Exception.Message)" -ForegroundColor Red
 }
 }
 } else {
 Write-Host "[-] No non-compliant members found in Pre-Windows 2000 Compatible Access group." -ForegroundColor Yellow
 }
} catch {
 Write-Error "Failed to remediate Pre-Windows 2000 Compatible Access group membership. Error: $($_.Exception.Message)"
}

2. Remediate LSA Registry Settings
$LsaPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"

$Settings = @{
 "EveryoneIncludesAnonymous" = 0
 "RestrictAnonymous" = 1
 "RestrictAnonymousSAM" = 1
}

foreach ($Key in $Settings.Keys) {
 try {
 if (-not (Test-Path $LsaPath)) {
 New-Item -Path $LsaPath -Force | Out-Null
 }

 $TargetValue = $Settings[$Key]
 Set-ItemProperty -Path $LsaPath -Name $Key -Value $TargetValue -Type DWord -ErrorAction Stop
 Write-Host "[+] Registry Key '$($Key)' successfully set to $($TargetValue)." -ForegroundColor Green
 } catch {
 Write-Error "Failed to apply LSA Registry Key '$($Key)'. Error: $($_.Exception.Message)"
 }
}
}
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Section 3.1.2 (Management of default groups and security options) \*  
\*\*CIS Benchmark\*\*: \* CIS Windows Server 2016 Benchmark v2.0.0 - Section 2.3.10.2 (Network access: Do not allow anonymous enumeration  
of SAM accounts) \* CIS Windows Server 2016 Benchmark v2.0.0 - Section 2.3.10.5 (Network access: Let Everyone permissions apply to  
anonymous users) \* CIS Windows Server 2016 Benchmark v2.0.0 - Section 2.3.10.10 (Network access: Restrict anonymous access to Named  
Pipes and Shares) \* \*\*Microsoft Security Guidance\*\*: Active Directory Pre-Windows 2000 Compatible Access security recommendations



# [REQ-ID-019] Enforce Smart Card Authentication for Privileged Users

## Target Scope \* \*\*Applicable Systems\*\*: Active Directory Domain Services (AD DS) user accounts, Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 (and above)

## Implementation Details \* \*\*Priority\*\*: High \* \*\*AD Configuration / PowerShell Attribute\*\*: \* \*\*AD User Attribute\*\*: `userAccountControl` flag `UF\_SMARTCARD\_REQUIRED` (value `0x40000` / `262144`) \* \*\*PowerShell AD Property\*\*: `SmartcardRequired` set to `true`

## Rationale Enforcing smart card authentication on privileged accounts significantly mitigates the risk of credential theft, lateral movement, and offline password cracking:

1. **Hash and Password Extraction Prevention:** By checking "Smart card is required for interactive logon" on an Active Directory user account, AD automatically rotates the account's password to a cryptographically strong, random 120-character string that is unknown to the user. This effectively invalidates the traditional NTHash and LMHash authentication methods, preventing attackers from performing password-spraying or brute-force attacks against administrative logins.
2. **Replay and Relay Mitigation:** Password-based authentication relies on credentials that can be captured, logged, or relay-attacked. Using smart cards (or physical tokens like YubiKeys) shifts authentication to Kerberos PKINIT (Public Key Cryptography for Initial Authentication in Kerberos). This protocol relies on private keys stored in the card's secure hardware element, ensuring credentials cannot be copied or replayed.
3. **Physical Presence Factor:** Combining a hardware token (something you have) and a PIN (something you know) establishes true multi-factor authentication (MFA) for administrative activities, preventing unauthorized access by remote network attackers who have compromised a password.

## Legacy Impact & Compatibility \* \*\*Password Logons Disabled\*\*: Privileged administrators will no longer be able to log on using standard username and password combinations. They must have physical/virtual smart cards and card readers actively connected to the workstation. \* \*\*PKI Dependency\*\*: A fully operational Enterprise PKI (Active Directory Certificate Services - AD CS) must be deployed. Domain Controllers must be issued valid certificates supporting KDC authentication. If the PKI is unavailable or certificates expire, users will be unable to log on. \* \*\*Service Account & Script Breaks\*\*: Administrators must not use personal administrative accounts to run automated scripts or services that rely on static password authentication. Such configurations will break and must be migrated to Group Managed Service Accounts (gMSAs) or managed via scheduled tasks utilizing certificate-based operations. \* \*\*Break-Glass (Emergency) Accounts\*\*: Break-glass accounts must be excluded from this policy if they do not have active smart card tokens, or they must be assigned dedicated hardware smart cards stored in physical security safes under dual-custody.

## Implementation Steps

### Option A: Active Directory Administrative Tools (Preferred)

To configure smart card requirement for individual privileged users or OUs:

1. Open **Active Directory Users and Computers** ( `dsa.msc` ) on a domain controller or administrative host.
2. Navigate to the Organizational Unit (OU) containing your administrative users.
3. Right-click the target administrator account and select **Properties**.
4. Click on the **Account** tab.
5. In the **Account options** window, scroll down and check the box: **Smart card is required for interactive logon**.
6. Click **Apply** and then **OK**.

Alternatively, you can select multiple users at once, right-click, select **Properties**, click the **Account** tab, select the check box next to **Smart card is required for interactive logon** under **Account options**, and click **OK** to apply the policy in batch.

### Option B: PowerShell Script (Remediation / Automation)

Use these scripts locally on a Domain Controller or a management machine with Active Directory administrative tools installed to enforce or audit the smart card requirement.

[Download Script: Configure-PrivilegedSmartCard.ps1](#)

```
Configure-PrivilegedSmartCard.ps1
Description: Enforces the 'Smart card is required for interactive logon' flag on a specified user group.
Target Engine: Windows PowerShell 5.1

Write-Host "Applying smart card logon requirement to administrative accounts..." -ForegroundColor Cyan

if (-not (Get-Module -ListAvailable -Name ActiveDirectory)) {
 Write-Error "ActiveDirectory PowerShell module is not available. Please run this script on a system with AD DS RSAT tools."
 exit 1
}

Import-Module ActiveDirectory

$GroupName = "Tier0_Administrators"
$Group = Get-ADGroup -Filter "Name -eq '$GroupName'"
if (-not $Group) {
 Write-Host "Group $GroupName not found. Defaulting to Domain Admins..." -ForegroundColor Yellow
 $GroupName = "Domain Admins"
}

$Members = Get-ADGroupMember -Identity $GroupName -Recursive | Where-Object { $_.objectClass -eq "user" }

if (-not $Members) {
 Write-Host "No users found in group $GroupName." -ForegroundColor Yellow
 exit 0
}

foreach ($Member in $Members) {
 $User = Get-ADUser -Identity $Member.distinguishedName -Properties SmartcardRequired
 if (-not $User.SmartcardRequired) {
 Write-Host "Enforcing smart card requirement for user: $($User.SamAccountName)" -ForegroundColor Cyan
 Set-ADUser -Identity $User.distinguishedName -SmartcardRequired $true
 } else {
 Write-Host "User $($User.SamAccountName) already requires smart card." -ForegroundColor Green
 }
}

Write-Host "Smart card requirement configuration complete." -ForegroundColor Green
```

*To audit the smart card requirement on privileged users:*

[Download Script: Test-PrivilegedSmartCard.ps1](#)

```
Test-PrivilegedSmartCard.ps1
Description: Audits if all members of the specified administrative group require smart card for logon.
Target Engine: Windows PowerShell 5.1

Write-Host "--- Auditing Privileged User Smart Card Requirements ---" -ForegroundColor Cyan

if (-not (Get-Module -ListAvailable -Name ActiveDirectory)) {
 Write-Warning "ActiveDirectory PowerShell module is not available. Please install RSAT AD DS tools."
 exit 0
}

Import-Module ActiveDirectory

$GroupName = "Tier0_Administrators"
$Group = Get-ADGroup -Filter "Name -eq '$GroupName'"
if (-not $Group) {
 Write-Host "Group $GroupName not found. Defaulting audit to Domain Admins..." -ForegroundColor Yellow
 $GroupName = "Domain Admins"
}

$Vulnerable = $false
$Members = Get-ADGroupMember -Identity $GroupName -Recursive | Where-Object { $_.objectClass -eq "user" }

if (-not $Members) {
 Write-Host "No users found in group $GroupName to audit." -ForegroundColor Yellow
} else {
 foreach ($Member in $Members) {
 $User = Get-ADUser -Identity $Member.distinguishedName -Properties SmartcardRequired
 if (-not $User.SmartcardRequired) {
 Write-Host " - User: $($User.SamAccountName) | Actual: Password Allowed (Expected: Smart Card Required)" -ForegroundColor Red
 $Vulnerable = $true
 } else {
 Write-Host " - User: $($User.SamAccountName) | Actual: Smart Card Required (Expected: Smart Card Required)" -ForegroundColor Green
 }
 }
}

if ($Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
}
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendations on administrator identification, multi-factor authentication, and password hash isolation. \* \*\*CIS Microsoft Windows Server Benchmark\*\*: Section 1.2 (Account policies) and Section 2.3.9.5 (Interactive logon: Smart card removal behavior). \* \*\*Microsoft Security Guidance\*\*: Protecting high-privilege credentials in Active Directory.

# [REQ-ID-020] Clean Up Legacy Group Policy Preferences and SYSVOL Passwords

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows Server 2025

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: None (Domain-wide SYSVOL cleanup of legacy policy preference files and logon/startup scripts)

## Rationale Historically, administrators utilized Group Policy Preferences (GPP) to automate account creation, service configurations, drive mappings, and local administrator password rotations. When credentials were saved within a GPP, the password was stored as an encrypted string under the `cpassword` attribute in XML configuration files (e.g., `Groups.xml`, `Services.xml`, `ScheduledTasks.xml`) inside the domain-wide `SYSVOL` share.

Although Microsoft encrypted the password with AES-256, the static AES decryption key was published on MSDN. Because any authenticated user (or trust) has read access to the `SYSVOL` share, any domain user can read the preference XML files, extract the `cpassword` value, and decrypt it to obtain cleartext credentials.

Microsoft patched this vulnerability in May 2014 via **MS14-025 (KB2962486)**, which blocks the Group Policy Management Console (GPMC) from creating or updating policies that contain password fields. However, **the patch does not delete existing GPP XML files with passwords from SYSVOL**. Consequently, legacy preferences with encrypted credentials remain in the `SYSVOL` directory and continue to be a primary target for adversary credential harvesting.

Additionally, administrators historically deployed custom login or management scripts (e.g., `.vbs`, `.bat`, `.cmd`, `.ps1`) in `SYSVOL` with cleartext passwords hardcoded. These must also be identified and purged.

## Legacy Impact & Compatibility \* \*\*Configuration Breakage\*\*: Removing credentials or stripping `cpassword` properties from active Group Policy Preferences will prevent those specific objects (drives, tasks, services) from authenticating and executing. \* \*\*Transition Prerequisite\*\*: Ensure that all systems requiring local administrator password rotation are migrated to Windows LAPS or Classic LAPS ([REQ-ID-002](#03-identities-services-enable-laps-md)) and all service/task authentication is migrated to Group Managed Service Accounts (gMSAs) ([REQ-ID-003](#03-identities-services-harden-service-accounts-md)) prior to cleaning up GPP passwords.

#### ## Implementation Steps

##### #### Option A: Active Directory Group Policy Management (GUI)

To manually locate and clean up credential fields:

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a management server.
2. Review all active GPOs that configure Preferences (specifically under **Computer Configuration** or **User Configuration** → **Preferences** → **Control Panel Settings** → **Local Users and Groups, Scheduled Tasks, or Services**).
3. If an active policy contains an account configuration with a password, recreate the policy option without specifying credentials (use LAPS or gMSA as alternatives).
4. For any GPOs no longer in use, delete them using GPMC to clean up their folders from the `SYSVOL` directory.

##### #### Option B: PowerShell & Remediation (Non-GPO / Script-based)

Use these scripts to scan and automatically remediate GPP configuration files in the `SYSVOL` share.

[Download Script: Remove-GPPSYSVOLPasswords.ps1](#)

```
Remove-GPPSYSVOLPasswords.ps1
Description: Removes cpassword attributes from Group Policy Preference XML files in SYSVOL and backs up original files.
Target Engine: Windows PowerShell 5.1

Write-Host "--- Remediation: Cleaning Up GPP cpassword Credentials in SYSVOL ---" -ForegroundColor Cyan

Retrieve local SYSVOL path from registry
$SysvolReg = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Netlogon\Parameters" -Name "Sysvol" -ErrorAction SilentlyContinue
if (-not $SysvolReg) {
 Write-Host "[*] SYSVOL registry path not found. Checking standard share path..." -ForegroundColor Yellow
 $SysvolPath = "C:\Windows\SYSVOL\sysvol"
} else {
 $SysvolPath = $SysvolReg.Sysvol
}

if (-not (Test-Path -Path $SysvolPath)) {
 Write-Host "[-] SYSVOL folder not found at path: $SysvolPath. Nothing to remediate." -ForegroundColor Red
 exit 0
}

1. Scan and remediate GPP XML files
$GppXmIs = Get-ChildItem -Path $SysvolPath -Filter *.xml -Recurse -File -ErrorAction SilentlyContinue

foreach ($file in $GppXmIs) {
 $content = Get-Content -Path $file.FullName -Raw -ErrorAction SilentlyContinue
 if ($content -and $content.Contains("cpassword")) {
 Write-Host "[*] Found GPP file with cpassword: $($file.FullName)" -ForegroundColor Yellow

 # Create Backup
 $Timestamp = Get-Date -Format "yyyyMMddHHmmss"
 $BackupPath = "$($file.FullName).bak_{$Timestamp}"
 Copy-Item -Path $file.FullName -Destination $BackupPath -Force -ErrorAction SilentlyContinue
 Write-Host " [+] Created backup at: $BackupPath" -ForegroundColor Gray

 # Load XML
 [xml]$xml = New-Object System.Xml.XmlDocument
 try {
 $xml.Load($file.FullName)

 # Find all elements with cpassword attribute using XPath
 $Nodes = $xml.SelectNodes("//*[@cpassword]")
 if ($Nodes.Count -gt 0) {
 foreach ($Node in $Nodes) {
 Write-Host " [+] Stripping cpassword attribute from XML node: $($Node.Name)" -ForegroundColor White
 $Node.RemoveAttribute("cpassword")
 # If username exists, log it to help administrator identify what was affected
 if ($Node.Attributes["username"]) {
 Write-Host " [!] Note: Node was configured for username '$($Node.Attributes["username"].Value)'" -ForegroundColor Yellow
 }
 }
 $xml.Save($file.FullName)
 Write-Host " [+] Successfully stripped cpassword from: $($file.FullName)" -ForegroundColor Green
 }
 } catch {
 Write-Host " [-] Failed to parse or save XML: $_.Exception.Message" -ForegroundColor Red
 }
 }
}

2. Alert for scripts (Do not auto-remediate script files to prevent code syntax breakage)
$ScriptFiles = Get-ChildItem -Path $SysvolPath -Include *.vbs, *.ps1, *.bat, *.cmd -Recurse -File -ErrorAction SilentlyContinue
$CredentialPattern = '(?i)\b(password|pwd|adminpwd|syspwd|adminpassword|localadminpwd)\s*=\s*["\']["\']*\s*\b'

foreach ($file in $ScriptFiles) {
 # Skip our own audit and implementation scripts
 if ($file.Name -like "Get-GPPSYSVOLPasswords*" -or $file.Name -like "Remove-GPPSYSVOLPasswords*" -or $file.Name -like "SYSVOL HoneyPot*") {
 continue
 }

 $content = Get-Content -Path $file.FullName -Raw -ErrorAction SilentlyContinue
}
```

```
 if ($content -and $content -match $CredentialPattern) {
 Write-Host "[WARNING] Script contains hardcoded credential pattern: $($file.FullName)" -ForegroundColor Red
 Write-Host " Manual intervention is required. Review and delete/rotate credentials in this script." -ForegroundColor
r Yellow
 }
 }

 Write-Host "[+] SYSVOL password remediation processing completed." -ForegroundColor Green
```

*To verify that no cpassword files or cleartext script credentials exist in SYSVOL:*

[Download Script: Get-GPPSYSVOLPasswords.ps1](#)

```
Get-GPPSYSVOLPasswords.ps1
Description: Audits the SYSVOL directory for legacy Group Policy Preference files containing cpassword values and scripts containi
ng cleartext credentials.
Target Engine: Windows PowerShell 5.1

Write-Host "--- Auditing SYSVOL for Group Policy Preference and Script Passwords ---" -ForegroundColor Cyan
$script:Vulnerable = $false

Retrieve local SYSVOL path from registry
$SysvolReg = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Netlogon\Parameters" -Name "Sysvol" -ErrorAction SilentlyContinue
if (-not $SysvolReg) {
 Write-Host "[*] SYSVOL registry path not found. Checking standard share path..." -ForegroundColor Yellow
 $SysvolPath = "C:\Windows\SYSVOL\sysvol"
} else {
 $SysvolPath = $SysvolReg.Sysvol
}

if (-not (Test-Path -Path $SysvolPath)) {
 Write-Host "[-] SYSVOL folder not found at path: $SysvolPath" -ForegroundColor Red
 exit 0 # Not a Domain Controller or SYSVOL not configured, nothing to audit
}

Write-Host "[*] Scanning SYSVOL directory: $SysvolPath" -ForegroundColor White

1. Scan for XML files containing 'cpassword'
$GppXmIs = Get-ChildItem -Path $SysvolPath -Filter *.xml -Recurse -File -ErrorAction SilentlyContinue
foreach ($file in $GppXmIs) {
 # Read the file content safely
 $content = Get-Content -Path $file.FullName -Raw -ErrorAction SilentlyContinue
 if ($content -and $content.Contains("cpassword")) {
 Write-Host "[!] VULNERABLE: Group Policy Preference file contains cpassword attribute: $($file.FullName)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
}

2. Scan for script files containing cleartext credentials
Scripts: .vbs, .ps1, .bat, .cmd
$ScriptFiles = Get-ChildItem -Path $SysvolPath -Include *.vbs, *.ps1, *.bat, *.cmd -Recurse -File -ErrorAction SilentlyContinue

Regex pattern for credentials (e.g. password=, pwd=, adminpwd=)
$CredentialPattern = '(?i)\b(password|pwd|adminpwd|syspwd|adminpassword|localadminpwd)\s*=\s*["'']["'']*\s*\b'

foreach ($file in $ScriptFiles) {
 # Check if the file is our own audit or implementation scripts (skip those)
 if ($file.Name -like "*Get-GPPSYSVOLPasswords*" -or $file.Name -like "*Remove-GPPSYSVOLPasswords*" -or $file.Name -like "*SYSVOL
Honeypot*") {
 continue
 }

 $content = Get-Content -Path $file.FullName -Raw -ErrorAction SilentlyContinue
 if ($content -and $content -match $CredentialPattern) {
 Write-Host "[!] VULNERABLE: Script file contains potential cleartext password: $($file.FullName)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "[+] No Group Policy Preference passwords or cleartext scripts found in SYSVOL." -ForegroundColor Green
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
}
```

## Sources & Compliance References \* \*\*Microsoft Security Bulletin\*\*: [MS14-025 - Vulnerability in Group Policy Preferences Could Allow Elevation of Privilege (2962486)](<https://learn.microsoft.com/en-us/security-updates/securitybulletins/2014/ms14-025>) \* \*\*ANSI Active Directory Hardening Guide\*\*: Recommendations on local account password delegation and account cleanup. \* \*\*CIS Control\*\*: CIS Control 5.2 - Unique passwords for administrative accounts; CIS Control 6.4 - Delete orphan/defunct policies.

## # Module 4: Network Configuration & Firewalling

This directory contains network security architectures, active directory port configurations, and network isolation boundaries.

### ## Technical Hardening Controls

1. [REQ-NET-001 - Configure Active Directory Port Matrix](#) Establishes the minimum permitted ports for Domain Controllers, Member Servers, and Client Workstations, ensuring perimeter and local firewalls block unauthorized inbound traffic.
2. [REQ-NET-002 - Restrict RPC Dynamic Ports](#) Binds core directory services (NTDS, Netlogon, DFSR) to specific static ports to allow precise network firewall controls while maintaining the system-wide dynamic RPC port range at default values to prevent socket exhaustion.
3. [REQ-NET-003 - Configure Workstation and Server Isolation](#) Configures local firewall rules on workstations and servers to block inbound SMB, RPC, RDP, and WinRM from peer systems to prevent lateral movement.
4. [REQ-NET-004 - Configure IPsec Domain Isolation](#) Enforces IPsec Connection Security Rules to authenticate and encrypt traffic within the domain boundary.
5. [REQ-NET-005 - Harden IPsec Cryptographic Configurations](#) Restricts permitted IPsec cryptography suites to secure options (AES-256 and DH Group 19/20) for Phase 1 and Phase 2 negotiations.
6. [REQ-NET-006 - Harden TLS Protocols, Cipher Suites, and Elliptic Curves](#) Disables legacy SSL/TLS versions, enforces TLS 1.2/1.3, orders strong cipher suites, and prioritizes secure elliptic curves.
7. [REQ-NET-007 - Enforce SMBv3 Security and Digitally Sign/Encrypt Communications](#) Disables legacy SMB dialects, enforces SMBv3, and mandates message signing and encryption to protect communications and prevent relay attacks.
8. [REQ-NET-008 - Configure Firewall Logging and Operational Settings](#) Enforces Windows Defender Firewall state, sets default inbound block policies, disables local rule merging on Domain Controllers, and configures detailed dropped packet logging to improve security visibility and forensic capabilities.
9. [REQ-NET-009 - Configure Hardened UNC Paths and LDAP Client Signing](#) Enforces mutual authentication and SMB signing for GPO folder structures (SYSVOL/NETLOGON), restricts workstation guest logons, and requires outgoing LDAP client signing.
10. [REQ-NET-010 - Harden WinRM Service and Restrict Remote RPC Clients](#) Disables Basic and Digest authentication, forces encrypted WinRM communications, restricts WinRM credential caching, and blocks anonymous RPC connections.
11. [REQ-NET-011 - Configure WMI Static Port](#) Binds the WMI service to static TCP port 24158 and isolates it to a standalone host process with packet privacy enabled to limit remote lateral movement exposure.
12. [REQ-NET-012 - Configure RPC Filters for Named Pipes](#) Enforces Windows Firewall RPC Filters to block administrative queries and code execution over SMB named pipes (e.g. SCM, Task Scheduler) from unauthorized subnets.
13. [REQ-NET-013 - Block Management Traffic Between Domain Controllers](#) Excludes Domain Controller IP addresses from allowed management rules (RDP, WinRM, WMI, ADWS) to prevent lateral movement between Tier 0 directory servers.



**# [REQ-NET-001] Configure Active Directory Port Matrix**

**## Target Scope** \* **Applicable Systems**: Domain Controllers, Member Servers, Tier 2 Client Workstations. \* **Operating Systems**: Windows Server 2016 (and above), Windows 10 (and above) Enterprise/Professional.

**## Implementation Details** \* **Priority**: High \* **GPO Path / Registry Location**: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Windows Defender Firewall with Advanced Security`

**## Rationale** Active Directory services require several ports to function, including DNS, Kerberos, LDAP, SMB, and RPC. If firewalls are not configured to restrict traffic to only these essential ports, adversaries can perform internal network scanning, identify open services, exploit vulnerabilities in unhardened services, or pivot across systems.

Restricting network communications to the minimum required AD Port Matrix ensures:

1. **Attack Surface Reduction**: Unused services are blocked from receiving network connections.
2. **Reconnaissance Mitigation**: Internal port scanning returns blocked states, slowing down discovery.
3. **Lateral Movement Containment**: Compromised endpoints cannot arbitrary query services on domain controllers or other member systems.

**## Legacy Impact & Compatibility** \* **Legacy Protocols**: Disabling NetBIOS ports (UDP 137/138, TCP 139) will break legacy name resolution and applications relying on NetBIOS API. \* **RPC Communication**: Dynamic RPC ranges must be synchronized between firewall rules and system settings. If firewalls restrict the RPC range but the operating system is not configured to bind RPC to that range, legitimate RPC traffic (like replication and group policy processing) will fail. \* **Application Dependency**: Member servers hosting multi-tier applications must have custom inbound firewall rules created for their specific application ports.

**## Implementation Steps****### Option A: Group Policy Object (GPO) Configuration (Preferred)**

**####** 1. Enforce Default Inbound Block 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Create or edit a GPO linked to the target systems (e.g., `GPO\_Hardening\_Firewall\_Baseline`). 3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security` 4. Right-click **Windows Defender Firewall with Advanced Security** and select **Properties**. 5. For the **Domain Profile**, **Private Profile**, and **Public Profile** tabs, set: \* **State**: `On (recommended)` \* **Inbound connections**: `Block (default)` \* **Outbound connections**: `Allow (default)`

**####** 2. Create Inbound Allow Rules for the AD Port Matrix (Domain Controllers GPO) For GPOs targeting Domain Controllers, configure the following inbound rules under **Inbound Rules**:

PROTOCOL	PORT	SOURCE SUBNET	DESCRIPTION
TCP / UDP	53	Any / Client Subnets	DNS Resolution
UDP	123	Any / Client Subnets	NTP Time Sync
TCP / UDP	88	Any / Client Subnets	Kerberos Authentication
TCP / UDP	464	Any / Client Subnets	Kerberos Password Change
TCP / UDP	389	Any / Client Subnets	LDAP Directory Queries
TCP	636	Any / Client Subnets	LDAPS (Secure LDAP)
TCP	3268	Any / Client Subnets	Global Catalog Query
TCP	3269	Any / Client Subnets	Global Catalog SSL
TCP	135	Any / Client Subnets	RPC Endpoint Mapper
TCP	445	Any / Client Subnets	SMB (SYSVOL / GPO)
TCP	38901	Any / Client Subnets	NTDS Static RPC Port
TCP	38902	Any / Client Subnets	Netlogon Static RPC Port
TCP	5722	Domain Controller Subnets	DFSR Static RPC Port
TCP	3389	PAW / Jump Host Subnets	RDP (Management)
TCP	5985 / 5986	PAW / Jump Host Subnets	WinRM (Management)

**WARNING:****STATIC RPC PORTS REQUIREMENT**

: The NTDS (TCP 38901), Netlogon (TCP 38902), and DFSR (TCP 5722) ports listed above require static port configurations to be active on the target Domain Controllers. If these ports are not explicitly configured to be static on the operating system, remote authentication and replication will fail. See [REQ-NET-002: Restrict RPC Dynamic Ports](#) for GPO and registry configurations to bind these services to their static endpoints.

---

**### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)**

Run the following scripts locally to audit and configure the firewall profiles and inbound port rules.

#### Remediation Script: [Download Script: Set-ADPortMatrixRules.ps1](implementation\_scripts/Set-ADPortMatrixRules.ps1)

```
Set-ADPortMatrixRules.ps1
Configures local Windows Defender Firewall profiles and applies basic AD port matrix baseline rules.

param(
 [string[]]$ManagementAddresses
)

If not passed, prompt user dynamically
if ($null -eq $ManagementAddresses) {
 if ([Environment]::UserInteractive) {
 $inputVal = Read-Host "Enter remote IP addresses/subnets for remote management (comma-separated, e.g. 10.0.0.0/24,192.168.1.50) [leave empty for LocalSubnet]"
 if ([string]::IsNullOrEmpty($inputVal)) {
 $ManagementAddresses = @"LocalSubnet"
 } else {
 $ManagementAddresses = $inputVal.Split(",") | ForEach-Object { $_.Trim() }
 }
 } else {
 $ManagementAddresses = @"LocalSubnet"
 }
}

Write-Host "Applying network firewall baseline policies..." -ForegroundColor Cyan

1. Enable firewall and set default block inbound
Set-NetFirewallProfile -Profile Domain, Public, Private -Enabled True -DefaultInboundAction Block -DefaultOutboundAction Allow
Write-Host "Firewall profiles enabled with Default Inbound Block." -ForegroundColor Green

2. Configure AD Port Matrix inbound rules (for local system role validation)
$Rules = @(
 @{ Name = "AD-DNS-TCP"; Port = 53; Proto = "TCP"; Remote = "Any" },
 @{ Name = "AD-DNS-UDP"; Port = 53; Proto = "UDP"; Remote = "Any" },
 @{ Name = "AD-Kerberos-TCP"; Port = 88; Proto = "TCP"; Remote = "Any" },
 @{ Name = "AD-Kerberos-UDP"; Port = 88; Proto = "UDP"; Remote = "Any" },
 @{ Name = "AD-NTP-UDP"; Port = 123; Proto = "UDP"; Remote = "Any" },
 @{ Name = "AD-RPC-Mapper-TCP"; Port = 135; Proto = "TCP"; Remote = "Any" },
 @{ Name = "AD-LDAP-TCP"; Port = 389; Proto = "TCP"; Remote = "Any" },
 @{ Name = "AD-LDAP-UDP"; Port = 389; Proto = "UDP"; Remote = "Any" },
 @{ Name = "AD-SMB-TCP"; Port = 445; Proto = "TCP"; Remote = "Any" },
 @{ Name = "AD-Kpwd-TCP"; Port = 464; Proto = "TCP"; Remote = "Any" },
 @{ Name = "AD-Kpwd-UDP"; Port = 464; Proto = "UDP"; Remote = "Any" },
 @{ Name = "AD-LDAPS-TCP"; Port = 636; Proto = "TCP"; Remote = "Any" },
 @{ Name = "AD-GC-TCP"; Port = 3268; Proto = "TCP"; Remote = "Any" },
 @{ Name = "AD-GC-SSL-TCP"; Port = 3269; Proto = "TCP"; Remote = "Any" },
 @{ Name = "AD-NTDS-Static-TCP"; Port = 38901; Proto = "TCP"; Remote = "Any" },
 @{ Name = "AD-Netlogon-Static-TCP"; Port = 38902; Proto = "TCP"; Remote = "Any" },
 @{ Name = "AD-DFS-R-Static-TCP"; Port = 5722; Proto = "TCP"; Remote = "Any" },
 @{ Name = "AD-RDP-TCP"; Port = 3389; Proto = "TCP"; Remote = $ManagementAddresses },
 @{ Name = "AD-WinRM-HTTP-TCP"; Port = 5985; Proto = "TCP"; Remote = $ManagementAddresses },
 @{ Name = "AD-WinRM-HTTPS-TCP"; Port = 5986; Proto = "TCP"; Remote = $ManagementAddresses }
)

foreach ($Rule in $Rules) {
 $Name = $Rule.Name
 $Port = $Rule.Port
 $Proto = $Rule.Proto
 $Remote = $Rule.Remote

 $Existing = Get-NetFirewallRule -Name $Name -ErrorAction SilentlyContinue
 if ($null -eq $Existing) {
 New-NetFirewallRule -Name $Name -DisplayName $Name `
 -Direction Inbound `
 -Action Allow `
 -Protocol $Proto `
 -LocalPort $Port `
 -RemoteAddress $Remote `
 -Profile Domain, Private `
 -Enabled True | Out-Null

 Write-Host "Inbound rule created: $($Name) on port $($Port) ($($Proto)) from remote address: $($Remote -join ', ')" -ForegroundColor Green
 } else {
 Set-NetFirewallRule -Name $Name -Enabled True -Action Allow -RemoteAddress $Remote | Out-Null
 Write-Host "Inbound rule verified: $($Name) restricted to remote address: $($Remote -join ', ')" -ForegroundColor Gray
 }
}
```

```
}

Write-Host "Firewall port matrix configuration completed successfully." -ForegroundColor Cyan
```

#### Audit Script: [Download Script: Test-ADPortMatrixRules.ps1](audit\_scripts/Test-ADPortMatrixRules.ps1)

```
Test-ADPortMatrixRules.ps1
Audits local firewall status and checks if default inbound traffic is blocked.

Write-Host "Auditing local network firewall status..." -ForegroundColor Cyan

1. Check Windows Defender Firewall State
$Profiles = Get-NetFirewallProfile
$AllProfilesSecure = $true

foreach ($FwProfile in $Profiles) {
 $Enabled = $FwProfile.Enabled
 $InAction = $FwProfile.DefaultInboundAction

 if ($Enabled -eq $true -and $InAction -eq "Block") {
 Write-Host "Profile: $($FwProfile.Name) | Enabled: True | InboundAction: Block" -ForegroundColor Green
 } else {
 Write-Host "Profile: $($FwProfile.Name) | Enabled: $($Enabled) | InboundAction: $($InAction) (INSECURE)" -ForegroundColor Red
 $AllProfilesSecure = $false
 }
}

if ($AllProfilesSecure) {
 Write-Host "Audit Result: Firewall state configuration is compliant." -ForegroundColor Green
} else {
 Write-Warning "Audit Result: Firewall profiles are not fully secured!"
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R8 (Administration network subnets) \* \*\*CIS Windows Server 2016 Benchmark\*\*: Section 19 (Windows Defender Firewall with Advanced Security) \* \*\*Microsoft Security Baseline\*\*: Domain Controller and Member Server Baselines \* \*\*DSInternals AD Firewall Guide (Michael Grafnetter)\*\*: [Active Directory Firewall - Domain Controller Firewall](https://firewall.dsinternals.com/ADDS/)

## # [REQ-NET-002] Restrict RPC Dynamic Ports

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers. \* \*\*Operating Systems\*\*: Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*NTDS RPC Port (DCs only)\*\*:

`HKLM\SYSTEM\CurrentControlSet\Services\NTDS\Parameters` \* Value Name: `TCP/IP Port` \* Value Type: `REG\_DWORD` \* Recommended Value: `38901` (Decimal) \* \*\*Netlogon RPC Port (DCs only)\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\Netlogon\Parameters` \* Value Name: `DCTcpipPort` \* Value Type: `REG\_DWORD` \* Recommended Value: `38902` (Decimal) \* \*\*DFSR Replication Port (DCs/DFS Members)\*\*: DFSR service WMI configuration or `dfsrdiag.exe`. \* Recommended Value: `5722` (Decimal)

## Rationale By default, the RPC runtime utilizes a massive dynamic range of high-order ports (TCP 49152-65535) for communication, including Active Directory replication, netlogon authentication, and DFS replication.

Opening this entire dynamic port range in network-based firewalls for all systems exposes an unmonitored attack surface. To mitigate this risk, key domain controller services (NTDS, Netlogon, and DFSR) must be bound to dedicated static ports (TCP 38901, 38902, and 5722 respectively). This allows network administrators to configure precise firewall rules permitting only these ports.

Crucially, **the system-wide dynamic RPC range must NOT be narrowed** (such as restricting it globally to 50000-50100). Restricting the global dynamic range introduces severe risks:

1. **Port Exhaustion:** Under standard server load, limiting the global ephemeral range to a small number of ports can exhaust available sockets, resulting in network failures and domain isolation outages.
2. **Replication Failure:** High volumes of directory transactions can exhaust localized RPC ports.
3. **No Added Security Value:** Narrowing the dynamic range globally does not mitigate standard exploits, and endpoints communicate with local security authority protocols (LSAD/SAMR) over SMB named pipes ( `\PIPE\lsass` ) rather than directly via dynamic TCP sockets.

Therefore, the system-wide dynamic RPC port range must remain at its default start port ( `49152` ) and number of ports ( `16384` ), and any narrowing configurations must be avoided.

## Legacy Impact & Compatibility \* \*\*Firewall Coordination\*\*: Network firewall rules permitting TCP ports 38901, 38902, and 5722 must be active and synchronized before configuring these ports, or replication and authentication failures will occur immediately. \* \*\*DFSR Management Tools\*\*: Configuring the DFSR static port dynamically requires the DFS Management Tools feature (`DfsMgmt`) to be installed on the local DC.

## ## Implementation Steps

## ### Option A: Group Policy Object (GPO) Configuration (Preferred)

#### 1. Define Static Ports for NTDS and Netlogon via GPO Registry Preferences 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Create or edit a GPO targeting Domain Controllers (e.g., `GPO\_Hardening\_DC\_RPC`). 3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry` 4. Define the following **Registry Items**: \* \*\*NTDS Static Port\*\*: \* \*\*Action\*\*: `Update` \* \*\*Hive\*\*: `HKEY\_LOCAL\_MACHINE` \* \*\*Key Path\*\*: `SYSTEM\CurrentControlSet\Services\NTDS\Parameters` \* \*\*Value Name\*\*: `TCP/IP Port` \* \*\*Value Type\*\*: `REG\_DWORD` \* \*\*Value Data\*\*: `38901` (Decimal) \* \*\*Netlogon Static Port\*\*: \* \*\*Action\*\*: `Update` \* \*\*Hive\*\*: `HKEY\_LOCAL\_MACHINE` \* \*\*Key Path\*\*: `SYSTEM\CurrentControlSet\Services\Netlogon\Parameters` \* \*\*Value Name\*\*: `DCTcpipPort` \* \*\*Value Type\*\*: `REG\_DWORD` \* \*\*Value Data\*\*: `38902` (Decimal)

#### 2. Ensure System-Wide RPC Port Narrowing is Disabled Ensure that the registry key `HKLM\SOFTWARE\Microsoft\Rpc\Internet` does **not** exist in your GPOs, as this key enforces restrictive dynamic range overrides. If present, delete the key to allow systems to fall back to the default Windows Server dynamic range (49152-65535).

## ### Option B: PowerShell &amp; Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to configure the RPC static ports and restore default dynamic ranges.

## #### Remediation Script: [Download Script: Set-RPCDynamicPorts.ps1](implementation\_scripts/Set-RPCDynamicPorts.ps1)

```
Set-RPCDynamicPorts.ps1
Description: Configures static RPC ports for NTDS, Netlogon, and DFSR, and ensures system-wide dynamic RPC ranges are at default values.

Write-Host "Configuring RPC dynamic port restrictions..." -ForegroundColor Cyan

$IsDC = (Get-CimInstance -ClassName Win32_ComputerSystem).Roles -contains "Primary_Domain_Controller"

if ($IsDC) {
 Write-Host "[+] Target is a Domain Controller. Configuring static ports..." -ForegroundColor Gray

 # NTDS Static Port → TCP 38901
 $NtdsPath = "HKLM:\SYSTEM\CurrentControlSet\Services\NTDS\Parameters"
 if (-not (Test-Path $NtdsPath)) {
 New-Item -Path $NtdsPath -Force | Out-Null
 }
 Set-ItemProperty -Path $NtdsPath -Name "TCP/IP Port" -Value 38901 -Type DWord
 Write-Host " NTDS Static Port set to TCP 38901." -ForegroundColor Green

 # Netlogon Static Port → TCP 38902
 $NetLogonPath = "HKLM:\SYSTEM\CurrentControlSet\Services\NetLogon\Parameters"
 if (-not (Test-Path $NetLogonPath)) {
 New-Item -Path $NetLogonPath -Force | Out-Null
 }
 Set-ItemProperty -Path $NetLogonPath -Name "DCTcpipPort" -Value 38902 -Type DWord
 Write-Host " Netlogon Static Port set to TCP 38902." -ForegroundColor Green

 # DFSR Static Port → TCP 5722 (if DFSR namespace is present)
 try {
 $DfsrConfig = Get-CimInstance -Namespace "root\MicrosoftDFS" -ClassName DfsrServiceConfiguration -ErrorAction Stop
 if ($null -ne $DfsrConfig) {
 Set-CimInstance -Query "Select * from DfsrServiceConfiguration" -Namespace "root\MicrosoftDFS" -Property @{ RpcPortAssignment = 5722 } -ErrorAction Stop | Out-Null
 Write-Host " DFSR Static Replication Port set to TCP 5722." -ForegroundColor Green
 }
 } catch {
 Write-Host " DFSR WMI configuration not accessible or role not installed. Skipping." -ForegroundColor Yellow
 }
}

Ensure system-wide dynamic RPC range is at default (start=49152, num=16384)
In accordance with Microsoft Directory Services guidelines to prevent port exhaustion.
Write-Host "[+] Resetting global dynamic RPC ports to default..." -ForegroundColor Gray

$ProcV4 = Start-Process netsh -ArgumentList "int ipv4 set dynamicport tcp start=49152 num=16384" -Wait -NoNewWindow -PassThru
if ($ProcV4.ExitCode -eq 0) {
 Write-Host " IPv4 Dynamic Port Range reset to default (49152-65535)." -ForegroundColor Green
} else {
 Write-Error " Failed to reset IPv4 dynamic port range."
}

$ProcV6 = Start-Process netsh -ArgumentList "int ipv6 set dynamicport tcp start=49152 num=16384" -Wait -NoNewWindow -PassThru
if ($ProcV6.ExitCode -eq 0) {
 Write-Host " IPv6 Dynamic Port Range reset to default (49152-65535)." -ForegroundColor Green
} else {
 Write-Error " Failed to reset IPv6 dynamic port range."
}

Clean HKLM\SOFTWARE\Microsoft\Rpc\Internet range narrowing if present
$RpcInternetPath = "HKLM:\SOFTWARE\Microsoft\Rpc\Internet"
if (Test-Path $RpcInternetPath) {
 Remove-Item -Path $RpcInternetPath -Force -Recurse | Out-Null
 Write-Host " Removed restrictive RPC Internet registry settings to restore defaults." -ForegroundColor Green
}

Write-Host "RPC dynamic port configuration applied successfully." -ForegroundColor Cyan
```

#### Audit Script: [Download Script: Test-RPCDynamicPorts.ps1](audit\_scripts/Test-RPCDynamicPorts.ps1)

```
Test-RPCDynamicPorts.ps1
Description: Audits dynamic RPC configurations and static ports.

Write-Host "Auditing dynamic RPC configurations..." -ForegroundColor Cyan

$IsDC = (Get-CimInstance -ClassName Win32_ComputerSystem).Roles -contains "Primary_Domain_Controller"
$vulnerable = $false

if ($IsDC) {
 # 1. NTDS Static Port Audit
 $NtdsPath = "HKLM:\SYSTEM\CurrentControlSet\Services\NTDS\Parameters"
 $NtdsVal = Get-ItemProperty -Path $NtdsPath -Name "TCP/IP Port" -ErrorAction SilentlyContinue
 $NtdsPort = if ($NtdsVal) { $NtdsVal."TCP/IP Port" } else { 0 }

 $NtdsColor = if ($NtdsPort -eq 38901) { "Green" } else { "Red"; $vulnerable = $true }
 Write-Host " - NTDS Static Port: $($NtdsPort) (Expected = 38901)" -ForegroundColor $NtdsColor

 # 2. Netlogon Static Port Audit
 $NetlogonPath = "HKLM:\SYSTEM\CurrentControlSet\Services\NetLogon\Parameters"
 $NetlogonVal = Get-ItemProperty -Path $NetlogonPath -Name "DCTcpipPort" -ErrorAction SilentlyContinue
 $NetlogonPort = if ($NetlogonVal) { $NetlogonVal.DCTcpipPort } else { 0 }

 $NetlogonColor = if ($NetlogonPort -eq 38902) { "Green" } else { "Red"; $vulnerable = $true }
 Write-Host " - Netlogon Static Port: $($NetlogonPort) (Expected = 38902)" -ForegroundColor $NetlogonColor

 # 3. DFSR Port Audit
 try {
 $DfsrConfig = Get-CimInstance -Namespace "root\MicrosoftDFS" -ClassName DfsrServiceConfiguration -ErrorAction Stop
 if ($null -ne $DfsrConfig) {
 $DfsrPort = $DfsrConfig.RpcPortAssignment
 $DfsrColor = if ($DfsrPort -eq 5722) { "Green" } else { "Red"; $vulnerable = $true }
 Write-Host " - DFSR Replication Port: $($DfsrPort) (Expected = 5722)" -ForegroundColor $DfsrColor
 }
 } catch {
 Write-Host " - DFSR role not active or WMI inaccessible." -ForegroundColor Gray
 }
}

4. Global Dynamic Port Audit (Netsh query)
Write-Host "[+] Querying active TCP dynamic port settings..." -ForegroundColor Yellow

$IPv4Ports = netsh int ipv4 show dynamicport tcp
$IPv6Ports = netsh int ipv6 show dynamicport tcp

Write-Host "--- IPv4 Dynamic Port Output ---" -ForegroundColor Gray
$IPv4Ports | Out-String | Write-Host -ForegroundColor DarkGray
Write-Host "--- IPv6 Dynamic Port Output ---" -ForegroundColor Gray
$IPv6Ports | Out-String | Write-Host -ForegroundColor DarkGray

Verify if dynamic ranges are narrowed
$RpcInternetPath = "HKLM:\SOFTWARE\Microsoft\Rpc\Internet"
if (Test-Path $RpcInternetPath) {
 Write-Host "[!] VULNERABLE: Restrictive RPC Internet registry settings found. Narrowing dynamic ranges is not recommended." -ForegroundColor Red
 $vulnerable = $true
} else {
 Write-Host "[+] Restrictive RPC Internet registry settings are absent." -ForegroundColor Green
}

if ($vulnerable) {
 Write-Host "Audit result: NON-COMPLIANT" -ForegroundColor Red
} else {
 Write-Host "Audit result: COMPLIANT" -ForegroundColor Green
}
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R8 (Administration network subnets) \* \*\*Microsoft Security Guidance\*\*: Restricting Active Directory RPC Traffic to a Specific Port \* \*\*CIS Windows Server 2016 Benchmark\*\*: Section 19 (Windows Defender Firewall with Advanced Security) \* \*\*DSInternals AD Firewall Guide (Michael Grafnetter)\*\*: [Active Directory Firewall - Domain Controller Firewall](https://firewall.dsinternals.com/ADDS/)



**# [REQ-NET-003] Configure Workstation and Server Isolation**

**## Target Scope** \* **Applicable Systems**: Tier 2 Client Workstations, Member Servers. \* **Operating Systems**: Windows Server 2016 (and above), Windows 10 (and above) Enterprise/Professional.

---

**## Implementation Details** \* **Priority**: High \* **GPO Path / Registry Location**: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Windows Defender Firewall with Advanced Security`

---

**## Rationale** Once an adversary establishes initial access on a Tier 2 client workstation or a Member Server, they will attempt to move laterally across the network to identify high-value targets, harvest credentials, and locate Tier 0 administrative pathways.

Lateral movement commonly relies on standard management and remote connection protocols, including SMB (TCP 445), RPC (TCP 135 and dynamic ports), RDP (TCP 3389), and WinRM (TCP 5985/5986). In a standard enterprise design, workstations do not require inbound connections from other workstations, and member servers rarely require inbound connections from peer member servers in the same tier.

Configuring local firewalls via Group Policy to explicitly block inbound SMB, RPC, RDP, and WinRM traffic originating from peer subnets (while maintaining administrative exceptions from authorized management subnets and Domain Controllers) stops host-to-host lateral propagation and containment is maintained.

---

**## Legacy Impact & Compatibility** \* **Peer-to-Peer Sharing**: Local network resource sharing (such as peer-to-peer file folders, local network printer sharing, or remote system diagnostics) will fail. \* **Administrative Jump Hosts**: System administrators must use designated Privileged Access Workstations (PAWs) or Management Jump Hosts located in the allowed administrative subnets to manage these systems. \* **Application Clustered Services**: Member servers configured in clusters or running distributed applications that require peer-to-peer communication will require explicit, highly restricted exclusions to permit replication and synchronization ports.

---

**## Implementation Steps****### Option A: Group Policy Object (GPO) Configuration (Preferred)**

**#### 1. Configure Peer Isolation GPO Rules** 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Create or edit a GPO linked to the workstations OU (e.g., `GPO\_Hardening\_Workstation\_Isolation`) or Member Servers OU. 3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security` 4. Under **Inbound Rules**, create new custom rules to block peer traffic: \* **Rule 1: Block Inbound SMB from Peers**: \* **Action**: `Block the connection` \* **Protocol**: `TCP` | **Local Port**: `445` \* **Remote Address**: `[Insert Local Client / Peer Subnets, e.g., 10.20.0.0/16]` \* **Profile**: `Domain, Private` \* **Rule 2: Block Inbound RDP from Peers**: \* **Action**: `Block the connection` \* **Protocol**: `TCP` | **Local Port**: `3389` \* **Remote Address**: `[Insert Local Client / Peer Subnets]` \* **Profile**: `Domain, Private` \* **Rule 3: Block Inbound WinRM from Peers**: \* **Action**: `Block the connection` \* **Protocol**: `TCP` | **Local Port**: `5985, 5986` \* **Remote Address**: `[Insert Local Client / Peer Subnets]` \* **Profile**: `Domain, Private` \* **Rule 4: Block Inbound RPC from Peers**: \* **Action**: `Block the connection` \* **Protocol**: `TCP` | **Local Port**: `135, 49152-65535` \* **Remote Address**: `[Insert Local Client / Peer Subnets]` \* **Profile**: `Domain, Private`

**#### 2. Create Management Allow Exceptions** In the same GPO, ensure there are priority inbound **Allow** rules configured to permit administration from dedicated administrative paths: \* **Allow Inbound Administration**: \* **Action**: `Allow the connection` \* **Protocol**: `TCP` | **Local Port**: `445, 3389, 5985, 5986` \* **Remote Address**: `[Insert Administrative Subnet (PAW / Jump Hosts), e.g., 10.10.0.0/24]` \* **Profile**: `Domain`

---

**### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)**

Run the following scripts locally to audit and apply workstation/server isolation rules.

#### Remediation Script: [Download Script: Set-WorkstationIsolation.ps1](implementation\_scripts/Set-WorkstationIsolation.ps1)

```
Set-WorkstationIsolation.ps1
Configures local firewall rules to block inbound SMB, RPC, and RDP from peer subnets.
Allows access only from designated Domain Controller and Admin Management subnets.

Adjust subnets for your local environment
$AdminSubnet = "10.10.0.0/24" # PAW / Jump Host / DC Subnet
$PeerSubnet = "10.20.0.0/16" # Local client/member peer subnet

Write-Host "Applying Workstation and Server Isolation Firewall Rules..." -ForegroundColor Cyan

1. Enable firewall profiles
Set-NetFirewallProfile -Profile Domain, Public, Private -Enabled True
Write-Host "All firewall profiles enabled." -ForegroundColor Green

2. Block Inbound SMB (TCP 445) from peer subnet
New-NetFirewallRule -DisplayName "Hardening: Block Inbound SMB from Peers" `
 -Direction Inbound `
 -Action Block `
 -Protocol TCP `
 -LocalPort 445 `
 -RemoteAddress $PeerSubnet `
 -Profile Domain, Private `
 -Enabled True | Out-Null
Write-Host "SMB peer blocking rule created." -ForegroundColor Green

3. Block Inbound RDP (TCP 3389) from peer subnet
New-NetFirewallRule -DisplayName "Hardening: Block Inbound RDP from Peers" `
 -Direction Inbound `
 -Action Block `
 -Protocol TCP `
 -LocalPort 3389 `
 -RemoteAddress $PeerSubnet `
 -Profile Domain, Private `
 -Enabled True | Out-Null
Write-Host "RDP peer blocking rule created." -ForegroundColor Green

4. Block Inbound WinRM (TCP 5985, 5986) from peer subnet
New-NetFirewallRule -DisplayName "Hardening: Block Inbound WinRM from Peers" `
 -Direction Inbound `
 -Action Block `
 -Protocol TCP `
 -LocalPort @(5985, 5986) `
 -RemoteAddress $PeerSubnet `
 -Profile Domain, Private `
 -Enabled True | Out-Null
Write-Host "WinRM peer blocking rule created." -ForegroundColor Green

5. Block Inbound RPC (TCP 135) from peer subnet
New-NetFirewallRule -DisplayName "Hardening: Block Inbound RPC Mapper from Peers" `
 -Direction Inbound `
 -Action Block `
 -Protocol TCP `
 -LocalPort 135 `
 -RemoteAddress $PeerSubnet `
 -Profile Domain, Private `
 -Enabled True | Out-Null
Write-Host "RPC Endpoint Mapper peer blocking rule created." -ForegroundColor Green

6. Allow Inbound Administration from Management Subnet (RDP, WinRM, SMB)
New-NetFirewallRule -DisplayName "Hardening: Allow Admin Management Inbound" `
 -Direction Inbound `
 -Action Allow `
 -Protocol TCP `
 -LocalPort @(445, 3389, 5985, 5986) `
 -RemoteAddress $AdminSubnet `
 -Profile Domain `
 -Enabled True | Out-Null
Write-Host "Management subnet inbound allowance rule created." -ForegroundColor Green

Write-Host "Workstation and Server isolation firewall rules applied successfully." -ForegroundColor Cyan
```

#### Audit Script: [Download Script: Test-WorkstationIsolation.ps1](audit\_scripts/Test-WorkstationIsolation.ps1)

```
Test-WorkstationIsolation.ps1
Audits the presence of isolation blocking rules on local firewall profiles.

Write-Host "Auditing workstation and server peer isolation rules..." -ForegroundColor Cyan

$PortsToVerify = @(445, 3389, 135)
$FailedChecks = 0

foreach ($Port in $PortsToVerify) {
 # Query rules that block inbound traffic on specified ports
 $BlockRules = Get-NetFirewallRule -ErrorAction SilentlyContinue | Where-Object {
 $_.Direction -eq "Inbound" -and
 $_.Action -eq "Block" -and
 $_.Enabled -eq $true
 }

 $HasPortBlock = $false
 foreach ($Rule in $BlockRules) {
 # Check filter associated with the rule to resolve local port
 $Filter = Get-NetFirewallPortFilter -AssociatedNetFirewallRule $Rule -ErrorAction SilentlyContinue
 if ($Filter -and $Filter.LocalPort -eq [string]$Port) {
 $HasPortBlock = $true
 }
 }

 if ($HasPortBlock) {
 Write-Host " - Isolation block rule for Port $($Port): FOUND (Compliant)" -ForegroundColor Green
 } else {
 Write-Host " - Isolation block rule for Port $($Port): NOT FOUND (Non-Compliant)" -ForegroundColor Red
 $FailedChecks++
 }
}

if ($FailedChecks -eq 0) {
 Write-Host "Audit Result: Peer isolation firewall rules are verified." -ForegroundColor Green
} else {
 Write-Warning "Audit Result: Missing peer isolation rules detected!"
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R8 (Administration network subnets) \* \*\*CIS Windows Server 2016 Benchmark\*\*: Section 19 (Windows Defender Firewall with Advanced Security) \* \*\*CIS Windows 10 Benchmark\*\*: Section 19 (Windows Defender Firewall with Advanced Security)

## # [REQ-NET-004] Configure IPsec Domain Isolation

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers, PAWs, Tier 2 Client Workstations. \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10 (and above) Enterprise/Professional.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Connection Security Rules`

## Rationale In environments without hardware-enforced line-encryption, an attacker who gains physical or logical access to internal network switches can perform Man-in-the-Middle (MitM) attacks (e.g., ARP spoofing, DHCP spoofing) or passive packet sniffing.

Implementing IPsec Transport Mode using Connection Security Rules ensures that domain-joined hosts cryptographically authenticate each other before transmitting payloads.

Benefits of IPsec isolation include:

1. **Host Authentication:** Ensures only trusted, domain-joined systems communicating via Kerberos V5 or certificates can exchange packets with critical servers.
2. **Data Integrity & Confidentiality:** Prevents packet tampering and sniffing on the wire. For DC-to-DC replication, mandating ESP encryption secures highly sensitive directory updates.
3. **Mitigation of Relay Attacks:** Even if credentials are intercepted, they cannot be easily replayed to services protected by IPsec isolation rules.

### Bootstrap & Kerberos Authentication Challenge In an IPsec-enforced environment, domain members require Kerberos tickets to authenticate and establish IPsec Security Associations (SAs). However, clients must communicate with Domain Controllers to obtain these Kerberos tickets. If both endpoints and Domain Controllers strictly enforce IPsec authentication from the start, a deadlock (chicken-and-egg lockout) occurs.

Therefore, Domain Controllers must allow unauthenticated initial requests (Request mode) to bootstrap clients, and clients must use Request mode or have explicit exemptions for the Domain Controllers during their boot sequence.

## Legacy Impact & Compatibility \* \*\*Non-Windows and Standalone Systems\*\*: Linux/Unix servers, network appliances, IP cameras, and network printers that do not participate in Active Directory Kerberos authentication will fail to connect. An **IPsec Boundary Group** (exemption list) must be configured to allow these hosts to communicate in cleartext. \* \*\*Network Performance\*\*: IPsec encryption and authentication overhead may slightly increase CPU usage on older hardware, though modern CPUs with AES-NI support experience negligible latency. \* \*\*Deployment Sequence\*\*: Connection Security Rules should always be set to **Request** authentication first. Once audit logs confirm all legitimate systems are successfully authenticating, the rules can be safely transitioned to **Require** authentication (inbound) to prevent self-lockout. \* \*\*BitLocker Network Unlock Compatibility\*\*: BitLocker Network Unlock sends a key-unlock payload in a cleartext DHCP request from the UEFI network stack in the pre-boot environment. Because this exchange happens before the Windows operating system and its IPsec driver are loaded, the client cannot negotiate IPsec SAs. If the Windows Deployment Services (WDS) server hosting the Network Unlock provider requires IPsec authentication for all incoming traffic, it will drop the unauthenticated UEFI client packets. To prevent this, DHCP traffic (UDP ports 67 and 68) must be exempted from IPsec enforcement on the WDS server, or the WDS server's IP address must be added to the IPsec exemption list for DHCP communications. \* \*\*Network Firewall Routing\*\*: Internal hardware network firewalls and router access control lists (ACLs) separating subnets must permit the transmission of IPsec negotiation and transport traffic between domain member hosts. Specifically, UDP port 500 (Internet Key Exchange / IKE), UDP port 4500 (IPsec NAT-Traversal / NAT-T, if applicable), and IP Protocol 50 (Encapsulating Security Payload / ESP) must be permitted across subnet boundaries. \* \*\*Hyper-V Virtualization Hosts\*\*: If Hyper-V virtual machines are running on a host, the host requires DNS query traffic (TCP/UDP port 53) to be allowed inbound on all virtual network switch interfaces to ensure VMs can reach local DNS services for name resolution.

## ## Implementation Steps

### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

To implement domain isolation successfully, two separate GPO policies must be created and linked: one targeting standard Endpoints (Member Servers and Workstations) and one targeting Domain Controllers.

#### 1. Endpoint Domain Isolation GPO (Isolated Domain) This GPO targets all standard domain assets (Workstations, Member Servers, PAWs).

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Create or edit a GPO targeting the target OUs (e.g., `GPO_Hardening_IPsec_Isolation_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Connection Security Rules`
4. Create the general Isolation Rule:
  - Right-click **Connection Security Rules** and select **New Rule...**
  - **Rule Type:** `Isolation`
  - **Requirements:** Select `Request authentication for inbound and outbound connections` (to allow cleartext fallback).
  - **Authentication Method:** `Computer (Kerberos V5)`
  - **Profile:** `Domain`

- **Name:** Hardening: IPsec Domain Isolation

5. Configure Boot Exemptions (to prevent lockouts): Create a new rule for essential infrastructure services:

- **Rule Type:** Exemption
- **IP Addresses:** Add the IP addresses of your DHCP servers and DNS servers (if external or required for initial boot name resolution).
- **Name:** Hardening: IPsec Infrastructure Exemptions

*Note: Domain Controllers must **not** be added to the exemption list. If DCs are exempted, all endpoint-to-DC traffic will bypass IPsec entirely. Instead, by keeping DCs subject to the general rule with Request outbound requirements, clients can fall back to cleartext during boot to obtain a Kerberos ticket, and will automatically establish an encrypted IPsec SA for all subsequent DC communications once authenticated. Once the domain isolation environment is stable and all domain assets have active SAs, the GPO requirement for endpoints can be upgraded to Require authentication for inbound connections and request authentication for outbound connections.*

#### 2. Domain Controller IPsec GPO This GPO targets only the Domain Controllers OU. Because DCs must process bootstrap authentication from unjoined or booting machines, they cannot require IPsec for client access.

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Create or edit a GPO targeting the Domain Controllers OU (e.g., `GPO_Hardening_IPsec_DCs` ).
3. Navigate to Connection Security Rules.
4. Create the DC Client Access Rule:
  - **Rule Type:** Isolation
  - **Requirements:** Request authentication for inbound and outbound connections
  - **Authentication Method:** Computer (Kerberos V5)
  - **Profile:** Domain
  - **Name:** Hardening: IPsec DC Client Access

5. Create the DC-to-DC Replication Encryption Rule:

- **Rule Type:** Isolation
- **Requirements:** Require authentication for inbound and outbound connections
- **Authentication Method:** Computer (Kerberos V5)
- **Protocols and Ports:** Set protocol to TCP , local port to 49152-65535 (or your restricted RPC port, e.g., 50000-50100 ).
- **Action:** Under advanced settings, require **ESP encryption** for this connection rule.
- **Name:** Hardening: IPsec DC-to-DC Replication

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to audit and configure Connection Security Rules.

#### Remediation Script: [Download Script: Set-IPsecDomainIsolation.ps1](implementation\_scripts/Set-IPsecDomainIsolation.ps1)

```

Set-IPsecDomainIsolation.ps1
Configures local IPsec Connection Security Rules for Domain Isolation.
Detects role (DC vs Endpoint) and applies appropriate isolation policies.

Write-Host "Configuring IPsec Connection Security Rules..." -ForegroundColor Cyan

1. Determine local machine role (Domain Controller vs Endpoint/Member Server)
$IsDomainController = $false
try {
 $ComputerSystem = Get-CimInstance -ClassName Win32_ComputerSystem -ErrorAction Stop
 if ($ComputerSystem.DomainRole -eq 4 -or $ComputerSystem.DomainRole -eq 5) {
 $IsDomainController = $true
 }
} catch {
 # Fallback to checking NTDS service or environment variables if CimInstance fails
 if (Get-Service -Name NTDS -ErrorAction SilentlyContinue) {
 $IsDomainController = $true
 }
}

if ($IsDomainController) {
 Write-Host "Local system identified as a Domain Controller." -ForegroundColor Yellow

 # Define Rule Names
 $GeneralRuleName = "Hardening: IPsec DC Client Access"
 $DCDCRuleName = "Hardening: IPsec DC-to-DC Replication"

 # A. DC General Client Access Rule: Inbound/Outbound set to Request
 # Allows initial cleartext bootstrap (Kerberos, DNS, LDAP) for clients, then promotes to IPsec
 $ExistingGeneral = Get-NetIPsecRule -DisplayName $GeneralRuleName -ErrorAction SilentlyContinue
 if ($null -eq $ExistingGeneral) {
 New-NetIPsecRule -DisplayName $GeneralRuleName `
 -InboundSecurity Request `
 -OutboundSecurity Request `
 -Phase1AuthSet "ComputerKerberos" `
 -Enabled True | Out-Null
 Write-Host "Created general DC IPsec rule (Request mode)." -ForegroundColor Green
 } else {
 Set-NetIPsecRule -DisplayName $GeneralRuleName `
 -InboundSecurity Request `
 -OutboundSecurity Request `
 -Phase1AuthSet "ComputerKerberos" `
 -Enabled True | Out-Null
 Write-Host "Updated general DC IPsec rule (Request mode)." -ForegroundColor Gray
 }

 # B. DC-to-DC Replication Rule: Require authentication and require encryption
 # Targets replication ports or remote DC subnets
 $ExistingDCDC = Get-NetIPsecRule -DisplayName $DCDCRuleName -ErrorAction SilentlyContinue

 # Attempt to retrieve other DCs in the domain for remote IP targeting
 $DCIps = @()
 try {
 if (Get-Module -ListAvailable -Name ActiveDirectory) {
 Import-Module ActiveDirectory -ErrorAction Stop
 $DCIps = Get-ADDomainController -Filter * | Where-Object { $_.IPv4Address -ne (Get-NetIPAddress -AddressFamily IPv4 | Se
lect-Object -ExpandProperty IPAddress) } | Select-Object -ExpandProperty IPv4Address
 }
 } catch {
 Write-Host "Could not query AD for other DC IP addresses. Rule will apply generally to replication ports." -ForegroundColor
Yellow
 }

 # Configure the rule targeting replication traffic (TCP 49152-65535 or custom RPC)
 # Require authentication (forces IPsec) for DC-to-DC communication
 $Params = @{
 DisplayName = $DCDCRuleName
 InboundSecurity = "Require"
 OutboundSecurity = "Require"
 Phase1AuthSet = "ComputerKerberos"
 Protocol = "TCP"
 LocalPort = "49152-65535"
 Enabled = "True"
 }
}

```

```

if ($DCIps.Count -gt 0) {
 $Params["RemoteAddress"] = $DCIps
}

if ($null -eq $ExistingDCDC) {
 New-NetIPsecRule @Params | Out-Null
 Write-Host "Created DC-to-DC replication encryption rule (Require mode)." -ForegroundColor Green
} else {
 # RemoteAddress cannot be passed empty if we update, so omit if empty
 if ($null -eq $Params["RemoteAddress"]) {
 Set-NetIPsecRule -DisplayName $DCDCRuleName `
 -InboundSecurity Require `
 -OutboundSecurity Require `
 -Phase1AuthSet "ComputerKerberos" `
 -Protocol TCP `
 -LocalPort "49152-65535" `
 -Enabled True | Out-Null
 } else {
 Set-NetIPsecRule @Params | Out-Null
 }
 Write-Host "Updated DC-to-DC replication encryption rule (Require mode)." -ForegroundColor Gray
}
} else {
 Write-Host "Local system identified as an Endpoint/Member Server." -ForegroundColor Yellow

 $RuleName = "Hardening: IPsec Domain Isolation"
 $ExistingRule = Get-NetIPsecRule -DisplayName $RuleName -ErrorAction SilentlyContinue

 # Endpoints: Request inbound and Request outbound for safe deployment,
 # then promote to Require inbound and Request outbound (fallback to cleartext for DCs/Internet)
 if ($null -eq $ExistingRule) {
 New-NetIPsecRule -DisplayName $RuleName `
 -InboundSecurity Request `
 -OutboundSecurity Request `
 -Phase1AuthSet "ComputerKerberos" `
 -Enabled True | Out-Null
 Write-Host "Created general Endpoint IPsec rule (Request mode)." -ForegroundColor Green
 } else {
 Set-NetIPsecRule -DisplayName $RuleName `
 -InboundSecurity Request `
 -OutboundSecurity Request `
 -Phase1AuthSet "ComputerKerberos" `
 -Enabled True | Out-Null
 Write-Host "Updated general Endpoint IPsec rule (Request mode)." -ForegroundColor Gray
 }
}

Create Exemption Rules for DHCP and DNS to prevent boot lockouts
$DHCPRuleName = "Exempt: DHCP Traffic"
$DNSRuleName = "Exempt: DNS Traffic"

DHCP Rule (UDP 67, 68)
$ExistingDHCP = Get-NetIPsecRule -DisplayName $DHCPRuleName -ErrorAction SilentlyContinue
if ($null -eq $ExistingDHCP) {
 New-NetIPsecRule -DisplayName $DHCPRuleName `
 -InboundSecurity None `
 -OutboundSecurity None `
 -Protocol UDP `
 -LocalPort @("67", "68") `
 -Enabled True | Out-Null
 Write-Host "Created DHCP exemption rule." -ForegroundColor Green
}

DNS Rule (UDP/TCP 53)
$ExistingDNS = Get-NetIPsecRule -DisplayName $DNSRuleName -ErrorAction SilentlyContinue
if ($null -eq $ExistingDNS) {
 New-NetIPsecRule -DisplayName $DNSRuleName `
 -InboundSecurity None `
 -OutboundSecurity None `
 -Protocol UDP `
 -RemotePort "53" `
 -Enabled True | Out-Null
 New-NetIPsecRule -DisplayName "$DNSRuleName (TCP)" `
 -InboundSecurity None `
 -OutboundSecurity None `
 -Protocol TCP `
 -RemotePort "53" `
 -Enabled True | Out-Null
}

```



```
 Write-Host "Created DNS exemption rules." -ForegroundColor Green
 }
}
```

#### Audit Script: [Download Script: Test-IPsecDomainIsolation.ps1](audit\_scripts/Test-IPsecDomainIsolation.ps1)

```

Test-IPsecDomainIsolation.ps1
Checks the state of local IPsec Connection Security Rules.
Accounts for role-specific rules (DC vs Endpoint).

Write-Host "Auditing IPsec Connection Security Rules..." -ForegroundColor Cyan

1. Determine local machine role
$IsDomainController = $false
try {
 $ComputerSystem = Get-CimInstance -ClassName Win32_ComputerSystem -ErrorAction Stop
 if ($ComputerSystem.DomainRole -eq 4 -or $ComputerSystem.DomainRole -eq 5) {
 $IsDomainController = $true
 }
} catch {
 if (Get-Service -Name NTDS -ErrorAction SilentlyContinue) {
 $IsDomainController = $true
 }
}

$NonCompliant = $false

if ($IsDomainController) {
 Write-Host "Auditing Domain Controller IPsec rules..." -ForegroundColor Yellow

 # DC should have a Client Access rule set to Request (or better)
 $GeneralRule = Get-NetIPsecRule -DisplayName "Hardening: IPsec DC Client Access" -ErrorAction SilentlyContinue
 if ($null -eq $GeneralRule -or $GeneralRule.Enabled -ne $true) {
 Write-Host " - General DC IPsec rule: NOT FOUND or DISABLED (Non-Compliant)" -ForegroundColor Red
 $NonCompliant = $true
 } else {
 $InboundSec = $GeneralRule.InboundSecurity
 $OutboundSec = $GeneralRule.OutboundSecurity
 if ($InboundSec -ne "Request" -and $InboundSec -ne "Require") {
 Write-Host " - General DC IPsec Inbound Security is '$InboundSec' (Non-Compliant, should be Request)" -ForegroundColor Red
 $NonCompliant = $true
 } else {
 Write-Host " - General DC IPsec Inbound Security: $InboundSec (Compliant)" -ForegroundColor Green
 }
 }

 # DC should have a DC-to-DC replication rule set to Require
 $DCDCRule = Get-NetIPsecRule -DisplayName "Hardening: IPsec DC-to-DC Replication" -ErrorAction SilentlyContinue
 if ($null -eq $DCDCRule -or $DCDCRule.Enabled -ne $true) {
 Write-Host " - DC-to-DC Replication rule: NOT FOUND or DISABLED (Non-Compliant)" -ForegroundColor Red
 $NonCompliant = $true
 } else {
 $InboundSec = $DCDCRule.InboundSecurity
 $OutboundSec = $DCDCRule.OutboundSecurity
 if ($InboundSec -ne "Require" -or $OutboundSec -ne "Require") {
 Write-Host " - DC-to-DC IPsec Inbound/Outbound is '$InboundSec'/'$OutboundSec' (Non-Compliant, should be Require)" -ForegroundColor Red
 $NonCompliant = $true
 } else {
 Write-Host " - DC-to-DC IPsec rule: Require (Compliant)" -ForegroundColor Green
 }
 }
} else {
 Write-Host "Auditing Endpoint/Member Server IPsec rules..." -ForegroundColor Yellow

 # Endpoint should have a general Domain Isolation rule set to Request (transition) or Require (inbound) / Request (outbound)
 $GeneralRule = Get-NetIPsecRule -DisplayName "Hardening: IPsec Domain Isolation" -ErrorAction SilentlyContinue
 if ($null -eq $GeneralRule -or $GeneralRule.Enabled -ne $true) {
 Write-Host " - Endpoint Domain Isolation rule: NOT FOUND or DISABLED (Non-Compliant)" -ForegroundColor Red
 $NonCompliant = $true
 } else {
 $InboundSec = $GeneralRule.InboundSecurity
 $OutboundSec = $GeneralRule.OutboundSecurity

 # Request/Request or Require/Request are acceptable depending on transition phase
 if ($InboundSec -eq "None" -or $OutboundSec -eq "None") {
 Write-Host " - Endpoint Domain Isolation is set to None (Non-Compliant)" -ForegroundColor Red
 $NonCompliant = $true
 } else {

```

```

 Write-Host " - Endpoint Domain Isolation rule: Enabled (Inbound: $InboundSec, Outbound: $OutboundSec) (Compliant)" -ForegroundColor Green
 }
}

DHCP and DNS exemptions should be configured if outbound is strict
$DHCPRule = Get-NetIPsecRule -DisplayName "Exempt: DHCP Traffic" -ErrorAction SilentlyContinue
if ($null -eq $DHCPRule) {
 Write-Host " - DHCP Exemption rule: NOT FOUND (Warning: highly recommended to prevent DHCP issues)" -ForegroundColor Yellow
} else {
 Write-Host " - DHCP Exemption rule: FOUND (Compliant)" -ForegroundColor Green
}

$DNSRule = Get-NetIPsecRule -DisplayName "Exempt: DNS Traffic" -ErrorAction SilentlyContinue
if ($null -eq $DNSRule) {
 Write-Host " - DNS Exemption rule: NOT FOUND (Warning: highly recommended to prevent DNS name resolution failures during boot)" -ForegroundColor Yellow
} else {
 Write-Host " - DNS Exemption rule: FOUND (Compliant)" -ForegroundColor Green
}
}

if ($NonCompliant) {
 Write-Host "IPsec Domain Isolation Audit: Non-Compliant." -ForegroundColor Red
 exit 1
} else {
 Write-Host "IPsec Domain Isolation Audit: Compliant." -ForegroundColor Green
 exit 0
}
}

```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R7 (IPsec transport mode for domain isolation) \*  
 \*\*CIS Windows Server 2016 Benchmark\*\*: Section 19 (Windows Defender Firewall with Advanced Security) \* \*\*Microsoft Security  
 Guidance\*\*: IPsec Domain Isolation Policies

## # [REQ-NET-005] Harden IPsec Cryptographic Configurations

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers, PAWs, Tier 2 Client Workstations. \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10 (and above) Enterprise/Professional.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Windows Defender Firewall with Advanced Security - [LDAP]` (Properties → IPsec Settings → IPsec Defaults → Customize)

## Rationale Default IPsec configurations in older Windows deployments or standard configurations allow weak encryption and integrity algorithms (such as 3DES, DES, MD5, SHA-1, and Diffie-Hellman Groups 1, 2, or 5). These legacy algorithms are vulnerable to key recovery attacks, pre-computation table attacks, and cryptographic collisions. To maintain secure communications through 2030 and beyond, IPsec configurations must be restricted to modern cryptographic standards. Utilizing Advanced Encryption Standard (AES-256) and Elliptic Curve Diffie-Hellman (ECDH) Group 19 (256-bit) or Group 20 (384-bit) provides strong mathematical protection against decryption and ensures perfect forward secrecy (PFS). Mandating these suites prevents protocol downgrade attacks and secures sensitive domain replication and management traffic.

## Legacy Impact & Compatibility \* \*\*Legacy OS Support\*\*: Outdated operating systems (such as Windows 7, Windows Server 2008 R2, or old Linux kernels that do not support modern DH groups or AES-256) will fail to establish IPsec security associations (SAs). Ensure all domain member systems are upgraded to supported OS versions before applying this policy. \* \*\*Non-Domain / Third-Party Appliances\*\*: Network devices, storage systems, and Linux servers that communicate within the IPsec domain boundary must support these strong suites. If they do not, they must be added to the IPsec exemption list (Boundary Group) to maintain cleartext communications. \* \*\*Performance Impact\*\*: Modern CPUs include hardware acceleration (AES-NI) for AES encryption, meaning the performance overhead of moving from AES-128 or 3DES to AES-256 is negligible.

### ## Implementation Steps

#### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Create a new GPO or edit an existing one (e.g., `GPO_Hardening_IPsec_Cryptography` ) targeting the appropriate Organizational Units.
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Windows Defender Firewall with Advanced Security - [LDAP]`
4. Right-click **Windows Defender Firewall with Advanced Security - [LDAP]** and select **Properties**.
5. Select the **IPsec Settings** tab.
6. Under **IPsec defaults**, click **Customize...**
7. Under **Key exchange (Main Mode)**, select **Advanced** and click **Customize...**
8. Configure the security methods:
  - Remove any entries referencing SHA-1, MD5, 3DES, DES, or DH Groups 1, 2, or 5.
  - Add or reorder methods so the preferred method is first:
    - **Encryption**: `AES-256`
    - **Integrity**: `SHA-256` or `SHA-384`
    - **Key exchange algorithm**: `Elliptic Curve Diffie-Hellman Group 19` (or `Group 20` )
9. Click **OK**.
10. Under **Data protection (Quick Mode)**, select **Advanced** and click **Customize...**
11. Check **Require encryption for all connection security rules that use these settings**.
12. Configure the data integrity and encryption rules:
  - Remove any rules using SHA-1, MD5, 3DES, or DES.
  - Add a custom rule:
    - **Protocol**: `ESP`
    - **Encryption**: `AES-256` (or `AES-GCM 256` )
    - **Integrity**: `SHA-256` (or `None` if utilizing GCM mode)
13. Click **OK** on all dialog boxes to save the configurations.

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to audit and configure custom cryptographic sets on individual systems.

#### Remediation Script: [Download Script: Set-IPsecCryptography.ps1](implementation\_scripts/Set-IPsecCryptography.ps1)

```
Set-IPsecCryptography.ps1
Description: Configures local IPsec Main Mode and Quick Mode custom cryptographic configurations.

Write-Host "Configuring hardened IPsec cryptographic settings..." -ForegroundColor Cyan

Phase 1: Define Main Mode cryptographic proposal
$MMProposal = New-NetIPsecMainModeCryptoProposal -Encryption AES256 -Hash SHA256 -KeyExchange DH19

Manage Main Mode Crypto Set
$MMSetName = "Hardened_MM_CryptoSet"
$ExistingMM = Get-NetIPsecMainModeCryptoSet -DisplayName $MMSetName -ErrorAction SilentlyContinue

if ($null -eq $ExistingMM) {
 New-NetIPsecMainModeCryptoSet -DisplayName $MMSetName -Proposal $MMProposal | Out-Null
 Write-Host "Created Main Mode crypto set." -ForegroundColor Green
} else {
 Set-NetIPsecMainModeCryptoSet -DisplayName $MMSetName -Proposal $MMProposal | Out-Null
 Write-Host "Updated Main Mode crypto set." -ForegroundColor Gray
}

Phase 2: Define Quick Mode cryptographic proposal
$QMProposal = New-NetIPsecQuickModeCryptoProposal -Encapsulation ESP -Encryption AES256 -ESPHash SHA256

Manage Quick Mode Crypto Set
$QMSetName = "Hardened_QM_CryptoSet"
$ExistingQM = Get-NetIPsecQuickModeCryptoSet -DisplayName $QMSetName -ErrorAction SilentlyContinue

if ($null -eq $ExistingQM) {
 New-NetIPsecQuickModeCryptoSet -DisplayName $QMSetName -Proposal $QMProposal | Out-Null
 Write-Host "Created Quick Mode crypto set." -ForegroundColor Green
} else {
 Set-NetIPsecQuickModeCryptoSet -DisplayName $QMSetName -Proposal $QMProposal | Out-Null
 Write-Host "Updated Quick Mode crypto set." -ForegroundColor Gray
}

Associate sets with all local Connection Security Rules and Main Mode Rules
$Rules = Get-NetIPsecRule -ErrorAction SilentlyContinue
if ($null -ne $Rules) {
 foreach ($Rule in $Rules) {
 Set-NetIPsecRule -DisplayName $Rule.DisplayName -QuickModeCryptoSet $QMSetName -ErrorAction SilentlyContinue | Out-Null
 }
}

$MMRules = Get-NetIPsecMainModeRule -ErrorAction SilentlyContinue
if ($null -ne $MMRules) {
 foreach ($MMRule in $MMRules) {
 Set-NetIPsecMainModeRule -DisplayName $MMRule.DisplayName -MainModeCryptoSet $MMSetName -ErrorAction SilentlyContinue | Out-Null
 }
}

Write-Host "IPsec cryptography configuration applied." -ForegroundColor Green
```

#### Audit Script: [Download Script: Test-IPsecCryptography.ps1](audit\_scripts/Test-IPsecCryptography.ps1)

```
Test-IPsecCryptography.ps1
Description: Checks that IPsec rules only utilize strong cryptographic sets.

Write-Host "Auditing IPsec cryptographic configurations..." -ForegroundColor Cyan

$NonCompliantCount = 0
$Rules = Get-NetIPsecRule -ErrorAction SilentlyContinue

if ($null -eq $Rules -or $Rules.Count -eq 0) {
 Write-Host " - No IPsec rules found to audit." -ForegroundColor Gray
} else {
 foreach ($Rule in $Rules) {
 $QMSetName = $Rule.QuickModeCryptoSet
 if ($null -eq $QMSetName -or $QMSetName -eq "") {
 Write-Host " - Rule '$($Rule.DisplayName)' is using default/unconfigured Quick Mode cryptography (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
 } else {
 $QMSet = Get-NetIPsecQuickModeCryptoSet -Name $QMSetName -ErrorAction SilentlyContinue
 if ($null -eq $QMSet) {
 Write-Host " - Rule '$($Rule.DisplayName)' references non-existent crypto set: $($QMSetName) (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
 } else {
 $Compliant = $true
 foreach ($Proposal in $QMSet.Proposals) {
 if ($Proposal.Encryption -ne "AES256" -and $Proposal.Encryption -ne "AESGCM256") {
 $Compliant = $false
 }
 if ($Proposal.ESPHash -ne "SHA256" -and $Proposal.ESPHash -ne "SHA384") {
 $Compliant = $false
 }
 }

 if ($Compliant) {
 Write-Host " - Rule '$($Rule.DisplayName)' matches cryptographic standards (Compliant)." -ForegroundColor Green
 } else {
 Write-Host " - Rule '$($Rule.DisplayName)' uses weak encryption or hashing methods (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
 }
 }
 }
 }
}

if ($NonCompliantCount -eq 0) {
 Write-Host "IPsec Cryptography Audit: Compliant." -ForegroundColor Green
} else {
 Write-Host "IPsec Cryptography Audit: Non-Compliant." -ForegroundColor Red
}
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R7 (IPsec transport mode for domain isolation) \* \*\*ANSSI General Security Rules (RGS)\*\*: Annex B1 (Cryptographic mechanisms) \* \*\*CIS Windows Server 2016 Benchmark\*\*: Section 19 (Windows Defender Firewall with Advanced Security) \* \*\*NIST Special Publication 800-131A\*\*: Transitions to Stronger Cryptographic Algorithms

## # [REQ-NET-006] Harden TLS Protocols, Cipher Suites, and Elliptic Curves

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers, PAWs, Tier 2 Client Workstations. \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10 (and above).

## ## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Protocols\*\*:

`HKLM:\SYSTEM\CurrentControlSet\Control\SecurityProviders\SCHANNEL\Protocols` \* \*\*Cipher Suite Order\*\*: `Computer Configuration\Administrative Templates\Network\SSL Configuration Settings` → \*\*SSL Cipher Suite Order\*\* (Registry: `HKLM:\SOFTWARE\Policies\Microsoft\Cryptography\Configuration\SSL\00010002` → `Functions`) \* \*\*ECC Curve Order\*\*: `Computer Configuration\Administrative Templates\Network\SSL Configuration Settings` → \*\*ECC Curve Order\*\* (Registry: `HKLM:\SOFTWARE\Policies\Microsoft\Cryptography\Configuration\SSL\00010002` → `EccCurves`) \* \*\*.NET strong cryptography support\*\*: \* `HKLM:\SOFTWARE\Microsoft\.NETFramework\v2.0.50727` and `v4.0.30319` (and Wow6432Node equivalents) \* `SchUseStrongCrypto` = `1` (REG\_DWORD) \* `SystemDefaultTlsVersions` = `1` (REG\_DWORD) \* \*\*Disable .NET strong-name bypass\*\*: \* `HKLM:\SOFTWARE\Microsoft\.NETFramework` (and Wow6432Node equivalent) \* `AllowStrongNameBypass` = `0` (REG\_DWORD) \* \*\*WinHTTP TLS 1.2 support\*\*: \* `HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\WinHttp` (and Wow6432Node equivalent) \* `DefaultSecureProtocols` = `2048` (REG\_DWORD, 0x00000800)

## Rationale Legacy versions of SSL (2.0 and 3.0) and TLS (1.0 and 1.1) are cryptographically weak and vulnerable to various attacks (such as BEAST, POODLE, and SWEET32) that can lead to credential exposure and session hijacking. Domain services, including LDAPS and WinRM, must enforce the usage of TLS 1.2 and TLS 1.3 (where supported) to prevent protocol downgrade attacks.

In addition to disabling insecure protocol versions, the TLS cipher suites and Elliptic Curves must be restricted to modern, collision-resistant options. CBC (Cipher Block Chaining) mode and RC4 ciphers are prone to padding oracle attacks. Restricting configurations to AES-GCM (Galois/Counter Mode) authenticated encryption suites combined with strong Elliptic Curve Diffie-Hellman (ECDHE) curves (such as Curve25519 and NIST P-384) guarantees confidentiality, integrity, and perfect forward secrecy (PFS).

By default, legacy .NET Framework applications (targeting .NET 3.5 or earlier) and WinHTTP-based APIs do not enforce TLS 1.2 or TLS 1.3, potentially defaulting to weak protocols like SSL 3.0 or TLS 1.0. Enabling `SchUseStrongCrypto` and `SystemDefaultTlsVersions` forces .NET to use the system default secure TLS versions. Restricting `DefaultSecureProtocols` forces WinHTTP clients to use TLS 1.2. Disabling the strong-name bypass (`AllowStrongNameBypass` = 0) prevents full-trust assemblies from skipping signature validation, protecting the .NET runtime from loading tampered assemblies.

## Legacy Impact & Compatibility \* \*\*Legacy Clients\*\*: Operating systems prior to Windows 7/Server 2008 R2 (without updates enabling TLS 1.2) or outdated third-party management agents, appliances, and legacy printers will fail to establish secure connections (LDAPS, HTTPS, RDP, WinRM) with hardened hosts. \* \*\*Domain Controller Reachability\*\*: Active Directory replication between DCs remains unaffected as it relies on RPC (Kerberos) rather than TLS. However, client-facing services such as secure LDAP (LDAPS) require TLS; clients must support TLS 1.2. \* \*\*Administrative Access\*\*: Admin connections (such as WinRM over HTTPS or RDP) will require TLS 1.2. Administrative machines (PAWs) must be configured to support the same cipher suites. \* \*\*.NET Framework Applications\*\*: Forcing strong cryptography may break communication with legacy servers or web services that do not support TLS 1.2. Applications targeting .NET 3.5 or older should be tested for compatibility.

## ## Implementation Steps

## ### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### 1. Enforce SSL Cipher Suite Order and ECC Curve Order via GPO 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Create or edit a GPO targeting all domain assets (e.g., `GPO_Hardening_TLS_Schannel`). 3. Navigate to: `Computer Configuration\Administrative Templates\Network\SSL Configuration Settings` 4. Double-click the **SSL Cipher Suite Order** policy: \* Set the policy to **Enabled**. \* In the **SSL Cipher Suites** text box, enter the hardened comma-separated cipher suite string:

`TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384,TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_RSA_WITH_AES_256_GCM`

5. Double-click the **ECC Curve Order** policy: \* Set the policy to **Enabled**. \* In the **ECC Curves** text box, enter the curves in order of preference: `curve25519,nistP384,nistP256` 6. Click **OK**.

##### 2. Deploy Schannel Protocol Registry settings via GPO Preferences Since Schannel protocol versions (disabling TLS 1.0/1.1 and enabling TLS 1.2/1.3) are not exposed via default ADMX templates, configure them using **Registry Preferences**: 1. Within the same GPO, navigate to: `Computer Configuration\Preferences\Windows Settings\Registry` 2. Right-click **Registry**, select **New** → **Registry Item**. 3. Create registry values under: `HKLM:\SYSTEM\CurrentControlSet\Control\SecurityProviders\SCHANNEL\Protocols` 4. Configure key pairs for each protocol (`SSL 2.0`, `SSL 3.0`, `TLS 1.0`, `TLS 1.1`, `TLS 1.2`, `TLS 1.3`) under both `Client` and `Server` subkeys: \* \*\*To Disable (SSL 2.0, SSL 3.0, TLS 1.0, TLS 1.1)\*\*: \* Value Name: `Enabled` | Type: `REG_DWORD` | Value Data: `0` \* Value Name: `DisabledByDefault` | Type: `REG_DWORD` | Value Data: `1` \* \*\*To Enable (TLS 1.2, TLS 1.3)\*\*: \* Value Name: `Enabled` | Type: `REG_DWORD` | Value Data: `1` \* Value Name: `DisabledByDefault` | Type: `REG_DWORD` | Value Data: `0`

*Note: Systems must be rebooted for Schannel protocol and cipher settings to take effect.*

##### 3. Deploy .NET and WinHTTP Registry Settings via GPO Preferences 1. Within the same GPO, navigate to: `Computer Configuration\Preferences\Windows Settings\Registry` 2. Configure the following registry entries (as Registry Items): \* \*\*.NET 4.0 (32-bit & 64-bit)\*\*: \* Key: `HKLM\SOFTWARE\Microsoft\.NETFramework\v4.0.30319` | Value: `SchUseStrongCrypto` = `1` (REG\_DWORD) \* Key: `HKLM\SOFTWARE\Microsoft\.NETFramework\v4.0.30319` | Value: `SystemDefaultTlsVersions` = `1` (REG\_DWORD) \* Key: `HKLM\SOFTWARE\Wow6432Node\Microsoft\.NETFramework\v4.0.30319` | Value: `SchUseStrongCrypto` = `1` (REG\_DWORD) \* Key: `HKLM\SOFTWARE\Wow6432Node\Microsoft\.NETFramework\v4.0.30319` | Value: `SystemDefaultTlsVersions` = `1` (REG\_DWORD) \* \*\*.NET

```

2.0 (32-bit & 64-bit)**: * Key: `HKLM\SOFTWARE\Microsoft\NETFramework\v2.0.50727` | Value: `SchUseStrongCrypto` = `1` (REG_DWORD) *
Key: `HKLM\SOFTWARE\Microsoft\NETFramework\v2.0.50727` | Value: `SystemDefaultTlsVersions` = `1` (REG_DWORD) * Key:
`HKLM\SOFTWARE\Wow6432Node\Microsoft\NETFramework\v2.0.50727` | Value: `SchUseStrongCrypto` = `1` (REG_DWORD) * Key:
`HKLM\SOFTWARE\Wow6432Node\Microsoft\NETFramework\v2.0.50727` | Value: `SystemDefaultTlsVersions` = `1` (REG_DWORD) * **.NET
Strong-Name Bypass**: * Key: `HKLM\SOFTWARE\Microsoft\NETFramework` | Value: `AllowStrongNameBypass` = `0` (REG_DWORD) * Key:
`HKLM\SOFTWARE\Wow6432Node\Microsoft\NETFramework` | Value: `AllowStrongNameBypass` = `0` (REG_DWORD) * **WinHTTP (32-bit &
64-bit)**: * Key: `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\WinHttp` | Value: `DefaultSecureProtocols` = `2048`
(REG_DWORD) * Key: `HKLM\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Internet Settings\WinHttp` | Value:
`DefaultSecureProtocols` = `2048` (REG_DWORD)

```

---

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally on testing hosts or non-GPO-managed systems.



#### Remediation Script: [Download Script: Set-TLSConfiguration.ps1](implementation\_scripts/Set-TLSConfiguration.ps1)

```

Set-TLSConfiguration.ps1
Description: Hardens TLS/Schannel protocols, prioritizes modern cipher suites, and orders ECC curves.

Write-Host "Configuring Schannel protocols, cipher suites, and ECC curves..." -ForegroundColor Cyan

Define Protocols to disable
$ProtocolsToDisable = @("SSL 2.0", "SSL 3.0", "TLS 1.0", "TLS 1.1")
Define Protocols to enable
$ProtocolsToEnable = @("TLS 1.2", "TLS 1.3")

1. Configure Protocols
$SchannelRoot = "HKLM:\SYSTEM\CurrentControlSet\Control\SecurityProviders\SCHANNEL\Protocols"

foreach ($Proto in $ProtocolsToDisable) {
 $Subkeys = @("Client", "Server")
 foreach ($Subkey in $Subkeys) {
 $Path = "$($SchannelRoot)\$($Proto)\$($Subkey)"
 if (-not (Test-Path $Path)) {
 New-Item -Path $Path -Force | Out-Null
 }
 Set-ItemProperty -Path $Path -Name "Enabled" -Value 0 -Type DWord -Force | Out-Null
 Set-ItemProperty -Path $Path -Name "DisabledByDefault" -Value 1 -Type DWord -Force | Out-Null
 }
}

foreach ($Proto in $ProtocolsToEnable) {
 $Subkeys = @("Client", "Server")
 foreach ($Subkey in $Subkeys) {
 $Path = "$($SchannelRoot)\$($Proto)\$($Subkey)"
 if (-not (Test-Path $Path)) {
 New-Item -Path $Path -Force | Out-Null
 }
 Set-ItemProperty -Path $Path -Name "Enabled" -Value 1 -Type DWord -Force | Out-Null
 Set-ItemProperty -Path $Path -Name "DisabledByDefault" -Value 0 -Type DWord -Force | Out-Null
 }
}

2. Configure SSL Cipher Suite Order and ECC Curve Order Policy
$SSLConfigPath = "HKLM:\SOFTWARE\Policies\Microsoft\Cryptography\Configuration\SSL\00010002"
if (-not (Test-Path $SSLConfigPath)) {
 New-Item -Path $SSLConfigPath -Force | Out-Null
}

Define cipher suites list
$CipherSuites = @(
 "TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384",
 "TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256",
 "TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384",
 "TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256",
 "TLS_DHE_RSA_WITH_AES_256_GCM_SHA384",
 "TLS_DHE_RSA_WITH_AES_128_GCM_SHA256"
)

Define ECC curves list
$EccCurves = @(
 "curve25519",
 "nistP384",
 "nistP256"
)

Apply policy properties
Set-ItemProperty -Path $SSLConfigPath -Name "Functions" -Value $CipherSuites -Type MultiString -Force | Out-Null
Set-ItemProperty -Path $SSLConfigPath -Name "EccCurves" -Value $EccCurves -Type MultiString -Force | Out-Null

3. Configure .NET strong cryptography, strong-name bypass, and WinHTTP TLS
$RegistryTargets = @(
 @{ Path = "HKLM:\SOFTWARE\Microsoft\.NETFramework\v2.0.50727"; Name = "SchUseStrongCrypto"; Value = 1; Type = "DWord" }
 @{ Path = "HKLM:\SOFTWARE\Microsoft\.NETFramework\v2.0.50727"; Name = "SystemDefaultTlsVersions"; Value = 1; Type = "DWord" }
 @{ Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\.NETFramework\v2.0.50727"; Name = "SchUseStrongCrypto"; Value = 1; Type = "DWord" }
 @{ Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\.NETFramework\v2.0.50727"; Name = "SystemDefaultTlsVersions"; Value = 1; Type = "DWord" }
 @{ Path = "HKLM:\SOFTWARE\Microsoft\.NETFramework\v4.0.30319"; Name = "SchUseStrongCrypto"; Value = 1; Type = "DWord" }
)

```

```

@{ Path = "HKLM:\SOFTWARE\Microsoft\.NETFramework\v4.0.30319"; Name = "SystemDefaultTlsVersions"; Value = 1; Type = "DWord" }
@{ Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\.NETFramework\v4.0.30319"; Name = "SchUseStrongCrypto"; Value = 1; Type = "DWord" }
@{ Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\.NETFramework\v4.0.30319"; Name = "SystemDefaultTlsVersions"; Value = 1; Type = "DWord" }

@{ Path = "HKLM:\SOFTWARE\Microsoft\.NETFramework"; Name = "AllowStrongNameBypass"; Value = 0; Type = "DWord" }
@{ Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\.NETFramework"; Name = "AllowStrongNameBypass"; Value = 0; Type = "DWord" }

@{ Path = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\WinHttp"; Name = "DefaultSecureProtocols"; Value = 2048; Type = "DWord" }
@{ Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Internet Settings\WinHttp"; Name = "DefaultSecureProtocols"; Value = 2048; Type = "DWord" }
}

foreach ($target in $RegistryTargets) {
 if (-not (Test-Path $target.Path)) {
 New-Item -Path $target.Path -Force | Out-Null
 }
 Set-ItemProperty -Path $target.Path -Name $target.Name -Value $target.Value -Type $target.Type -Force | Out-Null
}

Write-Host "Schannel configuration applied. A system reboot is required to apply changes." -ForegroundColor Green

```

#### Audit Script: [Download Script: Test-TLSConfiguration.ps1](audit\_scripts/Test-TLSConfiguration.ps1)

```

Test-TLSConfiguration.ps1
Description: Audits TLS/Schannel protocol configuration, cipher suites, and ECC curves.

Write-Host "Auditing TLS and Cryptographic configurations..." -ForegroundColor Cyan

$NonCompliantCount = 0
$SchannelRoot = "HKLM:\SYSTEM\CurrentControlSet\Control\SecurityProviders\SCHANNEL\Protocols"

1. Audit Protocols
$ProtocolsToDisable = @("SSL 2.0", "SSL 3.0", "TLS 1.0", "TLS 1.1")
foreach ($Proto in $ProtocolsToDisable) {
 $Subkeys = @("Client", "Server")
 foreach ($Subkey in $Subkeys) {
 $Path = "$($SchannelRoot)\($Proto)\($Subkey)"
 if (Test-Path $Path) {
 $Enabled = Get-ItemPropertyValue -Path $Path -Name "Enabled" -ErrorAction SilentlyContinue
 if ($Enabled -ne 0) {
 Write-Host " - Protocol $($Proto) ($($Subkey)) is not disabled (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
 } else {
 Write-Host " - Protocol $($Proto) ($($Subkey)) is disabled (Compliant)." -ForegroundColor Green
 }
 } else {
 Write-Host " - Protocol $($Proto) ($($Subkey)) registry key does not exist (Compliant)." -ForegroundColor Green
 }
 }
}

2. Audit SSL Policy and ECC Curves
$SSLConfigPath = "HKLM:\SOFTWARE\Policies\Microsoft\Cryptography\Configuration\SSL\00010002"
if (Test-Path $SSLConfigPath) {
 $Functions = Get-ItemPropertyValue -Path $SSLConfigPath -Name "Functions" -ErrorAction SilentlyContinue
 $EccCurves = Get-ItemPropertyValue -Path $SSLConfigPath -Name "EccCurves" -ErrorAction SilentlyContinue

 if ($null -eq $Functions -or $Functions.Length -eq 0) {
 Write-Host " - SSL Cipher Suite order policy is not configured (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
 } else {
 if ($Functions[0] -match "GCM") {
 Write-Host " - SSL Cipher Suite priority matches standards (Compliant)." -ForegroundColor Green
 } else {
 Write-Host " - SSL Cipher Suite priority does not favor secure GCM suites first (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
 }
 }

 if ($null -eq $EccCurves -or $EccCurves.Length -eq 0) {
 Write-Host " - ECC Curve priority policy is not configured (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
 } else {
 if ($EccCurves[0] -eq "curve25519" -or $EccCurves[0] -eq "nistP384") {
 Write-Host " - ECC Curve priority matches standards (Compliant)." -ForegroundColor Green
 } else {
 Write-Host " - ECC Curve priority does not favor curve25519/nistP384 (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
 }
 }
} else {
 Write-Host " - Custom SSL configuration policies are not configured (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
}

3. Audit .NET and WinHTTP registry configurations
$RegistryAudits = @(
 @{ Path = "HKLM:\SOFTWARE\Microsoft\.NETFramework\v2.0.50727"; Name = "SchUseStrongCrypto"; Value = 1 }
 @{ Path = "HKLM:\SOFTWARE\Microsoft\.NETFramework\v2.0.50727"; Name = "SystemDefaultTlsVersions"; Value = 1 }
 @{ Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\.NETFramework\v2.0.50727"; Name = "SchUseStrongCrypto"; Value = 1 }
 @{ Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\.NETFramework\v2.0.50727"; Name = "SystemDefaultTlsVersions"; Value = 1 }

 @{ Path = "HKLM:\SOFTWARE\Microsoft\.NETFramework\v4.0.30319"; Name = "SchUseStrongCrypto"; Value = 1 }
 @{ Path = "HKLM:\SOFTWARE\Microsoft\.NETFramework\v4.0.30319"; Name = "SystemDefaultTlsVersions"; Value = 1 }
 @{ Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\.NETFramework\v4.0.30319"; Name = "SchUseStrongCrypto"; Value = 1 }
 @{ Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\.NETFramework\v4.0.30319"; Name = "SystemDefaultTlsVersions"; Value = 1 }
)

```

```

@{ Path = "HKLM:\SOFTWARE\Microsoft\.NETFramework"; Name = "AllowStrongNameBypass"; Value = 0 }
@{ Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\.NETFramework"; Name = "AllowStrongNameBypass"; Value = 0 }

@{ Path = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\WinHttp"; Name = "DefaultSecureProtocols"; Value =
2048 }
@{ Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Internet Settings\WinHttp"; Name = "DefaultSecureProtocols"; Value = 2048 }
)

foreach ($target in $RegistryAudits) {
 if (Test-Path $target.Path) {
 $val = Get-ItemPropertyValue -Path $target.Path -Name $target.Name -ErrorAction SilentlyContinue
 if ($val -eq $target.Value) {
 Write-Host " - Registry Setting: $($target.Path)\$($target.Name) is Compliant." -ForegroundColor Green
 } else {
 Write-Host " - Registry Setting: $($target.Path)\$($target.Name) is Non-Compliant (Actual: '$val', Expected: '$($target.Value)')." -ForegroundColor Red
 $NonCompliantCount++
 }
 } else {
 Write-Host " - Registry Key: $($target.Path) does not exist (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
 }
}

if ($NonCompliantCount -eq 0) {
 Write-Host "TLS and Cryptographic configuration: Compliant." -ForegroundColor Green
} else {
 Write-Host "TLS and Cryptographic configuration: Non-Compliant ($($NonCompliantCount) issue(s) detected)." -ForegroundColor Red
}

```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*[: Recommendation R18 \(Schannel configuration\)](#) \* \*\*ANSSI General Security Rules (RGS)[: Annex B1 \(Cryptographic mechanisms\)](#) \* \*\*CIS Windows Server 2016 Benchmark[: Section 18.10 \(SSL Configuration Settings\)](#) \* \*\*Microsoft Security Guidance[: Transport Layer Security \(TLS\) best practices](#)

**# [REQ-NET-007] Enforce SMBv3 Security and Digitally Sign/Encrypt Communications**

**## Target Scope** \* **Applicable Systems**: Domain Controllers, Member Servers, PAWs, Tier 2 Client Workstations. \* **Operating Systems**: Windows Server 2016 (and above), Windows 10 (and above).

---

**## Implementation Details** \* **Priority**: High \* **GPO Path / Registry Location**: \* **Signing Policies**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` \* Microsoft network client: Digitally sign communications (always) \* Microsoft network client: Digitally sign communications (if server agrees) \* Microsoft network server: Digitally sign communications (always) \* Microsoft network server: Digitally sign communications (if client agrees) \* **Dialect Policies**: `Computer Configuration\Administrative Templates\Network\Lanman Server` and `Lanman Workstation` → **Mandate the minimum version of SMB** \* **Registry Locations**: \* `HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters` \* `HKLM:\SYSTEM\CurrentControlSet\Services\LanmanWorkstation\Parameters`

---

**## Rationale** Server Message Block (SMB) version 1.0 (SMBv1) is obsolete, highly insecure, and vulnerable to critical exploits (such as MS17-010 / EternalBlue, which enabled the global spread of WannaCry and NotPetya). SMBv2, while newer, lacks modern cryptographic protection and is prone to Man-in-the-Middle (MitM) interception and NTLM relaying.

Enforcing SMBv3 (minimum version 3.0.0 or 3.1.1) provides significant security advantages:

1. **AES-GCM Encryption**: Protects data in transit from passive sniffing and tampering. Enforcing encryption is critical on Domain Controllers (specifically for Sysvol and Netlogon shares) and servers hosting sensitive business files.
  2. **Pre-Authentication Integrity**: Prevents tampering with SMB negotiation packets (mitigating downgrade attacks).
  3. **SMB Signing**: Adds a cryptographic signature to all packets. Mandating SMB signing (specifically **Digitally sign communications (always)**) protects against SMB Relay attacks, where an attacker intercepts a NTLM authentication hash on the local network and replays it to a target server to gain unauthorized access.
- 

**## Legacy Impact & Compatibility** \* **Legacy Systems**: Any client or server operating system that does not support SMB 3.x (such as Windows Server 2008, Windows Vista, or old versions of Linux/Samba) will fail to establish connections if SMBv3 is mandated or if SMB encryption is required. \* **Multifunction Printers and NAS**: Older network scanners, printers, and NAS storage appliances that only support SMBv1 or SMBv2 will no longer be able to write files to network shares. These devices must be updated or isolated to a dedicated segment with specialized access rules. \* **SYSVOL Replication**: For Domain Controllers, ensuring DFS Replication is migrated to DFSR (which supports SMBv3) is a prerequisite.

---

**## Implementation Steps****### Option A: Group Policy Object (GPO) Configuration (Preferred)**

**#### 1. Configure SMB Signing Policies** 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Create or edit a GPO targeting all domain assets (e.g., `GPO\_Hardening\_SMB\_Security`). 3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` 4. Configure the following four policies: \* **Microsoft network client: Digitally sign communications (always)**: `Enabled` \* **Microsoft network client: Digitally sign communications (if server agrees)**: `Enabled` \* **Microsoft network server: Digitally sign communications (always)**: `Enabled` \* **Microsoft network server: Digitally sign communications (if client agrees)**: `Enabled`

**#### 2. Mandate Minimum SMB Dialects** On systems that support ADMX templates for SMB dialects (Windows 11 / Server 2022+): 1. Navigate to: `Computer Configuration\Administrative Templates\Network\Lanman Server` 2. Double-click **Mandate the minimum version of SMB**: \* Set the policy to **Enabled**. \* Set the minimum version to `SMB 3.0.0` (or `SMB 3.1.1` to require the latest dialect). 3. Navigate to: `Computer Configuration\Administrative Templates\Network\Lanman Workstation` 4. Double-click **Mandate the minimum version of SMB**: \* Set the policy to **Enabled**. \* Set the minimum version to `SMB 3.0.0` (or `SMB 3.1.1`).

**#### 3. Disable SMBv1 Driver via GPO Preferences** To ensure the SMBv1 driver is disabled on older machines, deploy a registry change: 1. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry` 2. Right-click **Registry**, select **New** → **Registry Item**: \* **Action**: `Update` \* **Hive**: `HKEY\_LOCAL\_MACHINE` \* **Key Path**: `SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters` \* **Value name**: `SMB1` \* **Value type**: `REG\_DWORD` \* **Value data**: `0`

---

**### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)**

Run the following scripts locally to enforce SMBv3 standards.

#### Remediation Script: [Download Script: Set-SMBSecurity.ps1](implementation\_scripts/Set-SMBSecurity.ps1)

```
Set-SMBSecurity.ps1
Description: Disables SMBv1, mandates signing, sets SMBv3 as minimum dialect, and enforces encryption.

Write-Host "Enforcing SMBv3 security settings..." -ForegroundColor Cyan

1. Disable SMBv1 Protocol globally (Server side)
Set-SmbServerConfiguration -EnableSMB1Protocol $false -Confirm:$false | Out-Null
Write-Host "SMBv1 server protocol disabled." -ForegroundColor Green

2. Disable SMBv1 Driver (Windows Optional Feature)
$SMB1Feature = Get-WindowsOptionalFeature -Online -FeatureName "SMB1Protocol" -ErrorAction SilentlyContinue
if ($null -ne $SMB1Feature -and $SMB1Feature.State -eq "Enabled") {
 Disable-WindowsOptionalFeature -Online -FeatureName "SMB1Protocol" -NoRestart -WarningAction SilentlyContinue | Out-Null
 Write-Host "SMB1 optional feature disabled." -ForegroundColor Green
}

3. Configure SMB Signing & Encryption on Server
Set-SmbServerConfiguration -RequireSecuritySignature $true -EncryptData $true -Confirm:$false | Out-Null
Write-Host "SMB Server signing and encryption mandated." -ForegroundColor Green

4. Configure SMB Signing & Encryption on Client
Set-SmbClientConfiguration -RequireSecuritySignature $true -Confirm:$false | Out-Null
Write-Host "SMB Client signing mandated." -ForegroundColor Green

5. Enforce Minimum Dialects in Registry (Server and Client)
$ServerParamsPath = "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters"
$ClientParamsPath = "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanWorkstation\Parameters"

Write minimum dialect version 0x00000300 (SMB 3.0.0)
if (-not (Test-Path $ServerParamsPath)) {
 New-Item -Path $ServerParamsPath -Force | Out-Null
}
Set-ItemProperty -Path $ServerParamsPath -Name "MinSMB2Dialect" -Value 0x00000300 -Type DWord -Force | Out-Null

if (-not (Test-Path $ClientParamsPath)) {
 New-Item -Path $ClientParamsPath -Force | Out-Null
}
Set-ItemProperty -Path $ClientParamsPath -Name "MinSMB2Dialect" -Value 0x00000300 -Type DWord -Force | Out-Null

Disable legacy fallback protocols (e.g. NetBIOS over TCP/IP) if possible, but keep focus on SMBv3
Write-Host "SMBv3 minimum dialect rules configured." -ForegroundColor Green
```

#### Audit Script: [Download Script: Test-SMBSecurity.ps1](audit\_scripts/Test-SMBSecurity.ps1)



```
Test-SMBSecurity.ps1
Description: Audits local SMB configuration for signing, encryption, and dialects.

Write-Host "Auditing SMB security configuration..." -ForegroundColor Cyan

$NonCompliantCount = 0

Retrieve configurations
$ServerConfig = Get-SmbServerConfiguration
$ClientConfig = Get-SmbClientConfiguration

1. Audit SMBv1 Server status
if ($ServerConfig.EnableSMB1Protocol -eq $true) {
 Write-Host " - SMBv1 Server protocol is enabled (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
} else {
 Write-Host " - SMBv1 Server protocol is disabled (Compliant)." -ForegroundColor Green
}

2. Audit Signing Requirements
if ($ServerConfig.RequireSecuritySignature -ne $true) {
 Write-Host " - SMB Server signing is not required (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
} else {
 Write-Host " - SMB Server signing is mandated (Compliant)." -ForegroundColor Green
}

if ($ClientConfig.RequireSecuritySignature -ne $true) {
 Write-Host " - SMB Client signing is not required (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
} else {
 Write-Host " - SMB Client signing is mandated (Compliant)." -ForegroundColor Green
}

3. Audit Encryption Requirements
if ($ServerConfig.EncryptData -ne $true) {
 Write-Host " - SMB Server global data encryption is not enforced (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
} else {
 Write-Host " - SMB Server global data encryption is enforced (Compliant)." -ForegroundColor Green
}

4. Audit Registry Dialects
$ServerParamsPath = "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters"
$ClientParamsPath = "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanWorkstation\Parameters"

if (Test-Path $ServerParamsPath) {
 $ServerMinDialect = Get-ItemPropertyValue -Path $ServerParamsPath -Name "MinSMB2Dialect" -ErrorAction SilentlyContinue
 if ($null -eq $ServerMinDialect -or $ServerMinDialect -lt 0x00000300) {
 Write-Host " - Server minimum dialect is less than SMB 3.0 or not set (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
 } else {
 Write-Host " - Server minimum dialect is set to SMB 3.0+ (Compliant)." -ForegroundColor Green
 }
} else {
 Write-Host " - LanmanServer registry path is missing (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
}

if (Test-Path $ClientParamsPath) {
 $ClientMinDialect = Get-ItemPropertyValue -Path $ClientParamsPath -Name "MinSMB2Dialect" -ErrorAction SilentlyContinue
 if ($null -eq $ClientMinDialect -or $ClientMinDialect -lt 0x00000300) {
 Write-Host " - Client minimum dialect is less than SMB 3.0 or not set (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
 } else {
 Write-Host " - Client minimum dialect is set to SMB 3.0+ (Compliant)." -ForegroundColor Green
 }
} else {
 Write-Host " - LanmanWorkstation registry path is missing (Non-Compliant)." -ForegroundColor Red
 $NonCompliantCount++
}

if ($NonCompliantCount -eq 0) {
 Write-Host "SMB Security configuration: Compliant." -ForegroundColor Green
}
```

```
} else {
 Write-Host "SMB Security configuration: Non-Compliant ($($NonCompliantCount) issue(s) detected)." -ForegroundColor Red
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R21 (Disabling SMBv1), Recommendation R22 (SMB signing and encryption) \* \*\*CIS Windows Server 2016 Benchmark\*\*: Section 2.3.10.1 (Microsoft network client: Digitally sign communications (always)) and Section 2.3.10.2 (Microsoft network server: Digitally sign communications (always)) \* \*\*Microsoft Security Guidance\*\*: SMB security enhancements and dialect management

## # [REQ-NET-008] Configure Firewall Logging and Operational Settings

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers, Tier 2 Client Workstations. \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10 (and above) Enterprise/Professional.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Windows Defender Firewall with Advanced Security`

## Rationale Windows Defender Firewall with Advanced Security (WFAS) serves as the host-level stateful firewall protecting Active Directory resources from unauthorized network access. However, without correct logging and behavioral configuration, the firewall does not provide adequate defensive or diagnostic value:

1. **Visibility Gaps:** By default, Windows Defender Firewall does not log dropped packets. If logging is disabled, security administrators cannot detect failed connection attempts, reconnaissance scans (port scanning), or unauthorized network communications.
2. **Log Rotational Coverage:** The default firewall log size limit of 4096 KB (4 MB) is insufficient for enterprise environments. High volumes of traffic or network scanning will cause the log to roll over rapidly, destroying valuable historical entries needed for security audits and incident investigation.
3. **Deterministic Administrative Control:** Allowing local administrators to create local rules or local connection security rules (IPsec) on critical Tier 0 systems, such as Domain Controllers, risks bypassing centrally defined domain firewall GPOs. Restricting rule merging ensures a uniform security baseline.
4. **Behavioral Notification Control:** Disabling interactive firewall notifications prevents desktop alerts from prompting administrative users, reducing operational noise and social engineering opportunities.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Enabling logging of blocked packets carries negligible CPU and disk I/O overhead on modern storage hardware. Successful connections must not be logged in production, as doing so can trigger disk performance degradation due to logging high-volume network flows. \* \*\*Local Rule Merging on Domain Controllers\*\*: Setting "Apply local firewall rules" to "No" on Domain Controllers will cause the system to ignore any firewall rules created locally (e.g., via the local netsh, PowerShell, or third-party installers). All required inbound ports for application operations must be centrally managed and distributed via Group Policy. \* \*\*Administrative Subnet Access\*\*: If the default inbound action is set to block, all legitimate management traffic (such as RDP, WinRM, or SNMP) must have explicit inbound allow rules defined prior to deployment to prevent lockouts.

## ## Implementation Steps

### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a management host.
2. Create a new GPO or edit an existing one (e.g., `GPO_Hardening_Firewall_Baseline` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security`
4. Right-click **Windows Defender Firewall with Advanced Security** and select **Properties**.
5. Under the **Domain Profile** tab, configure the following settings:
  - **State:** `On (recommended)`
  - **Inbound connections:** `Block (default)`
  - **Outbound connections:** `Allow (default)`
  - Under **Settings**, click **Customize...**:
    - **Display a notification:** `No`
    - **Allow unicast response:** `Yes`
    - **Apply local firewall rules:** `No` (Set to `No` for Domain Controllers GPOs; set to `Yes` or `No` for Member Servers/Workstations depending on operational requirements)
    - **Apply local connection security rules:** `No` (Set to `No` for Domain Controllers GPOs; set to `Yes` or `No` for Member Servers/Workstations depending on operational requirements)
    - Click **OK**.
  - Under **Logging**, click **Customize...**:
    - **Name:** `%SystemRoot%\System32\LogFiles\Firewall\pfirewall.log`
    - **Size limit (KB):** `32768`
    - **Log dropped packets:** `Yes`
    - **Log successful connections:** `No`
    - Click **OK**.
6. Repeat the exact configuration steps (Step 5) for the **Private Profile** and **Public Profile** tabs.
7. Click **Apply** and then click **OK**.
8. Link the GPO to the appropriate Organizational Unit (OU) containing the target assets.

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally (for testing or standalone systems) or if the control is not manageable via standard GPO GUI interfaces.

[Download Script: Set-FirewallLoggingAndSettings.ps1](#)

```
Set-FirewallLoggingAndSettings.ps1
Description: Configures Windows Defender Firewall settings, log size, and log permissions for all profiles.

Write-Host "Applying Windows Defender Firewall hardening settings..." -ForegroundColor Cyan

1. Detect if the local system is a Domain Controller (ProductType = 2 is Domain Controller)
$IsDomainController = $false
$OSInfo = Get-CimInstance -ClassName Win32_OperatingSystem -ErrorAction SilentlyContinue
if ($null -ne $OSInfo) {
 if ($OSInfo.ProductType -eq 2) {
 $IsDomainController = $true
 }
}

2. Configure Domain, Private, and Public profiles
$Profiles = @("Domain", "Private", "Public")

foreach ($FwProfile in $Profiles) {
 Write-Host "Configuring Profile: $($FwProfile)..." -ForegroundColor Cyan

 # Enable firewall and set default inbound/outbound behavior and notifications
 Set-NetFirewallProfile -Profile $FwProfile `
 -Enabled True `
 -DefaultInboundAction Block `
 -DefaultOutboundAction Allow `
 -NotifyOnListen False `
 -AllowUnicastResponseToMulticast True | Out-Null

 # Configure logging: log dropped packets, disable successful logs, set size to 32MB (32768 KB)
 Set-NetFirewallProfile -Profile $FwProfile `
 -LogBlocked True `
 -LogAllowed False `
 -LogMaxSizeKilobytes 32768 `
 -LogFileName "%SystemRoot%\System32\LogFiles\Firewall\pfirewall.log" | Out-Null

 # Disable local rule merging only on Domain Controllers to prevent bypasses
 if ($IsDomainController) {
 Set-NetFirewallProfile -Profile $FwProfile `
 -AllowLocalFirewallRules False `
 -AllowLocalIPsecRules False | Out-Null
 Write-Host "Disabled local rule merging on DC for profile: $($FwProfile)" -ForegroundColor Green
 } else {
 Set-NetFirewallProfile -Profile $FwProfile `
 -AllowLocalFirewallRules True `
 -AllowLocalIPsecRules True | Out-Null
 Write-Host "Configured profile: $($FwProfile) with local rule merging allowed" -ForegroundColor Green
 }
}

Write-Host "Firewall logging and operational settings configuration completed successfully." -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-FirewallLoggingAndSettingsStatus.ps1](#)

```
Get-FirewallLoggingAndSettingsStatus.ps1
Get-NetFirewallProfile | Select-Object Name, Enabled, DefaultInboundAction, DefaultOutboundAction, NotifyOnListen, LogBlocked, LogAllowed, LogMaxSizeKilobytes, LogFileName, AllowLocalFirewallRules, AllowLocalIPsecRules
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R7 (Filtering and IPsec on Domain Controllers), Recommendation R8 (Administration network subnets / filtering rules) \* \*\*CIS Windows Server 2016 Benchmark\*\*: Section 19.1 (Domain

Profile), Section 19.2 (Private Profile), Section 19.3 (Public Profile) \* \*\*Microsoft Security Baseline Focus\*\*: Windows Defender Firewall with Advanced Security settings

## # [REQ-NET-009] Configure Hardened UNC Paths and LDAP Client Signing

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers, Tier 2 Clients. \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Hardened UNC Paths (GPO)\*\*: `Computer Configuration\Policies\Administrative Templates\Network\Network Provider\Hardened UNC Paths` → Enabled \* Path: `\\\*\NETLOGON` | Value: `RequireIntegrity=1,RequireMutualAuthentication=1` \* Path: `\\\*\SYSVOL` | Value: `RequireIntegrity=1,RequireMutualAuthentication=1` \* \*\*Insecure Guest Logons (GPO)\*\*: `Computer Configuration\Policies\Administrative Templates\Network\Lanman Workstation\Enable insecure guest logons` → Disabled \* \*\*LDAP Client Signing (GPO)\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options\Network security: LDAP client signing requirements` → Require signing \* \*\*Registry Location (UNC Paths)\*\*: `HKLM\SOFTWARE\Policies\Microsoft\Windows\NetworkProvider\HardenedPaths` \* `\\\*\NETLOGON` = `RequireIntegrity=1,RequireMutualAuthentication=1` (REG\_SZ) \* `\\\*\SYSVOL` = `RequireIntegrity=1,RequireMutualAuthentication=1` (REG\_SZ) \* \*\*Registry Location (Guest Logons)\*\*: `HKLM\SOFTWARE\Policies\Microsoft\Windows\LanmanWorkstation` → `AllowInsecureGuestAuth` = `0` (REG\_DWORD) \* \*\*Registry Location (LDAP Client)\*\*: `HKLM\System\CurrentControlSet\Services\LDAP` → `Idapclientintegrity` = `2` (REG\_DWORD)

---

## Rationale Active Directory clients and servers routinely query Domain Controllers to retrieve Group Policy Objects (GPOs), startup/shutdown scripts, and user logon scripts from the `SYSVOL` and `NETLOGON` shares. By default, these connections are made over standard UNC paths and do not strictly enforce integrity validation (SMB signing) or mutual authentication.

Enforcing these channel-level controls mitigates the following threat vectors:

1. **GPO Spoofing and Execution Tampering:** An attacker positioned on the local network using man-in-the-middle (MitM) techniques (such as ARP spoofing or DNS poisoning) can intercept GPO retrieval traffic. Without Hardened UNC Paths, the attacker can spoof the Domain Controller and inject a malicious GPO or a modified script, which then executes with local SYSTEM privileges on the client host. Enforcing integrity and mutual authentication on `SYSVOL` and `NETLOGON` blocks this spoofing vector.
2. **Insecure Guest Logons:** Disabling insecure guest logons stops the workstation or server from automatically authenticating to untrusted remote SMB shares using guest credentials. This prevents attackers from setting up rogue SMB servers that trick hosts into leaking NetNTLM credentials or executing untrusted files.
3. **LDAP Session Hijacking:** Forcing outgoing LDAP client connections to negotiate signing protects directory queries made by Member Servers or Domain Controllers (acting as clients) from interception, packet tampering, or sniffing.

---

## Legacy Impact & Compatibility \* \*\*Legacy Client Access\*\*: Non-domain-joined systems or legacy operating systems (prior to Windows Vista/Server 2008) that cannot perform Kerberos mutual authentication will fail to access the `SYSVOL` or `NETLOGON` shares on Domain Controllers. \* \*\*Third-party SMB Implementation Compatibility\*\*: Virtual machines or network storage devices that access SYSVOL for GPO processing must support SMB v2/v3 with signing. \* \*\*LDAP Client Queries\*\*: Legacy third-party applications running on member servers that query AD directories over unencrypted LDAP (TCP port 389) without supporting signing negotiations will fail. These applications must be configured to utilize LDAPS (TCP port 636) or enable signing negotiation support.

---

## ## Implementation Steps

## ### Option A: Group Policy Object (GPO) Configuration (Preferred)

#### 1. Configure Hardened UNC Paths 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Create or edit a GPO linked to all target computers (e.g., `GPO\_Computer\_Hardening\_Baseline`). 3. Navigate to: `Computer Configuration\Policies\Administrative Templates\Network\Network Provider` 4. Double-click **Hardened UNC Paths**. 5. Set it to **Enabled**. 6. Click **Show...** in the options panel, and add the following entries: \* \*\*Value name\*\*: `\\\*\NETLOGON` | \*\*Value\*\*: `RequireIntegrity=1,RequireMutualAuthentication=1` \* \*\*Value name\*\*: `\\\*\SYSVOL` | \*\*Value\*\*: `RequireIntegrity=1,RequireMutualAuthentication=1` 7. Click **OK**.

#### 2. Disable Insecure Guest Logons 1. Navigate to: `Computer Configuration\Policies\Administrative Templates\Network\Lanman Workstation` 2. Double-click **Enable insecure guest logons**. 3. Set it to **Disabled** and click **OK**.

#### 3. Enforce LDAP Client Signing 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` 2. Double-click **Network security: LDAP client signing requirements**. 3. Set it to **Require signing** and click **OK**.

---

## ### Option B: PowerShell &amp; Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to apply the hardened network provider, Lanman Workstation, and LDAP client configurations.

[Download Script: Set-HardenedUNCAndClientSigning.ps1](#)

```
Set-HardenedUNCAndClientSigning.ps1
Description: Configures Hardened UNC Paths for SYSVOL/NETLOGON, disables insecure guest logons, and enforces LDAP client signing.

Write-Host "Applying network provider and client channel hardening..." -ForegroundColor Cyan

1. Hardened UNC Paths configuration
$UNCPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\NetworkProvider\HardenedPaths"
if (-not (Test-Path $UNCPath)) {
 New-Item -Path $UNCPath -Force | Out-Null
}
Set-ItemProperty -Path $UNCPath -Name "*\NETLOGON" -Value "RequireIntegrity=1,RequireMutualAuthentication=1" -Type String -ErrorAction Stop
Set-ItemProperty -Path $UNCPath -Name "*\SYSVOL" -Value "RequireIntegrity=1,RequireMutualAuthentication=1" -Type String -ErrorAction Stop
Write-Host "[+] Hardened UNC Paths for NETLOGON and SYSVOL configured." -ForegroundColor Green

2. Disable Insecure Guest Logons
$LanmanPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\LanmanWorkstation"
if (-not (Test-Path $LanmanPath)) {
 New-Item -Path $LanmanPath -Force | Out-Null
}
Set-ItemProperty -Path $LanmanPath -Name "AllowInsecureGuestAuth" -Value 0 -Type DWord -ErrorAction Stop
Write-Host "[+] Insecure guest logons disabled." -ForegroundColor Green

3. Enforce LDAP Client Signing Requirements
$LdapPath = "HKLM:\System\CurrentControlSet\Services\LDAP"
if (-not (Test-Path $LdapPath)) {
 New-Item -Path $LdapPath -Force | Out-Null
}
Set-ItemProperty -Path $LdapPath -Name "ldapclientintegrity" -Value 2 -Type DWord -ErrorAction Stop
Write-Host "[+] LDAP Client signing requirement set to Require signing." -ForegroundColor Green
```

To audit the network provider, Lanman workstation, and LDAP client signing configurations: [Download Script: Get-HardenedUNCAndClientSigningStatus.ps1](#)

```
Get-HardenedUNCAndClientSigningStatus.ps1
Description: Audits the registry configuration of Hardened UNC Paths, Lanman guest authentication, and LDAP Client signing.

Write-Host "--- Auditing Hardened UNC Paths and Client Signing status ---" -ForegroundColor Cyan

1. Audit UNC Paths
$UNCPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\NetworkProvider\HardenedPaths"
if (Test-Path $UNCPath) {
 $NetLogonVal = Get-ItemProperty -Path $UNCPath -Name "*\NETLOGON" -ErrorAction SilentlyContinue
 $SysvolVal = Get-ItemProperty -Path $UNCPath -Name "*\SYSVOL" -ErrorAction SilentlyContinue

 $NetColor = if ($NetLogonVal -and $NetLogonVal.'*\NETLOGON' -eq "RequireIntegrity=1,RequireMutualAuthentication=1") { "Green"
} else { "Red" }
 $SysColor = if ($SysvolVal -and $SysvolVal.'*\SYSVOL' -eq "RequireIntegrity=1,RequireMutualAuthentication=1") { "Green" } else
{ "Red" }

 Write-Host " - Hardened UNC NETLOGON: $($NetLogonVal.'*\NETLOGON') (Expected: RequireIntegrity=1,RequireMutualAuthenticatio
n=1)" -ForegroundColor $NetColor
 Write-Host " - Hardened UNC SYSVOL: $($SysvolVal.'*\SYSVOL') (Expected: RequireIntegrity=1,RequireMutualAuthentication=
1)" -ForegroundColor $SysColor
} else {
 Write-Host " - Hardened UNC registry path: NOT FOUND" -ForegroundColor Red
}

2. Audit Guest Logons
$LanmanPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\LanmanWorkstation"
$GuestVal = Get-ItemProperty -Path $LanmanPath -Name "AllowInsecureGuestAuth" -ErrorAction SilentlyContinue
$GuestSetting = if ($GuestVal) { $GuestVal.AllowInsecureGuestAuth } else { 1 }
$GuestColor = if ($GuestSetting -eq 0) { "Green" } else { "Red" }
Write-Host " - Allow Insecure Guest Logons: $GuestSetting (Expected: 0)" -ForegroundColor $GuestColor

3. Audit LDAP Client Signing
$LdapPath = "HKLM:\System\CurrentControlSet\Services\LDAP"
$LdapVal = Get-ItemProperty -Path $LdapPath -Name "ldapclientintegrity" -ErrorAction SilentlyContinue
$LdapSetting = if ($LdapVal) { $LdapVal.ldapclientintegrity } else { 0 }
$LdapColor = if ($LdapSetting -eq 2) { "Green" } else { "Red" }
Write-Host " - LDAP Client Integrity (Signing): $LdapSetting (Expected: 2 - Require)" -ForegroundColor $LdapColor
```

---

**## Sources & Compliance References \* \*\*ANSI AD Hardening Guide\*\*:** Recommendation R19 (Configuration des Hardened UNC Paths pour SYSVOL et NETLOGON) \* \*\*CIS Benchmark\*\*:

CIS Microsoft Windows Server Benchmark - Section 18.2.1 (Lanman Workstation), Section 18.1.1 (Network Provider) \* \*\*Microsoft Security Baseline Focus\*\*:

Windows Server and Client Group Policy Security Baselines



# [REQ-NET-010] Harden WinRM Service and Restrict Remote RPC Clients

## Target Scope \* \*\*Applicable Systems\*\* : Domain Controllers, Member Servers, Tier 2 Clients. \* \*\*Operating Systems\*\* : Windows Server 2016 (and above), Windows 10/11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\* : High \* \*\*GPO Path / Registry Location\*\* : \* \*\*WinRM Client GPO\*\* : `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Remote Management (WinRM)\WinRM Client` \* `Allow Basic authentication` → Disabled \* `Allow unencrypted traffic` → Disabled \* `Disallow Digest authentication` → Enabled \* \*\*WinRM Service GPO\*\* : `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Remote Management (WinRM)\WinRM Service` \* `Allow Basic authentication` → Disabled \* `Allow unencrypted traffic` → Disabled \* `Disallow WinRM from storing RunAs credentials` → Enabled \* \*\*Windows Remote Shell GPO\*\* : `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Remote Shell` \* `Allow Remote Shell Access` → Disabled \* \*\*RPC Client Restraints GPO\*\* : `Computer Configuration\Policies\Administrative Templates\System\Remote Procedure Call\Restrict Unauthenticated RPC clients` → Enabled (Set option to **Authenticated**) \* \*\*Registry Location (WinRM Client)\*\* : `HKLM\SOFTWARE\Policies\Microsoft\Windows\WinRM\Client` \* `AllowBasic` = `0` (REG\_DWORD) \* `AllowUnencryptedTraffic` = `0` (REG\_DWORD) \* `AllowDigest` = `0` (REG\_DWORD) \* \*\*Registry Location (WinRM Service)\*\* : `HKLM\SOFTWARE\Policies\Microsoft\Windows\WinRM\Service` \* `AllowBasic` = `0` (REG\_DWORD) \* `AllowUnencryptedTraffic` = `0` (REG\_DWORD) \* `DisableRunAs` = `1` (REG\_DWORD) \* \*\*Registry Location (Windows Remote Shell)\*\* : `HKLM\SOFTWARE\Policies\Microsoft\Windows\WinRM\Service\WinRS` \* `AllowRemoteShellAccess` = `0` (REG\_DWORD) \* \*\*Registry Location (RPC Clients)\*\* : `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Rpc` → `RestrictRemoteClients` = `1` (REG\_DWORD)

---

## Rationale Windows Remote Management (WinRM) and Remote Procedure Call (RPC) are standard management interfaces in Windows environments. However, by default, these interfaces allow backward-compatible configurations that pose significant security risks. Hardening these service channels blocks the following exploit vectors:

1. **Plaintext Credential Harvesting**: Allowing Basic authentication on WinRM clients and services allows transmission of administrative passwords in plaintext (or easily decodable formats) if secure channels are not established. Disabling Basic and Digest authentication forces the use of Kerberos or certificate-based authentication.
2. **Replay and Eavesdropping**: WinRM allows unencrypted traffic by default, which exposes administrative payloads and remote command execution streams to sniffing and hijacking. Forcing encryption protects the confidentiality and integrity of remote management sessions.
3. **RunAs Credential Exposure**: If WinRM is allowed to cache or store RunAs credentials for remote task execution, those credentials reside in the host's memory, where an administrative attacker can harvest them using memory extraction tools.
4. **Anonymous RPC Enumeration**: Restricting unauthenticated RPC clients prevents anonymous attackers from performing remote enumeration of active services, RPC interfaces, and registry endpoints, limiting remote reconnaissance capabilities.

---

## Legacy Impact & Compatibility \* \*\*Administrative Tooling\*\* : WinRM scripts, monitoring agents, or cross-domain management frameworks that rely on local accounts and Basic authentication to establish connections will fail. All administrative connections must be migrated to Kerberos or HTTPS with client certificate authentication. \* \*\*RPC Queries\*\* : Third-party scanners or legacy network devices that query RPC interfaces without authenticating will be blocked from accessing RPC data on hardened hosts.

---

## ## Implementation Steps

### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### 1. Configure WinRM Client Settings 1. Open the **Group Policy Management Console** ('gpmc.msc'). 2. Create or edit a GPO targeting all computers (e.g., 'GPO\_Computer\_Hardening\_Baseline'). 3. Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Remote Management (WinRM)\WinRM Client` 4. Configure the settings: \* \*\*Policy\*\* : `Allow Basic authentication` → **Disabled** \* \*\*Policy\*\* : `Allow unencrypted traffic` → **Disabled** \* \*\*Policy\*\* : `Disallow Digest authentication` → **Enabled**

##### 2. Configure WinRM Service Settings 1. Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Remote Management (WinRM)\WinRM Service` 2. Configure the settings: \* \*\*Policy\*\* : `Allow Basic authentication` → **Disabled** \* \*\*Policy\*\* : `Allow unencrypted traffic` → **Disabled** \* \*\*Policy\*\* : `Disallow WinRM from storing RunAs credentials` → **Enabled**

##### 3. Configure Windows Remote Shell Settings 1. Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Remote Shell` 2. Configure the settings: \* \*\*Policy\*\* : `Allow Remote Shell Access` → **Disabled**

##### 4. Configure RPC Client Restrictions 1. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Remote Procedure Call` 2. Double-click **Restrict Unauthenticated RPC clients**. 3. Set it to **Enabled**. 4. In the options dropdown, select **Authenticated** (corresponds to registry value '1'). 5. Click **OK**.

---

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to enforce WinRM client/service and RPC restrictions in the registry.

[Download Script: Set-WinRMAndRpcHardening.ps1](#)

```
Set-WinRMAndRpcHardening.ps1
Description: Hardens WinRM client/service parameters and restricts remote RPC clients.

Write-Host "Applying WinRM and RPC channel hardening settings..." -ForegroundColor Cyan

1. WinRM Client Hardening
$ClientPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WinRM\Client"
if (-not (Test-Path $ClientPath)) {
 New-Item -Path $ClientPath -Force | Out-Null
}
Set-ItemProperty -Path $ClientPath -Name "AllowBasic" -Value 0 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $ClientPath -Name "AllowUnencryptedTraffic" -Value 0 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $ClientPath -Name "AllowDigest" -Value 0 -Type DWord -ErrorAction Stop
Write-Host "[+] WinRM Client parameters hardened." -ForegroundColor Green

2. WinRM Service Hardening
$ServicePath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WinRM\Service"
if (-not (Test-Path $ServicePath)) {
 New-Item -Path $ServicePath -Force | Out-Null
}
Set-ItemProperty -Path $ServicePath -Name "AllowBasic" -Value 0 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $ServicePath -Name "AllowUnencryptedTraffic" -Value 0 -Type DWord -ErrorAction Stop
Set-ItemProperty -Path $ServicePath -Name "DisableRunAs" -Value 1 -Type DWord -ErrorAction Stop
Write-Host "[+] WinRM Service parameters hardened." -ForegroundColor Green

3. Windows Remote Shell Hardening
$WinRsPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WinRM\Service\WinRS"
if (-not (Test-Path $WinRsPath)) {
 New-Item -Path $WinRsPath -Force | Out-Null
}
Set-ItemProperty -Path $WinRsPath -Name "AllowRemoteShellAccess" -Value 0 -Type DWord -ErrorAction Stop
Write-Host "[+] Windows Remote Shell access disabled." -ForegroundColor Green

4. RPC Client Restrictions
$RpcPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Rpc"
if (-not (Test-Path $RpcPath)) {
 New-Item -Path $RpcPath -Force | Out-Null
}
Set-ItemProperty -Path $RpcPath -Name "RestrictRemoteClients" -Value 1 -Type DWord -ErrorAction Stop
Write-Host "[+] Unauthenticated RPC client restrictions enforced (RestrictRemoteClients = 1)." -ForegroundColor Green
```

To audit the WinRM client/service and RPC client settings status: [Download Script: Get-WinRMAndRpcHardeningStatus.ps1](#)

```
Get-WinRMAndRpcHardeningStatus.ps1
Description: Audits registry configuration of WinRM client/service options and RPC client restrictions.

Write-Host "--- Auditing WinRM and RPC Hardening Settings ---" -ForegroundColor Cyan

1. Audit WinRM Client
$ClientPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WinRM\Client"
$ExpectedClient = @{
 "AllowBasic" = 0
 "AllowUnencryptedTraffic" = 0
 "AllowDigest" = 0
}
if (Test-Path $ClientPath) {
 $ClientReg = Get-ItemProperty -Path $ClientPath -ErrorAction SilentlyContinue
 foreach ($S in $ExpectedClient.Keys) {
 $Val = $ClientReg.$S
 $Expected = $ExpectedClient[$S]
 $Color = if ($Val -eq $Expected) { "Green" } else { "Red" }
 Write-Host " - WinRM Client $($S): $Val (Expected: $Expected)" -ForegroundColor $Color
 }
} else {
 Write-Host " - WinRM Client Registry: NOT FOUND" -ForegroundColor Red
}

2. Audit WinRM Service
$ServicePath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WinRM\Service"
$ExpectedService = @{
 "AllowBasic" = 0
 "AllowUnencryptedTraffic" = 0
 "DisableRunAs" = 1
}
if (Test-Path $ServicePath) {
 $ServiceReg = Get-ItemProperty -Path $ServicePath -ErrorAction SilentlyContinue
 foreach ($S in $ExpectedService.Keys) {
 $Val = $ServiceReg.$S
 $Expected = $ExpectedService[$S]
 $Color = if ($Val -eq $Expected) { "Green" } else { "Red" }
 Write-Host " - WinRM Service $($S): $Val (Expected: $Expected)" -ForegroundColor $Color
 }
} else {
 Write-Host " - WinRM Service Registry: NOT FOUND" -ForegroundColor Red
}

3. Audit Windows Remote Shell
$WinRsPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WinRM\Service\WinRS"
if (Test-Path $WinRsPath) {
 $WinRsReg = Get-ItemProperty -Path $WinRsPath -ErrorAction SilentlyContinue
 $RsVal = $WinRsReg.AllowRemoteShellAccess
 $RsColor = if ($RsVal -eq 0) { "Green" } else { "Red" }
 Write-Host " - Windows Remote Shell AllowRemoteShellAccess: $RsVal (Expected: 0)" -ForegroundColor $RsColor
} else {
 Write-Host " - Windows Remote Shell Registry: NOT FOUND" -ForegroundColor Red
}

4. Audit RPC Clients
$RpcPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Rpc"
$RpcVal = Get-ItemProperty -Path $RpcPath -Name "RestrictRemoteClients" -ErrorAction SilentlyContinue
$RpcSetting = if ($RpcVal) { $RpcVal.RestrictRemoteClients } else { 0 }
$RpcColor = if ($RpcSetting -eq 1) { "Green" } else { "Red" }
Write-Host " - RPC RestrictRemoteClients: $RpcSetting (Expected: 1)" -ForegroundColor $RpcColor
```

## Sources & Compliance References \* \*\*CIS Benchmark\*\*: CIS Microsoft Windows Server Benchmark - Section 18.9 (Windows Remote Management), Section 18.1.2 (Remote Procedure Call) \* \*\*ANSSI AD Hardening Guide\*\*: Security guidelines regarding remote management and RPC access controls.

**# [REQ-NET-011] Configure WMI Static Port**

**## Target Scope** \* **\*\*Applicable Systems\*\***: Domain Controllers, Member Servers. \* **\*\*Operating Systems\*\***: Windows Server 2016 (and above).

**## Implementation Details** \* **\*\*Priority\*\***: Medium \* **\*\*GPO Path / Registry Location\*\***: \* **\*\*WMI DCOM Endpoints Registry Key\*\***:

`HKLM\SOFTWARE\Classes\AppID\{8BC3F05E-D86B-11D0-A075-00C04FB68820}` \* **Value Name**: `Endpoints` \* **Value Type**: `REG\_MULTI\_SZ` \* **Value Data**: `ncacn\_ip\_tcp,0,24158` \* **\*\*WMI Service Type Configuration\*\***: `HKLM\SYSTEM\CurrentControlSet\Services\winmgmt` \* **Value Name**: `Type` \* **Value Type**: `REG\_DWORD` \* **Value Data**: `16` (Decimal) (SERVICE\_WIN32\_OWN\_PROCESS)

**## Rationale** By default, Windows Management Instrumentation (WMI) runs in a shared svchost.exe process and dynamically allocates high-order TCP ports (49152-65535) for remote connections. This dynamic behavior prevents network administrators from implementing restrictive firewall policies, leaving the entire ephemeral port range exposed to the network.

Restricting WMI to a static port and process solves these security challenges:

1. **Attack Surface Reduction**: By configuring WMI to run on a static port (TCP 24158), administrators can block access to the generic high-order port range on perimeter firewalls while keeping WMI management accessible.
2. **Execution Isolation**: Moving WMI to a standalone host process prevents compromised shared services in the same svchost instance from tampering with management functions.
3. **Authentication Privacy**: Running `winmgmt.exe /standalonehost 6` sets the DCOM authentication level to `RPC_C_AUTHN_LEVEL_PKT_PRIVACY` (6). This enforces packet encryption for all WMI communications, mitigating man-in-the-middle (MITM) session hijacking and credential sniffing attacks.

**## Legacy Impact & Compatibility** \* **\*\*Service Interruption\*\***: Changing the WMI service type and endpoints requires a restart of the `winmgmt` service and its dependent services. Because multiple critical Windows infrastructure services depend on WMI (such as IP Helper, User Access Logging, and SMS Agent Host), a system reboot is recommended to apply this control cleanly. \* **\*\*Network Communication\*\***: Internal firewall appliances and host-based firewalls must have explicit allow rules permitting TCP port 24158 from the management subnets before applying this configuration.

**## Implementation Steps**

**#### Option A: Group Policy Object (GPO) Configuration (Preferred)**

**##### 1. Define Static WMI Port and Service Type via GPO Registry Preferences** 1. Open the **\*\*Group Policy Management Console\*\*** (`gpmc.msc`). 2. Create or edit a GPO targeting the target systems (e.g., `GPO\_Hardening\_Firewall\_Baseline`). 3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry` 4. Define the following **\*\*Registry Items\*\***: \* **\*\*WMI Static Port Assignment\*\***: \* **\*\*Action\*\***: `Update` \* **\*\*Hive\*\***: `HKEY\_LOCAL\_MACHINE` \* **\*\*Key Path\*\***: `SOFTWARE\Classes\AppID\{8BC3F05E-D86B-11D0-A075-00C04FB68820}` \* **\*\*Value Name\*\***: `Endpoints` \* **\*\*Value Type\*\***: `REG\_MULTI\_SZ` \* **\*\*Value Data\*\***: `ncacn\_ip\_tcp,0,24158` (Enter on a single line) \* **\*\*WMI Standalone Process Type\*\***: \* **\*\*Action\*\***: `Update` \* **\*\*Hive\*\***: `HKEY\_LOCAL\_MACHINE` \* **\*\*Key Path\*\***: `SYSTEM\CurrentControlSet\Services\winmgmt` \* **\*\*Value Name\*\***: `Type` \* **\*\*Value Type\*\***: `REG\_DWORD` \* **\*\*Value Data\*\***: `16` (Decimal) **##### 2. Configure Inbound Windows Firewall Rules** Ensure a GPO firewall rule permits inbound TCP port 24158 originating only from authorized management/PAW network ranges.

**#### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)**

Run the following scripts locally to apply the WMI static port configuration.

#### Remediation Script: [Download Script: Set-WMIStaticPort.ps1](implementation\_scripts/Set-WMIStaticPort.ps1)

```
Set-WMIStaticPort.ps1
Description: Configures WMI to run in a standalone host process on static TCP port 24158 with packet privacy.

Write-Host "Applying hardening requirement: Configure WMI Static Port..." -ForegroundColor Cyan

1. Configure the static TCP port 24158 for WMI AppID
$WmiAppIdPath = "HKLM:\SOFTWARE\Classes\AppID\{8BC3F05E-D86B-11D0-A075-00C04FB68820}"
if (-not (Test-Path $WmiAppIdPath)) {
 New-Item -Path $WmiAppIdPath -Force | Out-Null
}
Set Endpoints string array
Set-ItemProperty -Path $WmiAppIdPath -Name "Endpoints" -Value @("ncacn_ip_tcp,0,24158") -Type MultiString
Write-Host "[+] Configured WMI AppID static endpoint to TCP 24158." -ForegroundColor Green

2. Configure WMI to run in a standalone host process with packet privacy
This command sets HKLM\SYSTEM\CurrentControlSet\Services\winmgmt\Type to OWN_PROCESS (16)
and configures the default authentication level to PKT_PRIVACY (6).
Write-Host "[+] Configuring WMI service to run as standalone process..." -ForegroundColor Gray
$Proc = Start-Process -FilePath "winmgmt.exe" -ArgumentList "/standalonehost 6" -Wait -NoNewWindow -PassThru

if ($Proc.ExitCode -eq 0) {
 Write-Host "[+] WMI standalone host configuration completed successfully." -ForegroundColor Green
} else {
 Write-Warning "[-] WMI standalone host configuration exited with code $($Proc.ExitCode)."
}

Write-Host "[!] Note: A system reboot is recommended to cleanly apply WMI process changes and restart all dependencies." -ForegroundColor Yellow
```

#### Audit Script: [Download Script: Test-WMIStaticPort.ps1](audit\_scripts/Test-WMIStaticPort.ps1)

```
Test-WMIStaticPort.ps1
Description: Audits WMI static port registry configuration and standalone host settings.

Write-Host "Auditing WMI static port configuration..." -ForegroundColor Cyan
$Vulnerable = $false

1. Audit AppID Endpoints Registry Setting
$WmiAppIdPath = "HKLM:\SOFTWARE\Classes\AppID\{8BC3F05E-D86B-11D0-A075-00C04FB68820}"
$EndpointsVal = Get-ItemProperty -Path $WmiAppIdPath -Name "Endpoints" -ErrorAction SilentlyContinue

if ($EndpointsVal) {
 $Endpoints = $EndpointsVal.Endpoints
 if ($Endpoints -contains "ncacn_ip_tcp,0,24158") {
 Write-Host "[+] WMI static port registry endpoint is configured correctly (TCP 24158)." -ForegroundColor Green
 } else {
 Write-Host "[!] NON-COMPLIANT: WMI Endpoints registry value is: '$($Endpoints -join ', ')' (Expected: 'ncacn_ip_tcp,0,24158')" -ForegroundColor Red
 $Vulnerable = $true
 }
} else {
 Write-Host "[!] NON-COMPLIANT: Wmi AppID 'Endpoints' value is missing." -ForegroundColor Red
 $Vulnerable = $true
}

2. Audit WMI Service Execution Type
$WinmgmtPath = "HKLM:\SYSTEM\CurrentControlSet\Services\winmgmt"
$TypeVal = Get-ItemProperty -Path $WinmgmtPath -Name "Type" -ErrorAction SilentlyContinue

if ($TypeVal) {
 $Type = $TypeVal.Type
 if ($Type -eq 16) {
 Write-Host "[+] WMI is configured to run as a standalone process (Type = 16)." -ForegroundColor Green
 } else {
 Write-Host "[!] NON-COMPLIANT: WMI service execution type is: $Type (Expected: 16 [OWN_PROCESS])" -ForegroundColor Red
 $Vulnerable = $true
 }
} else {
 Write-Host "[!] NON-COMPLIANT: WMI service registry key is missing or inaccessible." -ForegroundColor Red
 $Vulnerable = $true
}

if ($Vulnerable) {
 Write-Host "Audit result: NON-COMPLIANT" -ForegroundColor Red
} else {
 Write-Host "Audit result: COMPLIANT" -ForegroundColor Green
}
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R7 (Filtering and IPsec on Domain Controllers), Recommendation R8 (Administration network subnets / filtering rules) \* \*\*CIS Windows Server Benchmark\*\*: Section 19 (Windows Defender Firewall with Advanced Security) \* \*\*DSInternals AD Firewall Guide (Michael Grafnetter)\*\*: [Active Directory Firewall - Domain Controller Firewall](https://firewall.dsinternals.com/ADDS/)

# [REQ-NET-012] Configure RPC Filters for Named Pipes

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers. \* \*\*Operating Systems\*\*: Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Startup Script Location\*\*: Computer Configuration\Policies\Windows Settings\Scripts (Startup) \* \*\*Netsh Configuration File\*\*: 'RpcNamedPipesFilters.txt' (imported via 'netsh.exe -f RpcNamedPipesFilters.txt')

## Rationale Active Directory domains heavily rely on the Server Message Block (SMB) protocol (TCP 445) for distributing group policies and replication via the SYSVOL and NETLOGON file shares. Therefore, SMB cannot simply be blocked at the network level on domain controllers. However, adversaries and common hacktools (e.g., PSEXec, Mimikatz) abuse this exposure. They establish connections to SMB named pipes (such as `\PIPE\svcm` for the Service Control Manager or `\PIPE\samr` for the Security Account Manager) to remotely execute code, manipulate services, or query administrative APIs.

Implementing Windows Firewall **RPC Filters** solves this vulnerability:

1. **Granular Control over Named Pipes:** Unlike standard L3/L4 firewall rules which can only allow or block all of TCP 445, RPC Filters inspect the RPC interface UUID inside the SMB payload.
2. **Selective Blocking:** Filters can block administrative RPC interfaces (like Service Control Manager `[MS-SCMR]`, Task Scheduler `[MS-TSCH]`, and Terminal Services runtime) over the `ncacn_np` (named pipe) transport, while leaving standard file sharing and replication functional.
3. **Exploit Containment:** By blocking interfaces like the Terminal Services runtime or MimiCom (Mimikatz interface), lateral movement and remote code execution techniques are contained.

## Legacy Impact & Compatibility \* \*\*Remote Administration Impact\*\*: Applying these filters will block remote administration of services ('sc.exe'), scheduled tasks ('schtasks.exe'), and event logs from unauthorized remote member servers using named pipes. Administrators must use secure TCP/IP management protocols (such as PowerShell Remoting/WinRM or remote MMC consoles utilizing `ncacn_ip_tcp` transport) from authorized PAWs. \* \*\*GPO Startup Dependencies\*\*: Since Windows does not provide a standard GPO interface for RPC Filters, they must be imported via startup scripts.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

#### 1. Create the RPC Filters Definition File Create a text file named 'RpcNamedPipesFilters.txt' with the following content: ``text rpc filter

```
add rule layer=um actiontype=block filterkey=d0c7640c-9355-4e52-8335-c12835559c10 add condition field=protocol matchtype=equal data=ncacn_np add condition field=if_uuid matchtype=equal data=367ABB81-9844-35F1-AD32-98F038001003 add filter
add rule layer=um actiontype=block filterkey=a43b9dd2-0866-4476-89dc-2e9b200762af add condition field=protocol matchtype=equal data=ncacn_np add condition field=if_uuid matchtype=equal data=86D35949-83C9-4044-B424-DB363231FD0C add filter
```

```
add rule layer=um actiontype=block filterkey=13518c11-e3d8-4f62-9461-eda11beb540a add condition field=if_uuid matchtype=equal data=1FF70682-0A51-30E8-076D-740BE8CEE98B add filter
```

```
add rule layer=um actiontype=block filterkey=1c079a18-e91f-4698-9868-68a121490636 add condition field=if_uuid matchtype=equal data=378E52B0-C0A9-11CF-822D-00AA0051E40F add filter
```

```
add rule layer=um actiontype=block filterkey=dedffabf-db89-4177-be77-1954aa2c0b95 add condition field=protocol matchtype=equal data=ncacn_np add condition field=if_uuid matchtype=equal data=f6beaff7-1e19-4fbb-9f8f-b89e2018337c add filter
```

```
add rule layer=um actiontype=block filterkey=f7f68868-5f50-4cda-a18c-6a7a549652e7 add condition field=if_uuid matchtype=equal data=82273FDC-E32A-18C3-3F78-827929DC23EA add filter
```

```
add rule layer=um actiontype=permit filterkey=43873c58-e130-4ffb-8858-d259a673a917 add condition field=if_uuid matchtype=equal data=4FC742E0-4A10-11CF-8273-00AA004AE673 add condition field=remote_user_token matchtype=equal data=D:(A;;CC;;;DA) add filter
```

```
add rule layer=um actiontype=block filterkey=0a239867-73db-45e6-b287-d006fe3c8b18 add condition field=if_uuid matchtype=equal data=4FC742E0-4A10-11CF-8273-00AA004AE673 add filter
```

```
add rule layer=um actiontype=block filterkey=7966512a-f2f4-4cb1-812d-d967ab83d28a add condition field=protocol matchtype=equal data=ncacn_np add condition field=if_uuid matchtype=equal data=12345678-1234-ABCD-EF00-0123456789AB add filter
```

```
add rule layer=um actiontype=permit filterkey=d71d00db-3eef-4935-bedf-20cf628abd9e add condition field=if_uuid matchtype=equal data=c681d488-d850-11d0-8c52-00c04fd90f7e add condition field=auth_type matchtype=equal data=16 add condition field=auth_level matchtype=equal data=6 add filter
```

```
add rule layer=um actiontype=block filterkey=3a4cce27-a7fa-4248-b8b8-ef6439a2c0ff add condition field=if_uuid matchtype=equal data=c681d488-d850-11d0-8c52-00c04fd90f7e add filter
```



```
add rule layer=um actiontype=permit filterkey=c5cf8020-c83c-4803-9241-8c7f3b10171f add condition field=if_uuid matchtype=equal data=df1941c5-fe89-4e79-bf10-463657acf44d add condition field=auth_type matchtype=equal data=16 add condition field=auth_level matchtype=equal data=6 add filter
```

```
add rule layer=um actiontype=block filterkey=9ad23a91-085d-4f99-ae15-85e0ad801278 add condition field=if_uuid matchtype=equal data=df1941c5-fe89-4e79-bf10-463657acf44d add filter
```

```
add rule layer=um actiontype=block filterkey=50754fe4-aa2d-42ff-8196-e90ea8fd2527 add condition field=protocol matchtype=equal data=ncacn_np add condition field=if_uuid matchtype=equal data=50abc2a4-574d-40b3-9d66-ee4fd5fba076 add filter
```

```
add rule layer=um actiontype=block filterkey=644291ca-9530-4066-b654-e7b838ebdc06 add condition field=if_uuid matchtype=equal data=17FC11E9-C258-4B8D-8D07-2F4125156244 add filter
```

```
add rule layer=um actiontype=block filterkey=5270da6b-67a8-4cbf-8b2c-fa5d0abcb975 add condition field=protocol matchtype=equal data=ncacn_np add condition field=if_uuid matchtype=equal data=a8e0653c-2744-4389-a61d-7373df8b2292 add filter
```

```
<div id="04-network-firewall-configure-rpc-named-pipe-filters-md-2-deploy-via-gpo-startup-script"></div>
2. Deploy via GPO Startup Script
1. Save 'RpcNamedPipesFilters.txt' inside the GPO's Startup folder.
2. Create a batch script 'Import-RpcFilters.bat' containing:
 ``cmd
 @echo off
 netsh.exe -f "%~dp0RpcNamedPipesFilters.txt"
```

3. Set the batch script as a Startup Script in the GPO targeting Domain Controllers under: [Computer Configuration\Policies\Windows Settings\Scripts \(Startup\)](#)

---

#### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to import and query RPC named pipe filters.



#### Remediation Script: [Download Script: Set-RpcNamedPipeFilters.ps1](implementation\_scripts/Set-RpcNamedPipeFilters.ps1)

```
Set-RpcNamedPipeFilters.ps1
Description: Generates and imports RPC filters to block remote management over SMB named pipes.

Write-Host "Applying hardening requirement: Configure RPC Named Pipe Filters..." -ForegroundColor Cyan

Define the path where the filters file will be written
$FilterFilePath = Join-Path $env:TEMP "RpcNamedPipesFilters.txt"

Write the netsh RPC filter commands
$FilterContent = @"
rpc filter
add rule layer=um actiontype=block filterkey=d0c7640c-9355-4e52-8335-c12835559c10
add condition field=protocol matchtype=equal data=ncacn_np
add condition field=if_uuid matchtype=equal data=367ABB81-9844-35F1-AD32-98F038001003
add filter

add rule layer=um actiontype=block filterkey=a43b9dd2-0866-4476-89dc-2e9b200762af
add condition field=protocol matchtype=equal data=ncacn_np
add condition field=if_uuid matchtype=equal data=86D35949-83C9-4044-B424-DB363231FD0C
add filter

add rule layer=um actiontype=block filterkey=13518c11-e3d8-4f62-9461-eda11beb540a
add condition field=if_uuid matchtype=equal data=1FF70682-0A51-30E8-076D-740BE8CEE98B
add filter

add rule layer=um actiontype=block filterkey=1c079a18-e91f-4698-9868-68a121490636
add condition field=if_uuid matchtype=equal data=378E52B0-C0A9-11CF-822D-00AA0051E40F
add filter

add rule layer=um actiontype=block filterkey=dedffabf-db89-4177-be77-1954aa2c0b95
add condition field=protocol matchtype=equal data=ncacn_np
add condition field=if_uuid matchtype=equal data=f6beaff7-1e19-4fbb-9f8f-b89e2018337c
add filter

add rule layer=um actiontype=block filterkey=f7f68868-5f50-4cda-a18c-6a7a549652e7
add condition field=if_uuid matchtype=equal data=82273FDC-E32A-18C3-3F78-827929DC23EA
add filter

add rule layer=um actiontype=permit filterkey=43873c58-e130-4ffb-8858-d259a673a917
add condition field=if_uuid matchtype=equal data=4FC742E0-4A10-11CF-8273-00AA004AE673
add condition field=remote_user_token matchtype=equal data=D:(A;;CC;;;DA)
add filter

add rule layer=um actiontype=block filterkey=0a239867-73db-45e6-b287-d006fe3c8b18
add condition field=if_uuid matchtype=equal data=4FC742E0-4A10-11CF-8273-00AA004AE673
add filter

add rule layer=um actiontype=block filterkey=7966512a-f2f4-4cb1-812d-d967ab83d28a
add condition field=protocol matchtype=equal data=ncacn_np
add condition field=if_uuid matchtype=equal data=12345678-1234-ABCD-EF00-0123456789AB
add filter

add rule layer=um actiontype=permit filterkey=d71d00db-3eef-4935-bedf-20cf628abd9e
add condition field=if_uuid matchtype=equal data=c681d488-d850-11d0-8c52-00c04fd90f7e
add condition field=auth_type matchtype=equal data=16
add condition field=auth_level matchtype=equal data=6
add filter

add rule layer=um actiontype=block filterkey=3a4cce27-a7fa-4248-b8b8-ef6439a2c0ff
add condition field=if_uuid matchtype=equal data=c681d488-d850-11d0-8c52-00c04fd90f7e
add filter

add rule layer=um actiontype=permit filterkey=c5cf8020-c83c-4803-9241-8c7f3b10171f
add condition field=if_uuid matchtype=equal data=df1941c5-fe89-4e79-bf10-463657acf44d
add condition field=auth_type matchtype=equal data=16
add condition field=auth_level matchtype=equal data=6
add filter

add rule layer=um actiontype=block filterkey=9ad23a91-085d-4f99-ae15-85e0ad801278
add condition field=if_uuid matchtype=equal data=df1941c5-fe89-4e79-bf10-463657acf44d
add filter

add rule layer=um actiontype=block filterkey=50754fe4-aa2d-42ff-8196-e90ea8fd2527
add condition field=protocol matchtype=equal data=ncacn_np
```

```

add condition field=if_uuid matchtype=equal data=50abc2a4-574d-40b3-9d66-ee4fd5fba076
add filter
add rule layer=um actiontype=block filterkey=644291ca-9530-4066-b654-e7b838ebdc06
add condition field=if_uuid matchtype=equal data=17FC11E9-C258-4B8D-8D07-2F4125156244
add filter

add rule layer=um actiontype=block filterkey=5270da6b-67a8-4cbf-8b2c-fa5d0abcb975
add condition field=protocol matchtype=equal data=ncacn_np
add condition field=if_uuid matchtype=equal data=a8e0653c-2744-4389-a61d-7373df8b2292
add filter
"@

Write filters to temp file
Set-Content -Path $FilterFilePath -Value $FilterContent -Encoding Ascii
Write-Host "[+] Generated RPC filters file at: $FilterFilePath" -ForegroundColor Gray

Import using netsh (requires elevation)
Write-Host "[+] Importing RPC filters using Netsh..." -ForegroundColor Gray
$Proc = Start-Process -FilePath "netsh.exe" -ArgumentList "rpc filter import `"$FilterFilePath`"" -Wait -NoNewWindow -PassThru

if ($Proc.ExitCode -eq 0) {
 Write-Host "[+] RPC filters imported successfully." -ForegroundColor Green
} else {
 Write-Error "[-] Failed to import RPC filters. Netsh exit code: $($Proc.ExitCode)."
}

Clean up temp file
if (Test-Path $FilterFilePath) {
 Remove-Item -Path $FilterFilePath -Force | Out-Null
}

```

#### Audit Script: [Download Script: Test-RpcNamedPipeFilters.ps1](audit\_scripts/Test-RpcNamedPipeFilters.ps1)

```
Test-RpcNamedPipeFilters.ps1
Description: Audits active RPC filters configuration using Netsh queries.

Write-Host "Auditing RPC filters configuration..." -ForegroundColor Cyan

Query RPC filters
$filters = netsh rpc filter show filter

Check for presence of SCM block rule filterkey, Mimikatz block rule filterkey, or MS-FSRVP block rule filterkey
$scmFound = $false
$mimiFound = $false
$fsrvpFound = $false

foreach ($line in $filters) {
 if ($line -like "*d0c7640c-9355-4e52-8335-c12835559c10*") {
 $scmFound = $true
 }
 if ($line -like "*644291ca-9530-4066-b654-e7b838ebdc06*") {
 $mimiFound = $true
 }
 if ($line -like "*5270da6b-67a8-4cbf-8b2c-fa5d0abcb975*") {
 $fsrvpFound = $true
 }
}

if ($scmFound -and $mimiFound -and $fsrvpFound) {
 Write-Host "[+] RPC Filters for named pipes are active (SCM, Mimikatz, and MS-FSRVP filterkeys found)." -ForegroundColor Green
 Write-Host "Audit result: COMPLIANT" -ForegroundColor Green
} else {
 Write-Host "[!] NON-COMPLIANT: Core RPC named pipe filters are missing or inactive." -ForegroundColor Red
 if (-not $scmFound) {
 Write-Host " - Missing SCM Named Pipe Filter (d0c7640c-9355-4e52-8335-c12835559c10)" -ForegroundColor Yellow
 }
 if (-not $mimiFound) {
 Write-Host " - Missing Mimikatz Filter (644291ca-9530-4066-b654-e7b838ebdc06)" -ForegroundColor Yellow
 }
 if (-not $fsrvpFound) {
 Write-Host " - Missing MS-FSRVP ShadowCoerce Filter (5270da6b-67a8-4cbf-8b2c-fa5d0abcb975)" -ForegroundColor Yellow
 }
 Write-Host "Audit result: NON-COMPLIANT" -ForegroundColor Red
}
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R7 (Filtering and IPsec on Domain Controllers), Recommendation R8 (Administration network subnets / filtering rules) \* \*\*CIS Benchmark\*\*: Section 19 (Windows Defender Firewall with Advanced Security) \* \*\*DSInternals AD Firewall Guide (Michael Grafnetter)\*\*: [Active Directory Firewall - Domain Controller Firewall] (<https://firewall.dsinternals.com/ADDS/>)

# [REQ-NET-013] Block Management Traffic Between Domain Controllers

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers. \* \*\*Operating Systems\*\*: Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Windows Firewall Inbound Rules\*\*: Exclusion of Domain Controller IP addresses from allowed remote management rules (RDP, WinRM, WMI, ADWS) in the Domain Controllers GPO.

## Rationale Domain Controllers (DCs) represent the Tier 0 security boundary of an Active Directory forest. In a multi-DC environment, security controls must prevent lateral movement and credential escalation between these core servers.

If an adversary gains administrative control of a single Domain Controller, they will immediately attempt to pivot to other DCs. By default, standard firewall configurations allow remote management protocols (RDP, WinRM, WMI) from all Tier 0 assets—including other Domain Controllers.

Enforcing intra-DC remote management blocking resolves this vector:

1. **Lateral Movement Containment:** Restricting management traffic between DCs (e.g. blocking RDP TCP 3389, WinRM TCP 5985/5986, WMI TCP 24158, and ADWS TCP 9389 from other DC IP addresses) prevents a compromised DC from being used to compromise other domain controllers.
2. **Replication Integrity:** Normal Active Directory replication and synchronization protocols (RPC replication, DNS, Kerberos, SMB) remain open and unaffected, while administrative logon and command execution protocols are blocked.
3. **Zero Trust Tiering:** Assumes that even Tier 0 systems must not trust other Tier 0 systems for remote command execution.

## Legacy Impact & Compatibility \* \*\*Administration Workflow\*\*: Administrators cannot RDP or execute remote PowerShell Remoting commands directly from one DC to another. They must manage each DC individually from authorized Privileged Access Workstations (PAWs) or dedicated Tier 0 jump hosts. \* \*\*Multi-DC Discovery\*\*: For non-GPO scripting configurations, the remediation script dynamically queries Active Directory to identify and block the IP addresses of other DCs.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

Configure the Domain Controllers GPO to restrict source IP addresses for remote management rules:

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Create or edit a GPO targeting Domain Controllers (e.g., `GPO_Hardening_DomainControllers_Firewall` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security`
4. Under **Inbound Rules**, configure rules for remote management (RDP, WinRM, WMI, ADWS) to:
  - Allow connections originating only from **Management/PAW subnets**.
  - Ensure that **Domain Controller IP addresses** are explicitly excluded from the allowed remote address list.
  - Alternatively, create explicit inbound **Block Rules** for ports `3389` , `5985` , `5986` , `24158` , and `9389` where the source IP addresses match the list of Domain Controllers.

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally on each DC to implement and audit the block rules.

#### Remediation Script: [Download Script: Set-IntraDcManagementBlocking.ps1](implementation\_scripts/Set-

## IntraDcManagementBlocking.ps1)

```
Set-IntraDcManagementBlocking.ps1
Description: Configures local Windows Firewall rules to block remote management traffic (RDP, WinRM, WMI, ADWS) originating from o
ther Domain Controllers to prevent lateral movement.

Write-Host "Applying hardening requirement: Block Management Traffic Between DCs..." -ForegroundColor Cyan

1. Get IP addresses of all Domain Controllers in the domain
$DcIPs = @()
try {
 $Dcs = Get-ADDomainController -Filter * -ErrorAction Stop
 foreach ($Dc in $Dcs) {
 # Skip local computer
 if ($Dc.Name -ne $env:COMPUTERNAME) {
 if ($Dc.IPv4Address) { $DcIPs += $Dc.IPv4Address }
 if ($Dc.IPv6Address) { $DcIPs += $Dc.IPv6Address }
 }
 }
} catch {
 Write-Host " Get-ADDomainController not available or not in AD domain. Skipping dynamic discovery." -ForegroundColor Yellow
}

if ($DcIPs.Count -eq 0) {
 Write-Host " No other Domain Controllers discovered. Blocking rule will be created but inactive." -ForegroundColor Yellow
 # Fallback to a dummy address to ensure rule structure is correct
 $DcIPs = @("255.255.255.255")
} else {
 Write-Host " Discovered other DCs: $($DcIPs -join ', ')" -ForegroundColor Gray
}

2. Configure local block rules for DC-to-DC remote management
$BlockRules = @(
 @{ Name = "AD-Block-IntraDC-RDP"; Port = 3389; Proto = "TCP" },
 @{ Name = "AD-Block-IntraDC-WinRM-HTTP"; Port = 5985; Proto = "TCP" },
 @{ Name = "AD-Block-IntraDC-WinRM-HTTPS"; Port = 5986; Proto = "TCP" },
 @{ Name = "AD-Block-IntraDC-WMI"; Port = 24158; Proto = "TCP" },
 @{ Name = "AD-Block-IntraDC-ADWS"; Port = 9389; Proto = "TCP" }
)

foreach ($Rule in $BlockRules) {
 $Name = $Rule.Name
 $Port = $Rule.Port
 $Proto = $Rule.Proto

 $Existing = Get-NetFirewallRule -Name $Name -ErrorAction SilentlyContinue
 if ($null -eq $Existing) {
 New-NetFirewallRule -Name $Name -DisplayName $Name `
 -Direction Inbound `
 -Action Block `
 -Protocol $Proto `
 -LocalPort $Port `
 -RemoteAddress $DcIPs `
 -Profile Domain, Private `
 -Enabled True | Out-Null
 Write-Host "Block rule created: $($Name) on port $($Port) ($($Proto)) from other DCs." -ForegroundColor Green
 } else {
 Set-NetFirewallRule -Name $Name -Enabled True -Action Block -RemoteAddress $DcIPs | Out-Null
 Write-Host "Block rule verified: $($Name) on port $($Port) ($($Proto)) from other DCs." -ForegroundColor Gray
 }
}

Write-Host "Intra-DC management blocking rules applied successfully." -ForegroundColor Cyan
```

#### Audit Script: [Download Script: Test-IntraDcManagementBlocking.ps1](audit\_scripts/Test-IntraDcManagementBlocking.ps1)

```
Test-IntraDcManagementBlocking.ps1
Description: Audits if management traffic from other Domain Controllers is blocked.

Write-Host "Auditing Intra-DC management blocking configurations..." -ForegroundColor Cyan

$vulnerable = $false
$BlockRules = @(
 "AD-Block-IntraDC-RDP",
 "AD-Block-IntraDC-WinRM-HTTP",
 "AD-Block-IntraDC-WinRM-HTTPS",
 "AD-Block-IntraDC-WMI",
 "AD-Block-IntraDC-ADWS"
)

foreach ($Name in $BlockRules) {
 $Rule = Get-NetFirewallRule -Name $Name -ErrorAction SilentlyContinue
 if ($Rule) {
 if ($Rule.Enabled -eq $true -and $Rule.Action -eq "Block") {
 Write-Host "[+] Rule $($Name) is active and configured to Block." -ForegroundColor Green
 } else {
 Write-Host "[!] NON-COMPLIANT: Rule $($Name) exists but is disabled or not set to Block." -ForegroundColor Red
 $vulnerable = $true
 }
 } else {
 Write-Host "[!] NON-COMPLIANT: Block rule $($Name) is missing." -ForegroundColor Red
 $vulnerable = $true
 }
}

if ($vulnerable) {
 Write-Host "Audit result: NON-COMPLIANT" -ForegroundColor Red
} else {
 Write-Host "Audit result: COMPLIANT" -ForegroundColor Green
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*\*: Recommendation R7 (Filtering and IPsec on Domain Controllers), Recommendation R8 (Administration network subnets / filtering rules) \* \*\*CIS Benchmark\*\*\*: Section 19 (Windows Defender Firewall with Advanced Security) \* \*\*DSInternals AD Firewall Guide (Michael Grafnetter)\*\*\*: [Active Directory Firewall - Domain Controller Firewall] (<https://firewall.dsinternals.com/ADDSD/>)

## # Module 5: Logging, Monitoring & SIEM

This directory contains configuration policies for security log auditing, PowerShell transcription, and host monitoring for detection systems in isolated networks.

1. [REQ-LOG-001 - Configure Advanced Security Audit Policies](#) Enforces granular Windows security audit policies (including logons, Kerberos authentication operations, group memberships, policy changes, and process execution) to log critical threat telemetry.
2. [REQ-LOG-002 - Configure PowerShell and Command-Line Auditing](#) Enforces process command-line argument auditing and verbose PowerShell logging (Script Block, Module, and Transcription logging) with a write-only, hardened transcript folder.
3. [REQ-LOG-003 - Deploy and Harden Microsoft Sysmon](#) Deploys Sysmon with a hardened telemetry configuration and configures aggressive service recovery settings to auto-restart the service if stopped by adversaries.
4. [REQ-LOG-004 - Configure Secure SIEM Log Shipping](#) Configures secured log shipping agents (Winlogbeat and Wazuh) utilizing TLS encryption, authenticated CA checks, local configuration file ACL protections, and buffer queue size limits to prevent local disk space exhaustion.
5. [REQ-LOG-005 - Configure Kerberoasting Honeypots and SIEM Detection Rules](#) Deploys decoy service accounts in Active Directory to attract Kerberoasting scans and provides high-fidelity SIEM alerting queries.
6. [REQ-LOG-006 - Configure SYSVOL Decoy XML Honeypot](#) Deploys a mock Group Policy folder structure and dummy preferences XML file with a Deny Read rule for Everyone and file access failure auditing.



**# [REQ-LOG-001] Configure Advanced Security Audit Policies**

**## Target Scope** \* **Applicable Systems**: Domain Controllers, Member Servers, Tier 2 Client Workstations \* **Operating Systems**: Windows Server 2016 (and above), Windows 10/11 Enterprise

---

**## Implementation Details** \* **Priority**: High \* **GPO Path / Registry Location**: \* **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` \* **Policy**: `Audit: Force audit policy subcategory settings (Windows Vista or later) to override audit policy category settings` \* **Setting**: `Enabled` \* **Registry Location**: `HKLM\System\CurrentControlSet\Control\Lsa\SCENoApplyLegacyAuditPolicy` = `1` (DWord)

---

**## Rationale** Standard Windows security event logging is basic and fails to capture critical event vectors, leading to visibility gaps during compromises. Enforcing refined subcategory audit policies ensures detailed Success and Failure logs for logon attempts, privilege use, process creations, and registry modifications without overloading log stores. This requirement acts as the primary logging baseline, enforcing category overrides and linking to profile-specific submodules matching each system's security tier.

---

**## Legacy Impact & Compatibility** \* **Event Log Volume**: Set local Security Event Log size to a minimum of 512MB to prevent premature rollover of security auditing data.

---

**## Modular Profile Audit Policies** This logging requirement is split into profile-specific advanced audit submodules mapping to each system's security tier: \* **[REQ-DC-135 - Configure Advanced Security Audit Policies for Domain Controllers](#02-domain-controllers-audit-policy-README-md)** \* **[REQ-PAW-129 - Configure Advanced Security Audit Policies for PAWs](#07-paws-audit-policy-README-md)** \* **[REQ-END-140 - Configure Advanced Security Audit Policies for Endpoints](#08-endpoints-audit-policy-README-md)**

---

**## Implementation Steps**

**### Option A: Group Policy Object (GPO) Configuration (Preferred)** 1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host. 2. Edit the corresponding baseline GPO (e.g. `GPO\_Hardening\_Baseline`). 3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` 4. Configure the following setting: \* **Policy**: `Audit: Force audit policy subcategory settings (Windows Vista or later) to override audit policy category settings` \* **Setting**: `Enabled`

---

**### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)** [Download Script: [Configure-AdvancedAuditPolicies.ps1](#)] ([implementation\\_scripts/Configure-AdvancedAuditPolicies.ps1](#))

```
Configure-AdvancedAuditPolicies.ps1
Description: Enforces Advanced Audit Policy Overrides registry value.

$RegPath = "reg:\HKLM\System\CurrentControlSet\Control\Lsa"
$ValueName = "SCENoApplyLegacyAuditPolicy"

Write-Host "Enforcing Advanced Security Audit Policies override settings..." -ForegroundColor Cyan

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value 1 -Type DWord -Force
Write-Host "Advanced Audit Policy overrides enforced successfully." -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-AdvancedAuditPoliciesStatus.ps1](#)

```
Get-AdvancedAuditPoliciesStatus.ps1
Description: Audits the Advanced Audit Policy Overrides registry value.

$RegPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
$ValueName = "SCENoApplyLegacyAuditPolicy"

$val = Get-ItemPropertyValue -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
if ($val -eq 1) {
 Write-Host "Advanced Security Audit Policy Overrides are correctly enabled." -ForegroundColor Green
} else {
 Write-Host "Advanced Security Audit Policy Overrides are disabled!" -ForegroundColor Red
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*\*: Recommendation R48 \* \*\*CIS Benchmark\*\*\*: Section 9 and 17

**# [REQ-LOG-002] Configure PowerShell and Command-Line Auditing**

**## Target Scope** \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers, Tier 2 Client Workstations. \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise.

---

**## Implementation Details** \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Process Command-Line\*\*: `Computer Configuration\Policies\Administrative Templates\System\Audit Process Creation\Include command line in process creation events` - Registry: `HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies\System\Audit\ProcessCreationIncludeCmdLine\_Enabled` (Value: 1) \* \*\*PowerShell Script Block Logging\*\*: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows PowerShell\Turn on PowerShell Script Block Logging` - Registry: `HKLM:\Software\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging\EnableScriptBlockLogging` (Value: 1) - Registry: `HKLM:\Software\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging\EnableScriptBlockInvocationLogging` (Value: 0) \* \*\*PowerShell Module Logging\*\*: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows PowerShell\Turn on PowerShell Module Logging` - Registry: `HKLM:\Software\Policies\Microsoft\Windows\PowerShell\ModuleLogging\EnableModuleLogging` (Value: 1) - Registry: `HKLM:\Software\Policies\Microsoft\Windows\PowerShell\ModuleLogging\ModuleNames` (Value: "") \* \*\*PowerShell Transcription\*\*: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows PowerShell\Turn on PowerShell Transcription` - Registry: `HKLM:\Software\Policies\Microsoft\Windows\PowerShell\Transcription\EnableTranscripting` (Value: 1) - Registry: `HKLM:\Software\Policies\Microsoft\Windows\PowerShell\Transcription\EnableInvocationHeader` (Value: 1) - Registry: `HKLM:\Software\Policies\Microsoft\Windows\PowerShell\Transcription\OutputDirectory` (Value: "C:\ProgramData\PowerShellTranscripts")

---

**## Rationale** Adversaries make extensive use of built-in system tools (LOLBins) and PowerShell script execution to perform reconnaissance, privilege escalation, and lateral movement. Because PowerShell code can be dynamically obfuscated or executed directly in memory without writing to disk, traditional file-based detection mechanisms are easily bypassed.

Enforcing advanced execution logging mitigates these threat vectors:

1. **Command Line Auditing**: Injecting command-line arguments into Security Event ID 4688 allows defenders to see parameters, file paths, and encoded arguments used during process execution.
2. **Script Block Logging**: Captures the full content of code blocks executed by PowerShell (Event ID 4104), logging the actual code after decryption and de-obfuscation at runtime.
3. **Module Logging**: Logs pipeline execution details and loaded modules (Event ID 4103) to map tool usage.
4. **Hardened Transcription**: Records all input commands and console outputs to local text transcripts. By restricting folder access permissions, users are blocked from reading previously typed commands (which may contain sensitive arguments, tokens, or credentials), while still allowing the system to log their actions.

---

**## Legacy Impact & Compatibility** \* \*\*Disk Space Utilization\*\*: Transcription files and verbose script block logs consume disk storage and create disk I/O. System administrators must ensure that the transcript destination directory is regularly audited, and that logs are forwarded to a SIEM and purged to prevent disk exhaustion. \* \*\*Sensitive Telemetry\*\*: Transcripts can accidentally log sensitive data like passwords typed in cleartext or API keys passed on the command line. Securing the transcription folder using strict NTFS ACLs is mandatory.

---

**## Implementation Steps****### Option A: Group Policy Object (GPO) Configuration (Preferred)**

**#### 1. Configure Process Command-Line Logging** 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Create or edit a GPO linked to target systems (e.g., `GPO\_Hardening\_Logging\_Baseline`). 3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Audit Process Creation` 4. Configure the setting: \* \*\*Policy\*\*: `Include command line in process creation events` \* \*\*Setting\*\*: `Enabled`

**#### 2. Configure PowerShell Audit Settings** 1. Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows PowerShell` 2. Configure \* \*\*Script Block Logging\*\*: \* \*\*Policy\*\*: `Turn on PowerShell Script Block Logging` \* \*\*Setting\*\*: `Enabled` (Ensure \* \*\*Log script block invocation start / stop events\*\* is **unchecked** / disabled to prevent operational log flooding) 3. Configure \* \*\*Module Logging\*\*: \* \*\*Policy\*\*: `Turn on PowerShell Module Logging` \* \*\*Setting\*\*: `Enabled` → Click **Show...** -> Add `\*` under Value. 4. Configure \* \*\*PowerShell Transcription\*\*: \* \*\*Policy\*\*: `Turn on PowerShell Transcription` \* \*\*Setting\*\*: `Enabled` \* \*\*Transcript output directory\*\*: `C:\ProgramData\PowerShellTranscripts` (Ensure this is a local path) \* \*\*Include invocation headers\*\*: Checked

---

**### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)**

Run the following scripts locally to configure command line process creation, PowerShell logging registry keys, and create the hardened transcript directory with write-only NTFS permissions.

[Download Script: Set-PowerShellAuditing.ps1](#)

```
Set-PowerShellAuditing.ps1
Configures command line auditing, PowerShell logging, transcription, and hardens folder ACLs.

Write-Host "--- Applying PowerShell & Command Line Auditing Remediation ---" -ForegroundColor Cyan

1. Enable Process Creation Command-Line Auditing
Write-Host "[+] Configuring Command Line Process Auditing..." -ForegroundColor Gray
$ProcAuditReg = "HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies\System\Audit"
if (-not (Test-Path $ProcAuditReg)) {
 New-Item -Path $ProcAuditReg -Force | Out-Null
}
Set-ItemProperty -Path $ProcAuditReg -Name "ProcessCreationIncludeCmdLine_Enabled" -Value 1 -Type DWord
Write-Host " Command line process auditing enabled." -ForegroundColor Green

2. Configure PowerShell Script Block Logging
Write-Host "[+] Configuring PowerShell Script Block Logging..." -ForegroundColor Gray
$ScriptBlockReg = "HKLM:\Software\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging"
if (-not (Test-Path $ScriptBlockReg)) {
 New-Item -Path $ScriptBlockReg -Force | Out-Null
}
Set-ItemProperty -Path $ScriptBlockReg -Name "EnableScriptBlockLogging" -Value 1 -Type DWord
Set-ItemProperty -Path $ScriptBlockReg -Name "EnableScriptBlockInvocationLogging" -Value 0 -Type DWord
Write-Host " PowerShell Script Block Logging enabled and invocation start/stop logging disabled." -ForegroundColor Green

3. Configure PowerShell Module Logging
Write-Host "[+] Configuring PowerShell Module Logging..." -ForegroundColor Gray
$ModuleReg = "HKLM:\Software\Policies\Microsoft\Windows\PowerShell\ModuleLogging"
if (-not (Test-Path $ModuleReg)) {
 New-Item -Path $ModuleReg -Force | Out-Null
}
Set-ItemProperty -Path $ModuleReg -Name "EnableModuleLogging" -Value 1 -Type DWord

$ModuleNamesReg = "HKLM:\Software\Policies\Microsoft\Windows\PowerShell\ModuleLogging\ModuleNames"
if (-not (Test-Path $ModuleNamesReg)) {
 New-Item -Path $ModuleNamesReg -Force | Out-Null
}
Set-ItemProperty -Path $ModuleNamesReg -Name "*" -Value "*" -Type String
Write-Host " PowerShell Module Logging enabled for all modules." -ForegroundColor Green

4. Configure PowerShell Transcription
Write-Host "[+] Configuring PowerShell Transcription Registry settings..." -ForegroundColor Gray
$TransReg = "HKLM:\Software\Policies\Microsoft\Windows\PowerShell\Transcription"
if (-not (Test-Path $TransReg)) {
 New-Item -Path $TransReg -Force | Out-Null
}
$TranscriptPath = "C:\ProgramData\PowerShellTranscripts"
Set-ItemProperty -Path $TransReg -Name "EnableTranscripting" -Value 1 -Type DWord
Set-ItemProperty -Path $TransReg -Name "EnableInvocationHeader" -Value 1 -Type DWord
Set-ItemProperty -Path $TransReg -Name "OutputDirectory" -Value $TranscriptPath -Type String
Write-Host " PowerShell Transcription registry keys configured." -ForegroundColor Green

5. Create and Harden PowerShell Transcript Folder
Write-Host "[+] Setting up hardened NTFS permissions on $($TranscriptPath)..." -ForegroundColor Gray
if (-not (Test-Path $TranscriptPath)) {
 New-Item -Path $TranscriptPath -ItemType Directory -Force | Out-Null
}

Fetch ACL and disable inheritance, copying existing rules
$Acl = Get-Acl -Path $TranscriptPath
$Acl.SetAccessRuleProtection($true, $true)
Set-Acl -Path $TranscriptPath -AclObject $Acl

Refresh ACL and remove user/authenticated user permissions
$Acl = Get-Acl -Path $TranscriptPath
$Rules = $Acl.Access
foreach ($Rule in $Rules) {
 $Identity = $Rule.IdentityReference.Value
 if ($Identity -like "*Users" -or $Identity -like "*Authenticated Users" -or $Identity -like "*Everyone") {
 $Acl.RemoveAccessRule($Rule) | Out-Null
 }
}

Define write-only permissions for Authenticated Users
CreateFiles/AppendData allows logging, while missing ListDirectory/ReadData blocks viewing transcripts.
```

```
$WriteRights = [System.Security.AccessControl.FileSystemRights]("CreateFiles, AppendData, ReadAttributes, WriteAttributes")
$InheritanceFlags = [System.Security.AccessControl.InheritanceFlags]("ContainerInherit, ObjectInherit")
$PropagationFlags = [System.Security.AccessControl.PropagationFlags]::None
$AccessType = [System.Security.AccessControl.AccessControlType]::Allow

$AccessRule = New-Object System.Security.AccessControl.FileSystemAccessRule("NT AUTHORITY\Authenticated Users", $WriteRights, $InheritanceFlags, $PropagationFlags, $AccessType)
$Acl.AddAccessRule($AccessRule)
Set-Acl -Path $TranscriptPath -AclObject $Acl
Write-Host " Hardened NTFS permissions applied to $($TranscriptPath) successfully." -ForegroundColor Green
```

*To verify the settings have been applied:*

[Download Script: Test-PowerShellAuditing.ps1](#)

```
Test-PowerShellAuditing.ps1
Audits command line process creation, PowerShell logging, and transcript folder permissions.

Write-Host "--- Auditing PowerShell & Command Line Auditing ---" -ForegroundColor Cyan

1. Audit Process Command-Line Logging
$ProcPath = "HKLM:\Software\Microsoft\Windows\CurrentVersion\Policies\System\Audit"
$CmdLineVal = Get-ItemProperty -Path $ProcPath -Name "ProcessCreationIncludeCmdLine_Enabled" -ErrorAction SilentlyContinue
$CmdSetting = 0
if ($CmdLineVal) {
 $CmdSetting = $CmdLineVal.ProcessCreationIncludeCmdLine_Enabled
}
$CmdColor = "Red"
if ($CmdSetting -eq 1) {
 $CmdColor = "Green"
}
Write-Host " - Command Line Process Auditing: $($CmdSetting) (Required = 1)" -ForegroundColor $CmdColor

2. Audit Script Block Logging
$SBPath = "HKLM:\Software\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging"
$SBVal = Get-ItemProperty -Path $SBPath -Name "EnableScriptBlockLogging" -ErrorAction SilentlyContinue
$SBSetting = 0
if ($SBVal) {
 $SBSetting = $SBVal.EnableScriptBlockLogging
}
$SBColor = "Red"
if ($SBSetting -eq 1) {
 $SBColor = "Green"
}
Write-Host " - PowerShell Script Block Logging: $($SBSetting) (Required = 1)" -ForegroundColor $SBColor

$SBInvVal = Get-ItemProperty -Path $SBPath -Name "EnableScriptBlockInvocationLogging" -ErrorAction SilentlyContinue
$SBInvSetting = 0
if ($SBInvVal) {
 $SBInvSetting = $SBInvVal.EnableScriptBlockInvocationLogging
}
$SBInvColor = "Red"
if ($SBInvSetting -eq 0) {
 $SBInvColor = "Green"
}
Write-Host " - PowerShell Script Block Invocation Logging (Start/Stop): $($SBInvSetting) (Required = 0)" -ForegroundColor $SBInvColor

3. Audit Module Logging
$ModPath = "HKLM:\Software\Policies\Microsoft\Windows\PowerShell\ModuleLogging"
$ModVal = Get-ItemProperty -Path $ModPath -Name "EnableModuleLogging" -ErrorAction SilentlyContinue
$ModSetting = 0
if ($ModVal) {
 $ModSetting = $ModVal.EnableModuleLogging
}
$ModColor = "Red"
if ($ModSetting -eq 1) {
 $ModColor = "Green"
}
Write-Host " - PowerShell Module Logging: $($ModSetting) (Required = 1)" -ForegroundColor $ModColor

4. Audit Transcription Setup
$TransPath = "HKLM:\Software\Policies\Microsoft\Windows\PowerShell\Transcription"
$TransVal = Get-ItemProperty -Path $TransPath -Name "EnableTranscripting" -ErrorAction SilentlyContinue
$TransSetting = 0
if ($TransVal) {
 $TransSetting = $TransVal.EnableTranscripting
}
$TransColor = "Red"
if ($TransSetting -eq 1) {
 $TransColor = "Green"
}
Write-Host " - PowerShell Transcription Enabled: $($TransSetting) (Required = 1)" -ForegroundColor $TransColor

$TransDirVal = Get-ItemProperty -Path $TransPath -Name "OutputDirectory" -ErrorAction SilentlyContinue
$TransDir = ""
if ($TransDirVal) {
 $TransDir = $TransDirVal.OutputDirectory
}
}
```

```

$DirColor = if ($TransDir) { "Green" } else { "Red" }
Write-Host " - PowerShell Transcription Directory: $($TransDir)" -ForegroundColor $DirColor

5. Audit Transcript Folder Security Permissions
if ($TransDir) {
 if (-not (Test-Path $TransDir)) {
 Write-Host " - Transcription directory does not exist locally." -ForegroundColor Red
 } else {
 $Acl = Get-Acl -Path $TransDir
 $Rules = $Acl.Access
 $HasUnsafeRead = $false
 $HasWriteOnly = $false

 foreach ($Rule in $Rules) {
 $Identity = $Rule.IdentityReference.Value
 $Rights = $Rule.FileSystemRights
 $Type = $Rule.AccessControlType

 if ($Type -eq "Allow" -and ($Identity -like "*Authenticated Users" -or $Identity -like "*Users" -or $Identity -like "*Everyone")) {
 # Look for read access
 $UnsafeRights = @("ReadData", "ListDirectory", "FullControl", "Modify", "Delete", "ReadAndExecute", "Read")
 foreach ($Right in $UnsafeRights) {
 if ($Rights.ToString() -match $Right) {
 $HasUnsafeRead = $true
 }
 }

 # Check for write-only parameters
 if ($Rights.ToString() -match "CreateFiles" -and $Rights.ToString() -match "AppendData" -and -not ($Rights.ToString() -match "ReadData")) {
 $HasWriteOnly = $true
 }
 }
 }

 $AclColor = "Red"
 if ($HasWriteOnly -and -not $HasUnsafeRead) {
 $AclColor = "Green"
 }
 Write-Host " - Transcript Folder Security: WriteOnly=$($HasWriteOnly), UnsafeReadAccess=$($HasUnsafeRead)" -ForegroundColor $AclColor
 }
}

```

## Sources & Compliance References \* \*\*ANSI AD Hardening Guide\*\*: Recommendation R50 (PowerShell logging configuration) \* \*\*CIS Benchmark\*\*: CIS Windows Server 2016 Benchmark v2.0.0 - Section 18.8 (PowerShell Logging) \* \*\*Microsoft Security Baseline Focus\*\*: Windows Administrative Templates (PowerShell Script Block Logging / Transcription)

# [REQ-LOG-003] Deploy and Harden Microsoft Sysmon

## Target Scope \* \*\*Applicable Systems\*\* : Domain Controllers, Member Servers, Tier 2 Client Workstations. \* \*\*Operating Systems\*\* : Windows Server 2016 (and above), Windows 10/11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\* : Medium \* \*\*GPO Path / Registry Location\*\* : \* \*\*Service Configurations\*\* : Local service parameters managed via startup script or scheduled task. \* \*\*Registry Location\*\* : `HKLM\SYSTEM\CurrentControlSet\Services\Sysmon` and `HKLM\SYSTEM\CurrentControlSet\Services\SysmonDrv`

---

## Rationale Windows Security event logs lack detailed telemetry on low-level operating system actions, such as process memory reads (e.g., LSASS dumping via Mimikatz), thread injection, network connections associated with process IDs, and driver loading. Microsoft Sysmon (System Monitor) bridges this gap by writing rich system monitoring telemetry to the `Microsoft-Windows-Sysmon/Operational` event log channel.

Because Sysmon is a critical detection source, adversaries actively target it by attempting to unload its filter driver ( `sysmon -u` or `fltmc unload SysmonDrv` ) or stopping/disabling the Sysmon service ( `sc stop Sysmon` ).

Hardening Sysmon involves:

1. **Service Recovery Configuration**: Forcing the operating system to automatically restart the Sysmon service on failure or termination.
2. **Telemetry Filtering**: Deploying a hardened, security-focused XML configuration template that filters out noise while logging process creation (Event ID 1), remote threads (Event ID 8), LSASS memory access (Event ID 10), and suspicious file drops (Event ID 11).

---

## Legacy Impact & Compatibility \* \*\*Driver Performance\*\* : Sysmon loads a kernel-mode filter driver (`SysmonDrv`). This driver must be tested in staging environments on Domain Controllers and high-load servers to ensure compatibility and that it does not introduce performance bottlenecks. \* \*\*Log Rotation\*\* : Sysmon logs generate substantial volume. The `Microsoft-Windows-Sysmon/Operational` event log channel must be sized to at least 1GB on domain controllers and critical systems to prevent loss of telemetry due to wrap-around log overwriting.

---

## Implementation Steps

### Option A: Installation and Service Configuration

#### 1. Install Sysmon with Hardened XML Base Configuration Deploy Sysmon using the command line with a local XML configuration file.



Save the following template as `sysmon-config.xml` in a secure administrative path:

```
<Sysmon schemaversion="4.50">
 <HashAlgorithms>md5,sha256,imphash</HashAlgorithms>
 <EventFiltering>
 <!-- Rule Group: Process Creation (Event ID 1) -->
 <RuleGroup name="Process Creation" groupRelation="or">
 <ProcessCreate onmatch="exclude" />
 </RuleGroup>

 <!-- Rule Group: Network Connections (Event ID 3) -->
 <RuleGroup name="Network Connection" groupRelation="or">
 <NetworkConnect onmatch="include">
 <DestinationPort condition="is">22</DestinationPort>
 <DestinationPort condition="is">3389</DestinationPort>
 <DestinationPort condition="is">445</DestinationPort>
 <DestinationPort condition="is">5985</DestinationPort>
 <DestinationPort condition="is">5986</DestinationPort>
 </NetworkConnect>
 </RuleGroup>

 <!-- Rule Group: Driver Load (Event ID 6) -->
 <RuleGroup name="Driver Load" groupRelation="or">
 <DriverLoad onmatch="exclude" />
 </RuleGroup>

 <!-- Rule Group: CreateRemoteThread (Event ID 8) -->
 <RuleGroup name="CreateRemoteThread" groupRelation="or">
 <CreateRemoteThread onmatch="include">
 <TargetImage condition="end with">lsass.exe</TargetImage>
 <TargetImage condition="end with">spoolsv.exe</TargetImage>
 </CreateRemoteThread>
 </RuleGroup>

 <!-- Rule Group: Process Access (Event ID 10) -->
 <RuleGroup name="Process Access" groupRelation="or">
 <ProcessAccess onmatch="include">
 <TargetImage condition="end with">lsass.exe</TargetImage>
 </ProcessAccess>
 </RuleGroup>

 <!-- Rule Group: File Creation (Event ID 11) -->
 <RuleGroup name="File Creation" groupRelation="or">
 <FileCreate onmatch="include">
 <TargetFilename condition="contains">\Startup\</TargetFilename>
 <TargetFilename condition="end with">.exe</TargetFilename>
 <TargetFilename condition="end with">.ps1</TargetFilename>
 <TargetFilename condition="end with">.bat</TargetFilename>
 </FileCreate>
 </RuleGroup>
 </EventFiltering>
</Sysmon>
```

Run the installer via administrative command line:

```
Sysmon64.exe -i sysmon-config.xml -accepteula
```

#### 2. Enforce Service Recovery settings Configure the Windows Service Control Manager to automatically restart the Sysmon service if it is terminated: ``cmd sc.exe failure Sysmon actions= restart/60000/restart/60000/restart/60000 reset= 86400 `` \*(Note: Ensure there is a space after the `=` sign for both parameters).\*

#### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to deploy/configure Sysmon and verify its operational state.

[Download Script: Set-SysmonHardening.ps1](#)

```
Set-SysmonHardening.ps1
Configures Sysmon service recovery settings.

Write-Host "--- Hardening Sysmon Service Recovery Settings ---" -ForegroundColor Cyan

1. Ensure Sysmon Service is installed and configured
$SysmonService = Get-Service -Name "Sysmon" -ErrorAction SilentlyContinue
if (-not $SysmonService) {
 Write-Warning "Sysmon service is not currently installed. Run the Sysmon installer first."
 exit 1
}

2. Configure Service Failure Recovery options via sc.exe
Write-Host "[+] Configuring service failure recovery actions for Sysmon..." -ForegroundColor Gray
$ScArgs = "failure Sysmon actions= restart/60000/restart/60000/restart/60000 reset= 86400"
$Process = Start-Process sc.exe -ArgumentList $ScArgs -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Sysmon service recovery actions successfully set to auto-restart." -ForegroundColor Green
} else {
 Write-Error " Failed to set service recovery settings. Exit Code: $($Process.ExitCode)"
}
}
```

To verify the settings have been applied:

[Download Script: Test-SysmonHardening.ps1](#)

```
Test-SysmonHardening.ps1
Audits Sysmon service, driver execution, and recovery actions.

Write-Host "--- Auditing Sysmon Hardening State ---" -ForegroundColor Cyan

1. Verify Sysmon Service status
$SysmonService = Get-Service -Name "Sysmon" -ErrorAction SilentlyContinue
$ServiceStatus = "Stopped"
if ($SysmonService) {
 $ServiceStatus = $SysmonService.Status
}
$ServiceColor = if ($ServiceStatus -eq "Running") { "Green" } else { "Red" }
Write-Host " - Sysmon Service Status: $($ServiceStatus) (Required = Running)" -ForegroundColor $ServiceColor

2. Verify Sysmon Filter Driver (SysmonDrv)
$DriverRunning = $false
$DriverCheck = fltmc.exe filters
foreach ($Line in $DriverCheck) {
 if ($Line -match "SysmonDrv") {
 $DriverRunning = $true
 }
}
$DriverColor = if ($DriverRunning) { "Green" } else { "Red" }
Write-Host " - Sysmon Kernel Driver Loaded: $($DriverRunning) (Required = True)" -ForegroundColor $DriverColor

3. Verify Service Failure Recovery Options
$FailureInfo = sc.exe qfailure Sysmon
$HasReset = $false
$HasRestart = $false
foreach ($Line in $FailureInfo) {
 if ($Line -match "RESET_PERIOD\s+:\s+86400") {
 $HasReset = $true
 }
 if ($Line -match "FAILURE_ACTIONS\s+:\s+RESTART") {
 $HasRestart = $true
 }
}

$RecoveryColor = if ($HasReset -and $HasRestart) { "Green" } else { "Red" }
Write-Host " - Recovery Configuration: ResetConfigured=$(($HasReset)), RestartActionsConfigured=$(($HasRestart))" -ForegroundColor $RecoveryColor
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R52 (Use of Sysmon and host log analysis) \* \*\*CIS Benchmark\*\*: Recommended baseline practice for advanced security monitoring \* \*\*Microsoft Security Guidance\*\*: Sysmon configuration best practices

## # [REQ-LOG-004] Configure Secure SIEM Log Shipping

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers, Tier 2 Client Workstations. \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Configuration Files\*\*: - Winlogbeat: `%ProgramFiles%\Winlogbeat\winlogbeat.yml` - Wazuh Agent: `%ProgramFiles(x86)%\ossec-agent\ossec.conf` \* \*\*System Services\*\*: Winlogbeat, WazuhSvc

## Rationale In high-security, isolated environments, local log storage is vulnerable to tampering. Attackers who obtain elevated privileges will attempt to clear or modify the Security Event Logs (e.g., via `wevtutil cl Security`) to destroy evidence of their activities. Shipping event logs in real-time to a dedicated offline SIEM (such as an ELK Stack or Wazuh Manager) ensures that forensic logs are preserved. To prevent adversaries from intercepting, redirecting, or tampering with log telemetry, log shippers must be hardened:

1. **Secure Transportation (TLS):** Enforce TLS 1.2 or TLS 1.3 encryption with strict verification of the server certificate authority. This prevents man-in-the-middle attacks where an adversary redirects logs to a rogue listener.
2. **Buffer and Queue Management:** Limit memory and disk spool queues for the shipping agents. If the SIEM receiver goes offline during maintenance or network failure, the agents must cache logs safely without causing memory leaks, high CPU overhead, or local disk exhaustion.
3. **Hardened Configuration Files:** Agent configurations contain hostnames, ports, and potentially credentials or internal CA paths. Restricting access to these configuration files prevents standard users from discovering SIEM endpoints or tampering with configuration parameters.

## Legacy Impact & Compatibility \* \*\*PKI Dependencies\*\*: Configuring full verification of the SSL/TLS certificate chain requires that the internal Active Directory Certificate Services (AD CS) CA certificate is distributed to all shipping agents and that the SIEM receiver possesses a certificate signed by this CA. \* \*\*Network Overhead\*\*: Encrypting and transmitting large volumes of security event logs introduces minor CPU and network utilization. Network links, especially in isolated subnets, must be monitored.

## ## Implementation Steps

## ### Option A: Agent Configuration

#### 1. Secure Winlogbeat Configuration Edit `winlogbeat.yml` (located by default under `%ProgramFiles%\Winlogbeat\winlogbeat.yml`) to enforce TLS 1.2/1.3, configure local queue limits, and forward the key log channels:

```
winlogbeat.event_logs:
 - name: Security
 - name: System
 - name: Microsoft-Windows-Sysmon/Operational
 - name: Microsoft-Windows-PowerShell/Operational

Enforce disk-assisted memory queue limits to prevent memory exhaustion
queue.mem:
 events: 4096
 flush.min_events: 2048
 flush.timeout: 1s

Output to Logstash/SIEM with TLS settings
output.logstash:
 hosts: ["local-logstash.internal.local:5044"]
 ssl.supported_protocols: [TLSv1.2, TLSv1.3]
 ssl.verification_mode: full
 ssl.certificate_authorities: ["C:\\ProgramData\\Winlogbeat\\certs\\ca.crt"]
 ssl.certificate: "C:\\ProgramData\\Winlogbeat\\certs\\client.crt"
 ssl.key: "C:\\ProgramData\\Winlogbeat\\certs\\client.key"
```

#### 2. Secure Wazuh Agent Configuration Edit `ossec.conf` (located by default under `%ProgramFiles(x86)%\ossec-agent\ossec.conf`) to

secure log verification and specify localized channels:

```
<ossec_config>
<client>
 <server>
 <address>local-wazuh.internal.local</address>
 <port>1514</port>
 <protocol>tcp</protocol>
 </server>
 <crypto_method>aes</crypto_method>
 <!-- Configure enrollment with validation -->
 <enrollment>
 <enabled>yes</enabled>
 <server_address>local-wazuh.internal.local</server_address>
 <port>1515</port>
 <ssl_cipher>HIGH</ssl_cipher>
 <ssl_verify_host>yes</ssl_verify_host>
 <ssl_cacert>C:\Program Files (x86)\ossec-agent\certs\wpk_root.pem</ssl_cacert>
 </enrollment>
</client>

<!-- Channels to Monitor -->
<localfile>
 <location>Security</location>
 <log_format>eventlog</log_format>
</localfile>
<localfile>
 <location>System</location>
 <log_format>eventlog</log_format>
</localfile>
<localfile>
 <location>Microsoft-Windows-Sysmon/Operational</location>
 <log_format>eventlog</log_format>
</localfile>
<localfile>
 <location>Microsoft-Windows-PowerShell/Operational</location>
 <log_format>eventlog</log_format>
</localfile>
</ossec_config>
```

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to secure agent configuration files and verify SIEM shipping services status.

[Download Script: Set-SiemLogShipping.ps1](#)

```

Set-SiemLogShipping.ps1
Secures Winlogbeat and Wazuh log shipping configuration file ACLs.

Write-Host "--- Hardening SIEM Shipping Agent Configurations ---" -ForegroundColor Cyan

$ConfigFiles = @(
 "C:\Program Files\Winlogbeat\winlogbeat.yml",
 "C:\Program Files (x86)\ossec-agent\ossec.conf"
)

foreach ($File in $ConfigFiles) {
 if (Test-Path $File) {
 Write-Host "[+] Applying hardened NTFS permissions to $($File)..." -ForegroundColor Gray

 # Get ACL
 $Acl = Get-Acl -Path $File
 # Disable inheritance and copy existing rules
 $Acl.SetAccessRuleProtection($true, $true)
 Set-Acl -Path $File -AclObject $Acl

 # Refresh ACL
 $Acl = Get-Acl -Path $File
 $Rules = $Acl.Access

 # Remove any access rules for Users, Authenticated Users, Everyone
 foreach ($Rule in $Rules) {
 $Identity = $Rule.IdentityReference.Value
 if ($Identity -like "*Users" -or $Identity -like "*Authenticated Users" -or $Identity -like "*Everyone") {
 $Acl.RemoveAccessRule($Rule) | Out-Null
 }
 }

 # Explicitly ensure Administrators and SYSTEM have Full Control
 $FullRights = [System.Security.AccessControl.FileSystemRights]::FullControl
 $InheritanceFlags = [System.Security.AccessControl.InheritanceFlags]::None
 $PropagationFlags = [System.Security.AccessControl.PropagationFlags]::None
 $AccessType = [System.Security.AccessControl.AccessControlType]::Allow

 $AdminRule = New-Object System.Security.AccessControl.FileSystemAccessRule("BUILTIN\Administrators", $FullRights, $InheritanceFlags, $PropagationFlags, $AccessType)
 $SystemRule = New-Object System.Security.AccessControl.FileSystemAccessRule("NT AUTHORITY\SYSTEM", $FullRights, $InheritanceFlags, $PropagationFlags, $AccessType)

 $Acl.AddAccessRule($AdminRule)
 $Acl.AddAccessRule($SystemRule)

 Set-Acl -Path $File -AclObject $Acl
 Write-Host " Permissions successfully secured for $($File)." -ForegroundColor Green
 } else {
 Write-Verbose " File $($File) not found, skipping."
 }
}

```

*To verify the settings have been applied:*

[Download Script: Test-SiemLogShipping.ps1](#)

```
Test-SiemLogShipping.ps1
Audits SIEM shipping agents, configuration permissions, and security.

Write-Host "--- Auditing SIEM Log Shipping Agents ---" -ForegroundColor Cyan

1. Audit Agent Services
$Services = @("winlogbeat", "WazuhSvc")
foreach ($SvcName in $Services) {
 $Svc = Get-Service -Name $SvcName -ErrorAction SilentlyContinue
 $Status = "Not Installed"
 if ($Svc) {
 $Status = $Svc.Status
 }
 $Color = if ($Status -eq "Running") { "Green" } else { "Yellow" }
 Write-Host " - Agent Service '$($SvcName)': $($Status)" -ForegroundColor $Color
}

2. Audit Config File Access Permissions
$ConfigFiles = @(
 "C:\Program Files\Winlogbeat\winlogbeat.yml",
 "C:\Program Files (x86)\ossec-agent\ossec.conf"
)

foreach ($File in $ConfigFiles) {
 if (Test-Path $File) {
 $Acl = Get-Acl -Path $File
 $Rules = $Acl.Access
 $HasUnsafeAccess = $false

 foreach ($Rule in $Rules) {
 $Identity = $Rule.IdentityReference.Value
 $Type = $Rule.AccessControlType

 # Verify if users, authenticated users, or everyone has read/write
 if ($Type -eq "Allow" -and ($Identity -like "*Users" -or $Identity -like "*Authenticated Users" -or $Identity -like "*Everyone")) {
 $HasUnsafeAccess = $true
 }
 }

 $FileColor = if (-not $HasUnsafeAccess) { "Green" } else { "Red" }
 Write-Host " - Configuration File: $($File) | UnsafeAccessAllowed=$($HasUnsafeAccess)" -ForegroundColor $FileColor
 } else {
 Write-Host " - Configuration File: $($File) | Status: NOT FOUND" -ForegroundColor Yellow
 }
}
}
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R52 (Sysmon and log shipping recommendations) \*  
 \*\*CIS Benchmark\*\*: Recommended settings for centralized log shipping configurations \* \*\*Wazuh Security Hardening Guidelines\*\*:  
 Transport encryption and configuration protection

# [REQ-LOG-005] Configure Kerberoasting Honeypots and SIEM Detection Rules

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: Active Directory Object Management (Account attributes: `servicePrincipalName`, `adminCount`, `LogonWorkstations`)

## Rationale Kerberoasting allows an authenticated user to request a Kerberos service ticket (TGS) for any service account mapped to a Service Principal Name (SPN). Because the ticket is encrypted using the service account's password hash, the attacker can extract the encrypted ticket from memory and attempt to crack the password offline using brute-force dictionaries or GPU arrays.

To mitigate and detect this vector, two strategies must be implemented:

1. **Service Account Honeypots (Decoy Accounts):** By creating a fake Active Directory user account and registering a decoy SPN on it, security teams establish a high-fidelity trap. Since this decoy account is not tied to any legitimate application or service, no standard user or system has any reason to request a Kerberos ticket for it.
  - Set the `adminCount` attribute to `1` so the account appears in attacker search queries searching for high-privilege targets (e.g., Domain Admins).
  - Any ticket request (Event ID 4769) for the decoy SPN is a definitive indicator of a Kerberoasting attempt, providing a zero-false-positive alert with the attacker's client IP.
2. **SIEM Filtering and Correlation:** Standard Event ID 4769 logging generates millions of events daily. Filtering this telemetry down to anomalous behavior is necessary to catch broad Kerberoasting scans:
  - **Ticket Encryption Type:** Focus on encryption type `0x17` (RC4-HMAC-MD5) or legacy DES (`0x1`, `0x2`, `0x3`) as modern Windows environments negotiate AES (`0x11` or `0x12`) by default.
  - **Account Exclusions:** Filter out usernames ending with `$` (which represent computer accounts, trusts, or managed service accounts that feature automatically rotated, high-entropy passwords).
  - **Anomalous Patterns:** Trigger alerts when a single user account requests RC4 or DES tickets for multiple distinct SPNs within a short timeframe (e.g., less than 5 seconds).

## Legacy Impact & Compatibility \* \*\*Disabled Interactive Logon\*\*: The honeypot account is a decoy and must never be allowed to log on to any system. Enforcing a logon workstation restriction to a non-existent host (e.g., `HONEYPOT-VOID-HOST`) ensures the account cannot be actively abused for lateral movement or authentication even if the password is leaked or cracked. \* \*\*TGS Ticket Issuance\*\*: Active Directory KDC will still issue service tickets for a user account even when logon workstation restrictions are active, ensuring the security audit event (Event ID 4769) is generated for the SIEM to alert on.

## ## Implementation Steps

### ### Option A: Active Directory Users and Computers (GUI)

##### 1. Create the Decoy Account 1. Log on to a Domain Controller or administrative workstation with Domain Admin privileges. 2. Open **Active Directory Users and Computers** (`dsa.msc`). 3. Create a new User Object (e.g., named `krbtgt\_honey`). 4. Set a strong, complex 120-character password. 5. In the account options: - Ensure **Account is enabled** is checked. - Check **Password never expires**. 6. Open the user properties and navigate to the **Account** tab: - Click **Log On To...** under Logon Workstations. - Select **The following computers** and enter a non-existent hostname (e.g., `HONEYPOT-VOID-HOST`). - Click **Add** and then **OK**.

##### 2. Configure administrative attributes 1. In Active Directory Users and Computers, select **View** → **Advanced Features** to enable the Attribute Editor. 2. Open the properties of the decoy account and navigate to the **Attribute Editor** tab. 3. Locate the `adminCount` attribute and double-click it. 4. Set the value to `1` and click **OK**.

##### 3. Register the Decoy SPN 1. Open an elevated command prompt on the Domain Controller. 2. Register a unique fake SPN using the `setspn` tool: ``cmd setspn -s MSSQLSvc/sql-backup-prod.domain.local:1433 krbtgt\_honey ``

### ### Option B: PowerShell & Active Directory cmdlets

Use this method to automatically deploy the honeypot configuration and audit its compliance status.

[Download Script: New-KerberoastHoneyPot.ps1](#)



```
New-KerberoastHoneyPot.ps1
Description: Configures a decoy Kerberoasting HoneyPot account with a fake SPN, AdminCount=1, and restricted logon capabilities.
Target Engine: Windows PowerShell 5.1

[System.Diagnostics.CodeAnalysis.SuppressMessageAttribute("PSAvoidUsingConvertToSecureStringWithPlainText", "")]
param()

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Configure Kerberoasting HoneyPot..." -ForegroundColor Cyan

$HoneyPotName = "krbtgt_honey"
$HoneyPotSPN = "MSSQLSvc/sql-backup-prod.domain.local:1433"
$HoneyPotDescription = "Decoy service account for backup database monitoring."

1. Check if the honeyPot user account already exists
$ExistingUser = Get-ADUser -Filter "SamAccountName -eq '$HoneyPotName'"

if (-not $ExistingUser) {
 # Generate a complex 120-character password to prevent cracking
 $Characters = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#%&*()-_+= "
 $RandomPassword = ""
 for ($i = 0; $i -lt 120; $i++) {
 $Index = Get-Random -Minimum 0 -Maximum $Characters.Length
 $RandomPassword += $Characters[$Index]
 }

 $SecurePassword = ConvertTo-SecureString $RandomPassword -AsPlainText -Force

 # Create the AD User.
 # Set LogonWorkstations to a non-existent host to prevent logon attempts.
 # UPN domain suffix should match the root domain.
 $Domain = Get-ADDomain
 New-ADUser -Name $HoneyPotName `
 -SamAccountName $HoneyPotName `
 -UserPrincipalName "$HoneyPotName@$($Domain.DNSRoot)" `
 -AccountPassword $SecurePassword `
 -Enabled $true `
 -Description $HoneyPotDescription `
 -LogonWorkstations "HONEYPOT-VOID-HOST" `
 -PasswordNeverExpires $true

 Write-Host "[+] Decoy user account '$HoneyPotName' created with logon restrictions." -ForegroundColor Green
} else {
 Write-Host "[*] Decoy user account '$HoneyPotName' already exists." -ForegroundColor Yellow
}

2. Configure target attributes: AdminCount, ServicePrincipalName
$UserObj = Get-ADUser -Identity $HoneyPotName -Properties servicePrincipalName, adminCount

Set AdminCount to 1 (highly attractive to scanners)
if ($UserObj.adminCount -ne 1) {
 Set-ADUser -Identity $HoneyPotName -Replace @{adminCount = 1}
 Write-Host "[+] Set AdminCount to 1 on '$HoneyPotName'." -ForegroundColor Green
} else {
 Write-Host "[*] AdminCount is already set to 1." -ForegroundColor Yellow
}

Set Service Principal Name
$SPNExists = $UserObj.servicePrincipalName | Where-Object { $_ -eq $HoneyPotSPN }
if (-not $SPNExists) {
 Set-ADUser -Identity $HoneyPotName -Add @{servicePrincipalName = $HoneyPotSPN}
 Write-Host "[+] Service Principal Name '$HoneyPotSPN' registered on '$HoneyPotName'." -ForegroundColor Green
} else {
 Write-Host "[*] Service Principal Name '$HoneyPotSPN' already registered." -ForegroundColor Yellow
}

Write-Host "HoneyPot configuration applied successfully." -ForegroundColor Green
```

To verify the honeyPot configuration state:

[Download Script: Get-KerberoastHoneyPotStatus.ps1](#)

```
Get-KerberoastHoneypotStatus.ps1
Description: Audits the existence, SPN, AdminCount, and logon restrictions of the Kerberoasting honeypot account.
Target Engine: Windows PowerShell 5.1

Import-Module ActiveDirectory

Write-Host "--- Auditing Kerberoasting Honeypot Configuration ---" -ForegroundColor Cyan

$HoneypotName = "krbtgt_honey"
$HoneypotSPN = "MSSQLSvc/sql-backup-prod.domain.local:1433"

1. Retrieve the honeypot account
$User = Get-ADUser -Filter "SamAccountName -eq '$HoneypotName'" -Properties servicePrincipalName, adminCount, LogonWorkstations

if (-not $User) {
 Write-Host "[-] Decoy user account '$HoneypotName' does not exist." -ForegroundColor Red
 exit 1
}

$IsCompliant = $true

2. Verify SPN is registered
$SPNExists = $User.servicePrincipalName | Where-Object { $_ -eq $HoneypotSPN }
if ($SPNExists) {
 Write-Host "[+] Decoy SPN '$HoneypotSPN' is registered on '$HoneypotName'." -ForegroundColor Green
} else {
 Write-Host "[-] Decoy SPN '$HoneypotSPN' is NOT registered on '$HoneypotName'." -ForegroundColor Red
 $IsCompliant = $false
}

3. Verify AdminCount is set to 1
if ($User.adminCount -eq 1) {
 Write-Host "[+] AdminCount is set to 1 (deceptive marker active)." -ForegroundColor Green
} else {
 Write-Host "[-] AdminCount is NOT set to 1 on '$HoneypotName'." -ForegroundColor Red
 $IsCompliant = $false
}

4. Verify LogonWorkstations restriction
if ($User.LogonWorkstations -like "*HONEYPOT-VOID-HOST*") {
 Write-Host "[+] Logon restrictions enforced (LogonWorkstations contains 'HONEYPOT-VOID-HOST')." -ForegroundColor Green
} else {
 Write-Host "[-] Logon restrictions NOT enforced on '$HoneypotName' (LogonWorkstations: '$($User.LogonWorkstations)')." -ForegroundColor Red
 $IsCompliant = $false
}

if ($IsCompliant) {
 Write-Host "[+] Secure: Kerberoasting Honeypot is fully configured and active." -ForegroundColor Green
 exit 0
} else {
 Write-Host "[-] Non-Compliant: Kerberoasting Honeypot configurations are incomplete." -ForegroundColor Red
 exit 1
}
```

## ## SIEM Alerting and Query Configurations

Use these reference detection logic configurations to alert on the honeypot and correlate standard ticket request events.

### ### 1. Honeypot Access Alerts (High Severity - Zero False Positives)

#### Splunk Query ``splunk index=security EventCode=4769 ServiceName="sql-backup-prod" | table \_time, TargetUserName, IpAddress, ServiceName, TicketEncryptionType ``

#### Microsoft Sentinel / KQL Query ``kql SecurityEvent | where EventID == 4769 | where ServiceName has "sql-backup-prod" | project TimeGenerated, TargetUserName, IpAddress, ServiceName, TicketEncryptionType ``

### ### 2. Anomalous Kerberoasting Request Activity (Medium Severity)

Detects users requesting a high volume of RC4 service tickets in a short period (potential Kerberoasting scans).

#### Splunk Query ``splunk index=security EventCode=4769 TicketEncryptionType="0x17" NOT (ServiceName="\*\$") | stats dc(ServiceName) as UniqueServicesRequested, values(ServiceName) as ServicesRequested by \_time, TargetUserName, IpAddress | where UniqueServicesRequested > 5 ``

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R48 (Audit policy configuration and anomaly detection) \* \*\*Active Directory Security Guide\*\*: Detecting Kerberoasting Activity and Creating Service Account Honeypots (Sean Metcalf) \* \*\*MITRE ATT&CK\*\*: Technique T1558.003 (Steal or Forge Kerberos Tickets: Kerberoasting)

**# [REQ-LOG-006] Configure SYSVOL Decoy XML Honeypot**

**## Target Scope** \* **\*\*Applicable Systems\*\***: Domain Controllers \* **\*\*Operating Systems\*\***: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows Server 2025

**## Implementation Details** \* **\*\*Priority\*\***: Medium \* **\*\*GPO Path / Registry Location\*\***: File System Auditing (Decoy file path) and Advanced Security Audit Policies (Object Access → Audit File System)

**## Rationale** Adversaries seeking to elevate privileges within an Active Directory domain frequently scan the `SYSVOL` share for files containing legacy Group Policy Preference (GPP) credentials (specifically searching for the `cpassword` attribute in XML files) or startup/login scripts. This discovery scanning is often automated using script search commands (e.g., `findstr /S cpassword`) or administrative diagnostic frameworks (such as PowerSploit or BloodHound).

To detect these unauthorized discovery scans, security teams can deploy a **SYSVOL Decoy XML Honeypot**. This decoy consists of a mock Group Policy folder structure containing a dummy GPP `Groups.xml` file with fake credential properties.

Because this mock policy is not linked to any active Active Directory object, no legitimate system or user account has any reason to query or read this file.

By applying an explicit **NTFS Deny Read** rule to the `Everyone` group on the decoy file, any attempt by an attacker to scan or read it will immediately fail and generate a high-fidelity **Access Denied** event. By configuring file failure auditing on the decoy path, Windows generates **Event ID 4656** or **4663** in the Domain Controller's Security Log. These events capture the attacker's account details and client IP address, providing a low-noise, high-fidelity alert.

**## Legacy Impact & Compatibility** \* **\*\*Auditing Dependency\*\***: This honeypot requires Advanced File System Auditing to be enabled on the Domain Controllers. Ensure that 'Audit File System' is configured for **\*\*Failure\*\*** events under the Domain Controllers' auditing policies ([REQ-LOG-001](#05-logging-monitoring-configure-advanced-audit-policies-md)). \* **\*\*Replication Compatibility\*\***: The decoy folder is located within the `SYSVOL` share. Since it is explicitly denied read access to 'Everyone', legacy replication components (like FRS) or standard file synchronization tools might skip it. This behavior is expected and does not impact domain services since the decoy folder is not an active GPO.

**## Implementation Steps****### Option A: Active Directory Domain Controllers File Explorer (GUI)**

To manually configure the decoy file:

1. Log on to a Domain Controller with Domain Admin privileges.
2. Navigate to the local SYSVOL policies directory (typically `C:\Windows\SYSVOL\sysvol\<domain.local>\Policies`).
3. Create a new folder with a randomly generated GUID format (e.g., `{5B7853A8-1E74-4C56-AC89-E911A34D2E5B}`).
4. Inside this folder, create the directory structure: `Machine\Preferences\Groups`.
5. Create a new file named `Groups.xml` inside `Groups` with standard XML structure and a dummy `cpassword` attribute:

```
<?xml version="1.0" encoding="utf-8"?>
<Groups clsid="{312F64FA-EB90-4b2e-A6AE-E8C1FDD4A2C}">
 <User clsid="{15C200C5-AE9F-4a18-A372-FD51206104C1}" name="BuiltinAdminDecoy" image="0" changed="2026-07-02 20:56:00" uid="{B6396E70-2EA1-46B4-9F6D-E5D3AD3CD2BE}">
 <Properties action="U" newName="LocalAdministrator" changeLogon="0" noChange="1" neverExpires="1" disabled="0" cpassword="j1UyJ/k7S8248c8j838jjSjjSj2jJ29" description="Decoy local admin account for automation services"/>
 </User>
</Groups>
```

6. Configure the permissions to deny read access:
  - Right-click `Groups.xml` and select **Properties**.
  - Navigate to the **Security** tab and click **Advanced**.
  - Click **Add**. Set the Principal to `Everyone`. Set the Type to **Deny**. Check the permissions for **Read & execute** and **Read**. Click **OK**.
7. Configure File System Auditing on the decoy:
  - In the **Advanced Security Settings** window for `Groups.xml`, navigate to the **Auditing** tab.
  - Click **Add**. Set the Principal to `Everyone`. Set the Type to **Fail**. Check the permissions for **Read & execute** and **Read**. Click **OK**.
  - Click **Apply** and then **OK**.

**### Option B: PowerShell & Remediation (Non-GPO / Script-based)**

Use these scripts to deploy and audit the SYSVOL honeypot.

[Download Script: New-SYSVOLHoneypot.ps1](#)

```

New-SYSVOLHoneyPot.ps1
Description: Configures a decoy Group Policy Preferences XML file in SYSVOL with Everyone:Deny read permissions and file access failure auditing.
Target Engine: Windows PowerShell 5.1

Write-Host "Applying hardening requirement: Configure SYSVOL Decoy XML HoneyPot..." -ForegroundColor Cyan

1. Retrieve local SYSVOL path
$SysvolReg = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Netlogon\Parameters" -Name "Sysvol" -ErrorAction SilentlyContinue
if (-not $SysvolReg) {
 Write-Host "[*] SYSVOL registry path not found. Checking standard share path..." -ForegroundColor Yellow
 $SysvolPath = "C:\Windows\SYSVOL\sysvol"
} else {
 $SysvolPath = $SysvolReg.Sysvol
}

if (-not (Test-Path -Path $SysvolPath)) {
 Write-Host "[-] SYSVOL folder not found at path: $SysvolPath. HoneyPot cannot be deployed." -ForegroundColor Red
 exit 1
}

Resolve the active Policies folder path
$PoliciesPath = Get-ChildItem -Path $SysvolPath -Directory | ForEach-Object {
 Join-Path $_.FullName "Policies"
} | Where-Object { Test-Path $_ } | Select-Object -First 1

if (-not $PoliciesPath) {
 Write-Host "[-] GPO Policies folder not found under SYSVOL: $SysvolPath" -ForegroundColor Red
 exit 1
}

2. Check if a decoy is already registered
$RegPath = "HKLM:\SOFTWARE\ADHardening\SYSVOLHoneyPot"
$ExistingGuid = $null
$ExistingPath = $null

if (Test-Path $RegPath) {
 $ExistingGuid = (Get-ItemProperty -Path $RegPath -Name "DecoyGuid" -ErrorAction SilentlyContinue).DecoyGuid
 $ExistingPath = (Get-ItemProperty -Path $RegPath -Name "DecoyPath" -ErrorAction SilentlyContinue).DecoyPath
}

If it exists, verify it
$DeployNew = $true
if ($ExistingGuid -and $ExistingPath -and (Test-Path $ExistingPath)) {
 Write-Host "[*] Decoy GPO already registered in registry with GUID: $ExistingGuid" -ForegroundColor Yellow
 $DeployNew = $false
}

if ($DeployNew) {
 # Generate a new random GUID
 $Guid = [guid]::NewGuid().ToString("B").ToUpper()
 $DecoyGpoPath = Join-Path $PoliciesPath $Guid
 $DecoyGroupsPath = Join-Path $DecoyGpoPath "Machine\Preferences\Groups"
 $DecoyXmlPath = Join-Path $DecoyGroupsPath "Groups.xml"

 Write-Host "[*] Deploying new decoy GPO folder at: $DecoyGpoPath" -ForegroundColor White
 New-Item -ItemType Directory -Path $DecoyGroupsPath -Force | Out-Null

 # Create dummy XML file with decoy cpassword content
 $DecoyXmlContent = @"
<?xml version="1.0" encoding="utf-8">
<Groups clsid="{312F64FA-EB98-4b2e-A6AE-E8C1FCDD4A2C}">
 <User clsid="{15C200C5-AE9F-4a18-A372-FD51206104C1}" name="BuiltinAdminDecoy" image="0" changed="2026-07-02 20:56:00" uid="{B6396E70-2EA1-46B4-9F6D-E5D3AD3CD2BE}">
 <Properties action="U" newName="LocalAdministrator" changeLogon="0" noChange="1" neverExpires="1" disabled="0" cpassword="j1Uyj/k7S8248c8j838jjSjjSj2jJ29" description="Decoy local admin account for automation services"/>
 </User>
</Groups>
"@

 Set-Content -Path $DecoyXmlPath -Value $DecoyXmlContent -Force | Out-Null
 Write-Host "[+] Decoy GPP Groups.xml created." -ForegroundColor Green

 # Save to Registry

```

```

 if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
 }
 Set-ItemProperty -Path $RegPath -Name "DecoyGuid" -Value $Guid -Type String
 Set-ItemProperty -Path $RegPath -Name "DecoyPath" -Value $DecoyXmlPath -Type String
 Write-Host "[+] Registered Decoy Guid: $Guid in HKLM:\SOFTWARE\ADHardening\SYSVOLHoneyPot" -ForegroundColor Gray
} else {
 $DecoyXmlPath = $ExistingPath
}

3. Configure permissions: Deny Everyone Read access
Write-Host "[*] Enforcing Deny Read/Execute permissions for Everyone on decoy file..." -ForegroundColor White
$Acl = Get-Acl -Path $DecoyXmlPath

Check if Deny rule for Everyone already exists to avoid duplication
$HasDenyRule = $false
foreach ($rule in $Acl.GetAccessRules($true, $false, [System.Security.Principal.NTAccount])) {
 if ($rule.IdentityReference.Value -eq "Everyone" -and $rule.AccessControlType -eq [System.Security.AccessControl.AccessControlType]::Deny) {
 $HasDenyRule = $true
 break
 }
}

if (-not $HasDenyRule) {
 $Identity = "Everyone"
 $Rights = [System.Security.AccessControl.FileSystemRights]::ReadAndExecute -bor [System.Security.AccessControl.FileSystemRights]::Read
 $Inheritance = [System.Security.AccessControl.InheritanceFlags]::None
 $Propagation = [System.Security.AccessControl.PropagationFlags]::None
 $Type = [System.Security.AccessControl.AccessControlType]::Deny

 $DenyRule = New-Object System.Security.AccessControl.FileSystemAccessRule($Identity, $Rights, $Inheritance, $Propagation, $Type)
 $Acl.AddAccessRule($DenyRule)
 Set-Acl -Path $DecoyXmlPath -AclObject $Acl
 Write-Host "[+] Applied Deny Everyone rule successfully." -ForegroundColor Green
} else {
 Write-Host "[*] Deny Everyone rule is already present." -ForegroundColor Yellow
}

4. Configure Object Auditing for Failure (SACL)
Write-Host "[*] Configuring Failure Audit rule for Everyone on decoy file..." -ForegroundColor White

To set SACL, we must load the ACL with audit rules
$AclAudit = Get-Acl -Path $DecoyXmlPath -Audit

$HasAuditRule = $false
foreach ($rule in $AclAudit.GetAuditRules($true, $false, [System.Security.Principal.NTAccount])) {
 if ($rule.IdentityReference.Value -eq "Everyone" -and $rule.AuditFlags -eq [System.Security.AccessControl.AuditFlags]::Failure) {
 $HasAuditRule = $true
 break
 }
}

if (-not $HasAuditRule) {
 $AuditIdentity = "Everyone"
 $AuditRights = [System.Security.AccessControl.FileSystemRights]::ReadAndExecute -bor [System.Security.AccessControl.FileSystemRights]::Read
 $AuditInheritance = [System.Security.AccessControl.InheritanceFlags]::None
 $AuditPropagation = [System.Security.AccessControl.PropagationFlags]::None
 $AuditFlags = [System.Security.AccessControl.AuditFlags]::Failure

 $AuditRule = New-Object System.Security.AccessControl.FileSystemAuditRule($AuditIdentity, $AuditRights, $AuditInheritance, $AuditPropagation, $AuditFlags)
 $AclAudit.AddAuditRule($AuditRule)

 # Set the ACL with audit rules back to the file
 Set-Acl -Path $DecoyXmlPath -AclObject $AclAudit
 Write-Host "[+] Applied Failure Audit rule successfully." -ForegroundColor Green
} else {
 Write-Host "[*] Failure Audit rule is already present." -ForegroundColor Yellow
}

Write-Host "SYSVOL Decoy XML HoneyPot configuration completed successfully." -ForegroundColor Green

```

*To verify the honeypot configuration status:*

[Download Script: Get-SYSVOLHoneypotStatus.ps1](#)

```

Get-SYSVOLHoneyPotStatus.ps1
Description: Checks the configuration status of the SYSVOL Decoy XML HoneyPot.
Target Engine: Windows PowerShell 5.1

Write-Host "--- Auditing SYSVOL Decoy XML HoneyPot Configuration ---" -ForegroundColor Cyan
$script:Vulnerable = $false

$RegPath = "HKLM:\SOFTWARE\ADHardening\SYSVOLHoneyPot"

if (Test-Path $RegPath) {
 $DecoyGuid = (Get-ItemProperty -Path $RegPath -Name "DecoyGuid" -ErrorAction SilentlyContinue).DecoyGuid
 $DecoyPath = (Get-ItemProperty -Path $RegPath -Name "DecoyPath" -ErrorAction SilentlyContinue).DecoyPath

 if (-not $DecoyGuid) {
 Write-Host "[-] Decoy GUID is missing in the registry." -ForegroundColor Red
 $script:Vulnerable = $true
 }

 if (-not $DecoyPath) {
 Write-Host "[-] Decoy file path is missing in the registry." -ForegroundColor Red
 $script:Vulnerable = $true
 } else {
 if (-not (Test-Path $DecoyPath)) {
 Write-Host "[-] Decoy XML file does not exist at registered path: $DecoyPath" -ForegroundColor Red
 $script:Vulnerable = $true
 } else {
 Write-Host "[+] Decoy XML file found: $DecoyPath" -ForegroundColor Green

 # Check Deny ACL rule
 $Acl = Get-Acl -Path $DecoyPath
 $HasDenyRule = $false
 foreach ($rule in $Acl.GetAccessRules($true, $false, [System.Security.Principal.NTAccount])) {
 if ($rule.IdentityReference.Value -eq "Everyone" -and $rule.AccessControlType -eq [System.Security.AccessControl.AccessControlType]::Deny) {
 $HasDenyRule = $true
 break
 }
 }

 if ($HasDenyRule) {
 Write-Host " - Everyone Deny Read rule: CONFIGURED" -ForegroundColor White
 } else {
 Write-Host " - Everyone Deny Read rule: NOT CONFIGURED" -ForegroundColor Red
 $script:Vulnerable = $true
 }

 # Check Audit failure rule (SACL)
 $AclAudit = Get-Acl -Path $DecoyPath -Audit
 $HasAuditRule = $false
 foreach ($rule in $AclAudit.GetAuditRules($true, $false, [System.Security.Principal.NTAccount])) {
 if ($rule.IdentityReference.Value -eq "Everyone" -and $rule.AuditFlags -eq [System.Security.AccessControl.AuditFlags]::Failure) {
 $HasAuditRule = $true
 break
 }
 }

 if ($HasAuditRule) {
 Write-Host " - Everyone Failure Audit rule: CONFIGURED" -ForegroundColor White
 } else {
 Write-Host " - Everyone Failure Audit rule: NOT CONFIGURED" -ForegroundColor Red
 $script:Vulnerable = $true
 }
 }
 }
} else {
 Write-Host "[-] Decoy registry key not found under HKLM:\SOFTWARE\ADHardening\SYSVOLHoneyPot" -ForegroundColor Red
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {

```



```
Write-Host "Audit Result: SECURE" -ForegroundColor Green
exit 0
}
```

---

## Sources & Compliance References \* \*\*Sean Metcalf (ADSecurity)\*\*: [Finding Passwords in SYSVOL & Exploiting Group Policy Preferences] (<https://adsecurity.org/?p=2288>) (Honey\_pot detection strategy) \* \*\*ANSSI Active Directory Hardening Guide\*\*<sup>2</sup>: Recommendations on logging and security auditing policies. \* \*\*CIS Controls\*\*<sup>3</sup>: CIS Control 8 - Audit Logs Management; CIS Control 10.5 - Configure host-based firewalls and detection sensors.

## # Module 6: Secure Operations & Maintenance

This directory contains operational procedures and configuration baselines for system backups, offline patch distribution, and regular security auditing.

### ## Technical Hardening Controls

1. [REQ-OPS-001 - Enforce KRBTGT Password Rotation](#) Enforces and audits periodic rotation of the domain KRBTGT account password to prevent Golden Ticket attacks.
2. [REQ-OPS-002 - Enable and Configure the Active Directory Recycle Bin](#) Enables the forest-wide Recycle Bin optional feature to preserve all link-valued attributes and permit rapid recovery of deleted objects.
3. [REQ-OPS-003 - Establish and Maintain Group Policy ADMX Central Store](#) Centralizes ADMX administrative templates within the SYSVOL share to prevent console drift and version mismatches.
4. [REQ-OPS-004 - Implement Third-Party and Custom GPO Templates for COTS Hardening](#) Enforces standardized configuration templates to lock down third-party application configurations (browsers, reader software, security guides).
5. [REQ-OPS-005 - Configure Dedicated WSUS for Tier 0](#) Establishes and secures dedicated WSUS update server endpoints for Tier 0 assets to prevent cross-tier update spoofing.
6. [REQ-OPS-006 - Redirect Default Users and Computers Containers](#) Redirects newly created user and computer objects to dedicated, deletion-protected Organizational Units (OUs) to enforce policy application.
7. [REQ-OPS-007 - Mandate Naming Conventions for GPOs, OUs, and User Accounts](#) Enforces consistent prefixes and structures for directory objects and mandates a standard GPO description template to support programmatic auditing.
8. [REQ-OPS-008 - Configure Daily System State Backups](#) Enforces daily DC System State backups to secure, offline storage to enable clean bare-metal recovery.
9. [REQ-OPS-009 - Implement Offline Patch Management via WSUS](#) Establishes secure, offline wsusutil metadata import/export processes for air-gapped systems.
10. [REQ-OPS-010 - Establish Continuous Security Assessments](#) Implements a scheduled operational program of offline directory reviews using PingCastle, BloodHound/SharpHound, and ORADAD.
11. [REQ-OPS-011 - Enable Detailed BSOD Stop Parameters for Crash Control](#) Enables detailed crash display screens to facilitate local hardware/system troubleshooting in isolated infrastructures.
12. [REQ-OPS-012 - Implement Automated Inactive Computer and User Account Cleanup](#) Disables and moves inactive user (180 days) and computer (90 days) accounts to a stale OU.
13. [REQ-OPS-013 - Clean Up Staged Install From Media \(IFM\) Data](#) Deletes temporary ntds.dit datasets immediately after Domain Controller promotions.

# [REQ-OPS-001] Enforce KRBtgt Password Rotation

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: Active Directory Object Management (krbtgt account object in the Users container: `CN=krbtgt,CN=Users,DC=[Domain]`)

**WARNING:**

The `krbtgt` account password must be rotated periodically to limit the lifespan of potentially compromised Ticket Granting Tickets (TGTs).

- **STANDARD FREQUENCY**

: Reset the password at least every 180 days (semi-annually) in accordance with the DoD STIG requirement.

- **HIGH-SECURITY FREQUENCY**

: High-security baselines (such as the ANSSI Active Directory hardening guide) recommend rotating the password every 40 days in high-security environments.

- **AD-HOC ROTATION**

: Perform an immediate two-step rotation in the event of a suspected Active Directory compromise, or following the departure of key administrative staff with Tier 0 access.

## Rationale The 'krbtgt' account is a built-in local service account that serves as the Key Distribution Center (KDC) service account in Active Directory. The password hash of the 'krbtgt' account is used to sign and encrypt all Kerberos Ticket Granting Tickets (TGTs) issued within the domain.

If an attacker compromises the Active Directory database (e.g., via NTDS.dit extraction) or obtains Domain Admin privileges, they can dump the `krbtgt` password hash and use it to craft forged Kerberos TGTs (a Golden Ticket attack). Golden Tickets allow attackers to authenticate as any user (including Domain Admins) to any service in the forest, bypassing password changes and maintaining persistent, virtually invisible administrative access.

To mitigate the risk of Golden Ticket persistence, the `krbtgt` password must be rotated periodically. Active Directory stores the current password (history index 0) and the immediately previous password (history index 1) for the `krbtgt` account. This design allows existing tickets to remain valid until they expire or are renewed, avoiding service disruptions.

Therefore, a complete rotation requires resetting the password twice. The resets must be separated by a sufficient time window to allow the first reset to replicate across all domain controllers and for existing Kerberos ticket lifetimes (default of 10 hours) to expire.

## Legacy Impact & Compatibility \* \*\*Authentication Outages\*\*: Resetting the 'krbtgt' password twice in rapid succession (without waiting for replication and Kerberos ticket lifetimes to elapse) will invalidate all active TGTs immediately. This will cause widespread authentication failures across the domain for all users, services, and trust relationships. \* \*\*Operational Wait Time\*\*: A minimum of 10 hours (24 hours is recommended in production) must be observed between the first and second reset to allow replication to converge and existing tickets to naturally expire or renew. \* \*\*Trust Relationships\*\*: Forest trusts and external trusts will not be impacted, but the ticket lifetime window must be strictly respected to prevent inter-forest Kerberos authentication issues.

### ## Implementation Steps

#### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

Note: This control cannot be implemented or managed via standard Group Policy Objects (GPO). The KRBtgt password rotation is an operational database task performed directly on the Active Directory objects.

To execute the manual graphical procedure using Active Directory Users and Computers (ADUC):

1. Log on to a Domain Controller or management workstation with Domain Admin credentials.
2. Open **Active Directory Users and Computers** ( `dsa.msc` ).
3. Click the **View** menu at the top and ensure **Advanced Features** is enabled.
4. Navigate to the **Users** container (or the custom Organizational Unit where the `krbtgt` account resides).
5. Locate the **krbtgt** account.
6. Right-click the **krbtgt** account and select **Reset Password**.
7. Enter a strong, randomly generated, complex password (minimum 128 characters) and confirm it. Click **OK**.
8. Verify that Active Directory replication is healthy across all Domain Controllers.
9. Wait at least 10 to 24 hours (allowing the default 10-hour Kerberos ticket lifetime to elapse).
10. Repeat steps 4-7 to perform the second password reset, retiring the compromised key from the password history.

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use the following scripts to audit the age of the `krbtgt` password and programmatically perform a reset.

[Download Script: Reset-KrbtgtPassword.ps1](#)

```
Reset-KrbtgtPassword.ps1
Description: Resets the password of the krbtgt account with a strong, random password.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: KRBTGT Password Rotation..." -ForegroundColor Cyan

1. Retrieve the krbtgt account
$Krbtgt = Get-ADUser -Filter "Name -eq 'krbtgt'" -Properties PasswordLastSet, Enabled
if (-not $Krbtgt) {
 Write-Error "KRBTGT account not found in the Active Directory domain."
 return
}

Write-Host "Current KRBTGT Password Last Set: $($Krbtgt.PasswordLastSet)" -ForegroundColor White

2. Generate a strong random password (128 characters)
$Length = 128
$Chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#%&*()_+=="
$RandomPassword = -join (1..$Length | ForEach-Object { $Chars[(Get-Random -Maximum $Chars.Length)] })

$SecurePassword = New-Object System.Security.SecurityString
foreach ($Char in $RandomPassword.ToCharArray()) {
 $SecurePassword.AppendChar($Char)
}

3. Apply the password change
try {
 Set-ADAccountPassword -Identity $Krbtgt -NewPassword $SecurePassword -Reset -ErrorAction Stop
 Write-Host "[OK] KRBTGT password has been successfully reset." -ForegroundColor Green
 Write-Host "[IMPORTANT] This is a single password reset." -ForegroundColor Yellow
 Write-Host "[IMPORTANT] To fully invalidate old Kerberos tickets (e.g., to recover from Golden Ticket compromise)," -ForegroundColorC
olor Yellow
 Write-Host " you MUST perform a second reset AFTER all domain controllers have replicated the first reset" -Foregroun
dColor Yellow
 Write-Host " and the maximum Kerberos ticket lifetime (default 10 hours) has elapsed." -ForegroundColor Yellow
} catch {
 Write-Error "Failed to reset KRBTGT password: $($_.Exception.Message)"
}
```

To audit the password last set age of the KRBTGT account: [Download Script: Get-KrbtgtRotationStatus.ps1](#)

```
Get-KrbtgtRotationStatus.ps1
Description: Audits the password age of the krbtgt account.

Import-Module ActiveDirectory

Write-Host "--- Auditing KRBTGT Password Rotation Status ---" -ForegroundColor Cyan

$Krbtgt = Get-ADUser -Filter "Name -eq 'krbtgt'" -Properties PasswordLastSet, PasswordExpired, Enabled

if ($Krbtgt) {
 $PasswordLastSet = $Krbtgt.PasswordLastSet
 if ($null -ne $PasswordLastSet) {
 $AgeDays = (New-TimeSpan -Start $PasswordLastSet -End (Get-Date)).Days
 $MaxAgeDays = 180 # STIG requirement threshold

 Write-Host " - Account Name: $($Krbtgt.Name)" -ForegroundColor White
 Write-Host " - Enabled: $($Krbtgt.Enabled)" -ForegroundColor White
 Write-Host " - Password Last Set: $PasswordLastSet ($AgeDays days ago)" -ForegroundColor White

 if ($AgeDays -gt $MaxAgeDays) {
 Write-Host " - Status: WARNING - KRBTGT password has not been rotated in $AgeDays days (Threshold: $MaxAgeDays days)." -ForegroundColor Red
 } else {
 Write-Host " - Status: OK - KRBTGT password was rotated recently ($AgeDays days ago)." -ForegroundColor Green
 }
 } else {
 Write-Host " - Status: WARNING - PasswordLastSet is not set for the krbtgt account." -ForegroundColor Red
 }
} else {
 Write-Error "KRBTGT account not found in the Active Directory domain."
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*\*: Section on secrets renewal (Renouvellement des secrets de l'Active Directory) \* \*\*CIS Benchmark\*\*\*: CIS Microsoft Windows Server Benchmark (General guidance on periodic Kerberos key rotation) \* \*\*DoD STIG\*\*\*: Windows Server Domain Controller STIG (V-205877, V-225006, V-254427) - The krbtgt account password must be reset at least every 180 days. \* \*\*Microsoft Security Guidance\*\*\*: Securing the KRBTGT Account / AD Forest Recovery Guide

# [REQ-OPS-002] Enable and Configure the Active Directory Recycle Bin

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers (Forest-wide configuration) \* \*\*Operating Systems\*\*: Windows Server 2016 and above

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: Active Directory Optional Features Configuration

## Rationale The Active Directory (AD) Recycle Bin is a critical availability and recovery component in an enterprise environment. It addresses risks related to accidental deletions, administrative scripting errors, and malicious insider sabotage targeting directory objects.

Enabling this feature provides several key benefits:

1. **Preserving Object State:** Unlike legacy "tombstone reanimation", the AD Recycle Bin preserves all link-valued attributes (such as group memberships, manager relationships, and direct reports) of deleted objects. When an object is restored, its security context and access privileges are reinstated exactly as they were, preventing security gaps associated with manual reconstruction.
2. **Operational Efficiency:** It allows administrators to restore objects without taking a Domain Controller offline to perform a Directory Services Restore Mode (DSRM) authoritative restore. This reduces downtime and maintains service availability.
3. **Data Lifecycle Alignment:** By explicitly defining the Deleted Object Lifetime (DOL), organizations ensure that deleted items are retained for a designated period before permanent sanitization.

## Legacy Impact & Compatibility \* \*\*Irreversibility\*\*: Enabling the Active Directory Recycle Bin is a permanent forest-wide action that cannot be disabled once activated. \* \*\*Functional Level Pre-requisite\*\*: The Forest Functional Level must be Windows Server 2008 R2 or higher. \* \*\*Database Size\*\*: Enabling the Recycle Bin increases the size of the Active Directory database ('ntds.dit') because deleted objects are not immediately stripped of their attributes. Ensure Domain Controllers have sufficient storage space.

## Implementation Steps

#### Option A: Active Directory Administrative Center (Preferred)

1. Log on to a Domain Controller with **Enterprise Admins** credentials.
2. Open **Active Directory Administrative Center** ( `dsac.exe` ).
3. In the left navigation pane, select the local domain name.
4. In the **Tasks** pane on the right side, click **Enable Recycle Bin...**
5. In the confirmation warning dialog, click **OK**.
6. A notification dialog will state that the change will begin replicating across all Domain Controllers in the forest. Click **OK**.
7. Refresh the Administrative Center interface; the "Enable Recycle Bin..." link will now be grayed out.

#### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts to audit and activate the optional feature forest-wide.

#### 1. Local Audit (Audit-ADRecycleBin.ps1)

[Download Script: Audit-ADRecycleBin.ps1](#)

```
Audit-ADRecycleBin.ps1
Description: Audits the enablement status of the AD Recycle Bin.

Import-Module ActiveDirectory

Write-Host "--- Auditing Active Directory Recycle Bin ---" -ForegroundColor Cyan

try {
 $Forest = Get-ADForest -ErrorAction Stop
 $EnabledFeatures = Get-ADOptionalFeature -Filter "Name -eq 'Recycle Bin Feature'" -Properties EnabledScopes | Select-Object -ExpandProperty EnabledScopes

 if ($EnabledFeatures) {
 Write-Host "`nStatus: Compliant. Active Directory Recycle Bin is enabled in the forest '$($Forest.Name)'." -ForegroundColor Green
 } else {
 Write-Host "`nVULNERABLE: Active Directory Recycle Bin is NOT enabled in the forest '$($Forest.Name)'." -ForegroundColor Red
 }
} catch {
 Write-Host "VULNERABLE: Could not query optional features. Error: $($_.Exception.Message)" -ForegroundColor Red
}
```

## #### 2. Local Remediation (Enable-ADRecycleBin.ps1)

[Download Script: Enable-ADRecycleBin.ps1](#)

```
Enable-ADRecycleBin.ps1
Description: Enables the Active Directory Recycle Bin optional feature forest-wide.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Enable Active Directory Recycle Bin..." -ForegroundColor Cyan

try {
 $Forest = Get-ADForest -ErrorAction Stop
 $RecycleBinFeature = Get-ADOptionalFeature -Filter "Name -eq 'Recycle Bin Feature'" -ErrorAction Stop
 $EnabledFeatures = Get-ADOptionalFeature -Filter "Name -eq 'Recycle Bin Feature'" -Properties EnabledScopes | Select-Object -ExpandProperty EnabledScopes

 if (-not $EnabledFeatures) {
 Write-Host "[+] Enabling Recycle Bin Feature in forest '$($Forest.Name)'..." -ForegroundColor Yellow
 Enable-ADOptionalFeature -Identity $RecycleBinFeature -Scope ForestOrConfigurationSet -Target $Forest.Name -Confirm:$false -ErrorAction Stop
 Write-Host "[+] Active Directory Recycle Bin enabled successfully." -ForegroundColor Green
 } else {
 Write-Host "[+] Active Directory Recycle Bin is already enabled." -ForegroundColor Green
 }
} catch {
 Write-Error "Failed to enable AD Recycle Bin. Error: $($_.Exception.Message)"
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*\*: Section on Active Directory Recycle Bin and backup/restore. \*  
 \*\*Microsoft Best Practices\*\*\*: Active Directory Recycle Bin Step-by-Step Guide.

# [REQ-OPS-003] Establish and Maintain Group Policy ADMX Central Store

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers (SYSVOL Share) \* \*\*Operating Systems\*\*: Windows Server 2016 and above

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: SYSVOL PolicyDefinitions Store  
(`\\SYSVOL\Policies\PolicyDefinitions`)

## Rationale Group Policy Objects (GPOs) rely on XML-based Administrative Template files (`.admx`) and language-specific resource files (`.adml`) to display registry-based policy settings within administrative tools.

By default, the Group Policy Management Editor loads templates from the local computer's `%SystemRoot%\PolicyDefinitions` folder. In an enterprise AD environment, this behavior introduces several security and operational risks:

1. **Configuration Drift:** If different Domain Controllers or management workstations have different template versions installed, editing GPOs can result in missing configurations, corrupted settings, or inadvertent reversion of newer security settings.
2. **Missing Security Controls:** As operating systems evolve, new security controls (such as disabling legacy name resolution or enforcing LSA protection) are introduced in newer templates. Without updated templates, administrators cannot manage these settings via the GPMC GUI.

Establishing the **Central Store** in the SYSVOL share ensures that all administrators edit GPOs using a single, authoritative set of administrative templates.

## Legacy Impact & Compatibility \* \*\*Replication Load\*\*: Placing templates in the SYSVOL share triggers File Replication Service (FRS) or DFS Replication (DFSR) to replicate the files to all Domain Controllers. The store typically contains 10–50 MB of data, which has minimal impact on modern replication networks. \* \*\*Consoles Compatibility\*\*: Older administrative consoles (such as Windows Server 2008 R2) may display errors when opening GPOs edited with newer templates, but the settings themselves will still apply correctly to target machines.

#### ## Implementation Steps

##### #### Option A: Manual Central Store Establishment (Preferred)

1. Log on to a Domain Controller with **Schema Admins** or **Domain Admins** credentials.
2. Open File Explorer and navigate to the local SYSVOL Policies folder: `C:\Windows\SYSVOL\sysvol\<Domain_FQDN>\Policies` (or use the UNC path: `\\localhost\SYSVOL\<Domain_FQDN>\Policies`).
3. Create a new folder named `PolicyDefinitions`.
4. Create language-specific subdirectories inside it based on your environment (e.g. `en-US`).
5. Copy the contents of the local administrative template directory `C:\Windows\PolicyDefinitions` (all `.admx` files) into the newly created `PolicyDefinitions` folder in SYSVOL.
6. Copy the local `.adml` files from `C:\Windows\PolicyDefinitions\en-US` into the `en-US` subfolder in SYSVOL.
7. To update templates in an offline, air-gapped environment, manually transfer the latest Administrative Templates (downloaded as `.msi` packages from Microsoft) to the domain controller, extract them, and copy the new `.admx` and `.adml` files into the SYSVOL Central Store, replacing older versions.

##### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts to audit and initialize the Central Store folder structure.

##### #### 1. Local Audit (Audit-GPOCentralStore.ps1)

[Download Script: Audit-GPOCentralStore.ps1](#)



```
Audit-GPOCentralStore.ps1
Description: Audits the existence of the GPO Central Store in SYSVOL.

Import-Module ActiveDirectory

Write-Host "--- Auditing Group Policy Central Store ---" -ForegroundColor Cyan

try {
 $Domain = Get-ADDomain -ErrorAction Stop
 $CentralStorePath = "\\${$Domain.DNSRoot}\SYSVOL\${$Domain.DNSRoot}\Policies\PolicyDefinitions"

 if (Test-Path -Path $CentralStorePath) {
 Write-Host "`nStatus: Compliant. Group Policy Central Store is established at:" -ForegroundColor Green
 Write-Host " $CentralStorePath" -ForegroundColor White

 $AdmxFiles = Get-ChildItem -Path $CentralStorePath -Filter *.admx
 Write-Host " Found $($AdmxFiles.Count) ADMX templates in the store." -ForegroundColor Green
 } else {
 Write-Host "`nVULNERABLE: Group Policy Central Store does NOT exist. Expected location:" -ForegroundColor Red
 Write-Host " $CentralStorePath" -ForegroundColor Red
 }
} catch {
 Write-Host "VULNERABLE: Could not query Active Directory for SYSVOL path. Error: $($_.Exception.Message)" -ForegroundColor Red
}
```

#### #### 2. Local Remediation (Create-GPOCentralStore.ps1)

[Download Script: Create-GPOCentralStore.ps1](#)

```
Create-GPOCentralStore.ps1
Description: Initializes the Central Store directory structure in SYSVOL.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Initialize GPO Central Store..." -ForegroundColor Cyan

try {
 $Domain = Get-ADDomain -ErrorAction Stop
 $CentralStorePath = "\\${$Domain.DNSRoot}\SYSVOL\${$Domain.DNSRoot}\Policies\PolicyDefinitions"

 if (-not (Test-Path -Path $CentralStorePath)) {
 New-Item -ItemType Directory -Path $CentralStorePath -Force -ErrorAction Stop | Out-Null
 # Create standard language folder
 New-Item -ItemType Directory -Path (Join-Path $CentralStorePath "en-US") -Force -ErrorAction Stop | Out-Null

 Write-Host "[+] Group Policy Central Store initialized successfully." -ForegroundColor Green
 Write-Host " Path: $CentralStorePath" -ForegroundColor White
 Write-Host " Please copy the latest .admx and .adml files to this directory." -ForegroundColor Yellow
 } else {
 Write-Host "[+] Central Store is already initialized at: $CentralStorePath" -ForegroundColor Green
 }
} catch {
 Write-Error "Failed to initialize GPO Central Store. Error: $($_.Exception.Message)"
}
```

## Sources & Compliance References \* \*\*Microsoft Guidance\*\*: How to create and manage the Central Store for Group Policy Administrative Templates. \* \*\*ANSI AD Hardening Guide\*\*: Section on Group Policy template maintenance and update management.

# [REQ-OPS-004] Implement Third-Party and Custom GPO Templates for COTS Hardening

## Target Scope \* \*\*Applicable Systems\*\*: Domain Members (Clients and Servers) \* \*\*Operating Systems\*\*: Windows 10/11, Windows Server 2016 and above

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: SYSVOL PolicyDefinitions Store / Local Administrative Templates

## Rationale Group Policy Objects (GPOs) natively manage core Windows operating system components but lack administrative control definitions for third-party Commercial Off-The-Shelf (COTS) software (such as Google Chrome, Microsoft Edge, Adobe Acrobat Reader) and advanced security guide extensions.

Implementing third-party and custom GPO templates provides the following benefits:

1. **Centralized Configuration:** Administrators can enforce security configurations across all enterprise workstations and member servers (e.g. disabling insecure browser protocols, locking PDF execution properties) directly from the Group Policy Management Console.
2. **Reduced Attack Surface:** Custom templates (such as [Security-ADMX GitHub Repository](#) or the Microsoft Security Guide template) expose hidden or advanced registry configurations, allowing administrators to restrict features like WDigest authentication or LSA credential caching that are not exposed in standard out-of-the-box Windows templates.
3. **Consistency:** Linking COTS hardening GPOs ensures that third-party applications remain compliant with corporate security baselines, preventing local user overrides.

## Legacy Impact & Compatibility \* \*\*Application Interoperability\*\*: Enforcing strict configurations on third-party software (such as blocking legacy TLS, enforcing browser extension whitelists, or restricting PDF javascript execution) may impact business applications or legacy intranet sites. All templates and settings must be fully validated in a testing sandbox prior to production deployment. \* \*\*Template Updates\*\*: As applications update, vendors release newer ADMX template versions. Administrators must periodically update the templates in the Central Store to support newer settings.

## ## Implementation Steps

### #### Option A: Manual Central Store Importing (Preferred)

1. Log on to a management workstation or Domain Controller with **Domain Admins** credentials.
2. Download the official Administrative Templates from the software manufacturer's website (e.g. Microsoft Edge Enterprise templates, Google Chrome templates).
3. Extract the downloaded files to locate the `.admx` files and matching `.adml` language-specific resource files (typically in `en-US` subfolders).
4. Navigate to the Central Store on a Domain Controller: `\\<Domain_FQDN>\SYSVOL\<Domain_FQDN>\Policies\PolicyDefinitions`
5. Copy the `.admx` files into the root of the `PolicyDefinitions` directory.
6. Copy the `.adml` files into the language subfolder matching the language (e.g. `PolicyDefinitions\en-US`).
7. Open the **Group Policy Management Console** ( `gpmc.msc` ) and edit a target hardening GPO. The new settings will appear under **Computer Configuration > Policies > Administrative Templates > [Software Name]**.

### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts to audit and import custom templates.

#### #### 1. Local Audit (Audit-ThirdPartyTemplates.ps1)

[Download Script: Audit-ThirdPartyTemplates.ps1](#)

```
Audit-ThirdPartyTemplates.ps1
Description: Checks the GPO Central Store for common third-party templates.

Import-Module ActiveDirectory

Write-Host "--- Auditing Third-Party GPO Templates ---" -ForegroundColor Cyan

try {
 $Domain = Get-ADDomain -ErrorAction Stop
 $CentralStorePath = "\\${$Domain.DNSRoot}\SYSVOL\${$Domain.DNSRoot}\Policies\PolicyDefinitions"

 if (Test-Path -Path $CentralStorePath) {
 $Templates = @{
 "Microsoft Edge" = "msedge.admx"
 "Google Chrome" = "chrome.admx"
 "Adobe Acrobat" = "Acrobat.admx"
 "MS Security Guide" = "SecGuide.admx"
 }

 Write-Host "`nChecking for common templates in Central Store:" -ForegroundColor Yellow
 foreach ($key in $Templates.Keys) {
 $file = $Templates[$key]
 $fullPath = Join-Path $CentralStorePath $file

 if (Test-Path -Path $fullPath) {
 Write-Host " - [FOUND] $key ($file)" -ForegroundColor Green
 } else {
 Write-Host " - [MISSING] $key ($file)" -ForegroundColor Yellow
 }
 }
 } else {
 Write-Host "VULNERABLE: Group Policy Central Store does not exist. Cannot audit templates." -ForegroundColor Red
 }
} catch {
 Write-Host "VULNERABLE: Could not query Active Directory. Error: $($_.Exception.Message)" -ForegroundColor Red
}
```

#### #### 2. Local Remediation (Import-ThirdPartyTemplate.ps1)

[Download Script: Import-ThirdPartyTemplate.ps1](#)

```
Import-ThirdPartyTemplate.ps1
Description: Copies a specified ADMX and ADML template to the Central Store.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Copy GPO Templates to Central Store..." -ForegroundColor Cyan

Define local source paths for templates (to be populated by administrator)
$SourceAdmx = "C:\SourceTemplates\msedge.admx"
$SourceAdml = "C:\SourceTemplates\en-US\msedge.adml"

try {
 $Domain = Get-ADDomain -ErrorAction Stop
 $CentralStorePath = "\\${$Domain.DNSRoot}\SYSVOL\${$Domain.DNSRoot}\Policies\PolicyDefinitions"

 if (-not (Test-Path -Path $CentralStorePath)) {
 Write-Error "GPO Central Store is not initialized. Please establish the Central Store first."
 exit 1
 }

 if ((Test-Path -Path $SourceAdmx) -and (Test-Path -Path $SourceAdml)) {
 # Copy ADMX file
 Copy-Item -Path $SourceAdmx -Destination $CentralStorePath -Force -ErrorAction Stop
 Write-Host "[+] Copied ADMX: $(Split-Path $SourceAdmx -Leaf) to Central Store." -ForegroundColor Green

 # Copy ADML file to matching subfolder
 $LangDir = Join-Path $CentralStorePath "en-US"
 if (-not (Test-Path -Path $LangDir)) {
 New-Item -ItemType Directory -Path $LangDir -Force -ErrorAction Stop | Out-Null
 }
 Copy-Item -Path $SourceAdml -Destination $LangDir -Force -ErrorAction Stop
 Write-Host "[+] Copied ADML: $(Split-Path $SourceAdml -Leaf) to Central Store en-US subfolder." -ForegroundColor Green
 } else {
 Write-Warning "Source template files not found at specified paths. Please ensure templates are downloaded locally."
 }
} catch {
 Write-Error "Failed to copy template files. Error: $($_.Exception.Message)"
}
```

---

**## Sources & Compliance References** \* **Security-ADMX Project**: Custom GPO templates for hardening Windows client and server environments. Available at the [Security-ADMX GitHub Repository](https://github.com/Harvester57/Security-ADMX). \* **CIS Microsoft Windows Server Benchmarks**: Sections recommending the use of administrative templates to control third-party browser settings. \* **ANSSI AD Hardening Guide**: Recommendations on secure configuration templates for operating systems and software.

# [REQ-OPS-005] Configure Dedicated WSUS for Tier 0

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Tier 0 Administration Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10/11 Enterprise

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Update\Specify intranet Microsoft update service location` \* \*\*Registry Location\*\*: `HKLM:\Software\Policies\Microsoft\Windows\WindowsUpdate`

---

## Rationale The Windows Server Update Services (WSUS) role allows administrators to centralize the approval and distribution of security updates and patches. However, update services execute code with system privileges.

If a shared, mutualized WSUS server (managed by Tier 1 or Tier 2 administrators) is used to patch Tier 0 Domain Controllers:

1. **Lateral Movement Target:** A compromise of the shared WSUS server or its database allows an attacker to inject malicious metadata, forcing Domain Controllers to execute arbitrary code or load compromised updates.
2. **Bypasses Administration Isolation:** Standard network administrators could inadvertently or maliciously deploy payloads to Tier 0 servers.
3. **HTTP Traffic Manipulation:** If WSUS communication is configured over cleartext HTTP (the default port 8530), attackers inside the network can perform man-in-the-middle attacks to inject custom update packages.

To mitigate these threats, Tier 0 Domain Controllers and PAWs must pull updates from a dedicated WSUS server located inside the Tier 0 security boundary, configured exclusively with SSL/TLS encryption.

---

## Legacy Impact & Compatibility \* \*\*Operational Overhead\*\*: Dedicating a separate WSUS server for Tier 0 assets means updates must be reviewed, approved, and synchronized separately from standard enterprise updates. \* \*\*Network Traffic\*\*: Ensure that firewall rules permit the dedicated Tier 0 WSUS server to connect outbound to Microsoft Update servers to synchronize updates. \* \*\*Certificate Management\*\*: Enabling SSL/TLS on WSUS requires generating and trusting a certificate on all Tier 0 client systems (DCs and PAWs).

---

#### ## Implementation Steps

##### #### Option A: Group Policy Object (GPO) Configuration

##### 1. Enforce HTTPS for WSUS in GPO 1. Open **Group Policy Management** (`gpmc.msc`). 2. Create or edit a GPO targeting Tier 0 systems (e.g., `GPO\_Hardening\_DomainControllers`). 3. Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Update` 4. Double-click the **Specify intranet Microsoft update service location** policy. 5. Set it to **Enabled**. 6. Set the intranet update service and status server properties to point to the dedicated, secure Tier 0 WSUS server: \* **Set the intranet update service for detecting updates**: `https://wsust0.corp.local:8531` \* **Set the intranet statistics server**: `https://wsust0.corp.local:8531` \* **Set the alternate download server**: `https://wsust0.corp.local:8531` 7. Click **OK** and link the GPO to the Domain Controllers and PAW OUs.

##### 2. Configure SSL on the WSUS Server On the dedicated Tier 0 WSUS server: 1. Open **IIS Manager** (`inetmgr.exe`). 2. Bind an SSL certificate (issued by a trusted PKI) to port 8531 on the WSUS Administration Web Site. 3. In the middle pane, double-click **SSL Settings** on the virtual directories (`SimpleAuthWebService`, `DSSAuthWebService`, `ClientWebService`, `APIRemoting30`). 4. Check **Require SSL** and click **Apply**. 5. Execute the WSUS configuration command to activate SSL bindings: `C:\Program Files\Update Services\Tools\wsusutil.exe configuressl wsust0.corp.local`

---

##### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script block to apply the dedicated WSUS target server configuration locally via registry parameters.

[Download Script: Set-LocalWsusServer.ps1](#)

```
Set-LocalWsusServer.ps1
Description: Configures the local client registry to utilize the dedicated Tier 0 WSUS over HTTPS.

Write-Host "Applying hardening requirement: Configure Dedicated WSUS for Tier 0..." -ForegroundColor Cyan

$WsusRegPath = "HKLM:\Software\Policies\Microsoft\Windows\WindowsUpdate"
$WsusServerUrl = "https://wsust0.corp.local:8531"

if (-not (Test-Path $WsusRegPath)) {
 New-Item -Path $WsusRegPath -Force | Out-Null
}

1. Configure target WSUS server values
Set-ItemProperty -Path $WsusRegPath -Name "WUServer" -Value $WsusServerUrl -Type String -ErrorAction Stop
Set-ItemProperty -Path $WsusRegPath -Name "WUStatusServer" -Value $WsusServerUrl -Type String -ErrorAction Stop

2. Force Windows Update configuration to use local settings
$UpdateAuPath = "HKLM:\Software\Policies\Microsoft\Windows\WindowsUpdate\AU"
if (-not (Test-Path $UpdateAuPath)) {
 New-Item -Path $UpdateAuPath -Force | Out-Null
}
Set-ItemProperty -Path $UpdateAuPath -Name "UseWUServer" -Value 1 -Type DWord -ErrorAction Stop

Write-Host "[+] Local system configured to use secure WSUS server: $WsusServerUrl" -ForegroundColor Green
```

To verify active WSUS configurations: [Download Script: Get-WsusConfigStatus.ps1](#)

```
Get-WsusConfigStatus.ps1
Description: Audits local WSUS configuration settings.

$WsusRegPath = "HKLM:\Software\Policies\Microsoft\Windows\WindowsUpdate"

Write-Host "Checking Windows Update registry parameters..." -ForegroundColor Cyan

if (Test-Path $WsusRegPath) {
 $WusVal = Get-ItemProperty -Path $WsusRegPath -Name "WUServer" -ErrorAction SilentlyContinue
 if ($null -ne $WusVal) {
 $WusServer = $WusVal.WUServer

 # Check if using HTTPS
 if ($WusServer -like "https://*") {
 Write-Host "[+] WUServer: $WusServer (Secure HTTPS Connection)." -ForegroundColor Green
 } else {
 Write-Host "[-] WUServer: $WusServer (Insecure HTTP Connection - Action Required)." -ForegroundColor Red
 }
 } else {
 Write-Host "[-] WUServer is not configured." -ForegroundColor Yellow
 }
} else {
 Write-Host "[-] Windows Update policies are not defined." -ForegroundColor Yellow
}
```

---

## Sources & Compliance References \* **\*\*ANSI AD Hardening Guide\*\***: Section 3.6.2 (Windows Server Update Services), Section 9 \* **\*\*ANSI Remediation of Active Directory Tier 0 Guide\*\***: Section 3.d (Page 23), Section 7 (Page 46) \* **\*\*Microsoft Security Guidance\*\***: Configure WSUS in a Multi-Tier Environment Securely

# [REQ-OPS-006] Redirect Default Users and Computers Containers

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 and above

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: Active Directory Domain Well-Known Objects

## Rationale By default, new user and computer objects created in an Active Directory domain are placed in the default containers: \*

\*\*Users\*\*: `CN=Users,DC=domain,DC=com` \* \*\*Computers\*\*: `CN=Computers,DC=domain,DC=com`

These default paths present significant security and operational issues:

1. **Lack of GPO Enforcement:** Default containers are generic container objects, not Organizational Units (OUs). Consequently, Group Policy Objects (GPOs) cannot be linked directly to them. Any newly joined computer or newly created user remains unhardened and outside the scope of organizational baseline policies until an administrator manually moves them to an OU.
2. **Insecure Window of Exposure:** The period between the initial domain join/creation and the manual movement of the object creates a window of vulnerability where systems run without mandatory security controls (e.g., endpoint firewall rules, AppLocker, audit configurations, credential protection).
3. **Absence of Deletion Protection:** Default containers do not possess the standard accidental deletion protection attributes that can be applied to OUs, increasing the risk of administrative errors.

Redirecting the default creation paths of users and computers to dedicated, hardened OUs ensures that newly created objects immediately inherit appropriate GPOs and are protected against accidental deletion from the moment of creation.

## Legacy Impact & Compatibility \* \*\*Domain Join Permissions\*\*: Under default AD behavior, authenticated users can join up to 10 computer objects to the domain in the `CN=Computers` container (subject to `ms-DS-MachineAccountQuota`). When `CN=Computers` is redirected to an OU, standard users will lack the permission to create computer objects in that new OU by default. However, since the Machine Account Quota is disabled (`ms-DS-MachineAccountQuota` set to `0` as per [[REQ-ID-017]](/03-identities-services/disable-machine-account-quota.md)), only delegated administrators can join systems, making this change consistent with the overall security model. \* \*\*Legacy Application Mappings\*\*: Some legacy line-of-business applications, provisioning scripts, or identity synchronizers (such as old Entra Connect/Azure AD Connect configurations) might be hardcoded to query or write to `CN=Users` or `CN=Computers`. These systems must be audited and updated to reference the new OUs before enabling redirection. \* \*\*Domain Functional Level\*\*: The Active Directory domain functional level must be Windows Server 2003 or higher to support redirection.

## ## Implementation Steps

### ### Option A: Active Directory Administrative Tools & CLI (Preferred)

1. Log on to a Domain Controller or a management host with **Domain Admins** or **Enterprise Admins** credentials.
2. Open **Active Directory Users and Computers** ( `dsa.msc` ).
3. Create two new Organizational Units (OUs) at the domain root:
  - **Name:** `New-Users`
  - **Name:** `New-Computers`
4. For both OUs, ensure that accidental deletion protection is enabled:
  - Right-click the OU, select **Properties**.
  - Navigate to the **Object** tab (ensure **View → Advanced Features** is enabled in `dsa.msc` to see this tab).
  - Check **Protect object from accidental deletion** and click **OK**.
5. Open an elevated command prompt on a Domain Controller.
6. Run the following command to redirect the default container for new computer objects:

```
redircmp.exe OU=New-Computers,DC=domain,DC=local
```

(Replace `DC=domain,DC=local` with the actual Distinguished Name of your domain).

7. Run the following command to redirect the default container for new user objects:

```
redirusr.exe OU=New-Users,DC=domain,DC=local
```

(Replace `DC=domain,DC=local` with the actual Distinguished Name of your domain).

8. Verify that the output of both commands states: `Redirection was successful.`

---

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts on a Domain Controller or administrative system with remote AD management tools.

#### #### 1. Local Audit

[Download Script: Audit-DefaultContainers.ps1](#)



```
Audit-DefaultContainers.ps1
Description: Audits if the default user and computer containers have been redirected to protected OUs.

Import-Module ActiveDirectory

Write-Host "--- Auditing Default User and Computer Containers Redirection ---" -ForegroundColor Cyan

try {
 $Domain = Get-ADDomain -ErrorAction Stop
 $DomainDN = $Domain.DistinguishedName
 $DefaultComputersDN = "CN=Computers,$($DomainDN)"
 $DefaultUsersDN = "CN=Users,$($DomainDN)"

 $Compliant = $true

 # 1. Check Computers Container
 Write-Host "[+] Current Computers Container: $($Domain.ComputersContainer)" -ForegroundColor Gray
 if ($Domain.ComputersContainer -eq $DefaultComputersDN) {
 Write-Host "VULNERABLE: Default Computers container is NOT redirected." -ForegroundColor Red
 $Compliant = $false
 } else {
 # Verify the redirected container is an OU and is protected from deletion
 $CompOU = Get-ADOrganizationalUnit -Identity $Domain.ComputersContainer -Properties ProtectedFromAccidentalDeletion -ErrorAction SilentlyContinue
 if ($CompOU) {
 if ($CompOU.ProtectedFromAccidentalDeletion) {
 Write-Host "[+] Computers Container redirected to a protected OU (Compliant)." -ForegroundColor Green
 } else {
 Write-Host "VULNERABLE: Computers Container redirected to OU '$($CompOU.Name)' but Accidental Deletion Protection is DISABLED." -ForegroundColor Red
 $Compliant = $false
 }
 } else {
 Write-Host "VULNERABLE: Computers Container is redirected to a non-OU object or the target container does not exist." -ForegroundColor Red
 $Compliant = $false
 }
 }

 # 2. Check Users Container
 Write-Host "[+] Current Users Container: $($Domain.UsersContainer)" -ForegroundColor Gray
 if ($Domain.UsersContainer -eq $DefaultUsersDN) {
 Write-Host "VULNERABLE: Default Users container is NOT redirected." -ForegroundColor Red
 $Compliant = $false
 } else {
 # Verify the redirected container is an OU and is protected from deletion
 $UserOU = Get-ADOrganizationalUnit -Identity $Domain.UsersContainer -Properties ProtectedFromAccidentalDeletion -ErrorAction SilentlyContinue
 if ($UserOU) {
 if ($UserOU.ProtectedFromAccidentalDeletion) {
 Write-Host "[+] Users Container redirected to a protected OU (Compliant)." -ForegroundColor Green
 } else {
 Write-Host "VULNERABLE: Users Container redirected to OU '$($UserOU.Name)' but Accidental Deletion Protection is DISABLED." -ForegroundColor Red
 $Compliant = $false
 }
 } else {
 Write-Host "VULNERABLE: Users Container is redirected to a non-OU object or the target container does not exist." -ForegroundColor Red
 $Compliant = $false
 }
 }

 if ($Compliant) {
 Write-Host "`nStatus: Compliant. User and Computer default containers are redirected to deletion-protected OUs." -ForegroundColor Green
 exit 0
 } else {
 Write-Host "`nStatus: Non-Compliant. Default containers redirection needs to be configured or corrected." -ForegroundColor Red
 exit 1
 }
} catch {
 Write-Host "VULNERABLE: Could not query Active Directory Domain settings. Error: $($_.Exception.Message)" -ForegroundColor Red
}
```

```
 exit 1
}
```

#### 2. Local Remediation

[Download Script: Set-DefaultContainersRedirection.ps1](#)

```
Set-DefaultContainersRedirection.ps1
Description: Creates new OUs, protects them from accidental deletion, and redirects default users/computers containers.

Import-Module ActiveDirectory

Write-Host "Applying hardening requirement: Redirect Default Containers to Protected OUs..." -ForegroundColor Cyan

try {
 $Domain = Get-ADDomain -ErrorAction Stop
 $DomainDN = $Domain.DistinguishedName
 $TargetComputersOU = "OU=New-Computers,$($DomainDN)"
 $TargetUsersOU = "OU=New-Users,$($DomainDN)"

 # 1. Create and protect target Computers OU
 if (-not (Get-ADOrganizationalUnit -Filter "DistinguishedName -eq '$TargetComputersOU'")) {
 Write-Host "[+] Creating target Computers OU: New-Computers" -ForegroundColor Yellow
 New-ADOrganizationalUnit -Name "New-Computers" -Path $DomainDN -ProtectedFromAccidentalDeletion $true -ErrorAction Stop
 Write-Host "[+] Computers OU created and protected successfully." -ForegroundColor Green
 } else {
 Write-Host "[+] Target Computers OU already exists. Ensuring accidental deletion protection is enabled..." -ForegroundColor Yellow
 Set-ADOrganizationalUnit -Identity $TargetComputersOU -ProtectedFromAccidentalDeletion $true -ErrorAction Stop
 Write-Host "[+] Accidental deletion protection verified on target Computers OU." -ForegroundColor Green
 }

 # 2. Create and protect target Users OU
 if (-not (Get-ADOrganizationalUnit -Filter "DistinguishedName -eq '$TargetUsersOU'")) {
 Write-Host "[+] Creating target Users OU: New-Users" -ForegroundColor Yellow
 New-ADOrganizationalUnit -Name "New-Users" -Path $DomainDN -ProtectedFromAccidentalDeletion $true -ErrorAction Stop
 Write-Host "[+] Users OU created and protected successfully." -ForegroundColor Green
 } else {
 Write-Host "[+] Target Users OU already exists. Ensuring accidental deletion protection is enabled..." -ForegroundColor Yellow
 Set-ADOrganizationalUnit -Identity $TargetUsersOU -ProtectedFromAccidentalDeletion $true -ErrorAction Stop
 Write-Host "[+] Accidental deletion protection verified on target Users OU." -ForegroundColor Green
 }

 # 3. Perform Computers Redirection
 if ($Domain.ComputersContainer -ne $TargetComputersOU) {
 Write-Host "[+] Redirecting default Computers container to $TargetComputersOU..." -ForegroundColor Yellow
 $Output = & redircmp.exe $TargetComputersOU 2>&1
 if ($LASTEXITCODE -eq 0) {
 Write-Host "[+] Computers container redirected successfully." -ForegroundColor Green
 } else {
 throw "Failed to redirect Computers container: $($Output)"
 }
 } else {
 Write-Host "[+] Computers container is already redirected to target OU." -ForegroundColor Green
 }

 # 4. Perform Users Redirection
 if ($Domain.UsersContainer -ne $TargetUsersOU) {
 Write-Host "[+] Redirecting default Users container to $TargetUsersOU..." -ForegroundColor Yellow
 $Output = & redirusr.exe $TargetUsersOU 2>&1
 if ($LASTEXITCODE -eq 0) {
 Write-Host "[+] Users container redirected successfully." -ForegroundColor Green
 } else {
 throw "Failed to redirect Users container: $($Output)"
 }
 } else {
 Write-Host "[+] Users container is already redirected to target OU." -ForegroundColor Green
 }

 Write-Host "`n[+] Redirection operations completed successfully." -ForegroundColor Green
} catch {
 Write-Error "Failed to apply redirection. Error: $($_.Exception.Message)"
 exit 1
}
```

## Sources & Compliance References \* \*\*Microsoft Technical Reference\*\*: [Redirecting Users and Computers Containers in Active Directory

Domains](<https://learn.microsoft.com/en-us/troubleshoot/windows-server/active-directory/redirect-users-computers-containers>) \* \*\*ANSI  
AD Hardening Guide\*\*\*: Section on Active Directory structure and logical partitioning.

# [REQ-OPS-007] Mandate Naming Conventions for GPOs, OUs, and User Accounts

## Target Scope \* **Applicable Systems**: Active Directory Domain Services (Logical Structure), Management Stations, Domain Controllers \* **Operating Systems**: Windows Server 2016+, Windows 10 Enterprise, Windows 11 Enterprise

## Implementation Details \* **Priority**: Medium \* **GPO Path / Registry Location**: Active Directory Directory Objects (Logical Policy)

## Rationale Active Directory environments, particularly those with administrative tiering, require strict logical organization to maintain security boundaries and prevent operational errors. The lack of standard naming conventions leads to several security and operational risks: 1.

**Administrative Confusions and Misconfigurations**: Without clear identifiers, administrators might link a highly restrictive Tier 0 GPO to a Tier 2 Client Workstations OU, causing system outages or security bypasses. 2. **Audit and Monitoring Gaps**: Security monitoring tools and SIEM parsers rely on predictable account and resource patterns (such as `a0-` for Tier 0 admin actions) to flag abnormal logons or lateral movement attempts. 3. **Privilege Escalation**: Predictable naming conventions for standard accounts, combined with clear tier prefixes for administrative accounts, prevent users from mistakenly allocating administrative permissions to non-admin accounts. 4. **Configuration Drift**: Group Policy Objects without descriptions or identifiers become "orphaned" or modified by different teams without clear change tracking, leading to undocumented changes that weaken the security posture.

Enforcing structured GPO, OU, and Account naming conventions, combined with a mandatory description template for GPOs, establishes self-documenting metadata that can be programmatically audited to ensure long-term directory integrity.

## Legacy Impact & Compatibility Applying naming conventions is a logical and administrative change, meaning it does not break underlying cryptographic protocols or network services. However: 1. **Renaming Existing Objects**: Renaming existing user accounts, OUs, or GPOs can break existing automation scripts, application binds (LDAP queries using hardcoded DN paths), and GPO links if not updated simultaneously. 2. **Phased Migration**: Rather than renaming existing legacy structures abruptly, organizations must adopt a phased approach: \* Enforce conventions on all new GPOs, OUs, and accounts. \* Run automated audit scripts to inventory non-compliant objects. \* Coordinate scheduled maintenance windows to update and rename legacy objects.

## ## Implementation Steps

### ### Option A: Manual Logical Configuration (GUI)

#### 1. Organizational Unit (OU) Hierarchy Design Establish a clear, tiered OU hierarchy in **Active Directory Users and Computers** (`dsa.msc`). All custom OUs must follow the format: `--` \* **Tier**: 'T0' (Tier 0), 'T1' (Tier 1), 'T2' (Tier 2), or 'Global' (common infrastructure/domain-wide settings). \* **ObjectType**: 'Computers', 'Users', 'Groups', or 'ServiceAccounts'. \* **Function**: Descriptive PascalCase text indicating the target scope.

Examples of compliant OUs:

- `T0-Computers-DomainControllers` (for DCs)
- `T1-Computers-ApplicationServers` (for Tier 1 member servers)
- `T2-Computers-Workstations` (for Tier 2 client computers)
- `T0-Users-Admins` (for Tier 0 administrative users)
- `T1-Users-ServiceAccounts` (for Tier 1 application/service accounts)

#### 2. Group Policy Object (GPO) Naming Scheme Create GPOs in the **Group Policy Management Console** (`gpmc.msc`) using the following naming structure: `GPO\_` \* **Scope**: 'T0' (Tier 0), 'T1' (Tier 1), 'T2' (Tier 2), or 'Global' (domain-wide). \* **Class**: 'Hardening' (security settings), 'Config' (operational settings), 'Restricted' (restricted logons/groups), or 'Software' (installations). \* **Name**: Descriptive PascalCase text indicating policy focus.

Examples of compliant GPOs:

- `GPO_T0_Hardening_DomainControllers`
- `GPO_T2_Config_WorkstationIsolation`
- `GPO_Global_Hardening_DefaultDomain`

#### 3. GPO Description Metadata Template Every GPO must have its **Description** field populated in `gpmc.msc` properties using the

following structured template. This template uses a YAML-compatible layout to support programmatic validation:

```

RequirementID: [ID of corresponding hardening control, e.g., REQ-OPS-007]
Owner: [Administrative team/role responsible, e.g., Domain Admins]
CreatedBy: [Account name of creator, e.g., a0-sysadmin]
CreatedDate: [YYYY-MM-DD]
LastModifiedBy: [Account name of editor, e.g., a0-sysadmin]
LastModifiedDate: [YYYY-MM-DD]
Purpose: [A clear, concise summary of the settings applied and their intent]
ApprovalRef: [Change management ticket or authorization reference, e.g., CR-84920]
Tier: [T0 / T1 / T2 / Global]

```

#### 4. Active Directory Account Naming Scheme Configure and provision accounts using standardized prefixes: \* \*\*Tier 0 Administrative Accounts\*\*: Prefix `a0-` (e.g., `a0-florian`). \* \*\*Tier 1 Administrative Accounts\*\*: Prefix `a1-` (e.g., `a1-florian`). \* \*\*Tier 2 Administrative Accounts\*\*: Prefix `a2-` (e.g., `a2-florian`). \* \*\*Tier 0 Service Accounts / gMSAs\*\*: Prefix `s0-` / `g0-` (e.g., `s0-backup`, `g0-adbackup\$`). \* \*\*Tier 1 Service Accounts / gMSAs\*\*: Prefix `s1-` / `g1-` (e.g., `s1-sql`, `g1-sqlservice\$`). \* \*\*Tier 2 Service Accounts / gMSAs\*\*: Prefix `s2-` / `g2-` (e.g., `s2-print`, `g2-printservice\$`). \* \*\*Emergency (Break-Glass) Accounts\*\*: Prefix `bg-` (e.g., `bg-admin1`).

### Option B: PowerShell & Administrative Scripting (Remediation / Auditing)

#### 1. Configuring GPO Description Metadata Use the following remediation script to programmatically write or update the structured metadata template to a specified GPO's description field.

[Download Script: Configure-GPODescription.ps1](#)

```
Configure-GPODescription.ps1
Description: Configures structured metadata in a GPO's description field.

param (
 [Parameter(Mandatory = $true)]
 [string]$GPOName,

 [Parameter(Mandatory = $true)]
 [string]$RequirementID,

 [Parameter(Mandatory = $true)]
 [string]$Owner,

 [Parameter(Mandatory = $true)]
 [string]$CreatedBy,

 [Parameter(Mandatory = $true)]
 [string]$ApprovalRef,

 [Parameter(Mandatory = $true)]
 [ValidateSet("T0", "T1", "T2", "Global")]
 [string]$Tier,

 [Parameter(Mandatory = $true)]
 [string]$Purpose
)

Import-Module GroupPolicy -ErrorAction SilentlyContinue

Write-Host "--- Configuring Structured GPO Description Metadata ---" -ForegroundColor Cyan

Verify if GPO exists
$gpo = Get-GPO -Name $GPOName -ErrorAction SilentlyContinue
if (-not $gpo) {
 Write-Error "Group Policy Object '$GPOName' was not found in the domain."
 exit 1
}

$CurrentDate = Get-Date -Format "yyyy-MM-dd"

Build YAML-style metadata string
$DescriptionText = @"

RequirementID: $RequirementID
Owner: $Owner
CreatedBy: $CreatedBy
CreatedDate: $CurrentDate
LastModifiedBy: $CreatedBy
LastModifiedDate: $CurrentDate
Purpose: $Purpose
ApprovalRef: $ApprovalRef
Tier: $Tier

"@

try {
 # Set the GPO description
 Set-GPO -Name $GPOName -Description $DescriptionText -ErrorAction Stop
 Write-Host "[+] Successfully configured description metadata for GPO: $GPOName" -ForegroundColor Green
 exit 0
} catch {
 Write-Error "Failed to update description for GPO '$GPOName'. Error: $($_.Exception.Message)"
 exit 1
}
```

To verify directory alignment and find non-compliant OUs, GPOs, and Accounts:

[Download Script: Audit-NamingConventions.ps1](#)

```

Audit-NamingConventions.ps1
Description: Audits GPOs, OUs, and User Accounts for compliance with standard naming conventions and metadata description template
s.

Import-Module ActiveDirectory -ErrorAction SilentlyContinue
Import-Module GroupPolicy -ErrorAction SilentlyContinue

Write-Host "--- Auditing Active Directory Naming Conventions & Metadata ---" -ForegroundColor Cyan

$Compliant = $true

Define Regex Patterns
$gpoNameRegex = "^GPO_(T[0-2]|Global)_(Hardening|Config|Restricted|Software)_[A-Za-z0-9]+$"
$souNameRegex = "^(T[0-2]-(Computers|Users|Groups|ServiceAccounts|Servers|Endpoints|Admins)-[A-Za-z0-9-]+|Domain Controllers|System|
Builtin|ForeignSecurityPrincipals|LostAndFound|NTDS Quotas|Program Data)$"

1. Audit GPO Names and Description Fields
Write-Host "`n[+] Checking Group Policy Objects..." -ForegroundColor Yellow
try {
 $gpos = Get-GPO -All -ErrorAction Stop
 foreach ($gpo in $gpos) {
 $gpoName = $gpo.DisplayName

 # Check Name format
 if ($gpoName -notmatch $gpoNameRegex) {
 Write-Host " [-] NON-COMPLIANT GPO NAME: '$gpoName' (does not match expected structure)" -ForegroundColor Red
 $Compliant = $false
 } else {
 Write-Host " [+] GPO Name OK: '$gpoName'" -ForegroundColor Green
 }

 # Check Description metadata
 $desc = $gpo.Description
 if ([string]::IsNullOrEmpty($desc)) {
 Write-Host " [-] NON-COMPLIANT GPO DESCRIPTION: '$gpoName' (Description field is empty)" -ForegroundColor Red
 $Compliant = $false
 } else {
 # Verify YAML structure has key metadata fields
 $hasReqId = $desc -match "RequirementID:"
 $hasOwner = $desc -match "Owner:"
 $hasTier = $desc -match "Tier:"
 $hasPurpose = $desc -match "Purpose:"

 if ($hasReqId -and $hasOwner -and $hasTier -and $hasPurpose) {
 Write-Host " [+] GPO Description Template OK: '$gpoName'" -ForegroundColor Green
 } else {
 Write-Host " [-] NON-COMPLIANT GPO DESCRIPTION FORMAT: '$gpoName' (Structured fields are missing or malformed)" -Fo
 regroundColor Red
 $Compliant = $false
 }
 }
 }
} catch {
 Write-Warning "Could not retrieve GPOs from domain. Ensure GPMC RSAT tools are installed and domain is reachable."
}

2. Audit Organizational Units
Write-Host "`n[+] Checking Organizational Units (OUs)..." -ForegroundColor Yellow
try {
 $ous = Get-ADOrganizationalUnit -Filter * -ErrorAction Stop
 foreach ($ou in $ous) {
 $ouName = $ou.Name
 if ($ouName -notmatch $souNameRegex) {
 Write-Host " [-] NON-COMPLIANT OU NAME: '$ouName' (DistinguishedName: $($ou.DistinguishedName))" -ForegroundColor Red
 $Compliant = $false
 } else {
 Write-Host " [+] OU Name OK: '$ouName'" -ForegroundColor Green
 }
 }
} catch {
 Write-Warning "Could not retrieve OUs from Active Directory. Ensure AD RSAT tools are installed."
}

3. Audit Account Naming Conventions (Tiered / Service / Emergency)

```



```

Write-Host "`n[+] Checking Account Naming Conventions..." -ForegroundColor Yellow
try {
 # Check Administrative Group memberships to see if administrative accounts are properly prefixed
 $privilegedGroups = @("Domain Admins", "Enterprise Admins", "Schema Admins", "Administrators")
 foreach ($group in $privilegedGroups) {
 $members = Get-ADGroupMember -Identity $group -ErrorAction SilentlyContinue
 foreach ($member in $members) {
 if ($member.objectClass -eq "user") {
 $sam = $member.SamAccountName
 # Tier 0 admins must have 'a0-' or 'bg-' or be standard default Administrator
 if ($sam -ne "Administrator" -and $sam -notlike "a0-*" -and $sam -notlike "bg-*") {
 Write-Host " [-] NON-COMPLIANT TIER 0 ACCOUNT NAME: '$sam' (Member of privileged group: $group, lacks 'a0-' or
'bg-' prefix)" -ForegroundColor Red
 $Compliant = $false
 } else {
 Write-Host " [+] Admin Account Prefix OK: '$sam' ($group)" -ForegroundColor Green
 }
 }
 }
 }
}

Audit Service Accounts and gMSAs
$serviceAccounts = Get-ADServiceAccount -Filter * -ErrorAction SilentlyContinue
foreach ($sa in $serviceAccounts) {
 $sam = $sa.SamAccountName
 if ($sam -notlike "g0-*" -and $sam -notlike "g1-*" -and $sam -notlike "g2-*") {
 Write-Host " [-] NON-COMPLIANT gMSA NAME: '$sam' (lacks 'g0-', 'g1-', or 'g2-' prefix)" -ForegroundColor Red
 $Compliant = $false
 } else {
 Write-Host " [+] gMSA Prefix OK: '$sam'" -ForegroundColor Green
 }
}
} catch {
 Write-Warning "Could not query account or group memberships. Ensure AD RSAT tools are installed and domain is reachable."
}

4. Final Compliance Verdict
if ($Compliant) {
 Write-Host "`nStatus: Compliant. GPOs, OUs, and Accounts conform to standard conventions." -ForegroundColor Green
 exit 0
} else {
 Write-Host "`nStatus: Non-Compliant. Naming conventions drift detected." -ForegroundColor Red
 exit 1
}
}

```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R12 (Naming and description convention of GPOs) \*  
 \*\*CIS Microsoft Windows Server 2016 Benchmark v2.0.0\*\*: Section 1.1.1 (Security policy and resource management) \* \*\*Microsoft Security  
 Best Practices\*\*: Administrative Account Lifecycle and Group Policy Management Strategy

## # [REQ-OPS-008] Configure Daily System State Backups

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: Windows Server Backup feature and local backup policies

## Rationale Disaster Recovery is a core pillar of Active Directory security. If Domain Controllers are corrupted or compromised, administrators must recover from trusted, clean states.

The System State contains the Active Directory database (ntds.dit), the SYSVOL share, registry settings, certificates, and DNS records.

To ensure resilience:

1. **Daily Frequency:** Create System State Backups daily on at least two Domain Controllers to minimize data loss.
2. **Storage Isolation (Offline/Immutable):** Backups must be stored on separate physical or virtual storage. In high-security systems, enforce write-once-read-many (WORM) storage or store backups in an offline, physically secured media rotation to prevent modifications by compromised accounts.
3. **Regular Validation:** Run recovery exercises quarterly in an isolated network sandbox to verify that restored DCs are functional and free of replication loops.

## Legacy Impact & Compatibility \* \*\*Performance Impact\*\*: Creating a system state backup utilizes significant disk I/O and CPU resources. It should be scheduled during off-peak hours to avoid affecting authentication response times. \* \*\*Storage Requirements\*\*: System State backups consume considerable storage space (typically 10-20 GB or more depending on database size). Ensure target volumes have adequate capacity and are configured to automatically prune older backups. \* \*\*Access Control\*\*: Backups contain the Active Directory database (including credential hashes). The backup destination volume and network shares must be strictly restricted: \* For local folders, configure NTFS ACLs to grant access only to 'BUILTIN\Administrators' and 'NT AUTHORITY\SYSTEM'. \* For remote network shares (UNC paths), configure share permissions and NTFS ACLs on the target storage server to restrict read/write access exclusively to Tier 0 Administrators (e.g., Domain Admins, Enterprise Admins) and dedicated backup service accounts, blocking all other domain accounts.

## ## Implementation Steps

## ### Option A: Graphical User Interface (GUI) Configuration

1. Log in to the Domain Controller and open **Server Manager**.
2. Click **Manage** → **Add Roles and Features**.
3. Advance to the **Features** step, check **Windows Server Backup**, and complete the installation.
4. Open the administrative tool **Windows Server Backup** (wbadmin.msc).
5. In the Actions pane, click **Backup Schedule....**
6. In the Backup Schedule Wizard, click **Next** on the Getting Started page.
7. Select **Custom** configuration and click **Next**.
8. Click **Add Items**, check **System State**, and click **OK**.
9. Specify the time and frequency (once a day, during off-peak hours) and click **Next**.
10. Choose the destination type (dedicated backup disk, volume, or shared network folder) and complete the wizard.

## ### Option B: PowerShell &amp; Registry Configuration (Remediation / Non-GPO)

Run the following script block to install the Windows Server Backup feature, configure a System State backup policy, and execute an immediate System State backup to a designated disk volume.

[Download Script: Set-ADSystemStateBackup.ps1](#)

```
Set-ADSystemStateBackup.ps1
Installs Windows Server Backup and executes a System State backup.

Write-Host "--- Initializing System State Backup ---" -ForegroundColor Cyan

1. Install Windows Server Backup feature if missing
$feature = Get-WindowsFeature -Name Windows-Server-Backup
if ($feature.Installed -eq $false) {
 Write-Host "[+] Installing Windows Server Backup feature..." -ForegroundColor Gray
 Install-WindowsFeature -Name Windows-Server-Backup -IncludeAllSubFeature | Out-Null
 Write-Host " Feature installed successfully." -ForegroundColor Green
} else {
 Write-Host "[+] Windows Server Backup feature is already installed." -ForegroundColor Green
}

Import WSB module
Import-Module WindowsServerBackup

2. Define Backup Volume Target
$BackupVolumePath = "E:\" # Replace with your designated offline backup storage disk
if (-not (Test-Path $BackupVolumePath)) {
 Write-Error "Backup target volume '$BackupVolumePath' does not exist. Please specify a valid volume."
 exit 1
}

3. Create Backup Policy
Write-Host "[+] Configuring System State Backup Policy..." -ForegroundColor Gray
$policy = New-WBPolicy
Add-WBSystemState -Policy $policy | Out-Null

$BackupTarget = New-WBBackupTarget -VolumePath $BackupVolumePath
Add-WBBackupTarget -Policy $policy -Target $BackupTarget | Out-Null

Write-Host " Backup Policy created (Target: $BackupVolumePath, Subject: SystemState)." -ForegroundColor Green

4. Execute Backup Job
Write-Host "[+] Starting System State Backup. This process can take several minutes..." -ForegroundColor Yellow
$BackupJob = Start-WBBackup -Policy $policy -Async

Monitor backup job status
while ($BackupJob.State -eq "Running" -or $BackupJob.State -eq "Verifying") {
 Write-Host " - Backup Progress: $($BackupJob.PercentComplete)% complete..." -ForegroundColor Gray
 Start-Sleep -Seconds 10
 # Refresh backup job status
 $BackupJob = Get-WBJob
}

Final output
$finalJob = Get-WBJob -Previous 1
if ($finalJob.JobState -eq "Completed") {
 Write-Host "`nSystem State Backup Completed successfully!" -ForegroundColor Green
} else {
 Write-Error "`nBackup failed with status: $($finalJob.JobState). Error: $($finalJob.ErrorDescription)"
}
}
```

To verify active backup configurations:

[Download Script: Audit-ADBackupStatus.ps1](#)

```
Audit-ADBackupStatus.ps1
Audits the status of local system state backups.

Import-Module WindowsServerBackup -ErrorAction SilentlyContinue

Write-Host "--- Auditing System State Backup Status ---" -ForegroundColor Cyan

Check if Windows Server Backup feature is installed
$feature = Get-WindowsFeature -Name Windows-Server-Backup -ErrorAction SilentlyContinue
if ($feature -and $feature.Installed -eq $false) {
 Write-Warning "Windows Server Backup feature is NOT installed on this machine."
 exit 1
}

Retrieve history of local backups
try {
 $backups = Get-WBBackupSet -ErrorAction Stop
 Write-Host "`n[+] Found $($backups.Count) recorded backup sets." -ForegroundColor Yellow

 # Sort and output the most recent backups
 $sortedBackups = $backups | Sort-Object -Property BackupTime -Descending
 foreach ($bk in $sortedBackups | Select-Object -First 5) {
 $containsSystemState = $bk.CatalogFlags -match "SystemState"
 $statusColor = if ($containsSystemState) { "Green" } else { "Yellow" }
 Write-Host " - Backup Time: $($bk.BackupTime) | Location: $($bk.VolumePath) | Contains SystemState: $containsSystemState"
 -ForegroundColor $statusColor
 }
} catch {
 Write-Host "[-] No backup records found on the system. System state backups may not be configured." -ForegroundColor Red
}

Audit Backup Target Access Control List
$policy = Get-WBPolicy -ErrorAction SilentlyContinue
if ($policy) {
 $targets = Get-WBBackupTarget -Policy $policy -ErrorAction SilentlyContinue
 foreach ($target in $targets) {
 $path = $null
 if ($target.VolumePath) {
 $path = $target.VolumePath
 } elseif ($target.NetworkPath) {
 $path = $target.NetworkPath
 }

 if ($path) {
 Write-Host "`n[*] Auditing backup target permissions: $path" -ForegroundColor Gray
 if (Test-Path $path) {
 $acl = Get-Acl -Path $path -ErrorAction SilentlyContinue
 if ($acl) {
 $unauthorized = $false
 foreach ($access in $acl.Access) {
 $identity = $access.IdentityReference.Value
 $rights = $access.FileSystemRights
 $type = $access.AccessControlType

 if ($type -eq "Allow") {
 if ($identity -notmatch "SYSTEM|Administrators|Domain Admins|Enterprise Admins|Creator Owner|NT AUTHORITY\SYSTEM|BUILTIN\Administrators") {
 if ($rights -match "Read|Write|Modify|FullControl") {
 Write-Host " [!] WARNING: Unauthorized identity '$identity' has '$rights' access to backup target." -ForegroundColor Red
 $unauthorized = $true
 }
 }
 }
 }
 }
 }
 if (-not $unauthorized) {
 Write-Host " [+] Target directory ACL is securely restricted." -ForegroundColor Green
 }
 } else {
 Write-Warning " Could not retrieve ACL for backup target."
 }
 }
} else {
 Write-Host " [-] Backup target path is currently offline or unreachable." -ForegroundColor Yellow
}
}
```

```
 }
 }
}
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R54 (Domain Controller backup and disaster recovery) \* \*\*Microsoft Security Guidance\*\*: Active Directory Backup and Restore Best Practices

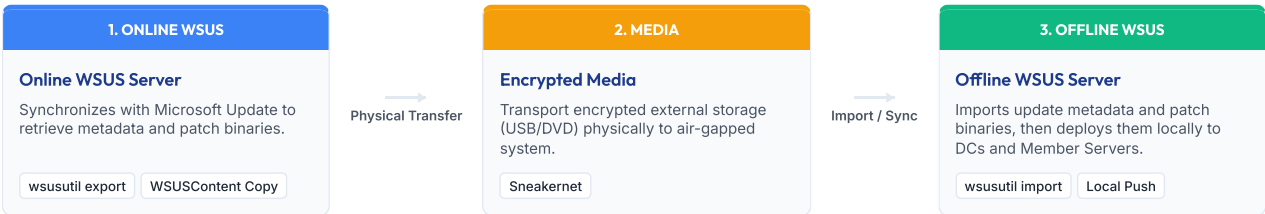
# [REQ-OPS-009] Implement Offline Patch Management via WSUS  
## Target Scope \* \*\*Applicable Systems\*\*: Dedicated WSUS Update Servers (Tier 0 & Tier 1/2) \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: Offline update synchronization and media transfer procedure

## Rationale Keeping Domain Controllers, member servers, and clients patched is critical to resolve OS and RPC vulnerabilities. In isolated, air-gapped networks, direct connection to Microsoft Update servers is impossible, requiring all patches to be imported offline. Establishing an offline WSUS sync protocol:

- 1. **Prevents Network exposure:** Domain Controllers and administrative hosts do not require access to external network zones.
- 2. **Maintains Integrity:** Allows checking update metadata and approvals in a controlled sandbox environment before propagating them to production.
- 3. **Automates Distribution:** Uses standard WSUS client policies to distribute approved updates locally with minimal network overhead.

### WSUS Import/Export Protocol



## Legacy Impact & Compatibility \* \*\*Operational Overhead\*\*: Performing sneakernet synchronization requires regular manual intervention by administrators to export updates on internet-connected networks, perform virus scans, write to encrypted media, and import onto the air-gapped WSUS server. \* \*\*Storage Requirements\*\*: Storing full update files on a WSUS server requires substantial storage space (typically several hundred gigabytes for multiple operating systems and products). \* \*\*Time Drift\*\*: There will be a latency between when patches are released and when they are imported and deployed to air-gapped systems. High-severity patches must be prioritized during scheduled transfer windows.

## Implementation Steps

### WSUS Categories & Classifications Configuration Before performing the import/export process, both the online (source) and offline (destination) WSUS servers must be configured with identical Products and Classifications. If the destination server does not have the corresponding categories enabled, the imported update metadata for those categories will be ignored.

#### 1. Graphical User Interface (GUI) Configuration On both the online and offline WSUS administration consoles: 1. Open the **Windows Server Update Services** console (`wsus.msc`). 2. Expand the server name and click on **Options** → **Products and Classifications**. 3. In the **Products** tab, ensure the exact operating systems present in the environment are selected: **Windows Server 2016**, **Windows Server 2019**, **Windows Server 2022**, **Windows 10**, **Windows 11**. 4. In the **Classifications** tab, ensure the following classifications are selected: **Critical Updates**, **Security Updates**, **Definition Updates** (mandatory if distributing Windows Defender signatures), **Update Rollups** (standard cumulative updates). 5. Click **Apply** and then click **OK**.

#### 2. PowerShell Configuration Run the following PowerShell commands as Administrator on both WSUS servers to align Categories and

Classifications programmatically:

```
Import WSUS module
Import-Module UpdateServices

Write-Host "--- Aligning WSUS Categories & Classifications ---" -ForegroundColor Cyan

Define targets
$TargetClassifications = @("Critical Updates", "Security Updates", "Definition Updates", "Update Rollups")
$TargetProducts = @("Windows Server 2016", "Windows Server 2019", "Windows Server 2022", "Windows 10", "Windows 11")

Enable Classifications
Write-Host "[+] Configuring WSUS Classifications..." -ForegroundColor Gray
Get-WsusClassification | Where-Object { $TargetClassifications -contains $_.Classification.Title } | Set-WsusClassification

Enable Products
Write-Host "[+] Configuring WSUS Products..." -ForegroundColor Gray
Get-WsusProduct | Where-Object { $TargetProducts -contains $_.Product.Title } | Set-WsusProduct

Write-Host "[+] WSUS categories and classifications updated successfully." -ForegroundColor Green
```

---

#### ### Option A: Command-line Execution (wsusutil)

##### 1. On the Internet-Connected WSUS Server 1. Wait for standard synchronization to complete, or force a sync. 2. Open a Command Prompt as Administrator. 3. Export the WSUS metadata file: ``cmd wsusutil.exe export C:\export\metadata.xml.gz C:\export\export.log `` 4. Copy the metadata file ('metadata.xml.gz') and the entire 'WSUSContent' directory (containing the update binaries) onto an encrypted and scanned storage medium.

##### 2. On the Air-Gapped WSUS Server 1. Connect the transfer storage medium and copy the files to a local staging directory (e.g., 'C:\import'). 2. Open a Command Prompt as Administrator. 3. Import the update metadata: ``cmd wsusutil.exe import C:\import\metadata.xml.gz C:\import\import.log `` 4. Copy the update binary files from the storage medium into the WSUS content directory of the air-gapped server.

---

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use the following PowerShell script to programmatically import WSUS metadata from a specified location using the native WSUS utility.

[Download Script: Invoke-OfflineWsusImport.ps1](#)

```
Invoke-OfflineWsusImport.ps1
Description: Imports WSUS update metadata from an offline backup file.

param (
 [Parameter(Mandatory = $true)]
 [string]$MetadataPath,

 [Parameter(Mandatory = $true)]
 [string]$LogPath
)

Write-Host "--- Performing Offline WSUS Import ---" -ForegroundColor Cyan

if (-not (Test-Path $MetadataPath)) {
 Write-Error "Metadata file not found at: $MetadataPath"
 exit 1
}

Run wsusutil import
$WsusUtil = "C:\Program Files\Update Services\Tools\wsusutil.exe"
if (-not (Test-Path $WsusUtil)) {
 Write-Error "wsusutil.exe not found at default location."
 exit 1
}

Write-Host "[+] Running wsusutil import..." -ForegroundColor Yellow
$Process = Start-Process -FilePath $WsusUtil -ArgumentList "import `"$MetadataPath`" `"$LogPath`"" -Wait -NoNewWindow -PassThru

if ($Process.ExitCode -eq 0) {
 Write-Host "[+] Offline WSUS Import completed successfully." -ForegroundColor Green
} else {
 Write-Error "wsusutil import failed with exit code $($Process.ExitCode)."
 exit 1
}
```

*To verify synchronization state of the local WSUS server:*

[Download Script: Get-OfflineWsusSyncStatus.ps1](#)



```
Get-OfflineWsusSyncStatus.ps1
Description: Checks the synchronization history of the local WSUS server.

Import-Module UpdateServices -ErrorAction SilentlyContinue

Write-Host "--- Auditing WSUS Offline Synchronization Status ---" -ForegroundColor Cyan

try {
 # Connect to the local WSUS server
 $wsus = [Microsoft.UpdateServices.Administration.AdminProxy]::GetUpdateServer()
 $history = $wsus.GetSubscription().GetSynchronizationHistory()

 if ($history.Count -gt 0) {
 $lastSync = $history[0]
 Write-Host "[+] Last WSUS Sync Time: $($lastSync.EndTime)" -ForegroundColor Green
 Write-Host "[+] Last WSUS Sync Result: $($lastSync.Result)" -ForegroundColor Green
 if ($lastSync.Result -eq "Succeeded") {
 exit 0
 } else {
 Write-Warning "Last WSUS Sync did not succeed."
 exit 1
 }
 } else {
 Write-Host "[-] No WSUS synchronization history found." -ForegroundColor Red
 exit 1
 }
} catch {
 Write-Host "[-] Failed to retrieve WSUS synchronization history. Ensure the WSUS role is installed and services are running." -ForegroundColor Red
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*\*: Section 3.6.2 (Windows Server Update Services) \* \*\*ANSSI Remediation of Active Directory Tier 0 Guide\*\*\*: Section 7 (Page 46)

# [REQ-OPS-010] Establish Continuous Security Assessments

## Target Scope \* \*\*Applicable Systems\*\*: Active Directory Domain Services, Management Workstations \* \*\*Operating Systems\*\*: Windows Server 2016+, Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: Administrative diagnostic tools configuration and execution plan

## Rationale Active Directory configurations naturally drift over time as a result of administrative changes, new trust relationships, and changing group policies. Administrators must actively search for misconfigurations, weak permissions, and signs of compromise.

In isolated, air-gapped networks, online security analysis services cannot be reached. Therefore:

1. **Periodic Scans:** Execute security audits locally using offline-compatible tools (such as PingCastle, BloodHound/SharpHound, Locksmith, or ORADAD).
2. **Directory Health Monitoring:** Run PingCastle monthly to generate local XML/HTML reports indicating domain vulnerabilities and tracking AD configuration health.
3. **Lateral Movement Auditing:** Execute SharpHound quarterly to construct lateral movement path graphs, enabling defenders to identify complex trust relationships or delegation chains leading to Tier 0 compromise.
4. **Active Directory Certificate Services Auditing:** Run Locksmith monthly to identify misconfigured certificate templates, insecure enrollment settings, and potential AD CS privilege escalation paths.
5. **Data Isolation:** Transfer the diagnostic reports, CSV exports, and export ZIPs out of production to secure, offline assessment platforms to limit exposure of sensitive configuration data.

## Legacy Impact & Compatibility \* \*\*LDAP & Domain Controller Load\*\*: SharpHound queries Active Directory via LDAP and SAMR protocols, which can generate thousands of queries depending on the directory's size. Schedule directory-wide collectors during low-activity windows to avoid latency. \* \*\*Security Alarm Generation\*\*: SharpHound, Locksmith, and PingCastle execution looks like reconnaissance activity and can trigger alerts in local Antivirus or Endpoint Detection and Response (EDR) agents. Administrators must coordinate scans with security operations teams and authorize the diagnostic tools. \* \*\*Data Sensitivity\*\*: BloodHound export ZIPs, PingCastle reports, and Locksmith CSV exports contain highly sensitive structural details of the directory. These files must be stored with strict access controls (Domain Admins only) and purged from administrative workstations after completion.

## Implementation Steps

### Option A: Manual Diagnostic Tool Execution

#### 1. Running PingCastle Audits 1. Place 'PingCastle.exe' in a secure local diagnostic directory (e.g., 'C:\Diagnostics'). 2. Open a Command Prompt as Administrator on a domain-joined workstation. 3. Run the following command to generate the HTML report and XML output: ``cmd PingCastle.exe --server target.domain.local --level level\_Default --xml --no\_update `` 4. Collect the generated files from the directory and transfer them to a secure analysis terminal.

#### 2. Running BloodHound/SharpHound Collectors 1. Place the 'SharpHound.exe' collector binary in the diagnostic directory. 2. Open a Command Prompt as Administrator on a domain-joined workstation. 3. Run the collector using standard active directory enumeration methods: ``cmd SharpHound.exe --CollectionMethods All --Domain target.domain.local --ZipFileName AD\_BloodHound\_Export.zip `` 4. Copy the resulting zip file to the offline BloodHound graph database dashboard to query and audit lateral movement paths.

#### 3. Running Locksmith Audits (AD CS) 1. Download the 'Invoke-Locksmith.ps1' script from the official repository and place it in the diagnostic directory (e.g., 'C:\Diagnostics'). 2. Open PowerShell as Administrator on a domain-joined workstation. 3. Run the script using the CSV export mode (Mode 2) to audit AD CS and save findings: ``powershell Set-Location -Path C:\Diagnostics .\Invoke-Locksmith.ps1 -Mode 2 `` 4. Copy the generated 'ADCSIssues.CSV' report to a secure analysis terminal for offline review.

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use the following PowerShell script to automate the execution of PingCastle, SharpHound, and Locksmith collectors inside a specified diagnostic workspace.

[Download Script: Start-OfflineAssessments.ps1](#)

```
Start-OfflineAssessments.ps1
Description: Triggers PingCastle, SharpHound, and Locksmith security audit scans.

param (
 [string]$DiagnosticsPath = "C:\Diagnostics",
 [string]$DomainName = "target.domain.local"
)

Write-Host "--- Starting AD Hardening Offline Assessment Scans ---" -ForegroundColor Cyan

if (-not (Test-Path $DiagnosticsPath)) {
 New-Item -Path $DiagnosticsPath -ItemType Directory -Force | Out-Null
}

$PingCastlePath = Join-Path $DiagnosticsPath "PingCastle.exe"
$SharpHoundPath = Join-Path $DiagnosticsPath "SharpHound.exe"
$LocksmithPath = Join-Path $DiagnosticsPath "Invoke-Locksmith.ps1"

1. Execute PingCastle
if (Test-Path $PingCastlePath) {
 Write-Host "[+] Executing PingCastle..." -ForegroundColor Yellow
 $params = @(
 "--server", $DomainName,
 "--level", "level_Default",
 "--xml",
 "--no_update",
 "--output", $DiagnosticsPath
)
 Start-Process -FilePath $PingCastlePath -ArgumentList $params -Wait -NoNewWindow
 Write-Host "[+] PingCastle scan complete." -ForegroundColor Green
} else {
 Write-Error "PingCastle.exe not found at $PingCastlePath. Please place the binary to execute."
}

2. Execute SharpHound
if (Test-Path $SharpHoundPath) {
 Write-Host "[+] Executing SharpHound..." -ForegroundColor Yellow
 $zipName = "AD_BloodHound_Export_" + (Get-Date -Format "yyyyMMdd") + ".zip"
 $zipPath = Join-Path $DiagnosticsPath $zipName

 $params = @(
 "--CollectionMethods", "All",
 "--Domain", $DomainName,
 "--ZipFileName", $zipPath
)
 Start-Process -FilePath $SharpHoundPath -ArgumentList $params -Wait -NoNewWindow
 Write-Host "[+] SharpHound collection complete. Saved to: $zipPath" -ForegroundColor Green
} else {
 Write-Error "SharpHound.exe not found at $SharpHoundPath. Please place the binary to execute."
}

3. Execute Locksmith
if (Test-Path $LocksmithPath) {
 Write-Host "[+] Executing Locksmith..." -ForegroundColor Yellow
 $originalLocation = Get-Location
 Set-Location -Path $DiagnosticsPath
 try {
 & $LocksmithPath -Mode 2
 Write-Host (" [+] Locksmith scan complete. Saved to: " + (Join-Path $DiagnosticsPath "ADCSIssues.CSV")) -ForegroundColor Green
 }
 catch {
 Write-Error "Failed to execute Locksmith: $($_.Exception.Message)"
 }
 finally {
 Set-Location -Path $originalLocation
 }
} else {
 Write-Error "Invoke-Locksmith.ps1 not found at $LocksmithPath. Please place the script to execute."
}
}
```

*To verify that diagnostic tools and recent reports exist on the auditing system:*

[Download Script: Get-OfflineAssessmentStatus.ps1](#)

```

Get-OfflineAssessmentStatus.ps1
Description: Audits the presence of PingCastle, SharpHound, and Locksmith tools and the age of recent reports.

Write-Host "--- Auditing Active Directory Security Assessment Tools ---" -ForegroundColor Cyan

$DiagnosticsPath = "C:\Diagnostics" # Common diagnostics path
$PingCastlePath = Join-Path $DiagnosticsPath "PingCastle.exe"
$SharpHoundPath = Join-Path $DiagnosticsPath "SharpHound.exe"
$LocksmithPath = Join-Path $DiagnosticsPath "Invoke-Locksmith.ps1"
$ReportAgeDays = 30

$Compliant = $true

1. Check PingCastle
if (Test-Path $PingCastlePath) {
 Write-Host "[+] PingCastle executable found: $PingCastlePath" -ForegroundColor Green

 # Check if reports have been generated in the last 30 days
 $reports = Get-ChildItem -Path $DiagnosticsPath -Filter "*pingcastle*.xml" -ErrorAction SilentlyContinue | Where-Object { $_.LastWriteTime -ge (Get-Date).AddDays(-$ReportAgeDays) }
 if ($reports) {
 Write-Host " [+] Found $($reports.Count) recent PingCastle report(s) (within last $ReportAgeDays days)." -ForegroundColor Green
 } else {
 Write-Warning " [-] No recent PingCastle report found (older than $ReportAgeDays days)."
 $Compliant = $false
 }
} else {
 Write-Warning "[-] PingCastle executable NOT found at: $PingCastlePath"
 $Compliant = $false
}

2. Check SharpHound
if (Test-Path $SharpHoundPath) {
 Write-Host "[+] SharpHound executable found: $SharpHoundPath" -ForegroundColor Green

 # Check if data exports exist
 $exports = Get-ChildItem -Path $DiagnosticsPath -Filter "*BloodHound*.zip" -ErrorAction SilentlyContinue | Where-Object { $_.LastWriteTime -ge (Get-Date).AddDays(-$ReportAgeDays) }
 if ($exports) {
 Write-Host " [+] Found $($exports.Count) recent SharpHound export(s) (within last $ReportAgeDays days)." -ForegroundColor Green
 } else {
 Write-Warning " [-] No recent SharpHound export found (older than $ReportAgeDays days)."
 $Compliant = $false
 }
} else {
 Write-Warning "[-] SharpHound executable NOT found at: $SharpHoundPath"
 $Compliant = $false
}

3. Check Locksmith
if (Test-Path $LocksmithPath) {
 Write-Host "[+] Locksmith script found: $LocksmithPath" -ForegroundColor Green

 # Check if Locksmith CSV output exists and was generated in the last 30 days
 $locksmithReport = Get-ChildItem -Path $DiagnosticsPath -Filter "ADCSIssues.CSV" -ErrorAction SilentlyContinue | Where-Object { $_.LastWriteTime -ge (Get-Date).AddDays(-$ReportAgeDays) }
 if ($locksmithReport) {
 Write-Host " [+] Found recent Locksmith report (within last $ReportAgeDays days)." -ForegroundColor Green
 } else {
 Write-Warning " [-] No recent Locksmith report (ADCSIssues.CSV) found (older than $ReportAgeDays days)."
 $Compliant = $false
 }
} else {
 Write-Warning "[-] Locksmith script (Invoke-Locksmith.ps1) NOT found at: $LocksmithPath"
 $Compliant = $false
}

if ($Compliant) {
 Write-Host "`nStatus: Compliant. Diagnostics tools and recent reports are present." -ForegroundColor Green
 exit 0
} else {
 Write-Host "`nStatus: Non-Compliant. Action required." -ForegroundColor Red
}

```

```
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSI AD Hardening Guide\*\*: Recommendation R57 (Perform continuous security assessments and audits) \* \*\*CIS Microsoft Windows Server 2016 Benchmark v2.0.0\*\*: Section 18.9 (Administrative tools and logging controls) \* \*\*Locksmith Repository\*\*: [Trimarc Locksmith AD CS Auditing Tool](<https://github.com/jakehildreth/Locksmith>)

# [REQ-OPS-011] Enable Detailed BSOD Stop Parameters for Crash Control

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers, Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022, Windows 10, Windows 11 Enterprise

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Path\*\*:  
'HKLM\SYSTEM\CurrentControlSet\Control\CrashControl' \* \*\*Registry Value\*\*: 'DisplayParameters' (REG\_DWORD = '1')

## Rationale During critical system failures (such as a kernel panic or Blue Screen of Death - BSOD), Windows by default displays a simplified error screen intended for general consumers, hiding the actual bugcheck code and parameters.

Enabling detailed stop error parameters is crucial because:

1. **Offline Diagnostics:** In air-gapped, isolated environments, administrators cannot easily query online resources, transmit automated memory dumps, or contact cloud support.
2. **Immediate Visibility:** Having the exact stop code (e.g., `0x0000000A`) and the four parameters visible on the screen or virtual console allows operators to diagnose driver, memory, or hardware issues immediately.
3. **Improves Mean Time to Recovery (MTTR):** Speeds up troubleshooting during disaster recovery or critical server restore processes.

## Legacy Impact & Compatibility \* \*\*User Experience\*\*: The screen display layout changes slightly when a crash occurs to show standard troubleshooting parameters. There is no impact on active system performance, running services, or application operations. \*

\* \*\*Troubleshooting dependencies\*\*: Enabling this value does not affect the creation of memory dump files (minidumps or full dumps), which remain governed by other CrashControl parameters.

### ## Implementation Steps

#### ### Option A: Group Policy Object (GPO) Configuration

Because this parameter resides in the system registry and is not exposed as a default administrative template, it must be deployed via Group Policy Preferences (GPP):

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a management station.
2. Create a new GPO targeting all domain computers (e.g., `GPO_Global_Config_SystemHardening` ) or edit an existing policy.
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Right-click the **Registry** node and select **New** → **Registry Item**.
5. Configure the following properties:
  - **Action:** Update
  - **Hive:** `HKEY_LOCAL_MACHINE`
  - **Key Path:** `SYSTEM\CurrentControlSet\Control\CrashControl`
  - **Value Name:** `DisplayParameters`
  - **Value Type:** `REG_DWORD`
  - **Value Data:** `1`
6. Click **OK**.
7. Link the GPO to the top-level OU structure containing computers, servers, and Domain Controllers.

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script block to enable detailed stop error parameters locally in the registry.

[Download Script: Set-CrashControl.ps1](#)

```
Set-CrashControl.ps1
Description: Enables detailed BSOD parameters in the registry.

Write-Host "--- Configuring Detailed BSOD Parameters ---" -ForegroundColor Cyan

$Path = "HKLM:\SYSTEM\CurrentControlSet\Control\CrashControl"
if (-not (Test-Path $Path)) {
 New-Item -Path $Path -Force | Out-Null
}

Set-ItemProperty -Path $Path -Name "DisplayParameters" -Value 1 -Type DWord -Force | Out-Null
Write-Host "[+] Detailed BSOD stop parameters configured successfully." -ForegroundColor Green
```

To verify that detailed stop parameters are enabled:

[Download Script: Audit-CrashControl.ps1](#)

```
Audit-CrashControl.ps1
Description: Audits whether detailed BSOD parameters are enabled in the registry.

Write-Host "--- Auditing Detailed BSOD Parameters ---" -ForegroundColor Cyan

$Path = "HKLM:\SYSTEM\CurrentControlSet\Control\CrashControl"
if (Test-Path $Path) {
 $Val = Get-ItemProperty -Path $Path -Name "DisplayParameters" -ErrorAction SilentlyContinue
 if ($Val -and $Val.DisplayParameters -eq 1) {
 Write-Host "[+] Detailed BSOD stop parameters are ENABLED (Compliant)." -ForegroundColor Green
 } else {
 Write-Host "[-] Detailed BSOD stop parameters are DISABLED (Non-Compliant)." -ForegroundColor Red
 exit 1
 }
} else {
 Write-Host "[-] Registry path HKLM:\SYSTEM\CurrentControlSet\Control\CrashControl not found." -ForegroundColor Red
 exit 1
}
```

---

## Sources & Compliance References \* \*\*Microsoft Security Guidance\*\*: Windows CrashControl Registry Reference \* \*\*ANSSI AD Hardening Guide\*\*: Section 9 (Operations & Troubleshooting guidance)



# [REQ-OPS-012] Implement Automated Inactive Computer and User Account Cleanup

## Target Scope \* \*\*Applicable Systems\*\*: Active Directory User and Computer Accounts (Tiers 0, 1, and 2). \* \*\*Operating Systems\*\*: Active Directory Domain Services (All supported functional levels).

## Implementation Details \* \*\*Priority\*\*: Medium (Operational mitigation against lateral movement and backdoor persistence). \* \*\*GPO Path / Registry Location\*\*: N/A (Implemented via an automated PowerShell maintenance script scheduled to run periodically on a management host).

## Rationale Inactive computer and user accounts remain in the directory due to gaps in the employee offboarding or machine decommissioning processes.

These stale accounts represent a significant security risk:

1. **Backdoor Persistence:** Attackers targeting a domain can take control of inactive accounts (especially stale administrative accounts or service accounts with never-expiring passwords) to establish persistent, quiet access that is rarely monitored.
2. **Resource-Based Delegation Exploits:** Stale computer accounts can be targeted by attackers to construct resource-based constrained delegation (RBCD) attacks, allowing them to impersonate high-privilege services and eventually compromise the domain.
3. **Password Aging bypass:** Standard computers automatically change their passwords every 30 days. If a machine is powered off or disconnected, its password age increases. If computer passwords are not rotated, or if a stale computer account remains enabled, it increases the risk of offline password dumping and hash-relay.

Automatically disabling and isolating user accounts after 180 days of inactivity, and computer accounts after 90 days of inactivity, minimizes the active attack surface of the directory.

## Legacy Impact & Compatibility \* \*\*Temporary Absence\*\*: Employees on long-term leave (e.g., parental or medical leave) may have their accounts disabled automatically. Standard procedures must exist for the service desk to quickly re-enable accounts upon validation. \* \*\*Lab and Staging Systems\*\*: Test or staging servers that are powered down for long periods might have their computer accounts disabled. These accounts should be placed in excluded Organizational Units (OUs) or excluded via the script parameters.

#### ## Implementation Steps

##### #### Option A: Scheduled Maintenance Task Setup

1. Place the decommissioning script ( `Decommission-InactiveAccounts.ps1` ) in a secure administrative directory on a domain management host (e.g., `C:\ADMaintenance\Scripts\` ).
2. Open **Task Scheduler** ( `taskschd.msc` ) on the management host.
3. Create a new task with the following properties:
  - **Account:** Run as a dedicated service account or `NT AUTHORITY\SYSTEM` (with delegated rights to modify user and computer accounts in target OUs).
  - **Trigger:** Weekly (e.g., every Sunday at 02:00 AM).
  - **Action:** Start a program:
    - **Program/script:** `powershell.exe`
    - **Arguments:** `-NoProfile -ExecutionPolicy Bypass -File "C:\ADMaintenance\Scripts\Decommission-InactiveAccounts.ps1" -LogPath "C:\ADMaintenance\Logs\"`
4. Enable logging and monitor execution logs to verify that accounts are correctly identified, disabled, and moved to the Stale OU.

##### #### Option B: PowerShell & Active Directory Configuration (Remediation / Non-GPO)

To automate the identification, disabling, and isolation of inactive accounts:

[Download Script: Decommission-InactiveAccounts.ps1](#)

```
Decommission-InactiveAccounts.ps1
Description: Disables and moves inactive user (180 days) and computer (90 days) accounts to a stale OU.

Import-Module ActiveDirectory

Define thresholds
$UserInactivityDays = 180
$ComputerInactivityDays = 90

$UserCutoffDate = (Get-Date).AddDays(-$UserInactivityDays)
$ComputerCutoffDate = (Get-Date).AddDays(-$ComputerInactivityDays)

Target OU for stale objects (adjust to your environment)
$StaleOU = "OU=StaleObjects,DC=domain,DC=local"
$Exclusions = @("Domain Controllers") # Exclude OUs containing DCs

if (-not (Get-ADOrganizationalUnit -Identity $StaleOU -ErrorAction SilentlyContinue)) {
 Write-Host "[] Stale OU '$StaleOU' does not exist. Creating it." -ForegroundColor Yellow
 New-ADOrganizationalUnit -Name "StaleObjects" -Path (Get-ADDomain).DistinguishedName -Verbose
}

Write-Host "Scanning for inactive user accounts (no logon in last $UserInactivityDays days)..." -ForegroundColor Cyan
$StaleUsers = Get-ADUser -Filter {Enabled -eq $true -and LastLogonDate -lt $UserCutoffDate -and Name -ne "Administrator" -and Name -ne "Guest"} -Properties LastLogonDate

foreach ($user in $StaleUsers) {
 # Verify the user is not in excluded paths
 $isExcluded = $false
 foreach ($ex in $Exclusions) {
 if ($user.DistinguishedName -like "*$ex*") { $isExcluded = $true }
 }
 if ($isExcluded) { continue }

 Write-Host "Disabling and moving inactive user: $($user.SamAccountName) (Last Logon: $($user.LastLogonDate))" -ForegroundColor Yellow
 Set-ADUser -Identity $user -Enabled $false -Description "Disabled by AD Decommissioning Script - Inactive for $UserInactivityDays days"
 Move-ADObject -Identity $user -TargetPath $StaleOU
}

Write-Host "`nScanning for inactive computer accounts (no logon in last $ComputerInactivityDays days)..." -ForegroundColor Cyan
$StaleComputers = Get-ADComputer -Filter {Enabled -eq $true -and LastLogonDate -lt $ComputerCutoffDate} -Properties LastLogonDate

foreach ($comp in $StaleComputers) {
 $isExcluded = $false
 foreach ($ex in $Exclusions) {
 if ($comp.DistinguishedName -like "*$ex*") { $isExcluded = $true }
 }
 if ($isExcluded) { continue }

 Write-Host "Disabling and moving inactive computer: $($comp.Name) (Last Logon: $($comp.LastLogonDate))" -ForegroundColor Yellow
 Set-ADComputer -Identity $comp -Enabled $false -Description "Disabled by AD Decommissioning Script - Inactive for $ComputerInactivityDays days"
 Move-ADObject -Identity $comp -TargetPath $StaleOU
}

Write-Host "`nDecommissioning process completed." -ForegroundColor Green
```

To audit the domain for stale user/computer accounts:

[Download Script: Get-InactiveAccountsStatus.ps1](#)

```
Get-InactiveAccountsStatus.ps1
Audits the directory for enabled but inactive user (180 days) and computer (90 days) accounts.

Import-Module ActiveDirectory

$UserInactivityDays = 180
$ComputerInactivityDays = 90

$UserCutoffDate = (Get-Date).AddDays(-$UserInactivityDays)
$ComputerCutoffDate = (Get-Date).AddDays(-$ComputerInactivityDays)

$StaleUsers = Get-ADUser -Filter {Enabled -eq $true -and LastLogonDate -lt $UserCutoffDate -and Name -ne "Administrator" -and Name -ne "Guest"} -Properties LastLogonDate
$StaleComputers = Get-ADComputer -Filter {Enabled -eq $true -and LastLogonDate -lt $ComputerCutoffDate} -Properties LastLogonDate

$Exclusions = @("Domain Controllers")

$nonCompliantCount = 0

foreach ($user in $StaleUsers) {
 $isExcluded = $false
 foreach ($ex in $Exclusions) {
 if ($user.DistinguishedName -like "*$ex*") { $isExcluded = $true }
 }
 if ($isExcluded) { continue }

 Write-Host "[!] NON-COMPLIANT: User account '$($user.SamAccountName)' is enabled but inactive since $($user.LastLogonDate)" -ForegroundColor Red
 $nonCompliantCount++
}

foreach ($comp in $StaleComputers) {
 $isExcluded = $false
 foreach ($ex in $Exclusions) {
 if ($comp.DistinguishedName -like "*$ex*") { $isExcluded = $true }
 }
 if ($isExcluded) { continue }

 Write-Host "[!] NON-COMPLIANT: Computer account '$($comp.Name)' is enabled but inactive since $($comp.LastLogonDate)" -ForegroundColor Red
 $nonCompliantCount++
}

if ($nonCompliantCount -eq 0) {
 Write-Host "[+] COMPLIANT: No stale enabled user or computer accounts detected." -ForegroundColor Green
 exit 0
} else {
 Write-Host "[!] NON-COMPLIANT: Detected $nonCompliantCount stale enabled accounts that need to be decommissioned." -ForegroundColor Red
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*\*: Recommendation R45 (Auditing and disabling stale user/computer accounts) \* \*\*CIS Benchmark\*\*\*: Section 5.1 (User Account Control and password expirations) \* \*\*PingCastle Rules\*\*\*: \* `S-C-Inactive` (Inactive computer check) \* `P-Inactive` (Check for inactive administrator accounts) \* `S-Inactive` (Inactive account check) \* `S-PwdLastSet-90` (Check if all computers have changed their passwords in the last 3 months)

## # [REQ-OPS-013] Clean Up Staged Install From Media (IFM) Data

## Target Scope \* \*\*Applicable Systems\*\*: Domain Controllers, Member Servers \* \*\*Operating Systems\*\*: Windows Server 2016, Windows Server 2019, Windows Server 2022

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: Administrative Standards / Scheduled Tasks

## Rationale The Install From Media (IFM) feature allows administrators to promote a new Domain Controller using an offline backup dataset rather than copying the entire Active Directory database over the network.

To generate this dataset, administrators run the `ntdsutil` tool (e.g., `ntdsutil "ac i ntds" "ifm" "create full c:\Staging" q q`). This process generates a staging folder containing:

1. The Active Directory database file ( `ntds.dit` ).
2. Copies of the `SYSTEM` and `SECURITY` registry hives (which contain the boot key needed to decrypt the database file).

If these staged IFM folders are left behind on member servers, administrative shares, or staging volumes, any user or attacker who compromises that machine can copy the files and extract all Active Directory password hashes offline. Securing active directory requires ensuring that temporary IFM datasets are deleted immediately after the new Domain Controller is promoted.

## Legacy Impact & Compatibility \* \*\*Operational Readiness\*\*: The only operational impact is that the IFM data is no longer available on the staging host. If another Domain Controller needs to be promoted using IFM, a new dataset must be generated. \* \*\*Service Interruption\*\*: This cleanup process has zero impact on active Active Directory services.

### ## Implementation Steps

#### ### Option A: Manual Search and Deletion (GUI)

To manually locate and clean up staged IFM folders:

1. Log on to the staging Member Server or Domain Controller with administrative privileges.
2. Open **File Explorer** and search all local disk volumes (e.g., `C:\` , `D:\` ) for files named `ntds.dit` .
3. Verify the location of each found file:
  - **Domain Controllers:** The authorized directory is the Active Directory database path (typically `C:\Windows\NTDS\ntds.dit` as specified in the registry value `DSA Database file` under `HKLM\SYSTEM\CurrentControlSet\Services\NTDS\Parameters` ).
  - **Member Servers:** There are no authorized directories. Any instance of `ntds.dit` is unauthorized.
4. If an unauthorized `ntds.dit` file is located:
  - Select the folder containing the `ntds.dit` file (which usually also contains a `registry` sub-folder).
  - Delete the folder and clear it from the Recycle Bin.

#### ### Option B: PowerShell & Remediation (Non-GPO / Script-based)

Use these scripts to audit and automatically delete staged IFM datasets.

[Download Script: Remove-StagedIFM.ps1](#)

```
Remove-StagedIFM.ps1
Description: Searches for unauthorized copies of ntds.dit and deletes them.
Target Engine: Windows PowerShell 5.1

Write-Host "Applying hardening requirement: Clean Up Staged IFM Data..." -ForegroundColor Cyan

1. Resolve standard active directory database path (only applicable to DCs)
$StandardDir = $null
$StandardAdPath = (Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\NTDS\Parameters" -Name "DSA Database file" -Error
Action SilentlyContinue)."DSA Database file"
if ($StandardAdPath) {
 $StandardDir = [System.IO.Path]::GetDirectoryName($StandardAdPath)
}

2. Recursive search function excluding standard heavy system directories
function Search-UnauthorizedDIT ($SearchPath) {
 if (-not (Test-Path $SearchPath)) { return @() }

 $Found = @()
 $Items = Get-ChildItem -Path $SearchPath -ErrorAction SilentlyContinue
 foreach ($Item in $Items) {
 if ($Item.Attributes -match "Directory") {
 # Skip system/program directories to optimize speed
 if ($Item.Name -match "^(Windows|Program Files|Program Files \(\x86\)|\$Recycle\.Bin|System Volume Information|AppData)
$") {
 continue
 }
 $Found += Search-UnauthorizedDIT $Item.FullName
 } elseif ($Item.Name -ieq "ntds.dit") {
 # Skip authorized DC database folder
 if ($null -ne $StandardDir -and $Item.DirectoryName -eq $StandardDir) {
 continue
 }
 $Found += $Item.FullName
 }
 }
 return $Found
}

3. Scan all local drives
$Drives = Get-PSDrive -PSProvider FileSystem
$VulnerableFiles = @()

foreach ($Drive in $Drives) {
 $Root = $Drive.Root
 Write-Host "[*] Scanning drive $Root for unauthorized ntds.dit files..." -ForegroundColor Gray
 $VulnerableFiles += Search-UnauthorizedDIT $Root
}

4. Delete unauthorized directories
if ($VulnerableFiles.Count -gt 0) {
 Write-Host "[-] Found $($VulnerableFiles.Count) unauthorized ntds.dit file(s)." -ForegroundColor Yellow
 foreach ($File in $VulnerableFiles) {
 $FolderToDelete = [System.IO.Path]::GetDirectoryName($File)
 Write-Host "[-] Deleting staging directory: $FolderToDelete" -ForegroundColor Yellow
 try {
 Remove-Item -Path $FolderToDelete -Recurse -Force -ErrorAction Stop
 Write-Host "[+] Directory $FolderToDelete successfully deleted." -ForegroundColor Green
 } catch {
 Write-Error "Failed to delete directory: $FolderToDelete. Error: $($_.Exception.Message)"
 }
 }
} else {
 Write-Host "[+] No unauthorized staged ntds.dit files found on local drives." -ForegroundColor Green
}
}
```

*To audit the system for staged IFM directories:*

[Download Script: Get-StagedIFMStatus.ps1](#)

```

Get-StagedIFMStatus.ps1
Description: Audits local drives for unauthorized staged ntds.dit databases.
Target Engine: Windows PowerShell 5.1

Write-Host "--- Auditing Staged IFM Data ---" -ForegroundColor Cyan

1. Resolve standard active directory database path (only applicable to DCs)
$StandardDir = $null
$StandardAdPath = (Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\NTDS\Parameters" -Name "DSA Database file" -Error
Action SilentlyContinue)."DSA Database file"
if ($StandardAdPath) {
 $StandardDir = [System.IO.Path]::GetDirectoryName($StandardAdPath)
}

2. Recursive search function
function Search-UnauthorizedDIT ($SearchPath) {
 if (-not (Test-Path $SearchPath)) { return @() }

 $Found = @()
 $Items = Get-ChildItem -Path $SearchPath -ErrorAction SilentlyContinue
 foreach ($Item in $Items) {
 if ($Item.Attributes -match "Directory") {
 # Skip system/program directories to optimize speed
 if ($Item.Name -match "^(Windows|Program Files|Program Files \(\x86\)|\$Recycle\Bin|System Volume Information|AppData)
$") {
 continue
 }
 $Found += Search-UnauthorizedDIT $Item.FullName
 } elseif ($Item.Name -ieq "ntds.dit") {
 # Skip authorized DC database folder
 if ($null -ne $StandardDir -and $Item.DirectoryName -eq $StandardDir) {
 continue
 }
 $Found += $Item.FullName
 }
 }
 return $Found
}

3. Scan local drives
$Drives = Get-PSDrive -PSProvider FileSystem
$VulnerableFiles = @()

foreach ($Drive in $Drives) {
 $Root = $Drive.Root
 $VulnerableFiles += Search-UnauthorizedDIT $Root
}

4. Evaluate status
if ($VulnerableFiles.Count -gt 0) {
 foreach ($File in $VulnerableFiles) {
 Write-Host "[!] VULNERABLE: Unauthorized ntds.dit database found at: $File" -ForegroundColor Red
 }
 exit 1
} else {
 Write-Host "[+] Secure: No unauthorized staged ntds.dit database files found." -ForegroundColor Green
 exit 0
}

```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*<sup>2</sup>: Recommendations on protecting domain backups and credentials storage \* \*\*Microsoft Security Guidance\*\*<sup>3</sup>: Active Directory Install From Media (IFM) Security best practices

## # Module 7: Privileged Access Workstations (PAWs) Hardening

This directory contains the physical isolation policies and operating system security configurations required to protect Tier 0 administrative workstations.

### ## Technical Hardening Controls

1. [REQ-PAW-001 - Configure AppLocker Policies for PAWs](#) Enforces strict AppLocker application control policies, restricting execution of unauthorized binaries to approved administrative groups.
2. [REQ-PAW-002 - Enable LSA Protection for PAWs](#) Configures LSASS to run as a protected process (PPL) to block credential dumping tools from harvesting secrets from LSA memory.
3. [REQ-PAW-003 - Restrict Local Administrators Group for PAWs](#) Restricts and audits membership in the local Administrators group on PAWs to prevent unauthorized local administrative access.
4. [REQ-PAW-004 - Enforce BitLocker with TPM and Startup PIN for PAWs](#) Configures highly stringent BitLocker policies specifically for PAWs, requiring TPM + pre-boot Startup PIN (no Network Unlock allowed), disabling sleep/standby states (S1-S3) to prevent DMA attacks, enabling Kernel DMA Protection, and enforcing enhanced PIN rules and automatic AD recovery password rotation.
5. [REQ-PAW-005 - UEFI Firmware Security Hardening](#) Enforces UEFI firmware locking, setting a strong BIOS administrator password, disabling CSM/Legacy boot, locking the boot order, and protecting against BIOS rollbacks.
6. [REQ-PAW-006 - Enable Hardware Virtualization and DMA Protection](#) Enables hardware CPU virtualization, IOMMU/DMA protection at the firmware level, and TPM 2.0 to provide the necessary platform integrity foundation for Virtualization-Based Security (VBS).
7. [REQ-PAW-007 - Disable Windows Platform Binary Table \(WPBT\)](#) Disables execution of binaries supplied by the Windows Platform Binary Table (WPBT) ACPI firmware table to mitigate boot-level security bypasses.
8. [REQ-PAW-008 - Windows Defender Antivirus PAW Baseline and Exploit Guard](#) Configures Windows Defender Antivirus on PAWs, enabling real-time scanning, behavioral monitoring, preventing local exclusion modifications, enforcing all ASR rules in strict Block mode, activating Tamper Protection, and enabling AppContainer sandbox isolation.
  - [REQ-PAW-057 - Disable Real-Time Monitoring and Behavior Monitoring Override for PAWs](#)
  - [REQ-PAW-058 - Configure Potentially Unwanted Applications \(PUA\) Protection for PAWs](#)
  - [REQ-PAW-059 - Prevent Local List Merging and Exclusions Configuration for PAWs](#)
  - [REQ-PAW-060 - Configure Auto Exclusions Configuration for PAWs](#)
  - [REQ-PAW-061 - Enable EDR in Block Mode for PAWs](#)
  - [REQ-PAW-062 - Allow Network Protection on Windows Server for PAWs](#)
  - [REQ-PAW-063 - Enable File Hash Computation for PAWs](#)
  - [REQ-PAW-064 - Configure Network Inspection System \(NIS\) settings for PAWs](#)
  - [REQ-PAW-065 - Configure OOB Real-Time Protection and Security Intelligence for PAWs](#)
  - [REQ-PAW-066 - Enable Dynamic Signature Dropped Event Reporting for PAWs](#)
  - [REQ-PAW-067 - Configure Quick Scan and Scanning Exclusions for PAWs](#)
  - [REQ-PAW-068 - Configure Scheduled Scan Parameters for PAWs](#)
  - [REQ-PAW-069 - Configure Security Intelligence Update Schedule for PAWs](#)
  - [REQ-PAW-070 - Configure Attack Surface Reduction Rules for PAWs](#)
    - [REQ-PAW-076 - ASR: Block abuse of exploited vulnerable signed drivers for PAWs](#)
    - [REQ-PAW-077 - ASR: Block Adobe Reader from creating child processes for PAWs](#)
    - [REQ-PAW-078 - ASR: Block all Office applications from creating child processes for PAWs](#)
    - [REQ-PAW-079 - ASR: Block credential stealing from the Windows local security authority subsystem for PAWs](#)
    - [REQ-PAW-080 - ASR: Block executable content from email client and webmail for PAWs](#)
    - [REQ-PAW-081 - ASR: Block executable files from running unless they meet a prevalence, age, or trusted list criterion for PAWs](#)
    - [REQ-PAW-082 - ASR: Block execution of potentially obfuscated scripts for PAWs](#)
    - [REQ-PAW-083 - ASR: Block JavaScript or VBScript from launching downloaded executable content for PAWs](#)
    - [REQ-PAW-084 - ASR: Block Office applications from creating executable content for PAWs](#)
    - [REQ-PAW-085 - ASR: Block Office applications from injecting code into other processes for PAWs](#)
    - [REQ-PAW-086 - ASR: Block Office communication application from creating child processes for PAWs](#)
    - [REQ-PAW-087 - ASR: Block persistence through WMI event subscription for PAWs](#)
    - [REQ-PAW-088 - ASR: Block process creations originating from PSEXEC and WMI commands for PAWs](#)

- [REQ-PAW-089 - ASR: Block untrusted and unsigned processes that run from USB for PAWs](#)
  - [REQ-PAW-090 - ASR: Block Win32 API calls from Office macros for PAWs](#)
  - [REQ-PAW-091 - ASR: Use advanced protection against ransomware for PAWs](#)
  - [REQ-PAW-071 - Configure Threat Severity Default Quarantine Actions for PAWs](#)
  - [REQ-PAW-072 - Configure Family Options UI Lockdown for PAWs](#)
  - [REQ-PAW-073 - Configure Tamper Protection for PAWs](#)
  - [REQ-PAW-074 - Configure Sandbox Execution Environment for PAWs](#)
  - [REQ-PAW-075 - Configure AMSI Authenticode Signature Verification for PAWs](#)
9. [REQ-PAW-009 - Configure User Rights Assignments for PAWs](#) Restricts critical user rights assignments (URAs) such as debugging programs, token impersonation, and denying network/interactive logon permissions for standard accounts on PAWs.
- [REQ-PAW-092 - Configure User Rights: Access Credential Manager as a trusted caller for PAWs](#)
  - [REQ-PAW-093 - Configure User Rights: Access this computer from the network for PAWs](#)
  - [REQ-PAW-094 - Configure User Rights: Act as part of the operating system for PAWs](#)
  - [REQ-PAW-095 - Configure User Rights: Allow log on locally for PAWs](#)
  - [REQ-PAW-096 - Configure User Rights: Back up files and directories for PAWs](#)
  - [REQ-PAW-097 - Configure User Rights: Create a pagefile for PAWs](#)
  - [REQ-PAW-098 - Configure User Rights: Create a token object for PAWs](#)
  - [REQ-PAW-099 - Configure User Rights: Create global objects for PAWs](#)
  - [REQ-PAW-100 - Configure User Rights: Create permanent shared objects for PAWs](#)
  - [REQ-PAW-101 - Configure User Rights: Debug programs for PAWs](#)
  - [REQ-PAW-102 - Configure User Rights: Enable computer and user accounts to be trusted for delegation for PAWs](#)
  - [REQ-PAW-103 - Configure User Rights: Force shutdown from a remote system for PAWs](#)
  - [REQ-PAW-104 - Configure User Rights: Impersonate a client after authentication for PAWs](#)
  - [REQ-PAW-105 - Configure User Rights: Load and unload device drivers for PAWs](#)
  - [REQ-PAW-106 - Configure User Rights: Lock pages in memory for PAWs](#)
  - [REQ-PAW-107 - Configure User Rights: Manage auditing and security log for PAWs](#)
  - [REQ-PAW-108 - Configure User Rights: Modify firmware environment values for PAWs](#)
  - [REQ-PAW-109 - Configure User Rights: Perform volume maintenance tasks for PAWs](#)
  - [REQ-PAW-110 - Configure User Rights: Profile single process for PAWs](#)
  - [REQ-PAW-111 - Configure User Rights: Restore files and directories for PAWs](#)
  - [REQ-PAW-112 - Configure User Rights: Take ownership of files or other objects for PAWs](#)
  - [REQ-PAW-113 - Configure User Rights: Deny access to this computer from the network for PAWs](#)
  - [REQ-PAW-114 - Configure User Rights: Deny log on through Remote Desktop Services for PAWs](#)
10. [REQ-PAW-010 - Enable VBS and Credential Guard for PAWs](#) Configures Virtualization-Based Security (VBS), Credential Guard (with UEFI Lock), System Guard Secure Launch, and memory protections to shield LSASS from credential dumping attacks on PAWs.
11. [REQ-PAW-011 - Harden DMA and Physical Security for PAWs](#) Mitigates physical access threat vectors by disabling sleep standby states (S1-S3), disabling external DMA device enumeration under lock, enforcing a strict block-all device enumeration policy, and blocking legacy SBP-2 device classes.
12. [REQ-PAW-012 - Enable WDAC Driver Blocklist](#) Enforces the Microsoft Vulnerable Driver Blocklist using Windows Defender Application Control (WDAC) to protect the kernel from Bring Your Own Vulnerable Driver (BYOVD) attacks.
13. [REQ-PAW-013 - Configure Account and Password Policies for PAWs](#) Configures robust local account lockout, local password complexity, and 20-character minimum length policies, and references Active Directory Fine-Grained Password Policies (FGPP) for Tier 0 Administrators.
14. [REQ-PAW-014 - Configure Early Launch Antimalware \(ELAM\) Policy for PAWs](#) Configures the Early Launch Antimalware (ELAM) driver initialization policy to ensure only signed, trusted boot drivers execute.
15. [REQ-PAW-015 - Configure Secure Printing and Print Spooler Policies for PAWs](#) Enforces disabling the Print Spooler service and configuring Point and Print restrictions to prevent print-related exploits.



16. [REQ-PAW-016 - Configure Untrusted Font Blocking for PAWs](#) Configures the Untrusted Font Blocking mitigation on PAWs to prevent font parsing exploits.
17. [REQ-PAW-017 - Configure svchost.exe Mitigation Options for PAWs](#) Configures svchost.exe mitigation options on PAWs to restrict binary loading to Microsoft-signed code and block dynamic code execution.
18. [REQ-PAW-018 - Enable Kernel-Mode Hardware-Enforced Stack Protection for PAWs](#) Configures Kernel-mode Hardware-enforced Stack Protection to enforce hardware-backed control-flow integrity and mitigate kernel Return-Oriented Programming (ROP) execution hijacks.
19. [REQ-PAW-019 - Harden Network Parameters and Disable Legacy Name Resolution](#) Disables Link-Local Multicast Name Resolution (LLMNR), NetBIOS over TCP/IP, and mDNS, and secures TCP/IP parameters on PAWs to prevent credential harvesting and protocol exploits.
20. [REQ-PAW-020 - Configure User Account Control Policies for PAWs](#) Enforces maximum UAC security behavior, requiring credential entry on the secure desktop for administrators and automatically denying elevation prompts.
21. [REQ-PAW-021 - Disable AutoPlay and AutoRun for PAWs](#) Turns off AutoPlay and AutoRun features across all drive types to prevent automatic execution of files and payloads from external media.
22. [REQ-PAW-022 - Disable Incoming Remote Desktop Access for PAWs](#) Strictly denies incoming RDP and Remote Assistance connections to administrative workstations to block lateral movement.
23. [REQ-PAW-023 - WSUS Client Configuration for PAWs](#) Enforces update client registry baselines to ensure workstations pull OS patches and security signatures exclusively from the local, offline WSUS server.
24. [REQ-PAW-024 - Configure User Profile and System Restrictions for PAWs](#) Locks down user profile registry settings and key system security policies including inactivity timeouts, secondary logon, and ASLR force.
  - [REQ-PAW-115 - User Profile: Toast Notifications Lock Screen Restrictions for PAWs](#)
  - [REQ-PAW-116 - User Profile: Spotlight and Consumer Features Restrictions for PAWs](#)
  - [REQ-PAW-117 - User Profile: Windows Copilot Restrictions for PAWs](#)
  - [REQ-PAW-118 - User Profile: In-Place Sharing Restrictions for PAWs](#)
  - [REQ-PAW-119 - User Profile: Shell RunAs User Suppression for PAWs](#)
  - [REQ-PAW-120 - User Profile: Personalization and Privacy Restrictions for PAWs](#)
  - [REQ-PAW-121 - User Profile: Group Policy Processing Behaviors for PAWs](#)
  - [REQ-PAW-122 - User Profile: Telemetry and Inventory Collection Restrictions for PAWs](#)
  - [REQ-PAW-123 - User Profile: Explorer Security and Memory Protections for PAWs](#)
  - [REQ-PAW-124 - User Profile: Internet Explorer Options and Feeds Restrictions for PAWs](#)
  - [REQ-PAW-125 - User Profile: Interactive Logon Warning Banners for PAWs](#)
  - [REQ-PAW-126 - User Profile: Interactive Logon Inactivity Timeout for PAWs](#)
  - [REQ-PAW-127 - User Profile: Windows Installer Hardening for PAWs](#)
  - [REQ-PAW-128 - User Profile: Secondary Logon Service Lockdown for PAWs](#)
  - [REQ-PAW-140 - User Profile: Structured Exception Handling Overwrite Protection \(SEOP\) for PAWs](#)
  - [REQ-PAW-141 - User Profile: Directory Protection Mode for PAWs](#)
  - [REQ-PAW-142 - User Profile: Address Space Layout Randomization \(ASLR\) Image Relocation for PAWs](#)
  - [REQ-PAW-143 - User Profile: Speculative Execution Mitigations \(Spectre/Meltdown\) for PAWs](#)
  - [REQ-PAW-144 - User Profile: Authenticode Certificate Padding Check for PAWs](#)
  - [REQ-PAW-145 - User Profile: Command Processor Batch File Locking for PAWs](#)
  - [REQ-PAW-146 - User Profile: Time-Travel Debugging \(TTD\) Recording Policy for PAWs](#)
  - [REQ-PAW-147 - User Profile: Trusted Root Store Protected Roots Certificate Restriction for PAWs](#)
  - [REQ-PAW-148 - User Profile: Disabling Injection of AppInit DLLs for PAWs](#)
  - [REQ-PAW-149 - User Profile: Preservation of Attachment Zone Information for PAWs](#)
  - [REQ-PAW-150 - User Profile: Disable Windows Game DVR for PAWs](#)
  - [REQ-PAW-151 - User Profile: Restrict Windows Ink Workspace on Lock Screen for PAWs](#)
25. [REQ-PAW-025 - Configure Exploit Protection Profile for PAWs](#) Configures and enforces a system-wide Microsoft Defender Exploit Protection profile to apply advanced memory mitigations (DEP, ASLR, CFG, SEHOP, Heap Integrity) on all PAWs.
26. [REQ-PAW-026 - Restrict Safe Mode Access to Administrators on PAWs](#) Prevents standard (non-administrative) users from logging into the system while in Safe Mode by setting SafeModeBlockNonAdmins to 1.

27. [REQ-PAW-027 - Configure Windows Defender Firewall and Block LOLBins for PAWs](#) Configures host firewall profiles and outbound rules to block known Living Off the Land Binaries (LOLBins) from initiating outgoing network connections.
28. [REQ-PAW-028 - Disable Unnecessary System Services for PAWs](#) Disables unnecessary and high-risk system services to minimize the attack surface of administrative endpoints.
- [REQ-PAW-037 - Disable Computer Browser Service for PAWs \(Browser\)](#)
  - [REQ-PAW-038 - Disable Infrared Monitor Service for PAWs \(irmon\)](#)
  - [REQ-PAW-039 - Disable Internet Connection Sharing \(ICS\) Service for PAWs \(SharedAccess\)](#)
  - [REQ-PAW-040 - Disable LxssManager Service for PAWs \(LxssManager\)](#)
  - [REQ-PAW-041 - Disable Microsoft FTP Service for PAWs \(FTPSVC\)](#)
  - [REQ-PAW-042 - Disable OpenSSH SSH Server Service for PAWs \(sshd\)](#)
  - [REQ-PAW-043 - Disable Remote Procedure Call \(RPC\) Locator Service for PAWs \(RpcLocator\)](#)
  - [REQ-PAW-044 - Disable Routing and Remote Access Service for PAWs \(RemoteAccess\)](#)
  - [REQ-PAW-045 - Disable Simple TCP/IP Services for PAWs \(simptcp\)](#)
  - [REQ-PAW-046 - Disable Special Administration Console Helper Service for PAWs \(sacsvr\)](#)
  - [REQ-PAW-047 - Disable SSDP Discovery Service for PAWs \(SSDPSRV\)](#)
  - [REQ-PAW-048 - Disable UPnP Device Host Service for PAWs \(upnphost\)](#)
  - [REQ-PAW-049 - Disable Web Management Service for PAWs \(WMSvc\)](#)
  - [REQ-PAW-050 - Disable Windows Media Player Network Sharing Service for PAWs \(WMPNetworkSvc\)](#)
  - [REQ-PAW-051 - Disable Windows Mobile Hotspot Service for PAWs \(icssvc\)](#)
  - [REQ-PAW-052 - Disable World Wide Web Publishing Service for PAWs \(W3SVC\)](#)
  - [REQ-PAW-053 - Disable Xbox Accessory Management Service for PAWs \(XboxGipSvc\)](#)
  - [REQ-PAW-054 - Disable Xbox Live Auth Manager Service for PAWs \(XblAuthManager\)](#)
  - [REQ-PAW-055 - Disable Xbox Live Game Save Service for PAWs \(XblGameSave\)](#)
  - [REQ-PAW-056 - Disable Xbox Live Networking Service for PAWs \(XboxNetApiSvc\)](#)
29. [REQ-PAW-029 - Configure System Administrative Templates for PAWs](#) Enforces custom administrative template settings including SMBv1 driver blocks and event log size extensions.
30. [REQ-PAW-030 - Enable UEFI Secure Boot for PAWs](#) Mandates hardware-rooted platform integrity checks, verifying that UEFI Secure Boot is active on the operating system for PAWs.
31. [REQ-PAW-031 - Enforce Smart Card Logon for PAWs](#) Enforces the 'Interactive logon: Require smart card' GPO policy locally to suppress username/password fields and force hardware-bound authentication.
32. [REQ-PAW-032 - Disable Unused Windows Features and PowerShell 2.0 Engine](#) Disables legacy, unused Windows optional features, including PowerShell 2.0, .NET Framework 3.5, and SMBv1 to minimize the workstation attack surface.
33. [REQ-PAW-033 - Configure Microsoft Office Security and Block OLE Packages](#) Blocks VBA macros from running in Office files downloaded from the Internet, enforces macro digital signing warnings, and disables OLE Package execution in Outlook to prevent initial access exploits.
34. [REQ-PAW-034 - Disable Windows Script Host and Remap Scripting Extensions](#) Disables Windows Script Host execution globally and remaps standard scripting extensions (.vbs, .js, etc.) to open in Notepad by default to prevent execution by double-click.
35. [REQ-PAW-035 - Configure Secure Boot Revocations and Bootloader Updates for PAWs](#) Configures and enforces BlackLotus revocation updates and bootloader integrity verification policy variables in system firmware for PAWs.
36. [REQ-PAW-036 - Configure Windows Defender Application Control](#) Deploys Windows Defender Application Control (WDAC) on PAWs in Audit Mode to block unauthorized system-level binaries and scripts.
37. [REQ-PAW-129 - Configure Advanced Security Audit Policies for PAWs](#) Enforces granular Windows security audit policies (including logons, group memberships, registry access, and system events) to log critical threat telemetry on privileged access workstations.
- [REQ-PAW-130 - Audit Policy: Advanced Audit Policy Overrides for PAWs](#)
  - [REQ-PAW-131 - Audit Policy: Account Logon Auditing for PAWs](#)
  - [REQ-PAW-132 - Audit Policy: Account Management Auditing for PAWs](#)
  - [REQ-PAW-133 - Audit Policy: Detailed Tracking Auditing for PAWs](#)
  - [REQ-PAW-134 - Audit Policy: Logon and Logoff Auditing for PAWs](#)
  - [REQ-PAW-135 - Audit Policy: Object Access Auditing for PAWs](#)

- [REQ-PAW-136 - Audit Policy: Policy Change Auditing for PAWs](#)
- [REQ-PAW-137 - Audit Policy: Privilege Use Auditing for PAWs](#)
- [REQ-PAW-138 - Audit Policy: System Events Auditing for PAWs](#)

# [REQ-PAW-001] Configure AppLocker Policies for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: Computer Configuration\Policies\Windows Settings\Security Settings\Application Control Policies\AppLocker

## Rationale Privileged Access Workstations (PAWs) host highly sensitive Tier 0 credentials. If administrative workstations are allowed to execute arbitrary binaries, scripts, or installation packages, they become highly susceptible to malware infections, remote access trojans, and credential harvesting tools (like Mimikatz).

Enforcing strict execution controls via AppLocker ensures that:

1. **Execution Control:** Only signed operating system files, approved software binaries, and scripts are allowed to execute.
2. **Standard User Restrictions:** Any standard users or unauthorized accounts cannot run executable files or installers from writeable directories (like `%TEMP%` or `%USERPROFILE%`).
3. **Defends Against AppLocker Bypasses:** Abusing trusted, signed Microsoft binaries (such as `msbuild.exe`, `installutil.exe`, `regasm.exe`, `regsvcs.exe`, `mshta.exe`, `regsvr32.exe`, `rundll32.exe`) allows attackers to execute arbitrary code bypassing default AppLocker rules. This control blocks these "Living off the Land" binaries (LOLBins) and prevents execution from user-writeable paths under `%WINDIR%` (such as `Tasks`, `Temp`, `tracing`, `spool\drivers\color`, etc.).
4. **Defense-in-Depth:** Even if an administrator is tricked into downloading a malicious file, AppLocker blocks the execution of the binary, preventing the compromise of the endpoint.

## Legacy Impact & Compatibility \* \*\*Authorized Software Only\*\*: Only administrative tools, management consoles, and approved software can be run on the PAW. Users will be unable to run portable utilities or install unapproved tools. \* \*\*AppLocker Service Requirement\*\*: The Application Identity service ('AppIDSvc') must be configured to start automatically and run continuously to enforce AppLocker policies. If the service is stopped, rules are not enforced. \* \*\*Administrative Overhead\*\*: Creating and maintaining AppLocker rules requires cataloging administrative tool requirements and updating rules when new utilities are introduced.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Create or edit the GPO linked to the PAWs Organizational Unit (OU) (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Application Control Policies\AppLocker`
4. Configure AppLocker Enforcement:
  - Right-click **AppLocker** and select **Properties**.
  - On the **Enforcement** tab, check **Configured** under:
    - **Executable rules** → Select **Enforce rules**
    - **Windows Installer rules** → Select **Enforce rules**
    - **Script rules** → Select **Enforce rules**
    - **Packaged app rules** → Select **Enforce rules**
5. Right-click **Executable Rules** and select **Create Default Rules** (this permits Windows files and program files).
6. Delete the default rule allowing "Everyone" to run files in all locations, and replace it with a rule allowing only authorized administrative groups (e.g., `Tier0-Admins` ) to run binaries outside the default system locations.
7. Per **ANSSI R2** recommendation, do not create standalone Deny rules. Instead, configure the following path **Exceptions** on the default Allow rule for the Windows folder:
  - Right-click the rule (Default Rule) `All files located in the Windows folder` and select **Properties**.
  - On the **Exceptions** tab, add path exceptions for writeable directories under `%WINDIR%` :
    - `%WINDIR%\Tasks\*`
    - `%WINDIR%\Temp\*`
    - `%WINDIR%\tracing\*`
    - `%WINDIR%\System32\spool\drivers\color\*`
    - `%WINDIR%\System32\Tasks\Microsoft\Windows\SyncCenter\*`
    - `%WINDIR%\SysWOW64\Tasks\Microsoft\Windows\SyncCenter\*`
  - On the same **Exceptions** tab, add path exceptions for the following bypass binaries (LOLBins):
    - `*\msbuild.exe`
    - `*\installutil.exe`
    - `*\mshta.exe`

- \*\\regasm.exe
- \*\\regsvcs.exe
- \*\\regsvr32.exe
- \*\\rundll32.exe
- \*\\bginfo.exe
- \*\\cdb.exe
- \*\\cmstp.exe
- \*\\control.exe
- \*\\csi.exe
- \*\\dfsvc.exe
- \*\\dnx.exe
- \*\\fsi.exe
- \*\\ie4unit.exe
- \*\\ieexec.exe
- \*\\infdefaultinstall.exe
- \*\\mavinject.exe
- \*\\msdeploy.exe
- \*\\msdt.exe
- \*\\msxsl.exe
- \*\\odbccconf.exe
- \*\\presentationhost.exe
- \*\\rcsi.exe
- \*\\rsi.exe
- \*\\runscripthelper.exe
- \*\\te.exe
- \*\\tracker.exe
- \*\\xwizard.exe

8. Repeat the process for **Script Rules** by creating default rules and adding exceptions to the `%WINDIR%\*`. Allow rule for script execution from user-writeable paths (such as `%WINDIR%\Temp\*` and `%WINDIR%\Tasks\*`).

9. Disable NTVDM (16-bit application support) to prevent AppLocker bypasses via 16-bit binaries:

- Navigate to: `Computer Configuration\Administrative Templates\System\16-bit Application Compatibility`
- Configure **Prevent access to 16-bit applications** to **Enabled**.

10. Link the GPO to the PAWs Organizational Unit (OU).

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Configure the Application Identity service ( `AppIDSvc` ) and import the robust AppLocker policy locally.

[Download Script: Configure-PawAppLockerService.ps1](#)

```
Configure-PawAppLockerService.ps1
Description: Configures the Application Identity service (AppIDSvc) to start automatically and imports a robust AppLocker XML policy.

Write-Host "Applying AppLocker Identity service hardening..." -ForegroundColor Cyan

1. Enable Application Identity service (AppIDSvc)
$AppLockerService = Get-Service -Name AppIDSvc -ErrorAction SilentlyContinue
if ($AppLockerService) {
 Set-Service -Name AppIDSvc -StartupType Automatic
 Start-Service -Name AppIDSvc -ErrorAction SilentlyContinue
 Write-Host "[+] Application Identity Service (AppIDSvc) set to Automatic and started." -ForegroundColor Green
} else {
 Write-Warning "[-] Application Identity Service not found on this machine."
}

2. Configure local AppLocker policy XML content
$AppLockerXml = @"
<AppLockerPolicy Version="1">
 <RuleCollection Type="Exe" EnforcementMode="Enabled">
 <FilePathRule Id="921cc481-6e1e-453f-b3a5-bc4f4a38674d" Name="(Default Rule) All files located in the Program Files folder" Description="Allows members of the Everyone group to run applications that are located in the Program Files folder." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePathCondition Path="%PROGRAMFILES%*" />
 </Conditions>
 </FilePathRule>
 <FilePathRule Id="a61c8b2c-6d8f-4ad9-acbc-467b78a7f7b4" Name="(Default Rule) All files located in the Windows folder" Description="Allows members of the Everyone group to run applications that are located in the Windows folder." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePathCondition Path="%WINDIR%*" />
 </Conditions>
 <Exceptions>
 <FilePathCondition Path="%WINDIR%\Temp*" />
 <FilePathCondition Path="%WINDIR%\Tasks*" />
 <FilePathCondition Path="%WINDIR%\tracing*" />
 <FilePathCondition Path="%WINDIR%\System32\spool\drivers\color*" />
 <FilePathCondition Path="%WINDIR%\System32\Tasks\Microsoft\Windows\SyncCenter*" />
 <FilePathCondition Path="%WINDIR%\SysWOW64\Tasks\Microsoft\Windows\SyncCenter*" />
 <FilePathCondition Path="*\msbuild.exe" />
 <FilePathCondition Path="*\installutil.exe" />
 <FilePathCondition Path="*\mshta.exe" />
 <FilePathCondition Path="*\regasm.exe" />
 <FilePathCondition Path="*\regsvcs.exe" />
 <FilePathCondition Path="*\regsvr32.exe" />
 <FilePathCondition Path="*\rundll32.exe" />
 <FilePathCondition Path="*\bginfo.exe" />
 <FilePathCondition Path="*\cdb.exe" />
 <FilePathCondition Path="*\cmstp.exe" />
 <FilePathCondition Path="*\control.exe" />
 <FilePathCondition Path="*\csi.exe" />
 <FilePathCondition Path="*\dfsvc.exe" />
 <FilePathCondition Path="*\dnx.exe" />
 <FilePathCondition Path="*\fsi.exe" />
 <FilePathCondition Path="*\ie4unit.exe" />
 <FilePathCondition Path="*\ieexec.exe" />
 <FilePathCondition Path="*\infdefaultinstall.exe" />
 <FilePathCondition Path="*\mavinject.exe" />
 <FilePathCondition Path="*\msdeploy.exe" />
 <FilePathCondition Path="*\msdt.exe" />
 <FilePathCondition Path="*\msxml.exe" />
 <FilePathCondition Path="*\odbccconf.exe" />
 <FilePathCondition Path="*\presentationhost.exe" />
 <FilePathCondition Path="*\rcsi.exe" />
 <FilePathCondition Path="*\rsi.exe" />
 <FilePathCondition Path="*\runscripthelper.exe" />
 <FilePathCondition Path="*\te.exe" />
 <FilePathCondition Path="*\tracker.exe" />
 <FilePathCondition Path="*\xwizard.exe" />
 </Exceptions>
 </FilePathRule>
 <FilePathRule Id="fd686d83-a829-4351-8ff4-27c1de5732e9" Name="(Default Rule) All files" Description="Allows members of the local Administrators group to run all applications." UserOrGroupSid="S-1-5-32-544" Action="Allow">

```

```

 <Conditions>
 <FilePathCondition Path="*" />
 </Conditions>
 </FilePathRule>
</RuleCollection>
<RuleCollection Type="Msi" EnforcementMode="Enabled">
 <FilePathRule Id="5b8fa8b3-3a5e-4c7a-9cb8-b223ff9db271" Name="(Default Rule) All Windows Installer files in Program Files" Description="Allows everyone to run Windows Installer files in Program Files." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePathCondition Path="%PROGRAMFILES%*" />
 </Conditions>
 </FilePathRule>
 <FilePathRule Id="6b8fa8b3-3a5e-4c7a-9cb8-b223ff9db272" Name="(Default Rule) All Windows Installer files in Windows" Description="Allows everyone to run Windows Installer files in Windows." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePathCondition Path="%WINDIR%*" />
 </Conditions>
 </FilePathRule>
 <FilePathRule Id="7b8fa8b3-3a5e-4c7a-9cb8-b223ff9db273" Name="(Default Rule) All Windows Installer files" Description="Allows administrators to run all Windows Installer files." UserOrGroupSid="S-1-5-32-544" Action="Allow">
 <Conditions>
 <FilePathCondition Path="*" />
 </Conditions>
 </FilePathRule>
</RuleCollection>
<RuleCollection Type="Script" EnforcementMode="Enabled">
 <FilePathRule Id="1c8fa8b3-3a5e-4c7a-9cb8-b223ff9db274" Name="(Default Rule) All scripts located in the Program Files folder" Description="Allows everyone to run scripts in Program Files." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePathCondition Path="%PROGRAMFILES%*" />
 </Conditions>
 </FilePathRule>
 <FilePathRule Id="2c8fa8b3-3a5e-4c7a-9cb8-b223ff9db275" Name="(Default Rule) All scripts located in the Windows folder" Description="Allows everyone to run scripts in Windows." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePathCondition Path="%WINDIR%*" />
 </Conditions>
 <Exceptions>
 <FilePathCondition Path="%WINDIR%\Temp%*" />
 <FilePathCondition Path="%WINDIR%\Tasks%*" />
 </Exceptions>
 </FilePathRule>
 <FilePathRule Id="3c8fa8b3-3a5e-4c7a-9cb8-b223ff9db276" Name="(Default Rule) All scripts" Description="Allows administrators to run all scripts." UserOrGroupSid="S-1-5-32-544" Action="Allow">
 <Conditions>
 <FilePathCondition Path="*" />
 </Conditions>
 </FilePathRule>
</RuleCollection>
<RuleCollection Type="Appx" EnforcementMode="Enabled">
 <FilePublisherRule Id="1d8fa8b3-3a5e-4c7a-9cb8-b223ff9db279" Name="(Default Rule) All signed packaged apps" Description="Allows everyone to run signed packaged apps." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePublisherCondition PublisherName="*" ProductName="*" BinaryName="*">
 <BinaryVersionRange LowSection="0.0.0.0" HighSection="*" />
 </FilePublisherCondition>
 </Conditions>
 </FilePublisherRule>
</RuleCollection>
</AppLockerPolicy>
"@

Write the temporary XML and import it
$TempPath = Join-Path -Path $env:TEMP -ChildPath "AppLockerPawPolicy.xml"
$AppLockerXml | Out-File -FilePath $TempPath -Encoding UTF8 -Force

3. Validate policy using Test-AppLockerPolicy before importing
try {
 Import-Module AppLocker -ErrorAction Stop
} catch {
 Write-Error "AppLocker module is not available on this system. Cannot configure or validate policy."
 if (Test-Path $TempPath) {
 Remove-Item -Path $TempPath -Force
 }
 return
}

```

```

$TestPaths = @(
 # Expected: Allowed
 "$env:windir\System32\cmd.exe",
 "$env:windir\System32\WindowsPowerShell\v1.0\powershell.exe",
 # Expected: DeniedByDefault or ExplicitlyDenied (since it is an exception to an Allow rule)
 "$env:USERPROFILE\Downloads\tool.exe",
 "$env:windir\Temp\malware.exe",
 "$env:windir\Tasks\evil.exe",
 "$env:windir\System32\msbuild.exe"
)

$ValidationFailed = $false
try {
 $TestResults = Test-AppLockerPolicy -XmlPolicy $TempPath -Path $TestPaths -User Everyone -ErrorAction Stop
 $ExpectedAllow = @(
 "$env:windir\System32\cmd.exe",
 "$env:windir\System32\WindowsPowerShell\v1.0\powershell.exe"
)
 $ExpectedDeny = @(
 "$env:USERPROFILE\Downloads\tool.exe",
 "$env:windir\Temp\malware.exe",
 "$env:windir\Tasks\evil.exe",
 "$env:windir\System32\msbuild.exe"
)

 foreach ($Result in $TestResults) {
 $Path = $Result.FilePath
 $Decision = $Result.PolicyDecision
 if ($ExpectedAllow -contains $Path) {
 if ($Decision -ne "Allowed") {
 Write-Warning "[VALIDATION FAIL] Expected Allow for: $Path (got: $Decision)"
 $ValidationFailed = $true
 }
 }
 if ($ExpectedDeny -contains $Path) {
 if ($Decision -eq "Allowed") {
 Write-Warning "[VALIDATION FAIL] Expected Deny/Not Allowed for: $Path (got: $Decision)"
 $ValidationFailed = $true
 }
 }
 }
} catch {
 Write-Warning "Could not perform policy validation tests: $($_.Exception.Message)"
 $ValidationFailed = $true
}

if ($ValidationFailed) {
 Write-Error "AppLocker policy validation failed. Policy was NOT imported."
 if (Test-Path $TempPath) {
 Remove-Item -Path $TempPath -Force
 }
 return
}

Write-Host "[+] AppLocker policy validation passed. Proceeding with import." -ForegroundColor Green

4. Import the validated AppLocker policy
try {
 Set-AppLockerPolicy -XmlPolicy $TempPath -ErrorAction Stop
 Write-Host "[+] Local AppLocker policy imported and enforced successfully." -ForegroundColor Green
} catch {
 Write-Error "Failed to import AppLocker policy: $($_.Exception.Message)"
} finally {
 if (Test-Path $TempPath) {
 Remove-Item -Path $TempPath -Force
 }
}

5. Disable NTVDM (16-bit compatibility) via Registry
$Ntvdmpath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppCompat"
if (-not (Test-Path $Ntvdmpath)) {
 New-Item -Path $Ntvdmpath -Force | Out-Null
}
Set-ItemProperty -Path $Ntvdmpath -Name "Prevent16BitApp" -Value 1 -Type DWord
Write-Host "[+] 16-bit NTVDM compatibility disabled in registry." -ForegroundColor Green

```



To verify the AppLocker service status:

[Download Script: Test-PawAppLockerStatus.ps1](#)

```
Test-PawAppLockerStatus.ps1
Description: Checks the current configuration and operational status of the Application Identity service.

Write-Host "--- Auditing AppLocker Service Status ---" -ForegroundColor Cyan

1. Audit service state
$AppIDSvc = Get-Service -Name AppIDSvc -ErrorAction SilentlyContinue

if ($AppIDSvc) {
 if ($AppIDSvc.Status -eq "Running" -and $AppIDSvc.StartType -eq "Automatic") {
 Write-Host " - AppLocker Service Status: Running | Startup: Automatic (Secure)" -ForegroundColor Green
 } else {
 Write-Host " - VULNERABLE: AppLocker Service Status: $($AppIDSvc.Status) | Startup: $($AppIDSvc.StartType) (Should be Running/Automatic)" -ForegroundColor Red
 }
} else {
 Write-Host " - VULNERABLE: Application Identity Service (AppIDSvc) is not installed." -ForegroundColor Red
}

2. Audit enforcement registry settings
$SrpPath = "HKLM:\Software\Policies\Microsoft\Windows\SrpV2"
$Collections = @("Exe", "Msi", "Script", "Appx")

if (Test-Path $SrpPath) {
 foreach ($Col in $Collections) {
 $ColPath = "$SrpPath\$Col"
 if (Test-Path $ColPath) {
 $Val = Get-ItemProperty -Path $ColPath -Name "EnforcementMode" -ErrorAction SilentlyContinue
 if ($null -ne $Val) {
 $Mode = if ($Val.EnforcementMode -eq 1) { "Enforced" } else { "Audit Only" }
 $Color = if ($Val.EnforcementMode -eq 1) { "Green" } else { "Yellow" }
 Write-Host " - Collection $Col Enforcement: $Mode (Value: $($Val.EnforcementMode))" -ForegroundColor $Color
 } else {
 Write-Host " - Collection $Col Enforcement: NOT CONFIGURED" -ForegroundColor Red
 }
 } else {
 Write-Host " - Collection $Col Path: NOT FOUND" -ForegroundColor Red
 }
 }
} else {
 Write-Host "[-] AppLocker registry base path (SrpV2) not found. Policy is not deployed." -ForegroundColor Red
}

3. Audit NTVDM Disable Status
$NtvdmPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppCompat"
if (Test-Path $NtvdmPath) {
 $AppCompatVal = Get-ItemProperty -Path $NtvdmPath -Name "Prevent16BitApp" -ErrorAction SilentlyContinue
 if ($null -ne $AppCompatVal -and $AppCompatVal.Prevent16BitApp -eq 1) {
 Write-Host " - NTVDM (16-bit AppCompat): Disabled (Secure)" -ForegroundColor Green
 } else {
 Write-Host " - NTVDM (16-bit AppCompat): Enabled or Not Configured (Expected: Disabled)" -ForegroundColor Yellow
 }
} else {
 Write-Host " - NTVDM (16-bit AppCompat): Not Configured (Expected: Disabled)" -ForegroundColor Yellow
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*\*: Recommendation R58 (Use of Privileged Access Workstations), DAT-NT-13 Note Technique (R2, R8, R10, R15, R16, R20) \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*\*: Section 18.9 (AppLocker Application Control) \* \*\*Microsoft Security Baselines\*\*\*: AppLocker deployment guidance for high-security environments. \* \*\*Ultimate AppLocker Bypass List\*\*\*: Generic & Verified AppLocker Bypasses

# [REQ-PAW-002] Enable LSA Protection for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: Computer Configuration\Policies\Administrative Templates\System\Local Security Authority\Configure LSASS to run as a protected process \* \*\*Registry Location\*\*: HKLM\SYSTEM\CurrentControlSet\Control\Lsa \* 'RunAsPPL' = '1' (REG\_DWORD)

## Rationale The Local Security Authority Subsystem Service (LSASS) process manages security policies, user authentication, and credential tokens on Windows systems. Attackers targeting administrative workstations commonly attempt to extract plain-text credentials or NT hashes from LSASS memory using debugging tools (e.g., Mimikatz, Procdump).

Enabling LSA Protection ensures that:

1. **Protected Process Light (PPL)**: The LSASS process runs as a Protected Process Light (PPL).
2. **Access Restriction**: Only verified, digitally signed code can load into LSASS, and standard processes (even those running as local system/administrator) cannot read the memory space of LSASS or inject code into it.
3. **Mitigating Dump Attacks**: Credential harvesting tools cannot dump LSASS memory to disk or scrape keys from LSA memory blocks.

## Legacy Impact & Compatibility \* \*\*Driver Signing\*\*: Any custom authentication packages or third-party smart card drivers that load into LSASS must be digitally signed with a Microsoft signature. Unsigned drivers will be blocked from loading. \* \*\*Reboot Required\*\*: Enabling or disabling LSA Protection requires a system restart to take effect. \* \*\*Audit Mode Available\*\*: LSA Protection can be run in audit mode (using registry value 'AuditLevel' under the Lsa key) to test for unsigned driver compatibility prior to full enforcement.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the GPO linked to the PAWs Organizational Unit (OU) (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Local Security Authority`
4. Configure the setting:
  - **Policy**: `Configure LSASS to run as a protected process`
  - **Setting**: `Enabled`
  - **Configure LSA to run as a protected process**: `Enabled with UEFI Lock` (or `Enabled without UEFI Lock` depending on management needs)
5. Link the GPO to the PAWs Organizational Unit (OU).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Configure the local registry key on the PAW to run LSASS as a protected process.

[Download Script: Configure-PawLsaProtection.ps1](#)

```
Configure-PawLsaProtection.ps1
Description: Configures the RunAsPPL registry key to enable LSA Protection on PAWs.

Write-Host "Applying LSA Protection registry hardening..." -ForegroundColor Cyan

$LsaPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"

if (-not (Test-Path $LsaPath)) {
 New-Item -Path $LsaPath -Force | Out-Null
}

Set-ItemProperty -Path $LsaPath -Name "RunAsPPL" -Value 1 -Type DWord
Write-Host "[+] LSA Protection (RunAsPPL) enabled in registry. (Reboot required)." -ForegroundColor Green
```

To verify the local LSA Protection state:

[Download Script: Test-PawLsaProtection.ps1](#)

```
Test-PawLsaProtection.ps1
Description: Checks the registry settings and running process state to verify LSA Protection is active.

Write-Host "--- Auditing LSA Protection Status ---" -ForegroundColor Cyan

$LsaPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"
$RunAsPPL = (Get-ItemProperty -Path $LsaPath -Name "RunAsPPL" -ErrorAction SilentlyContinue).RunAsPPL

if ($RunAsPPL -eq 1) {
 Write-Host " - LSA Protection (RunAsPPL): Enabled (Secure)" -ForegroundColor Green
} else {
 Write-Host " - VULNERABLE: LSA Protection (RunAsPPL) is not configured or disabled (Value: $($RunAsPPL))" -ForegroundColor Red
}
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Section on administrative workstation protection and LSASS security.  
\* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*: Section 18.2.1 (LSA Protection) \* \*\*Microsoft Security Baselines\*\*: Windows Defender Credential Guard and LSA Protection guidelines.

# [REQ-PAW-003] Restrict Local Administrators Group for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: Computer Configuration\Policies\Windows Settings\Security Settings\Restricted Groups

---

## Rationale Privileged Access Workstations (PAWs) represent Tier 0 administrative assets. Any user or group that has local administrative rights on a PAW can bypass operating system security boundaries, disable system protections, capture keystrokes, or extract cached credentials.

Restricting local Administrators group membership ensures that:

1. **Administrative Rights Restriction:** Standard domain users and lower-tiered administrators (e.g., workstation support admins) are strictly prevented from executing code in an elevated context on the PAW.
  2. **Tier Separation:** Administrative credentials from lower security tiers cannot compromise the PAW. Only dedicated Tier 0 administrators are allowed local administrative access.
  3. **Authorized Control:** Membership is strictly controlled and reset periodically via Group Policy (Restricted Groups), preventing persistent privilege creep.
- 

## Legacy Impact & Compatibility \* \*\*Daily Work Limitation\*\*: Lower-tier support personnel cannot log on to or troubleshoot the PAW. \* \*\*Dedicated Administrative Accounts\*\*: Administrators must authenticate using their dedicated Tier 0 administrative account (e.g., 'a0-admin'). Standard accounts are blocked from gaining administrative access. \* \*\*LAPS Integration\*\*: The local built-in Administrator account (RID 500) should be managed via Windows LAPS to ensure password rotation and secure retrieval.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

Deploy Restricted Groups via GPO to enforce local administrators group memberships:

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
  2. Edit the GPO linked to the PAWs Organizational Unit (OU) (e.g., `GPO_Hardening_PAW` ).
  3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Restricted Groups`
  4. Right-click **Restricted Groups** and select **Add Group**.
  5. Type `Administrators` (or click Browse to find the local group).
  6. Under **Members of this group**, define the allowed members:
    - `Administrator` (the built-in local administrator account)
    - `DomainName\Tier0-Admins` (dedicated Tier 0 administrative group)
    - *Do NOT include any standard domain users or lower-tier administrative groups.*
  7. Link the GPO to the PAWs Organizational Unit (OU).
- 

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to audit and remediate unauthorized administrative accounts in the local Administrators group.

[Download Script: Clean-PawLocalAdministrators.ps1](#)

```
Clean-PawLocalAdministrators.ps1
Description: Removes unauthorized accounts from the local Administrators group on PAWs.

Write-Host "--- Restricting Local Administrators Group on PAW ---" -ForegroundColor Cyan

Define the list of authorized members
Built-in Administrator and Tier 0 admin groups
$AuthorizedMembers = @("Administrator", "Tier0-Admins")

$LocalAdmins = Get-LocalGroupMember -Group "Administrators" -ErrorAction SilentlyContinue

if ($LocalAdmins) {
 foreach ($Member in $LocalAdmins) {
 $Match = $false
 foreach ($Auth in $AuthorizedMembers) {
 if ($Member.Name -eq $Auth -or $Member.Name -like "*\$Auth" -or $Member.Name -eq "$env:COMPUTERNAME\$Auth") {
 $Match = $true
 break
 }
 }

 if (-not $Match) {
 Write-Host "[-] Removing unauthorized member from Administrators: $($Member.Name) (Source: $($Member.PrincipalSource))"
 -ForegroundColor Yellow
 try {
 Remove-LocalGroupMember -Group "Administrators" -Member $Member.Name -ErrorAction Stop
 Write-Host " Successfully removed: $($Member.Name)" -ForegroundColor Green
 } catch {
 Write-Error " Failed to remove: $($Member.Name). Error: $_.Exception.Message"
 }
 } else {
 Write-Host "[+] Member authorized: $($Member.Name)" -ForegroundColor Green
 }
 }
} else {
 Write-Error "Could not retrieve members of local Administrators group."
}
}
```

*To audit local Administrators group memberships on the PAW:*

[Download Script: Test-PawLocalAdministrators.ps1](#)

```
Test-PawLocalAdministrators.ps1
Description: Audits local Administrators group memberships to ensure only authorized Tier 0 accounts are present.

Write-Host "--- Auditing PAW Local Administrators Group ---" -ForegroundColor Cyan

Define the authorized domain/local patterns
$AuthorizedMembers = @("Administrator", "Tier0-Admins")

$LocalAdmins = Get-LocalGroupMember -Group "Administrators" -ErrorAction SilentlyContinue

if ($LocalAdmins) {
 Write-Host "[*] Current members of local Administrators group:" -ForegroundColor Yellow
 foreach ($Member in $LocalAdmins) {
 $Match = $false
 foreach ($Auth in $AuthorizedMembers) {
 if ($Member.Name -eq $Auth -or $Member.Name -like "*\$Auth" -or $Member.Name -eq "$env:COMPUTERNAME\$Auth") {
 $Match = $true
 break
 }
 }

 if (-not $Match) {
 Write-Host " - VULNERABLE: Unauthorized account '$($Member.Name)' (Source: $($Member.PrincipalSource)) has administrative access." -ForegroundColor Red
 } else {
 Write-Host " - Member: $($Member.Name) | Source: $($Member.PrincipalSource) | Class: $($Member.ObjectClass) (Authorized)" -ForegroundColor Green
 }
 }
} else {
 Write-Error "Failed to retrieve local Administrators group members."
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendations on administrative isolation and local admin restriction. \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*: Section 5.5 (Ensure only authorized accounts are members of the Administrators group) \* \*\*Microsoft Security Baselines\*\*: Restricted Groups settings for high-security domain systems.

# [REQ-PAW-004] Enforce BitLocker with TPM and Startup PIN for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Tier 0 Privileged Access Workstations (PAWs). \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* Computer Configuration\Administrative Templates\Windows Components\BitLocker Drive Encryption \* Computer Configuration\Administrative Templates\Windows Components\BitLocker Drive Encryption\Operating System Drives \* Computer Configuration\Administrative Templates\System\Power Management\Sleep Settings \* Computer Configuration\Administrative Templates\System\Kernel DMA Protection \* HKLM\SOFTWARE\Policies\Microsoft\FVE \* HKLM\SOFTWARE\Policies\Microsoft\Power\PowerSettings\abfc251b-215d-4f10-ae40-e226dbe3c6a3 \* HKLM\SOFTWARE\Policies\Microsoft\Windows\KernelDMAProtection

---

## Rationale Privileged Access Workstations (PAWs) serve as the secure root of trust for administering Tier 0 Active Directory resources. Because these devices are physical endpoints, they are susceptible to theft, loss, and unauthorized physical access.

To achieve maximum protection, the PAW BitLocker configuration enforces a significantly more stringent baseline than standard client endpoints:

1. **TPM and Startup PIN:** Enforcing a pre-boot Startup PIN combined with TPM validation ensures that the drive cannot be unlocked or booted without explicit administrator presence. Network Unlock is prohibited on PAWs to prevent automatic decryption when connected to a local switch, ensuring physical presence verification is mandatory for every boot.
2. **Disabling Sleep/Standby States (S1-S3):** When a system enters a standby/sleep state, the BitLocker volume decryption keys remain stored in the volatile memory (RAM). An attacker with brief physical access to a sleeping PAW can exploit Direct Memory Access (DMA) interfaces (such as Thunderbolt, FireWire, or PCIe slots) or perform a cold-boot attack to extract the decryption keys directly from the RAM. Disabling S1-S3 standby states forces the system to either shut down (S5) or hibernate (S4), writing the RAM contents back to the encrypted disk and purging the keys from volatile memory.
3. **Kernel DMA Protection:** This blocks peripheral devices (Thunderbolt, PCIe) from initiating DMA requests unless the OS is fully booted, authorized, and running driver-level Input-Output Memory Management Unit (IOMMU) protection, preventing DMA memory extraction during the pre-boot and OS load phases.
4. **Enhanced Startup PINs:** This allows administrators to use alphanumeric characters, symbols, uppercase and lowercase letters, and spaces in their pre-boot Startup PIN rather than just numbers, increasing entropy and resistance to PIN-guessing attacks.
5. **Active Directory Backup & Recovery Password Rotation:** Ensures all BitLocker recovery keys are automatically backed up to Active Directory before encryption begins. In addition, when a recovery key is used to unlock a PAW, it must be automatically rotated and updated in Active Directory to prevent the reuse of compromised recovery keys.

---

## Legacy Impact & Compatibility \* \*\*Administrator Overhead\*\*: PAW administrators must enter the Startup PIN on every boot and every resume from hibernation. \* \*\*Hibernation Support\*\*: Hardware must support hibernation, and it must be enabled on the operating system ('powercfg /h on'). \* \*\*Resume Time\*\*: Waking the workstation from hibernation or cold boot takes slightly longer than resuming from standard standby sleep, but this latency is necessary to satisfy the Tier 0 threat model. \* \*\*Hardware Pre-requisites\*\*: A physical TPM 2.0 chip and motherboard support for Kernel DMA Protection (IOMMU) are required.

---

## ## Implementation Steps

### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### Step 1: Enforce BitLocker Encryption Strength 1. Open the \*\*Group Policy Management Console\*\* ('gpmc.msc'). 2. Edit the GPO linked to the PAWs Organizational Unit (e.g., 'GPO\_Hardening\_PAW'). 3. Navigate to: 'Computer Configuration\Administrative Templates\Windows Components\BitLocker Drive Encryption' 4. Configure the following setting: \* \*\*Policy\*\*: 'Choose drive encryption method and cipher strength (Windows 10 [Version 1511] and later)' \* \*\*Setting\*\*: 'Enabled' \* \*\*Select the encryption method\*\*: 'XTS-AES 256-bit' (for Operating System drives)

##### Step 2: Enforce TPM + Startup PIN and Enhanced PIN Policies 1. In the same GPO, navigate to: 'Computer Configuration\Administrative Templates\Windows Components\BitLocker Drive Encryption\Operating System Drives' 2. Configure the following settings: \* \*\*Policy\*\*: 'Require additional authentication at startup' \* \*\*Setting\*\*: 'Enabled' \* \*\*Configure Options\*\*: \* Set 'Configure TPM startup': 'Require TPM' \* Set 'Configure TPM startup PIN': 'Require startup PIN with TPM' \* Set 'Configure TPM startup key': 'Do not allow startup key with TPM' \* Set 'Configure TPM startup key and PIN': 'Do not allow startup key and PIN with TPM' \* Check 'Allow BitLocker without a compatible TPM': 'Disabled' \* \*\*Policy\*\*: 'Configure use of enhanced PINs for startup' \* \*\*Setting\*\*: 'Enabled' \* \*\*Policy\*\*: 'Minimum PIN length for startup' \* \*\*Setting\*\*: 'Enabled' \* \*\*Minimum characters\*\*: '8' (or higher depending on local organizational policy)

##### Step 3: Configure Active Directory Backup and Key Rotation 1. In the same OS Drives folder, configure the following settings: \* \*\*Policy\*\*: 'Choose how BitLocker-protected operating system drives can be recovered' \* \*\*Setting\*\*: 'Enabled' \* \*\*Configure Options\*\*: \* Check 'Allow data recovery agent' \* Check 'Save BitLocker recovery information to Active Directory Domain Services' \* Set 'Configure user storage of BitLocker recovery information': 'Store recovery passwords and key packages' \* Check 'Do not enable BitLocker until recovery information is stored in AD DS for operating system drives' \* \*\*Policy\*\*: 'Configure recovery password rotation for AD DS-joined computers' \* \*\*Setting\*\*: 'Enabled' \* \*\*Rotation options\*\*: 'Must rotate the recovery password for OS drives and fixed data drives'

##### Step 4: Disable Sleep/Standby States (S1-S3) 1. In the GPO, navigate to: 'Computer Configuration\Administrative Templates\System\Power Management\Sleep Settings' 2. Configure the following settings: \* \*\*Policy\*\*: 'Allow standby states (S1-S3) when sleeping (plugged in)' \* \*\*Setting\*\*: 'Disabled' \* \*\*Policy\*\*: 'Allow standby states (S1-S3) when sleeping (on battery)' \* \*\*Setting\*\*:

`Disabled`

##### Step 5: Enable Kernel DMA Protection 1. In the GPO, navigate to: `Computer Configuration\Administrative Templates\System\Kernel DMA Protection` 2. Configure the following setting: \* \*\*Policy\*\*\*: `Enable Kernel DMA Protection` \* \*\*Setting\*\*\*: `Enabled`

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally on the PAW to apply registry configuration baselines and enable BitLocker.

[Download Script: Set-PAWBitLockerEncryption.ps1](#)



```
Set-PAWBitLockerEncryption.ps1
Configures registry settings for PAW BitLocker, disables sleep states, and enables encryption.

Write-Host "--- Enforcing Stringent PAW BitLocker Baseline ---" -ForegroundColor Cyan

1. Enforce encryption strength (XTS-AES 256 = 7)
$FvePath = "HKLM:\SOFTWARE\Policies\Microsoft\FVE"
if (-not (Test-Path $FvePath)) {
 New-Item -Path $FvePath -Force | Out-Null
}
Set-ItemProperty -Path $FvePath -Name "EncryptionMethodWithXts0s" -Value 7 -Type DWord

2. Configure TPM + Startup PIN, AD Backup, and Enhanced PINs in registry
Set-ItemProperty -Path $FvePath -Name "UseAdvancedStartup" -Value 1 -Type DWord
Set-ItemProperty -Path $FvePath -Name "EnableNonTpm" -Value 0 -Type DWord
Set-ItemProperty -Path $FvePath -Name "UseTPM" -Value 2 -Type DWord # 2 = Require
Set-ItemProperty -Path $FvePath -Name "UseTPMPIN" -Value 2 -Type DWord # 2 = Require
Set-ItemProperty -Path $FvePath -Name "UseEnhancedPINs" -Value 1 -Type DWord
Set-ItemProperty -Path $FvePath -Name "MinPINLength" -Value 8 -Type DWord
Set-ItemProperty -Path $FvePath -Name "OSRecovery" -Value 1 -Type DWord
Set-ItemProperty -Path $FvePath -Name "OSRecoveryPassword" -Value 1 -Type DWord
Set-ItemProperty -Path $FvePath -Name "OSBackupSaveSource" -Value 1 -Type DWord
Set-ItemProperty -Path $FvePath -Name "OSActiveDirectoryBackup" -Value 1 -Type DWord
Set-ItemProperty -Path $FvePath -Name "OSRequireActiveDirectoryBackup" -Value 1 -Type DWord
Set-ItemProperty -Path $FvePath -Name "OSRecoveryPasswordRotation" -Value 1 -Type DWord # 1 = Enforce rotation

3. Disable Sleep States S1-S3 via GPO Registry override
$PowerSleepPath = "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\abfc251b-215d-4f10-ae40-e226dbe3c6a3"
if (-not (Test-Path $PowerSleepPath)) {
 New-Item -Path $PowerSleepPath -Force | Out-Null
}
Set-ItemProperty -Path $PowerSleepPath -Name "ACSettingIndex" -Value 0 -Type DWord
Set-ItemProperty -Path $PowerSleepPath -Name "DCSettingIndex" -Value 0 -Type DWord

Enforce hibernate locally using powercfg
powercfg /hibernate on
Write-Host "[+] Sleep states S1-S3 disabled, and Hibernation enabled." -ForegroundColor Green

4. Enable Kernel DMA Protection in registry
$DmaPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\KernelDMAProtection"
if (-not (Test-Path $DmaPath)) {
 New-Item -Path $DmaPath -Force | Out-Null
}
Set-ItemProperty -Path $DmaPath -Name "DeviceEnumerationPolicy" -Value 0 -Type DWord
Write-Host "[+] Kernel DMA Protection registry configuration applied." -ForegroundColor Green

5. Enable BitLocker on C: drive using TPM and Startup PIN
$Volume = Get-BitLockerVolume -MountPoint "C:" -ErrorAction SilentlyContinue
if ($Volume.ProtectionStatus -eq "Off") {
 Write-Host "[+] Activating BitLocker on C: volume..." -ForegroundColor Gray

 # We must first define a temporary PIN to enable startup PIN protection programmatically
 # The administrator must change this PIN immediately on next reboot
 $TempPin = "P@ssw0rdPIN1"
 $SecurePin = New-Object System.Security.SecureString
 foreach ($Char in $TempPin.ToCharArray()) {
 $SecurePin.AppendChar($Char)
 }

 Enable-BitLocker -MountPoint "C:" `
 -EncryptionMethod XtsAes256 `
 -UsedSpaceOnly `
 -Pin $SecurePin `
 -TpmAndPinProtector `
 -AdBackupRequired

 Write-Host "[+] BitLocker initiated with TPM and Startup PIN. Recovery keys sent to AD." -ForegroundColor Green
} else {
 Write-Host "[+] BitLocker is already enabled on C: (Protection Status: $($Volume.ProtectionStatus))." -ForegroundColor Green
}
}
```

To audit the PAW BitLocker status and security parameters: [Download Script: Test-PAWBitLockerStatus.ps1](#)

```
Test-PAWBitLockerStatus.ps1
Audits current BitLocker configuration, active protectors, sleep state, and DMA protection.

Write-Host "--- Auditing PAW BitLocker Security Parameters ---" -ForegroundColor Cyan

1. Query BitLocker protection and key protector types
$Volume = Get-BitLockerVolume -MountPoint "C:" -ErrorAction SilentlyContinue
if ($Volume) {
 $StatusColor = if ($Volume.ProtectionStatus -eq "On") { "Green" } else { "Red" }
 Write-Host " - Protection Status: $($Volume.ProtectionStatus)" -ForegroundColor $StatusColor
 Write-Host " - Encryption Method: $($Volume.EncryptionMethod)" -ForegroundColor White

 $HasTpmPin = $false
 foreach ($Protector in $Volume.KeyProtector) {
 if ($Protector.KeyProtectorType -eq "TpmAndPin") {
 $HasTpmPin = $true
 }
 Write-Host " - Active Protector: $($Protector.KeyProtectorType)" -ForegroundColor White
 }

 if ($HasTpmPin) {
 Write-Host " [+] TPM and Startup PIN is ACTIVE." -ForegroundColor Green
 } else {
 Write-Host " [-] TPM and Startup PIN is MISSING." -ForegroundColor Red
 }
} else {
 Write-Error "BitLocker volume information could not be retrieved."
}

2. Check Sleep State S1-S3 status
$SleepVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\abfc251b-215d-4f10-ae40-e226dbe3c6a3" -Name "ACSettingIndex" -ErrorAction SilentlyContinue
if ($SleepVal -and $SleepVal.ACSettingIndex -eq 0) {
 Write-Host " [+] Standby Sleep States (S1-S3) are disabled." -ForegroundColor Green
} else {
 Write-Host " [-] Standby Sleep States (S1-S3) are enabled (Risk of DMA attack)." -ForegroundColor Red
}

3. Check Kernel DMA Protection
$DmaVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\KernelDMAProtection" -Name "DeviceEnumerationPolicy" -ErrorAction SilentlyContinue
if ($DmaVal -and $DmaVal.DeviceEnumerationPolicy -eq 0) {
 Write-Host " [+] Kernel DMA Protection is enabled." -ForegroundColor Green
} else {
 Write-Host " [-] Kernel DMA Protection is disabled." -ForegroundColor Red
}
}
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R58 (Use of Privileged Access Workstations), Recommendation R9 (LAPS context for BitLocker recovery keys protection). \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*: Section 18.2.1.1 (Require additional authentication at startup), Section 18.2.1.2 (Configure use of enhanced PINs for startup), Section 18.2.1.3 (Configure minimum PIN length for startup), Section 18.2.1.4 (Configure recovery password rotation). \* \*\*Microsoft Security Guidelines\*\*: Device Guard and DMA protection reference architecture guides.

## # [REQ-PAW-005] UEFI Firmware Security Hardening

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* UEFI Firmware Configuration Menu \*

HKLM\SYSTEM\CurrentControlSet\Control \* `PEFirmwareType` = `2` (REG\_DWORD) \*

HKLM\SYSTEM\CurrentControlSet\Control\SecureBoot\State \* `UEFISecureBootEnabled` = `1` (REG\_DWORD)

## Rationale Privileged Access Workstations (PAWs) form the administrative root of trust for the Active Directory forest. If an attacker gains physical access to a PAW, they can attempt to compromise the operating system offline, bypass disk encryption, or load malicious code prior to the OS boot phase.

Securing the firmware level ensures:

1. **Firmware Lockdown:** Setting a strong UEFI administrator password prevents unauthorized local users or attackers with physical access from disabling hardware security configurations (such as TPM 2.0, Secure Boot, or virtualization extensions).
2. **Boot Integrity:** Disabling the Compatibility Support Module (CSM) or Legacy BIOS options forces native UEFI mode, which is a hard prerequisite for UEFI Secure Boot and Virtualization-Based Security (VBS).
3. **Execution Prevention:** Restricting the boot order to the primary internal OS drive prevents booting from unauthorized external media (USB flash drives, external SSDs, or local network PXE servers) containing diagnostics, password reset tools, or malicious secondary operating systems.
4. **Downgrade Attack Mitigation:** Restricting firmware rollbacks prevents attackers from flashing older, vulnerable firmware versions that might contain known UEFI security bypasses.
5. **Virtualization-Based Security Foundation:** Enabling CPU Virtualization Extensions (Intel VT-x / AMD-V) and IOMMU (Intel VT-d / AMD-Vi) at the firmware level establishes the mandatory hardware isolation required by the Windows Hypervisor to run VBS, Credential Guard, and Kernel DMA Protection.
6. **Platform Measurement Integrity:** Disabling Fast Boot forces the firmware to execute full hardware initialization, device checks, and complete TPM self-tests/PCR measurements at every boot, ensuring platform integrity and correct state validation.

## Legacy Impact & Compatibility \* \*\*Administrative Overhead\*\*: Technicians must enter the UEFI administrator password to make hardware changes or perform local diagnostics. This password must be securely generated and stored in a central, encrypted vault. \* \*\*Legacy OS Incompatibility\*\*: Operating systems or recovery environments that do not support native UEFI boot will fail to start. This is acceptable as PAWs must only run modern, authorized Windows Enterprise installations. \* \*\*Partition Format\*\*: Converting an existing Legacy BIOS installation to UEFI requires repartitioning the primary storage device from Master Boot Record (MBR) to GUID Partition Table (GPT) using utility tools like MBR2GPT.exe.

## ## Implementation Steps

## ### Option A: Manual UEFI Firmware Configuration (Preferred)

UEFI settings must be configured directly within the hardware platform firmware interface during system startup.

1. Turn on or restart the workstation and access the UEFI utility screen by pressing the vendor-specific key during POST (typically Delete, F2, F10, or F12).
2. Navigate to the **Security** or **Authentication** section:
  - Select the option to set the **Administrator Password** (also referred to as the **Supervisor Password**). Do not configure a User Password, as that prompts for authentication on every boot rather than only when entering configuration settings.
  - Enter a strong, complex password. Record this password in the team's secure credential repository.
3. Navigate to the **Boot** or **System Configuration** section:
  - Locate the **Boot Mode** setting and set it to **UEFI Only** or **Native UEFI**.
  - Locate **CSM (Compatibility Support Module)** or **Legacy Boot Support** and set it to **Disabled**.
  - Locate **Fast Boot** or **Quick Boot** and set it to **Disabled** (forcing complete POST diagnostics and full TPM initialization on every boot).
  - Locate **Boot Order** (or **Boot Priority**):
    - Set the primary boot option to the internal system storage drive (typically containing the Windows Boot Manager partition).
    - Disable all other boot options (such as USB, SD Card, Optical Drive, and Network PXE Boot) or set them to disabled in the boot menu.
    - Enable the option to prompt for the UEFI administrator password if a user attempts to access the boot override menu (typically F12 or F8).
4. Navigate to the **Advanced**, **CPU Configuration**, or **Security Chip** section:
  - Locate **Intel Virtualization Technology (VT-x)** or **AMD-V** and set it to **Enabled**.
  - Locate **Intel VT for Directed I/O (VT-d)** or **AMD IOMMU** and set it to **Enabled** (required for IOMMU/Kernel DMA Protection).
  - Locate **TPM 2.0 Device** (or **Security Chip / Intel PTT / AMD fTPM**) and set it to **Enabled** or **Active** (with SHA-256 PCR bank).

- Locate **Memory Overwrite Request Control Lock** (or **MOR Lock**) and set it to **Enabled**.
5. Navigate to the **Security** or **Secure Boot** section:
- Ensure **Secure Boot** is **Enabled** and the **Secure Boot Mode** is set to **Deployed** or **User Mode**.
  - Harden the certificates allowlist:
    - **Key Exchange Key (KEK)**: Must only contain "Microsoft Corporation KEK CA 2011" and "Microsoft Corporation KEK 2K CA 2023".
    - **Signature Database (db)**: Must only contain "Microsoft Windows Production PCA 2011" and "Windows UEFI CA 2023". Remove "Microsoft UEFI CA 2011" and "Microsoft Option ROM UEFI CA 2023" unless strictly required by specific physical PCIe expansion hardware.
6. Navigate to the **Advanced** or **Firmware Update** section:
- Locate the option for **BIOS Flash Protection** or **Firmware Rollback Protection** and set it to **Enabled** or **Block Downgrades**.
7. Save the configuration and restart the workstation.

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Since firmware password and boot order configurations are set at the hardware level, they cannot be directly configured from within the Windows operating system. However, the system's UEFI boot environment and BIOS specifications must be programmatically audited.

Run the following script to verify the native boot mode, Secure Boot support, and retrieve BIOS vendor information:

[Download Script: Audit-UEFIsecurity.ps1](#)

```
Audit-UEFIsecurity.ps1
Description: Audits local boot environment and BIOS firmware properties.

Write-Host "--- Auditing UEFI Security Baseline ---" -ForegroundColor Cyan

1. Verify boot environment type
$RegPath = "HKLM:\System\CurrentControlSet\Control"
$FirmwareProperty = Get-ItemProperty -Path $RegPath -Name "PEFirmwareType" -ErrorAction SilentlyContinue

if ($FirmwareProperty) {
 $FirmwareValue = $FirmwareProperty.PEFirmwareType
 if ($FirmwareValue -eq 2) {
 Write-Host "Status: Native UEFI mode is active." -ForegroundColor Green
 } else {
 Write-Host "VULNERABLE: System booted in Legacy BIOS mode (CSM enabled). Value: $($FirmwareValue)" -ForegroundColor Red
 }
} else {
 Write-Host "VULNERABLE: Boot environment type could not be read from registry." -ForegroundColor Red
}

2. Audit Secure Boot status
try {
 $SecureBootActive = Confirm-SecureBootUEFI -ErrorAction Stop
 if ($SecureBootActive -eq $true) {
 Write-Host "Status: UEFI Secure Boot is enabled." -ForegroundColor Green
 } else {
 Write-Host "VULNERABLE: UEFI Secure Boot is supported but disabled in firmware." -ForegroundColor Red
 }
} catch [System.PlatformNotSupportedException] {
 Write-Host "VULNERABLE: UEFI Secure Boot is not supported on this platform." -ForegroundColor Red
} catch {
 Write-Host "VULNERABLE: UEFI Secure Boot validation failed. Error: ($_.Exception.Message)" -ForegroundColor Red
}

3. Retrieve BIOS details
$BiosDetails = Get-CimInstance -ClassName Win32_Bios -ErrorAction SilentlyContinue
if ($BiosDetails) {
 Write-Host "Firmware Manufacturer: $($BiosDetails.Manufacturer)" -ForegroundColor White
 Write-Host "Firmware Version: $($BiosDetails.SMBIOSBIOSVersion)" -ForegroundColor White
 Write-Host "Firmware Release Date: $($BiosDetails.ReleaseDate)" -ForegroundColor White
} else {
 Write-Host "Warning: BIOS details could not be retrieved via WMI." -ForegroundColor Yellow
}
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R58 (Use of Privileged Access Workstations) \*

**\*\*CIS Microsoft Windows 10/11 Benchmark\*\***: Section 18.8 (Device Guard/VBS prerequisites) \* **\*\*Microsoft Security Guidelines\*\***: UEFI Firmware Security and Device Guard Deployment

# [REQ-PAW-006] Enable Hardware Virtualization and DMA Protection

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: Computer Configuration\Administrative Templates\System\Kernel DMA Protection \* \*\*Registry Location\*\*: HKLM\SOFTWARE\Policies\Microsoft\Windows\KernelDMAProtection \* `DeviceEnumerationPolicy` = `0` (REG\_DWORD)

## Rationale Virtualization-Based Security (VBS) and Windows Defender Credential Guard isolate sensitive security processes (like LSA) inside a hardware-virtualized container to prevent memory dumping and credential harvesting. However, these OS-level security boundaries are entirely reliant on hardware-level protections.

Enabling hardware virtualization and DMA protection guarantees:

1. **Isolated Execution Environment:** Enforcing CPU Virtualization Extensions (Intel VT-x or AMD-V) in the UEFI allows the hypervisor to isolate the VBS secure kernel from the host Windows operating system.
2. **Physical DMA Protection:** Enforcing IOMMU (Intel VT-d or AMD-Vi) at the firmware level enables Kernel DMA Protection. This blocks malicious peripherals (e.g., PCIe cards or Thunderbolt devices) from executing unauthorized Direct Memory Access (DMA) attacks to read or write to host system memory, preventing attackers from extracting BitLocker keys or credential secrets directly from RAM.
3. **Hardware Root of Trust:** Activating the Trusted Platform Module (TPM) 2.0 enables cryptographic boot measurement logging (PCR banks). TPM 2.0 ensures that the system boot configuration has not been modified prior to unsealing the BitLocker volume decryption keys.

## Legacy Impact & Compatibility \* \*\*Hardware Baseline\*\*: Systems must support CPU virtualization, Second Level Address Translation (SLAT), and input-output memory management (IOMMU). Unsupported hardware will prevent VBS and Credential Guard from initiating. \* \*\*Peripheral Compatibility\*\*: Older or non-certified Thunderbolt/PCIe devices that do not support DMA-remapping (DMA routing checks) may be blocked from functioning when Kernel DMA Protection is active. This is acceptable for a PAW, where external hardware connections must be strictly restricted. \* \*\*Nested Virtualization\*\*: Enabling hardware virtualization for VBS may conflict with older third-party hypervisors (such as legacy versions of VirtualBox or VMware Workstation) that do not support nesting under Microsoft Hyper-V.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

To enforce Kernel DMA Protection across the PAW infrastructure, implement the following GPO settings:

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the GPO linked to the PAWs Organizational Unit (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Administrative Templates\System\Kernel DMA Protection`
4. Configure the following setting:
  - **Policy:** `Enable Kernel DMA Protection`
  - **Setting:** `Enabled`
5. Link the GPO to the appropriate OU containing target PAWs.

*Note: Enforce hardware-level CPU Virtualization (VT-x/AMD-V), IOMMU (VT-d/AMD-Vi), and TPM 2.0 manually in the UEFI menu of each device.*

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Configure local registry keys to enforce Kernel DMA Protection and programmatically audit the hardware security baseline.

#### 1. Local Remediation (Enforce Kernel DMA Protection)

Run the following script to enforce the DMA Protection policy locally:

[Download Script: Configure-KernelDMAProtection.ps1](#)

```
Configure-KernelDMAProtection.ps1
Description: Configures registry keys to enable Kernel DMA Protection.

Write-Host "--- Enforcing Kernel DMA Protection ---" -ForegroundColor Cyan

$RegPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\KernelDMAProtection"
if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

DeviceEnumerationPolicy = 0 (Block all DMA until user logs on)
Set-ItemProperty -Path $RegPath -Name "DeviceEnumerationPolicy" -Value 0 -Type DWord
Write-Host "Status: Kernel DMA Protection registry configuration applied." -ForegroundColor Green
```

#### #### 2. Local Audit (TPM, Virtualization, and DMA Support)

Run the following script to audit the status of the required hardware security components:

[Download Script: Audit-HardwareSecurityFeatures.ps1](#)

```
Audit-HardwareSecurityFeatures.ps1
Description: Audits TPM 2.0, CPU Virtualization, and IOMMU/DMA status.

Write-Host "--- Auditing Hardware Security Features ---" -ForegroundColor Cyan

1. Audit TPM 2.0 Status
$Tpm = Get-Tpm -ErrorAction SilentlyContinue
if ($Tpm) {
 if ($Tpm.TpmPresent -eq $true) {
 $TpmColor = "Red"
 if ($Tpm.TpmReady -eq $true) {
 $TpmColor = "Green"
 }
 Write-Host "Status: TPM Present: $($Tpm.TpmPresent) | Ready: $($Tpm.TpmReady)" -ForegroundColor $TpmColor
 } else {
 Write-Host "VULNERABLE: TPM 2.0 is not detected on this system." -ForegroundColor Red
 }
} else {
 Write-Host "VULNERABLE: TPM verification cmdlet failed." -ForegroundColor Red
}

2. Audit VBS and DMA Status via Win32_DeviceGuard
try {
 $DG = Get-CimInstance -Namespace "Root\Microsoft\Windows\DeviceGuard" -ClassName "Win32_DeviceGuard" -ErrorAction Stop

 # VirtualizationBasedSecurityStatus: 2 = Running
 $VbsStatus = $DG.VirtualizationBasedSecurityStatus
 $VbsColor = "Red"
 if ($VbsStatus -eq 2) {
 $VbsColor = "Green"
 }
 Write-Host "Status: Virtualization-Based Security Status: $($VbsStatus) (Required = 2 [Running])" -ForegroundColor $VbsColor

 # AvailableSecurityProperties: 3 = DMA Protection (IOMMU)
 $DmaSupported = $DG.AvailableSecurityProperties -contains 3
 $DmaColor = "Red"
 if ($DmaSupported -eq $true) {
 $DmaColor = "Green"
 }
 Write-Host "Status: Hardware IOMMU/DMA Protection: $($DmaSupported)" -ForegroundColor $DmaColor

 # RequiredSecurityProperties: 3 = DMA Protection enforced
 $DmaEnforced = $DG.RequiredSecurityProperties -contains 3
 $EnforcedColor = "Red"
 if ($DmaEnforced -eq $true) {
 $EnforcedColor = "Green"
 }
 Write-Host "Status: DMA Protection Policy Enforced: $($DmaEnforced)" -ForegroundColor $EnforcedColor
} catch {
 Write-Host "VULNERABLE: Win32_DeviceGuard WMI class could not be queried. VBS is likely inactive." -ForegroundColor Red
}
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R58 (Use of Privileged Access Workstations) \*  
 \*\*CIS Microsoft Windows 10/11 Benchmark\*\*: Section 18.8.19.1 (Configure Enable Kernel DMA Protection), Section 18.8.14.1 (Turn On  
 Virtualization Based Security) \* \*\*Microsoft Security Guidelines\*\*: Kernel DMA Protection reference architecture



# [REQ-PAW-007] Disable Windows Platform Binary Table (WPBT)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* Computer Configuration\Preferences\Windows Settings\Registry \* HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\DisableWpbtExecution

## Rationale The Windows Platform Binary Table (WPBT) is an ACPI firmware table that allows hardware manufacturers (OEMs) to execute proprietary binaries in kernel space during the Windows boot phase. Windows automatically extracts the binary from the table and runs it with system privileges before security software, third-party agents, or standard driver verifications are fully initialized.

While designed to facilitate automated driver provisioning and anti-theft services, this mechanism represents a significant security risk:

1. **Firmware-to-OS Attack Vector:** Malicious actors utilizing UEFI rootkits, physical firmware flashing tools, or supply-chain firmware implants can compromise the WPBT table to execute arbitrary code at boot, bypassing Secure Boot and operating system-level integrity checks.
2. **Privilege Escalation Risks:** Historically, OEM software delivered via the WPBT has introduced high-severity local privilege escalation and remote code execution vulnerabilities due to inadequate code review or poor permission management.
3. **Control and Transparency:** Executing firmware-rooted binaries without administrative visibility or operating system validation bypasses normal software lifecycle and endpoint protection policies.

Disabling WPBT execution prevents Windows from parsing the ACPI table and running the embedded software, mitigating boot-level integrity bypasses.

## Legacy Impact & Compatibility \* \*\*OEM Software Functionality\*\*: Disabling the WPBT will stop manufacturer-embedded software (such as automated support assistants, system registration tools, or OEM-specific recovery software) from installing on a fresh OS deployment. System installation pipelines must manually deploy any validated, business-essential hardware utility packages rather than relying on automatic firmware injection. \* \*\*Deployment Timing\*\*: The registry setting 'DisableWpbtExecution' must be present prior to the initial Windows boot sequence to completely block WPBT payload execution on a newly installed OS. Applying the registry key via GPO will prevent subsequent runs or updates but will not retroactively clean up files that were already executed during the initial setup. For maximum protection, this registry modification should be integrated directly into reference installation media (e.g., via 'autounattend.xml' or custom WIM injection).

## ## Implementation Steps

### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

Because there is no default ADMX administrative template to manage WPBT execution, the setting must be configured as a Registry Preference under the PAW policy:

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a domain management host.
2. Edit the GPO linked to your PAWs Organizational Unit (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Right-click **Registry**, select **New → Registry Item**.
5. Configure the following properties:
  - **Action:** `Update`
  - **Hive:** `HKEY_LOCAL_MACHINE`
  - **Key Path:** `SYSTEM\CurrentControlSet\Control\Session Manager`
  - **Value name:** `DisableWpbtExecution`
  - **Value type:** `REG_DWORD`
  - **Value data:** `1`

6. Click **OK** to save the preference.

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script to configure the registry setting locally on the system:

[Download Script: Configure-DisableWpbt.ps1](#)

```
Configure-DisableWpbt.ps1
Description: Disables Windows Platform Binary Table (WPBT) execution in the registry.

Write-Host "Applying hardening requirement: Disable WPBT Execution..." -ForegroundColor Cyan

$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager"
$ValueName = "DisableWpbtExecution"
$ValueData = 1

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value $ValueData -Type DWord
Write-Host "Registry setting DisableWpbtExecution configured to 1." -ForegroundColor Green
```

*To verify that the registry value is correctly enforced:*

[Download Script: Get-WpbtStatus.ps1](#)

```
Get-WpbtStatus.ps1
Description: Audits the registry state for WPBT execution prevention.

Write-Host "--- Auditing WPBT Security Posture ---" -ForegroundColor Cyan

$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager"
$ValueName = "DisableWpbtExecution"

$RegistryValue = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue

if ($RegistryValue) {
 $Setting = $RegistryValue.DisableWpbtExecution
 if ($Setting -eq 1) {
 Write-Host "Status: WPBT execution is disabled (DisableWpbtExecution = 1)." -ForegroundColor Green
 } else {
 Write-Host "VULNERABLE: WPBT execution is enabled. Value is $($Setting)." -ForegroundColor Red
 }
} else {
 Write-Host "VULNERABLE: DisableWpbtExecution registry value is not configured (defaulting to execution enabled)." -ForegroundColor Red
}
}
```

---

## Sources & Compliance References \* **\*\*ANSI AD Hardening Guide\*\***: Recommendations regarding hardware platform integrity. \*  
**\*\*Microsoft Windows Security\*\***: Device Guard and UEFI Platform Security guidelines.

# Windows Defender Antivirus PAW Baseline and Exploit Guard

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus

---

## Rationale Privileged Access Workstations (PAWs) represent the highest security boundary on the endpoint layer, serving as isolated systems dedicated solely to Tier 0 directory administration. If a PAW is compromised, the entire AD forest is compromised. Therefore, the built-in antimalware and exploit prevention controls must be hardened to their absolute maximum threshold. This submodule contains individual requirement controls for each advanced protective mechanism and configuration parameter within Windows Defender Antivirus for PAWs.

---

## Legacy Impact & Compatibility \* \*\*Administrative Operations\*\*: Enabling the WMI/PSEXec block rule means administrative scripts must be run locally or orchestrated via secure WinRM endpoints. Traditional PSEXec commands from remote management consoles will be blocked, enforcing proper tier-isolated remote administration. \* \*\*Execution Restrictions\*\*: Since productivity suites (e.g., Office, Outlook) are strictly banned from PAWs, ASR rules targeting Microsoft Office applications are enforced as a defensive measure to prevent shadow installations or bypasses. \* \*\*AMSI Provider Signatures\*\*: Any third-party security agents registering as AMSI providers must have a valid, trusted Authenticode signature. Unsigned or self-signed providers will be prevented from loading.

---

## Defender Hardening Requirements for PAWs

The following Defender Antivirus configurations must be enforced on PAWs:

1. [REQ-PAW-057 - Disable Real-Time Monitoring and Behavior Monitoring Override for PAWs](#)
  2. [REQ-PAW-058 - Configure Potentially Unwanted Applications \(PUA\) Protection for PAWs](#)
  3. [REQ-PAW-059 - Prevent Local List Merging and Exclusions Configuration for PAWs](#)
  4. [REQ-PAW-060 - Configure Auto Exclusions Configuration for PAWs](#)
  5. [REQ-PAW-061 - Enable EDR in Block Mode for PAWs](#)
  6. [REQ-PAW-062 - Allow Network Protection on Windows Server for PAWs](#)
  7. [REQ-PAW-063 - Enable File Hash Computation for PAWs](#)
  8. [REQ-PAW-064 - Configure Network Inspection System \(NIS\) settings for PAWs](#)
  9. [REQ-PAW-065 - Configure OOBE Real-Time Protection and Security Intelligence for PAWs](#)
  10. [REQ-PAW-066 - Enable Dynamic Signature Dropped Event Reporting for PAWs](#)
  11. [REQ-PAW-067 - Configure Quick Scan and Scanning Exclusions for PAWs](#)
  12. [REQ-PAW-068 - Configure Scheduled Scan Parameters for PAWs](#)
  13. [REQ-PAW-069 - Configure Security Intelligence Update Schedule for PAWs](#)
  14. [REQ-PAW-070 - Configure Attack Surface Reduction Rules for PAWs](#)
  15. [REQ-PAW-071 - Configure Threat Severity Default Quarantine Actions for PAWs](#)
  16. [REQ-PAW-072 - Configure Family Options UI Lockdown for PAWs](#)
  17. [REQ-PAW-073 - Configure Tamper Protection for PAWs](#)
  18. [REQ-PAW-074 - Configure Sandbox Execution Environment for PAWs](#)
  19. [REQ-PAW-075 - Configure AMSI Authenticode Signature Verification for PAWs](#)
- 

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-057] Disable Real-Time Monitoring and Behavior Monitoring Override for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Real-time Protection` \* \*\*Registry Location\*\*: \*  
 `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` \* `DisableRealtimeMonitoring` = `0` (REG\_DWORD) \*  
 `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` \* `DisableBehaviorMonitoring` = `0` (REG\_DWORD) \*  
 `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` \* `DisableIOAVProtection` = `0` (REG\_DWORD) \*  
 `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` \* `DisableScriptScanning` = `0` (REG\_DWORD)

---

## Rationale Real-time scanning, behavior monitoring, and script checking are the core dynamic defense mechanisms of Windows Defender. Disabling or bypassing these controls allows malicious scripts, file-based attacks, and unauthorized in-memory activities to execute undetected.

---

## Legacy Impact & Compatibility Negligible operational impact. Real-time scanning introduces standard CPU overhead during file read/write operations.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Real-time Protection 2. Set 'Turn off real-time protection' to 'Disabled' 3. Set 'Turn on behavior monitoring' to 'Enabled' 4. Set 'Scan all downloaded files and attachments' to 'Enabled' 5. Set 'Turn on script scanning' to 'Enabled'

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderRtp.ps1](#)] ([../implementation\\_scripts/Configure-PawDefenderRtp.ps1](#))

```
Configure-PawDefenderRtp.ps1
Set-MpPreference -DisableRealtimeMonitoring $false
Set-MpPreference -DisableBehaviorMonitoring $false
Set-MpPreference -DisableIOAVProtection $false
Set-MpPreference -DisableScriptScanning $false
```

To audit the hardening status: [Download Script: Get-PawDefenderRtpStatus.ps1](#)

```
Get-PawDefenderRtpStatus.ps1
$Pref = Get-MpPreference
if ($Pref.DisableRealtimeMonitoring -eq $false -and $Pref.DisableBehaviorMonitoring -eq $false -and $Pref.DisableIOAVProtection -eq $false -and $Pref.DisableScriptScanning -eq $false) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-058] Configure Potentially Unwanted Applications (PUA) Protection for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender` \* `PUAProtection` = `1` (REG\_DWORD)

## Rationale Potentially Unwanted Applications (PUA) include adware, torrent clients, cryptominers, and system optimizers that increase risk and resource consumption. Forcing PUA blocking stops standard vectors of shadow IT and unauthorized utility tool execution.

## Legacy Impact & Compatibility Legitimate but unrecognized administrative utilities may be blocked. Standard exclusions must be configured for approved utilities.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus 2. Set 'Configure detection for potentially unwanted applications' to 'Enabled' 3. Select 'Block' in the options dropdown list

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderPUA.ps1](#)] ([./implementation\\_scripts/Configure-PawDefenderPUA.ps1](#))

```
Configure-PawDefenderPUA.ps1
Set-MpPreference -PUAProtection 1
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender" -Name "PUAProtection" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderPUAStatus.ps1](#)

```
Get-PawDefenderPUAStatus.ps1
$Pref = Get-MpPreference
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender" -Name "PUAProtection" -ErrorAction SilentlyContinue
if ($Pref.PUAProtection -eq 1 -or ($Reg -and $Reg.PUAProtection -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-059] Prevent Local List Merging and Exclusions Configuration for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender` \* `DisableLocalAdminMerge` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender` \* `HideExclusionsFromLocalAdmins` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions` \* `DisableLocalAdminConfiguration` = `1` (REG\_DWORD)

## Rationale If local administrators or compromised administrative accounts can modify Defender exclusions or merge local lists, they can authorize malicious folders or tools. Restricting list configuration to central GPOs ensures consistent security enforcement.

## Legacy Impact & Compatibility Local administrators will be unable to add folders or scripts to Defender exclusions. All exclusions must be managed centrally via GPO.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus 2. Set 'Configure local administrator merge behavior for lists' to 'Disabled' 3. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Exclusions 4. Set 'Prevent users from configuring exclusions' to 'Enabled' 5. Set 'Control whether or not exclusions are visible to Local Admins' to 'Enabled'

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderLocalExclusions.ps1](#)] ([../implementation\\_scripts/Configure-PawDefenderLocalExclusions.ps1](#))

```
Configure-PawDefenderLocalExclusions.ps1
Set-MpPreference -DisableLocalAdminMerge $true
Set-MpPreference -DisableExclusionRestriction $false
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender"
if (-not (Test-Path $Path)) { New-Item -Path $Path -Force | Out-Null }
Set-ItemProperty -Path $Path -Name "DisableLocalAdminMerge" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $Path -Name "HideExclusionsFromLocalAdmins" -Value 1 -Type DWord -Force
$ExclPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions"
if (-not (Test-Path $ExclPath)) { New-Item -Path $ExclPath -Force | Out-Null }
Set-ItemProperty -Path $ExclPath -Name "DisableLocalAdminConfiguration" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderLocalExclusionsStatus.ps1](#)

```
Get-PawDefenderLocalExclusionsStatus.ps1
$Pref = Get-MpPreference
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender" -Name "DisableLocalAdminMerge" -ErrorAction SilentlyContinue
$RegHide = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender" -Name "HideExclusionsFromLocalAdmins" -ErrorAction SilentlyContinue
$RegConfig = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions" -Name "DisableLocalAdminConfiguration" -ErrorAction SilentlyContinue
if (($Pref.DisableLocalAdminMerge -eq $true -or ($Reg -and $Reg.DisableLocalAdminMerge -eq 1)) -and
 ($RegHide -and $RegHide.HideExclusionsFromLocalAdmins -eq 1) -and
 ($RegConfig -and $RegConfig.DisableLocalAdminConfiguration -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-060] Configure Auto Exclusions Configuration for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Exclusions` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions` \* `DisableAutoExclusions` = `0` (REG\_DWORD)

## Rationale Auto Exclusions automatically configure exclusions for known safe system folders or server roles to reduce performance overhead. Enforcing that auto exclusions are not disabled ensures server performance stability and proper system scanning.

## Legacy Impact & Compatibility None.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Exclusions 2. Set 'Turn off Auto Exclusions' to 'Disabled'

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderAutoExclusions.ps1](#)] ([../implementation\\_scripts/Configure-PawDefenderAutoExclusions.ps1](#))

```
Configure-PawDefenderAutoExclusions.ps1
$ExclPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions"
if (-not (Test-Path $ExclPath)) { New-Item -Path $ExclPath -Force | Out-Null }
Set-ItemProperty -Path $ExclPath -Name "DisableAutoExclusions" -Value 0 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderAutoExclusionsStatus.ps1](#)

```
Get-PawDefenderAutoExclusionsStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions" -Name "DisableAutoExclusions" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.DisableAutoExclusions -eq 0) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-061] Enable EDR in Block Mode for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Features` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Features` \* `PassiveRemediation` = `1` (REG\_DWORD)

## Rationale Endpoint Detection and Response (EDR) in Block Mode allows Defender to take remediation actions on malicious artifacts detected by Microsoft Defender for Endpoint even if another non-Microsoft antivirus is primary. This establishes secondary defensive block capabilities.

## Legacy Impact & Compatibility None. Extends passive monitoring to execute block operations when necessary.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Features 2. Set 'Enable EDR in block mode' to 'Enabled'

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderEdrBlockMode.ps1](#)] (./implementation\_scripts/Configure-PawDefenderEdrBlockMode.ps1)

```
Configure-PawDefenderEdrBlockMode.ps1
$FeaturesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Features"
if (-not (Test-Path $FeaturesPath)) { New-Item -Path $FeaturesPath -Force | Out-Null }
Set-ItemProperty -Path $FeaturesPath -Name "PassiveRemediation" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderEdrBlockModeStatus.ps1](#)

```
Get-PawDefenderEdrBlockModeStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Features" -Name "PassiveRemediation" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.PassiveRemediation -eq 1) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations



# [REQ-PAW-062] Allow Network Protection on Windows Server for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Network Protection` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\Network Protection` \* `AllowNetworkProtectionOnWinServer` = `1` (REG\_DWORD)

## Rationale Network Protection blocks processes from accessing malicious domains, phishing sites, and host IP ranges. Allowing Network Protection on Windows Server ensures that member servers running server workloads possess the same IP filter protections as client platforms.

## Legacy Impact & Compatibility Unrecognized or legacy client-server connections to custom web resources may be filtered. Exclusions must be mapped under DFE.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Network Protection 2. Set 'This setting controls whether Network Protection is allowed to be configured into block or audit mode on Windows Server' to 'Enabled'

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-PawDefenderNetworkProtectionServer.ps1](../implementation\_scripts/Configure-PawDefenderNetworkProtectionServer.ps1)

```
Configure-PawDefenderNetworkProtectionServer.ps1
$NetProtPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\Network Protection"
if (-not (Test-Path $NetProtPath)) { New-Item -Path $NetProtPath -Force | Out-Null }
Set-ItemProperty -Path $NetProtPath -Name "AllowNetworkProtectionOnWinServer" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderNetworkProtectionServerStatus.ps1](#)

```
Get-PawDefenderNetworkProtectionServerStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\Network Protection"
-Name "AllowNetworkProtectionOnWinServer" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.AllowNetworkProtectionOnWinServer -eq 1) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-063] Enable File Hash Computation for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\MpEngine` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\MpEngine` \* `EnableFileHashComputation` = `1` (REG\_DWORD)

## Rationale Computing cryptographic file hashes allows Defender to pass hashes of scanned files to cloud and SIEM PAW platforms. This enables precise IOC matches, file tracking, and correlation with threat intelligence repositories.

## Legacy Impact & Compatibility Minor CPU/Disk I/O impact when scanning large directories.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\MpEngine 2. Set 'Enable file hash computation feature' to 'Enabled'

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderFileHash.ps1](#)] ([../implementation\\_scripts/Configure-PawDefenderFileHash.ps1](#))

```
Configure-PawDefenderFileHash.ps1
Set-MpPreference -EnableFileHashComputation $true
$MpEnginePath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\MpEngine"
if (-not (Test-Path $MpEnginePath)) { New-Item -Path $MpEnginePath -Force | Out-Null }
Set-ItemProperty -Path $MpEnginePath -Name "EnableFileHashComputation" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderFileHashStatus.ps1](#)

```
Get-PawDefenderFileHashStatus.ps1
$Pref = Get-MpPreference
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\MpEngine" -Name "EnableFileHashComputation" -Error
Action SilentlyContinue
if ($Pref.EnableFileHashComputation -eq $true -or ($Reg -and $Reg.EnableFileHashComputation -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-064] Configure Network Inspection System (NIS) settings for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Network Inspection System` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\NIS` \* `EnableConvertWarnToBlock` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\NIS` \* `AllowSwitchToAsyncInspection` = `1` (REG\_DWORD)

## Rationale The Network Inspection System (NIS) inspects network traffic patterns for known exploits. Converting warning verdicts to block enforces inline blocking of zero-day exploits, while allowing async inspection prevents performance overhead from slowing local network interfaces.

## Legacy Impact & Compatibility None.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Network Inspection System 2. Set 'Convert warn verdict to block' to 'Enabled' 3. Set 'Turn on asynchronous inspection' to 'Enabled'

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderNis.ps1](#)]  
(../implementation\_scripts/Configure-PawDefenderNis.ps1)

```
Configure-PawDefenderNis.ps1
$NisPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\NIS"
if (-not (Test-Path $NisPath)) { New-Item -Path $NisPath -Force | Out-Null }
Set-ItemProperty -Path $NisPath -Name "EnableConvertWarnToBlock" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $NisPath -Name "AllowSwitchToAsyncInspection" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderNisStatus.ps1](#)

```
Get-PawDefenderNisStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\NIS" -Name "EnableConvertWarnToBlock" -ErrorAction SilentlyContinue
$RegAsync = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\NIS" -Name "AllowSwitchToAsyncInspection" -ErrorAction SilentlyContinue
if (($Reg -and $Reg.EnableConvertWarnToBlock -eq 1) -and ($RegAsync -and $RegAsync.AllowSwitchToAsyncInspection -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-065] Configure OOBE Real-Time Protection and Security Intelligence for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Real-time Protection` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` \* `OobeEnableRtpAndSigUpdate` = `1` (REG\_DWORD)

## Rationale Enabling real-time protection and intelligence updates during the Out-of-Box Experience (OOBE) ensures that the system is fully updated and protected before the initial administrative user signs in or connects to enterprise network nodes.

## Legacy Impact & Compatibility Negligible. Minor network traffic during initial provisioning phases.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Real-time Protection 2. Set 'Configure real-time protection and Security Intelligence Updates during OOBE' to 'Enabled'

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderOobeRtp.ps1](#)] ([./implementation\\_scripts/Configure-PawDefenderOobeRtp.ps1](#))

```
Configure-PawDefenderOobeRtp.ps1
$RtpPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection"
if (-not (Test-Path $RtpPath)) { New-Item -Path $RtpPath -Force | Out-Null }
Set-ItemProperty -Path $RtpPath -Name "OobeEnableRtpAndSigUpdate" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderOobeRtpStatus.ps1](#)

```
Get-PawDefenderOobeRtpStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection" -Name "OobeEnableRtpAndSigUpdate" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.OobeEnableRtpAndSigUpdate -eq 1) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-066] Enable Dynamic Signature Dropped Event Reporting for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Low \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Reporting` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Reporting` \* `EnableDynamicSignatureDroppedEventReporting` = `1` (REG\_DWORD)

## Rationale Enabling this log report generation triggers explicit events when a dynamic scan ruleset signature is dropped. This ensures SIEM integrations can immediately log changes in the local threat signatures dataset.

## Legacy Impact & Compatibility None.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Reporting 2. Set 'Configure whether to report Dynamic Signature dropped events' to 'Enabled'

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderDynamicReporting.ps1](#)](../implementation\_scripts/Configure-PawDefenderDynamicReporting.ps1)

```
Configure-PawDefenderDynamicReporting.ps1
$RepPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Reporting"
if (-not (Test-Path $RepPath)) { New-Item -Path $RepPath -Force | Out-Null }
Set-ItemProperty -Path $RepPath -Name "EnableDynamicSignatureDroppedEventReporting" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderDynamicReportingStatus.ps1](#)

```
Get-PawDefenderDynamicReportingStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Reporting" -Name "EnableDynamicSignatureDroppedEventReporting" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.EnableDynamicSignatureDroppedEventReporting -eq 1) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-067] Configure Quick Scan and Scanning Exclusions for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` \* `QuickScanIncludeExclusions` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` \* `DisablePackedExeScanning` = `0` (REG\_DWORD)

## Rationale Malware frequently tries to establish persistence in excluded directories or inside packed/compressed executables. Forcing quick scans to include excluded files and ensuring packed file structures are recursively scanned prevents malware evasion.

## Legacy Impact & Compatibility Increased quick scan times depending on folder sizes and compression ratios.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan 2. Set 'Scan excluded files and directories during quick scans' to 'Enabled' 3. Set 'Turn off scanning of packed executables' to 'Disabled'

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderQuickScan.ps1](#)] ([./implementation\\_scripts/Configure-PawDefenderQuickScan.ps1](#))

```
Configure-PawDefenderQuickScan.ps1
Set-MpPreference -DisablePackedExeScanning $false
$ScanPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan"
if (-not (Test-Path $ScanPath)) { New-Item -Path $ScanPath -Force | Out-Null }
Set-ItemProperty -Path $ScanPath -Name "QuickScanIncludeExclusions" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $ScanPath -Name "DisablePackedExeScanning" -Value 0 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderQuickScanStatus.ps1](#)

```
Get-PawDefenderQuickScanStatus.ps1
$Pref = Get-MpPreference
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -Name "QuickScanIncludeExclusions" -ErrorAction SilentlyContinue
$RegPack = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -Name "DisablePackedExeScanning" -ErrorAction SilentlyContinue
if (($Pref.DisablePackedExeScanning -eq $false -or ($RegPack -and $RegPack.DisablePackedExeScanning -eq 0)) -and ($Reg -and $Reg.QuickScanIncludeExclusions -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-068] Configure Scheduled Scan Parameters for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan` \* \*\*Registry Location\*\*: \*  
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` \* `ScheduleDay` = `0` (REG\_DWORD) \*  
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` \* `DisableEmailScanning` = `0` (REG\_DWORD) \*  
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` \* `DisableHeuristics` = `0` (REG\_DWORD) \*  
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` \* `DaysWithoutCatchupQuickScan` = `7` (REG\_DWORD)

---

## Rationale Ensuring daily scheduled scans, enabling heuristics for behavioral anomaly detection, scan mail attachments, and forcing a catchup scan after at most 7 days ensures system integrity is continually validated.

---

## Legacy Impact & Compatibility Slight scanning overhead.

---

## Implementation Steps

#### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan 2. Set 'Specify the day of the week to run a scheduled scan' to 'Enabled' (Select 'Every day' or '0') 3. Set 'Turn on e-mail scanning' to 'Enabled' 4. Set 'Turn on heuristics' to 'Enabled' 5. Set 'Trigger a quick scan after X days without any scans' to 'Enabled' (7 days)

---

#### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderScheduledScan.ps1](#)] ([../implementation\\_scripts/Configure-PawDefenderScheduledScan.ps1](#))

---

```
Configure-PawDefenderScheduledScan.ps1
Set-MpPreference -DisableEmailScanning $false
Set-MpPreference -DisableHeuristics $false
$ScanPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan"
if (-not (Test-Path $ScanPath)) { New-Item -Path $ScanPath -Force | Out-Null }
Set-ItemProperty -Path $ScanPath -Name "ScheduleDay" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $ScanPath -Name "DisableEmailScanning" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $ScanPath -Name "DisableHeuristics" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $ScanPath -Name "DaysWithoutCatchupQuickScan" -Value 7 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderScheduledScanStatus.ps1](#)

```
Get-PawDefenderScheduledScanStatus.ps1
$Pref = Get-MpPreference
$RegDays = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -Name "DaysWithoutCatchupQuickScan" -ErrorAction SilentlyContinue
if ($Pref.DisableEmailScanning -eq $false -and $Pref.DisableHeuristics -eq $false -and ($RegDays -and $RegDays.DaysWithoutCatchupQuickScan -eq 7)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-069] Configure Security Intelligence Update Schedule for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Security Intelligence Updates` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates` \* `ASSignatureDue` = `7` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates` \* `AVSignatureDue` = `7` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates` \* `ScheduleDay` = `0` (REG\_DWORD)

## Rationale Antivirus signatures must remain fresh to block the latest published threats. Mandating daily checks for updates and marking signatures older than 7 days as out-of-date ensures continuous defense parity.

## Legacy Impact & Compatibility Requires continuous outbound connectivity to WSUS or Microsoft Update endpoints.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Security Intelligence Updates 2. Set 'Define the number of days before spyware security intelligence is considered out of date' to 'Enabled' (7 days) 3. Set 'Define the number of days before virus security intelligence is considered out of date' to 'Enabled' (7 days) 4. Set 'Specify the day of the week to check for security intelligence updates' to 'Enabled' (Every day or 0)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderUpdateSchedule.ps1](#)] ([../implementation\\_scripts/Configure-PawDefenderUpdateSchedule.ps1](#))

```
Configure-PawDefenderUpdateSchedule.ps1
$SigPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates"
if (-not (Test-Path $SigPath)) { New-Item -Path $SigPath -Force | Out-Null }
Set-ItemProperty -Path $SigPath -Name "ASSignatureDue" -Value 7 -Type DWord -Force
Set-ItemProperty -Path $SigPath -Name "AVSignatureDue" -Value 7 -Type DWord -Force
Set-ItemProperty -Path $SigPath -Name "ScheduleDay" -Value 0 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderUpdateScheduleStatus.ps1](#)

```
Get-PawDefenderUpdateScheduleStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates" -Name "ASSignatureDue" -ErrorAction SilentlyContinue
$RegAV = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates" -Name "AVSignatureDue" -ErrorAction SilentlyContinue
$RegDay = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates" -Name "ScheduleDay" -ErrorAction SilentlyContinue
if (($Reg -and $Reg.ASSignatureDue -eq 7) -and ($RegAV -and $RegAV.AVSignatureDue -eq 7) -and ($RegDay -and $RegDay.ScheduleDay -eq 0)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations



# Configure Attack Surface Reduction Rules for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction

---

## Rationale Attack Surface Reduction (ASR) rules are critical for PAWs to enforce administrative isolation. Spawning tools from Office documents, dumping memory from LSASS, or launching WMI/PSEXEC child commands represent standard post-exploitation vectors. This submodule contains individual requirement rules for each ASR control enforced on Privileged Access Workstations.

---

## Legacy Impact & Compatibility \* \*\*Strict Block Enforcement\*\*: Unlike standard workstations, all rules are enforced immediately in \*\*Block\*\* mode ('1') as there should be no productive Office files, macros, or legacy scripts active on directory management consoles.

---

## Enforced Attack Surface Reduction Rules on PAWs

The following individual ASR rules must be configured in Block mode:

1. [REQ-PAW-076 - ASR: Block abuse of exploited vulnerable signed drivers for PAWs](#)
  2. [REQ-PAW-077 - ASR: Block Adobe Reader from creating child processes for PAWs](#)
  3. [REQ-PAW-078 - ASR: Block all Office applications from creating child processes for PAWs](#)
  4. [REQ-PAW-079 - ASR: Block credential stealing from the Windows local security authority subsystem for PAWs](#)
  5. [REQ-PAW-080 - ASR: Block executable content from email client and webmail for PAWs](#)
  6. [REQ-PAW-081 - ASR: Block executable files from running unless they meet a prevalence, age, or trusted list criterion for PAWs](#)
  7. [REQ-PAW-082 - ASR: Block execution of potentially obfuscated scripts for PAWs](#)
  8. [REQ-PAW-083 - ASR: Block JavaScript or VBScript from launching downloaded executable content for PAWs](#)
  9. [REQ-PAW-084 - ASR: Block Office applications from creating executable content for PAWs](#)
  10. [REQ-PAW-085 - ASR: Block Office applications from injecting code into other processes for PAWs](#)
  11. [REQ-PAW-086 - ASR: Block Office communication application from creating child processes for PAWs](#)
  12. [REQ-PAW-087 - ASR: Block persistence through WMI event subscription for PAWs](#)
  13. [REQ-PAW-088 - ASR: Block process creations originating from PSEXEC and WMI commands for PAWs](#)
  14. [REQ-PAW-089 - ASR: Block untrusted and unsigned processes that run from USB for PAWs](#)
  15. [REQ-PAW-090 - ASR: Block Win32 API calls from Office macros for PAWs](#)
  16. [REQ-PAW-091 - ASR: Use advanced protection against ransomware for PAWs](#)
- 

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR Rules) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-076] ASR: Block abuse of exploited vulnerable signed drivers for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `56a863a9-875e-4185-98a7-b882c64b5ce5` = `1` (REG\_SZ)

## Rationale Prevents an application from writing a vulnerable signed driver to disk. Attackers use Bring Your Own Vulnerable Driver (BYOVD) techniques to bypass Windows kernel protections by loading legitimate, signed third-party drivers that contain known vulnerabilities, allowing them to disable security agents and gain kernel-level privileges.

## Legacy Impact & Compatibility Administrators or developers who deliberately load older or specific signed debugging/hardware diagnostics drivers containing known security bugs will be blocked. Approvals or exclusions must be configured for these specific drivers.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `56a863a9-875e-4185-98a7-b882c64b5ce5` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrVulnerableDrivers.ps1](#)] ([../implementation\\_scripts/Configure-PawAsrVulnerableDrivers.ps1](#))

```
Configure-PawAsrVulnerableDrivers.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "56a863a9-875e-4185-98a7-b882c64b5ce5" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrVulnerableDriversStatus.ps1](#)

```
Get-PawAsrVulnerableDriversStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "56a863a9-875e-4185-98a7-b882c64b5ce5" -ErrorAction SilentlyContinue
if ($Value -and ($Value."56a863a9-875e-4185-98a7-b882c64b5ce5" -eq "1" -or $Value."56a863a9-875e-4185-98a7-b882c64b5ce5" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-077] ASR: Block Adobe Reader from creating child processes for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `7674ba52-37eb-4a4f-a9a1-f0f9a1619a2c` = `1` (REG\_SZ)

## Rationale Prevents Adobe Reader from launching any child processes. Malicious PDF documents frequently attempt to exploit application vulnerabilities or trick users into executing embedded links, which spawns command shells (cmd.exe, powershell.exe) or scripting hosts (wscript.exe) to download and launch secondary malware payloads.

## Legacy Impact & Compatibility Adobe Reader will be unable to launch external helper applications directly. PDF forms that rely on spawning external applications or customized plugins that execute command-line helpers will fail to run.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `7674ba52-37eb-4a4f-a9a1-f0f9a1619a2c` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrAdobeChild.ps1](#)] ([../implementation\\_scripts/Configure-PawAsrAdobeChild.ps1](#))

```
Configure-PawAsrAdobeChild.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "7674ba52-37eb-4a4f-a9a1-f0f9a1619a2c" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrAdobeChildStatus.ps1](#)

```
Get-PawAsrAdobeChildStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "7674ba52-37eb-4a4f-a9a1-f0f9a1619a2c" -ErrorAction SilentlyContinue
if ($Value -and ($Value."7674ba52-37eb-4a4f-a9a1-f0f9a1619a2c" -eq "1" -or $Value."7674ba52-37eb-4a4f-a9a1-f0f9a1619a2c" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-078] ASR: Block all Office applications from creating child processes for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `d4f940ab-401b-4efc-aadc-ad5f3c50688a` = `1` (REG\_SZ)

## Rationale Blocks Microsoft Office applications (Word, Excel, PowerPoint) from creating child processes. This prevents malicious files containing embedded VBA macros or exploiting unpatched vulnerabilities (such as CVE-2021-40444) from launching scripting environments or system commands to download and execute code.

## Legacy Impact & Compatibility Office macros that legitimately launch external scripts, shell scripts, or third-party executable helpers will be blocked. Business processes relying on advanced inter-process macros must be rewritten to use modern API alternatives.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `d4f940ab-401b-4efc-aadc-ad5f3c50688a` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrOfficeChild.ps1](#)] ([./../implementation\\_scripts/Configure-PawAsrOfficeChild.ps1](#))

```
Configure-PawAsrOfficeChild.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "d4f940ab-401b-4efc-aadc-ad5f3c50688a" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrOfficeChildStatus.ps1](#)

```
Get-PawAsrOfficeChildStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "d4f940ab-401b-4efc-aadc-ad5f3c50688a" -ErrorAction SilentlyContinue
if ($Value -and ($Value."d4f940ab-401b-4efc-aadc-ad5f3c50688a" -eq "1" -or $Value."d4f940ab-401b-4efc-aadc-ad5f3c50688a" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-079] ASR: Block credential stealing from the Windows local security authority subsystem for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2` = `1` (REG\_SZ)

## Rationale Blocks attempts to open or dump the memory of the Local Security Authority Subsystem Service (lsass.exe). Attackers dump LSASS memory using tools like Mimikatz or Task Manager to extract plaintext credentials, Kerberos tickets, or NTLM password hashes from system memory for lateral movement.

## Legacy Impact & Compatibility Administrative diagnostic tools, customized authentication packages, or security scanners that attempt to open the LSASS process handle to read user tokens or perform access auditing will be blocked.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrLsassDump.ps1](#)] ([./../implementation\\_scripts/Configure-PawAsrLsassDump.ps1](#))

```
Configure-PawAsrLsassDump.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrLsassDumpStatus.ps1](#)

```
Get-PawAsrLsassDumpStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2" -ErrorAction SilentlyContinue
if ($Value -and ($Value."9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2" -eq "1" -or $Value."9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-080] ASR: Block executable content from email client and webmail for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `be9ba2d9-53ea-4cdc-84e5-9b1eeee46550` = `1` (REG\_SZ)

## Rationale Prevents executable files (such as .exe, .com, .scr, .vbs, .js, or .pif) from launching directly from email clients (like Outlook) or webmail accessed via browser sessions. This stops phishing attacks where users accidentally launch malicious attachments or download payloads directly from web-based email links.

## Legacy Impact & Compatibility Users will be blocked from directly launching attachments that contain scripts, setup files, or macros. They must first save the file to an authorized directory and run it from there (which allows standard real-time scans to run first).

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `be9ba2d9-53ea-4cdc-84e5-9b1eeee46550` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrEmailExecutable.ps1](#)] ([../implementation\\_scripts/Configure-PawAsrEmailExecutable.ps1](#))

```
Configure-PawAsrEmailExecutable.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "be9ba2d9-53ea-4cdc-84e5-9b1eeee46550" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrEmailExecutableStatus.ps1](#)

```
Get-PawAsrEmailExecutableStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "be9ba2d9-53ea-4cdc-84e5-9b1eeee46550" -ErrorAction SilentlyContinue
if ($Value -and ($Value."be9ba2d9-53ea-4cdc-84e5-9b1eeee46550" -eq "1" -or $Value."be9ba2d9-53ea-4cdc-84e5-9b1eeee46550" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-081] ASR: Block executable files from running unless they meet a prevalence, age, or trusted list criterion for PAWs  
 ## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `01443614-cd74-433a-b99e-2ecdc07bfc25` = `1` (REG\_SZ)

## Rationale Blocks execution of unrecognized, newly compiled, or low-prevalence executable files. This provides initial protection against zero-day malware campaigns and targeted custom payloads that have not yet established reputation telemetry in the Microsoft Cloud Protection network.

## Legacy Impact & Compatibility Developers building and testing custom internal binaries, or administrators installing niche, uncertified third-party software updates will face false positives. Exclusions must be configured for target test directories.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `01443614-cd74-433a-b99e-2ecdc07bfc25` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrLowPrevalence.ps1](#)] ([../implementation\\_scripts/Configure-PawAsrLowPrevalence.ps1](#))

```
Configure-PawAsrLowPrevalence.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "01443614-cd74-433a-b99e-2ecdc07bfc25" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrLowPrevalenceStatus.ps1](#)

```
Get-PawAsrLowPrevalenceStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "01443614-cd74-433a-b99e-2ecdc07bfc25" -ErrorAction SilentlyContinue
if ($Value -and ($Value."01443614-cd74-433a-b99e-2ecdc07bfc25" -eq "1" -or $Value."01443614-cd74-433a-b99e-2ecdc07bfc25" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations



# [REQ-PAW-082] ASR: Block execution of potentially obfuscated scripts for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `5beb7efe-fd9a-4556-801d-275e5ffc04cc` = `1` (REG\_SZ)

## Rationale Blocks execution of obfuscated or encrypted scripts (such as PowerShell, VBScript, or JavaScript). Threat actors obfuscate their scripts using base64 encoding, custom string manipulation, or encryption to hide the intent of their code and bypass static file scanning and network detection engines.

## Legacy Impact & Compatibility Legitimate administrative tools or third-party provisioning scripts that use heavy minification, obfuscation, or custom encoding wrappers will be blocked. Scripts must be refactored to use readable, signed code.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `5beb7efe-fd9a-4556-801d-275e5ffc04cc` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrObfuscatedScripts.ps1](#)] ([./../implementation\\_scripts/Configure-PawAsrObfuscatedScripts.ps1](#))

```
Configure-PawAsrObfuscatedScripts.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "5beb7efe-fd9a-4556-801d-275e5ffc04cc" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrObfuscatedScriptsStatus.ps1](#)

```
Get-PawAsrObfuscatedScriptsStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "5beb7efe-fd9a-4556-801d-275e5ffc04cc" -ErrorAction SilentlyContinue
if ($Value -and ($Value."5beb7efe-fd9a-4556-801d-275e5ffc04cc" -eq "1" -or $Value."5beb7efe-fd9a-4556-801d-275e5ffc04cc" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations



# [REQ-PAW-083] ASR: Block JavaScript or VBScript from launching downloaded executable content for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `d3e037e1-3eb8-44c8-a917-57927947596d` = `1` (REG\_SZ)

## Rationale Prevents JavaScript or VBScript running locally from launching executable binaries that were downloaded from the internet. Attackers use malicious scripts inside documents or web browsers to download payloads (like ransomware or trojans) to the disk and launch them using local script engines.

## Legacy Impact & Compatibility Local administrative script engines that perform software deployment or download updates and then launch installers will be blocked. These scripts must be replaced by structured configuration agents or signed installers.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `d3e037e1-3eb8-44c8-a917-57927947596d` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrScriptLaunchExe.ps1](#)] ([../implementation\\_scripts/Configure-PawAsrScriptLaunchExe.ps1](#))

```
Configure-PawAsrScriptLaunchExe.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "d3e037e1-3eb8-44c8-a917-57927947596d" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrScriptLaunchExeStatus.ps1](#)

```
Get-PawAsrScriptLaunchExeStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "d3e037e1-3eb8-44c8-a917-57927947596d" -ErrorAction SilentlyContinue
if ($Value -and ($Value."d3e037e1-3eb8-44c8-a917-57927947596d" -eq "1" -or $Value."d3e037e1-3eb8-44c8-a917-57927947596d" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-084] ASR: Block Office applications from creating executable content for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `3b576869-a4ec-4529-8536-b80a7769e899` = `1` (REG\_SZ)

## Rationale Prevents Microsoft Office applications (Word, Excel, PowerPoint) from creating or writing executable files (e.g., .exe, .dll, .scr) to the local filesystem. Malicious documents often attempt to drop payloads directly into the local temp folders or AppData directories before executing them.

## Legacy Impact & Compatibility Macros or add-ins that legitimately save external binaries or compile executable helper objects locally will be blocked.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `3b576869-a4ec-4529-8536-b80a7769e899` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrOfficeWriteExe.ps1](#)] ([../implementation\\_scripts/Configure-PawAsrOfficeWriteExe.ps1](#))

```
Configure-PawAsrOfficeWriteExe.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "3b576869-a4ec-4529-8536-b80a7769e899" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrOfficeWriteExeStatus.ps1](#)

```
Get-PawAsrOfficeWriteExeStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "3b576869-a4ec-4529-8536-b80a7769e899" -ErrorAction SilentlyContinue
if ($Value -and ($Value."3b576869-a4ec-4529-8536-b80a7769e899" -eq "1" -or $Value."3b576869-a4ec-4529-8536-b80a7769e899" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-085] ASR: Block Office applications from injecting code into other processes for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84` = `1` (REG\_SZ)

## Rationale Blocks Microsoft Office applications from writing code or injecting threads directly into external processes. Threat actors use code injection (such as process hollowing or remote thread creation) inside Office macros to hide execution under clean, trusted system binaries like explorer.exe or svchost.exe.

## Legacy Impact & Compatibility Office plugins or custom integration tools that inject threads or communicate with database engines using active injection libraries will be blocked.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrOfficeInjection.ps1](#)] ([../implementation\\_scripts/Configure-PawAsrOfficeInjection.ps1](#))

```
Configure-PawAsrOfficeInjection.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrOfficeInjectionStatus.ps1](#)

```
Get-PawAsrOfficeInjectionStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84" -ErrorAction SilentlyContinue
if ($Value -and ($Value."75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84" -eq "1" -or $Value."75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-086] ASR: Block Office communication application from creating child processes for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `26190899-1602-49e8-8b27-eb1d0a1ce869` = `1` (REG\_SZ)

## Rationale Blocks Microsoft Outlook or other Office communication applications (e.g., Teams, Skype) from creating child processes. This prevents malware payloads delivered through emails, chats, or calendar invites from spawning command-line utilities or scripting environments.

## Legacy Impact & Compatibility Outlook integrations or custom add-ins that legitimately spawn helper utilities, such as starting external browsers or custom document handlers directly, may be blocked.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `26190899-1602-49e8-8b27-eb1d0a1ce869` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrOutlookChild.ps1](#)] ([../implementation\\_scripts/Configure-PawAsrOutlookChild.ps1](#))

```
Configure-PawAsrOutlookChild.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "26190899-1602-49e8-8b27-eb1d0a1ce869" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrOutlookChildStatus.ps1](#)

```
Get-PawAsrOutlookChildStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "26190899-1602-49e8-8b27-eb1d0a1ce869" -ErrorAction SilentlyContinue
if ($Value -and ($Value."26190899-1602-49e8-8b27-eb1d0a1ce869" -eq "1" -or $Value."26190899-1602-49e8-8b27-eb1d0a1ce869" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-087] ASR: Block persistence through WMI event subscription for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `e6db77e5-3df2-4cf1-b95a-636979351e5b` = `1` (REG\_SZ)

## Rationale Blocks threat actors from achieving system persistence by registering permanent Windows Management Instrumentation (WMI) event subscriptions. WMI event subscriptions allow attackers to automatically launch malicious payloads when system triggers occur (like system boot or user logon) without using traditional startup registry keys.

## Legacy Impact & Compatibility Management software or diagnostic monitoring tools that legitimately register permanent WMI event filters to track system telemetry will be blocked. Alternatives like Windows services or scheduled tasks must be used.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `e6db77e5-3df2-4cf1-b95a-636979351e5b` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrWmiPersistence.ps1](#)] ([../implementation\\_scripts/Configure-PawAsrWmiPersistence.ps1](#))

```
Configure-PawAsrWmiPersistence.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "e6db77e5-3df2-4cf1-b95a-636979351e5b" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrWmiPersistenceStatus.ps1](#)

```
Get-PawAsrWmiPersistenceStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "e6db77e5-3df2-4cf1-b95a-636979351e5b" -ErrorAction SilentlyContinue
if ($Value -and ($Value."e6db77e5-3df2-4cf1-b95a-636979351e5b" -eq "1" -or $Value."e6db77e5-3df2-4cf1-b95a-636979351e5b" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-088] ASR: Block process creations originating from PSEXec and WMI commands for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `d1e49aac-8f56-4280-b9ba-993a6d77406c` = `1` (REG\_SZ)

## Rationale Blocks processes created via WMI commands or PSEXec remote execution utilities. This directly stops lateral movement attacks where compromised accounts or threat actors attempt to start commands, backdoors, or credential dumpers remotely across domain-joined servers and PAW platforms.

## Legacy Impact & Compatibility Disrupts remote administration tools, monitoring agents, and network scanners that rely on WMI/PSEXec. For Domain Controllers or critical servers, this rule is often set to \*\*Audit\*\* mode (`2`) rather than hard Block mode (`1`) to prevent administrative downtime.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `d1e49aac-8f56-4280-b9ba-993a6d77406c` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrPsexecWmi.ps1](#)] ([../implementation\\_scripts/Configure-PawAsrPsexecWmi.ps1](#))

```
Configure-PawAsrPsexecWmi.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "d1e49aac-8f56-4280-b9ba-993a6d77406c" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrPsexecWmiStatus.ps1](#)

```
Get-PawAsrPsexecWmiStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "d1e49aac-8f56-4280-b9ba-993a6d77406c" -ErrorAction SilentlyContinue
if ($Value -and ($Value."d1e49aac-8f56-4280-b9ba-993a6d77406c" -eq "1" -or $Value."d1e49aac-8f56-4280-b9ba-993a6d77406c" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-089] ASR: Block untrusted and unsigned processes that run from USB for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4` = `1` (REG\_SZ)

## Rationale Blocks the execution of unsigned or untrusted processes on removable storage devices (USB drives, external SSDs). This stops physical access vectors, rogue USB drops, and automated worm propagation techniques from running unauthorized installers or scripts.

## Legacy Impact & Compatibility Administrators or developers will be unable to run unsigned utilities directly from portable flash drives. Software must be copied to local trusted storage, or the binary must possess a trusted digital signature.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrUsbUnsigned.ps1](#)] ([../implementation\\_scripts/Configure-PawAsrUsbUnsigned.ps1](#))

```
Configure-PawAsrUsbUnsigned.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrUsbUnsignedStatus.ps1](#)

```
Get-PawAsrUsbUnsignedStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4" -ErrorAction SilentlyContinue
if ($Value -and ($Value."b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4" -eq "1" -or $Value."b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations



# [REQ-PAW-090] ASR: Block Win32 API calls from Office macros for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b` = `1` (REG\_SZ)

## Rationale Blocks VBA macros inside Microsoft Office documents from invoking Win32 API calls. Malicious documents use macros to call kernel memory functions (such as VirtualAlloc, WriteProcessMemory, or CreateThread) to load and execute shellcode in memory without dropping files to disk, bypassing file scanners.

## Legacy Impact & Compatibility Advanced business spreadsheets or database documents that utilize Win32 API calls for custom UI rendering, network calls, or system commands will fail. They must be rewritten using standard secure VBA structures or add-in APIs.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrOfficeWin32Calls.ps1](#)] ([../implementation\\_scripts/Configure-PawAsrOfficeWin32Calls.ps1](#))

```
Configure-PawAsrOfficeWin32Calls.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrOfficeWin32CallsStatus.ps1](#)

```
Get-PawAsrOfficeWin32CallsStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b" -ErrorAction SilentlyContinue
if ($Value -and ($Value."92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b" -eq "1" -or $Value."92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations



# [REQ-PAW-091] ASR: Use advanced protection against ransomware for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` \* `c1db55ab-c21a-4637-bb3f-a12568109d35` = `1` (REG\_SZ)

## Rationale Enables advanced behavioral heuristics and cloud analytics checks on files that attempt to modify multiple user files, detect signature-less encryption behavior, and block rapid write activity to prevent ransomware from encrypting system and user documents.

## Legacy Impact & Compatibility Minor performance overhead when scanning active files during large batch edits or heavy database updates.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `c1db55ab-c21a-4637-bb3f-a12568109d35` as Value Name, with Value set to `1` (Block).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAsrRansomware.ps1](#)] ([../implementation\\_scripts/Configure-PawAsrRansomware.ps1](#))

```
Configure-PawAsrRansomware.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "c1db55ab-c21a-4637-bb3f-a12568109d35" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawAsrRansomwareStatus.ps1](#)

```
Get-PawAsrRansomwareStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "c1db55ab-c21a-4637-bb3f-a12568109d35" -ErrorAction SilentlyContinue
if ($Value -and ($Value."c1db55ab-c21a-4637-bb3f-a12568109d35" -eq "1" -or $Value."c1db55ab-c21a-4637-bb3f-a12568109d35" -eq 1)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (ASR rules config) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-071] Configure Threat Severity Default Quarantine Actions for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Threats` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Threats` \* `Threats\_ThreatSeverityDefaultAction` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Threats\ThreatSeverityDefaultAction` \* `1` = `2` (REG\_DWORD)

## Rationale By default, Defender may prompt users or take actions (like clean/ignore) that leave malware remnants on the filesystem. Configuring default quarantine actions for all severities (low, medium, high, severe) ensures automated containment.

## Legacy Impact & Compatibility None.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Threats 2. Set 'Specify threat alert levels at which default action should not be taken when detected' to 'Enabled' 3. Click 'Show...' and enter threat levels (1 → 2, 2 → 2, 4 → 2, 5 → 2)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderThreatActions.ps1](#)] ([./implementation\\_scripts/Configure-PawDefenderThreatActions.ps1](#))

```
Configure-PawDefenderThreatActions.ps1
$ThreatsPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Threats"
if (-not (Test-Path $ThreatsPath)) { New-Item -Path $ThreatsPath -Force | Out-Null }
Set-ItemProperty -Path $ThreatsPath -Name "Threats_ThreatSeverityDefaultAction" -Value 1 -Type DWord -Force
$ThreatsSevPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Threats\ThreatSeverityDefaultAction"
if (-not (Test-Path $ThreatsSevPath)) { New-Item -Path $ThreatsSevPath -Force | Out-Null }
Set-ItemProperty -Path $ThreatsSevPath -Name "1" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $ThreatsSevPath -Name "2" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $ThreatsSevPath -Name "4" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $ThreatsSevPath -Name "5" -Value 2 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderThreatActionsStatus.ps1](#)

```
Get-PawDefenderThreatActionsStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Threats" -Name "Threats_ThreatSeverityDefaultAction" -ErrorAction SilentlyContinue
$RegSev = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Threats\ThreatSeverityDefaultAction" -ErrorAction SilentlyContinue
if (($Reg -and $Reg.Threats_ThreatSeverityDefaultAction -eq 1) -and
 ($RegSev -and $RegSev.1 -eq 2 -and $RegSev.2 -eq 2 -and $RegSev.4 -eq 2 -and $RegSev.5 -eq 2)) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-072] Configure Family Options UI Lockdown for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Low \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Security\Family options` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\Family options` \* `UILockdown` = `1` (REG\_DWORD)

## Rationale Locking down non-essential components of the Windows Security Center interface prevents users from tampering with parental or diagnostic UI controls on enterprise assets.

## Legacy Impact & Compatibility None.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Security\Family options 2. Set 'Hide the Family options area' to 'Enabled'

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderFamilyLockdown.ps1](#)] ([../implementation\\_scripts/Configure-PawDefenderFamilyLockdown.ps1](#))

```
Configure-PawDefenderFamilyLockdown.ps1
$FamilyPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\Family options"
if (-not (Test-Path $FamilyPath)) { New-Item -Path $FamilyPath -Force | Out-Null }
Set-ItemProperty -Path $FamilyPath -Name "UILockdown" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderFamilyLockdownStatus.ps1](#)

```
Get-PawDefenderFamilyLockdownStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\Family options" -Name "UILockdown"
-ErrorAction SilentlyContinue
if ($Reg -and $Reg.UILockdown -eq 1) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-073] Configure Tamper Protection for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\Windows Security\Tamper Protection` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Microsoft\Windows Defender\Features` \* `TamperProtection` = `5` (REG\_DWORD)

## Rationale Tamper Protection prevents local administrators or compromised system accounts from disabling Windows Defender services, real-time scanning, or modifying active exclusions locally. This blocks a primary malware persistence vector.

## Legacy Impact & Compatibility Local registry modifications to Defender configurations will be ignored. All changes must originate from central templates or cloud console panels.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Security\Tamper Protection 2. Set 'Protect Windows Security settings from tampering' to 'Enabled' (Block or On depending on ADMX version)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderTamperProtection.ps1](#)] ([./implementation\\_scripts/Configure-PawDefenderTamperProtection.ps1](#))

```
Configure-PawDefenderTamperProtection.ps1
$FeaturesPath = "HKLM:\SOFTWARE\Microsoft\Windows Defender\Features"
if (-not (Test-Path $FeaturesPath)) { New-Item -Path $FeaturesPath -Force | Out-Null }
try {
 Set-ItemProperty -Path $FeaturesPath -Name "TamperProtection" -Value 5 -Type DWord -ErrorAction Stop -Force
} catch {
 Write-Warning "Registry blocked. Tamper Protection registry key is normally protected by TrustedInstaller. Ensure GPO setting is applied."
}
```

To audit the hardening status: [Download Script: Get-PawDefenderTamperProtectionStatus.ps1](#)

```
Get-PawDefenderTamperProtectionStatus.ps1
$FeaturesPath = "HKLM:\SOFTWARE\Microsoft\Windows Defender\Features"
$TamperVal = Get-ItemProperty -Path $FeaturesPath -Name "TamperProtection" -ErrorAction SilentlyContinue
if ($TamperVal -and $TamperVal.TamperProtection -eq 5) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-074] Configure Sandbox Execution Environment for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Preferences\Windows Settings\Environment` \* \*\*Registry Location\*\*: \* `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Environment` \* `MP\_FORCE\_USE\_SANDBOX` = `1` (REG\_SZ)

## Rationale Forcing the Windows Defender scanning service (MsMpEng.exe) to run in a restricted AppContainer sandbox prevents privilege escalation. If an attacker exploits a parsing vulnerability in the engine, the compromise is contained inside the sandbox.

## Legacy Impact & Compatibility A system reboot is required to initialize the scanning process within the AppContainer sandbox.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Preferences\Windows Settings\Environment 2. Right-click and select New → Environment Variable 3. Configure Action: Update, Type: System, Name: MP\_FORCE\_USE\_SANDBOX, Value: 1

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderSandbox.ps1](#)] (./implementation\_scripts/Configure-PawDefenderSandbox.ps1)

```
Configure-PawDefenderSandbox.ps1
$EnvPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Environment"
if (-not (Test-Path $EnvPath)) { New-Item -Path $EnvPath -Force | Out-Null }
Set-ItemProperty -Path $EnvPath -Name "MP_FORCE_USE_SANDBOX" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderSandboxStatus.ps1](#)

```
Get-PawDefenderSandboxStatus.ps1
$EnvPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Environment"
$SandboxVar = Get-ItemProperty -Path $EnvPath -Name "MP_FORCE_USE_SANDBOX" -ErrorAction SilentlyContinue
if ($SandboxVar -and $SandboxVar.MP_FORCE_USE_SANDBOX -eq "1") {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# [REQ-PAW-075] Configure AMSI Authenticode Signature Verification for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Preferences\Windows Settings\Registry` \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Microsoft\AMSI` \* `FeatureBits` = `2` (REG\_DWORD)

## Rationale Enforcing signature checks on registered Antimalware Scan Interface (AMSI) providers blocks attackers from registering unsigned rogue AMSI provider DLLs to bypass script analysis.

## Legacy Impact & Compatibility Third-party antivirus/security providers must register using signed Authenticode binaries to prevent being blocked.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Preferences\Windows Settings\Registry 2. Right-click and select New → Registry Item 3. Configure Action: Update, Hive: HKEY\_LOCAL\_MACHINE, Key Path: SOFTWARE\Microsoft\AMSI, Value Name: FeatureBits, Value Type: REG\_DWORD, Value Data: 2

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawDefenderAmsiSignature.ps1](#)] ([../implementation\\_scripts/Configure-PawDefenderAmsiSignature.ps1](#))

```
Configure-PawDefenderAmsiSignature.ps1
$AmsiPath = "HKLM:\SOFTWARE\Microsoft\AMSI"
if (-not (Test-Path $AmsiPath)) { New-Item -Path $AmsiPath -Force | Out-Null }
Set-ItemProperty -Path $AmsiPath -Name "FeatureBits" -Value 2 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-PawDefenderAmsiSignatureStatus.ps1](#)

```
Get-PawDefenderAmsiSignatureStatus.ps1
$AmsiPath = "HKLM:\SOFTWARE\Microsoft\AMSI"
if (Test-Path $AmsiPath) {
 $AmsiBits = Get-ItemProperty -Path $AmsiPath -Name "FeatureBits" -ErrorAction SilentlyContinue
 if ($AmsiBits -and $AmsiBits.FeatureBits -eq 2) {
 Write-Output "Compliant"
 exit 0
 }
}
Write-Output "Non-Compliant"
exit 1
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.9 (Windows Defender Antivirus configuration parameters) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations

# Configure User Rights Assignments for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` \* \*\*Registry Location\*\*: Stored inside local security database under privilege definitions.

---

## Rationale Enforcing strict User Rights Assignments (URAs) on PAWs limits the execution footprint of administrative helper binaries, service accounts, and logon permissions to minimize the privilege footprint and prevent administrative impersonation. This submodule contains individual requirement rules for each User Rights Assignment control enforced on PAWs.

---

## Legacy Impact & Compatibility \* \*\*Maximum Console Isolation\*\*: Standard domain users and non-administrative services have no permission assignments on PAW consoles. Operational impact should be non-existent because PAWs are restricted to pure administrative functions.

---

## Enforced User Rights Assignments on PAWs

The following individual URA rules must be configured:

1. [REQ-PAW-092 - Configure User Rights: Access Credential Manager as a trusted caller for PAWs](#)
  2. [REQ-PAW-093 - Configure User Rights: Access this computer from the network for PAWs](#)
  3. [REQ-PAW-094 - Configure User Rights: Act as part of the operating system for PAWs](#)
  4. [REQ-PAW-095 - Configure User Rights: Allow log on locally for PAWs](#)
  5. [REQ-PAW-096 - Configure User Rights: Back up files and directories for PAWs](#)
  6. [REQ-PAW-097 - Configure User Rights: Create a pagefile for PAWs](#)
  7. [REQ-PAW-098 - Configure User Rights: Create a token object for PAWs](#)
  8. [REQ-PAW-099 - Configure User Rights: Create global objects for PAWs](#)
  9. [REQ-PAW-100 - Configure User Rights: Create permanent shared objects for PAWs](#)
  10. [REQ-PAW-101 - Configure User Rights: Debug programs for PAWs](#)
  11. [REQ-PAW-102 - Configure User Rights: Enable computer and user accounts to be trusted for delegation for PAWs](#)
  12. [REQ-PAW-103 - Configure User Rights: Force shutdown from a remote system for PAWs](#)
  13. [REQ-PAW-104 - Configure User Rights: Impersonate a client after authentication for PAWs](#)
  14. [REQ-PAW-105 - Configure User Rights: Load and unload device drivers for PAWs](#)
  15. [REQ-PAW-106 - Configure User Rights: Lock pages in memory for PAWs](#)
  16. [REQ-PAW-107 - Configure User Rights: Manage auditing and security log for PAWs](#)
  17. [REQ-PAW-108 - Configure User Rights: Modify firmware environment values for PAWs](#)
  18. [REQ-PAW-109 - Configure User Rights: Perform volume maintenance tasks for PAWs](#)
  19. [REQ-PAW-110 - Configure User Rights: Profile single process for PAWs](#)
  20. [REQ-PAW-111 - Configure User Rights: Restore files and directories for PAWs](#)
  21. [REQ-PAW-112 - Configure User Rights: Take ownership of files or other objects for PAWs](#)
  22. [REQ-PAW-113 - Configure User Rights: Deny access to this computer from the network for PAWs](#)
  23. [REQ-PAW-114 - Configure User Rights: Deny log on through Remote Desktop Services for PAWs](#)
- 

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-PAW-092] Configure User Rights: Access Credential Manager as a trusted caller for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Low \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Access Credential Manager as a trusted caller` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeTrustedCredManAccessPrivilege` set to `No one (Empty)`.

---

## Rationale No security principals should hold this privilege on PAWs. It prevents attackers from using compromised components to access stored credentials in the Credential Manager.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeTrustedCredManAccessPrivilege` to `No one (Empty)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Access Credential Manager as a trusted caller`. 3. Configure the security principal allocation to: `No one (Empty)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-



## PawUraSeTrustedCredManAccessPrivilege.ps1[../implementation\_scripts/Configure-PawUraSeTrustedCredManAccessPrivilege.ps1]

```
Configure-PawUraSeTrustedCredManAccessPrivilege.ps1
Configure-PawUraSeTrustedCredManAccessPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_settrustedcredmanaccessprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_settrustedcredmanaccessprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^([.]*)\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeTrustedCredManAccessPrivilege\s*=") {
 $NewLines += "SeTrustedCredManAccessPrivilege = "
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeTrustedCredManAccessPrivilege = ")
 } else {
 $NewLines += "SeTrustedCredManAccessPrivilege = "
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeTrustedCredManAccessPrivilegeStatus.ps1](#)

```
Get-PawUraSeTrustedCredManAccessPrivilegeStatus.ps1
Get-PawUraSeTrustedCredManAccessPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_settrustedcredmanaccessprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeTrustedCredManAccessPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-093] Configure User Rights: Access this computer from the network for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Access this computer from the network` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeNetworkLogonRight` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to connect to the computer over the network. Restricting this to Administrators and Remote Desktop Users prevents remote lateral movement by standard domain accounts.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeNetworkLogonRight` to `\*S-1-5-32-544 (Administrators)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Access this computer from the network`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUraSeNetworkLogonRight.ps1](#)]

(../implementation\_scripts/Configure-PawUraSeNetworkLogonRight.ps1)

```
Configure-PawUraSeNetworkLogonRight.ps1
Configure-PawUraSeNetworkLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_senetworklogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_senetworklogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\\[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^\\[\\.*\\]$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^\\s*SeNetworkLogonRight\\s*=") {
 $NewLines += "SeNetworkLogonRight = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeNetworkLogonRight = *S-1-5-32-544")
 } else {
 $NewLines += "SeNetworkLogonRight = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeNetworkLogonRightStatus.ps1](#)

```
Get-PawUraSeNetworkLogonRightStatus.ps1
Get-PawUraSeNetworkLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_senetworklogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" " -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeNetworkLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-094] Configure User Rights: Act as part of the operating system for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Act as part of the operating system` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeTcbPrivilege` set to `No one (Empty)`.

---

## Rationale Allows a process to impersonate any user without credentials. This privilege is equivalent to local SYSTEM and should be completely empty to prevent privilege escalation.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeTcbPrivilege` to `No one (Empty)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Act as part of the operating system`. 3. Configure the security principal allocation to: `No one (Empty)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUraSeTcbPrivilege.ps1](#)]

(../implementation\_scripts/Configure-PawUraSeTcbPrivilege.ps1)

```
Configure-PawUraSeTcbPrivilege.ps1
Configure-PawUraSeTcbPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_setcbprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_setcbprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^([.]*)\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeTcbPrivilege{s*}") {
 $NewLines += "SeTcbPrivilege = "
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeTcbPrivilege = ")
 } else {
 $NewLines += "SeTcbPrivilege = "
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeTcbPrivilegeStatus.ps1](#)

```
Get-PawUraSeTcbPrivilegeStatus.ps1
Get-PawUraSeTcbPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_setcbprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeTcbPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications



# [REQ-PAW-095] Configure User Rights: Allow log on locally for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Allow log on locally` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeInteractiveLogonRight` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to log on interactively at the computer console. Enforcing restriction to Administrators and standard local Users prevents unauthorized local sessions.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeInteractiveLogonRight` to `\*S-1-5-32-544 (Administrators)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Allow log on locally`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

PawUraSeInteractiveLogonRight.ps1][../implementation\_scripts/Configure-PawUraSeInteractiveLogonRight.ps1)

```
Configure-PawUraSeInteractiveLogonRight.ps1
Configure-PawUraSeInteractiveLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seinteractivelogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seinteractivelogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^([.]*)\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeInteractiveLogonRight\s*=") {
 $NewLines += "SeInteractiveLogonRight = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeInteractiveLogonRight = *S-1-5-32-544")
 } else {
 $NewLines += "SeInteractiveLogonRight = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeInteractiveLogonRightStatus.ps1](#)

```
Get-PawUraSeInteractiveLogonRightStatus.ps1
Get-PawUraSeInteractiveLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seinteractiveLogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeInteractiveLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-096] Configure User Rights: Back up files and directories for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Back up files and directories` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeBackupPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to bypass file permissions to read all files on the system. Restricting it to Administrators prevents data exfiltration by unauthorized users.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeBackupPrivilege` to `\*S-1-5-32-544 (Administrators)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Back up files and directories`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUraSeBackupPrivilege.ps1](#)]

(../implementation\_scripts/Configure-PawUraSeBackupPrivilege.ps1)

```
Configure-PawUraSeBackupPrivilege.ps1
Configure-PawUraSeBackupPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sebackupprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sebackupprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^([.]*)\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "\s*SeBackupPrivilege\s*=") {
 $NewLines += "SeBackupPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeBackupPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeBackupPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeBackupPrivilegeStatus.ps1](#)

```

Get-PawUraSeBackupPrivilegeStatus.ps1
Get-PawUraSeBackupPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sebackupprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" " -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeBackupPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}

```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-097] Configure User Rights: Create a pagefile for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Low \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Create a pagefile` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeCreatePagefilePrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to create and change the size of page files. Restricting this to Administrators prevents standard users from exhausting disk space or dumping system paging data.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeCreatePagefilePrivilege` to `\*S-1-5-32-544 (Administrators)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Create a pagefile`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

PawUraSeCreatePagefilePrivilege.ps1[../implementation\_scripts/Configure-PawUraSeCreatePagefilePrivilege.ps1]

```
Configure-PawUraSeCreatePagefilePrivilege.ps1
Configure-PawUraSeCreatePagefilePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_secreatepagefileprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_secreatepagefileprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\\[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^\\[\\.*\\]$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "\\s*SeCreatePagefilePrivilege\\s*=") {
 $NewLines += "SeCreatePagefilePrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeCreatePagefilePrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeCreatePagefilePrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeCreatePagefilePrivilegeStatus.ps1](#)



```
Get-PawUraSeCreatePagefilePrivilegeStatus.ps1
Get-PawUraSeCreatePagefilePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_secreatepagefileprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeCreatePagefilePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-098] Configure User Rights: Create a token object for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Create a token object` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeCreateTokenPrivilege` set to `No one (Empty)`.

---

## Rationale Allows a process to create access tokens. This can be used to escalate privileges to SYSTEM. The privilege must be completely empty.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeCreateTokenPrivilege` to `No one (Empty)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Create a token object`. 3. Configure the security principal allocation to: `No one (Empty)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUraSeCreateTokenPrivilege.ps1](#)]

(../implementation\_scripts/Configure-PawUraSeCreateTokenPrivilege.ps1)

```
Configure-PawUraSeCreateTokenPrivilege.ps1
Configure-PawUraSeCreateTokenPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_secreatetokenprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_secreatetokenprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\\[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^\\([.*]\\)$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "\\s*SeCreateTokenPrivilege\\s*=") {
 $NewLines += "SeCreateTokenPrivilege = "
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeCreateTokenPrivilege = ")
 } else {
 $NewLines += "SeCreateTokenPrivilege = "
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeCreateTokenPrivilegeStatus.ps1](#)

```
Get-PawUraSeCreateTokenPrivilegeStatus.ps1
Get-PawUraSeCreateTokenPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_secreatetokenprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeCreateTokenPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-099] Configure User Rights: Create global objects for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Create global objects` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeCreateGlobalPrivilege` set to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators), \*S-1-5-6 (Service)`.

---

## Rationale Allows processes to create global objects available to all sessions. Restricting this to system services and Administrators prevents local privilege escalation via object name collisions.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeCreateGlobalPrivilege` to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators), \*S-1-5-6 (Service)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Create global objects`. 3. Configure the security principal allocation to: `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators), \*S-1-5-6 (Service)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

PawUraSeCreateGlobalPrivilege.ps1[../implementation\_scripts/Configure-PawUraSeCreateGlobalPrivilege.ps1]

```
Configure-PawUraSeCreateGlobalPrivilege.ps1
Configure-PawUraSeCreateGlobalPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_secreateglobalprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_secreateglobalprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^([.*])$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "\s*SeCreateGlobalPrivilege\s*=") {
 $NewLines += "SeCreateGlobalPrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeCreateGlobalPrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6")
 } else {
 $NewLines += "SeCreateGlobalPrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeCreateGlobalPrivilegeStatus.ps1](#)

```
Get-PawUraSeCreateGlobalPrivilegeStatus.ps1
Get-PawUraSeCreateGlobalPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_secreateglobalprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeCreateGlobalPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-100] Configure User Rights: Create permanent shared objects for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Create permanent shared objects` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeCreatePermanentPrivilege` set to `No one (Empty)`.

---

## Rationale Allows a process to create directory objects in the object manager. It should be empty to prevent attackers from using it to hide processes or files.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeCreatePermanentPrivilege` to `No one (Empty)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Create permanent shared objects`. 3. Configure the security principal allocation to: `No one (Empty)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-



PawUraSeCreatePermanentPrivilege.ps1](../implementation\_scripts/Configure-PawUraSeCreatePermanentPrivilege.ps1)

```
Configure-PawUraSeCreatePermanentPrivilege.ps1
Configure-PawUraSeCreatePermanentPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_secreatepermanentprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_secreatepermanentprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^\[.*\]$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeCreatePermanentPrivilege\s*=") {
 $NewLines += "SeCreatePermanentPrivilege = "
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeCreatePermanentPrivilege = ")
 } else {
 $NewLines += "SeCreatePermanentPrivilege = "
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeCreatePermanentPrivilegeStatus.ps1](#)

```
Get-PawUraSeCreatePermanentPrivilegeStatus.ps1
Get-PawUraSeCreatePermanentPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_secreatepermanentprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeCreatePermanentPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-101] Configure User Rights: Debug programs for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Debug programs` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeDebugPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows processes to attach to and debug any system process (including lsass.exe). Disallowing it for standard accounts prevents credential dumping tools (Mimikatz) from reading LSASS memory.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeDebugPrivilege` to `\*S-1-5-32-544 (Administrators)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Debug programs`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUraSeDebugPrivilege.ps1](#)]

(../implementation\_scripts/Configure-PawUraSeDebugPrivilege.ps1)

```
Configure-PawUraSeDebugPrivilege.ps1
Configure-PawUraSeDebugPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sedebugprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sedebugprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^([.]*)\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeDebugPrivilege\s*=") {
 $NewLines += "SeDebugPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeDebugPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeDebugPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeDebugPrivilegeStatus.ps1](#)

```
Get-PawUraSeDebugPrivilegeStatus.ps1
Get-PawUraSeDebugPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sedebugprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeDebugPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-102] Configure User Rights: Enable computer and user accounts to be trusted for delegation for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer

Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Enable computer and user accounts to be trusted for delegation` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeEnableDelegationPrivilege` set to `No one (Empty)`.

---

## Rationale Allows users to change the delegation setting on a user or computer object in AD. This must be completely empty to prevent Kerberos delegation exploits (such as unconstrained delegation or coercion attacks).

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeEnableDelegationPrivilege` to `No one (Empty)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Enable computer and user accounts to be trusted for delegation`. 3. Configure the security principal allocation to: `No one (Empty)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

PawUraSeEnableDelegationPrivilege.ps1[../implementation\_scripts/Configure-PawUraSeEnableDelegationPrivilege.ps1]

```
Configure-PawUraSeEnableDelegationPrivilege.ps1
Configure-PawUraSeEnableDelegationPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seenabledelegationprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seenabledelegationprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^([.]*)\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeEnableDelegationPrivilege\s*=") {
 $NewLines += "SeEnableDelegationPrivilege = "
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeEnableDelegationPrivilege = ")
 } else {
 $NewLines += "SeEnableDelegationPrivilege = "
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeEnableDelegationPrivilegeStatus.ps1](#)

```
Get-PawUraSeEnableDelegationPrivilegeStatus.ps1
Get-PawUraSeEnableDelegationPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seenabledelegationprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeEnableDelegationPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications



# [REQ-PAW-103] Configure User Rights: Force shutdown from a remote system for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Force shutdown from a remote system` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeRemoteShutdownPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to shut down computers from a remote network location. Restricting this prevents denial-of-service shutdowns by compromised standard accounts.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeRemoteShutdownPrivilege` to `\*S-1-5-32-544 (Administrators)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Force shutdown from a remote system`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

PawUraSeRemoteShutdownPrivilege.ps1](../implementation\_scripts/Configure-PawUraSeRemoteShutdownPrivilege.ps1)

```
Configure-PawUraSeRemoteShutdownPrivilege.ps1
Configure-PawUraSeRemoteShutdownPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seremotesshutdownprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seremotesshutdownprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\\[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^\\([.*\\)]$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "\\s*SeRemoteShutdownPrivilege\\s*=") {
 $NewLines += "SeRemoteShutdownPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeRemoteShutdownPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeRemoteShutdownPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeRemoteShutdownPrivilegeStatus.ps1](#)

```
Get-PawUraSeRemoteShutdownPrivilegeStatus.ps1
Get-PawUraSeRemoteShutdownPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seremoteshutdownprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeRemoteShutdownPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-104] Configure User Rights: Impersonate a client after authentication for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Impersonate a client after authentication` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeImpersonatePrivilege` set to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators), \*S-1-5-6 (Service)`.

---

## Rationale Allows programs to impersonate clients. Restricting this to system service accounts and Administrators prevents local privilege escalation via print spooler or potato exploits.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeImpersonatePrivilege` to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators), \*S-1-5-6 (Service)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Impersonate a client after authentication`. 3. Configure the security principal allocation to: `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators), \*S-1-5-6 (Service)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUraSeImpersonatePrivilege.ps1](#)]

(../implementation\_scripts/Configure-PawUraSeImpersonatePrivilege.ps1)

```
Configure-PawUraSeImpersonatePrivilege.ps1
Configure-PawUraSeImpersonatePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seimpersonateprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seimpersonateprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[(.*)\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "\s*SeImpersonatePrivilege\s*=") {
 $NewLines += "SeImpersonatePrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeImpersonatePrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6")
 } else {
 $NewLines += "SeImpersonatePrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeImpersonatePrivilegeStatus.ps1](#)

```
Get-PawUraSeImpersonatePrivilegeStatus.ps1
Get-PawUraSeImpersonatePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seimpersonateprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" " -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeImpersonatePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-105] Configure User Rights: Load and unload device drivers for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Load and unload device drivers` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeLoadDriverPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to load device drivers in kernel mode. Restricting this prevents attackers from loading unsigned or vulnerable drivers (BYOVD) to execute kernel shellcode.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeLoadDriverPrivilege` to `\*S-1-5-32-544 (Administrators)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Load and unload device drivers`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUraSeLoadDriverPrivilege.ps1](#)]

(../implementation\_scripts/Configure-PawUraSeLoadDriverPrivilege.ps1)

```
Configure-PawUraSeLoadDriverPrivilege.ps1
Configure-PawUraSeLoadDriverPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seloaddriverprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seloaddriverprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^([.*])$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeLoadDriverPrivilege\s*=") {
 $NewLines += "SeLoadDriverPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeLoadDriverPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeLoadDriverPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeLoadDriverPrivilegeStatus.ps1](#)



```
Get-PawUraSeLoadDriverPrivilegeStatus.ps1
Get-PawUraSeLoadDriverPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seloaddriverprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeLoadDriverPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-106] Configure User Rights: Lock pages in memory for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Lock pages in memory` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeLockMemoryPrivilege` set to `No one (Empty)`.

---

## Rationale Allows processes to lock physical pages in memory, preventing the OS from swapping them to pagefile. It should be empty to prevent memory exhaustion DoS attacks.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeLockMemoryPrivilege` to `No one (Empty)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Lock pages in memory`. 3. Configure the security principal allocation to: `No one (Empty)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

PawUraSeLockMemoryPrivilege.ps1[../implementation\_scripts/Configure-PawUraSeLockMemoryPrivilege.ps1]

```
Configure-PawUraSeLockMemoryPrivilege.ps1
Configure-PawUraSeLockMemoryPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_selockmemoryprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_selockmemoryprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^([.]*)\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "\s*SeLockMemoryPrivilege\s*=") {
 $NewLines += "SeLockMemoryPrivilege = "
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeLockMemoryPrivilege = ")
 } else {
 $NewLines += "SeLockMemoryPrivilege = "
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeLockMemoryPrivilegeStatus.ps1](#)

```
Get-PawUraSeLockMemoryPrivilegeStatus.ps1
Get-PawUraSeLockMemoryPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_selockmemoryprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeLockMemoryPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-107] Configure User Rights: Manage auditing and security log for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Manage auditing and security log` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeSecurityPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to view, manage, and clear the Windows Security Event Log. Restricting this prevents attackers from clearing evidence of post-compromise activities.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeSecurityPrivilege` to `\*S-1-5-32-544 (Administrators)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Manage auditing and security log`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUraSeSecurityPrivilege.ps1](#)]

(../implementation\_scripts/Configure-PawUraSeSecurityPrivilege.ps1)

```
Configure-PawUraSeSecurityPrivilege.ps1
Configure-PawUraSeSecurityPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sesecurityprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sesecurityprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^([.]*)\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeSecurityPrivilege\s*=") {
 $NewLines += "SeSecurityPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeSecurityPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeSecurityPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeSecurityPrivilegeStatus.ps1](#)

```
Get-PawUraSeSecurityPrivilegeStatus.ps1
Get-PawUraSeSecurityPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sesecurityprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeSecurityPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-108] Configure User Rights: Modify firmware environment values for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Modify firmware environment values` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeSystemEnvironmentPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to configure firmware (UEFI) variables. Restricting this prevents attackers from disabling Secure Boot or modifying boot sector configurations.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeSystemEnvironmentPrivilege` to `\*S-1-5-32-544 (Administrators)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Modify firmware environment values`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-



PawUraSeSystemEnvironmentPrivilege.ps1](../implementation\_scripts/Configure-PawUraSeSystemEnvironmentPrivilege.ps1)

```
Configure-PawUraSeSystemEnvironmentPrivilege.ps1
Configure-PawUraSeSystemEnvironmentPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sesystemenvironmentprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sesystemenvironmentprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^\[.*\]$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^\s*SeSystemEnvironmentPrivilege\s*=") {
 $NewLines += "SeSystemEnvironmentPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeSystemEnvironmentPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeSystemEnvironmentPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeSystemEnvironmentPrivilegeStatus.ps1](#)

```
Get-PawUraSeSystemEnvironmentPrivilegeStatus.ps1
Get-PawUraSeSystemEnvironmentPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sesystemenvironmentprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeSystemEnvironmentPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-109] Configure User Rights: Perform volume maintenance tasks for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Perform volume maintenance tasks` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeManageVolumePrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows non-administrative users to run disk utilities. Restricting this to Administrators prevents unauthorized read/write access to raw volume sectors.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeManageVolumePrivilege` to `\*S-1-5-32-544 (Administrators)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Perform volume maintenance tasks`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

PawUraSeManageVolumePrivilege.ps1](../implementation\_scripts/Configure-PawUraSeManageVolumePrivilege.ps1)

```
Configure-PawUraSeManageVolumePrivilege.ps1
Configure-PawUraSeManageVolumePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_semanagevolumeprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_semanagevolumeprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^([.*])$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeManageVolumePrivilege\s*=") {
 $NewLines += "SeManageVolumePrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeManageVolumePrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeManageVolumePrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeManageVolumePrivilegeStatus.ps1](#)

```
Get-PawUraSeManageVolumePrivilegeStatus.ps1
Get-PawUraSeManageVolumePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_semanagevolumeprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeManageVolumePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-110] Configure User Rights: Profile single process for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Low \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Profile single process` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeProfileSingleProcessPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to profile non-system processes. Restricting this prevents attackers from profiling critical application components to locate vulnerability PAWs.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeProfileSingleProcessPrivilege` to `\*S-1-5-32-544 (Administrators)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Profile single process`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

PawUraSeProfileSingleProcessPrivilege.ps1](../implementation\_scripts/Configure-PawUraSeProfileSingleProcessPrivilege.ps1)

```
Configure-PawUraSeProfileSingleProcessPrivilege.ps1
Configure-PawUraSeProfileSingleProcessPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seprofilesinglprocessprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seprofilesinglprocessprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^([.]*)\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeProfileSingleProcessPrivilege\s*=") {
 $NewLines += "SeProfileSingleProcessPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeProfileSingleProcessPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeProfileSingleProcessPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeProfileSingleProcessPrivilegeStatus.ps1](#)

```
Get-PawUraSeProfileSingleProcessPrivilegeStatus.ps1
Get-PawUraSeProfileSingleProcessPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seprofilesinglprocessprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeProfileSingleProcessPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications



# [REQ-PAW-111] Configure User Rights: Restore files and directories for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Restore files and directories` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeRestorePrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to bypass file permissions when restoring files. Restricting this prevents attackers from overwriting critical system files to establish persistent access.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeRestorePrivilege` to `\*S-1-5-32-544 (Administrators)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Restore files and directories`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUraSeRestorePrivilege.ps1](#)]

(../implementation\_scripts/Configure-PawUraSeRestorePrivilege.ps1)

```
Configure-PawUraSeRestorePrivilege.ps1
Configure-PawUraSeRestorePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_serestoreprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_serestoreprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^\[.*\]$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "\s*SeRestorePrivilege\s*=") {
 $NewLines += "SeRestorePrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeRestorePrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeRestorePrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeRestorePrivilegeStatus.ps1](#)

```
Get-PawUraSeRestorePrivilegeStatus.ps1
Get-PawUraSeRestorePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_serestoreprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeRestorePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-112] Configure User Rights: Take ownership of files or other objects for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Take ownership of files or other objects` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeTakeOwnershipPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to take ownership of any system object. Restricting this prevents attackers from seizing control of critical files, services, or registry keys.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeTakeOwnershipPrivilege` to `\*S-1-5-32-544 (Administrators)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Take ownership of files or other objects`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

PawUraSeTakeOwnershipPrivilege.ps1](../implementation\_scripts/Configure-PawUraSeTakeOwnershipPrivilege.ps1)

```
Configure-PawUraSeTakeOwnershipPrivilege.ps1
Configure-PawUraSeTakeOwnershipPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_setakeownershipprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_setakeownershipprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\\[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^\\[\\.*\\]$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "\\s*SeTakeOwnershipPrivilege\\s*=") {
 $NewLines += "SeTakeOwnershipPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeTakeOwnershipPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeTakeOwnershipPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeTakeOwnershipPrivilegeStatus.ps1](#)

```
Get-PawUraSeTakeOwnershipPrivilegeStatus.ps1
Get-PawUraSeTakeOwnershipPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_setakeownershipprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeTakeOwnershipPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-113] Configure User Rights: Deny access to this computer from the network for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer

Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Deny access to this computer from the network` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeDenyNetworkLogonRight` set to `\*S-1-5-113 (Local Account), \*S-1-5-114 (Local Account and member of Administrators group)`.

---

## Rationale Explicitly blocks network logon for Local Accounts (S-1-5-113) and Local Administrators (S-1-5-114). This prevents attackers from performing remote lateral movement using compromised local account credentials.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeDenyNetworkLogonRight` to `\*S-1-5-113 (Local Account), \*S-1-5-114 (Local Account and member of Administrators group)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Deny access to this computer from the network`. 3. Configure the security principal allocation to: `\*S-1-5-113 (Local Account), \*S-1-5-114 (Local Account and member of Administrators group)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

PawUraSeDenyNetworkLogonRight.ps1][../implementation\_scripts/Configure-PawUraSeDenyNetworkLogonRight.ps1)

```
Configure-PawUraSeDenyNetworkLogonRight.ps1
Configure-PawUraSeDenyNetworkLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sedenynetworklogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sedenynetworklogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\\[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^\\([.*]\\)$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "\\s*SeDenyNetworkLogonRight\\s*=") {
 $NewLines += "SeDenyNetworkLogonRight = *S-1-5-113,*S-1-5-114"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeDenyNetworkLogonRight = *S-1-5-113,*S-1-5-114")
 } else {
 $NewLines += "SeDenyNetworkLogonRight = *S-1-5-113,*S-1-5-114"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeDenyNetworkLogonRightStatus.ps1](#)



```
Get-PawUraSeDenyNetworkLogonRightStatus.ps1
Get-PawUraSeDenyNetworkLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sedenynetworklogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeDenyNetworkLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-113,*S-1-5-114"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-114] Configure User Rights: Deny log on through Remote Desktop Services for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Deny log on through Remote Desktop Services` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeDenyRemoteInteractiveLogonRight` set to `\*S-1-5-113 (Local Account), \*S-1-5-114 (Local Account and member of Administrators group)`.

---

## Rationale Explicitly blocks Remote Desktop logons for Local Accounts (S-1-5-113) and Local Administrators (S-1-5-114), forcing administrators to connect using domain credentials over secure paths.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeDenyRemoteInteractiveLogonRight` to `\*S-1-5-113 (Local Account), \*S-1-5-114 (Local Account and member of Administrators group)` enforces maximum console and credential isolation. No productivity tools or standard non-administrative domain sessions should exist on PAW consoles.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Deny log on through Remote Desktop Services`. 3. Configure the security principal allocation to: `\*S-1-5-113 (Local Account), \*S-1-5-114 (Local Account and member of Administrators group)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

PawUraSeDenyRemoteInteractiveLogonRight.ps1][../implementation\_scripts/Configure-PawUraSeDenyRemoteInteractiveLogonRight.ps1)

```
Configure-PawUraSeDenyRemoteInteractiveLogonRight.ps1
Configure-PawUraSeDenyRemoteInteractiveLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sedenyremoteinteractivelogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sedenyremoteinteractivelogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^\[.*\]$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeDenyRemoteInteractiveLogonRight\s*=") {
 $NewLines += "SeDenyRemoteInteractiveLogonRight = *S-1-5-113,*S-1-5-114"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeDenyRemoteInteractiveLogonRight = *S-1-5-113,*S-1-5-114")
 } else {
 $NewLines += "SeDenyRemoteInteractiveLogonRight = *S-1-5-113,*S-1-5-114"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-PawUraSeDenyRemoteInteractiveLogonRightStatus.ps1](#)

```
Get-PawUraSeDenyRemoteInteractiveLogonRightStatus.ps1
Get-PawUraSeDenyRemoteInteractiveLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sedenyremoteinteractivelogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeDenyRemoteInteractiveLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-113,*S-1-5-114"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Protective controls baselines on Privileged Access Workstations \* \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-PAW-010] Enable VBS and Credential Guard for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: Computer Configuration\Administrative Templates\System\Device Guard\Turn On Virtualization Based Security \* \*\*Registry Location\*\*: HKLM\SOFTWARE\Policies\Microsoft\Windows\DeviceGuard \* `EnableVirtualizationBasedSecurity` = `1` (REG\_DWORD) \* `RequirePlatformSecurityFeatures` = `1` (REG\_DWORD, Secure Boot) \* `HypervisorEnforcedCodeIntegrity` = `1` (REG\_DWORD) \* `LsaCfgFlags` = `1` (REG\_DWORD, Credential Guard Enabled with UEFI Lock) \* `ConfigureSystemGuardLaunch` = `1` (REG\_DWORD, Secure Launch Enabled) \* `HVCIMATRequired` = `1` (REG\_DWORD, Require UEFI Memory Attributes Table)

---

## Rationale Privileged Access Workstations (PAWs) contain Tier 0 administrative tokens. A compromise of a PAW leads to a direct compromise of the Active Directory database (NTDS.dit) and full domain domain control. Mitigating credential dumping is the single most critical security objective for a PAW.

1. **Virtualization-Based Security (VBS)**: VBS establishes an isolated, secure kernel space using hypervisor hardware virtualization. This secure kernel is separated from the host operating system, preventing root-level exploits from accessing virtualized memory blocks.
  2. **Credential Guard**: Running within the VBS secure kernel, Credential Guard stores credential secrets (NTLM hashes, Kerberos TGTs) inside an isolated memory container. By shifting these secrets outside the standard Local Security Authority (LSA) process memory space, it blocks credential-dumping utilities (like Mimikatz) from harvesting secrets from memory.
  3. **Secure Launch**: System Guard Secure Launch protects firmware boot integrity by using hardware-enforced boot measurements. It isolates the hypervisor startup from potential rootkits or boot-level malware.
  4. **UEFI Memory Attributes Table (MAT)**: Enforcing UEFI MAT ensures that the bootloader validates page permissions in firmware, preventing buffer overflow or execution redirection vulnerabilities in pre-boot configurations.
- 

## Legacy Impact & Compatibility \* \*\*Firmware Requirements\*\*: PAWs must use modern UEFI firmware, native UEFI boot (Legacy CSM disabled), Secure Boot, IOMMU (Intel VT-d or AMD-Vi), CPU Virtualization (Intel VT-x or AMD-V), and TPM 2.0. Enabling Secure Boot and virtualization functions is a strict pre-requisite. Refer to [REQ-PAW-005 - UEFI Firmware Security Hardening](#07-paws-configure-uefi-security-md) and [REQ-PAW-006 - Enable Hardware Virtualization and DMA Protection](#07-paws-enable-hardware-virtualization-and-dma-protection-md) to secure these features in the firmware. If physical hardware does not meet these criteria, it is unfit for use as a PAW. \* \*\*Hypervisor Conflicts\*\*: Standard Windows virtualization layers will be required. Running non-compliant third-party hypervisors (such as older VirtualBox or VMware Workstation configurations) that do not support nested virtualization on Hyper-V will fail.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
  2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
  3. Navigate to: `Computer Configuration\Administrative Templates\System\Device Guard`
  4. Configure the setting:
    - **Policy**: `Turn On Virtualization Based Security`
    - **Setting**: `Enabled`
    - **Select Platform Security Level**: `Secure Boot`
    - **Virtualization Based Protection of Code Integrity**: `Enabled with UEFI Lock`
    - **Credential Guard Configuration**: `Enabled with UEFI Lock`
    - **Secure Launch Configuration**: `Enabled`
    - **Require UEFI Memory Attributes Table**: `Enabled`
  5. Link the GPO to the PAWs Organizational Unit (OU).
- 

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Configure the local registry parameters to activate VBS, Credential Guard, and Secure Launch.

[Download Script: Enable-PawVBSCredentialGuard.ps1](#)

```
Enable-PawVBSCredentialGuard.ps1
Description: Configures local registry keys to activate VBS and Credential Guard on PAWs.

Write-Host "--- Enforcing VBS & Credential Guard for PAWs ---" -ForegroundColor Cyan

$DeviceGuardPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeviceGuard"

if (-not (Test-Path $DeviceGuardPath)) {
 New-Item -Path $DeviceGuardPath -Force | Out-Null
}

Enable Virtualization-Based Security (VBS)
Set-ItemProperty -Path $DeviceGuardPath -Name "EnableVirtualizationBasedSecurity" -Value 1 -Type DWord
RequirePlatformSecurityFeatures = 1 (Secure Boot)
Set-ItemProperty -Path $DeviceGuardPath -Name "RequirePlatformSecurityFeatures" -Value 1 -Type DWord
HypervisorEnforcedCodeIntegrity = 1 (HvCI / Memory Integrity Enabled)
Set-ItemProperty -Path $DeviceGuardPath -Name "HypervisorEnforcedCodeIntegrity" -Value 1 -Type DWord
LsaCfgFlags = 1 (Credential Guard Enabled with UEFI Lock)
Set-ItemProperty -Path $DeviceGuardPath -Name "LsaCfgFlags" -Value 1 -Type DWord
ConfigureSystemGuardLaunch = 1 (Secure Launch Enabled)
Set-ItemProperty -Path $DeviceGuardPath -Name "ConfigureSystemGuardLaunch" -Value 1 -Type DWord
HVCIMATRequired = 1 (Require UEFI Memory Attributes Table)
Set-ItemProperty -Path $DeviceGuardPath -Name "HVCIMATRequired" -Value 1 -Type DWord

Write-Host "[+] PAW VBS and Credential Guard registry settings applied. (Reboot required)." -ForegroundColor Green
```

To audit VBS and Credential Guard status using WMI and Registry: [Download Script: Test-PawVBSCredentialGuard.ps1](#)

```
Test-PawVBSCredentialGuard.ps1
Description: Queries the local Win32_DeviceGuard class and registry settings to verify VBS protection states on PAWs.

Write-Host "--- Auditing PAW Virtualization-Based Security Baseline ---" -ForegroundColor Cyan

try {
 $DG = Get-CimInstance -Namespace "Root\Microsoft\Windows\DeviceGuard" -ClassName "Win32_DeviceGuard" -ErrorAction Stop

 # SecurityServicesRunning: 1 = Credential Guard, 2 = HvCI
 $CredGuardRunning = $DG.SecurityServicesRunning -contains 1
 $HvciRunning = $DG.SecurityServicesRunning -contains 2

 $VbsColor = if ($DG.VirtualizationBasedSecurityStatus -eq 2) { "Green" } else { "Red" }
 $CredColor = if ($CredGuardRunning) { "Green" } else { "Red" }
 $HvciColor = if ($HvciRunning) { "Green" } else { "Red" }

 Write-Host " - VBS Status: $($DG.VirtualizationBasedSecurityStatus) (Required = 2 [Running])" -ForegroundColor $VbsColor
 Write-Host " - Credential Guard Running: $CredGuardRunning (Required = True)" -ForegroundColor $CredColor
 Write-Host " - Hypervisor Code Integrity Running: $HvciRunning (Required = True)" -ForegroundColor $HvciColor

 # Query registry properties for System Guard and UEFI MAT
 $SystemGuard = (Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeviceGuard" -Name "ConfigureSystemGuardLaunch" -ErrorAction SilentlyContinue).ConfigureSystemGuardLaunch
 $MatRequired = (Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeviceGuard" -Name "HVCIMATRequired" -ErrorAction SilentlyContinue).HVCIMATRequired

 $SgColor = if ($SystemGuard -eq 1) { "Green" } else { "Red" }
 $MatColor = if ($MatRequired -eq 1) { "Green" } else { "Red" }

 Write-Host " - System Guard Secure Launch: $SystemGuard (Required = 1)" -ForegroundColor $SgColor
 Write-Host " - UEFI Memory Attributes Table Required: $MatRequired (Required = 1)" -ForegroundColor $MatColor
} catch {
 Write-Host " - VULNERABLE: DeviceGuard WMI class could not be queried. VBS is likely disabled." -ForegroundColor Red
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*\*: Section 18.8.14.1 (Turn On Virtualization Based Security), Section 18.8.14.2 (Turn On Virtualization Based Security: Credential Guard Configuration) \* \*\*ANSSI AD Hardening Guide\*\*\*: Recommendations regarding administrative workstation isolation and credential protections.

# [REQ-PAW-011] Harden DMA and Physical Security for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Paths / Registry Locations\*\*: \* \*\*GPO Paths\*\*: \* Computer Configuration\Administrative Templates\System\Power Management\Sleep Settings \* Computer Configuration\Administrative Templates\System\Device Installation\Device Installation Restrictions \* Computer Configuration\Administrative Templates\Windows Components\BitLocker Drive Encryption \* \*\*Registry Locations\*\*: \* HKLM\SOFTWARE\Policies\Microsoft\Power\PowerSettings\abfc2519-3608-4c2a-94ea-171b0ed546ab \* `ACSettingIndex` = `0` (REG\_DWORD, Disables standby plugged in) \* `DCSettingIndex` = `0` (REG\_DWORD, Disables standby on battery) \* HKLM\SOFTWARE\Policies\Microsoft\FVE \* `DisableExternalDMAUnderLock` = `1` (REG\_DWORD) \* `RDVDenyCrossOrg` = `0` (REG\_DWORD) \* HKLM\System\CurrentControlSet\Policies\Microsoft\FVE \* `RDVDenyWriteAccess` = `1` (REG\_DWORD) \* HKLM\SOFTWARE\Policies\Microsoft\Windows\DeviceInstall\Restrictions \* `DenyDeviceClasses` = `1` (REG\_DWORD) \* `DenyDeviceClassesRetroactive` = `1` (REG\_DWORD) \* HKLM\SOFTWARE\Policies\Microsoft\Windows\DeviceInstall\Restrictions\DenyDeviceClasses \* `1` = `{d48179be-ec20-11d1-b6b8-00c04fa372a7}` (REG\_SZ, SBP-2 device setup class) \* HKLM\SOFTWARE\Policies\Microsoft\Windows\KernelDMAProtection \* `DeviceEnumerationPolicy` = `0` (REG\_DWORD, Block all)

---

## Rationale Privileged Access Workstations (PAWs) represent Tier 0 boundary systems. Because they handle the highest levels of domain authorization, physical threat vectors must be mitigated to the absolute maximum threshold:

- Direct Memory Access (DMA) Defenses:** External interfaces (e.g. Thunderbolt, USB4, PCIe ExpressCard) allow attached devices to bypass the OS and read physical RAM contents directly. Attackers use physical DMA-hacking consoles to dump memory-resident Kerberos keys and NTLM credentials.
  - Disabling the SBP-2 setup class ( `{d48179be-ec20-11d1-b6b8-00c04fa372a7}` ) blocks FireWire/IEEE 1394 DMA controllers.
  - `DisableExternalDMAUnderLock` prevents DMA access when the workstation is locked.
  - Stricter Enumeration Policy on PAWs:** Standard workstations permit external DMA after a user logs on (value 1). On PAWs, this must be set to **Block all** (value 0). External DMA-capable expansion cards or devices are permanently blocked from memory access.
- Cold Boot Exploits:** RAM retention properties mean memory remains readable for seconds or minutes after power loss, especially if cooled. If a PAW enters standby states (S1-S3), the RAM remains powered. If stolen in standby, an attacker can extract cryptographic keys. Disabling standby forces the system to either shut down completely or hibernate, locking the BitLocker keys inside the TPM.
- USB Exfiltration Protection:** Restricting write access on removable drives ( `RDVDenyWriteAccess` ) prevents the data exfiltration of administrative materials or directory backups to local USB flash drives.

---

## Legacy Impact & Compatibility \* \*\*Standby and Resume\*\*: Standby states (S1-S3) are disabled. PAWs will hibernate when closed or idle. Session restoration will require a full TPM check and secure boot validation, which adds a brief delay during startup. \* \*\*External Device Blocking\*\*: External devices requiring DMA (e.g., external GPUs, specialized expansion boxes, or legacy docks) are permanently blocked. Administrators must use native motherboard ports and authorized docks. \* \*\*Removable Storage Blocks\*\*: Administrative files cannot be written to standard USB media. Files must be distributed via secure network endpoints or designated distribution shares.

---

## Implementation Steps

#### Option A: Group Policy Object (GPO) Configuration (Preferred)

- Open the **Group Policy Management Console** ( `gpmc.msc` ).
- Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
- Configure the following settings:

##### 1. Power Management (Disable Standby) Navigate to: `Computer Configuration\Administrative Templates\System\Power Management\Sleep Settings` \* \*\*Policy\*\*: `Allow standby states (S1-S3) when sleeping (plugged in)` → **Disabled** \* \*\*Policy\*\*: `Allow standby states (S1-S3) when sleeping (on battery)` → **Disabled** \* \*\*Policy\*\*: `Require a password when a computer wakes (plugged in)` → **Enabled** \* \*\*Policy\*\*: `Require a password when a computer wakes (on battery)` → **Enabled**

##### 2. BitLocker Removable Storage & DMA Navigate to: `Computer Configuration\Administrative Templates\Windows Components\BitLocker Drive Encryption` \* \*\*Policy\*\*: `Disallow new DMA devices when this computer is locked` → **Enabled**

Navigate to: `Computer Configuration\Administrative Templates\Windows Components\BitLocker Drive Encryption\Removable Data Drives`

- Policy:** `Deny write access to removable drives not protected by BitLocker` → **Enabled**
  - Check **Do not allow write access to devices configured in another organization** → **Disabled** (value 0 / False)

##### 3. Device Installation Restrictions (Block SBP-2 Setup Class) Navigate to: `Computer Configuration\Administrative Templates\System\Device Installation\Device Installation Restrictions` \* \*\*Policy\*\*: `Prevent installation of devices using drivers that match these device setup classes` → **Enabled** \* Click **Show...** and enter: `{d48179be-ec20-11d1-b6b8-00c04fa372a7}` \* Check **Also apply to matching devices that are already installed** → **Enabled** (value 1 / True)

##### 4. Kernel DMA Protection (Block All) Navigate to: `Computer Configuration\Administrative Templates\System\Kernel DMA Protection` \* \*\*Policy\*\*: `Enable Kernel DMA Protection` → **Enabled** \* \*\*Enumeration policy\*\*: Set to **Block all** (value 0)

---

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally on the PAW to apply DMA, Sleep, and BitLocker USB registry parameters.

[Download Script: Set-PawDMAPhysicalSecurity.ps1](#)

```
Set-PawDMAPhysicalSecurity.ps1
Description: Hardens local registry keys on PAWs to mitigate DMA attacks, disable standby sleep states, and restrict unencrypted U
SB writing.

Write-Host "Applying PAW DMA and physical security hardening..." -ForegroundColor Cyan

1. Disable Standby Sleep States (S1-S3)
$SleepPath = "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\abfc2519-3608-4c2a-94ea-171b0ed546ab"
if (-not (Test-Path $SleepPath)) {
 New-Item -Path $SleepPath -Force | Out-Null
}
Set-ItemProperty -Path $SleepPath -Name "ACSettingIndex" -Value 0 -Type DWord
Set-ItemProperty -Path $SleepPath -Name "DCSettingIndex" -Value 0 -Type DWord
Write-Host "[+] Standby sleep states (S1-S3) disabled." -ForegroundColor Green

2. Configure Wake Password Requirement
$WakePath = "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\0e796bdb-100d-47d6-a2d5-f7d2daa51f51"
if (-not (Test-Path $WakePath)) {
 New-Item -Path $WakePath -Force | Out-Null
}
Set-ItemProperty -Path $WakePath -Name "ACSettingIndex" -Value 1 -Type DWord
Set-ItemProperty -Path $WakePath -Name "DCSettingIndex" -Value 1 -Type DWord
Write-Host "[+] Wake password requirement enforced." -ForegroundColor Green

3. BitLocker DMA and Removable Storage Settings
$FvePath = "HKLM:\SOFTWARE\Policies\Microsoft\FVE"
if (-not (Test-Path $FvePath)) {
 New-Item -Path $FvePath -Force | Out-Null
}
Set-ItemProperty -Path $FvePath -Name "DisableExternalDMAUnderLock" -Value 1 -Type DWord
Set-ItemProperty -Path $FvePath -Name "RDVDenyCrossOrg" -Value 0 -Type DWord

$FvePolicyPath = "HKLM:\System\CurrentControlSet\Policies\Microsoft\FVE"
if (-not (Test-Path $FvePolicyPath)) {
 New-Item -Path $FvePolicyPath -Force | Out-Null
}
Set-ItemProperty -Path $FvePolicyPath -Name "RDVDenyWriteAccess" -Value 1 -Type DWord
Write-Host "[+] BitLocker DMA under lock and unencrypted USB write blocks configured." -ForegroundColor Green

4. Device Installation Restrictions (Block SBP-2 class)
$RestrictPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeviceInstall\Restrictions"
if (-not (Test-Path $RestrictPath)) {
 New-Item -Path $RestrictPath -Force | Out-Null
}
Set-ItemProperty -Path $RestrictPath -Name "DenyDeviceClasses" -Value 1 -Type DWord
Set-ItemProperty -Path $RestrictPath -Name "DenyDeviceClassesRetroactive" -Value 1 -Type DWord

$DenyClassPath = Join-Path $RestrictPath "DenyDeviceClasses"
if (-not (Test-Path $DenyClassPath)) {
 New-Item -Path $DenyClassPath -Force | Out-Null
}
Set-ItemProperty -Path $DenyClassPath -Name "1" -Value "{d48179be-ec20-11d1-b6b8-00c04fa372a7}" -Type String
Write-Host "[+] Device installation blocks for SBP-2 class enabled." -ForegroundColor Green

5. Kernel DMA Protection (Block all external DMA permanently for PAWs)
$KDmaPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\KernelDMAProtection"
if (-not (Test-Path $KDmaPath)) {
 New-Item -Path $KDmaPath -Force | Out-Null
}
Set-ItemProperty -Path $KDmaPath -Name "DeviceEnumerationPolicy" -Value 0 -Type DWord
Write-Host "[+] Kernel DMA Protection DeviceEnumerationPolicy set to 0 (Block all)." -ForegroundColor Green

Write-Host "PAW DMA and physical security settings applied successfully." -ForegroundColor Green
```

To audit local PAW DMA and physical security configuration: [Download Script: Test-PawDMAPhysicalSecurity.ps1](#)



```
Test-PawDMAPhysicalSecurity.ps1
Description: Audits local registry configuration for standby settings, DMA protection under lock, USB restrictions, and blocked de
vice setup classes on PAWs.

Write-Host "--- Auditing PAW DMA and Physical Security ---" -ForegroundColor Cyan

1. Audit Standby Settings
$SleepPath = "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\abfc2519-3608-4c2a-94ea-171b0ed546ab"
$AcSleep = Get-ItemProperty -Path $SleepPath -Name "ACSettingIndex" -ErrorAction SilentlyContinue
$DcSleep = Get-ItemProperty -Path $SleepPath -Name "DCSettingIndex" -ErrorAction SilentlyContinue

$AcSleepVal = if ($AcSleep) { $AcSleep.ACSettingIndex } else { 1 }
$DcSleepVal = if ($DcSleep) { $DcSleep.DCSettingIndex } else { 1 }

$AcSleepColor = if ($AcSleepVal -eq 0) { "Green" } else { "Red" }
$DcSleepColor = if ($DcSleepVal -eq 0) { "Green" } else { "Red" }

Write-Host " - Standby Sleep State (Plugged In) Setting: $AcSleepVal (Required = 0 [Disabled])" -ForegroundColor $AcSleepColor
Write-Host " - Standby Sleep State (On Battery) Setting: $DcSleepVal (Required = 0 [Disabled])" -ForegroundColor $DcSleepColor

2. Audit BitLocker Settings
$FvePath = "HKLM:\SOFTWARE\Policies\Microsoft\FVE"
$DmaLock = Get-ItemProperty -Path $FvePath -Name "DisableExternalDMAUnderLock" -ErrorAction SilentlyContinue
$DmaLockVal = if ($DmaLock) { $DmaLock.DisableExternalDMAUnderLock } else { 0 }
$DmaLockColor = if ($DmaLockVal -eq 1) { "Green" } else { "Red" }

$FvePolicyPath = "HKLM:\System\CurrentControlSet\Policies\Microsoft\FVE"
$UsbWrite = Get-ItemProperty -Path $FvePolicyPath -Name "RDVDenyWriteAccess" -ErrorAction SilentlyContinue
$UsbWriteVal = if ($UsbWrite) { $UsbWrite.RDVDenyWriteAccess } else { 0 }
$UsbWriteColor = if ($UsbWriteVal -eq 1) { "Green" } else { "Red" }

Write-Host " - Disable DMA Under Lock: $DmaLockVal (Required = 1)" -ForegroundColor $DmaLockColor
Write-Host " - USB Unencrypted Write Block: $UsbWriteVal (Required = 1)" -ForegroundColor $UsbWriteColor

3. Audit Device Restriction Settings
$RestrictPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeviceInstall\Restrictions"
$DenyDev = Get-ItemProperty -Path $RestrictPath -Name "DenyDeviceClasses" -ErrorAction SilentlyContinue
$DenyDevVal = if ($DenyDev) { $DenyDev.DenyDeviceClasses } else { 0 }
$DenyDevColor = if ($DenyDevVal -eq 1) { "Green" } else { "Red" }

Write-Host " - Prevent Device Setup Class Installation: $DenyDevVal (Required = 1)" -ForegroundColor $DenyDevColor

$DenyClassPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeviceInstall\Restrictions\DenyDeviceClasses"
$Sbp2 = Get-ItemProperty -Path $DenyClassPath -Name "1" -ErrorAction SilentlyContinue
$Sbp2Val = if ($Sbp2) { $Sbp2."1" } else { "" }
$Sbp2Color = if ($Sbp2Val -eq "{d48179be-ec20-11d1-b6b8-00c04fa372a7}") { "Green" } else { "Red" }

Write-Host " - Blocked SBP-2 Setup Class: '$Sbp2Val' (Required = '{d48179be-ec20-11d1-b6b8-00c04fa372a7}')" -ForegroundColor $Sbp2Color

4. Audit Kernel DMA Protection Setting (Stricter for PAWs)
$KDmaPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\KernelDMAProtection"
$EnumPol = Get-ItemProperty -Path $KDmaPath -Name "DeviceEnumerationPolicy" -ErrorAction SilentlyContinue
$EnumPolVal = if ($EnumPol) { $EnumPol.DeviceEnumerationPolicy } else { 2 }
$EnumPolColor = if ($EnumPolVal -eq 0) { "Green" } else { "Red" }

Write-Host " - Kernel DMA Protection Policy: $EnumPolVal (Required = 0 [Block all])" -ForegroundColor $EnumPolColor
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*\*: Section 18.2.1 (BitLocker settings), Section 18.8.19.1 (Kernel DMA Protection), Section 18.8.21.3 (Device Installation restrictions) \* \*\*ANSI AD Hardening Guide\*\*\*: Recommendations on storage encryption and hardware interface security for administrative workstations

# [REQ-PAW-012] Enable WDAC Driver Blocklist

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10, Windows 11 (Enterprise and Professional editions)

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: Computer Configuration\Administrative Templates\System\Device Guard\Deploy Windows Defender Application Control \* \*\*Registry Location\*\*:  
`HKLM\SYSTEM\CurrentControlSet\Control\CI\Config` → `VulnerableDriverBlocklistEnable` = `1` (REG\_DWORD)

## Rationale Attackers frequently employ "Bring Your Own Vulnerable Driver" (BYOVD) attacks to bypass Windows kernel protections on high-value administrative assets like Privileged Access Workstations (PAWs). In a BYOVD attack, an adversary with administrative privileges installs a legitimate, cryptographically signed third-party driver that contains a known, exploitable vulnerability. The attacker then exploits this vulnerability to execute arbitrary code with kernel privileges, allowing them to disable security agents, dump LSASS memory, or tamper with system integrity.

Enforcing the **Microsoft Vulnerable Driver Blocklist** via Windows Defender Application Control (WDAC) prevents known vulnerable or malicious drivers from loading in kernel space. By restricting the WDAC policy to **Kernel Mode Code Integrity (KMCI) only** (omitting user-mode enforcement), the control shields the system kernel from driver-based exploits on administrative hosts without introducing administrative overhead or blocking standard user-mode admin applications.

## Legacy Impact & Compatibility \* \*\*Pre-requisite (Memory Integrity/HVCI)\*\*: The vulnerable driver blocklist requires Hypervisor-Protected Code Integrity (HVCI) for secure, hypervisor-enforced validation. Refer to [REQ-PAW-010 - Enable VBS and Credential Guard for PAWs](#07-paws-enable-vbs-credential-guard-md) to ensure Virtualization-Based Security (VBS) and Memory Integrity (HVCI) are fully enabled. Secure Boot and CPU virtualization are strict pre-requisites; refer to [REQ-PAW-005 - UEFI Firmware Security Hardening](#07-paws-configure-uefi-security-md) and [REQ-PAW-006 - Enable Hardware Virtualization and DMA Protection](#07-paws-enable-hardware-virtualization-and-dma-protection-md) for firmware configuration. \* \*\*Compatibility with Legacy Drivers\*\*: Third-party backup, monitoring, or hardware administration software running deprecated, vulnerable drivers may fail to load. All such software must be updated to use secure, modern drivers. \* \*\*Deployment Testing\*\*: To prevent system instability, the WDAC blocklist policy should be deployed in **Audit Mode** initially to verify that no critical operational drivers are blocked in production before shifting to enforcement mode.

#### ## Implementation Steps

##### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

To enforce the driver blocklist across all PAWs, you can deploy the Microsoft recommended block rules as a custom WDAC policy.

1. Download the Microsoft recommended driver block rules XML from the official Microsoft documentation.
2. Edit the XML to ensure it operates in **Audit Mode** first, then convert the XML configuration into a binary format:

```
ConvertFrom-CIPolicy -XmlFilePath "C:\WDAC\DriverBlocklist.xml" -BinaryFilePath "C:\WDAC\SIPolicy.p7b"
```

3. Copy the compiled `SIPolicy.p7b` file to a secure local path on all target PAWs (e.g., `C:\Windows\System32\CodeIntegrity\SIPolicy.p7b`) or a share.
4. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a domain management host.
5. Create or edit a GPO linked to the PAWs OU (e.g., `GPO_Hardening_PAWs` ).
6. Navigate to: `Computer Configuration\Administrative Templates\System\Device Guard`
7. Configure the following setting:
  - o **Policy**: `Deploy Windows Defender Application Control`
  - o **Setting**: `Enabled`
  - o **Code Integrity Policy File Path**: Enter the local or network path to the policy file (e.g., `C:\Windows\System32\CodeIntegrity\SIPolicy.p7b` ).
8. To ensure the built-in system driver blocklist is active on modern builds, configure the following registry setting via Group Policy Preferences:
  - o **Path**: `Computer Configuration\Preferences\Windows Settings\Registry`
  - o **Hive**: `HKEY_LOCAL_MACHINE`
  - o **Key Path**: `SYSTEM\CurrentControlSet\Control\CI\Config`
  - o **Value Name**: `VulnerableDriverBlocklistEnable`
  - o **Value Type**: `REG_DWORD`
  - o **Value Data**: `1`

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to enable the Vulnerable Driver Blocklist registry key and ensure proper configuration.

[Download Script: Configure-DriverBlocklist.ps1](#)

```
Configure-DriverBlocklist.ps1
Description: Enables the Microsoft Vulnerable Driver Blocklist in the registry and validates VBS/HVCI settings.

Write-Host "Applying hardening requirement: Enable WDAC Driver Blocklist..." -ForegroundColor Cyan

1. Configure the registry settings to enable the blocklist
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\CI\Config"
$ValueName = "VulnerableDriverBlocklistEnable"

if (-not (Test-Path $RegPath)) {
 Write-Host "[+] Creating registry path: $RegPath" -ForegroundColor Gray
 New-Item -Path $RegPath -Force | Out-Null
}

Write-Host "[+] Setting registry value: $ValueName = 1" -ForegroundColor Gray
Set-ItemProperty -Path $RegPath -Name $ValueName -Value 1 -Type DWord -ErrorAction Stop

2. Validate VBS / HVCI Configuration
$ScenariosPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\HypervisorEnforcedCodeIntegrity"
if (Test-Path $ScenariosPath) {
 $HvciStatus = Get-ItemProperty -Path $ScenariosPath -Name "Enabled" -ErrorAction SilentlyContinue
 if ($null -ne $HvciStatus -and $HvciStatus.Enabled -eq 1) {
 Write-Host "[+] Pre-requisite Check: Memory Integrity (HVCI) is enabled." -ForegroundColor Green
 } else {
 Write-Host "[!] Warning: Memory Integrity (HVCI) is disabled. The blocklist requires HVCI for hypervisor enforcement." -ForegroundColor Yellow
 }
} else {
 Write-Host "[!] Warning: Memory Integrity scenario configuration not found. Check VBS settings." -ForegroundColor Yellow
}

Write-Host "[+] Configuration applied successfully. A reboot is required to activate the blocklist." -ForegroundColor Green
```

*To verify the setting has been applied:*

[Download Script: Get-DriverBlocklistStatus.ps1](#)

```
Get-DriverBlocklistStatus.ps1
Description: Audits the configuration of the Microsoft Vulnerable Driver Blocklist and HVCI state.

Write-Host "--- Auditing Vulnerable Driver Blocklist ---" -ForegroundColor Cyan
$Vulnerable = $false

1. Check registry value
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\CI\Config"
$ValueName = "VulnerableDriverBlocklistEnable"

if (Test-Path $RegPath) {
 $RegValue = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
 if ($null -ne $RegValue -and $RegValue.$ValueName -eq 1) {
 Write-Host "[+] Vulnerable Driver Blocklist is enabled in the registry." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Vulnerable Driver Blocklist is disabled or not set in the registry." -ForegroundColor Red
 $Vulnerable = $true
 }
} else {
 Write-Host "[!] VULNERABLE: Code Integrity Config registry key does not exist." -ForegroundColor Red
 $Vulnerable = $true
}

2. Check HVCI Status
$ScenariosPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\HypervisorEnforcedCodeIntegrity"
if (Test-Path $ScenariosPath) {
 $HvciStatus = Get-ItemProperty -Path $ScenariosPath -Name "Enabled" -ErrorAction SilentlyContinue
 if ($null -ne $HvciStatus -and $HvciStatus.Enabled -eq 1) {
 Write-Host "[+] Memory Integrity (HVCI) is enabled." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Memory Integrity (HVCI) is disabled in the registry." -ForegroundColor Red
 $Vulnerable = $true
 }
} else {
 Write-Host "[!] VULNERABLE: Memory Integrity scenario registry path does not exist." -ForegroundColor Red
 $Vulnerable = $true
}

3. Final Verdict
if ($Vulnerable) {
 Write-Host "`n[!] Verification FAILED: The Vulnerable Driver Blocklist is not fully secured." -ForegroundColor Red
} else {
 Write-Host "`n[+] Verification PASSED: The Vulnerable Driver Blocklist and HVCI are correctly configured." -ForegroundColor Green
}
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendations on system component code integrity and driver signature enforcement. \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*\*: Section 18.8.14.3 / 18.9.31.2 (Deploy Windows Defender Application Control / Memory Integrity). \* \*\*Microsoft Security Guidance\*\*\*: Microsoft recommended driver block rules documentation.

# [REQ-PAW-013] Configure Account and Password Policies for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) (Tier 0 Workstations) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Paths / Registry Locations\*\*: \* \*\*GPO Paths\*\*: \* `Computer Configuration\Policies\Windows Settings\Security Settings\Account Policies\Account Lockout Policy` \* `Computer Configuration\Policies\Windows Settings\Security Settings\Account Policies\Password Policy` \* `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` \* `Computer Configuration\Administrative Templates\System\PIN Complexity` \* `Computer Configuration\Administrative Templates\Windows Components\Microsoft Account` \* `Computer Configuration\Administrative Templates\Windows Components\Windows Hello for Business` \* \*\*Registry Locations\*\*: \* Configured via `GptTmpl.inf` (SecEdit System Access settings): \* `MinimumPasswordLength` = `20` (20 characters minimum) \* `PasswordComplexity` = `1` (Complexity enabled) \* `PasswordHistorySize` = `24` (24 passwords remembered) \* `MaxPasswordAge` = `0` (Password does not expire / disabled) \* `MinPasswordAge` = `1` (1 day minimum) \* `ClearTextPassword` = `0` (Reversible encryption disabled) \* `LockoutBadCount` = `5` (5 invalid logon attempts allowed) \* `ResetLockoutCount` = `30` (30 minutes lockout observation window) \* `LockoutDuration` = `30` (30 minutes lockout duration) \* `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon` \* `ScRemoveOption` = `1` (REG\_SZ, 1 = Lock Workstation) \* `CachedLogonsCount` = `0` (REG\_DWORD) \* `HKLM\SECURITY\Cache` \* `NL\$IterationCount` = `1954` (REG\_DWORD, 1954 = 2,000,896 rounds of PBKDF2-SHA1) \* `HKLM\System\CurrentControlSet\Control\Lsa` \* `LimitBlankPasswordUse` = `1` (REG\_DWORD) \* `NoLmHash` = `1` (REG\_DWORD) \* `LmCompatibilityLevel` = `5` (REG\_DWORD, Network security: LAN Manager authentication level - NTLMv2 only) \* `HKLM\System\CurrentControlSet\Control\SecurityProviders\WDigest` \* `UseLogonCredential` = `0` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows\System` \* `AllowDomainPINLogon` = `0` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\PassportForWork\PINComplexity` \* `MinimumPINLength` = `6` (REG\_DWORD) \* `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System` \* `MSAOptional` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\MicrosoftAccount` \* `DisableUserAuth` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\PassportForWork` \* `RequireSecurityDevice` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\PassportForWork\ExcludeSecurityDevices` \* `TPM12` = `0` (REG\_DWORD) \* `HKLM\System\CurrentControlSet\Services\Netlogon\Parameters` \* `RequireSignOrSeal` = `1` (REG\_DWORD) \* `SealSecureChannel` = `1` (REG\_DWORD) \* `SignSecureChannel` = `1` (REG\_DWORD) \* `DisablePasswordChange` = `0` (REG\_DWORD) \* `MaximumPasswordAge` = `30` (REG\_DWORD) \* `RequireStrongKey` = `1` (REG\_DWORD) \* `HKLM\System\CurrentControlSet\Services\LanmanWorkstation\Parameters` \* `EnablePlainTextPassword` = `0` (REG\_DWORD) \* `HKLM\System\CurrentControlSet\Control\Lsa\MSV1\_0` \* `allownullsessionfallback` = `0` (REG\_DWORD) \* `NTLMMinClientSec` = `537395200` (REG\_DWORD) \* `NTLMMinServerSec` = `537395200` (REG\_DWORD)

---

## Rationale Securing authentication parameters and account controls reduces the risk of password attacks and session hijackings on high-value administrative endpoints:

- Stricter Lockout Threshold ( `LockoutBadCount` )**: For PAWs, the lockout threshold is reduced to 5 attempts (compared to 10 for standard endpoints). This is necessary because PAWs are used exclusively by Tier 0 administrators, who are high-value targets. A stricter threshold prevents brute-force attempts on local fallback accounts.
- Robust Local Password Settings ( `MinimumPasswordLength` , `PasswordComplexity` )**: Local accounts on PAWs (such as fallback administrators) must use passwords of at least 20 characters with complexity enabled. This mitigates offline password cracking if database hashes or local SAM registries are dumped.
- No Password Expiration ( `MaxPasswordAge` = 0 )**: Periodic password expiration is disabled. Setting the password expiration interval to 0 prevents users from cycling to weaker password variants or writing credentials down, as a 20-character complex password is mathematically robust against current brute-forcing capabilities.
- Smart Card Removal Behavior ( `ScRemoveOption` )**: In secure environments using Smart Card or token-based authentication, removing the card must automatically lock the desktop session ( 1 ). If disabled, a user leaving their workstation with the card removed leaves the session exposed.
- Blank Passwords Limit ( `LimitBlankPasswordUse` )**: Restricting the use of blank passwords to physical console logons prevents attackers from using empty-password accounts to authenticate remotely over network shares or RDP.
- Logon Caching Restriction ( `CachedLogonsCount` = 0 ) and Hashing Complexity ( `NL$IterationCount` = 1954 )**: By default, Windows caches previous logons locally as MSCacheV2 hashes, derived using PBKDF2-SHA1. Setting `CachedLogonsCount` to 0 prevents the local storage of credentials for offline validation on standard workstations, forcing authentication against a DC. For systems where caching must be enabled (such as isolated member servers or laptops), the iteration count of the hashing algorithm should be increased using `NL$IterationCount` . Setting it to 1954 results in 2,000,896 rounds of PBKDF2-SHA1, increasing resistance to offline brute-force and GPU-accelerated cracking attacks.
- LSASS WDigest protection ( `UseLogonCredential` = 0 )**: Disabling WDigest credential caching prevents the LSASS process from storing cleartext passwords in memory.
- Microsoft Account and PIN bans**: Restricting Microsoft consumer account authentication and domain PIN logons ensures that standard enterprise credentials and secure Hello for Business PINs are the only mechanisms used.
- Secure Channel and NTLM session security**: Forcing secure channel signing, disabling plain text passwords, preventing null session fallbacks, requiring NTLMv2 and 128-bit encryption, and enforcing client-side NTLMv2-only authentication via `LmCompatibilityLevel` = 5 block legacy protocol exploitation and relay vectors.

**10. Fine-Grained Password Policies (FGPP):** While local accounts are secured on the machine, the Active Directory user accounts of the Tier 0 Administrators who logon to these PAWs must also be protected by a domain-level Fine-Grained Password Policy (FGPP / PSO) of at least 20 characters, as configured in [REQ-ID-001 - Enforce Fine-Grained Password Policies](#).

---

**## Legacy Impact & Compatibility** \* **Account Lockouts**: Legitimate administrators who forget their passwords may lock themselves out. Standard procedures must exist for administrative reset of locked accounts by another Tier 0 administrator. \* **Smart Card Removal**: Administrators must be trained to carry their smart cards with them, which automatically locks the session. Re-authenticating requires inserting the card and entering the PIN. \* **Reversible Encryption**: Disabling reversible encryption may break legacy applications that rely on reading cleartext password equivalents. These applications should not exist in the Tier 0 environment. \* **Ligon Caching (CachedLogonsCount = 0)**: PAWs must have active, real-time connectivity to a Domain Controller to allow users to log on. Off-domain logons (e.g., users traveling or working offline without a pre-boot VPN connection) will fail. Remote users must use pre-boot VPN tunnels or alternate remote access architectures. \* **No Password Expiration**: Removing periodic password changes minimizes helpdesk tickets and stops administrators from choosing predictable increments. However, the organization must still enforce credential revocation and manual rotation protocols in the event of a suspected credential leak.

---

## ## Implementation Steps

### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

**#### Step 1: Configure Lockout and Password Policies (Domain-wide or PAW GPO)** These settings must be configured in a dedicated GPO linked to the PAW Organizational Unit (OU) (e.g., 'GPO\_Hardening\_PAWs'): 1. Open the **Group Policy Management Console** ('gpmc.msc'). 2. Edit the PAW GPO. 3. Navigate to: 'Computer Configuration\Policies\Windows Settings\Security Settings\Account Policies' 4. Configure the settings: \* **Account Lockout Policy**: \* **Account lockout threshold**: '5' invalid logon attempts \* **Reset account lockout counter after**: '30' minutes \* **Account lockout duration**: '30' minutes \* **Password Policy**: \* **Enforce password history**: '24' passwords remembered \* **Maximum password age**: '0' days (never expire) \* **Minimum password age**: '1' day \* **Minimum password length**: '20' characters \* **Password must meet complexity requirements**: 'Enabled' \* **Store passwords using reversible encryption**: 'Disabled'

**#### Step 2: Configure Local Security Options** In the PAW GPO, navigate to: 'Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options' \* **Policy**: 'Interactive logon: Smart card removal behavior' → Set to **Lock Workstation** (value 1) \* **Policy**: 'Accounts: Limit local account use of blank passwords to console logon only' → Set to **Enabled** (value 1) \* **Policy**: 'Network security: Do not store LAN Manager hash value on next password change' → Set to **Enabled** (value 1) \* **Policy**: 'Interactive logon: Number of previous logons to cache (in case domain controller is not available)' → Set to **0** \* **Policy**: 'Network security: Minimum session security for NTLM SSP based (including secure RPC) clients' → Set to **Require NTLMv2 session security, Require 128-bit encryption** (value 537395200) \* **Policy**: 'Network security: Minimum session security for NTLM SSP based (including secure RPC) servers' → Set to **Require NTLMv2 session security, Require 128-bit encryption** (value 537395200) \* **Policy**: 'Network access: Allow anonymous SID/Name translation' → Set to **Disabled** (value 0) \* **Policy**: 'Network security: Allow LocalSystem NULL session fallback' → Set to **Disabled** (value 0)

**#### Step 3: Configure Hello for Business, PIN and Microsoft Account Policies** Navigate to: 'Computer Configuration\Administrative Templates\System\PIN Complexity' \* **Policy**: 'Minimum PIN length' → Set to **Enabled** with value **6**

Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Microsoft Account`

- **Policy:** `Block all consumer Microsoft account user authentication` → Set to **Enabled**

Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Windows Hello for Business`

- **Policy:** `Use a hardware security device` → Set to **Enabled**
- **Policy:** `Use convenience PIN sign-in` → Set to **Disabled** (value 0)
- **Policy:** `Allow Microsoft accounts to be optional` → Set to **Enabled** (value 1)

**#### Step 4: Configure PBKDF2 Iteration Count via GPO Preferences** Since the PBKDF2 iteration count setting is not exposed in standard ADMX templates, deploy it via Registry GPO Preferences: 1. Within the PAW GPO, navigate to: 'Computer Configuration\Preferences\Windows Settings\Registry' 2. Right-click **Registry**, select **New** → **Registry Item**. 3. Configure: \* **Action**: 'Update' \* **Hive**: 'HKEY\_LOCAL\_MACHINE' \* **Key Path**: 'SECURITY\Cache' \* **Value name**: 'NL\$IterationCount' \* **Value type**: 'REG\_DWORD' \* **Value data**: '1954' (Decimal)

---

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the settings locally (for testing or standalone PAW systems) or if the control is not manageable via standard GPO GUI interfaces.

[Download Script: Set-PAWAccountPolicies.ps1](#)



```
Set-PAWAccountPolicies.ps1
Configures local account lockout, password parameters, smart card removal behavior, blank password blocks, Hello for Business, Microsoft accounts, secure channel options, and NTLM session security options on PAWs.

Write-Host "Applying PAW account and password policies..." -ForegroundColor Cyan

1. Enforce local security options via Registry
$WinLogonPath = "HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Winlogon"
if (-not (Test-Path $WinLogonPath)) {
 New-Item -Path $WinLogonPath -Force | Out-Null
}
Set-ItemProperty -Path $WinLogonPath -Name "ScRemoveOption" -Value "1" -Type String -Force
Set-ItemProperty -Path $WinLogonPath -Name "CachedLogonsCount" -Value 0 -Type DWord -Force
Write-Host "[+] Smart card removal behavior and logon caching configured." -ForegroundColor Green

$LsaPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
if (-not (Test-Path $LsaPath)) {
 New-Item -Path $LsaPath -Force | Out-Null
}
Set-ItemProperty -Path $LsaPath -Name "LimitBlankPasswordUse" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $LsaPath -Name "NoLMHash" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $LsaPath -Name "LmCompatibilityLevel" -Value 5 -Type DWord -Force
Write-Host "[+] Blank password, NoLMHash, and client NTLMv2-only options enforced." -ForegroundColor Green

LSASS WDigest caching block
$WDigestPath = "HKLM:\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest"
if (-not (Test-Path $WDigestPath)) {
 New-Item -Path $WDigestPath -Force | Out-Null
}
Set-ItemProperty -Path $WDigestPath -Name "UseLogonCredential" -Value 0 -Type DWord -Force
Write-Host "[+] LSASS WDigest credential caching disabled." -ForegroundColor Green

PBKDF2 Iterations for Cached Logons
$CachePath = "HKLM:\SECURITY\Cache"
if (-not (Test-Path $CachePath)) {
 New-Item -Path $CachePath -Force | Out-Null
}
Set-ItemProperty -Path $CachePath -Name "NL`$IterationCount" -Value 1954 -Type DWord -Force
Write-Host "[+] PBKDF2 cached credentials iteration count configured." -ForegroundColor Green

Hello for Business, PIN and Microsoft Account policies
$SystemPolicyPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System"
if (-not (Test-Path $SystemPolicyPath)) {
 New-Item -Path $SystemPolicyPath -Force | Out-Null
}
Set-ItemProperty -Path $SystemPolicyPath -Name "AllowDomainPINLogon" -Value 0 -Type DWord -Force

$PinComplexityPath = "HKLM:\SOFTWARE\Policies\Microsoft\PassportForWork\PINComplexity"
if (-not (Test-Path $PinComplexityPath)) {
 New-Item -Path $PinComplexityPath -Force | Out-Null
}
Set-ItemProperty -Path $PinComplexityPath -Name "MinimumPINLength" -Value 6 -Type DWord -Force

$SystemPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
if (-not (Test-Path $SystemPath)) {
 New-Item -Path $SystemPath -Force | Out-Null
}
Set-ItemProperty -Path $SystemPath -Name "MSAOptional" -Value 1 -Type DWord -Force

$MsaPath = "HKLM:\SOFTWARE\Policies\Microsoft\MicrosoftAccount"
if (-not (Test-Path $MsaPath)) {
 New-Item -Path $MsaPath -Force | Out-Null
}
Set-ItemProperty -Path $MsaPath -Name "DisableUserAuth" -Value 1 -Type DWord -Force

$PassportPath = "HKLM:\SOFTWARE\Policies\Microsoft\PassportForWork"
if (-not (Test-Path $PassportPath)) {
 New-Item -Path $PassportPath -Force | Out-Null
}
Set-ItemProperty -Path $PassportPath -Name "RequireSecurityDevice" -Value 1 -Type DWord -Force

$ExcludeDevicesPath = "HKLM:\SOFTWARE\Policies\Microsoft\PassportForWork\ExcludeSecurityDevices"
if (-not (Test-Path $ExcludeDevicesPath)) {
 New-Item -Path $ExcludeDevicesPath -Force | Out-Null
}
```

```

}
Set-ItemProperty -Path $ExcludeDevicesPath -Name "TPM12" -Value 0 -Type DWord -Force
Write-Host "[+] Hello for Business, PIN and Microsoft Account options configured." -ForegroundColor Green

Domain Member Secure Channel netlogon settings
$NetLogonPath = "HKLM:\System\CurrentControlSet\Services\Netlogon\Parameters"
if (-not (Test-Path $NetLogonPath)) {
 New-Item -Path $NetLogonPath -Force | Out-Null
}
Set-ItemProperty -Path $NetLogonPath -Name "RequireSignOrSeal" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $NetLogonPath -Name "SealSecureChannel" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $NetLogonPath -Name "SignSecureChannel" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $NetLogonPath -Name "DisablePasswordChange" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $NetLogonPath -Name "MaximumPasswordAge" -Value 30 -Type DWord -Force
Set-ItemProperty -Path $NetLogonPath -Name "RequireStrongKey" -Value 1 -Type DWord -Force
Write-Host "[+] Domain Member secure channel configurations applied." -ForegroundColor Green

LanmanWorkstation plain text passwords block
$LanmanWorkPath = "HKLM:\System\CurrentControlSet\Services\LanmanWorkstation\Parameters"
if (-not (Test-Path $LanmanWorkPath)) {
 New-Item -Path $LanmanWorkPath -Force | Out-Null
}
Set-ItemProperty -Path $LanmanWorkPath -Name "EnablePlainTextPassword" -Value 0 -Type DWord -Force

NTLM SSP Client & Server security and Null Session Fallback
$MsvPath = "HKLM:\System\CurrentControlSet\Control\Lsa\MSV1_0"
if (-not (Test-Path $MsvPath)) {
 New-Item -Path $MsvPath -Force | Out-Null
}
Set-ItemProperty -Path $MsvPath -Name "allownullsessionfallback" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $MsvPath -Name "NTLMMinClientSec" -Value 537395200 -Type DWord -Force
Set-ItemProperty -Path $MsvPath -Name "NTLMMinServerSec" -Value 537395200 -Type DWord -Force
Write-Host "[+] Network authentication security and NTLM session settings applied." -ForegroundColor Green

2. Enforce Account Lockout and Password Policy via secdit
$SecTempDir = Join-Path $env:TEMP "PAWAccountSecurityTemplates"
if (-not (Test-Path $SecTempDir)) {
 New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null
}

$CfgFile = Join-Path $SecTempDir "paw_account_sec.cfg"
$LogFile = Join-Path $SecTempDir "secdit.log"
$DbFile = Join-Path $SecTempDir "secdit.sdb"

Export current db
$Process = Start-Process secdit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Error "Failed to export current configuration database."
 return
}

$ConfigText = Get-Content -Path $CfgFile -Raw
$HasSystemAccess = $ConfigText -match "[System Access\]"
if (-not $HasSystemAccess) {
 $ConfigText += "`r`n[System Access]`r`n"
}

Re-build [System Access] section line-by-line
$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InSystemAccess = $false

$AccountSettings = @{
 "LockoutBadCount" = 5
 "ResetLockoutCount" = 30
 "LockoutDuration" = 30
 "ClearTextPassword" = 0
 "MinimumPasswordLength" = 20
 "PasswordComplexity" = 1
 "PasswordHistorySize" = 24
 "MaxPasswordAge" = 0
 "MinPasswordAge" = 1
}

foreach ($Line in $Lines) {
 if ($Line -match "^\[(.*)\]$") {
 $SectionName = $Matches[1]
 }
}

```



```

 if ($SectionName -eq "System Access") {
 $InSystemAccess = $true
 $NewLines += $Line
 continue
 } else {
 $InSystemAccess = $false
 }
 }

 if ($InSystemAccess) {
 $IsManaged = $false
 foreach ($Key in $AccountSettings.Keys) {
 if ($Line -match "^s*$(($Key)\s*=)") {
 $IsManaged = $true
 break
 }
 }
 if (-not $IsManaged) {
 $NewLines += $Line
 }
 } else {
 $NewLines += $Line
 }
}

Append our settings
$FinalLines = @()
foreach ($Line in $NewLines) {
 $FinalLines += $Line
 if ($Line -eq "[System Access]") {
 foreach ($Key in $AccountSettings.Keys) {
 $Val = $AccountSettings[$Key]
 $FinalLines += "$($Key) = $($Val)"
 }
 }
}

$FinalLines -join "`r`n" | Out-File -FilePath $CfgFile -Encoding ascii -Force

Import
$Process = Start-Process secdit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas SECURITYPOLICY /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host "[+] Lockout and password policies applied locally." -ForegroundColor Green
} else {
 Write-Error "Failed to apply local account policies. Exit Code: $($Process.ExitCode)"
}

Remove-Item -Path $SecTempDir -Recurse -Force -ErrorAction SilentlyContinue

```

*To verify the settings have been applied:*

[Download Script: Test-PAWAccountPolicies.ps1](#)

```
Test-PAWAccountPolicies.ps1
Checks local registry and SecEdit settings for account lockout, password options, smart card removal behavior, PIN parameters, Hel
lo for Business, Microsoft account settings, secure channel properties, and NTLM session configuration on PAWs.

Write-Host "--- Auditing PAW Account and Password Policies ---" -ForegroundColor Cyan

$script:Vulnerable = $false

Helper function to audit registry properties
function Test-RegistryValue ($path, $name, $expectedValue) {
 $val = Get-ItemProperty -Path $path -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 $color = "Red"
 if ($actual -eq $expectedValue) {
 $color = "Green"
 } else {
 $script:Vulnerable = $true
 }
 Write-Host " - Registry Setting: $name | Actual: '$actual' (Expected: '$expectedValue')" -ForegroundColor $color
}

1. Audit Registry Settings
$WinlogonPath = "HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Winlogon"
Test-RegistryValue $WinlogonPath "ScRemoveOption" "1"
Test-RegistryValue $WinlogonPath "CachedLogonsCount" 0

$LsaPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
Test-RegistryValue $LsaPath "LimitBlankPasswordUse" 1
Test-RegistryValue $LsaPath "NoLMHash" 1
Test-RegistryValue $LsaPath "LmCompatibilityLevel" 5

$WDigestPath = "HKLM:\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest"
Test-RegistryValue $WDigestPath "UseLogonCredential" 0

$CachePath = "HKLM:\SECURITY\Cache"
Test-RegistryValue $CachePath "NL`$IterationCount" 1954

$SystemPolicyPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System"
Test-RegistryValue $SystemPolicyPath "AllowDomainPINLogon" 0

$PinComplexityPath = "HKLM:\SOFTWARE\Policies\Microsoft\PassportForWork\PINComplexity"
Test-RegistryValue $PinComplexityPath "MinimumPINLength" 6

$SystemPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
Test-RegistryValue $SystemPath "MSAOptional" 1

$MsaPath = "HKLM:\SOFTWARE\Policies\Microsoft\MicrosoftAccount"
Test-RegistryValue $MsaPath "DisableUserAuth" 1

$PassportPath = "HKLM:\SOFTWARE\Policies\Microsoft\PassportForWork"
Test-RegistryValue $PassportPath "RequireSecurityDevice" 1

$ExcludeDevicesPath = "HKLM:\SOFTWARE\Policies\Microsoft\PassportForWork\ExcludeSecurityDevices"
Test-RegistryValue $ExcludeDevicesPath "TPM12" 0

$NetlogonPath = "HKLM:\System\CurrentControlSet\Services\Netlogon\Parameters"
Test-RegistryValue $NetlogonPath "RequireSignOrSeal" 1
Test-RegistryValue $NetlogonPath "SealSecureChannel" 1
Test-RegistryValue $NetlogonPath "SignSecureChannel" 1
Test-RegistryValue $NetlogonPath "DisablePasswordChange" 0
Test-RegistryValue $NetlogonPath "MaximumPasswordAge" 30
Test-RegistryValue $NetlogonPath "RequireStrongKey" 1

$LanmanWorkPath = "HKLM:\System\CurrentControlSet\Services\LanmanWorkstation\Parameters"
Test-RegistryValue $LanmanWorkPath "EnablePlainTextPassword" 0

$MsvPath = "HKLM:\System\CurrentControlSet\Control\Lsa\MSV1_0"
Test-RegistryValue $MsvPath "allownullsessionfallback" 0
Test-RegistryValue $MsvPath "NTLMMinClientSec" 537395200
Test-RegistryValue $MsvPath "NTLMMinServerSec" 537395200

2. Audit SecEdit Settings
$SecTempDir = Join-Path $env:TEMP "PAWAccountAuditSecurityTemplates"
if (-not (Test-Path $SecTempDir)) {
```

```

 New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null
}

$CfgFile = Join-Path $SecTempDir "paw_account_audit.cfg"
$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Error "Failed to export current database."
 return
}

$ConfigContent = Get-Content -Path $CfgFile -Raw
$AccountSettings = @{
 "LockoutBadCount" = 5
 "ResetLockoutCount" = 30
 "LockoutDuration" = 30
 "ClearTextPassword" = 0
 "MinimumPasswordLength" = 20
 "PasswordComplexity" = 1
 "PasswordHistorySize" = 24
 "MaxPasswordAge" = 0
 "MinPasswordAge" = 1
}

foreach ($Key in $AccountSettings.Keys) {
 $Expected = $AccountSettings[$Key]
 if ($ConfigContent -match "(?m)^\s*${$Key}\s*=\s*(.*)\s*$") {
 $Actual = $Matches[1].Trim()
 } else {
 $Actual = ""
 }

 $Color = "Red"
 if ($Actual -eq [string]$Expected) {
 $Color = "Green"
 } else {
 $script:Vulnerable = $true
 }

 Write-Host " - System Access Setting: ${$Key} | Actual: '$($Actual)' (Expected: '$($Expected)')" -ForegroundColor $Color
}

Remove-Item -Path $SecTempDir -Recurse -Force -ErrorAction SilentlyContinue

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
}

```

## Sources & Compliance References \* \*\*ANSI AD Hardening Guide\*\*: Recommendations on password complexity, reversible encryption blocks, lockout management, and domain member secure channels. \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*: Section 1.1 (Password Policy), Section 1.2 (Account Lockout Policy), Section 2.3.7.3 (Accounts: Limit local account use of blank passwords...), Section 2.3.9.5 (Interactive logon: Smart card removal behavior), Section 2.3.10.2 (Microsoft network client: Send unencrypted password), Section 2.3.11.8 (Network access: Allow anonymous SID/Name translation), Section 2.3.11.10 (Network security: Allow LocalSystem NULL session fallback). \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Various account policy, PIN complexity, Windows Hello for Business, Microsoft account restrictions, WDigest disabled, and Netlogon secure channel parameters.

# [REQ-PAW-014] Configure Early Launch Antimalware (ELAM) Policy for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations. \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*ELAM GPO\*\*: `Computer Configuration\Policies\Administrative Templates\System\Early Launch Antimalware\Boot-Start Driver Initialization Policy` → Enabled (Select \*\*Good, unknown and bad but critical\*\* in options) \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Policies\EarlyLaunch` → `DriverLoadPolicy` = `3` (REG\_DWORD)

## Rationale Privileged Access Workstations (PAWs) require the highest level of boot integrity to prevent compromise of Tier 0 administrative tasks:

1. **Early Boot Rootkits**: Malicious boot-start drivers can execute before third-party endpoint protection or security agents initialize. Enforcing the Early Launch Antimalware (ELAM) driver initialization policy ensures that unknown, unsigned, or bad drivers are prevented from loading at boot time.
2. **Platform Integrity**: Securing the boot sequence with ELAM is a critical requirement of Virtualization-Based Security (VBS) and overall hardware-rooted protection.

## Legacy Impact & Compatibility \* \*\*Boot-Start Drivers\*\*: If third-party, custom, or legacy hardware monitoring drivers are unsigned, the ELAM policy will block them from loading, potentially resulting in blue screen (BSOD) or hardware failures. Verify all active drivers possess valid digital signatures prior to enforcement.

#### ## Implementation Steps

##### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the GPO applied to your PAWs (e.g., `GPO_Hardening_PAWs` ).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Early Launch Antimalware`
4. Double-click **Boot-Start Driver Initialization Policy**.
5. Set it to **Enabled**.
6. In the options dropdown, select **Good, unknown and bad but critical** (registry value `3` ).
7. Click **OK**.

##### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to apply the ELAM driver load policy.

[Download Script: Configure-ElamPolicy.ps1](#)

```
Configure-ElamPolicy.ps1
Description: Configures the Early Launch Antimalware (ELAM) boot-start driver load policy on the local system.

Write-Host "Applying ELAM Boot-Start driver initialization policy..." -ForegroundColor Cyan

$ElamPath = "HKLM:\SYSTEM\CurrentControlSet\Policies\EarlyLaunch"
if (-not (Test-Path $ElamPath)) {
 New-Item -Path $ElamPath -Force | Out-Null
}
Set-ItemProperty -Path $ElamPath -Name "DriverLoadPolicy" -Value 3 -Type DWord -ErrorAction Stop
Write-Host "[+] ELAM Boot-Start driver initialization policy set to Good, unknown and bad but critical." -ForegroundColor Green
```

*To audit the ELAM driver load policy status:*

[Download Script: Get-ElamPolicyStatus.ps1](#)

```
Get-ElamPolicyStatus.ps1
Description: Audits registry configuration of the Early Launch Antimalware (ELAM) policy.

Write-Host "--- Auditing ELAM Boot-Start Policy ---" -ForegroundColor Cyan

$script:Vulnerable = $false

$Path = "HKLM:\SYSTEM\CurrentControlSet\Policies\EarlyLaunch"
$Name = "DriverLoadPolicy"
$Expected = 3

if (Test-Path $Path) {
 $Reg = Get-ItemProperty -Path $Path -ErrorAction SilentlyContinue
 $Val = $Reg.$Name
 if ($Val -eq $Expected) {
 Write-Host " [+] Path $($Path) | $($Name): $Val (Expected: $Expected)" -ForegroundColor Green
 } else {
 Write-Host " [!] MISMATCH: Path $($Path) | $($Name): $Val (Expected: $Expected)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
} else {
 Write-Host " [!] NOT FOUND: Path $($Path) (Expected: $Name = $Expected)" -ForegroundColor Red
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Benchmark\*\*: CIS Microsoft Windows Client Benchmark - Section 18.2.2 (System Options / ELAM) \* \*\*ANSI AD Hardening Guide\*\*: Section addressing system and boot configuration integrity.

# [REQ-PAW-015] Configure Secure Printing and Print Spooler Policies for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations. \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*System Service GPO\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services\Print Spooler` → Service Startup Mode: **Disabled** \* \*\*Point and Print GPO\*\*: `Computer Configuration\Policies\Administrative Templates\Printers\Limits print driver installation to Administrators` → Enabled \* \*\*Registry Location (Service)\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\Spooler` → `Start` = `4` (REG\_DWORD) \* \*\*Registry Location (Printers)\*\*: `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Printers\PointAndPrint` → `RestrictDriverInstallationToAdministrators` = `1` (REG\_DWORD)

## Rationale The Windows Print Spooler service (`Spooler`) has been a recurring source of critical privilege escalation and coercion exploits (e.g., the PrintNightmare family).

To secure Privileged Access Workstations (PAWs), which host highly privileged Tier 0 credentials:

1. **Disable the Print Spooler:** PAWs should never act as print servers or need print capabilities. Disabling the `Spooler` service completely eliminates this massive attack surface.
2. **Point and Print Restrictions:** As a defense-in-depth fallback, restricting print driver installations and updates to Administrators ensures that even if the spooler service is temporarily enabled for maintenance, standard users cannot load arbitrary, untrusted drivers.

## Legacy Impact & Compatibility \* \*\*No Printing Support\*\*: Printing from PAWs is completely disabled. This aligns with Tier 0 isolation principles where PAWs are restricted to administration and are not used for general office productivity.

#### ## Implementation Steps

##### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### 1. Disable the Print Spooler Service 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Edit the GPO applied to your PAWs (e.g., `GPO\_Hardening\_PAWs`). 3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` 4. Double-click **Print Spooler**. 5. Check **Define this policy setting** and select **Disabled**. Click **OK**.

##### 2. Restrict Point and Print Driver Installations 1. Navigate to: `Computer Configuration\Policies\Administrative Templates\Printers` 2. Double-click **Limits print driver installation to Administrators**. 3. Set it to **Enabled** and click **OK**.

##### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to disable the Print Spooler service and enforce driver restrictions.

[Download Script: Configure-PrintingAndSpooler.ps1](#)

```
Configure-PrintingAndSpooler.ps1
Description: Disables the Print Spooler service and configures Point and Print driver installation restrictions on the local PAW.

Write-Host "Hardening Print Spooler and Printer configurations for PAWs..." -ForegroundColor Cyan

1. Disable the Print Spooler Service
if (Get-Service -Name "Spooler" -ErrorAction SilentlyContinue) {
 Set-Service -Name "Spooler" -StartupType Disabled -Confirm:$false
 Stop-Service -Name "Spooler" -Force -Confirm:$false
 Write-Host "[+] Print Spooler service has been stopped and disabled." -ForegroundColor Green
} else {
 Write-Host "[+] Print Spooler service not found on local machine." -ForegroundColor Gray
}

2. Limit Print Driver Installation to Administrators (Defense-in-Depth)
$PrinterPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers\PointAndPrint"
if (-not (Test-Path $PrinterPath)) {
 New-Item -Path $PrinterPath -Force | Out-Null
}
Set-ItemProperty -Path $PrinterPath -Name "RestrictDriverInstallationToAdministrators" -Value 1 -Type DWord -ErrorAction Stop
Write-Host "[+] Print driver installation restricted to Administrators." -ForegroundColor Green
```

To audit these printer security configurations on the PAW:

[Download Script: Get-PrintingAndSpoolerStatus.ps1](#)

```
Get-PrintingAndSpoolerStatus.ps1
Description: Audits print spooler status and Point and Print configurations on the local PAW.

Write-Host "--- Auditing PAW Secure Printing and Spooler Hardening ---" -ForegroundColor Cyan

$script:Vulnerable = $false

1. Audit Spooler Service Startup Type
$Service = Get-Service -Name "Spooler" -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 # Check StartupType
 $StartupType = (Get-CimInstance -ClassName Win32_Service -Filter "Name='Spooler'").StartMode
 # StartMode can be "Disabled", "Manual", "Auto"
 $Color = if ($StartupType -eq "Disabled") { "Green" } else { "Red" }
 Write-Host " [-] Print Spooler Service Startup: $StartupType (Expected: Disabled)" -ForegroundColor $Color
 if ($StartupType -ne "Disabled") {
 $script:Vulnerable = $true
 }
} else {
 Write-Host " [+] Print Spooler Service is not present on this machine." -ForegroundColor Green
}

2. Audit Point and Print Registry Restriction
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers\PointAndPrint"
$Name = "RestrictDriverInstallationToAdministrators"
$Expected = 1

if (Test-Path $Path) {
 $Reg = Get-ItemProperty -Path $Path -ErrorAction SilentlyContinue
 $Val = $Reg.$Name
 if ($Val -eq $Expected) {
 Write-Host " [+] Path $($Path) | $($Name): $Val (Expected: $Expected)" -ForegroundColor Green
 } else {
 Write-Host " [!] MISMATCH: Path $($Path) | $($Name): $Val (Expected: $Expected)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
} else {
 Write-Host " [!] NOT FOUND: Path $($Path) (Expected: $Name = $Expected)" -ForegroundColor Red
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Benchmark\*\*: CIS Microsoft Windows Client Benchmark - Section 18.7 (Printing settings) \*  
 \*\*ANSSI AD Hardening Guide\*\*: Section addressing system services minimization (disabling the Spooler service on sensitive systems).

# [REQ-PAW-016] Configure Untrusted Font Blocking for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs). \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Administrative Templates\System\Mitigation Options\Untrusted Font Blocking` \* \*\*Registry Location\*\*: `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\MitigationOptions` \* \*\*Value Name\*\*: `MitigationOptions\_FontBlocking` \* \*\*Value Type\*\*: `REG\_SZ` \* \*\*Value Data\*\*: `1000000000000`

## Rationale Font files (TrueType, OpenType, and others) are highly complex formats that require advanced parsing logic. Historically, font parsing in Windows was performed by the Graphics Device Interface (GDI) within the operating system kernel. Vulnerabilities in the kernel-mode font parser (such as buffer overflows or remote code execution) have been frequently exploited by threat actors to execute arbitrary code with kernel-level privileges.

Enabling Untrusted Font Blocking limits the attack surface of the graphics subsystem on Privileged Access Workstations (PAWs):

1. **Kernel Attack Surface Reduction:** Restricting the system to only load trusted fonts installed in the `%windir%\Fonts` system directory prevents the processing of malicious, web-delivered, or embedded font files.
2. **Mitigation of Document-Based Exploits:** Prevents malicious font files embedded in administrative documents, scripts, or web tools from triggering parsing vulnerabilities in the context of high-privileged administrative accounts.

## Legacy Impact & Compatibility \* \*\*Web Browsers & Web Fonts\*\*: Web browsers used for administrative tasks or SaaS consoles on PAWs that dynamically download custom fonts may fail to render those fonts, reverting to system default fallback fonts. This may result in styling anomalies or missing icons. \* \*\*Embedded Fonts in Documents\*\*: Administrative guides or documents utilizing custom embedded fonts that are not locally installed in the system directory will render using default system fonts, potentially affecting layout alignment. \* \*\*Deployment Validation\*\*: It is recommended to deploy this setting in Audit Mode (`3000000000000`) initially to monitor event log entries (Event ID 305 inside the `Microsoft-Windows-Security-Mitigations/KernelMode` log) before fully enforcing the block policy.

#### ## Implementation Steps

##### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a domain controller or management host.
2. Create a new GPO or edit an existing one (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Mitigation Options`
4. Configure the following setting:
  - **Policy:** `Untrusted Font Blocking`
  - **Setting:** `Enabled`
  - **Mitigation Options:** `Block untrusted fonts and log events`
5. Link the GPO to the PAW Organizational Unit (OU) containing the target workstations.

##### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally on standalone systems or during reference image build phases.

[Download Script: Configure-UntrustedFontBlocking.ps1](#)

```
Configure-UntrustedFontBlocking.ps1
Description: Configures Untrusted Font Blocking mitigation to block untrusted fonts and log events on PAWs.

$RegPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\MitigationOptions"
$ValueName = "MitigationOptions_FontBlocking"
$ValueData = "1000000000000"

Write-Host "Applying hardening requirement: Configure Untrusted Font Blocking..." -ForegroundColor Cyan

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value $ValueData -Type String -Force | Out-Null
Write-Host "Hardening applied successfully." -ForegroundColor Green
```

To verify the setting has been applied:

[Download Script: Get-UntrustedFontBlockingStatus.ps1](#)



```
Get-UntrustedFontBlockingStatus.ps1
Description: Checks the current configuration state of Untrusted Font Blocking registry setting on PAWs.

$RegPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\MitigationOptions"
$ValueName = "MitigationOptions_FontBocking"
$ExpectedValue = "10000000000000"

Write-Host "Auditing hardening requirement: Configure Untrusted Font Blocking..." -ForegroundColor Cyan

if (Test-Path $RegPath) {
 $value = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
 if ($null -ne $value -and $value.$ValueName -eq $ExpectedValue) {
 Write-Host "Audit Result: Compliant. Untrusted fonts are blocked and logged (($ValueName) = $($ExpectedValue))." -Foregroun
dColor Green
 exit 0
 }
}

Write-Host "Audit Result: Non-Compliant. Untrusted fonts are not configured to block and log." -ForegroundColor Red
exit 1
```

---

## Sources & Compliance References \* **Microsoft Learn**: Block untrusted fonts in an enterprise (MitigationOptions registry configurations)  
 \* **CIS Benchmark**: CIS Microsoft Windows Client Benchmark - Section 18.9.30.1 (System Options / Mitigation Options)

# [REQ-PAW-017] Configure svchost.exe Mitigation Options for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs). \* \*\*Operating Systems\*\*: Windows 10 (1903 and above), Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Administrative Templates\System\Service Control Manager Settings\Security Settings\Enable svchost.exe mitigation options` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Control\SCMConfig` \* \*\*Value Name\*\*: `EnableSvchostMitigationPolicy` \* \*\*Value Type\*\*: `REG\_DWORD` \* \*\*Value Data\*\*: `1`

## Rationale Privileged Access Workstations (PAWs) host highly sensitive administrative sessions and credentials. Securing the Service Host (`svchost.exe`) process is critical to preventing kernel-level security evasion and credential harvesting techniques.

Enabling `svchost.exe` mitigation options on PAWs restricts the behavior of the `svchost.exe` process to enhance security:

1. **Microsoft-Only Binary Enforcement:** Requires all binaries and dynamic-link libraries (DLLs) loaded into `svchost.exe` to be digitally signed by Microsoft. This prevents attackers from injecting custom, unsigned malicious DLLs into `svchost.exe` instances to tamper with administrative service processes.
2. **Dynamic Code Blocking:** Prevents the execution of dynamically generated code (such as Just-In-Time compiled code) within `svchost.exe` processes, neutralizing typical in-memory exploitation vectors.

## Legacy Impact & Compatibility \* \*\*Third-Party Compatibility\*\*: This policy requires all binaries loaded by `svchost.exe` to be Microsoft-signed. Since PAWs are strictly controlled, single-purpose administrative machines, they should run minimal third-party software. However, any security tools, smart card readers, or system drivers that attempt to run services inside the `svchost.exe` process space using non-Microsoft DLLs will fail to load. \* \*\*Operating System Support\*\*: This policy has no effect on Windows 10 versions prior to 1903. \* \*\*Deployment Validation\*\*: Ensure that any administrative agents or hardware verification drivers are fully certified and Microsoft-signed before enforcing this control.

#### ## Implementation Steps

##### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a domain controller or management host.
2. Create a new GPO or edit an existing one (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Service Control Manager Settings\Security Settings`
4. Configure the following setting:
  - **Policy:** `Enable svchost.exe mitigation options`
  - **Setting:** `Enabled`
5. Link the GPO to the PAW Organizational Unit (OU) containing the target systems.

##### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally on standalone systems or during reference image build phases.

[Download Script: Configure-SvchostMitigation.ps1](#)

```
Configure-SvchostMitigation.ps1
Description: Configures svchost.exe mitigation options to enforce Microsoft-signed binaries and block dynamic code on PAWs.

$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\SCMConfig"
$ValueName = "EnableSvchostMitigationPolicy"
$ValueData = 1

Write-Host "Applying hardening requirement: Configure svchost.exe mitigation options..." -ForegroundColor Cyan

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value $ValueData -Type DWord -Force | Out-Null
Write-Host "Hardening applied successfully." -ForegroundColor Green
```

*To verify the setting has been applied:*

[Download Script: Get-SvchostMitigationStatus.ps1](#)

```
Get-SvchostMitigationStatus.ps1
Description: Audits the configuration state of svchost.exe mitigation options on PAWs.

$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\SCM\Config"
$ValueName = "EnableSvchostMitigationPolicy"
$ExpectedValue = 1

Write-Host "Auditing hardening requirement: Configure svchost.exe mitigation options..." -ForegroundColor Cyan

if (Test-Path $RegPath) {
 $value = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
 if ($null -ne $value -and $value.$ValueName -eq $ExpectedValue) {
 Write-Host "Audit Result: Compliant. svchost.exe mitigation options are enabled." -ForegroundColor Green
 exit 0
 }
}

Write-Host "Audit Result: Non-Compliant. svchost.exe mitigation options are disabled or not configured." -ForegroundColor Red
exit 1
```

---

## Sources & Compliance References \* \*\*Microsoft Learn\*\*: Group Policy settings reference - Service Control Manager Settings \*  
\*\*Microsoft Security Guidance\*\*: Removing "Enable svchost.exe mitigation options" from baseline recommendations (for compatibility awareness) \* \*\*CIS Benchmark\*\*: CIS Microsoft Windows Client Benchmark (Section 18.9.30.1 - Mitigation Options reference)

# [REQ-PAW-018] Enable Kernel-Mode Hardware-Enforced Stack Protection for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 11 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: Computer Configuration\Administrative Templates\System\Device Guard\Turn On Virtualization Based Security → Set \*\*Kernel-level shadow stacks\*\* to \*\*Enabled\*\* \* \*\*Registry Location\*\*: HKLM\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\KernelShadowStacks \* 'Enabled' = '1' (REG\_DWORD)

## Rationale Kernel-mode Hardware-enforced Stack Protection uses CPU hardware features to protect the operating system kernel from memory corruption exploits, specifically Return-Oriented Programming (ROP) attacks.

On highly critical endpoints such as Privileged Access Workstations (PAWs), attackers aim to achieve kernel-mode execution to subvert administrative separation controls, bypass Endpoint Detection and Response (EDR) software, and extract domain credential secrets from isolated zones.

Intel Control-flow Enforcement Technology (CET) and AMD Shadow Stack technologies create a separate, hardware-secured copy of the call stack (the "shadow stack"). Before returning from a function, the CPU compares the return address on the standard stack with the address stored on the hardware-secured shadow stack. If a mismatch is detected, the processor terminates the thread or crashes the system, neutralizing control-flow hijacking attempts.

Deploying Kernel-mode Hardware-enforced Stack Protection on PAWs guarantees that the administrative gateway machines remain resilient against advanced kernel exploits.

## Legacy Impact & Compatibility \* \*\*Hardware Requirements\*\*: Systems must support hardware shadow stacks. This requires Intel 11th Gen Core (Tiger Lake) or newer processors, or AMD Zen 3 (Ryzen 5000) or newer processors. \* \*\*Firmware & OS Requirements\*\*: The system must run Windows 11 version 22H2 or newer. Additionally, the system must use UEFI BIOS with Secure Boot enabled. \* \*\*Dependencies\*\*: Virtualization-Based Security (VBS) and Hypervisor-Enforced Code Integrity (HVCI / Memory Integrity) must be enabled and active. \* \*\*Driver Compatibility\*\*: This feature enforces strict rules on kernel-mode code. Older, legacy, or improperly signed drivers—often associated with kernel-level anti-cheat software, legacy hardware, or debugging tools—will fail to load or may trigger system crashes (BSOD). PAW hardware should be carefully standardized on modern platforms to prevent driver compatibility issues.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the appropriate PAW GPO (e.g., `GPO_Hardening_PAWs` ).
3. Navigate to: `Computer Configuration\Administrative Templates\System\Device Guard`
4. Configure the setting:
  - **Policy**: `Turn On Virtualization Based Security` → Set to **Enabled**
  - Check **Kernel-level shadow stacks** → Set to **Enabled** (or set registry `Enabled` = `1` )

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script locally to configure the registry and activate Kernel-mode Hardware-enforced Stack Protection.

[Download Script: Enable-PawKernelShadowStacks.ps1](#)

```
Enable-PawKernelShadowStacks.ps1
Description: Configures HKLM registry to enable Kernel-mode Hardware-enforced Stack Protection (Kernel Shadow Stacks) for PAWs.

Write-Host "Enabling Kernel-mode Hardware-enforced Stack Protection for PAWs..." -ForegroundColor Cyan

$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\KernelShadowStacks"

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name "Enabled" -Value 1 -Type DWord
Write-Host "[+] Registry setting for PAW Kernel Shadow Stacks enabled. (Reboot required)." -ForegroundColor Green
```

*To audit the state of Kernel-mode Hardware-enforced Stack Protection:*

[Download Script: Test-PawKernelShadowStacks.ps1](#)

```
Test-PawKernelShadowStacks.ps1
Description: Audits the registry status of Kernel-mode Hardware-enforced Stack Protection (Kernel Shadow Stacks) for PAWs.

Write-Host "--- Auditing PAW Kernel-mode Hardware-enforced Stack Protection ---" -ForegroundColor Cyan

$script:Vulnerable = $false
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\KernelShadowStacks"

Check registry value
$val = Get-ItemProperty -Path $RegPath -Name "Enabled" -ErrorAction SilentlyContinue
$actual = if ($val) { $val.Enabled } else { "" }

if ($actual -eq 1) {
 Write-Host " - Registry Setting: KernelShadowStacks Enabled | Actual: '1' (Expected: '1')" -ForegroundColor Green
} else {
 $script:Vulnerable = $true
 Write-Host " - Registry Setting: KernelShadowStacks Enabled | Actual: '$actual' (Expected: '1')" -ForegroundColor Red
}

Verify VBS dependency is met
try {
 $DG = Get-CimInstance -Namespace "Root\Microsoft\Windows\DeviceGuard" -ClassName "Win32_DeviceGuard" -ErrorAction Stop
 if ($DG.VirtualizationBasedSecurityStatus -eq 2) {
 Write-Host " - VBS Status: Running" -ForegroundColor Green
 } else {
 $script:Vulnerable = $true
 Write-Host " - VBS Status: Not Running (VBS is required for Kernel Shadow Stacks)" -ForegroundColor Red
 }
} catch {
 $script:Vulnerable = $true
 Write-Host " - DeviceGuard WMI class query failed. VBS is likely disabled." -ForegroundColor Red
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
}
}
```

---

## Sources & Compliance References \* \*\*Microsoft Windows Security Baselines\*\*\*: Core Isolation Security Baseline \* \*\*ANSI AD Hardening Guide\*\*\*: Section on hardware virtualization and kernel exploitation mitigation \* \*\*DoD Windows 11 Computer STIG\*\*\*: Device Guard / Virtualization-Based Security policies

## # [REQ-PAW-019] Harden Network Parameters and Disable Legacy Name Resolution

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: \* Computer Configuration\Administrative Templates\Network\DNS Client\Turn off Multicast Name Resolution \* Computer Configuration\Administrative Templates\Network\DNS Client\Turn off default IPv6 DNS Servers \* Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security \* Computer Configuration\Administrative Templates\Network\Network Connections \* Computer Configuration\Administrative Templates\Network\Windows Connection Manager \* Computer Configuration\Administrative Templates\Network\WLAN Service\WLAN Settings \* Computer Configuration\Administrative Templates\Printers \* Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options \* Computer Configuration\Policies\Windows Settings\Security Settings\System Services\WinHTTP Web Proxy Auto-Discovery Service \* \*\*Registry Locations\*\*: \* HKLM\Software\Policies\Microsoft\Windows NT\DNSClient \* 'EnableMulticast' = '0' (REG\_DWORD, Disables LLMNR) \* 'EnableDNS' = '0' (REG\_DWORD, Disables mDNS) \* 'DisableIPv6DefaultDnsServers' = '1' (REG\_DWORD, Turn off default IPv6 DNS Servers) \* HKLM\SYSTEM\CurrentControlSet\Services\Netbt\Parameters \* 'NoNameReleaseOnDemand' = '1' (REG\_DWORD) \* 'NodeType' = '2' (REG\_DWORD) \* HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters \* 'EnableICMPRedirect' = '0' (REG\_DWORD) \* 'DisableIPSourceRouting' = '2' (REG\_DWORD) \* HKLM\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters \* 'DisableIPSourceRouting' = '2' (REG\_DWORD) \* HKLM\SOFTWARE\Policies\Microsoft\Windows\Network Connections \* 'NC\_ShowSharedAccessUI' = '0' (REG\_DWORD) \* 'NC\_AllowNetBridge\_NLA' = '0' (REG\_DWORD, Prohibit Network Bridge) \* 'NC\_StdDomainUserSetLocation' = '1' (REG\_DWORD, Require elevation to set network location) \* HKLM\SOFTWARE\Policies\Microsoft\Windows\WcmSvc\GroupPolicy \* 'fMinimizeConnections' = '3' (REG\_DWORD) \* 'fBlockNonDomain' = '1' (REG\_DWORD) \* HKLM\SOFTWARE\Microsoft\wcmSvc\wifinetworkmanager\config \* 'AutoConnectAllowedOEM' = '0' (REG\_DWORD) \* HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Printers \* 'DisableWebPnPDownload' = '1' (REG\_DWORD) \* 'DisableHTTPPrinting' = '1' (REG\_DWORD) \* HKLM\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters \* 'RestrictNullSessAccess' = '1' (REG\_DWORD) \* HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\Wpad \* 'WpadOverride' = '1' (REG\_DWORD, Disables WPAD) \* HKLM\SYSTEM\CurrentControlSet\Services\LanmanServer\DefaultSecurity \* 'SrvsvcSessionInfo' = (REG\_BINARY, Restricts Net Session Enumeration)

---

## Rationale Legacy name resolution protocols and insecure default network configurations are heavily targeted by attackers for credential harvesting and man-in-the-middle (MitM) positioning:

1. **Legacy Name Resolution (LLMNR / NetBIOS)**: LLMNR and NBT-NS serve as fallback protocols when DNS resolution fails. When a host queries an unresolvable name, it broadcasts requests over the local subnet. An attacker can spoof responses (e.g., using Responder) to capture NTLMv2 hashes or perform authentication relay attacks. NetBIOS name release requests can be forged to disrupt local names unless protected.
  2. **NetBIOS Node Type and Name Release**: Setting the Node Type to P-node (point-to-point, value 2) disables broadcast resolution fallbacks. Enabling name release protection ( `NoNameReleaseOnDemand` ) prevents attackers from spoofing name release requests to deregister local names.
  3. **ICMP Redirects**: ICMP redirect packets can be used by an attacker on the same subnet to dynamically redirect routing for specific hosts through the attacker's machine, enabling full MitM packet sniffing and modification. Disabling ICMP redirects prevents this vector.
  4. **IP Source Routing**: Source routing allows a sender to specify the exact network path a packet should follow. This is commonly abused to bypass firewall routing rules or establish communication paths that violate network segment isolation.
  5. **Disable Default IPv6 DNS Servers**: Disabling default IPv6 DNS servers prevents automated fallback to unauthenticated, dynamic local IPv6 DNS servers advertised by rogue routers or malicious tools (like mitm6), which would otherwise redirect query traffic and coerce NTLM or Kerberos authentication.
  6. **Disable Web Proxy Auto-Discovery (WPAD)**: Disabling WPAD removes another name resolution mechanism that Responder exploits to harvest credentials. By disabling the `WinHttpAutoProxySvc` service and configuring `WpadOverride = 1`, the workstation is protected from rogue web proxy configurations.
  7. **Restrict Net Session Enumeration (NetCease)**: By default, any authenticated domain user can query session information from remote hosts. Attackers utilize session enumeration to locate high-privileged user sessions (e.g., Domain Admins) across the network. Hardening the `SrvsvcSessionInfo` default security descriptor blocks this remote reconnaissance.
- 

## Legacy Impact & Compatibility \* \*\*DNS Dependency\*\*: Disabling LLMNR and NetBIOS requires a fully operational DNS infrastructure. Any internal local network resource names must be registered in the AD DNS zones. Ad-hoc name resolution (such as workgroup-based peer file sharing) will no longer function. \* \*\*Standby Subnets\*\*: Disabling ICMP redirects means systems will rely on static routing tables and default gateway definitions. This is the standard operational stance for secure enterprise subnets.

## ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

#### Step 1: Configure DNS Client Settings 1. Open the \*\*Group Policy Management Console\*\* ('gpmc.msc'). 2. Edit the PAW GPO (e.g., 'GPO\_Hardening\_PAW'). 3. Navigate to: 'Computer Configuration\Administrative Templates\Network\DNS Client' 4. Configure the settings: \* \*\*Policy\*\*: 'Turn off Multicast Name Resolution' → \*\*Enabled\*\* \* \*\*Policy\*\*: 'Configure multicast DNS (mDNS) protocol' → \*\*Enabled\*\* with option set to \*\*Disabled\*\* \* \*\*Policy\*\*: 'Turn off default IPv6 DNS Servers' → \*\*Enabled\*\*

#### Step 2: Configure Network Connections Policies 1. In the target GPO, navigate to: `Computer Configuration\Administrative Templates\Network\Network Connections` 2. Configure the settings: \* **Policy**: `Prohibit use of Internet Connection Sharing on your DNS domain network` → **Enabled** \* **Policy**: `Prohibit installation and configuration of Network Bridge on your DNS domain network` → **Enabled** \* **Policy**: `Require domain users to elevate when setting a network's location` → **Enabled**

#### Step 3: Configure Windows Connection Manager Policies 1. In the target GPO, navigate to: `Computer Configuration\Administrative Templates\Network\Windows Connection Manager` 2. Configure the settings: \* **Policy**: `Minimize the number of simultaneous connections to the Internet or a Windows Domain` → **Enabled** with option set to **3 = Prevent Wi-Fi when on Ethernet** \* **Policy**: `Prohibit connection to non-domain networks when connected to domain authenticated network` → **Enabled**

#### Step 4: Configure WLAN Settings (WiFi Sense) 1. In the target GPO, navigate to: `Computer Configuration\Administrative Templates\Network\WLAN Service\WLAN Settings` 2. Configure the settings: \* **Policy**: `Allow Windows to automatically connect to suggested open hotspots, to networks shared by contacts, and to hotspots offering paid services` → **Disabled**

#### Step 5: Configure Spooler and HTTP Printing Policies 1. In the target GPO, navigate to: `Computer Configuration\Administrative Templates\System\Internet Communication Management\Internet Communication settings` 2. Configure the settings: \* **Policy**: `Turn off downloading of print drivers over HTTP` → **Enabled** \* **Policy**: `Turn off printing over HTTP` → **Enabled**

#### Step 6: Configure Network Access Security settings 1. In the target GPO, navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` 2. Configure the settings: \* **Policy**: `Network access: Restrict anonymous access to Named Pipes and Shares` → **Enabled**

#### Step 7: Disable WinHTTP WPAD Service 1. In the target GPO, navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` 2. Scroll to **WinHTTP Web Proxy Auto-Discovery Service**, select **Define this service setting**, and set the service startup mode to **Disabled**.

#### Step 8: Configure Registry Network settings via GPO Preferences 1. Under the target GPO, navigate to: `Computer Configuration\Preferences\Windows Settings\Registry` 2. Right-click **Registry** and select **New → Registry Item** for each of the following:

- **NetBIOS Name Release Protection:**

- **Action:** Update
- **Hive:** HKEY\_LOCAL\_MACHINE
- **Key Path:** SYSTEM\CurrentControlSet\Services\Netbt\Parameters
- **Value Name:** NoNameReleaseOnDemand
- **Value Type:** REG\_DWORD
- **Value Data:** 1

- **NetBIOS P-Node Type:**

- **Action:** Update
- **Hive:** HKEY\_LOCAL\_MACHINE
- **Key Path:** SYSTEM\CurrentControlSet\Services\Netbt\Parameters
- **Value Name:** NodeType
- **Value Type:** REG\_DWORD
- **Value Data:** 2

- **Disable ICMP Redirects:**

- **Action:** Update
- **Hive:** HKEY\_LOCAL\_MACHINE
- **Key Path:** SYSTEM\CurrentControlSet\Services\Tcpip\Parameters
- **Value Name:** EnableICMPRedirect
- **Value Type:** REG\_DWORD
- **Value Data:** 0

- **Disable IP Source Routing (IPv4):**

- **Action:** Update
- **Hive:** HKEY\_LOCAL\_MACHINE
- **Key Path:** SYSTEM\CurrentControlSet\Services\Tcpip\Parameters
- **Value Name:** DisableIPSourceRouting
- **Value Type:** REG\_DWORD
- **Value Data:** 2

- **Disable IP Source Routing (IPv6):**

- **Action:** Update
- **Hive:** HKEY\_LOCAL\_MACHINE
- **Key Path:** SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters
- **Value Name:** DisableIPSourceRouting
- **Value Type:** REG\_DWORD
- **Value Data:** 2

```
* **Disable WPAD Override (User Preference)**:
* **Action**: `Update`
* **Hive**: `HKEY_CURRENT_USER`
* **Key Path**: `Software\Microsoft\Windows\CurrentVersion\Internet Settings\Wpad`
* **Value Name**: `WpadOverride`
* **Value Type**: `REG_DWORD`
* **Value Data**: `1`

* **Restrict Net Session Enumeration (NetCease SDDL)**:
* **Action**: `Update`
* **Hive**: `HKEY_LOCAL_MACHINE`
* **Key Path**: `SYSTEM\CurrentControlSet\Services\LanmanServer\DefaultSecurity`
* **Value Name**: `SrvsvcSessionInfo`
* **Value Type**: `REG_BINARY`
* **Value Data**: Generate via SDDL `D:(A;;;CC;;;BA)(A;;;CC;;;S0)(A;;;CC;;;PU)`
```

#### Step 9: Disable NetBIOS (via DHCP Scope Options) 1. Open the **DHCP Management Console** (`dhcpgmt.msc`). 2. Under Scope Options, select **Configure Options**. 3. Add **Option 043 (Vendor Specific Info)** and set the NetBIOS over TCP/IP value to `0x2` (Disable NetBIOS over TCP/IP).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to disable legacy resolution and enforce secure TCP/IP registry parameters.

[Download Script: Set-PawNetworkHardening.ps1](#)



```
Set-PawNetworkHardening.ps1
Description: Configures local registry keys to disable LLMNR/NetBIOS, harden TCP/IP stack, prevent dual-homing, block hotspot auto
-connect, print driver web downloads, HTTP printing, and limit anonymous share access on PAWs.

Write-Host "Applying network and name resolution hardening..." -ForegroundColor Cyan

Helper to configure registry keys
function Set-RegDWord {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$path,
 [string]$name,
 [int]$value
)
 if ($PSCmdlet.ShouldProcess($path, "Set registry DWORD value $name to $value")) {
 $parent = Split-Path -Path $path
 if (-not (Test-Path $parent)) {
 New-Item -Path $parent -Force | Out-Null
 }
 if (-not (Test-Path $path)) {
 New-Item -Path $path -Force | Out-Null
 }
 Set-ItemProperty -Path $path -Name $name -Value $value -Type DWord -Force
 }
}

1. Disable LLMNR, mDNS, and default IPv6 DNS Servers
Set-RegDWord "HKLM:\Software\Policies\Microsoft\Windows NT\DNSClient" "EnableMulticast" 0
Set-RegDWord "HKLM:\Software\Policies\Microsoft\Windows NT\DNSClient" "EnablemDNS" 0
Set-RegDWord "HKLM:\Software\Policies\Microsoft\Windows NT\DNSClient" "DisableIPv6DefaultDnsServers" 1
Write-Host "[+] LLMNR (Multicast Name Resolution), mDNS, and default IPv6 DNS Servers disabled." -ForegroundColor Green

2. Configure NetBIOS Parameters
$NetbtPath = "HKLM:\SYSTEM\CurrentControlSet\Services\Netbt\Parameters"
if (-not (Test-Path $NetbtPath)) {
 New-Item -Path $NetbtPath -Force | Out-Null
}
Set-ItemProperty -Path $NetbtPath -Name "NoNameReleaseOnDemand" -Value 1 -Type DWord
Set-ItemProperty -Path $NetbtPath -Name "NodeType" -Value 2 -Type DWord
Write-Host "[+] NetBIOS name release protection and P-node type configured." -ForegroundColor Green

3. Disable NetBIOS over TCP/IP on all active adapters
Write-Host "[+] Disabling NetBIOS on all active network adapters..." -ForegroundColor Gray
$Adapters = Get-CimInstance -ClassName Win32_NetworkAdapterConfiguration -ErrorAction SilentlyContinue | Where-Object { $_.IPEnabled -eq $true }
if ($Adapters) {
 foreach ($Adapter in $Adapters) {
 Invoke-CimMethod -InputObject $Adapter -MethodName SetTCPIPNetBIOS -Arguments @{ TcpipNetbiosOptions = 2 } | Out-Null
 }
 Write-Host " NetBIOS disabled on active network interfaces." -ForegroundColor Green
}

4. Harden TCP/IP Parameters
Set-RegDWord "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" "EnableICMPRedirect" 0
Set-RegDWord "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" "DisableIPSourceRouting" 2
Write-Host "[+] IPv4 TCP/IP parameter redirects and source routing disabled." -ForegroundColor Green

Set-RegDWord "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters" "DisableIPSourceRouting" 2
Write-Host "[+] IPv6 TCP/IP parameter source routing disabled." -ForegroundColor Green

5. Prevent Network Connection Sharing and Dual-Homing Bridging
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Network Connections" "NC_ShowSharedAccessUI" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Network Connections" "NC_AllowNetBridge_NLA" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Network Connections" "NC_StdDomainUserSetLocation" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WcmSvc\GroupPolicy" "fMinimizeConnections" 3
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WcmSvc\GroupPolicy" "fBlockNonDomain" 1
Set-RegDWord "HKLM:\SOFTWARE\Microsoft\wcmSvc\wifinetworkmanager\config" "AutoConnectAllowedOEM" 0
Write-Host "[+] Network connections, sharing, bridging, elevation, and hotspot settings configured." -ForegroundColor Green

6. Printing Spooler Web Downloads and HTTP printing block
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers" "DisableWebPnPDownload" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers" "DisableHTTPPrinting" 1
Write-Host "[+] Printing spooler HTTP and Web service options disabled." -ForegroundColor Green
```

```
7. Restrict anonymous access to SAM and Named Pipes/Shares
Set-RegDWord "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters" "RestrictNullSessAccess" 1
Write-Host "[+] Anonymous null session share access restricted." -ForegroundColor Green

8. Disable WPAD
Write-Host "[+] Disabling WinHTTP Auto-Proxy service..." -ForegroundColor Gray
Set-Service -Name "WinHttpAutoProxySvc" -StartupType Disabled -ErrorAction SilentlyContinue
Stop-Service -Name "WinHttpAutoProxySvc" -Force -ErrorAction SilentlyContinue

$WpadPath = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Internet Settings\Wpad"
if (-not (Test-Path $WpadPath)) {
 New-Item -Path $WpadPath -Force | Out-Null
}
Set-ItemProperty -Path $WpadPath -Name "WpadOverride" -Value 1 -Type DWord -Force
Write-Host "[+] WPAD auto-detection disabled in user preferences registry." -ForegroundColor Green

9. Restrict Net Session Enumeration (NetCease SDDL)
Write-Host "[+] Restricting Net Session Enumeration..." -ForegroundColor Gray
try {
 $SD = New-Object System.Security.AccessControl.CommonSecurityDescriptor($false, $false, "D:(A;;CC;;;BA)(A;;CC;;;SD)(A;;CC;;;PU)")
 $BinaryForm = New-Object byte[] $SD.BinaryLength
 $SD.GetBinaryForm($BinaryForm, 0)
 $LanmanSecPath = "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\DefaultSecurity"
 if (-not (Test-Path $LanmanSecPath)) {
 New-Item -Path $LanmanSecPath -Force | Out-Null
 }
 Set-ItemProperty -Path $LanmanSecPath -Name "SrvsvcSessionInfo" -Value $BinaryForm -Type Binary -Force
 Write-Host "[+] Net Session Enumeration restricted to Admins/Operators/Power Users." -ForegroundColor Green
} catch {
 Write-Error "Failed to apply Net Session Enumeration restrictions: $($_.Exception.Message)"
}

Write-Host "Network and name resolution hardening applied successfully." -ForegroundColor Green
```

*To audit the network and name resolution status:*

[Download Script: Test-PawNetworkHardeningStatus.ps1](#)

```
Test-PawNetworkHardeningStatus.ps1
Description: Audits LLMNR, NetBIOS parameters, NetBIOS adapter state, TCP/IP parameters, and print/network connection options on P
AWS.

Write-Host "--- Auditing Network and Name Resolution Baseline ---" -ForegroundColor Cyan

$script:Vulnerable = $false

Helper function to audit registry properties
function Test-RegistryValue ($path, $name, $expectedValue) {
 $val = Get-ItemProperty -Path $path -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 $color = "Red"
 if ($actual -eq $expectedValue) {
 $color = "Green"
 } else {
 $script:Vulnerable = $true
 }
 Write-Host " - Registry Setting: $name | Actual: '$actual' (Expected: '$expectedValue')" -ForegroundColor $color
}

1. Audit LLMNR, mDNS, and default IPv6 DNS Servers
$DnsPath = "HKLM:\Software\Policies\Microsoft\Windows NT\DNSClient"
Test-RegistryValue $DnsPath "EnableMulticast" 0
Test-RegistryValue $DnsPath "EnablemDNS" 0
Test-RegistryValue $DnsPath "DisableIPv6DefaultDnsServers" 1

2. Audit NetBIOS Parameters
$NetbtPath = "HKLM:\SYSTEM\CurrentControlSet\Services\Netbt\Parameters"
Test-RegistryValue $NetbtPath "NoNameReleaseOnDemand" 1
Test-RegistryValue $NetbtPath "NodeType" 2

3. Audit TCP/IP Parameters
$TcpipPath = "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters"
Test-RegistryValue $TcpipPath "EnableICMPRedirect" 0
Test-RegistryValue $TcpipPath "DisableIPSourceRouting" 2

$Tcpip6Path = "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters"
Test-RegistryValue $Tcpip6Path "DisableIPSourceRouting" 2

4. Audit Connection Sharing & Dual-Homing Settings
$NetConnPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Network Connections"
Test-RegistryValue $NetConnPath "NC_ShowSharedAccessUI" 0
Test-RegistryValue $NetConnPath "NC_AllowNetBridge_NLA" 0
Test-RegistryValue $NetConnPath "NC_StdDomainUserSetLocation" 1

$WcmPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WcmSvc\GroupPolicy"
Test-RegistryValue $WcmPath "fMinimizeConnections" 3
Test-RegistryValue $WcmPath "fBlockNonDomain" 1

$WifiPath = "HKLM:\SOFTWARE\Microsoft\wcmsvc\wifinetworkmanager\config"
Test-RegistryValue $WifiPath "AutoConnectAllowedOEM" 0

5. Audit HTTP Print Options
$PrinterPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers"
Test-RegistryValue $PrinterPath "DisableWebPnPDownload" 1
Test-RegistryValue $PrinterPath "DisableHTTPPPrinting" 1

6. Audit Null Session Share Restrict
$ServerPath = "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters"
Test-RegistryValue $ServerPath "RestrictNullSessAccess" 1

7. Audit WPAD Service and Registry Override
$WpadSvc = Get-Service -Name "WinHttpAutoProxySvc" -ErrorAction SilentlyContinue
if ($null -ne $WpadSvc) {
 if ($WpadSvc.StartType -eq "Disabled") {
 Write-Host " - WPAD Service State: Disabled (Secure)" -ForegroundColor Green
 } else {
 Write-Host " - VULNERABLE: WPAD Service StartType is $($WpadSvc.StartType) (Expected: Disabled)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
}
}

$WpadPath = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Internet Settings\Wpad"
```

```

Test-RegistryValue $WpadPath "WpadOverride" 1

8. Audit Net Session Enumeration (NetCease)
$LanmanSecPath = "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\DefaultSecurity"
if (Test-Path $LanmanSecPath) {
 $SrvsvcSessionInfo = (Get-ItemProperty -Path $LanmanSecPath -Name "SrvsvcSessionInfo" -ErrorAction SilentlyContinue).SrvsvcSessionInfo
 if ($null -ne $SrvsvcSessionInfo) {
 try {
 $SD = New-Object System.Security.AccessControl.CommonSecurityDescriptor($false, $false, $SrvsvcSessionInfo, 0)
 $Sddl = $SD.GetSddlForm("Dacl")
 if ($Sddl -eq "D:(A;;CC;;;BA)(A;;CC;;;SO)(A;;CC;;;PU)" {
 Write-Host " - Net Session Enumeration Security Descriptor: Hardened (Secure)" -ForegroundColor Green
 } else {
 Write-Host " - VULNERABLE: Net Session Enumeration Security Descriptor is '$Sddl' (Expected: 'D:(A;;CC;;;BA)(A;;CC;;;SO)(A;;CC;;;PU)')" -ForegroundColor Red
 $script:Vulnerable = $true
 }
 } catch {
 Write-Host " - VULNERABLE: Failed to parse Net Session Enumeration security descriptor." -ForegroundColor Red
 $script:Vulnerable = $true
 }
 } else {
 Write-Host " - VULNERABLE: SrvsvcSessionInfo registry value not found." -ForegroundColor Red
 $script:Vulnerable = $true
 }
} else {
 Write-Host " - VULNERABLE: LanmanServer\DefaultSecurity path not found." -ForegroundColor Red
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
}

```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10/11 Client Benchmark\*\*: Section 18.6.4.1 (EnablemDNS), Section 18.6.11.2 (NC\_AllowNetBridge\_NLA), Section 18.6.11.3 (NC\_ShowSharedAccessUI), Section 18.6.11.4 (NC\_StdDomainUserSetLocation), Section 18.6.21.1 & 18.6.21.2 (fMinimizeConnections & fBlockNonDomain), Section 18.6.23.2.1 (AutoConnectAllowedOEM). \* \*\*CIS Microsoft Windows 10/11 Client Benchmark\*\*: Section 9.1 (Disable LLNMR), Section 18.8.44.1 (Configure EnableICMPRedirect), Section 18.8.44.2 (Configure DisableIPSourceRouting). \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R19 (LDAP and name resolution security recommendations). \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Print driver download limits, HTTP printing blocks, internet connection sharing prohibitions, hotspot connection rules, and null session restrictions.

# [REQ-PAW-020] Configure User Account Control Policies for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: \* `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` \* `Computer Configuration\Administrative Templates\System` \* \*\*Registry Locations\*\*: \* `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System` \* `ConsentPromptBehaviorAdmin` = `1` (REG\_DWORD, Prompt for credentials on secure desktop) \* `ConsentPromptBehaviorUser` = `0` (REG\_DWORD, Automatically deny elevation requests) \* `EnableLUA` = `1` (REG\_DWORD, Enable User Account Control / Admin Approval Mode) \* `PromptOnSecureDesktop` = `1` (REG\_DWORD, Switch to secure desktop when prompting) \* `LocalAccountTokenFilterPolicy` = `0` (REG\_DWORD, Apply UAC restrictions to local accounts on network logons) \* `EnableInstallDetection` = `1` (REG\_DWORD, Detect application installations and prompt for elevation) \* `EnableVirtualization` = `1` (REG\_DWORD, Virtualize file and registry write failures to per-user locations) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows\Sudo` \* `Enabled` = `1` (REG\_DWORD, Sudo behavior set to force a new elevated window)

---

## Rationale User Account Control (UAC) is a fundamental defense mechanism in Windows. It limits the privilege levels of running applications, executing administrative actions with standard user tokens unless elevated privileges are explicitly approved.

Hardening UAC settings ensures:

1. **Secure Desktop Enforcement:** The elevation prompt is displayed on a separate, secure desktop environment that isolated system threads run on. This prevents third-party malware running in user space from intercepting credentials or programmatically clicking "Yes" to elevate itself.
2. **Auto-Denial of Standard User Elevation:** Standard users should not be allowed to request elevation. If a standard user triggers a task requiring administrative rights, the prompt should auto-deny rather than requesting an administrator password, preventing users from attempting to bypass controls or exposing local admin passwords on a non-secure user terminal.
3. **Admin Approval Mode:** Forcing built-in administrators to run in Admin Approval Mode ensures that even administrative users do not run web browsers or document editors with administrative tokens by default.
4. **Sudo Command Control:** The `sudo` command introduced in Windows 11 (24H2) allows users to run elevated commands from an unelevated console. Leaving this feature unconfigured or allowing execution within the current console session can expose elevated processes to command injection or token interception in the same console session. Restricting `sudo` to opening a new elevated window ( `1` ) or disabling it entirely ( `0` ) mitigates session hijacking risks.
5. **Network UAC Restrictions ( `LocalAccountTokenFilterPolicy` ):** Restricting the elevation of local accounts during network logons prevents lateral movement. When set to `0` , local accounts (except for the built-in Administrator RID 500 account) connecting remotely via network shares or administrative interfaces cannot obtain administrative tokens, neutralizing pass-the-hash attacks using secondary local administrative accounts.
6. **Installer Detection ( `EnableInstallDetection` ):** Detecting installer program behavior prevents silent software execution. When enabled, any execution of an install file or setup program by standard users or administrators triggers a UAC elevation prompt, preventing unauthorized silent program deployments.
7. **UAC Virtualization ( `EnableVirtualization` ):** Virtualizing writes redirection keeps the operating system directory space clean. It redirects legacy application registry and file writes targeting system folders (like `Program Files` or `System32` ) to user-profile-specific folders, allowing legacy applications to run without requiring administrative rights.

---

## Legacy Impact & Compatibility \* \*\*User Experience\*\*: Standard users will not be able to install software or change system settings that require administrative credentials. Support technicians must log on as local administrators to perform maintenance tasks or use remote tools. Note that interactive logons for standard users are already blocked on PAWs via User Rights Assignments. \* \*\*Script and Installer Behaviors\*\*: Legacy scripts and administrative install tasks that run programmatically without secure-desktop awareness may fail or hang if they trigger elevation prompts that cannot be programmatically bypassed. \* \*\*Network Admin Access\*\*: Network-based remote administration using local accounts will be restricted to the RID 500 account. Domain accounts must be used for remote administrative access (WinRM, SMB administration) on standard endpoints.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options`
4. Configure the following settings:
  - **Policy:** `User Account Control: Behavior of the elevation prompt for administrators in Admin Approval Mode` → **Prompt for credentials on the secure desktop**
  - **Policy:** `User Account Control: Behavior of the elevation prompt for standard users` → **Automatically deny elevation requests**
  - **Policy:** `User Account Control: Run all administrators in Admin Approval Mode` → **Enabled**
  - **Policy:** `User Account Control: Switch to the secure desktop when prompting for elevation` → **Enabled**

- **Policy:** User Account Control: Detect application installations and prompt for elevation → **Enabled**
- **Policy:** User Account Control: Virtualize file and registry write failures to per-user locations → **Enabled**

5. Since the UAC network restrictions policy is not directly exposed in standard GPO security templates, deploy the registry setting via GPO Preferences:

- Navigate to: Computer Configuration\Preferences\Windows Settings\Registry
- Create a new Registry Item:
  - **Action:** Update
  - **Hive:** HKEY\_LOCAL\_MACHINE
  - **Key Path:** SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System
  - **Value Name:** LocalAccountTokenFilterPolicy
  - **Value Type:** REG\_DWORD
  - **Value Data:** 0

6. Navigate to: Computer Configuration\Administrative Templates\System

- **Policy:** Configure the behavior of the sudo command → **Enabled** with options set to **Force a new elevated window**

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to configure maximum security parameters for UAC in the system registry.

[Download Script: Configure-PawUACPolicies.ps1](#)

```
Configure-PawUACPolicies.ps1
Description: Enforces hardened User Account Control (UAC) registry configuration values including network restrictions, installer
detection, and virtualization on PAWs.

Write-Host "--- Hardening User Account Control Policies ---" -ForegroundColor Cyan

$SystemPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"

if (-not (Test-Path $SystemPath)) {
 New-Item -Path $SystemPath -Force | Out-Null
}

ConsentPromptBehaviorAdmin = 1 (Prompt for credentials on secure desktop)
Set-ItemProperty -Path $SystemPath -Name "ConsentPromptBehaviorAdmin" -Value 1 -Type DWord -Force
ConsentPromptBehaviorUser = 0 (Automatically deny elevation requests)
Set-ItemProperty -Path $SystemPath -Name "ConsentPromptBehaviorUser" -Value 0 -Type DWord -Force
EnableLUA = 1 (Enable User Account Control / Admin Approval Mode)
Set-ItemProperty -Path $SystemPath -Name "EnableLUA" -Value 1 -Type DWord -Force
PromptOnSecureDesktop = 1 (Switch to secure desktop when prompting)
Set-ItemProperty -Path $SystemPath -Name "PromptOnSecureDesktop" -Value 1 -Type DWord -Force
LocalAccountTokenFilterPolicy = 0 (Apply UAC restrictions to local accounts on network logons)
Set-ItemProperty -Path $SystemPath -Name "LocalAccountTokenFilterPolicy" -Value 0 -Type DWord -Force
EnableInstallDetection = 1 (Detect application installations and prompt for elevation)
Set-ItemProperty -Path $SystemPath -Name "EnableInstallDetection" -Value 1 -Type DWord -Force
EnableVirtualization = 1 (Virtualize file and registry write failures to per-user locations)
Set-ItemProperty -Path $SystemPath -Name "EnableVirtualization" -Value 1 -Type DWord -Force

Configure Windows Sudo command behavior (Enabled = 1 [Force new elevated window])
$SudoPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Sudo"
if (-not (Test-Path $SudoPath)) {
 New-Item -Path $SudoPath -Force | Out-Null
}
Set-ItemProperty -Path $SudoPath -Name "Enabled" -Value 1 -Type DWord -Force

Write-Host "[+] UAC registry values configured successfully." -ForegroundColor Green
```

To audit UAC configurations on the PAW:

[Download Script: Test-PawUACPolicies.ps1](#)

```
Test-PawUACPolicies.ps1
Description: Verifies local system registry settings for User Account Control on PAWs.

Write-Host "--- Auditing User Account Control Policies ---" -ForegroundColor Cyan

$SystemPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
$script:Vulnerable = $false

function Test-UACRegistryValue ($name, $expected, $message) {
 $val = Get-ItemProperty -Path $SystemPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { $null }
 $color = "Red"
 if ($actual -eq $expected) {
 $color = "Green"
 } else {
 $script:Vulnerable = $true
 }
 Write-Host " - Registry Setting: $name | Actual: '$actual' (Expected: '$expected') | $message" -ForegroundColor $color
}

Test-UACRegistryValue "ConsentPromptBehaviorAdmin" 1 "Behavior of elevation prompt for administrators"
Test-UACRegistryValue "ConsentPromptBehaviorUser" 0 "Behavior of elevation prompt for standard users"
Test-UACRegistryValue "EnableLUA" 1 "Run all administrators in Admin Approval Mode"
Test-UACRegistryValue "PromptOnSecureDesktop" 1 "Switch to secure desktop when prompting"
Test-UACRegistryValue "LocalAccountTokenFilterPolicy" 0 "UAC network restrictions"
Test-UACRegistryValue "EnableInstallDetection" 1 "Installer detection"
Test-UACRegistryValue "EnableVirtualization" 1 "UAC virtualization"

Audit Sudo command
$SudoPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Sudo"
if (Test-Path $SudoPath) {
 $SudoState = Get-ItemProperty -Path $SudoPath -Name "Enabled" -ErrorAction SilentlyContinue
 $SudoVal = if ($SudoState) { $SudoState.Enabled } else { 0 }
 $SudoColor = if ($SudoVal -eq 0 -or $SudoVal -eq 1) { "Green" } else { "Red" }
 Write-Host " - Sudo Command Enabled state: $SudoVal (Required = 1 [New Window] or 0 [Disabled])" -ForegroundColor $SudoColor
 if ($SudoVal -ne 0 -and $SudoVal -ne 1) {
 $script:Vulnerable = $true
 }
} else {
 Write-Host " - Sudo Command Enabled state: Not Configured (Default/Compliant as it inherits disabled)" -ForegroundColor Green
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10/11 Client Benchmark\*\*:

- Section 2.3.17.1 (ConsentPromptBehaviorAdmin),
- Section 2.3.17.2 (ConsentPromptBehaviorUser),
- Section 2.3.17.4 (EnableInstallDetection),
- Section 2.3.17.5 (EnableLUA),
- Section 2.3.17.8 (EnableVirtualization)

\* \*\*CIS Microsoft Windows 10/11 Client Benchmark\*\*:

- Section 18.4.1 (LocalAccountTokenFilterPolicy)

\* \*\*Microsoft Security Baselines\*\*:

- Windows Client Security baseline registry settings.



# [REQ-PAW-021] Disable AutoPlay and AutoRun for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Administrative Templates\Windows Components\AutoPlay Policies` \* \*\*Registry Locations\*\*: \*  
`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer` \* `NoDriveTypeAutoRun` = `255` (0xFF, REG\_DWORD) \* `NoAutorun`  
= `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows\Explorer` \* `NoAutoplayfornonVolume` = `1` (REG\_DWORD)

---

## Rationale The AutoPlay and AutoRun features in Windows are designed to automatically execute programs or open media when a removable drive, network share, or CD-ROM is inserted or connected.

Attackers exploit these features by placing malicious scripts, payloads, or executables on USB drives or external storage media. If AutoPlay is enabled, connecting the drive triggers automatic execution of these scripts or programs without user interaction or approval, allowing malware to achieve immediate execution in the context of the logged-on user. Disabling AutoPlay across all drive types completely mitigates this physical transmission vector.

Additionally, non-volume devices (such as mobile phones, cameras, or media players) can still trigger AutoPlay behavior. Disallowing AutoPlay for non-volume devices ensures these devices do not introduce unauthorized execution pathways when plugged into standard client machines.

---

## Legacy Impact & Compatibility \* \*\*User Experience\*\*: Users will not see pop-up choices or automated actions when connecting USB devices or external drives. They must manually open File Explorer and navigate to the drive to read or open files. \* \*\*Installer Media\*\*: Software installers located on external media will not start automatically; users must double-click the setup application manually.

---

#### ## Implementation Steps

##### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
  2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
  3. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\AutoPlay Policies`
  4. Configure the following settings:
    - **Policy:** `Turn off AutoPlay`
      - **Setting:** `Enabled`
      - **Select Options:** `All drives`
    - **Policy:** `Set the default behavior for AutoRun`
      - **Setting:** `Enabled`
      - **Select Options:** `Do not execute any autorun commands`
    - **Policy:** `Disallow Autoplay for non-volume devices`
      - **Setting:** `Enabled`
- 

##### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to configure Explorer registry keys to disable AutoPlay, AutoRun, and AutoPlay for non-volume devices.

[Download Script: Disable-PawAutoPlay.ps1](#)



```
Disable-PawAutoPlay.ps1
Description: Disables AutoPlay/AutoRun registry settings globally on all drive types and non-volume devices on PAWs.

Write-Host "--- Disabling AutoPlay and AutoRun ---" -ForegroundColor Cyan

$ExplorerPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer"

if (-not (Test-Path $ExplorerPath)) {
 New-Item -Path $ExplorerPath -Force | Out-Null
}

NoDriveTypeAutoRun = 0xFF (255 in decimal) disables AutoRun on all types of drives
Set-ItemProperty -Path $ExplorerPath -Name "NoDriveTypeAutoRun" -Value 255 -Type DWord -Force

NoAutorun = 1 disables AutoRun commands in inf files
Set-ItemProperty -Path $ExplorerPath -Name "NoAutorun" -Value 1 -Type DWord -Force

Disallow Autoplay for non-volume devices (NoAutoplayfornonVolume = 1)
$PolExplorerPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Explorer"
if (-not (Test-Path $PolExplorerPath)) {
 New-Item -Path $PolExplorerPath -Force | Out-Null
}
Set-ItemProperty -Path $PolExplorerPath -Name "NoAutoplayfornonVolume" -Value 1 -Type DWord -Force

Write-Host "[+] AutoPlay and AutoRun registry parameters set." -ForegroundColor Green
```

*To audit AutoPlay configurations on the PAW:*

[Download Script: Test-PawAutoPlay.ps1](#)

```
Test-PawAutoPlay.ps1
Description: Audits local system registry parameters for AutoPlay status on PAWs.

Write-Host "--- Auditing AutoPlay Configuration ---" -ForegroundColor Cyan

$ExplorerPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer"
$PolExplorerPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Explorer"

$NoDriveAuto = Get-ItemProperty -Path $ExplorerPath -Name "NoDriveTypeAutoRun" -ErrorAction SilentlyContinue
$NoAutoCmd = Get-ItemProperty -Path $ExplorerPath -Name "NoAutorun" -ErrorAction SilentlyContinue
$NoNonVol = Get-ItemProperty -Path $PolExplorerPath -Name "NoAutoplayfornonVolume" -ErrorAction SilentlyContinue

$NoDriveVal = if ($NoDriveAuto) { $NoDriveAuto.NoDriveTypeAutoRun } else { 0 }
$NoAutoVal = if ($NoAutoCmd) { $NoAutoCmd.NoAutorun } else { 0 }
$NoNonVolVal = if ($NoNonVol) { $NoNonVol.NoAutoplayfornonVolume } else { 0 }

$NoDriveColor = if ($NoDriveVal -eq 255) { "Green" } else { "Red" }
$NoAutoColor = if ($NoAutoVal -eq 1) { "Green" } else { "Red" }
$NoNonVolColor = if ($NoNonVolVal -eq 1) { "Green" } else { "Red" }

Write-Host " - NoDriveTypeAutoRun: $NoDriveVal (Required = 255 to disable all drives)" -ForegroundColor $NoDriveColor
Write-Host " - NoAutorun: $NoAutoVal (Required = 1)" -ForegroundColor $NoAutoColor
Write-Host " - NoAutoplayfornonVolume: $NoNonVolVal (Required = 1)" -ForegroundColor $NoNonVolColor
```

---

**## Sources & Compliance References** \* **CIS Microsoft Windows 10 Benchmark**: Section 18.3.1 (Turn off AutoPlay), Section 18.3.2 (Set the default behavior for AutoRun) \* **Microsoft Security Baselines**: Windows Client Explorer configuration standards. \* **DoD Windows 11 STIG**: Disallow Autoplay for non-volume devices requirements.

# [REQ-PAW-022] Disable Incoming Remote Desktop Access for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: \* `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Connections` \* `Computer Configuration\Administrative Templates\System\Remote Assistance` \* \*\*Registry Locations\*\*: \* `HKLM\SYSTEM\CurrentControlSet\Control\Terminal Server` \* `fDenyTSConnections` = `1` (REG\_DWORD, Disabled) \* `fAllowToGetHelp` = `0` (REG\_DWORD, Solicited Remote Assistance Disabled) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services` \* `fAllowToGetHelp` = `0` (REG\_DWORD, Solicited Remote Assistance Policy Disabled)

---

## Rationale Remote Desktop Protocol (RDP) is one of the primary mechanisms used by attackers for lateral movement and administrative session hijacking.

For Privileged Access Workstations (PAWs), which manage Tier 0 administrative assets:

1. **Lateral Movement Prevention:** PAWs represent physical console endpoints used to administer the forest. They must never accept inbound network connections. Disabling incoming RDP connections prevents attackers from pivoting from compromised general workstations to the PAW.
2. **Session Security:** Eliminating RDP listener ports prevents credential sniffing, password spraying, and remote exploitation of remote desktop services vulnerabilities on the administrative root of trust.
3. **Remote Assistance Block:** Disabling solicited remote assistance prevents potential remote command execution or remote support hijacking.

---

## Legacy Impact & Compatibility \* \*\*No Remote Administration\*\*: Support technicians cannot connect to PAWs via RDP. All operations on a PAW must be performed directly at the physical console. \* \*\*User Assistance\*\*: Support sessions cannot be initiated via Remote Desktop or Remote Assistance. Local diagnostic procedures must be used.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

#### 1. Disable Inbound Remote Desktop Connections 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Edit the PAW GPO (e.g., `GPO\_Hardening\_PAW`). 3. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Connections` 4. Configure the setting: \* \*\*Policy\*\*: `Allow users to connect remotely by using Remote Desktop Services` \* \*\*Setting\*\*: `Disabled`

#### 2. Disable Solicited Remote Assistance 1. Navigate to: `Computer Configuration\Administrative Templates\System\Remote Assistance` 2. Configure the setting: \* \*\*Policy\*\*: `Configure Solicited Remote Assistance` \* \*\*Setting\*\*: `Disabled`

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to disable Remote Desktop and Remote Assistance, and enforce NLA and secure registry keys.

[Download Script: Disable-PawRemoteDesktop.ps1](#)

```
Disable-PawRemoteDesktop.ps1
Description: Disables Remote Desktop and Solicited Remote Assistance connections, sets NLA requirements, and cleans parameters on
PAWs.

Write-Host "--- Restricting Remote Desktop and Remote Assistance Access ---" -ForegroundColor Cyan

$RdpPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server"

1. Disable RDP Connections (fDenyTSConnections = 1)
Set-ItemProperty -Path $RdpPath -Name "fDenyTSConnections" -Value 1 -Type DWord -Force
Write-Host "[+] Inbound Remote Desktop connections disabled." -ForegroundColor Green

2. Enforce Network Level Authentication (UserAuthentication = 1)
$RdpSecPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp"
if (Test-Path $RdpSecPath) {
 Set-ItemProperty -Path $RdpSecPath -Name "UserAuthentication" -Value 1 -Type DWord -Force
 Write-Host "[+] Network Level Authentication (NLA) enforced." -ForegroundColor Green
}

3. Disable Remote Assistance (fAllowToGetHelp = 0)
Set-ItemProperty -Path $RdpPath -Name "fAllowToGetHelp" -Value 0 -Type DWord -Force

4. Disable and clean Solicited Remote Assistance Policies, set SSL, and delete temp folders
$TSPoliciesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services"
if (-not (Test-Path $TSPoliciesPath)) {
 New-Item -Path $TSPoliciesPath -Force | Out-Null
}
Set-ItemProperty -Path $TSPoliciesPath -Name "fAllowToGetHelp" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $TSPoliciesPath -Name "SecurityLayer" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $TSPoliciesPath -Name "DeleteTempDirsOnExit" -Value 1 -Type DWord -Force

$paramsToDelete = @("MaxTicketExpiryUnits", "MaxTicketExpiry", "fUseMailto", "fAllowFullControl")
foreach ($Param in $paramsToDelete) {
 if (Get-ItemProperty -Path $TSPoliciesPath -Name $Param -ErrorAction SilentlyContinue) {
 Remove-ItemProperty -Path $TSPoliciesPath -Name $Param -Force -ErrorAction SilentlyContinue
 }
}
Write-Host "[+] Remote Desktop policies (SSL, Temp folders, Solicited Help) configured and cleaned." -ForegroundColor Green
```

*To audit Remote Desktop and Remote Assistance status on the PAW:*

[Download Script: Test-PawRemoteDesktopStatus.ps1](#)

```
Test-PawRemoteDesktopStatus.ps1
Description: Audits local RDP, Remote Assistance, security layer, temp folders, and NLA registry configuration and listening firew
all ports on PAWs.

Write-Host "--- Auditing Remote Desktop Configuration ---" -ForegroundColor Cyan

$RdpPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server"
$RdpSecPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp"
$TSPoliciesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services"

$DenyTS = Get-ItemProperty -Path $RdpPath -Name "fDenyTSConnections" -ErrorAction SilentlyContinue
$DenyVal = if ($DenyTS) { $DenyTS.fDenyTSConnections } else { 1 }

$NlaProp = Get-ItemProperty -Path $RdpSecPath -Name "UserAuthentication" -ErrorAction SilentlyContinue
$NlaVal = if ($NlaProp) { $NlaProp.UserAuthentication } else { 0 }

$DenyColor = if ($DenyVal -eq 1) { "Green" } else { "Red" }
$NlaColor = if ($NlaVal -eq 1) { "Green" } else { "Red" }

Write-Host " - fDenyTSConnections: $DenyVal (Required = 1 to block all)" -ForegroundColor $DenyColor
Write-Host " - UserAuthentication (NLA): $NlaVal (Required = 1 if RDP is enabled)" -ForegroundColor $NlaColor

Check if port 3389 firewall rule is active and enabled
$RdpFirewall = Get-NetFirewallRule -Name "RemoteDesktop-UserMode-In-TCP" -ErrorAction SilentlyContinue
if ($RdpFirewall) {
 $FirewallColor = if ($RdpFirewall.Enabled -eq $true) { "Red" } else { "Green" }
 Write-Host " - RDP Inbound Firewall Rule Active: $($RdpFirewall.Enabled) (Expected = False)" -ForegroundColor $FirewallColor
}

Audit Remote Assistance
$GetHelpTS = Get-ItemProperty -Path $RdpPath -Name "fAllowToGetHelp" -ErrorAction SilentlyContinue
$GetHelpTSVal = if ($GetHelpTS) { $GetHelpTS.fAllowToGetHelp } else { 0 }
$HelpColor = if ($GetHelpTSVal -eq 0) { "Green" } else { "Red" }
Write-Host " - fAllowToGetHelp (Terminal Server): $GetHelpTSVal (Recommended = 0)" -ForegroundColor $HelpColor

Audit Solicited Remote Assistance Policy, Security Layer, and Temp Folders
if (Test-Path $TSPoliciesPath) {
 $PolGetHelp = Get-ItemProperty -Path $TSPoliciesPath -Name "fAllowToGetHelp" -ErrorAction SilentlyContinue
 $PolGetHelpVal = if ($PolGetHelp) { $PolGetHelp.fAllowToGetHelp } else { $null }

 $PolHelpColor = if ($PolGetHelpVal -eq 0) { "Green" } else { "Red" }
 Write-Host " - fAllowToGetHelp (Policies): $PolGetHelpVal (Recommended = 0)" -ForegroundColor $PolHelpColor

 $SecurityLayerProp = Get-ItemProperty -Path $TSPoliciesPath -Name "SecurityLayer" -ErrorAction SilentlyContinue
 $SecurityLayerVal = if ($SecurityLayerProp) { $SecurityLayerProp.SecurityLayer } else { $null }
 $SecLayerColor = if ($SecurityLayerVal -eq 2) { "Green" } else { "Red" }
 Write-Host " - SecurityLayer (SSL): $SecurityLayerVal (Required = 2)" -ForegroundColor $SecLayerColor

 $DeleteTempProp = Get-ItemProperty -Path $TSPoliciesPath -Name "DeleteTempDirsOnExit" -ErrorAction SilentlyContinue
 $DeleteTempVal = if ($DeleteTempProp) { $DeleteTempProp.DeleteTempDirsOnExit } else { $null }
 $DeleteTempColor = if ($DeleteTempVal -eq 1) { "Green" } else { "Red" }
 Write-Host " - DeleteTempDirsOnExit (Temp Folders): $DeleteTempVal (Required = 1)" -ForegroundColor $DeleteTempColor

 $Params = @("MaxTicketExpiryUnits", "MaxTicketExpiry", "fUseMailto", "fAllowFullControl")
 foreach ($Param in $Params) {
 $Val = (Get-ItemProperty -Path $TSPoliciesPath -Name $Param -ErrorAction SilentlyContinue).$Param
 if ($null -ne $Val) {
 Write-Host " - VULNERABLE: Solicited Remote Assistance parameter '$Param' is set to '$Val' (Expected: Deleted/Not Con
figured)" -ForegroundColor Red
 } else {
 Write-Host " - Parameter '$Param': Not Configured (Correct)" -ForegroundColor Green
 }
 }
} else {
 Write-Host " - Solicited Remote Assistance Policy Path does not exist" -ForegroundColor Red
}
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*: Section 18.2.1 (Require user authentication for remote connections by using Network Level Authentication) \* \*\*ANSSI AD Hardening Guide\*\*: Security guidelines regarding Remote Desktop access and management protocols on administrative systems. \* \*\*DoD Windows 11 STIG\*\*: Solicited Remote Assistance requirements.

## # [REQ-PAW-023] WSUS Client Configuration for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: \* `Computer Configuration\Administrative Templates\Windows Components\Windows Update` \* `Computer Configuration\Administrative Templates\Windows Components\Delivery Optimization` \* \*\*Registry Locations\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate` \* `WUserver` (REG\_SZ) \* `WUstatusServer` (REG\_SZ) \* `DoNotConnectToWindowsUpdateInternetLocations` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU` \* `NoAutoUpdate` = `0` (REG\_DWORD) \* `AUOptions` = `4` (REG\_DWORD) \* `UseWUserver` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows\DeliveryOptimization` \* `DODownloadMode` = `2` (REG\_DWORD)

---

## Rationale In an isolated, air-gapped network, workstations cannot connect directly to Microsoft's online Update servers. If the system is left in its default configuration: 1. **DNS/Firewall Pollution**: Workstations will continuously attempt to resolve and connect to public Windows Update URLs (e.g., `*.update.microsoft.com`), filling firewall and local DNS resolver cache logs with timeouts and block events. 2. **Missing Updates**: Workstations will fail to receive security patches, critical updates, and Windows Defender definitions. 3. **Control Bypass**: Attackers or unapproved software could attempt to install out-of-band features or packages if update routes are not explicitly locked to internal sources.

Enforcing the intranet update service location redirects all system update queries to the local WSUS server. Furthermore, enforcing Windows **Delivery Optimization** download mode to [Group \(2\)](#) limits peer-to-peer update sharing strictly to computers within the same active directory domain/group or local subnet boundaries, reducing bandwidth constraints on WAN/intranet segments and preventing unmanaged peer sharing.

---

## Legacy Impact & Compatibility \* \*\*WSUS Dependency\*\*: The local Windows Server Update Services (WSUS) server must remain online and functional. If the WSUS server is unreachable, workstations will be unable to query, download, or apply security patches. \* **Administrative Burden**: Enterprise administrators must manually synchronize the WSUS database (sneakernet transfer of metadata and updates) to ensure new patches are made available to clients. \* **Delivery Optimization limits**: Group mode restricts Delivery Optimization cache sharing to standard networks. If endpoints span multiple separated sites without route aggregation, update replication times could increase slightly.

---

## ## Implementation Steps

## ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
  2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
  3. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Windows Update`
  4. Configure the following settings:
    - **Policy**: `Configure Automatic Updates`
      - **Setting**: `Enabled`
      - **Configure automatic updating**: `4 - Auto download and schedule the install`
  5. Configure the service location:
    - **Policy**: `Specify intranet Microsoft update service location`
      - **Setting**: `Enabled`
      - **Set the intranet update service for detecting updates**: `http://local-wsus.domain.local:8530` (Replace with your internal WSUS FQDN or IP)
      - **Set the intranet statistics server**: `http://local-wsus.domain.local:8530`
  6. Navigate to:
    - **Policy**: `Do not connect to any Windows Update Internet locations`
      - **Setting**: `Enabled` (blocks fallback to public servers)
  7. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Delivery Optimization`
  8. Configure the following settings:
    - **Policy**: `Download Mode`
      - **Setting**: `Enabled`
      - **Download Mode**: `Group (2)`
- 

## ### Option B: PowerShell &amp; Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to configure registry keys to enforce local WSUS parameters and Delivery Optimization download mode.

[Download Script: Set-PawWSUSClientConfiguration.ps1](#)

```
Set-PawWSUSClientConfiguration.ps1
Description: Configures local registry keys to point the Windows Update client to the intranet WSUS server and enforces D0 Group mode on PAWs.

Write-Host "--- Configuring WSUS Client Settings ---" -ForegroundColor Cyan

$WUPPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate"
$WUAUPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU"
$DOPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeliveryOptimization"

1. Create keys if they do not exist
if (-not (Test-Path $WUPPath)) {
 New-Item -Path $WUPPath -Force | Out-Null
}
if (-not (Test-Path $WUAUPath)) {
 New-Item -Path $WUAUPath -Force | Out-Null
}
if (-not (Test-Path $DOPath)) {
 New-Item -Path $DOPath -Force | Out-Null
}

Define intranet WSUS URL
$WSUSServer = "http://local-wsus.domain.local:8530"

2. Configure update location and statistics server
Set-ItemProperty -Path $WUPPath -Name "WUServer" -Value $WSUSServer -Type String -Force
Set-ItemProperty -Path $WUPPath -Name "WUStatusServer" -Value $WSUSServer -Type String -Force
Set-ItemProperty -Path $WUPPath -Name "DoNotConnectToWindowsUpdateInternetLocations" -Value 1 -Type DWord -Force

3. Configure Automatic Updates behavior (AUOptions = 4: Auto Download & Schedule)
Set-ItemProperty -Path $WUAUPath -Name "NoAutoUpdate" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $WUAUPath -Name "AUOptions" -Value 4 -Type DWord -Force
Set-ItemProperty -Path $WUAUPath -Name "UseWUServer" -Value 1 -Type DWord -Force

4. Enforce D0DownloadMode = 2 (Group)
Set-ItemProperty -Path $DOPath -Name "D0DownloadMode" -Value 2 -Type DWord -Force

Write-Host "[+] Local WSUS parameters and Delivery Optimization download mode applied." -ForegroundColor Green
```

*To audit the WSUS client configuration status on the PAW:*

[Download Script: Test-PawWSUSClientStatus.ps1](#)

```
Test-PawWSUSClientStatus.ps1
Description: Audits registry values to verify WSUS server assignment and Delivery Optimization configuration on PAWs.

Write-Host "--- Auditing WSUS Client Settings ---" -ForegroundColor Cyan

$WUPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate"
$WUAUPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU"
$DOPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeliveryOptimization"

$WUServerProp = Get-ItemProperty -Path $WUPath -Name "WUServer" -ErrorAction SilentlyContinue
$WUStatusProp = Get-ItemProperty -Path $WUPath -Name "WUStatusServer" -ErrorAction SilentlyContinue
$UseWUServerProp = Get-ItemProperty -Path $WUAUPath -Name "UseWUServer" -ErrorAction SilentlyContinue

$WUServerVal = if ($WUServerProp) { $WUServerProp.WUServer } else { "" }
$WUStatusVal = if ($WUStatusProp) { $WUStatusProp.WUStatusServer } else { "" }
$UseWUVal = if ($UseWUServerProp) { $UseWUServerProp.UseWUServer } else { 0 }

$ServerColor = if ($WUServerVal -like "http*") { "Green" } else { "Red" }
$UseColor = if ($UseWUVal -eq 1) { "Green" } else { "Red" }

Write-Host " - Intranet WUServer: $WUServerVal" -ForegroundColor $ServerColor
Write-Host " - Intranet WUStatusServer: $WUStatusVal" -ForegroundColor $ServerColor
Write-Host " - UseWUServer Active: $UseWUVal (Required = 1)" -ForegroundColor $UseColor

Audit Delivery Optimization
$DOPVal = if (Test-Path $DOPath) { (Get-ItemProperty -Path $DOPath -Name "DODownloadMode" -ErrorAction SilentlyContinue).DODownloadMode } else { $null }
$DOPColor = if ($DOPVal -eq 2) { "Green" } else { "Red" }
Write-Host " - Delivery Optimization DODownloadMode: $DOPVal (Expected = 2)" -ForegroundColor $DOPColor
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*\*: Section 18.2.2 (Specify intranet Microsoft update service location), Section 18.2.3 (Do not connect to any Windows Update Internet locations) \* \*\*Microsoft Security Baselines\*\*\*: Windows Update client baseline policies. \* \*\*DoD Windows 11 STIG\*\*\*: Delivery Optimization DODownloadMode configuration requirements.

## # Configure User Profile Restrictions for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Paths / Registry Locations\*\*: \* \*\*GPO Paths\*\*: Multiple policies under Administrative Templates and Security Settings. \* \*\*Registry Locations\*\*: Stored inside local HKLM and HKCU security hives under custom settings.

---

## Rationale Enforcing User Profile Restrictions on PAWs prevents information disclosure on locked consoles, blocks advertising or unapproved telemetry/consumer features, and locks down installer paths to guarantee administrative console isolation. This submodule contains individual requirement rules for each User Profile restriction setting configured on Privileged Access Workstations.

---

## Legacy Impact & Compatibility \* \*\*Minimal Impact\*\*: Operational impact is non-existent because PAW consoles do not host standard user sessions or productivity applications. Custom drivers and management utilities must be pre-approved and signed.

---

## ## Enforced User Profile Restrictions on PAWs

The following individual User Profile restriction rules must be configured:

1. [REQ-PAW-115 - User Profile: Toast Notifications Lock Screen Restrictions for PAWs](#)
  2. [REQ-PAW-116 - User Profile: Spotlight and Consumer Features Restrictions for PAWs](#)
  3. [REQ-PAW-117 - User Profile: Windows Copilot Restrictions for PAWs](#)
  4. [REQ-PAW-118 - User Profile: In-Place Sharing Restrictions for PAWs](#)
  5. [REQ-PAW-119 - User Profile: Shell RunAs User Suppression for PAWs](#)
  6. [REQ-PAW-120 - User Profile: Personalization and Privacy Restrictions for PAWs](#)
  7. [REQ-PAW-121 - User Profile: Group Policy Processing Behaviors for PAWs](#)
  8. [REQ-PAW-122 - User Profile: Telemetry and Inventory Collection Restrictions for PAWs](#)
  9. [REQ-PAW-123 - User Profile: Explorer Security and Memory Protections for PAWs](#)
  10. [REQ-PAW-124 - User Profile: Internet Explorer Options and Feeds Restrictions for PAWs](#)
  11. [REQ-PAW-125 - User Profile: Interactive Logon Warning Banners for PAWs](#)
  12. [REQ-PAW-126 - User Profile: Interactive Logon Inactivity Timeout for PAWs](#)
  13. [REQ-PAW-127 - User Profile: Windows Installer Hardening for PAWs](#)
  14. [REQ-PAW-128 - User Profile: Secondary Logon Service Lockdown for PAWs](#)
  15. [REQ-PAW-140 - User Profile: Structured Exception Handling Overwrite Protection \(SEOP\) for PAWs](#)
  16. [REQ-PAW-141 - User Profile: Directory Protection Mode for PAWs](#)
  17. [REQ-PAW-142 - User Profile: Address Space Layout Randomization \(ASLR\) Image Relocation for PAWs](#)
  18. [REQ-PAW-143 - User Profile: Speculative Execution Mitigations \(Spectre/Meltdown\) for PAWs](#)
  19. [REQ-PAW-144 - User Profile: Authenticode Certificate Padding Check for PAWs](#)
  20. [REQ-PAW-145 - User Profile: Command Processor Batch File Locking for PAWs](#)
  21. [REQ-PAW-146 - User Profile: Time-Travel Debugging \(TTD\) Recording Policy for PAWs](#)
  22. [REQ-PAW-147 - User Profile: Trusted Root Store Protected Roots Certificate Restriction for PAWs](#)
  23. [REQ-PAW-148 - User Profile: Disabling Injection of AppInit DLLs for PAWs](#)
  24. [REQ-PAW-149 - User Profile: Preservation of Attachment Zone Information for PAWs](#)
  25. [REQ-PAW-150 - User Profile: Disable Windows Game DVR for PAWs](#)
  26. [REQ-PAW-151 - User Profile: Restrict Windows Ink Workspace on Lock Screen for PAWs](#)
- 

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*: PAW workstation restrictions \* \*\*ANSSI Active Directory Hardening Guide\*\*: Workstation baseline guide



# [REQ-PAW-115] User Profile: Toast Notifications Lock Screen Restrictions for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKCU\Software\Policies\Microsoft\Windows\CurrentVersion\PushNotifications\NoToastApplicationNotificationOnLockScreen` = `1` (DWord)

## Rationale Disables application notifications on the lock screen to prevent third parties from reading sensitive notification banners or MFA verification tokens on locked machines.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `User Configuration \ Administrative Templates \ Start Menu and Taskbar \ Notifications` 2. Configure the policy: \* \*\*Policy\*\*: `Turn off toast notifications on the lock screen` → Set to \*\*Enabled\*\*

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUptoastnotifications.ps1](#)] ([../implementation\\_scripts/Configure-PawUptoastnotifications.ps1](#))

```
Configure-PawUptoastnotifications.ps1
Configure-PawUptoastnotifications.ps1
Write-Host "Applying User Profile restriction: toast-notifications..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CurrentVersion\PushNotifications" "NoToastApplicationNotificationOnLockScreen" "1" "DWord"

Apply to Default User profile for new sessions
$DefaultHivePath = "C:\Users\Default\NTUSER.DAT"
if (Test-Path $DefaultHivePath) {
 reg load HKU\DefaultUser $DefaultHivePath | Out-Null
 $DefaultKey = "Registry::HKU\DefaultUser\Software\Policies\Microsoft\Windows\CurrentVersion\PushNotifications"
 if (-not (Test-Path $DefaultKey)) { New-Item -Path $DefaultKey -Force | Out-Null }
 Set-ItemProperty -Path $DefaultKey -Name "NoToastApplicationNotificationOnLockScreen" -Value "1" -Type DWord -Force
 [GC]::Collect()
 [GC]::WaitForPendingFinalizers()
 reg unload HKU\DefaultUser | Out-Null
}
```

To audit the hardening status: [Download Script: Get-PawUptoastnotificationsStatus.ps1](#)

```
Get-PawUptoastnotificationsStatus.ps1
Get-PawUptoastnotificationsStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CurrentVersion\PushNotifications" "NoToastApplicationNotificationOnLockScreen" "1"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: PAW workstation restrictions \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Workstation baseline guide

# [REQ-PAW-116] User Profile: Spotlight and Consumer Features Restrictions for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKCU\Software\Policies\Microsoft\Windows\CloudContent\DisableThirdPartySuggestions` = `1` (DWord) \*

`HKCU\Software\Policies\Microsoft\Windows\CloudContent\ConfigureWindowsSpotlight` = `2` (DWord) \*

`HKCU\Software\Policies\Microsoft\Windows\CloudContent\DisableSpotlightCollectionOnDesktop` = `1` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\CloudContent\DisableWindowsConsumerFeatures` = `1` (DWord)

---

## Rationale Disables Windows consumer features, Spotlight wallpaper suggestions, and third-party advertising in the shell to limit telemetry and prevent social-engineering application suggestions.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Cloud Content` 2. Configure the policy: \* \*\*Policy\*\*: `Turn off Microsoft consumer experiences` → Set to **Enabled** 3. Navigate to: `User Configuration \ Administrative Templates \ Windows Components \ Cloud Content` 4. Configure the policies: \* \*\*Policy\*\*: `Do not suggest third-party content in Windows spotlight` → Set to **Enabled** \* \*\*Policy\*\*: `Configure Windows spotlight on lock screen` → Set to **Enabled** (and select **Disabled** to turn off recommendations) 5. Deploy custom registry preference for desktop spotlight (HKCU): \* Navigate to: `User Configuration \ Preferences \ Windows Settings \ Registry` \* Create a new **Registry Item**: \* \*\*Action\*\*: Update \* \*\*Hive\*\*: HKEY\_CURRENT\_USER \* \*\*Key Path\*\*: `Software\Policies\Microsoft\Windows\CloudContent` \* \*\*Value name\*\*: `DisableSpotlightCollectionOnDesktop` \* \*\*Value type\*\*: REG\_DWORD \* \*\*Value data\*\*: `1`

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUpspotlightconsumer.ps1](#)]

(../implementation\_scripts/Configure-PawUpspotlightconsumer.ps1)

```
Configure-PawUpspotlightconsumer.ps1
Configure-PawUpspotlightconsumer.ps1
Write-Host "Applying User Profile restriction: spotlight-consumer..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CloudContent" "DisableThirdPartySuggestions" "1" "DWord"
Set-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CloudContent" "ConfigureWindowsSpotlight" "2" "DWord"
Set-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CloudContent" "DisableSpotlightCollectionOnDesktop" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\CloudContent" "DisableWindowsConsumerFeatures" "1" "DWord"

Apply to Default User profile for new sessions
$DefaultHivePath = "C:\Users\Default\NTUSER.DAT"
if (Test-Path $DefaultHivePath) {
 reg load HKU\DefaultUser $DefaultHivePath | Out-Null
 $DefaultKey = "Registry::HKU\DefaultUser\Software\Policies\Microsoft\Windows\CloudContent"
 if (-not (Test-Path $DefaultKey)) { New-Item -Path $DefaultKey -Force | Out-Null }
 Set-ItemProperty -Path $DefaultKey -Name "DisableThirdPartySuggestions" -Value "1" -Type DWord -Force
 $DefaultKey = "Registry::HKU\DefaultUser\Software\Policies\Microsoft\Windows\CloudContent"
 if (-not (Test-Path $DefaultKey)) { New-Item -Path $DefaultKey -Force | Out-Null }
 Set-ItemProperty -Path $DefaultKey -Name "ConfigureWindowsSpotlight" -Value "2" -Type DWord -Force
 $DefaultKey = "Registry::HKU\DefaultUser\Software\Policies\Microsoft\Windows\CloudContent"
 if (-not (Test-Path $DefaultKey)) { New-Item -Path $DefaultKey -Force | Out-Null }
 Set-ItemProperty -Path $DefaultKey -Name "DisableSpotlightCollectionOnDesktop" -Value "1" -Type DWord -Force
 [GC]::Collect()
 [GC]::WaitForPendingFinalizers()
 reg unload HKU\DefaultUser | Out-Null
}
}
```

To audit the hardening status: [Download Script: Get-PawUpspotlightconsumerStatus.ps1](#)

```
Get-PawUpspotlightconsumerStatus.ps1
Get-PawUpspotlightconsumerStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CloudContent" "DisableThirdPartySuggestions" "1"
Test-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CloudContent" "ConfigureWindowsSpotlight" "2"
Test-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CloudContent" "DisableSpotLightCollectionOnDesktop" "1"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\CloudContent" "DisableWindowsConsumerFeatures" "1"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: PAW workstation restrictions \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Workstation baseline guide

# [REQ-PAW-117] User Profile: Windows Copilot Restrictions for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*  
 'HKCU\Software\Policies\Microsoft\Windows\WindowsCopilot\TurnOffWindowsCopilot' = '1' (DWord)

## Rationale Disables the Windows Copilot assistant to prevent unauthorized data exfiltration or automated context parsing of local user activity.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: 'User Configuration \ Administrative Templates \ Windows Components \ Windows Copilot' 2. Configure the policy: \* \*\*Policy\*\*: 'Turn off Windows Copilot' → Set to \*\*Enabled\*\*

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUpwindowscopilot.ps1](#)] ([../implementation\\_scripts/Configure-PawUpwindowscopilot.ps1](#))

```
Configure-PawUpwindowscopilot.ps1
Configure-PawUpwindowscopilot.ps1
Write-Host "Applying User Profile restriction: windows-copilot..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\WindowsCopilot" "TurnOffWindowsCopilot" "1" "DWord"

Apply to Default User profile for new sessions
$DefaultHivePath = "C:\Users\Default\NTUSER.DAT"
if (Test-Path $DefaultHivePath) {
 reg load HKU\DefaultUser $DefaultHivePath | Out-Null
 $DefaultKey = "Registry::HKU\DefaultUser\Software\Policies\Microsoft\Windows\WindowsCopilot"
 if (-not (Test-Path $DefaultKey)) { New-Item -Path $DefaultKey -Force | Out-Null }
 Set-ItemProperty -Path $DefaultKey -Name "TurnOffWindowsCopilot" -Value "1" -Type DWord -Force
 [GC]::Collect()
 [GC]::WaitForPendingFinalizers()
 reg unload HKU\DefaultUser | Out-Null
}
```

To audit the hardening status: [Download Script: Get-PawUpwindowscopilotStatus.ps1](#)

```
Get-PawUpwindowscopilotStatus.ps1
Get-PawUpwindowscopilotStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\WindowsCopilot" "TurnOffWindowsCopilot" "1"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: PAW workstation restrictions \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Workstation baseline guide

# [REQ-PAW-118] User Profile: In-Place Sharing Restrictions for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*  
 `HKCU\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer\NoInplaceSharing` = `1` (DWord)

## Rationale Disables direct in-place shell context sharing actions to prevent rapid local file redistribution bypasses.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `User Configuration \ Administrative Templates \ Windows Components \ Network Sharing` 2. Configure the policy: \* \*\*Policy\*\*: `Prevent users from sharing files within their profile.` → Set to **Enabled**

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUpinplacesharing.ps1](#)] ([../implementation\\_scripts/Configure-PawUpinplacesharing.ps1](#))

```
Configure-PawUpinplacesharing.ps1
Configure-PawUpinplacesharing.ps1
Write-Host "Applying User Profile restriction: inplace-sharing..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKCU:" "Software\Microsoft\Windows\CurrentVersion\Policies\Explorer" "NoInplaceSharing" "1" "DWord"

Apply to Default User profile for new sessions
$DefaultHivePath = "C:\Users\Default\NTUSER.DAT"
if (Test-Path $DefaultHivePath) {
 reg load HKU\DefaultUser $DefaultHivePath | Out-Null
 $DefaultKey = "Registry::HKU\DefaultUser\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer"
 if (-not (Test-Path $DefaultKey)) { New-Item -Path $DefaultKey -Force | Out-Null }
 Set-ItemProperty -Path $DefaultKey -Name "NoInplaceSharing" -Value "1" -Type DWord -Force
 [GC]::Collect()
 [GC]::WaitForPendingFinalizers()
 reg unload HKU\DefaultUser | Out-Null
}
```

To audit the hardening status: [Download Script: Get-PawUpinplacesharingStatus.ps1](#)



```
Get-PawUpinplacesharingStatus.ps1
Get-PawUpinplacesharingStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKCU:" "Software\Microsoft\Windows\CurrentVersion\Policies\Explorer" "NoInplaceSharing" "1"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: PAW workstation restrictions \* \*\*ANSI Active Directory Hardening Guide\*\*\*: Workstation baseline guide

# [REQ-PAW-119] User Profile: Shell RunAs User Suppression for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

```
'HKLM\SOFTWARE\Classes\batfile\shell\runasuser\SuppressionPolicy' = '4096' (DWord) *
'HKLM\SOFTWARE\Classes\cmdfile\shell\runasuser\SuppressionPolicy' = '4096' (DWord) *
'HKLM\SOFTWARE\Classes\exefile\shell\runasuser\SuppressionPolicy' = '4096' (DWord) *
'HKLM\SOFTWARE\Classes\mscfile\shell\runasuser\SuppressionPolicy' = '4096' (DWord)
```

## Rationale Suppresses the 'Run as different user' shell context menu for scripts and executables to prevent users from bypassing local validation structures.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy custom registry preferences under: 'Computer Configuration \ Preferences \ Windows Settings \ Registry' 2. Create four new \*\*Registry Items\*\* for bat, cmd, exe, and msc file RunAs suppression: \* \*\*Registry Item 1\*\*: \* \*\*Key Path\*\*: 'SOFTWARE\Classes\batfile\shell\runasuser' \* \*\*Value name\*\*: 'SuppressionPolicy' \* \*\*Value type\*\*: REG\_DWORD \* \*\*Value data\*\*: '4096' \* \*\*Registry Item 2\*\*: \* \*\*Key Path\*\*: 'SOFTWARE\Classes\cmdfile\shell\runasuser' \* \*\*Value name\*\*: 'SuppressionPolicy' \* \*\*Value type\*\*: REG\_DWORD \* \*\*Value data\*\*: '4096' \* \*\*Registry Item 3\*\*: \* \*\*Key Path\*\*: 'SOFTWARE\Classes\exefile\shell\runasuser' \* \*\*Value name\*\*: 'SuppressionPolicy' \* \*\*Value type\*\*: REG\_DWORD \* \*\*Value data\*\*: '4096' \* \*\*Registry Item 4\*\*: \* \*\*Key Path\*\*: 'SOFTWARE\Classes\mscfile\shell\runasuser' \* \*\*Value name\*\*: 'SuppressionPolicy' \* \*\*Value type\*\*: REG\_DWORD \* \*\*Value data\*\*: '4096'

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUprunassuppression.ps1](#)] (./implementation\_scripts/Configure-PawUprunassuppression.ps1)

```
Configure-PawUprunassuppression.ps1
Configure-PawUprunassuppression.ps1
Write-Host "Applying User Profile restriction: runas-suppression..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Classes\batfile\shell\runasuser" "SuppressionPolicy" "4096" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Classes\cmdfile\shell\runasuser" "SuppressionPolicy" "4096" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Classes\exefile\shell\runasuser" "SuppressionPolicy" "4096" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Classes\mscfile\shell\runasuser" "SuppressionPolicy" "4096" "DWord"
```

To audit the hardening status: [Download Script: Get-PawUprunassuppressionStatus.ps1](#)

```
Get-PawUprunassuppressionStatus.ps1
Get-PawUprunassuppressionStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Classes\batfile\shell\runasuser" "SuppressionPolicy" "4096"
Test-RegValue "HKLM:" "SOFTWARE\Classes\cmdfile\shell\runasuser" "SuppressionPolicy" "4096"
Test-RegValue "HKLM:" "SOFTWARE\Classes\exefile\shell\runasuser" "SuppressionPolicy" "4096"
Test-RegValue "HKLM:" "SOFTWARE\Classes\mscfile\shell\runasuser" "SuppressionPolicy" "4096"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: PAW workstation restrictions \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Workstation baseline guide

# [REQ-PAW-120] User Profile: Personalization and Privacy Restrictions for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Personalization\NoLockScreenCamera` = `1` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Personalization\NoLockScreenSlideshow` = `1` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\AppPrivacy\LetAppsActivateWithVoiceAboveLock` = `2` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\InputPersonalization\AllowInputPersonalization` = `0` (DWord)

## Rationale Disables lock screen cameras, lock screen slideshows, voice activation above lock, and input personalization (speech/typing logs) to preserve local workstation privacy.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Control Panel \ Personalization` 2. Configure the policies: \* \*\*Policy\*\*: `Prevent enabling lock screen camera` → Set to **Enabled** \* \*\*Policy\*\*: `Prevent enabling lock screen slide show` → Set to **Enabled** 3. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ App Privacy` 4. Configure the policy: \* \*\*Policy\*\*: `Let Windows apps activate with voice while the system is locked` → Set to **Enabled** and select **Force Deny** (value 2) 5. Navigate to: `Computer Configuration \ Administrative Templates \ Control Panel \ Regional and Language Options` 6. Configure the policy: \* \*\*Policy\*\*: `Allow users to enable online speech recognition services` → Set to **Disabled** (value 0)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUppersonalizationprivacy.ps1](#)] (./implementation\_scripts/Configure-PawUppersonalizationprivacy.ps1)

```
Configure-PawUppersonalizationprivacy.ps1
Configure-PawUppersonalizationprivacy.ps1
Write-Host "Applying User Profile restriction: personalization-privacy..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Personalization" "NoLockScreenCamera" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Personalization" "NoLockScreenSlideshow" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\AppPrivacy" "LetAppsActivateWithVoiceAboveLock" "2" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\InputPersonalization" "AllowInputPersonalization" "0" "DWord"
```

To audit the hardening status: [Download Script: Get-PawUppersonalizationprivacyStatus.ps1](#)

```
Get-PawUppersonalizationprivacyStatus.ps1
Get-PawUppersonalizationprivacyStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Personalization" "NoLockScreenCamera" "1"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Personalization" "NoLockScreenSlideshow" "1"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\AppPrivacy" "LetAppsActivateWithVoiceAboveLock" "2"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\InputPersonalization" "AllowInputPersonalization" "0"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: PAW workstation restrictions \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Workstation baseline guide

# [REQ-PAW-121] User Profile: Group Policy Processing Behaviors for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}\NoBackgroundPolicy` = `0` (DWord) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}\NoGPListChanges` = `0` (DWord)

## Rationale Enforces that Group Policy background refreshes and policy changes are processed predictably on PAWs without user-directed bypasses.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ System \ Group Policy` 2. Configure the policy: \* \*\*Policy\*\*: `Configure Registry policy processing` → Set to **Enabled** \* Check **Process even if the Group Policy objects have not changed** (enforces NoGPListChanges = 0) \* Uncheck **Do not apply during periodic background processing** (enforces NoBackgroundPolicy = 0)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUpgpprocessing.ps1](#)] ([../implementation\\_scripts/Configure-PawUpgpprocessing.ps1](#))

```
Configure-PawUpgpprocessing.ps1
Configure-PawUpgpprocessing.ps1
Write-Host "Applying User Profile restriction: gp-processing..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" "NoBackgroundPolicy"
"0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" "NoGPListChanges"
"0" "DWord"
```

To audit the hardening status: [Download Script: Get-PawUpgpprocessingStatus.ps1](#)

```
Get-PawUpgpprocessingStatus.ps1
Get-PawUpgpprocessingStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" "NoBackgroundPolicy"
"0"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" "NoGPOListChanges"
"0"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: PAW workstation restrictions \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Workstation baseline guide

# [REQ-PAW-122] User Profile: Telemetry and Inventory Collection Restrictions for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\AppCompat\DisableInventory` = `1` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\DataCollection\AllowTelemetry` = `1` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\DataCollection\LimitEnhancedDiagnosticDataWindowsAnalytics` = `1` (DWord)

## Rationale Disables application compatibility inventory scans and restricts telemetry uploads to the minimum baseline levels.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Application Compatibility` 2. Configure the policy: \* \*\*Policy\*\*: `Turn off Inventory Collector` → Set to **Enabled** 3. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Data Collection and Preview Builds` 4. Configure the policies: \* \*\*Policy\*\*: `Allow Telemetry` → Set to **Enabled** and select **1 - Basic** (or **1 - Required** depending on OS version) \* \*\*Policy\*\*: `Limit Enhanced diagnostic data to the minimum required by Windows Analytics` → Set to **Enabled** and select **1 - Enable Windows Analytics settings**

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUptelemetryinventory.ps1](#)] ([../implementation\\_scripts/Configure-PawUptelemetryinventory.ps1](#))

```
Configure-PawUptelemetryinventory.ps1
Configure-PawUptelemetryinventory.ps1
Write-Host "Applying User Profile restriction: telemetry-inventory..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\AppCompat" "DisableInventory" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\DataCollection" "AllowTelemetry" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\DataCollection" "LimitEnhancedDiagnosticDataWindowsAnalytics" "1" "DWord"
```

To audit the hardening status: [Download Script: Get-PawUptelemetryinventoryStatus.ps1](#)



```
Get-PawUptelemetryinventoryStatus.ps1
Get-PawUptelemetryinventoryStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\AppCompat" "DisableInventory" "1"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\DataCollection" "AllowTelemetry" "1"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\DataCollection" "LimitEnhancedDiagnosticDataWindowsAnalytics" "1"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*: PAW workstation restrictions \* \*\*ANSI Active Directory Hardening Guide\*\*: Workstation baseline guide

# [REQ-PAW-123] User Profile: Explorer Security and Memory Protections for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Explorer\NoDataExecutionPrevention` = `0` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Explorer\NoHeapTerminationOnCorruption` = `0` (DWord) \*

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer\PreXPSP2ShellProtocolBehavior` = `0` (DWord)

## Rationale Enforces Data Execution Prevention (DEP) and Heap Termination on Corruption inside Windows Explorer and disables pre-XP SP2 shell protocol behaviors.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ File Explorer` 2. Configure the policies: \* \*\*Policy\*\*: `Turn off Data Execution Prevention for Explorer` → Set to **Disabled** (keeps DEP active) \* \*\*Policy\*\*: `Turn off heap termination on corruption` → Set to **Disabled** (keeps Heap Termination active) 3. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ File Explorer` (or User configuration equivalent) 4. Configure the policy: \* \*\*Policy\*\*: `Shell Protocol Protected Mode` → Set to **Enabled** (enforces PreXPSP2ShellProtocolBehavior = 0)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUpexplorersecurity.ps1](#)] ([../implementation\\_scripts/Configure-PawUpexplorersecurity.ps1](#))

```
Configure-PawUpexplorersecurity.ps1
Configure-PawUpexplorersecurity.ps1
Write-Host "Applying User Profile restriction: explorer-security..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Explorer" "NoDataExecutionPrevention" "0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Explorer" "NoHeapTerminationOnCorruption" "0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" "PreXPSP2ShellProtocolBehavior" "0" "DWord"
```

To audit the hardening status: [Download Script: Get-PawUpexplorersecurityStatus.ps1](#)

```
Get-PawUpexplorersecurityStatus.ps1
Get-PawUpexplorersecurityStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Explorer" "NoDataExecutionPrevention" "0"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Explorer" "NoHeapTerminationOnCorruption" "0"
Test-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" "PreXPSP2ShellProtocolBehavior" "0"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*: PAW workstation restrictions \* \*\*ANSI Active Directory Hardening Guide\*\*: Workstation baseline guide

# [REQ-PAW-124] User Profile: Internet Explorer Options and Feeds Restrictions for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Policies\Microsoft\Internet Explorer\Main\NotifyDisableIEOptions` = `0` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds\DisableEnclosureDownload` = `1` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds\AllowBasicAuthInClear` = `0` (DWord)

## Rationale Blocks basic cleartext authentication, disables feed enclosure downloads, and disables override options inside standard Internet Options panels.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Internet Explorer` 2. Configure the policy: \* \*\*Policy\*\*: `Disable Internet Explorer 11 as a standalone browser` → Set to **Enabled** and select **Always** 3. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ RSS Feeds` 4. Configure the policies: \* \*\*Policy\*\*: `Prevent downloading of enclosures` → Set to **Enabled** \* \*\*Policy\*\*: `Turn on Basic feed authentication over HTTP` → Set to **Disabled**

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUpiesecurity.ps1](#)] ([../implementation\\_scripts/Configure-PawUpiesecurity.ps1](#))

```
Configure-PawUpiesecurity.ps1
Configure-PawUpiesecurity.ps1
Write-Host "Applying User Profile restriction: ie-security..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Internet Explorer\Main" "NotifyDisableIEOptions" "0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" "DisableEnclosureDownload" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" "AllowBasicAuthInClear" "0" "DWord"
```

To audit the hardening status: [Download Script: Get-PawUpiesecurityStatus.ps1](#)

```
Get-PawUpiesecurityStatus.ps1
Get-PawUpiesecurityStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Internet Explorer\Main" "NotifyDisableIEOptions" "0"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" "DisableEnclosureDownload" "1"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" "AllowBasicAuthInClear" "0"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*: PAW workstation restrictions \* \*\*ANSSI Active Directory Hardening Guide\*\*: Workstation baseline guide

# [REQ-PAW-125] User Profile: Interactive Logon Warning Banners for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\DisableAutomaticRestartSignOn` = `1` (DWord) \*

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\LegalNoticeText` = `You are accessing a U.S. Government (USG) Information System (IS) that is provided for USG-authorized use only. By using this IS, you consent to routine monitoring.` (String) \*

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\LegalNoticeCaption` = `US Department of Defense Warning Statement` (String)

## Rationale Enforces legally binding interactive logon warning text and caption titles, and disables automatic restart sign-ons on client PAWs.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Policies \ Windows Settings \ Security Settings \ Local Policies \ Security Options` 2. Configure the policies: \* \*\*Policy\*\*: `Interactive logon: Message text for users attempting to log on` → Set to: `You are accessing a U.S. Government (USG) Information System (IS) that is provided for USG-authorized use only. By using this IS, you consent to routine monitoring.` \* \*\*Policy\*\*: `Interactive logon: Message title for users attempting to log on` → Set to: `US Department of Defense Warning Statement` 3. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Windows Logon Options` 4. Configure the policy: \* \*\*Policy\*\*: `Sign-in and lock last interactive user automatically after a restart` → Set to `Disabled`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUplogonbanners.ps1](#)] ([./implementation\\_scripts/Configure-PawUplogonbanners.ps1](#))

```
Configure-PawUplogonbanners.ps1
Configure-PawUplogonbanners.ps1
Write-Host "Applying User Profile restriction: logon-banners..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "DisableAutomaticRestartSignOn" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "LegalNoticeText" "You are accessing a U.S. Governm
ent (USG) Information System (IS) that is provided for USG-authorized use only. By using this IS, you consent to routine monitorin
g." "String"
Set-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "LegalNoticeCaption" "US Department of Defense Warn
ing Statement" "String"
```

To audit the hardening status: [Download Script: Get-PawUplogonbannersStatus.ps1](#)

```
Get-PawUplogonbannersStatus.ps1
Get-PawUplogonbannersStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "DisableAutomaticRestartSignOn" "1"
Test-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "LegalNoticeText" "You are accessing a U.S. Govern
ment (USG) Information System (IS) that is provided for USG-authorized use only. By using this IS, you consent to routine monitorin
g."
Test-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "LegalNoticeCaption" "US Department of Defense War
ning Statement"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: PAW workstation restrictions \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Workstation baseline guide

# [REQ-PAW-126] User Profile: Interactive Logon Inactivity Timeout for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*  
 'HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\InactivityTimeoutSecs' = '900' (DWord)

## Rationale Enforces a maximum inactivity limit of 15 minutes (900 seconds) before locking idle user sessions.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: 'Computer Configuration \ Policies \ Windows Settings \ Security Settings \ Local Policies \ Security Options' 2. Configure the policy: \* \*\*Policy\*\*: 'Interactive logon: Machine inactivity limit' → Set to \* \*\*900 seconds\*\* (15 minutes)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUpinactivitytimeout.ps1](#)] ([../implementation\\_scripts/Configure-PawUpinactivitytimeout.ps1](#))

```
Configure-PawUpinactivitytimeout.ps1
Configure-PawUpinactivitytimeout.ps1
Write-Host "Applying User Profile restriction: inactivity-timeout..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "InactivityTimeoutSecs" "900" "DWord"
```

To audit the hardening status: [Download Script: Get-PawUpinactivitytimeoutStatus.ps1](#)



```
Get-PawUpinactivitytimeoutStatus.ps1
Get-PawUpinactivitytimeoutStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "InactivityTimeoutSecs" "900"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: PAW workstation restrictions \* \*\*ANSI Active Directory Hardening Guide\*\*\*: Workstation baseline guide

# [REQ-PAW-127] User Profile: Windows Installer Hardening for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer\EnableUserControl` = `0` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer\AlwaysInstallElevated` = `0` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer\SafeForScripting` = `0` (DWord) \*

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Device Installer\DisableCoInstallers` = `1` (DWord)

## Rationale Enforces Windows Installer lockdowns by disabling AlwaysInstallElevated and blocking driver co-installer executions on PAWs.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Windows Installer` 2. Configure the policies: \* \*\*Policy\*\*: `Allow user control over installs` → Set to **Disabled** \* \*\*Policy\*\*: `Always install with elevated privileges` → Set to **Disabled** \* \*\*Policy\*\*: `Prevent Internet Explorer security prompt for Windows Installer scripts` → Set to **Disabled** 3. Deploy custom registry preference for blocking driver co-installers: \* Navigate to: `Computer Configuration \ Preferences \ Windows Settings \ Registry` \* Create a new **Registry Item**: \* \*\*Action\*\*: Update \* \*\*Hive\*\*: **HKEY\_LOCAL\_MACHINE** \* \*\*Key Path\*\*: **SOFTWARE\Microsoft\Windows\CurrentVersion\Device Installer** \* \*\*Value name\*\*: **DisableCoInstallers** \* \*\*Value type\*\*: **REG\_DWORD** \* \*\*Value data\*\*: **1**

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUpinstallerhardening.ps1](#)] ([../implementation\\_scripts/Configure-PawUpinstallerhardening.ps1](#))

```
Configure-PawUpinstallerhardening.ps1
Configure-PawUpinstallerhardening.ps1
Write-Host "Applying User Profile restriction: installer-hardening..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Installer" "EnableUserControl" "0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Installer" "AlwaysInstallElevated" "0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Installer" "SafeForScripting" "0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Device Installer" "DisableCoInstallers" "1" "DWord"
```

To audit the hardening status: [Download Script: Get-PawUpinstallerhardeningStatus.ps1](#)

```
Get-PawUpinstallerhardeningStatus.ps1
Get-PawUpinstallerhardeningStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Installer" "EnableUserControl" "0"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Installer" "AlwaysInstallElevated" "0"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Installer" "SafeForScripting" "0"
Test-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Device Installer" "DisableCoInstallers" "1"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: PAW workstation restrictions \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Workstation baseline guide

# [REQ-PAW-128] User Profile: Secondary Logon Service Lockdown for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*  
'HKLM\SYSTEM\CurrentControlSet\Services\seclogon\Start' = '4' (DWord)

## Rationale Disables the Secondary Logon service (seclogon) to prevent users from executing processes under alternate credentials via runas without proper validation.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts user customizations on administrative consoles. No operational impact is expected on dedicated PAW consoles.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure the registry values directly using Group Policy Preferences (Registry Items).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawUpseclogonservice.ps1](#)] ([../implementation\\_scripts/Configure-PawUpseclogonservice.ps1](#))

```
Configure-PawUpsecLogonservice.ps1
Configure-PawUpsecLogonservice.ps1
Write-Host "Applying User Profile restriction: seclogon-service..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}
Set-RegValue "HKLM:" "SYSTEM\CurrentControlSet\Services\seclogon" "Start" "4" "DWord"
```

To audit the hardening status: [Download Script: Get-PawUpseclogonserviceStatus.ps1](#)

```
Get-PawUpseclogonserviceStatus.ps1
Get-PawUpseclogonserviceStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SYSTEM\CurrentControlSet\Services\seclogon" "Start" "4"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: PAW workstation restrictions \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Workstation baseline guide

# [REQ-PAW-140] User Profile: Structured Exception Handling Overwrite Protection (SEHOP) for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\kernel\DisableExceptionChainValidation` = `0` (DWord)

## Rationale Structured Exception Handling Overwrite Protection (SEHOP) detects and thwarts exploits targeting structured exception handler corruption, a common stack exploitation technique.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\kernel\DisableExceptionChainValidation` = `0` (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditSehop.ps1](#)] ([../implementation\\_scripts/Configure-PawAuditSehop.ps1](#))

```
Configure-PawAuditSehop.ps1
Write-Host "Enforcing System Mitigation control: sehop..." -ForegroundColor Cyan

Set Registry value: DisableExceptionChainValidation
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\kernel")) { New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\kernel" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\kernel" -Name "DisableExceptionChainValidation" -Value 0 -Type DWord -Force
Write-Host " Enforced DisableExceptionChainValidation = 0" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-PawAuditSehopStatus.ps1](#)

```
Get-PawAuditSehopStatus.ps1
$script:Vulnerable = $false

Audit Registry value: DisableExceptionChainValidation
$RegVal = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\kernel" -Name "DisableExceptionChainValidation" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.DisableExceptionChainValidation -ne 0) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-PAW-141] User Profile: Directory Protection Mode for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: 'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\ProtectionMode' = '1' (DWord)

## Rationale Setting ProtectionMode to 1 restricts access to crucial system folders like System32, enforcing strict ACLs and protecting against write tampering.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: 'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\ProtectionMode' = '1' (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditProtectionmode.ps1](#)] ([../implementation\\_scripts/Configure-PawAuditProtectionmode.ps1](#))

```
Configure-PawAuditProtectionmode.ps1
Write-Host "Enforcing System Mitigation control: protection-mode..." -ForegroundColor Cyan

Set Registry value: ProtectionMode
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager")) { New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager" -Name "ProtectionMode" -Value 1 -Type DWord -Force
Write-Host " Enforced ProtectionMode = 1" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-PawAuditProtectionmodeStatus.ps1](#)

```
Get-PawAuditProtectionmodeStatus.ps1
$script:Vulnerable = $false

Audit Registry value: ProtectionMode
$RegVal = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager" -Name "ProtectionMode" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.ProtectionMode -ne 1) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-PAW-142] User Profile: Address Space Layout Randomization (ASLR) Image Relocation for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: 'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\MoveImages' = '4294967295' (DWord)

## Rationale Enforcing Address Space Layout Randomization (ASLR) image relocation (MoveImages) forces all dynamic modules to relocate, neutralizing hardcoded ROP payload chains.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: 'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\MoveImages' = '4294967295' (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditAslrrelocation.ps1](#)] (../implementation\_scripts/Configure-PawAuditAslrrelocation.ps1)

```
Configure-PawAuditAslrrelocation.ps1
Write-Host "Enforcing System Mitigation control: aslr-relocation..." -ForegroundColor Cyan

Set Registry value: MoveImages
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management")) { New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Name "MoveImages" -Value 4294967295 -Type DWord -Force
Write-Host " Enforced MoveImages = 4294967295" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-PawAuditAslrrelocationStatus.ps1](#)

```
Get-PawAuditAslrrelocationStatus.ps1
$script:Vulnerable = $false

Audit Registry value: MoveImages
$RegVal = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Name "MoveImages" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.MoveImages -ne 4294967295) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions



# [REQ-PAW-143] User Profile: Speculative Execution Mitigations (Spectre/Meltdown) for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry:

'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\FeatureSettingsOverride' = '72' (DWord) \* Registry:

'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\FeatureSettingsOverrideMask' = '3' (DWord)

## Rationale Enforces hardware-backed speculative execution mitigations (Spectre, Meltdown, MDS) to prevent side-channel leaks of sensitive kernel-space data.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: 'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\FeatureSettingsOverride' = '72' (DWord) \* Registry: 'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\FeatureSettingsOverrideMask' = '3' (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditSpeculativemitigations.ps1](#)] (./implementation\_scripts/Configure-PawAuditSpeculativemitigations.ps1)

```
Configure-PawAuditSpeculativemitigations.ps1
Write-Host "Enforcing System Mitigation control: speculative-mitigations..." -ForegroundColor Cyan

Set Registry value: FeatureSettingsOverride
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management")) { New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Name "FeatureSettingsOverride" -Value 72 -Type DWord -Force
Write-Host " Enforced FeatureSettingsOverride = 72" -ForegroundColor Green

Set Registry value: FeatureSettingsOverrideMask
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management")) { New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Name "FeatureSettingsOverrideMask" -Value 3 -Type DWord -Force
Write-Host " Enforced FeatureSettingsOverrideMask = 3" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-PawAuditSpeculativemitigationsStatus.ps1](#)

```
Get-PawAuditSpeculativemitigationsStatus.ps1
$script:Vulnerable = $false

Audit Registry value: FeatureSettingsOverride
$RegVal = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Name "FeatureSettingsOverride" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.FeatureSettingsOverride -ne 72) {
 $script:Vulnerable = $true
}

Audit Registry value: FeatureSettingsOverrideMask
$RegVal = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Name "FeatureSettingsOverrideMask" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.FeatureSettingsOverrideMask -ne 3) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*ANSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-PAW-144] User Profile: Authenticode Signature Certificate Padding Check for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry:

'HKLM\Software\Microsoft\Cryptography\Wintrust\Config\EnableCertPaddingCheck' = '1' (DWord) \* Registry:

'HKLM\Software\Wow6432Node\Microsoft\Cryptography\Wintrust\Config\EnableCertPaddingCheck' = '1' (DWord)

## Rationale Blocks padding execution attacks on Authenticode signed binaries by validating that there is no extra unverified payload appended to the certificate signature block.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: 'HKLM\Software\Microsoft\Cryptography\Wintrust\Config\EnableCertPaddingCheck' = '1' (DWord) \* Registry: 'HKLM\Software\Wow6432Node\Microsoft\Cryptography\Wintrust\Config\EnableCertPaddingCheck' = '1' (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditCertpadding.ps1](#)] (./implementation\_scripts/Configure-PawAuditCertpadding.ps1)

```
Configure-PawAuditCertpadding.ps1
Write-Host "Enforcing System Mitigation control: cert-padding..." -ForegroundColor Cyan

Set Registry value: EnableCertPaddingCheck
if (-not (Test-Path "HKLM:\Software\Microsoft\Cryptography\Wintrust\Config")) { New-Item -Path "HKLM:\Software\Microsoft\Cryptography\Wintrust\Config" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\Software\Microsoft\Cryptography\Wintrust\Config" -Name "EnableCertPaddingCheck" -Value 1 -Type DWord -Force
Write-Host " Enforced EnableCertPaddingCheck = 1" -ForegroundColor Green

Set Registry value: EnableCertPaddingCheck
if (-not (Test-Path "HKLM:\Software\Wow6432Node\Microsoft\Cryptography\Wintrust\Config")) { New-Item -Path "HKLM:\Software\Wow6432Node\Microsoft\Cryptography\Wintrust\Config" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\Software\Wow6432Node\Microsoft\Cryptography\Wintrust\Config" -Name "EnableCertPaddingCheck" -Value 1 -Type DWord -Force
Write-Host " Enforced EnableCertPaddingCheck = 1" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-PawAuditCertpaddingStatus.ps1](#)

```
Get-PawAuditCertpaddingStatus.ps1
$script:Vulnerable = $false

Audit Registry value: EnableCertPaddingCheck
$RegVal = Get-ItemProperty -Path "HKLM:\Software\Microsoft\Cryptography\Wintrust\Config" -Name "EnableCertPaddingCheck" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.EnableCertPaddingCheck -ne 1) {
 $script:Vulnerable = $true
}

Audit Registry value: EnableCertPaddingCheck
$RegVal = Get-ItemProperty -Path "HKLM:\Software\Wow6432Node\Microsoft\Cryptography\Wintrust\Config" -Name "EnableCertPaddingCheck" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.EnableCertPaddingCheck -ne 1) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*ANSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-PAW-145] User Profile: Command Processor Batch File Locking for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: 'HKLM\SOFTWARE\Microsoft\Command Processor\LockBatchFilesWhenInUse' = '1' (DWord)

## Rationale Locking batch scripts when executing prevents attackers or concurrent processes from rewriting script lines on-the-fly, neutralizing dynamic code modification hijacks.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: 'HKLM\SOFTWARE\Microsoft\Command Processor\LockBatchFilesWhenInUse' = '1' (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditLockbatchfiles.ps1](#)] ([../implementation\\_scripts/Configure-PawAuditLockbatchfiles.ps1](#))

```
Configure-PawAuditLockbatchfiles.ps1
Write-Host "Enforcing System Mitigation control: lock-batch-files..." -ForegroundColor Cyan

Set Registry value: LockBatchFilesWhenInUse
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\Command Processor")) { New-Item -Path "HKLM:\SOFTWARE\Microsoft\Command Processor" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Command Processor" -Name "LockBatchFilesWhenInUse" -Value 1 -Type DWord -Force
Write-Host " Enforced LockBatchFilesWhenInUse = 1" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-PawAuditLockbatchfilesStatus.ps1](#)

```
Get-PawAuditLockbatchfilesStatus.ps1
$script:Vulnerable = $false

Audit Registry value: LockBatchFilesWhenInUse
$RegVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Command Processor" -Name "LockBatchFilesWhenInUse" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.LockBatchFilesWhenInUse -ne 1) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-PAW-146] User Profile: Time-Travel Debugging (TTD) Recording Policy for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: 'HKLM\SOFTWARE\Microsoft\TTD\RecordingPolicy' = '2' (DWord)

## Rationale Disables and locks down user-mode Time-Travel Debugging (TTD) traces, preventing local adversaries from capturing memory dumps and private cryptographic structures from administrative processes.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: 'HKLM\SOFTWARE\Microsoft\TTD\RecordingPolicy' = '2' (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditTtdrecording.ps1](#)] ([../implementation\\_scripts/Configure-PawAuditTtdrecording.ps1](#))

```
Configure-PawAuditTtdrecording.ps1
Write-Host "Enforcing System Mitigation control: ttd-recording..." -ForegroundColor Cyan

Set Registry value: RecordingPolicy
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\TTD")) { New-Item -Path "HKLM:\SOFTWARE\Microsoft\TTD" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\TTD" -Name "RecordingPolicy" -Value 2 -Type DWord -Force
Write-Host " Enforced RecordingPolicy = 2" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-PawAuditTtdrecordingStatus.ps1](#)

```
Get-PawAuditTtdrecordingStatus.ps1
$script:Vulnerable = $false

Audit Registry value: RecordingPolicy
$RegVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\TTD" -Name "RecordingPolicy" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.RecordingPolicy -ne 2) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-PAW-147] User Profile: Trusted Root Store Protected Roots Certificate Restriction for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: 'HKLM\SOFTWARE\Policies\Microsoft\SystemCertificates\Root\ProtectedRoots\Flags' = '1' (DWord)

## Rationale Restricts users from installing root certificates into the trusted root store, preventing root CA hijack actions and man-in-the-middle proxy injection attacks.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: 'HKLM\SOFTWARE\Policies\Microsoft\SystemCertificates\Root\ProtectedRoots\Flags' = '1' (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditProtectedroots.ps1](#)] ([../implementation\\_scripts/Configure-PawAuditProtectedroots.ps1](#))

```
Configure-PawAuditProtectedroots.ps1
Write-Host "Enforcing System Mitigation control: protected-roots..." -ForegroundColor Cyan

Set Registry value: Flags
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\SystemCertificates\Root\ProtectedRoots")) { New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\SystemCertificates\Root\ProtectedRoots" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\SystemCertificates\Root\ProtectedRoots" -Name "Flags" -Value 1 -Type DWord -Force
Write-Host " Enforced Flags = 1" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-PawAuditProtectedrootsStatus.ps1](#)

```
Get-PawAuditProtectedrootsStatus.ps1
$script:Vulnerable = $false

Audit Registry value: Flags
$RegVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\SystemCertificates\Root\ProtectedRoots" -Name "Flags" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.Flags -ne 1) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-PAW-148] User Profile: Disabling Injection of AppInit DLLs for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Windows\LoadAppInit\_DLLs` = `0` (DWord)

## Rationale AppInit DLL injection is a legacy mechanism that loads arbitrary user DLLs into every process that links user32.dll. Enforcing a complete ban (value 0) prevents unauthorized injection hooks.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Windows\LoadAppInit\_DLLs` = `0` (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditAppinitDlls.ps1](#)] ([../implementation\\_scripts/Configure-PawAuditAppinitDlls.ps1](#))

```
Configure-PawAuditAppinitDlls.ps1
Write-Host "Enforcing System Mitigation control: appinit-dlls..." -ForegroundColor Cyan

Set Registry value: LoadAppInit_DLLs
if (-not (Test-Path "HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Windows")) { New-Item -Path "HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Windows" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Windows" -Name "LoadAppInit_DLLs" -Value 0 -Type DWord -Force
Write-Host " Enforced LoadAppInit_DLLs = 0" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-PawAuditAppinitDllsStatus.ps1](#)

```
Get-PawAuditAppinitDllsStatus.ps1
$script:Vulnerable = $false

Audit Registry value: LoadAppInit_DLLs
$RegVal = Get-ItemProperty -Path "HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Windows" -Name "LoadAppInit_DLLs" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.LoadAppInit_DLLs -ne 0) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions



# [REQ-PAW-149] User Profile: Preservation of Attachment Zone Information for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry:  
`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Attachments\SaveZoneInformation` = `2` (DWord)

## Rationale Ensures the Attachment Manager preserves the Zone.Identifier alternate data stream (ADS) marking files downloaded from untrusted web zones, enforcing SmartScreen check prompts.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `User Configuration \ Administrative Templates \ Windows Components \ Attachment Manager` 2. Configure the policy: \* \*\*Policy\*\*: `Do not preserve zone information in file attachments` → Set to **Disabled** (which configures `SaveZoneInformation` = `2` to preserve zone information)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditAttachmentzone.ps1](#)] ([../implementation\\_scripts/Configure-PawAuditAttachmentzone.ps1](#))

```
Configure-PawAuditAttachmentzone.ps1
Write-Host "Enforcing System Mitigation control: attachment-zone..." -ForegroundColor Cyan

Set Registry value: SaveZoneInformation
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Attachments")) { New-Item -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Attachments" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Attachments" -Name "SaveZoneInformation" -Value 2 -Type DWord -Force
Write-Host " Enforced SaveZoneInformation = 2" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-PawAuditAttachmentzoneStatus.ps1](#)

```
Get-PawAuditAttachmentzoneStatus.ps1
$script:Vulnerable = $false

Audit Registry value: SaveZoneInformation
$RegVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Attachments" -Name "SaveZoneInformation" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.SaveZoneInformation -ne 2) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-PAW-150] User Profile: Disable Windows Game DVR for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: `HKLM\SOFTWARE\Policies\Microsoft\Windows\GameDVR\AllowGameDVR` = `0` (DWord)

## Rationale Disabling Game DVR blocks background media recording agents, conserving local computing cycles and preventing administrative session leakage via broadcast APIs.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Windows Game Recording and Broadcasting` 2. Configure the policy: \* \*\*Policy\*\*: `Enables or disables Windows Game Recording and Broadcasting` → Set to **Disabled** (which configures `AllowGameDVR` = `0`)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditGamedvr.ps1](#)] ([../implementation\\_scripts/Configure-PawAuditGamedvr.ps1](#))

```
Configure-PawAuditGamedvr.ps1
Write-Host "Enforcing System Mitigation control: game-dvr..." -ForegroundColor Cyan

Set Registry value: AllowGameDVR
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\GameDVR")) { New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\GameDVR" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\GameDVR" -Name "AllowGameDVR" -Value 0 -Type DWord -Force
Write-Host " Enforced AllowGameDVR = 0" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-PawAuditGamedvrStatus.ps1](#)

```
Get-PawAuditGamedvrStatus.ps1
$script:Vulnerable = $false

Audit Registry value: AllowGameDVR
$RegVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\GameDVR" -Name "AllowGameDVR" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.AllowGameDVR -ne 0) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-PAW-151] User Profile: Restrict Windows Ink Workspace on Lock Screen for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: `HKLM\SOFTWARE\Policies\Microsoft\WindowsInkWorkspace\AllowWindowsInkWorkspace` = `1` (DWord)

## Rationale Disabling access to the Windows Ink Workspace on the lock screen prevents unauthorized physical users from invoking drawing tools, scripts, or apps without authenticating.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Windows Ink Workspace` 2. Configure the policy: \* \*\*Policy\*\*: `Allow Windows Ink Workspace` → Set to **Enabled** \* \*\*Action\*\*: Choose **On**, but disallow access above lock\*\* (which configures `AllowWindowsInkWorkspace` = `1`)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditInkworkspace.ps1](#)] ([../implementation\\_scripts/Configure-PawAuditInkworkspace.ps1](#))

```
Configure-PawAuditInkworkspace.ps1
Write-Host "Enforcing System Mitigation control: ink-workspace..." -ForegroundColor Cyan

Set Registry value: AllowWindowsInkWorkspace
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsInkWorkspace")) { New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsInkWorkspace" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsInkWorkspace" -Name "AllowWindowsInkWorkspace" -Value 1 -Type DWord -Force
Write-Host " Enforced AllowWindowsInkWorkspace = 1" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-PawAuditInkworkspaceStatus.ps1](#)

```
Get-PawAuditInkworkspaceStatus.ps1
$script:Vulnerable = $false

Audit Registry value: AllowWindowsInkWorkspace
$RegVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsInkWorkspace" -Name "AllowWindowsInkWorkspace" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.AllowWindowsInkWorkspace -ne 1) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-PAW-025] Configure Exploit Protection Profile for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: \* `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Exploit Guard\Exploit Protection` → \*\*Use a common set of exploit protection settings` \* `Computer Configuration\Administrative Templates\MS Security Guide` → \*\*Enable Certificate Padding\*\* \* `Computer Configuration\Administrative Templates\MS Security Guide` → \*\*Enable Structured Exception Handling Overwrite Protection (SEHOP)\*\* \* \*\*Registry Locations\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender ExploitGuard\Exploit Protection` \* `ExploitProtectionSettings` = `C:\ProgramData\ExploitProtection\ExploitProtectionSettings.xml` (REG\_SZ) \* `HKLM\SOFTWARE\Microsoft\Cryptography\Wintrust\Config` \* `EnableCertPaddingCheck` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Wow6432Node\Microsoft\Cryptography\Wintrust\Config` \* `EnableCertPaddingCheck` = `1` (REG\_DWORD) \* `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\kernel` \* `DisableExceptionChainValidation` = `0` (REG\_DWORD)

---

## Rationale Exploit Protection provides a set of advanced memory and vulnerability mitigations. These mitigations protect both the operating system and applications from memory corruption, buffer overflows, execution redirection, and process hijack attempts.

By enforcing system-wide mitigations:

1. **Data Execution Prevention (DEP)**: Enforces non-executable memory pages, preventing attackers from executing shellcode injected into data-only memory regions (such as the stack or heap).
2. **Address Space Layout Randomization (ASLR)**: Randomizes the locations where system components, executable code, and memory allocations are loaded. Enabling Mandatory ASLR (Force Relocate Images), Bottom-Up ASLR, and High Entropy ASLR makes memory structures unpredictable, thwarting return-oriented programming (ROP) exploits.
3. **Control Flow Guard (CFG)**: Verifies control flow integrity for indirect call targets at compile time, preventing attackers from hijacking indirect jumps to point to arbitrary payloads.
4. **Structured Exception Handler Overwrite Protection (SEHOP)**: Blocks exploits that overwrite Structured Exception Handlers (SEH) to gain control of execution paths during error handling.
5. **Heap Termination on Corruption**: Immediately terminates a process if corruption is detected in its heap. This blocks heap-based buffer overflow exploitation before execution control can be seized.

---

## Legacy Impact & Compatibility \* \*\*Application Crashes\*\*: Strict system-wide mitigations, particularly Mandatory ASLR ('ForceRelocateImages') and DEP, can cause legacy, proprietary, or unsigned applications that are not compiled to support dynamic base relocations to crash upon launching. \* \*\*Orchestration/Agent Conflicts\*\*: Certain older monitoring agents or custom management wrappers that perform memory hook injections may fail when CFG or strict memory mitigations are enforced.

---

## ## Implementation Steps

### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### Step 1: Generate the Reference XML Configuration File Before configuring the GPO, you must create a reference XML file containing the desired Exploit Protection mitigations: 1. On a reference workstation, open the \*\*Windows Security\*\* app. 2. Select \*\*App & browser control\*\* and click \*\*Exploit protection settings\*\*. 3. Under the \*\*System settings\*\* tab, configure the following: \* \*\*Control Flow Guard (CFG)\*\*: On by default \* \*\*Data Execution Prevention (DEP)\*\*: On by default \* \*\*Force randomization for images (Mandatory ASLR)\*\*: On by default \* \*\*Randomize memory allocations (Bottom-up ASLR)\*\*: On by default \* \*\*High-entropy ASLR\*\*: On by default \* \*\*Validate exception chains (SEHOP)\*\*: On by default \* \*\*Validate heap integrity\*\*: On by default 4. Select the \*\*Program settings\*\* tab and configure application-specific overrides for administrative utilities and scripting hosts ('powershell.exe', 'cmd.exe', etc.) to apply EAF, IAF, and ROP mitigations. 5. Scroll to the bottom of the page and click \*\*Export settings\*\*. 6. Save the file as 'ExploitProtectionSettings.xml'. 7. Copy this XML file to a location accessible by target endpoints, or distribute it to local target directories (e.g., 'C:\ProgramData\ExploitProtection\ExploitProtectionSettings.xml') via Group Policy Preferences (Files).

##### Step 2: Configure the Group Policy Setting 1. Open the \*\*Group Policy Management Console\*\* ('gpmc.msc') on a domain controller or management host. 2. Edit the PAW GPO (e.g., 'GPO\_Hardening\_PAW'). 3. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Exploit Guard\Exploit Protection` 4. Configure the following setting: \* \*\*Policy\*\*: 'Use a common set of exploit protection settings' \* \*\*Setting\*\*: 'Enabled' \* \*\*Options\*\*: Under the \*Path\* or \*Url\* field, enter the full path to the XML file (e.g., 'C:\ProgramData\ExploitProtection\ExploitProtectionSettings.xml' or a UNC share path). 5. If utilizing Microsoft Security Guide templates: \* Navigate to: `Computer Configuration\Administrative Templates\MS Security Guide` \* Configure policies: \* \*\*Policy\*\*: 'Enable Certificate Padding' → Set to \*\*Enabled\*\* \* \*\*Policy\*\*: 'Enable Structured Exception Handling Overwrite Protection (SEHOP)' → Set to \*\*Enabled\*\*

---

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the exploit protection profile locally on individual systems or standalone hosts.

[Download Script: Configure-PawExploitProtection.ps1](#)

```
Configure-PawExploitProtection.ps1
Description: Generates the system-wide and application-specific Exploit Protection XML profile, applies it locally, and configures
the policy registry keys on PAWS.

Write-Host "Applying Exploit Protection Profile..." -ForegroundColor Cyan

1. Define the XML content including System and App settings
$xmlContent = @"
<?xml version="1.0" encoding="utf-8">
<MitigationPolicy>
 <SystemConfig>
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 <SEHOP Enable="true" TelemetryOnly="false" />
 <Heap TerminateOnError="true" />
 </SystemConfig>
 <AppConfig Executable="wscript.exe">
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 <Payload EnableExportAddressFilter="true" AuditEnableExportAddressFilter="false" EnableExportAddressFilterPlus="true" AuditEnabl
eExportAddressFilterPlus="false" EnableImportAddressFilter="true" AuditEnableImportAddressFilter="false" EnableRopStackPivot="true"
AuditEnableRopStackPivot="false" EnableRopCallerCheck="true" AuditEnableRopCallerCheck="false" EnableRopSimExec="true" AuditEnableRo
pSimExec="false" />
 <ImageLoad BlockRemoteImageLoads="true" AuditBlockRemoteImageLoads="false" PreferSystem32="true" AuditPreferSystem32="false" />
 </AppConfig>
 <AppConfig Executable="cscript.exe">
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 <Payload EnableExportAddressFilter="true" AuditEnableExportAddressFilter="false" EnableExportAddressFilterPlus="true" AuditEnabl
eExportAddressFilterPlus="false" EnableImportAddressFilter="true" AuditEnableImportAddressFilter="false" EnableRopStackPivot="true"
AuditEnableRopStackPivot="false" EnableRopCallerCheck="true" AuditEnableRopCallerCheck="false" EnableRopSimExec="true" AuditEnableRo
pSimExec="false" />
 <ImageLoad BlockRemoteImageLoads="true" AuditBlockRemoteImageLoads="false" PreferSystem32="true" AuditPreferSystem32="false" />
 </AppConfig>
 <AppConfig Executable="powershell.exe">
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 </AppConfig>
</MitigationPolicy>
"@

2. Create the target directory and write the XML file
$TargetDir = "C:\ProgramData\ExploitProtection"
if (-not (Test-Path $TargetDir)) {
 New-Item -Path $TargetDir -ItemType Directory -Force | Out-Null
}

$xmlPath = "$TargetDir\ExploitProtectionSettings.xml"
Set-Content -Path $xmlPath -Value $xmlContent -Encoding UTF8
Write-Host "Exploit Protection XML profile written to $($xmlPath)" -ForegroundColor Gray

3. Apply the settings locally using the cmdlet
if (Get-Command Set-ProcessMitigation -ErrorAction SilentlyContinue) {
 Set-ProcessMitigation -PolicyFilePath $xmlPath
 Write-Host "[+] Exploit protection system settings applied locally." -ForegroundColor Green
} else {
 Write-Warning "Set-ProcessMitigation cmdlet is not available. Please ensure you are running Windows 10/11 or Windows Server 2016
+."
}

4. Configure policy registry keys to point to the XML file
$RegPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender ExploitGuard\Exploit Protection"
if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name "ExploitProtectionSettings" -Value $xmlPath -Type String -Force
Write-Host "[+] GPO policy registry values configured." -ForegroundColor Green

5. Configure MS Security Guide mitigations: Certificate Padding check and SEHOP registry keys
```

```
$WintrustPath = "HKLM:\SOFTWARE\Microsoft\Cryptography\Wintrust\Config"
if (-not (Test-Path $WintrustPath)) {
 New-Item -Path $WintrustPath -Force | Out-Null
}
Set-ItemProperty -Path $WintrustPath -Name "EnableCertPaddingCheck" -Value 1 -Type DWord -Force

$WintrustWow64Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\Cryptography\Wintrust\Config"
if (-not (Test-Path $WintrustWow64Path)) {
 New-Item -Path $WintrustWow64Path -Force | Out-Null
}
Set-ItemProperty -Path $WintrustWow64Path -Name "EnableCertPaddingCheck" -Value 1 -Type DWord -Force

$SessionKernelPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\kernel"
if (-not (Test-Path $SessionKernelPath)) {
 New-Item -Path $SessionKernelPath -Force | Out-Null
}
Set-ItemProperty -Path $SessionKernelPath -Name "DisableExceptionChainValidation" -Value 0 -Type DWord -Force
Write-Host "[+] Certificate Padding check and SEHOP registry keys applied." -ForegroundColor Green

Write-Host "Exploit Protection Profile application completed successfully." -ForegroundColor Cyan
```

*To verify the settings have been applied:*

[Download Script: Get-PawExploitProtectionStatus.ps1](#)

```
Get-PawExploitProtectionStatus.ps1
Description: Audits system-wide Exploit Protection settings against the recommended security baseline on PAWs.

Write-Host "Auditing system-wide Exploit Protection mitigations..." -ForegroundColor Cyan

$BaselineFailed = $false
$Mitigations = Get-ProcessMitigation -System

Helper function to evaluate and display status
function Test-MitigationSetting {
 param(
 [string]$MitigationName,
 [string]$CurrentValue,
 [string]$ExpectedValue
)
 if ($CurrentValue -eq $ExpectedValue) {
 Write-Host " [PASS] $($MitigationName): $($CurrentValue)" -ForegroundColor Green
 } else {
 Write-Host " [FAIL] $($MitigationName): $($CurrentValue) (Expected: $($ExpectedValue))" -ForegroundColor Red
 $script:BaselineFailed = $true
 }
}

Write-Host "`nSystem-wide Mitigations:" -ForegroundColor Gray

Audit DEP
Test-MitigationSetting -MitigationName "DEP Enable" -CurrentValue $Mitigations.DEP.Enable -ExpectedValue "ON"
Test-MitigationSetting -MitigationName "DEP EmulateAtlThunks" -CurrentValue $Mitigations.DEP.EmulateAtlThunks -ExpectedValue "OFF"

Audit ASLR
Test-MitigationSetting -MitigationName "ASLR ForceRelocateImages" -CurrentValue $Mitigations.ASLR.ForceRelocateImages -ExpectedValue "ON"
Test-MitigationSetting -MitigationName "ASLR BottomUp" -CurrentValue $Mitigations.ASLR.BottomUp -ExpectedValue "ON"
Test-MitigationSetting -MitigationName "ASLR HighEntropy" -CurrentValue $Mitigations.ASLR.HighEntropy -ExpectedValue "ON"

Audit CFG
Test-MitigationSetting -MitigationName "CFG Enable" -CurrentValue $Mitigations.CFG.Enable -ExpectedValue "ON"

Audit SEHOP
Test-MitigationSetting -MitigationName "SEHOP Enable" -CurrentValue $Mitigations.SEHOP.Enable -ExpectedValue "ON"

Audit Heap
Test-MitigationSetting -MitigationName "Heap TerminateOnError" -CurrentValue $Mitigations.Heap.TerminateOnError -ExpectedValue "ON"

Audit Registry Policy and XML Configuration
Write-Host "`nRegistry Policy and XML Configuration:" -ForegroundColor Gray
$RegPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender ExploitGuard\Exploit Protection"
if (Test-Path $RegPath) {
 $SettingsValue = Get-ItemProperty -Path $RegPath -Name "ExploitProtectionSettings" -ErrorAction SilentlyContinue
 if ($SettingsValue -and $SettingsValue.ExploitProtectionSettings -ne "") {
 $XmlPath = $SettingsValue.ExploitProtectionSettings
 Write-Host " [PASS] Exploit Protection Policy registry key is configured." -ForegroundColor Green
 Write-Host " Path: $XmlPath" -ForegroundColor Gray

 if (Test-Path $XmlPath) {
 Write-Host " [PASS] Exploit Protection XML file exists." -ForegroundColor Green
 try {
 [xml]$xml = Get-Content -Path $XmlPath -Raw -ErrorAction Stop

 # Verify SystemConfig block
 if ($xml.MitigationPolicy.SystemConfig) {
 Write-Host " [PASS] XML contains SystemConfig block." -ForegroundColor Green
 } else {
 Write-Host " [FAIL] XML is missing SystemConfig block." -ForegroundColor Red
 $BaselineFailed = $true
 }
 }

 # Verify major AppConfigs
 $ExpectedApps = @("wscript.exe", "cscript.exe", "powershell.exe")
 $ConfiguredApps = $xml.MitigationPolicy.AppConfig | ForEach-Object { $_.Executable }

 foreach ($app in $ExpectedApps) {
 if ($ConfiguredApps -contains $app) {
 Write-Host " [PASS] XML contains application profile for: $app" -ForegroundColor Green
 }
 }
 }
 }
}

```

```

 } else {
 Write-Host " [FAIL] XML is missing application profile for: $app" -ForegroundColor Red
 $BaselineFailed = $true
 }
 }
} catch {
 Write-Host " [FAIL] Failed to parse Exploit Protection XML. Error: $(_.Exception.Message)" -ForegroundColor Red
 $BaselineFailed = $true
}
} else {
 Write-Host " [FAIL] Exploit Protection XML file does not exist at specified path." -ForegroundColor Red
 $BaselineFailed = $true
}
} else {
 Write-Host " [FAIL] Exploit Protection Policy registry key is empty or missing." -ForegroundColor Red
 $BaselineFailed = $true
}
} else {
 Write-Host " [FAIL] Exploit Protection Policy registry path does not exist." -ForegroundColor Red
 $BaselineFailed = $true
}
}

Audit MS Security Guide Registry Settings
Write-Host "`nMS Security Guide Mitigations (Certificate Padding & SEHOP):" -ForegroundColor Gray

function Test-MitigationRegistryValue ($path, $name, $expectedValue) {
 $val = Get-ItemProperty -Path $path -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 $color = "Red"
 if ($actual -eq $expectedValue) {
 $color = "Green"
 } else {
 $script:BaselineFailed = $true
 }
 Write-Host " - Registry Setting: $name | Actual: '$actual' (Expected: '$expectedValue')" -ForegroundColor $color
}

$WintrustPath = "HKLM:\SOFTWARE\Microsoft\Cryptography\Wintrust\Config"
Test-MitigationRegistryValue $WintrustPath "EnableCertPaddingCheck" 1

$WintrustWow64Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\Cryptography\Wintrust\Config"
Test-MitigationRegistryValue $WintrustWow64Path "EnableCertPaddingCheck" 1

$SessionKernelPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\kernel"
Test-MitigationRegistryValue $SessionKernelPath "DisableExceptionChainValidation" 0

Write-Host ""
if ($BaselineFailed) {
 Write-Host "Auditing FAILED: One or more configurations do not match the secure baseline." -ForegroundColor Red
 exit 1
} else {
 Write-Host "Auditing PASSED: All configurations match the secure baseline." -ForegroundColor Green
 exit 0
}
}

```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10/11 Client Benchmark\*\*: Section 18.9.30 (ASR/Exploit Guard Mitigation Policy configurations) \* \*\*CIS Microsoft Windows 10/11 Client Benchmark\*\*: Section 18.4.4 (Enable Certificate Padding), Section 18.4.5 (Enable Structured Exception Handling Overwrite Protection (SEHOP)) \* \*\*Microsoft Security Baselines\*\*: Exploit Protection baseline templates and application configurations \* \*\*ANSI Active Directory Hardening Guide\*\*: Recommendations regarding endpoint protective controls



# [REQ-PAW-026] Restrict Safe Mode Access to Administrators on PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer

Configuration\Preferences\Windows Settings\Registry` \* \*\*Registry Location\*\*:

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System` \* \*\*Value Name\*\*: `SafeModeBlockNonAdmins` \* \*\*Value Type\*\*: `REG\_DWORD` \* \*\*Value Data\*\*: `1`

## Rationale Malicious actors with standard user credentials can potentially bypass local security policies, local endpoint detection and response (EDR) agents, and group policy restrictions by booting the system into Safe Mode. In Safe Mode, many security agents and services do not load, creating an environment where local controls can be circumvented.

By configuring `SafeModeBlockNonAdmins = 1`:

1. **Prevent Credential Bypass**: Standard users are blocked from logging in during Safe Mode, ensuring they cannot exploit the disabled security agents to execute unauthorized programs or extract system information.
2. **Maintenance Integrity**: Safe Mode remains accessible exclusively to system administrators for debugging and recovery, ensuring administrative capability is preserved while mitigating standard user risk.

## Legacy Impact & Compatibility \* \*\*Administrative Operations\*\*: This setting does not affect local or domain administrative accounts, which can still log in normally to perform repairs. \* \*\*Troubleshooting Availability\*\*: If a standard user experiences system issues that require Safe Mode, an administrator must log in to the workstation to troubleshoot. Since interactive logons for standard users are already disabled on PAWs, this is the default administrative stance.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

Because there is no native Administrative Template (ADMX) policy for this setting, it must be deployed using Group Policy Preferences (GPP) to configure the registry directly.

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a domain controller or management host.
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Right-click in the right pane, select **New > Registry Item**, and configure:
  - **Action**: `Update`
  - **Hive**: `HKEY_LOCAL_MACHINE`
  - **Key Path**: `SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System`
  - **Value Name**: `SafeModeBlockNonAdmins`
  - **Value Type**: `REG_DWORD`
  - **Value Data**: `1`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally on standalone systems or during reference image build phases.

[Download Script: Configure-PawDisableSafeModeNonAdmins.ps1](#)

```
Configure-PawDisableSafeModeNonAdmins.ps1
Description: Prevents standard users from logging into the system while in Safe Mode by setting SafeModeBlockNonAdmins to 1 on PAW
s.

$RegPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
$ValueName = "SafeModeBlockNonAdmins"
$ValueData = 1

Write-Host "Applying hardening requirement: Restrict Safe Mode access to administrators..." -ForegroundColor Cyan

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value $ValueData -Type DWord -Force | Out-Null
Write-Host "Hardening applied successfully." -ForegroundColor Green
```

To verify the setting has been applied:

[Download Script: Get-PawSafeModeNonAdminsStatus.ps1](#)

```
Get-PawSafeModeNonAdminsStatus.ps1
Description: Checks the current configuration state of SafeModeBlockNonAdmins registry setting on PAWs.

$RegPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
$ValueName = "SafeModeBlockNonAdmins"

Write-Host "Auditing hardening requirement: Restrict Safe Mode access to administrators..." -ForegroundColor Cyan

if (Test-Path $RegPath) {
 $value = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
 if ($null -ne $value -and $value.$ValueName -eq 1) {
 Write-Host "Audit Result: Compliant. Standard users are blocked from logging in during Safe Mode ($ValueName = 1)." -Foregro
undColor Green
 exit 0
 }
}

Write-Host "Audit Result: Non-Compliant. Standard users are allowed to log in during Safe Mode." -ForegroundColor Red
exit 1
```

---

## Sources & Compliance References \* \*\*Australian Cyber Security Centre (ACSC) / ASD\*\* : Windows Hardening Guidelines (SafeModeBlockNonAdmins configuration). \* \*\*Microsoft Learn\*\* : Windows policies and registry-based mitigations.

# [REQ-PAW-027] Configure Windows Defender Firewall and Block LOLBins for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: \* `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Windows Defender Firewall with Advanced Security` \* \*\*Registry Locations\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\WindowsFirewall\DomainProfile` \* `HKLM\SOFTWARE\Policies\Microsoft\WindowsFirewall\PrivateProfile` \* `HKLM\SOFTWARE\Policies\Microsoft\WindowsFirewall\PublicProfile` \* `HKLM\SYSTEM\CurrentControlSet\Services\SharedAccess\Parameters\FirewallPolicy\FirewallRules`

## Rationale Windows Defender Firewall is the primary host-based security control protecting endpoints from unauthorized incoming network connections and regulating outgoing network behaviors. A secure baseline requires enabling the firewall on all profiles, setting inbound connections to block by default, disabling notification prompts that can be bypassed by users, and implementing detailed auditing/logging to monitor network anomalies.

Additionally:

1. **Outbound LOLBins Blocking**: Malicious actors frequently abuse built-in Windows administrative utilities (known as Living Off the Land Binaries, or LOLBins) to download malicious payloads, exfiltrate sensitive data, and communicate with external command-and-control (C2) servers. Blocking outbound network communication for binaries that have no legitimate business requirement to connect to external networks (such as `mshta.exe`, `certutil.exe`, `bitsadmin.exe`, `regsvr32.exe`, `rundll32.exe`, `cscript.exe`, `wscript.exe`, and `hh.exe`) significantly mitigates these threat vectors.

## Legacy Impact & Compatibility \* \*\*Legitimate Administrative Scripts\*\*: Outbound rules will block administrative scripts, third-party software deployment installers, or internal software update tools that execute using `cscript.exe`, `wscript.exe`, or `mshta.exe` and require access to network resources. Proper scoping or exemptions based on authorized source paths or networks should be tested.

## ## Implementation Steps

### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### 1. Configure Profile States and Logging 1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain controller or management workstation. 2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW`). 3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Windows Defender Firewall with Advanced Security` 4. Right-click **Windows Defender Firewall with Advanced Security** and select **Properties**. 5. Configure the **Domain Profile** tab: \* \*\*Firewall state\*\*: `On (recommended)` \* \*\*Inbound connections\*\*: `Block (default)` \* \*\*Outbound connections\*\*: `Allow (default)` \* Click **Customize...** under **Settings**: \* \*\*Display a notification\*\*: `No` \* \*\*Apply local firewall rules\*\*: `No` \* \*\*Apply local connection security rules\*\*: `No` \* Click **Customize...** under **Logging**: \* \*\*Name\*\*: `%SystemRoot%\System32\logfiles\firewall\domainfw.log` \* \*\*Size limit (KB)\*\*: `16384` \* \*\*Log dropped packets\*\*: `Yes` \* \*\*Log successful connections\*\*: `No` 6. Configure the **Private Profile** tab: \* \*\*Firewall state\*\*: `On (recommended)` \* \*\*Inbound connections\*\*: `Block (default)` \* \*\*Outbound connections\*\*: `Allow (default)` \* Click **Customize...** under **Settings**: \* \*\*Display a notification\*\*: `No` \* \*\*Apply local firewall rules\*\*: `No` \* \*\*Apply local connection security rules\*\*: `No` \* Click **Customize...** under **Logging**: \* \*\*Name\*\*: `%SystemRoot%\System32\logfiles\firewall\privatefw.log` \* \*\*Size limit (KB)\*\*: `16384` \* \*\*Log dropped packets\*\*: `Yes` \* \*\*Log successful connections\*\*: `No` 7. Configure the **Public Profile** tab: \* \*\*Firewall state\*\*: `On (recommended)` \* \*\*Inbound connections\*\*: `Block (default)` \* \*\*Outbound connections\*\*: `Allow (default)` \* Click **Customize...** under **Settings**: \* \*\*Display a notification\*\*: `No` \* \*\*Apply local firewall rules\*\*: `No` \* \*\*Apply local connection security rules\*\*: `No` \* Click **Customize...** under **Logging**: \* \*\*Name\*\*: `%SystemRoot%\System32\logfiles\firewall\publicfw.log` \* \*\*Size limit (KB)\*\*: `16384` \* \*\*Log dropped packets\*\*: `Yes` \* \*\*Log successful connections\*\*: `No`

##### 2. Create Outbound Rules for Known LOLBins 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Windows Defender Firewall with Advanced Security\Outbound Rules` 2. Create a new rule for each target LOLBin binary (e.g., `mshta.exe`, `certutil.exe`, `bitsadmin.exe`, `regsvr32.exe`, `rundll32.exe`, `cscript.exe`, `wscript.exe`, `hh.exe`, `calc.exe`, `notepad.exe`, `conhost.exe`, `RunScriptHelper.exe`): \* Right-click **Outbound Rules** and select **New Rule...** \* \*\*Rule Type\*\*: `Program` \* \*\*Program\*\*: Choose `This program path` and enter the path matching the binary (both x64 and x86 paths if applicable, e.g., `%SystemRoot%\System32\mshta.exe` and `%SystemRoot%\SysWOW64\mshta.exe`). \* \*\*Action\*\*: `Block the connection` \* \*\*Profile\*\*: Select `Domain`, `Private`, and `Public`. \* \*\*Name\*\*: Specify a descriptive name (e.g., `Hardening: Block Outbound mshta.exe (x64)`).

### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to configure Windows Defender Firewall profiles, logging parameters, merge settings, and outbound LOLBins rules.

[Download Script: Configure-PawWindowsFirewall.ps1](#)

```
Configure-PawWindowsFirewall.ps1
Description: Configures Windows Defender Firewall profiles (Domain, Private, Public) and blocks outbound traffic for known LOLBins
on PAWs.

Write-Host "Configuring Windows Defender Firewall profiles..." -ForegroundColor Cyan

1. Configure profiles
$FWProfiles = @("Domain", "Private", "Public")
foreach ($FWProfile in $FWProfiles) {
 $LogFile = "$env:windir\System32\logfiles\firewall\$($FWProfile.ToLower())fw.log"

 Set-NetFirewallProfile -Profile $FWProfile `
 -Enabled True `
 -DefaultInboundAction Block `
 -DefaultOutboundAction Allow `
 -NotifyOnListen False `
 -AllowLocalPolicyMerge False `
 -AllowLocalIPsecPolicyMerge False `
 -LogFileName $LogFile `
 -LogMaxSizeKilobytes 16384 `
 -LogBlocked True `
 -LogAllowed False | Out-Null
 Write-Host "[+] Profile '$FWProfile' configured with logging and defaults." -ForegroundColor Green
}

2. Block outbound traffic for known LOLBins
$LoLbins = @(
 @{ Name = "mshta.exe (x64)"; Path = "%SystemRoot%\System32\mshta.exe" },
 @{ Name = "mshta.exe (x86)"; Path = "%SystemRoot%\SysWOW64\mshta.exe" },
 @{ Name = "certutil.exe (x64)"; Path = "%SystemRoot%\System32\certutil.exe" },
 @{ Name = "certutil.exe (x86)"; Path = "%SystemRoot%\SysWOW64\certutil.exe" },
 @{ Name = "bitsadmin.exe (x64)"; Path = "%SystemRoot%\System32\bitsadmin.exe" },
 @{ Name = "bitsadmin.exe (x86)"; Path = "%SystemRoot%\SysWOW64\bitsadmin.exe" },
 @{ Name = "regsvr32.exe (x64)"; Path = "%SystemRoot%\System32\regsvr32.exe" },
 @{ Name = "regsvr32.exe (x86)"; Path = "%SystemRoot%\SysWOW64\regsvr32.exe" },
 @{ Name = "rundll32.exe (x64)"; Path = "%SystemRoot%\System32\rundll32.exe" },
 @{ Name = "rundll32.exe (x86)"; Path = "%SystemRoot%\SysWOW64\rundll32.exe" },
 @{ Name = "cscript.exe (x64)"; Path = "%SystemRoot%\System32\cscript.exe" },
 @{ Name = "cscript.exe (x86)"; Path = "%SystemRoot%\SysWOW64\cscript.exe" },
 @{ Name = "wscript.exe (x64)"; Path = "%SystemRoot%\System32\wscript.exe" },
 @{ Name = "wscript.exe (x86)"; Path = "%SystemRoot%\SysWOW64\wscript.exe" },
 @{ Name = "hh.exe (x64)"; Path = "%SystemRoot%\hh.exe" },
 @{ Name = "hh.exe (x86)"; Path = "%SystemRoot%\SysWOW64\hh.exe" },
 @{ Name = "calc.exe (x64)"; Path = "%SystemRoot%\System32\calc.exe" },
 @{ Name = "calc.exe (x86)"; Path = "%SystemRoot%\SysWOW64\calc.exe" },
 @{ Name = "notepad.exe (x64)"; Path = "%SystemRoot%\System32\notepad.exe" },
 @{ Name = "notepad.exe (x86)"; Path = "%SystemRoot%\SysWOW64\notepad.exe" },
 @{ Name = "conhost.exe (x64)"; Path = "%SystemRoot%\System32\conhost.exe" },
 @{ Name = "conhost.exe (x86)"; Path = "%SystemRoot%\SysWOW64\conhost.exe" },
 @{ Name = "RunScriptHelper.exe (x64)"; Path = "%SystemRoot%\System32\RunScriptHelper.exe" },
 @{ Name = "RunScriptHelper.exe (x86)"; Path = "%SystemRoot%\SysWOW64\RunScriptHelper.exe" }
)

Write-Host "Configuring outbound firewall block rules for known LOLBins..." -ForegroundColor Cyan

foreach ($Bin in $LoLbins) {
 $DisplayName = "Hardening: Block Outbound $($Bin.Name)"
 $Existing = Get-NetFirewallRule -DisplayName $DisplayName -ErrorAction SilentlyContinue
 if ($null -eq $Existing) {
 New-NetFirewallRule -DisplayName $DisplayName `
 -Name $DisplayName `
 -Direction Outbound `
 -Action Block `
 -Program $Bin.Path `
 -Profile Any `
 -Enabled True | Out-Null
 Write-Host "[+] Outbound block rule created for $($Bin.Name)." -ForegroundColor Green
 } else {
 Set-NetFirewallRule -DisplayName $DisplayName -Action Block -Enabled True | Out-Null
 Write-Host "[~] Outbound block rule for $($Bin.Name) already exists, updated state to Enabled/Block." -ForegroundColor Gray
 }
}
}
```

```
Write-Host "Firewall profiles and LOLBins outbound rules configured successfully." -ForegroundColor Green
```

*To verify the settings have been applied:*

[Download Script: Get-PawWindowsFirewallStatus.ps1](#)

```
Get-PawWindowsFirewallStatus.ps1
Description: Audits Windows Defender Firewall profile configurations and outbound block rules for known LOLBins on PAWs.

Write-Host "--- Auditing Windows Defender Firewall Configuration ---" -ForegroundColor Cyan

$script:Vulnerable = $false

Helper function to audit firewall profiles
function Test-FirewallProfile ($ProfileName, $ExpectMergeLocal, $ExpectMergeIPsec) {
 $FWProfile = Get-NetFirewallProfile -Profile $ProfileName -ErrorAction SilentlyContinue
 if ($null -eq $FWProfile) {
 Write-Host " - Profile '$ProfileName' NOT FOUND" -ForegroundColor Red
 $script:Vulnerable = $true
 return
 }

 $EnabledColor = if ($FWProfile.Enabled -eq $true) { "Green" } else { "Red" }
 $InboundColor = if ($FWProfile.DefaultInboundAction -eq "Block") { "Green" } else { "Red" }
 $OutboundColor = if ($FWProfile.DefaultOutboundAction -eq "Allow") { "Green" } else { "Red" }
 $NotifyColor = if ($FWProfile.NotifyOnListen -eq $false) { "Green" } else { "Red" }

 Write-Host " * Profile: $ProfileName" -ForegroundColor Gray
 Write-Host " - Enabled: $($FWProfile.Enabled) (Expected: True)" -ForegroundColor $EnabledColor
 Write-Host " - DefaultInboundAction: $($FWProfile.DefaultInboundAction) (Expected: Block)" -ForegroundColor $InboundColor
 Write-Host " - DefaultOutboundAction: $($FWProfile.DefaultOutboundAction) (Expected: Allow)" -ForegroundColor $OutboundColor
 Write-Host " - NotifyOnListen: $($FWProfile.NotifyOnListen) (Expected: False)" -ForegroundColor $NotifyColor

 # Check log configurations
 $LogPath = "$env:windir\System32\logfiles\firewall\$($ProfileName.ToLower())fw.log"
 $LogPathColor = if ($FWProfile.LogFileName -eq $LogPath) { "Green" } else { "Red" }
 $LogSizeColor = if ($FWProfile.LogMaxSizeKilobytes -ge 16384) { "Green" } else { "Red" }
 $LogBlockedColor = if ($FWProfile.LogBlocked -eq $true) { "Green" } else { "Red" }
 $LogAllowedColor = if ($FWProfile.LogAllowed -eq $false) { "Green" } else { "Red" }

 Write-Host " - LogFileName: $($FWProfile.LogFileName) (Expected: $LogPath)" -ForegroundColor $LogPathColor
 Write-Host " - LogMaxSizeKilobytes: $($FWProfile.LogMaxSizeKilobytes) (Expected: ≥ 16384)" -ForegroundColor $LogSizeColor
 Write-Host " - LogBlocked: $($FWProfile.LogBlocked) (Expected: True)" -ForegroundColor $LogBlockedColor
 Write-Host " - LogAllowed: $($FWProfile.LogAllowed) (Expected: False)" -ForegroundColor $LogAllowedColor

 if ($FWProfile.Enabled -ne $true -or $FWProfile.DefaultInboundAction -ne "Block" -or $FWProfile.NotifyOnListen -ne $false -or $FWProfile.LogFileName -ne $LogPath -or $FWProfile.LogMaxSizeKilobytes -lt 16384 -or $FWProfile.LogBlocked -ne $true -or $FWProfile.LogAllowed -ne $false) {
 $script:Vulnerable = $true
 }

 if ($null -ne $ExpectMergeLocal) {
 $MergeLocalColor = if ($FWProfile.AllowLocalPolicyMerge -eq $ExpectMergeLocal) { "Green" } else { "Red" }
 Write-Host " - AllowLocalPolicyMerge: $($FWProfile.AllowLocalPolicyMerge) (Expected: $ExpectMergeLocal)" -ForegroundColor $MergeLocalColor
 if ($FWProfile.AllowLocalPolicyMerge -ne $ExpectMergeLocal) { $script:Vulnerable = $true }
 }

 if ($null -ne $ExpectMergeIPsec) {
 $MergeIPsecColor = if ($FWProfile.AllowLocalIPsecPolicyMerge -eq $ExpectMergeIPsec) { "Green" } else { "Red" }
 Write-Host " - AllowLocalIPsecPolicyMerge: $($FWProfile.AllowLocalIPsecPolicyMerge) (Expected: $ExpectMergeIPsec)" -ForegroundColor $MergeIPsecColor
 if ($FWProfile.AllowLocalIPsecPolicyMerge -ne $ExpectMergeIPsec) { $script:Vulnerable = $true }
 }
}

Write-Host "Auditing profiles..." -ForegroundColor Gray
Test-FirewallProfile -ProfileName "Domain" -ExpectMergeLocal $false -ExpectMergeIPsec $false
Test-FirewallProfile -ProfileName "Private" -ExpectMergeLocal $false -ExpectMergeIPsec $false
Test-FirewallProfile -ProfileName "Public" -ExpectMergeLocal $false -ExpectMergeIPsec $false

Audit outbound rules for known LOLBins
$LoLbins = @(
 @{ Name = "mshta.exe (x64)"; Path = "%SystemRoot%\System32\mshta.exe" },
 @{ Name = "mshta.exe (x86)"; Path = "%SystemRoot%\SysWOW64\mshta.exe" },
 @{ Name = "certutil.exe (x64)"; Path = "%SystemRoot%\System32\certutil.exe" },
 @{ Name = "certutil.exe (x86)"; Path = "%SystemRoot%\SysWOW64\certutil.exe" },
 @{ Name = "bitsadmin.exe (x64)"; Path = "%SystemRoot%\System32\bitsadmin.exe" },
 @{ Name = "bitsadmin.exe (x86)"; Path = "%SystemRoot%\SysWOW64\bitsadmin.exe" },
 @{ Name = "regsvr32.exe (x64)"; Path = "%SystemRoot%\System32\regsvr32.exe" },
 @{ Name = "regsvr32.exe (x86)"; Path = "%SystemRoot%\SysWOW64\regsvr32.exe" },

```

```

@{ Name = "rundll32.exe (x64)"; Path = "%SystemRoot%\System32\rundll32.exe" },
@{ Name = "rundll32.exe (x86)"; Path = "%SystemRoot%\SysWOW64\rundll32.exe" },
@{ Name = "cscript.exe (x64)"; Path = "%SystemRoot%\System32\cscript.exe" },
@{ Name = "cscript.exe (x86)"; Path = "%SystemRoot%\SysWOW64\cscript.exe" },
@{ Name = "wscript.exe (x64)"; Path = "%SystemRoot%\System32\wscript.exe" },
@{ Name = "wscript.exe (x86)"; Path = "%SystemRoot%\SysWOW64\wscript.exe" },
@{ Name = "hh.exe (x64)"; Path = "%SystemRoot%\hh.exe" },
@{ Name = "hh.exe (x86)"; Path = "%SystemRoot%\SysWOW64\hh.exe" },
@{ Name = "calc.exe (x64)"; Path = "%SystemRoot%\System32\calc.exe" },
@{ Name = "calc.exe (x86)"; Path = "%SystemRoot%\SysWOW64\calc.exe" },
@{ Name = "notepad.exe (x64)"; Path = "%SystemRoot%\System32\notepad.exe" },
@{ Name = "notepad.exe (x86)"; Path = "%SystemRoot%\SysWOW64\notepad.exe" },
@{ Name = "conhost.exe (x64)"; Path = "%SystemRoot%\System32\conhost.exe" },
@{ Name = "conhost.exe (x86)"; Path = "%SystemRoot%\SysWOW64\conhost.exe" },
@{ Name = "RunScriptHelper.exe (x64)"; Path = "%SystemRoot%\System32\RunScriptHelper.exe" },
@{ Name = "RunScriptHelper.exe (x86)"; Path = "%SystemRoot%\SysWOW64\RunScriptHelper.exe" }
)

Write-Host "Auditing outbound firewall rules for known LOLBins..." -ForegroundColor Gray

foreach ($Bin in $Lolbins) {
 $DisplayName = "Hardening: Block Outbound $($Bin.Name)"
 $Rule = Get-NetFirewallRule -DisplayName $DisplayName -ErrorAction SilentlyContinue
 $Color = "Red"

 if ($null -ne $Rule) {
 $ProgFilter = Get-NetFirewallApplicationFilter -AssociatedNetFirewallRule $Rule -ErrorAction SilentlyContinue
 $ProgPath = "None"
 if ($null -ne $ProgFilter) {
 $ProgPath = $ProgFilter.Program
 }

 if ($Rule.Enabled -eq $true -and $Rule.Direction -eq "Outbound" -and $Rule.Action -eq "Block" -and $ProgPath -eq $Bin.Path)
 {
 $Color = "Green"
 Write-Host " - Firewall Rule: $DisplayName | Enabled: True | Action: Block | Program: $ProgPath (Compliant)" -ForegroundColor $Color
 } else {
 $script:Vulnerable = $true
 Write-Host " - Firewall Rule: $DisplayName | Enabled: $($Rule.Enabled) | Action: $($Rule.Action) | Program: $ProgPath (Non-Compliant)" -ForegroundColor $Color
 }
 } else {
 $script:Vulnerable = $true
 Write-Host " - Firewall Rule: $DisplayName | NOT FOUND (Non-Compliant)" -ForegroundColor $Color
 }
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}

```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*\*: Section 9.1 (Domain Profile), Section 9.2 (Private Profile), Section 9.3 (Public Profile) \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendations on host-based firewalls and protocol filtering on administrative workstations. \* \*\*Microsoft Security Baselines\*\*\*: Windows Defender Firewall recommendations



# Disable Unnecessary System Services for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\Start`

---

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Operating system services that are not required for core administrative tasks or business functionality introduce unnecessary entry points for network exposure, resource utilization, and privilege escalation vulnerabilities. On Tier 0 administrative workstations (PAWs), enforcing the principle of least functionality is even more critical.

---

## Legacy Impact & Compatibility \* \*\*Infra Compatibility\*\*: Administrative functions relying on legacy RPC Locator or SSDP-based discovery of network printers/devices may be affected. \* \*\*WSL Functionality\*\*: Disabling `LxssManager` prevents running Windows Subsystem for Linux (WSL) containers on PAWs. This is the desired security behavior, as PAWs should not host developer Linux kernels or unvetted container instances. \* \*\*Mobile Hotspotting\*\*: Disabling `icssvc` and `SharedAccess` prevents users from creating cellular hot-spotting configurations or sharing their network cards. This is a desired security behavior.

---

## Service Hardening Requirements

The following services must be stopped and disabled:

1. [REQ-PAW-037 - Disable Computer Browser Service for PAWs \(Browser\)](#)
  2. [REQ-PAW-038 - Disable Infrared Monitor Service for PAWs \(irmon\)](#)
  3. [REQ-PAW-039 - Disable Internet Connection Sharing \(ICS\) Service for PAWs \(SharedAccess\)](#)
  4. [REQ-PAW-040 - Disable LxssManager Service for PAWs \(LxssManager\)](#)
  5. [REQ-PAW-041 - Disable Microsoft FTP Service for PAWs \(FTPSVC\)](#)
  6. [REQ-PAW-042 - Disable OpenSSH SSH Server Service for PAWs \(sshd\)](#)
  7. [REQ-PAW-043 - Disable Remote Procedure Call \(RPC\) Locator Service for PAWs \(RpcLocator\)](#)
  8. [REQ-PAW-044 - Disable Routing and Remote Access Service for PAWs \(RemoteAccess\)](#)
  9. [REQ-PAW-045 - Disable Simple TCP/IP Services for PAWs \(simptcp\)](#)
  10. [REQ-PAW-046 - Disable Special Administration Console Helper Service for PAWs \(sacsvr\)](#)
  11. [REQ-PAW-047 - Disable SSDP Discovery Service for PAWs \(SSDPSRV\)](#)
  12. [REQ-PAW-048 - Disable UPnP Device Host Service for PAWs \(upnphost\)](#)
  13. [REQ-PAW-049 - Disable Web Management Service for PAWs \(WMSvc\)](#)
  14. [REQ-PAW-050 - Disable Windows Media Player Network Sharing Service for PAWs \(WMPNetworkSvc\)](#)
  15. [REQ-PAW-051 - Disable Windows Mobile Hotspot Service for PAWs \(icssvc\)](#)
  16. [REQ-PAW-052 - Disable World Wide Web Publishing Service for PAWs \(W3SVC\)](#)
  17. [REQ-PAW-053 - Disable Xbox Accessory Management Service for PAWs \(XboxGipSvc\)](#)
  18. [REQ-PAW-054 - Disable Xbox Live Auth Manager Service for PAWs \(XblAuthManager\)](#)
  19. [REQ-PAW-055 - Disable Xbox Live Game Save Service for PAWs \(XblGameSave\)](#)
  20. [REQ-PAW-056 - Disable Xbox Live Networking Service for PAWs \(XboxNetApiSvc\)](#)
- 

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.3, 5.8, 5.9, 5.11, 5.12, 5.14, 5.25, 5.27, 5.29, 5.31, 5.32, 5.33, 5.34, 5.37, 5.38, 5.43, 5.44 to 5.47 \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive systems. \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Services disable requirements



# [REQ-PAW-037] Disable Computer Browser Service for PAWs (Browser)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\Browser\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Computer Browser (Browser) service directly supports this on Privileged Access Workstations:

1. Maintains list of computers on network; legacy, insecure, and cleartext name discovery.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Computer Browser` ( `Browser` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawBrowser.ps1](#)

```
Configure-DisablePawBrowser.ps1
Description: Disables the unnecessary Computer Browser (Browser) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Computer Browser service on PAW..." -ForegroundColor Cyan

$ServiceName = "Browser"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawBrowserStatus.ps1](#)

```
Get-PawBrowserStatus.ps1
Description: Audits the startup configuration of Computer Browser (Browser) service on the local PAW system.

Write-Host "--- Auditing Computer Browser (Browser) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "Browser"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.3 (Browser) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-PAW-038] Disable Infrared monitor service Service for PAWs (irmon)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\irmon\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Infrared monitor service (irmon) service directly supports this on Privileged Access Workstations:

1. Supports infrared device communications; obsolete and exposes unnecessary communication port.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Infrared monitor service (irmon)`, double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawirmon.ps1](#)

```
Configure-DisablePawirmon.ps1
Description: Disables the unnecessary Infrared monitor service (irmon) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Infrared monitor service service on PAW..." -ForegroundColor Cyan

$ServiceName = "irmon"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawirmonStatus.ps1](#)

```
Get-PawirmonStatus.ps1
Description: Audits the startup configuration of Infrared monitor service (irmon) service on the local PAW system.

Write-Host "--- Auditing Infrared monitor service (irmon) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "irmon"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.8 (irmon) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-PAW-039] Disable Internet Connection Sharing (ICS) Service for PAWs (SharedAccess)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\SharedAccess\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Internet Connection Sharing (ICS) (SharedAccess) service directly supports this on Privileged Access Workstations:

1. Provides NAT, addressing, and name resolution; represents security bridging risk.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Internet Connection Sharing (ICS) (SharedAccess)`, double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawSharedAccess.ps1](#)

```
Configure-DisablePawSharedAccess.ps1
Description: Disables the unnecessary Internet Connection Sharing (ICS) (SharedAccess) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Internet Connection Sharing (ICS) service on PAW..." -ForegroundColor Cyan

$ServiceName = "SharedAccess"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawSharedAccessStatus.ps1](#)

```
Get-PawSharedAccessStatus.ps1
Description: Audits the startup configuration of Internet Connection Sharing (ICS) (SharedAccess) service on the local PAW system.

Write-Host "--- Auditing Internet Connection Sharing (ICS) (SharedAccess) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "SharedAccess"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\${$ServiceName}"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '${$ServiceName}' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '${$ServiceName}' startup type is not Disabled (Start value is ${$Start})." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '${$ServiceName}' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '${$ServiceName}' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.9 (SharedAccess) \* \*\*ANSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-PAW-040] Disable LxssManager Service for PAWs (LxssManager)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\LxssManager\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the LxssManager (LxssManager) service directly supports this on Privileged Access Workstations:

1. Windows Subsystem for Linux (WSL); disables hosting unvetted container instances / developer Linux kernels on sensitive systems.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `LxssManager` ( `LxssManager` ), double-click to define the policy, and select **Disabled**.

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawLxssManager.ps1](#)

```
Configure-DisablePawLxssManager.ps1
Description: Disables the unnecessary LxssManager (LxssManager) service on the local PAW.

Write-Host "Applying hardening requirement: Disable LxssManager service on PAW..." -ForegroundColor Cyan

$ServiceName = "LxssManager"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawLxssManagerStatus.ps1](#)

```
Get-PawLxssManagerStatus.ps1
Description: Audits the startup configuration of LxssManager (LxssManager) service on the local PAW system.

Write-Host "--- Auditing LxssManager (LxssManager) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "LxssManager"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*\*: Section 5.11 (LxssManager) \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*\*: Unnecessary services restrictions



# [REQ-PAW-041] Disable Microsoft FTP Service Service for PAWs (FTPSVC)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\FTPSVC\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Microsoft FTP Service (FTPSVC) service directly supports this on Privileged Access Workstations:

1. FTP service; insecure cleartext file transport protocol that should never run on client hosts.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

#### ## Implementation Steps

##### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Microsoft FTP Service` ( `FTPSVC` ), double-click to define the policy, and select **Disabled**.

##### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawFTPSVC.ps1](#)

```
Configure-DisablePawFTPSVC.ps1
Description: Disables the unnecessary Microsoft FTP Service (FTPSVC) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Microsoft FTP Service service on PAW..." -ForegroundColor Cyan

$ServiceName = "FTPSVC"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawFTPSVCStatus.ps1](#)

```
Get-PawFTPSVCStatus.ps1
Description: Audits the startup configuration of Microsoft FTP Service (FTPSVC) service on the local PAW system.

Write-Host "--- Auditing Microsoft FTP Service (FTPSVC) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "FTPSVC"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\[${ServiceName}]"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '${ServiceName}' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '${ServiceName}' startup type is not Disabled (Start value is ${Start})." -Foregro
undColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '${ServiceName}' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '${ServiceName}' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.12 (FTPSVC) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-PAW-042] Disable OpenSSH SSH Server Service for PAWs (sshd)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\sshd\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the OpenSSH SSH Server (sshd) service directly supports this on Privileged Access Workstations:

1. OpenSSH server daemon; exposes command access ports to incoming connections.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `OpenSSH SSH Server` ( `sshd` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawsshd.ps1](#)

```
Configure-DisablePawsshd.ps1
Description: Disables the unnecessary OpenSSH SSH Server (sshd) service on the local PAW.

Write-Host "Applying hardening requirement: Disable OpenSSH SSH Server service on PAW..." -ForegroundColor Cyan

$ServiceName = "sshd"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawsshdStatus.ps1](#)

```
Get-PawsshdStatus.ps1
Description: Audits the startup configuration of OpenSSH SSH Server (sshd) service on the local PAW system.

Write-Host "--- Auditing OpenSSH SSH Server (sshd) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "sshd"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*\*: Section 5.14 (sshd) \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*\*: Unnecessary services restrictions

# [REQ-PAW-043] Disable Remote Procedure Call (RPC) Locator Service for PAWs (RpcLocator)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\RpcLocator\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Remote Procedure Call (RPC) Locator (RpcLocator) service directly supports this on Privileged Access Workstations:

1. Legacy RPC name locator; obsolete and exposes legacy RPC APIs.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Remote Procedure Call (RPC) Locator` ( `RpcLocator` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawRpcLocator.ps1](#)

```
Configure-DisablePawRpcLocator.ps1
Description: Disables the unnecessary Remote Procedure Call (RPC) Locator (RpcLocator) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Remote Procedure Call (RPC) Locator service on PAW..." -ForegroundColor Cyan

$ServiceName = "RpcLocator"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawRpcLocatorStatus.ps1](#)

```
Get-PawRpcLocatorStatus.ps1
Description: Audits the startup configuration of Remote Procedure Call (RPC) Locator (RpcLocator) service on the local PAW system.

Write-Host "--- Auditing Remote Procedure Call (RPC) Locator (RpcLocator) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "RpcLocator"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.25 (RpcLocator) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-PAW-044] Disable Routing and Remote Access Service for PAWs (RemoteAccess)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\RemoteAccess\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Routing and Remote Access (RemoteAccess) service directly supports this on Privileged Access Workstations:

1. Routing, NAT, and VPN; provides network routing capabilities that could bypass domain firewalls.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Routing and Remote Access` ( `RemoteAccess` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawRemoteAccess.ps1](#)

```
Configure-DisablePawRemoteAccess.ps1
Description: Disables the unnecessary Routing and Remote Access (RemoteAccess) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Routing and Remote Access service on PAW..." -ForegroundColor Cyan

$ServiceName = "RemoteAccess"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawRemoteAccessStatus.ps1](#)

```
Get-PawRemoteAccessStatus.ps1
Description: Audits the startup configuration of Routing and Remote Access (RemoteAccess) service on the local PAW system.

Write-Host "--- Auditing Routing and Remote Access (RemoteAccess) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "RemoteAccess"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.27 (RemoteAccess) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions



# [REQ-PAW-045] Disable Simple TCP/IP Services Service for PAWs (simptcp)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\simptcp\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Simple TCP/IP Services (simptcp) service directly supports this on Privileged Access Workstations:

1. Character Generator, Daytime, Discard, Echo, and Quote of the Day; legacy UDP/TCP test ports vulnerable to amplification DDoS.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

### ## Implementation Steps

#### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Simple TCP/IP Services` ( `simptcp` ), double-click to define the policy, and select **Disabled**.

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawsimptcp.ps1](#)

```
Configure-DisablePawsimptcp.ps1
Description: Disables the unnecessary Simple TCP/IP Services (simptcp) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Simple TCP/IP Services service on PAW..." -ForegroundColor Cyan

$ServiceName = "simptcp"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawsimptcpStatus.ps1](#)

```
Get-PawsimptcpStatus.ps1
Description: Audits the startup configuration of Simple TCP/IP Services (simptcp) service on the local PAW system.

Write-Host "--- Auditing Simple TCP/IP Services (simptcp) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "simptcp"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.29 (simptcp) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-PAW-046] Disable Special Administration Console Helper Service for PAWs (sacsvr)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\sacsvr\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Special Administration Console Helper (sacsvr) service directly supports this on Privileged Access Workstations:

1. Enables out-of-band serial port administration; unnecessary on endpoints.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Special Administration Console Helper` ( `sacsvr` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawsacsvr.ps1](#)

```
Configure-DisablePawsacsvr.ps1
Description: Disables the unnecessary Special Administration Console Helper (sacsvr) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Special Administration Console Helper service on PAW..." -ForegroundColor Cyan

$ServiceName = "sacsvr"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawsacsvrStatus.ps1](#)

```
Get-PawsacsvrStatus.ps1
Description: Audits the startup configuration of Special Administration Console Helper (sacsvr) service on the local PAW system.

Write-Host "--- Auditing Special Administration Console Helper (sacsvr) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "sacsvr"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.31 (sacsvr) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-PAW-047] Disable SSDP Discovery Service for PAWs (SSDPSRV)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\SSDPSRV\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the SSDP Discovery (SSDPSRV) service directly supports this on Privileged Access Workstations:

1. Simple Service Discovery Protocol; listens for broadcast UDP name query advertisements, vulnerable to reflect DDOS and name spoofing.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `SSDP Discovery` ( `SSDPSRV` ), double-click to define the policy, and select **Disabled**.

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawSSDPSRV.ps1](#)

```
Configure-DisablePawSSDPSRV.ps1
Description: Disables the unnecessary SSDP Discovery (SSDPSRV) service on the local PAW.

Write-Host "Applying hardening requirement: Disable SSDP Discovery service on PAW..." -ForegroundColor Cyan

$ServiceName = "SSDPSRV"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawSSDPSRVStatus.ps1](#)

```
Get-PawSSDPSRVStatus.ps1
Description: Audits the startup configuration of SSDP Discovery (SSDPSRV) service on the local PAW system.

Write-Host "--- Auditing SSDP Discovery (SSDPSRV) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "SSDPSRV"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\[${ServiceName}]"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '${ServiceName}' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '${ServiceName}' startup type is not Disabled (Start value is ${Start})." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '${ServiceName}' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '${ServiceName}' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*\*: Section 5.32 (SSDPSRV) \* \*\*ANSI Active Directory Hardening Guide\*\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*\*: Unnecessary services restrictions

# [REQ-PAW-048] Disable UPnP Device Host Service for PAWs (upnphost)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\upnphost\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the UPnP Device Host (upnphost) service directly supports this on Privileged Access Workstations:

1. Universal Plug and Play; allows local hosts to advertise and control UPnP devices, opening network exposure.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `UPnP Device Host` ( `upnphost` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawupnphost.ps1](#)

```
Configure-DisablePawupnphost.ps1
Description: Disables the unnecessary UPnP Device Host (upnphost) service on the local PAW.

Write-Host "Applying hardening requirement: Disable UPnP Device Host service on PAW..." -ForegroundColor Cyan

$ServiceName = "upnphost"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawupnphostStatus.ps1](#)

```
Get-PawupnphostStatus.ps1
Description: Audits the startup configuration of UPnP Device Host (upnphost) service on the local PAW system.

Write-Host "--- Auditing UPnP Device Host (upnphost) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "upnphost"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*\*: Section 5.33 (upnphost) \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*\*: Unnecessary services restrictions



# [REQ-PAW-049] Disable Web Management Service Service for PAWs (WMSvc)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\WMSvc\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Web Management Service (WMSvc) service directly supports this on Privileged Access Workstations:

1. IIS Web Management; unnecessary web console access daemon on endpoint systems.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Web Management Service` ( `WMSvc` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawWMSvc.ps1](#)

```
Configure-DisablePawWMSvc.ps1
Description: Disables the unnecessary Web Management Service (WMSvc) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Web Management Service service on PAW..." -ForegroundColor Cyan

$ServiceName = "WMSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawWMSvcStatus.ps1](#)

```
Get-PawWMSvcStatus.ps1
Description: Audits the startup configuration of Web Management Service (WMSvc) service on the local PAW system.

Write-Host "--- Auditing Web Management Service (WMSvc) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "WMSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.34 (WMSvc) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-PAW-050] Disable Windows Media Player Network Sharing Service Service for PAWs (WMPNetworkSvc)  
 ## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\WMPNetworkSvc\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Windows Media Player Network Sharing Service (WMPNetworkSvc) service directly supports this on Privileged Access Workstations:

1. Shares Windows Media Player libraries over network; unnecessary port listening.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

#### ## Implementation Steps

##### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Windows Media Player Network Sharing Service` ( `WMPNetworkSvc` ), double-click to define the policy, and select **Disabled**.

##### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawWMPNetworkSvc.ps1](#)

```
Configure-DisablePawWMPNetworkSvc.ps1
Description: Disables the unnecessary Windows Media Player Network Sharing Service (WMPNetworkSvc) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Windows Media Player Network Sharing Service service on PAW..." -ForegroundColor Cyan

$ServiceName = "WMPNetworkSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawWMPNetworkSvcStatus.ps1](#)

```
Get-PawWMPNetworkSvcStatus.ps1
Description: Audits the startup configuration of Windows Media Player Network Sharing Service (WMPNetworkSvc) service on the local
PAW system.

Write-Host "--- Auditing Windows Media Player Network Sharing Service (WMPNetworkSvc) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "WMPNetworkSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.37 (WMPNetworkSvc) \* \*\*ANSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-PAW-051] Disable Windows Mobile Hotspot Service Service for PAWs (icssvc)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\icssvc\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Windows Mobile Hotspot Service (icssvc) service directly supports this on Privileged Access Workstations:

1. Allows sharing Wi-Fi/Cellular network connections; exposes bridging hazards.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Windows Mobile Hotspot Service` ( `icssvc` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawicssvc.ps1](#)

```
Configure-DisablePawicssvc.ps1
Description: Disables the unnecessary Windows Mobile Hotspot Service (icssvc) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Windows Mobile Hotspot Service service on PAW..." -ForegroundColor Cyan

$ServiceName = "icssvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawicssvcStatus.ps1](#)

```
Get-PawicssvcStatus.ps1
Description: Audits the startup configuration of Windows Mobile Hotspot Service (icssvc) service on the local PAW system.

Write-Host "--- Auditing Windows Mobile Hotspot Service (icssvc) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "icssvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.38 (icssvc) \* \*\*ANSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-PAW-052] Disable World Wide Web Publishing Service Service for PAWs (W3SVC)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\W3SVC\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the World Wide Web Publishing Service (W3SVC) service directly supports this on Privileged Access Workstations:

1. IIS Web Server; web servers should never run on client workstations.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

#### ## Implementation Steps

##### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `World Wide Web Publishing Service ( W3SVC )`, double-click to define the policy, and select **Disabled**.

##### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawW3SVC.ps1](#)

```
Configure-DisablePawW3SVC.ps1
Description: Disables the unnecessary World Wide Web Publishing Service (W3SVC) service on the local PAW.

Write-Host "Applying hardening requirement: Disable World Wide Web Publishing Service service on PAW..." -ForegroundColor Cyan

$ServiceName = "W3SVC"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawW3SVCStatus.ps1](#)

```
Get-PawW3SVCStatus.ps1
Description: Audits the startup configuration of World Wide Web Publishing Service (W3SVC) service on the local PAW system.

Write-Host "--- Auditing World Wide Web Publishing Service (W3SVC) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "W3SVC"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.43 (W3SVC) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions



# [REQ-PAW-053] Disable Xbox Accessory Management Service Service for PAWs (XboxGipSvc)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\XboxGipSvc\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Xbox Accessory Management Service (XboxGipSvc) service directly supports this on Privileged Access Workstations:

1. Manages Xbox accessories; gaming components irrelevant to corporate environments.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Xbox Accessory Management Service` ( `XboxGipSvc` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawXboxGipSvc.ps1](#)

```
Configure-DisablePawXboxGipSvc.ps1
Description: Disables the unnecessary Xbox Accessory Management Service (XboxGipSvc) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Xbox Accessory Management Service service on PAW..." -ForegroundColor Cyan

$ServiceName = "XboxGipSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawXboxGipSvcStatus.ps1](#)

```
Get-PawXboxGipSvcStatus.ps1
Description: Audits the startup configuration of Xbox Accessory Management Service (XboxGipSvc) service on the local PAW system.

Write-Host "--- Auditing Xbox Accessory Management Service (XboxGipSvc) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "XboxGipSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.44 (XboxGipSvc) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-PAW-054] Disable Xbox Live Auth Manager Service for PAWs (XblAuthManager)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\XblAuthManager\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Xbox Live Auth Manager (XblAuthManager) service directly supports this on Privileged Access Workstations:

1. Xbox Live authentication; gaming components irrelevant to corporate environments.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

#### ## Implementation Steps

##### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Xbox Live Auth Manager` ( `XblAuthManager` ), double-click to define the policy, and select **Disabled**.

##### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawXblAuthManager.ps1](#)

```
Configure-DisablePawXblAuthManager.ps1
Description: Disables the unnecessary Xbox Live Auth Manager (XblAuthManager) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Xbox Live Auth Manager service on PAW..." -ForegroundColor Cyan

$ServiceName = "XblAuthManager"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawXblAuthManagerStatus.ps1](#)

```
Get-PawXblAuthManagerStatus.ps1
Description: Audits the startup configuration of Xbox Live Auth Manager (XblAuthManager) service on the local PAW system.

Write-Host "--- Auditing Xbox Live Auth Manager (XblAuthManager) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "XblAuthManager"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

**## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*:** Section 5.45 (XblAuthManager) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-PAW-055] Disable Xbox Live Game Save Service for PAWs (XblGameSave)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\XblGameSave\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Xbox Live Game Save (XblGameSave) service directly supports this on Privileged Access Workstations:

1. Xbox Live game save synchronization; gaming components irrelevant to corporate environments.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Xbox Live Game Save ( XblGameSave )`, double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawXblGameSave.ps1](#)

```
Configure-DisablePawXblGameSave.ps1
Description: Disables the unnecessary Xbox Live Game Save (XblGameSave) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Xbox Live Game Save service on PAW..." -ForegroundColor Cyan

$ServiceName = "XblGameSave"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawXblGameSaveStatus.ps1](#)

```
Get-PawXblGameSaveStatus.ps1
Description: Audits the startup configuration of Xbox Live Game Save (XblGameSave) service on the local PAW system.

Write-Host "--- Auditing Xbox Live Game Save (XblGameSave) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "XblGameSave"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.46 (XblGameSave) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-PAW-056] Disable Xbox Live Networking Service Service for PAWs (XboxNetApiSvc)

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\XboxNetApiSvc\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Xbox Live Networking Service (XboxNetApiSvc) service directly supports this on Privileged Access Workstations:

1. Xbox Live networking API; gaming components irrelevant to corporate environments.
2. On highly critical Tier 0 PAW systems, any running background service represents potential exploit surface. Restricting local capabilities to the absolute bare minimum is a primary security requirement.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for PAW administrative roles unless specific management software strictly relies on it. \* \*\*Engineering Exception\*\*: In the case of services like LxssManager (WSL), disabling is the expected secure baseline to prevent running unvetted Linux container namespaces on administrative workstations.

#### ## Implementation Steps

##### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Xbox Live Networking Service` ( `XboxNetApiSvc` ), double-click to define the policy, and select **Disabled**.

##### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisablePawXboxNetApiSvc.ps1](#)

```
Configure-DisablePawXboxNetApiSvc.ps1
Description: Disables the unnecessary Xbox Live Networking Service (XboxNetApiSvc) service on the local PAW.

Write-Host "Applying hardening requirement: Disable Xbox Live Networking Service service on PAW..." -ForegroundColor Cyan

$ServiceName = "XboxNetApiSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service on the PAW:

[Download Script: Get-PawXboxNetApiSvcStatus.ps1](#)

```
Get-PawXboxNetApiSvcStatus.ps1
Description: Audits the startup configuration of Xbox Live Networking Service (XboxNetApiSvc) service on the local PAW system.

Write-Host "--- Auditing Xbox Live Networking Service (XboxNetApiSvc) Service on PAW ---" -ForegroundColor Cyan

$ServiceName = "XboxNetApiSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.47 (XboxNetApiSvc) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on limiting active background services on sensitive hosts \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions



**# [REQ-PAW-029] Configure System Administrative Templates for PAWs**

**## Target Scope** \* **Applicable Systems**: Privileged Access Workstations (PAWs) used for Tier 0 directory administration. \* **Operating Systems**: Windows 10 Enterprise (1607+) and Windows 11 Enterprise.

**## Implementation Details** \* **Priority**: Medium \* **GPO Path / Registry Location**: \* **GPO Path**: `Computer Configuration\Policies\Administrative Templates\...` \* **Registry Location**: Multiple locations under `HKLM\SOFTWARE\Policies` and `HKLM\SYSTEM\CurrentControlSet` (see details below)

**## Rationale** Administrative templates govern system-wide capabilities, behaviors, and diagnostic logging. Hardening these configurations reduces the attack surface and mitigates privilege escalation, credential theft, and unauthorized software installation:

1. **Protocols Hardening**: Disabling legacy SMBv1 client/server components prevents exploitation of known protocol flaws. Configuring NetBIOS NodeType to P-Node prevents name resolution fallback issues.
2. **Data Collection & Telemetry**: Disabling Insider builds, telemetry feedback, widgets, Cortana, and OneSettings downloads blocks potential information disclosure paths and aligns with clean enterprise environments.
3. **App and Installer Restrictions**: Preventing non-admin users from installing packaged apps, limiting App Installer protocol handlers ( `ms-appinstaller` ), and disabling experimental installer features mitigates malware installation vectors.
4. **Event Log Sizes**: Increasing maximum log file sizes (Application/Setup/System to 32,768 KB, Security to 196,608 KB) ensures security events are retained long enough for compliance auditing and forensic analysis.
5. **Windows Update Controls**: Restricting update pauses and configuring daily update checks ensure client systems remain persistently patched.

**## Legacy Impact & Compatibility** \* **App Installers**: Disabling `ms-appinstaller` protocol handlers will block users from web-installing applications through the Appx installer interface. \* **IE11 Standalone**: Disabling Internet Explorer 11 blocks the standalone browser, redirecting users to Microsoft Edge.

**## Implementation Steps****### Option A: Group Policy Object (GPO) Configuration (Preferred)**

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the PAW GPO (e.g., `GPO_Hardening_PAW` ).
3. Configure the following policies grouped by their GPO nodes:

**#### Network & TCP/IP Settings** \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Network\Lanman Workstation` \* **Configure SMB v1 client driver**: Set to `Enabled`, select `Disable driver (recommended)` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Network\Lanman Server` \* **Configure SMB v1 server**: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Network\TCPIP Settings\Parameters` \* **NetBT NodeType configuration**: Set to `Enabled`, select `P-node (recommended)`

**#### Legacy Security Options (MSS Settings)** \* Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` \* Configure the following security settings (deploy via GPO Preferences Registry if Custom ADMX templates are not available): \* **MSS: (AutoAdminLogon) Enable Automatic Logon**: Set to `Disabled` \* **MSS: (DisableIPSourceRouting IPv6) IP source routing protection level**: Set to `Enabled: Highest protection, source routing is completely disabled` \* **MSS: (DisableIPSourceRouting) IP source routing protection level**: Set to `Enabled: Highest protection, source routing is completely disabled` \* **MSS: (EnableICMPRedirect) Allow ICMP redirects to override OSPF generated routes**: Set to `Disabled` \* **MSS: (NoNameReleaseOnDemand) Allow the computer to ignore NetBIOS name release requests except from WINS servers**: Set to `Enabled` \* **MSS: (SafeDllSearchMode) Enable Safe DLL search mode**: Set to `Enabled` \* **MSS: (ScreenSaverGracePeriod) The time in seconds before the screen saver grace period expires**: Set to `Enabled: 5 or fewer seconds` \* **MSS: (WarningLevel) Percentage threshold for the security event log at which the system will generate a warning**: Set to `Enabled: 90% or less`

**#### System & Group Policy Settings** \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Device Installation` \* **Prevent device metadata retrieval from the Internet**: Set to `Enabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Group Policy` \* **Configure registry policy processing**: Set to `Enabled` \* **Uncheck: 'Do not apply during periodic background processing'** \* **Check: 'Process even if the Group Policy objects have not changed'** \* **Configure security policy processing**: Set to `Enabled` \* **Uncheck: 'Do not apply during periodic background processing'** \* **Check: 'Process even if the Group Policy objects have not changed'** \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Cross-Device Experiences` \* **Continue experiences on this device**: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Internet Communication Management\Internet Communication settings` \* **Turn off downloading of print drivers over HTTP**: Set to `Enabled` \* **Turn off Internet download for Web publishing and online ordering wizards**: Set to `Enabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Local Security Authority` \* **Allow Custom SSPs and APs to be loaded into LSASS**: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Logon` \* **Block user from showing account details on sign-in**: Set to `Enabled` \* **Do not display network selection UI**: Set to `Enabled` \* **Do not enumerate connected users on domain-joined computers**: Set to `Enabled` \* **Turn off app notifications on the lock screen**: Set to `Enabled` \* **Turn off picture password sign-in**: Set to `Enabled` \* **Turn on convenience PIN sign-in**: Set to `Disabled` \* **Prevent the use of security questions for local accounts**: Set to `Enabled` \* **Configure the transmission of the user's password in the content of MPR notifications sent by winlogon**: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Power Management\Sleep`

Settings` \* \*\*Allow network connectivity during connected-standby (on battery)\*\*: Set to `Disabled` \* \*\*Allow network connectivity during connected-standby (plugged in)\*\*: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Remote Assistance` \* \*\*Configure Offer Remote Assistance\*\*: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Remote Procedure Call` \* \*\*Enable RPC Endpoint Mapper Client Authentication\*\*: Set to `Enabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Windows Time Service\Time Providers` \* \*\*Enable Windows NTP Client\*\*: Set to `Enabled` \* \*\*Enable Windows NTP Server\*\*: Set to `Disabled`

#### Windows Components Settings \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\App Package Deployment` \* \*\*Not allow per-user unsigned packages to install by default (requires explicitly allow per install)\*\*: Set to `Enabled` \* \*\*Prevent non-admin users from installing packaged Windows apps\*\*: Set to `Enabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Biometrics\Facial Features` \* \*\*Configure enhanced anti-spoofing\*\*: Set to `Enabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Cloud Content` \* \*\*Turn off cloud consumer account state content\*\*: Set to `Enabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Connect` \* \*\*Require pin for pairing\*\*: Set to `Enabled` (Select `First Time` or `Always`) \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Credential User Interface` \* \*\*Do not display the password reveal button\*\*: Set to `Enabled` \* \*\*Enumerate administrator accounts on elevation\*\*: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Data Collection and Preview Builds` \* \*\*Disable OneSettings Downloads\*\*: Set to `Enabled` \* \*\*Do not show feedback notifications\*\*: Set to `Enabled` \* \*\*Enable OneSettings Auditing\*\*: Set to `Enabled` \* \*\*Limit Diagnostic Log Collection\*\*: Set to `Enabled` \* \*\*Limit Dump Collection\*\*: Set to `Enabled` \* \*\*Toggle user control over Insider builds\*\*: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\App Installer` \* \*\*Enable App Installer Experimental Features\*\*: Set to `Disabled` \* \*\*Enable App Installer Hash Override\*\*: Set to `Disabled` \* \*\*Enable App Installer Local Archive Malware Scan Override\*\*: Set to `Disabled` \* \*\*Enable App Installer Microsoft Store Source Certificate Validation Bypass\*\*: Set to `Disabled` \* \*\*Enable App Installer ms-appinstaller protocol\*\*: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Event Log Service\Application` \* \*\*Control Event Log behavior when the log file reaches its maximum size\*\*: Set to `Disabled` \* \*\*Specify the maximum log file size (KB)\*\*: Set to `Enabled`, set maximum log size to `32768` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Event Log Service\Security` \* \*\*Control Event Log behavior when the log file reaches its maximum size\*\*: Set to `Disabled` \* \*\*Specify the maximum log file size (KB)\*\*: Set to `Enabled`, set maximum log size to `196608` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Event Log Service\Setup` \* \*\*Control Event Log behavior when the log file reaches its maximum size\*\*: Set to `Disabled` \* \*\*Specify the maximum log file size (KB)\*\*: Set to `Enabled`, set maximum log size to `32768` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Event Log Service\System` \* \*\*Control Event Log behavior when the log file reaches its maximum size\*\*: Set to `Disabled` \* \*\*Specify the maximum log file size (KB)\*\*: Set to `Enabled`, set maximum log size to `32768` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\File Explorer` \* \*\*Do not apply the Mark of the Web tag to files copied from insecure sources\*\*: Set to `Disabled` \* \*\*Turn off shell protocol protected mode\*\*: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Internet Explorer` \* \*\*Disable Internet Explorer 11 as a standalone browser\*\*: Set to `Enabled`, select `Always` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Internet Explorer\Feeds` \* \*\*Prevent downloading of enclosures\*\*: Set to `Enabled` \* \*\*Turn on Basic feed authentication over HTTP\*\*: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Defender Antivirus\Remediation\Behavioral Network Blocks\Brute Force Protection` \* \*\*Configure Remote Encryption Protection Mode\*\*: Set to `Enabled` (Select `Audit` or higher) \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan` \* \*\*Turn off scanning of packed executables\*\*: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Defender Security Center\App and Browser protection` \* \*\*Prevent users from modifying settings\*\*: Set to `Enabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Search` \* \*\*Allow Cortana\*\*: Set to `Disabled` \* \*\*Allow Cortana above lock screen\*\*: Set to `Disabled` \* \*\*Allow indexing of encrypted files\*\*: Set to `Disabled` \* \*\*Allow search and Cortana to use location\*\*: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Store` \* \*\*Turn off Automatic Download and Install of updates\*\*: Set to `Disabled` \* \*\*Turn off the offer to update to the latest version of Windows\*\*: Set to `Enabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Widgets` \* \*\*Allow widgets\*\*: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Logon Options` \* \*\*Sign-in and lock last interactive user automatically after a restart\*\*: Set to `Disabled` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Sandbox` \* \*\*Allow clipboard sharing with Windows Sandbox\*\*: Set to `Disabled` \* \*\*Allow networking in Windows Sandbox\*\*: Set to `Disabled`

#### Windows Update Settings \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Update` (or `Windows Update\Manage end user experience` depending on ADMX version) \* \*\*Remove access to "Pause updates" feature\*\*: Set to `Enabled` \* \*\*Manage preview builds\*\*: Set to `Disabled` \* \*\*Select when Preview Builds and Feature Updates are received\*\*: Set to `Enabled`, set `Defer Feature Updates Period in Days` to `180` (or more) \* \*\*Select when Quality Updates are received\*\*: Set to `Enabled`, set `Defer Quality Updates Period in Days` to `0` \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Update\Manage end user experience` (or standard `Windows Update\AU` depending on ADMX version) \* \*\*Configure Automatic Updates\*\*: Set to `Enabled`, select `Scheduled install day` = `0 - Every day` \* \*\*No auto-restart with logged on users for scheduled automatic updates installations\*\*: Set to `Disabled`

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script locally to configure the administrative templates registry values on PAWs.

[Download Script: Configure-PawSystemAdministrativeTemplates.ps1](#)

```
Configure-PawSystemAdministrativeTemplates.ps1
Description: Configures system and administrative template controls for PAWs.

Write-Host "Applying System Administrative Templates hardening..." -ForegroundColor Cyan

Key Path: HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon")) {
 New-Item -Path "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" -Name "AutoAdminLogon" -Value "0" -Type String
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" -Name "ScreenSaverGracePeriod" -Value 5 -Type D
Word

Key Path: HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\CredUI
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\CredUI")) {
 New-Item -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\CredUI" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\CredUI" -Name "EnumerateAdministrators" -Value 0 -T
ype DWord

Key Path: HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer")) {
 New-Item -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" -Name "NoWebServices" -Value 1 -Type DWor
d
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" -Name "PreXPSP2ShellProtocolBehavior" -Va
lue 0 -Type DWord

Key Path: HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System")) {
 New-Item -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" -Name "EnableMPR" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" -Name "DisableAutomaticRestartSignOn" -Valu
e 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Biometrics\FacialFeatures
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Biometrics\FacialFeatures")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Biometrics\FacialFeatures" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Biometrics\FacialFeatures" -Name "EnhancedAntiSpoofing" -Value 1 -Type DWo
rd

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Dsh
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Dsh")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Dsh" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Dsh" -Name "AllowNewsAndInterests" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" -Name "DisableEnclosureDownload" -Value 1 -Type D
Word

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Internet Explorer\Main
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Internet Explorer\Main")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Internet Explorer\Main" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Internet Explorer\Main" -Name "NotifyDisableIEOptions" -Value 0 -Type DWor
d

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Power\PowerSettings\{f15576e8-98b7-4186-b944-eafa664402d9}
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\{f15576e8-98b7-4186-b944-eafa664402d9}")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\{f15576e8-98b7-4186-b944-eafa664402d9}" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\{f15576e8-98b7-4186-b944-eafa664402d9}" -Name "DCSetting
Index" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\{f15576e8-98b7-4186-b944-eafa664402d9}" -Name "ACSetting
Index" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpClient
```

```

if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpClient")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpClient" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpClient" -Name "Enabled" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpServer
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpServer")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpServer" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpServer" -Name "Enabled" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\App and Browser protection
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\App and Browser protection")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\App and Browser protection" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\App and Browser protection" -Name "DisallowExploitProtectionOverride" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Brute Force Protection
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Brute Force Protection")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Brute Force Protection" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Brute Force Protection" -Name "BruteForceProtectionConfiguredState" -Value 2 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -Name "DisablePackedExeScanning" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Printers
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers" -Name "DisableWebPnPDownload" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Rpc
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Rpc")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Rpc" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Rpc" -Name "EnableAuthEpResolution" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services" -Name "fAllowUnsolicited" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\WindowsStore
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsStore")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsStore" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsStore" -Name "AutoDownload" -Value 4 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsStore" -Name "DisableOSUpgrade" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\AppInstaller
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -Name "EnableExperimentalFeatures" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -Name "EnableHashOverride" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -Name "EnableLocalArchiveMalwareScanOverride" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -Name "EnableBypassCertificatePinningForMicrosoftStore" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -Name "EnableMSAppInstallerProtocol" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Appx
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Appx")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Appx" -Force | Out-Null
}

```



```

}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Appx" -Name "DisablePerUserUnsignedPackagesByDefault" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Appx" -Name "BlockNonAdminUserInstall" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\CloudContent
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CloudContent")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CloudContent" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CloudContent" -Name "DisableConsumerAccountStateContent" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Connect
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Connect")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Connect" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Connect" -Name "RequirePinForPairing" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\CredUI
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CredUI")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CredUI" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CredUI" -Name "DisablePasswordReveal" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\DataCollection
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Name "DisableOneSettingsDownloads" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Name "DoNotShowFeedbackNotifications" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Name "EnableOneSettingsAuditing" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Name "LimitDiagnosticLogCollection" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Name "LimitDumpCollection" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Device Metadata
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Device Metadata")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Device Metadata" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Device Metadata" -Name "PreventDeviceMetadataFromNetwork" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\EventLog\Application
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Application")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Application" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Application" -Name "Retention" -Value "0" -Type String
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Application" -Name "MaxSize" -Value 32768 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\EventLog\Security
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Security")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Security" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Security" -Name "Retention" -Value "0" -Type String
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Security" -Name "MaxSize" -Value 196608 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup" -Name "Retention" -Value "0" -Type String
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup" -Name "MaxSize" -Value 32768 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\EventLog\System
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\System")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\System" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\System" -Name "Retention" -Value "0" -Type String
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\System" -Name "MaxSize" -Value 32768 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Explorer
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Explorer")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Explorer" -Force | Out-Null
}

```

```

Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Explorer" -Name "DisableMotWOnInsecurePathCopy" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBBCFA2}
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBBCFA2}")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBBCFA2}" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBBCFA2}" -Name "NoBackgroundPolicy" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBBCFA2}" -Name "NoGPOListChanges" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}" -Name "NoBackgroundPolicy" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}" -Name "NoGPOListChanges" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\PreviewBuilds
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PreviewBuilds")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PreviewBuilds" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PreviewBuilds" -Name "AllowBuildPreview" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Sandbox
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Sandbox")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Sandbox" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Sandbox" -Name "AllowClipboardRedirection" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Sandbox" -Name "AllowNetworking" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\System
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "EnableCdp" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "AllowCustomSSPsAPs" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "BlockUserFromShowingAccountDetailsOnSignIn" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "DontDisplayNetworkSelectionUI" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "DontEnumerateConnectedUsers" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "DisableLockScreenAppNotifications" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "BlockDomainPicturePassword" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "AllowDomainPINLogon" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "NoLocalPasswordResetQuestions" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Windows Search
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" -Name "AllowCortana" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" -Name "AllowCortanaAboveLock" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" -Name "AllowIndexingEncryptedStoresOrItems" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" -Name "AllowSearchToUseLocation" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Name "SetDisablePauseUXAccess" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Name "ManagePreviewBuildsPolicyValue" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Name "DeferFeatureUpdates" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Name "DeferFeatureUpdatesPeriodInDays" -Value 180 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Name "DeferQualityUpdates" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Name "DeferQualityUpdatesPeriodInDays" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU

```

```

if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU" -Name "NoAutoRebootWithLoggedOnUsers" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU" -Name "ScheduledInstallDay" -Value 0 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Control\Session Manager
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager" -Name "SafeDllSearchMode" -Value 1 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Services\Eventlog\Security
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Services\Eventlog\Security")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Eventlog\Security" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Eventlog\Security" -Name "WarningLevel" -Value 90 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters" -Name "SMB1" -Value 0 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Services\NetBT\Parameters
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Services\NetBT\Parameters")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Services\NetBT\Parameters" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\NetBT\Parameters" -Name "NodeType" -Value 2 -Type DWord
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\NetBT\Parameters" -Name "NoNameReleaseOnDemand" -Value 1 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters" -Name "DisableIPSourceRouting" -Value 2 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" -Name "DisableIPSourceRouting" -Value 2 -Type DWord
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" -Name "EnableICMPRedirect" -Value 0 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Services\mrxsmbl0
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Services\mrxsmbl0")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Services\mrxsmbl0" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\mrxsmbl0" -Name "Start" -Value 4 -Type DWord

Key Path: HKLM\Software\Policies\Microsoft\Internet Explorer\Feeds
if (-not (Test-Path "HKLM:\Software\Policies\Microsoft\Internet Explorer\Feeds")) {
 New-Item -Path "HKLM:\Software\Policies\Microsoft\Internet Explorer\Feeds" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\Software\Policies\Microsoft\Internet Explorer\Feeds" -Name "AllowBasicAuthInClear" -Value 0 -Type DWord

Write-Host "[+] PAW administrative templates configured successfully." -ForegroundColor Green

```

To verify the administrative template configuration on the PAW:

[Download Script: Get-PawSystemAdministrativeTemplatesStatus.ps1](#)



```
Get-PawSystemAdministrativeTemplatesStatus.ps1
Description: Audits 84 system and administrative template controls on the local PAW.

Write-Host "--- Auditing System Administrative Templates Hardening ---" -ForegroundColor Cyan
$script:Vulnerable = $false

function Test-RegValue {
 param(
 [string]$RecNum,
 [string]$Hive,
 [string]$KeyPath,
 [string]$ValueName,
 [object]$ExpectedValue
)
 $FullPath = "$($Hive)\$($KeyPath)"
 if (Test-Path $FullPath) {
 $Prop = Get-ItemProperty -Path $FullPath -Name $ValueName -ErrorAction SilentlyContinue
 if ($null -ne $Prop) {
 $ActualValue = $Prop.$ValueName
 if ($ActualValue -eq $ExpectedValue) {
 Write-Host " [+] $RecNum | $ValueName = $ActualValue (Secure)" -ForegroundColor Green
 } else {
 Write-Host " [!] MISMATCH: $RecNum | Path: $Hive\$KeyPath | Value: $ValueName | Current: $ActualValue (Expected: $ExpectedValue)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
 } else {
 Write-Host " [!] MISSING VALUE: $RecNum | Path: $Hive\$KeyPath | Value: $ValueName (Expected: $ExpectedValue)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
 } else {
 Write-Host " [!] MISSING KEY: $RecNum | Path: $Hive\$KeyPath (Expected: $ValueName = $ExpectedValue)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
}

Test-RegValue -RecNum "18.4.2" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\mrxsb10" -ValueName "Start" -ExpectedValue 4
Test-RegValue -RecNum "18.4.3" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters" -ValueName "SMB1" -ExpectedValue 0
Test-RegValue -RecNum "18.4.7" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\NetBT\Parameters" -ValueName "NodeType" -ExpectedValue 2
Test-RegValue -RecNum "18.5.1" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" -ValueName "AutoAdminLogon" -ExpectedValue "0"
Test-RegValue -RecNum "18.5.2" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters" -ValueName "DisableIPSourceRouting" -ExpectedValue 2
Test-RegValue -RecNum "18.5.3" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" -ValueName "DisableIPSourceRouting" -ExpectedValue 2
Test-RegValue -RecNum "18.5.5" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" -ValueName "EnableICMPRedirect" -ExpectedValue 0
Test-RegValue -RecNum "18.5.7" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\NetBT\Parameters" -ValueName "NoNameReleaseOnDemand" -ExpectedValue 1
Test-RegValue -RecNum "18.5.9" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Control\Session Manager" -ValueName "SafeDllSearchMode" -ExpectedValue 1
Test-RegValue -RecNum "18.5.10" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" -ValueName "ScreenSaveGracePeriod" -ExpectedValue 5
Test-RegValue -RecNum "18.5.13" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\Eventlog\Security" -ValueName "WarningLevel" -ExpectedValue 90
Test-RegValue -RecNum "18.9.7.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Device Metadata" -ValueName "PreventDeviceMetadataFromNetwork" -ExpectedValue 1
Test-RegValue -RecNum "18.9.19.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" -ValueName "NoBackgroundPolicy" -ExpectedValue 0
Test-RegValue -RecNum "18.9.19.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" -ValueName "NoGPListChanges" -ExpectedValue 0
Test-RegValue -RecNum "18.9.19.4" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}" -ValueName "NoBackgroundPolicy" -ExpectedValue 0
Test-RegValue -RecNum "18.9.19.5" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}" -ValueName "NoGPListChanges" -ExpectedValue 0
Test-RegValue -RecNum "18.9.19.6" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "EnableCdp" -ExpectedValue 0
Test-RegValue -RecNum "18.9.20.1.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows NT\Printers" -ValueName "DisableWebPrintDownload" -ExpectedValue 1
Test-RegValue -RecNum "18.9.20.1.6" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" -ValueName
```

```

"NoWebServices" -ExpectedValue 1
Test-RegValue -RecNum "18.9.26.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "AllowCustomSSPsAPs"
-ExpectedValue 0
Test-RegValue -RecNum "18.9.28.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "BlockUserFromShowingAccountDetailsOnSignIn" -ExpectedValue 1
Test-RegValue -RecNum "18.9.28.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "DontDisplayNetworkSelectionUI" -ExpectedValue 1
Test-RegValue -RecNum "18.9.28.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "DontEnumerateConnectedUsers" -ExpectedValue 1
Test-RegValue -RecNum "18.9.28.5" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "DisableLockScreenAppNotifications" -ExpectedValue 1
Test-RegValue -RecNum "18.9.28.6" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "BlockDomainPicturePassword" -ExpectedValue 1
Test-RegValue -RecNum "18.9.28.7" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "AllowDomainPINLogin" -ExpectedValue 0
Test-RegValue -RecNum "18.9.33.6.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Power\PowerSettings\15576e8-98b7-4186-b944-eafa664402d9" -ValueName "DCSettingIndex" -ExpectedValue 0
Test-RegValue -RecNum "18.9.33.6.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Power\PowerSettings\15576e8-98b7-4186-b944-eafa664402d9" -ValueName "ACSettingIndex" -ExpectedValue 0
Test-RegValue -RecNum "18.9.35.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services" -ValueName "fAllowUnsolicited" -ExpectedValue 0
Test-RegValue -RecNum "18.9.36.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows NT\Rpc" -ValueName "EnableAuthEpResolution" -ExpectedValue 1
Test-RegValue -RecNum "18.9.51.1.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpClient" -ValueName "Enabled" -ExpectedValue 1
Test-RegValue -RecNum "18.9.51.1.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpServer" -ValueName "Enabled" -ExpectedValue 0
Test-RegValue -RecNum "18.10.4.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Appx" -ValueName "DisablePerUserUnsigneDPackagesByDefault" -ExpectedValue 1
Test-RegValue -RecNum "18.10.4.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Appx" -ValueName "BlockNonAdminUserInstall" -ExpectedValue 1
Test-RegValue -RecNum "18.10.9.1.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Biometrics\FacialFeatures" -ValueName "EnhancedAntiSpoofing" -ExpectedValue 1
Test-RegValue -RecNum "18.10.13.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\CloudContent" -ValueName "DisableConsumerAccountStateContent" -ExpectedValue 1
Test-RegValue -RecNum "18.10.14.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Connect" -ValueName "RequirePinForPairing" -ExpectedValue 1
Test-RegValue -RecNum "18.10.15.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\CredUI" -ValueName "DisablePasswordReveal" -ExpectedValue 1
Test-RegValue -RecNum "18.10.15.2" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\CredUI" -ValueName "EnumerateAdministrators" -ExpectedValue 0
Test-RegValue -RecNum "18.10.15.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "NoLocalPasswordResetQuestions" -ExpectedValue 1
Test-RegValue -RecNum "18.10.16.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\DataCollection" -ValueName "DisableOneSettingsDownloads" -ExpectedValue 1
Test-RegValue -RecNum "18.10.16.4" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\DataCollection" -ValueName "DoNotShowFeedbackNotifications" -ExpectedValue 1
Test-RegValue -RecNum "18.10.16.5" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\DataCollection" -ValueName "EnableOneSettingsAuditing" -ExpectedValue 1
Test-RegValue -RecNum "18.10.16.6" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\DataCollection" -ValueName "LimitDiagnosticLogCollection" -ExpectedValue 1
Test-RegValue -RecNum "18.10.16.7" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\DataCollection" -ValueName "LimitDumpCollection" -ExpectedValue 1
Test-RegValue -RecNum "18.10.16.8" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\PreviewBuilds" -ValueName "AllowBuildPreview" -ExpectedValue 0
Test-RegValue -RecNum "18.10.18.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -ValueName "EnableExperimentalFeatures" -ExpectedValue 0
Test-RegValue -RecNum "18.10.18.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -ValueName "EnableHashOverride" -ExpectedValue 0
Test-RegValue -RecNum "18.10.18.4" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -ValueName "EnableLocalArchiveMalwareScanOverride" -ExpectedValue 0
Test-RegValue -RecNum "18.10.18.5" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -ValueName "EnableBypassCertificatePinningForMicrosoftStore" -ExpectedValue 0
Test-RegValue -RecNum "18.10.18.6" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -ValueName "EnableMSAppInstallerProtocol" -ExpectedValue 0
Test-RegValue -RecNum "18.10.26.1.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\Application" -ValueName "Retention" -ExpectedValue "0"
Test-RegValue -RecNum "18.10.26.1.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\Application" -ValueName "MaxSize" -ExpectedValue 32768
Test-RegValue -RecNum "18.10.26.2.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\Security" -ValueName "Retention" -ExpectedValue "0"
Test-RegValue -RecNum "18.10.26.2.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\Security" -ValueName "MaxSize" -ExpectedValue 196608
Test-RegValue -RecNum "18.10.26.3.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup" -ValueName "Retention" -ExpectedValue "0"
Test-RegValue -RecNum "18.10.26.3.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup" -ValueName "MaxSize" -ExpectedValue 32768

```

```

Test-RegValue -RecNum "18.10.26.4.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\System" -ValueName "Retention" -ExpectedValue "0"
Test-RegValue -RecNum "18.10.26.4.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\System" -ValueName "MaxSize" -ExpectedValue 32768
Test-RegValue -RecNum "18.10.29.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Explorer" -ValueName "DisableMotWOnInsecurePathCopy" -ExpectedValue 0
Test-RegValue -RecNum "18.10.29.5" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" -ValueName "PreXPSP2ShellProtocolBehavior" -ExpectedValue 0
Test-RegValue -RecNum "18.10.35.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Internet Explorer\Main" -ValueName "NotifyDisableIEOptions" -ExpectedValue 0
Test-RegValue -RecNum "18.10.58.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" -ValueName "DisableEnclosureDownload" -ExpectedValue 1
Test-RegValue -RecNum "18.10.58.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" -ValueName "AllowBasicAuthInClear" -ExpectedValue 0
Test-RegValue -RecNum "18.10.43.11.1.1.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\BehavioralNetwork Blocks\Brute Force Protection" -ValueName "BruteForceProtectionConfiguredState" -ExpectedValue 2
Test-RegValue -RecNum "18.10.43.13.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -ValueName "DisablePackedExeScanning" -ExpectedValue 0
Test-RegValue -RecNum "18.10.59.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Windows Search" -ValueName "AllowCortana" -ExpectedValue 0
Test-RegValue -RecNum "18.10.59.4" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Windows Search" -ValueName "AllowCortanaAboveLock" -ExpectedValue 0
Test-RegValue -RecNum "18.10.59.5" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Windows Search" -ValueName "AllowIndexingEncryptedStoresOrItems" -ExpectedValue 0
Test-RegValue -RecNum "18.10.59.6" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Windows Search" -ValueName "AllowSearchToUseLocation" -ExpectedValue 0
Test-RegValue -RecNum "18.10.66.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\WindowsStore" -ValueName "AutoDownload" -ExpectedValue 4
Test-RegValue -RecNum "18.10.66.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\WindowsStore" -ValueName "DisableOSUpgrade" -ExpectedValue 1
Test-RegValue -RecNum "18.10.72.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Dsh" -ValueName "AllowNewsAndInterests" -ExpectedValue 0
Test-RegValue -RecNum "18.10.82.1" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" -ValueName "EnableMPR" -ExpectedValue 0
Test-RegValue -RecNum "18.10.82.2" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" -ValueName "DisableAutomaticRestartSignOn" -ExpectedValue 1
Test-RegValue -RecNum "18.10.91.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Sandbox" -ValueName "AllowClipboardRedirection" -ExpectedValue 0
Test-RegValue -RecNum "18.10.91.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Sandbox" -ValueName "AllowNetworking" -ExpectedValue 0
Test-RegValue -RecNum "18.10.93.2.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -ValueName "SetDisablePauseUXAccess" -ExpectedValue 1
Test-RegValue -RecNum "18.10.93.4.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -ValueName "ManagePreviewBuildsPolicyValue" -ExpectedValue 1
Test-RegValue -RecNum "18.10.93.4.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -ValueName "DeferFeatureUpdates" -ExpectedValue 1
Test-RegValue -RecNum "18.10.93.4.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -ValueName "DeferQualityUpdates" -ExpectedValue 1
Test-RegValue -RecNum "18.10.93.2.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU" -ValueName "ScheduledInstallDay" -ExpectedValue 0
Test-RegValue -RecNum "18.10.93.1.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU" -ValueName "NoAutoRebootWithLoggedOnUsers" -ExpectedValue 0

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}

```

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10/11 Client Benchmark\*\*: Section 18.2 to 18.10 Administrative template rules. \* \*\*Microsoft Windows Security Baselines\*\*: Computer and updates client configuration parameters. \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Event log, update client, network parameters, and device install templates. \* \*\*ANSI AD Hardening Guide\*\*: Recommendations on disabling legacy protocols (SMBv1, etc.) and enforcing cryptographic checks.

# [REQ-PAW-030] Enable UEFI Secure Boot for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs). \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows 11 Enterprise.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* UEFI Firmware configuration (Hardware/BIOS level) \* HKLM\SYSTEM\CurrentControlSet\Control\SecureBoot\State \* `UEFISecureBootEnabled` = `1` (REG\_DWORD)

---

## Rationale Secure Boot is a security standard developed by members of the PC industry to help ensure that a device boots using only software that is trusted by the Original Equipment Manufacturer (OEM).

When the PC starts, the firmware checks the signature of each piece of boot software, including UEFI firmware drivers (also known as Option ROMs), EFI applications, and the operating system. If the signatures are valid, the PC boots, and the firmware gives control to the operating system.

If Secure Boot is disabled:

1. **Bootkits & Rootkits**: Attackers with physical access or local administrator privileges can replace the system bootloader with a malicious bootloader (bootkit). This bootkit executes before the Windows operating system loads, allowing it to bypass all Windows security controls, disable antivirus software, and run completely undetected.
2. **Virtualization-Based Security**: Advanced Windows defenses (like Credential Guard and Device Guard) depend on hardware-rooted trust. If Secure Boot is disabled, Virtualization-Based Security (VBS) cannot verify platform integrity, rendering these protections ineffective.

---

## Legacy Impact & Compatibility \* \*\*BIOS Mode Conversion\*\*: Systems running in legacy BIOS mode (Compatibility Support Module - CSM) instead of Native UEFI cannot use Secure Boot. Converting these systems requires changing partition styles from MBR to GPT (using tools like `MBR2GPT.exe`) and changing firmware settings; refer to [REQ-PAW-005 - UEFI Firmware Security Hardening](#07-paws-configure-uefi-security-md) for firmware settings. Improper conversion can cause boot failures if not executed correctly. \* \*\*Dual-Boot Systems\*\*: If the workstation dual-boots with unsigned Linux distributions or runs legacy recovery media, the firmware will reject the bootloader, preventing boot.

---

## Implementation Steps

### Option A: Manual UEFI Firmware Configuration (Preferred)

UEFI Secure Boot must be enabled in the hardware firmware menu directly (BIOS settings) during system startup:

1. Turn on or restart the workstation and access the UEFI utility screen by pressing the vendor-specific key during POST (typically Delete, F2, F10, or F12).
2. Navigate to the **Security** or **Secure Boot** section:
  - Ensure **Secure Boot** is set to **Enabled**.
  - Ensure the **Secure Boot Mode** is set to **Deployed** or **User Mode** (not Setup Mode).
3. Save the configuration and restart the workstation.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Since Secure Boot is a hardware firmware configuration, it cannot be turned on from within Windows using registry settings. However, you can programmatically audit the state of Secure Boot to flag non-compliant hardware.

Run the following script to check the status of Secure Boot on the local machine:

[Download Script: Audit-PawSecureBoot.ps1](#)

```
Audit-PawSecureBoot.ps1
Description: Queries UEFI Secure Boot parameters and audits UEFI Secure Boot status.

Write-Host "--- Auditing UEFI Secure Boot ---" -ForegroundColor Cyan

$script:NonCompliant = $false

1. Verify boot environment type
if ($env:firmware_type -eq "UEFI") {
 Write-Host " - Boot Environment Type: UEFI" -ForegroundColor Green
} else {
 Write-Host " - VULNERABLE: System booted in Legacy BIOS mode (CSM enabled) or firmware type is unrecognized." -ForegroundColor Red
 $script:NonCompliant = $true
}

2. Verify Secure Boot status
try {
 # Confirm-SecureBootUEFI returns $true if Secure Boot is active, $false if disabled,
 # and throws an exception if the platform does not support UEFI or Secure Boot.
 $SecureBootState = Confirm-SecureBootUEFI -ErrorAction Stop

 $Color = if ($SecureBootState -eq $true) { "Green" } else { "Red" }
 Write-Host " - Secure Boot Active: $SecureBootState" -ForegroundColor $Color
 if ($SecureBootState -eq $false) { $script:NonCompliant = $true }
} catch [System.PlatformNotSupportedException] {
 Write-Host " - VULNERABLE: UEFI Secure Boot is not supported on this platform (Legacy BIOS mode)." -ForegroundColor Red
 $script:NonCompliant = $true
} catch {
 # If cmdlet throws unauthorized access or not enabled error
 Write-Host " - VULNERABLE: Secure Boot is disabled in firmware or cannot be verified. Error: $($_.Exception.Message)" -ForegroundColor Red
 $script:NonCompliant = $true
}

if ($script:NonCompliant) {
 exit 1
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*\*: Section 18.8 (VBS Prerequisites) \* \*\*ANSSI AD Hardening Guide\*\*\*: Recommendations regarding hardware platform integrity.

# [REQ-PAW-031] Enforce Smart Card Logon for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) (Tier 0 Workstations) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options\Interactive logon: Require smart card` →  
 \*\*Enabled\*\* \* \*\*Registry Location\*\*: `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System` → `ScForceOption` = `1` (REG\_DWORD)

## Rationale Enforcing smart card requirements at the local operating system level on administrative workstations provides vital physical and logical isolation:

1. **Password Logon Interface Block:** Requiring a smart card at logon tells Winlogon to suppress the default username and password fields. This forces the credential provider to only accept certificate-based smart card inserts. An attacker who has somehow acquired a user's password (e.g. via social engineering or physical shoulder surfing) will be unable to log on interactively because the endpoint will not display the password input fields.
2. **Mitigation of Credential Replay Attacks:** Traditional password logons store NTHashes locally in the LSA database or cache, which can be extracted by dumping LSASS memory. By using smart card credentials, the logon process utilizes public-key cryptography (Kerberos PKINIT) where the private key never leaves the secure boundaries of the smart card's hardware security chip.
3. **Session Interruption and Removal Enforcement:** Enforcing smart card logons naturally aligns with the smart card removal behavior requirement. Removing the smart card locks the session, and re-entry is impossible without physically re-inserting the token and inputting the PIN.

## Legacy Impact & Compatibility \* \*\*Hard Block for Non-Smart Card Users\*\*: Administrators without active smart cards or security tokens (like YubiKeys) will be completely blocked from logging on to the PAW. Pre-provisioning and certificate enrollment procedures are strict prerequisites. \* \*\*Safe Mode Exemption\*\*: The local Administrator account is exempt from the smart card requirement only when the system is booted into Safe Mode or using the Recovery Console. In normal operation, local accounts cannot log on interactively if smart cards are enforced. \* \*\*System Component Trust\*\*: Workstations must trust the root and intermediate certification authorities issuing the smart card certificates, requiring active enrollment in domain CA policies.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

To enforce smart card logon on all PAWs via Group Policy:

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a domain controller or administrative host.
2. Edit your dedicated PAW Group Policy Object (e.g., `GPO_Hardening_PAWs` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options`
4. In the details pane, double-click **Interactive logon: Require smart card**.
5. Select **Enabled** and click **OK**.
6. Ensure the GPO is linked to the PAW Organizational Unit (OU).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use these scripts locally on a PAW endpoint to enforce or audit the local smart card requirement.

[Download Script: Set-PAWSmartCardEnforcement.ps1](#)

```
Set-PAWSmartCardEnforcement.ps1
Description: Configures the registry to require smart cards for interactive logons on PAWs.
Target Engine: Windows PowerShell 5.1

Write-Host "Enforcing smart card interactive logon requirement..." -ForegroundColor Cyan

$RegPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
$ValueName = "ScForceOption"
$ValueData = 1

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value $ValueData -Type DWord -Force
Write-Host "Smart card interactive logon requirement applied successfully." -ForegroundColor Green
```



To audit the local smart card requirement on PAWs:

[Download Script: Test-PAWSmartCardEnforcement.ps1](#)

```
Test-PAWSmartCardEnforcement.ps1
Description: Audits if the registry is configured to require smart cards for interactive logons on PAWs.
Target Engine: Windows PowerShell 5.1

Write-Host "--- Auditing PAW Smart Card Interactive Logon Requirement ---" -ForegroundColor Cyan

$RegPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
$ValueName = "ScForceOption"
$ExpectedValue = 1

$Vulnerable = $false

if (Test-Path $RegPath) {
 $Property = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
 if ($null -ne $Property -and $Property.$ValueName -eq $ExpectedValue) {
 Write-Host " - Registry Setting: $ValueName | Actual: $($Property.$ValueName) (Expected: $ExpectedValue)" -ForegroundColor Green
 } else {
 $ActualVal = if ($null -ne $Property) { $Property.$ValueName } else { "Not Found" }
 Write-Host " - Registry Setting: $ValueName | Actual: $ActualVal (Expected: $ExpectedValue)" -ForegroundColor Red
 $Vulnerable = $true
 }
} else {
 Write-Host " - Registry Path: $RegPath | Actual: Path Not Found (Expected: Path Exists)" -ForegroundColor Red
 $Vulnerable = $true
}

if ($Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10/11 Client Benchmark\*\*: Section 2.3.9.5 (Interactive logon: Smart card removal behavior) and Section 2.3.9.6 (Interactive logon: Require smart card). \* \*\*ANSSI AD Hardening Guide\*\*: Recommendations on workstation physical and logical isolation, and enforcing multi-factor authentication. \* \*\*Microsoft Security Guidance\*\*: Enforcing multi-factor authentication for high-value hosts.

# [REQ-PAW-032] Disable Unused Windows Features and PowerShell 2.0 Engine

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) (Tier 0 hosts). \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional.

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path (PowerShell 2.0 Compatibility)\*\*: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows PowerShell` \* \*\*Registry Location (PowerShell 2.0 Compatibility)\*\*: `HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\PowerShellV2` \* \*\*Feature Disablement\*\*: Deployed via Group Policy Startup Script or PowerShell Desired State Configuration (DSC) using DISM.

---

## Rationale To enforce strict isolation and minimize the attack surface of Tier 0 Privileged Access Workstations, all unnecessary legacy protocols, optional features, and runtime engines must be disabled.

1. **PowerShell 2.0 Engine:** Legacy PowerShell 2.0 does not support modern logging, transcription, or security monitoring mechanisms such as the Antimalware Scan Interface (AMSI). Attackers leverage "downgrade attacks" by executing PowerShell scripts using the `-version 2.0` parameter to bypass script block logging and security tooling. Disabling the engine and its parent runtimes eliminates this bypass vector.
2. **.NET Framework 3.5:** The .NET 3.5 Framework includes the runtime files for .NET 2.0 and 3.0. PowerShell 2.0 requires .NET 2.0/3.5 to run. Disabling `.NET Framework 3.5` removes legacy runtime binaries that are susceptible to downgrade attacks and removes support for older, unpatched software.
3. **SMBv1 Protocol:** The legacy SMBv1 protocol is cryptographically weak, lacks authentication integrity protection, and has been the target of catastrophic remote code execution attacks (such as EternalBlue). Leaving the SMBv1 driver active allows relaying and man-in-the-middle attacks.
4. **Internet Explorer 11:** Internet Explorer contains obsolete MSHTML render engine components. Disabling this legacy browser reduces vulnerability to web-based code execution.
5. **Work Folders, XPS, DirectPlay, and Client Protocols:** Services and tools such as Work Folders, XPS Viewer, DirectPlay, Telnet Client, TFTP Client, and Simple TCP/IP Services contain legacy network parsers and protocols that are completely unnecessary for a secure administrative system.
6. **WSL and Windows Sandbox:** Virtualization layers such as the Windows Subsystem for Linux (WSL) and Windows Sandbox allow the execution of unmonitored binaries, containers, and Linux utilities. On a PAW, these components present an unacceptable audit-bypass risk and must be disabled.

---

## Legacy Impact & Compatibility \* \*\*Script Dependencies\*\*: Any administrative scripts that depend on .NET Framework 2.0/3.0/3.5 will fail. All internal scripts must be updated to run on .NET 4.8 or modern .NET (Core) runtimes. \* \*\*Legacy Management Software\*\*: Old configuration managers or printers that require SMBv1 client capabilities to mount file shares will fail to connect. \* \*\*Developer Tools\*\*: Disabling WSL and Windows Sandbox on PAWs prevents the use of localized Docker configurations or container tests on the administrative device. PAWs must remain dedicated to administration, and developer workloads must be redirected to standard development endpoints.

---

## ## Implementation Steps

### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### Step 1: Disable PowerShell 2.0 Compatibility Policy 1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host. 2. Edit the target PAWs GPO (e.g., `GPO_Hardening_PAWs`). 3. Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows PowerShell` 4. Configure the following setting: \* \*\*Policy\*\*: `Turn on PowerShell 2.0 Compatibility Mode` \* \*\*Setting\*\*: `Disabled` 5. Link the GPO to the PAWs Organizational Unit (OU).

##### Step 2: Deploy Feature Disablement Startup Script Because Windows Optional Features are managed via DISM/Packages and lack direct GPO settings for feature removal, deploy the disablement script as a Computer Startup script: 1. In the same GPO, navigate to: `Computer Configuration\Policies\Windows Settings\Scripts (Startup/Shutdown)` 2. Double-click **Startup**, click the **PowerShell Scripts** tab. 3. Add the `Disable-PawUnusedFeatures.ps1` script to execute on system startup.

---

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally or run it as a startup script.

[Download Script: Disable-PawUnusedFeatures.ps1](#)



```
Disable-PawUnusedFeatures.ps1
Description: Disables unused legacy features, .NET 3.5, and PowerShell 2.0 on the local PAW system.

Write-Host "Disabling unused legacy features and PowerShell 2.0..." -ForegroundColor Cyan

Check if running as administrator
$isAdmin = ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
if (-not $isAdmin) {
 Write-Error "This script must be run as an Administrator."
 exit 1
}

Features to disable (Client OS DISM feature names)
$Features = @(
 "MicrosoftWindowsPowerShellV2",
 "MicrosoftWindowsPowerShellV2Root",
 "NetFx3", # .NET Framework 3.5
 "SMB1Protocol", # SMBv1 Client
 "Internet-Explorer-Optional-amd64", # Internet Explorer 11
 "WorkFolders-Client", # Work Folders Client
 "Xps-Viewer-Dependency", # XPS Viewer
 "DirectPlay", # DirectPlay
 "TelnetClient", # Telnet Client
 "TFTP", # TFTP Client
 "SimpleTCP", # Simple TCP/IP Services
 "Microsoft-Windows-Subsystem-Linux", # WSL (specifically disabled on PAWs)
 "Containers-DisposableClientVM" # Windows Sandbox (specifically disabled on PAWs)
)

foreach ($Feature in $Features) {
 $State = Get-WindowsOptionalFeature -Online -FeatureName $Feature -ErrorAction SilentlyContinue
 if ($null -ne $State) {
 if ($State.State -eq "Enabled" -or $State.State -eq "EnabledPendingRestart") {
 Write-Host "[*] Disabling feature: $Feature..." -ForegroundColor Yellow
 Disable-WindowsOptionalFeature -Online -FeatureName $Feature -NoRestart -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Feature '$Feature' has been disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Feature '$Feature' is already disabled." -ForegroundColor Gray
 }
 } else {
 Write-Host "[~] Feature '$Feature' is not present in this Windows image." -ForegroundColor Gray
 }
}

Write-Host "Optional features configuration completed." -ForegroundColor Green
```

To verify the state of unused features:

[Download Script: Get-PawUnusedFeaturesStatus.ps1](#)

```
Get-PawUnusedFeaturesStatus.ps1
Description: Audits the installation state of unused legacy features on the local PAW system.

Write-Host "--- Auditing Unused Windows Features ---" -ForegroundColor Cyan

Check if running as administrator
$isAdmin = ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
if (-not $isAdmin) {
 Write-Error "This script must be run as an Administrator."
 exit 1
}

$script:Vulnerable = $false

Features to check (Client OS DISM feature names)
$Features = @(
 "MicrosoftWindowsPowerShellV2",
 "MicrosoftWindowsPowerShellV2Root",
 "NetFx3", # .NET Framework 3.5
 "SMB1Protocol", # SMBv1 Client
 "Internet-Explorer-Optional-amd64", # Internet Explorer 11
 "WorkFolders-Client", # Work Folders Client
 "Xps-Viewer-Dependency", # XPS Viewer
 "DirectPlay", # DirectPlay
 "TelnetClient", # Telnet Client
 "TFTP", # TFTP Client
 "SimpleTCP", # Simple TCP/IP Services
 "Microsoft-Windows-Subsystem-Linux", # WSL (specifically disabled on PAWs)
 "Containers-DisposableClientVM" # Windows Sandbox (specifically disabled on PAWs)
)

foreach ($Feature in $Features) {
 $State = Get-WindowsOptionalFeature -Online -FeatureName $Feature -ErrorAction SilentlyContinue
 if ($null -ne $State) {
 $IsEnabled = ($State.State -eq "Enabled" -or $State.State -eq "EnabledPendingRestart")
 $Color = if (-not $IsEnabled) { "Green" } else { "Red" }
 Write-Host " - Feature: $Feature | State: $($State.State) (Expected: Disabled)" -ForegroundColor $Color

 if ($IsEnabled) {
 $script:Vulnerable = $true
 }
 } else {
 Write-Host " - Feature: $Feature | Not Present (Compliant)" -ForegroundColor Green
 }
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
}
```

---

## Sources & Compliance References \* \*\*DoD Windows 11 Computer STIG v2r6\*\*\*: V-220728 (PowerShell 2.0), V-253286 (SMBv1), V-219524 (.NET Framework 3.5) \* \*\*ANSSI AD Hardening Guide\*\*\*: Section on Endpoint Minimization and disabling legacy protocols \* \*\*CIS Microsoft Windows Client Benchmark\*\*\*: Section 18.9 (PowerShell restrictions)

## # [REQ-PAW-033] Configure Microsoft Office Security and Block OLE Packages

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) (Tier 0 Workstations) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: User Configuration\Policies\Administrative Templates\Microsoft Office 2016\Security Settings \* \*\*Registry Locations\*\*: \* `HKCU\software\policies\microsoft\office\16.0\common\security` \* `vbawarnings` = `3` (REG\_DWORD, Enforce macro signing / Block unsigned macros) \* `HKCU\software\policies\microsoft\office\16.0\excel\security` \* `blockcontentexecutionfrominternet` = `1` (REG\_DWORD, Block macros in files from the Internet) \* `HKCU\software\policies\microsoft\office\16.0\word\security` \* `blockcontentexecutionfrominternet` = `1` (REG\_DWORD) \* `HKCU\software\policies\microsoft\office\16.0\powerpoint\security` \* `blockcontentexecutionfrominternet` = `1` (REG\_DWORD) \* `HKCU\software\policies\microsoft\office\16.0\outlook\security` \* `ShowOLEPackageObj` = `0` (REG\_DWORD, Disable OLE package activation)

---

## Rationale Malicious documents (e.g., weaponized Word, Excel, or PowerPoint files) containing embedded VBA macros are a prevalent initial access and execution vector. Similarly, embedding malicious OLE packages inside Outlook items (such as RTF-formatted emails) allows attackers to trigger script execution or execute arbitrary packages via `packager.dll` when an administrator opens or previews the email. Hardening these settings ensures:

1. **Internet Macro Blocking:** VBA macros in files downloaded from the Internet or untrusted external attachments are blocked from executing, regardless of user consent.
  2. **Macro Code Signing:** Any locally run macros are restricted to trusted, digitally signed code, preventing the execution of ad-hoc unverified user scripts.
  3. **OLE Package Disablement:** Restricting Outlook OLE package activation ( `ShowOLEPackageObj = 0` ) blocks the execution of dangerous embedded objects in email messages.
- 

## Legacy Impact & Compatibility \* \*\*Office Macros\*\*: Business workflows relying on macro-enabled spreadsheets or documents downloaded from external sources (e.g., vendor portals) will be blocked. Unsigned local macros will also fail to run. Users must acquire certificates to digitally sign internal macro projects. \* \*\*Outlook Packages\*\*: Embedded document shortcuts or packages in emails will be displayed as static icons and cannot be double-clicked for direct execution.

---

## ## Implementation Steps

## ### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### Step 1: Enforce Macro Security in ADMX Templates 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Edit the PAW GPO. 3. Navigate to: `User Configuration\Policies\Administrative Templates\Microsoft Office 2016\Security Settings\Trust Center` 4. Set the following policies: \* \*\*Policy\*\*: `VBA Macro Notification Settings` → **Enabled** with option set to **Disable all except digitally signed macros** 5. For each application (Word, Excel, PowerPoint, Access), navigate to: `User Configuration\Policies\Administrative Templates\[Application] 2016\[Application] Options\Security\Trust Center` 6. Set the policy: \* \*\*Policy\*\*: `Block macros from running in Office files from the Internet` → **Enabled**

##### Step 2: Disable Outlook OLE Packages 1. Navigate to: `User Configuration\Policies\Administrative Templates\Microsoft Outlook 2016\Security` 2. Configure the setting: \* \*\*Policy\*\*: `Do not allow OLE package execution` → **Enabled**

---

## ### Option B: PowerShell &amp; Registry Configuration (Remediation / Non-GPO)

Configure the current user registry hives to enforce macro blocking and OLE package restrictions.

[Download Script: Configure-PawOfficeSecurity.ps1](#)

```
Configure-PawOfficeSecurity.ps1
Description: Configures registry settings under the HKCU hive to restrict VBA macros and block Outlook OLE package execution on PA
Ws.

Write-Host "Applying Microsoft Office security and OLE restrictions..." -ForegroundColor Cyan

Helper to configure User Registry DWORD values
function Set-UserRegDWord {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$Path,
 [string]$Name,
 [int]$Value
)
 if ($PSCmdlet.ShouldProcess($Path, "Set registry DWORD value $Name to $Value")) {
 $FullRegistryPath = "HKCU:\$Path"
 if (-not (Test-Path $FullRegistryPath)) {
 New-Item -Path $FullRegistryPath -Force | Out-Null
 }
 Set-ItemProperty -Path $FullRegistryPath -Name $Name -Value $Value -Type DWord -Force
 }
}

1. Enforce macro signing policy (common)
Set-UserRegDWord "software\policies\microsoft\office\16.0\common\security" "vbwarnings" 3
Write-Host "[+] Digital signing for Office macros enforced." -ForegroundColor Green

2. Block macros from the Internet for key Office applications
$Apps = @("excel", "word", "powerpoint", "access", "visio")
foreach ($App in $Apps) {
 Set-UserRegDWord "software\policies\microsoft\office\16.0\$App\security" "blockcontentexecutionfrominternet" 1
}
Write-Host "[+] VBA macro blocks from Internet applied to Office applications." -ForegroundColor Green

3. Disable OLE Package execution in Outlook (Policies and Preferences branches)
Set-UserRegDWord "software\policies\microsoft\office\16.0\outlook\security" "ShowOLEPackageObj" 0
Set-UserRegDWord "software\microsoft\office\16.0\outlook\security" "ShowOLEPackageObj" 0
Write-Host "[+] Outlook OLE Package execution blocked." -ForegroundColor Green
```

*To verify the current Office security settings:*

[Download Script: Get-PawOfficeSecurityStatus.ps1](#)

```
Get-PawOfficeSecurityStatus.ps1
Description: Audits Microsoft Office macro settings and Outlook OLE package restrictions on PAWs.

Write-Host "--- Auditing Microsoft Office Security BaseLine ---" -ForegroundColor Cyan

$script:Vulnerable = $false

Helper to audit registry values under HKCU
function Test-UserRegistryValue ($Path, $Name, $ExpectedValue) {
 $FullRegistryPath = "HKCU:\$Path"
 $Val = Get-ItemProperty -Path $FullRegistryPath -Name $Name -ErrorAction SilentlyContinue
 $Actual = if ($val) { $val.$Name } else { "" }
 $Color = "Red"
 if ($Actual -eq $ExpectedValue) {
 $Color = "Green"
 } else {
 $script:Vulnerable = $true
 }
 Write-Host " - User Registry: $Name | Actual: '$Actual' (Expected: '$ExpectedValue')" -ForegroundColor $Color
}

1. Audit macro signing warning
Test-UserRegistryValue "software\policies\microsoft\office\16.0\common\security" "vbawarnings" 3

2. Audit macro Internet blocks
$Apps = @("excel", "word", "powerpoint", "access", "visio")
foreach ($App in $Apps) {
 Test-UserRegistryValue "software\policies\microsoft\office\16.0\$App\security" "blockcontentexecutionfrominternet" 1
}

3. Audit Outlook OLE package block
Test-UserRegistryValue "software\policies\microsoft\office\16.0\outlook\security" "ShowOLEPackageObj" 0

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

**## Sources & Compliance References** \* **ANSI AD Hardening Guide**: Recommendations on restricting application execution, blocking external scripts, and application sandboxing. \* **CIS Microsoft Office Benchmark**: Section on Office Common Security settings, VBA warnings, and blocking macros in internet files. \* **Microsoft Security Baseline**: Recommended settings for Microsoft Office and Office 365 ProPlus Security. \* **DoD Microsoft Outlook STIG**: Restrictions on active content, remote attachments, and OLE execution behavior.

## # [REQ-PAW-034] Disable Windows Script Host and Remap Scripting Extensions

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) (Tier 0 Workstations) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path (WSH Disable)\*\*: Computer Configuration\Preferences\Windows Settings\Registry \* \*\*GPO Path (Associations)\*\*: User Configuration\Preferences\Control Panel Settings\Folder Options \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Microsoft\Windows Script Host\Settings` \* `Enabled` = `0` (REG\_DWORD) \* `HKCU\SOFTWARE\Microsoft\Windows Script Host\Settings` \* `Enabled` = `0` (REG\_DWORD)

## Rationale Windows Script Host (WSH), which executes VBScript and JScript files (`wscript.exe` and `cscript.exe`), is frequently targeted by threat actors in phishing campaigns and initial access vectors. By placing a script file (e.g., `.vbs`, `.js`, `.wsf`, `.hta`) in an email attachment or download path, attackers can execute arbitrary code on the system if a user opens the file.

Enforcing these deactivation controls ensures:

1. **Attack Surface Reduction:** Disabling WSH globally blocks the execution of JScript and VBScript files via the standard scripting engines on the workstation.
2. **Defense-in-Depth File Associations:** Remapping the default file handler for typical scripting extensions to `notepad.exe` ensures that if a scripting file is double-clicked by an administrator, the file opens as plaintext in Notepad for inspection rather than executing its contents.

## Legacy Impact & Compatibility \* \*\*Script Dependencies\*\*: Any legacy administrative scripts (such as logon scripts or backup routines) written in VBScript or JScript will fail to run. All internal workstation management scripts must be written in PowerShell 5.1+ and executed under secure execution policies. \* \*\*Explorer Associations\*\*: Double-clicking on a `.js` or `.vbs` configuration file will open Notepad instead of running the script.

## ## Implementation Steps

## ### Option A: Group Policy Object (GPO) Configuration (Preferred)

#### Step 1: Disable WSH via GPO Registry Preferences 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Edit the PAW GPO (e.g., `GPO\_Hardening\_PAW`). 3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry` 4. Create a new **Registry Item**: \* \*\*Action\*\*: `Update` \* \*\*Hive\*\*: `HKEY\_LOCAL\_MACHINE` \* \*\*Key Path\*\*: `SOFTWARE\Microsoft\Windows Script Host\Settings` \* \*\*Value Name\*\*: `Enabled` \* \*\*Value Type\*\*: `REG\_DWORD` \* \*\*Value Data\*\*: `0`

#### Step 2: Configure Script File Extensions to Open in Notepad 1. Navigate to: `User Configuration\Preferences\Control Panel Settings\Folder Options` 2. Right-click and select **New → Open With**: \* \*\*File Extension\*\*: `.vbs` \* \*\*Associated Program\*\*: `%SystemRoot%\System32\notepad.exe` \* \*\*Set as default\*\*: Check 3. Repeat the process for the following extensions: `.vbe`, `.js`, `.jse`, `.wsf`, `.wsh`, `.hta`.

## ### Option B: PowerShell &amp; Registry Configuration (Remediation / Non-GPO)

Configure the local registry settings to disable WSH and remap associations.

[Download Script: Disable-PawWsh.ps1](#)

```
Disable-PawWsh.ps1
Description: Disables Windows Script Host globally in HKLM and HKCU registry hives, and remaps script file associations to Notepad.

Write-Host "Applying Windows Script Host and file association hardening..." -ForegroundColor Cyan

1. Disable WSH globally
$RegistryHkLm = "HKLM:\SOFTWARE\Microsoft\Windows Script Host\Settings"
if (-not (Test-Path $RegistryHkLm)) {
 New-Item -Path $RegistryHkLm -Force | Out-Null
}
Set-ItemProperty -Path $RegistryHkLm -Name "Enabled" -Value 0 -Type DWord -Force
Write-Host "[+] WSH globally disabled in HKLM." -ForegroundColor Green

$RegistryHkcu = "HKCU:\SOFTWARE\Microsoft\Windows Script Host\Settings"
if (-not (Test-Path $RegistryHkcu)) {
 New-Item -Path $RegistryHkcu -Force | Out-Null
}
Set-ItemProperty -Path $RegistryHkcu -Name "Enabled" -Value 0 -Type DWord -Force
Write-Host "[+] WSH disabled in current user HKCU hive." -ForegroundColor Green

2. Remap script file extensions to notepad
$Extensions = @("vbs", "vbe", "js", "jse", "wsf", "wsh", "hta")
foreach ($Ext in $Extensions) {
 $ProgIdPath = "HKLM:\SOFTWARE\Classes\.$Ext"

 # Update Class Association to Notepad
 if (-not (Test-Path $ProgIdPath)) {
 New-Item -Path $ProgIdPath -Force | Out-Null
 }
 Set-ItemProperty -Path $ProgIdPath -Name "" -Value "txtfile" -Type String -Force
 Write-Host " Mapped .$Ext extension to txtfile handler." -ForegroundColor Gray
}
Write-Host "[+] Script file extension handlers mapped to Notepad." -ForegroundColor Green
```

To verify the WSH configuration state:

[Download Script: Get-PawWshStatus.ps1](#)

```
Get-PawWshStatus.ps1
Description: Audits Windows Script Host registry state and script file extension association handlers on PAWs.

Write-Host "--- Auditing Windows Script Host Hardening ---" -ForegroundColor Cyan

$script:Vulnerable = $false

1. Audit WSH Registry settings
$RegistryHkLm = "HKLM:\SOFTWARE\Microsoft\Windows Script Host\Settings"
if (Test-Path $RegistryHkLm) {
 $ValHkLm = (Get-ItemProperty -Path $RegistryHkLm -Name "Enabled" -ErrorAction SilentlyContinue).Enabled
 if ($ValHkLm -eq 0) {
 Write-Host " - HKLM WSH Enabled: 0 (Secure)" -ForegroundColor Green
 } else {
 Write-Host " - VULNERABLE: HKLM WSH is enabled or not configured (Value: '$ValHkLm')" -ForegroundColor Red
 $script:Vulnerable = $true
 }
} else {
 Write-Host " - VULNERABLE: HKLM WSH settings key is missing (Expected: Enabled = 0)" -ForegroundColor Red
 $script:Vulnerable = $true
}

2. Audit file associations
$Extensions = @("vbs", "vbe", "js", "jse", "wsf", "wsh", "hta")
foreach ($Ext in $Extensions) {
 $ProgIdPath = "HKLM:\SOFTWARE\Classes\.$Ext"
 if (Test-Path $ProgIdPath) {
 $Handler = (Get-ItemProperty -Path $ProgIdPath -Name "" -ErrorAction SilentlyContinue).""
 if ($Handler -eq "txtfile" -or $Handler -match "notepad") {
 Write-Host " - Extension .$Ext Handler: $Handler (Secure)" -ForegroundColor Green
 } else {
 Write-Host " - VULNERABLE: Extension .$Ext Handler is '$Handler' (Expected: txtfile/notepad)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
 } else {
 Write-Host " - VULNERABLE: Extension .$Ext Class Registry key not found." -ForegroundColor Red
 $script:Vulnerable = $true
 }
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendations on workstation OS minimization and script execution control. \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: V-219661 (Windows Script Host disablement and script restriction). \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 18.9 (Administrative templates for script execution safety).



# [REQ-PAW-035] Configure Secure Boot Revocations and Bootloader Updates for PAWs

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs). \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows 11 Enterprise.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*BlackLotus Revocation Updates (CVE-2023-24932)\*\*: 'HKLM\SYSTEM\CurrentControlSet\Control\Secureboot' \* 'AvailableUpdates' = '>= 0x4000' (REG\_DWORD)

## Rationale A vulnerability in the Windows Boot Manager allows an attacker with physical access or local administrative rights to bypass UEFI Secure Boot and execute unsigned code during the boot process (BlackLotus bootkit).

To fully mitigate this threat (CVE-2023-24932), Windows update revocations must be applied to the UEFI variables (DBX list) and code integrity SVN policies must be updated. This is managed via the `AvailableUpdates` registry key, which instructs the OS boot manager to write the revocation variables to firmware.

According to the latest Microsoft guidelines, the recommended trigger value for enterprise deployments to apply all security updates (including the new Windows UEFI CA 2023 certificates and boot manager updates) is `0x5944` (hex) / `22852` (decimal). As the OS processes this bitmask, the value is cleared incrementally, ending up at `0x4000` (hex) / `16384` (decimal) upon successful completion.

## Legacy Impact & Compatibility \* \*\*BlackLotus Mitigation Risks\*\*: Enforcing the BlackLotus DBX and SVN updates is a permanent, non-reversible action once written to the device firmware. If an administrator attempts to boot the machine using older, unpatched Windows installation media or recovery disks, the system will reject the boot manager and fail to boot. All recovery and deployment media must be updated with current security patches before applying these mitigations.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

To configure the update triggers for the DBX and Code Integrity boot manager revocations, define Registry GPO Preferences inside the PAW GPO:

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Create or edit a GPO linked to the PAWs OU (e.g., `GPO_Hardening_PAWs` ).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a registry item to deploy the `AvailableUpdates` DWORD under `HKLM\SYSTEM\CurrentControlSet\Control\Secureboot` .
  - **Action:** `Update`
  - **Hive:** `HKEY_LOCAL_MACHINE`
  - **Key Path:** `SYSTEM\CurrentControlSet\Control\Secureboot`
  - **Value Name:** `AvailableUpdates`
  - **Value Type:** `REG_DWORD`
  - **Value Data:** `22852` (Decimal) or `5944` (Hex)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script locally to configure the BlackLotus mitigation update trigger:

[Download Script: Set-PawSecureBootRevocations.ps1](#)

```
Set-PawSecureBootRevocations.ps1
Description: Triggers Secure Boot DBX and Code Integrity revocation updates for BlackLotus mitigation.

Write-Host "--- Configuring BlackLotus Secure Boot Mitigations ---" -ForegroundColor Cyan

$Path = "HKLM:\SYSTEM\CurrentControlSet\Control\Secureboot"
if (-not (Test-Path $Path)) {
 New-Item -Path $Path -Force | Out-Null
}

Trigger updates (0x5944 = 22852)
Set-ItemProperty -Path $Path -Name "AvailableUpdates" -Value 22852 -Type DWord -Force | Out-Null
Write-Host "[+] BlackLotus DBX and 2023 CA revocation updates configured in registry. A system reboot is required." -ForegroundColor Green
```

### Audit Script

Run the following script to check the status of Secure Boot revocations on the local machine:

## Download Script: Audit-PawSecureBootRevocations.ps1

```
Audit-PawSecureBootRevocations.ps1
Description: Queries UEFI Secure Boot parameters and audits BlackLotus mitigation registry settings.

Write-Host "--- Auditing BlackLotus Mitigations ---" -ForegroundColor Cyan

$script:NonCompliant = $false

1. Audit AvailableUpdates registry key
$Path = "HKLM:\SYSTEM\CurrentControlSet\Control\Secureboot"
if (Test-Path $Path) {
 $Val = Get-ItemProperty -Path $Path -Name "AvailableUpdates" -ErrorAction SilentlyContinue
 $UpdateVal = if ($Val) { $Val.AvailableUpdates } else { 0 }

 # Check if configured (≥ 0x4000 / 16384)
 if ($UpdateVal -ge 16384) {
 Write-Host " - BlackLotus Revocation Updates (AvailableUpdates): $UpdateVal (Compliant)" -ForegroundColor Green
 } else {
 Write-Host " - BlackLotus Revocation Updates (AvailableUpdates): $UpdateVal (Non-Compliant - DBX/SVN revocations not triggered)" -ForegroundColor Red
 $script:NonCompliant = $true
 }
} else {
 Write-Host " - BlackLotus Revocation Updates: Registry path not found (Non-Compliant)" -ForegroundColor Red
 $script:NonCompliant = $true
}

2. Audit UEFICA2023Status (if present)
$ServicingPath = "HKLM:\SYSTEM\CurrentControlSet\Control\SecureBoot\Servicing"
if (Test-Path $ServicingPath) {
 $ServVal = Get-ItemProperty -Path $ServicingPath -Name "UEFICA2023Status" -ErrorAction SilentlyContinue
 if ($ServVal) {
 $Status = $ServVal.UEFICA2023Status
 $Color = if ($Status -eq "Updated") { "Green" } else { "Yellow" }
 Write-Host " - UEFI CA 2023 Update Status: $Status" -ForegroundColor $Color
 }
}

if ($script:NonCompliant) {
 exit 1
}
```

## Sources & Compliance References \* \*\*Microsoft KB5068202\*\*: Registry key updates for Secure Boot: Windows devices with IT-managed updates \* \*\*ANSSI AD Hardening Guide\*\*: Recommendations regarding hardware platform integrity.

# [REQ-PAW-036] Configure Windows Defender Application Control

## Target Scope \* \*\*Applicable Systems\*\*: Privileged Access Workstations (PAWs) \* \*\*Operating Systems\*\*: Windows 10, Windows 11 (Enterprise and Professional editions)

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: Computer Configuration\Administrative Templates\System\Device Guard\Deploy Windows Defender Application Control \* \*\*Registry Location\*\*:  
'HKLM\SOFTWARE\Policies\Microsoft\Windows\DeviceGuard\CodeIntegrityPolicyPaths'

---

## Rationale Privileged Access Workstations (PAWs) are dedicated administrative hosts used to manage high-value assets such as Domain Controllers and identity systems. Because they handle Tier 0 administrative credentials, they are highly targeted by adversaries.

**Windows Defender Application Control (WDAC)** provides kernel-enforced application control to ensure that only trusted code executes on PAWs. Standard application control options like AppLocker operate primarily in user mode, whereas WDAC enforces integrity at both the kernel (KMCI) and user mode (UMCI) levels. Implementing a baseline WDAC policy that restricts software execution exclusively to Microsoft-signed code and trusted system components blocks unauthorized administrative tools, remote monitoring agents, and malicious payloads.

---

## Legacy Impact & Compatibility \* \*\*Pre-requisite (Memory Integrity/HVCI)\*\*: Hypervisor-Protected Code Integrity (HVCI) must be active to enforce code integrity policies at the hypervisor layer. Refer to [REQ-PAW-010 - Enable VBS and Credential Guard for PAWs](#07-paws-enable-vbs-credential-guard-md) to ensure Memory Integrity (HVCI) and Virtualization-Based Security (VBS) are fully active. \*

\* \*\*Administrative Overhead\*\*: Standard users and administrators cannot install or execute arbitrary software. Any administration tool, package, or script must be signed by a trusted publisher or explicitly allowed by the WDAC policy. \* \*\*Audit Mode Deployment\*\*: To prevent disruption of critical administrative tasks, the WDAC baseline policy must be deployed in **Audit Mode** initially. This allows tracking would-be blocks in the Event Viewer without interrupting daily operations, allowing the baseline to be fully tuned before enforcement.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

To deploy WDAC via Group Policy, the policy XML must first be generated, compiled, and placed in a secure shared intranet network path or local path on target hosts.

#### 1. Generate and Compile the Policy (on a Reference PAW Host) Run the following PowerShell commands to generate the Microsoft Default Windows baseline policy: ``powershell # Generate the baseline policy XML New-CIPolicy -MultiplePolicyFormat -Level FilePublisher -FilePath "C:\WDAC\PawBaselinePolicy.xml" -UserPEs

## Compile the XML policy into a binary CIP file

---

ConvertFrom-CIPolicy -XmlFilePath "C:\WDAC\PawBaselinePolicy.xml" -BinaryFilePath "C:\WDAC\PawBaselinePolicy.cip"

```
<div id="07-paws-configure-wdac-md-2-deploy-the-policy-via-gpo"></div>
2. Deploy the Policy via GPO
1. Copy the compiled `PawBaselinePolicy.cip` file to a local secure directory on all target PAWs (e.g., `C:\Windows\System32\CodeIntegrity\SIPolicy.p7b`) or host it on a network share.
2. Open the **Group Policy Management Console** (`gpmc.msc`).
3. Create or edit a GPO linked to the PAWs OU (e.g., `GPO_Hardening_PAWs`).
4. Navigate to:
 `Computer Configuration\Administrative Templates\System\Device Guard`
5. Configure the following setting:
 * **Policy**: `Deploy Windows Defender Application Control`
 * **Setting**: `Enabled`
 * **Code Integrity Policy File Path**: Enter the local path (e.g., `C:\Windows\System32\CodeIntegrity\SIPolicy.p7b`) or network share path.
```

```

```

```
<div id="07-paws-configure-wdac-md-option-b-powershell-registry-configuration-remediation-non-gpo"></div>
Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)
```

Run the following scripts locally to generate a baseline WDAC policy, enable Audit Mode, and configure local parameters.

```
<div id="07-paws-configure-wdac-md-configure-pawwdaclocalpolicy.ps1"></div>
Configure-PawWDACLocalPolicy.ps1

[Download Script: Configure-PawWDACLocalPolicy.ps1](implementation_scripts/Configure-PawWDACLocalPolicy.ps1)

```powershell
# Configure-PawWDACLocalPolicy.ps1
# Description: Generates a baseline local Code Integrity policy for PAWs, sets it to Audit Mode, and compiles it.

Write-Host "--- Configuring PAW WDAC Local Policy Baseline ---" -ForegroundColor Cyan

# Create working directories
$WdacDir = "C:\Windows\System32\CodeIntegrity"
if (-not (Test-Path $WdacDir)) {
    New-Item -Path $WdacDir -ItemType Directory -Force | Out-Null
}

# 1. Generate the Default Windows Policy
Write-Host "[+] Generating Default Windows code integrity rules..." -ForegroundColor Gray
$PolicyXml = "C:\Windows\Temp\PawDefaultWindows.xml"
$PolicyBin = "$WdacDir\SIPolicy.p7b"

# Create a policy based on Microsoft's default rules (trusts Windows, Store, and Driver files)
New-CIPolicy -FilePath $PolicyXml -Level Windows -UserPEs -ErrorAction Stop

# 2. Set Policy to Audit Mode (Rule Option 3 represents Audit Mode)
Write-Host "[+] Setting WDAC policy to Audit Mode for baseline logging..." -ForegroundColor Gray
Set-RuleOption -FilePath $PolicyXml -Option 3 -ErrorAction SilentlyContinue

# 3. Compile the XML into the binary policy expected by the bootloader
Write-Host "[+] Compiling Code Integrity XML into SIPolicy.p7b..." -ForegroundColor Gray
ConvertFrom-CIPolicy -XmlFilePath $PolicyXml -BinaryFilePath $PolicyBin -ErrorAction Stop

# Cleanup temp files
if (Test-Path $PolicyXml) { Remove-Item $PolicyXml -Force }

Write-Host "[+] Local WDAC baseline policy configured. Reboot required." -ForegroundColor Green
```

To verify the setting has been applied:

```
# Test-PawWDACStatus.ps1
```

[Download Script: Test-PawWDACStatus.ps1](#)

```
# Test-PawWDACStatus.ps1
# Description: Audits the local PAW to check if Code Integrity policies and HVCI are active.

Write-Host "--- Auditing PAW WDAC State ---" -ForegroundColor Cyan
$Vulnerable = $false

# 1. Query WMI class for Code Integrity status
try {
    $CI = Get-CimInstance -Namespace "Root\Microsoft\Windows\CI" -ClassName "MSFT_Sipolicy" -ErrorAction Stop
    if ($null -ne $CI -and $CI.Count -gt 0) {
        Write-Host "`n[+] Found $($CI.Count) active Code Integrity policies." -ForegroundColor Green
        foreach ($Policy in $CI) {
            Write-Host "    - Policy: $($Policy.FriendlyName) | ID: $($Policy.PolicyID) | Enforced: $($Policy.EnforcementMode)" -ForegroundColor Green
        }
    } else {
        Write-Host "`n[-] No active Code Integrity / WDAC policies detected via WMI." -ForegroundColor Yellow
    }
} catch {
    Write-Host "`n[-] Could not query WMI MSFT_Sipolicy. This is expected if no WDAC policies are currently deployed." -ForegroundColor Gray
}

# 2. Check Memory Integrity (HVCI) configuration
$ScenariosPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\HypervisorEnforcedCodeIntegrity"
if (Test-Path $ScenariosPath) {
    $HvciStatus = Get-ItemProperty -Path $ScenariosPath -Name "Enabled" -ErrorAction SilentlyContinue
    if ($null -ne $HvciStatus -and $HvciStatus.Enabled -eq 1) {
        Write-Host "[+] Memory Integrity (HVCI) is enabled." -ForegroundColor Green
    } else {
        Write-Host "[!] VULNERABLE: Memory Integrity (HVCI) is disabled in the registry." -ForegroundColor Red
        $Vulnerable = $true
    }
} else {
    Write-Host "[!] VULNERABLE: Memory Integrity scenario registry path does not exist." -ForegroundColor Red
    $Vulnerable = $true
}

# 3. Final Verdict
if ($Vulnerable) {
    Write-Host "`n[!] Verification FAILED: One or more driver security controls are not configured." -ForegroundColor Red
} else {
    Write-Host "`n[+] Verification PASSED: WDAC driver settings and HVCI are correctly configured." -ForegroundColor Green
}
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Recommendations on system component code integrity and application control restrictions. * **CIS Microsoft Windows 10/11 Benchmark***: Section 18.8.14.3 (Deploy Windows Defender Application Control / Memory Integrity). * **Microsoft Security Guidance***: Windows Defender Application Control Deployment Guide.

[REQ-PAW-129] Configure Advanced Security Audit Policies for PAWs

Target Scope * **Applicable Systems**: Privileged Access Workstations * **Operating Systems**: Windows 10/11 Enterprise

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` * **Policy**: `Audit: Force audit policy subcategory settings (Windows Vista or later) to override audit policy category settings` * **Setting**: `Enabled` * **Registry Location**: `HKLM\System\CurrentControlSet\Control\Lsa\SCENoApplyLegacyAuditPolicy` = `1` (DWord)

Rationale Enforcing category overrides on PAWs is a prerequisite to ensure that the refined audit subcategories (such as Logon, Object Access, System Events, etc.) are correctly logged and not masked by basic legacy policies.

Legacy Impact & Compatibility * **Event Log Volume**: Ensure the Security log size is at least 512MB to prevent rollover.

Submodule Requirements The child policies under this parent requirement: * **[REQ-PAW-130 - Audit Policy: Advanced Audit Policy Overrides for PAWs](#07-paws-audit-policy-configure-paw-audit-audit-override-md)** * **[REQ-PAW-131 - Audit Policy: Account Logon Auditing for PAWs](#07-paws-audit-policy-configure-paw-audit-account-logon-md)** * **[REQ-PAW-132 - Audit Policy: Account Management Auditing for PAWs](#07-paws-audit-policy-configure-paw-audit-account-management-md)** * **[REQ-PAW-133 - Audit Policy: Detailed Tracking Auditing for PAWs](#07-paws-audit-policy-configure-paw-audit-detailed-tracking-md)** * **[REQ-PAW-134 - Audit Policy: Logon and Logoff Auditing for PAWs](#07-paws-audit-policy-configure-paw-audit-logon-logoff-md)** * **[REQ-PAW-135 - Audit Policy: Object Access Auditing for PAWs](#07-paws-audit-policy-configure-paw-audit-object-access-md)** * **[REQ-PAW-136 - Audit Policy: Policy Change Auditing for PAWs](#07-paws-audit-policy-configure-paw-audit-policy-change-md)** * **[REQ-PAW-137 - Audit Policy: Privilege Use Auditing for PAWs](#07-paws-audit-policy-configure-paw-audit-privilege-use-md)** * **[REQ-PAW-138 - Audit Policy: System Events Auditing for PAWs](#07-paws-audit-policy-configure-paw-audit-system-events-md)**

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` 2. Configure the following setting: * **Policy**: `Audit: Force audit policy subcategory settings (Windows Vista or later) to override audit policy category settings` * **Setting**: `Enabled`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditPoliciesParent.ps1](#)] (./implementation_scripts/Configure-PawAuditPoliciesParent.ps1)

```
# Configure-PawAuditPoliciesParent.ps1
# Description: Enforces Advanced Audit Policy Overrides registry value.

$RegPath = "reg:\HKLM\System\CurrentControlSet\Control\Lsa"
$ValueName = "SCENoApplyLegacyAuditPolicy"

if (-not (Test-Path $RegPath)) {
    New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value 1 -Type DWord -Force
Write-Host "Enforced SCENoApplyLegacyAuditPolicy = 1 on PAW" -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-PawAuditPoliciesParentStatus.ps1](#)

```
# Get-PawAuditPoliciesParentStatus.ps1
# Description: Audits the Advanced Audit Policy Overrides registry value.

$RegPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
$ValueName = "SCENoApplyLegacyAuditPolicy"

$val = Get-ItemPropertyValue -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
if ($val -eq 1) {
    Write-Host "Advanced Security Audit Policy Overrides are correctly enabled." -ForegroundColor Green
} else {
    Write-Host "Advanced Security Audit Policy Overrides are disabled!" -ForegroundColor Red
}
```

Sources & Compliance References * **ANSSI AD Hardening Guide**: Recommendation R48 * **CIS Benchmark**: Section 9 and 17

[REQ-PAW-130] Audit Policy: Advanced Audit Policy Overrides for PAWs

Target Scope * **Applicable Systems**: Privileged Access Workstations (PAWs) * **Operating Systems**: Windows 10/11 Enterprise

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Paths**: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` * Registry: `HKLM\System\CurrentControlSet\Control\Lsa\SCENoApplyLegacyAuditPolicy` = `1` (DWord) * Registry: `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters\LogLevel` = `0` (DWord)

Rationale Enforcing advanced audit policy overrides prevents legacy category settings from overriding refined subcategory policies, and disabling verbose Kerberos logging ensures that event logs are not flooded with diagnostic events.

Legacy Impact & Compatibility * **Event Log Volume**: Ensure the local Security Event Log is sized to at least 512MB to accommodate the audit stream.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: * Registry: `HKLM\System\CurrentControlSet\Control\Lsa\SCENoApplyLegacyAuditPolicy` = `1` (DWord) * Registry: `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters\LogLevel` = `0` (DWord)

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditAuditoverride.ps1](#)] ([./implementation_scripts/Configure-PawAuditAuditoverride.ps1](#))

```
# Configure-PawAuditAuditoverride.ps1
Write-Host "Applying Audit Policy category: audit-override..." -ForegroundColor Cyan

# Set Registry Override: SCENoApplyLegacyAuditPolicy
if (-not (Test-Path "reg:\HKLM\System\CurrentControlSet\Control\Lsa")) { New-Item -Path "reg:\HKLM\System\CurrentControlSet\Control\Lsa" -Force | Out-Null }
Set-ItemProperty -Path "reg:\HKLM\System\CurrentControlSet\Control\Lsa" -Name "SCENoApplyLegacyAuditPolicy" -Value 1 -Type DWord -Force
Write-Host "    Enforced SCENoApplyLegacyAuditPolicy = 1" -ForegroundColor Green

# Set Registry Override: LogLevel
if (-not (Test-Path "reg:\HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters")) { New-Item -Path "reg:\HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters" -Force | Out-Null }
Set-ItemProperty -Path "reg:\HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters" -Name "LogLevel" -Value 0 -Type DWord -Force
Write-Host "    Enforced LogLevel = 0" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-PawAuditAuditoverrideStatus.ps1](#)

```
# Get-PawAuditAuditoverrideStatus.ps1
$script:Vulnerable = $false

# Audit Registry: SCENoApplyLegacyAuditPolicy
$RegVal = Get-ItemProperty -Path "reg:\HKLM\System\CurrentControlSet\Control\Lsa" -Name "SCENoApplyLegacyAuditPolicy" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.SCENoApplyLegacyAuditPolicy -ne 1) {
    $script:Vulnerable = $true
}

# Audit Registry: LogLevel
$RegVal = Get-ItemProperty -Path "reg:\HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters" -Name "LogLevel" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.LogLevel -ne 0) {
    $script:Vulnerable = $true
}

if ($script:Vulnerable) {
    Write-Output "Non-Compliant"
    exit 1
} else {
    Write-Output "Compliant"
    exit 0
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Client auditing baselines * **CIS Windows 10/11 Benchmark**: Section 17 (Advanced Audit Policy Configuration)

[REQ-PAW-131] Audit Policy: Account Logon Auditing for PAWs

Target Scope * **Applicable Systems**: Privileged Access Workstations (PAWs) * **Operating Systems**: Windows 10/11 Enterprise

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Paths**: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` * Subcategory: `Credential Validation` → `Success and Failure`

Rationale Auditing account logon events captures authentication requests processed by the local system or the workstation, which is critical for identifying Kerberoasting, NTLM relaying, and brute-force attempts.

Legacy Impact & Compatibility * **Event Log Volume**: Ensure the local Security Event Log is sized to at least 512MB to accommodate the audit stream.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: * Subcategory: `Credential Validation` → `Success and Failure`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditAccountlogon.ps1](#)] ([../implementation_scripts/Configure-PawAuditAccountlogon.ps1](#))

```
# Configure-PawAuditAccountlogon.ps1
Write-Host "Applying Audit Policy category: account-logon..." -ForegroundColor Cyan

# Set Audit Subcategory: Credential Validation
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Credential Validation`" /success:enable /failure:enable" -Wait
-NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Credential Validation to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Credential Validation. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-PawAuditAccountlogonStatus.ps1](#)

```
# Get-PawAuditAccountlogonStatus.ps1
$script:Vulnerable = $false

# Audit Subcategory: Credential Validation
$RawOutput = auditpol.exe /get /subcategory:"Credential Validation" /r
if ($RawOutput -notmatch ",Credential Validation,.*, (Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

if ($script:Vulnerable) {
    Write-Output "Non-Compliant"
    exit 1
} else {
    Write-Output "Compliant"
    exit 0
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Client auditing baselines * **CIS Windows 10/11 Benchmark**: Section 17 (Advanced Audit Policy Configuration)

[REQ-PAW-132] Audit Policy: Account Management Auditing for PAWs

Target Scope * **Applicable Systems**: Privileged Access Workstations (PAWs) * **Operating Systems**: Windows 10/11 Enterprise

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Paths**: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` * Subcategory: `User Account Management` → `Success and Failure` * Subcategory: `Security Group Management` → `Success and Failure` * Subcategory: `Computer Account Management` → `Success and Failure` * Subcategory: `Other Account Management Events` → `Success and Failure`

Rationale Auditing account management logs security principal modifications (creations, deletions, password resets, group modifications) to detect privilege escalation attempts on domain or local administrative groups.

Legacy Impact & Compatibility * **Event Log Volume**: Ensure the local Security Event Log is sized to at least 512MB to accommodate the audit stream.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: * Subcategory: `User Account Management` → `Success and Failure` * Subcategory: `Security Group Management` → `Success and Failure` * Subcategory: `Computer Account Management` → `Success and Failure` * Subcategory: `Other Account Management Events` → `Success and Failure`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditAccountmanagement.ps1](#)] ([../implementation_scripts/Configure-PawAuditAccountmanagement.ps1](#))

```
# Configure-PawAuditAccountmanagement.ps1
Write-Host "Applying Audit Policy category: account-management..." -ForegroundColor Cyan

# Set Audit Subcategory: User Account Management
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`User Account Management`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory User Account Management to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory User Account Management. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Security Group Management
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Security Group Management`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Security Group Management to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Security Group Management. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Computer Account Management
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Computer Account Management`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Computer Account Management to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Computer Account Management. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Other Account Management Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Other Account Management Events`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Other Account Management Events to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Other Account Management Events. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-PawAuditAccountmanagementStatus.ps1](#)

```
# Get-PawAuditAccountmanagementStatus.ps1
$script:Vulnerable = $false

# Audit Subcategory: User Account Management
$RawOutput = auditpol.exe /get /subcategory:"User Account Management" /r
if ($RawOutput -notmatch ",User Account Management,.*, (Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Security Group Management
$RawOutput = auditpol.exe /get /subcategory:"Security Group Management" /r
if ($RawOutput -notmatch ",Security Group Management,.*, (Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Computer Account Management
$RawOutput = auditpol.exe /get /subcategory:"Computer Account Management" /r
if ($RawOutput -notmatch ",Computer Account Management,.*, (Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Other Account Management Events
$RawOutput = auditpol.exe /get /subcategory:"Other Account Management Events" /r
if ($RawOutput -notmatch ",Other Account Management Events,.*, (Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

if ($script:Vulnerable) {
    Write-Output "Non-Compliant"
    exit 1
} else {
    Write-Output "Compliant"
    exit 0
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Client auditing baselines * **CIS Windows 10/11 Benchmark***: Section 17 (Advanced Audit Policy Configuration)

[REQ-PAW-133] Audit Policy: Detailed Tracking Auditing for PAWs

Target Scope * **Applicable Systems**: Privileged Access Workstations (PAWs) * **Operating Systems**: Windows 10/11 Enterprise

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Paths**: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` * Subcategory: `Process Creation` → `Success and Failure` * Subcategory: `DPAPI Activity` → `Success and Failure` * Subcategory: `PNP Activity` → `Success`

Rationale Detailed tracking records process creations and device arrivals to ensure EDR/SIEM visibility into executable command lines and hardware plug events.

Legacy Impact & Compatibility * **Event Log Volume**: Ensure the local Security Event Log is sized to at least 512MB to accommodate the audit stream.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: * Subcategory: `Process Creation` → `Success and Failure` * Subcategory: `DPAPI Activity` → `Success and Failure` * Subcategory: `PNP Activity` → `Success`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditDetailedtracking.ps1](#)] ([./implementation_scripts/Configure-PawAuditDetailedtracking.ps1](#))

```
# Configure-PawAuditDetailedtracking.ps1
Write-Host "Applying Audit Policy category: detailed-tracking..." -ForegroundColor Cyan

# Set Audit Subcategory: Process Creation
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Process Creation`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Process Creation to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Process Creation. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: DPAPI Activity
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`DPAPI Activity`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory DPAPI Activity to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory DPAPI Activity. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: PNP Activity
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`PNP Activity`" /success:enable /failure:disable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory PNP Activity to Success" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory PNP Activity. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-PawAuditDetailedtrackingStatus.ps1](#)

```
# Get-PawAuditDetailedtrackingStatus.ps1
$script:Vulnerable = $false

# Audit Subcategory: Process Creation
$RawOutput = auditpol.exe /get /subcategory:"Process Creation" /r
if ($RawOutput -notmatch ",Process Creation,.*(Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: DPAPI Activity
$RawOutput = auditpol.exe /get /subcategory:"DPAPI Activity" /r
if ($RawOutput -notmatch ",DPAPI Activity,.*(Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: PNP Activity
$RawOutput = auditpol.exe /get /subcategory:"PNP Activity" /r
if ($RawOutput -notmatch ",PNP Activity,.*,Success") {
    $script:Vulnerable = $true
}

if ($script:Vulnerable) {
    Write-Output "Non-Compliant"
    exit 1
} else {
    Write-Output "Compliant"
    exit 0
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Client auditing baselines * **CIS Windows 10/11 Benchmark***: Section 17 (Advanced Audit Policy Configuration)

[REQ-PAW-135] Audit Policy: Logon and Logoff Auditing for PAWs

Target Scope * **Applicable Systems**: Privileged Access Workstations (PAWs) * **Operating Systems**: Windows 10/11 Enterprise

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Paths**: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` * Subcategory: `Logon` → `Success and Failure` * Subcategory: `Logoff` → `Success` * Subcategory: `Special Logon` → `Success and Failure` * Subcategory: `Account Lockout` → `Success and Failure` * Subcategory: `Other Logon/Logoff Events` → `Success and Failure`

Rationale Auditing logon/logoff events monitors administrative session states, special elevations, and failed logon attempts, which is critical for finding unauthorized remote access or lateral movement.

Legacy Impact & Compatibility * **Event Log Volume**: Ensure the local Security Event Log is sized to at least 512MB to accommodate the audit stream.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: * Subcategory: `Logon` → `Success and Failure` * Subcategory: `Logoff` → `Success` * Subcategory: `Special Logon` → `Success and Failure` * Subcategory: `Account Lockout` → `Success and Failure` * Subcategory: `Other Logon/Logoff Events` → `Success and Failure`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditLogonlogoff.ps1](#)]

(../implementation_scripts/Configure-PawAuditLogonlogoff.ps1)

```
# Configure-PawAuditLogonLogoff.ps1
Write-Host "Applying Audit Policy category: logon-logoff..." -ForegroundColor Cyan

# Set Audit Subcategory: Logon
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Logon`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Logon to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Logon. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Logoff
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Logoff`" /success:enable /failure:disable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Logoff to Success" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Logoff. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Special Logon
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Special Logon`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Special Logon to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Special Logon. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Account Lockout
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Account Lockout`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Account Lockout to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Account Lockout. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Other Logon/Logoff Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Other Logon/Logoff Events`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Other Logon/Logoff Events to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Other Logon/Logoff Events. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-PawAuditLogonlogoffStatus.ps1](#)

```
# Get-PawAuditLogonLogoffStatus.ps1
$script:Vulnerable = $false

# Audit Subcategory: Logon
$RawOutput = auditpol.exe /get /subcategory:"Logon" /r
if ($RawOutput -notmatch ",Logon,.*, (Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Logoff
$RawOutput = auditpol.exe /get /subcategory:"Logoff" /r
if ($RawOutput -notmatch ",Logoff,.*, Success") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Special Logon
$RawOutput = auditpol.exe /get /subcategory:"Special Logon" /r
if ($RawOutput -notmatch ",Special Logon,.*, (Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Account Lockout
$RawOutput = auditpol.exe /get /subcategory:"Account Lockout" /r
if ($RawOutput -notmatch ",Account Lockout,.*, (Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Other Logon/Logoff Events
$RawOutput = auditpol.exe /get /subcategory:"Other Logon/Logoff Events" /r
if ($RawOutput -notmatch ",Other Logon/Logoff Events,.*, (Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

if ($script:Vulnerable) {
    Write-Output "Non-Compliant"
    exit 1
} else {
    Write-Output "Compliant"
    exit 0
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Client auditing baselines * **CIS Windows 10/11 Benchmark**: Section 17 (Advanced Audit Policy Configuration)

[REQ-PAW-136] Audit Policy: Object Access Auditing for PAWs

Target Scope * **Applicable Systems**: Privileged Access Workstations (PAWs) * **Operating Systems**: Windows 10/11 Enterprise

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Paths**: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` * Subcategory: `Handle Manipulation` → `Success and Failure` * Subcategory: `Registry` → `Success and Failure` * Subcategory: `File Share` → `Success and Failure` * Subcategory: `Detailed File Share` → `Failure` * Subcategory: `Other Object Access Events` → `Success and Failure`

Rationale Auditing object access (files, registry keys, and shares) helps monitor unauthorized modifications to system configuration files and access to restricted shares.

Legacy Impact & Compatibility * **Event Log Volume**: Ensure the local Security Event Log is sized to at least 512MB to accommodate the audit stream.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: * Subcategory: `Handle Manipulation` → `Success and Failure` * Subcategory: `Registry` → `Success and Failure` * Subcategory: `File Share` → `Success and Failure` * Subcategory: `Detailed File Share` → `Failure` * Subcategory: `Other Object Access Events` → `Success and Failure`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditObjectaccess.ps1](#)]

(../implementation_scripts/Configure-PawAuditObjectaccess.ps1)

```
# Configure-PawAuditObjectaccess.ps1
Write-Host "Applying Audit Policy category: object-access..." -ForegroundColor Cyan

# Set Audit Subcategory: Handle Manipulation
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Handle Manipulation`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Handle Manipulation to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Handle Manipulation. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Registry
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Registry`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Registry to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Registry. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: File Share
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"File Share`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory File Share to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory File Share. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Detailed File Share
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Detailed File Share`" /success:disable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Detailed File Share to Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Detailed File Share. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Other Object Access Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Other Object Access Events`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Other Object Access Events to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Other Object Access Events. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-PawAuditObjectaccessStatus.ps1](#)

```

# Get-PawAuditObjectaccessStatus.ps1
$script:Vulnerable = $false

# Audit Subcategory: Handle Manipulation
$RawOutput = auditpol.exe /get /subcategory:"Handle Manipulation" /r
if ($RawOutput -notmatch ",Handle Manipulation,.*(Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Registry
$RawOutput = auditpol.exe /get /subcategory:"Registry" /r
if ($RawOutput -notmatch ",Registry,.*(Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: File Share
$RawOutput = auditpol.exe /get /subcategory:"File Share" /r
if ($RawOutput -notmatch ",File Share,.*(Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Detailed File Share
$RawOutput = auditpol.exe /get /subcategory:"Detailed File Share" /r
if ($RawOutput -notmatch ",Detailed File Share,.*,Failure") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Other Object Access Events
$RawOutput = auditpol.exe /get /subcategory:"Other Object Access Events" /r
if ($RawOutput -notmatch ",Other Object Access Events,.*(Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

if ($script:Vulnerable) {
    Write-Output "Non-Compliant"
    exit 1
} else {
    Write-Output "Compliant"
    exit 0
}

```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Client auditing baselines * **CIS Windows 10/11 Benchmark***: Section 17 (Advanced Audit Policy Configuration)

[REQ-PAW-137] Audit Policy: Policy Change Auditing for PAWs

Target Scope * **Applicable Systems**: Privileged Access Workstations (PAWs) * **Operating Systems**: Windows 10/11 Enterprise

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Paths**: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` * Subcategory: `Policy Change` → `Success and Failure` * Subcategory: `Authentication Policy Change` → `Success` * Subcategory: `Authorization Policy Change` → `Success` * Subcategory: `MPSSVC Rule-Level Policy Change` → `Success and Failure` * Subcategory: `Other Policy Change Events` → `Failure`

Rationale Auditing policy changes tracks attempts to modify authorization policies, auditing configuration changes, or firewall rule alterations to hide adversarial tracks.

Legacy Impact & Compatibility * **Event Log Volume**: Ensure the local Security Event Log is sized to at least 512MB to accommodate the audit stream.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: * Subcategory: `Policy Change` → `Success and Failure` * Subcategory: `Authentication Policy Change` → `Success` * Subcategory: `Authorization Policy Change` → `Success` * Subcategory: `MPSSVC Rule-Level Policy Change` → `Success and Failure` * Subcategory: `Other Policy Change Events` → `Failure`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditPolicychange.ps1](#)]

(../implementation_scripts/Configure-PawAuditPolycychange.ps1)

```
# Configure-PawAuditPolycychange.ps1
Write-Host "Applying Audit Policy category: policy-change..." -ForegroundColor Cyan

# Set Audit Subcategory: Policy Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Policy Change`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Policy Change to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Policy Change. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Authentication Policy Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Authentication Policy Change`" /success:enable /failure:disable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Authentication Policy Change to Success" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Authentication Policy Change. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Authorization Policy Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Authorization Policy Change`" /success:enable /failure:disable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Authorization Policy Change to Success" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Authorization Policy Change. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: MPSSVC Rule-Level Policy Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`MPSSVC Rule-Level Policy Change`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory MPSSVC Rule-Level Policy Change to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory MPSSVC Rule-Level Policy Change. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Other Policy Change Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Other Policy Change Events`" /success:disable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Other Policy Change Events to Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Other Policy Change Events. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-PawAuditPolycychangeStatus.ps1](#)

```

# Get-PawAuditPolicychangeStatus.ps1
$script:Vulnerable = $false

# Audit Subcategory: Policy Change
$RawOutput = auditpol.exe /get /subcategory:"Policy Change" /r
if ($RawOutput -notmatch ",Policy Change,.*(Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Authentication Policy Change
$RawOutput = auditpol.exe /get /subcategory:"Authentication Policy Change" /r
if ($RawOutput -notmatch ",Authentication Policy Change,.*,Success") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Authorization Policy Change
$RawOutput = auditpol.exe /get /subcategory:"Authorization Policy Change" /r
if ($RawOutput -notmatch ",Authorization Policy Change,.*,Success") {
    $script:Vulnerable = $true
}

# Audit Subcategory: MPSSVC Rule-Level Policy Change
$RawOutput = auditpol.exe /get /subcategory:"MPSSVC Rule-Level Policy Change" /r
if ($RawOutput -notmatch ",MPSSVC Rule-Level Policy Change,.*(Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Other Policy Change Events
$RawOutput = auditpol.exe /get /subcategory:"Other Policy Change Events" /r
if ($RawOutput -notmatch ",Other Policy Change Events,.*,Failure") {
    $script:Vulnerable = $true
}

if ($script:Vulnerable) {
    Write-Output "Non-Compliant"
    exit 1
} else {
    Write-Output "Compliant"
    exit 0
}

```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Client auditing baselines * **CIS Windows 10/11 Benchmark***: Section 17 (Advanced Audit Policy Configuration)

[REQ-PAW-138] Audit Policy: Privilege Use Auditing for PAWs

Target Scope * **Applicable Systems**: Privileged Access Workstations (PAWs) * **Operating Systems**: Windows 10/11 Enterprise

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Paths**: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` * Subcategory: `Sensitive Privilege Use` → `Success and Failure`

Rationale Auditing sensitive privilege use logs attempts by processes or users to exercise rights like ActAsPartOfTypeOperatingSystem or LoadDrivers, identifying potential privilege escalations.

Legacy Impact & Compatibility * **Event Log Volume**: Ensure the local Security Event Log is sized to at least 512MB to accommodate the audit stream.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: * Subcategory: `Sensitive Privilege Use` → `Success and Failure`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditPrivilegeuse.ps1](#)] ([../implementation_scripts/Configure-PawAuditPrivilegeuse.ps1](#))

```
# Configure-PawAuditPrivilegeuse.ps1
Write-Host "Applying Audit Policy category: privilege-use..." -ForegroundColor Cyan

# Set Audit Subcategory: Sensitive Privilege Use
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Sensitive Privilege Use`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Sensitive Privilege Use to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Sensitive Privilege Use. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-PawAuditPrivilegeuseStatus.ps1](#)

```
# Get-PawAuditPrivilegeuseStatus.ps1
$script:Vulnerable = $false

# Audit Subcategory: Sensitive Privilege Use
$RawOutput = auditpol.exe /get /subcategory:"Sensitive Privilege Use" /r
if ($RawOutput -notmatch ",Sensitive Privilege Use,.*, (Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

if ($script:Vulnerable) {
    Write-Output "Non-Compliant"
    exit 1
} else {
    Write-Output "Compliant"
    exit 0
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide**: Client auditing baselines * **CIS Windows 10/11 Benchmark**: Section 17 (Advanced Audit Policy Configuration)

[REQ-PAW-139] Audit Policy: System Events Auditing for PAWs

Target Scope * **Applicable Systems**: Privileged Access Workstations (PAWs) * **Operating Systems**: Windows 10/11 Enterprise

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Paths**: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` * Subcategory: `IPsec Driver` → `Success and Failure` * Subcategory: `Other System Events` → `Success and Failure` * Subcategory: `Security State Change` → `Success and Failure` * Subcategory: `Security System Extension` → `Success and Failure` * Subcategory: `System Integrity` → `Success and Failure`

Rationale Auditing system security extensions, integrity violations, and driver arrivals monitors boot health and tampering of host security services.

Legacy Impact & Compatibility * **Event Log Volume**: Ensure the local Security Event Log is sized to at least 512MB to accommodate the audit stream.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: * Subcategory: `IPsec Driver` → `Success and Failure` * Subcategory: `Other System Events` → `Success and Failure` * Subcategory: `Security State Change` → `Success and Failure` * Subcategory: `Security System Extension` → `Success and Failure` * Subcategory: `System Integrity` → `Success and Failure`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-PawAuditSystemevents.ps1](#)]

(../implementation_scripts/Configure-PawAuditSystemevents.ps1)

```
# Configure-PawAuditSystemevents.ps1
Write-Host "Applying Audit Policy category: system-events..." -ForegroundColor Cyan

# Set Audit Subcategory: IPsec Driver
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`IPsec Driver`" /success:enable /failure:enable" -Wait -NoNewWin
dow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory IPsec Driver to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory IPsec Driver. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Other System Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Other System Events`" /success:enable /failure:enable" -Wait -N
oNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Other System Events to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Other System Events. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Security State Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Security State Change`" /success:enable /failure:enable" -Wait
-NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Security State Change to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Security State Change. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: Security System Extension
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Security System Extension`" /success:enable /failure:enable" -W
ait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory Security System Extension to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory Security System Extension. Exit Code: $($Process.ExitCode)"
}

# Set Audit Subcategory: System Integrity
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`System Integrity`" /success:enable /failure:enable" -Wait -NoNe
wWindow -PassThru
if ($Process.ExitCode -eq 0) {
    Write-Host "    Enforced subcategory System Integrity to Success and Failure" -ForegroundColor Green
} else {
    Write-Error "    Failed to configure subcategory System Integrity. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-PawAuditSystemeventsStatus.ps1](#)

```
# Get-PawAuditSystemeventsStatus.ps1
$script:Vulnerable = $false

# Audit Subcategory: IPsec Driver
$RawOutput = auditpol.exe /get /subcategory:"IPsec Driver" /r
if ($RawOutput -notmatch ",IPsec Driver,.*(Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Other System Events
$RawOutput = auditpol.exe /get /subcategory:"Other System Events" /r
if ($RawOutput -notmatch ",Other System Events,.*(Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Security State Change
$RawOutput = auditpol.exe /get /subcategory:"Security State Change" /r
if ($RawOutput -notmatch ",Security State Change,.*(Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: Security System Extension
$RawOutput = auditpol.exe /get /subcategory:"Security System Extension" /r
if ($RawOutput -notmatch ",Security System Extension,.*(Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

# Audit Subcategory: System Integrity
$RawOutput = auditpol.exe /get /subcategory:"System Integrity" /r
if ($RawOutput -notmatch ",System Integrity,.*(Success and Failure|Success & Failure)") {
    $script:Vulnerable = $true
}

if ($script:Vulnerable) {
    Write-Output "Non-Compliant"
    exit 1
} else {
    Write-Output "Compliant"
    exit 0
}
```

Sources & Compliance References * **ANSSI Active Directory Hardening Guide***: Client auditing baselines * **CIS Windows 10/11 Benchmark***: Section 17 (Advanced Audit Policy Configuration)

Module 8: Endpoint Hardening

This directory defines the technical security baselines for standard client workstations (Tier 2 endpoints) operating in isolated, air-gapped domains.

To prevent initial access and lateral movement, the following unitary technical hardening controls must be implemented:

Technical Hardening Controls

1. [REQ-END-001 - Harden Network Parameters and Disable Legacy Name Resolution](#) Disables Link-Local Multicast Name Resolution (LLMNR), NetBIOS over TCP/IP, and mDNS, and secures TCP/IP parameters to prevent local credential harvesting and protocol exploits.
2. [REQ-END-002 - Configure User Account Control Policies](#) Enforces maximum UAC security behavior, requiring credential entry on the secure desktop for administrators and automatically denying elevation prompts for standard users.
3. [REQ-END-003 - Disable AutoPlay and AutoRun](#) Turns off AutoPlay and AutoRun features across all drive types to prevent automatic execution of files and payloads from external media.
4. [REQ-END-004 - Block Removable Storage](#) Blocks read and write access to USB drives and other removable media classes to mitigate data leakage and malware propagation.
5. [REQ-END-005 - Restrict Remote Desktop Access](#) Blocks incoming RDP connections to standard workstations by default, or restricts allowed connection sources to authorized administrative subnets with Network Level Authentication (NLA) enabled.
6. [REQ-END-006 - Restrict Local Administrators Group](#) Locks down local workstation administrative privileges, removing standard domain users and enforcing administrative segregation utilizing LAPS.
7. [REQ-END-007 - Windows Defender Antivirus Baseline and Exploit Guard](#) Configures Windows Defender Antivirus, enabling real-time scanning, behavioral monitoring, preventing local exclusion modifications, enforcing Attack Surface Reduction (ASR) rules, activating Tamper Protection, and enabling AppContainer sandbox isolation.
 - [REQ-END-057 - Disable Real-Time Monitoring and Behavior Monitoring Override](#)
 - [REQ-END-058 - Configure Potentially Unwanted Applications \(PUA\) Protection](#)
 - [REQ-END-059 - Prevent Local List Merging and Exclusions Configuration](#)
 - [REQ-END-060 - Configure Auto Exclusions Configuration](#)
 - [REQ-END-061 - Prevent MAPS Local Setting Override](#)
 - [REQ-END-062 - Enable EDR in Block Mode](#)
 - [REQ-END-063 - Allow Network Protection on Windows Server](#)
 - [REQ-END-064 - Enable File Hash Computation](#)
 - [REQ-END-065 - Configure Network Inspection System \(NIS\) settings](#)
 - [REQ-END-066 - Configure OOBE Real-Time Protection and Security Intelligence](#)
 - [REQ-END-067 - Enable Dynamic Signature Dropped Event Reporting](#)
 - [REQ-END-068 - Configure Quick Scan and Scanning Exclusions](#)
 - [REQ-END-069 - Configure Scheduled Scan Parameters](#)
 - [REQ-END-070 - Configure Security Intelligence Update Schedule](#)
 - [REQ-END-071 - Configure Attack Surface Reduction Rules](#)
 - [REQ-END-080 - ASR: Block abuse of exploited vulnerable signed drivers](#)
 - [REQ-END-081 - ASR: Block Adobe Reader from creating child processes](#)
 - [REQ-END-082 - ASR: Block all Office applications from creating child processes](#)
 - [REQ-END-083 - ASR: Block credential stealing from the Windows local security authority subsystem](#)
 - [REQ-END-084 - ASR: Block executable content from email client and webmail](#)
 - [REQ-END-085 - ASR: Block executable files from running unless they meet a prevalence, age, or trusted list criterion](#)
 - [REQ-END-086 - ASR: Block execution of potentially obfuscated scripts](#)
 - [REQ-END-087 - ASR: Block JavaScript or VBScript from launching downloaded executable content](#)
 - [REQ-END-088 - ASR: Block Office applications from creating executable content](#)
 - [REQ-END-089 - ASR: Block Office applications from injecting code into other processes](#)
 - [REQ-END-090 - ASR: Block Office communication application from creating child processes](#)
 - [REQ-END-091 - ASR: Block persistence through WMI event subscription](#)
 - [REQ-END-092 - ASR: Block process creations originating from PSEXEC and WMI commands](#)
 - [REQ-END-093 - ASR: Block untrusted and unsigned processes that run from USB](#)

- [REQ-END-094 - ASR: Block Win32 API calls from Office macros](#)
 - [REQ-END-095 - ASR: Use advanced protection against ransomware](#)
 - [REQ-END-072 - Configure Threat Severity Default Quarantine Actions](#)
 - [REQ-END-073 - Configure Family Options UI Lockdown](#)
 - [REQ-END-074 - Configure Tamper Protection](#)
 - [REQ-END-075 - Configure Sandbox Execution Environment](#)
 - [REQ-END-076 - Configure AMSI Authenticode Signature Verification](#)
 - [REQ-END-077 - Configure File Explorer SmartScreen](#)
 - [REQ-END-078 - Disable OneDrive File Sync](#)
 - [REQ-END-079 - Enforce Antivirus Scan on Opening Attachments](#)
8. [REQ-END-008 - WSUS Client Configuration](#) Enforces update client registry baselines to ensure workstations pull OS patches and security signatures exclusively from the local, offline WSUS server.
 9. [REQ-END-009 - Enable UEFI Secure Boot](#) Mandates hardware-rooted platform integrity checks, verifying that UEFI Secure Boot is active on the operating system.
 10. [REQ-END-010 - Enable VBS and Credential Guard](#) Activates Virtualization-Based Security (VBS) and Credential Guard to protect password hashes and Kerberos tickets in an isolated virtual container, mitigating LSASS dumping.
 11. [REQ-END-011 - Configure Windows Defender Application Control](#) Deploys application control baselines and the Microsoft Vulnerable Driver Blocklist to enforce code integrity policies, restricting the system to run only signed, authorized binaries, scripts, and secure drivers.
 12. [REQ-END-012 - Enable BitLocker and Network Unlock](#) Enforces full disk encryption with TPM and enables secure Network Unlock capabilities for standard client workstations.
 13. [REQ-END-013 - UEFI Firmware Security Hardening](#) Enforces password protection, disables Compatibility Support Module (CSM)/Legacy Boot, locks boot order, and configures secure firmware update policies.
 14. [REQ-END-014 - Enable Hardware Virtualization and DMA Protection](#) Enables CPU virtualization (VT-x/AMD-V) and IOMMU (VT-d/AMD-Vi) to provide the hardware-rooted platform integrity required for VBS and Kernel DMA protection.
 15. [REQ-END-015 - Disable Windows Platform Binary Table \(WPBT\)](#) Disables execution of binaries supplied by the Windows Platform Binary Table (WPBT) ACPI firmware table to mitigate boot-level security bypasses.
 16. [REQ-END-016 - Configure User Rights Assignments](#) Restricts critical user rights assignments (URAs) such as debugging programs, token impersonation, and local logon permissions on standard client endpoints.
 - [REQ-END-096 - Configure User Rights: Access Credential Manager as a trusted caller](#)
 - [REQ-END-097 - Configure User Rights: Access this computer from the network](#)
 - [REQ-END-098 - Configure User Rights: Act as part of the operating system](#)
 - [REQ-END-099 - Configure User Rights: Allow log on locally](#)
 - [REQ-END-100 - Configure User Rights: Back up files and directories](#)
 - [REQ-END-101 - Configure User Rights: Change the system time](#)
 - [REQ-END-102 - Configure User Rights: Change the time zone](#)
 - [REQ-END-103 - Configure User Rights: Create a pagefile](#)
 - [REQ-END-104 - Configure User Rights: Create a token object](#)
 - [REQ-END-105 - Configure User Rights: Create global objects](#)
 - [REQ-END-106 - Configure User Rights: Create permanent shared objects](#)
 - [REQ-END-107 - Configure User Rights: Create symbolic links](#)
 - [REQ-END-108 - Configure User Rights: Debug programs](#)
 - [REQ-END-109 - Configure User Rights: Enable computer and user accounts to be trusted for delegation](#)
 - [REQ-END-110 - Configure User Rights: Force shutdown from a remote system](#)
 - [REQ-END-111 - Configure User Rights: Impersonate a client after authentication](#)
 - [REQ-END-112 - Configure User Rights: Increase scheduling priority](#)
 - [REQ-END-113 - Configure User Rights: Load and unload device drivers](#)
 - [REQ-END-114 - Configure User Rights: Lock pages in memory](#)
 - [REQ-END-115 - Configure User Rights: Manage auditing and security log](#)

- [REQ-END-116 - Configure User Rights: Modify firmware environment values](#)
 - [REQ-END-117 - Configure User Rights: Perform volume maintenance tasks](#)
 - [REQ-END-118 - Configure User Rights: Profile single process](#)
 - [REQ-END-119 - Configure User Rights: Profile system performance](#)
 - [REQ-END-120 - Configure User Rights: Replace a process level token](#)
 - [REQ-END-121 - Configure User Rights: Restore files and directories](#)
 - [REQ-END-122 - Configure User Rights: Take ownership of files or other objects](#)
 - [REQ-END-123 - Configure User Rights: Modify an object label](#)
 - [REQ-END-124 - Configure User Rights: Deny access to this computer from the network](#)
 - [REQ-END-125 - Configure User Rights: Deny log on through Remote Desktop Services](#)
17. [REQ-END-017 - Harden DMA and Physical Security](#) Mitigates physical access threat vectors by disabling standby sleep states (S1-S3), disabling external DMA device enumeration under lock, blocking legacy SBP-2 device classes, and denying write access to removable drives without BitLocker protection.
18. [REQ-END-018 - Configure Account and Password Policies](#) Enforces local and domain-wide account settings, including account lockout thresholds, lockout observation windows, smart card removal actions, and disabling reversible password encryption.
19. [REQ-END-019 - Configure User Profile Restrictions](#) Locks down user profile registry settings (HKCU) to disable toast notifications on the lock screen and block third-party application suggestions.
- [REQ-END-126 - User Profile: Toast Notifications Lock Screen Restrictions](#)
 - [REQ-END-127 - User Profile: Spotlight and Consumer Features Restrictions](#)
 - [REQ-END-128 - User Profile: Windows Copilot Restrictions](#)
 - [REQ-END-129 - User Profile: In-Place Sharing Restrictions](#)
 - [REQ-END-130 - User Profile: Shell RunAs User Suppression](#)
 - [REQ-END-131 - User Profile: Personalization and Privacy Restrictions](#)
 - [REQ-END-132 - User Profile: Group Policy Processing Behaviors](#)
 - [REQ-END-133 - User Profile: Telemetry and Inventory Collection Restrictions](#)
 - [REQ-END-134 - User Profile: Explorer Security and Memory Protections](#)
 - [REQ-END-135 - User Profile: Internet Explorer Options and Feeds Restrictions](#)
 - [REQ-END-136 - User Profile: Interactive Logon Warning Banners](#)
 - [REQ-END-137 - User Profile: Interactive Logon Inactivity Timeout](#)
 - [REQ-END-138 - User Profile: Windows Installer Hardening](#)
 - [REQ-END-139 - User Profile: Secondary Logon Service Lockdown](#)
 - [REQ-END-151 - User Profile: Structured Exception Handling Overwrite Protection \(SEOP\) for Endpoints](#)
 - [REQ-END-152 - User Profile: Directory Protection Mode for Endpoints](#)
 - [REQ-END-153 - User Profile: Address Space Layout Randomization \(ASLR\) Image Relocation for Endpoints](#)
 - [REQ-END-154 - User Profile: Speculative Execution Mitigations \(Spectre/Meltdown\) for Endpoints](#)
 - [REQ-END-155 - User Profile: Authenticode Certificate Padding Check for Endpoints](#)
 - [REQ-END-156 - User Profile: Command Processor Batch File Locking for Endpoints](#)
 - [REQ-END-157 - User Profile: Time-Travel Debugging \(TTD\) Recording Policy for Endpoints](#)
 - [REQ-END-158 - User Profile: Trusted Root Store Protected Roots Certificate Restriction for Endpoints](#)
 - [REQ-END-159 - User Profile: Disabling Injection of Applnit DLLs for Endpoints](#)
 - [REQ-END-160 - User Profile: Preservation of Attachment Zone Information for Endpoints](#)
 - [REQ-END-161 - User Profile: Disable Windows Game DVR for Endpoints](#)
 - [REQ-END-162 - User Profile: Restrict Windows Ink Workspace on Lock Screen for Endpoints](#)
20. [REQ-END-020 - Configure Exploit Protection Profile](#) Configures and enforces a system-wide Microsoft Defender Exploit Protection profile to apply advanced memory mitigations (DEP, ASLR, CFG, SEHOP, Heap Integrity) on all endpoints.
21. [REQ-END-021 - Restrict Safe Mode Access to Administrators](#) Prevents standard (non-administrative) users from logging into the system while in Safe Mode by setting SafeModeBlockNonAdmins to 1.
22. [REQ-END-022 - Configure Windows Defender Firewall and Block LOLBins](#) Configures Domain, Private, and Public firewall profile states, logging, and notifications, and enforces outbound rules to block known Living Off the Land Binaries (LOLBins) from initiating outgoing

network connections.

23. [REQ-END-023 - Enable LSA Protection with UEFI Lock](#) Configures the LSA Protection setting to run the LSASS process as a Protected Process Light (PPL) with UEFI Lock, preventing credential harvesting from LSASS memory.
24. [REQ-END-024 - Disable Unnecessary System Services](#) Disables unnecessary and high-risk system services to minimize the attack surface of standard client endpoints and member servers.
 - [REQ-END-037 - Disable Computer Browser Service \(Browser\)](#)
 - [REQ-END-038 - Disable Infrared Monitor Service \(irmon\)](#)
 - [REQ-END-039 - Disable Internet Connection Sharing \(ICS\) Service \(SharedAccess\)](#)
 - [REQ-END-040 - Disable LxssManager Service \(LxssManager\)](#)
 - [REQ-END-041 - Disable Microsoft FTP Service \(FTPSVC\)](#)
 - [REQ-END-042 - Disable OpenSSH SSH Server Service \(sshd\)](#)
 - [REQ-END-043 - Disable Remote Procedure Call \(RPC\) Locator Service \(RpcLocator\)](#)
 - [REQ-END-044 - Disable Routing and Remote Access Service \(RemoteAccess\)](#)
 - [REQ-END-045 - Disable Simple TCP/IP Services \(simptcp\)](#)
 - [REQ-END-046 - Disable Special Administration Console Helper Service \(sacsvr\)](#)
 - [REQ-END-047 - Disable SSDP Discovery Service \(SSDPSRV\)](#)
 - [REQ-END-048 - Disable UPnP Device Host Service \(upnphost\)](#)
 - [REQ-END-049 - Disable Web Management Service \(WMSvc\)](#)
 - [REQ-END-050 - Disable Windows Media Player Network Sharing Service \(WMPNetworkSvc\)](#)
 - [REQ-END-051 - Disable Windows Mobile Hotspot Service \(icssvc\)](#)
 - [REQ-END-052 - Disable World Wide Web Publishing Service \(W3SVC\)](#)
 - [REQ-END-053 - Disable Xbox Accessory Management Service \(XboxGipSvc\)](#)
 - [REQ-END-054 - Disable Xbox Live Auth Manager Service \(XblAuthManager\)](#)
 - [REQ-END-055 - Disable Xbox Live Game Save Service \(XblGameSave\)](#)
 - [REQ-END-056 - Disable Xbox Live Networking Service \(XboxNetApiSvc\)](#)
25. [REQ-END-025 - Configure Secure Printing and Print Spooler Policies](#) Configures printing security, RPC over TCP communication, Point and Print restrictions, and Redirection Guard, and disables incoming print spooler connections.
26. [REQ-END-026 - Configure System Administrative Templates](#) Enforces 91 administrative template settings including SMBv1 driver blocks, event log size extensions, and Windows Update scheduling.
27. [REQ-END-027 - Configure AppLocker Policies](#) Deploys AppLocker application control policies to restrict unauthorized software and script execution, and prevents default AppLocker bypasses.
28. [REQ-END-028 - Configure Early Launch Antimalware \(ELAM\) Policy](#) Configures the Early Launch Antimalware (ELAM) driver initialization policy to ensure only signed, trusted boot drivers execute.
29. [REQ-END-029 - Configure Untrusted Font Blocking](#) Configures the Untrusted Font Blocking mitigation to prevent loading of fonts outside the system fonts directory.
30. [REQ-END-030 - Configure svchost.exe Mitigation Options](#) Configures svchost.exe mitigation options on Tier 2 client workstations to restrict binary loading to Microsoft-signed code and block dynamic code execution.
31. [REQ-END-031 - Enable Kernel-Mode Hardware-Enforced Stack Protection](#) Configures Kernel-mode Hardware-enforced Stack Protection to enforce hardware-backed control-flow integrity and mitigate kernel Return-Oriented Programming (ROP) execution hijacks.
32. [REQ-END-032 - Disable Unused Windows Features and PowerShell 2.0 Engine](#) Disables legacy, unused Windows optional features, including PowerShell 2.0, .NET Framework 3.5, and SMBv1 to minimize the client attack surface.
33. [REQ-END-033 - Configure Microsoft Office Security and Block OLE Packages](#) Blocks VBA macros from running in Office files downloaded from the Internet, enforces macro digital signing warnings, and disables OLE Package execution in Outlook to prevent initial access exploits.
34. [REQ-END-034 - Disable Windows Script Host and Remap Scripting Extensions](#) Disables Windows Script Host execution globally and remaps standard scripting extensions (.vbs, .js, etc.) to open in Notepad by default to prevent execution by double-click.
35. [REQ-END-035 - Configure Secure Boot Revocations and Bootloader Updates](#) Configures and enforces BlackLotus revocation updates and bootloader integrity verification policy variables in system firmware.

36. [REQ-END-036 - Enable WDAC Driver Blocklist](#) Enforces the Microsoft Vulnerable Driver Blocklist via Windows Defender Application Control (WDAC) to prevent known vulnerable or malicious drivers from loading in kernel space, mitigating Bring Your Own Vulnerable Driver (BYOVD) attacks.
37. [REQ-END-140 - Configure Advanced Security Audit Policies for Endpoints](#) Enforces granular Windows security audit policies (including logons, group memberships, registry access, and system events) to log critical threat telemetry on Tier 2 client workstations.
- [REQ-END-141 - Audit Policy: Advanced Audit Policy Overrides for Endpoints](#)
 - [REQ-END-142 - Audit Policy: Account Logon Auditing for Endpoints](#)
 - [REQ-END-143 - Audit Policy: Account Management Auditing for Endpoints](#)
 - [REQ-END-144 - Audit Policy: Detailed Tracking Auditing for Endpoints](#)
 - [REQ-END-145 - Audit Policy: Logon and Logoff Auditing for Endpoints](#)
 - [REQ-END-146 - Audit Policy: Object Access Auditing for Endpoints](#)
 - [REQ-END-147 - Audit Policy: Policy Change Auditing for Endpoints](#)
 - [REQ-END-148 - Audit Policy: Privilege Use Auditing for Endpoints](#)
 - [REQ-END-149 - Audit Policy: System Events Auditing for Endpoints](#)

[REQ-END-001] Harden Network Parameters and Disable Legacy Name Resolution

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Paths**: * Computer Configuration\Administrative Templates\Network\DNS Client\Turn off Multicast Name Resolution * Computer Configuration\Administrative Templates\Network\DNS Client\Turn off default IPv6 DNS Servers * Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security * Computer Configuration\Administrative Templates\Network\Network Connections * Computer Configuration\Administrative Templates\Network\Windows Connection Manager * Computer Configuration\Administrative Templates\Network\WLAN Service\WLAN Settings * Computer Configuration\Administrative Templates\Printers * Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options * Computer Configuration\Policies\Windows Settings\Security Settings\System Services\WinHTTP Web Proxy Auto-Discovery Service * **Registry Locations**: * HKLM\Software\Policies\Microsoft\Windows NT\DNSClient * 'EnableMulticast' = '0' (REG_DWORD, Disables LLMNR) * 'EnableDNS' = '0' (REG_DWORD, Disables mDNS) * 'DisableIPv6DefaultDnsServers' = '1' (REG_DWORD, Turn off default IPv6 DNS Servers) * HKLM\SYSTEM\CurrentControlSet\Services\Netbt\Parameters * 'NoNameReleaseOnDemand' = '1' (REG_DWORD) * 'NodeType' = '2' (REG_DWORD) * HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters * 'EnableICMPRedirect' = '0' (REG_DWORD) * 'DisableIPSourceRouting' = '2' (REG_DWORD) * HKLM\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters * 'DisableIPSourceRouting' = '2' (REG_DWORD) * HKLM\SOFTWARE\Policies\Microsoft\Windows\Network Connections * 'NC_ShowSharedAccessUI' = '0' (REG_DWORD) * 'NC_AllowNetBridge_NLA' = '0' (REG_DWORD, Prohibit Network Bridge) * 'NC_StdDomainUserSetLocation' = '1' (REG_DWORD, Require elevation to set network location) * HKLM\SOFTWARE\Policies\Microsoft\Windows\WcmSvc\GroupPolicy * 'fMinimizeConnections' = '3' (REG_DWORD) * 'fBlockNonDomain' = '1' (REG_DWORD) * HKLM\SOFTWARE\Microsoft\wcmSvc\wifinetworkmanager\config * 'AutoConnectAllowedOEM' = '0' (REG_DWORD) * HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Printers * 'DisableWebPnPDownload' = '1' (REG_DWORD) * 'DisableHTTPPrinting' = '1' (REG_DWORD) * HKLM\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters * 'RestrictNullSessAccess' = '1' (REG_DWORD) * HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings\Wpad * 'WpadOverride' = '1' (REG_DWORD, Disables WPAD) * HKLM\SYSTEM\CurrentControlSet\Services\LanmanServer\DefaultSecurity * 'SrvsvcSessionInfo' = (REG_BINARY, Restricts Net Session Enumeration)

Rationale Legacy name resolution protocols and insecure default network configurations are heavily targeted by attackers for credential harvesting and man-in-the-middle (MitM) positioning:

1. **Legacy Name Resolution (LLMNR / NetBIOS)**: LLMNR and NBT-NS serve as fallback protocols when DNS resolution fails. When a host queries an unresolvable name, it broadcasts requests over the local subnet. An attacker can spoof responses (e.g., using Responder) to capture NTLMv2 hashes or perform authentication relay attacks. NetBIOS name release requests can be forged to disrupt local names unless protected.
 2. **NetBIOS Node Type and Name Release**: Setting the Node Type to P-node (point-to-point, value 2) disables broadcast resolution fallbacks. Enabling name release protection (`NoNameReleaseOnDemand`) prevents attackers from spoofing name release requests to deregister local names.
 3. **ICMP Redirects**: ICMP redirect packets can be used by an attacker on the same subnet to dynamically redirect routing for specific hosts through the attacker's machine, enabling full MitM packet sniffing and modification. Disabling ICMP redirects prevents this vector.
 4. **IP Source Routing**: Source routing allows a sender to specify the exact network path a packet should follow. This is commonly abused to bypass firewall routing rules or establish communication paths that violate network segment isolation.
 5. **Disable Default IPv6 DNS Servers**: Disabling default IPv6 DNS servers prevents automated fallback to unauthenticated, dynamic local IPv6 DNS servers advertised by rogue routers or malicious tools (like mitm6), which would otherwise redirect query traffic and coerce NTLM or Kerberos authentication.
 6. **Disable Web Proxy Auto-Discovery (WPAD)**: Disabling WPAD removes another name resolution mechanism that Responder exploits to harvest credentials. By disabling the `WinHttpAutoProxySvc` service and configuring `WpadOverride = 1`, the workstation is protected from rogue web proxy configurations.
 7. **Restrict Net Session Enumeration (NetCease)**: By default, any authenticated domain user can query session information from remote hosts. Attackers utilize session enumeration to locate high-privileged user sessions (e.g., Domain Admins) across the network. Hardening the `SrvsvcSessionInfo` default security descriptor blocks this remote reconnaissance.
-

Legacy Impact & Compatibility * **DNS Dependency**: Disabling LLMNR and NetBIOS requires a fully operational DNS infrastructure. Any internal local network resource names must be registered in the AD DNS zones. Ad-hoc name resolution (such as workgroup-based peer file sharing) will no longer function. * **Standby Subnets**: Disabling ICMP redirects means systems will rely on static routing tables and default gateway definitions. This is the standard operational stance for secure enterprise subnets.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Step 1: Configure DNS Client Settings 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Edit the target endpoints GPO. 3. Navigate to: 'Computer Configuration\Administrative Templates\Network\DNS Client' 4. Configure the settings: * **Policy**: 'Turn off Multicast Name Resolution' → **Enabled** * **Policy**: 'Configure multicast DNS (mDNS) protocol' → **Enabled** with option set to **Disabled** * **Policy**: 'Turn off default IPv6 DNS Servers' → **Enabled**

Step 2: Configure Network Connections Policies 1. In the target GPO, navigate to: `Computer Configuration\Administrative Templates\Network\Network Connections` 2. Configure the settings: * **Policy**: `Prohibit use of Internet Connection Sharing on your DNS domain network` → **Enabled** * **Policy**: `Prohibit installation and configuration of Network Bridge on your DNS domain network` → **Enabled** * **Policy**: `Require domain users to elevate when setting a network's location` → **Enabled**

Step 3: Configure Windows Connection Manager Policies 1. In the target GPO, navigate to: `Computer Configuration\Administrative Templates\Network\Windows Connection Manager` 2. Configure the settings: * **Policy**: `Minimize the number of simultaneous connections to the Internet or a Windows Domain` → **Enabled** with option set to *3 = Prevent Wi-Fi when on Ethernet* * **Policy**: `Prohibit connection to non-domain networks when connected to domain authenticated network` → **Enabled**

Step 4: Configure WLAN Settings (WiFi Sense) 1. In the target GPO, navigate to: `Computer Configuration\Administrative Templates\Network\WLAN Service\WLAN Settings` 2. Configure the settings: * **Policy**: `Allow Windows to automatically connect to suggested open hotspots, to networks shared by contacts, and to hotspots offering paid services` → **Disabled**

Step 5: Configure Spooler and HTTP Printing Policies 1. In the target GPO, navigate to: `Computer Configuration\Administrative Templates\System\Internet Communication Management\Internet Communication settings` 2. Configure the settings: * **Policy**: `Turn off downloading of print drivers over HTTP` → **Enabled** * **Policy**: `Turn off printing over HTTP` → **Enabled**

Step 6: Configure Network Access Security settings 1. In the target GPO, navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` 2. Configure the settings: * **Policy**: `Network access: Restrict anonymous access to Named Pipes and Shares` → **Enabled**

Step 7: Disable WinHTTP WPAD Service 1. In the target GPO, navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` 2. Scroll to **WinHTTP Web Proxy Auto-Discovery Service**, select **Define this service setting**, and set the service startup mode to **Disabled**.

Step 8: Configure Registry Network settings via GPO Preferences 1. Under the target GPO, navigate to: `Computer Configuration\Preferences\Windows Settings\Registry` 2. Right-click **Registry** and select **New → Registry Item** for each of the following:

- **NetBIOS Name Release Protection:**

- **Action:** Update
- **Hive:** HKEY_LOCAL_MACHINE
- **Key Path:** SYSTEM\CurrentControlSet\Services\Netbt\Parameters
- **Value Name:** NoNameReleaseOnDemand
- **Value Type:** REG_DWORD
- **Value Data:** 1

- **NetBIOS P-Node Type:**

- **Action:** Update
- **Hive:** HKEY_LOCAL_MACHINE
- **Key Path:** SYSTEM\CurrentControlSet\Services\Netbt\Parameters
- **Value Name:** NodeType
- **Value Type:** REG_DWORD
- **Value Data:** 2

- **Disable ICMP Redirects:**

- **Action:** Update
- **Hive:** HKEY_LOCAL_MACHINE
- **Key Path:** SYSTEM\CurrentControlSet\Services\Tcpip\Parameters
- **Value Name:** EnableICMPRedirect
- **Value Type:** REG_DWORD
- **Value Data:** 0

- **Disable IP Source Routing (IPv4):**

- **Action:** Update
- **Hive:** HKEY_LOCAL_MACHINE
- **Key Path:** SYSTEM\CurrentControlSet\Services\Tcpip\Parameters
- **Value Name:** DisableIPSourceRouting
- **Value Type:** REG_DWORD
- **Value Data:** 2

- **Disable IP Source Routing (IPv6):**

- **Action:** Update
- **Hive:** HKEY_LOCAL_MACHINE
- **Key Path:** SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters
- **Value Name:** DisableIPSourceRouting
- **Value Type:** REG_DWORD
- **Value Data:** 2

```
* **Disable WPAD Override (User Preference)**:
* **Action**: `Update`
* **Hive**: `HKEY_CURRENT_USER`
* **Key Path**: `Software\Microsoft\Windows\CurrentVersion\Internet Settings\Wpad`
* **Value Name**: `WpadOverride`
* **Value Type**: `REG_DWORD`
* **Value Data**: `1`

* **Restrict Net Session Enumeration (NetCease SDDL)**:
* **Action**: `Update`
* **Hive**: `HKEY_LOCAL_MACHINE`
* **Key Path**: `SYSTEM\CurrentControlSet\Services\LanmanServer\DefaultSecurity`
* **Value Name**: `SrvsvcSessionInfo`
* **Value Type**: `REG_BINARY`
* **Value Data**: Generate via SDDL `D:(A;;CC;;;BA)(A;;CC;;;S0)(A;;CC;;;PU)`
```

Step 9: Disable NetBIOS (via DHCP Scope Options) 1. Open the **DHCP Management Console** (`dhcpgmt.msc`). 2. Under Scope Options, select **Configure Options**. 3. Add **Option 043 (Vendor Specific Info)** and set the NetBIOS over TCP/IP value to `0x2` (Disable NetBIOS over TCP/IP).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to disable legacy resolution and enforce secure TCP/IP registry parameters.

[Download Script: Set-NetworkHardeningSettings.ps1](#)

```
# Set-NetworkHardeningSettings.ps1
# Description: Configures local registry keys to disable LLNMR/NetBIOS, harden TCP/IP stack, prevent dual-homing, block hotspot auto
-connect, print driver web downloads, HTTP printing, and limit anonymous share access.

Write-Host "Applying network and name resolution hardening..." -ForegroundColor Cyan

# Helper to configure registry keys
function Set-RegDWord {
    [CmdletBinding(SupportsShouldProcess)]
    param (
        [string]$path,
        [string]$name,
        [int]$value
    )
    if ($PSCmdlet.ShouldProcess($path, "Set registry DWORD value $name to $value")) {
        $parent = Split-Path -Path $path
        if (-not (Test-Path $parent)) {
            New-Item -Path $parent -Force | Out-Null
        }
        if (-not (Test-Path $path)) {
            New-Item -Path $path -Force | Out-Null
        }
        Set-ItemProperty -Path $path -Name $name -Value $value -Type DWord -Force
    }
}

# 1. Disable LLNMR, mDNS, and default IPv6 DNS Servers
Set-RegDWord "HKLM:\Software\Policies\Microsoft\Windows NT\DNSClient" "EnableMulticast" 0
Set-RegDWord "HKLM:\Software\Policies\Microsoft\Windows NT\DNSClient" "EnablemDNS" 0
Set-RegDWord "HKLM:\Software\Policies\Microsoft\Windows NT\DNSClient" "DisableIPv6DefaultDnsServers" 1
Write-Host "[+] LLNMR (Multicast Name Resolution), mDNS, and default IPv6 DNS Servers disabled." -ForegroundColor Green

# 2. Configure NetBIOS Parameters
$NetbtPath = "HKLM:\SYSTEM\CurrentControlSet\Services\Netbt\Parameters"
if (-not (Test-Path $NetbtPath)) {
    New-Item -Path $NetbtPath -Force | Out-Null
}
Set-ItemProperty -Path $NetbtPath -Name "NoNameReleaseOnDemand" -Value 1 -Type DWord
Set-ItemProperty -Path $NetbtPath -Name "NodeType" -Value 2 -Type DWord
Write-Host "[+] NetBIOS name release protection and P-node type configured." -ForegroundColor Green

# 3. Disable NetBIOS over TCP/IP on all active adapters
Write-Host "[+] Disabling NetBIOS on all active network adapters..." -ForegroundColor Gray
$Adapters = Get-CimInstance -ClassName Win32_NetworkAdapterConfiguration -ErrorAction SilentlyContinue | Where-Object { $_.IPEnabled -eq $true }
if ($Adapters) {
    foreach ($Adapter in $Adapters) {
        Invoke-CimMethod -InputObject $Adapter -MethodName SetTcpipNetbios -Arguments @{ TcpipNetbiosOptions = 2 } | Out-Null
    }
    Write-Host "NetBIOS disabled on active network interfaces." -ForegroundColor Green
}

# 4. Harden TCP/IP Parameters
Set-RegDWord "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" "EnableICMPRedirect" 0
Set-RegDWord "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" "DisableIPSourceRouting" 2
Write-Host "[+] IPv4 TCP/IP parameter redirects and source routing disabled." -ForegroundColor Green

Set-RegDWord "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters" "DisableIPSourceRouting" 2
Write-Host "[+] IPv6 TCP/IP parameter source routing disabled." -ForegroundColor Green

# 5. Prevent Network Connection Sharing and Dual-Homing Bridging
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Network Connections" "NC_ShowSharedAccessUI" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Network Connections" "NC_AllowNetBridge_NLA" 0
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Network Connections" "NC_StdDomainUserSetLocation" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WcmSvc\GroupPolicy" "fMinimizeConnections" 3
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WcmSvc\GroupPolicy" "fBlockNonDomain" 1
Set-RegDWord "HKLM:\SOFTWARE\Microsoft\wcmSvc\wifinetworkmanager\config" "AutoConnectAllowedOEM" 0
Write-Host "[+] Network connections, sharing, bridging, elevation, and hotspot settings configured." -ForegroundColor Green

# 6. Printing Spooler Web Downloads and HTTP printing block
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers" "DisableWebPnPDownload" 1
Set-RegDWord "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers" "DisableHTTPPrinting" 1
Write-Host "[+] Printing spooler HTTP and Web service options disabled." -ForegroundColor Green
```

```
# 7. Restrict anonymous access to SAM and Named Pipes/Shares
Set-RegDWord "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters" "RestrictNullSessAccess" 1
Write-Host "[+] Anonymous null session share access restricted." -ForegroundColor Green

# 8. Disable WPAD
Write-Host "[+] Disabling WinHTTP Auto-Proxy service..." -ForegroundColor Gray
Set-Service -Name "WinHttpAutoProxySvc" -StartupType Disabled -ErrorAction SilentlyContinue
Stop-Service -Name "WinHttpAutoProxySvc" -Force -ErrorAction SilentlyContinue

$WpadPath = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Internet Settings\Wpad"
if (-not (Test-Path $WpadPath)) {
    New-Item -Path $WpadPath -Force | Out-Null
}
Set-ItemProperty -Path $WpadPath -Name "WpadOverride" -Value 1 -Type DWord -Force
Write-Host "[+] WPAD auto-detection disabled in user preferences registry." -ForegroundColor Green

# 9. Restrict Net Session Enumeration (NetCease SDDL)
Write-Host "[+] Restricting Net Session Enumeration..." -ForegroundColor Gray
try {
    $SD = New-Object System.Security.AccessControl.CommonSecurityDescriptor($false, $false, "D:(A;;CC;;;BA)(A;;CC;;;SD)(A;;CC;;;PU)")
    $BinaryForm = New-Object byte[] $SD.BinaryLength
    $SD.GetBinaryForm($BinaryForm, 0)
    $LanmanSecPath = "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\DefaultSecurity"
    if (-not (Test-Path $LanmanSecPath)) {
        New-Item -Path $LanmanSecPath -Force | Out-Null
    }
    Set-ItemProperty -Path $LanmanSecPath -Name "SrvsvcSessionInfo" -Value $BinaryForm -Type Binary -Force
    Write-Host "[+] Net Session Enumeration restricted to Admins/Operators/Power Users." -ForegroundColor Green
} catch {
    Write-Error "Failed to apply Net Session Enumeration restrictions: $($_.Exception.Message)"
}

Write-Host "Network and name resolution hardening applied successfully." -ForegroundColor Green
```

To audit the network and name resolution status: [Download Script: Test-NetworkHardeningStatus.ps1](#)

```
# Test-NetworkHardeningStatus.ps1
# Description: Audits LLMNR, NetBIOS parameters, NetBIOS adapter state, TCP/IP parameters, and STIG print / network connection options.

Write-Host "--- Auditing Network and Name Resolution Baseline ---" -ForegroundColor Cyan

$script:Vulnerable = $false

# Helper function to audit registry properties
function Test-RegistryValue ($path, $name, $expectedValue) {
    $val = Get-ItemProperty -Path $path -Name $name -ErrorAction SilentlyContinue
    $actual = if ($val) { $val.$name } else { "" }
    $color = "Red"
    if ($actual -eq $expectedValue) {
        $color = "Green"
    } else {
        $script:Vulnerable = $true
    }
    Write-Host "    - Registry Setting: $name | Actual: '$actual' (Expected: '$expectedValue')" -ForegroundColor $color
}

# 1. Audit LLMNR, mDNS, and default IPv6 DNS Servers
$DnsPath = "HKLM:\Software\Policies\Microsoft\Windows NT\DNSClient"
Test-RegistryValue $DnsPath "EnableMulticast" 0
Test-RegistryValue $DnsPath "EnablemDNS" 0
Test-RegistryValue $DnsPath "DisableIPv6DefaultDnsServers" 1

# 2. Audit NetBIOS Parameters
$NetbtPath = "HKLM:\SYSTEM\CurrentControlSet\Services\Netbt\Parameters"
Test-RegistryValue $NetbtPath "NoNameReleaseOnDemand" 1
Test-RegistryValue $NetbtPath "NodeType" 2

# 3. Audit TCP/IP Parameters
$TcpipPath = "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters"
Test-RegistryValue $TcpipPath "EnableICMPRedirect" 0
Test-RegistryValue $TcpipPath "DisableIPSourceRouting" 2

$Tcpip6Path = "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters"
Test-RegistryValue $Tcpip6Path "DisableIPSourceRouting" 2

# 4. Audit Connection Sharing & Dual-Homing Settings
$NetConnPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Network Connections"
Test-RegistryValue $NetConnPath "NC_ShowSharedAccessUI" 0
Test-RegistryValue $NetConnPath "NC_AllowNetBridge_NLA" 0
Test-RegistryValue $NetConnPath "NC_StdDomainUserSetLocation" 1

$WcmPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WcmSvc\GroupPolicy"
Test-RegistryValue $WcmPath "fMinimizeConnections" 3
Test-RegistryValue $WcmPath "fBlockNonDomain" 1

$WifiPath = "HKLM:\SOFTWARE\Microsoft\wcmsvc\wifinetworkmanager\config"
Test-RegistryValue $WifiPath "AutoConnectAllowedOEM" 0

# 5. Audit HTTP Print Options
$PrinterPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers"
Test-RegistryValue $PrinterPath "DisableWebPnPDownload" 1
Test-RegistryValue $PrinterPath "DisableHTTPPPrinting" 1

# 6. Audit Null Session Share Restrict
$ServerPath = "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters"
Test-RegistryValue $ServerPath "RestrictNullSessAccess" 1

# 7. Audit WPAD Service and Registry Override
$WpadSvc = Get-Service -Name "WinHttpAutoProxySvc" -ErrorAction SilentlyContinue
if ($null -ne $WpadSvc) {
    if ($WpadSvc.StartType -eq "Disabled") {
        Write-Host "    - WPAD Service State: Disabled (Secure)" -ForegroundColor Green
    } else {
        Write-Host "    - VULNERABLE: WPAD Service StartType is $($WpadSvc.StartType) (Expected: Disabled)" -ForegroundColor Red
        $script:Vulnerable = $true
    }
}
}

$WpadPath = "HKCU:\Software\Microsoft\Windows\CurrentVersion\Internet Settings\Wpad"
```

```

Test-RegistryValue $WpadPath "WpadOverride" 1

# 8. Audit Net Session Enumeration (NetCease)
$LanmanSecPath = "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\DefaultSecurity"
if (Test-Path $LanmanSecPath) {
    $SrvsvcSessionInfo = (Get-ItemProperty -Path $LanmanSecPath -Name "SrvsvcSessionInfo" -ErrorAction SilentlyContinue).SrvsvcSessionInfo
    if ($null -ne $SrvsvcSessionInfo) {
        try {
            $SD = New-Object System.Security.AccessControl.CommonSecurityDescriptor($false, $false, $SrvsvcSessionInfo, 0)
            $Sddl = $SD.GetSddlForm("Dacl")
            if ($Sddl -eq "D:(A;;CC;;;BA)(A;;CC;;;SO)(A;;CC;;;PU)" {
                Write-Host "    - Net Session Enumeration Security Descriptor: Hardened (Secure)" -ForegroundColor Green
            } else {
                Write-Host "    - VULNERABLE: Net Session Enumeration Security Descriptor is '$Sddl' (Expected: 'D:(A;;CC;;;BA)(A;;CC;;;SO)(A;;CC;;;PU)')" -ForegroundColor Red
                $script:Vulnerable = $true
            }
        } catch {
            Write-Host "    - VULNERABLE: Failed to parse Net Session Enumeration security descriptor." -ForegroundColor Red
            $script:Vulnerable = $true
        }
    } else {
        Write-Host "    - VULNERABLE: SrvsvcSessionInfo registry value not found." -ForegroundColor Red
        $script:Vulnerable = $true
    }
} else {
    Write-Host "    - VULNERABLE: LanmanServer\DefaultSecurity path not found." -ForegroundColor Red
    $script:Vulnerable = $true
}

if ($script:Vulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
}

```

Sources & Compliance References * **CIS Microsoft Windows 10/11 Client Benchmark**: Section 18.6.4.1 (EnablemDNS), Section 18.6.11.2 (NC_AllowNetBridge_NLA), Section 18.6.11.3 (NC_ShowSharedAccessUI), Section 18.6.11.4 (NC_StdDomainUserSetLocation), Section 18.6.21.1 & 18.6.21.2 (fMinimizeConnections & fBlockNonDomain), Section 18.6.23.2.1 (AutoConnectAllowedOEM). * **CIS Microsoft Windows 10/11 Client Benchmark**: Section 9.1 (Disable LLMNR), Section 18.8.44.1 (Configure EnableICMPRedirect), Section 18.8.44.2 (Configure DisableIPSourceRouting). * **ANSSI AD Hardening Guide**: Recommendation R19 (LDAP and name resolution security recommendations). * **DoD Windows 11 Computer STIG v2r6**: Various print driver download limits, HTTP printing blocks, internet connection sharing prohibitions, hotspot connection rules, and null session restrictions.

[REQ-END-002] Configure User Account Control Policies

Target Scope * ****Applicable Systems****: Tier 2 client workstations and member servers. * ****Operating Systems****: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * ****Priority****: High * ****GPO Path / Registry Location****: * ****GPO Paths****: * `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` * `Computer Configuration\Administrative Templates\System` * ****Registry Locations****: * `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System` * `ConsentPromptBehaviorAdmin` = `1` (REG_DWORD, Prompt for credentials on secure desktop) * `ConsentPromptBehaviorUser` = `0` (REG_DWORD, Automatically deny elevation requests) * `EnableLUA` = `1` (REG_DWORD, Enable User Account Control / Admin Approval Mode) * `PromptOnSecureDesktop` = `1` (REG_DWORD, Switch to secure desktop when prompting) * `LocalAccountTokenFilterPolicy` = `0` (REG_DWORD, Apply UAC restrictions to local accounts on network logons) * `EnableInstallDetection` = `1` (REG_DWORD, Detect application installations and prompt for elevation) * `EnableVirtualization` = `1` (REG_DWORD, Virtualize file and registry write failures to per-user locations) * `HKLM\SOFTWARE\Policies\Microsoft\Windows\Sudo` * `Enabled` = `1` (REG_DWORD, Sudo behavior set to force a new elevated window)

Rationale User Account Control (UAC) is a fundamental defense mechanism in Windows. It limits the privilege levels of running applications, executing administrative actions with standard user tokens unless elevated privileges are explicitly approved.

Hardening UAC settings ensures:

1. **Secure Desktop Enforcement**: The elevation prompt is displayed on a separate, secure desktop environment that isolated system threads run on. This prevents third-party malware running in user space from intercepting credentials or programmatically clicking "Yes" to elevate itself.
2. **Auto-Denial of Standard User Elevation**: Standard users should not be allowed to request elevation. If a standard user triggers a task requiring administrative rights, the prompt should auto-deny rather than requesting an administrator password, preventing users from attempting to bypass controls or exposing local admin passwords on a non-secure user terminal.
3. **Admin Approval Mode**: Forcing built-in administrators to run in Admin Approval Mode ensures that even administrative users do not run web browsers or document editors with administrative tokens by default.
4. **Sudo Command Control**: The `sudo` command introduced in Windows 11 (24H2) allows users to run elevated commands from an unelevated console. Leaving this feature unconfigured or allowing execution within the current console session can expose elevated processes to command injection or token interception in the same console session. Restricting `sudo` to opening a new elevated window (`1`) or disabling it entirely (`0`) mitigates session hijacking risks.
5. **Network UAC Restrictions (`LocalAccountTokenFilterPolicy`)**: Restricting the elevation of local accounts during network logons prevents lateral movement. When set to `0`, local accounts (except for the built-in Administrator RID 500 account) connecting remotely via network shares or administrative interfaces cannot obtain administrative tokens, neutralizing pass-the-hash attacks using secondary local administrative accounts.
6. **Installer Detection (`EnableInstallDetection`)**: Detecting installer program behavior prevents silent software execution. When enabled, any execution of an install file or setup program by standard users or administrators triggers a UAC elevation prompt, preventing unauthorized silent program deployments.
7. **UAC Virtualization (`EnableVirtualization`)**: Virtualizing writes redirection keeps the operating system directory space clean. It redirects legacy application registry and file writes targeting system folders (like `Program Files` or `System32`) to user-profile-specific folders, allowing legacy applications to run without requiring administrative rights.

Legacy Impact & Compatibility * ****User Experience****: Standard users will not be able to install software or change system settings that require administrative credentials. Support technicians must log on as local administrators to perform maintenance tasks or use remote tools. * ****Script and Installer Behaviors****: Legacy scripts and administrative install tasks that run programmatically without secure-desktop awareness may fail or hang if they trigger elevation prompts that cannot be programmatically bypassed. * ****Network Admin Access****: Network-based remote administration using local accounts will be restricted to the RID 500 account. Domain accounts must be used for remote administrative access (WinRM, SMB administration) on standard endpoints.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`).
2. Create or edit a GPO linked to the workstations OU (e.g., `GPO_Hardening_Workstations`).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options`
4. Configure the following settings:
 - **Policy**: `User Account Control: Behavior of the elevation prompt for administrators in Admin Approval Mode` → **Prompt for credentials on the secure desktop**
 - **Policy**: `User Account Control: Behavior of the elevation prompt for standard users` → **Automatically deny elevation requests**
 - **Policy**: `User Account Control: Run all administrators in Admin Approval Mode` → **Enabled**
 - **Policy**: `User Account Control: Switch to the secure desktop when prompting for elevation` → **Enabled**
 - **Policy**: `User Account Control: Detect application installations and prompt for elevation` → **Enabled**

- **Policy:** `User Account Control: Virtualize file and registry write failures to per-user locations` → **Enabled**

5. Since the UAC network restrictions policy is not directly exposed in standard GPO security templates, deploy the registry setting via GPO Preferences:

- Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
- Create a new Registry Item:
 - **Action:** `Update`
 - **Hive:** `HKEY_LOCAL_MACHINE`
 - **Key Path:** `SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System`
 - **Value Name:** `LocalAccountTokenFilterPolicy`
 - **Value Type:** `REG_DWORD`
 - **Value Data:** `0`

6. Navigate to: `Computer Configuration\Administrative Templates\System`

- **Policy:** `Configure the behavior of the sudo command` → **Enabled** with options set to **Force a new elevated window**

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to configure maximum security parameters for UAC in the system registry.

[Download Script: Configure-UACPolicies.ps1](#)

```
# Configure-UACPolicies.ps1
# Enforces hardened User Account Control (UAC) registry configuration values including network restrictions, installer detection, and virtualization.

Write-Host "--- Hardening User Account Control Policies ---" -ForegroundColor Cyan

$SystemPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"

if (-not (Test-Path $SystemPath)) {
    New-Item -Path $SystemPath -Force | Out-Null
}

# ConsentPromptBehaviorAdmin = 1 (Prompt for credentials on secure desktop)
Set-ItemProperty -Path $SystemPath -Name "ConsentPromptBehaviorAdmin" -Value 1 -Type DWord -Force
# ConsentPromptBehaviorUser = 0 (Automatically deny elevation requests)
Set-ItemProperty -Path $SystemPath -Name "ConsentPromptBehaviorUser" -Value 0 -Type DWord -Force
# EnableLUA = 1 (Enable User Account Control / Admin Approval Mode)
Set-ItemProperty -Path $SystemPath -Name "EnableLUA" -Value 1 -Type DWord -Force
# PromptOnSecureDesktop = 1 (Switch to secure desktop when prompting)
Set-ItemProperty -Path $SystemPath -Name "PromptOnSecureDesktop" -Value 1 -Type DWord -Force
# LocalAccountTokenFilterPolicy = 0 (Apply UAC restrictions to local accounts on network logons)
Set-ItemProperty -Path $SystemPath -Name "LocalAccountTokenFilterPolicy" -Value 0 -Type DWord -Force
# EnableInstallDetection = 1 (Detect application installations and prompt for elevation)
Set-ItemProperty -Path $SystemPath -Name "EnableInstallDetection" -Value 1 -Type DWord -Force
# EnableVirtualization = 1 (Virtualize file and registry write failures to per-user locations)
Set-ItemProperty -Path $SystemPath -Name "EnableVirtualization" -Value 1 -Type DWord -Force

# Configure Windows Sudo command behavior (Enabled = 1 [Force new elevated window])
$SudoPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Sudo"
if (-not (Test-Path $SudoPath)) {
    New-Item -Path $SudoPath -Force | Out-Null
}
Set-ItemProperty -Path $SudoPath -Name "Enabled" -Value 1 -Type DWord -Force

Write-Host "[+] UAC registry values configured successfully." -ForegroundColor Green
```

To audit UAC configurations:

[Download Script: Test-UACPolicies.ps1](#)


```
# Test-UACPolicies.ps1
# Verifies local system registry settings for User Account Control.

Write-Host "--- Auditing User Account Control Policies ---" -ForegroundColor Cyan

$SystemPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
$script:Vulnerable = $false

function Test-UACRegistryValue ($name, $expected, $message) {
    $val = Get-ItemProperty -Path $SystemPath -Name $name -ErrorAction SilentlyContinue
    $actual = if ($val) { $val.$name } else { $null }
    $color = "Red"
    if ($actual -eq $expected) {
        $color = "Green"
    } else {
        $script:Vulnerable = $true
    }
    Write-Host "    - Registry Setting: $name | Actual: '$actual' (Expected: '$expected') | $message" -ForegroundColor $color
}

Test-UACRegistryValue "ConsentPromptBehaviorAdmin" 1 "Behavior of elevation prompt for administrators"
Test-UACRegistryValue "ConsentPromptBehaviorUser" 0 "Behavior of elevation prompt for standard users"
Test-UACRegistryValue "EnableLUA" 1 "Run all administrators in Admin Approval Mode"
Test-UACRegistryValue "PromptOnSecureDesktop" 1 "Switch to secure desktop when prompting"
Test-UACRegistryValue "LocalAccountTokenFilterPolicy" 0 "UAC network restrictions"
Test-UACRegistryValue "EnableInstallDetection" 1 "Installer detection"
Test-UACRegistryValue "EnableVirtualization" 1 "UAC virtualization"

# Audit Sudo command
$SudoPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Sudo"
if (Test-Path $SudoPath) {
    $SudoState = Get-ItemProperty -Path $SudoPath -Name "Enabled" -ErrorAction SilentlyContinue
    $SudoVal = if ($SudoState) { $SudoState.Enabled } else { 0 }
    $SudoColor = if ($SudoVal -eq 0 -or $SudoVal -eq 1) { "Green" } else { "Red" }
    Write-Host "    - Sudo Command Enabled state: $SudoVal (Required = 1 [New Window] or 0 [Disabled])" -ForegroundColor $SudoColor
    if ($SudoVal -ne 0 -and $SudoVal -ne 1) {
        $script:Vulnerable = $true
    }
} else {
    Write-Host "    - Sudo Command Enabled state: Not Configured (Default/Compliant as it inherits disabled)" -ForegroundColor Green
}

if ($script:Vulnerable) {
    Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
    exit 1
} else {
    Write-Host "Audit Result: SECURE" -ForegroundColor Green
    exit 0
}
}
```

📖 Sources & Compliance References * **CIS Microsoft Windows 10/11 Client Benchmark***: Section 2.3.17.1 (ConsentPromptBehaviorAdmin), Section 2.3.17.2 (ConsentPromptBehaviorUser), Section 2.3.17.4 (EnableInstallDetection), Section 2.3.17.5 (EnableLUA), Section 2.3.17.8 (EnableVirtualization) * **CIS Microsoft Windows 10/11 Client Benchmark***: Section 18.4.1 (LocalAccountTokenFilterPolicy) * **Microsoft Security Baselines***: Windows Client Security baseline registry settings.

[REQ-END-003] Disable AutoPlay and AutoRun

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\AutoPlay Policies` * **Registry Locations**: * `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer` * `NoDriveTypeAutoRun` = `255` (0xFF, REG_DWORD) * `NoAutorun` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows\Explorer` * `NoAutoplayfornonVolume` = `1` (REG_DWORD)

Rationale The AutoPlay and AutoRun features in Windows are designed to automatically execute programs or open media when a removable drive, network share, or CD-ROM is inserted or connected.

Attackers exploit these features by placing malicious scripts, payloads, or executables on USB drives or external storage media. If AutoPlay is enabled, connecting the drive triggers automatic execution of these scripts or programs without user interaction or approval, allowing malware to achieve immediate execution in the context of the logged-on user. Disabling AutoPlay across all drive types completely mitigates this physical transmission vector.

Additionally, non-volume devices (such as mobile phones, cameras, or media players) can still trigger AutoPlay behavior. Disallowing AutoPlay for non-volume devices ensures these devices do not introduce unauthorized execution pathways when plugged into standard client machines.

Legacy Impact & Compatibility * **User Experience**: Users will not see pop-up choices or automated actions when connecting USB sticks, DVDs, or external drives. They must manually open File Explorer and navigate to the drive to read or open files. * **Installer Media**: Software installers located on optical disks or external media will not start automatically; users must double-click the setup application manually.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`).
 2. Create or edit a GPO linked to the workstations OU (e.g., `GPO_Hardening_Workstations`).
 3. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\AutoPlay Policies`
 4. Configure the following settings:
 - **Policy:** `Turn off AutoPlay`
 - **Setting:** `Enabled`
 - **Select Options:** `All drives`
 - **Policy:** `Set the default behavior for AutoRun`
 - **Setting:** `Enabled`
 - **Select Options:** `Do not execute any autorun commands`
 - **Policy:** `Disallow Autoplay for non-volume devices`
 - **Setting:** `Enabled`
-

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to configure Explorer registry keys to disable AutoPlay, AutoRun, and AutoPlay for non-volume devices.

[Download Script: Disable-AutoPlay.ps1](#)

```
# Disable-AutoPlay.ps1
# Disables AutoPlay/AutoRun registry settings globally on all drive types and non-volume devices.

Write-Host "--- Disabling AutoPlay and AutoRun ---" -ForegroundColor Cyan

$ExplorerPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer"

if (-not (Test-Path $ExplorerPath)) {
    New-Item -Path $ExplorerPath -Force | Out-Null
}

# NoDriveTypeAutoRun = 0xFF (255 in decimal) disables AutoRun on all types of drives
Set-ItemProperty -Path $ExplorerPath -Name "NoDriveTypeAutoRun" -Value 255 -Type DWord -Force

# NoAutorun = 1 disables AutoRun commands in inf files
Set-ItemProperty -Path $ExplorerPath -Name "NoAutorun" -Value 1 -Type DWord -Force

# Disallow Autoplay for non-volume devices (NoAutoplayfornonVolume = 1)
$PolExplorerPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Explorer"
if (-not (Test-Path $PolExplorerPath)) {
    New-Item -Path $PolExplorerPath -Force | Out-Null
}
Set-ItemProperty -Path $PolExplorerPath -Name "NoAutoplayfornonVolume" -Value 1 -Type DWord -Force

Write-Host "[+] AutoPlay and AutoRun registry parameters set." -ForegroundColor Green
```

To audit AutoPlay configurations: [Download Script: Test-AutoPlay.ps1](#)

```
# Test-AutoPlay.ps1
# Audits local system registry parameters for AutoPlay status.

Write-Host "--- Auditing AutoPlay Configuration ---" -ForegroundColor Cyan

$ExplorerPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer"
$PolExplorerPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Explorer"

$NoDriveAuto = Get-ItemProperty -Path $ExplorerPath -Name "NoDriveTypeAutoRun" -ErrorAction SilentlyContinue
$NoAutoCmd = Get-ItemProperty -Path $ExplorerPath -Name "NoAutorun" -ErrorAction SilentlyContinue
$NoNonVol = Get-ItemProperty -Path $PolExplorerPath -Name "NoAutoplayfornonVolume" -ErrorAction SilentlyContinue

$NoDriveVal = if ($NoDriveAuto) { $NoDriveAuto.NoDriveTypeAutoRun } else { 0 }
$NoAutoVal = if ($NoAutoCmd) { $NoAutoCmd.NoAutorun } else { 0 }
$NoNonVolVal = if ($NoNonVol) { $NoNonVol.NoAutoplayfornonVolume } else { 0 }

$NoDriveColor = if ($NoDriveVal -eq 255) { "Green" } else { "Red" }
$NoAutoColor = if ($NoAutoVal -eq 1) { "Green" } else { "Red" }
$NoNonVolColor = if ($NoNonVolVal -eq 1) { "Green" } else { "Red" }

Write-Host "    - NoDriveTypeAutoRun: $NoDriveVal (Required = 255 to disable all drives)" -ForegroundColor $NoDriveColor
Write-Host "    - NoAutorun: $NoAutoVal (Required = 1)" -ForegroundColor $NoAutoColor
Write-Host "    - NoAutoplayfornonVolume: $NoNonVolVal (Required = 1)" -ForegroundColor $NoNonVolColor
```

📖 Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.3.1 (Turn off AutoPlay), Section 18.3.2 (Set the default behavior for AutoRun) * **Microsoft Security Baselines**: Windows Client Explorer configuration standards. * **DoD Windows 11 STIG**: Disallow Autoplay for non-volume devices requirements.

[REQ-END-004] Block Removable Storage

Target Scope * **Applicable Systems**: Tier 2 client workstations. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional.

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * Computer Configuration\Administrative Templates\System\Removable Storage Access * HKLM\SOFTWARE\Policies\Microsoft\Windows\RemovableStorageDevices

Rationale Removable storage media, such as USB flash drives, external SSDs, and optical discs, represent a significant risk vector for corporate network environments.

Attackers use USB drives to bypass network-based security boundaries (such as firewalls and intrusion detection systems), introducing malware directly onto local workstation hosts via physical sneakernets. USB drives are also a primary tool for insider threat data exfiltration, enabling users to copy proprietary or sensitive information off company terminals onto untracked hardware. Restricting removable storage access at the operating system level prevents both unauthorized data ingress (malware infection) and data egress (unauthorized data copying).

Legacy Impact & Compatibility * **User Restrictions**: Users cannot read from or write to external USB storage devices, SD cards, or external CD/DVD readers. Any connected mass storage device will be rejected by the file system. * **Administrative Tools**: Support technicians must use network-based shares, administrative shares, or secure file-transfer systems to deploy scripts or updates to client workstations. * **Exceptions**: Keyboards, mice, printers, and smart card readers are not blocked, as they do not register under the Removable Storage device class.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`).
2. Create or edit a GPO linked to the workstations OU (e.g., `GPO_Hardening_Workstations`).
3. Navigate to: `Computer Configuration\Administrative Templates\System\Removable Storage Access`
4. Configure the setting:
 - **Policy:** `All Removable Storage classes: Deny all access`
 - **Setting:** `Enabled`

Alternatively, if you only want to block write access while allowing read-only access (for specific profiles), configure:

- **Policy:** `Removable Disks: Deny write access` → `Enabled`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to configure registry keys to block all removable storage devices.

[Download Script: Block-RemovableStorage.ps1](#)

```
# Block-RemovableStorage.ps1
# Configures local registry parameters to deny access to all removable storage classes.

Write-Host "--- Restricting Removable Storage Devices ---" -ForegroundColor Cyan

$RemovableStoragePath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\RemovableStorageDevices"

if (-not (Test-Path $RemovableStoragePath)) {
    New-Item -Path $RemovableStoragePath -Force | Out-Null
}

# Deny_All = 1 blocks all removable storage classes
Set-ItemProperty -Path $RemovableStoragePath -Name "Deny_All" -Value 1 -Type DWord

Write-Host "[+] Removable storage block configured." -ForegroundColor Green
```

To audit removable storage block configurations: [Download Script: Test-RemovableStorage.ps1](#)

```
# Test-RemovableStorage.ps1
# Audits registry values for removable storage blocks.

Write-Host "--- Auditing Removable Storage Restrictions ---" -ForegroundColor Cyan

$RemovableStoragePath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\RemovableStorageDevices"

$DenyAllProp = Get-ItemProperty -Path $RemovableStoragePath -Name "Deny_All" -ErrorAction SilentlyContinue
$DenyAllVal = if ($DenyAllProp) { $DenyAllProp.Deny_All } else { 0 }
$DenyColor = if ($DenyAllVal -eq 1) { "Green" } else { "Red" }

Write-Host "    - Removable Storage Deny_All: $DenyAllVal (Required = 1)" -ForegroundColor $DenyColor
```

 Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9.82 (All Removable Storage classes: Deny all access) * **ANSSI AD Hardening Guide**: Section on hardware and external communication control.

[REQ-END-005] Restrict Remote Desktop Access

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Paths**: * `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Connections` * `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Security` * `Computer Configuration\Administrative Templates\System\Remote Assistance` * **Registry Locations**: * `HKLM\SYSTEM\CurrentControlSet\Control\Terminal Server` * `fDenyTSConnections` = `1` (REG_DWORD, Disabled) * `fAllowToGetHelp` = `0` (REG_DWORD, Solicited Remote Assistance Disabled) * `HKLM\SYSTEM\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp` * `UserAuthentication` = `1` (REG_DWORD, NLA Enabled) * `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services` * `fAllowToGetHelp` = `0` (REG_DWORD, Solicited Remote Assistance Disabled) * `MaxTicketExpiryUnits` = (Delete / Not Configured) * `MaxTicketExpiry` = (Delete / Not Configured) * `fUseMailto` = (Delete / Not Configured) * `fAllowFullControl` = (Delete / Not Configured)

Rationale Remote Desktop Protocol (RDP) is one of the primary mechanisms used by attackers for lateral movement and administrative session hijacking. If inbound RDP is enabled globally on workstations: 1. **Lateral Movement**: An attacker who compromises a single standard user's credentials with administrative permissions on other machines can RDP from workstation to workstation across the network. 2. **Session Hijacking**: Attackers can hijack existing administrative RDP sessions using built-in command-line tools (such as `tscon.exe`) if they obtain administrator privileges on the system. 3. **Password Spraying**: Open RDP ports allow attackers to attempt password spraying or brute-force attacks against local administrative accounts.

Furthermore, Windows **Remote Assistance** allows helper connections that can lead to remote code execution or unauthorized access if not properly restricted. Disabling Solicited Remote Assistance and removing any legacy configuration values limits the workstation's attack surface.

The safest configuration is to disable Remote Desktop Services and Remote Assistance entirely on all Tier 2 workstations. If RDP is strictly necessary for remote technical support, it must require Network Level Authentication (NLA) and the listening firewall rules must restrict access to authorized management subnets only.

Legacy Impact & Compatibility * **Remote Administration**: Support technicians cannot connect to workstations via RDP unless they connect from an IP address inside the authorized administrative subnet (e.g., from a PAW or Jump Host). * **User Assistance**: Standard users cannot use Remote Desktop or Remote Assistance to share screens or assist one another. Alternate secure remote assistance tools (which require local user approval and do not open listener ports) must be used.

Implementation Steps**### Option A: Group Policy Object (GPO) Configuration (Preferred)**

1. Disable Inbound Remote Desktop Connections (Default Hardening) 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Create or edit a GPO linked to the workstations OU (e.g., `GPO\_Hardening\_Workstations`). 3. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Connections` 4. Configure the setting: * **Policy**: `Allow users to connect remotely by using Remote Desktop Services` * **Setting**: `Disabled`

2. Enforce NLA and High Encryption (If RDP is Required for Admins) If RDP is strictly required, enable it but restrict it using the following settings: 1. Under the same path: * **Policy**: `Allow users to connect remotely by using Remote Desktop Services` → `Enabled` 2. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Security` 3. Configure the following settings: * **Policy**: `Require user authentication for remote connections by using Network Level Authentication` * **Setting**: `Enabled` * **Policy**: `Set client connection encryption level` * **Setting**: `Enabled` (Select `High Level` in the options dropdown) * **Policy**: `Require use of specific security layer for remote (RDP) connections` * **Setting**: `Enabled: SSL` 4. Deploy local firewall rules via GPO to restrict TCP port 3389 inbound to administrative subnet ranges only.

3. Configure Temporary Folders Deletion on Exit 1. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Temporary Folders` 2. Configure the setting: * **Policy**: `Do not delete temp folders upon exit` * **Setting**: `Disabled` (ensures session temporary directories are deleted when users log off)

4. Disable Solicited Remote Assistance 1. Navigate to: `Computer Configuration\Administrative Templates\System\Remote Assistance` 2. Configure the setting: * **Policy**: `Configure Solicited Remote Assistance` * **Setting**: `Disabled`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to disable Remote Desktop and Remote Assistance, and enforce NLA and secure registry keys.

[Download Script: Disable-RemoteDesktop.ps1](#)

```
# Disable-RemoteDesktop.ps1
# Disables Remote Desktop and Solicited Remote Assistance connections, sets NLA requirements, sets Security Layer to SSL, configures
temp folder deletion, and cleans parameters.

Write-Host "--- Restricting Remote Desktop and Remote Assistance Access ---" -ForegroundColor Cyan

$RdpPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server"

# 1. Disable RDP Connections (fDenyTSConnections = 1)
Set-ItemProperty -Path $RdpPath -Name "fDenyTSConnections" -Value 1 -Type DWord -Force
Write-Host "[+] Inbound Remote Desktop connections disabled." -ForegroundColor Green

# 2. Enforce Network Level Authentication (UserAuthentication = 1)
$RdpSecPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp"
if (Test-Path $RdpSecPath) {
    Set-ItemProperty -Path $RdpSecPath -Name "UserAuthentication" -Value 1 -Type DWord -Force
    Write-Host "[+] Network Level Authentication (NLA) enforced." -ForegroundColor Green
}

# 3. Disable Remote Assistance (fAllowToGetHelp = 0)
Set-ItemProperty -Path $RdpPath -Name "fAllowToGetHelp" -Value 0 -Type DWord -Force

# 4. Disable and clean Solicited Remote Assistance Policies, set SSL, and delete temp folders
$TSPoliciesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services"
if (-not (Test-Path $TSPoliciesPath)) {
    New-Item -Path $TSPoliciesPath -Force | Out-Null
}
Set-ItemProperty -Path $TSPoliciesPath -Name "fAllowToGetHelp" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $TSPoliciesPath -Name "SecurityLayer" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $TSPoliciesPath -Name "DeleteTempDirsOnExit" -Value 1 -Type DWord -Force

$paramsToDelete = @("MaxTicketExpiryUnits", "MaxTicketExpiry", "fUseMailto", "fAllowFullControl")
foreach ($Param in $paramsToDelete) {
    if (Get-ItemProperty -Path $TSPoliciesPath -Name $Param -ErrorAction SilentlyContinue) {
        Remove-ItemProperty -Path $TSPoliciesPath -Name $Param -Force -ErrorAction SilentlyContinue
    }
}
Write-Host "[+] Remote Desktop policies (SSL, Temp folders, Solicited Help) configured and cleaned." -ForegroundColor Green
```

To audit Remote Desktop and Remote Assistance status: [Download Script: Test-RemoteDesktopStatus.ps1](#)

```
# Test-RemoteDesktopStatus.ps1
# Audits local RDP, Remote Assistance, security layer, temp folders, and NLA registry configuration and listening firewall ports.

Write-Host "--- Auditing Remote Desktop Configuration ---" -ForegroundColor Cyan

$RdpPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server"
$RdpSecPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Terminal Server\WinStations\RDP-Tcp"
$TSPoliciesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services"

$DenyTS = Get-ItemProperty -Path $RdpPath -Name "fDenyTSConnections" -ErrorAction SilentlyContinue
$DenyVal = if ($DenyTS) { $DenyTS.fDenyTSConnections } else { 1 }

$NlaProp = Get-ItemProperty -Path $RdpSecPath -Name "UserAuthentication" -ErrorAction SilentlyContinue
$NlaVal = if ($NlaProp) { $NlaProp.UserAuthentication } else { 0 }

$DenyColor = if ($DenyVal -eq 1) { "Green" } else { "Yellow" }
$NlaColor = if ($NlaVal -eq 1) { "Green" } else { "Red" }

Write-Host "    - fDenyTSConnections: $DenyVal (Recommended = 1 to block all)" -ForegroundColor $DenyColor
Write-Host "    - UserAuthentication (NLA): $NlaVal (Required = 1 if RDP is enabled)" -ForegroundColor $NlaColor

# Check if port 3389 firewall rule is active and enabled
$RdpFirewall = Get-NetFirewallRule -Name "RemoteDesktop-UserMode-In-TCP" -ErrorAction SilentlyContinue
if ($RdpFirewall) {
    $FirewallColor = if ($RdpFirewall.Enabled -eq $true) { "Yellow" } else { "Green" }
    Write-Host "    - RDP Inbound Firewall Rule Active: $($RdpFirewall.Enabled)" -ForegroundColor $FirewallColor
}

# Audit Remote Assistance
$GetHelpTS = Get-ItemProperty -Path $RdpPath -Name "fAllowToGetHelp" -ErrorAction SilentlyContinue
$GetHelpTSVal = if ($GetHelpTS) { $GetHelpTS.fAllowToGetHelp } else { 0 }
$HelpColor = if ($GetHelpTSVal -eq 0) { "Green" } else { "Red" }
Write-Host "    - fAllowToGetHelp (Terminal Server): $GetHelpTSVal (Recommended = 0)" -ForegroundColor $HelpColor

# Audit Solicited Remote Assistance Policy, Security Layer, and Temp Folders
if (Test-Path $TSPoliciesPath) {
    $PolGetHelp = Get-ItemProperty -Path $TSPoliciesPath -Name "fAllowToGetHelp" -ErrorAction SilentlyContinue
    $PolGetHelpVal = if ($PolGetHelp) { $PolGetHelp.fAllowToGetHelp } else { $null }

    $PolHelpColor = if ($PolGetHelpVal -eq 0) { "Green" } else { "Red" }
    Write-Host "    - fAllowToGetHelp (Policies): $PolGetHelpVal (Recommended = 0)" -ForegroundColor $PolHelpColor

    $SecurityLayerProp = Get-ItemProperty -Path $TSPoliciesPath -Name "SecurityLayer" -ErrorAction SilentlyContinue
    $SecurityLayerVal = if ($SecurityLayerProp) { $SecurityLayerProp.SecurityLayer } else { $null }
    $SecLayerColor = if ($SecurityLayerVal -eq 2) { "Green" } else { "Red" }
    Write-Host "    - SecurityLayer (SSL): $SecurityLayerVal (Required = 2)" -ForegroundColor $SecLayerColor

    $DeleteTempProp = Get-ItemProperty -Path $TSPoliciesPath -Name "DeleteTempDirsOnExit" -ErrorAction SilentlyContinue
    $DeleteTempVal = if ($DeleteTempProp) { $DeleteTempProp.DeleteTempDirsOnExit } else { $null }
    $DeleteTempColor = if ($DeleteTempVal -eq 1) { "Green" } else { "Red" }
    Write-Host "    - DeleteTempDirsOnExit (Temp Folders): $DeleteTempVal (Required = 1)" -ForegroundColor $DeleteTempColor

    $Params = @("MaxTicketExpiryUnits", "MaxTicketExpiry", "fUseMailto", "fAllowFullControl")
    foreach ($Param in $Params) {
        $Val = (Get-ItemProperty -Path $TSPoliciesPath -Name $Param -ErrorAction SilentlyContinue).$Param
        if ($null -ne $Val) {
            Write-Host "    - VULNERABLE: Solicited Remote Assistance parameter '$Param' is set to '$Val' (Expected: Deleted/Not Configured)" -ForegroundColor Red
        } else {
            Write-Host "    - Parameter '$Param': Not Configured (Correct)" -ForegroundColor Green
        }
    }
} else {
    Write-Host "    - Solicited Remote Assistance Policy Path does not exist" -ForegroundColor Red
}
}
```

📄 Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.2.1 (Require user authentication for remote connections by using Network Level Authentication) * **ANSSI AD Hardening Guide**: Security guidelines regarding Remote Desktop access and management protocols. * **DoD Windows 11 STIG**: Solicited Remote Assistance requirements.

[REQ-END-006] Restrict Local Administrators Group

Target Scope * **Applicable Systems**: Tier 2 client workstations. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional.

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: Computer Configuration\Policies\Windows Settings\Security Settings\Restricted Groups * **Registry Location**: HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System * `LocalAccountTokenFilterPolicy` = `0` (REG_DWORD, Disabled)

Rationale Local administrator rights on workstations are a significant source of operational vulnerability. If standard end-users run as local administrators: 1. **Malware Propagation**: Malware executed by the user runs in an administrative context, allowing it to bypass local firewalls, alter registry hives, disable security controls (like Windows Defender), and persist across reboots. 2. **Credential Harvesting**: Compromised local admin accounts allow attackers to execute memory-dumping tools (e.g., Mimikatz) to harvest stored domain credentials of other users who have logged on to that machine. 3. **Software Control Bypass**: Users can install arbitrary, unapproved software, introducing license compliance risks and unmonitored security vulnerabilities.

Securing the local Administrators group ensures only local security accounts (like the local `Administrator` managed by LAPS) or dedicated workstation support accounts are members. The default `Domain Users` or standard domain accounts must never be allowed local administrative rights.

Legacy Impact & Compatibility * **User Restrictions**: Users cannot install software, update system drivers, or modify local network configurations. Support staff must assist users with system modifications or use automated software distribution channels. * **Legacy Apps**: Applications that require local administrator privileges to write data directly to the `%ProgramFiles%` directory or local registry keys will fail. These applications must be reconfigured to write to user-profile locations (e.g., `%APPDATA%`) or run under service account privileges.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

Enforce local Administrators group membership and restrict remote execution via local accounts (UAC remote restrictions):

1. Open the **Group Policy Management Console** (`gpmc.msc`).
2. Create or edit a GPO linked to the workstations OU (e.g., `GPO_Hardening_Workstations`).
3. **Configure Restricted Groups**:
 - Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Restricted Groups`
 - Right-click **Restricted Groups** and select **Add Group**.
 - Type `Administrators` (or click Browse to find the local group).
 - Under **Members of this group**, define the allowed members:
 - `Administrator` (the built-in local administrator account)
 - `DomainName\Workstation-Support-Admins` (dedicated Tier 2 support team group, if used)
 - *Leave out `Domain Users` or any other general domain accounts.*
 - Applying this GPO will overwrite the membership of the local Administrators group, immediately removing any account not explicitly listed.
4. **Configure Local Account Token Filter Policy**:
 - Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
 - Right-click **Registry** and select **New** → **Registry Item**.
 - Set **Action**: `Update`
 - Set **Hive**: `HKEY_LOCAL_MACHINE`
 - Set **Key Path**: `SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System`
 - Set **Value name**: `LocalAccountTokenFilterPolicy`
 - Set **Value type**: `REG_DWORD`
 - Set **Value data**: `0`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to audit and remediate unauthorized administrative accounts in the local Administrators group and enforce local account remote token restrictions.

[Download Script: Clean-LocalAdministrators.ps1](#)

```
# Clean-LocalAdministrators.ps1
# Removes unauthorized domain or local accounts from the local Administrators group and disables local account remote token filtering bypass.

Write-Host "--- Restricting Local Administrators Group ---" -ForegroundColor Cyan

# Define the list of authorized members
# The built-in Administrator account (RID 500) and authorized domain support groups.
$AuthorizedMembers = @("Administrator", "Workstation-Support-Admins")

$LocalAdmins = Get-LocalGroupMember -Group "Administrators" -ErrorAction SilentlyContinue

if ($LocalAdmins) {
    foreach ($Member in $LocalAdmins) {
        # Check if the member is not in the authorized list
        $Match = $false
        foreach ($Auth in $AuthorizedMembers) {
            # Check for exact matches or matches against SAM / SID formats
            if ($Member.Name -eq $Auth -or $Member.Name -like "*\$Auth" -or $Member.Name -eq "$env:COMPUTERNAME\$Auth") {
                $Match = $true
                break
            }
        }

        if (-not $Match) {
            Write-Host "[-] Removing unauthorized member: $($Member.Name) (Source: $($Member.PrincipalSource))" -ForegroundColor Yellow
            low
            try {
                Remove-LocalGroupMember -Group "Administrators" -Member $Member.Name -ErrorAction Stop
                Write-Host "    Successfully removed: $($Member.Name)" -ForegroundColor Green
            } catch {
                Write-Error "    Failed to remove: $($Member.Name). Error: $_.Exception.Message"
            }
        } else {
            Write-Host "[+] Member authorized: $($Member.Name)" -ForegroundColor Green
        }
    }
} else {
    Write-Error "Could not retrieve members of local Administrators group."
}

# Enforce LocalAccountTokenFilterPolicy = 0
$RegistryPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
$ValueName = "LocalAccountTokenFilterPolicy"

try {
    if (-not (Test-Path $RegistryPath)) {
        New-Item -Path $RegistryPath -Force | Out-Null
    }
    Set-ItemProperty -Path $RegistryPath -Name $ValueName -Value 0 -Type DWord -Force | Out-Null
    Write-Host "[+] LocalAccountTokenFilterPolicy configured to 0." -ForegroundColor Green
} catch {
    Write-Error "Failed to configure LocalAccountTokenFilterPolicy. Error: $_.Exception.Message"
}
}
```

To audit local Administrators group memberships and UAC token filtering policy: [Download Script: Test-LocalAdministrators.ps1](#)

```
# Test-LocalAdministrators.ps1
# Audits membership of the local Administrators group and checks local account remote token filtering bypass policy.

Write-Host "--- Auditing Local Administrators Group ---" -ForegroundColor Cyan

$LocalAdmins = Get-LocalGroupMember -Group "Administrators" -ErrorAction SilentlyContinue

if ($LocalAdmins) {
    Write-Host "[*] Current members of local Administrators group:" -ForegroundColor Yellow
    foreach ($Member in $LocalAdmins) {
        # Flag any domain user accounts that might have been added to administrators group
        $StatusColor = "Green"
        if ($Member.PrincipalSource -eq "ActiveDirectory" -and $Member.Name -notmatch "Workstation-Support-Admins") {
            $StatusColor = "Red"
            Write-Host "    - VULNERABLE: Domain Account '$($Member.Name)' has local admin rights." -ForegroundColor $StatusColor
        } else {
            Write-Host "    - Member: $($Member.Name) | Source: $($Member.PrincipalSource) | Class: $($Member.ObjectClass)" -Foregro
undColor $StatusColor
        }
    }
} else {
    Write-Error "Failed to retrieve local Administrators group members."
}

# Audit LocalAccountTokenFilterPolicy
$RegistryPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
$ValueName = "LocalAccountTokenFilterPolicy"

if (Test-Path $RegistryPath) {
    $Val = (Get-ItemProperty -Path $RegistryPath -Name $ValueName -ErrorAction SilentlyContinue).$ValueName
    if ($Val -eq 0) {
        Write-Host "[+] LocalAccountTokenFilterPolicy is configured correctly (0)." -ForegroundColor Green
    } elseif ($null -eq $Val) {
        Write-Host "[-] LocalAccountTokenFilterPolicy is not explicitly set (Expected: 0)." -ForegroundColor Red
    } else {
        Write-Host "[-] LocalAccountTokenFilterPolicy is vulnerable: $Val (Expected: 0)." -ForegroundColor Red
    }
} else {
    Write-Host "[-] LocalAccountTokenFilterPolicy is not configured (Expected: 0)." -ForegroundColor Red
}
}
```

 Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 5.5 (Ensure only authorized accounts are members of the Administrators group) * **ANSSI AD Hardening Guide**: Recommendations regarding local administration restriction and tiering boundaries.

Windows Defender Antivirus Baseline and Exploit Guard

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus

Rationale Windows Defender Antivirus is the primary endpoint protection suite on Windows platforms. To establish a robust defense-in-depth posture against modern endpoint threat vectors, the basic protection must be augmented with Exploit Guard, Tamper Protection, SmartScreen, and process isolation.

This submodule contains individual requirement controls for each advanced protective mechanism and configuration parameter within Windows Defender Antivirus.

Legacy Impact & Compatibility * **ASR Administrative Impact**: Enabling ASR rules can block legacy administrative scripts or third-party orchestration tools that rely on WMI/PSEXec or execute obfuscated administrative wrappers. Extensive audit testing is recommended prior to broad enforcement. * **Office Application Rules**: Rules related to Microsoft Office (e.g., blocking child processes) apply only to endpoints where productivity suites are installed. They will have no impact on member servers without Office. * **Sandbox Boot Overhead**: Setting 'MP_FORCE_USE_SANDBOX' requires a reboot to initialize the scanning process within the AppContainer sandbox. There is negligible performance overhead once initialized. * **SmartScreen Impact**: Standard users will be blocked from launching unrecognized software. Support staff must assist with authorizing internal or uncertified custom business applications. * **AMSI Provider Signatures**: Any third-party antimalware software that registers itself as an AMSI provider must possess a valid, trusted Authenticode signature. Unsigned or self-signed providers will be blocked from loading.

Defender Hardening Requirements

The following Defender Antivirus configurations must be enforced:

1. [REQ-END-057 - Disable Real-Time Monitoring and Behavior Monitoring Override](#)
 2. [REQ-END-058 - Configure Potentially Unwanted Applications \(PUA\) Protection](#)
 3. [REQ-END-059 - Prevent Local List Merging and Exclusions Configuration](#)
 4. [REQ-END-060 - Configure Auto Exclusions Configuration](#)
 5. [REQ-END-061 - Prevent MAPS Local Setting Override](#)
 6. [REQ-END-062 - Enable EDR in Block Mode](#)
 7. [REQ-END-063 - Allow Network Protection on Windows Server](#)
 8. [REQ-END-064 - Enable File Hash Computation](#)
 9. [REQ-END-065 - Configure Network Inspection System \(NIS\) settings](#)
 10. [REQ-END-066 - Configure OOBE Real-Time Protection and Security Intelligence](#)
 11. [REQ-END-067 - Enable Dynamic Signature Dropped Event Reporting](#)
 12. [REQ-END-068 - Configure Quick Scan and Scanning Exclusions](#)
 13. [REQ-END-069 - Configure Scheduled Scan Parameters](#)
 14. [REQ-END-070 - Configure Security Intelligence Update Schedule](#)
 15. [REQ-END-071 - Configure Attack Surface Reduction Rules](#)
 16. [REQ-END-072 - Configure Threat Severity Default Quarantine Actions](#)
 17. [REQ-END-073 - Configure Family Options UI Lockdown](#)
 18. [REQ-END-074 - Configure Tamper Protection](#)
 19. [REQ-END-075 - Configure Sandbox Execution Environment](#)
 20. [REQ-END-076 - Configure AMSI Authenticode Signature Verification](#)
 21. [REQ-END-077 - Configure File Explorer SmartScreen](#)
 22. [REQ-END-078 - Disable OneDrive File Sync](#)
 23. [REQ-END-079 - Enforce Antivirus Scan on Opening Attachments](#)
-

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-057] Disable Real-Time Monitoring and Behavior Monitoring Override

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Real-time Protection` * **Registry Location**: *
 `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` * `DisableRealtimeMonitoring` = `0` (REG_DWORD) *
 `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` * `DisableBehaviorMonitoring` = `0` (REG_DWORD) *
 `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` * `DisableIOAVProtection` = `0` (REG_DWORD) *
 `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` * `DisableScriptScanning` = `0` (REG_DWORD)

Rationale Real-time scanning, behavior monitoring, and script checking are the core dynamic defense mechanisms of Windows Defender. Disabling or bypassing these controls allows malicious scripts, file-based attacks, and unauthorized in-memory activities to execute undetected.

Legacy Impact & Compatibility Negligible operational impact. Real-time scanning introduces standard CPU overhead during file read/write operations.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Real-time Protection 2. Set 'Turn off real-time protection' to 'Disabled' 3. Set 'Turn on behavior monitoring' to 'Enabled' 4. Set 'Scan all downloaded files and attachments' to 'Enabled' 5. Set 'Turn on script scanning' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderRtp.ps1](#)] ([../implementation_scripts/Configure-DefenderRtp.ps1](#))

```
# Configure-DefenderRtp.ps1
Set-MpPreference -DisableRealtimeMonitoring $false
Set-MpPreference -DisableBehaviorMonitoring $false
Set-MpPreference -DisableIOAVProtection $false
Set-MpPreference -DisableScriptScanning $false
```

To audit the hardening status: [Download Script: Get-DefenderRtpStatus.ps1](#)

```
# Get-DefenderRtpStatus.ps1
$Pref = Get-MpPreference
if ($Pref.DisableRealtimeMonitoring -eq $false -and $Pref.DisableBehaviorMonitoring -eq $false -and $Pref.DisableIOAVProtection -eq $false -and $Pref.DisableScriptScanning -eq $false) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-058] Configure Potentially Unwanted Applications (PUA) Protection

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender` * `PUAProtection` = `1` (REG_DWORD)

Rationale Potentially Unwanted Applications (PUA) include adware, torrent clients, cryptominers, and system optimizers that increase risk and resource consumption. Forcing PUA blocking stops standard vectors of shadow IT and unauthorized utility tool execution.

Legacy Impact & Compatibility Legitimate but unrecognized administrative utilities may be blocked. Standard exclusions must be configured for approved utilities.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus 2. Set 'Configure detection for potentially unwanted applications' to 'Enabled' 3. Select 'Block' in the options dropdown list

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderPUA.ps1](#)] ([./implementation_scripts/Configure-DefenderPUA.ps1](#))

```
# Configure-DefenderPUA.ps1
Set-MpPreference -PUAProtection 1
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender" -Name "PUAProtection" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderPUAStatus.ps1](#)

```
# Get-DefenderPUAStatus.ps1
$Pref = Get-MpPreference
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender" -Name "PUAProtection" -ErrorAction SilentlyContinue
if ($Pref.PUAProtection -eq 1 -or ($Reg -and $Reg.PUAProtection -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-059] Prevent Local List Merging and Exclusions Configuration

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender` * `DisableLocalAdminMerge` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender` * `HideExclusionsFromLocalAdmins` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions` * `DisableLocalAdminConfiguration` = `1` (REG_DWORD)

Rationale If local administrators or compromised administrative accounts can modify Defender exclusions or merge local lists, they can authorize malicious folders or tools. Restricting list configuration to central GPOs ensures consistent security enforcement.

Legacy Impact & Compatibility Local administrators will be unable to add folders or scripts to Defender exclusions. All exclusions must be managed centrally via GPO.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus 2. Set 'Configure local administrator merge behavior for lists' to 'Disabled' 3. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Exclusions 4. Set 'Prevent users from configuring exclusions' to 'Enabled' 5. Set 'Control whether or not exclusions are visible to Local Admins' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderLocalExclusions.ps1](#)] ([./implementation_scripts/Configure-DefenderLocalExclusions.ps1](#))

```
# Configure-DefenderLocalExclusions.ps1
Set-MpPreference -DisableLocalAdminMerge $true
Set-MpPreference -DisableExclusionRestriction $false
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender"
if (-not (Test-Path $Path)) { New-Item -Path $Path -Force | Out-Null }
Set-ItemProperty -Path $Path -Name "DisableLocalAdminMerge" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $Path -Name "HideExclusionsFromLocalAdmins" -Value 1 -Type DWord -Force
$ExclPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions"
if (-not (Test-Path $ExclPath)) { New-Item -Path $ExclPath -Force | Out-Null }
Set-ItemProperty -Path $ExclPath -Name "DisableLocalAdminConfiguration" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderLocalExclusionsStatus.ps1](#)

```
# Get-DefenderLocalExclusionsStatus.ps1
$Pref = Get-MpPreference
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender" -Name "DisableLocalAdminMerge" -ErrorAction SilentlyContinue
$RegHide = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender" -Name "HideExclusionsFromLocalAdmins" -ErrorAction SilentlyContinue
$RegConfig = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions" -Name "DisableLocalAdminConfiguration" -ErrorAction SilentlyContinue
if (($Pref.DisableLocalAdminMerge -eq $true -or ($Reg -and $Reg.DisableLocalAdminMerge -eq 1)) -and
    ($RegHide -and $RegHide.HideExclusionsFromLocalAdmins -eq 1) -and
    ($RegConfig -and $RegConfig.DisableLocalAdminConfiguration -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-060] Configure Auto Exclusions Configuration

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Exclusions` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions` * `DisableAutoExclusions` = `0` (REG_DWORD)

Rationale Auto Exclusions automatically configure exclusions for known safe system folders or server roles to reduce performance overhead. Enforcing that auto exclusions are not disabled ensures server performance stability and proper system scanning.

Legacy Impact & Compatibility None.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Exclusions 2. Set 'Turn off Auto Exclusions' to 'Disabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderAutoExclusions.ps1](#)] ([../implementation_scripts/Configure-DefenderAutoExclusions.ps1](#))

```
# Configure-DefenderAutoExclusions.ps1
$ExclPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions"
if (-not (Test-Path $ExclPath)) { New-Item -Path $ExclPath -Force | Out-Null }
Set-ItemProperty -Path $ExclPath -Name "DisableAutoExclusions" -Value 0 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderAutoExclusionsStatus.ps1](#)

```
# Get-DefenderAutoExclusionsStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Exclusions" -Name "DisableAutoExclusions" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.DisableAutoExclusions -eq 0) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-061] Prevent MAPS Local Setting Override

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\MAPS` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Spynet` * `LocalSettingOverrideSpynetReporting` = `0` (REG_DWORD)

Rationale Preventing local overrides of Microsoft Active Protection Service (MAPS) reporting ensures that endpoints consistently report telemetry and signature feedback to cloud resources, preserving centralized protective visibility.

Legacy Impact & Compatibility Local system administrators cannot opt-out of telemetry reporting.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\MAPS 2. Set 'Configure local setting override for reporting to Microsoft MAPS' to 'Disabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderMapsOverride.ps1](#)] ([../implementation_scripts/Configure-DefenderMapsOverride.ps1](#))

```
# Configure-DefenderMapsOverride.ps1
$SpynetPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Spynet"
if (-not (Test-Path $SpynetPath)) { New-Item -Path $SpynetPath -Force | Out-Null }
Set-ItemProperty -Path $SpynetPath -Name "LocalSettingOverrideSpynetReporting" -Value 0 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderMapsOverrideStatus.ps1](#)

```
# Get-DefenderMapsOverrideStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Spynet" -Name "LocalSettingOverrideSpynetReporting" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.LocalSettingOverrideSpynetReporting -eq 0) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-062] Enable EDR in Block Mode

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Features` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Features` * `PassiveRemediation` = `1` (REG_DWORD)

Rationale Endpoint Detection and Response (EDR) in Block Mode allows Defender to take remediation actions on malicious artifacts detected by Microsoft Defender for Endpoint even if another non-Microsoft antivirus is primary. This establishes secondary defensive block capabilities.

Legacy Impact & Compatibility None. Extends passive monitoring to execute block operations when necessary.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Features 2. Set 'Enable EDR in block mode' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderEdrBlockMode.ps1](#)] (./implementation_scripts/Configure-DefenderEdrBlockMode.ps1)

```
# Configure-DefenderEdrBlockMode.ps1
$FeaturesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Features"
if (-not (Test-Path $FeaturesPath)) { New-Item -Path $FeaturesPath -Force | Out-Null }
Set-ItemProperty -Path $FeaturesPath -Name "PassiveRemediation" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderEdrBlockModeStatus.ps1](#)

```
# Get-DefenderEdrBlockModeStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Features" -Name "PassiveRemediation" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.PassiveRemediation -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-063] Allow Network Protection on Windows Server

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Network Protection` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\Network Protection` * `AllowNetworkProtectionOnWinServer` = `1` (REG_DWORD)

Rationale Network Protection blocks processes from accessing malicious domains, phishing sites, and host IP ranges. Allowing Network Protection on Windows Server ensures that member servers running server workloads possess the same IP filter protections as client platforms.

Legacy Impact & Compatibility Unrecognized or legacy client-server connections to custom web resources may be filtered. Exclusions must be mapped under DFE.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Network Protection 2. Set 'This setting controls whether Network Protection is allowed to be configured into block or audit mode on Windows Server' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-DefenderNetworkProtectionServer.ps1](../implementation_scripts/Configure-DefenderNetworkProtectionServer.ps1)

```
# Configure-DefenderNetworkProtectionServer.ps1
$NetProtPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\Network Protection"
if (-not (Test-Path $NetProtPath)) { New-Item -Path $NetProtPath -Force | Out-Null }
Set-ItemProperty -Path $NetProtPath -Name "AllowNetworkProtectionOnWinServer" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderNetworkProtectionServerStatus.ps1](#)

```
# Get-DefenderNetworkProtectionServerStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\Network Protection" -Name "AllowNetworkProtectionOnWinServer" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.AllowNetworkProtectionOnWinServer -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-064] Enable File Hash Computation

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\MpEngine` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\MpEngine` * `EnableFileHashComputation` = `1` (REG_DWORD)

Rationale Computing cryptographic file hashes allows Defender to pass hashes of scanned files to cloud and SIEM endpoints. This enables precise IOC matches, file tracking, and correlation with threat intelligence repositories.

Legacy Impact & Compatibility Minor CPU/Disk I/O impact when scanning large directories.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\MpEngine 2. Set 'Enable file hash computation feature' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderFileHash.ps1](#)] ([../implementation_scripts/Configure-DefenderFileHash.ps1](#))

```
# Configure-DefenderFileHash.ps1
Set-MpPreference -EnableFileHashComputation $true
$MpEnginePath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\MpEngine"
if (-not (Test-Path $MpEnginePath)) { New-Item -Path $MpEnginePath -Force | Out-Null }
Set-ItemProperty -Path $MpEnginePath -Name "EnableFileHashComputation" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderFileHashStatus.ps1](#)

```
# Get-DefenderFileHashStatus.ps1
$Pref = Get-MpPreference
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\MpEngine" -Name "EnableFileHashComputation" -Error
Action SilentlyContinue
if ($Pref.EnableFileHashComputation -eq $true -or ($Reg -and $Reg.EnableFileHashComputation -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-065] Configure Network Inspection System (NIS) settings

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Network Inspection System` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\NIS` * `EnableConvertWarnToBlock` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\NIS` * `AllowSwitchToAsyncInspection` = `1` (REG_DWORD)

Rationale The Network Inspection System (NIS) inspects network traffic patterns for known exploits. Converting warning verdicts to block enforces inline blocking of zero-day exploits, while allowing async inspection prevents performance overhead from slowing local network interfaces.

Legacy Impact & Compatibility None.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Network Inspection System 2. Set 'Convert warn verdict to block' to 'Enabled' 3. Set 'Turn on asynchronous inspection' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderNis.ps1](#)] ([../implementation_scripts/Configure-DefenderNis.ps1](#))

```
# Configure-DefenderNis.ps1
$NisPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\NIS"
if (-not (Test-Path $NisPath)) { New-Item -Path $NisPath -Force | Out-Null }
Set-ItemProperty -Path $NisPath -Name "EnableConvertWarnToBlock" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $NisPath -Name "AllowSwitchToAsyncInspection" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderNisStatus.ps1](#)

```
# Get-DefenderNisStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\NIS" -Name "EnableConvertWarnToBlock" -ErrorAction SilentlyContinue
$RegAsync = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\NIS" -Name "AllowSwitchToAsyncInspection" -ErrorAction SilentlyContinue
if (($Reg -and $Reg.EnableConvertWarnToBlock -eq 1) -and ($RegAsync -and $RegAsync.AllowSwitchToAsyncInspection -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-066] Configure OOBE Real-Time Protection and Security Intelligence

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Real-time Protection` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection` * `OobeEnableRtpAndSigUpdate` = `1` (REG_DWORD)

Rationale Enabling real-time protection and intelligence updates during the Out-of-Box Experience (OOBE) ensures that the system is fully updated and protected before the initial administrative user signs in or connects to enterprise network nodes.

Legacy Impact & Compatibility Negligible. Minor network traffic during initial provisioning phases.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Real-time Protection 2. Set 'Configure real-time protection and Security Intelligence Updates during OOBE' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderOobeRtp.ps1](#)] ([./implementation_scripts/Configure-DefenderOobeRtp.ps1](#))

```
# Configure-DefenderOobeRtp.ps1
$RtpPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection"
if (-not (Test-Path $RtpPath)) { New-Item -Path $RtpPath -Force | Out-Null }
Set-ItemProperty -Path $RtpPath -Name "OobeEnableRtpAndSigUpdate" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderOobeRtpStatus.ps1](#)

```
# Get-DefenderOobeRtpStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Real-Time Protection" -Name "OobeEnableRtpAndSigUpdate" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.OobeEnableRtpAndSigUpdate -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-067] Enable Dynamic Signature Dropped Event Reporting

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: Low * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Reporting` * **Registry Location**: *
`HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Reporting` * `EnableDynamicSignatureDroppedEventReporting` = `1` (REG_DWORD)

Rationale Enabling this log report generation triggers explicit events when a dynamic scan ruleset signature is dropped. This ensures SIEM integrations can immediately log changes in the local threat signatures dataset.

Legacy Impact & Compatibility None.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Reporting 2. Set 'Configure whether to report Dynamic Signature dropped events' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderDynamicReporting.ps1](#)] ([../implementation_scripts/Configure-DefenderDynamicReporting.ps1](#))

```
# Configure-DefenderDynamicReporting.ps1
$RepPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Reporting"
if (-not (Test-Path $RepPath)) { New-Item -Path $RepPath -Force | Out-Null }
Set-ItemProperty -Path $RepPath -Name "EnableDynamicSignatureDroppedEventReporting" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderDynamicReportingStatus.ps1](#)

```
# Get-DefenderDynamicReportingStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Reporting" -Name "EnableDynamicSignatureDroppedEventReporting" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.EnableDynamicSignatureDroppedEventReporting -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-068] Configure Quick Scan and Scanning Exclusions

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` * `QuickScanIncludeExclusions` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` * `DisablePackedExeScanning` = `0` (REG_DWORD)

Rationale Malware frequently tries to establish persistence in excluded directories or inside packed/compressed executables. Forcing quick scans to include excluded files and ensuring packed file structures are recursively scanned prevents malware evasion.

Legacy Impact & Compatibility Increased quick scan times depending on folder sizes and compression ratios.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan 2. Set 'Scan excluded files and directories during quick scans' to 'Enabled' 3. Set 'Turn off scanning of packed executables' to 'Disabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderQuickScan.ps1](#)] (./implementation_scripts/Configure-DefenderQuickScan.ps1)

```
# Configure-DefenderQuickScan.ps1
Set-MpPreference -DisablePackedExeScanning $false
$ScanPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan"
if (-not (Test-Path $ScanPath)) { New-Item -Path $ScanPath -Force | Out-Null }
Set-ItemProperty -Path $ScanPath -Name "QuickScanIncludeExclusions" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $ScanPath -Name "DisablePackedExeScanning" -Value 0 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderQuickScanStatus.ps1](#)

```
# Get-DefenderQuickScanStatus.ps1
$Pref = Get-MpPreference
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -Name "QuickScanIncludeExclusions" -ErrorAction SilentlyContinue
$RegPack = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -Name "DisablePackedExeScanning" -ErrorAction SilentlyContinue
if (($Pref.DisablePackedExeScanning -eq $false -or ($RegPack -and $RegPack.DisablePackedExeScanning -eq 0)) -and ($Reg -and $Reg.QuickScanIncludeExclusions -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-069] Configure Scheduled Scan Parameters

Target Scope * ****Applicable Systems****: Tier 2 client workstations and member servers. * ****Operating Systems****: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * ****Priority****: Medium * ****GPO Path / Registry Location****: * ****GPO Path****: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan` * ****Registry Location****: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` * `ScheduleDay` = `0` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` * `DisableEmailScanning` = `0` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` * `DisableHeuristics` = `0` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan` * `DaysWithoutCatchupQuickScan` = `7` (REG_DWORD)

Rationale Ensuring daily scheduled scans, enabling heuristics for behavioral anomaly detection, scan mail attachments, and forcing a catchup scan after at most 7 days ensures system integrity is continually validated.

Legacy Impact & Compatibility Slight scanning overhead.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan 2. Set 'Specify the day of the week to run a scheduled scan' to 'Enabled' (Select 'Every day' or '0') 3. Set 'Turn on e-mail scanning' to 'Enabled' 4. Set 'Turn on heuristics' to 'Enabled' 5. Set 'Trigger a quick scan after X days without any scans' to 'Enabled' (7 days)

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderScheduledScan.ps1](#)] ([../implementation_scripts/Configure-DefenderScheduledScan.ps1](#))

```
# Configure-DefenderScheduledScan.ps1
Set-MpPreference -DisableEmailScanning $false
Set-MpPreference -DisableHeuristics $false
$ScanPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan"
if (-not (Test-Path $ScanPath)) { New-Item -Path $ScanPath -Force | Out-Null }
Set-ItemProperty -Path $ScanPath -Name "ScheduleDay" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $ScanPath -Name "DisableEmailScanning" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $ScanPath -Name "DisableHeuristics" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $ScanPath -Name "DaysWithoutCatchupQuickScan" -Value 7 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderScheduledScanStatus.ps1](#)

```
# Get-DefenderScheduledScanStatus.ps1
$Pref = Get-MpPreference
$RegDays = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -Name "DaysWithoutCatchupQuickScan" -ErrorAction SilentlyContinue
if ($Pref.DisableEmailScanning -eq $false -and $Pref.DisableHeuristics -eq $false -and ($RegDays -and $RegDays.DaysWithoutCatchupQuickScan -eq 7)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * ****CIS Microsoft Windows 10 Benchmark****: Section 18.9 (Windows Defender Antivirus configuration parameters) * ****ANSSI Active Directory Hardening Guide****: Protective controls baselines on client workstations

[REQ-END-070] Configure Security Intelligence Update Schedule

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Security Intelligence Updates` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates` * `ASSignatureDue` = `7` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates` * `AVSignatureDue` = `7` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates` * `ScheduleDay` = `0` (REG_DWORD)

Rationale Antivirus signatures must remain fresh to block the latest published threats. Mandating daily checks for updates and marking signatures older than 7 days as out-of-date ensures continuous defense parity.

Legacy Impact & Compatibility Requires continuous outbound connectivity to WSUS or Microsoft Update endpoints.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Security Intelligence Updates 2. Set 'Define the number of days before spyware security intelligence is considered out of date' to 'Enabled' (7 days) 3. Set 'Define the number of days before virus security intelligence is considered out of date' to 'Enabled' (7 days) 4. Set 'Specify the day of the week to check for security intelligence updates' to 'Enabled' (Every day or 0)

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderUpdateSchedule.ps1](#)] ([../implementation_scripts/Configure-DefenderUpdateSchedule.ps1](#))

```
# Configure-DefenderUpdateSchedule.ps1
$SigPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates"
if (-not (Test-Path $SigPath)) { New-Item -Path $SigPath -Force | Out-Null }
Set-ItemProperty -Path $SigPath -Name "ASSignatureDue" -Value 7 -Type DWord -Force
Set-ItemProperty -Path $SigPath -Name "AVSignatureDue" -Value 7 -Type DWord -Force
Set-ItemProperty -Path $SigPath -Name "ScheduleDay" -Value 0 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderUpdateScheduleStatus.ps1](#)

```
# Get-DefenderUpdateScheduleStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates" -Name "ASSignatureDue" -ErrorAction SilentlyContinue
$RegAV = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates" -Name "AVSignatureDue" -ErrorAction SilentlyContinue
$RegDay = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Signature Updates" -Name "ScheduleDay" -ErrorAction SilentlyContinue
if (($Reg -and $Reg.ASSignatureDue -eq 7) -and ($RegAV -and $RegAV.AVSignatureDue -eq 7) -and ($RegDay -and $RegDay.ScheduleDay -eq 0)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

Configure Attack Surface Reduction Rules

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction

Rationale Attack Surface Reduction (ASR) rules targets software behaviors that are frequently abused by malware, such as launching executable files from email attachments, spawning child processes from Office apps or PDF viewers, credential theft from LSASS, and writing unsigned files from USB.

This submodule contains individual requirement rules for each ASR control enforced on standard workstations.

Legacy Impact & Compatibility * **Detailed Audit Phase**: ASR rules should first be deployed in **Audit** mode to collect telemetry in the Windows Event Logs (Event ID '1121' or '1122'). This allows administrators to configure exclusions before switching to **Block** mode ('1'). * **Macro/Office Dependencies**: Organizations relying on advanced inter-process Office communication or macros running system API calls will experience disruptions. Software configurations or ASR exclusion groups must be updated.

Enforced Attack Surface Reduction Rules

The following individual ASR rules must be configured in Block mode:

1. [REQ-END-080 - ASR: Block abuse of exploited vulnerable signed drivers](#)
2. [REQ-END-081 - ASR: Block Adobe Reader from creating child processes](#)
3. [REQ-END-082 - ASR: Block all Office applications from creating child processes](#)
4. [REQ-END-083 - ASR: Block credential stealing from the Windows local security authority subsystem](#)
5. [REQ-END-084 - ASR: Block executable content from email client and webmail](#)
6. [REQ-END-085 - ASR: Block executable files from running unless they meet a prevalence, age, or trusted list criterion](#)
7. [REQ-END-086 - ASR: Block execution of potentially obfuscated scripts](#)
8. [REQ-END-087 - ASR: Block JavaScript or VBScript from launching downloaded executable content](#)
9. [REQ-END-088 - ASR: Block Office applications from creating executable content](#)
10. [REQ-END-089 - ASR: Block Office applications from injecting code into other processes](#)
11. [REQ-END-090 - ASR: Block Office communication application from creating child processes](#)
12. [REQ-END-091 - ASR: Block persistence through WMI event subscription](#)
13. [REQ-END-092 - ASR: Block process creations originating from PSEXEC and WMI commands](#)
14. [REQ-END-093 - ASR: Block untrusted and unsigned processes that run from USB](#)
15. [REQ-END-094 - ASR: Block Win32 API calls from Office macros](#)
16. [REQ-END-095 - ASR: Use advanced protection against ransomware](#)

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR Rules) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-080] ASR: Block abuse of exploited vulnerable signed drivers

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `56a863a9-875e-4185-98a7-b882c64b5ce5` = `1` (REG_SZ)

Rationale Prevents an application from writing a vulnerable signed driver to disk. Attackers use Bring Your Own Vulnerable Driver (BYOVD) techniques to bypass Windows kernel protections by loading legitimate, signed third-party drivers that contain known vulnerabilities, allowing them to disable security agents and gain kernel-level privileges.

Legacy Impact & Compatibility Administrators or developers who deliberately load older or specific signed debugging/hardware diagnostics drivers containing known security bugs will be blocked. Approvals or exclusions must be configured for these specific drivers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `56a863a9-875e-4185-98a7-b882c64b5ce5` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrVulnerableDrivers.ps1](#)] ([../implementation_scripts/Configure-AsrVulnerableDrivers.ps1](#))

```
# Configure-AsrVulnerableDrivers.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "56a863a9-875e-4185-98a7-b882c64b5ce5" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrVulnerableDriversStatus.ps1](#)

```
# Get-AsrVulnerableDriversStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "56a863a9-875e-4185-98a7-b882c64b5ce5" -ErrorAction SilentlyContinue
if ($Value -and ($Value."56a863a9-875e-4185-98a7-b882c64b5ce5" -eq "1" -or $Value."56a863a9-875e-4185-98a7-b882c64b5ce5" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-081] ASR: Block Adobe Reader from creating child processes

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `7674ba52-37eb-4a4f-a9a1-f0f9a1619a2c` = `1` (REG_SZ)

Rationale Prevents Adobe Reader from launching any child processes. Malicious PDF documents frequently attempt to exploit application vulnerabilities or trick users into executing embedded links, which spawns command shells (cmd.exe, powershell.exe) or scripting hosts (wscript.exe) to download and launch secondary malware payloads.

Legacy Impact & Compatibility Adobe Reader will be unable to launch external helper applications directly. PDF forms that rely on spawning external applications or customized plugins that execute command-line helpers will fail to run.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `7674ba52-37eb-4a4f-a9a1-f0f9a1619a2c` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrAdobeChild.ps1](#)] ([../implementation_scripts/Configure-AsrAdobeChild.ps1](#))

```
# Configure-AsrAdobeChild.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "7674ba52-37eb-4a4f-a9a1-f0f9a1619a2c" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrAdobeChildStatus.ps1](#)

```
# Get-AsrAdobeChildStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "7674ba52-37eb-4a4f-a9a1-f0f9a1619a2c" -ErrorAction SilentlyContinue
if ($Value -and ($Value."7674ba52-37eb-4a4f-a9a1-f0f9a1619a2c" -eq "1" -or $Value."7674ba52-37eb-4a4f-a9a1-f0f9a1619a2c" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-082] ASR: Block all Office applications from creating child processes

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `d4f940ab-401b-4efc-aadc-ad5f3c50688a` = `1` (REG_SZ)

Rationale Blocks Microsoft Office applications (Word, Excel, PowerPoint) from creating child processes. This prevents malicious files containing embedded VBA macros or exploiting unpatched vulnerabilities (such as CVE-2021-40444) from launching scripting environments or system commands to download and execute code.

Legacy Impact & Compatibility Office macros that legitimately launch external scripts, shell scripts, or third-party executable helpers will be blocked. Business processes relying on advanced inter-process macros must be rewritten to use modern API alternatives.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `d4f940ab-401b-4efc-aadc-ad5f3c50688a` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrOfficeChild.ps1](#)] ([./../implementation_scripts/Configure-AsrOfficeChild.ps1](#))

```
# Configure-AsrOfficeChild.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "d4f940ab-401b-4efc-aadc-ad5f3c50688a" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrOfficeChildStatus.ps1](#)

```
# Get-AsrOfficeChildStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "d4f940ab-401b-4efc-aadc-ad5f3c50688a" -ErrorAction SilentlyContinue
if ($Value -and ($Value."d4f940ab-401b-4efc-aadc-ad5f3c50688a" -eq "1" -or $Value."d4f940ab-401b-4efc-aadc-ad5f3c50688a" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-083] ASR: Block credential stealing from the Windows local security authority subsystem

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2` = `1` (REG_SZ)

Rationale Blocks attempts to open or dump the memory of the Local Security Authority Subsystem Service (lsass.exe). Attackers dump LSASS memory using tools like Mimikatz or Task Manager to extract plaintext credentials, Kerberos tickets, or NTLM password hashes from system memory for lateral movement.

Legacy Impact & Compatibility Administrative diagnostic tools, customized authentication packages, or security scanners that attempt to open the LSASS process handle to read user tokens or perform access auditing will be blocked.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrLsassDump.ps1](#)] ([../implementation_scripts/Configure-AsrLsassDump.ps1](#))

```
# Configure-AsrLsassDump.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrLsassDumpStatus.ps1](#)

```
# Get-AsrLsassDumpStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2" -ErrorAction SilentlyContinue
if ($Value -and ($Value."9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2" -eq "1" -or $Value."9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-084] ASR: Block executable content from email client and webmail

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `be9ba2d9-53ea-4cdc-84e5-9b1eeee46550` = `1` (REG_SZ)

Rationale Prevents executable files (such as .exe, .com, .scr, .vbs, .js, or .pif) from launching directly from email clients (like Outlook) or webmail accessed via browser sessions. This stops phishing attacks where users accidentally launch malicious attachments or download payloads directly from web-based email links.

Legacy Impact & Compatibility Users will be blocked from directly launching attachments that contain scripts, setup files, or macros. They must first save the file to an authorized directory and run it from there (which allows standard real-time scans to run first).

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `be9ba2d9-53ea-4cdc-84e5-9b1eeee46550` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrEmailExecutable.ps1](#)] ([../implementation_scripts/Configure-AsrEmailExecutable.ps1](#))

```
# Configure-AsrEmailExecutable.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "be9ba2d9-53ea-4cdc-84e5-9b1eeee46550" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrEmailExecutableStatus.ps1](#)

```
# Get-AsrEmailExecutableStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "be9ba2d9-53ea-4cdc-84e5-9b1eeee46550" -ErrorAction SilentlyContinue
if ($Value -and ($Value."be9ba2d9-53ea-4cdc-84e5-9b1eeee46550" -eq "1" -or $Value."be9ba2d9-53ea-4cdc-84e5-9b1eeee46550" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-085] ASR: Block executable files from running unless they meet a prevalence, age, or trusted list criterion
 ## Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `01443614-cd74-433a-b99e-2ecdc07bfc25` = `1` (REG_SZ)

Rationale Blocks execution of unrecognized, newly compiled, or low-prevalence executable files. This provides initial protection against zero-day malware campaigns and targeted custom payloads that have not yet established reputation telemetry in the Microsoft Cloud Protection network.

Legacy Impact & Compatibility Developers building and testing custom internal binaries, or administrators installing niche, uncertified third-party software updates will face false positives. Exclusions must be configured for target test directories.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `01443614-cd74-433a-b99e-2ecdc07bfc25` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrLowPrevalence.ps1](#)] ([../implementation_scripts/Configure-AsrLowPrevalence.ps1](#))

```
# Configure-AsrLowPrevalence.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "01443614-cd74-433a-b99e-2ecdc07bfc25" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrLowPrevalenceStatus.ps1](#)

```
# Get-AsrLowPrevalenceStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "01443614-cd74-433a-b99e-2ecdc07bfc25" -ErrorAction SilentlyContinue
if ($Value -and ($Value."01443614-cd74-433a-b99e-2ecdc07bfc25" -eq "1" -or $Value."01443614-cd74-433a-b99e-2ecdc07bfc25" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-086] ASR: Block execution of potentially obfuscated scripts

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `5beb7efe-fd9a-4556-801d-275e5ffc04cc` = `1` (REG_SZ)

Rationale Blocks execution of obfuscated or encrypted scripts (such as PowerShell, VBScript, or JavaScript). Threat actors obfuscate their scripts using base64 encoding, custom string manipulation, or encryption to hide the intent of their code and bypass static file scanning and network detection engines.

Legacy Impact & Compatibility Legitimate administrative tools or third-party provisioning scripts that use heavy minification, obfuscation, or custom encoding wrappers will be blocked. Scripts must be refactored to use readable, signed code.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `5beb7efe-fd9a-4556-801d-275e5ffc04cc` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrObfuscatedScripts.ps1](#)] (../implementation_scripts/Configure-AsrObfuscatedScripts.ps1)

```
# Configure-AsrObfuscatedScripts.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "5beb7efe-fd9a-4556-801d-275e5ffc04cc" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrObfuscatedScriptsStatus.ps1](#)

```
# Get-AsrObfuscatedScriptsStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "5beb7efe-fd9a-4556-801d-275e5ffc04cc" -ErrorAction SilentlyContinue
if ($Value -and ($Value."5beb7efe-fd9a-4556-801d-275e5ffc04cc" -eq "1" -or $Value."5beb7efe-fd9a-4556-801d-275e5ffc04cc" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-087] ASR: Block JavaScript or VBScript from launching downloaded executable content

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `d3e037e1-3eb8-44c8-a917-57927947596d` = `1` (REG_SZ)

Rationale Prevents JavaScript or VBScript running locally from launching executable binaries that were downloaded from the internet. Attackers use malicious scripts inside documents or web browsers to download payloads (like ransomware or trojans) to the disk and launch them using local script engines.

Legacy Impact & Compatibility Local administrative script engines that perform software deployment or download updates and then launch installers will be blocked. These scripts must be replaced by structured configuration agents or signed installers.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `d3e037e1-3eb8-44c8-a917-57927947596d` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrScriptLaunchExe.ps1](#)] ([./../implementation_scripts/Configure-AsrScriptLaunchExe.ps1](#))

```
# Configure-AsrScriptLaunchExe.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "d3e037e1-3eb8-44c8-a917-57927947596d" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrScriptLaunchExeStatus.ps1](#)

```
# Get-AsrScriptLaunchExeStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "d3e037e1-3eb8-44c8-a917-57927947596d" -ErrorAction SilentlyContinue
if ($Value -and ($Value."d3e037e1-3eb8-44c8-a917-57927947596d" -eq "1" -or $Value."d3e037e1-3eb8-44c8-a917-57927947596d" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-088] ASR: Block Office applications from creating executable content

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `3b576869-a4ec-4529-8536-b80a7769e899` = `1` (REG_SZ)

Rationale Prevents Microsoft Office applications (Word, Excel, PowerPoint) from creating or writing executable files (e.g., .exe, .dll, .scr) to the local filesystem. Malicious documents often attempt to drop payloads directly into the local temp folders or AppData directories before executing them.

Legacy Impact & Compatibility Macros or add-ins that legitimately save external binaries or compile executable helper objects locally will be blocked.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `3b576869-a4ec-4529-8536-b80a7769e899` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrOfficeWriteExe.ps1](#)] ([./../implementation_scripts/Configure-AsrOfficeWriteExe.ps1](#))

```
# Configure-AsrOfficeWriteExe.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "3b576869-a4ec-4529-8536-b80a7769e899" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrOfficeWriteExeStatus.ps1](#)

```
# Get-AsrOfficeWriteExeStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "3b576869-a4ec-4529-8536-b80a7769e899" -ErrorAction SilentlyContinue
if ($Value -and ($Value."3b576869-a4ec-4529-8536-b80a7769e899" -eq "1" -or $Value."3b576869-a4ec-4529-8536-b80a7769e899" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-089] ASR: Block Office applications from injecting code into other processes

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84` = `1` (REG_SZ)

Rationale Blocks Microsoft Office applications from writing code or injecting threads directly into external processes. Threat actors use code injection (such as process hollowing or remote thread creation) inside Office macros to hide execution under clean, trusted system binaries like explorer.exe or svchost.exe.

Legacy Impact & Compatibility Office plugins or custom integration tools that inject threads or communicate with database engines using active injection libraries will be blocked.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrOfficeInjection.ps1](#)] ([../implementation_scripts/Configure-AsrOfficeInjection.ps1](#))

```
# Configure-AsrOfficeInjection.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrOfficeInjectionStatus.ps1](#)

```
# Get-AsrOfficeInjectionStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84" -ErrorAction SilentlyContinue
if ($Value -and ($Value."75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84" -eq "1" -or $Value."75668c1f-73b5-4cf0-bb93-3ecf5cb7cc84" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-090] ASR: Block Office communication application from creating child processes

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `26190899-1602-49e8-8b27-eb1d0a1ce869` = `1` (REG_SZ)

Rationale Blocks Microsoft Outlook or other Office communication applications (e.g., Teams, Skype) from creating child processes. This prevents malware payloads delivered through emails, chats, or calendar invites from spawning command-line utilities or scripting environments.

Legacy Impact & Compatibility Outlook integrations or custom add-ins that legitimately spawn helper utilities, such as starting external browsers or custom document handlers directly, may be blocked.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `26190899-1602-49e8-8b27-eb1d0a1ce869` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrOutlookChild.ps1](#)] ([../implementation_scripts/Configure-AsrOutlookChild.ps1](#))

```
# Configure-AsrOutlookChild.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "26190899-1602-49e8-8b27-eb1d0a1ce869" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrOutlookChildStatus.ps1](#)

```
# Get-AsrOutlookChildStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "26190899-1602-49e8-8b27-eb1d0a1ce869" -ErrorAction SilentlyContinue
if ($Value -and ($Value."26190899-1602-49e8-8b27-eb1d0a1ce869" -eq "1" -or $Value."26190899-1602-49e8-8b27-eb1d0a1ce869" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-091] ASR: Block persistence through WMI event subscription

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `e6db77e5-3df2-4cf1-b95a-636979351e5b` = `1` (REG_SZ)

Rationale Blocks threat actors from achieving system persistence by registering permanent Windows Management Instrumentation (WMI) event subscriptions. WMI event subscriptions allow attackers to automatically launch malicious payloads when system triggers occur (like system boot or user logon) without using traditional startup registry keys.

Legacy Impact & Compatibility Management software or diagnostic monitoring tools that legitimately register permanent WMI event filters to track system telemetry will be blocked. Alternatives like Windows services or scheduled tasks must be used.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `e6db77e5-3df2-4cf1-b95a-636979351e5b` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrWmiPersistence.ps1](#)] ([../implementation_scripts/Configure-AsrWmiPersistence.ps1](#))

```
# Configure-AsrWmiPersistence.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "e6db77e5-3df2-4cf1-b95a-636979351e5b" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrWmiPersistenceStatus.ps1](#)

```
# Get-AsrWmiPersistenceStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "e6db77e5-3df2-4cf1-b95a-636979351e5b" -ErrorAction SilentlyContinue
if ($Value -and ($Value."e6db77e5-3df2-4cf1-b95a-636979351e5b" -eq "1" -or $Value."e6db77e5-3df2-4cf1-b95a-636979351e5b" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-092] ASR: Block process creations originating from PSEXEC and WMI commands

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `d1e49aac-8f56-4280-b9ba-993a6d77406c` = `1` (REG_SZ)

Rationale Blocks processes created via WMI commands or PSEXEC remote execution utilities. This directly stops lateral movement attacks where compromised accounts or threat actors attempt to start commands, backdoors, or credential dumpers remotely across domain-joined servers and workstations.

Legacy Impact & Compatibility Disrupts remote administration tools, monitoring agents, and network scanners that rely on WMI/PSEXEC. For Domain Controllers or critical servers, this rule is often set to **Audit** mode (`2`) rather than hard Block mode (`1`) to prevent administrative downtime.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `d1e49aac-8f56-4280-b9ba-993a6d77406c` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrPsexecWmi.ps1](#)] ([../implementation_scripts/Configure-AsrPsexecWmi.ps1](#))

```
# Configure-AsrPsexecWmi.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "d1e49aac-8f56-4280-b9ba-993a6d77406c" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrPsexecWmiStatus.ps1](#)

```
# Get-AsrPsexecWmiStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "d1e49aac-8f56-4280-b9ba-993a6d77406c" -ErrorAction SilentlyContinue
if ($Value -and ($Value."d1e49aac-8f56-4280-b9ba-993a6d77406c" -eq "1" -or $Value."d1e49aac-8f56-4280-b9ba-993a6d77406c" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-093] ASR: Block untrusted and unsigned processes that run from USB

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4` = `1` (REG_SZ)

Rationale Blocks the execution of unsigned or untrusted processes on removable storage devices (USB drives, external SSDs). This stops physical access vectors, rogue USB drops, and automated worm propagation techniques from running unauthorized installers or scripts.

Legacy Impact & Compatibility Administrators or developers will be unable to run unsigned utilities directly from portable flash drives. Software must be copied to local trusted storage, or the binary must possess a trusted digital signature.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrUsbUnsigned.ps1](#)] ([../implementation_scripts/Configure-AsrUsbUnsigned.ps1](#))

```
# Configure-AsrUsbUnsigned.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrUsbUnsignedStatus.ps1](#)

```
# Get-AsrUsbUnsignedStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4" -ErrorAction SilentlyContinue
if ($Value -and ($Value."b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4" -eq "1" -or $Value."b2b3f03d-6a65-4f7b-a9c7-1c7ef74a9ba4" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-094] ASR: Block Win32 API calls from Office macros

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b` = `1` (REG_SZ)

Rationale Blocks VBA macros inside Microsoft Office documents from invoking Win32 API calls. Malicious documents use macros to call kernel memory functions (such as VirtualAlloc, WriteProcessMemory, or CreateThread) to load and execute shellcode in memory without dropping files to disk, bypassing file scanners.

Legacy Impact & Compatibility Advanced business spreadsheets or database documents that utilize Win32 API calls for custom UI rendering, network calls, or system commands will fail. They must be rewritten using standard secure VBA structures or add-in APIs.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-AsrOfficeWin32Calls.ps1] (../implementation_scripts/Configure-AsrOfficeWin32Calls.ps1)

```
# Configure-AsrOfficeWin32Calls.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrOfficeWin32CallsStatus.ps1](#)

```
# Get-AsrOfficeWin32CallsStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b" -ErrorAction SilentlyContinue
if ($Value -and ($Value."92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b" -eq "1" -or $Value."92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-095] ASR: Use advanced protection against ransomware

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules` * `c1db55ab-c21a-4637-bb3f-a12568109d35` = `1` (REG_SZ)

Rationale Enables advanced behavioral heuristics and cloud analytics checks on files that attempt to modify multiple user files, detect signature-less encryption behavior, and block rapid write activity to prevent ransomware from encrypting system and user documents.

Legacy Impact & Compatibility Minor performance overhead when scanning active files during large batch edits or heavy database updates.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Windows Defender Exploit Guard\Attack Surface Reduction 2. Set 'Configure Attack Surface Reduction rules' to 'Enabled' 3. Click 'Show...' and add the rule GUID `c1db55ab-c21a-4637-bb3f-a12568109d35` as Value Name, with Value set to `1` (Block).

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-AsrRansomware.ps1](#)] ([../implementation_scripts/Configure-AsrRansomware.ps1](#))

```
# Configure-AsrRansomware.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
if (-not (Test-Path $AsrRulesPath)) { New-Item -Path $AsrRulesPath -Force | Out-Null }
Set-ItemProperty -Path $AsrRulesPath -Name "c1db55ab-c21a-4637-bb3f-a12568109d35" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-AsrRansomwareStatus.ps1](#)

```
# Get-AsrRansomwareStatus.ps1
$AsrRulesPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Windows Defender Exploit Guard\ASR\Rules"
$Value = Get-ItemProperty -Path $AsrRulesPath -Name "c1db55ab-c21a-4637-bb3f-a12568109d35" -ErrorAction SilentlyContinue
if ($Value -and ($Value."c1db55ab-c21a-4637-bb3f-a12568109d35" -eq "1" -or $Value."c1db55ab-c21a-4637-bb3f-a12568109d35" -eq 1)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (ASR rules config) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-072] Configure Threat Severity Default Quarantine Actions

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Threats` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Threats` * `Threats\_ThreatSeverityDefaultAction` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Threats\ThreatSeverityDefaultAction` * `1` = `2` (REG_DWORD)

Rationale By default, Defender may prompt users or take actions (like clean/ignore) that leave malware remnants on the filesystem. Configuring default quarantine actions for all severities (low, medium, high, severe) ensures automated containment.

Legacy Impact & Compatibility None.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Defender Antivirus\Threats 2. Set 'Specify threat alert levels at which default action should not be taken when detected' to 'Enabled' 3. Click 'Show...' and enter threat levels (1 → 2, 2 → 2, 4 → 2, 5 → 2)

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderThreatActions.ps1](#)] (./implementation_scripts/Configure-DefenderThreatActions.ps1)

```
# Configure-DefenderThreatActions.ps1
$ThreatsPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Threats"
if (-not (Test-Path $ThreatsPath)) { New-Item -Path $ThreatsPath -Force | Out-Null }
Set-ItemProperty -Path $ThreatsPath -Name "Threats_ThreatSeverityDefaultAction" -Value 1 -Type DWord -Force
$ThreatsSevPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Threats\ThreatSeverityDefaultAction"
if (-not (Test-Path $ThreatsSevPath)) { New-Item -Path $ThreatsSevPath -Force | Out-Null }
Set-ItemProperty -Path $ThreatsSevPath -Name "1" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $ThreatsSevPath -Name "2" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $ThreatsSevPath -Name "4" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $ThreatsSevPath -Name "5" -Value 2 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderThreatActionsStatus.ps1](#)

```
# Get-DefenderThreatActionsStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Threats" -Name "Threats_ThreatSeverityDefaultAction" -ErrorAction SilentlyContinue
$RegSev = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Threats\ThreatSeverityDefaultAction" -ErrorAction SilentlyContinue
if (($Reg -and $Reg.Threats_ThreatSeverityDefaultAction -eq 1) -and
    ($RegSev -and $RegSev.1 -eq 2 -and $RegSev.2 -eq 2 -and $RegSev.4 -eq 2 -and $RegSev.5 -eq 2)) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-073] Configure Family Options UI Lockdown

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: Low * **GPO Path / Registry Location**: * **GPO Path**: 'Computer Configuration\Administrative Templates\Windows Components\Windows Security\Family options' * **Registry Location**: * 'HKLM\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\Family options' * 'UILockdown' = '1' (REG_DWORD)

Rationale Locking down non-essential components of the Windows Security Center interface prevents users from tampering with parental or diagnostic UI controls on enterprise assets.

Legacy Impact & Compatibility None.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Security\Family options 2. Set 'Hide the Family options area' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderFamilyLockdown.ps1](#)] ([../implementation_scripts/Configure-DefenderFamilyLockdown.ps1](#))

```
# Configure-DefenderFamilyLockdown.ps1
$FamilyPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\Family options"
if (-not (Test-Path $FamilyPath)) { New-Item -Path $FamilyPath -Force | Out-Null }
Set-ItemProperty -Path $FamilyPath -Name "UILockdown" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderFamilyLockdownStatus.ps1](#)

```
# Get-DefenderFamilyLockdownStatus.ps1
$Reg = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\Family options" -Name "UILockdown"
-ErrorAction SilentlyContinue
if ($Reg -and $Reg.UILockdown -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-074] Configure Tamper Protection

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\Windows Security\Tamper Protection` * **Registry Location**: * `HKLM\SOFTWARE\Microsoft\Windows Defender\Features` * `TamperProtection` = `5` (REG_DWORD)

Rationale Tamper Protection prevents local administrators or compromised system accounts from disabling Windows Defender services, real-time scanning, or modifying active exclusions locally. This blocks a primary malware persistence vector.

Legacy Impact & Compatibility Local registry modifications to Defender configurations will be ignored. All changes must originate from central templates or cloud console panels.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\Windows Security\Tamper Protection 2. Set 'Protect Windows Security settings from tampering' to 'Enabled' (Block or On depending on ADMX version)

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderTamperProtection.ps1](#)] ([./implementation_scripts/Configure-DefenderTamperProtection.ps1](#))

```
# Configure-DefenderTamperProtection.ps1
$FeaturesPath = "HKLM:\SOFTWARE\Microsoft\Windows Defender\Features"
if (-not (Test-Path $FeaturesPath)) { New-Item -Path $FeaturesPath -Force | Out-Null }
try {
    Set-ItemProperty -Path $FeaturesPath -Name "TamperProtection" -Value 5 -Type DWord -ErrorAction Stop -Force
} catch {
    Write-Warning "Registry blocked. Tamper Protection registry key is normally protected by TrustedInstaller. Ensure GPO setting is applied."
}
```

To audit the hardening status: [Download Script: Get-DefenderTamperProtectionStatus.ps1](#)

```
# Get-DefenderTamperProtectionStatus.ps1
$FeaturesPath = "HKLM:\SOFTWARE\Microsoft\Windows Defender\Features"
$TamperVal = Get-ItemProperty -Path $FeaturesPath -Name "TamperProtection" -ErrorAction SilentlyContinue
if ($TamperVal -and $TamperVal.TamperProtection -eq 5) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-075] Configure Sandbox Execution Environment

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Environment` * **Registry Location**: * `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Environment` * `MP\_FORCE\_USE\_SANDBOX` = `1` (REG_SZ)

Rationale Forcing the Windows Defender scanning service (MsMpEng.exe) to run in a restricted AppContainer sandbox prevents privilege escalation. If an attacker exploits a parsing vulnerability in the engine, the compromise is contained inside the sandbox.

Legacy Impact & Compatibility A system reboot is required to initialize the scanning process within the AppContainer sandbox.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Preferences\Windows Settings\Environment 2. Right-click and select New → Environment Variable 3. Configure Action: Update, Type: System, Name: MP_FORCE_USE_SANDBOX, Value: 1

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderSandbox.ps1](#)] ([./implementation_scripts/Configure-DefenderSandbox.ps1](#))

```
# Configure-DefenderSandbox.ps1
$EnvPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Environment"
if (-not (Test-Path $EnvPath)) { New-Item -Path $EnvPath -Force | Out-Null }
Set-ItemProperty -Path $EnvPath -Name "MP_FORCE_USE_SANDBOX" -Value "1" -Type String -Force
```

To audit the hardening status: [Download Script: Get-DefenderSandboxStatus.ps1](#)

```
# Get-DefenderSandboxStatus.ps1
$EnvPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Environment"
$SandboxVar = Get-ItemProperty -Path $EnvPath -Name "MP_FORCE_USE_SANDBOX" -ErrorAction SilentlyContinue
if ($SandboxVar -and $SandboxVar.MP_FORCE_USE_SANDBOX -eq "1") {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-076] Configure AMSI Authenticode Signature Verification

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Preferences\Windows Settings\Registry` * **Registry Location**: * `HKLM\SOFTWARE\Microsoft\AMSI` * `FeatureBits` = `2` (REG_DWORD)

Rationale Enforcing signature checks on registered Antimalware Scan Interface (AMSI) providers blocks attackers from registering unsigned rogue AMSI provider DLLs to bypass script analysis.

Legacy Impact & Compatibility Third-party antivirus/security providers must register using signed Authenticode binaries to prevent being blocked.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Preferences\Windows Settings\Registry 2. Right-click and select New → Registry Item 3. Configure Action: Update, Hive: HKEY_LOCAL_MACHINE, Key Path: SOFTWARE\Microsoft\AMSI, Value Name: FeatureBits, Value Type: REG_DWORD, Value Data: 2

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderAmsiSignature.ps1](#)] ([./implementation_scripts/Configure-DefenderAmsiSignature.ps1](#))

```
# Configure-DefenderAmsiSignature.ps1
$AmsiPath = "HKLM:\SOFTWARE\Microsoft\AMSI"
if (-not (Test-Path $AmsiPath)) { New-Item -Path $AmsiPath -Force | Out-Null }
Set-ItemProperty -Path $AmsiPath -Name "FeatureBits" -Value 2 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderAmsiSignatureStatus.ps1](#)

```
# Get-DefenderAmsiSignatureStatus.ps1
$AmsiPath = "HKLM:\SOFTWARE\Microsoft\AMSI"
if (Test-Path $AmsiPath) {
    $AmsiBits = Get-ItemProperty -Path $AmsiPath -Name "FeatureBits" -ErrorAction SilentlyContinue
    if ($AmsiBits -and $AmsiBits.FeatureBits -eq 2) {
        Write-Output "Compliant"
        exit 0
    }
}
Write-Output "Non-Compliant"
exit 1
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-077] Configure File Explorer SmartScreen

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\File Explorer` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows\System` * `EnableSmartScreen` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows\System` * `ShellSmartScreenLevel` = `Block` (REG_SZ)

Rationale Windows Defender SmartScreen protects users from running unrecognized or potentially malicious applications downloaded from the internet. Configuring the level to 'Block' prevents users from bypassing security warnings.

Legacy Impact & Compatibility Users will be unable to run unrecognized custom scripts or executables without administrative approval.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\File Explorer 2. Set 'Configure Windows Defender SmartScreen' to 'Enabled' 3. Select 'Require approval from an administrator before running unrecognized software' under Options

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderSmartScreen.ps1](#)] ([./implementation_scripts/Configure-DefenderSmartScreen.ps1](#))

```
# Configure-DefenderSmartScreen.ps1
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System"
if (-not (Test-Path $Path)) { New-Item -Path $Path -Force | Out-Null }
Set-ItemProperty -Path $Path -Name "EnableSmartScreen" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $Path -Name "ShellSmartScreenLevel" -Value "Block" -Type String -Force
```

To audit the hardening status: [Download Script: Get-DefenderSmartScreenStatus.ps1](#)

```
# Get-DefenderSmartScreenStatus.ps1
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System"
if (Test-Path $Path) {
    $Enable = Get-ItemProperty -Path $Path -Name "EnableSmartScreen" -ErrorAction SilentlyContinue
    $Level = Get-ItemProperty -Path $Path -Name "ShellSmartScreenLevel" -ErrorAction SilentlyContinue
    if ($Enable -and $Enable.EnableSmartScreen -eq 1 -and $Level -and $Level.ShellSmartScreenLevel -eq "Block") {
        Write-Output "Compliant"
        exit 0
    }
}
Write-Output "Non-Compliant"
exit 1
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-078] Disable OneDrive File Sync

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: Medium * **GPO Path / Registry Location**: * **GPO Path**: `Computer Configuration\Administrative Templates\Windows Components\OneDrive` * **Registry Location**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows\OneDrive` * `DisableFileSyncNGSC` = `1` (REG_DWORD)

Rationale Preventing OneDrive files from syncing automatically protects endpoints against automated synchronization of encrypted files during a ransomware event, and prevents unauthorized data exfiltration.

Legacy Impact & Compatibility OneDrive file synchronization will be completely disabled. Alternate storage paths must be used.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: Computer Configuration\Administrative Templates\Windows Components\OneDrive 2. Set 'Prevent the usage of OneDrive for file storage' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DisableOneDriveSync.ps1](#)] ([../implementation_scripts/Configure-DisableOneDriveSync.ps1](#))

```
# Configure-DisableOneDriveSync.ps1
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\OneDrive"
if (-not (Test-Path $Path)) { New-Item -Path $Path -Force | Out-Null }
Set-ItemProperty -Path $Path -Name "DisableFileSyncNGSC" -Value 1 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DisableOneDriveSyncStatus.ps1](#)

```
# Get-DisableOneDriveSyncStatus.ps1
$Path = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\OneDrive"
$Reg = Get-ItemProperty -Path $Path -Name "DisableFileSyncNGSC" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.DisableFileSyncNGSC -eq 1) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-079] Enforce Antivirus Scan on Opening Attachments

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: `User Configuration\Administrative Templates\Windows Components\Attachment Manager` * **Registry Location**: * `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Attachments` * `ScanWithAntiVirus` = `3` (REG_DWORD)

Rationale Forcing the Attachment Manager to notify the registered antivirus product when a user opens files downloaded from the web or email clients prevents initial access vectors.

Legacy Impact & Compatibility Opening unrecognized email attachments may have a slight latency while the file is analyzed.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: User Configuration\Administrative Templates\Windows Components\Attachment Manager 2. Set 'Notify antivirus programs when opening attachments' to 'Enabled'

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-DefenderAttachmentScan.ps1](#)] ([../implementation_scripts/Configure-DefenderAttachmentScan.ps1](#))

```
# Configure-DefenderAttachmentScan.ps1
$Path = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Attachments"
if (-not (Test-Path $Path)) { New-Item -Path $Path -Force | Out-Null }
Set-ItemProperty -Path $Path -Name "ScanWithAntiVirus" -Value 3 -Type DWord -Force
```

To audit the hardening status: [Download Script: Get-DefenderAttachmentScanStatus.ps1](#)

```
# Get-DefenderAttachmentScanStatus.ps1
$Path = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Attachments"
$Reg = Get-ItemProperty -Path $Path -Name "ScanWithAntiVirus" -ErrorAction SilentlyContinue
if ($Reg -and $Reg.ScanWithAntiVirus -eq 3) {
    Write-Output "Compliant"
    exit 0
} else {
    Write-Output "Non-Compliant"
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark**: Section 18.9 (Windows Defender Antivirus configuration parameters) * **ANSSI Active Directory Hardening Guide**: Protective controls baselines on client workstations

[REQ-END-008] WSUS Client Configuration

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Paths**: * `Computer Configuration\Administrative Templates\Windows Components\Windows Update` * `Computer Configuration\Administrative Templates\Windows Components\Delivery Optimization` * **Registry Locations**: * `HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate` * `WUServer` (REG_SZ) * `WUStatusServer` (REG_SZ) * `DoNotConnectToWindowsUpdateInternetLocations` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU` * `NoAutoUpdate` = `0` (REG_DWORD) * `AUOptions` = `4` (REG_DWORD) * `UseWUServer` = `1` (REG_DWORD) * `HKLM\SOFTWARE\Policies\Microsoft\Windows\DeliveryOptimization` * `DODownloadMode` = `2` (REG_DWORD)

Rationale In an isolated, air-gapped network, workstations cannot connect directly to Microsoft's online Update servers. If the system is left in its default configuration: 1. **DNS/Firewall Pollution**: Workstations will continuously attempt to resolve and connect to public Windows Update URLs (e.g., `*.update.microsoft.com`), filling firewall and local DNS resolver cache logs with timeouts and block events. 2. **Missing Updates**: Workstations will fail to receive security patches, critical updates, and Windows Defender definitions. 3. **Control Bypass**: Attackers or unapproved software could attempt to install out-of-band features or packages if update routes are not explicitly locked to internal sources.

Enforcing the intranet update service location redirects all system update queries to the local WSUS server. Furthermore, enforcing Windows **Delivery Optimization** download mode to [Group \(2\)](#) limits peer-to-peer update sharing strictly to computers within the same active directory domain/group or local subnet boundaries, reducing bandwidth constraints on WAN/intranet segments and preventing unmanaged peer sharing.

Legacy Impact & Compatibility * **WSUS Dependency**: The local Windows Server Update Services (WSUS) server must remain online and functional. If the WSUS server is unreachable, workstations will be unable to query, download, or apply security patches. * **Administrative Burden**: Enterprise administrators must manually synchronize the WSUS database (sneakernet transfer of metadata and updates) to ensure new patches are made available to clients. * **Delivery Optimization limits**: Group mode restricts Delivery Optimization cache sharing to standard networks. If endpoints span multiple separated sites without route aggregation, update replication times could increase slightly.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`).
2. Create or edit a GPO linked to the workstations OU (e.g., `GPO_Hardening_Workstations`).
3. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Windows Update`
4. Configure the following settings:
 - **Policy**: `Configure Automatic Updates`
 - **Setting**: `Enabled`
 - **Configure automatic updating**: `4 - Auto download and schedule the install`
5. Configure the service location:
 - **Policy**: `Specify intranet Microsoft update service location`
 - **Setting**: `Enabled`
 - **Set the intranet update service for detecting updates**: `http://local-wsus.domain.local:8530` (Replace with your internal WSUS FQDN or IP)
 - **Set the intranet statistics server**: `http://local-wsus.domain.local:8530`
6. Navigate to:
 - **Policy**: `Do not connect to any Windows Update Internet locations`
 - **Setting**: `Enabled` (blocks fallback to public servers)
7. Navigate to: `Computer Configuration\Administrative Templates\Windows Components\Delivery Optimization`
8. Configure the following settings:
 - **Policy**: `Download Mode`
 - **Setting**: `Enabled`
 - **Download Mode**: `Group (2)`

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to configure registry keys to enforce local WSUS parameters and Delivery Optimization download mode.

[Download Script: Set-WSUSClientConfiguration.ps1](#)

```
# Set-WSUSClientConfiguration.ps1
# Configures local registry keys to point the Windows Update client to the intranet WSUS server and enforces DO Group mode.

Write-Host "--- Configuring WSUS Client Settings ---" -ForegroundColor Cyan

$WUPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate"
$WUAUPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU"
$DOPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeliveryOptimization"

# 1. Create keys if they do not exist
if (-not (Test-Path $WUPath)) {
    New-Item -Path $WUPath -Force | Out-Null
}
if (-not (Test-Path $WUAUPath)) {
    New-Item -Path $WUAUPath -Force | Out-Null
}
if (-not (Test-Path $DOPath)) {
    New-Item -Path $DOPath -Force | Out-Null
}

# Define intranet WSUS URL
$WSUSServer = "http://local-wsus.domain.local:8530"

# 2. Configure update location and statistics server
Set-ItemProperty -Path $WUPath -Name "WUServer" -Value $WSUSServer -Type String -Force
Set-ItemProperty -Path $WUPath -Name "WUStatusServer" -Value $WSUSServer -Type String -Force
Set-ItemProperty -Path $WUPath -Name "DoNotConnectToWindowsUpdateInternetLocations" -Value 1 -Type DWord -Force

# 3. Configure Automatic Updates behavior (AUOptions = 4: Auto Download & Schedule)
Set-ItemProperty -Path $WUAUPath -Name "NoAutoUpdate" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $WUAUPath -Name "AUOptions" -Value 4 -Type DWord -Force
Set-ItemProperty -Path $WUAUPath -Name "UseWUServer" -Value 1 -Type DWord -Force

# 4. Enforce DODownloadMode = 2 (Group)
Set-ItemProperty -Path $DOPath -Name "DODownloadMode" -Value 2 -Type DWord -Force

Write-Host "[+] Local WSUS parameters and Delivery Optimization download mode applied." -ForegroundColor Green
```

To audit the WSUS client configuration status: [Download Script: Test-WSUSClientStatus.ps1](#)

```
# Test-WSUSClientStatus.ps1
# Audits registry values to verify WSUS server assignment and Delivery Optimization configuration.

Write-Host "--- Auditing WSUS Client Settings ---" -ForegroundColor Cyan

$WUPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate"
$WUAUPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU"
$DOPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeliveryOptimization"

$WUServerProp = Get-ItemProperty -Path $WUPath -Name "WUServer" -ErrorAction SilentlyContinue
$WUStatusProp = Get-ItemProperty -Path $WUPath -Name "WUStatusServer" -ErrorAction SilentlyContinue
$UseWUServerProp = Get-ItemProperty -Path $WUAUPath -Name "UseWUServer" -ErrorAction SilentlyContinue

$WUServerVal = if ($WUServerProp) { $WUServerProp.WUServer } else { "" }
$WUStatusVal = if ($WUStatusProp) { $WUStatusProp.WUStatusServer } else { "" }
$UseWUVal = if ($UseWUServerProp) { $UseWUServerProp.UseWUServer } else { 0 }

$ServerColor = if ($WUServerVal -like "http*") { "Green" } else { "Red" }
$UseColor = if ($UseWUVal -eq 1) { "Green" } else { "Red" }

Write-Host "    - Intranet WUServer: $WUServerVal" -ForegroundColor $ServerColor
Write-Host "    - Intranet WUStatusServer: $WUStatusVal" -ForegroundColor $ServerColor
Write-Host "    - UseWUServer Active: $UseWUVal (Required = 1)" -ForegroundColor $UseColor

# Audit Delivery Optimization
$DOPVal = if (Test-Path $DOPath) { (Get-ItemProperty -Path $DOPath -Name "DODownloadMode" -ErrorAction SilentlyContinue).DODownloadMode } else { $null }
$DOPColor = if ($DOPVal -eq 2) { "Green" } else { "Red" }
Write-Host "    - Delivery Optimization DODownloadMode: $($DOPVal | Out-String).Trim() (Expected = 2)" -ForegroundColor $DOPColor
```

 Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark***: Section 18.2.2 (Specify intranet Microsoft update service location), Section 18.2.3 (Do not connect to any Windows Update Internet locations) * **Microsoft Security Baselines***: Windows Update client baseline policies. * **DoD Windows 11 STIG***: Delivery Optimization DODownloadMode configuration requirements.

[REQ-END-009] Enable UEFI Secure Boot

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * UEFI Firmware configuration (Hardware/BIOS level) * HKLM\SYSTEM\CurrentControlSet\Control\SecureBoot\State * `UEFISecureBootEnabled` = `1` (REG_DWORD)

Rationale Secure Boot is a security standard developed by members of the PC industry to help ensure that a device boots using only software that is trusted by the Original Equipment Manufacturer (OEM).

When the PC starts, the firmware checks the signature of each piece of boot software, including UEFI firmware drivers (also known as Option ROMs), EFI applications, and the operating system. If the signatures are valid, the PC boots, and the firmware gives control to the operating system.

If Secure Boot is disabled:

1. **Bootkits & Rootkits:** Attackers with physical access or local administrator privileges can replace the system bootloader with a malicious bootloader (bootkit). This bootkit executes before the Windows operating system loads, allowing it to bypass all Windows security controls, disable antivirus software, and run completely undetected.
2. **Virtualization-Based Security:** Advanced Windows defenses (like Credential Guard and Device Guard) depend on hardware-rooted trust. If Secure Boot is disabled, Virtualization-Based Security (VBS) cannot verify platform integrity, rendering these protections ineffective.

Legacy Impact & Compatibility * **BIOS Mode Conversion**: Systems running in legacy BIOS mode (Compatibility Support Module - CSM) instead of Native UEFI cannot use Secure Boot. Converting these systems requires changing partition styles from MBR to GPT (using tools like `MBR2GPT.exe`) and changing firmware settings; refer to [REQ-END-013 - UEFI Firmware Security Hardening](#08-endpoints-configure-uefi-security-md) for firmware settings. Improper conversion can cause boot failures if not executed correctly. * **Dual-Boot Systems**: If the workstation dual-boots with unsigned Linux distributions or runs legacy recovery media, the firmware will reject the bootloader, preventing boot.

Implementation Steps

Option A: Manual UEFI Firmware Configuration (Preferred)

UEFI Secure Boot must be enabled in the hardware firmware menu directly (BIOS settings) during system startup:

1. Turn on or restart the workstation and access the UEFI utility screen by pressing the vendor-specific key during POST (typically Delete, F2, F10, or F12).
2. Navigate to the **Security** or **Secure Boot** section:
 - Ensure **Secure Boot** is set to **Enabled**.
 - Ensure the **Secure Boot Mode** is set to **Deployed** or **User Mode** (not Setup Mode).
3. Save the configuration and restart the workstation.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Since Secure Boot is a hardware firmware configuration, it cannot be turned on from within Windows using registry settings. However, you can programmatically audit the state of Secure Boot to flag non-compliant hardware.

Run the following script to check the status of Secure Boot on the local machine:

[Download Script: Audit-SecureBoot.ps1](#)

```
# Audit-SecureBoot.ps1
# Description: Queries UEFI Secure Boot parameters and audits UEFI Secure Boot status.

Write-Host "--- Auditing UEFI Secure Boot ---" -ForegroundColor Cyan

$script:NonCompliant = $false

# 1. Verify boot environment type
if ($env:firmware_type -eq "UEFI") {
    Write-Host "    - Boot Environment Type: UEFI" -ForegroundColor Green
} else {
    Write-Host "    - VULNERABLE: System booted in Legacy BIOS mode (CSM enabled) or firmware type is unrecognized." -ForegroundColor Red
    $script:NonCompliant = $true
}

# 2. Verify Secure Boot status
try {
    # Confirm-SecureBootUEFI returns $true if Secure Boot is active, $false if disabled,
    # and throws an exception if the platform does not support UEFI or Secure Boot.
    $SecureBootState = Confirm-SecureBootUEFI -ErrorAction Stop

    $Color = if ($SecureBootState -eq $true) { "Green" } else { "Red" }
    Write-Host "    - Secure Boot Active: $SecureBootState" -ForegroundColor $Color
    if ($SecureBootState -eq $false) { $script:NonCompliant = $true }
} catch [System.PlatformNotSupportedException] {
    Write-Host "    - VULNERABLE: UEFI Secure Boot is not supported on this platform (Legacy BIOS mode)." -ForegroundColor Red
    $script:NonCompliant = $true
} catch {
    # If cmdlet throws unauthorized access or not enabled error
    Write-Host "    - VULNERABLE: Secure Boot is disabled in firmware or cannot be verified. Error: $($_.Exception.Message)" -ForegroundColor Red
    $script:NonCompliant = $true
}

if ($script:NonCompliant) {
    exit 1
}
```

Sources & Compliance References * **CIS Microsoft Windows 10/11 Benchmark:** Section 18.8 (VBS Prerequisites) * ****ANSSI AD Hardening Guide**:** Recommendations regarding hardware platform integrity.

[REQ-END-010] Enable VBS and Credential Guard

Target Scope * **Applicable Systems**: Tier 2 client workstations and member servers. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: Computer Configuration\Administrative Templates\System\Device Guard\Turn On Virtualization Based Security * **Registry Location**: HKLM\SOFTWARE\Policies\Microsoft\Windows\DeviceGuard * `EnableVirtualizationBasedSecurity` = `1` (REG_DWORD) * `RequirePlatformSecurityFeatures` = `1` (REG_DWORD, Secure Boot) * `HypervisorEnforcedCodeIntegrity` = `1` (REG_DWORD) * `LsaCfgFlags` = `1` (REG_DWORD, Credential Guard Enabled with UEFI Lock) * `ConfigureSystemGuardLaunch` = `1` (REG_DWORD, Secure Launch Enabled) * `HVCIMATRequired` = `1` (REG_DWORD, Require UEFI Memory Attributes Table)

Rationale Virtualization-Based Security (VBS) uses hardware virtualization features to create and isolate a secure region of memory from the normal operating system.

Windows Defender **Credential Guard** runs inside this isolated VBS environment (known as the secure kernel). By moving NTLM password hashes, Kerberos Ticket Granting Tickets (TGTs), and other domain credentials into this virtualized container, Credential Guard ensures they are inaccessible to the standard operating system.

If VBS and Credential Guard are not enabled:

1. **LSASS Access**: Attackers running with administrative rights on the workstation can query the LSASS process memory space and extract Active Directory authentication tokens using memory-dumping tools (e.g., Mimikatz).
2. **Pass-the-Hash / Pass-the-Ticket**: Attackers can use the extracted hashes or Kerberos tickets to log on to other domain systems, leading to rapid lateral movement and domain compromise.

Enforcing VBS and Credential Guard prevents in-memory credential harvesting, breaking the primary lateral movement escalation vector.

Legacy Impact & Compatibility * **Virtualization Conflicts**: Third-party virtualization software (such as legacy versions of VMware Workstation or VirtualBox) that do not support nested virtualization or integration with Windows Hyper-V will fail to run when VBS is active. * **Hardware Requirements**: Systems must support CPU virtualization (Intel VT-x or AMD-V), Second Level Address Translation (SLAT), and have secure firmware (UEFI, Secure Boot, IOMMU / DMA protection) as strict pre-requisites. Refer to [REQ-END-013 - UEFI Firmware Security Hardening](#08-endpoints-configure-uefi-security-md) and [REQ-END-014 - Enable Hardware Virtualization and DMA Protection](#08-endpoints-enable-hardware-virtualization-and-dma-protection-md) to ensure these platform security features are fully enabled in the firmware. Older client hardware that does not support these specifications cannot run Credential Guard.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** (`gpmc.msc`).
2. Create or edit a GPO linked to the workstations OU (e.g., `GPO_Hardening_Workstations`).
3. Navigate to: `Computer Configuration\Administrative Templates\System\Device Guard`
4. Configure the setting:
 - **Policy**: `Turn On Virtualization Based Security`
 - **Setting**: `Enabled`
 - **Select Platform Security Level**: `Secure Boot`
 - **Virtualization Based Protection of Code Integrity**: `Enabled with UEFI lock`
 - **Credential Guard Configuration**: `Enabled with UEFI lock`
 - **Secure Launch Configuration**: `Enabled`
 - **Require UEFI Memory Attributes Table**: `Enabled`

Note: The "Enabled with UEFI lock" setting ensures that an administrator cannot remotely disable Credential Guard via registry changes alone; it requires physical access to the machine to clear UEFI variables on reboot.

Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to configure registry keys to enable VBS and Credential Guard.

[Download Script: Enable-VBSCredentialGuard.ps1](#)

```
# Enable-VBSCredentialGuard.ps1
# Configures local registry keys to activate VBS and Credential Guard.

Write-Host "--- Enforcing VBS & Credential Guard ---" -ForegroundColor Cyan

$DeviceGuardPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeviceGuard"

if (-not (Test-Path $DeviceGuardPath)) {
    New-Item -Path $DeviceGuardPath -Force | Out-Null
}

# Enable Virtualization-Based Security (VBS)
Set-ItemProperty -Path $DeviceGuardPath -Name "EnableVirtualizationBasedSecurity" -Value 1 -Type DWord
# RequirePlatformSecurityFeatures = 1 (Secure Boot)
Set-ItemProperty -Path $DeviceGuardPath -Name "RequirePlatformSecurityFeatures" -Value 1 -Type DWord
# HypervisorEnforcedCodeIntegrity = 1 (HVCI / Memory Integrity Enabled)
Set-ItemProperty -Path $DeviceGuardPath -Name "HypervisorEnforcedCodeIntegrity" -Value 1 -Type DWord
# LsaCfgFlags = 1 (Credential Guard Enabled with UEFI Lock)
Set-ItemProperty -Path $DeviceGuardPath -Name "LsaCfgFlags" -Value 1 -Type DWord
# ConfigureSystemGuardLaunch = 1 (Secure Launch Enabled)
Set-ItemProperty -Path $DeviceGuardPath -Name "ConfigureSystemGuardLaunch" -Value 1 -Type DWord
# HVCIMATRequired = 1 (Require UEFI Memory Attributes Table)
Set-ItemProperty -Path $DeviceGuardPath -Name "HVCIMATRequired" -Value 1 -Type DWord

Write-Host "[+] VBS and Credential Guard registry settings applied. (Reboot required)." -ForegroundColor Green
```

To audit VBS and Credential Guard status using WMI: [Download Script: Test-VBSCredentialGuard.ps1](#)

```
# Test-VBSCredentialGuard.ps1
# Queries the local Win32_DeviceGuard class to verify active protection states.

Write-Host "--- Auditing Virtualization-Based Security Baseline ---" -ForegroundColor Cyan

try {
    $DG = Get-CimInstance -Namespace "Root\Microsoft\Windows\DeviceGuard" -ClassName "Win32_DeviceGuard" -ErrorAction Stop

    # SecurityServicesRunning: 1 = Credential Guard, 2 = HVCI
    $CredGuardRunning = $DG.SecurityServicesRunning -contains 1
    $HvciRunning = $DG.SecurityServicesRunning -contains 2

    $VbsColor = if ($DG.VirtualizationBasedSecurityStatus -eq 2) { "Green" } else { "Red" }
    $CredColor = if ($CredGuardRunning) { "Green" } else { "Red" }
    $HvciColor = if ($HvciRunning) { "Green" } else { "Red" }

    Write-Host "    - VBS Status: $($DG.VirtualizationBasedSecurityStatus) (Required = 2 [Running])" -ForegroundColor $VbsColor
    Write-Host "    - Credential Guard Running: $CredGuardRunning (Required = True)" -ForegroundColor $CredColor
    Write-Host "    - Hypervisor Code Integrity Running: $HvciRunning (Required = True)" -ForegroundColor $HvciColor

    # Query registry properties for System Guard and UEFI MAT
    $SystemGuard = (Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeviceGuard" -Name "ConfigureSystemGuardLaunch" -ErrorAction SilentlyContinue).ConfigureSystemGuardLaunch
    $MatRequired = (Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeviceGuard" -Name "HVCIMATRequired" -ErrorAction SilentlyContinue).HVCIMATRequired

    $SgColor = if ($SystemGuard -eq 1) { "Green" } else { "Red" }
    $MatColor = if ($MatRequired -eq 1) { "Green" } else { "Red" }

    Write-Host "    - System Guard Secure Launch: $SystemGuard (Required = 1)" -ForegroundColor $SgColor
    Write-Host "    - UEFI Memory Attributes Table Required: $MatRequired (Required = 1)" -ForegroundColor $MatColor
} catch {
    Write-Host "    - VULNERABLE: DeviceGuard WMI class could not be queried. VBS is likely disabled." -ForegroundColor Red
}
```

📄 Sources & Compliance References * **CIS Microsoft Windows 10 Benchmark***: Section 18.8.14.1 (Turn On Virtualization Based Security), Section 18.8.14.2 (Turn On Virtualization Based Security: Credential Guard Configuration) * **ANSSI AD Hardening Guide***: Recommendations regarding LSA Protection and Credential Guard deployment.

[REQ-END-011] Configure Windows Defender Application Control

Target Scope * **Applicable Systems**: Tier 2 client workstations. * **Operating Systems**: Windows 10 (and above) Enterprise/Professional.

Implementation Details * **Priority**: High * **GPO Path / Registry Location**: * **GPO Path**: Computer Configuration\Administrative Templates\System\Device Guard\Deploy Windows Defender Application Control * **Registry Location**: `HKLM\SOFTWARE\Policies\Microsoft\Windows\DeviceGuard\CodeIntegrityPolicyPaths`

Rationale Traditional signature-based antivirus solutions scan for known malware patterns. However, they are easily bypassed by custom, compiled executables, dynamic scripts, or zero-day payloads.

Windows Defender Application Control (WDAC) is a kernel-enforced application control framework. Instead of asking "Is this file malicious?", WDAC asks "Is this file explicitly trusted?".

If WDAC is not configured:

1. **Payload Execution:** Standard users can execute downloaded scripts (e.g., PowerShell, VBScript) or binary files, facilitating initial access.
2. **Antivirus Bypass:** Attackers can run obfuscated code, compile payloads on the target endpoint using built-in Windows compilers (e.g., `csc.exe`), or run memory injection scripts that standard antivirus signatures miss.

Deploying a strict WDAC baseline ensures that only binaries and scripts signed by Microsoft, trusted system developers, or located in protected directories (such as Windows system folders) are allowed to execute.

Legacy Impact & Compatibility * **Pre-requisite (Memory Integrity/HVCI)**: Enforcing Windows Defender Application Control policies securely requires Hypervisor-Protected Code Integrity (HVCI) for secure, hypervisor-enforced validation. Refer to [REQ-END-010 - Enable VBS and Credential Guard](#08-endpoints-enable-vbs-credential-guard-md) to ensure Virtualization-Based Security (VBS) and Memory Integrity (HVCI) are fully enabled. Secure Boot and CPU virtualization are strict pre-requisites; refer to [REQ-END-013 - UEFI Firmware Security Hardening](#08-endpoints-configure-uefi-security-md) and [REQ-END-014 - Enable Hardware Virtualization and DMA Protection](#08-endpoints-enable-hardware-virtualization-and-dma-protection-md) for firmware setup. * **Administrative Overhead**: Any new enterprise software must be added to the code integrity trust policy (by digital signature or folder exceptions). Deploying unapproved third-party software will trigger blocks. * **User Script Blocks**: Administrators and power users cannot write and run custom PowerShell or VBS scripts locally unless the scripts are digitally signed by a trusted certificate in the WDAC policy or run in a directory excluded by the rules. * **Audit Phase Mandate**: To prevent severe business disruption, WDAC policies must always be deployed in **Audit Mode** first. This logs would-be blocks to the Event Viewer without interrupting execution, allowing administrators to gather a list of required applications and construct rules before shifting to **Enforced Mode**.

Implementation Steps

Option A: Group Policy Object (GPO) Configuration (Preferred)

To deploy WDAC via Group Policy, the policy XML must first be generated, compiled, and placed in a shared intranet network share.

1. Generate and Compile the Policy (on a Reference Machine) Run the following PowerShell commands to generate the Microsoft Default Windows baseline policy: ``powershell # Generate the baseline policy XML New-CIPolicy -MultiplePolicyFormat -Level FilePublisher -FilePath "C:\WDAC\BaselinePolicy.xml" -UserPEs

Compile the XML policy into a binary CIP file

ConvertFrom-CIPolicy -XmlFilePath "C:\WDAC\BaselinePolicy.xml" -BinaryFilePath "C:\WDAC\BaselinePolicy.cip"

```
<div id="08-endpoints-configure-wdac-md-2-deploy-the-policy-via-gpo"></div>
#### 2. Deploy the Policy via GPO
1. Copy the compiled 'BaselinePolicy.cip' file to a local secure directory on the target clients (e.g., 'C:\Windows\System32\CodeIntegrity\SIPolicy.p7b') or host it on a network path.
2. Open the **Group Policy Management Console** ('gpmc.msc').
3. Create or edit the GPO linked to your workstations OU (e.g., 'GPO_Hardening_Workstations').
4. Navigate to:
    'Computer Configuration\Administrative Templates\System\Device Guard'
5. Configure the setting:
    * **Policy**: 'Deploy Windows Defender Application Control'
    * **Setting**: 'Enabled'
    * **Code Integrity Policy File Path**: Enter the path to the policy file (e.g., 'C:\Windows\System32\CodeIntegrity\SIPolicy.p7b' or a UNC share path).

---

<div id="08-endpoints-configure-wdac-md-option-b-powershell-registry-configuration-remediation-non-gpo"></div>
### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to generate a baseline WDAC policy, enable Audit Mode, and configure local registry parameters.

<div id="08-endpoints-configure-wdac-md-configure-wdaclocalpolicy.ps1"></div>
# Configure-WDACLocalPolicy.ps1

[Download Script: Configure-WDACLocalPolicy.ps1](implementation_scripts/Configure-WDACLocalPolicy.ps1)

```powershell
Configure-WDACLocalPolicy.ps1
Description: Generates a baseline local Code Integrity policy, sets it to Audit Mode, and compiles it.

Write-Host "--- Configuring WDAC Local Policy Baseline ---" -ForegroundColor Cyan

Create working directories
$WdacDir = "C:\Windows\System32\CodeIntegrity"
if (-not (Test-Path $WdacDir)) {
 New-Item -Path $WdacDir -ItemType Directory -Force | Out-Null
}

1. Generate the Default Windows Policy
Write-Host "[+] Generating Default Windows code integrity rules..." -ForegroundColor Gray
$PolicyXml = "C:\Windows\Temp\DefaultWindows.xml"
$PolicyBin = "$WdacDir\SIPolicy.p7b"

Create a policy based on Microsoft's default rules (trusts Windows, Store, and Driver files)
New-CIPolicy -FilePath $PolicyXml -Level Windows -UserPEs -ErrorAction Stop

2. Set Policy to Audit Mode (Rule Option 3 represents Audit Mode)
Write-Host "[+] Setting WDAC policy to Audit Mode for baseline logging..." -ForegroundColor Gray
Set-RuleOption -FilePath $PolicyXml -Option 3 -ErrorAction SilentlyContinue

3. Compile the XML into the binary policy expected by the bootloader
Write-Host "[+] Compiling Code Integrity XML into SIPolicy.p7b..." -ForegroundColor Gray
ConvertFrom-CIPolicy -XmlFilePath $PolicyXml -BinaryFilePath $PolicyBin -ErrorAction Stop

Cleanup temp files
if (Test-Path $PolicyXml) { Remove-Item $PolicyXml -Force }

Write-Host "[+] Local WDAC baseline policy configured. Reboot required." -ForegroundColor Green
```

*To audit the running WDAC policy states:*

```
Test-WDACStatus.ps1
Download Script: Test-WDACStatus.ps1
```

```

Test-WDACStatus.ps1
Description: Audits the local system to check if Code Integrity policies and HVCI are active.

Write-Host "--- Auditing WDAC State ---" -ForegroundColor Cyan
$Vulnerable = $false

1. Query WMI class for Code Integrity status
try {
 $CI = Get-CimInstance -Namespace "Root\Microsoft\Windows\CI" -ClassName "MSFT_Sipolicy" -ErrorAction Stop
 if ($null -ne $CI -and $CI.Count -gt 0) {
 Write-Host "`n[+] Found $($CI.Count) active Code Integrity policies." -ForegroundColor Green
 foreach ($Policy in $CI) {
 Write-Host " - Policy: $($Policy.FriendlyName) | ID: $($Policy.PolicyID) | Enforced: $($Policy.EnforcementMode)" -ForegroundColor Green
 }
 } else {
 Write-Host "`n[-] No active Code Integrity / WDAC policies detected via WMI." -ForegroundColor Yellow
 }
} catch {
 Write-Host "`n[-] Could not query WMI MSFT_Sipolicy. This is expected if no WDAC policies are currently deployed." -ForegroundColor Gray
}

2. Check Memory Integrity (HVCI) configuration
$ScenariosPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\HypervisorEnforcedCodeIntegrity"
if (Test-Path $ScenariosPath) {
 $HvciStatus = Get-ItemProperty -Path $ScenariosPath -Name "Enabled" -ErrorAction SilentlyContinue
 if ($null -ne $HvciStatus -and $HvciStatus.Enabled -eq 1) {
 Write-Host "[+] Memory Integrity (HVCI) is enabled." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Memory Integrity (HVCI) is disabled in the registry." -ForegroundColor Red
 $Vulnerable = $true
 }
} else {
 Write-Host "[!] VULNERABLE: Memory Integrity scenario registry path does not exist." -ForegroundColor Red
 $Vulnerable = $true
}

3. Final Verdict
if ($Vulnerable) {
 Write-Host "`n[!] Verification FAILED: One or more driver security controls are not configured." -ForegroundColor Red
} else {
 Write-Host "`n[+] Verification PASSED: WDAC driver settings and HVCI are correctly configured." -ForegroundColor Green
}

```

## 📖 Sources & Compliance References \* \*\*CIS Microsoft Windows 10 Benchmark\*\*\*: Section 18.8.14.3 (Deploy Windows Defender Application Control) \* \*\*ANSSI AD Hardening Guide\*\*\*: Recommendations regarding application security and driver/code signing enforcement.

# [REQ-END-012] Enable BitLocker and Network Unlock

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional.

---

## Implementation Details \* \*\*GPO Paths / Registry Locations\*\*: \* \*\*GPO Paths\*\*: \* `Computer Configuration\Administrative Templates\Windows Components\BitLocker Drive Encryption` (General and OS, Fixed, and Removable subkeys) \* `Computer Configuration\Policies\Windows Settings\Security Settings\Public Key Policies\BitLocker Network Unlock` \* \*\*Registry Locations\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\FVE` (General Startup & Encryption Policies) \* `AllowNetworkUnlock` = `1` (REG\_DWORD) \* `MinimumPIN` = `6` (REG\_DWORD) \* `UseEnhancedPin` = `1` (REG\_DWORD) \* `UseTPM` = `2` (REG\_DWORD) \* `UseTPMPIN` = `2` (REG\_DWORD) \* `UseTPMKey` = `2` (REG\_DWORD) \* `UseTPMKeyPIN` = `2` (REG\_DWORD) \* `UseAdvancedStartup` = `1` (REG\_DWORD) \* `EnableBDEWithNoTPM` = `0` (REG\_DWORD, Enforce TPM) \* `HKLM\SOFTWARE\Policies\Microsoft\FVE` (Operating System Drives `OS...` keys) \* `OSAllowSecureBootForIntegrity` = `1` (REG\_DWORD) \* `OSRecovery` = `1` (REG\_DWORD) \* `OSManageDRA` = `0` (REG\_DWORD) \* `OSRecoveryPassword` = `1` (REG\_DWORD, Require 48-digit) \* `OSRecoveryKey` = `0` (REG\_DWORD, Do not allow) \* `OSHideRecoveryPage` = `1` (REG\_DWORD) \* `OSActiveDirectoryBackup` = `1` (REG\_DWORD) \* `OSActiveDirectoryInfoToStore` = `1` (REG\_DWORD) \* `OSRequireActiveDirectoryBackup` = `1` (REG\_DWORD) \* `OSHardwareEncryption` = `0` (REG\_DWORD, Disable hardware encryption) \* `OSPassphrase` = `0` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\FVE` (Fixed Data Drives `FDV...` keys) \* `FDVDiscoveryVolumeType` = `""` (REG\_SZ, blank string) \* `FDVRecovery` = `1` (REG\_DWORD) \* `FDVManageDRA` = `1` (REG\_DWORD) \* `FDVRecoveryPassword` = `2` (REG\_DWORD, Allow 48-digit) \* `FDVRecoveryKey` = `2` (REG\_DWORD, Allow 256-bit) \* `FDVHideRecoveryPage` = `1` (REG\_DWORD) \* `FDVActiveDirectoryBackup` = `1` (REG\_DWORD, Backup to AD DS enabled) \* `FDVActiveDirectoryInfoToStore` = `1` (REG\_DWORD) \* `FDVRequireActiveDirectoryBackup` = `1` (REG\_DWORD, Require AD backup) \* `FDVHardwareEncryption` = `0` (REG\_DWORD, Disable hardware encryption) \* `FDVPassphrase` = `0` (REG\_DWORD) \* `FDVAllowUserCert` = `1` (REG\_DWORD) \* `FDVEnforceUserCert` = `1` (REG\_DWORD, Require smart cards) \* `HKLM\SOFTWARE\Policies\Microsoft\FVE` (Removable Data Drives `RDV...` keys) \* `RDVDiscoveryVolumeType` = `""` (REG\_SZ, blank string) \* `RDVRecovery` = `1` (REG\_DWORD) \* `RDVManageDRA` = `1` (REG\_DWORD) \* `RDVRecoveryPassword` = `0` (REG\_DWORD, Do not allow) \* `RDVRecoveryKey` = `0` (REG\_DWORD, Do not allow) \* `RDVHideRecoveryPage` = `1` (REG\_DWORD) \* `RDVActiveDirectoryBackup` = `0` (REG\_DWORD) \* `RDVActiveDirectoryInfoToStore` = `1` (REG\_DWORD) \* `RDVRequireActiveDirectoryBackup` = `0` (REG\_DWORD) \* `RDVHardwareEncryption` = `0` (REG\_DWORD) \* `RDVPassphrase` = `0` (REG\_DWORD) \* `RDVAllowUserCert` = `1` (REG\_DWORD) \* `RDVEnforceUserCert` = `1` (REG\_DWORD) \* `RDVDenyCrossOrg` = `0` (REG\_DWORD) \* `HKLM\System\CurrentControlSet\Policies\Microsoft\FVE` \* `RDVDenyWriteAccess` = `1` (REG\_DWORD)

---

## Rationale BitLocker Drive Encryption protects the operating system volume from offline attacks, data tampering, and data theft when the device is powered off or stolen. Without full disk encryption, an attacker with physical access to a workstation can extract the hard drive, mount it on a non-secure system, bypass operating system security controls, dump local password databases (SAM), and access cached domain credentials.

Enforcing specific BitLocker startup parameters, such as a minimum PIN length of at least 6 characters, ensures that if a startup PIN is used, it cannot be easily brute-forced. Restricting how the TPM, startup keys, and PINs are configured ensures consistent security policy application.

To maximize security, standard endpoints (Tier 2) should use **BitLocker Network Unlock** to prevent operational overhead in managing startup PINs for thousands of workstations.

### How BitLocker Network Unlock Works BitLocker Network Unlock allows domain-joined workstations connected to the wired corporate LAN to automatically unlock their OS drives on reboot, while still requiring a backup PIN or recovery key when disconnected from the corporate network.

```
[Client PC Boot (UEFI)]
|
+-- Sends DHCP Request + Encrypted Key Payload (via Wired Ethernet)
 |
 v
[WDS Server (Network Unlock Role)]
|
+-- Decrypts Payload using Network Unlock Certificate Private Key
+-- Sends Decryption Key back via DHCP Reply Option
 |
 v
[Client PC]
|
+-- Automatically Unlocks OS Volume & Boots Windows
```

If the workstation is stolen or boots outside the local LAN (e.g., on a public network, Wi-Fi, or offline), the DHCP payload request goes unanswered, the Network Unlock fails, and the workstation falls back to prompting the user for a Startup PIN or Recovery Key.

---

## Legacy Impact & Compatibility \* \*\*Wired Network Required\*\*: Network Unlock operates in the pre-boot UEFI phase. Wireless network adapters are not active at this stage; workstations must be connected to the physical corporate switch via an Ethernet cable. \* \*\*UEFI and



**TPM Requirements\*\*:** Client computers must support UEFI DHCP drivers, native UEFI boot (Legacy CSM disabled), and have an active TPM 1.2 or 2.0 chip. **\*\*PKI Infrastructure\*\*:** Deploying Network Unlock requires a functioning Active Directory Certificate Services (AD CS) instance to issue and manage the Network Unlock certificate.

## ## Implementation Steps

### #### Option A: Group Policy and Server Configuration (Preferred)

**##### Step 1:** Configure the WDS Server for Network Unlock 1. Install the **\*\*Windows Deployment Services (WDS)\*\*** role on an internal Windows Server. 2. In Server Manager, select **\*\*Add Roles and Features\*\*** and check **\*\*BitLocker Network Unlock\*\*** under Features. 3. Open the Local PKI CA console (`certsrv.msc`) and issue a certificate using the **\*\*BitLocker Network Unlock\*\*** template. 4. Export the certificate public key (`.cer` file) and export the private key (`.pfx` file). 5. Import the `.pfx` private key certificate into the local WDS server's **\*\*Local Computer\Personal\*\*** certificate store. 6. Restart the WDS service (`wddsvc`).

**##### Step 2:** Distribute the Network Unlock Certificate via GPO 1. Open the **\*\*Group Policy Management Console\*\*** (`gpmc.msc`). 2. Edit your GPO linked to the workstations OU (e.g., `GPO_Hardening_Workstations`). 3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Public Key Policies` 4. Right-click **\*\*BitLocker Network Unlock\*\*** and select **\*\*Add Network Unlock Certificate\*\***. 5. Import the public `.cer` file exported in Step 1.

**##### Step 3:** Enforce GPO BitLocker Settings Navigate to `Computer Configuration\Administrative Templates\Windows Components\BitLocker Drive Encryption` and configure:

**##### 1. General Settings** \* **\*\*Policy\*\*:** `'Allow Network Unlock at startup'` → **\*\*Enabled\*\*** \* **\*\*Policy\*\*:** `'Configure minimum PIN length for startup'` → **\*\*Enabled\*\*** (Minimum characters: **\*\*6\*\***) \* **\*\*Policy\*\*:** `'Choose drive encryption method and cipher strength (Windows 10 [Version 1511] and later)'` → **\*\*Enabled\*\*** (OS, Fixed, and Removable: **\*\*XTS-AES 256-bit\*\***)

**##### 2. Operating System Drives** Navigate to `'Operating System Drives'` subfolder: \* **\*\*Policy\*\*:** `'Require additional authentication at startup'` → **\*\*Enabled\*\*** \* `'Allow BitLocker without a compatible TPM'` → **\*\*Disabled\*\*** (unchecked) \* `'Configure TPM startup'` → **\*\*Require TPM\*\*** \* `'Configure TPM startup PIN'` → **\*\*Allow startup PIN with TPM\*\*** (Allows Network Unlock auto-unlock) \* `'Configure TPM startup key'` → **\*\*Allow startup key with TPM\*\*** \* `'Configure TPM startup key and PIN'` → **\*\*Allow startup key and PIN with TPM\*\*** \* **\*\*Policy\*\*:** `'Allow enhanced PINs for startup'` → **\*\*Enabled\*\*** \* **\*\*Policy\*\*:** `'Allow Secure Boot for integrity validation'` → **\*\*Enabled\*\*** \* **\*\*Policy\*\*:** `'Choose how BitLocker-protected operating system drives can be recovered'` → **\*\*Enabled\*\*** \* `'Allow data recovery agent'` → **\*\*Disabled\*\*** (unchecked) \* `'Configure user storage of BitLocker recovery information'` → **\*\*Require 48-digit recovery password\*\*** \* `'Configure user storage of BitLocker recovery key'` → **\*\*Do not allow 256-bit recovery key\*\*** \* `'Omit recovery options from the BitLocker setup wizard'` → **\*\*Enabled\*\*** (checked) \* `'Save BitLocker recovery information to AD DS for operating system drives'` → **\*\*Enabled\*\*** (checked) \* `'Configure storage of BitLocker recovery information to AD DS'` → **\*\*Store recovery passwords and key packages\*\*** \* `'Do not enable BitLocker until recovery information is stored to AD DS for operating system drives'` → **\*\*Enabled\*\*** (checked) \* **\*\*Policy\*\*:** `'Configure use of hardware-based encryption for operating system drives'` → **\*\*Disabled\*\*** \* **\*\*Policy\*\*:** `'Configure use of passwords for operating system drives'` → **\*\*Disabled\*\***

**##### 3. Fixed Data Drives** Navigate to `'Fixed Data Drives'` subfolder: \* **\*\*Policy\*\*:** `'Allow access to BitLocker-protected fixed data drives from earlier versions of Windows'` → **\*\*Disabled\*\*** \* **\*\*Policy\*\*:** `'Choose how BitLocker-protected fixed drives can be recovered'` → **\*\*Enabled\*\*** \* `'Allow data recovery agent'` → **\*\*Enabled\*\*** (checked) \* `'Configure user storage of BitLocker recovery information'` → **\*\*Allow 48-digit recovery password\*\*** \* `'Configure user storage of BitLocker recovery key'` → **\*\*Allow 256-bit recovery key\*\*** \* `'Omit recovery options from the BitLocker setup wizard'` → **\*\*Enabled\*\*** (checked) \* `'Save BitLocker recovery information to AD DS for fixed data drives'` → **\*\*Enabled\*\*** (checked - overridden per user decision) \* `'Configure storage of BitLocker recovery information to AD DS'` → **\*\*Backup recovery passwords and key packages\*\*** \* `'Do not enable BitLocker until recovery information is stored to AD DS for fixed data drives'` → **\*\*Enabled\*\*** (checked - overridden to enforce AD backup) \* **\*\*Policy\*\*:** `'Configure use of hardware-based encryption for fixed data drives'` → **\*\*Disabled\*\*** \* **\*\*Policy\*\*:** `'Configure use of passwords for fixed data drives'` → **\*\*Disabled\*\*** \* **\*\*Policy\*\*:** `'Configure use of smart cards on fixed data drives'` → **\*\*Enabled\*\*** \* `'Require use of smart cards on fixed data drives'` → **\*\*Enabled\*\*** (checked)

**##### 4. Removable Data Drives** Navigate to `'Removable Data Drives'` subfolder: \* **\*\*Policy\*\*:** `'Allow access to BitLocker-protected removable data drives from earlier versions of Windows'` → **\*\*Disabled\*\*** \* **\*\*Policy\*\*:** `'Choose how BitLocker-protected removable drives can be recovered'` → **\*\*Enabled\*\*** \* `'Allow data recovery agent'` → **\*\*Enabled\*\*** (checked) \* `'Configure user storage of BitLocker recovery information'` → **\*\*Do not allow 48-digit recovery password\*\*** \* `'Configure user storage of BitLocker recovery key'` → **\*\*Do not allow 256-bit recovery key\*\*** \* `'Omit recovery options from the BitLocker setup wizard'` → **\*\*Enabled\*\*** (checked) \* `'Save BitLocker recovery information to AD DS for removable data drives'` → **\*\*Disabled\*\*** (unchecked) \* `'Configure storage of BitLocker recovery information to AD DS'` → **\*\*Backup recovery passwords and key packages\*\*** \* `'Do not enable BitLocker until recovery information is stored to AD DS for removable data drives'` → **\*\*Disabled\*\*** (unchecked) \* **\*\*Policy\*\*:** `'Configure use of hardware-based encryption for removable data drives'` → **\*\*Disabled\*\*** \* **\*\*Policy\*\*:** `'Configure use of passwords for removable data drives'` → **\*\*Disabled\*\*** \* **\*\*Policy\*\*:** `'Configure use of smart cards on removable data drives'` → **\*\*Enabled\*\*** \* `'Require use of smart cards on removable data drives'` → **\*\*Enabled\*\*** (checked) \* **\*\*Policy\*\*:** `'Deny write access to removable drives not protected by BitLocker'` → **\*\*Enabled\*\*** \* `'Do not allow write access to devices configured in another organization'` → **\*\*Disabled\*\*** (unchecked)

### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to audit and configure BitLocker parameters.

[Download Script: Set-BitLockerEncryption.ps1](#)

```
Set-BitLockerEncryption.ps1
Enables BitLocker encryption locally, configures startup policies/PIN lengths, and backs up recovery keys to AD.

Write-Host "--- Enforcing BitLocker Drive Encryption ---" -ForegroundColor Cyan

1. Configure FVE Registry settings
$FveRegPath = "HKLM:\SOFTWARE\Policies\Microsoft\FVE"
if (-not (Test-Path $FveRegPath)) {
 New-Item -Path $FveRegPath -Force | Out-Null
}

General Startup and Network Unlock settings
Set-ItemProperty -Path $FveRegPath -Name "AllowNetworkUnlock" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "MinimumPIN" -Value 6 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "UseTPM" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "UseTPMPIN" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "UseTPMKey" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "UseTPMKeyPIN" -Value 2 -Type DWord -Force

OS Drive Settings (18.10.10.2.x)
Set-ItemProperty -Path $FveRegPath -Name "UseEnhancedPin" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "OSAllowSecureBootForIntegrity" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "OSRecovery" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "OSManageDRA" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "OSRecoveryPassword" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "OSRecoveryKey" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "OSHideRecoveryPage" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "OSActiveDirectoryBackup" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "OSActiveDirectoryInfoToStore" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "OSRequireActiveDirectoryBackup" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "OSHHardwareEncryption" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "OSPassphrase" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "UseAdvancedStartup" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "EnableBDEWithNoTPM" -Value 0 -Type DWord -Force

Fixed Drive Settings (18.10.10.1.x)
Set-ItemProperty -Path $FveRegPath -Name "FDVDiscoveryVolumeType" -Value "" -Type String -Force
Set-ItemProperty -Path $FveRegPath -Name "FDVRecovery" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "FDVManageDRA" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "FDVRecoveryPassword" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "FDVRecoveryKey" -Value 2 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "FDVHideRecoveryPage" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "FDVActiveDirectoryBackup" -Value 1 -Type DWord -Force # Overridden to enable AD backups
Set-ItemProperty -Path $FveRegPath -Name "FDVActiveDirectoryInfoToStore" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "FDVRequireActiveDirectoryBackup" -Value 1 -Type DWord -Force # Overridden to require AD b
ackups
Set-ItemProperty -Path $FveRegPath -Name "FDVHardwareEncryption" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "FDVPassphrase" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "FDVAllowUserCert" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "FDVEnforceUserCert" -Value 1 -Type DWord -Force

Removable Drive Settings (18.10.10.3.x)
Set-ItemProperty -Path $FveRegPath -Name "RDVDiscoveryVolumeType" -Value "" -Type String -Force
Set-ItemProperty -Path $FveRegPath -Name "RDVRecovery" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "RDVManageDRA" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "RDVRecoveryPassword" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "RDVRecoveryKey" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "RDVHideRecoveryPage" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "RDVActiveDirectoryBackup" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "RDVActiveDirectoryInfoToStore" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "RDVRequireActiveDirectoryBackup" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "RDVHardwareEncryption" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "RDVPassphrase" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "RDVAllowUserCert" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "RDVEnforceUserCert" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $FveRegPath -Name "RDVDenyCrossOrg" -Value 0 -Type DWord -Force

Removable Drive Write Blocks (System FVE Policies)
$FveSystemPath = "HKLM:\System\CurrentControlSet\Policies\Microsoft\FVE"
if (-not (Test-Path $FveSystemPath)) {
 New-Item -Path $FveSystemPath -Force | Out-Null
}
Set-ItemProperty -Path $FveSystemPath -Name "RDVDenyWriteAccess" -Value 1 -Type DWord -Force
```



```
Write-Host "[+] BitLocker startup authentication and volume encryption policies configured." -ForegroundColor Green

2. Enable BitLocker on C: drive using TPM protection
$Volume = Get-BitLockerVolume -MountPoint "C:"

Check if protection is already active
if ($Volume.ProtectionStatus -eq "Off") {
 Write-Host "[+] Activating BitLocker on C: drive using XTS-AES 256 encryption..." -ForegroundColor Gray

 # Enable BitLocker and backup recovery password protector to Active Directory
 Enable-BitLocker -MountPoint "C:" `
 -EncryptionMethod XtsAes256 `
 -UsedSpaceOnly `
 -TpmProtector `
 -AdBackupRequired

 Write-Host "[+] BitLocker encryption initiated. Recovery key backed up to AD." -ForegroundColor Green
} else {
 Write-Host "[+] BitLocker is already enabled on C: (Protection Status: $($Volume.ProtectionStatus))." -ForegroundColor Green
}
```

To audit local BitLocker and Network Unlock registry settings: [Download Script: Test-BitLockerStatus.ps1](#)

```

Test-BitLockerStatus.ps1
Audits current BitLocker protection state, key protector types, and Network Unlock/Startup PIN configuration.

Write-Host "--- Auditing BitLocker Status ---" -ForegroundColor Cyan

1. Query local BitLocker state
$Volume = Get-BitLockerVolume -MountPoint "C:" -ErrorAction SilentlyContinue
if ($Volume) {
 $StatusColor = if ($Volume.ProtectionStatus -eq "On") { "Green" } else { "Red" }
 Write-Host " Protection Status: $($Volume.ProtectionStatus)" -ForegroundColor $StatusColor
 Write-Host " Encryption Method: $($Volume.EncryptionMethod)" -ForegroundColor White

 Write-Host "`n[+] Active Key Protectors:" -ForegroundColor Yellow
 foreach ($Protector in $Volume.KeyProtector) {
 Write-Host " Type: $($Protector.KeyProtectorType) | ID: $($Protector.KeyProtectorId)" -ForegroundColor White
 }
} else {
 Write-Error "BitLocker volume information could not be retrieved."
}

2. Check Network Unlock and Startup Authentication registry configuration
$FveRegPath = "HKLM:\SOFTWARE\Policies\Microsoft\FVE"
$FveSysPath = "HKLM:\System\CurrentControlSet\Policies\Microsoft\FVE"

$Params = @(
 @{ Name = "AllowNetworkUnlock"; Expected = 1; Path = $FveRegPath },
 @{ Name = "MinimumPIN"; Expected = 6; Path = $FveRegPath },
 @{ Name = "UseTPM"; Expected = 2; Path = $FveRegPath },
 @{ Name = "UseTPMPIN"; Expected = 2; Path = $FveRegPath },
 @{ Name = "UseTPMKey"; Expected = 2; Path = $FveRegPath },
 @{ Name = "UseTPMKeyPIN"; Expected = 2; Path = $FveRegPath },

 # OS Drives
 @{ Name = "UseEnhancedPin"; Expected = 1; Path = $FveRegPath },
 @{ Name = "OSAllowSecureBootForIntegrity"; Expected = 1; Path = $FveRegPath },
 @{ Name = "OSRecovery"; Expected = 1; Path = $FveRegPath },
 @{ Name = "OSManageDRA"; Expected = 0; Path = $FveRegPath },
 @{ Name = "OSRecoveryPassword"; Expected = 1; Path = $FveRegPath },
 @{ Name = "OSRecoveryKey"; Expected = 0; Path = $FveRegPath },
 @{ Name = "OSHideRecoveryPage"; Expected = 1; Path = $FveRegPath },
 @{ Name = "OSActiveDirectoryBackup"; Expected = 1; Path = $FveRegPath },
 @{ Name = "OSActiveDirectoryInfoToStore"; Expected = 1; Path = $FveRegPath },
 @{ Name = "OSRequireActiveDirectoryBackup"; Expected = 1; Path = $FveRegPath },
 @{ Name = "OSHardwareEncryption"; Expected = 0; Path = $FveRegPath },
 @{ Name = "OSPassphrase"; Expected = 0; Path = $FveRegPath },
 @{ Name = "UseAdvancedStartup"; Expected = 1; Path = $FveRegPath },
 @{ Name = "EnableBDEWithNoTPM"; Expected = 0; Path = $FveRegPath },

 # Fixed Drives
 @{ Name = "FDVDiscoveryVolumeType"; Expected = ""; Path = $FveRegPath },
 @{ Name = "FDVRecovery"; Expected = 1; Path = $FveRegPath },
 @{ Name = "FDVManagedDRA"; Expected = 1; Path = $FveRegPath },
 @{ Name = "FDVRecoveryPassword"; Expected = 2; Path = $FveRegPath },
 @{ Name = "FDVRecoveryKey"; Expected = 2; Path = $FveRegPath },
 @{ Name = "FDVHideRecoveryPage"; Expected = 1; Path = $FveRegPath },
 @{ Name = "FDVActiveDirectoryBackup"; Expected = 1; Path = $FveRegPath },
 @{ Name = "FDVActiveDirectoryInfoToStore"; Expected = 1; Path = $FveRegPath },
 @{ Name = "FDVRequireActiveDirectoryBackup"; Expected = 1; Path = $FveRegPath },
 @{ Name = "FDVHardwareEncryption"; Expected = 0; Path = $FveRegPath },
 @{ Name = "FDVPassphrase"; Expected = 0; Path = $FveRegPath },
 @{ Name = "FDVAllowUserCert"; Expected = 1; Path = $FveRegPath },
 @{ Name = "FDVEnforceUserCert"; Expected = 1; Path = $FveRegPath },

 # Removable Drives
 @{ Name = "RDVDiscoveryVolumeType"; Expected = ""; Path = $FveRegPath },
 @{ Name = "RDVRecovery"; Expected = 1; Path = $FveRegPath },
 @{ Name = "RDVManagedDRA"; Expected = 1; Path = $FveRegPath },
 @{ Name = "RDVRecoveryPassword"; Expected = 0; Path = $FveRegPath },
 @{ Name = "RDVRecoveryKey"; Expected = 0; Path = $FveRegPath },
 @{ Name = "RDVHideRecoveryPage"; Expected = 1; Path = $FveRegPath },
 @{ Name = "RDVActiveDirectoryBackup"; Expected = 0; Path = $FveRegPath },
 @{ Name = "RDVActiveDirectoryInfoToStore"; Expected = 1; Path = $FveRegPath },
 @{ Name = "RDVRequireActiveDirectoryBackup"; Expected = 0; Path = $FveRegPath },
 @{ Name = "RDVHardwareEncryption"; Expected = 0; Path = $FveRegPath },

```

```

@{ Name = "RDVPassphrase"; Expected = 0; Path = $FveRegPath },
@{ Name = "RDVAllowUserCert"; Expected = 1; Path = $FveRegPath },
@{ Name = "RDVEnforceUserCert"; Expected = 1; Path = $FveRegPath },
@{ Name = "RDVDenyCrossOrg"; Expected = 0; Path = $FveRegPath },
@{ Name = "RDVDenyWriteAccess"; Expected = 1; Path = $FveSysPath }
)

Write-Host "`n[*] Auditing BitLocker settings:" -ForegroundColor Yellow
$script:Vulnerable = $false

foreach ($Param in $Params) {
 if (Test-Path $Param.Path) {
 $Val = Get-ItemProperty -Path $Param.Path -Name $Param.Name -ErrorAction SilentlyContinue
 $ActualVal = if ($Val) { $Val.$($Param.Name) } else { $null }

 $IsMatch = $false
 if ($Param.Expected -eq "") {
 $IsMatch = ($null -eq $ActualVal -or $ActualVal -eq "")
 } else {
 $IsMatch = ($ActualVal -eq $Param.Expected)
 }

 $Color = if ($IsMatch) { "Green" } else { "Red" }
 if (-not $IsMatch) { $script:Vulnerable = $true }

 Write-Host " - $($Param.Name): $ActualVal (Expected = $($Param.Expected))" -ForegroundColor $Color
 } else {
 Write-Host " - Policy key $($Param.Path) does not exist." -ForegroundColor Red
 $script:Vulnerable = $true
 }
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
}

```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*\*: Section 18.2.1 (BitLocker Drive Encryption settings) \*  
 \*\*ANSSI AD Hardening Guide\*\*\*: Recommendations regarding endpoint encryption and physical key storage security. \* \*\*DoD Windows 11  
 STIG\*\*\*: BitLocker startup option requirements and fixed/removable drive recovery rules.

## # [REQ-END-013] UEFI Firmware Security Hardening

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* UEFI Firmware Configuration Menu \* HKLM\SYSTEM\CurrentControlSet\Control\SecureBoot\State \* `UEFISecureBootEnabled` = `1` (REG\_DWORD)

---

## Rationale Standard Tier 2 endpoints (such as employee laptops and workstations) and member servers are frequently exposed to physical theft, loss, and unauthorized local access in branch offices or remote environments. If the firmware on these systems is left unsecured, an attacker can modify boot settings, bypass operating system security controls, or execute physical DMA and offline decryption attacks. Securing the firmware level ensures:

1. **Firmware Integrity:** Enforcing a UEFI administrator password prevents unauthorized configuration modifications, such as disabling Secure Boot, TPM, or hardware virtualization features.
2. **Native UEFI Boot:** Disabling the Compatibility Support Module (CSM) or Legacy BIOS options forces native UEFI mode, which is mandatory for activating UEFI Secure Boot and Virtualization-Based Security (VBS).
3. **Restricted Boot Paths:** Restricting the boot order to the primary internal storage device prevents users or attackers from booting unauthorized operating systems or diagnostic tools from USB media or untrusted local networks.
4. **BIOS Rollback Prevention:** Enforcing firmware update signature validation and blocking BIOS rollbacks mitigates the risk of downgrade attacks targeting known firmware vulnerabilities.
5. **Virtualization-Based Security Foundation:** Enabling CPU Virtualization Extensions (Intel VT-x / AMD-V) and IOMMU (Intel VT-d / AMD-Vi) at the firmware level establishes the mandatory hardware isolation required by the Windows Hypervisor to run VBS, Credential Guard, and Kernel DMA Protection.
6. **Platform Measurement Integrity:** Disabling Fast Boot forces the firmware to execute full hardware initialization, device checks, and complete TPM self-tests/PCR measurements at every boot, ensuring platform integrity and correct state validation.

---

## Legacy Impact & Compatibility \* \*\*Administrative Overhead\*\*: Technicians must enter the UEFI administrator password to make hardware changes or perform local diagnostics. This password must be securely generated and stored in a central, encrypted vault. \* \*\*Legacy OS Incompatibility\*\*: Operating systems or recovery environments that do not support native UEFI boot will fail to start. This is acceptable as Tier 2 systems must only run modern, authorized Windows 10/11 Enterprise installations. \* \*\*Partition Format\*\*: Converting an existing Legacy BIOS installation to UEFI requires repartitioning the primary storage device from Master Boot Record (MBR) to GUID Partition Table (GPT) using utility tools like MBR2GPT.exe.

---

### ## Implementation Steps

#### ### Option A: Manual UEFI Firmware Configuration (Preferred)

UEFI settings must be configured directly within the hardware platform firmware interface during system startup.

1. Turn on or restart the workstation and access the UEFI utility screen by pressing the vendor-specific key during POST (typically Delete, F2, F10, or F12).
2. Navigate to the **Security** or **Authentication** section:
  - Select the option to set the **Administrator Password** (also referred to as the **Supervisor Password**). Do not configure a User Password, as that prompts for authentication on every boot rather than only when entering configuration settings.
  - Enter a strong, complex password. Record this password in the team's secure credential repository.
3. Navigate to the **Boot** or **System Configuration** section:
  - Locate the **Boot Mode** setting and set it to **UEFI Only** or **Native UEFI**.
  - Locate **CSM (Compatibility Support Module)** or **Legacy Boot Support** and set it to **Disabled**.
  - Locate **Fast Boot** or **Quick Boot** and set it to **Disabled** (forcing complete POST diagnostics and full TPM initialization on every boot).
  - Locate **Boot Order** (or **Boot Priority**):
    - Set the primary boot option to the internal system storage drive (typically containing the Windows Boot Manager partition).
    - Disable all other boot options (such as USB, SD Card, Optical Drive, and Network PXE Boot) or set them to disabled in the boot menu.
    - Enable the option to prompt for the UEFI administrator password if a user attempts to access the boot override menu (typically F12 or F8).
4. Navigate to the **Advanced**, **CPU Configuration**, or **Security Chip** section:
  - Locate **Intel Virtualization Technology (VT-x)** or **AMD-V** and set it to **Enabled**.
  - Locate **Intel VT for Directed I/O (VT-d)** or **AMD IOMMU** and set it to **Enabled** (required for IOMMU/Kernel DMA Protection).
  - Locate **TPM 2.0 Device** (or **Security Chip / Intel PTT / AMD fTPM**) and set it to **Enabled** or **Active** (with SHA-256 PCR bank).
  - Locate **Memory Overwrite Request Control Lock** (or **MOR Lock**) and set it to **Enabled**.
5. Navigate to the **Security** or **Secure Boot** section:

- Ensure **Secure Boot** is **Enabled** and the **Secure Boot Mode** is set to **Deployed** or **User Mode**.
  - Harden the certificates allowlist:
    - **Key Exchange Key (KEK)**: Must only contain "Microsoft Corporation KEK CA 2011" and "Microsoft Corporation KEK 2K CA 2023".
    - **Signature Database (db)**: Must only contain "Microsoft Windows Production PCA 2011" and "Windows UEFI CA 2023". Remove "Microsoft UEFI CA 2011" and "Microsoft Option ROM UEFI CA 2023" unless strictly required by specific physical PCIe expansion hardware.
6. Navigate to the **Advanced** or **Firmware Update** section:
- Locate the option for **BIOS Flash Protection** or **Firmware Rollback Protection** and set it to **Enabled** or **Block Downgrades**.
7. Save the configuration and restart the workstation.

---

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

While hardware firmware settings cannot be directly written via standard Windows registry keys, the current state of the local UEFI boot environment and system BIOS characteristics can be programmatically audited.

Run the following script to verify native UEFI boot, Secure Boot state, and retrieve system BIOS properties:

[Download Script: Audit-UEFISecurity.ps1](#)

```
Audit-UEFISecurity.ps1
Description: Audits local boot environment and BIOS firmware properties.

Write-Host "--- Auditing UEFI Security Baseline ---" -ForegroundColor Cyan

1. Verify boot environment type
if ($env:firmware_type -eq "UEFI") {
 Write-Host "Status: Native UEFI mode is active." -ForegroundColor Green
} else {
 Write-Host "VULNERABLE: System booted in Legacy BIOS mode (CSM enabled) or firmware type is unrecognized." -ForegroundColor Red
}

2. Audit Secure Boot status
try {
 $SecureBootActive = Confirm-SecureBootUEFI -ErrorAction Stop
 if ($SecureBootActive -eq $true) {
 Write-Host "Status: UEFI Secure Boot is enabled." -ForegroundColor Green
 } else {
 Write-Host "VULNERABLE: UEFI Secure Boot is supported but disabled in firmware." -ForegroundColor Red
 }
} catch [System.PlatformNotSupportedException] {
 Write-Host "VULNERABLE: UEFI Secure Boot is not supported on this platform." -ForegroundColor Red
} catch {
 Write-Host "VULNERABLE: UEFI Secure Boot validation failed. Error: $($_.Exception.Message)" -ForegroundColor Red
}

3. Retrieve BIOS details
$BiosDetails = Get-CimInstance -ClassName Win32_Bios -ErrorAction SilentlyContinue
if ($BiosDetails) {
 Write-Host "Firmware Manufacturer: $($BiosDetails.Manufacturer)" -ForegroundColor White
 Write-Host "Firmware Version: $($BiosDetails.SMBIOSBIOSVersion)" -ForegroundColor White
 Write-Host "Firmware Release Date: $($BiosDetails.ReleaseDate)" -ForegroundColor White
} else {
 Write-Host "Warning: BIOS details could not be retrieved via WMI." -ForegroundColor Yellow
}
```

---

## Sources & Compliance References \* \*\*ANSI AD Hardening Guide\*\*\*: Recommendations regarding hardware platform integrity. \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*\*: Section 18.8 (Device Guard/VBS prerequisites) \* \*\*Microsoft Security Guidelines\*\*\*: UEFI Firmware Security and Device Guard Deployment

# [REQ-END-014] Enable Hardware Virtualization and DMA Protection

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: Computer Configuration\Administrative Templates\System\Kernel DMA Protection \* \*\*Registry Location\*\*: HKLM\SOFTWARE\Policies\Microsoft\Windows\KernelDMAProtection \* `DeviceEnumerationPolicy` = `0` (REG\_DWORD)

---

## Rationale Operating-system-level security layers such as Virtualization-Based Security (VBS) and Windows Defender Credential Guard are designed to isolate credentials and kernel code integrity in a virtual secure environment. However, these layers rely entirely on the security of the underlying hardware platform and motherboard architecture.

Enabling hardware virtualization and DMA protection ensures:

1. **Hypervisor Memory Isolation:** Activating CPU Virtualization Extensions (Intel VT-x or AMD-V) in the UEFI allows the Windows hypervisor to separate the host operating system from the secure kernel.
2. **Direct Memory Access (DMA) Protection:** Enforcing IOMMU (Intel VT-d or AMD-Vi) at the firmware level enables Kernel DMA Protection. This blocks malicious peripherals connected to hot-plug ports (such as Thunderbolt, USB4, or PCIe expansion slots) from executing unauthorized DMA requests to read or modify physical RAM, mitigating physical attacks to extract BitLocker encryption keys or in-memory credential tokens.
3. **Hardware Trust Anchoring:** Enabling TPM 2.0 provides the physical root of trust needed to verify boot integrity (via PCR measurements) and safely store cryptographic keys, preventing the operating system from booting or unsealing disk encryption if the hardware state has been tampered with.

---

## Legacy Impact & Compatibility \* \*\*Hardware Requirements\*\*: Systems must support CPU virtualization, Second Level Address Translation (SLAT), and input-output memory management (IOMMU). Modern hardware supports these features by default, but older server models or legacy clients may need upgrades. \* \*\*External Device Blocking\*\*: Enforcing Kernel DMA Protection may prevent non-compliant hot-plug peripherals (such as legacy Thunderbolt docking stations or older external GPUs) from running if they do not support DMA remapping. To balance usability on standard endpoints, policies can be configured to allow external devices only after a user logs on and unlocks the session. \* \*\*Third-Party Virtualization\*\*: Activating hardware virtualization for VBS may cause conflicts with older third-party virtualization software that does not support nested execution under Microsoft Hyper-V.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

To enforce Kernel DMA Protection across standard client workstations and member servers, implement the following GPO settings:

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Create or edit a GPO linked to the target workstations and servers OUs (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Administrative Templates\System\Kernel DMA Protection`
4. Configure the following settings:
  - **Policy:** `Enable Kernel DMA Protection` → **Enabled**
  - **Policy:** `Enumeration policy for external devices incompatible with Kernel DMA Protection` → **Enabled: Block All** (value 0)
5. Link the GPO to the appropriate OUs.

*Note: In addition to the GPO policy, ensure CPU Virtualization (VT-x/AMD-V), IOMMU (VT-d/AMD-Vi), and TPM 2.0 are manually enabled in the UEFI configuration menu of the systems.*

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Configure local registry keys to enforce Kernel DMA Protection and programmatically audit the hardware security baseline.

#### 1. Local Remediation (Enforce Kernel DMA Protection)

Run the following script to configure the Kernel DMA Protection policy locally:

[Download Script: Configure-KernelDMAProtection.ps1](#)

```
Configure-KernelDMAProtection.ps1
Description: Configures registry keys to enable Kernel DMA Protection.

Write-Host "--- Enforcing Kernel DMA Protection ---" -ForegroundColor Cyan

$RegPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\KernelDMAProtection"
if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

DeviceEnumerationPolicy = 0 (Block all external DMA devices)
Set-ItemProperty -Path $RegPath -Name "DeviceEnumerationPolicy" -Value 0 -Type DWord
Write-Host "Status: Kernel DMA Protection registry configuration applied." -ForegroundColor Green
```

#### #### 2. Local Audit (TPM, Virtualization, and DMA Support)

Run the following script to audit the status of the required hardware security components:

[Download Script: Audit-HardwareSecurityFeatures.ps1](#)

```
Audit-HardwareSecurityFeatures.ps1
Description: Audits TPM 2.0, CPU Virtualization, and IOMMU/DMA status.

Write-Host "--- Auditing Hardware Security Features ---" -ForegroundColor Cyan

1. Audit TPM 2.0 Status
$Tpm = Get-Tpm -ErrorAction SilentlyContinue
if ($Tpm) {
 if ($Tpm.TpmPresent -eq $true) {
 $TpmColor = "Red"
 if ($Tpm.TpmReady -eq $true) {
 $TpmColor = "Green"
 }
 Write-Host "Status: TPM Present: $($Tpm.TpmPresent) | Ready: $($Tpm.TpmReady)" -ForegroundColor $TpmColor
 } else {
 Write-Host "VULNERABLE: TPM 2.0 is not detected on this system." -ForegroundColor Red
 }
} else {
 Write-Host "VULNERABLE: TPM verification cmdlet failed." -ForegroundColor Red
}

2. Audit VBS and DMA Status via Win32_DeviceGuard
try {
 $DG = Get-CimInstance -Namespace "Root\Microsoft\Windows\DeviceGuard" -ClassName "Win32_DeviceGuard" -ErrorAction Stop

 # VirtualizationBasedSecurityStatus: 2 = Running
 $VbsStatus = $DG.VirtualizationBasedSecurityStatus
 $VbsColor = "Red"
 if ($VbsStatus -eq 2) {
 $VbsColor = "Green"
 }
 Write-Host "Status: Virtualization-Based Security Status: $($VbsStatus) (Required = 2 [Running])" -ForegroundColor $VbsColor

 # AvailableSecurityProperties: 3 = DMA Protection (IOMMU)
 $DmaSupported = $DG.AvailableSecurityProperties -contains 3
 $DmaColor = "Red"
 if ($DmaSupported -eq $true) {
 $DmaColor = "Green"
 }
 Write-Host "Status: Hardware IOMMU/DMA Protection: $($DmaSupported)" -ForegroundColor $DmaColor

 # RequiredSecurityProperties: 3 = DMA Protection enforced
 $DmaEnforced = $DG.RequiredSecurityProperties -contains 3
 $EnforcedColor = "Red"
 if ($DmaEnforced -eq $true) {
 $EnforcedColor = "Green"
 }
 Write-Host "Status: DMA Protection Policy Enforced: $($DmaEnforced)" -ForegroundColor $EnforcedColor
} catch {
 Write-Host "VULNERABLE: Win32_DeviceGuard WMI class could not be queried. VBS is likely inactive." -ForegroundColor Red
}
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: Recommendations regarding hardware platform integrity. \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*: Section 18.8.19.1 (Configure Enable Kernel DMA Protection), Section 18.8.14.1 (Turn On Virtualization Based Security) \* \*\*Microsoft Security Guidelines\*\*: Kernel DMA Protection reference architecture



# [REQ-END-015] Disable Windows Platform Binary Table (WPBT)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* Computer Configuration\Preferences\Windows Settings\Registry \* HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\DisableWpbtExecution

## Rationale The Windows Platform Binary Table (WPBT) is an ACPI firmware table that allows hardware manufacturers (OEMs) to execute proprietary binaries in kernel space during the Windows boot phase. Windows automatically extracts the binary from the table and runs it with system privileges before security software, third-party agents, or standard driver verifications are fully initialized.

While designed to facilitate automated driver provisioning and anti-theft services, this mechanism represents a significant security risk:

1. **Firmware-to-OS Attack Vector:** Malicious actors utilizing UEFI rootkits, physical firmware flashing tools, or supply-chain firmware implants can compromise the WPBT table to execute arbitrary code at boot, bypassing Secure Boot and operating system-level integrity checks.
2. **Privilege Escalation Risks:** Historically, OEM software delivered via the WPBT has introduced high-severity local privilege escalation and remote code execution vulnerabilities due to inadequate code review or poor permission management.
3. **Control and Transparency:** Executing firmware-rooted binaries without administrative visibility or operating system validation bypasses normal software lifecycle and endpoint protection policies.

Disabling WPBT execution prevents Windows from parsing the ACPI table and running the embedded software, mitigating boot-level integrity bypasses.

## Legacy Impact & Compatibility \* \*\*OEM Software Functionality\*\*: Disabling the WPBT will stop manufacturer-embedded software (such as automated support assistants, system registration tools, or OEM-specific recovery software) from installing on a fresh OS deployment. System installation pipelines must manually deploy any validated, business-essential hardware utility packages rather than relying on automatic firmware injection. \* \*\*Deployment Timing\*\*: The registry setting 'DisableWpbtExecution' must be present prior to the initial Windows boot sequence to completely block WPBT payload execution on a newly installed OS. Applying the registry key via GPO will prevent subsequent runs or updates but will not retroactively clean up files that were already executed during the initial setup. For maximum protection, this registry modification should be integrated directly into reference installation media (e.g., via 'autounattend.xml' or custom WIM injection).

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

Because there is no default ADMX administrative template to manage WPBT execution, the setting must be configured as a Registry Preference under the endpoint policy:

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a domain management host.
2. Edit the GPO linked to your workstations Organizational Unit (e.g., `GPO_Hardening_Workstations` ).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Right-click **Registry**, select **New → Registry Item**.
5. Configure the following properties:
  - **Action:** `Update`
  - **Hive:** `HKEY_LOCAL_MACHINE`
  - **Key Path:** `SYSTEM\CurrentControlSet\Control\Session Manager`
  - **Value name:** `DisableWpbtExecution`
  - **Value type:** `REG_DWORD`
  - **Value data:** `1`

6. Click **OK** to save the preference.

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script to configure the registry setting locally on the system:

[Download Script: Configure-DisableWpbt.ps1](#)

```
Configure-DisableWpbt.ps1
Description: Disables Windows Platform Binary Table (WPBT) execution in the registry.

Write-Host "Applying hardening requirement: Disable WPBT Execution..." -ForegroundColor Cyan

$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager"
$ValueName = "DisableWpbtExecution"
$ValueData = 1

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value $ValueData -Type DWord
Write-Host "Registry setting DisableWpbtExecution configured to 1." -ForegroundColor Green
```

*To verify that the registry value is correctly enforced:*

[Download Script: Get-WpbtStatus.ps1](#)

```
Get-WpbtStatus.ps1
Description: Audits the registry state for WPBT execution prevention.

Write-Host "--- Auditing WPBT Security Posture ---" -ForegroundColor Cyan

$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager"
$ValueName = "DisableWpbtExecution"

$RegistryValue = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue

if ($RegistryValue) {
 $Setting = $RegistryValue.DisableWpbtExecution
 if ($Setting -eq 1) {
 Write-Host "Status: WPBT execution is disabled (DisableWpbtExecution = 1)." -ForegroundColor Green
 } else {
 Write-Host "VULNERABLE: WPBT execution is enabled. Value is $($Setting)." -ForegroundColor Red
 }
} else {
 Write-Host "VULNERABLE: DisableWpbtExecution registry value is not configured (defaulting to execution enabled)." -ForegroundColor Red
}
}
```

---

## Sources & Compliance References \* **\*\*ANSI AD Hardening Guide\*\***: Recommendations regarding hardware platform integrity. \*  
**\*\*Microsoft Windows Security\*\***: Device Guard and UEFI Platform Security guidelines.

**# Configure User Rights Assignments**

**## Target Scope** \* **Applicable Systems**: Member Servers, Tier 2 Clients (Windows 10/11) \* **Operating Systems**: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

**## Implementation Details** \* **Priority**: High \* **GPO Path / Registry Location**: \* **GPO Path**: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` \* **Registry Location**: Stored inside local security database under privilege definitions.

**## Rationale** User Rights Assignments (URAs) govern the specific actions that security principals (users, groups, and service accounts) can perform on a system. Insecure default URA mappings can be abused by attackers to elevate privileges, compromise credentials, or establish persistence.

This submodule contains individual requirement rules for each User Rights Assignment control enforced on standard workstations.

**## Legacy Impact & Compatibility** \* **Operational Impact**: Restricting privileges restricts custom services or applications that depend on local execution rights. Validate all applications in audit staging environments before implementing block rules.

**## Enforced User Rights Assignments**

The following individual URA rules must be configured:

1. [REQ-END-096 - Configure User Rights: Access Credential Manager as a trusted caller](#)
2. [REQ-END-097 - Configure User Rights: Access this computer from the network](#)
3. [REQ-END-098 - Configure User Rights: Act as part of the operating system](#)
4. [REQ-END-099 - Configure User Rights: Allow log on locally](#)
5. [REQ-END-100 - Configure User Rights: Back up files and directories](#)
6. [REQ-END-101 - Configure User Rights: Change the system time](#)
7. [REQ-END-102 - Configure User Rights: Change the time zone](#)
8. [REQ-END-103 - Configure User Rights: Create a pagefile](#)
9. [REQ-END-104 - Configure User Rights: Create a token object](#)
10. [REQ-END-105 - Configure User Rights: Create global objects](#)
11. [REQ-END-106 - Configure User Rights: Create permanent shared objects](#)
12. [REQ-END-107 - Configure User Rights: Create symbolic links](#)
13. [REQ-END-108 - Configure User Rights: Debug programs](#)
14. [REQ-END-109 - Configure User Rights: Enable computer and user accounts to be trusted for delegation](#)
15. [REQ-END-110 - Configure User Rights: Force shutdown from a remote system](#)
16. [REQ-END-111 - Configure User Rights: Impersonate a client after authentication](#)
17. [REQ-END-112 - Configure User Rights: Increase scheduling priority](#)
18. [REQ-END-113 - Configure User Rights: Load and unload device drivers](#)
19. [REQ-END-114 - Configure User Rights: Lock pages in memory](#)
20. [REQ-END-115 - Configure User Rights: Manage auditing and security log](#)
21. [REQ-END-116 - Configure User Rights: Modify firmware environment values](#)
22. [REQ-END-117 - Configure User Rights: Perform volume maintenance tasks](#)
23. [REQ-END-118 - Configure User Rights: Profile single process](#)
24. [REQ-END-119 - Configure User Rights: Profile system performance](#)
25. [REQ-END-120 - Configure User Rights: Replace a process level token](#)
26. [REQ-END-121 - Configure User Rights: Restore files and directories](#)
27. [REQ-END-122 - Configure User Rights: Take ownership of files or other objects](#)
28. [REQ-END-123 - Configure User Rights: Modify an object label](#)
29. [REQ-END-124 - Configure User Rights: Deny access to this computer from the network](#)
30. [REQ-END-125 - Configure User Rights: Deny log on through Remote Desktop Services](#)

**## Sources & Compliance References** \* **ANSSI Active Directory Hardening Guide**: User Rights Assignment protective controls \* **Microsoft Security Baseline**: User Rights Configuration specifications

# [REQ-END-096] Configure User Rights: Access Credential Manager as a trusted caller

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Low \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Access Credential Manager as a trusted caller` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeTrustedCredManAccessPrivilege` set to `No one (Empty)`.

---

## Rationale No security principals should hold this privilege on endpoints. It prevents attackers from using compromised components to access stored credentials in the Credential Manager.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeTrustedCredManAccessPrivilege` to `No one (Empty)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Access Credential Manager as a trusted caller`. 3. Configure the security principal allocation to: `No one (Empty)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

UraSeTrustedCredManAccessPrivilege.ps1](../implementation\_scripts/Configure-UraSeTrustedCredManAccessPrivilege.ps1)

```
Configure-UraSeTrustedCredManAccessPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_settrustedcredmanaccessprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_settrustedcredmanaccessprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeTrustedCredManAccessPrivilege\s*=") {
 $NewLines += "SeTrustedCredManAccessPrivilege = "
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeTrustedCredManAccessPrivilege = ")
 } else {
 $NewLines += "SeTrustedCredManAccessPrivilege = "
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeTrustedCredManAccessPrivilegeStatus.ps1](#)

```
Get-UraSeTrustedCredManAccessPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_settrustedcredmanaccessprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeTrustedCredManAccessPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-097] Configure User Rights: Access this computer from the network

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Access this computer from the network` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeNetworkLogonRight` set to `\*S-1-5-32-544 (Administrators), \*S-1-5-32-555 (Remote Desktop Users)`.

---

## Rationale Allows users to connect to the computer over the network. Restricting this to Administrators and Remote Desktop Users prevents remote lateral movement by standard domain accounts.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeNetworkLogonRight` to `\*S-1-5-32-544 (Administrators), \*S-1-5-32-555 (Remote Desktop Users)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Access this computer from the network`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators), \*S-1-5-32-555 (Remote Desktop Users)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeNetworkLogonRight.ps1](#)]

(../implementation\_scripts/Configure-UraSeNetworkLogonRight.ps1)

```
Configure-UraSeNetworkLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_senetworklogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_senetworklogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\s]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeNetworkLogonRight\s*=") {
 $NewLines += "SeNetworkLogonRight = *S-1-5-32-544,*S-1-5-32-555"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeNetworkLogonRight = *S-1-5-32-544,*S-1-5-32-555")
 } else {
 $NewLines += "SeNetworkLogonRight = *S-1-5-32-544,*S-1-5-32-555"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeNetworkLogonRightStatus.ps1](#)



```
Get-UraSeNetworkLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_senetworklogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeNetworkLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544,*S-1-5-32-555"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-098] Configure User Rights: Act as part of the operating system

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Act as part of the operating system` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeTcbPrivilege` set to `No one (Empty)`.

---

## Rationale Allows a process to impersonate any user without credentials. This privilege is equivalent to local SYSTEM and should be completely empty to prevent privilege escalation.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeTcbPrivilege` to `No one (Empty)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Act as part of the operating system`. 3. Configure the security principal allocation to: `No one (Empty)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeTcbPrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeTcbPrivilege.ps1)

```
Configure-UraSeTcbPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_setcbprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_setcbprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]\\`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\s]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^\\s*SeTcbPrivilege\\s*=") {
 $NewLines += "SeTcbPrivilege = "
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeTcbPrivilege = ")
 } else {
 $NewLines += "SeTcbPrivilege = "
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeTcbPrivilegeStatus.ps1](#)

```
Get-UraSeTcbPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_setcbprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeTcbPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-END-099] Configure User Rights: Allow log on locally

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Allow log on locally` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeInteractiveLogonRight` set to `\*S-1-5-32-544 (Administrators), \*S-1-5-32-545 (Users)`.

---

## Rationale Allows users to log on interactively at the computer console. Enforcing restriction to Administrators and standard local Users prevents unauthorized local sessions.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeInteractiveLogonRight` to `\*S-1-5-32-544 (Administrators), \*S-1-5-32-545 (Users)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Allow log on locally`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators), \*S-1-5-32-545 (Users)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeInteractiveLogonRight.ps1](#)]

(../implementation\_scripts/Configure-UraSeInteractiveLogonRight.ps1)

```
Configure-UraSeInteractiveLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seinteractivelogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seinteractivelogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeInteractiveLogonRight\s*=") {
 $NewLines += "SeInteractiveLogonRight = *S-1-5-32-544,*S-1-5-32-545"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeInteractiveLogonRight = *S-1-5-32-544,*S-1-5-32-545")
 } else {
 $NewLines += "SeInteractiveLogonRight = *S-1-5-32-544,*S-1-5-32-545"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeInteractiveLogonRightStatus.ps1](#)

```
Get-UraSeInteractiveLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seinteractivelogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeInteractiveLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544,*S-1-5-32-545"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-END-100] Configure User Rights: Back up files and directories

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Back up files and directories` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeBackupPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to bypass file permissions to read all files on the system. Restricting it to Administrators prevents data exfiltration by unauthorized users.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeBackupPrivilege` to `\*S-1-5-32-544 (Administrators)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Back up files and directories`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeBackupPrivilege.ps1](#)]



(../implementation\_scripts/Configure-UraSeBackupPrivilege.ps1)

```
Configure-UraSeBackupPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sebackupprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sebackupprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeBackupPrivilege\s*=") {
 $NewLines += "SeBackupPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeBackupPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeBackupPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeBackupPrivilegeStatus.ps1](#)

```
Get-UraSeBackupPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sebackupprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeBackupPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-END-101] Configure User Rights: Change the system time

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Change the system time` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeSystemtimePrivilege` set to `\*S-1-5-32-544 (Administrators), \*S-1-5-19 (LocalService)`.

---

## Rationale Allows users to change the internal system clock. Restricting this to Administrators and Local Service prevents attackers from disrupting time synchronization, which breaks Kerberos authentication.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeSystemtimePrivilege` to `\*S-1-5-32-544 (Administrators), \*S-1-5-19 (LocalService)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Change the system time`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators), \*S-1-5-19 (LocalService)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeSystemtimePrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeSystemtimePrivilege.ps1)

```
Configure-UraSeSystemtimePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sesystemtimeprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sesystemtimeprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]\\`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^\\[(.*)\\]$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^\\s*SeSystemtimePrivilege\\s*=") {
 $NewLines += "SeSystemtimePrivilege = *S-1-5-32-544,*S-1-5-19"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeSystemtimePrivilege = *S-1-5-32-544,*S-1-5-19")
 } else {
 $NewLines += "SeSystemtimePrivilege = *S-1-5-32-544,*S-1-5-19"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeSystemtimePrivilegeStatus.ps1](#)

```
Get-UraSeSystemtimePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sesystemtimeprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeSystemtimePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544,*S-1-5-19"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-END-102] Configure User Rights: Change the time zone

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Low \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Change the time zone` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeTimeZonePrivilege` set to `\*S-1-5-32-544 (Administrators), \*S-1-5-19 (LocalService), \*S-1-5-32-545 (Users)`.

---

## Rationale Allows users to change the local time zone. Restricting this prevents users from generating misleading event log timestamps.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeTimeZonePrivilege` to `\*S-1-5-32-544 (Administrators), \*S-1-5-19 (LocalService), \*S-1-5-32-545 (Users)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Change the time zone`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators), \*S-1-5-19 (LocalService), \*S-1-5-32-545 (Users)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeTimeZonePrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeTimeZonePrivilege.ps1)

```
Configure-UraSeTimeZonePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_setimezoneprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_setimezoneprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^\[(.*)\]$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^\s*SeTimeZonePrivilege\s*=") {
 $NewLines += "SeTimeZonePrivilege = *S-1-5-32-544,*S-1-5-19,*S-1-5-32-545"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeTimeZonePrivilege = *S-1-5-32-544,*S-1-5-19,*S-1-5-32-545")
 } else {
 $NewLines += "SeTimeZonePrivilege = *S-1-5-32-544,*S-1-5-19,*S-1-5-32-545"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeTimeZonePrivilegeStatus.ps1](#)

```
Get-UraSeTimeZonePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_setimezoneprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeTimeZonePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544,*S-1-5-19,*S-1-5-32-545"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications



# [REQ-END-103] Configure User Rights: Create a pagefile

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Low \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Create a pagefile` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeCreatePagefilePrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to create and change the size of page files. Restricting this to Administrators prevents standard users from exhausting disk space or dumping system paging data.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeCreatePagefilePrivilege` to `\*S-1-5-32-544 (Administrators)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Create a pagefile`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeCreatePagefilePrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeCreatePagefilePrivilege.ps1)

```
Configure-UraSeCreatePagefilePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_secreatepagefileprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_secreatepagefileprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeCreatePagefilePrivilege\s*=") {
 $NewLines += "SeCreatePagefilePrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeCreatePagefilePrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeCreatePagefilePrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeCreatePagefilePrivilegeStatus.ps1](#)

```
Get-UraSeCreatePagefilePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_secreatepagefileprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeCreatePagefilePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-104] Configure User Rights: Create a token object

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Create a token object` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeCreateTokenPrivilege` set to `No one (Empty)`.

---

## Rationale Allows a process to create access tokens. This can be used to escalate privileges to SYSTEM. The privilege must be completely empty.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeCreateTokenPrivilege` to `No one (Empty)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Create a token object`. 3. Configure the security principal allocation to: `No one (Empty)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeCreateTokenPrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeCreateTokenPrivilege.ps1)

```
Configure-UraSeCreateTokenPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_secreatetokenprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_secreatetokenprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\s]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeCreateTokenPrivilege\s*=") {
 $NewLines += "SeCreateTokenPrivilege = "
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeCreateTokenPrivilege = ")
 } else {
 $NewLines += "SeCreateTokenPrivilege = "
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeCreateTokenPrivilegeStatus.ps1](#)

```
Get-UraSeCreateTokenPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_secreatetokenprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeCreateTokenPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-105] Configure User Rights: Create global objects

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Create global objects` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeCreateGlobalPrivilege` set to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators), \*S-1-5-6 (Service)`.

---

## Rationale Allows processes to create global objects available to all sessions. Restricting this to system services and Administrators prevents local privilege escalation via object name collisions.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeCreateGlobalPrivilege` to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators), \*S-1-5-6 (Service)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Create global objects`. 3. Configure the security principal allocation to: `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators), \*S-1-5-6 (Service)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeCreateGlobalPrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeCreateGlobalPrivilege.ps1)

```
Configure-UraSeCreateGlobalPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_secreateglobalprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_secreateglobalprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeCreateGlobalPrivilege\s*=") {
 $NewLines += "SeCreateGlobalPrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeCreateGlobalPrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6")
 } else {
 $NewLines += "SeCreateGlobalPrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeCreateGlobalPrivilegeStatus.ps1](#)



```
Get-UraSeCreateGlobalPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_secreateglobalprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeCreateGlobalPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-END-106] Configure User Rights: Create permanent shared objects

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Create permanent shared objects` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeCreatePermanentPrivilege` set to `No one (Empty)`.

---

## Rationale Allows a process to create directory objects in the object manager. It should be empty to prevent attackers from using it to hide processes or files.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeCreatePermanentPrivilege` to `No one (Empty)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Create permanent shared objects`. 3. Configure the security principal allocation to: `No one (Empty)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

UraSeCreatePermanentPrivilege.ps1](../implementation\_scripts/Configure-UraSeCreatePermanentPrivilege.ps1)

```
Configure-UraSeCreatePermanentPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_secreatepermanentprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_secreatepermanentprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeCreatePermanentPrivilege\s*=") {
 $NewLines += "SeCreatePermanentPrivilege = "
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeCreatePermanentPrivilege = ")
 } else {
 $NewLines += "SeCreatePermanentPrivilege = "
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeCreatePermanentPrivilegeStatus.ps1](#)

```
Get-UraSeCreatePermanentPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_secreatepermanentprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeCreatePermanentPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-107] Configure User Rights: Create symbolic links

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Create symbolic links` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeCreateSymbolicLinkPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to create symbolic links. Restricting this prevents symbolic link attacks that can delete or overwrite critical system files.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeCreateSymbolicLinkPrivilege` to `\*S-1-5-32-544 (Administrators)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Create symbolic links`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

UraSeCreateSymbolicLinkPrivilege.ps1[../implementation\_scripts/Configure-UraSeCreateSymbolicLinkPrivilege.ps1]

```
Configure-UraSeCreateSymbolicLinkPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_secreatesymboliclinkprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_secreatesymboliclinkprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*\\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeCreateSymbolicLinkPrivilege\s*=") {
 $NewLines += "SeCreateSymbolicLinkPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeCreateSymbolicLinkPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeCreateSymbolicLinkPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeCreateSymbolicLinkPrivilegeStatus.ps1](#)

```
Get-UraSeCreateSymbolicLinkPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_secreatesymboliclinkprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeCreateSymbolicLinkPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-108] Configure User Rights: Debug programs

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Debug programs` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeDebugPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows processes to attach to and debug any system process (including lsass.exe). Disallowing it for standard accounts prevents credential dumping tools (Mimikatz) from reading LSASS memory.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeDebugPrivilege` to `\*S-1-5-32-544 (Administrators)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Debug programs`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeDebugPrivilege.ps1](#)]



(../implementation\_scripts/Configure-UraSeDebugPrivilege.ps1)

```
Configure-UraSeDebugPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sedebugprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sedebugprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*\\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeDebugPrivilege\s*=") {
 $NewLines += "SeDebugPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeDebugPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeDebugPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeDebugPrivilegeStatus.ps1](#)

```
Get-UraSeDebugPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sedebugprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeDebugPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-END-109] Configure User Rights: Enable computer and user accounts to be trusted for delegation

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Enable computer and user accounts to be trusted for delegation` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeEnableDelegationPrivilege` set to `No one (Empty)`.

---

## Rationale Allows users to change the delegation setting on a user or computer object in AD. This must be completely empty to prevent Kerberos delegation exploits (such as unconstrained delegation or coercion attacks).

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeEnableDelegationPrivilege` to `No one (Empty)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Enable computer and user accounts to be trusted for delegation`. 3. Configure the security principal allocation to: `No one (Empty)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

UraSeEnableDelegationPrivilege.ps1[../implementation\_scripts/Configure-UraSeEnableDelegationPrivilege.ps1]

```
Configure-UraSeEnableDelegationPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seenabledelegationprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seenabledelegationprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*\\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeEnableDelegationPrivilege\s*=") {
 $NewLines += "SeEnableDelegationPrivilege = "
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeEnableDelegationPrivilege = ")
 } else {
 $NewLines += "SeEnableDelegationPrivilege = "
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeEnableDelegationPrivilegeStatus.ps1](#)

```
Get-UraSeEnableDelegationPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seenabledelegationprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeEnableDelegationPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-110] Configure User Rights: Force shutdown from a remote system

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Force shutdown from a remote system` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeRemoteShutdownPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to shut down computers from a remote network location. Restricting this prevents denial-of-service shutdowns by compromised standard accounts.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeRemoteShutdownPrivilege` to `\*S-1-5-32-544 (Administrators)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Force shutdown from a remote system`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

UraSeRemoteShutdownPrivilege.ps1][../implementation\_scripts/Configure-UraSeRemoteShutdownPrivilege.ps1)

```
Configure-UraSeRemoteShutdownPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seremotesshutdownprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seremotesshutdownprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*\\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeRemoteShutdownPrivilege\s*=") {
 $NewLines += "SeRemoteShutdownPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeRemoteShutdownPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeRemoteShutdownPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeRemoteShutdownPrivilegeStatus.ps1](#)

```
Get-UraSeRemoteShutdownPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seremotesshutdownprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeRemoteShutdownPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications



# [REQ-END-111] Configure User Rights: Impersonate a client after authentication

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Impersonate a client after authentication` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeImpersonatePrivilege` set to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators), \*S-1-5-6 (Service)`.

---

## Rationale Allows programs to impersonate clients. Restricting this to system service accounts and Administrators prevents local privilege escalation via print spooler or potato exploits.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeImpersonatePrivilege` to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators), \*S-1-5-6 (Service)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Impersonate a client after authentication`. 3. Configure the security principal allocation to: `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService), \*S-1-5-32-544 (Administrators), \*S-1-5-6 (Service)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeImpersonatePrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeImpersonatePrivilege.ps1)

```
Configure-UraSeImpersonatePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seimpersonateprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seimpersonateprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*\\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^\\s*SeImpersonatePrivilege\\s*=") {
 $NewLines += "SeImpersonatePrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeImpersonatePrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6")
 } else {
 $NewLines += "SeImpersonatePrivilege = *S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeImpersonatePrivilegeStatus.ps1](#)

```
Get-UraSeImpersonatePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seimpersonateprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeImpersonatePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-19,*S-1-5-20,*S-1-5-32-544,*S-1-5-6"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-END-112] Configure User Rights: Increase scheduling priority

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Low \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Increase scheduling priority` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeIncreaseBasePriorityPrivilege` set to `\*S-1-5-32-544 (Administrators), \*S-1-5-90-0 (Window Manager Group)`.

---

## Rationale Allows processes to increase scheduling execution priority. Restricting this prevents process priority manipulation resulting in system resource starvation.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeIncreaseBasePriorityPrivilege` to `\*S-1-5-32-544 (Administrators), \*S-1-5-90-0 (Window Manager Group)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Increase scheduling priority`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators), \*S-1-5-90-0 (Window Manager Group)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

UraSeIncreaseBasePriorityPrivilege.ps1)[./implementation\_scripts/Configure-UraSeIncreaseBasePriorityPrivilege.ps1)

```
Configure-UraSeIncreaseBasePriorityPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seincreasebasepriorityprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seincreasebasepriorityprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`n[Privilege Rights]"
}

$Lines = $ConfigText -split "`r?"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeIncreaseBasePriorityPrivilege\s*=") {
 $NewLines += "SeIncreaseBasePriorityPrivilege = *S-1-5-32-544,*S-1-5-90-0"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeIncreaseBasePriorityPrivilege = *S-1-5-32-544,*S-1-5-90-0")
 } else {
 $NewLines += "SeIncreaseBasePriorityPrivilege = *S-1-5-32-544,*S-1-5-90-0"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeIncreaseBasePriorityPrivilegeStatus.ps1](#)

```
Get-UraSeIncreaseBasePriorityPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seincreasebasepriorityprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeIncreaseBasePriorityPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544,*S-1-5-90-0"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-113] Configure User Rights: Load and unload device drivers

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Load and unload device drivers` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeLoadDriverPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to load device drivers in kernel mode. Restricting this prevents attackers from loading unsigned or vulnerable drivers (BYOVD) to execute kernel shellcode.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeLoadDriverPrivilege` to `\*S-1-5-32-544 (Administrators)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Load and unload device drivers`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeLoadDriverPrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeLoadDriverPrivilege.ps1)

```
Configure-UraSeLoadDriverPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seloaddriverprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seloaddriverprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeLoadDriverPrivilege*s*") {
 $NewLines += "SeLoadDriverPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeLoadDriverPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeLoadDriverPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeLoadDriverPrivilegeStatus.ps1](#)



```
Get-UraSeLoadDriverPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seloaddriverprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeLoadDriverPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-114] Configure User Rights: Lock pages in memory

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Lock pages in memory` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeLockMemoryPrivilege` set to `No one (Empty)`.

---

## Rationale Allows processes to lock physical pages in memory, preventing the OS from swapping them to pagefile. It should be empty to prevent memory exhaustion DoS attacks.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeLockMemoryPrivilege` to `No one (Empty)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Lock pages in memory`. 3. Configure the security principal allocation to: `No one (Empty)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeLockMemoryPrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeLockMemoryPrivilege.ps1)

```
Configure-UraSeLockMemoryPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_selockmemoryprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_selockmemoryprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "\[Privilege Rights\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\s]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^\\s*SeLockMemoryPrivilege\\s*=") {
 $NewLines += "SeLockMemoryPrivilege = "
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeLockMemoryPrivilege = ")
 } else {
 $NewLines += "SeLockMemoryPrivilege = "
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeLockMemoryPrivilegeStatus.ps1](#)

```
Get-UraSeLockMemoryPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_selockmemoryprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeLockMemoryPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-END-115] Configure User Rights: Manage auditing and security log

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Manage auditing and security log` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeSecurityPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to view, manage, and clear the Windows Security Event Log. Restricting this prevents attackers from clearing evidence of post-compromise activities.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeSecurityPrivilege` to `\*S-1-5-32-544 (Administrators)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Manage auditing and security log`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeSecurityPrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeSecurityPrivilege.ps1)

```
Configure-UraSeSecurityPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sesecurityprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sesecurityprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\s]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeSecurityPrivilege\s*=") {
 $NewLines += "SeSecurityPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeSecurityPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeSecurityPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeSecurityPrivilegeStatus.ps1](#)

```
Get-UraSeSecurityPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sesecurityprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeSecurityPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-116] Configure User Rights: Modify firmware environment values

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Modify firmware environment values` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeSystemEnvironmentPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to configure firmware (UEFI) variables. Restricting this prevents attackers from disabling Secure Boot or modifying boot sector configurations.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeSystemEnvironmentPrivilege` to `\*S-1-5-32-544 (Administrators)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Modify firmware environment values`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-



UraSeSystemEnvironmentPrivilege.ps1[../implementation\_scripts/Configure-UraSeSystemEnvironmentPrivilege.ps1]

```
Configure-UraSeSystemEnvironmentPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sesystemenvironmentprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sesystemenvironmentprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*\\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeSystemEnvironmentPrivilege\s*=") {
 $NewLines += "SeSystemEnvironmentPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeSystemEnvironmentPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeSystemEnvironmentPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeSystemEnvironmentPrivilegeStatus.ps1](#)

```
Get-UraSeSystemEnvironmentPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sesystemenvironmentprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeSystemEnvironmentPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-117] Configure User Rights: Perform volume maintenance tasks

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Perform volume maintenance tasks` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeManageVolumePrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows non-administrative users to run disk utilities. Restricting this to Administrators prevents unauthorized read/write access to raw volume sectors.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeManageVolumePrivilege` to `\*S-1-5-32-544 (Administrators)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Perform volume maintenance tasks`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeManageVolumePrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeManageVolumePrivilege.ps1)

```
Configure-UraSeManageVolumePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_semanagevolumeprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_semanagevolumeprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\s]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^\\s*SeManageVolumePrivilege\\s*=") {
 $NewLines += "SeManageVolumePrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeManageVolumePrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeManageVolumePrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeManageVolumePrivilegeStatus.ps1](#)

```
Get-UraSeManageVolumePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_semanagevolumeprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeManageVolumePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-118] Configure User Rights: Profile single process

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Low \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Profile single process` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeProfileSingleProcessPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to profile non-system processes. Restricting this prevents attackers from profiling critical application components to locate vulnerability endpoints.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeProfileSingleProcessPrivilege` to `\*S-1-5-32-544 (Administrators)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Profile single process`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

UraSeProfileSingleProcessPrivilege.ps1][../implementation\_scripts/Configure-UraSeProfileSingleProcessPrivilege.ps1)

```
Configure-UraSeProfileSingleProcessPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seprofilesinglprocessprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seprofilesinglprocessprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeProfileSingleProcessPrivilege\s*=") {
 $NewLines += "SeProfileSingleProcessPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeProfileSingleProcessPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeProfileSingleProcessPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeProfileSingleProcessPrivilegeStatus.ps1](#)

```
Get-UraSeProfileSingleProcessPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seprofilesingprocessprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeProfileSingleProcessPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications



# [REQ-END-119] Configure User Rights: Profile system performance

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Low \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Profile system performance` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeSystemProfilePrivilege` set to `\*S-1-5-32-544 (Administrators), \*S-1-5-80-3139157870-2983391045-3678747466-658725712-1809340420 (WdiServiceHost)`.

---

## Rationale Allows users to profile system performance. Restricting this prevents unauthorized profiling of kernel components.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeSystemProfilePrivilege` to `\*S-1-5-32-544 (Administrators), \*S-1-5-80-3139157870-2983391045-3678747466-658725712-1809340420 (WdiServiceHost)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Profile system performance`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators), \*S-1-5-80-3139157870-2983391045-3678747466-658725712-1809340420 (WdiServiceHost)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeSystemProfilePrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeSystemProfilePrivilege.ps1)

```
Configure-UraSeSystemProfilePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sesystemprofileprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sesystemprofileprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*\\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^\\s*SeSystemProfilePrivilege\\s*=") {
 $NewLines += "SeSystemProfilePrivilege = *S-1-5-32-544,*S-1-5-80-3139157870-2983391045-3678747466-658725712-1809340420"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeSystemProfilePrivilege = *S-1-5-32-544,*S-1-5-80-3139157870-2983391045-3678747466-658725712-1809340420")
 } else {
 $NewLines += "SeSystemProfilePrivilege = *S-1-5-32-544,*S-1-5-80-3139157870-2983391045-3678747466-658725712-1809340420"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeSystemProfilePrivilegeStatus.ps1](#)

```
Get-UraSeSystemProfilePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sesystemprofileprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeSystemProfilePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544,*S-1-5-80-3139157870-2983391045-3678747466-658725712-1809340420"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-END-120] Configure User Rights: Replace a process level token

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Replace a process level token` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeAssignPrimaryTokenPrivilege` set to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService)`.

---

## Rationale Allows a process to replace the default access token associated with a subprocess. Restricting this to Local Service and Network Service prevents local privilege escalation.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeAssignPrimaryTokenPrivilege` to `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Replace a process level token`. 3. Configure the security principal allocation to: `\*S-1-5-19 (LocalService), \*S-1-5-20 (NetworkService)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-](#)

UraSeAssignPrimaryTokenPrivilege.ps1](../implementation\_scripts/Configure-UraSeAssignPrimaryTokenPrivilege.ps1)

```
Configure-UraSeAssignPrimaryTokenPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_seassignprimarytokenprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_seassignprimarytokenprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*\\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeAssignPrimaryTokenPrivilege\s*=") {
 $NewLines += "SeAssignPrimaryTokenPrivilege = *S-1-5-19,*S-1-5-20"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeAssignPrimaryTokenPrivilege = *S-1-5-19,*S-1-5-20")
 } else {
 $NewLines += "SeAssignPrimaryTokenPrivilege = *S-1-5-19,*S-1-5-20"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeAssignPrimaryTokenPrivilegeStatus.ps1](#)

```
Get-UraSeAssignPrimaryTokenPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_seassignprimarytokenprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeAssignPrimaryTokenPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-19,*S-1-5-20"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-121] Configure User Rights: Restore files and directories

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Restore files and directories` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeRestorePrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to bypass file permissions when restoring files. Restricting this prevents attackers from overwriting critical system files to establish persistent access.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeRestorePrivilege` to `\*S-1-5-32-544 (Administrators)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Restore files and directories`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeRestorePrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeRestorePrivilege.ps1)

```
Configure-UraSeRestorePrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_serestoreprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_serestoreprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\s]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^\\s*SeRestorePrivilege\\s*=") {
 $NewLines += "SeRestorePrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeRestorePrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeRestorePrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeRestorePrivilegeStatus.ps1](#)



```
Get-UraSeRestorePrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_serestoreprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeRestorePrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-END-122] Configure User Rights: Take ownership of files or other objects

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Take ownership of files or other objects` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeTakeOwnershipPrivilege` set to `\*S-1-5-32-544 (Administrators)`.

---

## Rationale Allows users to take ownership of any system object. Restricting this prevents attackers from seizing control of critical files, services, or registry keys.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeTakeOwnershipPrivilege` to `\*S-1-5-32-544 (Administrators)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Take ownership of files or other objects`. 3. Configure the security principal allocation to: `\*S-1-5-32-544 (Administrators)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeTakeOwnershipPrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeTakeOwnershipPrivilege.ps1)

```
Configure-UraSeTakeOwnershipPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_setakeownershipprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_setakeownershipprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\s]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeTakeOwnershipPrivilege\s*=") {
 $NewLines += "SeTakeOwnershipPrivilege = *S-1-5-32-544"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeTakeOwnershipPrivilege = *S-1-5-32-544")
 } else {
 $NewLines += "SeTakeOwnershipPrivilege = *S-1-5-32-544"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeTakeOwnershipPrivilegeStatus.ps1](#)

```
Get-UraSeTakeOwnershipPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_setakeownershipprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeTakeOwnershipPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-32-544"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-END-123] Configure User Rights: Modify an object label

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Modify an object label` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeRelabelPrivilege` set to `No one (Empty)`.

---

## Rationale Allows a process to change the integrity label of files or other objects. It must be empty to prevent attackers from lowering file integrity to bypass security boundaries.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeRelabelPrivilege` to `No one (Empty)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Modify an object label`. 3. Configure the security principal allocation to: `No one (Empty)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeRelabelPrivilege.ps1](#)]

(../implementation\_scripts/Configure-UraSeRelabelPrivilege.ps1)

```
Configure-UraSeRelabelPrivilege.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_serelabelprivilege.cfg"
$DbFile = Join-Path $SecTempDir "secedit_serelabelprivilege.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\s]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^\\s*SeRelabelPrivilege\\s*=") {
 $NewLines += "SeRelabelPrivilege = "
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeRelabelPrivilege = ")
 } else {
 $NewLines += "SeRelabelPrivilege = "
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeRelabelPrivilegeStatus.ps1](#)

```
Get-UraSeRelabelPrivilegeStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sereLabelprivilege.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeRelabelPrivilege\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = ""
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-END-124] Configure User Rights: Deny access to this computer from the network

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Deny access to this computer from the network` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeDenyNetworkLogonRight` set to `\*S-1-5-113 (Local Account), \*S-1-5-114 (Local Account and member of Administrators group)`.

---

## Rationale Explicitly blocks network logon for Local Accounts (S-1-5-113) and Local Administrators (S-1-5-114). This prevents attackers from performing remote lateral movement using compromised local account credentials.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeDenyNetworkLogonRight` to `\*S-1-5-113 (Local Account), \*S-1-5-114 (Local Account and member of Administrators group)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Deny access to this computer from the network`. 3. Configure the security principal allocation to: `\*S-1-5-113 (Local Account), \*S-1-5-114 (Local Account and member of Administrators group)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-UraSeDenyNetworkLogonRight.ps1](#)]



(../implementation\_scripts/Configure-UraSeDenyNetworkLogonRight.ps1)

```
Configure-UraSeDenyNetworkLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sedenynetworklogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sedenynetworklogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^s*SeDenyNetworkLogonRight\s*=") {
 $NewLines += "SeDenyNetworkLogonRight = *S-1-5-113,*S-1-5-114"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeDenyNetworkLogonRight = *S-1-5-113,*S-1-5-114")
 } else {
 $NewLines += "SeDenyNetworkLogonRight = *S-1-5-113,*S-1-5-114"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeDenyNetworkLogonRightStatus.ps1](#)

```
Get-UraSeDenyNetworkLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sedenynetworklogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeDenyNetworkLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-113,*S-1-5-114"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*\*: User Rights Configuration specifications

# [REQ-END-125] Configure User Rights: Deny log on through Remote Desktop Services

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Windows 10/11) \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment\Deny log on through Remote Desktop Services` \* \*\*Registry Location\*\*: Stored inside local security database under privilege `SeDenyRemoteInteractiveLogonRight` set to `\*S-1-5-113 (Local Account), \*S-1-5-114 (Local Account and member of Administrators group)`.

---

## Rationale Explicitly blocks Remote Desktop logons for Local Accounts (S-1-5-113) and Local Administrators (S-1-5-114), forcing administrators to connect using domain credentials over secure paths.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricting `SeDenyRemoteInteractiveLogonRight` to `\*S-1-5-113 (Local Account), \*S-1-5-114 (Local Account and member of Administrators group)` prevents unauthorized local or network actions. Verify if custom service accounts require this privilege before deploying.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\User Rights Assignment` 2. Open the policy `Deny log on through Remote Desktop Services`. 3. Configure the security principal allocation to: `\*S-1-5-113 (Local Account), \*S-1-5-114 (Local Account and member of Administrators group)`.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: Configure-

UraSeDenyRemoteInteractiveLogonRight.ps1[../implementation\_scripts/Configure-UraSeDenyRemoteInteractiveLogonRight.ps1)

```
Configure-UraSeDenyRemoteInteractiveLogonRight.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_sedenyremoteinteractivelogonright.cfg"
$DbFile = Join-Path $SecTempDir "secedit_sedenyremoteinteractivelogonright.sdb"
$LogFile = Join-Path $SecTempDir "secedit.log"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) { Throw "Failed to export security template" }

$ConfigText = Get-Content -Path $CfgFile -Raw
if ($ConfigText -notmatch "[Privilege Rights\\]") {
 $ConfigText += "`r`n[Privilege Rights]`r`n"
}

$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InPriv = $false
$KeyAdded = $false

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*\\$") {
 if ($Matches[1] -eq "Privilege Rights") {
 $InPriv = $true
 } else {
 $InPriv = $false
 }
 }
 if ($InPriv -and $Line -match "^\\s*SeDenyRemoteInteractiveLogonRight\\s*=") {
 $NewLines += "SeDenyRemoteInteractiveLogonRight = *S-1-5-113,*S-1-5-114"
 $KeyAdded = $true
 } else {
 $NewLines += $Line
 }
}

if (-not $KeyAdded) {
 # Find Privilege Rights section index and insert it right after
 $Idx = $NewLines.IndexOf("[Privilege Rights]")
 if ($Idx -ge 0) {
 $NewLines.Insert($Idx + 1, "SeDenyRemoteInteractiveLogonRight = *S-1-5-113,*S-1-5-114")
 } else {
 $NewLines += "SeDenyRemoteInteractiveLogonRight = *S-1-5-113,*S-1-5-114"
 }
}

$NewLines | Set-Content -Path $CfgFile -Force
$Proc = Start-Process secedit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas USER_RIGHTS /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Proc.ExitCode -ne 0) { Throw "Failed to configure secedit user rights" }
Remove-Item -Path $CfgFile, $DbFile -ErrorAction SilentlyContinue
```

To audit the hardening status: [Download Script: Get-UraSeDenyRemoteInteractiveLogonRightStatus.ps1](#)

```
Get-UraSeDenyRemoteInteractiveLogonRightStatus.ps1
$SecTempDir = Join-Path $env:TEMP "SecurityTemplates"
if (-not (Test-Path $SecTempDir)) { New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null }
$CfgFile = Join-Path $SecTempDir "ura_audit_sedenyremoteinteractivelogonright.cfg"

$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Output "Non-Compliant"
 exit 1
}

$ConfigText = Get-Content -Path $CfgFile -Raw
Remove-Item -Path $CfgFile -ErrorAction SilentlyContinue

$Match = $ConfigText -match "(?mi)^\s*SeDenyRemoteInteractiveLogonRight\s*=\s*(.*)$"
$CurrentValue = ""
if ($Match) {
 $CurrentValue = $Matches[1].Trim()
}

$Expected = "*S-1-5-113,*S-1-5-114"
if ($CurrentValue -eq $Expected) {
 Write-Output "Compliant"
 exit 0
} else {
 Write-Output "Non-Compliant"
 exit 1
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: User Rights Assignment protective controls \*  
 \*\*Microsoft Security Baseline\*\*: User Rights Configuration specifications

# [REQ-END-017] Harden DMA and Physical Security

## Target Scope \* \*\*Applicable Systems\*\*: Member Servers, Tier 2 Clients (Workstations / Laptops) \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above)

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Paths / Registry Locations\*\*: \* \*\*GPO Paths\*\*: \* Computer Configuration\Administrative Templates\System\Power Management\Sleep Settings \* Computer Configuration\Administrative Templates\System\Device Installation\Device Installation Restrictions \* Computer Configuration\Administrative Templates\Windows Components\BitLocker Drive Encryption \* \*\*Registry Locations\*\*: \* HKLM\SOFTWARE\Policies\Microsoft\Power\PowerSettings\abfc2519-3608-4c2a-94ea-171b0ed546ab \* `ACSettingIndex` = `0` (REG\_DWORD, Disables standby plugged in) \* `DCSettingIndex` = `0` (REG\_DWORD, Disables standby on battery) \* HKLM\SOFTWARE\Policies\Microsoft\FVE \* `DisableExternalDMAUnderLock` = `1` (REG\_DWORD) \* `RDVDenyCrossOrg` = `0` (REG\_DWORD) \* HKLM\System\CurrentControlSet\Policies\Microsoft\FVE \* `RDVDenyWriteAccess` = `1` (REG\_DWORD) \* HKLM\SOFTWARE\Policies\Microsoft\Windows\DeviceInstall\Restrictions \* `DenyDeviceClasses` = `1` (REG\_DWORD) \* `DenyDeviceClassesRetroactive` = `1` (REG\_DWORD) \* `DenyDeviceIDs` = `1` (REG\_DWORD) \* `DenyDeviceIDsRetroactive` = `1` (REG\_DWORD) \* HKLM\SOFTWARE\Policies\Microsoft\Windows\DeviceInstall\Restrictions\DenyDeviceClasses \* `1` = `{d48179be-ec20-11d1-b6b8-00c04fa372a7}` (REG\_SZ, SBP-2 device setup class) \* HKLM\SOFTWARE\Policies\Microsoft\Windows\DeviceInstall\Restrictions\DenyDeviceIDs \* `1` = `PCI\CC\_0C0A` (REG\_SZ, Blocks Thunderbolt 1 and 2 controllers) \* `2` = `PCI\CC\_0C0010` (REG\_SZ, Blocks IEEE 1394 OHCI compliant Firewire controllers) \* HKLM\SOFTWARE\Policies\Microsoft\Windows\KernelDMAProtection \* `DeviceEnumerationPolicy` = `0` (REG\_DWORD, Block all)

---

## Rationale Physical access to an endpoint introduces distinct attack vectors that bypass traditional OS privilege separation:

1. **Direct Memory Access (DMA) Attacks:** Hot-plug buses (like FireWire, Thunderbolt, and USB4) permit devices to read and write directly to system memory without operating system mediation. Attackers connect specialized hardware (e.g., PCILeech) to hot-plug ports to extract BitLocker encryption keys or session tokens directly from RAM.
  - Disabling the Serial Bus Protocol 2 (SBP-2) setup class ( `{d48179be-ec20-11d1-b6b8-00c04fa372a7}` ) blocks FireWire/IEEE 1394 DMA controllers.
  - Enforcing `DisableExternalDMAUnderLock` prevents DMA requests when the screen is locked.
  - `DeviceEnumerationPolicy` restricts external DMA execution (set to Block all).
2. **Cold Boot Attacks:** When a system enters standby sleep states (S1-S3), the system RAM remains powered. If a laptop is stolen while in standby, an attacker can quickly reboot the machine or cool the RAM chips to dump their contents, extracting credentials or disk encryption keys. Disabling standby forces the system to either remain fully active or enter Hibernation (S4)/Shutdown, where memory contents are encrypted on disk or cleared.
3. **USB Data Exfiltration:** Blocking write access to removable drives unless they are encrypted with BitLocker ( `RDVDenyWriteAccess` ) prevents users or malicious agents from copying confidential data to unauthorized USB media.

---

## Legacy Impact & Compatibility \* \*\*Standby Disabled\*\*: Workstations and laptops will bypass standby states and enter hibernation when closed or idle. This preserves battery but may increase the time required to resume user sessions by a few seconds. \* \*\*DMA Device Support\*\*: Non-compliant external peripherals (such as legacy docking stations or external display adapters) that require DMA without supporting remapping may not work until the user logs on, or may be blocked entirely. \* \*\*USB Writing\*\*: Standard USB flash drives will be read-only unless encrypted via BitLocker on the endpoint. This requires training users on BitLocker To Go deployment.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Create or edit a GPO targeting endpoints (e.g., `GPO_Hardening_DMA_Physical` ).
3. Configure the following settings:

#### 1. Power Management (Disable Standby) Navigate to: `Computer Configuration\Administrative Templates\System\Power Management\Sleep Settings` \* \*\*Policy\*\*: `Allow standby states (S1-S3) when sleeping (plugged in)` → **Disabled** \* \*\*Policy\*\*: `Allow standby states (S1-S3) when sleeping (on battery)` → **Disabled** \* \*\*Policy\*\*: `Require a password when a computer wakes (plugged in)` → **Enabled** \* \*\*Policy\*\*: `Require a password when a computer wakes (on battery)` → **Enabled**

#### 2. BitLocker Removable Storage & DMA Navigate to: `Computer Configuration\Administrative Templates\Windows Components\BitLocker Drive Encryption` \* \*\*Policy\*\*: `Disable new DMA devices when this computer is locked` → **Enabled**

Navigate to: `Computer Configuration\Administrative Templates\Windows Components\BitLocker Drive Encryption\Removable Data Drives`

- **Policy:** `Deny write access to removable drives not protected by BitLocker` → **Enabled**
  - Check **Do not allow write access to devices configured in another organization** → **Disabled** (value 0 / False)

#### 3. Device Installation Restrictions (Block SBP-2 Setup Class & PCI Device IDs) Navigate to: `Computer Configuration\Administrative Templates\System\Device Installation\Device Installation Restrictions` \* \*\*Policy\*\*: `Prevent installation of devices using drivers that match these device setup classes` → **Enabled** \* Click **Show...** and enter: `{d48179be-ec20-11d1-b6b8-00c04fa372a7}` \* Check **Also apply to matching devices that are already installed** → **Enabled** (value 1 / True) \* \*\*Policy\*\*: `Prevent installation of devices that match any of these device IDs` → **Enabled** \* Click **Show...** and enter: `PCI\CC_0C0A` and `PCI\CC_0C0010` \* Check **Also apply to matching**

devices that are already installed\*\* → \*\*Enabled\*\* (value 1 / True)

#### 4. Kernel DMA Protection Navigate to: `Computer Configuration\Administrative Templates\System\Kernel DMA Protection` \* \*\*Policy\*\*:  
`Enable Kernel DMA Protection` → \*\*Enabled\*\* \* \*\*Enumeration policy\*\*: Set to \*\*Block all\*\* (value 0)

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to apply DMA, Sleep, and BitLocker USB registry parameters.

[Download Script: Set-DMAPhysicalSecurity.ps1](#)

```
Set-DMAPhysicalSecurity.ps1
Description: Hardens local registry keys to mitigate DMA attacks, disable standby sleep states, and restrict unencrypted USB writing.

Write-Host "Applying DMA and physical security hardening..." -ForegroundColor Cyan

1. Disable Standby Sleep States (S1-S3)
$SleepPath = "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\abfc2519-3608-4c2a-94ea-171b0ed546ab"
if (-not (Test-Path $SleepPath)) {
 New-Item -Path $SleepPath -Force | Out-Null
}
Set-ItemProperty -Path $SleepPath -Name "ACSettingIndex" -Value 0 -Type DWord
Set-ItemProperty -Path $SleepPath -Name "DCSettingIndex" -Value 0 -Type DWord
Write-Host "[+] Standby sleep states (S1-S3) disabled." -ForegroundColor Green

2. Configure Wake Password Requirement
$WakePath = "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\0e796bdb-100d-47d6-a2d5-f7d2daa51f51"
if (-not (Test-Path $WakePath)) {
 New-Item -Path $WakePath -Force | Out-Null
}
Set-ItemProperty -Path $WakePath -Name "ACSettingIndex" -Value 1 -Type DWord
Set-ItemProperty -Path $WakePath -Name "DCSettingIndex" -Value 1 -Type DWord
Write-Host "[+] Wake password requirement enforced." -ForegroundColor Green

3. BitLocker DMA and Removable Storage Settings
$FvePath = "HKLM:\SOFTWARE\Policies\Microsoft\FVE"
if (-not (Test-Path $FvePath)) {
 New-Item -Path $FvePath -Force | Out-Null
}
Set-ItemProperty -Path $FvePath -Name "DisableExternalDMAUnderLock" -Value 1 -Type DWord
Set-ItemProperty -Path $FvePath -Name "RDVDenyCrossOrg" -Value 0 -Type DWord

$FvePolicyPath = "HKLM:\System\CurrentControlSet\Policies\Microsoft\FVE"
if (-not (Test-Path $FvePolicyPath)) {
 New-Item -Path $FvePolicyPath -Force | Out-Null
}
Set-ItemProperty -Path $FvePolicyPath -Name "RDVDenyWriteAccess" -Value 1 -Type DWord
Write-Host "[+] BitLocker DMA under lock and unencrypted USB write blocks configured." -ForegroundColor Green

4. Device Installation Restrictions (Block SBP-2 class and PCI\CC_0C0A device ID)
$RestrictPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeviceInstall\Restrictions"
if (-not (Test-Path $RestrictPath)) {
 New-Item -Path $RestrictPath -Force | Out-Null
}
Set-ItemProperty -Path $RestrictPath -Name "DenyDeviceClasses" -Value 1 -Type DWord
Set-ItemProperty -Path $RestrictPath -Name "DenyDeviceClassesRetroactive" -Value 1 -Type DWord
Set-ItemProperty -Path $RestrictPath -Name "DenyDeviceIDs" -Value 1 -Type DWord
Set-ItemProperty -Path $RestrictPath -Name "DenyDeviceIDsRetroactive" -Value 1 -Type DWord

$DenyClassPath = Join-Path $RestrictPath "DenyDeviceClasses"
if (-not (Test-Path $DenyClassPath)) {
 New-Item -Path $DenyClassPath -Force | Out-Null
}
Set-ItemProperty -Path $DenyClassPath -Name "1" -Value "{d48179be-ec20-11d1-b6b8-00c04fa372a7}" -Type String

$DenyIDPath = Join-Path $RestrictPath "DenyDeviceIDs"
if (-not (Test-Path $DenyIDPath)) {
 New-Item -Path $DenyIDPath -Force | Out-Null
}
Set-ItemProperty -Path $DenyIDPath -Name "1" -Value "PCI\CC_0C0A" -Type String
Set-ItemProperty -Path $DenyIDPath -Name "2" -Value "PCI\CC_0C0010" -Type String
Write-Host "[+] Device installation blocks for SBP-2 class, PCI\CC_0C0A, and PCI\CC_0C0010 enabled." -ForegroundColor Green

5. Kernel DMA Protection (Block all external DMA)
$KDmaPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\KernelDMAProtection"
if (-not (Test-Path $KDmaPath)) {
 New-Item -Path $KDmaPath -Force | Out-Null
}
Set-ItemProperty -Path $KDmaPath -Name "DeviceEnumerationPolicy" -Value 0 -Type DWord
Write-Host "[+] Kernel DMA Protection DeviceEnumerationPolicy set to 0 (Block all)." -ForegroundColor Green

Write-Host "DMA and physical security settings applied successfully." -ForegroundColor Green
```



To audit local DMA and physical security configuration: [Download Script: Test-DMAPhysicalSecurity.ps1](#)

```
Test-DMAPhysicalSecurity.ps1
Description: Audits local registry configuration for standby settings, DMA protection under lock, USB restrictions, and blocked device setup classes.

Write-Host "--- Auditing DMA and Physical Security ---" -ForegroundColor Cyan

1. Audit Standby Settings
$SleepPath = "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\abfc2519-3608-4c2a-94ea-171b0ed546ab"
$AcSleep = Get-ItemProperty -Path $SleepPath -Name "ACSettingIndex" -ErrorAction SilentlyContinue
$DcSleep = Get-ItemProperty -Path $SleepPath -Name "DCSettingIndex" -ErrorAction SilentlyContinue

$AcSleepVal = if ($AcSleep) { $AcSleep.ACSettingIndex } else { 1 }
$DcSleepVal = if ($DcSleep) { $DcSleep.DCSettingIndex } else { 1 }

$AcSleepColor = if ($AcSleepVal -eq 0) { "Green" } else { "Red" }
$DcSleepColor = if ($DcSleepVal -eq 0) { "Green" } else { "Red" }

Write-Host " - Standby Sleep State (Plugged In) Setting: $AcSleepVal (Required = 0 [Disabled])" -ForegroundColor $AcSleepColor
Write-Host " - Standby Sleep State (On Battery) Setting: $DcSleepVal (Required = 0 [Disabled])" -ForegroundColor $DcSleepColor

2. Audit BitLocker Settings
$FvePath = "HKLM:\SOFTWARE\Policies\Microsoft\FVE"
$DmaLock = Get-ItemProperty -Path $FvePath -Name "DisableExternalDMAUnderLock" -ErrorAction SilentlyContinue
$DmaLockVal = if ($DmaLock) { $DmaLock.DisableExternalDMAUnderLock } else { 0 }
$DmaLockColor = if ($DmaLockVal -eq 1) { "Green" } else { "Red" }

$FvePolicyPath = "HKLM:\System\CurrentControlSet\Policies\Microsoft\FVE"
$UsbWrite = Get-ItemProperty -Path $FvePolicyPath -Name "RDVDenyWriteAccess" -ErrorAction SilentlyContinue
$UsbWriteVal = if ($UsbWrite) { $UsbWrite.RDVDenyWriteAccess } else { 0 }
$UsbWriteColor = if ($UsbWriteVal -eq 1) { "Green" } else { "Red" }

Write-Host " - Disable DMA Under Lock: $DmaLockVal (Required = 1)" -ForegroundColor $DmaLockColor
Write-Host " - USB Unencrypted Write Block: $UsbWriteVal (Required = 1)" -ForegroundColor $UsbWriteColor

3. Audit Device Restriction Settings
$RestrictPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeviceInstall\Restrictions"
$DenyDev = Get-ItemProperty -Path $RestrictPath -Name "DenyDeviceClasses" -ErrorAction SilentlyContinue
$DenyDevVal = if ($DenyDev) { $DenyDev.DenyDeviceClasses } else { 0 }
$DenyDevColor = if ($DenyDevVal -eq 1) { "Green" } else { "Red" }

$DenyID = Get-ItemProperty -Path $RestrictPath -Name "DenyDeviceIDs" -ErrorAction SilentlyContinue
$DenyIDVal = if ($DenyID) { $DenyID.DenyDeviceIDs } else { 0 }
$DenyIDColor = if ($DenyIDVal -eq 1) { "Green" } else { "Red" }

Write-Host " - Prevent Device Setup Class Installation: $DenyDevVal (Required = 1)" -ForegroundColor $DenyDevColor
Write-Host " - Prevent Device ID Installation: $DenyIDVal (Required = 1)" -ForegroundColor $DenyIDColor

$DenyClassPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeviceInstall\Restrictions\DenyDeviceClasses"
$Sbp2 = Get-ItemProperty -Path $DenyClassPath -Name "1" -ErrorAction SilentlyContinue
$Sbp2Val = if ($Sbp2) { $Sbp2."1" } else { "" }
$Sbp2Color = if ($Sbp2Val -eq "{d48179be-ec20-11d1-b6b8-00c04fa372a7}") { "Green" } else { "Red" }

Write-Host " - Blocked SBP-2 Setup Class: '$Sbp2Val' (Required = '{d48179be-ec20-11d1-b6b8-00c04fa372a7}')" -ForegroundColor $Sbp2Color

$DenyIDPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DeviceInstall\Restrictions\DenyDeviceIDs"
$DId1 = Get-ItemProperty -Path $DenyIDPath -Name "1" -ErrorAction SilentlyContinue
$DId1Val = if ($DId1) { $DId1."1" } else { "" }
$DId1Color = if ($DId1Val -eq "PCI\CC_0C0A") { "Green" } else { "Red" }

$DId2 = Get-ItemProperty -Path $DenyIDPath -Name "2" -ErrorAction SilentlyContinue
$DId2Val = if ($DId2) { $DId2."2" } else { "" }
$DId2Color = if ($DId2Val -eq "PCI\CC_0C0010") { "Green" } else { "Red" }

Write-Host " - Blocked Device ID PCI\CC_0C0A: '$DId1Val' (Required = 'PCI\CC_0C0A')" -ForegroundColor $DId1Color
Write-Host " - Blocked Device ID PCI\CC_0C0010: '$DId2Val' (Required = 'PCI\CC_0C0010')" -ForegroundColor $DId2Color

4. Audit Kernel DMA Protection Setting
$KdmaPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\KernelDMAProtection"
$EnumPol = Get-ItemProperty -Path $KdmaPath -Name "DeviceEnumerationPolicy" -ErrorAction SilentlyContinue
$EnumPolVal = if ($EnumPol) { $EnumPol.DeviceEnumerationPolicy } else { 2 }
$EnumPolColor = if ($EnumPolVal -eq 0) { "Green" } else { "Red" }
```

```
Write-Host " - Kernel DMA Protection Policy: $EnumPolVal (Required = 0 [Block all])" -ForegroundColor $EnumPolColor
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*\*: Section 18.2.1 (BitLocker settings), Section 18.8.19.1 (Kernel DMA Protection), Section 18.8.21.3 (Device Installation restrictions) \* \*\*ANSSI AD Hardening Guide\*\*\*: Recommendations on workstation storage encryption and hardware interface security

# [REQ-END-018] Configure Account and Password Policies

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations, Member Servers, Domain Controllers \* \*\*Operating Systems\*\*: Windows Server 2016 (and above), Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Paths / Registry Locations\*\*: \* \*\*GPO Paths\*\*: \* `Computer Configuration\Policies\Windows Settings\Security Settings\Account Policies\Account Lockout Policy` \* `Computer Configuration\Policies\Windows Settings\Security Settings\Account Policies>Password Policy` \* `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` \* `Computer Configuration\Administrative Templates\System\PIN Complexity` \* `Computer Configuration\Administrative Templates\Windows Components\Microsoft Account` \* `Computer Configuration\Administrative Templates\Windows Components\Windows Hello for Business` \* \*\*Registry Locations\*\*: \* Configured via `GptTmpl.inf` (SecEdit System Access settings): \* `MinimumPasswordLength` = `14` (14 characters minimum) \* `PasswordComplexity` = `1` (Complexity enabled) \* `PasswordHistorySize` = `24` (24 passwords remembered) \* `MaxPasswordAge` = `0` (Password does not expire / disabled) \* `MinPasswordAge` = `1` (1 day minimum) \* `ClearTextPassword` = `0` (Reversible encryption disabled) \* `RelaxMinPasswordLengthLimits` = `1` (Relax minimum password length limits enabled) \* `AllowAdministratorLockout` = `1` (Allow Administrator account lockout enabled) \* `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon` \* `ScRemoveOption` = `1` (REG\_SZ, 1 = Lock Workstation) \* `CachedLogonsCount` = `0` (REG\_DWORD) \* `PasswordExpiryWarning` = `14` (REG\_DWORD, prompt user to change password before expiration) \* `HKLM\SECURITY\Cache` \* `NL\$IterationCount` = `1954` (REG\_DWORD, 1954 = 2,000,896 rounds of PBKDF2-SHA1) \* `HKLM\System\CurrentControlSet\Control\Lsa` \* `LimitBlankPasswordUse` = `1` (REG\_DWORD) \* `NoLMHash` = `1` (REG\_DWORD) \* `RestrictAnonymousSAM` = `1` (REG\_DWORD, restrict anonymous enumeration of SAM) \* `RestrictAnonymous` = `1` (REG\_DWORD, restrict anonymous enumeration of shares) \* `ForceNetworkLogon` = `0` (REG\_DWORD, sharing and security model Classic) \* `ObaseCaseInsensitive` = `1` (REG\_DWORD, require case insensitivity for non-Windows subsystems) \* `LmCompatibilityLevel` = `5` (REG\_DWORD, Network security: LAN Manager authentication level - NTLMv2 only) \* `HKLM\System\CurrentControlSet\Control\Lsa\Kerberos\Parameters` \* `AllowPKU2U` = `0` (REG\_DWORD, Allow PKU2U requests disabled) \* `HKLM\System\CurrentControlSet\Control\SecurityProviders\WDigest` \* `UseLogonCredential` = `0` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows\System` \* `AllowDomainPINLogon` = `0` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\PassportForWork\PINComplexity` \* `MinimumPINLength` = `6` (REG\_DWORD) \* `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System` \* `MSAOptional` = `1` (REG\_DWORD) \* `MaxDevicePasswordFailedAttempts` = `10` (REG\_DWORD, 10 or fewer invalid logon attempts, but not 0) \* `CrashOnAuditFail` = `0` (REG\_DWORD, shut down system if unable to log audits disabled) \* `DisableCAD` = `0` (REG\_DWORD, CTRL+ALT+DEL required) \* `DontDisplayLastUserName` = `1` (REG\_DWORD, Don't display last signed-in enabled) \* `HKLM\SOFTWARE\Policies\Microsoft\MicrosoftAccount` \* `DisableUserAuth` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\PassportForWork` \* `RequireSecurityDevice` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Policies\Microsoft\PassportForWork\ExcludeSecurityDevices` \* `TPM12` = `0` (REG\_DWORD) \* `HKLM\System\CurrentControlSet\Services\Netlogon\Parameters` \* `RequireSignOrSeal` = `1` (REG\_DWORD) \* `SealSecureChannel` = `1` (REG\_DWORD) \* `SignSecureChannel` = `1` (REG\_DWORD) \* `DisablePasswordChange` = `0` (REG\_DWORD) \* `MaximumPasswordAge` = `30` (REG\_DWORD) \* `RequireStrongKey` = `1` (REG\_DWORD) \* `ForceLogoffWhenHourExpire` = `1` (REG\_DWORD, force logoff when logon hours expire enabled) \* `HKLM\System\CurrentControlSet\Services\LanmanWorkstation\Parameters` \* `EnablePlainTextPassword` = `0` (REG\_DWORD) \* `HKLM\System\CurrentControlSet\Services\LanmanServer\Parameters` \* `AutoDisconnect` = `15` (REG\_DWORD, idle time before suspending session in minutes) \* `EnableForcedLogoff` = `1` (REG\_DWORD, disconnect clients when logon hours expire enabled) \* `NullSessionShares` = `""` (REG\_MULTI\_SZ, shares that can be accessed anonymously → none) \* `HKLM\System\CurrentControlSet\Control\Lsa\MSV1\_0` \* `allownullsessionfallback` = `0` (REG\_DWORD) \* `NTLMMinClientSec` = `537395200` (REG\_DWORD) \* `NTLMMinServerSec` = `537395200` (REG\_DWORD)

---

## Rationale Securing authentication parameters and account controls reduces the risk of password attacks and session hijackings:

1. **Account Lockout Threshold ( `LockoutBadCount` )**: Brute-force and password-spraying attacks target user accounts to discover credentials. If no lockout threshold is configured, an attacker can make infinite password attempts. Configuring a lockout threshold of 10 bad attempts mitigates online brute-force attacks.
2. **Account Lockout Reset ( `ResetLockoutCount` )**: This policy dictates how long the failed logon counter persists before resetting. Setting this to 15 minutes ensures that password attempts are restricted over time without introducing major helpdesk overhead.
3. **Reversible Encryption ( `ClearTextPassword` )**: Storing passwords using reversible encryption is equivalent to storing cleartext passwords in the directory database. This key option exists only for legacy application support (such as CHAP authentication) and must be disabled to prevent database dumping/credential recovery.
4. **Local Password Complexity and Minimum Length ( `MinimumPasswordLength` , `PasswordComplexity` )**: Setting the minimum password length to 14 characters and enabling complexity makes offline brute-force and dictionary attacks significantly harder. A length of 14 characters is the standard baseline recommended by ANSSI and CIS for general workstations.
5. **Password History and Age ( `PasswordHistorySize` , `MaxPasswordAge` , `MinPasswordAge` )**: Mandating a history of 24 prevents users from cycling between a few similar passwords. Disabling password expiration ( `MaxPasswordAge` = 0 ) aligns with NIST SP 800-63B and modern ANSSI guidelines. Since 14-character complex passwords are sufficiently robust, forcing periodic changes is deprecated as it often leads users to write down passwords or choose predictable variations. A minimum age of 1 day prevents users from immediately bypassing the history requirement by changing their password 24 times in succession.
6. **Smart Card Removal Behavior ( `ScRemoveOption` )**: In secure environments using Smart Card or token-based authentication, removing the card must automatically lock the desktop session ( 1 ). If disabled, a user leaving their workstation with the card removed leaves the

session exposed.

7. **Blank Passwords Limit ( `LimitBlankPasswordUse` )**: Restricting the use of blank passwords to physical console logons prevents attackers from using empty-password accounts to authenticate remotely over network shares or RDP.
8. **Logon Caching Restriction ( `CachedLogonsCount` = 0 ) and Hashing Complexity ( `NL$IterationCount` = 1954 )**: By default, Windows caches previous logons locally as MSCacheV2 hashes, derived using PBKDF2-SHA1. Setting `CachedLogonsCount` to 0 prevents the local storage of credentials for offline validation on standard workstations, forcing authentication against a DC. For systems where caching must be enabled (such as isolated member servers or laptops), the iteration count of the hashing algorithm should be increased using `NL$IterationCount`. Setting it to 1954 results in 2,000,896 rounds of PBKDF2-SHA1, dramatically increasing resistance to offline brute-force and GPU-accelerated cracking attacks (like RTX 4090 models).
9. **LSASS WDigest protection ( `UseLogonCredential` = 0 )**: Disabling WDigest credential caching prevents the LSASS process from storing cleartext passwords in memory.
10. **Microsoft Account and PIN bans**: Restricting Microsoft consumer account authentication and domain PIN logons ensures that standard enterprise credentials and secure Hello for Business PINs are the only mechanisms used.
11. **Secure Channel and NTLM session security**: Forcing secure channel signing, disabling plain text passwords, preventing null session fallbacks, requiring NTLMv2 and 128-bit encryption, and enforcing client-side NTLMv2-only authentication via `LmCompatibilityLevel` = 5 block legacy protocol exploitation and relay vectors.
12. **Kerberos Security Policy**: Restricting Kerberos ticket lifetimes (e.g. 10 hours max ticket lifetime, 7 days max renewal, 600 minutes max service ticket lifetime) and clock skew tolerance (5 minutes) limits the window of opportunity for stolen ticket abuse (Pass-the-Ticket) and ensures synchronization integrity.
13. **GPO Background Refresh Security**: Forcing regular Group Policy background reapplication prevents persistent local configuration changes or drift.
14. **WMI Class Minimization**: Avoiding WMI queries to `Win32_Product` avoids unintended re-installation checks of all MSI packages during GPO processing, protecting host CPU and disk health.
15. **GPO Commentary**: Requiring descriptive GPO comments ensures structural accountability and documentation parity.

---

## Legacy Impact & Compatibility \* **Account Lockouts**: Legitimate users who forget their passwords may lock themselves out. Standard procedures must exist for administrative reset of locked accounts. \* **Smart Card Removal**: Users must be trained to carry their smart cards with them, which automatically locks the session. Re-authenticating requires inserting the card and entering the PIN. \* **Reversible Encryption**: Disabling reversible encryption may break legacy applications that rely on reading cleartext password equivalents. These applications should be modernized to support modern Kerberos or SAML/OIDC federations. \* **Minimum Password Length (14 characters)**: Users with short passwords will be forced to choose a longer password (at least 14 characters) during their next password change. Systems or service accounts with hardcoded shorter credentials must be updated before enforcing this policy. \* **No Password Expiration (MaxPasswordAge = 0)**: Users will no longer be prompted to periodically change their passwords, reducing helpdesk calls related to expired password lockouts and discouraging the use of weak incremental password schemes. \* **Logon Caching (CachedLogonsCount = 0)**: Workstations must have active, real-time connectivity to a Domain Controller to allow users to log on. Off-domain logons (e.g., users traveling or working offline without a pre-boot VPN connection) will fail. Remote users must use pre-boot VPN tunnels or alternate remote access architectures. \* **PBKDF2 Iteration Overhead**: Increasing the iteration count raises the CPU processing time required during logons that use cached credentials. A value of 1954 may introduce a slight delay (typically under 1 second) during interactive logins on older hardware.

---

## ## Implementation Steps

### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### Step 1: Configure Lockout, Password and Kerberos Policies (Domain-wide) These settings must be configured directly within the **Default Domain Policy** to apply domain-wide: 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Edit the **Default Domain Policy**. 3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Account Policies`. 4. Configure the settings: \* **Account Lockout Policy**: \* **Account lockout threshold**: '10' invalid logon attempts \* **Reset account lockout counter after**: '15' minutes \* **Account lockout duration**: '15' minutes (Must be greater than or equal to reset time) \* **Allow Administrator account lockout**: 'Enabled' \* **Password Policy**: \* **Enforce password history**: '24' passwords remembered \* **Maximum password age**: '0' days (never expire) \* **Minimum password age**: '1' day \* **Minimum password length**: '14' characters \* **Password must meet complexity requirements**: 'Enabled' \* **Store passwords using reversible encryption**: 'Disabled' \* **Relax minimum password length limits**: 'Enabled' \* **Kerberos Policy**: \* **Enforce user logon restrictions**: 'Enabled' \* **Maximum lifetime for service ticket**: '600' minutes \* **Maximum lifetime for user ticket**: '10' hours \* **Maximum lifetime for user ticket renewal**: '7' days \* **Maximum tolerance for computer clock synchronization**: '5' minutes

##### Step 2: Configure GPO Security Filtering & Delegation (MS16-072 Compliance) Following the MS16-072 security update, user-scoped GPOs require computer accounts to have Read access to process the policy. When using Security Filtering to restrict GPOs to specific user groups: 1. In GPMC, select your user-targeted GPO. 2. Under the **Scope** tab, in the **Security Filtering** section, select **Authenticated Users** and click **Remove**. 3. Click **Add**, select your target user security group, and click **OK**. 4. Click on the **Delegation** tab. 5. Click **Add**, search for 'Domain Computers' (change Object Types to include Computers), and click **OK**. 6. Set the permissions to **Read** and click **OK**.

##### Step 3: Group Policy WMI Filtering Best Practices When using WMI filters to target GPOs: \* **DO NOT** query the 'Win32\_Product' WMI class. Calling this class triggers Windows Installer to perform a self-repair validation on every installed MSI package on the system,

causing severe CPU spikes and delays during startup and logon. Use lightweight queries such as `Win32\_OperatingSystem` instead.

#### Step 4: Mandate GPO Comments for Accountability For all GPOs created, add a comment in the **Comment** tab of the GPO properties describing the purpose, author, date, and requirement ID.

#### Step 5: Configure Local Security Options In the endpoints GPO (e.g., `GPO\_Hardening\_Workstations`), navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` \* **Policy**: `Interactive logon: Smart card removal behavior` → Set to **Lock Workstation** (value 1) \* **Policy**: `Accounts: Limit local account use of blank passwords to console logon only` → Set to **Enabled** (value 1) \* **Policy**: `Network security: Do not store LAN Manager hash value on next password change` → Set to **Enabled** (value 1) \* **Policy**: `Interactive logon: Number of previous logons to cache (in case domain controller is not available)` → Set to **0** \* **Policy**: `Network security: Minimum session security for NTLM SSP based (including secure RPC) clients` → Set to **Require NTLMv2 session security, Require 128-bit encryption** (value 537395200) \* **Policy**: `Network security: Minimum session security for NTLM SSP based (including secure RPC) servers` → Set to **Require NTLMv2 session security, Require 128-bit encryption** (value 537395200) \* **Policy**: `Network access: Allow anonymous SID/Name translation` → Set to **Disabled** (value 0) \* **Policy**: `Network security: Allow LocalSystem NULL session fallback` → Set to **Disabled** (value 0) \* **Policy**: `Interactive logon: Machine account lockout threshold` → Set to **10** (or fewer invalid logon attempts, but not 0) \* **Policy**: `Audit: Shut down system immediately if unable to log security audits` → Set to **Disabled** (value 0) \* **Policy**: `Interactive logon: Do not require CTRL+ALT+DEL` → Set to **Disabled** (value 0) \* **Policy**: `Interactive logon: Don't display last signed-in` → Set to **Enabled** (value 1) \* **Policy**: `Interactive logon: Prompt user to change password before expiration` → Set to **14** days (or between 5 and 14 days) \* **Policy**: `Microsoft network client: Send unencrypted password to third-party SMB servers` → Set to **Disabled** (value 0) \* **Policy**: `Microsoft network server: Amount of idle time required before suspending session` → Set to **15** or fewer minutes \* **Policy**: `Microsoft network server: Disconnect clients when logon hours expire` → Set to **Enabled** (value 1) \* **Policy**: `Network access: Do not allow anonymous enumeration of SAM accounts and shares` → Set to **Enabled** (value 1) \* **Policy**: `Network access: Shares that can be accessed anonymously` → Set to **None** \* **Policy**: `Network access: Sharing and security model for local accounts` → Set to **Classic - local users authenticate as themselves** \* **Policy**: `Network Security: Allow PKU2U authentication requests to this computer to use online identities` → Set to **Disabled** (value 0) \* **Policy**: `Network security: Force logoff when logon hours expire` → Set to **Enabled** (value 1) \* **Policy**: `System objects: Require case insensitivity for non-Windows subsystems` → Set to **Enabled** (value 1)

#### Step 3: Configure Hello for Business, PIN and Microsoft Account Policies Navigate to: `Computer Configuration\Administrative Templates\System\PIN Complexity` \* **Policy**: `Minimum PIN length` → Set to **Enabled** with value **6**

Navigate to: [Computer Configuration\Administrative Templates\Windows Components\Microsoft Account](#)

- **Policy:** [Block all consumer Microsoft account user authentication](#) → Set to **Enabled**

Navigate to: [Computer Configuration\Administrative Templates\Windows Components\Windows Hello for Business](#)

- **Policy:** [Use a hardware security device](#) → Set to **Enabled**
- **Policy:** [Use convenience PIN sign-in](#) → Set to **Disabled** (value 0)
- **Policy:** [Allow Microsoft accounts to be optional](#) → Set to **Enabled** (value 1)

#### Step 4: Configure PBKDF2 Iteration Count via GPO Preferences Since the PBKDF2 iteration count setting is not exposed in standard ADMX templates, deploy it via Registry GPO Preferences: 1. Within the endpoints GPO, navigate to: `Computer Configuration\Preferences\Windows Settings\Registry` 2. Right-click **Registry**, select **New** → **Registry Item**. 3. Configure: \* **Action**: `Update` \* **Hive**: `HKEY\_LOCAL\_MACHINE` \* **Key Path**: `SECURITY\Cache` \* **Value name**: `NL\$IterationCount` \* **Value type**: `REG\_DWORD` \* **Value data**: `1954` (Decimal)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Enforce the local security settings and SecEdit configuration locally.

[Download Script: Set-AccountPolicies.ps1](#)



```
Set-AccountPolicies.ps1
Configures local account lockout, password parameters, smart card removal behavior, blank password blocks, Hello for Business, Microsoft accounts, secure channel options, and NTLM session security options.

Write-Host "Applying account and password policies..." -ForegroundColor Cyan

1. Enforce local security options via Registry
$WinlogonPath = "HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Winlogon"
if (-not (Test-Path $WinlogonPath)) {
 New-Item -Path $WinlogonPath -Force | Out-Null
}
Set-ItemProperty -Path $WinlogonPath -Name "ScRemoveOption" -Value "1" -Type String -Force
Set-ItemProperty -Path $WinlogonPath -Name "CachedLogonsCount" -Value 0 -Type DWord -Force
Write-Host "[+] Smart card removal behavior and logon caching configured." -ForegroundColor Green

$LsaPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
if (-not (Test-Path $LsaPath)) {
 New-Item -Path $LsaPath -Force | Out-Null
}
Set-ItemProperty -Path $LsaPath -Name "LimitBlankPasswordUse" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $LsaPath -Name "NoLMHash" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $LsaPath -Name "LmCompatibilityLevel" -Value 5 -Type DWord -Force
Write-Host "[+] Blank password, NoLMHash, and client NTLMv2-only options enforced." -ForegroundColor Green

LSASS WDigest caching block
$WDigestPath = "HKLM:\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest"
if (-not (Test-Path $WDigestPath)) {
 New-Item -Path $WDigestPath -Force | Out-Null
}
Set-ItemProperty -Path $WDigestPath -Name "UseLogonCredential" -Value 0 -Type DWord -Force
Write-Host "[+] LSASS WDigest credential caching disabled." -ForegroundColor Green

PBKDF2 Iterations for Cached Logons
$CachePath = "HKLM:\SECURITY\Cache"
if (-not (Test-Path $CachePath)) {
 New-Item -Path $CachePath -Force | Out-Null
}
Set-ItemProperty -Path $CachePath -Name "NL`$IterationCount" -Value 1954 -Type DWord -Force
Write-Host "[+] PBKDF2 cached credentials iteration count configured." -ForegroundColor Green

Hello for Business, PIN and Microsoft Account policies
$SystemPolicyPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System"
if (-not (Test-Path $SystemPolicyPath)) {
 New-Item -Path $SystemPolicyPath -Force | Out-Null
}
Set-ItemProperty -Path $SystemPolicyPath -Name "AllowDomainPINLogon" -Value 0 -Type DWord -Force

$PinComplexityPath = "HKLM:\SOFTWARE\Policies\Microsoft\PassportForWork\PINComplexity"
if (-not (Test-Path $PinComplexityPath)) {
 New-Item -Path $PinComplexityPath -Force | Out-Null
}
Set-ItemProperty -Path $PinComplexityPath -Name "MinimumPINLength" -Value 6 -Type DWord -Force

$SystemPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
if (-not (Test-Path $SystemPath)) {
 New-Item -Path $SystemPath -Force | Out-Null
}
Set-ItemProperty -Path $SystemPath -Name "MSAOptional" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $SystemPath -Name "MaxDevicePasswordFailedAttempts" -Value 10 -Type DWord -Force

$MsaPath = "HKLM:\SOFTWARE\Policies\Microsoft\MicrosoftAccount"
if (-not (Test-Path $MsaPath)) {
 New-Item -Path $MsaPath -Force | Out-Null
}
Set-ItemProperty -Path $MsaPath -Name "DisableUserAuth" -Value 1 -Type DWord -Force

$PassportPath = "HKLM:\SOFTWARE\Policies\Microsoft\PassportForWork"
if (-not (Test-Path $PassportPath)) {
 New-Item -Path $PassportPath -Force | Out-Null
}
Set-ItemProperty -Path $PassportPath -Name "RequireSecurityDevice" -Value 1 -Type DWord -Force

$ExcludeDevicesPath = "HKLM:\SOFTWARE\Policies\Microsoft\PassportForWork\ExcludeSecurityDevices"
if (-not (Test-Path $ExcludeDevicesPath)) {
```

```

 New-Item -Path $ExcludeDevicesPath -Force | Out-Null
}
Set-ItemProperty -Path $ExcludeDevicesPath -Name "TPM12" -Value 0 -Type DWord -Force
Write-Host "[+] Hello for Business, PIN and Microsoft Account options configured." -ForegroundColor Green

Domain Member Secure Channel netlogon settings
$NetlogonPath = "HKLM:\System\CurrentControlSet\Services\Netlogon\Parameters"
if (-not (Test-Path $NetlogonPath)) {
 New-Item -Path $NetlogonPath -Force | Out-Null
}
Set-ItemProperty -Path $NetlogonPath -Name "RequireSignOrSeal" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $NetlogonPath -Name "SealSecureChannel" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $NetlogonPath -Name "SignSecureChannel" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $NetlogonPath -Name "DisablePasswordChange" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $NetlogonPath -Name "MaximumPasswordAge" -Value 30 -Type DWord -Force
Set-ItemProperty -Path $NetlogonPath -Name "RequireStrongKey" -Value 1 -Type DWord -Force
Write-Host "[+] Domain Member secure channel configurations applied." -ForegroundColor Green

LanmanWorkstation plain text passwords block
$LanmanWorkPath = "HKLM:\System\CurrentControlSet\Services\LanmanWorkstation\Parameters"
if (-not (Test-Path $LanmanWorkPath)) {
 New-Item -Path $LanmanWorkPath -Force | Out-Null
}
Set-ItemProperty -Path $LanmanWorkPath -Name "EnablePlainTextPassword" -Value 0 -Type DWord -Force

NTLM SSP Client & Server security and Null Session Fallback
$MsvPath = "HKLM:\System\CurrentControlSet\Control\Lsa\MSV1_0"
if (-not (Test-Path $MsvPath)) {
 New-Item -Path $MsvPath -Force | Out-Null
}
Set-ItemProperty -Path $MsvPath -Name "allownullsessionfallback" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $MsvPath -Name "NTLMMinClientSec" -Value 537395200 -Type DWord -Force
Set-ItemProperty -Path $MsvPath -Name "NTLMMinServerSec" -Value 537395200 -Type DWord -Force
Write-Host "[+] Network authentication security and NTLM session settings applied." -ForegroundColor Green

Additional local security options for endpoints
$SystemPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
if (-not (Test-Path $SystemPath)) {
 New-Item -Path $SystemPath -Force | Out-Null
}
Set-ItemProperty -Path $SystemPath -Name "CrashOnAuditFail" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $SystemPath -Name "DisableCAD" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $SystemPath -Name "DontDisplayLastUserName" -Value 1 -Type DWord -Force

Set-ItemProperty -Path $WinLogonPath -Name "PasswordExpiryWarning" -Value 14 -Type DWord -Force

Set-ItemProperty -Path $LsaPath -Name "RestrictAnonymousSAM" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $LsaPath -Name "RestrictAnonymous" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $LsaPath -Name "ForceNetworkLogon" -Value 0 -Type DWord -Force
Set-ItemProperty -Path $LsaPath -Name "ObaseCaseInsensitive" -Value 1 -Type DWord -Force

$KerbParamsPath = "HKLM:\System\CurrentControlSet\Control\Lsa\Kerberos\Parameters"
if (-not (Test-Path $KerbParamsPath)) {
 New-Item -Path $KerbParamsPath -Force | Out-Null
}
Set-ItemProperty -Path $KerbParamsPath -Name "AllowPKU2U" -Value 0 -Type DWord -Force

$LanmanServerPath = "HKLM:\System\CurrentControlSet\Services\LanmanServer\Parameters"
if (-not (Test-Path $LanmanServerPath)) {
 New-Item -Path $LanmanServerPath -Force | Out-Null
}
Set-ItemProperty -Path $LanmanServerPath -Name "AutoDisconnect" -Value 15 -Type DWord -Force
Set-ItemProperty -Path $LanmanServerPath -Name "EnableForcedLogoff" -Value 1 -Type DWord -Force
Set-ItemProperty -Path $LanmanServerPath -Name "NullSessionShares" -Value @() -Type MultiString -Force

Set-ItemProperty -Path $NetlogonPath -Name "ForceLogoffWhenHourExpire" -Value 1 -Type DWord -Force
Write-Host "[+] Additional network and interactive security options applied." -ForegroundColor Green

2. Enforce Account Lockout and Password Policy via secedit
$SecTempDir = Join-Path $env:TEMP "AccountSecurityTemplates"
if (-not (Test-Path $SecTempDir)) {
 New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null
}

$CfgFile = Join-Path $SecTempDir "account_sec.cfg"
$LogFile = Join-Path $SecTempDir "secedit.log"
$DbFile = Join-Path $SecTempDir "secedit.sdb"

```



```

Export current db
$Process = Start-Process secedit -ArgumentList "/export /cfg `"$CfgFile`"" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Error "Failed to export current configuration database."
 return
}

$ConfigText = Get-Content -Path $CfgFile -Raw
$HasSystemAccess = $ConfigText -match "\[System Access\]"
if (-not $HasSystemAccess) {
 $ConfigText += "`r`n[System Access]`r`n"
}

Re-build [System Access] section line-by-line
$Lines = $ConfigText -split "`r?`n"
$NewLines = @()
$InSystemAccess = $false

$AccountSettings = @{
 "LockoutBadCount" = 10
 "ResetLockoutCount" = 15
 "LockoutDuration" = 15
 "ClearTextPassword" = 0
 "MinimumPasswordLength" = 14
 "PasswordComplexity" = 1
 "PasswordHistorySize" = 24
 "MaxPasswordAge" = 0
 "MinPasswordAge" = 1
 "RelaxMinPasswordLengthLimits" = 1
 "AllowAdministratorLockout" = 1
 "MaxServiceTicketAge" = 600
 "MaxTicketAge" = 10
 "MaxRenewAge" = 7
 "MaxClockSkew" = 5
 "TicketValidateClient" = 1
}

foreach ($Line in $Lines) {
 if ($Line -match "^[^\\]*\\$") {
 $SectionName = $Matches[1]
 if ($SectionName -eq "System Access") {
 $InSystemAccess = $true
 $NewLines += $Line
 continue
 } else {
 $InSystemAccess = $false
 }
 }

 if ($InSystemAccess) {
 $IsManaged = $false
 foreach ($Key in $AccountSettings.Keys) {
 if ($Line -match "^\\s*$(($Key)\\s*=)") {
 $IsManaged = $true
 break
 }
 }
 if (-not $IsManaged) {
 $NewLines += $Line
 }
 } else {
 $NewLines += $Line
 }
}

Append our settings
$FinalLines = @()
foreach ($Line in $NewLines) {
 $FinalLines += $Line
 if ($Line -eq "[System Access]") {
 foreach ($Key in $AccountSettings.Keys) {
 $Val = $AccountSettings[$Key]
 $FinalLines += "$(($Key)) = $($Val)"
 }
 }
}

```

```
$FinalLines -join "`r`n" | Out-File -FilePath $CfgFile -Encoding ascii -Force

Import
$Process = Start-Process secdit -ArgumentList "/configure /db `"$DbFile`" /cfg `"$CfgFile`" /areas SECURITYPOLICY /log `"$LogFile`" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host "[+] Lockout and password policies applied locally." -ForegroundColor Green
} else {
 Write-Error "Failed to apply local account policies. Exit Code: $($Process.ExitCode)"
}

Remove-Item -Path $SecTempDir -Recurse -Force -ErrorAction SilentlyContinue
```

To audit local account and password policies: [Download Script: Test-AccountPolicies.ps1](#)

```
Test-AccountPolicies.ps1
Checks local registry and SecEdit settings for account lockout, password options, smart card removal behavior, PIN parameters, Hel
lo for Business, Microsoft account settings, secure channel properties, and NTLM session configuration.

Write-Host "--- Auditing Account and Password Policies ---" -ForegroundColor Cyan

$script:Vulnerable = $false

Helper function to audit registry properties
function Test-RegistryValue ($path, $name, $expectedValue) {
 $val = Get-ItemProperty -Path $path -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 $color = "Red"
 if ($actual -eq $expectedValue) {
 $color = "Green"
 } else {
 $script:Vulnerable = $true
 }
 Write-Host " - Registry Setting: $name | Actual: '$actual' (Expected: '$expectedValue')" -ForegroundColor $color
}

1. Audit Registry Settings
$WinlogonPath = "HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Winlogon"
Test-RegistryValue $WinlogonPath "ScRemoveOption" "1"
Test-RegistryValue $WinlogonPath "CachedLogonsCount" 0

$LsaPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
Test-RegistryValue $LsaPath "LimitBlankPasswordUse" 1
Test-RegistryValue $LsaPath "NoLMHash" 1

$WDigestPath = "HKLM:\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest"
Test-RegistryValue $WDigestPath "UseLogonCredential" 0

$CachePath = "HKLM:\SECURITY\Cache"
Test-RegistryValue $CachePath "NL`$IterationCount" 1954

$SystemPolicyPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System"
Test-RegistryValue $SystemPolicyPath "AllowDomainPINLogon" 0

$PinComplexityPath = "HKLM:\SOFTWARE\Policies\Microsoft\PassportForWork\PINComplexity"
Test-RegistryValue $PinComplexityPath "MinimumPINLength" 6

$SystemPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
Test-RegistryValue $SystemPath "MSAOptional" 1
Test-RegistryValue $SystemPath "MaxDevicePasswordFailedAttempts" 10

$MsaPath = "HKLM:\SOFTWARE\Policies\Microsoft\MicrosoftAccount"
Test-RegistryValue $MsaPath "DisableUserAuth" 1

$PassportPath = "HKLM:\SOFTWARE\Policies\Microsoft\PassportForWork"
Test-RegistryValue $PassportPath "RequireSecurityDevice" 1

$ExcludeDevicesPath = "HKLM:\SOFTWARE\Policies\Microsoft\PassportForWork\ExcludeSecurityDevices"
Test-RegistryValue $ExcludeDevicesPath "TPM12" 0

$NetlogonPath = "HKLM:\System\CurrentControlSet\Services\Netlogon\Parameters"
Test-RegistryValue $NetlogonPath "RequireSignOrSeal" 1
Test-RegistryValue $NetlogonPath "SealSecureChannel" 1
Test-RegistryValue $NetlogonPath "SignSecureChannel" 1
Test-RegistryValue $NetlogonPath "DisablePasswordChange" 0
Test-RegistryValue $NetlogonPath "MaximumPasswordAge" 30
Test-RegistryValue $NetlogonPath "RequireStrongKey" 1

$LanmanWorkPath = "HKLM:\System\CurrentControlSet\Services\LanmanWorkstation\Parameters"
Test-RegistryValue $LanmanWorkPath "EnablePlainTextPassword" 0

$MsvPath = "HKLM:\System\CurrentControlSet\Control\Lsa\MSV1_0"
Test-RegistryValue $MsvPath "allownullsessionfallback" 0
Test-RegistryValue $MsvPath "NTLMMinClientSec" 537395200
Test-RegistryValue $MsvPath "NTLMMinServerSec" 537395200

Audit Additional security options
$SystemPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
Test-RegistryValue $SystemPath "CrashOnAuditFail" 0
```

```

Test-RegistryValue $SystemPath "DisableCAD" 0
Test-RegistryValue $SystemPath "DontDisplayLastUserName" 1

Test-RegistryValue $WinlogonPath "PasswordExpiryWarning" 14

$LsaPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
Test-RegistryValue $LsaPath "RestrictAnonymousSAM" 1
Test-RegistryValue $LsaPath "RestrictAnonymous" 1
Test-RegistryValue $LsaPath "ForceNetworkLogon" 0
Test-RegistryValue $LsaPath "ObaseCaseInsensitive" 1
Test-RegistryValue $LsaPath "LmCompatibilityLevel" 5

$KerberosPath = "HKLM:\System\CurrentControlSet\Control\Lsa\Kerberos\Parameters"
Test-RegistryValue $KerberosPath "AllowPKU2U" 0

$LanmanServerPath = "HKLM:\System\CurrentControlSet\Services\LanmanServer\Parameters"
Test-RegistryValue $LanmanServerPath "AutoDisconnect" 15
Test-RegistryValue $LanmanServerPath "EnableForcedLogoff" 1

Audit NullSessionShares
$NullSessionVal = Get-ItemProperty -Path $LanmanServerPath -Name "NullSessionShares" -ErrorAction SilentlyContinue
$NullSessionActual = if ($NullSessionVal) { $NullSessionVal.NullSessionShares } else { "" }
$NullSessionColor = "Red"
if ($Null -eq $NullSessionActual -or $NullSessionActual -eq "" -or $NullSessionActual.Count -eq 0 -or ($NullSessionActual.Count -eq 1 -and $NullSessionActual[0] -eq "")) {
 $NullSessionColor = "Green"
} else {
 $script:Vulnerable = $true
}
Write-Host " - Registry Setting: NullSessionShares | Actual: '$NullSessionActual' (Expected: empty/None)" -ForegroundColor $NullSessionColor

Test-RegistryValue $NetlogonPath "ForceLogoffWhenHourExpire" 1

2. Audit SecEdit Settings
$SecTempDir = Join-Path $env:TEMP "AccountAuditSecurityTemplates"
if (-not (Test-Path $SecTempDir)) {
 New-Item -Path $SecTempDir -ItemType Directory -Force | Out-Null
}

$CfgFile = Join-Path $SecTempDir "account_audit.cfg"
$Process = Start-Process secedit -ArgumentList "/export /cfg '$CfgFile'" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -ne 0) {
 Write-Error "Failed to export current database."
 return
}

$ConfigContent = Get-Content -Path $CfgFile -Raw
$AccountSettings = @{
 "LockoutBadCount" = 10
 "ResetLockoutCount" = 15
 "LockoutDuration" = 15
 "ClearTextPassword" = 0
 "MinimumPasswordLength" = 14
 "PasswordComplexity" = 1
 "PasswordHistorySize" = 24
 "MaxPasswordAge" = 0
 "MinPasswordAge" = 1
 "RelaxMinPasswordLengthLimits" = 1
 "AllowAdministratorLockout" = 1
 "MaxServiceTicketAge" = 600
 "MaxTicketAge" = 10
 "MaxRenewAge" = 7
 "MaxClockSkew" = 5
 "TicketValidateClient" = 1
}

foreach ($Key in $AccountSettings.Keys) {
 $Expected = $AccountSettings[$Key]
 if ($ConfigContent -match "(?m)^\s*$(($Key)\s*=\s*(.*)\s*$)") {
 $Actual = $Matches[1].Trim()
 } else {
 $Actual = ""
 }

 $Color = "Red"
 if ($Actual -eq [string]$Expected) {

```

```

 $Color = "Green"
 } else {
 $script:Vulnerable = $true
 }
 Write-Host " - System Access Setting: $($Key) | Actual: '$($Actual)' (Expected: '$($Expected)')" -ForegroundColor $Color
}

Remove-Item -Path $SecTempDir -Recurse -Force -ErrorAction SilentlyContinue

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
}

```

---

**## Sources & Compliance References** \* **CIS Microsoft Windows 10/11 Client Benchmark**: Section 1.1 (Password Policy), Section 1.2 (Account Lockout Policy), Section 2.3.2.2 (CrashOnAuditFail), Section 2.3.7.1 (DisableCAD), Section 2.3.7.2 (DontDisplayLastUserName), Section 2.3.7.3 (blank passwords), Section 2.3.7.8 (PasswordExpiryWarning), Section 2.3.9.1 (AutoDisconnect), Section 2.3.9.4 (EnableForcedLogoff), Section 2.3.9.5 (Smart card behavior), Section 2.3.10.3 (RestrictAnonymousSAM), Section 2.3.10.11 (NullSessionShares), Section 2.3.10.12 (ForceNetworkLogon), Section 2.3.11.3 (AllowPKU2U), Section 2.3.11.6 (ForceLogoffWhenHourExpire), Section 2.3.11.8 (anonymous translation), Section 2.3.11.10 (null session fallback), Section 2.3.15.1 (ObaseCaseInsensitive). \* **CIS Microsoft Windows 10/11 Client Benchmark**: Section 1.1.6 (RelaxMinPasswordLengthLimits), Section 1.2.3 (AllowAdministratorLockout). \* **ANSSI AD Hardening Guide**: Recommendations on password complexity, reversible encryption blocks, lockout management, and domain member secure channels. \* **DoD Windows 11 Computer STIG v2r6**: Various account policy, PIN complexity, Windows Hello for Business, Microsoft account restrictions, WDigest disabled, and Netlogon secure channel parameters.

## # Configure User Profile Restrictions

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Paths / Registry Locations\*\*: \* \*\*GPO Paths\*\*: Multiple policies under Administrative Templates and Security Settings. \* \*\*Registry Locations\*\*: Stored inside local HKLM and HKCU security hives under custom settings.

---

## Rationale Securing user profile characteristics and administrative explorer behaviors prevents exposure of sensitive information, restricts arbitrary file execution pathways, disables unapproved telemetry/consumer features, and locks down potential privilege escalation points. This submodule contains individual requirement rules for each User Profile restriction setting configured on standard client workstations.

---

## Legacy Impact & Compatibility \* \*\*User Customization Limits\*\*: Users cannot customize lock screen slideshows, cameras, and Windows consumer feature recommendations. Legacy applications that rely on custom DLL loads via Applnit\_DLLs or dynamically self-modifying batch processes may require exclusions or manual policy configurations.

---

## ## Enforced User Profile Restrictions

The following individual User Profile restriction rules must be configured:

1. [REQ-END-126 - User Profile: Toast Notifications Lock Screen Restrictions](#)
  2. [REQ-END-127 - User Profile: Spotlight and Consumer Features Restrictions](#)
  3. [REQ-END-128 - User Profile: Windows Copilot Restrictions](#)
  4. [REQ-END-129 - User Profile: In-Place Sharing Restrictions](#)
  5. [REQ-END-130 - User Profile: Shell RunAs User Suppression](#)
  6. [REQ-END-131 - User Profile: Personalization and Privacy Restrictions](#)
  7. [REQ-END-132 - User Profile: Group Policy Processing Behaviors](#)
  8. [REQ-END-133 - User Profile: Telemetry and Inventory Collection Restrictions](#)
  9. [REQ-END-134 - User Profile: Explorer Security and Memory Protections](#)
  10. [REQ-END-135 - User Profile: Internet Explorer Options and Feeds Restrictions](#)
  11. [REQ-END-136 - User Profile: Interactive Logon Warning Banners](#)
  12. [REQ-END-137 - User Profile: Interactive Logon Inactivity Timeout](#)
  13. [REQ-END-138 - User Profile: Windows Installer Hardening](#)
  14. [REQ-END-139 - User Profile: Secondary Logon Service Lockdown](#)
  15. [REQ-END-150 - User Profile: Structured Exception Handling Overwrite Protection \(SEOP\) for Endpoints](#)
  16. [REQ-END-151 - User Profile: Directory Protection Mode for Endpoints](#)
  17. [REQ-END-152 - User Profile: Address Space Layout Randomization \(ASLR\) Image Relocation for Endpoints](#)
  18. [REQ-END-153 - User Profile: Speculative Execution Mitigations \(Spectre/Meltdown\) for Endpoints](#)
  19. [REQ-END-154 - User Profile: Authenticode Certificate Padding Check for Endpoints](#)
  20. [REQ-END-155 - User Profile: Command Processor Batch File Locking for Endpoints](#)
  21. [REQ-END-156 - User Profile: Time-Travel Debugging \(TTD\) Recording Policy for Endpoints](#)
  22. [REQ-END-157 - User Profile: Trusted Root Store Protected Roots Certificate Restriction for Endpoints](#)
  23. [REQ-END-158 - User Profile: Disabling Injection of Applnit DLLs for Endpoints](#)
  24. [REQ-END-159 - User Profile: Preservation of Attachment Zone Information for Endpoints](#)
  25. [REQ-END-160 - User Profile: Disable Windows Game DVR for Endpoints](#)
  26. [REQ-END-161 - User Profile: Restrict Windows Ink Workspace on Lock Screen for Endpoints](#)
- 

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*: Section 18.8 (User Profile Settings) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines

# [REQ-END-126] User Profile: Toast Notifications Lock Screen Restrictions

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*  
 'HKCU\Software\Policies\Microsoft\Windows\CurrentVersion\PushNotifications\NoToastApplicationNotificationOnLockScreen' = '1' (DWord)

## Rationale Disables application notifications on the lock screen to prevent third parties from reading sensitive notification banners or MFA verification tokens on locked machines.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: 'User Configuration \ Administrative Templates \ Start Menu and Taskbar \ Notifications' 2. Configure the policy: \* \*\*Policy\*\*: 'Turn off toast notifications on the lock screen' → Set to \*\*Enabled\*\*

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Uptoastnotifications.ps1](#)] ([../implementation\\_scripts/Configure-Uptoastnotifications.ps1](#))

```
Configure-Uptoastnotifications.ps1
Write-Host "Applying User Profile restriction: toast-notifications..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CurrentVersion\PushNotifications" "NoToastApplicationNotificationOnLockScreen" "1" "DWord"

Apply to Default User profile for new sessions
$DefaultHivePath = "C:\Users\Default\NTUSER.DAT"
if (Test-Path $DefaultHivePath) {
 reg load HKU\DefaultUser $DefaultHivePath | Out-Null
 $DefaultKey = "Registry::HKU\DefaultUser\Software\Policies\Microsoft\Windows\CurrentVersion\PushNotifications"
 if (-not (Test-Path $DefaultKey)) { New-Item -Path $DefaultKey -Force | Out-Null }
 Set-ItemProperty -Path $DefaultKey -Name "NoToastApplicationNotificationOnLockScreen" -Value "1" -Type DWord -Force
 [GC]::Collect()
 [GC]::WaitForPendingFinalizers()
 reg unload HKU\DefaultUser | Out-Null
}
```

To audit the hardening status: [Download Script: Get-UptoastnotificationsStatus.ps1](#)

```
Get-UptoastnotificationsStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CurrentVersion\PushNotifications" "NoToastApplicationNotificationOnLockScreen" "1"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: User profile privacy and shell lockdown controls \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Client security baselines



# [REQ-END-127] User Profile: Spotlight and Consumer Features Restrictions

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKCU\Software\Policies\Microsoft\Windows\CloudContent\DisableThirdPartySuggestions` = `1` (DWord) \*

`HKCU\Software\Policies\Microsoft\Windows\CloudContent\ConfigureWindowsSpotlight` = `2` (DWord) \*

`HKCU\Software\Policies\Microsoft\Windows\CloudContent\DisableSpotlightCollectionOnDesktop` = `1` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\CloudContent\DisableWindowsConsumerFeatures` = `1` (DWord)

---

## Rationale Disables Windows consumer features, Spotlight wallpaper suggestions, and third-party advertising in the shell to limit telemetry and prevent social-engineering application suggestions.

---

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Cloud Content` 2. Configure the policy: \* \*\*Policy\*\*: `Turn off Microsoft consumer experiences` → Set to **Enabled** 3. Navigate to: `User Configuration \ Administrative Templates \ Windows Components \ Cloud Content` 4. Configure the policies: \* \*\*Policy\*\*: `Do not suggest third-party content in Windows spotlight` → Set to **Enabled** \* \*\*Policy\*\*: `Configure Windows spotlight on lock screen` → Set to **Enabled** (and select **Disabled** to turn off recommendations) 5. Deploy custom registry preference for desktop spotlight (HKCU): \* Navigate to: `User Configuration \ Preferences \ Windows Settings \ Registry` \* Create a new **Registry Item**: \* \*\*Action\*\*: Update \* \*\*Hive\*\*: HKEY\_CURRENT\_USER \* \*\*Key Path\*\*: `Software\Policies\Microsoft\Windows\CloudContent` \* \*\*Value name\*\*: `DisableSpotlightCollectionOnDesktop` \* \*\*Value type\*\*: REG\_DWORD \* \*\*Value data\*\*: `1`

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Upspotlightconsumer.ps1](#)]

(../implementation\_scripts/Configure-Upspotlightconsumer.ps1)

```
Configure-Upspotlightconsumer.ps1
Write-Host "Applying User Profile restriction: spotlight-consumer..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CloudContent" "DisableThirdPartySuggestions" "1" "DWord"
Set-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CloudContent" "ConfigureWindowsSpotlight" "2" "DWord"
Set-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CloudContent" "DisableSpotlightCollectionOnDesktop" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\CloudContent" "DisableWindowsConsumerFeatures" "1" "DWord"

Apply to Default User profile for new sessions
$DefaultHivePath = "C:\Users\Default\NTUSER.DAT"
if (Test-Path $DefaultHivePath) {
 reg load HKU\DefaultUser $DefaultHivePath | Out-Null
 $DefaultKey = "Registry::HKU\DefaultUser\Software\Policies\Microsoft\Windows\CloudContent"
 if (-not (Test-Path $DefaultKey)) { New-Item -Path $DefaultKey -Force | Out-Null }
 Set-ItemProperty -Path $DefaultKey -Name "DisableThirdPartySuggestions" -Value "1" -Type DWord -Force
 $DefaultKey = "Registry::HKU\DefaultUser\Software\Policies\Microsoft\Windows\CloudContent"
 if (-not (Test-Path $DefaultKey)) { New-Item -Path $DefaultKey -Force | Out-Null }
 Set-ItemProperty -Path $DefaultKey -Name "ConfigureWindowsSpotlight" -Value "2" -Type DWord -Force
 $DefaultKey = "Registry::HKU\DefaultUser\Software\Policies\Microsoft\Windows\CloudContent"
 if (-not (Test-Path $DefaultKey)) { New-Item -Path $DefaultKey -Force | Out-Null }
 Set-ItemProperty -Path $DefaultKey -Name "DisableSpotlightCollectionOnDesktop" -Value "1" -Type DWord -Force
 [GC]::Collect()
 [GC]::WaitForPendingFinalizers()
 reg unload HKU\DefaultUser | Out-Null
}
}
```

To audit the hardening status: [Download Script: Get-UpspotlightconsumerStatus.ps1](#)

```
Get-UpspotlightconsumerStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CloudContent" "DisableThirdPartySuggestions" "1"
Test-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CloudContent" "ConfigureWindowsSpotlight" "2"
Test-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\CloudContent" "DisableSpotlightCollectionOnDesktop" "1"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\CloudContent" "DisableWindowsConsumerFeatures" "1"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*: User profile privacy and shell lockdown controls \* \*\*ANSI Active Directory Hardening Guide\*\*: Client security baselines

# [REQ-END-128] User Profile: Windows Copilot Restrictions

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*  
`HKCU\Software\Policies\Microsoft\Windows\WindowsCopilot\TurnOffWindowsCopilot` = `1` (DWord)

## Rationale Disables the Windows Copilot assistant to prevent unauthorized data exfiltration or automated context parsing of local user activity.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `User Configuration \ Administrative Templates \ Windows Components \ Windows Copilot` 2. Configure the policy: \* \*\*Policy\*\*: `Turn off Windows Copilot` → Set to \*\*Enabled\*\*

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Upwindownscopilot.ps1](#)] ([../implementation\\_scripts/Configure-Upwindownscopilot.ps1](#))

```
Configure-Upwindownscopilot.ps1
Write-Host "Applying User Profile restriction: windows-copilot..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\WindowsCopilot" "TurnOffWindowsCopilot" "1" "DWord"

Apply to Default User profile for new sessions
$DefaultHivePath = "C:\Users\Default\NTUSER.DAT"
if (Test-Path $DefaultHivePath) {
 reg load HKU\DefaultUser $DefaultHivePath | Out-Null
 $DefaultKey = "Registry::HKU\DefaultUser\Software\Policies\Microsoft\Windows\WindowsCopilot"
 if (-not (Test-Path $DefaultKey)) { New-Item -Path $DefaultKey -Force | Out-Null }
 Set-ItemProperty -Path $DefaultKey -Name "TurnOffWindowsCopilot" -Value "1" -Type DWord -Force
 [GC]::Collect()
 [GC]::WaitForPendingFinalizers()
 reg unload HKU\DefaultUser | Out-Null
}
```

To audit the hardening status: [Download Script: Get-UpwindownscopilotStatus.ps1](#)

```
Get-UpwindscopilotStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKCU:" "Software\Policies\Microsoft\Windows\WindowsCopilot" "TurnOffWindowsCopilot" "1"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: User profile privacy and shell lockdown controls \* \*\*ANSSI  
Active Directory Hardening Guide\*\*\*: Client security baselines

# [REQ-END-129] User Profile: In-Place Sharing Restrictions

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*  
`HKCU\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer\NoInplaceSharing` = `1` (DWord)

## Rationale Disables direct in-place shell context sharing actions to prevent rapid local file redistribution bypasses.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `User Configuration \ Administrative Templates \ Windows Components \ Network Sharing` 2. Configure the policy: \* \*\*Policy\*\*: `Prevent users from sharing files within their profile.` → Set to **Enabled**

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Upinplacesharing.ps1](#)] ([../implementation\\_scripts/Configure-Upinplacesharing.ps1](#))

```
Configure-Upinplacesharing.ps1
Write-Host "Applying User Profile restriction: inplace-sharing..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKCU:" "Software\Microsoft\Windows\CurrentVersion\Policies\Explorer" "NoInplaceSharing" "1" "DWord"

Apply to Default User profile for new sessions
$DefaultHivePath = "C:\Users\Default\NTUSER.DAT"
if (Test-Path $DefaultHivePath) {
 reg load HKU\DefaultUser $DefaultHivePath | Out-Null
 $DefaultKey = "Registry::HKU\DefaultUser\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer"
 if (-not (Test-Path $DefaultKey)) { New-Item -Path $DefaultKey -Force | Out-Null }
 Set-ItemProperty -Path $DefaultKey -Name "NoInplaceSharing" -Value "1" -Type DWord -Force
 [GC]::Collect()
 [GC]::WaitForPendingFinalizers()
 reg unload HKU\DefaultUser | Out-Null
}
```

To audit the hardening status: [Download Script: Get-UpinplacesharingStatus.ps1](#)

```
Get-UpinplacesharingStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKCU:" "Software\Microsoft\Windows\CurrentVersion\Policies\Explorer" "NoInplaceSharing" "1"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: User profile privacy and shell lockdown controls \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Client security baselines

# [REQ-END-130] User Profile: Shell RunAs User Suppression

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

```
'HKLM\SOFTWARE\Classes\batfile\shell\runasuser\SuppressionPolicy' = '4096' (DWord) *
'HKLM\SOFTWARE\Classes\cmdfile\shell\runasuser\SuppressionPolicy' = '4096' (DWord) *
'HKLM\SOFTWARE\Classes\exefile\shell\runasuser\SuppressionPolicy' = '4096' (DWord) *
'HKLM\SOFTWARE\Classes\mscfile\shell\runasuser\SuppressionPolicy' = '4096' (DWord)
```

## Rationale Suppresses the 'Run as different user' shell context menu for scripts and executables to prevent users from bypassing local validation structures.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy custom registry preferences under: 'Computer Configuration \ Preferences \ Windows Settings \ Registry' 2. Create four new \*\*Registry Items\*\* for bat, cmd, exe, and msc file RunAs suppression: \* \*\*Registry Item 1\*\*: \* \*\*Key Path\*\*: 'SOFTWARE\Classes\batfile\shell\runasuser' \* \*\*Value name\*\*: 'SuppressionPolicy' \* \*\*Value type\*\*: REG\_DWORD \* \*\*Value data\*\*: '4096' \* \*\*Registry Item 2\*\*: \* \*\*Key Path\*\*: 'SOFTWARE\Classes\cmdfile\shell\runasuser' \* \*\*Value name\*\*: 'SuppressionPolicy' \* \*\*Value type\*\*: REG\_DWORD \* \*\*Value data\*\*: '4096' \* \*\*Registry Item 3\*\*: \* \*\*Key Path\*\*: 'SOFTWARE\Classes\exefile\shell\runasuser' \* \*\*Value name\*\*: 'SuppressionPolicy' \* \*\*Value type\*\*: REG\_DWORD \* \*\*Value data\*\*: '4096' \* \*\*Registry Item 4\*\*: \* \*\*Key Path\*\*: 'SOFTWARE\Classes\mscfile\shell\runasuser' \* \*\*Value name\*\*: 'SuppressionPolicy' \* \*\*Value type\*\*: REG\_DWORD \* \*\*Value data\*\*: '4096'

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Uprunassuppression.ps1](#)] ([../implementation\\_scripts/Configure-Uprunassuppression.ps1](#))

```
Configure-Uprunassuppression.ps1
Write-Host "Applying User Profile restriction: runas-suppression..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Classes\batfile\shell\runasuser" "SuppressionPolicy" "4096" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Classes\cmdfile\shell\runasuser" "SuppressionPolicy" "4096" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Classes\exefile\shell\runasuser" "SuppressionPolicy" "4096" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Classes\mscfile\shell\runasuser" "SuppressionPolicy" "4096" "DWord"
```

To audit the hardening status: [Download Script: Get-UprunassuppressionStatus.ps1](#)



```
Get-UprunassuppressionStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Classes\batfile\shell\runasuser" "SuppressionPolicy" "4096"
Test-RegValue "HKLM:" "SOFTWARE\Classes\cmdfile\shell\runasuser" "SuppressionPolicy" "4096"
Test-RegValue "HKLM:" "SOFTWARE\Classes\exefile\shell\runasuser" "SuppressionPolicy" "4096"
Test-RegValue "HKLM:" "SOFTWARE\Classes\mscfile\shell\runasuser" "SuppressionPolicy" "4096"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*: User profile privacy and shell lockdown controls \* \*\*ANSI Active Directory Hardening Guide\*\*: Client security baselines

# [REQ-END-131] User Profile: Personalization and Privacy Restrictions

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Personalization\NoLockScreenCamera` = `1` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Personalization\NoLockScreenSlideshow` = `1` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\AppPrivacy\LetAppsActivateWithVoiceAboveLock` = `2` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\InputPersonalization\AllowInputPersonalization` = `0` (DWord)

## Rationale Disables lock screen cameras, lock screen slideshows, voice activation above lock, and input personalization (speech/typing logs) to preserve local workstation privacy.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Control Panel \ Personalization` 2. Configure the policies: \* \*\*Policy\*\*: `Prevent enabling lock screen camera` → Set to **Enabled** \* \*\*Policy\*\*: `Prevent enabling lock screen slide show` → Set to **Enabled** 3. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ App Privacy` 4. Configure the policy: \* \*\*Policy\*\*: `Let Windows apps activate with voice while the system is locked` → Set to **Enabled** and select **Force Deny** (value 2) 5. Navigate to: `Computer Configuration \ Administrative Templates \ Control Panel \ Regional and Language Options` 6. Configure the policy: \* \*\*Policy\*\*: `Allow users to enable online speech recognition services` → Set to **Disabled** (value 0)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Uppersonalizationprivacy.ps1](#)] (./implementation\_scripts/Configure-Uppersonalizationprivacy.ps1)

```
Configure-Uppersonalizationprivacy.ps1
Write-Host "Applying User Profile restriction: personalization-privacy..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Personalization" "NoLockScreenCamera" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Personalization" "NoLockScreenSlideshow" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\AppPrivacy" "LetAppsActivateWithVoiceAboveLock" "2" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\InputPersonalization" "AllowInputPersonalization" "0" "DWord"
```

To audit the hardening status: [Download Script: Get-UppersonalizationprivacyStatus.ps1](#)

```
Get-UppersonalizationprivacyStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Personalization" "NoLockScreenCamera" "1"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Personalization" "NoLockScreenSlideshow" "1"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\AppPrivacy" "LetAppsActivateWithVoiceAboveLock" "2"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\InputPersonalization" "AllowInputPersonalization" "0"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: User profile privacy and shell lockdown controls \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Client security baselines

# [REQ-END-132] User Profile: Group Policy Processing Behaviors

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}\NoBackgroundPolicy` = `0` (DWord) \* `HKLM\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}\NoGPListChanges` = `0` (DWord)

## Rationale Enforces that Group Policy background refreshes and policy changes are processed predictably on workstations without user-directed bypasses.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ System \ Group Policy` 2. Configure the policy: \* \*\*Policy\*\*: `Configure Registry policy processing` → Set to **Enabled** \* Check **Process even if the Group Policy objects have not changed** (enforces NoGPListChanges = 0) \* Uncheck **Do not apply during periodic background processing** (enforces NoBackgroundPolicy = 0)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Upgpprocessing.ps1](#)] ([../implementation\\_scripts/Configure-Upgpprocessing.ps1](#))

```
Configure-Upgpprocessing.ps1
Write-Host "Applying User Profile restriction: gp-processing..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" "NoBackgroundPolicy"
"0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" "NoGPListChanges"
"0" "DWord"
```

To audit the hardening status: [Download Script: Get-UpgpprocessingStatus.ps1](#)

```
Get-UpgpprocessingStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" "NoBackgroundPolicy"
"0"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" "NoGPOListChanges"
"0"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*: User profile privacy and shell lockdown controls \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines

# [REQ-END-133] User Profile: Telemetry and Inventory Collection Restrictions

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\AppCompat\DisableInventory` = `1` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\DataCollection\AllowTelemetry` = `1` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\DataCollection\LimitEnhancedDiagnosticDataWindowsAnalytics` = `1` (DWord)

## Rationale Disables application compatibility inventory scans and restricts telemetry uploads to the minimum baseline levels.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Application Compatibility` 2. Configure the policy: \* \*\*Policy\*\*: `Turn off Inventory Collector` → Set to **Enabled** 3. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Data Collection and Preview Builds` 4. Configure the policies: \* \*\*Policy\*\*: `Allow Telemetry` → Set to **Enabled** and select **1 - Basic** (or **1 - Required** depending on OS version) \* \*\*Policy\*\*: `Limit Enhanced diagnostic data to the minimum required by Windows Analytics` → Set to **Enabled** and select **1 - Enable Windows Analytics settings**

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Uptelemetryinventory.ps1](#)] ([../implementation\\_scripts/Configure-Uptelemetryinventory.ps1](#))

```
Configure-Uptelemetryinventory.ps1
Write-Host "Applying User Profile restriction: telemetry-inventory..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\AppCompat" "DisableInventory" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\DataCollection" "AllowTelemetry" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\DataCollection" "LimitEnhancedDiagnosticDataWindowsAnalytics" "1" "DWord"
```

To audit the hardening status: [Download Script: Get-UptelemetryinventoryStatus.ps1](#)

```
Get-UptelemetryinventoryStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\AppCompat" "DisableInventory" "1"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\DataCollection" "AllowTelemetry" "1"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\DataCollection" "LimitEnhancedDiagnosticDataWindowsAnalytics" "1"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: User profile privacy and shell lockdown controls \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Client security baselines

# [REQ-END-134] User Profile: Explorer Security and Memory Protections

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Explorer\NoDataExecutionPrevention` = `0` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Explorer\NoHeapTerminationOnCorruption` = `0` (DWord) \*

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer\PreXPSP2ShellProtocolBehavior` = `0` (DWord)

## Rationale Enforces Data Execution Prevention (DEP) and Heap Termination on Corruption inside Windows Explorer and disables pre-XP SP2 shell protocol behaviors.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ File Explorer` 2. Configure the policies: \* \*\*Policy\*\*: `Turn off Data Execution Prevention for Explorer` → Set to **Disabled** (keeps DEP active) \* \*\*Policy\*\*: `Turn off heap termination on corruption` → Set to **Disabled** (keeps Heap Termination active) 3. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ File Explorer` (or User configuration equivalent) 4. Configure the policy: \* \*\*Policy\*\*: `Shell Protocol Protected Mode` → Set to **Enabled** (enforces PreXPSP2ShellProtocolBehavior = 0)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Upexplorersecurity.ps1](#)] ([../implementation\\_scripts/Configure-Upexplorersecurity.ps1](#))

```
Configure-Upexplorersecurity.ps1
Write-Host "Applying User Profile restriction: explorer-security..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Explorer" "NoDataExecutionPrevention" "0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Explorer" "NoHeapTerminationOnCorruption" "0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" "PreXPSP2ShellProtocolBehavior" "0" "DWord"
```

To audit the hardening status: [Download Script: Get-UpexplorersecurityStatus.ps1](#)



```
Get-UpexplorersecurityStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Explorer" "NoDataExecutionPrevention" "0"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Explorer" "NoHeapTerminationOnCorruption" "0"
Test-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" "PreXPSP2ShellProtocolBehavior" "0"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: User profile privacy and shell lockdown controls \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Client security baselines

# [REQ-END-135] User Profile: Internet Explorer Options and Feeds Restrictions

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Policies\Microsoft\Internet Explorer\Main\NotifyDisableIEOptions` = `0` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds\DisableEnclosureDownload` = `1` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds\AllowBasicAuthInClear` = `0` (DWord)

## Rationale Blocks basic cleartext authentication, disables feed enclosure downloads, and disables override options inside standard Internet Options panels.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Internet Explorer` 2. Configure the policy: \* \*\*Policy\*\*: `Disable Internet Explorer 11 as a standalone browser` → Set to **Enabled** and select **Always** 3. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ RSS Feeds` 4. Configure the policies: \* \*\*Policy\*\*: `Prevent downloading of enclosures` → Set to **Enabled** \* \*\*Policy\*\*: `Turn on Basic feed authentication over HTTP` → Set to **Disabled**

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Upiesecurity.ps1](#)] ([../implementation\\_scripts/Configure-Upiesecurity.ps1](#))

```
Configure-Upiesecurity.ps1
Write-Host "Applying User Profile restriction: ie-security..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Internet Explorer\Main" "NotifyDisableIEOptions" "0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" "DisableEnclosureDownload" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" "AllowBasicAuthInClear" "0" "DWord"
```

To audit the hardening status: [Download Script: Get-UpiesecurityStatus.ps1](#)

```
Get-UpiesecurityStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Internet Explorer\Main" "NotifyDisableIEOptions" "0"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" "DisableEnclosureDownload" "1"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" "AllowBasicAuthInClear" "0"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: User profile privacy and shell lockdown controls \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Client security baselines

# [REQ-END-136] User Profile: Interactive Logon Warning Banners

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\DisableAutomaticRestartSignOn` = `1` (DWord) \*

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\LegalNoticeText` = `You are accessing a U.S. Government (USG) Information System (IS) that is provided for USG-authorized use only. By using this IS, you consent to routine monitoring.` (String) \*

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\LegalNoticeCaption` = `US Department of Defense Warning Statement` (String)

## Rationale Enforces legally binding interactive logon warning text and caption titles, and disables automatic restart sign-ons on client endpoints.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Policies \ Windows Settings \ Security Settings \ Local Policies \ Security Options` 2. Configure the policies: \* \*\*Policy\*\*: `Interactive logon: Message text for users attempting to log on` → Set to: `You are accessing a U.S. Government (USG) Information System (IS) that is provided for USG-authorized use only. By using this IS, you consent to routine monitoring.` \* \*\*Policy\*\*: `Interactive logon: Message title for users attempting to log on` → Set to: `US Department of Defense Warning Statement` 3. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Windows Logon Options` 4. Configure the policy: \* \*\*Policy\*\*: `Sign-in and lock last interactive user automatically after a restart` → Set to `Disabled`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Uplogonbanners.ps1](#)] ([../implementation\\_scripts/Configure-Uplogonbanners.ps1](#))

```
Configure-Uplogonbanners.ps1
Write-Host "Applying User Profile restriction: logon-banners..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "DisableAutomaticRestartSignOn" "1" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "LegalNoticeText" "You are accessing a U.S. Governm
ent (USG) Information System (IS) that is provided for USG-authorized use only. By using this IS, you consent to routine monitorin
g." "String"
Set-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "LegalNoticeCaption" "US Department of Defense Warn
ing Statement" "String"
```

To audit the hardening status: [Download Script: Get-UplogonbannersStatus.ps1](#)

```
Get-UplogonbannersStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "DisableAutomaticRestartSignOn" "1"
Test-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "LegalNoticeText" "You are accessing a U.S. Government (USG) Information System (IS) that is provided for USG-authorized use only. By using this IS, you consent to routine monitoring."
Test-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "LegalNoticeCaption" "US Department of Defense Warning Statement"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: User profile privacy and shell lockdown controls \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Client security baselines

# [REQ-END-137] User Profile: Interactive Logon Inactivity Timeout

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*  
 'HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\InactivityTimeoutSecs' = '900' (DWord)

## Rationale Enforces a maximum inactivity limit of 15 minutes (900 seconds) before locking idle user sessions.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: 'Computer Configuration \ Policies \ Windows Settings \ Security Settings \ Local Policies \ Security Options' 2. Configure the policy: \* \*\*Policy\*\*: 'Interactive logon: Machine inactivity limit' → Set to \*\*900 seconds\*\* (15 minutes)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Upinactivitytimeout.ps1](#)] ([../implementation\\_scripts/Configure-Upinactivitytimeout.ps1](#))

```
Configure-Upinactivitytimeout.ps1
Write-Host "Applying User Profile restriction: inactivity-timeout..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "InactivityTimeoutSecs" "900" "DWord"
```

To audit the hardening status: [Download Script: Get-UpinactivitytimeoutStatus.ps1](#)

```
Get-UpinactivitytimeoutStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" "InactivityTimeoutSecs" "900"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: User profile privacy and shell lockdown controls \* \*\*ANSSI  
Active Directory Hardening Guide\*\*\*: Client security baselines

# [REQ-END-138] User Profile: Windows Installer Hardening

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer\EnableUserControl` = `0` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer\AlwaysInstallElevated` = `0` (DWord) \*

`HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer\SafeForScripting` = `0` (DWord) \*

`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Device Installer\DisableCoInstallers` = `1` (DWord)

## Rationale Enforces Windows Installer lockdowns by disabling AlwaysInstallElevated and blocking driver co-installer executions on endpoints.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Windows Installer` 2. Configure the policies: \* \*\*Policy\*\*: `Allow user control over installs` → Set to **Disabled** \* \*\*Policy\*\*: `Always install with elevated privileges` → Set to **Disabled** \* \*\*Policy\*\*: `Prevent Internet Explorer security prompt for Windows Installer scripts` → Set to **Disabled** 3. Deploy custom registry preference for blocking driver co-installers: \* Navigate to: `Computer Configuration \ Preferences \ Windows Settings \ Registry` \* Create a new **Registry Item**: \* \*\*Action\*\*: Update \* \*\*Hive\*\*: HKEY\_LOCAL\_MACHINE \* \*\*Key Path\*\*: `SOFTWARE\Microsoft\Windows\CurrentVersion\Device Installer` \* \*\*Value name\*\*: `DisableCoInstallers` \* \*\*Value type\*\*: REG\_DWORD \* \*\*Value data\*\*: `1`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Upinstallerhardening.ps1](#)] ([./implementation\\_scripts/Configure-Upinstallerhardening.ps1](#))

```
Configure-Upinstallerhardening.ps1
Write-Host "Applying User Profile restriction: installer-hardening..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Installer" "EnableUserControl" "0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Installer" "AlwaysInstallElevated" "0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Installer" "SafeForScripting" "0" "DWord"
Set-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Device Installer" "DisableCoInstallers" "1" "DWord"
```

To audit the hardening status: [Download Script: Get-UpinstallerhardeningStatus.ps1](#)



```
Get-UpinstallerhardeningStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Installer" "EnableUserControl" "0"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Installer" "AlwaysInstallElevated" "0"
Test-RegValue "HKLM:" "SOFTWARE\Policies\Microsoft\Windows\Installer" "SafeForScripting" "0"
Test-RegValue "HKLM:" "SOFTWARE\Microsoft\Windows\CurrentVersion\Device Installer" "DisableCoInstallers" "1"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*: User profile privacy and shell lockdown controls \* \*\*ANSI Active Directory Hardening Guide\*\*: Client security baselines

# [REQ-END-139] User Profile: Secondary Logon Service Lockdown

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Locations\*\*: \*  
'HKLM\SYSTEM\CurrentControlSet\Services\seclogon\Start' = '4' (DWord)

## Rationale Disables the Secondary Logon service (seclogon) to prevent users from executing processes under alternate credentials via runas without proper validation.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts client customization features. Verify compatibility in staging environments.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure the registry values directly using Group Policy Preferences (Registry Items).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-Upseclogonservice.ps1](#)] ([../implementation\\_scripts/Configure-Upseclogonservice.ps1](#))

```
Configure-Upseclogonservice.ps1
Write-Host "Applying User Profile restriction: seclogon-service..." -ForegroundColor Cyan

function Set-RegValue {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$value,
 [string]$type
)
 if ($PSCmdlet.ShouldProcess("$hive\$keyPath", "Set registry value $name to $value")) {
 $fullPath = "$hive\$keyPath"
 $parent = Split-Path -Path $fullPath
 if (-not (Test-Path $parent)) { New-Item -Path $parent -Force | Out-Null }
 if (-not (Test-Path $fullPath)) { New-Item -Path $fullPath -Force | Out-Null }
 Set-ItemProperty -Path $fullPath -Name $name -Value $value -Type $type -Force
 }
}

Set-RegValue "HKLM:" "SYSTEM\CurrentControlSet\Services\seclogon" "Start" "4" "DWord"
```

To audit the hardening status: [Download Script: Get-UpseclogonserviceStatus.ps1](#)

```
Get-UpseclogonserviceStatus.ps1
$script:Vulnerable = $false

function Test-RegValue {
 param (
 [string]$hive,
 [string]$keyPath,
 [string]$name,
 [string]$expected
)
 $fullPath = "$hive\$keyPath"
 $val = Get-ItemProperty -Path $fullPath -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 if ($actual -ne $expected) {
 $script:Vulnerable = $true
 }
}

Test-RegValue "HKLM:" "SYSTEM\CurrentControlSet\Services\seclogon" "Start" "4"

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Benchmark\*\*\*: User profile privacy and shell lockdown controls \* \*\*ANSSI  
Active Directory Hardening Guide\*\*\*: Client security baselines

# [REQ-END-151] User Profile: Structured Exception Handling Overwrite Protection (SEHOP) for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\kernel\DisableExceptionChainValidation` = `0` (DWord)

## Rationale Structured Exception Handling Overwrite Protection (SEHOP) detects and thwarts exploits targeting structured exception handler corruption, a common stack exploitation technique.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\kernel\DisableExceptionChainValidation` = `0` (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditSehop.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditSehop.ps1](#))

```
Configure-EndAuditSehop.ps1
Write-Host "Enforcing System Mitigation control: sehops..." -ForegroundColor Cyan

Set Registry value: DisableExceptionChainValidation
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\kernel")) { New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\kernel" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\kernel" -Name "DisableExceptionChainValidation" -Value 0 -Type DWord -Force
Write-Host " Enforced DisableExceptionChainValidation = 0" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-EndAuditSehopStatus.ps1](#)

```
Get-EndAuditSehopStatus.ps1
$script:Vulnerable = $false

Audit Registry value: DisableExceptionChainValidation
$RegVal = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\kernel" -Name "DisableExceptionChainValidation" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.DisableExceptionChainValidation -ne 0) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-END-152] User Profile: Directory Protection Mode for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: 'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\ProtectionMode' = '1' (DWord)

## Rationale Setting ProtectionMode to 1 restricts access to crucial system folders like System32, enforcing strict ACLs and protecting against write tampering.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

#### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: 'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\ProtectionMode' = '1' (DWord)

#### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditProtectionmode.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditProtectionmode.ps1](#))

```
Configure-EndAuditProtectionmode.ps1
Write-Host "Enforcing System Mitigation control: protection-mode..." -ForegroundColor Cyan

Set Registry value: ProtectionMode
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager")) { New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager" -Name "ProtectionMode" -Value 1 -Type DWord -Force
Write-Host " Enforced ProtectionMode = 1" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-EndAuditProtectionmodeStatus.ps1](#)

```
Get-EndAuditProtectionmodeStatus.ps1
$script:Vulnerable = $false

Audit Registry value: ProtectionMode
$RegVal = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager" -Name "ProtectionMode" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.ProtectionMode -ne 1) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-END-153] User Profile: Address Space Layout Randomization (ASLR) Image Relocation for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\MoveImages` = `4294967295` (DWord)

## Rationale Enforcing Address Space Layout Randomization (ASLR) image relocation (MoveImages) forces all dynamic modules to relocate, neutralizing hardcoded ROP payload chains.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\MoveImages` = `4294967295` (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditAslrrelocation.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditAslrrelocation.ps1](#))

```
Configure-EndAuditAslrrelocation.ps1
Write-Host "Enforcing System Mitigation control: aslr-relocation..." -ForegroundColor Cyan

Set Registry value: MoveImages
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management")) { New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Name "MoveImages" -Value 4294967295 -Type DWord -Force
Write-Host " Enforced MoveImages = 4294967295" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-EndAuditAslrrelocationStatus.ps1](#)

```
Get-EndAuditAslrrelocationStatus.ps1
$script:Vulnerable = $false

Audit Registry value: MoveImages
$RegVal = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Name "MoveImages" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.MoveImages -ne 4294967295) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-END-154] User Profile: Speculative Execution Mitigations (Spectre/Meltdown) for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry:

'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\FeatureSettingsOverride' = '72' (DWord) \* Registry:

'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\FeatureSettingsOverrideMask' = '3' (DWord)

## Rationale Enforces hardware-backed speculative execution mitigations (Spectre, Meltdown, MDS) to prevent side-channel leaks of sensitive kernel-space data.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: 'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\FeatureSettingsOverride' = '72' (DWord) \* Registry: 'HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\FeatureSettingsOverrideMask' = '3' (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditSpeculativemitigations.ps1](#)] (./implementation\_scripts/Configure-EndAuditSpeculativemitigations.ps1)

```
Configure-EndAuditSpeculativemitigations.ps1
Write-Host "Enforcing System Mitigation control: speculative-mitigations..." -ForegroundColor Cyan

Set Registry value: FeatureSettingsOverride
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management")) { New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Name "FeatureSettingsOverride" -Value 72 -Type DWord -Force
Write-Host " Enforced FeatureSettingsOverride = 72" -ForegroundColor Green

Set Registry value: FeatureSettingsOverrideMask
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management")) { New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Name "FeatureSettingsOverrideMask" -Value 3 -Type DWord -Force
Write-Host " Enforced FeatureSettingsOverrideMask = 3" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-EndAuditSpeculativemitigationsStatus.ps1](#)

```
Get-EndAuditSpeculativemitigationsStatus.ps1
$script:Vulnerable = $false

Audit Registry value: FeatureSettingsOverride
$RegVal = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Name "FeatureSettingsOverride" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.FeatureSettingsOverride -ne 72) {
 $script:Vulnerable = $true
}

Audit Registry value: FeatureSettingsOverrideMask
$RegVal = Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management" -Name "FeatureSettingsOverrideMask" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.FeatureSettingsOverrideMask -ne 3) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*ANSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions



# [REQ-END-155] User Profile: Authenticode Signature Certificate Padding Check for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry:

'HKLM\Software\Microsoft\Cryptography\Wintrust\Config\EnableCertPaddingCheck' = '1' (DWord) \* Registry:

'HKLM\Software\Wow6432Node\Microsoft\Cryptography\Wintrust\Config\EnableCertPaddingCheck' = '1' (DWord)

## Rationale Blocks padding execution attacks on Authenticode signed binaries by validating that there is no extra unverified payload appended to the certificate signature block.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: 'HKLM\Software\Microsoft\Cryptography\Wintrust\Config\EnableCertPaddingCheck' = '1' (DWord) \* Registry: 'HKLM\Software\Wow6432Node\Microsoft\Cryptography\Wintrust\Config\EnableCertPaddingCheck' = '1' (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditCertpadding.ps1](#)] (./implementation\_scripts/Configure-EndAuditCertpadding.ps1)

```
Configure-EndAuditCertpadding.ps1
Write-Host "Enforcing System Mitigation control: cert-padding..." -ForegroundColor Cyan

Set Registry value: EnableCertPaddingCheck
if (-not (Test-Path "HKLM:\Software\Microsoft\Cryptography\Wintrust\Config")) { New-Item -Path "HKLM:\Software\Microsoft\Cryptography\Wintrust\Config" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\Software\Microsoft\Cryptography\Wintrust\Config" -Name "EnableCertPaddingCheck" -Value 1 -Type DWord -Force
Write-Host " Enforced EnableCertPaddingCheck = 1" -ForegroundColor Green

Set Registry value: EnableCertPaddingCheck
if (-not (Test-Path "HKLM:\Software\Wow6432Node\Microsoft\Cryptography\Wintrust\Config")) { New-Item -Path "HKLM:\Software\Wow6432Node\Microsoft\Cryptography\Wintrust\Config" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\Software\Wow6432Node\Microsoft\Cryptography\Wintrust\Config" -Name "EnableCertPaddingCheck" -Value 1 -Type DWord -Force
Write-Host " Enforced EnableCertPaddingCheck = 1" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-EndAuditCertpaddingStatus.ps1](#)

```
Get-EndAuditCertpaddingStatus.ps1
$script:Vulnerable = $false

Audit Registry value: EnableCertPaddingCheck
$RegVal = Get-ItemProperty -Path "HKLM:\Software\Microsoft\Cryptography\Wintrust\Config" -Name "EnableCertPaddingCheck" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.EnableCertPaddingCheck -ne 1) {
 $script:Vulnerable = $true
}

Audit Registry value: EnableCertPaddingCheck
$RegVal = Get-ItemProperty -Path "HKLM:\Software\Wow6432Node\Microsoft\Cryptography\Wintrust\Config" -Name "EnableCertPaddingCheck" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.EnableCertPaddingCheck -ne 1) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*ANSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-END-156] User Profile: Command Processor Batch File Locking for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: 'HKLM\SOFTWARE\Microsoft\Command Processor\LockBatchFilesWhenInUse' = '1' (DWord)

## Rationale Locking batch scripts when executing prevents attackers or concurrent processes from rewriting script lines on-the-fly, neutralizing dynamic code modification hijacks.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

#### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: 'HKLM\SOFTWARE\Microsoft\Command Processor\LockBatchFilesWhenInUse' = '1' (DWord)

#### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditLockbatchfiles.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditLockbatchfiles.ps1](#))

```
Configure-EndAuditLockbatchfiles.ps1
Write-Host "Enforcing System Mitigation control: lock-batch-files..." -ForegroundColor Cyan

Set Registry value: LockBatchFilesWhenInUse
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\Command Processor")) { New-Item -Path "HKLM:\SOFTWARE\Microsoft\Command Processor" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Command Processor" -Name "LockBatchFilesWhenInUse" -Value 1 -Type DWord -Force
Write-Host " Enforced LockBatchFilesWhenInUse = 1" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-EndAuditLockbatchfilesStatus.ps1](#)

```
Get-EndAuditLockbatchfilesStatus.ps1
$script:Vulnerable = $false

Audit Registry value: LockBatchFilesWhenInUse
$RegVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Command Processor" -Name "LockBatchFilesWhenInUse" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.LockBatchFilesWhenInUse -ne 1) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-END-157] User Profile: Time-Travel Debugging (TTD) Recording Policy for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: 'HKLM\SOFTWARE\Microsoft\TTD\RecordingPolicy' = '2' (DWord)

## Rationale Disables and locks down user-mode Time-Travel Debugging (TTD) traces, preventing local adversaries from capturing memory dumps and private cryptographic structures from administrative processes.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: 'HKLM\SOFTWARE\Microsoft\TTD\RecordingPolicy' = '2' (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditTtdrecording.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditTtdrecording.ps1](#))

```
Configure-EndAuditTtdrecording.ps1
Write-Host "Enforcing System Mitigation control: ttd-recording..." -ForegroundColor Cyan

Set Registry value: RecordingPolicy
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\TTD")) { New-Item -Path "HKLM:\SOFTWARE\Microsoft\TTD" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\TTD" -Name "RecordingPolicy" -Value 2 -Type DWord -Force
Write-Host " Enforced RecordingPolicy = 2" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-EndAuditTtdrecordingStatus.ps1](#)

```
Get-EndAuditTtdrecordingStatus.ps1
$script:Vulnerable = $false

Audit Registry value: RecordingPolicy
$RegVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\TTD" -Name "RecordingPolicy" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.RecordingPolicy -ne 2) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-END-158] User Profile: Trusted Root Store Protected Roots Certificate Restriction for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: 'HKLM\SOFTWARE\Policies\Microsoft\SystemCertificates\Root\ProtectedRoots\Flags' = '1' (DWord)

## Rationale Restricts users from installing root certificates into the trusted root store, preventing root CA hijack actions and man-in-the-middle proxy injection attacks.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: 'HKLM\SOFTWARE\Policies\Microsoft\SystemCertificates\Root\ProtectedRoots\Flags' = '1' (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditProtectedroots.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditProtectedroots.ps1](#))

```
Configure-EndAuditProtectedroots.ps1
Write-Host "Enforcing System Mitigation control: protected-roots..." -ForegroundColor Cyan

Set Registry value: Flags
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\SystemCertificates\Root\ProtectedRoots")) { New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\SystemCertificates\Root\ProtectedRoots" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\SystemCertificates\Root\ProtectedRoots" -Name "Flags" -Value 1 -Type DWord -Force
Write-Host " Enforced Flags = 1" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-EndAuditProtectedrootsStatus.ps1](#)

```
Get-EndAuditProtectedrootsStatus.ps1
$script:Vulnerable = $false

Audit Registry value: Flags
$RegVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\SystemCertificates\Root\ProtectedRoots" -Name "Flags" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.Flags -ne 1) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-END-159] User Profile: Disabling Injection of AppInit DLLs for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Windows\LoadAppInit\_DLLs` = `0` (DWord)

## Rationale AppInit DLL injection is a legacy mechanism that loads arbitrary user DLLs into every process that links user32.dll. Enforcing a complete ban (value 0) prevents unauthorized injection hooks.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Deploy the following Registry settings using Group Policy Preferences (Registry Extension): \* Registry: `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Windows\LoadAppInit\_DLLs` = `0` (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditAppInitDlls.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditAppInitDlls.ps1](#))

```
Configure-EndAuditAppInitDlls.ps1
Write-Host "Enforcing System Mitigation control: appinit-dlls..." -ForegroundColor Cyan

Set Registry value: LoadAppInit_DLLs
if (-not (Test-Path "HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Windows")) { New-Item -Path "HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Windows" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Windows" -Name "LoadAppInit_DLLs" -Value 0 -Type DWord -Force
Write-Host " Enforced LoadAppInit_DLLs = 0" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-EndAuditAppInitDllsStatus.ps1](#)

```
Get-EndAuditAppInitDllsStatus.ps1
$script:Vulnerable = $false

Audit Registry value: LoadAppInit_DLLs
$RegVal = Get-ItemProperty -Path "HKLM:\Software\Microsoft\Windows NT\CurrentVersion\Windows" -Name "LoadAppInit_DLLs" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.LoadAppInit_DLLs -ne 0) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-END-160] User Profile: Preservation of Attachment Zone Information for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Attachments\SaveZoneInformation` = `2` (DWord)

## Rationale Ensures the Attachment Manager preserves the Zone.Identifier alternate data stream (ADS) marking files downloaded from untrusted web zones, enforcing SmartScreen check prompts.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `User Configuration \ Administrative Templates \ Windows Components \ Attachment Manager` 2. Configure the policy: \* \*\*Policy\*\*: `Do not preserve zone information in file attachments` → Set to **Disabled** (which configures `SaveZoneInformation` = `2` to preserve zone information)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditAttachmentzone.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditAttachmentzone.ps1](#))

```
Configure-EndAuditAttachmentzone.ps1
Write-Host "Enforcing System Mitigation control: attachment-zone..." -ForegroundColor Cyan

Set Registry value: SaveZoneInformation
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Attachments")) { New-Item -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Attachments" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Attachments" -Name "SaveZoneInformation" -Value 2 -Type DWord -Force
Write-Host " Enforced SaveZoneInformation = 2" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-EndAuditAttachmentzoneStatus.ps1](#)

```
Get-EndAuditAttachmentzoneStatus.ps1
$script:Vulnerable = $false

Audit Registry value: SaveZoneInformation
$RegVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Attachments" -Name "SaveZoneInformation" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.SaveZoneInformation -ne 2) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

# [REQ-END-161] User Profile: Disable Windows Game DVR for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: `HKLM\SOFTWARE\Policies\Microsoft\Windows\GameDVR\AllowGameDVR` = `0` (DWord)

## Rationale Disabling Game DVR blocks background media recording agents, conserving local computing cycles and preventing administrative session leakage via broadcast APIs.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Windows Game Recording and Broadcasting` 2. Configure the policy: \* \*\*Policy\*\*: `Enables or disables Windows Game Recording and Broadcasting` → Set to **Disabled** (which configures `AllowGameDVR` = `0`)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditGamedvr.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditGamedvr.ps1](#))

```
Configure-EndAuditGamedvr.ps1
Write-Host "Enforcing System Mitigation control: game-dvr..." -ForegroundColor Cyan

Set Registry value: AllowGameDVR
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\GameDVR")) { New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\GameDVR" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\GameDVR" -Name "AllowGameDVR" -Value 0 -Type DWord -Force
Write-Host " Enforced AllowGameDVR = 0" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-EndAuditGamedvrStatus.ps1](#)

```
Get-EndAuditGamedvrStatus.ps1
$script:Vulnerable = $false

Audit Registry value: AllowGameDVR
$RegVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\GameDVR" -Name "AllowGameDVR" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.AllowGameDVR -ne 0) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions



# [REQ-END-162] User Profile: Restrict Windows Ink Workspace on Lock Screen for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*Registry Settings\*\*: \* Registry: `HKLM\SOFTWARE\Policies\Microsoft\WindowsInkWorkspace\AllowWindowsInkWorkspace` = `1` (DWord)

## Rationale Disabling access to the Windows Ink Workspace on the lock screen prevents unauthorized physical users from invoking drawing tools, scripts, or apps without authenticating.

## Legacy Impact & Compatibility \* \*\*Operational Impact\*\*: Restricts legacy application hooks or diagnostic modes. Ensure testing in a representative staging environment prior to wide deployment.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Navigate to: `Computer Configuration \ Administrative Templates \ Windows Components \ Windows Ink Workspace` 2. Configure the policy: \* \*\*Policy\*\*: `Allow Windows Ink Workspace` → Set to **Enabled** \* \*\*Action\*\*: Choose **On**, but disallow access above lock\*\* (which configures `AllowWindowsInkWorkspace` = `1`)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditInkworkspace.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditInkworkspace.ps1](#))

```
Configure-EndAuditInkworkspace.ps1
Write-Host "Enforcing System Mitigation control: ink-workspace..." -ForegroundColor Cyan

Set Registry value: AllowWindowsInkWorkspace
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsInkWorkspace")) { New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsInkWorkspace" -Force | Out-Null }
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsInkWorkspace" -Name "AllowWindowsInkWorkspace" -Value 1 -Type DWord -Force
Write-Host " Enforced AllowWindowsInkWorkspace = 1" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-EndAuditInkworkspaceStatus.ps1](#)

```
Get-EndAuditInkworkspaceStatus.ps1
$script:Vulnerable = $false

Audit Registry value: AllowWindowsInkWorkspace
$RegVal = Get-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsInkWorkspace" -Name "AllowWindowsInkWorkspace" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.AllowWindowsInkWorkspace -ne 1) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client security baselines \* \*\*CIS Windows 10/11 Client Benchmark\*\*: Section 18.9 (Administrative Templates: System \ Mitigations) and Registry restrictions

## # [REQ-END-020] Configure Exploit Protection Profile

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: \* `Computer Configuration\Administrative Templates\Windows Components\Windows Defender Exploit Guard\Exploit Protection` → \*\*Use a common set of exploit protection settings\*\* \* `Computer Configuration\Administrative Templates\MS Security Guide` → \*\*Enable Certificate Padding\*\* \* `Computer Configuration\Administrative Templates\MS Security Guide` → \*\*Enable Structured Exception Handling Overwrite Protection (SEHOP)\*\* \* \*\*Registry Locations\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\Windows Defender ExploitGuard\Exploit Protection` \* `ExploitProtectionSettings` = `C:\ProgramData\ExploitProtection\ExploitProtectionSettings.xml` (REG\_SZ) \* `HKLM\SOFTWARE\Microsoft\Cryptography\Wintrust\Config` \* `EnableCertPaddingCheck` = `1` (REG\_DWORD) \* `HKLM\SOFTWARE\Wow6432Node\Microsoft\Cryptography\Wintrust\Config` \* `EnableCertPaddingCheck` = `1` (REG\_DWORD) \* `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\kernel` \* `DisableExceptionChainValidation` = `0` (REG\_DWORD)

## Rationale Exploit Protection (the successor to the Enhanced Mitigation Experience Toolkit, or EMET) provides a set of advanced memory and vulnerability mitigations. These mitigations protect both the operating system and applications from memory corruption, buffer overflows, execution redirection, and process hijack attempts.

By enforcing system-wide mitigations:

1. **Data Execution Prevention (DEP)**: Enforces non-executable memory pages, preventing attackers from executing shellcode injected into data-only memory regions (such as the stack or heap).
2. **Address Space Layout Randomization (ASLR)**: Randomizes the locations where system components, executable code, and memory allocations are loaded. Enabling Mandatory ASLR (Force Relocate Images), Bottom-Up ASLR, and High Entropy ASLR makes memory structures unpredictable, thwarting return-oriented programming (ROP) exploits.
3. **Control Flow Guard (CFG)**: Verifies control flow integrity for indirect call targets at compile time, preventing attackers from hijacking indirect jumps to point to arbitrary payloads.
4. **Structured Exception Handler Overwrite Protection (SEHOP)**: Blocks exploits that overwrite Structured Exception Handlers (SEH) to gain control of execution paths during error handling.
5. **Heap Termination on Corruption**: Immediately terminates a process if corruption is detected in its heap. This blocks heap-based buffer overflow exploitation before execution control can be seized.

### Application-Specific Hardening In addition to global default policies, specific security mitigations are applied to common productivity applications, browsers, document viewers, and scripting runtimes that represent the primary initial access target vectors: \* \*\*Microsoft Office (Word, Excel, PowerPoint, Outlook, Access, Publisher, Visio, Lync/Skype)\*\*: High-risk entry points. Enabling advanced Payload protections (Export/Import Address Filtering and Return-Oriented Programming mitigations) prevents memory bypass techniques used in malicious macro files and document exploits. Image loading policies are hardened to block loading of remote libraries from UNC shares. \* \*\*Adobe Acrobat and Reader\*\*: Document parsing vulnerability targets. Applies EAF, IAF, and ROP mitigations to block malicious PDF parsing exploits. \* \*\*Web Browsers (Chrome, Edge, Firefox)\*\*: Standard workspace web execution clients. Applies core DEP, ASLR, and CFG mitigations to secure dynamic code execution and sandbox interfaces without interfering with dynamic browser script generation engines. \* \*\*Scripting Engines and Utilities (wscript.exe, cscript.exe, powershell.exe)\*\*: Common execution mechanisms for malicious scripts. Applies strict memory relocation, code verification, and ROP defenses to prevent execution hijacking. \* \*\*Java Runtime Environment (java.exe, javaw.exe, javaws.exe)\*\*: Frequently targeted by older or legacy JIT exploits. Applies ROP mitigations to validate runtime stack operations.

## Legacy Impact & Compatibility \* \*\*Application Crashes\*\*: Strict system-wide mitigations, particularly Mandatory ASLR ('ForceRelocateImages') and DEP, can cause legacy, proprietary, or unsigned applications that are not compiled to support dynamic base relocations to crash upon launching. \* \*\*Orchestration/Agent Conflicts\*\*: Certain older monitoring agents or custom management wrappers that perform memory hook injections may fail when CFG or strict memory mitigations are enforced. \* \*\*Staged Deployment\*\*: It is critical to pilot configurations on a representative sample of workstations and member servers to identify application compatibility issues. If a specific critical business application is incompatible, administrators can define application-specific bypasses (') within the XML configuration rather than disabling system-wide protections.

## ## Implementation Steps

### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### Step 1: Generate the Reference XML Configuration File Before configuring the GPO, you must create a reference XML file containing the desired Exploit Protection mitigations: 1. On a reference workstation, open the \*\*Windows Security\*\* app. 2. Select \*\*App & browser control\*\* and click \*\*Exploit protection settings\*\*. 3. Under the \*\*System settings\*\* tab, configure the following: \* \*\*Control Flow Guard (CFG)\*\*: On by default \* \*\*Data Execution Prevention (DEP)\*\*: On by default \* \*\*Force randomization for images (Mandatory ASLR)\*\*: On by default \* \*\*Randomize memory allocations (Bottom-up ASLR)\*\*: On by default \* \*\*High-entropy ASLR\*\*: On by default \* \*\*Validate exception chains (SEHOP)\*\*: On by default \* \*\*Validate heap integrity\*\*: On by default 4. Select the \*\*Program settings\*\* tab and configure application-specific overrides for Microsoft Office ('winword.exe', 'excel.exe', etc.), Web Browsers, and PDF viewers to apply EAF, IAF, and ROP mitigations. 5. Scroll to the bottom of the page and click \*\*Export settings\*\*. 6. Save the file as 'ExploitProtectionSettings.xml'. 7. Copy this XML file to a location accessible by target endpoints, or distribute it to local target directories (e.g., 'C:\ProgramData\ExploitProtection\ExploitProtectionSettings.xml') via Group Policy Preferences (Files).

#### Step 2: Configure the Group Policy Setting 1. Open the **Group Policy Management Console** ('gpmc.msc') on a management workstation. 2. Create a new GPO or edit an existing one (e.g., 'GPO\_Hardening\_Endpoints'). 3. Navigate to: 'Computer Configuration\Administrative Templates\Windows Components\Windows Defender Exploit Guard\Exploit Protection' 4. Configure the following setting: \* **Policy**: 'Use a common set of exploit protection settings' \* **Setting**: 'Enabled' \* **Options**: Under the \*Path\* or \*Url\* field, enter the full path to the XML file (e.g., 'C:\ProgramData\ExploitProtection\ExploitProtectionSettings.xml' or a UNC share path). 5. If utilizing Microsoft Security Guide templates: \* Navigate to: 'Computer Configuration\Administrative Templates\MS Security Guide' \* Configure policies: \* **Policy**: 'Enable Certificate Padding' → Set to **Enabled** \* **Policy**: 'Enable Structured Exception Handling Overwrite Protection (SEHOP)' → Set to **Enabled** 6. Link the GPO to the appropriate Organizational Unit (OU) containing the target client endpoints and member servers.

---

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the exploit protection profile locally on individual systems or standalone hosts.

[Download Script: Configure-ExploitProtection.ps1](#)

```
Configure-ExploitProtection.ps1
Description: Generates the system-wide and application-specific Exploit Protection XML profile, applies it locally, and configures
the policy registry keys.

Write-Host "Applying Exploit Protection Profile..." -ForegroundColor Cyan

1. Define the XML content including System and App settings
$xmlContent = @"
<?xml version="1.0" encoding="utf-8"?>
<MitigationPolicy>
 <SystemConfig>
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 <SEHOP Enable="true" TelemetryOnly="false" />
 <Heap TerminateOnError="true" />
 </SystemConfig>
 <AppConfig Executable="WINWORD.EXE">
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 <Payload EnableExportAddressFilter="true" AuditEnableExportAddressFilter="false" EnableExportAddressFilterPlus="true" AuditEnabl
eExportAddressFilterPlus="false" EnableImportAddressFilter="true" AuditEnableImportAddressFilter="false" EnableRopStackPivot="true"
AuditEnableRopStackPivot="false" EnableRopCallerCheck="true" AuditEnableRopCallerCheck="false" EnableRopSimExec="true" AuditEnableRo
pSimExec="false" />
 <ImageLoad BlockRemoteImageLoads="true" AuditBlockRemoteImageLoads="false" PreferSystem32="true" AuditPreferSystem32="false" />
 </AppConfig>
 <AppConfig Executable="EXCEL.EXE">
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 <Payload EnableExportAddressFilter="true" AuditEnableExportAddressFilter="false" EnableExportAddressFilterPlus="true" AuditEnabl
eExportAddressFilterPlus="false" EnableImportAddressFilter="true" AuditEnableImportAddressFilter="false" EnableRopStackPivot="true"
AuditEnableRopStackPivot="false" EnableRopCallerCheck="true" AuditEnableRopCallerCheck="false" EnableRopSimExec="true" AuditEnableRo
pSimExec="false" />
 <ImageLoad BlockRemoteImageLoads="true" AuditBlockRemoteImageLoads="false" PreferSystem32="true" AuditPreferSystem32="false" />
 </AppConfig>
 <AppConfig Executable="POWERPNT.EXE">
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 <Payload EnableExportAddressFilter="true" AuditEnableExportAddressFilter="false" EnableExportAddressFilterPlus="true" AuditEnabl
eExportAddressFilterPlus="false" EnableImportAddressFilter="true" AuditEnableImportAddressFilter="false" EnableRopStackPivot="true"
AuditEnableRopStackPivot="false" EnableRopCallerCheck="true" AuditEnableRopCallerCheck="false" EnableRopSimExec="true" AuditEnableRo
pSimExec="false" />
 <ImageLoad BlockRemoteImageLoads="true" AuditBlockRemoteImageLoads="false" PreferSystem32="true" AuditPreferSystem32="false" />
 </AppConfig>
 <AppConfig Executable="OUTLOOK.EXE">
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 <Payload EnableExportAddressFilter="true" AuditEnableExportAddressFilter="false" EnableExportAddressFilterPlus="true" AuditEnabl
eExportAddressFilterPlus="false" EnableImportAddressFilter="true" AuditEnableImportAddressFilter="false" EnableRopStackPivot="true"
AuditEnableRopStackPivot="false" EnableRopCallerCheck="true" AuditEnableRopCallerCheck="false" EnableRopSimExec="true" AuditEnableRo
pSimExec="false" />
 <ImageLoad BlockRemoteImageLoads="true" AuditBlockRemoteImageLoads="false" PreferSystem32="true" AuditPreferSystem32="false" />
 </AppConfig>
 <AppConfig Executable="MSACCESS.EXE">
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 <Payload EnableExportAddressFilter="true" AuditEnableExportAddressFilter="false" EnableExportAddressFilterPlus="true" AuditEnabl
eExportAddressFilterPlus="false" EnableImportAddressFilter="true" AuditEnableImportAddressFilter="false" EnableRopStackPivot="true"
AuditEnableRopStackPivot="false" EnableRopCallerCheck="true" AuditEnableRopCallerCheck="false" EnableRopSimExec="true" AuditEnableRo
pSimExec="false" />
 <ImageLoad BlockRemoteImageLoads="true" AuditBlockRemoteImageLoads="false" PreferSystem32="true" AuditPreferSystem32="false" />
 </AppConfig>
 <AppConfig Executable="MSPUB.EXE">
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 <Payload EnableExportAddressFilter="true" AuditEnableExportAddressFilter="false" EnableExportAddressFilterPlus="true" AuditEnabl
eExportAddressFilterPlus="false" EnableImportAddressFilter="true" AuditEnableImportAddressFilter="false" EnableRopStackPivot="true"
AuditEnableRopStackPivot="false" EnableRopCallerCheck="true" AuditEnableRopCallerCheck="false" EnableRopSimExec="true" AuditEnableRo
pSimExec="false" />
 </AppConfig>
</MitigationPolicy>
"@
```

[illegible]

```

</AppConfig>
<AppConfig Executable="powershell.exe">
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
</AppConfig>
<AppConfig Executable="java.exe">
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 <Payload EnableRopStackPivot="true" AuditEnableRopStackPivot="false" EnableRopCallerCheck="true" AuditEnableRopCallerCheck="false" EnableRopSimExec="true" AuditEnableRopSimExec="false" />
</AppConfig>
<AppConfig Executable="javaw.exe">
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 <Payload EnableRopStackPivot="true" AuditEnableRopStackPivot="false" EnableRopCallerCheck="true" AuditEnableRopCallerCheck="false" EnableRopSimExec="true" AuditEnableRopSimExec="false" />
</AppConfig>
<AppConfig Executable="javaws.exe">
 <DEP Enable="true" EmulateAtlThunks="false" />
 <ASLR ForceRelocateImages="true" RequireInfo="false" BottomUp="true" HighEntropy="true" />
 <ControlFlowGuard Enable="true" SuppressExports="false" />
 <Payload EnableRopStackPivot="true" AuditEnableRopStackPivot="false" EnableRopCallerCheck="true" AuditEnableRopCallerCheck="false" EnableRopSimExec="true" AuditEnableRopSimExec="false" />
</AppConfig>
</MitigationPolicy>
"@

2. Create the target directory and write the XML file
$TargetDir = "C:\ProgramData\ExploitProtection"
if (-not (Test-Path $TargetDir)) {
 New-Item -Path $TargetDir -ItemType Directory -Force | Out-Null
}

$xmlPath = "$TargetDir\ExploitProtectionSettings.xml"
Set-Content -Path $xmlPath -Value $xmlContent -Encoding UTF8
Write-Host "Exploit Protection XML profile written to $($xmlPath)" -ForegroundColor Gray

3. Apply the settings locally using the cmdlet
if (Get-Command Set-ProcessMitigation -ErrorAction SilentlyContinue) {
 Set-ProcessMitigation -PolicyFilePath $xmlPath
 Write-Host "[+] Exploit protection system settings applied locally." -ForegroundColor Green
} else {
 Write-Warning "Set-ProcessMitigation cmdlet is not available. Please ensure you are running Windows 10/11 or Windows Server 2016+."
}

4. Configure policy registry keys to point to the XML file
$RegPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender ExploitGuard\Exploit Protection"
if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name "ExploitProtectionSettings" -Value $xmlPath -Type String -Force
Write-Host "[+] GPO policy registry values configured." -ForegroundColor Green

5. Configure MS Security Guide mitigations: Certificate Padding check and SEHOP registry keys
$WintrustPath = "HKLM:\SOFTWARE\Microsoft\Cryptography\Wintrust\Config"
if (-not (Test-Path $WintrustPath)) {
 New-Item -Path $WintrustPath -Force | Out-Null
}
Set-ItemProperty -Path $WintrustPath -Name "EnableCertPaddingCheck" -Value 1 -Type DWord -Force

$WintrustWow64Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\Cryptography\Wintrust\Config"
if (-not (Test-Path $WintrustWow64Path)) {
 New-Item -Path $WintrustWow64Path -Force | Out-Null
}
Set-ItemProperty -Path $WintrustWow64Path -Name "EnableCertPaddingCheck" -Value 1 -Type DWord -Force

$SessionKernelPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\kernel"
if (-not (Test-Path $SessionKernelPath)) {
 New-Item -Path $SessionKernelPath -Force | Out-Null
}
Set-ItemProperty -Path $SessionKernelPath -Name "DisableExceptionChainValidation" -Value 0 -Type DWord -Force
Write-Host "[+] Certificate Padding check and SEHOP registry keys applied." -ForegroundColor Green

```

```
Write-Host "Exploit Protection Profile application completed successfully." -ForegroundColor Cyan
```

*To verify the settings have been applied:*

[Download Script: Get-ExploitProtectionStatus.ps1](#)



```

Get-ExploitProtectionStatus.ps1
Description: Audits the system-wide Exploit Protection settings against the recommended security baseline.

Write-Host "Auditing system-wide Exploit Protection mitigations..." -ForegroundColor Cyan

$BaselineFailed = $false
$Mitigations = Get-ProcessMitigation -System

Helper function to evaluate and display status
function Test-MitigationSetting {
 param(
 [string]$MitigationName,
 [string]$CurrentValue,
 [string]$ExpectedValue
)
 if ($CurrentValue -eq $ExpectedValue) {
 Write-Host " [PASS] $($MitigationName): $($CurrentValue)" -ForegroundColor Green
 } else {
 Write-Host " [FAIL] $($MitigationName): $($CurrentValue) (Expected: $($ExpectedValue))" -ForegroundColor Red
 $script:BaselineFailed = $true
 }
}

Write-Host "`nSystem-wide Mitigations:" -ForegroundColor Gray

Audit DEP
Test-MitigationSetting -MitigationName "DEP Enable" -CurrentValue $Mitigations.DEP.Enable -ExpectedValue "ON"
Test-MitigationSetting -MitigationName "DEP EmulateAtlThunks" -CurrentValue $Mitigations.DEP.EmulateAtlThunks -ExpectedValue "OFF"

Audit ASLR
Test-MitigationSetting -MitigationName "ASLR ForceRelocateImages" -CurrentValue $Mitigations.ASLR.ForceRelocateImages -ExpectedValue "ON"
Test-MitigationSetting -MitigationName "ASLR BottomUp" -CurrentValue $Mitigations.ASLR.BottomUp -ExpectedValue "ON"
Test-MitigationSetting -MitigationName "ASLR HighEntropy" -CurrentValue $Mitigations.ASLR.HighEntropy -ExpectedValue "ON"

Audit CFG
Test-MitigationSetting -MitigationName "CFG Enable" -CurrentValue $Mitigations.CFG.Enable -ExpectedValue "ON"

Audit SEHOP
Test-MitigationSetting -MitigationName "SEHOP Enable" -CurrentValue $Mitigations.SEHOP.Enable -ExpectedValue "ON"

Audit Heap
Test-MitigationSetting -MitigationName "Heap TerminateOnError" -CurrentValue $Mitigations.Heap.TerminateOnError -ExpectedValue "ON"

Audit Registry Policy and XML Configuration
Write-Host "`nRegistry Policy and XML Configuration:" -ForegroundColor Gray
$RegPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender ExploitGuard\Exploit Protection"
if (Test-Path $RegPath) {
 $SettingsValue = Get-ItemProperty -Path $RegPath -Name "ExploitProtectionSettings" -ErrorAction SilentlyContinue
 if ($SettingsValue -and $SettingsValue.ExploitProtectionSettings -ne "") {
 $XmlPath = $SettingsValue.ExploitProtectionSettings
 Write-Host " [PASS] Exploit Protection Policy registry key is configured." -ForegroundColor Green
 Write-Host " Path: $XmlPath" -ForegroundColor Gray

 if (Test-Path $XmlPath) {
 Write-Host " [PASS] Exploit Protection XML file exists." -ForegroundColor Green
 try {
 [xml]$xml = Get-Content -Path $XmlPath -Raw -ErrorAction Stop

 # Verify SystemConfig block
 if ($xml.MitigationPolicy.SystemConfig) {
 Write-Host " [PASS] XML contains SystemConfig block." -ForegroundColor Green
 } else {
 Write-Host " [FAIL] XML is missing SystemConfig block." -ForegroundColor Red
 $BaselineFailed = $true
 }
 }

 # Verify major AppConfigs
 $ExpectedApps = @"(WINWORD.EXE", "EXCEL.EXE", "chrome.exe", "msedge.exe", "AcroRd32.exe", "java.exe)"
 $ConfiguredApps = $xml.MitigationPolicy.AppConfig | ForEach-Object { $_.Executable }

 foreach ($app in $ExpectedApps) {
 if ($ConfiguredApps -contains $app) {
 Write-Host " [PASS] XML contains application profile for: $app" -ForegroundColor Green
 }
 }
 }
 }
}

```



```

 } else {
 Write-Host " [FAIL] XML is missing application profile for: $app" -ForegroundColor Red
 $BaselineFailed = $true
 }
 }
} catch {
 Write-Host " [FAIL] Failed to parse Exploit Protection XML. Error: $(_.Exception.Message)" -ForegroundColor Red
 $BaselineFailed = $true
}
} else {
 Write-Host " [FAIL] Exploit Protection XML file does not exist at specified path." -ForegroundColor Red
 $BaselineFailed = $true
}
} else {
 Write-Host " [FAIL] Exploit Protection Policy registry key is empty or missing." -ForegroundColor Red
 $BaselineFailed = $true
}
} else {
 Write-Host " [FAIL] Exploit Protection Policy registry path does not exist." -ForegroundColor Red
 $BaselineFailed = $true
}
}

Audit MS Security Guide Registry Settings
Write-Host "`nMS Security Guide Mitigations (Certificate Padding & SEHOP):" -ForegroundColor Gray

function Test-MitigationRegistryValue ($path, $name, $expectedValue) {
 $val = Get-ItemProperty -Path $path -Name $name -ErrorAction SilentlyContinue
 $actual = if ($val) { $val.$name } else { "" }
 $color = "Red"
 if ($actual -eq $expectedValue) {
 $color = "Green"
 } else {
 $script:BaselineFailed = $true
 }
 Write-Host " - Registry Setting: $name | Actual: '$actual' (Expected: '$expectedValue')" -ForegroundColor $color
}

$WintrustPath = "HKLM:\SOFTWARE\Microsoft\Cryptography\Wintrust\Config"
Test-MitigationRegistryValue $WintrustPath "EnableCertPaddingCheck" 1

$WintrustWow64Path = "HKLM:\SOFTWARE\Wow6432Node\Microsoft\Cryptography\Wintrust\Config"
Test-MitigationRegistryValue $WintrustWow64Path "EnableCertPaddingCheck" 1

$SessionKernelPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\kernel"
Test-MitigationRegistryValue $SessionKernelPath "DisableExceptionChainValidation" 0

Write-Host ""
if ($BaselineFailed) {
 Write-Host "Auditing FAILED: One or more configurations do not match the secure baseline." -ForegroundColor Red
 exit 1
} else {
 Write-Host "Auditing PASSED: All configurations match the secure baseline." -ForegroundColor Green
 exit 0
}
}

```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows 10/11 Client Benchmark\*\*: Section 18.9.30 (ASR/Exploit Guard Mitigation Policy configurations) \* \*\*CIS Microsoft Windows 10/11 Client Benchmark\*\*: Section 18.4.4 (Enable Certificate Padding), Section 18.4.5 (Enable Structured Exception Handling Overwrite Protection (SEHOP)) \* \*\*Microsoft Security Baselines\*\*: Exploit Protection baseline templates and application configurations \* \*\*ANSI Active Directory Hardening Guide\*\*: Recommendations regarding endpoint protective controls

# [REQ-END-021] Restrict Safe Mode Access to Administrators

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Preferences\Windows Settings\Registry` \* \*\*Registry Location\*\*:  
`HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System` \* \*\*Value Name\*\*: `SafeModeBlockNonAdmins` \* \*\*Value Type\*\*:  
`REG\_DWORD` \* \*\*Value Data\*\*: `1`

---

## Rationale Malicious actors with standard user credentials can potentially bypass local security policies, local endpoint detection and response (EDR) agents, and group policy restrictions by booting the system into Safe Mode. In Safe Mode, many security agents and services do not load, creating an environment where local controls can be circumvented.

By configuring `SafeModeBlockNonAdmins = 1`:

1. **Prevent Credential Bypass**: Standard users are blocked from logging in during Safe Mode, ensuring they cannot exploit the disabled security agents to execute unauthorized programs or extract system information.
2. **Maintenance Integrity**: Safe Mode remains accessible exclusively to system administrators for debugging and recovery, ensuring administrative capability is preserved while mitigating standard user risk.

---

## Legacy Impact & Compatibility \* \*\*Administrative Operations\*\*: This setting does not affect local or domain administrative accounts, which can still log in normally to perform repairs. \* \*\*Troubleshooting Availability\*\*: If a standard user experiences system issues that require Safe Mode, an administrator must log in to the workstation to troubleshoot, which may slightly increase administrative overhead.

---

#### ## Implementation Steps

##### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

Because there is no native Administrative Template (ADMX) policy for this setting, it must be deployed using Group Policy Preferences (GPP) to configure the registry directly.

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a domain controller or management host.
2. Create a new GPO or edit an existing one (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Right-click in the right pane, select **New > Registry Item**, and configure:
  - **Action**: `Update`
  - **Hive**: `HKEY_LOCAL_MACHINE`
  - **Key Path**: `SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System`
  - **Value Name**: `SafeModeBlockNonAdmins`
  - **Value Type**: `REG_DWORD`
  - **Value Data**: `1`
5. Link the GPO to the appropriate Organizational Unit (OU) containing the target client endpoints and member servers.

---

##### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally on standalone systems or during reference image build phases.

[Download Script: Configure-DisableSafeModeNonAdmins.ps1](#)

```
Configure-DisableSafeModeNonAdmins.ps1
Description: Prevents standard users from logging into the system while in Safe Mode by setting SafeModeBlockNonAdmins to 1.

$RegPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
$ValueName = "SafeModeBlockNonAdmins"
$ValueData = 1

Write-Host "Applying hardening requirement: Restrict Safe Mode access to administrators..." -ForegroundColor Cyan

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value $ValueData -Type DWord -Force | Out-Null
Write-Host "Hardening applied successfully." -ForegroundColor Green
```

To verify the setting has been applied:

[Download Script: Get-SafeModeNonAdminsStatus.ps1](#)

```
Get-SafeModeNonAdminsStatus.ps1
Description: Checks the current configuration state of SafeModeBlockNonAdmins registry setting.

$RegPath = "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
$ValueName = "SafeModeBlockNonAdmins"

Write-Host "Auditing hardening requirement: Restrict Safe Mode access to administrators..." -ForegroundColor Cyan

if (Test-Path $RegPath) {
 $value = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
 if ($null -ne $value -and $value.$ValueName -eq 1) {
 Write-Host "Audit Result: Compliant. Standard users are blocked from logging in during Safe Mode ($ValueName = 1)." -Foregro
undColor Green
 exit 0
 }
}

Write-Host "Audit Result: Non-Compliant. Standard users are allowed to log in during Safe Mode." -ForegroundColor Red
exit 1
```

---

## Sources & Compliance References \* \*\*Australian Cyber Security Centre (ACSC) / ASD\*\* : Windows Hardening Guidelines (SafeModeBlockNonAdmins configuration). \* \*\*Microsoft Learn\*\* : Windows policies and registry-based mitigations.

# [REQ-END-022] Configure Windows Defender Firewall and Block LOLBins

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: \* `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Windows Defender Firewall with Advanced Security` \* \*\*Registry Locations\*\*: \* `HKLM\SOFTWARE\Policies\Microsoft\WindowsFirewall\DomainProfile` \* `HKLM\SOFTWARE\Policies\Microsoft\WindowsFirewall\PrivateProfile` \* `HKLM\SOFTWARE\Policies\Microsoft\WindowsFirewall\PublicProfile` \* `HKLM\SYSTEM\CurrentControlSet\Services\SharedAccess\Parameters\FirewallPolicy\FirewallRules`

## Rationale Windows Defender Firewall is the primary host-based security control protecting endpoints from unauthorized incoming network connections and regulating outgoing network behaviors. A secure baseline requires enabling the firewall on all profiles, setting inbound connections to block by default, disabling notification prompts that can be bypassed by users, and implementing detailed auditing/logging to monitor network anomalies.

Additionally:

1. **Outbound LOLBins Blocking**: Malicious actors frequently abuse built-in Windows administrative utilities (known as Living Off the Land Binaries, or LOLBins) to download malicious payloads, exfiltrate sensitive data, and communicate with external command-and-control (C2) servers. Blocking outbound network communication for binaries that have no legitimate business requirement to connect to external networks (such as `mshta.exe`, `certutil.exe`, `bitsadmin.exe`, `regsvr32.exe`, `rundll32.exe`, `cscript.exe`, `wscript.exe`, and `hh.exe`) significantly mitigates these threat vectors.

## Legacy Impact & Compatibility \* \*\*Legitimate Administrative Scripts\*\*: Outbound rules will block administrative scripts, third-party software deployment installers, or internal software update tools that execute using `cscript.exe`, `wscript.exe`, or `mshta.exe` and require access to network resources. Proper scoping or exemptions based on authorized source paths or networks should be tested.

#### ## Implementation Steps

##### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### 1. Configure Profile States and Logging 1. Open the **Group Policy Management Console** (`gpmc.msc`) on a domain controller or management workstation. 2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints`). 3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Windows Defender Firewall with Advanced Security` 4. Right-click **Windows Defender Firewall with Advanced Security** and select **Properties**. 5. Configure the **Domain Profile** tab: \* \*\*Firewall state\*\*: `On (recommended)` \* \*\*Inbound connections\*\*: `Block (default)` \* \*\*Outbound connections\*\*: `Allow (default)` \* Click **Customize...** under **Settings**: \* \*\*Display a notification\*\*: `No` \* \*\*Apply local firewall rules\*\*: `No` \* \*\*Apply local connection security rules\*\*: `No` \* Click **Customize...** under **Logging**: \* \*\*Name\*\*: `%SystemRoot%\System32\logfiles\firewall\domainfw.log` \* \*\*Size limit (KB)\*\*: `16384` \* \*\*Log dropped packets\*\*: `Yes` \* \*\*Log successful connections\*\*: `No` 6. Configure the **Private Profile** tab: \* \*\*Firewall state\*\*: `On (recommended)` \* \*\*Inbound connections\*\*: `Block (default)` \* \*\*Outbound connections\*\*: `Allow (default)` \* Click **Customize...** under **Settings**: \* \*\*Display a notification\*\*: `No` \* \*\*Apply local firewall rules\*\*: `No` \* \*\*Apply local connection security rules\*\*: `No` \* Click **Customize...** under **Logging**: \* \*\*Name\*\*: `%SystemRoot%\System32\logfiles\firewall\privatefw.log` \* \*\*Size limit (KB)\*\*: `16384` \* \*\*Log dropped packets\*\*: `Yes` \* \*\*Log successful connections\*\*: `No` 7. Configure the **Public Profile** tab: \* \*\*Firewall state\*\*: `On (recommended)` \* \*\*Inbound connections\*\*: `Block (default)` \* \*\*Outbound connections\*\*: `Allow (default)` \* Click **Customize...** under **Settings**: \* \*\*Display a notification\*\*: `No` \* \*\*Apply local firewall rules\*\*: `No` \* \*\*Apply local connection security rules\*\*: `No` \* Click **Customize...** under **Logging**: \* \*\*Name\*\*: `%SystemRoot%\System32\logfiles\firewall\publicfw.log` \* \*\*Size limit (KB)\*\*: `16384` \* \*\*Log dropped packets\*\*: `Yes` \* \*\*Log successful connections\*\*: `No`

##### 2. Create Outbound Rules for Known LOLBins 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Windows Defender Firewall with Advanced Security\Windows Defender Firewall with Advanced Security\Outbound Rules` 2. Create a new rule for each target LOLBin binary (e.g., `mshta.exe`, `certutil.exe`, `bitsadmin.exe`, `regsvr32.exe`, `rundll32.exe`, `cscript.exe`, `wscript.exe`, `hh.exe`, `calc.exe`, `notepad.exe`, `conhost.exe`, `RunScriptHelper.exe`): \* Right-click **Outbound Rules** and select **New Rule...** \* \*\*Rule Type\*\*: `Program` \* \*\*Program\*\*: Choose `This program path` and enter the path matching the binary (both x64 and x86 paths if applicable, e.g., `%SystemRoot%\System32\mshta.exe` and `%SystemRoot%\SysWOW64\mshta.exe`). \* \*\*Action\*\*: `Block the connection` \* \*\*Profile\*\*: Select `Domain`, `Private`, and `Public`. \* \*\*Name\*\*: Specify a descriptive name (e.g., `Hardening: Block Outbound mshta.exe (x64)`).

##### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to configure Windows Defender Firewall profiles, logging parameters, merge settings, and outbound LOLBins rules.

[Download Script: Configure-WindowsFirewall.ps1](#)

```
Configure-WindowsFirewall.ps1
Description: Configures Windows Defender Firewall profiles (Domain, Private, Public) and blocks outbound traffic for known LOLBins.

Write-Host "Configuring Windows Defender Firewall profiles..." -ForegroundColor Cyan

1. Configure profiles
$FWProfiles = @("Domain", "Private", "Public")
foreach ($FWProfile in $FWProfiles) {
 $LogFile = "$env:windir\System32\logfiles\firewall\$($FWProfile.ToLower())fw.log"

 Set-NetFirewallProfile -Profile $FWProfile `
 -Enabled True `
 -DefaultInboundAction Block `
 -DefaultOutboundAction Allow `
 -NotifyOnListen False `
 -AllowLocalPolicyMerge False `
 -AllowLocalIPsecPolicyMerge False `
 -LogFileName $LogFile `
 -LogMaxSizeKilobytes 16384 `
 -LogBlocked True `
 -LogAllowed False | Out-Null
 Write-Host "[+] Profile '$FWProfile' configured with logging and defaults." -ForegroundColor Green
}

2. Block outbound traffic for known LOLBins
$LoLbins = @(
 @{ Name = "mshta.exe (x64)"; Path = "%SystemRoot%\System32\mshta.exe" },
 @{ Name = "mshta.exe (x86)"; Path = "%SystemRoot%\SysWOW64\mshta.exe" },
 @{ Name = "certutil.exe (x64)"; Path = "%SystemRoot%\System32\certutil.exe" },
 @{ Name = "certutil.exe (x86)"; Path = "%SystemRoot%\SysWOW64\certutil.exe" },
 @{ Name = "bitsadmin.exe (x64)"; Path = "%SystemRoot%\System32\bitsadmin.exe" },
 @{ Name = "bitsadmin.exe (x86)"; Path = "%SystemRoot%\SysWOW64\bitsadmin.exe" },
 @{ Name = "regsvr32.exe (x64)"; Path = "%SystemRoot%\System32\regsvr32.exe" },
 @{ Name = "regsvr32.exe (x86)"; Path = "%SystemRoot%\SysWOW64\regsvr32.exe" },
 @{ Name = "rundll32.exe (x64)"; Path = "%SystemRoot%\System32\rundll32.exe" },
 @{ Name = "rundll32.exe (x86)"; Path = "%SystemRoot%\SysWOW64\rundll32.exe" },
 @{ Name = "cscript.exe (x64)"; Path = "%SystemRoot%\System32\cscript.exe" },
 @{ Name = "cscript.exe (x86)"; Path = "%SystemRoot%\SysWOW64\cscript.exe" },
 @{ Name = "wscript.exe (x64)"; Path = "%SystemRoot%\System32\wscript.exe" },
 @{ Name = "wscript.exe (x86)"; Path = "%SystemRoot%\SysWOW64\wscript.exe" },
 @{ Name = "hh.exe (x64)"; Path = "%SystemRoot%\hh.exe" },
 @{ Name = "hh.exe (x86)"; Path = "%SystemRoot%\SysWOW64\hh.exe" },
 @{ Name = "calc.exe (x64)"; Path = "%SystemRoot%\System32\calc.exe" },
 @{ Name = "calc.exe (x86)"; Path = "%SystemRoot%\SysWOW64\calc.exe" },
 @{ Name = "notepad.exe (x64)"; Path = "%SystemRoot%\System32\notepad.exe" },
 @{ Name = "notepad.exe (x86)"; Path = "%SystemRoot%\SysWOW64\notepad.exe" },
 @{ Name = "conhost.exe (x64)"; Path = "%SystemRoot%\System32\conhost.exe" },
 @{ Name = "conhost.exe (x86)"; Path = "%SystemRoot%\SysWOW64\conhost.exe" },
 @{ Name = "RunScriptHelper.exe (x64)"; Path = "%SystemRoot%\System32\RunScriptHelper.exe" },
 @{ Name = "RunScriptHelper.exe (x86)"; Path = "%SystemRoot%\SysWOW64\RunScriptHelper.exe" }
)

Write-Host "Configuring outbound firewall block rules for known LOLBins..." -ForegroundColor Cyan

foreach ($Bin in $LoLbins) {
 $DisplayName = "Hardening: Block Outbound $($Bin.Name)"
 $Existing = Get-NetFirewallRule -DisplayName $DisplayName -ErrorAction SilentlyContinue
 if ($null -eq $Existing) {
 New-NetFirewallRule -DisplayName $DisplayName `
 -Name $DisplayName `
 -Direction Outbound `
 -Action Block `
 -Program $Bin.Path `
 -Profile Any `
 -Enabled True | Out-Null
 Write-Host "[+] Outbound block rule created for $($Bin.Name)." -ForegroundColor Green
 } else {
 Set-NetFirewallRule -DisplayName $DisplayName -Action Block -Enabled True | Out-Null
 Write-Host "[~] Outbound block rule for $($Bin.Name) already exists, updated state to Enabled/Block." -ForegroundColor Gray
 }
}
}
```

```
Write-Host "Firewall profiles and LOLBins outbound rules configured successfully." -ForegroundColor Green
```

*To verify the settings have been applied:*

[Download Script: Get-WindowsFirewallStatus.ps1](#)

```
Get-WindowsFirewallStatus.ps1
Description: Audits Windows Defender Firewall profile configurations and outbound block rules for known LOLBins.

Write-Host "--- Auditing Windows Defender Firewall Configuration ---" -ForegroundColor Cyan

$script:Vulnerable = $false

Helper function to audit firewall profiles
function Test-FirewallProfile ($ProfileName, $ExpectMergeLocal, $ExpectMergeIPsec) {
 $FWProfile = Get-NetFirewallProfile -Profile $ProfileName -ErrorAction SilentlyContinue
 if ($null -eq $FWProfile) {
 Write-Host " - Profile '$ProfileName' NOT FOUND" -ForegroundColor Red
 $script:Vulnerable = $true
 return
 }

 $EnabledColor = if ($FWProfile.Enabled -eq $true) { "Green" } else { "Red" }
 $InboundColor = if ($FWProfile.DefaultInboundAction -eq "Block") { "Green" } else { "Red" }
 $OutboundColor = if ($FWProfile.DefaultOutboundAction -eq "Allow") { "Green" } else { "Red" }
 $NotifyColor = if ($FWProfile.NotifyOnListen -eq $false) { "Green" } else { "Red" }

 Write-Host " * Profile: $ProfileName" -ForegroundColor Gray
 Write-Host " - Enabled: $($FWProfile.Enabled) (Expected: True)" -ForegroundColor $EnabledColor
 Write-Host " - DefaultInboundAction: $($FWProfile.DefaultInboundAction) (Expected: Block)" -ForegroundColor $InboundColor
 Write-Host " - DefaultOutboundAction: $($FWProfile.DefaultOutboundAction) (Expected: Allow)" -ForegroundColor $OutboundColor
 Write-Host " - NotifyOnListen: $($FWProfile.NotifyOnListen) (Expected: False)" -ForegroundColor $NotifyColor

 # Check log configurations
 $LogPath = "$env:windir\System32\logfiles\firewall\$($ProfileName.ToLower())fw.log"
 $LogPathColor = if ($FWProfile.LogFileName -eq $LogPath) { "Green" } else { "Red" }
 $LogSizeColor = if ($FWProfile.LogMaxSizeKilobytes -ge 16384) { "Green" } else { "Red" }
 $LogBlockedColor = if ($FWProfile.LogBlocked -eq $true) { "Green" } else { "Red" }
 $LogAllowedColor = if ($FWProfile.LogAllowed -eq $false) { "Green" } else { "Red" }

 Write-Host " - LogFileName: $($FWProfile.LogFileName) (Expected: $LogPath)" -ForegroundColor $LogPathColor
 Write-Host " - LogMaxSizeKilobytes: $($FWProfile.LogMaxSizeKilobytes) (Expected: ≥ 16384)" -ForegroundColor $LogSizeColor
 Write-Host " - LogBlocked: $($FWProfile.LogBlocked) (Expected: True)" -ForegroundColor $LogBlockedColor
 Write-Host " - LogAllowed: $($FWProfile.LogAllowed) (Expected: False)" -ForegroundColor $LogAllowedColor

 if ($FWProfile.Enabled -ne $true -or $FWProfile.DefaultInboundAction -ne "Block" -or $FWProfile.NotifyOnListen -ne $false -or $FWProfile.LogFileName -ne $LogPath -or $FWProfile.LogMaxSizeKilobytes -lt 16384 -or $FWProfile.LogBlocked -ne $true -or $FWProfile.LogAllowed -ne $false) {
 $script:Vulnerable = $true
 }

 if ($null -ne $ExpectMergeLocal) {
 $MergeLocalColor = if ($FWProfile.AllowLocalPolicyMerge -eq $ExpectMergeLocal) { "Green" } else { "Red" }
 Write-Host " - AllowLocalPolicyMerge: $($FWProfile.AllowLocalPolicyMerge) (Expected: $ExpectMergeLocal)" -ForegroundColor $MergeLocalColor
 if ($FWProfile.AllowLocalPolicyMerge -ne $ExpectMergeLocal) { $script:Vulnerable = $true }
 }

 if ($null -ne $ExpectMergeIPsec) {
 $MergeIPsecColor = if ($FWProfile.AllowLocalIPsecPolicyMerge -eq $ExpectMergeIPsec) { "Green" } else { "Red" }
 Write-Host " - AllowLocalIPsecPolicyMerge: $($FWProfile.AllowLocalIPsecPolicyMerge) (Expected: $ExpectMergeIPsec)" -ForegroundColor $MergeIPsecColor
 if ($FWProfile.AllowLocalIPsecPolicyMerge -ne $ExpectMergeIPsec) { $script:Vulnerable = $true }
 }
}

Write-Host "Auditing profiles..." -ForegroundColor Gray
Test-FirewallProfile -ProfileName "Domain" -ExpectMergeLocal $false -ExpectMergeIPsec $false
Test-FirewallProfile -ProfileName "Private" -ExpectMergeLocal $false -ExpectMergeIPsec $false
Test-FirewallProfile -ProfileName "Public" -ExpectMergeLocal $false -ExpectMergeIPsec $false

Audit outbound rules for known LOLBins
$LoLbins = @(
 @{ Name = "mshta.exe (x64)"; Path = "%SystemRoot%\System32\mshta.exe" },
 @{ Name = "mshta.exe (x86)"; Path = "%SystemRoot%\SysWOW64\mshta.exe" },
 @{ Name = "certutil.exe (x64)"; Path = "%SystemRoot%\System32\certutil.exe" },
 @{ Name = "certutil.exe (x86)"; Path = "%SystemRoot%\SysWOW64\certutil.exe" },
 @{ Name = "bitsadmin.exe (x64)"; Path = "%SystemRoot%\System32\bitsadmin.exe" },
 @{ Name = "bitsadmin.exe (x86)"; Path = "%SystemRoot%\SysWOW64\bitsadmin.exe" },
 @{ Name = "regsvr32.exe (x64)"; Path = "%SystemRoot%\System32\regsvr32.exe" },
 @{ Name = "regsvr32.exe (x86)"; Path = "%SystemRoot%\SysWOW64\regsvr32.exe" },

```

```

@{ Name = "rundll32.exe (x64)"; Path = "%SystemRoot%\System32\rundll32.exe" },
@{ Name = "rundll32.exe (x86)"; Path = "%SystemRoot%\SysWOW64\rundll32.exe" },
@{ Name = "cscript.exe (x64)"; Path = "%SystemRoot%\System32\cscript.exe" },
@{ Name = "cscript.exe (x86)"; Path = "%SystemRoot%\SysWOW64\cscript.exe" },
@{ Name = "wscript.exe (x64)"; Path = "%SystemRoot%\System32\wscript.exe" },
@{ Name = "wscript.exe (x86)"; Path = "%SystemRoot%\SysWOW64\wscript.exe" },
@{ Name = "hh.exe (x64)"; Path = "%SystemRoot%\hh.exe" },
@{ Name = "hh.exe (x86)"; Path = "%SystemRoot%\SysWOW64\hh.exe" },
@{ Name = "calc.exe (x64)"; Path = "%SystemRoot%\System32\calc.exe" },
@{ Name = "calc.exe (x86)"; Path = "%SystemRoot%\SysWOW64\calc.exe" },
@{ Name = "notepad.exe (x64)"; Path = "%SystemRoot%\System32\notepad.exe" },
@{ Name = "notepad.exe (x86)"; Path = "%SystemRoot%\SysWOW64\notepad.exe" },
@{ Name = "conhost.exe (x64)"; Path = "%SystemRoot%\System32\conhost.exe" },
@{ Name = "conhost.exe (x86)"; Path = "%SystemRoot%\SysWOW64\conhost.exe" },
@{ Name = "RunScriptHelper.exe (x64)"; Path = "%SystemRoot%\System32\RunScriptHelper.exe" },
@{ Name = "RunScriptHelper.exe (x86)"; Path = "%SystemRoot%\SysWOW64\RunScriptHelper.exe" }
)

Write-Host "Auditing outbound firewall rules for known LOLBins..." -ForegroundColor Gray

foreach ($Bin in $Lolbins) {
 $DisplayName = "Hardening: Block Outbound $($Bin.Name)"
 $Rule = Get-NetFirewallRule -DisplayName $DisplayName -ErrorAction SilentlyContinue
 $Color = "Red"

 if ($null -ne $Rule) {
 $ProgFilter = Get-NetFirewallApplicationFilter -AssociatedNetFirewallRule $Rule -ErrorAction SilentlyContinue
 $ProgPath = "None"
 if ($null -ne $ProgFilter) {
 $ProgPath = $ProgFilter.Program
 }

 if ($Rule.Enabled -eq $true -and $Rule.Direction -eq "Outbound" -and $Rule.Action -eq "Block" -and $ProgPath -eq $Bin.Path)
 {
 $Color = "Green"
 Write-Host " - Firewall Rule: $DisplayName | Enabled: True | Action: Block | Program: $ProgPath (Compliant)" -ForegroundColor $Color
 } else {
 $script:Vulnerable = $true
 Write-Host " - Firewall Rule: $DisplayName | Enabled: $($Rule.Enabled) | Action: $($Rule.Action) | Program: $ProgPath (Non-Compliant)" -ForegroundColor $Color
 }
 } else {
 $script:Vulnerable = $true
 Write-Host " - Firewall Rule: $DisplayName | NOT FOUND (Non-Compliant)" -ForegroundColor $Color
 }
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}

```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*\*: Section 9.1 (Domain Profile), Section 9.2 (Private Profile), Section 9.3 (Public Profile) \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendations on host-based firewalls and protocol filtering \* \*\*Microsoft Security Baselines\*\*\*: Windows Defender Firewall recommendations



# [REQ-END-023] Enable LSA Protection with UEFI Lock

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 Enterprise (1607+) and Windows 11 Enterprise/Professional.

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: Computer Configuration\Policies\Administrative Templates\System\Local Security Authority\Configures LSASS to run as a protected process \* \*\*Registry Location\*\*: HKLM\SYSTEM\CurrentControlSet\Control\Lsa \* 'RunAsPPL' = '1' (REG\_DWORD, Enabled with UEFI Lock)

## Rationale The Local Security Authority Subsystem Service (LSASS) process manages security policies, user authentication, and credential tokens on Windows systems. Attackers targeting workstations commonly attempt to extract plain-text credentials or NT hashes from LSASS memory using debugging tools (e.g., Mimikatz, Procdump).

Enabling LSA Protection ensures that:

1. **Protected Process Light (PPL)**: The LSASS process runs as a Protected Process Light (PPL).
2. **Access Restriction**: Only verified, digitally signed code can load into LSASS, and standard processes (even those running as local system/administrator) cannot read the memory space of LSASS or inject code into it.
3. **Mitigating Dump Attacks**: Credential harvesting tools cannot dump LSASS memory to disk or scrape keys from LSA memory blocks.

## Legacy Impact & Compatibility \* \*\*Driver Signing\*\*: Any custom authentication packages or third-party smart card drivers that load into LSASS must be digitally signed with a Microsoft signature. Unsigned drivers will be blocked from loading. \* \*\*Reboot Required\*\*: Enabling or disabling LSA Protection requires a system restart to take effect. \* \*\*Audit Mode\*\*: LSA Protection can be run in audit mode (using registry value AuditLevel under the Lsa key) to test for unsigned driver compatibility prior to full enforcement.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the GPO linked to the workstations Organizational Unit (OU) (e.g., `GPO_Hardening_Workstations` ).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Local Security Authority`
4. Configure the setting:
  - **Policy**: `Configures LSASS to run as a protected process`
  - **Setting**: `Enabled`
  - **Configure LSA to run as a protected process**: `Enabled with UEFI Lock`
5. Link the GPO to the target workstations Organizational Unit (OU).

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Configure the local registry key on the workstation to run LSASS as a protected process with UEFI Lock.

[Download Script: Configure-LsaProtection.ps1](#)

```
Configure-LsaProtection.ps1
Description: Configures the RunAsPPL registry key to enable LSA Protection on workstations.

Write-Host "Applying LSA Protection registry hardening..." -ForegroundColor Cyan

$LsaPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"

if (-not (Test-Path $LsaPath)) {
 New-Item -Path $LsaPath -Force | Out-Null
}

Set-ItemProperty -Path $LsaPath -Name "RunAsPPL" -Value 1 -Type DWord
Write-Host "[+] LSA Protection (RunAsPPL) enabled in registry. (Reboot required)." -ForegroundColor Green
```

To verify the local LSA Protection state:

[Download Script: Get-LsaProtectionStatus.ps1](#)

```
Get-LsaProtectionStatus.ps1
Description: Checks the registry settings and running process state to verify LSA Protection is active.

Write-Host "--- Auditing LSA Protection Status ---" -ForegroundColor Cyan

$LsaPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa"
$RunAsPPL = (Get-ItemProperty -Path $LsaPath -Name "RunAsPPL" -ErrorAction SilentlyContinue).RunAsPPL

if ($RunAsPPL -eq 1) {
 Write-Host " - LSA Protection (RunAsPPL): Enabled (Secure)" -ForegroundColor Green
} else {
 Write-Host " - VULNERABLE: LSA Protection (RunAsPPL) is not configured or disabled (Value: $($RunAsPPL))" -ForegroundColor Red
}
}
```

---

## Sources & Compliance References \* \*\*ANSI AD Hardening Guide\*\*: Recommendations regarding LSASS protection and LSA credential security. \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*: Section 18.2.1 (LSA Protection) \* \*\*Microsoft Security Baselines\*\*: Windows Defender Credential Guard and LSA Protection guidelines.

## # Disable Unnecessary System Services

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Operating system services that are not required for core administrative tasks or business functionality introduce unnecessary entry points for network exposure, resource utilization, and privilege escalation vulnerabilities. This submodule contains the individual hardening controls for each unnecessary system service.

## Legacy Impact & Compatibility \* \*\*Infra Compatibility\*\*: Administrative functions relying on legacy RPC Locator or SSDP-based discovery of network printers/devices may be affected. \* \*\*WSL Functionality\*\*: Disabling `LxssManager` prevents users from running Windows Subsystem for Linux (WSL) containers on their endpoints. If WSL is strictly required for engineering roles, this service should be excluded from the policy. \* \*\*Mobile Hotspotting\*\*: Disabling mobile hotspotting services prevents users from creating cellular hot-spotting configurations. This is a desired security behavior but can generate support tickets for mobile employees.

## ## Service Hardening Requirements

The following services must be stopped and disabled:

1. [REQ-END-037 - Disable Computer Browser Service \(Browser\)](#)
2. [REQ-END-038 - Disable Infrared Monitor Service \(irmon\)](#)
3. [REQ-END-039 - Disable Internet Connection Sharing \(ICS\) Service \(SharedAccess\)](#)
4. [REQ-END-040 - Disable LxssManager Service \(LxssManager\)](#)
5. [REQ-END-041 - Disable Microsoft FTP Service \(FTPSVC\)](#)
6. [REQ-END-042 - Disable OpenSSH SSH Server Service \(sshd\)](#)
7. [REQ-END-043 - Disable Remote Procedure Call \(RPC\) Locator Service \(RpcLocator\)](#)
8. [REQ-END-044 - Disable Routing and Remote Access Service \(RemoteAccess\)](#)
9. [REQ-END-045 - Disable Simple TCP/IP Services \(simptcp\)](#)
10. [REQ-END-046 - Disable Special Administration Console Helper Service \(sacsvr\)](#)
11. [REQ-END-047 - Disable SSDP Discovery Service \(SSDPsrv\)](#)
12. [REQ-END-048 - Disable UPnP Device Host Service \(upnphost\)](#)
13. [REQ-END-049 - Disable Web Management Service \(WMSvc\)](#)
14. [REQ-END-050 - Disable Windows Media Player Network Sharing Service \(WMPNetworkSvc\)](#)
15. [REQ-END-051 - Disable Windows Mobile Hotspot Service \(icssvc\)](#)
16. [REQ-END-052 - Disable World Wide Web Publishing Service \(W3SVC\)](#)
17. [REQ-END-053 - Disable Xbox Accessory Management Service \(XboxGipSvc\)](#)
18. [REQ-END-054 - Disable Xbox Live Auth Manager Service \(XblAuthManager\)](#)
19. [REQ-END-055 - Disable Xbox Live Game Save Service \(XblGameSave\)](#)
20. [REQ-END-056 - Disable Xbox Live Networking Service \(XboxNetApiSvc\)](#)

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.3, 5.8, 5.9, 5.11, 5.12, 5.14, 5.25, 5.27, 5.29, 5.31, 5.32, 5.33, 5.34, 5.37, 5.38, 5.43, 5.44 to 5.47 \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Services disable requirements

# [REQ-END-037] Disable Computer Browser Service (Browser)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\Browser\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Computer Browser (Browser) service directly supports this:

1. Maintains list of computers on network; legacy, insecure, and cleartext name discovery.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Computer Browser capabilities before domain-wide enforcement.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Computer Browser` ( `Browser` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableBrowser.ps1](#)

```
Configure-DisableBrowser.ps1
Description: Disables the unnecessary Computer Browser (Browser) service.

Write-Host "Applying hardening requirement: Disable Computer Browser service..." -ForegroundColor Cyan

$ServiceName = "Browser"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-BrowserStatus.ps1](#)

```
Get-BrowserStatus.ps1
Description: Audits the startup configuration of Computer Browser (Browser) service.

Write-Host "--- Auditing Computer Browser (Browser) Service ---" -ForegroundColor Cyan

$ServiceName = "Browser"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.3 (Browser) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-038] Disable Infrared monitor service Service (irmon)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\irmon\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Infrared monitor service (irmon) service directly supports this:

1. Supports infrared device communications; obsolete and exposes unnecessary communication port.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Infrared monitor service capabilities before domain-wide enforcement.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Infrared monitor service` ( `irmon` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disableirmon.ps1](#)

```
Configure-Disableirmon.ps1
Description: Disables the unnecessary Infrared monitor service (irmon) service.

Write-Host "Applying hardening requirement: Disable Infrared monitor service service..." -ForegroundColor Cyan

$ServiceName = "irmon"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-irmonStatus.ps1](#)

```
Get-irmonStatus.ps1
Description: Audits the startup configuration of Infrared monitor service (irmon) service.

Write-Host "--- Auditing Infrared monitor service (irmon) Service ---" -ForegroundColor Cyan

$ServiceName = "irmon"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.8 (irmon) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-039] Disable Internet Connection Sharing (ICS) Service (SharedAccess)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\SharedAccess\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Internet Connection Sharing (ICS) (SharedAccess) service directly supports this:

1. Provides NAT, addressing, and name resolution; represents security bridging risk.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Internet Connection Sharing (ICS) capabilities before domain-wide enforcement.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Internet Connection Sharing (ICS) (SharedAccess)`, double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableSharedAccess.ps1](#)

```
Configure-DisableSharedAccess.ps1
Description: Disables the unnecessary Internet Connection Sharing (ICS) (SharedAccess) service.

Write-Host "Applying hardening requirement: Disable Internet Connection Sharing (ICS) service..." -ForegroundColor Cyan

$ServiceName = "SharedAccess"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-SharedAccessStatus.ps1](#)



```
Get-SharedAccessStatus.ps1
Description: Audits the startup configuration of Internet Connection Sharing (ICS) (SharedAccess) service.

Write-Host "--- Auditing Internet Connection Sharing (ICS) (SharedAccess) Service ---" -ForegroundColor Cyan

$ServiceName = "SharedAccess"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\${$ServiceName}"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '${$ServiceName}' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '${$ServiceName}' startup type is not Disabled (Start value is ${$Start})." -Foregro
undColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '${$ServiceName}' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '${$ServiceName}' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.9 (SharedAccess) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-040] Disable LxssManager Service (LxssManager)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\LxssManager\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the LxssManager (LxssManager) service directly supports this:

1. Windows Subsystem for Linux (WSL); disables hosting unvetted container instances / developer Linux kernels on sensitive systems.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local LxssManager capabilities before domain-wide enforcement.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `LxssManager` ( `LxssManager` ), double-click to define the policy, and select **Disabled**.

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableLxssManager.ps1](#)

```
Configure-DisableLxssManager.ps1
Description: Disables the unnecessary LxssManager (LxssManager) service.

Write-Host "Applying hardening requirement: Disable LxssManager service..." -ForegroundColor Cyan

$ServiceName = "LxssManager"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-LxssManagerStatus.ps1](#)

```
Get-LxssManagerStatus.ps1
Description: Audits the startup configuration of LxssManager (LxssManager) service.

Write-Host "--- Auditing LxssManager (LxssManager) Service ---" -ForegroundColor Cyan

$ServiceName = "LxssManager"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*\*: Section 5.11 (LxssManager) \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*\*: Unnecessary services restrictions

# [REQ-END-041] Disable Microsoft FTP Service Service (FTPSVC)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\FTPSVC\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Microsoft FTP Service (FTPSVC) service directly supports this:

1. FTP service; insecure cleartext file transport protocol that should never run on client hosts.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Microsoft FTP Service capabilities before domain-wide enforcement.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Microsoft FTP Service` ( `FTPSVC` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableFTPSVC.ps1](#)

```
Configure-DisableFTPSVC.ps1
Description: Disables the unnecessary Microsoft FTP Service (FTPSVC) service.

Write-Host "Applying hardening requirement: Disable Microsoft FTP Service service..." -ForegroundColor Cyan

$ServiceName = "FTPSVC"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-FTPSVCStatus.ps1](#)

```
Get-FTPSVCStatus.ps1
Description: Audits the startup configuration of Microsoft FTP Service (FTPSVC) service.

Write-Host "--- Auditing Microsoft FTP Service (FTPSVC) Service ---" -ForegroundColor Cyan

$ServiceName = "FTPSVC"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.12 (FTPSVC) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-042] Disable OpenSSH SSH Server Service (sshd)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\sshd\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the OpenSSH SSH Server (sshd) service directly supports this:

1. OpenSSH server daemon; exposes command access ports to incoming connections.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local OpenSSH SSH Server capabilities before domain-wide enforcement.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `OpenSSH SSH Server` ( `sshd` ), double-click to define the policy, and select **Disabled**.

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disablesshd.ps1](#)

```
Configure-Disablesshd.ps1
Description: Disables the unnecessary OpenSSH SSH Server (sshd) service.

Write-Host "Applying hardening requirement: Disable OpenSSH SSH Server service..." -ForegroundColor Cyan

$ServiceName = "sshd"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-sshdStatus.ps1](#)

```
Get-sshdStatus.ps1
Description: Audits the startup configuration of OpenSSH SSH Server (sshd) service.

Write-Host "--- Auditing OpenSSH SSH Server (sshd) Service ---" -ForegroundColor Cyan

$ServiceName = "sshd"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.14 (sshd) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-043] Disable Remote Procedure Call (RPC) Locator Service (RpcLocator)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\RpcLocator\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Remote Procedure Call (RPC) Locator (RpcLocator) service directly supports this:

1. Legacy RPC name locator; obsolete and exposes legacy RPC APIs.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Remote Procedure Call (RPC) Locator capabilities before domain-wide enforcement.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Remote Procedure Call (RPC) Locator` ( `RpcLocator` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableRpcLocator.ps1](#)

```
Configure-DisableRpcLocator.ps1
Description: Disables the unnecessary Remote Procedure Call (RPC) Locator (RpcLocator) service.

Write-Host "Applying hardening requirement: Disable Remote Procedure Call (RPC) Locator service..." -ForegroundColor Cyan

$ServiceName = "RpcLocator"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-RpcLocatorStatus.ps1](#)



```
Get-RpcLocatorStatus.ps1
Description: Audits the startup configuration of Remote Procedure Call (RPC) Locator (RpcLocator) service.

Write-Host "--- Auditing Remote Procedure Call (RPC) Locator (RpcLocator) Service ---" -ForegroundColor Cyan

$ServiceName = "RpcLocator"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*\*: Section 5.25 (RpcLocator) \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*\*: Unnecessary services restrictions

# [REQ-END-044] Disable Routing and Remote Access Service (RemoteAccess)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

---

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\RemoteAccess\Start`

---

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Routing and Remote Access (RemoteAccess) service directly supports this:

1. Routing, NAT, and VPN; provides network routing capabilities that could bypass domain firewalls.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

---

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Routing and Remote Access capabilities before domain-wide enforcement.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
  2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
  3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
  4. Locate `Routing and Remote Access` ( `RemoteAccess` ), double-click to define the policy, and select **Disabled**.
- 

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableRemoteAccess.ps1](#)

```
Configure-DisableRemoteAccess.ps1
Description: Disables the unnecessary Routing and Remote Access (RemoteAccess) service.

Write-Host "Applying hardening requirement: Disable Routing and Remote Access service..." -ForegroundColor Cyan

$ServiceName = "RemoteAccess"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-RemoteAccessStatus.ps1](#)

```
Get-RemoteAccessStatus.ps1
Description: Audits the startup configuration of Routing and Remote Access (RemoteAccess) service.

Write-Host "--- Auditing Routing and Remote Access (RemoteAccess) Service ---" -ForegroundColor Cyan

$ServiceName = "RemoteAccess"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.27 (RemoteAccess) \* \*\*ANSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-045] Disable Simple TCP/IP Services Service (simptcp)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\simptcp\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Simple TCP/IP Services (simptcp) service directly supports this:

1. Character Generator, Daytime, Discard, Echo, and Quote of the Day; legacy UDP/TCP test ports vulnerable to amplification DDoS.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Simple TCP/IP Services capabilities before domain-wide enforcement.

### ## Implementation Steps

#### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Simple TCP/IP Services` ( `simptcp` ), double-click to define the policy, and select **Disabled**.

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disablesimptcp.ps1](#)

```
Configure-Disablesimptcp.ps1
Description: Disables the unnecessary Simple TCP/IP Services (simptcp) service.

Write-Host "Applying hardening requirement: Disable Simple TCP/IP Services service..." -ForegroundColor Cyan

$ServiceName = "simptcp"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-simptcpStatus.ps1](#)

```
Get-simptcpStatus.ps1
Description: Audits the startup configuration of Simple TCP/IP Services (simptcp) service.

Write-Host "--- Auditing Simple TCP/IP Services (simptcp) Service ---" -ForegroundColor Cyan

$ServiceName = "simptcp"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.29 (simptcp) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-046] Disable Special Administration Console Helper Service (sacsvr)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\sacsvr\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Special Administration Console Helper (sacsvr) service directly supports this:

1. Enables out-of-band serial port administration; unnecessary on endpoints.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Special Administration Console Helper capabilities before domain-wide enforcement.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Special Administration Console Helper` ( `sacsvr` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disablesacsvr.ps1](#)

```
Configure-Disablesacsvr.ps1
Description: Disables the unnecessary Special Administration Console Helper (sacsvr) service.

Write-Host "Applying hardening requirement: Disable Special Administration Console Helper service..." -ForegroundColor Cyan

$ServiceName = "sacsvr"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-sacsvrStatus.ps1](#)

```
Get-sacsvrStatus.ps1
Description: Audits the startup configuration of Special Administration Console Helper (sacsvr) service.

Write-Host "--- Auditing Special Administration Console Helper (sacsvr) Service ---" -ForegroundColor Cyan

$ServiceName = "sacsvr"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.31 (sacsvr) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-047] Disable SSDP Discovery Service (SSDPSRV)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\SSDPSRV\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the SSDP Discovery (SSDPSRV) service directly supports this:

1. Simple Service Discovery Protocol; listens for broadcast UDP name query advertisements, vulnerable to reflect DDOS and name spoofing.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local SSDP Discovery capabilities before domain-wide enforcement.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `SSDP Discovery` ( `SSDPSRV` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableSSDPSRV.ps1](#)

```
Configure-DisableSSDPSRV.ps1
Description: Disables the unnecessary SSDP Discovery (SSDPSRV) service.

Write-Host "Applying hardening requirement: Disable SSDP Discovery service..." -ForegroundColor Cyan

$ServiceName = "SSDPSRV"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-SSDPSRVStatus.ps1](#)



```
Get-SSDPSRVStatus.ps1
Description: Audits the startup configuration of SSDP Discovery (SSDPSRV) service.

Write-Host "--- Auditing SSDP Discovery (SSDPSRV) Service ---" -ForegroundColor Cyan

$ServiceName = "SSDPSRV"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.32 (SSDPSRV) \* \*\*ANSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-048] Disable UPnP Device Host Service (upnphost)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\upnphost\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the UPnP Device Host (upnphost) service directly supports this:

1. Universal Plug and Play; allows local hosts to advertise and control UPnP devices, opening network exposure.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local UPnP Device Host capabilities before domain-wide enforcement.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `UPnP Device Host` ( `upnphost` ), double-click to define the policy, and select **Disabled**.

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disableupnphost.ps1](#)

```
Configure-Disableupnphost.ps1
Description: Disables the unnecessary UPnP Device Host (upnphost) service.

Write-Host "Applying hardening requirement: Disable UPnP Device Host service..." -ForegroundColor Cyan

$ServiceName = "upnphost"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-upnphostStatus.ps1](#)

```
Get-upnphostStatus.ps1
Description: Audits the startup configuration of UPnP Device Host (upnphost) service.

Write-Host "--- Auditing UPnP Device Host (upnphost) Service ---" -ForegroundColor Cyan

$ServiceName = "upnphost"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.33 (upnphost) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-049] Disable Web Management Service Service (WMSvc)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\WMSvc\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Web Management Service (WMSvc) service directly supports this:

1. IIS Web Management; unnecessary web console access daemon on endpoint systems.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Web Management Service capabilities before domain-wide enforcement.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Web Management Service` ( `WMSvc` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableWMSvc.ps1](#)

```
Configure-DisableWMSvc.ps1
Description: Disables the unnecessary Web Management Service (WMSvc) service.

Write-Host "Applying hardening requirement: Disable Web Management Service service..." -ForegroundColor Cyan

$ServiceName = "WMSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-WMSvcStatus.ps1](#)

```
Get-WMSvcStatus.ps1
Description: Audits the startup configuration of Web Management Service (WMSvc) service.

Write-Host "--- Auditing Web Management Service (WMSvc) Service ---" -ForegroundColor Cyan

$ServiceName = "WMSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.34 (WMSvc) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-050] Disable Windows Media Player Network Sharing Service Service (WMPNetworkSvc)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\WMPNetworkSvc\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Windows Media Player Network Sharing Service (WMPNetworkSvc) service directly supports this:

1. Shares Windows Media Player libraries over network; unnecessary port listening.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Windows Media Player Network Sharing Service capabilities before domain-wide enforcement.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Windows Media Player Network Sharing Service (WMPNetworkSvc)`, double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableWMPNetworkSvc.ps1](#)

```
Configure-DisableWMPNetworkSvc.ps1
Description: Disables the unnecessary Windows Media Player Network Sharing Service (WMPNetworkSvc) service.

Write-Host "Applying hardening requirement: Disable Windows Media Player Network Sharing Service service..." -ForegroundColor Cyan

$ServiceName = "WMPNetworkSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-WMPNetworkSvcStatus.ps1](#)

```
Get-WMPNetworkSvcStatus.ps1
Description: Audits the startup configuration of Windows Media Player Network Sharing Service (WMPNetworkSvc) service.

Write-Host "--- Auditing Windows Media Player Network Sharing Service (WMPNetworkSvc) Service ---" -ForegroundColor Cyan

$ServiceName = "WMPNetworkSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.37 (WMPNetworkSvc) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-051] Disable Windows Mobile Hotspot Service Service (icssvc)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\icssvc\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Windows Mobile Hotspot Service (icssvc) service directly supports this:

1. Allows sharing Wi-Fi/Cellular network connections; exposes bridging hazards.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Windows Mobile Hotspot Service capabilities before domain-wide enforcement.

### ## Implementation Steps

#### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Windows Mobile Hotspot Service` ( `icssvc` ), double-click to define the policy, and select **Disabled**.

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-Disableicssvc.ps1](#)

```
Configure-Disableicssvc.ps1
Description: Disables the unnecessary Windows Mobile Hotspot Service (icssvc) service.

Write-Host "Applying hardening requirement: Disable Windows Mobile Hotspot Service service..." -ForegroundColor Cyan

$ServiceName = "icssvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-icssvcStatus.ps1](#)



```
Get-icssvcStatus.ps1
Description: Audits the startup configuration of Windows Mobile Hotspot Service (icssvc) service.

Write-Host "--- Auditing Windows Mobile Hotspot Service (icssvc) Service ---" -ForegroundColor Cyan

$ServiceName = "icssvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -Foregro
undColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.38 (icssvc) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-052] Disable World Wide Web Publishing Service Service (W3SVC)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\W3SVC\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the World Wide Web Publishing Service (W3SVC) service directly supports this:

1. IIS Web Server; web servers should never run on client workstations.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local World Wide Web Publishing Service capabilities before domain-wide enforcement.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `World Wide Web Publishing Service ( W3SVC )`, double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableW3SVC.ps1](#)

```
Configure-DisableW3SVC.ps1
Description: Disables the unnecessary World Wide Web Publishing Service (W3SVC) service.

Write-Host "Applying hardening requirement: Disable World Wide Web Publishing Service service..." -ForegroundColor Cyan

$ServiceName = "W3SVC"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-W3SVCStatus.ps1](#)

```
Get-W3SVCStatus.ps1
Description: Audits the startup configuration of World Wide Web Publishing Service (W3SVC) service.

Write-Host "--- Auditing World Wide Web Publishing Service (W3SVC) Service ---" -ForegroundColor Cyan

$ServiceName = "W3SVC"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*\*: Section 5.43 (W3SVC) \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*\*: Unnecessary services restrictions

# [REQ-END-053] Disable Xbox Accessory Management Service (XboxGipSvc)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\XboxGipSvc\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Xbox Accessory Management Service (XboxGipSvc) service directly supports this:

1. Manages Xbox accessories; gaming components irrelevant to corporate environments.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Xbox Accessory Management Service capabilities before domain-wide enforcement.

### ## Implementation Steps

#### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Xbox Accessory Management Service (XboxGipSvc)`, double-click to define the policy, and select **Disabled**.

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableXboxGipSvc.ps1](#)

```
Configure-DisableXboxGipSvc.ps1
Description: Disables the unnecessary Xbox Accessory Management Service (XboxGipSvc) service.

Write-Host "Applying hardening requirement: Disable Xbox Accessory Management Service service..." -ForegroundColor Cyan

$ServiceName = "XboxGipSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-XboxGipSvcStatus.ps1](#)

```
Get-XboxGipSvcStatus.ps1
Description: Audits the startup configuration of Xbox Accessory Management Service (XboxGipSvc) service.

Write-Host "--- Auditing Xbox Accessory Management Service (XboxGipSvc) Service ---" -ForegroundColor Cyan

$ServiceName = "XboxGipSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*\*: Section 5.44 (XboxGipSvc) \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*\*: Unnecessary services restrictions

# [REQ-END-054] Disable Xbox Live Auth Manager Service (XblAuthManager)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\XblAuthManager\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Xbox Live Auth Manager (XblAuthManager) service directly supports this:

1. Xbox Live authentication; gaming components irrelevant to corporate environments.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Xbox Live Auth Manager capabilities before domain-wide enforcement.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Xbox Live Auth Manager` ( `XblAuthManager` ), double-click to define the policy, and select **Disabled**.

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableXblAuthManager.ps1](#)

```
Configure-DisableXblAuthManager.ps1
Description: Disables the unnecessary Xbox Live Auth Manager (XblAuthManager) service.

Write-Host "Applying hardening requirement: Disable Xbox Live Auth Manager service..." -ForegroundColor Cyan

$ServiceName = "XblAuthManager"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-XblAuthManagerStatus.ps1](#)

```
Get-XblAuthManagerStatus.ps1
Description: Audits the startup configuration of Xbox Live Auth Manager (XblAuthManager) service.

Write-Host "--- Auditing Xbox Live Auth Manager (XblAuthManager) Service ---" -ForegroundColor Cyan

$ServiceName = "XblAuthManager"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.45 (XblAuthManager) \* \*\*ANSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-055] Disable Xbox Live Game Save Service (XblGameSave)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\XblGameSave\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Xbox Live Game Save (XblGameSave) service directly supports this:

1. Xbox Live game save synchronization; gaming components irrelevant to corporate environments.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Xbox Live Game Save capabilities before domain-wide enforcement.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Xbox Live Game Save` ( `XblGameSave` ), double-click to define the policy, and select **Disabled**.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableXblGameSave.ps1](#)

```
Configure-DisableXblGameSave.ps1
Description: Disables the unnecessary Xbox Live Game Save (XblGameSave) service.

Write-Host "Applying hardening requirement: Disable Xbox Live Game Save service..." -ForegroundColor Cyan

$ServiceName = "XblGameSave"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-XblGameSaveStatus.ps1](#)



```
Get-XblGameSaveStatus.ps1
Description: Audits the startup configuration of Xbox Live Game Save (XblGameSave) service.

Write-Host "--- Auditing Xbox Live Game Save (XblGameSave) Service ---" -ForegroundColor Cyan

$ServiceName = "XblGameSave"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.46 (XblGameSave) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

# [REQ-END-056] Disable Xbox Live Networking Service Service (XboxNetApiSvc)

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services` \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Services\XboxNetApiSvc\Start`

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary system services must be disabled. Disabling the Xbox Live Networking Service (XboxNetApiSvc) service directly supports this:

1. Xbox Live networking API; gaming components irrelevant to corporate environments.
2. By ensuring this service is disabled, we remove a potentially vulnerable network listener or local subsystem, decreasing both the remote and local exposure.

## Legacy Impact & Compatibility \* \*\*Normal Operations\*\*: Disabling this service is expected to be transparent for standard Active Directory domain operations unless specific business functionality requires the service. \* \*\*Verification\*\*: Administrators must verify that client operations do not rely on local Xbox Live Networking Service capabilities before domain-wide enforcement.

### ## Implementation Steps

#### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Locate `Xbox Live Networking Service` ( `XboxNetApiSvc` ), double-click to define the policy, and select **Disabled**.

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

[Download Script: Configure-DisableXboxNetApiSvc.ps1](#)

```
Configure-DisableXboxNetApiSvc.ps1
Description: Disables the unnecessary Xbox Live Networking Service (XboxNetApiSvc) service.

Write-Host "Applying hardening requirement: Disable Xbox Live Networking Service service..." -ForegroundColor Cyan

$ServiceName = "XboxNetApiSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"

$Service = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 if ($Service.StartType -ne "Disabled") {
 if ($Service.Status -eq "Running") {
 Stop-Service -Name $ServiceName -Force -ErrorAction SilentlyContinue | Out-Null
 }
 Set-Service -Name $ServiceName -StartupType Disabled -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Service '$($ServiceName)' stopped and disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Service '$($ServiceName)' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Service '$($ServiceName)' is not installed." -ForegroundColor Gray
}

if (Test-Path -Path $RegPath) {
 Set-ItemProperty -Path $RegPath -Name "Start" -Value 4 -Type DWord -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Registry Start value set to 4 (Disabled) for service '$($ServiceName)'." -ForegroundColor Green
}
```

To verify the startup type of this unnecessary service:

[Download Script: Get-XboxNetApiSvcStatus.ps1](#)

```
Get-XboxNetApiSvcStatus.ps1
Description: Audits the startup configuration of Xbox Live Networking Service (XboxNetApiSvc) service.

Write-Host "--- Auditing Xbox Live Networking Service (XboxNetApiSvc) Service ---" -ForegroundColor Cyan

$ServiceName = "XboxNetApiSvc"
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$($ServiceName)"
$IsVulnerable = $false

if (Test-Path -Path $RegPath) {
 $StartVal = Get-ItemProperty -Path $RegPath -Name "Start" -ErrorAction SilentlyContinue
 if ($null -ne $StartVal) {
 $Start = $StartVal.Start
 if ($Start -eq 4) {
 Write-Host "[+] Service '$($ServiceName)' is secure (Disabled)." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' startup type is not Disabled (Start value is $($Start))." -ForegroundColor Red
 $IsVulnerable = $true
 }
 } else {
 Write-Host "[!] VULNERABLE: Service '$($ServiceName)' exists but Start registry value is missing." -ForegroundColor Red
 $IsVulnerable = $true
 }
} else {
 Write-Host "[+] Service '$($ServiceName)' is not installed (Secure)." -ForegroundColor Green
}

if ($IsVulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 5.47 (XboxNetApiSvc) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on host service minimization \* \*\*DoD Windows 11 Computer STIG v2r6\*\*: Unnecessary services restrictions

## # [REQ-END-025] Configure Secure Printing and Print Spooler Policies

## Target Scope \* \*\*Applicable Systems\*\* : Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\* : Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\* : High \* \*\*GPO Path / Registry Location\*\* : \* \*\*System Service GPO\*\* : 'Computer Configuration\Policies\Windows Settings\Security Settings\System Services\Print Spooler' → Service Startup Mode: \*\*Disabled\*\* \* \*\*Limits print driver installation to Administrators GPO\*\* : 'Computer Configuration\Policies\Administrative Templates\Printers\Limits print driver installation to Administrators' → Enabled \* \*\*Registry Location (Service)\*\* : 'HKLM\SYSTEM\CurrentControlSet\Services\Spooler' → 'Start' = '4' (REG\_DWORD) \* \*\*Registry Location (Printers)\*\* : 'HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Printers' \* \*\*Registry Location (Point and Print)\*\* : 'HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Printers\PointAndPrint' → 'RestrictDriverInstallationToAdministrators' = '1' (REG\_DWORD) \* \*\*Registry Location (Print Control)\*\* : 'HKLM\SYSTEM\CurrentControlSet\Control\Print'

## Rationale The Windows Print Spooler service ('Spooler') has been the source of numerous high-severity vulnerabilities (such as the PrintNightmare family - CVE-2021-1675 and CVE-2021-34527). Attackers exploit the Print Spooler to coerce authentication or execute arbitrary code with SYSTEM privileges.

To secure standard client endpoints and member servers, the primary defense is to **completely disable the Print Spooler service**. This eliminates the service's attack surface. As a secondary defense-in-depth, additional printer registry configurations (such as restricting driver installation to administrators, Redirection Guard, and RPC connection configurations) are enforced to ensure that even if the spooler service is temporarily running, the subsystem remains hardened.

1. **Remote Connections Block**: By disabling remote client connections to the print spooler, standard client endpoints are prevented from acting as print servers. Outbound printing remains unaffected, but external hosts can no longer target the workstation's print spooler over the network.
2. **Redirection Guard**: Enabling Redirection Guard prevents print spooler processing from being redirected via symbolic links or junction points, mitigating local privilege escalation vectors that abuse file system paths during printer driver mapping.
3. **RPC over TCP**: Forcing both incoming and outgoing RPC connections to use TCP instead of legacy Named Pipes (which can be easily hijacked or relayed) reduces the attack surface. Forcing packet-level privacy and authentication protocols ensures printing traffic is encrypted and authenticated.
4. **Point and Print Restrictions**: Restricting driver installation and update prompts to administrators prevents non-privileged users from installing malicious or unverified printer drivers.

## Legacy Impact & Compatibility \* \*\*No Printing Support\*\* : Printing from client workstations is completely disabled. Standard users will be unable to print documents to network or local printers. This matches the security requirements of high-security air-gapped environments where physical document flows must be restricted.

### ## Implementation Steps

#### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### 1. Disable the Print Spooler Service 1. Open the **Group Policy Management Console** ('gpmc.msc'). 2. Edit the GPO applied to client endpoints (e.g., 'GPO\_Hardening\_Endpoints'). 3. Navigate to: 'Computer Configuration\Policies\Windows Settings\Security Settings\System Services' 4. Double-click **Print Spooler**. 5. Check **Define this policy setting** and select **Disabled**. Click **OK**.

##### 2. Configure Printers GPO Hardening (Secondary Defense) 1. Navigate to: 'Computer Configuration\Policies\Administrative Templates\Printers' 2. Configure the following policies: \* **Allow Print Spooler to accept client connections**: Set to 'Disabled' \* **Configure Redirection Guard**: Set to 'Enabled', select 'Redirection Guard Enabled' \* **Configure RPC connection settings**: Set to 'Enabled' \* Protocol to use for outgoing RPC connections: 'RPC over TCP' \* Use authentication for outgoing RPC connections: 'Default' \* **Configure RPC listener settings**: Set to 'Enabled' \* Protocols to allow for incoming RPC connections: 'RPC over TCP' \* Authentication protocol to use for incoming RPC connections: 'Negotiate' (or higher) \* **Configure RPC over TCP port**: Set to 'Enabled', set Port to '0' (dynamic port allocation) \* **Configure RPC packet level privacy setting for incoming connections**: Set to 'Enabled' \*(Note: Requires 'SecGuide.admx' template)\* \* **Manage processing of Queue-specific files**: Set to 'Enabled', select 'Limit Queue-specific files to Color profiles' \* **Point and Print Restrictions**: Set to 'Enabled' \* When installing drivers for a new connection: 'Show warning and elevation prompt' \* When updating drivers for an existing connection: 'Show warning and elevation prompt' \* **Limits print driver installation to Administrators**: Set to 'Enabled'

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script locally to configure registry keys for print spooler hardening.

[Download Script: Configure-PrintingAndSpooler.ps1](#)

```
Configure-PrintingAndSpooler.ps1
Description: Disables the Print Spooler service and configures secondary print registry hardening parameters on standard endpoint
s.

Write-Host "Applying Print Spooler security hardening..." -ForegroundColor Cyan

1. Disable the Print Spooler Service
if (Get-Service -Name "Spooler" -ErrorAction SilentlyContinue) {
 Set-Service -Name "Spooler" -StartupType Disabled -Confirm:$false
 Stop-Service -Name "Spooler" -Force -Confirm:$false
 Write-Host "[+] Print Spooler service has been stopped and disabled." -ForegroundColor Green
}

1. Base Printers Path Policies
$PrintersPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers"
if (-not (Test-Path $PrintersPath)) {
 New-Item -Path $PrintersPath -Force | Out-Null
}

Allow Print Spooler to accept client connections → Disabled
Set-ItemProperty -Path $PrintersPath -Name "RegisterSpoolerRemoteRpcEndPoint" -Value 2 -Type Dword
Set-ItemProperty -Path $PrintersPath -Name "RegisterSpoolerRemoteSubsystem" -Value 0 -Type Dword

Configure Redirection Guard → Enabled: Redirection Guard Enabled
Set-ItemProperty -Path $PrintersPath -Name "RedirectionGuardPolicy" -Value 1 -Type Dword

Manage processing of Queue-specific files → Enabled: Limit Queue-specific files to Color profiles
Set-ItemProperty -Path $PrintersPath -Name "CopyFilesPolicy" -Value 1 -Type Dword

2. Printers RPC Connection and Listener Policies
$PrintersRpcPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers\RPC"
if (-not (Test-Path $PrintersRpcPath)) {
 New-Item -Path $PrintersRpcPath -Force | Out-Null
}

Protocol to use for outgoing RPC connections → RPC over TCP (0)
Set-ItemProperty -Path $PrintersRpcPath -Name "RpcUseNamedPipeProtocol" -Value 0 -Type Dword

Use authentication for outgoing RPC connections → Default (0)
Set-ItemProperty -Path $PrintersRpcPath -Name "RpcAuthentication" -Value 0 -Type Dword

Protocols to allow for incoming RPC connections → RPC over TCP (5)
Set-ItemProperty -Path $PrintersRpcPath -Name "RpcProtocols" -Value 5 -Type Dword

Authentication protocol to use for incoming RPC connections → Negotiate (0)
Set-ItemProperty -Path $PrintersRpcPath -Name "ForceKerberosForRpc" -Value 0 -Type Dword

Configure RPC over TCP port → 0
Set-ItemProperty -Path $PrintersRpcPath -Name "RpcTcpPort" -Value 0 -Type Dword

3. System Print Control Privacy Setting
$PrintControlPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Print"
if (-not (Test-Path $PrintControlPath)) {
 New-Item -Path $PrintControlPath -Force | Out-Null
}

Configure RPC packet level privacy setting for incoming connections → Enabled
Set-ItemProperty -Path $PrintControlPath -Name "RpcAuthnLevelPrivacyEnabled" -Value 1 -Type Dword

4. Point and Print Restrictions
$PointPrintPath = "HKLM:\Software\Policies\Microsoft\Windows NT\Printers\PointAndPrint"
if (-not (Test-Path $PointPrintPath)) {
 New-Item -Path $PointPrintPath -Force | Out-Null
}

Set-ItemProperty -Path $PointPrintPath -Name "RestrictPointAndPrint" -Value 1 -Type Dword
Set-ItemProperty -Path $PointPrintPath -Name "NoWarningNoElevationOnInstall" -Value 0 -Type Dword
Set-ItemProperty -Path $PointPrintPath -Name "UpdatePromptSettings" -Value 0 -Type Dword
Set-ItemProperty -Path $PointPrintPath -Name "RestrictDriverInstallationToAdministrators" -Value 1 -Type Dword

Write-Host "[+] Print Spooler and Printer configurations hardened successfully." -ForegroundColor Green
```

*To verify the print spooler policy settings:*

[Download Script: Get-PrintingAndSpoolerStatus.ps1](#)

```

Get-PrintingAndSpoolerStatus.ps1
Description: Audits print spooler and printer security configurations on the local system.

Write-Host "--- Auditing Printing and Spooler Hardening ---" -ForegroundColor Cyan

$script:Vulnerable = $false

1. Audit Spooler Service Startup Type
$Service = Get-Service -Name "Spooler" -ErrorAction SilentlyContinue
if ($null -ne $Service) {
 $StartupType = (Get-CimInstance -ClassName Win32_Service -Filter "Name='Spooler'").StartMode
 $Color = if ($StartupType -eq "Disabled") { "Green" } else { "Red" }
 Write-Host " [-] Print Spooler Service Startup: $StartupType (Expected: Disabled)" -ForegroundColor $Color
 if ($StartupType -ne "Disabled") {
 $script:Vulnerable = $true
 }
} else {
 Write-Host " [+] Print Spooler Service is not present on this machine." -ForegroundColor Green
}

Helper function to audit registry properties
function Test-RegistryValue {
 param(
 [string]$Path,
 [string]$Name,
 [object]$ExpectedValue
)
 if (Test-Path $Path) {
 $Val = Get-ItemProperty -Path $Path -Name $Name -ErrorAction SilentlyContinue
 if ($null -ne $Val) {
 $Actual = $Val.$Name
 if ($Actual -eq $ExpectedValue) {
 Write-Host " - Path: $Path | Value: $Name | Current: $Actual (Expected: $ExpectedValue)" -ForegroundColor Green
 } else {
 Write-Host " [!] MISMATCH: Path: $Path | Value: $Name | Current: $Actual (Expected: $ExpectedValue)" -ForegroundColorRed
 $script:Vulnerable = $true
 }
 } else {
 Write-Host " [!] MISSING VALUE: Path: $Path | Value: $Name (Expected: $ExpectedValue)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
 } else {
 Write-Host " [!] MISSING KEY: Path: $Path (Expected: $Name = $ExpectedValue)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
}

Audit base printer settings
$PrintersPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers"
Test-RegistryValue -Path $PrintersPath -Name "RegisterSpoolerRemoteRpcEndPoint" -ExpectedValue 2
Test-RegistryValue -Path $PrintersPath -Name "RegisterSpoolerRemoteSubsystem" -ExpectedValue 0
Test-RegistryValue -Path $PrintersPath -Name "RedirectionGuardPolicy" -ExpectedValue 1
Test-RegistryValue -Path $PrintersPath -Name "CopyFilesPolicy" -ExpectedValue 1

Audit RPC settings
$PrintersRpcPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers\RPC"
Test-RegistryValue -Path $PrintersRpcPath -Name "RpcUseNamedPipeProtocol" -ExpectedValue 0
Test-RegistryValue -Path $PrintersRpcPath -Name "RpcAuthentication" -ExpectedValue 0
Test-RegistryValue -Path $PrintersRpcPath -Name "RpcProtocols" -ExpectedValue 5
Test-RegistryValue -Path $PrintersRpcPath -Name "ForceKerberosForRpc" -ExpectedValue 0
Test-RegistryValue -Path $PrintersRpcPath -Name "RpcTcpPort" -ExpectedValue 0

Audit Print Control
$PrintControlPath = "HKLM:\SYSTEM\CurrentControlSet\Control\Print"
Test-RegistryValue -Path $PrintControlPath -Name "RpcAuthnLevelPrivacyEnabled" -ExpectedValue 1

Audit Point and Print
$PointPrintPath = "HKLM:\Software\Policies\Microsoft\Windows NT\Printers\PointAndPrint"
Test-RegistryValue -Path $PointPrintPath -Name "RestrictPointAndPrint" -ExpectedValue 1
Test-RegistryValue -Path $PointPrintPath -Name "NoWarningNoElevationOnInstall" -ExpectedValue 0
Test-RegistryValue -Path $PointPrintPath -Name "UpdatePromptSettings" -ExpectedValue 0
Test-RegistryValue -Path $PointPrintPath -Name "RestrictDriverInstallationToAdministrators" -ExpectedValue 1

```

```
if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 18.7.1 to 18.7.8, Section 18.7.10 to 18.7.12 (Printing security settings) \* \*\*ANSSI Active Directory Hardening Guide\*\*: Recommendations on disabling spooler remote calls to prevent coercive authentication \* \*\*Microsoft Security Guidance\*\*: Point and Print Restrictions (CVE-2021-34527 mitigation details)



**# [REQ-END-026] Configure System Administrative Templates**

**## Target Scope** \* **Applicable Systems**: Tier 2 client workstations and member servers. \* **Operating Systems**: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

**## Implementation Details** \* **Priority**: Medium \* **GPO Path / Registry Location**: \* **GPO Path**: `Computer Configuration\Policies\Administrative Templates\...` \* **Registry Location**: Multiple locations under `HKLM\SOFTWARE\Policies` and `HKLM\SYSTEM\CurrentControlSet` (see details below)

**## Rationale** Administrative templates govern system-wide capabilities, behaviors, and diagnostic logging. Hardening these configurations reduces the attack surface and mitigates privilege escalation, credential theft, and unauthorized software installation:

1. **Protocols Hardening**: Disabling legacy SMBv1 client/server components prevents exploitation of known protocol flaws. Configuring NetBIOS NodeType to P-Node prevents name resolution fallback issues.
2. **Data Collection & Telemetry**: Disabling Insider builds, telemetry feedback, widgets, Cortana, and OneSettings downloads blocks potential information disclosure paths and aligns with clean enterprise environments.
3. **App and Installer Restrictions**: Preventing non-admin users from installing packaged apps, limiting App Installer protocol handlers ( `ms-appinstaller` ), and disabling experimental installer features mitigates malware installation vectors.
4. **Event Log Sizes**: Increasing maximum log file sizes (Application/Setup/System to 32,768 KB, Security to 196,608 KB) ensures security events are retained long enough for compliance auditing and forensic analysis.
5. **Windows Update Controls**: Restricting update pauses and configuring daily update checks ensure client systems remain persistently patched.

**## Legacy Impact & Compatibility** \* **App Installers**: Disabling `ms-appinstaller` protocol handlers will block users from web-installing applications through the Appx installer interface. \* **IE11 Standalone**: Disabling Internet Explorer 11 blocks the standalone browser, redirecting users to Microsoft Edge.

**## Implementation Steps****### Option A: Group Policy Object (GPO) Configuration (Preferred)**

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on an administrative workstation.
2. Edit or create a GPO linked to endpoints (e.g., `GPO_Hardening_Endpoints_SystemTemplates` ).
3. Configure the following policies grouped by their GPO nodes:

**#### Network & TCP/IP Settings** \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Network\Lanman Workstation` \* **Configure SMB v1 client driver**: Set to `Enabled`, select `Disable driver (recommended)` (Recommendation 18.4.2) \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Network\Lanman Server` \* **Configure SMB v1 server**: Set to `Disabled` (Recommendation 18.4.3) \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Network\TCPIP Settings\Parameters` \* **NetBT NodeType configuration**: Set to `Enabled`, select `P-node (recommended)` (Recommendation 18.4.7)

**#### Legacy Security Options (MSS Settings)** \* Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` \* Configure the following security settings (alternatively, deploy via GPO Preferences Registry if Custom ADMX templates are not available): \* **MSS: (AutoAdminLogon) Enable Automatic Logon**: Set to `Disabled` (Recommendation 18.5.1) \* **MSS: (DisableIPSourceRouting IPv6) IP source routing protection level**: Set to `Enabled: Highest protection, source routing is completely disabled` (Recommendation 18.5.2) \* **MSS: (DisableIPSourceRouting) IP source routing protection level**: Set to `Enabled: Highest protection, source routing is completely disabled` (Recommendation 18.5.3) \* **MSS: (EnableICMPRedirect) Allow ICMP redirects to override OSPF generated routes**: Set to `Disabled` (Recommendation 18.5.5) \* **MSS: (NoNameReleaseOnDemand) Allow the computer to ignore NetBIOS name release requests except from WINS servers**: Set to `Enabled` (Recommendation 18.5.7) \* **MSS: (SafeDllSearchMode) Enable Safe DLL search mode**: Set to `Enabled` (Recommendation 18.5.9) \* **MSS: (ScreenSaverGracePeriod) The time in seconds before the screen saver grace period expires**: Set to `Enabled: 5 or fewer seconds` (Recommendation 18.5.10) \* **MSS: (WarningLevel) Percentage threshold for the security event log at which the system will generate a warning**: Set to `Enabled: 90% or less` (Recommendation 18.5.13)

**#### System & Group Policy Settings** \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Device Installation` \* **Prevent device metadata retrieval from the Internet**: Set to `Enabled` (Recommendation 18.9.7.2) \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Group Policy` \* **Configure registry policy processing**: Set to `Enabled` \* Uncheck: `Do not apply during periodic background processing` (Recommendation 18.9.19.2) \* Check: `Process even if the Group Policy objects have not changed` (Recommendation 18.9.19.3) \* **Configure security policy processing**: Set to `Enabled` \* Uncheck: `Do not apply during periodic background processing` (Recommendation 18.9.19.4) \* Check: `Process even if the Group Policy objects have not changed` (Recommendation 18.9.19.5) \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Cross-Device Experiences` \* **Continue experiences on this device**: Set to `Disabled` (Recommendation 18.9.19.6) \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Internet Communication Management\Internet Communication settings` \* **Turn off downloading of print drivers over HTTP**: Set to `Enabled` (Recommendation 18.9.20.1.2) \* **Turn off Internet download for Web publishing and online ordering wizards**: Set to `Enabled` (Recommendation 18.9.20.1.6) \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Local Security Authority` \* **Allow Custom SSPs and APs to be loaded into LSASS**: Set to `Disabled` (Recommendation 18.9.26.1) \* Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Logon` \* **Block user from showing account details on sign-in**: Set to `Enabled` (Recommendation 18.9.28.1) \* **Do not display network selection UI**: Set to `Enabled` (Recommendation 18.9.28.2) \* **Do not enumerate connected users on domain-joined computers**: Set to `Enabled` (Recommendation 18.9.28.3) \* **Turn off app notifications on the lock**

screen\*\*: Set to 'Enabled' (Recommendation 18.9.28.5) \* \*\*Turn off picture password sign-in\*\*: Set to 'Enabled' (Recommendation 18.9.28.6) \* \*\*Turn on convenience PIN sign-in\*\*: Set to 'Disabled' (Recommendation 18.9.28.7) \* \*\*Prevent the use of security questions for local accounts\*\*: Set to 'Enabled' (Recommendation 18.10.15.3) \* \*\*Configure the transmission of the user's password in the content of MPR notifications sent by winlogon\*\*: Set to 'Disabled' (Recommendation 18.10.82.1) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\System\Power Management\Sleep Settings' \* \*\*Allow network connectivity during connected-standby (on battery)\*\*: Set to 'Disabled' (Recommendation 18.9.33.6.1) \* \*\*Allow network connectivity during connected-standby (plugged in)\*\*: Set to 'Disabled' (Recommendation 18.9.33.6.2) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\System\Remote Assistance' \* \*\*Configure Offer Remote Assistance\*\*: Set to 'Disabled' (Recommendation 18.9.35.1) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\System\Remote Procedure Call' \* \*\*Enable RPC Endpoint Mapper Client Authentication\*\*: Set to 'Enabled' (Recommendation 18.9.36.1) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\System\Windows Time Service\Time Providers' \* \*\*Enable Windows NTP Client\*\*: Set to 'Enabled' (Recommendation 18.9.51.1.1) \* \*\*Enable Windows NTP Server\*\*: Set to 'Disabled' (Recommendation 18.9.51.1.2)

#### Windows Components Settings \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\App Package Deployment' \* \*\*Not allow per-user unsigned packages to install by default (requires explicitly allow per install)\*\*: Set to 'Enabled' (Recommendation 18.10.4.2) \* \*\*Prevent non-admin users from installing packaged Windows apps\*\*: Set to 'Enabled' (Recommendation 18.10.4.3) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Biometrics\Facial Features' \* \*\*Configure enhanced anti-spoofing\*\*: Set to 'Enabled' (Recommendation 18.10.9.1.1) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Cloud Content' \* \*\*Turn off cloud consumer account state content\*\*: Set to 'Enabled' (Recommendation 18.10.13.1) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Connect' \* \*\*Require pin for pairing\*\*: Set to 'Enabled' (Select 'First Time' or 'Always') (Recommendation 18.10.14.1) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Credential User Interface' \* \*\*Do not display the password reveal button\*\*: Set to 'Enabled' (Recommendation 18.10.15.1) \* \*\*Enumerate administrator accounts on elevation\*\*: Set to 'Disabled' (Recommendation 18.10.15.2) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Data Collection and Preview Builds' \* \*\*Disable OneSettings Downloads\*\*: Set to 'Enabled' (Recommendation 18.10.16.3) \* \*\*Do not show feedback notifications\*\*: Set to 'Enabled' (Recommendation 18.10.16.4) \* \*\*Enable OneSettings Auditing\*\*: Set to 'Enabled' (Recommendation 18.10.16.5) \* \*\*Limit Diagnostic Log Collection\*\*: Set to 'Enabled' (Recommendation 18.10.16.6) \* \*\*Limit Dump Collection\*\*: Set to 'Enabled' (Recommendation 18.10.16.7) \* \*\*Toggle user control over Insider builds\*\*: Set to 'Disabled' (Recommendation 18.10.16.8) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\App Installer' \* \*\*Enable App Installer Experimental Features\*\*: Set to 'Disabled' (Recommendation 18.10.18.2) \* \*\*Enable App Installer Hash Override\*\*: Set to 'Disabled' (Recommendation 18.10.18.3) \* \*\*Enable App Installer Local Archive Malware Scan Override\*\*: Set to 'Disabled' (Recommendation 18.10.18.4) \* \*\*Enable App Installer Microsoft Store Source Certificate Validation Bypass\*\*: Set to 'Disabled' (Recommendation 18.10.18.5) \* \*\*Enable App Installer ms-appinstaller protocol\*\*: Set to 'Disabled' (Recommendation 18.10.18.6) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Event Log Service\Application' \* \*\*Control Event Log behavior when the log file reaches its maximum size\*\*: Set to 'Disabled' (Recommendation 18.10.26.1.1) \* \*\*Specify the maximum log file size (KB)\*\*: Set to 'Enabled', set maximum log size to '32768' (Recommendation 18.10.26.1.2) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Event Log Service\Security' \* \*\*Control Event Log behavior when the log file reaches its maximum size\*\*: Set to 'Disabled' (Recommendation 18.10.26.2.1) \* \*\*Specify the maximum log file size (KB)\*\*: Set to 'Enabled', set maximum log size to '196608' (Recommendation 18.10.26.2.2) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Event Log Service\Setup' \* \*\*Control Event Log behavior when the log file reaches its maximum size\*\*: Set to 'Disabled' (Recommendation 18.10.26.3.1) \* \*\*Specify the maximum log file size (KB)\*\*: Set to 'Enabled', set maximum log size to '32768' (Recommendation 18.10.26.3.2) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Event Log Service\System' \* \*\*Control Event Log behavior when the log file reaches its maximum size\*\*: Set to 'Disabled' (Recommendation 18.10.26.4.1) \* \*\*Specify the maximum log file size (KB)\*\*: Set to 'Enabled', set maximum log size to '32768' (Recommendation 18.10.26.4.2) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\File Explorer' \* \*\*Do not apply the Mark of the Web tag to files copied from insecure sources\*\*: Set to 'Disabled' (Recommendation 18.10.29.3) \* \*\*Turn off shell protocol protected mode\*\*: Set to 'Disabled' (Recommendation 18.10.29.5) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Internet Explorer' \* \*\*Disable Internet Explorer 11 as a standalone browser\*\*: Set to 'Enabled', select 'Always' (Recommendation 18.10.35.1) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Internet Explorer\Feeds' \* \*\*Prevent downloading of enclosures\*\*: Set to 'Enabled' (Recommendation 18.10.58.1) \* \*\*Turn on Basic feed authentication over HTTP\*\*: Set to 'Disabled' (Recommendation 18.10.58.2) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Defender Antivirus\Remediation\Behavioral Network Blocks\Brute Force Protection' \* \*\*Configure Remote Encryption Protection Mode\*\*: Set to 'Enabled' (Select 'Audit' or higher) (Recommendation 18.10.43.11.1.2) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Defender Antivirus\Scan' \* \*\*Turn off scanning of packed executables\*\*: Set to 'Disabled' (Recommendation 18.10.43.13.2) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Defender Security Center\App and Browser protection' \* \*\*Prevent users from modifying settings\*\*: Set to 'Enabled' (Recommendation 18.10.92.2.1) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Search' \* \*\*Allow Cortana\*\*: Set to 'Disabled' (Recommendation 18.10.59.3) \* \*\*Allow Cortana above lock screen\*\*: Set to 'Disabled' (Recommendation 18.10.59.4) \* \*\*Allow indexing of encrypted files\*\*: Set to 'Disabled' (Recommendation 18.10.59.5) \* \*\*Allow search and Cortana to use location\*\*: Set to 'Disabled' (Recommendation 18.10.59.6) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Store' \* \*\*Turn off Automatic Download and Install of updates\*\*: Set to 'Disabled' (Recommendation 18.10.66.2) \* \*\*Turn off the offer to update to the latest version of Windows\*\*: Set to 'Enabled' (Recommendation 18.10.66.3) \* Navigate to: 'Computer Configuration\Policies\Administrative Templates\Windows Components\Widgets' \* \*\*Allow widgets\*\*: Set to 'Disabled' (Recommendation 18.10.72.1) \* Navigate to: 'Computer

Configuration\Policies\Administrative Templates\Windows Components\Windows Logon Options` \* \*\*Sign-in and lock last interactive user automatically after a restart\*\*`: Set to `Disabled` (Recommendation 18.10.82.2) \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Sandbox` \* \*\*Allow clipboard sharing with Windows Sandbox\*\*`: Set to `Disabled` (Recommendation 18.10.91.1) \* \*\*Allow networking in Windows Sandbox\*\*`: Set to `Disabled` (Recommendation 18.10.91.2)

#### Windows Update Settings \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Update` (or `Windows Update\Manage end user experience` depending on ADMX version) \* \*\*Remove access to "Pause updates" feature\*\*`: Set to `Enabled` (Recommendation 18.10.93.2.3) \* \*\*Manage preview builds\*\*`: Set to `Disabled` (Recommendation 18.10.93.4.1) \* \*\*Select when Preview Builds and Feature Updates are received\*\*`: Set to `Enabled`, set *Defer Feature Updates Period in Days* to `180` (or more) (Recommendation 18.10.93.4.2) \* \*\*Select when Quality Updates are received\*\*`: Set to `Enabled`, set *Defer Quality Updates Period in Days* to `0` (Recommendation 18.10.93.4.3) \* Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows Update\Manage end user experience` (or standard `Windows Update\AU` depending on ADMX version) \* \*\*Configure Automatic Updates\*\*`: Set to `Enabled`, select *Scheduled install day* = `0 - Every day` (Recommendation 18.10.93.2.2) \* \*\*No auto-restart with logged on users for scheduled automatic updates installations\*\*`: Set to `Disabled` (Recommendation 18.10.93.1.1)

4. Link the GPO to the target Organizational Unit (OU) containing workstations and member servers.

---

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script locally to configure the administrative templates registry values.

[Download Script: Configure-SystemAdministrativeTemplates.ps1](#)

```
Configure-SystemAdministrativeTemplates.ps1
Description: Configures 84 system and administrative template controls for Windows Client hardening.

Write-Host "Applying System Administrative Templates hardening..." -ForegroundColor Cyan

Key Path: HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon")) {
 New-Item -Path "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" -Name "AutoAdminLogon" -Value "0" -Type String
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" -Name "ScreenSaverGracePeriod" -Value 5 -Type DWord

Key Path: HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\CredUI
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\CredUI")) {
 New-Item -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\CredUI" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\CredUI" -Name "EnumerateAdministrators" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer")) {
 New-Item -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" -Name "NoWebServices" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" -Name "PreXPSP2ShellProtocolBehavior" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System
if (-not (Test-Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System")) {
 New-Item -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" -Name "EnableMPR" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" -Name "DisableAutomaticRestartSignOn" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Biometrics\FacialFeatures
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Biometrics\FacialFeatures")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Biometrics\FacialFeatures" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Biometrics\FacialFeatures" -Name "EnhancedAntiSpoofing" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Dsh
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Dsh")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Dsh" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Dsh" -Name "AllowNewsAndInterests" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" -Name "DisableEnclosureDownload" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Internet Explorer\Main
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Internet Explorer\Main")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Internet Explorer\Main" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Internet Explorer\Main" -Name "NotifyDisableIEOptions" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Power\PowerSettings\{f15576e8-98b7-4186-b944-eafa664402d9}
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\{f15576e8-98b7-4186-b944-eafa664402d9}")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\{f15576e8-98b7-4186-b944-eafa664402d9}" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\{f15576e8-98b7-4186-b944-eafa664402d9}" -Name "DCSettingIndex" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Power\PowerSettings\{f15576e8-98b7-4186-b944-eafa664402d9}" -Name "ACSettingIndex" -Value 0 -Type DWord
```



```

Key Path: HKLM\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpClient
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpClient")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpClient" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpClient" -Name "Enabled" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpServer
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpServer")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpServer" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpServer" -Name "Enabled" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\App and Browser protection
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\App and Browser protection")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\App and Browser protection" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender Security Center\App and Browser protection" -Name "DisallowExploitProtectionOverride" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Brute Force Protection
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Brute Force Protection")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Brute Force Protection" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\Behavioral Network Blocks\Brute Force Protection" -Name "BruteForceProtectionConfiguredState" -Value 2 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows Defender\Scan
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -Name "DisablePackedExeScanning" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Printers
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Printers" -Name "DisableWebPnPDownload" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Rpc
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Rpc")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Rpc" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Rpc" -Name "EnableAuthEpResolution" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services" -Name "fAllowUnsolicited" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\WindowsStore
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsStore")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsStore" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsStore" -Name "AutoDownload" -Value 4 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\WindowsStore" -Name "DisableOSUpgrade" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\AppInstaller
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -Name "EnableExperimentalFeatures" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -Name "EnableHashOverride" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -Name "EnableLocalArchiveMalwareScanOverride" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -Name "EnableBypassCertificatePinningForMicrosoftStore" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -Name "EnableMSAppInstallerProtocol" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Appx
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Appx")) {

```

```

 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Appx" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Appx" -Name "DisablePerUserUnsignedPackagesByDefault" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Appx" -Name "BlockNonAdminUserInstall" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\CloudContent
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CloudContent")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CloudContent" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CloudContent" -Name "DisableConsumerAccountStateContent" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Connect
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Connect")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Connect" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Connect" -Name "RequirePinForPairing" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\CredUI
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CredUI")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CredUI" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\CredUI" -Name "DisablePasswordReveal" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\DataCollection
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Name "DisableOneSettingsDownloads" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Name "DoNotShowFeedbackNotifications" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Name "EnableOneSettingsAuditing" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Name "LimitDiagnosticLogCollection" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Name "LimitDumpCollection" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Device Metadata
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Device Metadata")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Device Metadata" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Device Metadata" -Name "PreventDeviceMetadataFromNetwork" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\EventLog\Application
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Application")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Application" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Application" -Name "Retention" -Value "0" -Type String
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Application" -Name "MaxSize" -Value 32768 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\EventLog\Security
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Security")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Security" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Security" -Name "Retention" -Value "0" -Type String
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Security" -Name "MaxSize" -Value 196608 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup" -Name "Retention" -Value "0" -Type String
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup" -Name "MaxSize" -Value 32768 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\EventLog\System
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\System")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\System" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\System" -Name "Retention" -Value "0" -Type String
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\EventLog\System" -Name "MaxSize" -Value 32768 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Explorer
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Explorer")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Explorer" -Force | Out-Null
}

```

```

}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Explorer" -Name "DisableMotWOnInsecurePathCopy" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" -Name "NoBackgroundPolicy" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" -Name "NoGPOListChanges" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}" -Name "NoBackgroundPolicy" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}" -Name "NoGPOListChanges" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\PreviewBuilds
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PreviewBuilds")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PreviewBuilds" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PreviewBuilds" -Name "AllowBuildPreview" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Sandbox
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Sandbox")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Sandbox" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Sandbox" -Name "AllowClipboardRedirection" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Sandbox" -Name "AllowNetworking" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\System
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "EnableCdp" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "AllowCustomSSPsAPs" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "BlockUserFromShowingAccountDetailsOnSignin" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "DontDisplayNetworkSelectionUI" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "DontEnumerateConnectedUsers" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "DisableLockScreenAppNotifications" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "BlockDomainPicturePassword" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "AllowDomainPINLogon" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\System" -Name "NoLocalPasswordResetQuestions" -Value 1 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\Windows Search
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" -Name "AllowCortana" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" -Name "AllowCortanaAboveLock" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" -Name "AllowIndexingEncryptedStoresOrItems" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\Windows Search" -Name "AllowSearchToUseLocation" -Value 0 -Type DWord

Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Name "SetDisablePauseUXAccess" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Name "ManagePreviewBuildsPolicyValue" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Name "DeferFeatureUpdates" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Name "DeferFeatureUpdatesPeriodInDays" -Value 180 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Name "DeferQualityUpdates" -Value 1 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Name "DeferQualityUpdatesPeriodInDays" -Value 0 -Type DWord

```

```
Key Path: HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU
if (-not (Test-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU")) {
 New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU" -Name "NoAutoRebootWithLoggedOnUsers" -Value 0 -Type DWord
Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU" -Name "ScheduledInstallDay" -Value 0 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Control\Session Manager
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager" -Name "SafeDllSearchMode" -Value 1 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Services\Eventlog\Security
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Services\Eventlog\Security")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Eventlog\Security" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Eventlog\Security" -Name "WarningLevel" -Value 90 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters" -Name "SMB1" -Value 0 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Services\NetBT\Parameters
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Services\NetBT\Parameters")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Services\NetBT\Parameters" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\NetBT\Parameters" -Name "NodeType" -Value 2 -Type DWord
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\NetBT\Parameters" -Name "NoNameReleaseOnDemand" -Value 1 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters" -Name "DisableIPSourceRouting" -Value 2 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" -Name "DisableIPSourceRouting" -Value 2 -Type DWord
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" -Name "EnableICMPRedirect" -Value 0 -Type DWord

Key Path: HKLM\SYSTEM\CurrentControlSet\Services\mrxsmbl
if (-not (Test-Path "HKLM:\SYSTEM\CurrentControlSet\Services\mrxsmbl")) {
 New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Services\mrxsmbl" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\mrxsmbl" -Name "Start" -Value 4 -Type DWord

Key Path: HKLM\Software\Policies\Microsoft\Internet Explorer\Feeds
if (-not (Test-Path "HKLM:\Software\Policies\Microsoft\Internet Explorer\Feeds")) {
 New-Item -Path "HKLM:\Software\Policies\Microsoft\Internet Explorer\Feeds" -Force | Out-Null
}
Set-ItemProperty -Path "HKLM:\Software\Policies\Microsoft\Internet Explorer\Feeds" -Name "AllowBasicAuthInClear" -Value 0 -Type DWord

Write-Host "[+] System administrative templates configured successfully." -ForegroundColor Green
```

To verify the administrative template configuration:

[Download Script: Get-SystemAdministrativeTemplatesStatus.ps1](#)



```
Get-SystemAdministrativeTemplatesStatus.ps1
Description: Audits 84 system and administrative template controls on the local machine.

Write-Host "--- Auditing System Administrative Templates Hardening ---" -ForegroundColor Cyan
$script:Vulnerable = $false

function Test-RegValue {
 param(
 [string]$RecNum,
 [string]$Hive,
 [string]$KeyPath,
 [string]$ValueName,
 [object]$ExpectedValue
)
 $FullPath = "$($Hive)\$($KeyPath)"
 if (Test-Path $FullPath) {
 $Prop = Get-ItemProperty -Path $FullPath -Name $ValueName -ErrorAction SilentlyContinue
 if ($null -ne $Prop) {
 $ActualValue = $Prop.$ValueName
 if ($ActualValue -eq $ExpectedValue) {
 Write-Host " [+] $RecNum | $ValueName = $ActualValue (Secure)" -ForegroundColor Green
 } else {
 Write-Host " [!] MISMATCH: $RecNum | Path: $Hive\$KeyPath | Value: $ValueName | Current: $ActualValue (Expected: $ExpectedValue)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
 } else {
 Write-Host " [!] MISSING VALUE: $RecNum | Path: $Hive\$KeyPath | Value: $ValueName (Expected: $ExpectedValue)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
 } else {
 Write-Host " [!] MISSING KEY: $RecNum | Path: $Hive\$KeyPath (Expected: $ValueName = $ExpectedValue)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
}

Test-RegValue -RecNum "18.4.2" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\mrxsb10" -ValueName "Start" -ExpectedValue 4
Test-RegValue -RecNum "18.4.3" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters" -ValueName "SMB1" -ExpectedValue 0
Test-RegValue -RecNum "18.4.7" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\NetBT\Parameters" -ValueName "NodeType" -ExpectedValue 2
Test-RegValue -RecNum "18.5.1" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" -ValueName "AutoAdminLogon" -ExpectedValue "0"
Test-RegValue -RecNum "18.5.2" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\Tcpip6\Parameters" -ValueName "DisableIPSourceRouting" -ExpectedValue 2
Test-RegValue -RecNum "18.5.3" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" -ValueName "DisableIPSourceRouting" -ExpectedValue 2
Test-RegValue -RecNum "18.5.5" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\Tcpip\Parameters" -ValueName "EnableICMPRedirect" -ExpectedValue 0
Test-RegValue -RecNum "18.5.7" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\NetBT\Parameters" -ValueName "NoNameReleaseOnDemand" -ExpectedValue 1
Test-RegValue -RecNum "18.5.9" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Control\Session Manager" -ValueName "SafeDllSearchMode" -ExpectedValue 1
Test-RegValue -RecNum "18.5.10" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon" -ValueName "ScreenSaveGracePeriod" -ExpectedValue 5
Test-RegValue -RecNum "18.5.13" -Hive "HKLM" -KeyPath "SYSTEM\CurrentControlSet\Services\Eventlog\Security" -ValueName "WarningLevel" -ExpectedValue 90
Test-RegValue -RecNum "18.9.7.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Device Metadata" -ValueName "PreventDeviceMetadataFromNetwork" -ExpectedValue 1
Test-RegValue -RecNum "18.9.19.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" -ValueName "NoBackgroundPolicy" -ExpectedValue 0
Test-RegValue -RecNum "18.9.19.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{35378EAC-683F-11D2-A89A-00C04FBBCFA2}" -ValueName "NoGPListChanges" -ExpectedValue 0
Test-RegValue -RecNum "18.9.19.4" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}" -ValueName "NoBackgroundPolicy" -ExpectedValue 0
Test-RegValue -RecNum "18.9.19.5" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Group Policy\{827D319E-6EAC-11D2-A4EA-00C04F79F83A}" -ValueName "NoGPListChanges" -ExpectedValue 0
Test-RegValue -RecNum "18.9.19.6" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "EnableCdp" -ExpectedValue 0
Test-RegValue -RecNum "18.9.20.1.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows NT\Printers" -ValueName "DisableWebPrintDownload" -ExpectedValue 1
Test-RegValue -RecNum "18.9.20.1.6" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" -ValueName
```

```

"NoWebServices" -ExpectedValue 1
Test-RegValue -RecNum "18.9.26.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "AllowCustomSSPsAPs"
-ExpectedValue 0
Test-RegValue -RecNum "18.9.28.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "BlockUserFromShowingAccountDetailsOnSignIn" -ExpectedValue 1
Test-RegValue -RecNum "18.9.28.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "DontDisplayNetworkSelectionUI" -ExpectedValue 1
Test-RegValue -RecNum "18.9.28.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "DontEnumerateConnectedUsers" -ExpectedValue 1
Test-RegValue -RecNum "18.9.28.5" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "DisableLockScreenAppNotifications" -ExpectedValue 1
Test-RegValue -RecNum "18.9.28.6" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "BlockDomainPicturePassword" -ExpectedValue 1
Test-RegValue -RecNum "18.9.28.7" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "AllowDomainPINLogin" -ExpectedValue 0
Test-RegValue -RecNum "18.9.33.6.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Power\PowerSettings\15576e8-98b7-4186-b944-efaf664402d9" -ValueName "DCSettingIndex" -ExpectedValue 0
Test-RegValue -RecNum "18.9.33.6.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Power\PowerSettings\15576e8-98b7-4186-b944-efaf664402d9" -ValueName "ACSettingIndex" -ExpectedValue 0
Test-RegValue -RecNum "18.9.35.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services" -ValueName "fAllowUnsolicited" -ExpectedValue 0
Test-RegValue -RecNum "18.9.36.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows NT\Rpc" -ValueName "EnableAuthEpResolution" -ExpectedValue 1
Test-RegValue -RecNum "18.9.51.1.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpClient" -ValueName "Enabled" -ExpectedValue 1
Test-RegValue -RecNum "18.9.51.1.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\W32Time\TimeProviders\NtpServer" -ValueName "Enabled" -ExpectedValue 0
Test-RegValue -RecNum "18.10.4.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Appx" -ValueName "DisablePerUserUnsigneDPackagesByDefault" -ExpectedValue 1
Test-RegValue -RecNum "18.10.4.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Appx" -ValueName "BlockNonAdminUserInstall" -ExpectedValue 1
Test-RegValue -RecNum "18.10.9.1.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Biometrics\FacialFeatures" -ValueName "EnhancedAntiSpoofing" -ExpectedValue 1
Test-RegValue -RecNum "18.10.13.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\CloudContent" -ValueName "DisableConsumerAccountStateContent" -ExpectedValue 1
Test-RegValue -RecNum "18.10.14.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Connect" -ValueName "RequirePinForPairing" -ExpectedValue 1
Test-RegValue -RecNum "18.10.15.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\CredUI" -ValueName "DisablePasswordReveal" -ExpectedValue 1
Test-RegValue -RecNum "18.10.15.2" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\CredUI" -ValueName "EnumerateAdministrators" -ExpectedValue 0
Test-RegValue -RecNum "18.10.15.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\System" -ValueName "NoLocalPasswordResetQuestions" -ExpectedValue 1
Test-RegValue -RecNum "18.10.16.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\DataCollection" -ValueName "DisableOneSettingsDownloads" -ExpectedValue 1
Test-RegValue -RecNum "18.10.16.4" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\DataCollection" -ValueName "DoNotShowFeedbackNotifications" -ExpectedValue 1
Test-RegValue -RecNum "18.10.16.5" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\DataCollection" -ValueName "EnableOneSettingsAuditing" -ExpectedValue 1
Test-RegValue -RecNum "18.10.16.6" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\DataCollection" -ValueName "LimitDiagnosticLogCollection" -ExpectedValue 1
Test-RegValue -RecNum "18.10.16.7" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\DataCollection" -ValueName "LimitDumpCollection" -ExpectedValue 1
Test-RegValue -RecNum "18.10.16.8" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\PreviewBuilds" -ValueName "AllowBuildPreview" -ExpectedValue 0
Test-RegValue -RecNum "18.10.18.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -ValueName "EnableExperimentalFeatures" -ExpectedValue 0
Test-RegValue -RecNum "18.10.18.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -ValueName "EnableHashOverride" -ExpectedValue 0
Test-RegValue -RecNum "18.10.18.4" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -ValueName "EnableLocalArchiveMalwareScanOverride" -ExpectedValue 0
Test-RegValue -RecNum "18.10.18.5" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -ValueName "EnableBypassCertificatePinningForMicrosoftStore" -ExpectedValue 0
Test-RegValue -RecNum "18.10.18.6" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\AppInstaller" -ValueName "EnableMSAppInstallerProtocol" -ExpectedValue 0
Test-RegValue -RecNum "18.10.26.1.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\Application" -ValueName "Retention" -ExpectedValue "0"
Test-RegValue -RecNum "18.10.26.1.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\Application" -ValueName "MaxSize" -ExpectedValue 32768
Test-RegValue -RecNum "18.10.26.2.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\Security" -ValueName "Retention" -ExpectedValue "0"
Test-RegValue -RecNum "18.10.26.2.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\Security" -ValueName "MaxSize" -ExpectedValue 196608
Test-RegValue -RecNum "18.10.26.3.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup" -ValueName "Retention" -ExpectedValue "0"
Test-RegValue -RecNum "18.10.26.3.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\Setup" -ValueName "MaxSize" -ExpectedValue 32768

```

```

Test-RegValue -RecNum "18.10.26.4.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\System" -ValueName "Retention" -ExpectedValue "0"
Test-RegValue -RecNum "18.10.26.4.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\EventLog\System" -ValueName "MaxSize" -ExpectedValue 32768
Test-RegValue -RecNum "18.10.29.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Explorer" -ValueName "DisableMotWOnInsecurePathCopy" -ExpectedValue 0
Test-RegValue -RecNum "18.10.29.5" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\Explorer" -ValueName "PreXPSP2ShellProtocolBehavior" -ExpectedValue 0
Test-RegValue -RecNum "18.10.35.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Internet Explorer\Main" -ValueName "NotifyDisableIEOptions" -ExpectedValue 0
Test-RegValue -RecNum "18.10.43.11.1.1.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows Defender\Remediation\BehavioralNetwork Blocks\Brute Force Protection" -ValueName "BruteForceProtectionConfiguredState" -ExpectedValue 2
Test-RegValue -RecNum "18.10.43.13.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows Defender\Scan" -ValueName "DisablePackedExeScanning" -ExpectedValue 0
Test-RegValue -RecNum "18.10.58.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Internet Explorer\Feeds" -ValueName "DisableEnclosureDownload" -ExpectedValue 1
Test-RegValue -RecNum "18.10.58.2" -Hive "HKLM" -KeyPath "Software\Policies\Microsoft\Internet Explorer\Feeds" -ValueName "AllowBasicAuthInClear" -ExpectedValue 0
Test-RegValue -RecNum "18.10.59.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Windows Search" -ValueName "AllowCortana" -ExpectedValue 0
Test-RegValue -RecNum "18.10.59.4" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Windows Search" -ValueName "AllowCortanaAboveLock" -ExpectedValue 0
Test-RegValue -RecNum "18.10.59.5" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Windows Search" -ValueName "AllowIndexingEncryptedStoresOrItems" -ExpectedValue 0
Test-RegValue -RecNum "18.10.59.6" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Windows Search" -ValueName "AllowSearchToUseLocation" -ExpectedValue 0
Test-RegValue -RecNum "18.10.66.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\WindowsStore" -ValueName "AutoDownload" -ExpectedValue 4
Test-RegValue -RecNum "18.10.66.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\WindowsStore" -ValueName "DisableOSUpgrade" -ExpectedValue 1
Test-RegValue -RecNum "18.10.72.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Dsh" -ValueName "AllowNewsAndInterests" -ExpectedValue 0
Test-RegValue -RecNum "18.10.82.1" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" -ValueName "EnableMPR" -ExpectedValue 0
Test-RegValue -RecNum "18.10.82.2" -Hive "HKLM" -KeyPath "SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System" -ValueName "DisableAutomaticRestartSignOn" -ExpectedValue 1
Test-RegValue -RecNum "18.10.91.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Sandbox" -ValueName "AllowClipboardRedirection" -ExpectedValue 0
Test-RegValue -RecNum "18.10.91.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\Sandbox" -ValueName "AllowNetworking" -ExpectedValue 0
Test-RegValue -RecNum "18.10.92.2.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows Defender Security Center\App and Browser protection" -ValueName "DisallowExploitProtectionOverride" -ExpectedValue 1
Test-RegValue -RecNum "18.10.93.1.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU" -ValueName "NoAutoRebootWithLoggedOnUsers" -ExpectedValue 0
Test-RegValue -RecNum "18.10.93.2.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate\AU" -ValueName "ScheduledInstallDay" -ExpectedValue 0
Test-RegValue -RecNum "18.10.93.2.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -ValueName "SetDisablePauseUXAccess" -ExpectedValue 1
Test-RegValue -RecNum "18.10.93.4.1" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -ValueName "ManagePreviewBuildsPolicyValue" -ExpectedValue 1
Test-RegValue -RecNum "18.10.93.4.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -ValueName "DeferFeatureUpdates" -ExpectedValue 1
Test-RegValue -RecNum "18.10.93.4.2" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -ValueName "DeferFeatureUpdatesPeriodInDays" -ExpectedValue 180
Test-RegValue -RecNum "18.10.93.4.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -ValueName "DeferQualityUpdates" -ExpectedValue 1
Test-RegValue -RecNum "18.10.93.4.3" -Hive "HKLM" -KeyPath "SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -ValueName "DeferQualityUpdatesPeriodInDays" -ExpectedValue 0

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}

```

## Sources & Compliance References \* \*\*CIS Microsoft Windows Client Benchmark\*\*: Section 18.4, 18.5, 18.9, and 18.10 (System Administrative Templates settings) \* \*\*ANSI Active Directory Hardening Guide\*\*: Recommendations on local account password management and protocol minimization \* \*\*Microsoft Security Baseline\*\*: Windows Client Security Baseline recommendations

## # [REQ-END-027] Configure AppLocker Policies

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 Enterprise/Professional (and above), Windows 11 Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: Computer Configuration\Policies\Windows Settings\Security Settings\Application Control Policies\AppLocker

## Rationale Endpoints and general member servers are the most common entry points for malware, ransomware, and administrative account compromise. Standard users running with non-administrative accounts can download and execute malicious executables or scripts in writeable directories (like '%TEMP%' or '%USERPROFILE%') to bypass traditional signature-based antivirus solutions.

Enforcing strict application control via AppLocker on endpoints ensures that:

1. **Malware Prevention:** Standard users are blocked from executing unauthorized binaries and installers.
2. **Defends Against AppLocker Bypasses:** Abusing trusted, signed Microsoft binaries (such as `msbuild.exe`, `installutil.exe`, `regasm.exe`, `regsvcs.exe`, `mshta.exe`, `regsvr32.exe`, `rundll32.exe`) allows attackers to execute arbitrary code bypassing default AppLocker rules. This control blocks these "Living off the Land" binaries (LOLBins) and prevents execution from user-writeable paths under `%WINDIR%` (such as `Tasks`, `Temp`, `tracing`, `spool\drivers\color`, etc.).
3. **Restricts Interpreted Codes:** Block unauthorized execution of scripts (PowerShell, VBScript, Batch) from writeable locations.
4. **Defense-in-Depth:** Limits the lateral movement of adversaries who pivot from one compromised endpoint to another.

## Legacy Impact & Compatibility \* \*\*Authorized Software Only\*\*: Users will be unable to run arbitrary executables, portable apps, or custom scripts. Standard deployment mechanisms (like SCCM, Microsoft Intune, or active software deployment tools) must be used, or explicit rules must be maintained. \* \*\*Developer & Power User Impact\*\*: Developers and power users who need to compile code locally or run custom utilities will be impacted. Dedicated whitelists or exceptions based on certificate publisher must be implemented. \* \*\*Audit Mode Deployment\*\*: Due to the high potential for system disruption, the policy must first be deployed in **Audit Only** mode for a validation period to collect logs and verify that no legitimate applications are blocked.

## ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Create or edit the GPO linked to the workstations/member servers Organizational Unit (OU) (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\System Services`
4. Double-click **Application Identity**.
5. Select **Define this policy setting** and configure the startup mode to **Automatic**.
6. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Application Control Policies\AppLocker`
7. Configure AppLocker Enforcement:
  - Right-click **AppLocker** and select **Properties**.
  - On the **Enforcement** tab, check **Configured** under:
    - **Executable rules** → Select **Enforce rules** (or **Audit only** for testing)
    - **Windows Installer rules** → Select **Enforce rules**
    - **Script rules** → Select **Enforce rules**
    - **Packaged app rules** → Select **Enforce rules**
8. Right-click **Executable Rules** and select **Create Default Rules** (this permits Windows files and program files).
9. Delete the default rule allowing "Everyone" to run files in all locations, and replace it with a rule allowing only authorized administrative groups (e.g., `Domain Admins`, `Local Administrators`) to run binaries outside the default system locations.
10. Per **ANSSI R2** recommendation, do not create standalone Deny rules. Instead, configure the following path **Exceptions** on the default Allow rule for the Windows folder:
  - Right-click the rule `(Default Rule) All files located in the Windows folder` and select **Properties**.
  - On the **Exceptions** tab, add path exceptions for writeable directories under `%WINDIR%` :
    - `%WINDIR%\Tasks\*`
    - `%WINDIR%\Temp\*`
    - `%WINDIR%\tracing\*`
    - `%WINDIR%\System32\spool\drivers\color\*`
    - `%WINDIR%\System32\Tasks\Microsoft\Windows\SyncCenter\*`
    - `%WINDIR%\SysWOW64\Tasks\Microsoft\Windows\SyncCenter\*`
  - On the same **Exceptions** tab, add path exceptions for the following bypass binaries (LOLBins):

- \*`\msbuild.exe`
- \*`\installutil.exe`
- \*`\mshta.exe`
- \*`\regasm.exe`
- \*`\regsvcs.exe`
- \*`\regsvr32.exe`
- \*`\rundll32.exe`
- \*`\bginfo.exe`
- \*`\cdb.exe`
- \*`\cmstp.exe`
- \*`\control.exe`
- \*`\csi.exe`
- \*`\dfsvc.exe`
- \*`\dnx.exe`
- \*`\fsi.exe`
- \*`\ie4unit.exe`
- \*`\ieexec.exe`
- \*`\infdefaultinstall.exe`
- \*`\mavinject.exe`
- \*`\msdeploy.exe`
- \*`\msdt.exe`
- \*`\msxsl.exe`
- \*`\odbcconf.exe`
- \*`\presentationhost.exe`
- \*`\rcsi.exe`
- \*`\rsi.exe`
- \*`\runscripthelper.exe`
- \*`\te.exe`
- \*`\tracker.exe`
- \*`\xwizard.exe`

11. Repeat the process for **Script Rules** by creating default rules and adding exceptions to the `%WINDIR%\*`. Allow rule for script execution from user-writable paths (such as `%WINDIR%\Temp\*` and `%WINDIR%\Tasks\*`).

12. Disable NTVDM (16-bit application support) to prevent AppLocker bypasses via 16-bit binaries:

- Navigate to: `Computer Configuration\Administrative Templates\System\16-bit Application Compatibility`
- Configure **Prevent access to 16-bit applications** to **Enabled**.

13. Link the GPO to the Endpoints Organizational Unit (OU).

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Configure the Application Identity service ( `AppIDSvc` ) and import the robust AppLocker policy locally.

[Download Script: Configure-EndpointAppLocker.ps1](#)

```
Configure-EndpointAppLocker.ps1
Description: Configures the Application Identity service (AppIDSvc) to start automatically and imports a robust AppLocker XML policy.

Write-Host "Applying AppLocker Identity service hardening..." -ForegroundColor Cyan

1. Enable Application Identity service (AppIDSvc)
$AppLockerService = Get-Service -Name AppIDSvc -ErrorAction SilentlyContinue
if ($AppLockerService) {
 Set-Service -Name AppIDSvc -StartupType Automatic
 Start-Service -Name AppIDSvc -ErrorAction SilentlyContinue
 Write-Host "[+] Application Identity Service (AppIDSvc) set to Automatic and started." -ForegroundColor Green
} else {
 Write-Warning "[-] Application Identity Service not found on this machine."
}

2. Configure local AppLocker policy XML content
$AppLockerXml = @"
<AppLockerPolicy Version="1">
 <RuleCollection Type="Exe" EnforcementMode="Enabled">
 <FilePathRule Id="921cc481-6e1e-453f-b3a5-bc4f4a38674d" Name="(Default Rule) All files located in the Program Files folder" Description="Allows members of the Everyone group to run applications that are located in the Program Files folder." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePathCondition Path="%PROGRAMFILES%*" />
 </Conditions>
 </FilePathRule>
 <FilePathRule Id="a61c8b2c-6d8f-4ad9-acbc-467b78a7f7b4" Name="(Default Rule) All files located in the Windows folder" Description="Allows members of the Everyone group to run applications that are located in the Windows folder." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePathCondition Path="%WINDIR%*" />
 </Conditions>
 <Exceptions>
 <FilePathCondition Path="%WINDIR%\Temp*" />
 <FilePathCondition Path="%WINDIR%\Tasks*" />
 <FilePathCondition Path="%WINDIR%\tracing*" />
 <FilePathCondition Path="%WINDIR%\System32\spool\drivers\color*" />
 <FilePathCondition Path="%WINDIR%\System32\Tasks\Microsoft\Windows\SyncCenter*" />
 <FilePathCondition Path="%WINDIR%\SysWOW64\Tasks\Microsoft\Windows\SyncCenter*" />
 <FilePathCondition Path="*\msbuild.exe" />
 <FilePathCondition Path="*\installutil.exe" />
 <FilePathCondition Path="*\mshta.exe" />
 <FilePathCondition Path="*\regasm.exe" />
 <FilePathCondition Path="*\regsvcs.exe" />
 <FilePathCondition Path="*\regsvr32.exe" />
 <FilePathCondition Path="*\rundll32.exe" />
 <FilePathCondition Path="*\bginfo.exe" />
 <FilePathCondition Path="*\cdb.exe" />
 <FilePathCondition Path="*\cmstp.exe" />
 <FilePathCondition Path="*\control.exe" />
 <FilePathCondition Path="*\csi.exe" />
 <FilePathCondition Path="*\dfsvc.exe" />
 <FilePathCondition Path="*\dnx.exe" />
 <FilePathCondition Path="*\fsi.exe" />
 <FilePathCondition Path="*\ie4unit.exe" />
 <FilePathCondition Path="*\ieexec.exe" />
 <FilePathCondition Path="*\infdefaultinstall.exe" />
 <FilePathCondition Path="*\mavinject.exe" />
 <FilePathCondition Path="*\msdeploy.exe" />
 <FilePathCondition Path="*\msdt.exe" />
 <FilePathCondition Path="*\msxml.exe" />
 <FilePathCondition Path="*\odbcconf.exe" />
 <FilePathCondition Path="*\presentationhost.exe" />
 <FilePathCondition Path="*\rcsi.exe" />
 <FilePathCondition Path="*\rsi.exe" />
 <FilePathCondition Path="*\runscripthelper.exe" />
 <FilePathCondition Path="*\te.exe" />
 <FilePathCondition Path="*\tracker.exe" />
 <FilePathCondition Path="*\xwizard.exe" />
 </Exceptions>
 </FilePathRule>
 <FilePathRule Id="fd686d83-a829-4351-8ff4-27c1de5732e9" Name="(Default Rule) All files" Description="Allows members of the local Administrators group to run all applications." UserOrGroupSid="S-1-5-32-544" Action="Allow">

```



```

 <Conditions>
 <FilePathCondition Path="*" />
 </Conditions>
 </FilePathRule>
</RuleCollection>
<RuleCollection Type="Msi" EnforcementMode="Enabled">
 <FilePathRule Id="5b8fa8b3-3a5e-4c7a-9cb8-b223ff9db271" Name="(Default Rule) All Windows Installer files in Program Files" Description="Allows everyone to run Windows Installer files in Program Files." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePathCondition Path="%PROGRAMFILES%*" />
 </Conditions>
 </FilePathRule>
 <FilePathRule Id="6b8fa8b3-3a5e-4c7a-9cb8-b223ff9db272" Name="(Default Rule) All Windows Installer files in Windows" Description="Allows everyone to run Windows Installer files in Windows." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePathCondition Path="%WINDIR%*" />
 </Conditions>
 </FilePathRule>
 <FilePathRule Id="7b8fa8b3-3a5e-4c7a-9cb8-b223ff9db273" Name="(Default Rule) All Windows Installer files" Description="Allows administrators to run all Windows Installer files." UserOrGroupSid="S-1-5-32-544" Action="Allow">
 <Conditions>
 <FilePathCondition Path="*" />
 </Conditions>
 </FilePathRule>
</RuleCollection>
<RuleCollection Type="Script" EnforcementMode="Enabled">
 <FilePathRule Id="1c8fa8b3-3a5e-4c7a-9cb8-b223ff9db274" Name="(Default Rule) All scripts located in the Program Files folder" Description="Allows everyone to run scripts in Program Files." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePathCondition Path="%PROGRAMFILES%*" />
 </Conditions>
 </FilePathRule>
 <FilePathRule Id="2c8fa8b3-3a5e-4c7a-9cb8-b223ff9db275" Name="(Default Rule) All scripts located in the Windows folder" Description="Allows everyone to run scripts in Windows." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePathCondition Path="%WINDIR%*" />
 </Conditions>
 <Exceptions>
 <FilePathCondition Path="%WINDIR%\Temp%*" />
 <FilePathCondition Path="%WINDIR%\Tasks%*" />
 </Exceptions>
 </FilePathRule>
 <FilePathRule Id="3c8fa8b3-3a5e-4c7a-9cb8-b223ff9db276" Name="(Default Rule) All scripts" Description="Allows administrators to run all scripts." UserOrGroupSid="S-1-5-32-544" Action="Allow">
 <Conditions>
 <FilePathCondition Path="*" />
 </Conditions>
 </FilePathRule>
</RuleCollection>
<RuleCollection Type="Appx" EnforcementMode="Enabled">
 <FilePublisherRule Id="1d8fa8b3-3a5e-4c7a-9cb8-b223ff9db279" Name="(Default Rule) All signed packaged apps" Description="Allows everyone to run signed packaged apps." UserOrGroupSid="S-1-1-0" Action="Allow">
 <Conditions>
 <FilePublisherCondition PublisherName="*" ProductName="*" BinaryName="*">
 <BinaryVersionRange LowSection="0.0.0.0" HighSection="*" />
 </FilePublisherCondition>
 </Conditions>
 </FilePublisherRule>
</RuleCollection>
</AppLockerPolicy>
"@

```

```

Write the temporary XML and import it
$TempPath = Join-Path -Path $env:TEMP -ChildPath "AppLockerEndpointPolicy.xml"
$AppLockerXml | Out-File -FilePath $TempPath -Encoding UTF8 -Force

3. Validate policy using Test-AppLockerPolicy before importing
try {
 Import-Module AppLocker -ErrorAction Stop
} catch {
 Write-Error "AppLocker module is not available on this system. Cannot configure or validate policy."
 if (Test-Path $TempPath) {
 Remove-Item -Path $TempPath -Force
 }
 return
}

```

```

$TestPaths = @(
 # Expected: Allowed
 "$env:windir\System32\cmd.exe",
 "$env:windir\System32\WindowsPowerShell\v1.0\powershell.exe",
 # Expected: DeniedByDefault or ExplicitlyDenied (since it is an exception to an Allow rule)
 "$env:USERPROFILE\Downloads\tool.exe",
 "$env:windir\Temp\malware.exe",
 "$env:windir\Tasks\evil.exe",
 "$env:windir\System32\msbuild.exe"
)

$ValidationFailed = $false
try {
 $TestResults = Test-AppLockerPolicy -XmlPolicy $TempPath -Path $TestPaths -User Everyone -ErrorAction Stop
 $ExpectedAllow = @(
 "$env:windir\System32\cmd.exe",
 "$env:windir\System32\WindowsPowerShell\v1.0\powershell.exe"
)
 $ExpectedDeny = @(
 "$env:USERPROFILE\Downloads\tool.exe",
 "$env:windir\Temp\malware.exe",
 "$env:windir\Tasks\evil.exe",
 "$env:windir\System32\msbuild.exe"
)

 foreach ($Result in $TestResults) {
 $Path = $Result.FilePath
 $Decision = $Result.PolicyDecision
 if ($ExpectedAllow -contains $Path) {
 if ($Decision -ne "Allowed") {
 Write-Warning "[VALIDATION FAIL] Expected Allow for: $Path (got: $Decision)"
 $ValidationFailed = $true
 }
 }
 if ($ExpectedDeny -contains $Path) {
 if ($Decision -eq "Allowed") {
 Write-Warning "[VALIDATION FAIL] Expected Deny/Not Allowed for: $Path (got: $Decision)"
 $ValidationFailed = $true
 }
 }
 }
} catch {
 Write-Warning "Could not perform policy validation tests: $($_.Exception.Message)"
 $ValidationFailed = $true
}

if ($ValidationFailed) {
 Write-Error "AppLocker policy validation failed. Policy was NOT imported."
 if (Test-Path $TempPath) {
 Remove-Item -Path $TempPath -Force
 }
 return
}

Write-Host "[+] AppLocker policy validation passed. Proceeding with import." -ForegroundColor Green

4. Import the validated AppLocker policy
try {
 Set-AppLockerPolicy -XmlPolicy $TempPath -ErrorAction Stop
 Write-Host "[+] Local AppLocker policy imported and enforced successfully." -ForegroundColor Green
} catch {
 Write-Error "Failed to import AppLocker policy: $($_.Exception.Message)"
} finally {
 if (Test-Path $TempPath) {
 Remove-Item -Path $TempPath -Force
 }
}

5. Disable NTVDM (16-bit compatibility) via Registry
$Ntvdmpath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppCompat"
if (-not (Test-Path $Ntvdmpath)) {
 New-Item -Path $Ntvdmpath -Force | Out-Null
}
Set-ItemProperty -Path $Ntvdmpath -Name "Prevent16BitApp" -Value 1 -Type DWord
Write-Host "[+] 16-bit NTVDM compatibility disabled in registry." -ForegroundColor Green

```



To verify the AppLocker service status:

[Download Script: Test-EndpointAppLockerStatus.ps1](#)

```
Test-EndpointAppLockerStatus.ps1
Description: Checks the current configuration and operational status of the Application Identity service.

Write-Host "--- Auditing AppLocker Service Status ---" -ForegroundColor Cyan

1. Audit service state
$AppIDSvc = Get-Service -Name AppIDSvc -ErrorAction SilentlyContinue

if ($AppIDSvc) {
 if ($AppIDSvc.Status -eq "Running" -and $AppIDSvc.StartType -eq "Automatic") {
 Write-Host " - AppLocker Service Status: Running | Startup: Automatic (Secure)" -ForegroundColor Green
 } else {
 Write-Host " - VULNERABLE: AppLocker Service Status: $($AppIDSvc.Status) | Startup: $($AppIDSvc.StartType) (Should be Running/Automatic)" -ForegroundColor Red
 }
} else {
 Write-Host " - VULNERABLE: Application Identity Service (AppIDSvc) is not installed." -ForegroundColor Red
}

2. Audit enforcement registry settings
$SrpPath = "HKLM:\Software\Policies\Microsoft\Windows\SrpV2"
$Collections = @("Exe", "Msi", "Script", "Appx")

if (Test-Path $SrpPath) {
 foreach ($Col in $Collections) {
 $ColPath = "$SrpPath\$Col"
 if (Test-Path $ColPath) {
 $Val = Get-ItemProperty -Path $ColPath -Name "EnforcementMode" -ErrorAction SilentlyContinue
 if ($null -ne $Val) {
 $Mode = if ($Val.EnforcementMode -eq 1) { "Enforced" } else { "Audit Only" }
 $Color = if ($Val.EnforcementMode -eq 1) { "Green" } else { "Yellow" }
 Write-Host " - Collection $Col Enforcement: $Mode (Value: $($Val.EnforcementMode))" -ForegroundColor $Color
 } else {
 Write-Host " - Collection $Col Enforcement: NOT CONFIGURED" -ForegroundColor Red
 }
 } else {
 Write-Host " - Collection $Col Path: NOT FOUND" -ForegroundColor Red
 }
 }
} else {
 Write-Host "[-] AppLocker registry base path (SrpV2) not found. Policy is not deployed." -ForegroundColor Red
}

3. Audit NTVDM Disable Status
$NtvdmPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows\AppCompat"
if (Test-Path $NtvdmPath) {
 $AppCompatVal = Get-ItemProperty -Path $NtvdmPath -Name "Prevent16BitApp" -ErrorAction SilentlyContinue
 if ($null -ne $AppCompatVal -and $AppCompatVal.Prevent16BitApp -eq 1) {
 Write-Host " - NTVDM (16-bit AppCompat): Disabled (Secure)" -ForegroundColor Green
 } else {
 Write-Host " - NTVDM (16-bit AppCompat): Enabled or Not Configured (Expected: Disabled)" -ForegroundColor Yellow
 }
} else {
 Write-Host " - NTVDM (16-bit AppCompat): Not Configured (Expected: Disabled)" -ForegroundColor Yellow
}
```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*: DAT-NT-13 Note Technique (R2, R8, R10, R15, R16, R20) \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*: Section 18.9 (AppLocker Application Control) \* \*\*Microsoft Security Baselines\*\*: AppLocker deployment guidance for client environments. \* \*\*Ultimate AppLocker Bypass List\*\*: Generic & Verified AppLocker Bypasses

## # [REQ-END-028] Configure Early Launch Antimalware (ELAM) Policy

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*ELAM GPO\*\*: `Computer Configuration\Policies\Administrative Templates\System\Early Launch Antimalware\Boot-Start Driver Initialization Policy` → Enabled (Select \*\*Good, unknown and bad but critical\*\* in options) \* \*\*Registry Location\*\*: `HKLM\SYSTEM\CurrentControlSet\Policies\EarlyLaunch` → `DriverLoadPolicy` = `3` (REG\_DWORD)

## Rationale Boot-level integrity is crucial for endpoints to ensure malware cannot bypass OS-level antimalware defenses by loading malicious kernel-mode drivers early in the boot sequence:

1. **Early Boot Rootkits**: Malicious boot-start drivers can execute before third-party endpoint protection or security agents initialize. Enforcing the Early Launch Antimalware (ELAM) driver initialization policy ensures that unknown, unsigned, or bad drivers are prevented from loading at boot time.
2. **Platform Integrity**: Securing the boot sequence with ELAM is a critical requirement of Virtualization-Based Security (VBS) and overall hardware-rooted protection.

## Legacy Impact & Compatibility \* \*\*Boot-Start Drivers\*\*: If third-party, custom, or legacy hardware monitoring drivers are unsigned, the ELAM policy will block them from loading, potentially resulting in blue screen (BSOD) or hardware failures. Verify all active drivers possess valid digital signatures prior to enforcement.

### ## Implementation Steps

#### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the GPO applied to your standard endpoints (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Early Launch Antimalware`
4. Double-click **Boot-Start Driver Initialization Policy**.
5. Set it to **Enabled**.
6. In the options dropdown, select **Good, unknown and bad but critical** (registry value `3` ).
7. Click **OK**.

#### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to apply the ELAM driver load policy.

[Download Script: Configure-ElamPolicy.ps1](#)

```
Configure-ElamPolicy.ps1
Description: Configures the Early Launch Antimalware (ELAM) boot-start driver load policy on the local system.

Write-Host "Applying ELAM Boot-Start driver initialization policy..." -ForegroundColor Cyan

$ElamPath = "HKLM:\SYSTEM\CurrentControlSet\Policies\EarlyLaunch"
if (-not (Test-Path $ElamPath)) {
 New-Item -Path $ElamPath -Force | Out-Null
}
Set-ItemProperty -Path $ElamPath -Name "DriverLoadPolicy" -Value 3 -Type DWord -ErrorAction Stop
Write-Host "[+] ELAM Boot-Start driver initialization policy set to Good, unknown and bad but critical." -ForegroundColor Green
```

*To audit the ELAM driver load policy status:*

[Download Script: Get-ElamPolicyStatus.ps1](#)

```
Get-ElamPolicyStatus.ps1
Description: Audits registry configuration of the Early Launch Antimalware (ELAM) policy.

Write-Host "--- Auditing ELAM Boot-Start Policy ---" -ForegroundColor Cyan

$script:Vulnerable = $false

$Path = "HKLM:\SYSTEM\CurrentControlSet\Policies\EarlyLaunch"
$Name = "DriverLoadPolicy"
$Expected = 3

if (Test-Path $Path) {
 $Reg = Get-ItemProperty -Path $Path -ErrorAction SilentlyContinue
 $Val = $Reg.$Name
 if ($Val -eq $Expected) {
 Write-Host " [+] Path $($Path) | $($Name): $Val (Expected: $Expected)" -ForegroundColor Green
 } else {
 Write-Host " [!] MISMATCH: Path $($Path) | $($Name): $Val (Expected: $Expected)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
} else {
 Write-Host " [!] NOT FOUND: Path $($Path) (Expected: $Name = $Expected)" -ForegroundColor Red
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*CIS Benchmark\*\*: CIS Microsoft Windows Client Benchmark - Section 18.2.2 (System Options / ELAM) \* \*\*ANSSI AD Hardening Guide\*\*: Section addressing system and boot configuration integrity.

## # [REQ-END-029] Configure Untrusted Font Blocking

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

## Implementation Details \* \*\*Priority\*\*: Medium \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Administrative Templates\System\Mitigation Options\Untrusted Font Blocking` \* \*\*Registry Location\*\*: `HKLM\SOFTWARE\Policies\Microsoft\Windows NT\MitigationOptions` \* \*\*Value Name\*\*: `MitigationOptions\_FontBlocking` \* \*\*Value Type\*\*: `REG\_SZ` \* \*\*Value Data\*\*: `1000000000000`

## Rationale Font files (TrueType, OpenType, and others) are highly complex formats that require advanced parsing logic. Historically, font parsing in Windows was performed by the Graphics Device Interface (GDI) within the operating system kernel. Vulnerabilities in the kernel-mode font parser (such as buffer overflows or remote code execution) have been frequently exploited by threat actors to execute arbitrary code with kernel-level privileges.

Enabling Untrusted Font Blocking limits the attack surface of the graphics subsystem:

1. **Kernel Attack Surface Reduction**: Restricting the system to only load trusted fonts installed in the `%windir%\Fonts` system directory prevents the processing of malicious, web-delivered, or embedded font files.
2. **Mitigation of Document-Based Exploits**: Prevents malicious font files embedded in Microsoft Office documents, PDFs, or web pages from triggering parsing vulnerabilities in the context of the current user.

## Legacy Impact & Compatibility \* \*\*Web Browsers & Web Fonts\*\*: Web browsers (such as Microsoft Edge or Google Chrome) and web applications that dynamically download custom fonts (e.g., Google Fonts, font icon libraries like FontAwesome) may fail to render those fonts, reverting to system default fallback fonts. This may result in styling anomalies or missing icons. \* \*\*Embedded Fonts in Documents\*\*: Microsoft Office files or PDFs utilizing custom embedded fonts that are not locally installed in the system directory will render using default system fonts, potentially affecting layout alignment. \* \*\*Deployment Validation\*\*: It is recommended to deploy this setting in Audit Mode (`3000000000000`) initially to monitor event log entries (Event ID 305 inside the `Microsoft-Windows-Security-Mitigations\KernelMode` log) before fully enforcing the block policy.

### ## Implementation Steps

#### #### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a domain controller or management host.
2. Create a new GPO or edit an existing one (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Mitigation Options`
4. Configure the following setting:
  - **Policy**: `Untrusted Font Blocking`
  - **Setting**: `Enabled`
  - **Mitigation Options**: `Block untrusted fonts and log events`
5. Link the GPO to the appropriate Organizational Unit (OU) containing the target client endpoints and member servers.

#### #### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally on standalone systems or during reference image build phases.

[Download Script: Configure-UntrustedFontBlocking.ps1](#)

```
Configure-UntrustedFontBlocking.ps1
Description: Configures Untrusted Font Blocking mitigation to block untrusted fonts and log events.

$RegPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\MitigationOptions"
$ValueName = "MitigationOptions_FontBlocking"
$ValueData = "1000000000000"

Write-Host "Applying hardening requirement: Configure Untrusted Font Blocking..." -ForegroundColor Cyan

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value $ValueData -Type String -Force | Out-Null
Write-Host "Hardening applied successfully." -ForegroundColor Green
```

*To verify the setting has been applied:*

[Download Script: Get-UntrustedFontBlockingStatus.ps1](#)

```
Get-UntrustedFontBlockingStatus.ps1
Description: Checks the current configuration state of Untrusted Font Blocking registry setting.

$RegPath = "HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\MitigationOptions"
$ValueName = "MitigationOptions_FontBlocking"
$ExpectedValue = "10000000000000"

Write-Host "Auditing hardening requirement: Configure Untrusted Font Blocking..." -ForegroundColor Cyan

if (Test-Path $RegPath) {
 $value = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
 if ($null -ne $value -and $value.$ValueName -eq $ExpectedValue) {
 Write-Host "Audit Result: Compliant. Untrusted fonts are blocked and logged ($($ValueName) = $($ExpectedValue))." -Foregroun
dColor Green
 exit 0
 }
}

Write-Host "Audit Result: Non-Compliant. Untrusted fonts are not configured to block and log." -ForegroundColor Red
exit 1
```

---

## Sources & Compliance References \* \*\*Microsoft Learn\*\*: Block untrusted fonts in an enterprise (MitigationOptions registry configurations)  
\* \*\*CIS Benchmark\*\*: CIS Microsoft Windows Client Benchmark - Section 18.9.30.1 (System Options / Mitigation Options)

**# [REQ-END-030] Configure svchost.exe Mitigation Options**

**## Target Scope** \* **Applicable Systems**: Tier 2 Client Workstations. \* **Operating Systems**: Windows 10 (1903 and above), Windows 11 Enterprise/Professional.

**## Implementation Details** \* **Priority**: Medium \* **GPO Path / Registry Location**: \* **GPO Path**: `Computer Configuration\Policies\Administrative Templates\System\Service Control Manager Settings\Security Settings\Enable svchost.exe mitigation options` \* **Registry Location**: `HKLM\SYSTEM\CurrentControlSet\Control\SCMConfig` \* **Value Name**: `EnableSvchostMitigationPolicy` \* **Value Type**: `REG\_DWORD` \* **Value Data**: `1`

**## Rationale** The Service Host (`svchost.exe`) process runs multiple system services. Because these services execute with high privileges, they are frequently targeted for process injection, hollowing, or spoofing to execute arbitrary code or harvest credentials.

Enabling `svchost.exe` mitigation options restricts the behavior of the `svchost.exe` process to enhance security:

1. **Microsoft-Only Binary Enforcement**: Requires all binaries and dynamic-link libraries (DLLs) loaded into `svchost.exe` to be digitally signed by Microsoft. This prevents attackers from injecting custom, unsigned malicious DLLs into `svchost.exe` instances.
2. **Dynamic Code Blocking**: Prevents the execution of dynamically generated code (such as Just-In-Time compiled code) within `svchost.exe` processes, neutralizing typical in-memory exploitation vectors.

**## Legacy Impact & Compatibility** \* **Third-Party Compatibility**: This policy requires all binaries loaded by `svchost.exe` to be Microsoft-signed. Any third-party software, security agents, or system drivers that attempt to run services inside the `svchost.exe` process space using non-Microsoft DLLs will fail to load. This has historically caused issues with legacy antivirus, third-party authentication plugins, or specialized management utilities on client systems. \* **Operating System Support**: This policy has no effect on Windows 10 versions prior to 1903. \* **Deployment Validation**: It is recommended to perform extensive baseline testing on a representative subset of workstations running third-party software before deploying this configuration across the entire production domain.

**## Implementation Steps****### Option A: Group Policy Object (GPO) Configuration (Preferred)**

1. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a domain controller or management host.
2. Create a new GPO or edit an existing one (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Policies\Administrative Templates\System\Service Control Manager Settings\Security Settings`
4. Configure the following setting:
  - **Policy**: `Enable svchost.exe mitigation options`
  - **Setting**: `Enabled`
5. Link the GPO to the appropriate Organizational Unit (OU) containing the target client endpoints.

**### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)**

Use this method to apply the setting locally on standalone systems or during reference image build phases.

**Download Script: [Configure-SvchostMitigation.ps1](#)**

```
Configure-SvchostMitigation.ps1
Description: Configures svchost.exe mitigation options to enforce Microsoft-signed binaries and block dynamic code.

$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\SCMConfig"
$ValueName = "EnableSvchostMitigationPolicy"
$ValueData = 1

Write-Host "Applying hardening requirement: Configure svchost.exe mitigation options..." -ForegroundColor Cyan

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value $ValueData -Type DWord -Force | Out-Null
Write-Host "Hardening applied successfully." -ForegroundColor Green
```

To verify the setting has been applied:

**Download Script: [Get-SvchostMitigationStatus.ps1](#)**

```
Get-SvchostMitigationStatus.ps1
Description: Audits the configuration state of svchost.exe mitigation options.

$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\SCMConfig"
$ValueName = "EnableSvchostMitigationPolicy"
$ExpectedValue = 1

Write-Host "Auditing hardening requirement: Configure svchost.exe mitigation options..." -ForegroundColor Cyan

if (Test-Path $RegPath) {
 $value = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
 if ($null -ne $value -and $value.$ValueName -eq $ExpectedValue) {
 Write-Host "Audit Result: Compliant. svchost.exe mitigation options are enabled." -ForegroundColor Green
 exit 0
 }
}

Write-Host "Audit Result: Non-Compliant. svchost.exe mitigation options are disabled or not configured." -ForegroundColor Red
exit 1
```

---

## Sources & Compliance References \* \*\*Microsoft Learn\*\*: Group Policy settings reference - Service Control Manager Settings \*  
\*\*Microsoft Security Guidance\*\*: Removing "Enable svchost.exe mitigation options" from baseline recommendations (for compatibility awareness) \* \*\*CIS Benchmark\*\*: CIS Microsoft Windows Client Benchmark (Section 18.9.30.1 - Mitigation Options reference)

# [REQ-END-031] Enable Kernel-Mode Hardware-Enforced Stack Protection

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 11 (and above) Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: Computer Configuration\Administrative Templates\System\Device Guard\Turn On Virtualization Based Security → Set \*\*Kernel-level shadow stacks\*\* to \*\*Enabled\*\* \* \*\*Registry Location\*\*: HKLM\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\KernelShadowStacks \* 'Enabled' = '1' (REG\_DWORD)

## Rationale Kernel-mode Hardware-enforced Stack Protection uses CPU hardware features to protect the operating system kernel from memory corruption exploits, specifically Return-Oriented Programming (ROP) attacks.

An adversary attempting privilege escalation or remote code execution often hijacks the control flow of kernel-mode components by overwriting return addresses on the stack. Intel Control-flow Enforcement Technology (CET) and AMD Shadow Stack technologies create a separate, hardware-secured copy of the call stack (the "shadow stack").

Before returning from a function, the CPU compares the return address on the standard stack with the address stored on the hardware-secured shadow stack. If the addresses do not match, the processor detects a control flow violation, terminates the process, or triggers a system crash to prevent execution of malicious payloads.

Enforcing Kernel-mode Hardware-enforced Stack Protection provides hardware-backed control-flow integrity, neutralizing key vectors of kernel exploitation.

## Legacy Impact & Compatibility \* \*\*Hardware Requirements\*\*: Systems must support hardware shadow stacks. This requires Intel 11th Gen Core (Tiger Lake) or newer processors, or AMD Zen 3 (Ryzen 5000) or newer processors. \* \*\*Firmware & OS Requirements\*\*: The system must run Windows 11 version 22H2 or newer. Additionally, the system must use UEFI BIOS with Secure Boot enabled. \* \*\*Dependencies\*\*: Virtualization-Based Security (VBS) and Hypervisor-Enforced Code Integrity (HVCI / Memory Integrity) must be enabled and active. \* \*\*Driver Compatibility\*\*: This feature enforces strict rules on kernel-mode code. Older, legacy, or improperly signed drivers—often associated with kernel-level anti-cheat software, legacy hardware, or debugging tools—will fail to load or may trigger system crashes (BSOD). Verify driver compatibility in a representative sandbox environment prior to enterprise-wide enforcement.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Edit the appropriate endpoint GPO (e.g., `GPO_Hardening_Endpoints` ).
3. Navigate to: `Computer Configuration\Administrative Templates\System\Device Guard`
4. Configure the setting:
  - **Policy:** `Turn On Virtualization Based Security` → Set to **Enabled**
  - Check **Kernel-level shadow stacks** → Set to **Enabled** (or set registry `Enabled` = `1` )

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following script locally to configure the registry and activate Kernel-mode Hardware-enforced Stack Protection.

[Download Script: Enable-KernelShadowStacks.ps1](#)

```
Enable-KernelShadowStacks.ps1
Description: Configures HKLM registry to enable Kernel-mode Hardware-enforced Stack Protection (Kernel Shadow Stacks).

Write-Host "Enabling Kernel-mode Hardware-enforced Stack Protection..." -ForegroundColor Cyan

$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\KernelShadowStacks"

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name "Enabled" -Value 1 -Type DWord
Write-Host "[+] Registry setting for Kernel Shadow Stacks enabled. (Reboot required)." -ForegroundColor Green
```

*To audit the state of Kernel-mode Hardware-enforced Stack Protection:*

[Download Script: Test-KernelShadowStacks.ps1](#)



```
Test-KernelShadowStacks.ps1
Description: Audits the registry status of Kernel-mode Hardware-enforced Stack Protection (Kernel Shadow Stacks).

Write-Host "--- Auditing Kernel-mode Hardware-enforced Stack Protection ---" -ForegroundColor Cyan

$script:Vulnerable = $false
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\KernelShadowStacks"

Check registry value
$val = Get-ItemProperty -Path $RegPath -Name "Enabled" -ErrorAction SilentlyContinue
$actual = if ($val) { $val.Enabled } else { "" }

if ($actual -eq 1) {
 Write-Host " - Registry Setting: KernelShadowStacks Enabled | Actual: '1' (Expected: '1')" -ForegroundColor Green
} else {
 $script:Vulnerable = $true
 Write-Host " - Registry Setting: KernelShadowStacks Enabled | Actual: '$actual' (Expected: '1')" -ForegroundColor Red
}

Verify VBS dependency is met
try {
 $DG = Get-CimInstance -Namespace "Root\Microsoft\Windows\DeviceGuard" -ClassName "Win32_DeviceGuard" -ErrorAction Stop
 if ($DG.VirtualizationBasedSecurityStatus -eq 2) {
 Write-Host " - VBS Status: Running" -ForegroundColor Green
 } else {
 $script:Vulnerable = $true
 Write-Host " - VBS Status: Not Running (VBS is required for Kernel Shadow Stacks)" -ForegroundColor Red
 }
} catch {
 $script:Vulnerable = $true
 Write-Host " - DeviceGuard WMI class query failed. VBS is likely disabled." -ForegroundColor Red
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
}
}
```

---

## Sources & Compliance References \* \*\*Microsoft Windows Security Baselines\*\*: Core Isolation Security Baseline \* \*\*ANSI AD Hardening Guide\*\*: Section on hardware virtualization and kernel exploitation mitigation \* \*\*DoD Windows 11 Computer STIG\*\*: Device Guard / Virtualization-Based Security policies

# [REQ-END-032] Disable Unused Windows Features and PowerShell 2.0 Engine

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path (PowerShell 2.0 Compatibility)\*\*: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows PowerShell` \* \*\*Registry Location (PowerShell 2.0 Compatibility)\*\*: `HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\PowerShellV2` \* \*\*Feature Disablement\*\*: Deployed via Group Policy Startup Script or PowerShell Desired State Configuration (DSC) using DISM/ServerManager.

---

## Rationale To minimize the attack surface of standard client endpoints and member servers, all unnecessary legacy protocols, optional features, and runtime engines must be disabled:

1. **PowerShell 2.0 Engine:** Legacy PowerShell 2.0 does not support modern logging, transcription, or security monitoring mechanisms such as the Antimalware Scan Interface (AMSI). Attackers leverage "downgrade attacks" by executing PowerShell scripts using the `-version 2.0` parameter to bypass script block logging and security tooling. Disabling the engine and its parent runtimes eliminates this bypass vector.
2. **.NET Framework 3.5:** The .NET 3.5 Framework includes the runtime files for .NET 2.0 and 3.0. PowerShell 2.0 requires .NET 2.0/3.5 to run. Disabling `.NET Framework 3.5` removes legacy runtime binaries that are susceptible to downgrade attacks and removes support for older, unpatched software.
3. **SMBv1 Protocol:** The legacy SMBv1 protocol is cryptographically weak, lacks authentication integrity protection, and has been the target of catastrophic remote code execution attacks (such as EternalBlue). Leaving the SMBv1 driver active allows relaying and man-in-the-middle attacks.
4. **Internet Explorer 11:** Internet Explorer contains obsolete MSHTML render engine components. Disabling this legacy browser reduces vulnerability to web-based code execution.
5. **Work Folders, XPS, DirectPlay, and Client Protocols:** Services and tools such as Work Folders, XPS Viewer, DirectPlay, Telnet Client, TFTP Client, and Simple TCP/IP Services contain legacy network parsers and protocols that are completely unnecessary for a secure administrative system.

---

## Legacy Impact & Compatibility \* \*\*Script Dependencies\*\*: Any administrative scripts that depend on .NET Framework 2.0/3.0/3.5 will fail. All internal scripts must be updated to run on .NET 4.8 or modern .NET (Core) runtimes. \* \*\*Legacy Management Software\*\*: Old configuration managers or printers that require SMBv1 client capabilities to mount file shares will fail to connect. \* \*\*Compatibility Exclusions\*\*: Windows Subsystem for Linux (WSL) and Windows Sandbox are excluded from this baseline on standard client endpoints, as they may be required for development and testing activities by standard users.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### Step 1: Disable PowerShell 2.0 Compatibility Policy 1. Open the **Group Policy Management Console** (`gpmc.msc`) on a management host. 2. Edit the target endpoints GPO (e.g., `GPO_Hardening_Endpoints`). 3. Navigate to: `Computer Configuration\Policies\Administrative Templates\Windows Components\Windows PowerShell` 4. Configure the following setting: \* \*\*Policy\*\*: `Turn on PowerShell 2.0 Compatibility Mode` \* \*\*Setting\*\*: `Disabled` 5. Link the GPO to the Endpoints and Member Servers OUs.

##### Step 2: Deploy Feature Disablement Startup Script Because Windows Optional Features are managed via DISM/Packages and lack direct GPO settings for feature removal, deploy the disablement script as a Computer Startup script: 1. In the same GPO, navigate to: `Computer Configuration\Policies\Windows Settings\Scripts (Startup/Shutdown)` 2. Double-click **Startup**, click the **PowerShell Scripts** tab. 3. Add the `Disable-UnusedFeatures.ps1` script to execute on system startup.

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Use this method to apply the setting locally or run it as a startup script.

[Download Script: Disable-UnusedFeatures.ps1](#)

```
Disable-UnusedFeatures.ps1
Description: Disables unused legacy features, .NET 3.5, and PowerShell 2.0 on the local system.

Write-Host "Disabling unused legacy features and PowerShell 2.0..." -ForegroundColor Cyan

Check if running as administrator
$isAdmin = ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
if (-not $isAdmin) {
 Write-Error "This script must be run as an Administrator."
 exit 1
}

Feature lists
$DismFeatures = @(
 "MicrosoftWindowsPowerShellV2",
 "MicrosoftWindowsPowerShellV2Root",
 "NetFx3",
 "SMB1Protocol",
 "Internet-Explorer-Optional-amd64",
 "WorkFolders-Client",
 "Xps-Viewer-Dependency",
 "DirectPlay",
 "TelnetClient",
 "TFTP",
 "SimpleTCP"
 # .NET Framework 3.5
 # SMBv1 Client
 # Internet Explorer 11
 # Work Folders Client
 # XPS Viewer
 # DirectPlay
 # Telnet Client
 # TFTP Client
 # Simple TCP/IP Services
)

$ServerFeatures = @(
 "PowerShell-V2",
 "NET-Framework-Core",
 "FS-SMB1",
 "Internet-Explorer-Optional-amd64",
 "WorkFolders-Client",
 "Xps-Viewer-Dependency",
 "DirectPlay",
 "Telnet-Client",
 "TFTP-Client",
 "Simple-TCP/IP"
)

if (Get-Command Get-WindowsFeature -ErrorAction SilentlyContinue) {
 # Windows Server path
 foreach ($FeatName in $ServerFeatures) {
 $Feat = Get-WindowsFeature -Name $FeatName -ErrorAction SilentlyContinue
 if ($null -ne $Feat) {
 if ($Feat.Installed) {
 Write-Host "[*] Removing Server feature: $FeatName..." -ForegroundColor Yellow
 Uninstall-WindowsFeature -Name $FeatName -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Feature '$FeatName' uninstalled." -ForegroundColor Green
 } else {
 Write-Host "[~] Feature '$FeatName' is already uninstalled." -ForegroundColor Gray
 }
 } else {
 # Try DISM fallback
 $DismFeat = Get-WindowsOptionalFeature -Online -FeatureName $FeatName -ErrorAction SilentlyContinue
 if ($null -ne $DismFeat) {
 if ($DismFeat.State -eq "Enabled" -or $DismFeat.State -eq "EnabledPendingRestart") {
 Write-Host "[*] Disabling optional feature: $FeatName..." -ForegroundColor Yellow
 Disable-WindowsOptionalFeature -Online -FeatureName $FeatName -NoRestart -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Feature '$FeatName' disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Feature '$FeatName' is already disabled." -ForegroundColor Gray
 }
 } else {
 Write-Host "[~] Feature '$FeatName' is not present in the system." -ForegroundColor Gray
 }
 }
 }
} else {
 # Windows Client path
 foreach ($Feature in $DismFeatures) {
 $State = Get-WindowsOptionalFeature -Online -FeatureName $Feature -ErrorAction SilentlyContinue
 }
}
```

```
if ($null -ne $State) {
 if ($State.State -eq "Enabled" -or $State.State -eq "EnabledPendingRestart") {
 Write-Host "[*] Disabling feature: $Feature..." -ForegroundColor Yellow
 Disable-WindowsOptionalFeature -Online -FeatureName $Feature -NoRestart -ErrorAction SilentlyContinue | Out-Null
 Write-Host "[+] Feature '$Feature' has been disabled." -ForegroundColor Green
 } else {
 Write-Host "[~] Feature '$Feature' is already disabled." -ForegroundColor Gray
 }
} else {
 Write-Host "[~] Feature '$Feature' is not present in this Windows image." -ForegroundColor Gray
}
}

Write-Host "Optional features configuration completed." -ForegroundColor Green
```

*To verify the state of unused features:*

[Download Script: Get-UnusedFeaturesStatus.ps1](#)

```
Get-UnusedFeaturesStatus.ps1
Description: Audits the installation state of unused legacy features on the local system.

Write-Host "--- Auditing Unused Windows Features ---" -ForegroundColor Cyan

Check if running as administrator
$isAdmin = ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
if (-not $isAdmin) {
 Write-Error "This script must be run as an Administrator."
 exit 1
}

$script:Vulnerable = $false

$DismFeatures = @(
 "MicrosoftWindowsPowerShellV2",
 "MicrosoftWindowsPowerShellV2Root",
 "NetFx3",
 "SMB1Protocol",
 "Internet-Explorer-Optional-amd64",
 "WorkFolders-Client",
 "Xps-Viewer-Dependency",
 "DirectPlay",
 "TelnetClient",
 "TFTP",
 "SimpleTCP"
 # .NET Framework 3.5
 # SMBv1 Client
 # Internet Explorer 11
 # Work Folders Client
 # XPS Viewer
 # DirectPlay
 # Telnet Client
 # TFTP Client
 # Simple TCP/IP Services
)

$ServerFeatures = @(
 "PowerShell-V2",
 "NET-Framework-Core",
 "FS-SMB1",
 "Internet-Explorer-Optional-amd64",
 "WorkFolders-Client",
 "Xps-Viewer-Dependency",
 "DirectPlay",
 "Telnet-Client",
 "TFTP-Client",
 "Simple-TCP/IP"
)

if (Get-Command Get-WindowsFeature -ErrorAction SilentlyContinue) {
 # Windows Server path
 foreach ($FeatName in $ServerFeatures) {
 $Feat = Get-WindowsFeature -Name $FeatName -ErrorAction SilentlyContinue
 if ($null -ne $Feat) {
 $Color = if ($Feat.Installed -eq $false) { "Green" } else { "Red" }
 Write-Host " - Feature: $FeatName | Installed: $($Feat.Installed) (Expected: False)" -ForegroundColor $Color
 if ($Feat.Installed) {
 $script:Vulnerable = $true
 }
 } else {
 # Try DISM fallback
 $DismFeat = Get-WindowsOptionalFeature -Online -FeatureName $FeatName -ErrorAction SilentlyContinue
 if ($null -ne $DismFeat) {
 $IsEnabled = ($DismFeat.State -eq "Enabled" -or $DismFeat.State -eq "EnabledPendingRestart")
 $Color = if (-not $IsEnabled) { "Green" } else { "Red" }
 Write-Host " - Feature: $FeatName (DISM) | State: $($DismFeat.State) (Expected: Disabled)" -ForegroundColor $Color
 if ($IsEnabled) {
 $script:Vulnerable = $true
 }
 } else {
 Write-Host " - Feature: $FeatName | Not Present (Compliant)" -ForegroundColor Green
 }
 }
 }
} else {
 # Windows Client path
 foreach ($Feature in $DismFeatures) {
 $State = Get-WindowsOptionalFeature -Online -FeatureName $Feature -ErrorAction SilentlyContinue
 if ($null -ne $State) {
 $IsEnabled = ($State.State -eq "Enabled" -or $State.State -eq "EnabledPendingRestart")
 }
 }
}
```

```
$Color = if (-not $IsEnabled) { "Green" } else { "Red" }
Write-Host " - Feature: $Feature | State: $($State.State) (Expected: Disabled)" -ForegroundColor $Color

if ($IsEnabled) {
 $script:Vulnerable = $true
}
} else {
 Write-Host " - Feature: $Feature | Not Present (Compliant)" -ForegroundColor Green
}
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
 exit 1
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
 exit 0
}
```

---

## Sources & Compliance References \* \*\*DoD Windows 11 Computer STIG v2r6\*\*\*: V-220728 (PowerShell 2.0), V-253286 (SMBv1), V-219524 (.NET Framework 3.5) \* \*\*ANSSI AD Hardening Guide\*\*\*: Section on Endpoint Minimization and disabling legacy protocols \* \*\*CIS Microsoft Windows Client Benchmark\*\*\*: Section 18.9 (PowerShell restrictions)

## # [REQ-END-033] Configure Microsoft Office Security and Block OLE Packages

## Target Scope \* \*\*Applicable Systems\*\*: Member Workstations (Endpoints) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: User Configuration\Policies\Administrative Templates\Microsoft Office 2016\Security Settings \* \*\*Registry Locations\*\*: \* `HKCU\software\policies\microsoft\office\16.0\common\security` \* `vbawarnings` = `3` (REG\_DWORD, Enforce macro signing / Block unsigned macros) \* `HKCU\software\policies\microsoft\office\16.0\excel\security` \* `blockcontentexecutionfrominternet` = `1` (REG\_DWORD, Block macros in files from the Internet) \* `HKCU\software\policies\microsoft\office\16.0\word\security` \* `blockcontentexecutionfrominternet` = `1` (REG\_DWORD) \* `HKCU\software\policies\microsoft\office\16.0\powerpoint\security` \* `blockcontentexecutionfrominternet` = `1` (REG\_DWORD) \* `HKCU\software\policies\microsoft\office\16.0\outlook\security` \* `ShowOLEPackageObj` = `0` (REG\_DWORD, Disable OLE package activation)

---

## Rationale Malicious documents (e.g., weaponized Word, Excel, or PowerPoint files) containing embedded VBA macros are a prevalent initial access and execution vector. Similarly, embedding malicious OLE packages inside Outlook items (such as RTF-formatted emails) allows attackers to trigger script execution or execute arbitrary packages via `packager.dll` when an administrator or standard user opens or previews the email.

Hardening these settings ensures:

1. **Internet Macro Blocking:** VBA macros in files downloaded from the Internet or untrusted external attachments are blocked from executing, regardless of user consent.
  2. **Macro Code Signing:** Any locally run macros are restricted to trusted, digitally signed code, preventing the execution of ad-hoc unverified user scripts.
  3. **OLE Package Disablement:** Restricting Outlook OLE package activation ( `ShowOLEPackageObj = 0` ) blocks the execution of dangerous embedded objects in email messages.
- 

## Legacy Impact & Compatibility \* \*\*Office Macros\*\*: Business workflows relying on macro-enabled spreadsheets or documents downloaded from external sources (e.g., vendor portals) will be blocked. Unsigned local macros will also fail to run. Users must acquire certificates to digitally sign internal macro projects. \* \*\*Outlook Packages\*\*: Embedded document shortcuts or packages in emails will be displayed as static icons and cannot be double-clicked for direct execution.

---

## ## Implementation Steps

## #### Option A: Group Policy Object (GPO) Configuration (Preferred)

##### Step 1: Enforce Macro Security in ADMX Templates 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Edit the target endpoints GPO. 3. Navigate to: `User Configuration\Policies\Administrative Templates\Microsoft Office 2016\Security Settings\Trust Center` 4. Set the following policies: \* \*\*Policy\*\*: `VBA Macro Notification Settings` → **Enabled** with option set to **Disable all except digitally signed macros** 5. For each application (Word, Excel, PowerPoint, Access), navigate to: `User Configuration\Policies\Administrative Templates\[Application] 2016\[Application] Options\Security\Trust Center` 6. Set the policy: \* \*\*Policy\*\*: `Block macros from running in Office files from the Internet` → **Enabled**

##### Step 2: Disable Outlook OLE Packages 1. Navigate to: `User Configuration\Policies\Administrative Templates\Microsoft Outlook 2016\Security` 2. Configure the setting: \* \*\*Policy\*\*: `Do not allow OLE package execution` → **Enabled**

---

## #### Option B: PowerShell &amp; Registry Configuration (Remediation / Non-GPO)

Configure the current user registry hives to enforce macro blocking and OLE package restrictions.

[Download Script: Configure-OfficeSecurity.ps1](#)

```
Configure-OfficeSecurity.ps1
Description: Configures registry settings under the HKCU hive to restrict VBA macros and block Outlook OLE package execution.

Write-Host "Applying Microsoft Office security and OLE restrictions..." -ForegroundColor Cyan

Helper to configure User Registry DWORD values
function Set-UserRegDWord {
 [CmdletBinding(SupportsShouldProcess)]
 param (
 [string]$Path,
 [string]$Name,
 [int]$Value
)
 if ($PSCmdlet.ShouldProcess($Path, "Set registry DWORD value $Name to $Value")) {
 $FullRegistryPath = "HKCU:\$Path"
 if (-not (Test-Path $FullRegistryPath)) {
 New-Item -Path $FullRegistryPath -Force | Out-Null
 }
 Set-ItemProperty -Path $FullRegistryPath -Name $Name -Value $Value -Type DWord -Force
 }
}

1. Enforce macro signing policy (common)
Set-UserRegDWord "software\policies\microsoft\office\16.0\common\security" "vbawarnings" 3
Write-Host "[+] Digital signing for Office macros enforced." -ForegroundColor Green

2. Block macros from the Internet for key Office applications
$Apps = @("excel", "word", "powerpoint", "access", "visio")
foreach ($App in $Apps) {
 Set-UserRegDWord "software\policies\microsoft\office\16.0\$App\security" "blockcontentexecutionfrominternet" 1
}
Write-Host "[+] VBA macro blocks from Internet applied to Office applications." -ForegroundColor Green

3. Disable OLE Package execution in Outlook (Policies and Preferences branches)
Set-UserRegDWord "software\policies\microsoft\office\16.0\outlook\security" "ShowOLEPackageObj" 0
Set-UserRegDWord "software\microsoft\office\16.0\outlook\security" "ShowOLEPackageObj" 0
Write-Host "[+] Outlook OLE Package execution blocked." -ForegroundColor Green
```

To verify the current Office security settings:

[Download Script: Get-OfficeSecurityStatus.ps1](#)



```
Get-OfficeSecurityStatus.ps1
Description: Audits Microsoft Office macro settings and Outlook OLE package restrictions.

Write-Host "--- Auditing Microsoft Office Security Baseline ---" -ForegroundColor Cyan

$script:Vulnerable = $false

Helper to audit registry values under HKCU
function Test-UserRegistryValue ($Path, $Name, $ExpectedValue) {
 $FullRegistryPath = "HKCU:\$Path"
 $Val = Get-ItemProperty -Path $FullRegistryPath -Name $Name -ErrorAction SilentlyContinue
 $Actual = if ($Val) { $Val.$Name } else { "" }
 $Color = "Red"
 if ($Actual -eq $ExpectedValue) {
 $Color = "Green"
 } else {
 $script:Vulnerable = $true
 }
 Write-Host " - User Registry: $Name | Actual: '$Actual' (Expected: '$ExpectedValue')" -ForegroundColor $Color
}

1. Audit macro signing warning
Test-UserRegistryValue "software\policies\microsoft\office\16.0\common\security" "vbawarnings" 3

2. Audit macro Internet blocks
$Apps = @("excel", "word", "powerpoint", "access", "visio")
foreach ($App in $Apps) {
 Test-UserRegistryValue "software\policies\microsoft\office\16.0\$App\security" "blockcontentexecutionfrominternet" 1
}

3. Audit Outlook OLE package block
Test-UserRegistryValue "software\policies\microsoft\office\16.0\outlook\security" "ShowOLEPackageObj" 0

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
}
```

---

**## Sources & Compliance References** \* **ANSI AD Hardening Guide**: Recommendations on restricting application execution, blocking external scripts, and application sandboxing. \* **CIS Microsoft Office Benchmark**: Section on Office Common Security settings, VBA warnings, and blocking macros in internet files. \* **Microsoft Security Baseline**: Recommended settings for Microsoft Office and Office 365 ProPlus Security. \* **DoD Microsoft Outlook STIG**: Restrictions on active content, remote attachments, and OLE execution behavior.

# [REQ-END-034] Disable Windows Script Host and Remap Scripting Extensions

## Target Scope \* \*\*Applicable Systems\*\*: Member Workstations (Endpoints) \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path (WSH Disable)\*\*: Computer Configuration\Preferences\Windows Settings\Registry \* \*\*GPO Path (Associations)\*\*: User Configuration\Preferences\Control Panel Settings\Folder Options \* \*\*Registry Location\*\*: \* `HKLM\SOFTWARE\Microsoft\Windows Script Host\Settings` \* `Enabled` = `0` (REG\_DWORD) \* `HKCU\SOFTWARE\Microsoft\Windows Script Host\Settings` \* `Enabled` = `0` (REG\_DWORD)

## Rationale Windows Script Host (WSH), which executes VBScript and JScript files (`wscript.exe` and `cscript.exe`), is frequently targeted by threat actors in phishing campaigns and initial access vectors. By placing a script file (e.g., `.vbs`, `.js`, `.wsf`, `.hta`) in an email attachment or download path, attackers can execute arbitrary code on the system if a user opens the file.

Enforcing these deactivation controls ensures:

1. **Attack Surface Reduction:** Disabling WSH globally blocks the execution of JScript and VBScript files via the standard scripting engines on the workstation.
2. **Defense-in-Depth File Associations:** Remapping the default file handler for typical scripting extensions to `notepad.exe` ensures that if a scripting file is double-clicked by an administrator or user, the file opens as plaintext in Notepad for inspection rather than executing its contents.

## Legacy Impact & Compatibility \* \*\*Script Dependencies\*\*: Any legacy administrative scripts (such as logon scripts or backup routines) written in VBScript or JScript will fail to run. All internal workstation management scripts must be written in PowerShell 5.1+ and executed under secure execution policies. \* \*\*Explorer Associations\*\*: Double-clicking on a `.js` or `.vbs` configuration file will open Notepad instead of running the script.

#### ## Implementation Steps

##### ### Option A: Group Policy Object (GPO) Configuration (Preferred)

#### Step 1: Disable WSH via GPO Registry Preferences 1. Open the **Group Policy Management Console** (`gpmc.msc`). 2. Edit the Endpoint GPO (e.g., `GPO\_Hardening\_Endpoints`). 3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry` 4. Create a new **Registry Item**: \* \*\*Action\*\*: `Update` \* \*\*Hive\*\*: `HKEY\_LOCAL\_MACHINE` \* \*\*Key Path\*\*: `SOFTWARE\Microsoft\Windows Script Host\Settings` \* \*\*Value Name\*\*: `Enabled` \* \*\*Value Type\*\*: `REG\_DWORD` \* \*\*Value Data\*\*: `0`

#### Step 2: Configure Script File Extensions to Open in Notepad 1. Navigate to: `User Configuration\Preferences\Control Panel Settings\Folder Options` 2. Right-click and select **New → Open With**: \* \*\*File Extension\*\*: `vbs` \* \*\*Associated Program\*\*: `%SystemRoot%\System32\notepad.exe` \* \*\*Set as default\*\*: Check 3. Repeat the process for the following extensions: `vbe`, `js`, `jse`, `wsf`, `wsh`, `hta`.

##### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Configure the local registry settings to disable WSH and remap associations.

[Download Script: Disable-Wsh.ps1](#)

```
Disable-Wsh.ps1
Description: Disables Windows Script Host globally in HKLM and HKCU registry hives, and remaps script file associations to Notepad.

Write-Host "Applying Windows Script Host and file association hardening..." -ForegroundColor Cyan

1. Disable WSH globally
$RegistryHkLm = "HKLM:\SOFTWARE\Microsoft\Windows Script Host\Settings"
if (-not (Test-Path $RegistryHkLm)) {
 New-Item -Path $RegistryHkLm -Force | Out-Null
}
Set-ItemProperty -Path $RegistryHkLm -Name "Enabled" -Value 0 -Type DWord -Force
Write-Host "[+] WSH globally disabled in HKLM." -ForegroundColor Green

$RegistryHkcu = "HKCU:\SOFTWARE\Microsoft\Windows Script Host\Settings"
if (-not (Test-Path $RegistryHkcu)) {
 New-Item -Path $RegistryHkcu -Force | Out-Null
}
Set-ItemProperty -Path $RegistryHkcu -Name "Enabled" -Value 0 -Type DWord -Force
Write-Host "[+] WSH disabled in current user HKCU hive." -ForegroundColor Green

2. Remap script file extensions to notepad
$Extensions = @("vbs", "vbe", "js", "jse", "wsf", "wsh", "hta")
foreach ($Ext in $Extensions) {
 $ProgIdPath = "HKLM:\SOFTWARE\Classes\.$Ext"

 # Update Class Association to Notepad
 if (-not (Test-Path $ProgIdPath)) {
 New-Item -Path $ProgIdPath -Force | Out-Null
 }
 Set-ItemProperty -Path $ProgIdPath -Name "" -Value "txtfile" -Type String -Force
 Write-Host " Mapped .$Ext extension to txtfile handler." -ForegroundColor Gray
}
Write-Host "[+] Script file extension handlers mapped to Notepad." -ForegroundColor Green
```

To verify the WSH configuration state:

[Download Script: Get-WshStatus.ps1](#)

```

Get-WshStatus.ps1
Description: Audits Windows Script Host registry state and script file extension association handlers.

Write-Host "--- Auditing Windows Script Host Hardening ---" -ForegroundColor Cyan

$script:Vulnerable = $false

1. Audit WSH Registry settings
$RegistryHkLm = "HKLM:\SOFTWARE\Microsoft\Windows Script Host\Settings"
if (Test-Path $RegistryHkLm) {
 $ValHkLm = (Get-ItemProperty -Path $RegistryHkLm -Name "Enabled" -ErrorAction SilentlyContinue).Enabled
 if ($ValHkLm -eq 0) {
 Write-Host " - HKLM WSH Enabled: 0 (Secure)" -ForegroundColor Green
 } else {
 Write-Host " - VULNERABLE: HKLM WSH is enabled or not configured (Value: '$ValHkLm')" -ForegroundColor Red
 $script:Vulnerable = $true
 }
} else {
 Write-Host " - VULNERABLE: HKLM WSH settings key is missing (Expected: Enabled = 0)" -ForegroundColor Red
 $script:Vulnerable = $true
}

2. Audit file associations
$Extensions = @("vbs", "vbe", "js", "jse", "wsf", "wsh", "hta")
foreach ($Ext in $Extensions) {
 $ProgIdPath = "HKLM:\SOFTWARE\Classes\.$Ext"
 if (Test-Path $ProgIdPath) {
 $Handler = (Get-ItemProperty -Path $ProgIdPath -Name "" -ErrorAction SilentlyContinue).""
 if ($Handler -eq "txtfile" -or $Handler -match "notepad") {
 Write-Host " - Extension .$Ext Handler: $Handler (Secure)" -ForegroundColor Green
 } else {
 Write-Host " - VULNERABLE: Extension .$Ext Handler is '$Handler' (Expected: txtfile/notepad)" -ForegroundColor Red
 $script:Vulnerable = $true
 }
 } else {
 Write-Host " - VULNERABLE: Extension .$Ext Class Registry key not found." -ForegroundColor Red
 $script:Vulnerable = $true
 }
}

if ($script:Vulnerable) {
 Write-Host "Audit Result: VULNERABLE" -ForegroundColor Red
} else {
 Write-Host "Audit Result: SECURE" -ForegroundColor Green
}

```

## Sources & Compliance References \* \*\*ANSSI AD Hardening Guide\*\*\*: Recommendations on workstation OS minimization and script execution control. \* \*\*DoD Windows 11 Computer STIG v2r6\*\*\*: V-219661 (Windows Script Host disablement and script restriction). \* \*\*CIS Microsoft Windows Client Benchmark\*\*\*: Section 18.9 (Administrative templates for script execution safety).

**# [REQ-END-035] Configure Secure Boot Revocations and Bootloader Updates**

**## Target Scope** \* \*\*Applicable Systems\*\*: Tier 2 client workstations and member servers. \* \*\*Operating Systems\*\*: Windows 10 (and above) Enterprise/Professional, Windows Server 2016 (and above).

**## Implementation Details** \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*BlackLotus Revocation Updates (CVE-2023-24932)\*\*: 'HKLM\SYSTEM\CurrentControlSet\Control\Secureboot' \* 'AvailableUpdates' = '>= 0x4000' (REG\_DWORD)

**## Rationale** A vulnerability in the Windows Boot Manager allows an attacker with physical access or local administrative rights to bypass UEFI Secure Boot and execute unsigned code during the boot process (BlackLotus bootkit).

To fully mitigate this threat (CVE-2023-24932), Windows update revocations must be applied to the UEFI variables (DBX list) and code integrity SVN policies must be updated. This is managed via the `AvailableUpdates` registry key, which instructs the OS boot manager to write the revocation variables to firmware.

According to the latest Microsoft guidelines, the recommended trigger value for enterprise deployments to apply all security updates (including the new Windows UEFI CA 2023 certificates and boot manager updates) is `0x5944` (hex) / `22852` (decimal). As the OS processes this bitmask, the value is cleared incrementally, ending up at `0x4000` (hex) / `16384` (decimal) upon successful completion.

**## Legacy Impact & Compatibility** \* \*\*BlackLotus Mitigation Risks\*\*: Enforcing the BlackLotus DBX and SVN updates is a permanent, non-reversible action once written to the device firmware. If an administrator attempts to boot the machine using older, unpatched Windows installation media or recovery disks, the system will reject the boot manager and fail to boot. All recovery and deployment media must be updated with current security patches before applying these mitigations.

**## Implementation Steps****### Option A: Group Policy Object (GPO) Configuration (Preferred)**

To configure the update triggers for the DBX and Code Integrity boot manager revocations, define Registry GPO Preferences inside the endpoints GPO:

1. Open the **Group Policy Management Console** ( `gpmc.msc` ).
2. Create or edit a GPO linked to the workstations OU (e.g., `GPO_Hardening_Workstations` ).
3. Navigate to: `Computer Configuration\Preferences\Windows Settings\Registry`
4. Create a registry item to deploy the `AvailableUpdates` DWORD under `HKLM\SYSTEM\CurrentControlSet\Control\Secureboot` .
  - **Action:** `Update`
  - **Hive:** `HKEY_LOCAL_MACHINE`
  - **Key Path:** `SYSTEM\CurrentControlSet\Control\Secureboot`
  - **Value Name:** `AvailableUpdates`
  - **Value Type:** `REG_DWORD`
  - **Value Data:** `22852` (Decimal) or `5944` (Hex)

**### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)**

Run the following script locally to configure the BlackLotus mitigation update trigger:

[Download Script: Set-SecureBootRevocations.ps1](#)

```
Set-SecureBootRevocations.ps1
Description: Triggers Secure Boot DBX and Code Integrity revocation updates for BlackLotus mitigation.

Write-Host "--- Configuring BlackLotus Secure Boot Mitigations ---" -ForegroundColor Cyan

$Path = "HKLM:\SYSTEM\CurrentControlSet\Control\Secureboot"
if (-not (Test-Path $Path)) {
 New-Item -Path $Path -Force | Out-Null
}

Trigger updates (0x5944 = 22852)
Set-ItemProperty -Path $Path -Name "AvailableUpdates" -Value 22852 -Type DWord -Force | Out-Null
Write-Host "[+] BlackLotus DBX and 2023 CA revocation updates configured in registry. A system reboot is required." -ForegroundColor Green
```

**### Audit Script**

Run the following script to check the status of Secure Boot revocations on the local machine:

[Download Script: Audit-SecureBootRevocations.ps1](#)

```
Audit-SecureBootRevocations.ps1
Description: Queries UEFI Secure Boot parameters and audits BlackLotus mitigation registry settings.

Write-Host "--- Auditing BlackLotus Mitigations ---" -ForegroundColor Cyan

$script:NonCompliant = $false

1. Audit AvailableUpdates registry key
$Path = "HKLM:\SYSTEM\CurrentControlSet\Control\Secureboot"
if (Test-Path $Path) {
 $Val = Get-ItemProperty -Path $Path -Name "AvailableUpdates" -ErrorAction SilentlyContinue
 $UpdateVal = if ($Val) { $Val.AvailableUpdates } else { 0 }

 # Check if configured (≥ 0x4000 / 16384)
 if ($UpdateVal -ge 16384) {
 Write-Host " - BlackLotus Revocation Updates (AvailableUpdates): $UpdateVal (Compliant)" -ForegroundColor Green
 } else {
 Write-Host " - BlackLotus Revocation Updates (AvailableUpdates): $UpdateVal (Non-Compliant - DBX/SVN revocations not triggered)" -ForegroundColor Red
 $script:NonCompliant = $true
 }
} else {
 Write-Host " - BlackLotus Revocation Updates: Registry path not found (Non-Compliant)" -ForegroundColor Red
 $script:NonCompliant = $true
}

2. Audit UEFICA2023Status (if present)
$ServicingPath = "HKLM:\SYSTEM\CurrentControlSet\Control\SecureBoot\Servicing"
if (Test-Path $ServicingPath) {
 $ServVal = Get-ItemProperty -Path $ServicingPath -Name "UEFICA2023Status" -ErrorAction SilentlyContinue
 if ($ServVal) {
 $Status = $ServVal.UEFICA2023Status
 $Color = if ($Status -eq "Updated") { "Green" } else { "Yellow" }
 Write-Host " - UEFI CA 2023 Update Status: $Status" -ForegroundColor $Color
 }
}

if ($script:NonCompliant) {
 exit 1
}
```

## Sources & Compliance References \* \*\*Microsoft KB5068202\*\*: Registry key updates for Secure Boot: Windows devices with IT-managed updates \* \*\*ANSSI AD Hardening Guide\*\*: Recommendations regarding hardware platform integrity.

## # [REQ-END-036] Enable WDAC Driver Blocklist

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 client workstations \* \*\*Operating Systems\*\*: Windows 10, Windows 11 (Enterprise and Professional editions)

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: Computer Configuration\Administrative Templates\System\Device Guard\Deploy Windows Defender Application Control \* \*\*Registry Location\*\*: 'HKLM\SYSTEM\CurrentControlSet\Control\CI\Config' → 'VulnerableDriverBlocklistEnable' = '1' (REG\_DWORD)

## Rationale Attackers frequently employ "Bring Your Own Vulnerable Driver" (BYOVD) attacks to bypass Windows kernel protections on standard endpoints. In a BYOVD attack, an adversary with administrative privileges installs a legitimate, cryptographically signed third-party driver that contains a known, exploitable vulnerability. The attacker then exploits this vulnerability to execute arbitrary code with kernel privileges, allowing them to disable security agents, dump LSASS memory, or tamper with system integrity. Enforcing the **Microsoft Vulnerable Driver Blocklist** via Windows Defender Application Control (WDAC) prevents known vulnerable or malicious drivers from loading in kernel space. By restricting the WDAC policy to **Kernel Mode Code Integrity (KMCI) only** (omitting user-mode enforcement), the control shields the system kernel from driver-based exploits on endpoint hosts without introducing administrative overhead or blocking standard user-mode applications.

## Legacy Impact & Compatibility \* \*\*Pre-requisite (Memory Integrity/HVCI)\*\*: The vulnerable driver blocklist requires Hypervisor-Protected Code Integrity (HVCI) for secure, hypervisor-enforced validation. Refer to [REQ-END-010 - Enable VBS and Credential Guard](#08-endpoints-enable-vbs-credential-guard-md) to ensure Virtualization-Based Security (VBS) and Memory Integrity (HVCI) are fully enabled. Secure Boot and CPU virtualization are strict pre-requisites; refer to [REQ-END-013 - UEFI Firmware Security Hardening](#08-endpoints-configure-uefi-security-md) and [REQ-END-014 - Enable Hardware Virtualization and DMA Protection](#08-endpoints-enable-hardware-virtualization-and-dma-protection-md) for firmware configuration. \* \*\*Compatibility with Legacy Drivers\*\*: Third-party backup, monitoring, or hardware administration software running deprecated, vulnerable drivers may fail to load. All such software must be updated to use secure, modern drivers. \* \*\*Deployment Testing\*\*: To prevent system instability, the WDAC blocklist policy should be deployed in **Audit Mode** initially to verify that no critical operational drivers are blocked in production before shifting to enforcement mode.

## ## Implementation Steps

## ### Option A: Group Policy Object (GPO) Configuration (Preferred)

To enforce the driver blocklist across all endpoints, you can deploy the Microsoft recommended block rules as a custom WDAC policy.

1. Download the Microsoft recommended driver block rules XML from the official Microsoft documentation.
2. Edit the XML to ensure it operates in **Audit Mode** first, then convert the XML configuration into a binary format:

```
ConvertFrom-CIPolicy -XmlFilePath "C:\WDAC\DriverBlocklist.xml" -BinaryFilePath "C:\WDAC\SIPolicy.p7b"
```

3. Copy the compiled `SIPolicy.p7b` file to a secure local path on all target endpoints (e.g., `C:\Windows\System32\CodeIntegrity\SIPolicy.p7b`) or a share.
4. Open the **Group Policy Management Console** ( `gpmc.msc` ) on a domain management host.
5. Create or edit a GPO linked to the workstations OU (e.g., `GPO_Hardening_Workstations` ).
6. Navigate to: `Computer Configuration\Administrative Templates\System\Device Guard`
7. Configure the following setting:
  - **Policy:** `Deploy Windows Defender Application Control`
  - **Setting:** `Enabled`
  - **Code Integrity Policy File Path:** Enter the local or network path to the policy file (e.g., `C:\Windows\System32\CodeIntegrity\SIPolicy.p7b` ).
8. To ensure the built-in system driver blocklist is active on modern builds, configure the following registry setting via Group Policy Preferences:
  - **Path:** `Computer Configuration\Preferences\Windows Settings\Registry`
  - **Hive:** `HKEY_LOCAL_MACHINE`
  - **Key Path:** `SYSTEM\CurrentControlSet\Control\CI\Config`
  - **Value Name:** `VulnerableDriverBlocklistEnable`
  - **Value Type:** `REG_DWORD`
  - **Value Data:** `1`

### ### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)

Run the following scripts locally to enable the Vulnerable Driver Blocklist registry key and ensure proper configuration.

#### # Configure-DriverBlocklist.ps1

[Download Script: Configure-DriverBlocklist.ps1](#)

```
Configure-DriverBlocklist.ps1
Description: Enables the Microsoft Vulnerable Driver Blocklist in the registry and validates VBS/HVCI settings.

Write-Host "Applying hardening requirement: Enable WDAC Driver Blocklist..." -ForegroundColor Cyan

1. Configure the registry settings to enable the blocklist
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\CI\Config"
$ValueName = "VulnerableDriverBlocklistEnable"

if (-not (Test-Path $RegPath)) {
 Write-Host "[+] Creating registry path: $RegPath" -ForegroundColor Gray
 New-Item -Path $RegPath -Force | Out-Null
}

Write-Host "[+] Setting registry value: $ValueName = 1" -ForegroundColor Gray
Set-ItemProperty -Path $RegPath -Name $ValueName -Value 1 -Type DWord -ErrorAction Stop

2. Validate VBS / HVCI Configuration
$ScenariosPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\HypervisorEnforcedCodeIntegrity"
if (Test-Path $ScenariosPath) {
 $HvciStatus = Get-ItemProperty -Path $ScenariosPath -Name "Enabled" -ErrorAction SilentlyContinue
 if ($null -ne $HvciStatus -and $HvciStatus.Enabled -eq 1) {
 Write-Host "[+] Pre-requisite Check: Memory Integrity (HVCI) is enabled." -ForegroundColor Green
 } else {
 Write-Host "[!] Warning: Memory Integrity (HVCI) is disabled. The blocklist requires HVCI for hypervisor enforcement." -ForegroundColor Yellow
 }
} else {
 Write-Host "[!] Warning: Memory Integrity scenario configuration not found. Check VBS settings." -ForegroundColor Yellow
}

Write-Host "[+] Configuration applied successfully. A reboot is required to activate the blocklist." -ForegroundColor Green
```

*To verify the setting has been applied:*

#### # Get-DriverBlocklistStatus.ps1

[Download Script: Get-DriverBlocklistStatus.ps1](#)



```
Get-DriverBlocklistStatus.ps1
Description: Audits the configuration of the Microsoft Vulnerable Driver Blocklist and HVCI state.

Write-Host "--- Auditing Vulnerable Driver Blocklist ---" -ForegroundColor Cyan
$Vulnerable = $false

1. Check registry value
$RegPath = "HKLM:\SYSTEM\CurrentControlSet\Control\CI\Config"
$ValueName = "VulnerableDriverBlocklistEnable"

if (Test-Path $RegPath) {
 $RegValue = Get-ItemProperty -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
 if ($null -ne $RegValue -and $RegValue.$ValueName -eq 1) {
 Write-Host "[+] Vulnerable Driver Blocklist is enabled in the registry." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Vulnerable Driver Blocklist is disabled or not set in the registry." -ForegroundColor Red
 $Vulnerable = $true
 }
} else {
 Write-Host "[!] VULNERABLE: Code Integrity Config registry key does not exist." -ForegroundColor Red
 $Vulnerable = $true
}

2. Check HVCI Status
$ScenariosPath = "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\HypervisorEnforcedCodeIntegrity"
if (Test-Path $ScenariosPath) {
 $HvciStatus = Get-ItemProperty -Path $ScenariosPath -Name "Enabled" -ErrorAction SilentlyContinue
 if ($null -ne $HvciStatus -and $HvciStatus.Enabled -eq 1) {
 Write-Host "[+] Memory Integrity (HVCI) is enabled." -ForegroundColor Green
 } else {
 Write-Host "[!] VULNERABLE: Memory Integrity (HVCI) is disabled in the registry." -ForegroundColor Red
 $Vulnerable = $true
 }
} else {
 Write-Host "[!] VULNERABLE: Memory Integrity scenario registry path does not exist." -ForegroundColor Red
 $Vulnerable = $true
}

3. Final Verdict
if ($Vulnerable) {
 Write-Host "`n[!] Verification FAILED: The Vulnerable Driver Blocklist is not fully secured." -ForegroundColor Red
} else {
 Write-Host "`n[+] Verification PASSED: The Vulnerable Driver Blocklist and HVCI are correctly configured." -ForegroundColor Green
}
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Recommendations on system component code integrity and driver signature enforcement. \* \*\*CIS Microsoft Windows 10/11 Benchmark\*\*\*: Section 18.8.14.3 / 18.9.31.2 (Deploy Windows Defender Application Control / Memory Integrity). \* \*\*Microsoft Security Guidance\*\*\*: Microsoft recommended driver block rules documentation.

**# [REQ-END-140] Configure Advanced Security Audit Policies for Endpoints****## Target Scope** \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

**## Implementation Details** \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Path\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` \* \*\*Policy\*\*: `Audit: Force audit policy subcategory settings (Windows Vista or later) to override audit policy category settings` \* \*\*Setting\*\*: `Enabled` \* \*\*Registry Location\*\*: `HKLM\System\CurrentControlSet\Control\Lsa\SCENoApplyLegacyAuditPolicy` = `1` (DWord)

**## Rationale** Enforcing category overrides on client workstations is a prerequisite to ensure that the refined audit subcategories (such as Logon, Object Access, System Events, etc.) are correctly logged and not masked by basic legacy policies.

**## Legacy Impact & Compatibility** \* \*\*Event Log Volume\*\*: Ensure the Security log size is at least 512MB to prevent rollover.

**## Submodule Requirements** The child policies under this parent requirement: \* \*\*[REQ-END-141 - Audit Policy: Advanced Audit Policy Overrides for Endpoints](#08-endpoints-audit-policy-configure-end-audit-audit-override-md)\*\* \* \*\*[REQ-END-142 - Audit Policy: Account Logon Auditing for Endpoints](#08-endpoints-audit-policy-configure-end-audit-account-logon-md)\*\* \* \*\*[REQ-END-143 - Audit Policy: Account Management Auditing for Endpoints](#08-endpoints-audit-policy-configure-end-audit-account-management-md)\*\* \* \*\*[REQ-END-144 - Audit Policy: Detailed Tracking Auditing for Endpoints](#08-endpoints-audit-policy-configure-end-audit-detailed-tracking-md)\*\* \* \*\*[REQ-END-145 - Audit Policy: Logon and Logoff Auditing for Endpoints](#08-endpoints-audit-policy-configure-end-audit-logon-logoff-md)\*\* \* \*\*[REQ-END-146 - Audit Policy: Object Access Auditing for Endpoints](#08-endpoints-audit-policy-configure-end-audit-object-access-md)\*\* \* \*\*[REQ-END-147 - Audit Policy: Policy Change Auditing for Endpoints](#08-endpoints-audit-policy-configure-end-audit-policy-change-md)\*\* \* \*\*[REQ-END-148 - Audit Policy: Privilege Use Auditing for Endpoints](#08-endpoints-audit-policy-configure-end-audit-privilege-use-md)\*\* \* \*\*[REQ-END-149 - Audit Policy: System Events Auditing for Endpoints](#08-endpoints-audit-policy-configure-end-audit-system-events-md)\*\*

**## Implementation Steps**

**### Option A: Group Policy Object (GPO) Configuration (Preferred)** 1. Navigate to: `Computer Configuration\Policies\Windows Settings\Security Settings\Local Policies\Security Options` 2. Configure the following setting: \* \*\*Policy\*\*: `Audit: Force audit policy subcategory settings (Windows Vista or later) to override audit policy category settings` \* \*\*Setting\*\*: `Enabled`

**### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO)** [Download Script: [Configure-EndAuditPoliciesParent.ps1](#)] ([./implementation\\_scripts/Configure-EndAuditPoliciesParent.ps1](#))

```
Configure-EndAuditPoliciesParent.ps1
Description: Enforces Advanced Audit Policy Overrides registry value.

$RegPath = "reg:\HKLM\System\CurrentControlSet\Control\Lsa"
$ValueName = "SCENoApplyLegacyAuditPolicy"

if (-not (Test-Path $RegPath)) {
 New-Item -Path $RegPath -Force | Out-Null
}

Set-ItemProperty -Path $RegPath -Name $ValueName -Value 1 -Type DWord -Force
Write-Host "Enforced SCENoApplyLegacyAuditPolicy = 1 on Endpoint" -ForegroundColor Green
```

To verify the setting has been applied: [Download Script: Get-EndAuditPoliciesParentStatus.ps1](#)

```
Get-EndAuditPoliciesParentStatus.ps1
Description: Audits the Advanced Audit Policy Overrides registry value.

$RegPath = "HKLM:\System\CurrentControlSet\Control\Lsa"
$ValueName = "SCENoApplyLegacyAuditPolicy"

$val = Get-ItemPropertyValue -Path $RegPath -Name $ValueName -ErrorAction SilentlyContinue
if ($val -eq 1) {
 Write-Host "Advanced Security Audit Policy Overrides are correctly enabled." -ForegroundColor Green
} else {
 Write-Host "Advanced Security Audit Policy Overrides are disabled!" -ForegroundColor Red
}
```

**## Sources & Compliance References** \* \*\*ANSSI AD Hardening Guide\*\*: Recommendation R48 \* \*\*CIS Benchmark\*\*: Section 9 and 17

# [REQ-END-141] Audit Policy: Advanced Audit Policy Overrides for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Registry: `HKLM\System\CurrentControlSet\Control\Lsa\ SCENoApplyLegacyAuditPolicy` = `1` (DWord) \* Registry: `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters\ LogLevel` = `0` (DWord)

## Rationale Enforcing advanced audit policy overrides prevents legacy category settings from overriding refined subcategory policies, and disabling verbose Kerberos logging ensures that event logs are not flooded with diagnostic events.

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Set local Security Event Log size to a minimum of 512MB to prevent premature rollover of security auditing data.

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Registry: `HKLM\System\CurrentControlSet\Control\Lsa\ SCENoApplyLegacyAuditPolicy` = `1` (DWord) \* Registry: `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters\ LogLevel` = `0` (DWord)

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditAuditoverride.ps1](#)] ([./implementation\\_scripts/Configure-EndAuditAuditoverride.ps1](#))

```
Configure-EndAuditAuditoverride.ps1
Write-Host "Applying Audit Policy category: audit-override..." -ForegroundColor Cyan

Set Registry Override: SCENoApplyLegacyAuditPolicy
if (-not (Test-Path "reg:\HKLM\System\CurrentControlSet\Control\Lsa")) { New-Item -Path "reg:\HKLM\System\CurrentControlSet\Control\Lsa" -Force | Out-Null }
Set-ItemProperty -Path "reg:\HKLM\System\CurrentControlSet\Control\Lsa" -Name "SCENoApplyLegacyAuditPolicy" -Value 1 -Type DWord -Force
Write-Host " Enforced SCENoApplyLegacyAuditPolicy = 1" -ForegroundColor Green

Set Registry Override: LogLevel
if (-not (Test-Path "reg:\HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters")) { New-Item -Path "reg:\HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters" -Force | Out-Null }
Set-ItemProperty -Path "reg:\HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters" -Name "LogLevel" -Value 0 -Type DWord -Force
Write-Host " Enforced LogLevel = 0" -ForegroundColor Green
```

To audit the hardening status: [Download Script: Get-EndAuditAuditoverrideStatus.ps1](#)

```
Get-EndAuditAuditoverrideStatus.ps1
$script:Vulnerable = $false

Audit Registry: SCENoApplyLegacyAuditPolicy
$RegVal = Get-ItemProperty -Path "reg:\HKLM\System\CurrentControlSet\Control\Lsa" -Name "SCENoApplyLegacyAuditPolicy" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.SCENoApplyLegacyAuditPolicy -ne 1) {
 $script:Vulnerable = $true
}

Audit Registry: LogLevel
$RegVal = Get-ItemProperty -Path "reg:\HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters" -Name "LogLevel" -ErrorAction SilentlyContinue
if (-not $RegVal -or $RegVal.LogLevel -ne 0) {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client auditing baselines \* \*\*CIS Windows 10/11 Benchmark\*\*: Section 17 (Advanced Audit Policy Configuration)

# [REQ-END-142] Audit Policy: Account Logon Auditing for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Credential Validation` → `Success and Failure`

## Rationale Auditing account logon events captures authentication requests processed by the local system or the domain controller, which is critical for identifying Kerberoasting, NTLM relaying, and brute-force attempts.

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Set local Security Event Log size to a minimum of 512MB to prevent premature rollover of security auditing data.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Credential Validation` → `Success and Failure`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditAccountlogon.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditAccountlogon.ps1](#))

```
Configure-EndAuditAccountlogon.ps1
Write-Host "Applying Audit Policy category: account-logon..." -ForegroundColor Cyan

Set Audit Subcategory: Credential Validation
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Credential Validation`" /success:enable /failure:enable" -Wait
-NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Credential Validation to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Credential Validation. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-EndAuditAccountlogonStatus.ps1](#)

```
Get-EndAuditAccountlogonStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Credential Validation
$RawOutput = auditpol.exe /get /subcategory:"Credential Validation" /r
if ($RawOutput -notmatch ",Credential Validation,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client auditing baselines \* \*\*CIS Windows 10/11 Benchmark\*\*: Section 17 (Advanced Audit Policy Configuration)

# [REQ-END-143] Audit Policy: Account Management Auditing for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `User Account Management` → `Success and Failure` \* Subcategory: `Security Group Management` → `Success and Failure` \* Subcategory: `Other Account Management Events` → `Success and Failure`

## Rationale Auditing account management logs security principal modifications (creations, deletions, password resets, group modifications) to detect privilege escalation attempts on domain or local administrative groups.

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Set local Security Event Log size to a minimum of 512MB to prevent premature rollover of security auditing data.

### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `User Account Management` → `Success and Failure` \* Subcategory: `Security Group Management` → `Success and Failure` \* Subcategory: `Other Account Management Events` → `Success and Failure`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditAccountmanagement.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditAccountmanagement.ps1](#))

```
Configure-EndAuditAccountmanagement.ps1
Write-Host "Applying Audit Policy category: account-management..." -ForegroundColor Cyan

Set Audit Subcategory: User Account Management
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`User Account Management`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory User Account Management to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory User Account Management. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Security Group Management
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Security Group Management`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Security Group Management to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Security Group Management. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Other Account Management Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Other Account Management Events`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Other Account Management Events to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Other Account Management Events. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-EndAuditAccountmanagementStatus.ps1](#)

```
Get-EndAuditAccountmanagementStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: User Account Management
$RawOutput = auditpol.exe /get /subcategory:"User Account Management" /r
if ($RawOutput -notmatch ",User Account Management,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Security Group Management
$RawOutput = auditpol.exe /get /subcategory:"Security Group Management" /r
if ($RawOutput -notmatch ",Security Group Management,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Other Account Management Events
$RawOutput = auditpol.exe /get /subcategory:"Other Account Management Events" /r
if ($RawOutput -notmatch ",Other Account Management Events,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client auditing baselines \* \*\*CIS Windows 10/11 Benchmark\*\*: Section 17 (Advanced Audit Policy Configuration)

# [REQ-END-144] Audit Policy: Detailed Tracking Auditing for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Process Creation` → `Success and Failure` \* Subcategory: `DPAPI Activity` → `Success and Failure` \* Subcategory: `PNP Activity` → `Success`

## Rationale Detailed tracking records process creations and device arrivals to ensure EDR/SIEM visibility into executable command lines and hardware plug events.

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Set local Security Event Log size to a minimum of 512MB to prevent premature rollover of security auditing data.

### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Process Creation` → `Success and Failure` \* Subcategory: `DPAPI Activity` → `Success and Failure` \* Subcategory: `PNP Activity` → `Success`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditDetailedtracking.ps1](#)] (./implementation\_scripts/Configure-EndAuditDetailedtracking.ps1)

```
Configure-EndAuditDetailedtracking.ps1
Write-Host "Applying Audit Policy category: detailed-tracking..." -ForegroundColor Cyan

Set Audit Subcategory: Process Creation
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Process Creation`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Process Creation to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Process Creation. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: DPAPI Activity
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`DPAPI Activity`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory DPAPI Activity to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory DPAPI Activity. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: PNP Activity
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`PNP Activity`" /success:enable /failure:disable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory PNP Activity to Success" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory PNP Activity. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-EndAuditDetailedtrackingStatus.ps1](#)



```
Get-EndAuditDetailedtrackingStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Process Creation
$RawOutput = auditpol.exe /get /subcategory:"Process Creation" /r
if ($RawOutput -notmatch ",Process Creation,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: DPAPI Activity
$RawOutput = auditpol.exe /get /subcategory:"DPAPI Activity" /r
if ($RawOutput -notmatch ",DPAPI Activity,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: PNP Activity
$RawOutput = auditpol.exe /get /subcategory:"PNP Activity" /r
if ($RawOutput -notmatch ",PNP Activity,.*,Success") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Client auditing baselines \* \*\*CIS Windows 10/11 Benchmark\*\*\*: Section 17 (Advanced Audit Policy Configuration)

# [REQ-END-146] Audit Policy: Logon and Logoff Auditing for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Logon` → `Success and Failure` \* Subcategory: `Logoff` → `Success` \* Subcategory: `Special Logon` → `Success and Failure` \* Subcategory: `Account Lockout` → `Success and Failure` \* Subcategory: `Other Logon/Logoff Events` → `Success and Failure`

---

## Rationale Auditing logon/logoff events monitors administrative session states, special elevations, and failed logon attempts, which is critical for finding unauthorized remote access or lateral movement.

---

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Set local Security Event Log size to a minimum of 512MB to prevent premature rollover of security auditing data.

---

#### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Logon` → `Success and Failure` \* Subcategory: `Logoff` → `Success` \* Subcategory: `Special Logon` → `Success and Failure` \* Subcategory: `Account Lockout` → `Success and Failure` \* Subcategory: `Other Logon/Logoff Events` → `Success and Failure`

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditLogonlogoff.ps1](#)]

(../implementation\_scripts/Configure-EndAuditLogonlogoff.ps1)

```
Configure-EndAuditLogonLogoff.ps1
Write-Host "Applying Audit Policy category: logon-logoff..." -ForegroundColor Cyan

Set Audit Subcategory: Logon
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Logon`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Logon to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Logon. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Logoff
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Logoff`" /success:enable /failure:disable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Logoff to Success" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Logoff. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Special Logon
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Special Logon`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Special Logon to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Special Logon. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Account Lockout
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Account Lockout`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Account Lockout to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Account Lockout. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Other Logon/Logoff Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Other Logon/Logoff Events`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Other Logon/Logoff Events to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Other Logon/Logoff Events. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-EndAuditLogonlogoffStatus.ps1](#)

```

Get-EndAuditLogonLogoffStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Logon
$RawOutput = auditpol.exe /get /subcategory:"Logon" /r
if ($RawOutput -notmatch ",Logon,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Logoff
$RawOutput = auditpol.exe /get /subcategory:"Logoff" /r
if ($RawOutput -notmatch ",Logoff,.*, Success") {
 $script:Vulnerable = $true
}

Audit Subcategory: Special Logon
$RawOutput = auditpol.exe /get /subcategory:"Special Logon" /r
if ($RawOutput -notmatch ",Special Logon,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Account Lockout
$RawOutput = auditpol.exe /get /subcategory:"Account Lockout" /r
if ($RawOutput -notmatch ",Account Lockout,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Other Logon/Logoff Events
$RawOutput = auditpol.exe /get /subcategory:"Other Logon/Logoff Events" /r
if ($RawOutput -notmatch ",Other Logon/Logoff Events,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}

```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client auditing baselines \* \*\*CIS Windows 10/11 Benchmark\*\*: Section 17 (Advanced Audit Policy Configuration)

# [REQ-END-147] Audit Policy: Object Access Auditing for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Registry` → `Failure` \* Subcategory: `File Share` → `Failure` \* Subcategory: `Detailed File Share` → `Failure` \* Subcategory: `Handle Manipulation` → `Failure`

## Rationale Auditing object access (files, registry keys, and shares) helps monitor unauthorized modifications to system configuration files and access to restricted shares.

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Set local Security Event Log size to a minimum of 512MB to prevent premature rollover of security auditing data.

### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Registry` → `Failure` \* Subcategory: `File Share` → `Failure` \* Subcategory: `Detailed File Share` → `Failure` \* Subcategory: `Handle Manipulation` → `Failure`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditObjectaccess.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditObjectaccess.ps1](#))

```
Configure-EndAuditObjectaccess.ps1
Write-Host "Applying Audit Policy category: object-access..." -ForegroundColor Cyan

Set Audit Subcategory: Registry
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Registry`" /success:disable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Registry to Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Registry. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: File Share
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"File Share`" /success:disable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory File Share to Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory File Share. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Detailed File Share
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Detailed File Share`" /success:disable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Detailed File Share to Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Detailed File Share. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Handle Manipulation
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Handle Manipulation`" /success:disable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Handle Manipulation to Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Handle Manipulation. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-EndAuditObjectaccessStatus.ps1](#)

```

Get-EndAuditObjectAccessStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Registry
$RawOutput = auditpol.exe /get /subcategory:"Registry" /r
if ($RawOutput -notmatch ",Registry,.*,Failure") {
 $script:Vulnerable = $true
}

Audit Subcategory: File Share
$RawOutput = auditpol.exe /get /subcategory:"File Share" /r
if ($RawOutput -notmatch ",File Share,.*,Failure") {
 $script:Vulnerable = $true
}

Audit Subcategory: Detailed File Share
$RawOutput = auditpol.exe /get /subcategory:"Detailed File Share" /r
if ($RawOutput -notmatch ",Detailed File Share,.*,Failure") {
 $script:Vulnerable = $true
}

Audit Subcategory: Handle Manipulation
$RawOutput = auditpol.exe /get /subcategory:"Handle Manipulation" /r
if ($RawOutput -notmatch ",Handle Manipulation,.*,Failure") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}

```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Client auditing baselines \* \*\*CIS Windows 10/11 Benchmark\*\*\*: Section 17 (Advanced Audit Policy Configuration)

# [REQ-END-148] Audit Policy: Policy Change Auditing for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

---

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer

Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Policy Change`  
→ `Success and Failure` \* Subcategory: `Authentication Policy Change` → `Success` \* Subcategory: `Authorization Policy Change` → `Success`  
\* Subcategory: `MPSSVC Rule-Level Policy Change` → `Success and Failure` \* Subcategory: `Other Policy Change Events` → `Failure`

---

## Rationale Auditing policy changes tracks attempts to modify authorization policies, auditing configuration changes, or firewall rule alterations to hide adversarial tracks.

---

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Set local Security Event Log size to a minimum of 512MB to prevent premature rollover of security auditing data.

---

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Policy Change` → `Success and Failure` \* Subcategory: `Authentication Policy Change` → `Success` \* Subcategory: `Authorization Policy Change` → `Success` \* Subcategory: `MPSSVC Rule-Level Policy Change` → `Success and Failure` \* Subcategory: `Other Policy Change Events` → `Failure`

---

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditPolicychange.ps1](#)]

(../implementation\_scripts/Configure-EndAuditPolicychange.ps1)

```
Configure-EndAuditPolicychange.ps1
Write-Host "Applying Audit Policy category: policy-change..." -ForegroundColor Cyan

Set Audit Subcategory: Policy Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Policy Change`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Policy Change to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Policy Change. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Authentication Policy Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Authentication Policy Change`" /success:enable /failure:disable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Authentication Policy Change to Success" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Authentication Policy Change. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Authorization Policy Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Authorization Policy Change`" /success:enable /failure:disable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Authorization Policy Change to Success" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Authorization Policy Change. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: MPSSVC Rule-Level Policy Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"MPSSVC Rule-Level Policy Change`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory MPSSVC Rule-Level Policy Change to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory MPSSVC Rule-Level Policy Change. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Other Policy Change Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`"Other Policy Change Events`" /success:disable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Other Policy Change Events to Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Other Policy Change Events. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-EndAuditPolicychangeStatus.ps1](#)



```

Get-EndAuditPolicychangeStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Policy Change
$RawOutput = auditpol.exe /get /subcategory:"Policy Change" /r
if ($RawOutput -notmatch ",Policy Change,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Authentication Policy Change
$RawOutput = auditpol.exe /get /subcategory:"Authentication Policy Change" /r
if ($RawOutput -notmatch ",Authentication Policy Change,.*,Success") {
 $script:Vulnerable = $true
}

Audit Subcategory: Authorization Policy Change
$RawOutput = auditpol.exe /get /subcategory:"Authorization Policy Change" /r
if ($RawOutput -notmatch ",Authorization Policy Change,.*,Success") {
 $script:Vulnerable = $true
}

Audit Subcategory: MPSSVC Rule-Level Policy Change
$RawOutput = auditpol.exe /get /subcategory:"MPSSVC Rule-Level Policy Change" /r
if ($RawOutput -notmatch ",MPSSVC Rule-Level Policy Change,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Other Policy Change Events
$RawOutput = auditpol.exe /get /subcategory:"Other Policy Change Events" /r
if ($RawOutput -notmatch ",Other Policy Change Events,.*,Failure") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}

```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Client auditing baselines \* \*\*CIS Windows 10/11 Benchmark\*\*\*: Section 17 (Advanced Audit Policy Configuration)

# [REQ-END-149] Audit Policy: Privilege Use Auditing for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Sensitive Privilege Use` → `Success and Failure`

## Rationale Auditing sensitive privilege use logs attempts by processes or users to exercise rights like ActAsPartOfTypeOperatingSystem or LoadDrivers, identifying potential privilege escalations.

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Set local Security Event Log size to a minimum of 512MB to prevent premature rollover of security auditing data.

## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Sensitive Privilege Use` → `Success and Failure`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditPrivilegeuse.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditPrivilegeuse.ps1](#))

```
Configure-EndAuditPrivilegeuse.ps1
Write-Host "Applying Audit Policy category: privilege-use..." -ForegroundColor Cyan

Set Audit Subcategory: Sensitive Privilege Use
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Sensitive Privilege Use`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Sensitive Privilege Use to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Sensitive Privilege Use. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-EndAuditPrivilegeuseStatus.ps1](#)

```
Get-EndAuditPrivilegeuseStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Sensitive Privilege Use
$RawOutput = auditpol.exe /get /subcategory:"Sensitive Privilege Use" /r
if ($RawOutput -notmatch ",Sensitive Privilege Use,.*, (Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}
```

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*: Client auditing baselines \* \*\*CIS Windows 10/11 Benchmark\*\*: Section 17 (Advanced Audit Policy Configuration)

# [REQ-END-150] Audit Policy: System Events Auditing for Endpoints

## Target Scope \* \*\*Applicable Systems\*\*: Tier 2 Client Workstations \* \*\*Operating Systems\*\*: Windows 10/11 Enterprise/Professional

## Implementation Details \* \*\*Priority\*\*: High \* \*\*GPO Path / Registry Location\*\*: \* \*\*GPO Paths\*\*: `Computer Configuration\Policies\Windows Settings\Security Settings\Advanced Audit Policy Configuration\Audit Policies` \* Subcategory: `Other System Events` → `Success and Failure` \* Subcategory: `Security State Change` → `Success and Failure` \* Subcategory: `Security System Extension` -> `Success and Failure` \* Subcategory: `System Integrity` → `Success and Failure`

## Rationale Auditing system security extensions, integrity violations, and driver arrivals monitors boot health and tampering of host security services.

## Legacy Impact & Compatibility \* \*\*Event Log Volume\*\*: Set local Security Event Log size to a minimum of 512MB to prevent premature rollover of security auditing data.

### ## Implementation Steps

### Option A: Group Policy Object (GPO) Configuration (Preferred) 1. Configure Advanced Audit Policy subcategory or registry override preferences matching: \* Subcategory: `Other System Events` → `Success and Failure` \* Subcategory: `Security State Change` → `Success and Failure` \* Subcategory: `Security System Extension` → `Success and Failure` \* Subcategory: `System Integrity` → `Success and Failure`

### Option B: PowerShell & Registry Configuration (Remediation / Non-GPO) [Download Script: [Configure-EndAuditSystemevents.ps1](#)] ([../implementation\\_scripts/Configure-EndAuditSystemevents.ps1](#))

```
Configure-EndAuditSystemevents.ps1
Write-Host "Applying Audit Policy category: system-events..." -ForegroundColor Cyan

Set Audit Subcategory: Other System Events
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Other System Events`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Other System Events to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Other System Events. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Security State Change
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Security State Change`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Security State Change to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Security State Change. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: Security System Extension
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`Security System Extension`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory Security System Extension to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory Security System Extension. Exit Code: $($Process.ExitCode)"
}

Set Audit Subcategory: System Integrity
$Process = Start-Process auditpol -ArgumentList "/set /subcategory:`System Integrity`" /success:enable /failure:enable" -Wait -NoNewWindow -PassThru
if ($Process.ExitCode -eq 0) {
 Write-Host " Enforced subcategory System Integrity to Success and Failure" -ForegroundColor Green
} else {
 Write-Error " Failed to configure subcategory System Integrity. Exit Code: $($Process.ExitCode)"
}
```

To audit the hardening status: [Download Script: Get-EndAuditSystemeventsStatus.ps1](#)

```

Get-EndAuditSystemeventsStatus.ps1
$script:Vulnerable = $false

Audit Subcategory: Other System Events
$RawOutput = auditpol.exe /get /subcategory:"Other System Events" /r
if ($RawOutput -notmatch ",Other System Events,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Security State Change
$RawOutput = auditpol.exe /get /subcategory:"Security State Change" /r
if ($RawOutput -notmatch ",Security State Change,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: Security System Extension
$RawOutput = auditpol.exe /get /subcategory:"Security System Extension" /r
if ($RawOutput -notmatch ",Security System Extension,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

Audit Subcategory: System Integrity
$RawOutput = auditpol.exe /get /subcategory:"System Integrity" /r
if ($RawOutput -notmatch ",System Integrity,.*(Success and Failure|Success & Failure)") {
 $script:Vulnerable = $true
}

if ($script:Vulnerable) {
 Write-Output "Non-Compliant"
 exit 1
} else {
 Write-Output "Compliant"
 exit 0
}

```

---

## Sources & Compliance References \* \*\*ANSSI Active Directory Hardening Guide\*\*\*: Client auditing baselines \* \*\*CIS Windows 10/11 Benchmark\*\*\*: Section 17 (Advanced Audit Policy Configuration)

## # Implementation Plan and Prioritized Roadmap

This document outlines the prioritized implementation plan for Domain Controllers, Endpoints, and Privileged Access Workstations (PAWs). Hardening a production Active Directory environment requires balancing security posture improvements against operational disruption and engineering effort.

To achieve this, the security controls in this guidebook are organized into five sequential phases based on their real-world impact on preventing compromise, ease of implementation, and potential compatibility impact.

---

### ## Target Scope

- **Domain Controllers:** High-security Windows Server instances hosting directory services (Tier 0).
  - **Endpoints:** Standard member client workstations (Tier 2).
  - **Privileged Access Workstations (PAWs):** High-security, isolated administration workstations (Tier 0).
  - **Administrative Architecture:** Directory layout, administrative tiering boundaries, and trusts (Tier 0 to Tier 2).
  - **Operations & Maintenance:** Backup, disaster recovery, patch shipping, and continuous monitoring procedures.
- 

### ## Phase 0: Architectural Foundation & Administrative Tiering

Architectural choices must be made and established **before** implementing individual host-level hardening measures. Without a secure administrative tiering structure, hardening client hosts or Domain Controllers is easily bypassed. This phase establishes administrative boundaries and prepares the GPO structure.

#### Security Posture Impact \* Establishes the three-tier administrative model (Tier 0, Tier 1, Tier 2) to prevent credential exposure from high-privilege domains to lower-security zones. \* Denies administrative logon rights across boundaries to block lateral movement and credential harvesting. \* Secures AD trust relationships to prevent domain containment breaches.

#### Architectural Requirements \* \*\*[REQ-ARCH-001 - Implement Active Directory Administrative Tiering Model](#01-architecture-implement-administrative-tiering-model-md)\*\*: Enforces GPO logon restrictions to isolate administrative tiers. \* \*\*[REQ-ARCH-002 - Restrict Administrative Management Protocols](#01-architecture-restrict-mgmt-protocols-md)\*\*: Secures management access paths. \* \*\*[REQ-ARCH-003 - Audit Privileged Groups](#01-architecture-audit-privileged-groups-md)\*\*: Enforces monitoring and strict controls on Tier 0 group memberships. \* \*\*[REQ-ARCH-004 - Keep Domain and Forest Functional Levels Up-To-Date](#01-architecture-keep-functional-levels-up-to-date-md)\*\*: Ensures modern AD security features are active. \* \*\*[REQ-ARCH-005 - Default Domain and Domain Controllers Policies Management](#01-architecture-default-policies-recommendations-md)\*\*: Standardizes GPO hierarchy and separates default policy links. \* \*\*[REQ-ARCH-006 - Harden Active Directory Domain Trusts](#01-architecture-harden-domain-trusts-md)\*\*: Configures secure trust filters and disables SID history routing where appropriate. \* \*\*[REQ-ARCH-007 - Harden Microsoft Exchange Active Directory Permissions](#01-architecture-harden-exchange-permissions-md)\*\*: Removes WriteDacl and WriteOwner permissions for Exchange groups on the domain root.

#### Identities & Services Requirements \* \*\*[REQ-ID-010 - Restrict Schema Administrators Group Membership](#03-identities-services-restrict-schema-admins-md)\*\*: Restricts membership of the Schema Administrators group to minimize administrative privilege footprint. \* \*\*[REQ-ID-011 - Enforce Accidental Deletion Protection on Organizational Units](#03-identities-services-prevent-accidental-deletion-ous-md)\*\*: Enables accidental deletion protection on all active Organizational Units to prevent directory data loss.

#### Network & Firewall Requirements \* \*\*[REQ-NET-001 - Configure Active Directory Port Matrix](#04-network-firewall-configure-ad-port-matrix-md)\*\*: Standardizes the AD port matrix firewall rules for domain controllers and member servers.

---

### ## Phase 1: Critical Risk Reduction & Operational Baselines

This phase targets the elimination of immediately exploitable vulnerability classes, including coercive authentication, protocol relaying, name resolution poisoning, default password reuse, and PKI template vulnerabilities. Crucially, it also implements the core backup, auditing, and LAPS baselines to establish recovery and visibility before deeper hardening parameters are deployed.

#### Security Posture Impact \* Enforces backup and restore procedures to protect against fatal misconfigurations or ransomware. \* Blocks coercion techniques (such as PetitPotam or DFSCoerce) that allow attackers to instantly compromise Domain Controllers from standard domain accounts. \* Neutralizes LLMNR/mDNS spoofing (such as Responder attacks) that capture hashes on local network segments. \* Standardizes naming schemas and establishes foundational AD CS and local administrator password management (LAPS). \* Implements essential security auditing for PowerShell and process creation to ensure visibility.

#### Operations & Maintenance Requirements \* \*\*[REQ-OPS-001 - Enforce KRBTGT Password Rotation](#06-operations-maintenance-enforce-krbtgt-password-rotation-md)\*\*: Implements standard 2-step rotation of the domain key ticket account. \* \*\*[REQ-OPS-002 - Enable and Configure the Active Directory Recycle Bin](#06-operations-maintenance-enable-recycle-bin-md)\*\*: Enforces forest-wide Recycle Bin for rapid recovery of deleted objects. \* \*\*[REQ-OPS-003 - Establish and Maintain Group Policy ADMX Central Store](#06-operations-maintenance-maintain-gpo-templates-md)\*\*: Prevents version drift across consoles. \* \*\*[REQ-OPS-007 - Mandate Naming Conventions for GPOs, OUs, and User Accounts](#06-operations-maintenance-mandate-naming-conventions-md)\*\*: Enforces GPO/OU prefix metadata supporting GPO auditing. \* \*\*[REQ-OPS-008 - Configure Daily System State Backups](#06-operations-maintenance-configure-system-state-backups-md)\*\*: Implements daily AD System State backup, offline/immutable backup isolation, and quarterly recovery drills. \* \*\*[REQ-OPS-013 - Clean Up Staged Install From Media (IFM) Data](#06-operations-maintenance-cleanup-staged-ifm-files-md)\*\*: Deletes temporary ntfs.dit datasets immediately after Domain Controller promotions.

#### Domain Controller Requirements \* \*\*[REQ-DC-001 - Disable SMBv1](#02-domain-controllers-disable-smbv1-md)\*\*: Disables legacy, vulnerable file sharing protocol drivers. \* \*\*[REQ-DC-002 - Disable Multicast Name Resolution](#02-domain-controllers-disable-multicast-

name-resolution-md)\*\*: Prevents LLMNR/mDNS spoofing on DCs. \* \*\*[REQ-DC-003 - Disable NTLMv1](#02-domain-controllers-disable-ntlmv1-md)\*\*: Mitigates weak cryptographic authentication. \* \*\*[REQ-DC-008 - Disable Print Spooler Service](#02-domain-controllers-disable-print-spooler-md)\*\*: Disables spooler to block PrintNightmare and coercion. \* \*\*[REQ-DC-015 - Migrate SYSVOL Replication to DFSR](#02-domain-controllers-migrate-sysvol-replication-dfsr-md)\*\*: Retires legacy FRS replication. \* \*\*[REQ-DC-016 - Harden adminSDHolder Permissions](#02-domain-controllers-harden-adminsdholder-permissions-md)\*\*: Blocks permission changes to high-privilege templates. \* \*\*[REQ-DC-024 - Configure dSHeuristics Attribute](#02-domain-controllers-configure-dsheuristics-md)\*\*: Restricts anonymous directory access. \* \*\*[REQ-DC-030 - Secure Directory Services Restore Mode (DSRM) and Recovery Parameters](#02-domain-controllers-harden-dsrm-recovery-mode-md)\*\*: Configures DsrAdminLogonBehavior to restrict network logons. \* \*\*[REQ-DC-031 - Configure NTP Time Synchronization on the PDC Emulator](#02-domain-controllers-configure-pdc-time-sync-md)\*\*: Configures w32time parameters and external time sync on the PDC Emulator.

### Identities & Services Requirements \* \*\*[REQ-ID-002 - Enable Local Administrator Password Solution (LAPS)](#03-identities-services-enable-laps-md)\*\*: Implements Windows LAPS or Classic LAPS to rotate local administrator passwords periodically. \* \*\*[REQ-ID-006 - Rename and Disable Default Administrator and Guest Accounts](#03-identities-services-harden-default-accounts-md)\*\*: Disables the default Guest account and renames the default Administrator account. \* \*\*[REQ-ID-015 - Harden Active Directory Certificate Services (ADCS) and PKI](#03-identities-services-harden-adcs-pki-md)\*\*: Hardens Active Directory Certificate Services (ADCS) and PKI templates against privilege escalation. \* \*\*[REQ-ID-020 - Clean Up Legacy Group Policy Preferences and SYSVOL Passwords](#03-identities-services-cleanup-gpp-sysvol-passwords-md)\*\*: Cleans up GPP credentials and insecure scripts from SYSVOL.

### Logging & Monitoring Requirements \* \*\*[REQ-LOG-001 - Configure Advanced Security Audit Policies](#05-logging-monitoring-configure-advanced-audit-policies-md)\*\*: Configures advanced security audit policies to enable comprehensive event logging. \* \*\*[REQ-LOG-002 - Configure PowerShell and Command-Line Auditing](#05-logging-monitoring-configure-powershell-and-command-line-auditing-md)\*\*: Enforces PowerShell script block and command-line process auditing.

### PAW Requirements \* \*\*[REQ-PAW-003 - Restrict Local Administrators Group for PAWs](#07-paws-restrict-local-administrators-md)\*\*: Removes standard users from local administrators. \* \*\*[REQ-PAW-015 - Configure Secure Printing and Print Spooler Policies for PAWs](#07-paws-configure-printing-and-spooler-md)\*\*: Disables print spooler on administrative hosts. \* \*\*[REQ-PAW-019 - Harden Network Parameters and Disable Legacy Name Resolution](#07-paws-harden-network-and-name-resolution-md)\*\*: Disables LLMNR/mDNS and NetBIOS on PAWs. \* \*\*[REQ-PAW-021 - Disable AutoPlay and AutoRun for PAWs](#07-paws-disable-autoplay-autorun-md)\*\*: Stops auto-execution from removable media.

### Endpoint Requirements \* \*\*[REQ-END-001 - Harden Network Parameters and Disable Legacy Name Resolution](#08-endpoints-harden-network-and-name-resolution-md)\*\*: Disables local name resolution protocol spoofing. \* \*\*[REQ-END-003 - Disable AutoPlay and AutoRun](#08-endpoints-disable-autoplay-autorun-md)\*\*: Disables automated optical or flash drive execution. \* \*\*[REQ-END-006 - Restrict Local Administrators Group](#08-endpoints-restrict-local-admins-md)\*\*: Enforces administrative segregation and limits local admin rights. \* \*\*[REQ-END-025 - Configure Secure Printing and Print Spooler Policies](#08-endpoints-configure-printing-and-spooler-md)\*\*: Blocks incoming print spooler calls on client hosts.

## ## Phase 2: Credential & Session Isolation (High Impact)

This phase focuses on isolating credentials inside memory and network packets to block credential dumping tools (like Mimikatz) and session hijacking. It also integrates isolated update deployment mechanisms, restricts Kerberos delegations, secures network authentication protocols, and implements robust service account policies.

### Security Posture Impact \* Enforces network cryptographic integrity via SMB signing and LDAP channel binding, stopping man-in-the-middle relays. \* Protects the Local Security Authority Subsystem Service (LSASS) process from debugging and memory reading. \* Redirects default directory containers to ensure new objects receive security policies automatically. \* Restricts Kerberos delegation and deploys fine-grained password and service account policies.

### Operations & Maintenance Requirements \* \*\*[REQ-OPS-005 - Configure Dedicated WSUS for Tier 0](#06-operations-maintenance-configure-dedicated-tier0-wsus-md)\*\*: Secures dedicated patch servers to prevent cross-tier update spoofing. \* \*\*[REQ-OPS-006 - Redirect Default Users and Computers Containers](#06-operations-maintenance-redirect-default-containers-md)\*\*: Prevents newly joined machines from staying in unmanaged default OUs. \* \*\*[REQ-OPS-009 - Implement Offline Patch Management via WSUS](#06-operations-maintenance-implement-offline-patch-management-md)\*\*: Implements offline WSUS metadata imports/exports (sneakernet transport). \* \*\*[REQ-OPS-012 - Implement Automated Inactive Computer and User Account Cleanup](#06-operations-maintenance-decommission-inactive-accounts-md)\*\*: Disables and moves inactive user (180 days) and computer (90 days) accounts to a stale OU.

### Domain Controller Requirements \* \*\*[REQ-DC-004 - Enforce LDAP Server Signing](#02-domain-controllers-enforce-ldap-signing-md)\*\*: Restricts cleartext un-signed LDAP operations. \* \*\*[REQ-DC-005 - Enforce LDAP Channel Binding](#02-domain-controllers-enforce-ldap-channel-binding-md)\*\*: Enforces channel binding for LDAP over SSL. \* \*\*[REQ-DC-006 - Enable LSA Protection](#02-domain-controllers-enable-lsa-protection-md)\*\*: Enables Protected Process Light (PPL) for LSASS. \* \*\*[REQ-DC-007 - Disable Credential Guard](#02-domain-controllers-disable-credential-guard-md)\*\*: Disables Credential Guard on DCs while maintaining VBS configurations to maintain compatibility. \* \*\*[REQ-DC-009 - Enforce SMB Message Signing](#02-domain-controllers-enforce-smb-signing-md)\*\*: Mandates SMB signing to block NTLM relaying. \* \*\*[REQ-DC-011 - Restrict Remote SAM API Access](#02-domain-controllers-restrict-ntds-sam-api-md)\*\*: Restricts remote account listing. \* \*\*[REQ-DC-019 - Enforce RDP Restricted Admin Mode](#02-domain-controllers-enforce-rdp-restricted-admin-md)\*\*: Stops administrative credential caching during RDP. \* \*\*[REQ-DC-025 - Configure Security Options for Domain Controllers](#02-domain-controllers-configure-security-options-md)\*\*: Locks anonymous pipe access and credential parameters. \* \*\*[REQ-DC-032 - Enable UEFI Secure Boot](#02-domain-controllers-enable-secure-boot-md)\*\*: Requirement to enforce hardware-rooted platform integrity checks, verifying that UEFI Secure Boot is active on Domain Controllers.

### Identities & Services Requirements \* \*\*[REQ-ID-001 - Enforce Fine-Grained Password Policies](#03-identities-services-enforce-fgpp-



md)\*\*: Enforces password complexity, length, and lockout policies using Fine-Grained Password Policies. \* \*\*[REQ-ID-003 - Implement Group Managed Service Accounts (gMSA)](#03-identities-services-harden-service-accounts-md)\*\*: Deploys group Managed Service Accounts (gMSAs) to automate password management for services. \* \*\*[REQ-ID-004 - Restrict Kerberos Delegation](#03-identities-services-restrict-kerberos-delegation-md)\*\*: Restricts Kerberos delegation configurations to prevent delegation and relay-to-delegation attacks. \* \*\*[REQ-ID-005 - Configure and Populate Protected Users Group](#03-identities-services-configure-protected-users-group-md)\*\*: Configures the Protected Users security group to restrict credential caching and weak delegation. \* \*\*[REQ-ID-007 - Restrict Interactive Logons for Service Accounts](#03-identities-services-restrict-service-account-logons-md)\*\*: Blocks interactive and remote logons for service accounts using GPO user rights assignments. \* \*\*[REQ-ID-009 - Enforce Kerberos Pre-Authentication](#03-identities-services-enforce-kerberos-preauthentication-md)\*\*: Configures accounts to mandate Kerberos pre-authentication, mitigating offline AS-REP roasting. \* \*\*[REQ-ID-013 - Clean Up adminCount Attribute Orphans](#03-identities-services-cleanup-admincount-orphans-md)\*\*: Cleans up the adminCount attribute on non-privileged accounts to prevent administrative ACL persistence. \* \*\*[REQ-ID-014 - Renew KDS Root Keys and gMSA Secrets](#03-identities-services-renew-kds-keys-gmsa-secrets-md)\*\*: Automatically renews KDS root keys and gMSA secrets to maintain cryptographic validity. \* \*\*[REQ-ID-018 - Restrict Pre-Windows 2000 Compatible Access Group](#03-identities-services-restrict-pre-windows-2000-compatible-access-group-md)\*\*: Restricts the Pre-Windows 2000 Compatible Access group to block anonymous directory listings.

#### Network & Firewall Requirements \* \*\*[REQ-NET-007 - Enforce SMBv3 Security and Digitally Sign/Encrypt Communications](#04-network-firewall-enforce-smbv3-security-md)\*\*: Enforces SMBv3 security parameters, including message signing and encryption. \* \*\*[REQ-NET-009 - Configure Hardened UNC Paths and LDAP Client Signing](#04-network-firewall-configure-hardened-unc-paths-md)\*\*: Hardens UNC paths (UNC Hardening) and mandates LDAP client signing parameters.

#### Logging & Monitoring Requirements \* \*\*[REQ-LOG-003 - Deploy and Harden Microsoft Sysmon](#05-logging-monitoring-deploy-and-harden-sysmon-md)\*\*: Deploys and hardens Microsoft Sysmon to detect advanced host-based anomalies. \* \*\*[REQ-LOG-004 - Configure Secure SIEM Log Shipping](#05-logging-monitoring-configure-siem-log-shipping-md)\*\*: Enforces secure, encrypted SIEM log shipping for central log correlation. \* \*\*[REQ-LOG-005 - Configure Kerberoasting Honeypots and SIEM Detection Rules](#05-logging-monitoring-implement-kerberoasting-honeypot-md)\*\*: Configures decoy accounts and SIEM alerts to capture Kerberoasting attacks. \* \*\*[REQ-LOG-006 - Configure SYSVOL Decoy XML Honeypot](#05-logging-monitoring-implement-sysvol-honeypot-md)\*\*: Deploys a decoy GPO XML file with Deny access rules to detect active credential harvesting scans.

#### PAW Requirements \* \*\*[REQ-PAW-002 - Enable LSA Protection for PAWs](#07-paws-enable-lsa-protection-md)\*\*: Blocks LSASS memory reading on PAWs. \* \*\*[REQ-PAW-030 - Enable UEFI Secure Boot for PAWs](#07-paws-enable-secure-boot-md)\*\*: Mandates hardware-rooted platform integrity checks, verifying that UEFI Secure Boot is active on the operating system for PAWs. \* \*\*[REQ-PAW-010 - Enable VBS and Credential Guard for PAWs](#07-paws-enable-vbs-credential-guard-md)\*\*: Uses hypervisor isolation to protect administrative tokens. \* \*\*[REQ-PAW-020 - Configure User Account Control Policies for PAWs](#07-paws-configure-uac-policies-md)\*\*: Configures UAC secure prompt restrictions. \* \*\*[REQ-PAW-031 - Enforce Smart Card Logon for PAWs](#07-paws-enforce-smartcard-logon-paws-md)\*\*: Requires hardware-backed administrative logon.

#### Endpoint Requirements \* \*\*[REQ-END-002 - Configure User Account Control Policies](#08-endpoints-configure-uac-policies-md)\*\*: Restricts local administrator prompt behavior. \* \*\*[REQ-END-009 - Enable UEFI Secure Boot](#08-endpoints-enable-secure-boot-md)\*\*: Mandates hardware-rooted platform integrity checks, verifying that UEFI Secure Boot is active on the operating system. \* \*\*[REQ-END-010 - Enable VBS and Credential Guard](#08-endpoints-enable-vbs-credential-guard-md)\*\*: Isolates LSASS secrets on Tier 2 endpoints. \* \*\*[REQ-END-018 - Configure Account and Password Policies](#08-endpoints-configure-account-policies-md)\*\*: Sets local lockout and complexity guidelines. \* \*\*[REQ-END-023 - Enable LSA Protection with UEFI Lock](#08-endpoints-enable-lsa-protection-md)\*\*: Locks LSASS protection with UEFI firmware configuration.

## ## Phase 3: Tiering & Hardware-Rooted Protections (Strategic)

This phase establishes the physical boundaries, hardware-based trust mechanisms, and deep tiering segmentations that form the foundations of Tier 0 administrative workstations (PAWs) and secure network isolation.

#### Security Posture Impact \* Assures that administrative systems cannot be modified by offline attacks (via BitLocker and firmware locks). \* Ensures the boot sequence is verified from a hardware trust anchor (TPM 2.0). \* Cryptographically isolates tiers on the network using IPsec domain isolation. \* Enforces smart card requirements for administrative accounts.

#### Domain Controller Requirements \* \*\*[REQ-DC-010 - Restrict Kerberos Encryption Types](#02-domain-controllers-restrict-kerberos-encryption-md)\*\*: Enforces AES-only encryption for Kerberos. \* \*\*[REQ-DC-017 - Harden Microsoft DNS AD Container Permissions](#02-domain-controllers-harden-dns-container-permissions-md)\*\*: Prevents server-level DNS hijack DLLs. \* \*\*[REQ-DC-018 - Harden Virtualization Hosts for Domain Controllers](#02-domain-controllers-harden-dc-virtualization-hosts-md)\*\*: Places virtualized domain controllers inside a secure Tier 0 host boundary. \* \*\*[REQ-DC-023 - Configure User Rights Assignments for Domain Controllers](#02-domain-controllers-configure-user-rights-assignments-md)\*\*: Limits operator rights on DCs. \* \*\*[REQ-DC-033 - Configure Secure Boot Revocations and Bootloader Updates](#02-domain-controllers-configure-secure-boot-revocations-md)\*\*: Requirement to configure and enforce BlackLotus revocation updates and bootloader integrity verification policy variables in system firmware.

#### Identities & Services Requirements \* \*\*[REQ-ID-008 - Enforce User and Service Account Kerberos Encryption (AES-Only)](#03-identities-services-enforce-user-aes-encryption-md)\*\*: Restricts user and service account Kerberos encryption to AES-only, disabling weak DES/RC4. \* \*\*[REQ-ID-012 - Configure Active Directory Authentication Silos and Policies](#03-identities-services-configure-authentication-silos-md)\*\*: Establishes Authentication Silos and Policies to isolate Tier 0 administrative account sessions. \* \*\*[REQ-ID-016 - Configure Logon Screen and Credentials Delegation](#03-identities-services-configure-credential-delegation-md)\*\*: Disables credential delegation and configures logon screen security parameters. \* \*\*[REQ-ID-019 - Enforce Smart Card Authentication for Privileged Users](#03-identities-services-enforce-smartcard-privileged-users-md)\*\*: Mandates smart card logon requirements for administrative accounts to enforce hardware-based MFA.

#### Network & Firewall Requirements \* \*\*[REQ-NET-003 - Configure Workstation and Server Isolation](#04-network-firewall-configure-

workstation-isolation-md)\*\*: Segregates workstations and member servers to prevent peer-to-peer lateral movement. \* \*\*[REQ-NET-004 - Configure IPsec Domain Isolation](#04-network-firewall-configure-ipsec-domain-isolation-md)\*\*: Enforces IPsec-based domain isolation to cryptographically segment administrative tiers. \* \*\*[REQ-NET-005 - Harden IPsec Cryptographic Configurations](#04-network-firewall-harden-ipsec-cryptography-md)\*\*: Hardens IPsec cryptographic algorithms and parameters to ensure network integrity.

### PAW Requirements \* \*\*[REQ-PAW-004 - Enforce BitLocker with TPM and Startup PIN for PAWs](#07-paws-enable-bitlocker-md)\*\*: Demands BitLocker with a pre-boot startup PIN. \* \*\*[REQ-PAW-005 - UEFI Firmware Security Hardening](#07-paws-configure-uefi-security-md)\*\*: Implements strong UEFI passwords and locks the boot order. \* \*\*[REQ-PAW-006 - Enable Hardware Virtualization and DMA Protection](#07-paws-enable-hardware-virtualization-and-dma-protection-md)\*\*: Configures IOMMU and virtualization flags. \* \*\*[REQ-PAW-009 - Configure User Rights Assignments for PAWs](#07-paws-configure-user-rights-assignments-md)\*\*: Restricts debugging, impersonation, and interactive logins on PAWs. \* \*\*[REQ-PAW-011 - Harden DMA and Physical Security for PAWs](#07-paws-harden-dma-and-physical-security-md)\*\*: Blocks sleep states and limits external bus operations. \* \*\*[REQ-PAW-022 - Disable Incoming Remote Desktop Access for PAWs](#07-paws-restrict-rdp-access-md)\*\*: Blocks remote lateral logins to administrative devices. \* \*\*[REQ-PAW-035 - Configure Secure Boot Revocations and Bootloader Updates for PAWs](#07-paws-configure-secure-boot-revocations-md)\*\*: Configures and enforces BlackLotus revocation updates and bootloader integrity verification policy variables in system firmware for PAWs.

### Endpoint Requirements \* \*\*[REQ-END-005 - Restrict Remote Desktop Access](#08-endpoints-restrict-rdp-access-md)\*\*: Prevents incoming RDP connections. \* \*\*[REQ-END-012 - Enable BitLocker and Network Unlock](#08-endpoints-enable-bitlocker-md)\*\*: Protects client data storage using BitLocker. \* \*\*[REQ-END-013 - UEFI Firmware Security Hardening](#08-endpoints-configure-uefi-security-md)\*\*: Secures UEFI parameters on client workstations. \* \*\*[REQ-END-014 - Enable Hardware Virtualization and DMA Protection](#08-endpoints-enable-hardware-virtualization-and-dma-protection-md)\*\*: Enables hardware VBS requisites. \* \*\*[REQ-END-016 - Configure User Rights Assignments](#08-endpoints-configure-user-rights-assignments-md)\*\*: Disables token impersonation and program debugging for standard users. \* \*\*[REQ-END-017 - Harden DMA and Physical Security](#08-endpoints-harden-dma-and-physical-security-md)\*\*: Disables client standby states and limits DMA peripherals. \* \*\*[REQ-END-035 - Configure Secure Boot Revocations and Bootloader Updates](#08-endpoints-configure-secure-boot-revocations-md)\*\*: Configures and enforces BlackLotus revocation updates and bootloader integrity verification policy variables in system firmware.

#### ## Phase 4: Advanced Restrictions & Fine-Tuning (Continuous Hardening)

This phase introduces strict operational controls, software restrictions (AppLocker/WDAC), logging updates, and services minimization. Additionally, continuous offline assessment loops are established to audit security controls on an ongoing basis.

### Security Posture Impact \* Prevents the execution of unauthorized binaries, scripts, or malicious installers via application blocklists/allowlists. \* Restricts system executables ('svchost.exe') from loading arbitrary non-Microsoft binaries. \* Audits Active Directory configurations monthly via offline scanners. \* Tightens firewall configurations and disables unnecessary system services.

### Operations & Maintenance Requirements \* \*\*[REQ-OPS-004 - Implement Third-Party and Custom GPO Templates for COTS Hardening](#06-operations-maintenance-use-third-party-templates-md)\*\*: Standardizes security settings for custom application baselines. \* \*\*[REQ-OPS-010 - Establish Continuous Security Assessments](#06-operations-maintenance-establish-continuous-security-assessments-md)\*\*: Integrates monthly PingCastle scans, quarterly BloodHound analyses, and semi-annual offline database checks. \* \*\*[REQ-OPS-011 - Enable Detailed BSOD Stop Parameters for Crash Control](#06-operations-maintenance-enable-detailed-bsod-parameters-md)\*\*: Enables detailed crash diagnostics for air-gapped system recovery.

### Domain Controller Requirements \* \*\*[REQ-DC-012 - Disable Unnecessary Services on Domain Controllers](#02-domain-controllers-disable-unnecessary-services-md)\*\*: Stops non-essential system functions. \* \*\*[REQ-DC-013 - Enable Kerberos Armoring](#02-domain-controllers-enable-kerberos-armoring-md)\*\*: Protects pre-authentication traffic using FAST. \* \*\*[REQ-DC-014 - Restrict NTLM](#02-domain-controllers-restrict-ntlm-md)\*\*: Restricts NTLM protocol fallback domain-wide. \* \*\*[REQ-DC-020 - Windows Defender Antivirus Domain Controller Baseline and Exploit Guard](#02-domain-controllers-defender-antivirus-md)\*\*: Applies real-time scanning and server exclusions. \* \*\*[REQ-DC-021 - Configure AppLocker Policies on Domain Controllers](#02-domain-controllers-configure-applocker-policies-md)\*\*: Controls binary execution on DCs. \* \*\*[REQ-DC-022 - Enable WDAC Driver Blocklist](#02-domain-controllers-enable-wdac-driver-blocklist-md)\*\*: Blocks known vulnerable drivers. \* \*\*[REQ-DC-026 - Configure TCP/IP and Network Parameter Hardening for Domain Controllers](#02-domain-controllers-harden-network-parameters-md)\*\*: Optimizes network protocol parameters. \* \*\*[REQ-DC-027 - Configure Telemetry, Diagnostics and Privacy Options for Domain Controllers](#02-domain-controllers-configure-telemetry-privacy-md)\*\*: Limits diagnostics collection. \* \*\*[REQ-DC-028 - Configure Untrusted Font Blocking for Domain Controllers](#02-domain-controllers-configure-untrusted-font-blocking-md)\*\*: Protects kernel font parser. \* \*\*[REQ-DC-029 - Configure svchost.exe Mitigation Options](#02-domain-controllers-configure-svchost-mitigation-md)\*\*: Limits svchost sub-process execution. \* \*\*[REQ-DC-034 - Configure Windows Defender Application Control](#02-domain-controllers-configure-wdac-md)\*\*: Deploys code integrity rules.

### Identities & Services Requirements \* \*\*[REQ-ID-017 - Disable Machine Account Quota](#03-identities-services-disable-machine-account-quota-md)\*\*: Reduces the Machine Account Quota (ms-DS-MachineAccountQuota) to 0 to prevent unauthorized machine domain joins.

### Network & Firewall Requirements \* \*\*[REQ-NET-002 - Restrict RPC Dynamic Ports](#04-network-firewall-restrict-rpc-dynamic-ports-md)\*\*: Limits RPC dynamic ports to restrict the active network service attack surface. \* \*\*[REQ-NET-006 - Harden TLS Protocols, Cipher Suites, and Elliptic Curves](#04-network-firewall-harden-tls-configuration-md)\*\*: Disables weak TLS protocols and configures secure cipher suites for system-wide channels. \* \*\*[REQ-NET-008 - Configure Firewall Logging and Operational Settings](#04-network-firewall-configure-firewall-logging-md)\*\*: Configures Windows Defender Firewall logging parameters to maintain full network audit records. \* \*\*[REQ-NET-010 - Harden WinRM Service and Restrict Remote RPC Clients](#04-network-firewall-harden-winrm-service-md)\*\*: Restricts WinRM remote management and limits remote RPC client connections. \* \*\*[REQ-NET-011 - Configure WMI Static Port](#04-network-firewall-configure-wmi-static-port-md)\*\*: Restricts WMI service connections to a dedicated static network port. \* \*\*[REQ-NET-012 - Configure RPC Filters for Named



Pipes](#04-network-firewall-configure-rpc-named-pipe-filters-md)\*\*: Restricts remote RPC named pipe connections to standard system pipes. \* \*\*[REQ-NET-013 - Block Management Traffic Between Domain Controllers](#04-network-firewall-block-intra-dc-management-md)\*\*: Restricts network management traffic between Domain Controllers to prevent intra-tier attacks.

### PAW Requirements \* \*\*[REQ-PAW-001 - Configure AppLocker Policies for PAWs](#07-paws-configure-applocker-policies-md)\*\*: Allows administration utilities. \* \*\*[REQ-PAW-007 - Disable Windows Platform Binary Table (WPBT)](#07-paws-disable-wpbt-md)\*\*: Protects kernel boot from firmware binary injection. \* \*\*[REQ-PAW-008 - Windows Defender Antivirus PAW Baseline and Exploit Guard](#07-paws-defender-antivirus-md)\*\*: Enforces real-time protection and ASR blocks. \* \*\*[REQ-PAW-012 - Enable WDAC Driver Blocklist](#07-paws-enable-wdac-driver-blocklist-md)\*\*: Restricts bypass drivers on PAWs. \* \*\*[REQ-PAW-013 - Configure Account and Password Policies for PAWs](#07-paws-configure-account-policies-md)\*\*: Configures administrative account rules. \* \*\*[REQ-PAW-014 - Configure Early Launch Antimalware (ELAM) Policy for PAWs](#07-paws-configure-elam-md)\*\*: Validates boot driver signatures. \* \*\*[REQ-PAW-016 - Configure Untrusted Font Blocking for PAWs](#07-paws-configure-untrusted-font-blocking-md)\*\*: Font isolation on administration hosts. \* \*\*[REQ-PAW-017 - Configure svchost.exe Mitigation Options for PAWs](#07-paws-configure-svchost-mitigation-md)\*\*: Enforces Microsoft signature check on svchost. \* \*\*[REQ-PAW-018 - Enable Kernel-Mode Hardware-Enforced Stack Protection for PAWs](#07-paws-enable-kernel-shadow-stacks-md)\*\*: Enforces hardware-backed ROP mitigation. \* \*\*[REQ-PAW-023 - WSUS Client Configuration for PAWs](#07-paws-wsus-client-config-md)\*\*: Directs updates to local WSUS servers. \* \*\*[REQ-PAW-024 - Configure User Profile Restrictions](#07-paws-configure-user-profile-restrictions-md)\*\*: Locks administrative host profiles. \* \*\*[REQ-PAW-025 - Configure Exploit Protection Profile for PAWs](#07-paws-configure-exploit-protection-md)\*\*: System-wide DEP and ASLR configurations. \* \*\*[REQ-PAW-026 - Restrict Safe Mode Access to Administrators on PAWs](#07-paws-disable-safe-mode-for-standard-users-md)\*\*: Disables standard user access in Safe Mode. \* \*\*[REQ-PAW-027 - Configure Windows Defender Firewall and Block LOLBins for PAWs](#07-paws-configure-windows-firewall-md)\*\*: Firewalls and LOLBin traffic limits. \* \*\*[REQ-PAW-028 - Disable Unnecessary System Services for PAWs](#07-paws-disable-unnecessary-system-services-md)\*\*: Reduces active service footprints. \* \*\*[REQ-PAW-029 - Configure System Administrative Templates for PAWs](#07-paws-configure-system-administrative-templates-md)\*\*: Custom registry rules. \* \*\*[REQ-PAW-032 - Disable Unused Windows Features and PowerShell 2.0 Engine](#07-paws-disable-unused-features-md)\*\*: Disables legacy .NET 3.5, PowerShell 2.0, SMBv1, and unused platform features. \* \*\*[REQ-PAW-033 - Configure Microsoft Office Security and Block OLE Packages](#07-paws-configure-office-security-md)\*\*: Blocks VBA macros in external files and Outlook OLE packages. \* \*\*[REQ-PAW-034 - Disable Windows Script Host and Remap Scripting Extensions](#07-paws-disable-windows-script-host-md)\*\*: Disables WSH execution and maps extension defaults to Notepad. \* \*\*[REQ-PAW-036 - Configure Windows Defender Application Control](#07-paws-configure-wdac-md)\*\*: Deploys code integrity rules.

### Endpoint Requirements \* \*\*[REQ-END-004 - Block Removable Storage](#08-endpoints-block-removable-storage-md)\*\*: Prevents data exfiltration and USB storage execution. \* \*\*[REQ-END-007 - Windows Defender Antivirus Baseline and Exploit Guard](#08-endpoints-defender-antivirus-md)\*\*: Baseline real-time monitoring and ASR blocks. \* \*\*[REQ-END-008 - WSUS Client Configuration](#08-endpoints-wsus-client-config-md)\*\*: Updates from local offline servers only. \* \*\*[REQ-END-011 - Configure Windows Defender Application Control](#08-endpoints-configure-wdac-md)\*\*: Deploys code integrity rules. \* \*\*[REQ-END-015 - Disable Windows Platform Binary Table (WPBT)](#08-endpoints-disable-wpbt-md)\*\*: Mitigates firmware-based binary insertion. \* \*\*[REQ-END-019 - Configure User Profile Restrictions](#08-endpoints-configure-user-profile-restrictions-md)\*\*: Restricts HKCU features and notifications. \* \*\*[REQ-END-020 - Configure Exploit Protection Profile](#08-endpoints-configure-exploit-protection-md)\*\*: Memory protection profiles on client endpoints. \* \*\*[REQ-END-021 - Restrict Safe Mode Access to Administrators](#08-endpoints-disable-safe-mode-for-standard-users-md)\*\*: Denies standard user Safe Mode login. \* \*\*[REQ-END-022 - Configure Windows Defender Firewall and Block LOLBins](#08-endpoints-configure-windows-firewall-md)\*\*: Firewalls and outbound execution blocks. \* \*\*[REQ-END-024 - Disable Unnecessary System Services](#08-endpoints-disable-unnecessary-system-services-md)\*\*: Minimizes active service list on endpoints. \* \*\*[REQ-END-026 - Configure System Administrative Templates](#08-endpoints-configure-system-administrative-templates-md)\*\*: Client registry parameter sets. \* \*\*[REQ-END-027 - Configure AppLocker Policies](#08-endpoints-configure-applocker-policies-md)\*\*: Software execution restrictions on clients. \* \*\*[REQ-END-028 - Configure Early Launch Antimalware (ELAM) Policy](#08-endpoints-configure-elam-md)\*\*: Verifies driver startup list. \* \*\*[REQ-END-029 - Configure Untrusted Font Blocking](#08-endpoints-configure-untrusted-font-blocking-md)\*\*: Disables third-party font libraries. \* \*\*[REQ-END-030 - Configure svchost.exe Mitigation Options](#08-endpoints-configure-svchost-mitigation-md)\*\*: Restricts binary loading to Microsoft-signed code. \* \*\*[REQ-END-031 - Enable Kernel-Mode Hardware-Enforced Stack Protection](#08-endpoints-enable-kernel-shadow-stacks-md)\*\*: Mitigates Return-Oriented Programming (ROP) exploits. \* \*\*[REQ-END-032 - Disable Unused Windows Features and PowerShell 2.0 Engine](#08-endpoints-disable-unused-features-md)\*\*: Disables legacy .NET 3.5, PowerShell 2.0, SMBv1, and unused optional features. \* \*\*[REQ-END-033 - Configure Microsoft Office Security and Block OLE Packages](#08-endpoints-configure-office-security-md)\*\*: Blocks VBA macros in external files and Outlook OLE packages. \* \*\*[REQ-END-034 - Disable Windows Script Host and Remap Scripting Extensions](#08-endpoints-disable-windows-script-host-md)\*\*: Disables WSH execution and maps extension defaults to Notepad. \* \*\*[REQ-END-036 - Enable WDAC Driver Blocklist](#08-endpoints-enable-wdac-driver-blocklist-md)\*\*: Blocks known vulnerable drivers on Endpoints.

## ## Maintenance and Extension of the Implementation Plan

When a new technical hardening requirement is added to this guidebook, this implementation plan must be updated to maintain synchronization. Follow this process to integrate new requirements:

### 1. Security Impact & Disruption Assessment Evaluate the new control against the following criteria to determine its priority phase:

- **Phase 0 (Architectural Foundation & Tiering):**
  - Does the control establish global directories structures, trusts boundaries, or Tier restrictions?
- **Phase 1 (Immediate / Low Disruption or Critical Risk):**
  - Does the control mitigate an actively exploited vulnerability class (such as coercion or name spoofing)?
  - Can it be applied with near-zero likelihood of breaking legacy systems or applications?

- **Phase 2 (Credential & Session Isolation):**
  - Does the control isolate credential storage (LSASS) or protect authentication exchanges over the network?
- **Phase 3 (Tiering & Hardware-Rooted Protections):**
  - Does the control require hardware features (TPM, UEFI, BitLocker, IOMMU) or establish critical tiering boundaries?
- **Phase 4 (Advanced Restrictions & Fine-Tuning):**
  - Does the control involve strict application blocklisting/allowlisting (AppLocker/WDAC), network segmentation, blocking LOLBins, disabling services, or fine-tuning diagnostic parameters that require extensive verification?

### 2. Document Integration \* Open `roadmap/implementation-plan.md`. \* Locate the chosen phase section. \* Identify the correct target scope subsection (Architectural, Operations, Domain Controller, PAW, or Endpoint). \* Insert the new requirement. Ensure it is sorted in alphanumeric order by its Requirement ID. \* Use the relative markdown link syntax: `\* \*\*[REQ-XXX-### - Requirement Title] (.../module-dir/file-name.md)\*\*: Brief explanation.` (Note: remove the space between the bracket and parenthesis in your actual implementation).

### 3. Verify Links and Guidebook Build After editing, run the automated verification script from a PowerShell console to ensure links resolve properly: ``powershell .\Verify-ADHardeningDocs.ps1 ``

Once verification passes, run the compilation script to update the unified guidebook file:

```
python scripts/compile_docs.py
```

#### # ANSSI Active Directory Hardening Compliance Matrix

This document maps the recommendations of the **ANSSI (French National Agency for the Security of Information Systems)** Active Directory Hardening Guide and related secure administration guides to the technical security controls present in this guidebook.

## Mapped ANSSI Recommendations

ID	RECOMMENDATION DESCRIPTION	CATEGORY	STATUS	MAPPED TECHNICAL CONTROL(S)
R1	Define and implement a logical partitioning model (Tiering)	Tiering / Admin Boundaries	Covered	REQ-ARCH-001, REQ-ARCH-002, REQ-ARCH-003
R2	Limit and control administrative privileges	Account Restrictions	Covered	REQ-ARCH-001, REQ-END-006, REQ-PAW-003
R3	Use dedicated administrative accounts and secure stations	PAW & Admin Accounts	Covered	REQ-ARCH-001, REQ-PAW-001, REQ-PAW-002, REQ-PAW-003, REQ-PAW-004, REQ-PAW-005, REQ-PAW-006, REQ-PAW-007, REQ-PAW-008, REQ-PAW-009, REQ-PAW-010, REQ-PAW-011, REQ-PAW-012, REQ-PAW-013, REQ-PAW-036
R4	Minimize services and software on Domain Controllers	Service Minimization	Covered	REQ-DC-008, REQ-DC-012
R5	Keep Forest and Domain functional levels up-to-date	Functional Levels	Covered	REQ-ARCH-004
R6	Restrict membership of default administrative groups	Privileged Group Audit	Covered	REQ-ARCH-003, REQ-ID-010
R7	Configure IPsec transport mode for domain isolation	Network Cryptography	Covered	REQ-NET-004, REQ-NET-005
R8	Restrict administration protocols to dedicated subnets and jump hosts	Management Ports & Subnets	Covered	REQ-NET-001, REQ-NET-002, REQ-NET-003, REQ-ARCH-002
R9	Deploy Local Administrator Password Solution (LAPS)	Local Admin Passwords	Covered	REQ-ID-002
R10	Restrict authentication delegation and administrative tool execution	AppLocker / Delegation	Covered	REQ-DC-021, REQ-DC-034, REQ-PAW-001, REQ-PAW-036, REQ-END-011, REQ-END-027
R11	Enforce NTLM restriction policies	NTLM Restriction	Covered	REQ-DC-014
R12	Disable obsolete name resolution protocols (LLMNR/NetBIOS)	Name Resolution	Covered	REQ-DC-002, REQ-END-001
R13	Deprecate legacy protocols and enforce transport security	Obsolete Protocols	Covered	REQ-DC-001, REQ-DC-003, REQ-DC-010
R14	Enforce LSA Protection and Credential Guard	Credential Isolation	Covered	REQ-DC-006, REQ-DC-007, REQ-PAW-002, REQ-PAW-010, REQ-END-010, REQ-END-023
R15	Prohibit unconstrained Kerberos delegation	Kerberos Delegation	Covered	REQ-ID-004
R16	Restrict constrained Kerberos delegation	Kerberos Delegation	Covered	REQ-ID-004
R17	Enforce strong Kerberos encryption algorithms (AES-only)	Kerberos Encryption	Covered	REQ-DC-010, REQ-ID-008
R18	Harden TLS protocols, cipher suites, and elliptic curves (Schannel)	TLS / Cryptography	Covered	REQ-NET-006
R19	Enforce LDAP server signing and client-side resolution settings	LDAP Security	Covered	REQ-DC-004, REQ-DC-024, REQ-NET-009

ID	RECOMMENDATION DESCRIPTION	CATEGORY	STATUS	MAPPED TECHNICAL CONTROL(S)
R20	Enforce LDAP Channel Binding and Kerberos Armoring	LDAP Channel Binding	Covered	<a href="#">REQ-DC-005</a> , <a href="#">REQ-DC-013</a>
R21	Disable SMBv1 on all network nodes	SMB Security	Covered	<a href="#">REQ-DC-001</a> , <a href="#">REQ-END-026</a>
R22	Enforce SMB signing and encryption	SMB Security	Covered	<a href="#">REQ-DC-009</a> , <a href="#">REQ-NET-007</a>
R23	Harden and protect adminSDHolder permissions	adminSDHolder Permissions	Covered	<a href="#">REQ-DC-016</a> , <a href="#">REQ-DC-024</a> , <a href="#">REQ-ID-013</a>
R24	Harden Active Directory Domain Trusts (SID history/filtering)	Domain Trusts	Covered	<a href="#">REQ-ARCH-006</a>
R35	Implement Group Managed Service Accounts (gMSA)	Service Account Hardening	Covered	<a href="#">REQ-ID-003</a> , <a href="#">REQ-ID-014</a>
R36	Harden Active Directory Certificate Services (ADCS) and PKI	ADCS / PKI Hardening	Covered	<a href="#">REQ-ID-015</a>
R37	Revoke obsolete and insecure certificate templates	ADCS / PKI Hardening	Covered	<a href="#">REQ-ID-015</a>
R47	Harden virtualization hosts for Domain Controllers	DC Virtualization	Covered	<a href="#">REQ-DC-018</a>
R48	Configure advanced security audit policies	Audit Policies	Covered	<a href="#">REQ-LOG-001</a>
R50	Configure PowerShell and command-line auditing	PowerShell Auditing	Covered	<a href="#">REQ-LOG-002</a>
R52	Deploy and harden Sysmon and configure SIEM log shipping	Monitoring & Shipping	Covered	<a href="#">REQ-LOG-003</a> , <a href="#">REQ-LOG-004</a>
R54	Establish secure Domain Controller backup and disaster recovery	Disaster Recovery	Covered	<a href="#">REQ-OPS-002</a> , <a href="#">REQ-OPS-008</a>
R57	Perform continuous security assessments and privileged group audits	Security Assessments	Covered	<a href="#">REQ-ARCH-003</a> , <a href="#">REQ-OPS-010</a>
R58	Deploy and harden Privileged Access Workstations (PAWs)	PAW Deployment	Covered	<a href="#">REQ-PAW-001</a> , <a href="#">REQ-PAW-004</a> , <a href="#">REQ-PAW-005</a> , <a href="#">REQ-PAW-006</a>
R64	Configure Active Directory Authentication Silos and Policies	Authentication Silos	Covered	<a href="#">REQ-ID-012</a>
R80	Configure Authentication Policies for administrative groups	Authentication Silos	Covered	<a href="#">REQ-ID-012</a>

#### ## Administrative / Operational Recommendations (Out of Scope)

The following ANSSI secure administration recommendations relate to organizational policy, administrative processes, physical security, or user training, which are outside the scope of technical GPO or PowerShell controls:

ID	RECOMMENDATION DESCRIPTION	CATEGORY	STATUS	NOTES
R26	Inform administrators of security policies	Governance	Not Covered	Organizational policy / administrative training.
R27	Maintain an up-to-date registry of administrative actions	Operations	Not Covered	Operational procedure / manual record-keeping.
R30	Periodically audit administration workstations	Operations	Not Covered	Process-based vulnerability scans and compliance checks.
R31	Physically secure administration environments	Physical Security	Not Covered	Server room access controls, locked server racks, etc.
R32	Establish separate domain names for administration zones	Architecture	Not Covered	Organizational DNS design recommendation.

## # CIS Benchmarks Compliance Mapping Matrix

This document maps the **CIS Benchmarks** sections for Windows Server (2016, 2019, 2022) and Windows 10/11 Client systems to the technical security controls present in this guidebook.

## ## Mapped CIS Benchmark Sections

CIS SECTION	SECTION TITLE / SCOPE	BENCHMARK CATEGORY	STATUS	MAPPED TECHNICAL CONTROL(S)
1.1	Password Policy (Complexity, Length, Age, History)	Account Policies	Covered	REQ-ID-001, REQ-PAW-013, REQ-END-018
1.2	Account Lockout Policy (Threshold, Duration)	Account Policies	Covered	REQ-PAW-013, REQ-END-018
1.3	Kerberos Policy (Ticket Lifetimes, Clock Tolerance)	Account Policies	Covered	REQ-END-018
2.2	User Rights Assignment (Deny logons, Allow logons, DC Operator Restrictions)	Local Policies	Covered	REQ-ARCH-001, REQ-ID-007, REQ-DC-023, REQ-PAW-009, REQ-END-016
2.3	Security Options (LSA, LAN Manager, LDAP Signing, SMB Signing)	Local Policies	Covered	REQ-DC-003, REQ-DC-004, REQ-DC-005, REQ-DC-006, REQ-DC-009, REQ-DC-010, REQ-DC-011, REQ-DC-014, REQ-DC-024, REQ-DC-025
9.1	Windows Defender Firewall Profiles (Domain, Private, Public)	Firewall	Covered	REQ-NET-001, REQ-NET-008, REQ-END-022
18.2	Local Administrator Password Solution (LAPS) settings	Administrative Templates	Covered	REQ-ID-002
18.3	AutoPlay and AutoRun settings	Administrative Templates	Covered	REQ-END-003
18.5	Network parameters (KeepAliveTime, perform router discovery, TCP retransmissions)	Administrative Templates	Covered	REQ-DC-026, REQ-END-001
18.6	DNS Client, Fonts, LLTD, Peer-to-Peer, WCN settings	Administrative Templates	Covered	REQ-DC-002, REQ-DC-026, REQ-END-001
18.8	PowerShell Logging, Device Guard, and Virtualization-Based Security	Administrative Templates	Covered	REQ-LOG-002, REQ-PAW-006, REQ-PAW-010, REQ-END-010
18.9	System Services (Print Spooler), AppLocker, WDAC, and GP Refresh settings	Administrative Templates	Covered	REQ-ARCH-005, REQ-DC-001, REQ-DC-008, REQ-DC-021, REQ-PAW-001, REQ-END-011, REQ-END-024, REQ-END-025, REQ-END-027
18.10	Windows Components (Defender, RDP Session Limits, Search, KMS Client, WinRS)	Administrative Templates	Covered	REQ-DC-019, REQ-DC-020, REQ-DC-027, REQ-NET-010, REQ-END-005, REQ-END-007, REQ-END-026
19.1	Windows Defender Firewall port configurations and isolation rules	Firewall Advanced Security	Covered	REQ-NET-001, REQ-NET-003, REQ-NET-008, REQ-END-022
19.6 / 19.7	User configuration (Help Experience, Cloud Content, spotlight, WMP playback)	Administrative Templates	Covered	REQ-DC-027, REQ-END-019
Public Key	Encrypting File System (EFS) Settings	Public Key Policies	Covered	REQ-ARCH-005

## ## CIS Benchmark Sections Outside Active Directory Scope

The following CIS Benchmark sections are not covered by this guidebook because they concern standalone workstation settings, user-level privacy options, or non-security features:



SECTION	TITLE / SCOPE	CATEGORY	STATUS	NOTES
18.8.2	Toast Notifications / Cortana Settings	User Experience	Not Covered	Operational/productivity settings, low security impact.
18.9.1	Background Intelligent Transfer Service	System Services	Not Covered	General operating system background task tuning.

#### ## Explicit Hardening Exclusions

The following specific CIS Level 2 controls have been explicitly excluded from this guidebook based on operational safety and environment compatibility constraints:

1. **Disable IPv6 Components (CIS 18.6.19.2.1):** Excluded to prevent network malfunctions on loopback, DNS resolution, and domain replication dependencies.
2. **Disable WinRM Server Automatic Listener (CIS 18.10.89.2.2):** Excluded to preserve automatic listener provisioning for remote management.

## # Microsoft Security Baselines Compliance Mapping Matrix

This document maps the focus areas of the **Microsoft Security Baselines** (Domain Controller, Member Server, and Windows Client baselines) to the technical security controls present in this guidebook.

## ## Mapped Microsoft Security Baseline Focus Areas

FOCUS AREA	BASELINE REQUIREMENT DESCRIPTION	BASELINE CATEGORY	STATUS	MAPPED TECHNICAL CONTROL(S)
<b>Credential Guard</b>	Deploy Windows Defender Credential Guard to isolate and protect LSASS credentials (disabled on Domain Controllers as per Microsoft recommendations).	Credential Isolation	<b>Covered</b>	<a href="#">REQ-DC-007</a> , <a href="#">REQ-PAW-010</a> , <a href="#">REQ-END-010</a>
<b>LSA Protection</b>	Configure Local Security Authority (LSA) to run as a protected process (LSA Protection).	Credential Isolation	<b>Covered</b>	<a href="#">REQ-DC-006</a> , <a href="#">REQ-PAW-002</a> , <a href="#">REQ-END-023</a>
<b>Protocol Deprecation</b>	Disable legacy protocols (SMBv1, NTLMv1, digest authentication) across servers and endpoints.	Legacy Protocols	<b>Covered</b>	<a href="#">REQ-DC-001</a> , <a href="#">REQ-DC-003</a> , <a href="#">REQ-DC-014</a> , <a href="#">REQ-END-026</a>
<b>AppLocker</b>	Deploy AppLocker application control policies to restrict unauthorized software execution.	Application Control	<b>Covered</b>	<a href="#">REQ-DC-021</a> , <a href="#">REQ-PAW-001</a> , <a href="#">REQ-END-027</a>
<b>Windows Defender Application Control</b>	Deploy Windows Defender Application Control (WDAC) and Driver Blocklists.	Application Control	<b>Covered</b>	<a href="#">REQ-DC-022</a> , <a href="#">REQ-DC-034</a> , <a href="#">REQ-PAW-012</a> , <a href="#">REQ-PAW-036</a> , <a href="#">REQ-END-011</a> , <a href="#">REQ-END-036</a>
<b>BitLocker</b>	Enforce BitLocker drive encryption with TPM and Startup PIN configurations.	Data Protection	<b>Covered</b>	<a href="#">REQ-PAW-004</a> , <a href="#">REQ-END-012</a>
<b>DMA Protection</b>	Enable hardware virtualization-based security and Kernel DMA Protection.	Hardware Integrity	<b>Covered</b>	<a href="#">REQ-PAW-006</a> , <a href="#">REQ-END-014</a>
<b>Secure Boot</b>	Enforce UEFI Secure Boot and hardware platform security settings.	Hardware Integrity	<b>Covered</b>	<a href="#">REQ-PAW-005</a> , <a href="#">REQ-END-009</a>
<b>Audit Policies</b>	Configure advanced security audit policies (DC, member server, and client baselines).	Auditing & Logging	<b>Covered</b>	<a href="#">REQ-LOG-001</a>
<b>PowerShell Logging</b>	Configure PowerShell script block logging and transcription.	Auditing & Logging	<b>Covered</b>	<a href="#">REQ-LOG-002</a>
<b>UAC Policies</b>	Configure User Account Control (UAC) baseline settings.	Account Controls	<b>Covered</b>	<a href="#">REQ-END-002</a>
<b>Removable Storage</b>	Deploy GPOs to block external removable storage devices (USB mass storage).	Data Protection	<b>Covered</b>	<a href="#">REQ-END-004</a>
<b>Point and Print</b>	Configure Point and Print restrictions to prevent PrintNightmare exploits.	Services Hardening	<b>Covered</b>	<a href="#">REQ-END-025</a> , <a href="#">REQ-PAW-015</a>
<b>SYSVOL replication</b>	Migrate SYSVOL replication from FRS to DFS Replication (DFSR).	DC Hardening	<b>Covered</b>	<a href="#">REQ-DC-015</a>
<b>adminSDHolder</b>	Harden adminSDHolder object permissions to prevent privilege persistence.	DC Hardening	<b>Covered</b>	<a href="#">REQ-DC-016</a> , <a href="#">REQ-DC-024</a>
<b>Services minimization</b>	Disable unnecessary system services on Domain Controllers and Endpoints.	Services Hardening	<b>Covered</b>	<a href="#">REQ-DC-012</a> , <a href="#">REQ-END-024</a>
<b>dSHeuristics Hardening</b>	Configure the forest-wide dSHeuristics attribute to block anonymous operations, secure adminSDHolder, and enforce KB5008383 protections.	DC Hardening	<b>Covered</b>	<a href="#">REQ-DC-024</a>

## ## Microsoft Baseline Controls Outside Guidebook Scope

The following Microsoft Security Baseline recommendations are not covered by this guidebook as they are more suited for cloud-managed, internet-connected, or hybrid environments:

FOCUS AREA	BASELINE REQUIREMENT DESCRIPTION	CATEGORY	STATUS	NOTES
Microsoft Defender for Endpoint	Cloud-based EDR enrollment and configuration	Endpoint Defense	Not Covered	Out of scope due to the strict air-gapped (offline) requirement of the environment.
Windows Update for Business	Cloud-based patch management and updates	Patch Management	Not Covered	Out of scope. Patching must be managed via offline WSUS tiering.
Microsoft Defender SmartScreen	Cloud-based reputation screening for downloads	Edge / Web Security	Not Covered	No internet connection is present to contact Microsoft reputation services.